

Go

K19G

2026-09-07

Table of contents

Everything Go - Comprehensive Syllabus	115
How to read this book (pick a path)	115
Map of overlapping parts (twins)	116
Related books	116
1. Foundations & Mindset	116
2. Core Language Essentials	117
3. Pointers & Memory Management	117
4. Advanced Error Handling	117
5. Concurrency & Parallelism	117
6. Web Development & Modern Stacks	118
7. Performance, Profiling & Tooling	118
8. Infrastructure & Deployment	118
9. Security & Hardening	119
10. Specialized Applications	119
11. Production Deep Dives	119
12. Modern Go Book Synthesis	119
13. Go Deep Dives (Runtime, Toolchain & Production)	120
14. Concurrency From the Ground Up	120
15. Web Development in Go (Bookstore track)	120
16. Golang CLI (stdlib → Cobra)	120
17. Standard Library Tour	121
18. Projects	121
 Foundations	 122
 Why Go Exists	 123
Overview	123
The Problems Go Was Designed to Solve	123
1. Slow Compilation Times	123
2. Complexity in Modern Languages	123
3. Difficulty with Concurrency	123
4. Dependency Management Chaos	123
Go's Design Philosophy	124
Key Principles	124
What Makes Go Different	124
Compiled, Statically Typed, Fast	124
Built-in Concurrency	124
Single Binary Output	125
Standard Formatting	125

Who Uses Go	125
When to Choose Go	125
Summary	125
Related Topics	126
More examples	126
Example: clarity over cleverness	126
Example: static types catch mistakes early	127
Runnable example	127
Learning Go in 2026	130
Why Go is Worth Learning in 2026	130
The Go Mindset (Most People Get This Wrong)	130
The Fastest Roadmap to Learn Go	130
1. Learn Only the Essentials First	130
2. Learn Concurrency Early	131
3. Build Tiny Programs Every Day	131
The 3-Layer Practice System	131
Go Projects That Make You Job-Ready	131
Why Go is a DevOps Superpower	131
Memory & Consistency	132
Go 1.27 Migration Checklist (Shared)	132
1. Pin Toolchain and Module Versions	132
2. Run Modernization and Safety Baseline	132
3. Re-Baseline Tests and Benchmarks	133
4. Language and stdlib opt-ins	133
5. Enable Experiments Deliberately	133
6. Observe Production After Rollout	133
Summary	133
Related Topics	134
More examples	134
Example: API health-check style probe (no network)	134
Example: composition over inheritance	134
Runnable example	135
The Go Philosophy	138
The Go Philosophy: Why “Boring” is a Superpower	139
Boring Software is Reliable Software	139
1. Readability Over Conciseness	139
2. One Way To Do It	139
3. Stability is a Feature	139
The 2026 Context: AI and Complexity	140
Key Principles Summarized	140
Conclusion	140
More examples	140
Example: one obvious loop	140
Example: no overloading—distinct names	141
Runnable example	142

The 3-Layer Practice System	144
The 3-Layer Practice System: Read, Write, Ship	145
Layer 1: Read (Input)	145
What to Read	145
How to Read	145
Layer 2: Write (Output)	145
The “Clone” Technique	145
Constraints	146
Layer 3: Ship (Feedback)	146
The Feedback Loop	146
Summary	146
More examples	146
Example: clone <code>cat</code> (Layer 2 write)	146
Example: intentional breakage (Layer 2 write)	147
Runnable example	148
Transitioning to Go	151
Transitioning to Go: For Java and Node.js Developers	152
For the Java Developer	152
The “Spring” Trap	153
For the Node.js Developer	153
The “Concurrency” Trap	153
Universal Truths in Go	154
Summary	154
More examples	154
Example: explicit constructor DI (not Spring)	154
Example: value vs pointer is explicit	155
Runnable example	156
Installing and Running Go	158
Overview	158
Installation	158
macOS	158
Linux	158
Windows	158
Verify Installation	159
Environment Variables	159
Your First Go Program	159
The <code>go run</code> Command	159
How It Works	160
Running Multiple Files	160
The <code>go build</code> Command	160
Common Build Options	160
Cross-Compilation	161
The Build Lifecycle	161

IDE and Editor Setup	161
Visual Studio Code	161
Recommended Tools (installed automatically)	161
GoLand	162
Neovim/Vim	162
Project Structure Basics	162
Common Pitfalls	162
GOPATH vs Modules	162
Executable vs Library	162
Summary	163
Exercises	163
Related Topics	163
More examples	163
Example: classic Hello, World	163
Example: exit codes and stderr	164
Runnable example	164
How Go Code Is Organized	167
Overview	167
Packages: The Building Blocks	167
Package Rules	167
The <code>main</code> Package	167
Import System	168
Basic Imports	168
Import Block (Preferred Style)	168
Import Aliases	168
Blank Imports (Side Effects Only)	169
Dot Imports (Avoid in Production)	169
Visibility: Exported vs Unexported	169
Modules: Modern Dependency Management	170
Creating a Module	170
Module Structure	170
Internal Packages	170
Standard Project Layout	171
<code>cmd/</code> Directory	171
Package Naming Conventions	171
Avoid Meaningless Names	172
Circular Import Prevention	172
Summary	172
Exercises	172
Related Topics	172
More examples	173
Example: exported vs unexported names	173
Example: package main entry point	173
Runnable example	174
Core Go Commands	176
Overview	176

Command Summary	176
go build - Compiling Code	176
Basic Usage	176
Build Flags	177
Cross-Compilation	177
Linker Flags	177
go run - Quick Execution	178
go test - Testing	178
go get - Dependency Management	179
go mod - Module Operations	179
go fmt / gofmt - Formatting	180
go vet - Static Analysis	180
go doc - Documentation	180
go install - Installing Binaries	181
go clean - Cleanup	181
go env - Environment	182
go list - Package Information	182
go generate - Code Generation	182
Tool Installation Best Practices	183
Common Workflows	183
Development Cycle	183
Release Build	183
Summary	183
Related Topics	184
More examples	184
Example: flags for go run experiments	184
Example: build-time GOOS/GOARCH report	185
Runnable example	185
Formatting, Vetting, and Documentation	187
Overview	187
Code Formatting with gofmt	187
Why Gofmt Exists	187
Basic Usage	187
Formatting Rules	187
Code Simplification	188
Import Formatting with goimports	188
Static Analysis with go vet	189
What Vet Catches	189
Common Issues Detected	189
Running Specific Checks	190
Enhanced Linting with staticcheck	190
Documentation with godoc	191
Writing Documentation	191
Documentation Conventions	191
Code Examples in Docs	192
Viewing Documentation	192
Package Comments	193

Combining Tools in Workflow	194
Pre-commit Hook	194
Makefile Integration	194
CI Pipeline (GitHub Actions)	195
Summary	195
Related Topics	195
More examples	195
Example: godoc-style exported comments	195
Example: a construct <code>go vet</code> cares about	196
Runnable example	197
Dependency Management	199
Overview	199
Understanding Go Modules	199
Creating a Module	199
The <code>go.mod</code> File	199
Anatomy of <code>go.mod</code>	199
Directives Explained	200
The <code>go.sum</code> File	200
Adding Dependencies	201
Using <code>go get</code>	201
Automatic Detection	201
Updating Dependencies	201
Semantic Versioning	202
Major Version Suffixes	202
Managing <code>go.mod</code>	202
<code>go mod tidy</code>	202
<code>go mod download</code>	202
<code>go mod verify</code>	203
<code>go mod why</code>	203
<code>go mod graph</code>	203
Vendoring	203
Replace Directive	204
Local Development	204
Fork Substitution	204
Version Override	204
Private Modules	204
GOPRIVATE	204
Git Credentials	204
Dependency Management Best Practices	205
1. Pin Dependencies	205
2. Regular Updates	205
3. Minimal Dependencies	205
4. Security Scanning	205
Common Issues	205
Module Not Found	205
Version Conflicts	206
Checksum Mismatch	206

Summary	206
Related Topics	206
More examples	206
Example: prefer stdlib before <code>go get</code>	206
Example: semantic version labels as data	207
Runnable example	208
Workspaces and Multi-Module Development	210
Overview	210
The Problem Workspaces Solve	210
Without Workspaces	210
With Workspaces	210
Creating a Workspace	211
Initial Setup	211
The <code>go.work</code> File	211
Workspace Commands	211
<code>go work init</code>	211
<code>go work use</code>	212
<code>go work edit</code>	212
<code>go work sync</code>	212
Multi-Module Development Workflow	212
Project Structure	212
Setting Up	213
Development Flow	213
Replace vs Workspace	213
When to Use <code>replace</code>	213
When to Use Workspace	213
Workspace with Replace	214
Best Practices	214
1. Git Ignore <code>go.work</code>	214
2. Document Setup	214
3. CI Without Workspaces	215
4. Shared Workspace for Monorepos	215
Common Patterns	215
Microservices Development	215
Library Development	216
Troubleshooting	216
Module Not Found in Workspace	216
Wrong Version Used	216
Workspace Mode Detection	216
Summary	217
Exercises	217
Related Topics	217
More examples	217
Example: local “library” function in one module	217
Example: replace-path thinking with build info	218
Runnable example	218

Core	221
Predeclared Types	222
Overview	222
Boolean Type	222
Boolean Expressions	222
Numeric Types	222
Integer Types	222
Unsigned Integers	223
Type Aliases	223
Floating-Point Types	223
Complex Numbers	224
String Type	224
Strings vs Runes	224
The Error Type	225
Type Conversions	225
Common Conversion Pitfalls	226
Type Information	226
Numeric Literals	226
Summary	226
Related Topics	227
More examples	227
Example: integer and float basics	227
Example: bool and rune	227
Runnable example	228
Zero Values and Initialization	230
Overview	230
Zero Values by Type	230
Demonstrating Zero Values	230
Struct Zero Values	231
Nested Structs	231
Array Zero Values	232
Why Zero Values Matter	232
1. Eliminates Undefined Behavior	232
2. Reduces Boilerplate	232
3. Enables “Usable Zero Value” Pattern	233
Initialization Methods	233
Short Variable Declaration	233
Explicit Type Declaration	233
Type Inference (var)	233
Multiple Variable Declaration	234
Composite Literal Initialization	234
nil vs Zero Value	234
nil Slice vs Empty Slice	234
nil Map Danger	235
The new vs make Functions	235
new(T)	235

<code>make(T, args)</code>	235
Summary	236
Related Topics	236
More examples	236
Example: zero values of common types	236
Example: nil map vs empty map	237
Runnable example	237
Constants and Literals	240
Overview	240
Declaring Constants	240
Basic Declaration	240
Constant Block	240
Typed vs Untyped Constants	240
Untyped Constants	241
Why This Matters	241
Constant Kinds	241
The <code>iota</code> Constant Generator	241
Basic Usage	242
Skip Values	242
Bit Flags with <code>iota</code>	242
Multiple <code>iota</code> in One Line	242
Reset <code>iota</code>	243
Constant Expressions	243
Allowed in Constant Expressions	243
Not Allowed	243
Numeric Literals	244
Integer Literals	244
Floating-Point Literals	244
Hexadecimal Floats (Go 1.13+)	244
Imaginary Literals	244
Digit Separators	244
String and Rune Literals	245
Interpreted Strings	245
Escape Sequences	245
Raw String Literals	245
Rune Literals	245
Common Patterns	246
Enum Pattern	246
Configuration Constants	246
Summary	246
Related Topics	247
More examples	247
Example: <code>iota</code> enums	247
Example: untyped constant flexibility	247
Runnable example	248

Variables and Scope	250
Overview	250
Variable Declaration	250
Using var	250
Short Variable Declaration (:=)	250
When to Use var vs :=	250
Multiple Variables	251
Variable Block	251
Redeclaration and Shadowing	251
Redeclaration with :=	251
Variable Shadowing	252
Scope Levels	253
Package Scope	253
File Scope (Imports)	253
Function Scope	253
Block Scope	253
Visibility (Exported vs Unexported)	254
The Blank Identifier (_)	254
Variable Lifetime	255
Stack vs Heap	255
Garbage Collection	255
Common Patterns	255
Zero Value Initialization	255
Conditional Initialization	256
Multiple Assignment	256
Common Pitfalls	256
Accidental Shadowing	256
Loop Variable Capture	257
Summary	257
Related Topics	257
More examples	258
Example: short declaration scope	258
Example: if with short statement	258
Runnable example	259
 Conditional Logic	 261
Overview	261
The if Statement	261
Basic Syntax	261
With else	261
With else if	261
Initialization Statement	262
No Parentheses Required	262
Braces Required	262
Boolean Expressions	263
Comparison Operators	263
Logical Operators	263

The switch Statement	263
Basic Switch	263
No Fall-Through by Default	264
Explicit Fallthrough	264
Switch with Initialization	264
Expression-less Switch	265
Type Switch	265
Common Patterns	266
Error Checking	266
Boolean Assignment	266
Guard Clauses	266
ok-idiom	267
Comma-ok Pattern	267
Switch vs If/Else	267
Prefer Switch When:	267
Prefer If/Else When:	268
Summary	268
Related Topics	268
More examples	268
Example: if / else if ladder	268
Example: switch on strings	269
Runnable example	270
Loops and Iteration	273
Overview	273
The Basic for Loop	273
The while Loop (Condition Only)	273
The Infinite Loop	273
The range Loop	274
Slices and Arrays	274
Maps	274
Strings	274
Channels	274
Iterators (Go 1.23+)	274
Sequence Iterators	275
Pull Iterators	275
Loop Control	275
break and continue	275
Labels for Nested Loops	276
Common Patterns	276
Summary	276
Related Topics	276
More examples	277
Example: classic for and range	277
Example: break and continue	277
Runnable example	278

Arrays and Slices	281
Overview	281
Arrays	281
Declaration	281
Properties	281
Slices	281
Creation	281
From Array	282
Slice Internals & Reallocation	282
Length and Capacity	282
Slice Operations	282
Append	282
Slicing	282
Copy	283
The <code>slices</code> Package (Go 1.21+)	283
Modern Loop Pattern (Go 1.23+)	283
<code>nil</code> vs Empty Slice	283
Common Patterns	284
Remove Element	284
Insert Element	284
Stack	284
Summary	284
Related Topics	284
More examples	284
Example: array vs slice	284
Example: append and capacity growth	285
Runnable example	285
 Working with Strings	 288
Overview	288
String Basics	288
Immutability	288
Bytes vs Runes	288
Iterating	288
String Operations	289
Concatenation	289
Comparison	289
<code>strings</code> Package	289
<code>strconv</code> Package	290
<code>fmt</code> Package	290
Rune Operations	290
Summary	290
Related Topics	291
More examples	291
Example: bytes vs runes	291
Example: <code>strings.Builder</code>	291
Runnable example	292

Maps	295
Overview	295
Creating Maps	295
Basic Operations	295
Set	295
Get	295
Delete	295
Check Existence	296
nil Map Behavior	296
Iteration	296
Common Patterns	297
Set (Unique Values)	297
Counter	297
Grouping	297
Default Value	297
Concurrency Warning	297
The <code>maps</code> Package (Go 1.21+)	298
Modern Loop Pattern (Go 1.23+)	298
Map Internals	298
Summary	298
Related Topics	299
More examples	299
Example: comma-ok lookup	299
Example: delete and range	299
Runnable example	300
 Structs and Data Modeling	 303
Overview	303
Defining Structs	303
Creating Instances	303
Accessing Fields	304
Anonymous Structs	304
Embedding (Composition)	304
Promoted field keys in literals (Go 1.27+)	305
Struct Tags	305
Reading Tags	306
Comparison	306
Constructors	306
Summary	306
Related Topics	307
More examples	307
Example: literal and field access	307
Example: embedding promotes fields	307
Runnable example	308
 Time and Scheduling	 311
Overview	311
Current Time	311

Duration	311
Sleeping	311
Timers	311
Tickers	312
Formatting	312
Parsing	312
Time Zones	312
Comparisons	312
Summary	313
Related Topics	313
More examples	313
Example: duration arithmetic	313
Example: parse and format	314
Runnable example	314

Memory 317

Functions in Depth 318

Overview	318
Function Declaration	318
Multiple Return Values	318
Named Return Values	319
Variadic Functions	319
Closures	319
Defer	319
Defer Order (LIFO)	320
Function Signatures	320
Summary	320
Related Topics	321
More examples	321
Example: multiple returns and named results	321
Example: defer runs LIFO	321
Runnable example	322

Functions as Values 324

Overview	324
Function Variables	324
Function Types	324
Higher-Order Functions	325
Functions as Parameters	325
Functions as Return Values	325
Common Patterns	325
Callback Pattern	325
Option Pattern	326
Middleware Pattern	326
Anonymous Functions	327
Summary	327

Related Topics	327
More examples	327
Example: swap function variables	327
Example: closure captures state	328
Runnable example	328
Methods and Receivers	331
Overview	331
Method Syntax	331
Value vs Pointer Receivers	331
Value Receiver	331
Pointer Receiver	332
When to Use Pointer Receivers	332
Methods on Any Type	332
Automatic Dereferencing	332
Embedding and Method Promotion	333
Method Override	333
Generic Methods (Go 1.27+)	333
Summary	334
Related Topics	334
More examples	335
Example: value receiver cannot mutate	335
Example: pointer receiver mutates	335
Runnable example	336
Understanding Memory in Go	338
Overview	338
Stack vs Heap	338
Stack	338
Heap	338
Escape Analysis	338
Check Escape Analysis	339
What Causes Escape	339
Memory Layout	339
Value Types	339
Reference Types	340
Best Practices	340
Reduce Allocations	340
Preallocate Slices	340
Use Value Semantics When Possible	340
Summary	341
Related Topics	341
More examples	341
Example: value stays local	341
Example: interface argument often forces heap	342
Runnable example	342

Pointers Explained	345
Overview	345
Pointer Basics	345
Declaration	345
Why Use Pointers	346
1. Modify Variables in Functions	346
2. Avoid Copying Large Structs	346
3. Share Data	346
nil Pointers	347
Pointer to Pointer	347
Pointers and Slices/Maps	347
Common Patterns	348
Return Pointer for “No Result”	348
Constructor Pattern	348
Summary	348
Related Topics	348
More examples	349
Example: address and dereference	349
Example: nil pointer check	349
Runnable example	350
 Mutability and Data Sharing	 352
Overview	352
Value vs Reference Semantics	352
Value Types (Copied)	352
Reference Types (Shared)	352
Controlling Mutability	353
Immutable by Design	353
Defensive Copy	353
Deep Copy	353
Function Parameters	353
Struct Field Mutability	354
Slice Gotchas	354
Summary	354
Related Topics	354
More examples	355
Example: slice header shares backing array	355
Example: copy to isolate mutation	355
Runnable example	356
 Garbage Collection	 359
Overview	359
Modern GC Architecture	359
Concurrent Mark and Sweep	359
Go 1.26: Green Tea GC (GTGC)	359
Go 1.27: size-specialized small allocations	360
GOGC and Memory Limits	360
Tuning with GOGC	360

Tuning with GOMEMLIMIT	360
Memory Stats	361
Reducing GC Pressure	361
1. Reduce Allocations	361
2. Preallocate	362
3. Use Value Types	362
4. Avoid String Concatenation in Loops	362
Profiling	362
Summary	363
Related Topics	363
More examples	363
Example: preallocate slice capacity	363
Example: strings.Builder vs naive concat	364
Runnable example	364
Stack Mechanics & Optimizations	367
Understanding Memory: Stack vs. Heap in Go	368
The Two Worlds	368
1. The Stack	368
2. The Heap	368
Go's Optimization Goal	368
Stack Growth (Contiguous Stacks)	368
2026: The "Mid-Stack" Inlining	369
Visualization	369
Practical Rule	369
More examples	369
Example: stack-friendly local work	369
Example: escape when a pointer outlives the frame	370
Runnable example	371
Escape Analysis & Alignment	373
Escape Analysis & Memory Alignment	374
Part 1: Escape Analysis	374
Asking the Compiler	374
Common Escape Triggers	374
The "Mid-Stack" Trick	374
Part 2: Memory Alignment (Padding)	375
The Struct Layout Game	375
Tools used in 2026	375
When does this matter?	375
More examples	376
Example: measure padding with <code>unsafe.Sizeof</code>	376
Example: reuse buffer vs allocate every loop	376
Runnable example	377

Generics	380
Designing with Interfaces	381
Overview	381
Interface Basics	381
Implicit Satisfaction	381
The Empty Interface	382
Interface Values	382
nil Interface vs nil Value	382
Common Patterns	382
Accept Interface, Return Concrete	382
Small Interfaces	383
Assert Interface Compliance	383
Summary	383
Related Topics	383
More examples	383
Example: implicit satisfaction	383
Example: empty interface / any	384
Runnable example	385
Interface Best Practices	387
Overview	387
Keep Interfaces Small	387
Define Interfaces at Consumer	387
Accept Interfaces, Return Structs	388
Interface Composition	388
Avoid Interface Pollution	388
Testing with Interfaces	389
Summary	389
Related Topics	389
More examples	389
Example: accept interfaces, return structs	389
Example: small interfaces compose	390
Runnable example	391
Type Assertions and Reflection Boundaries	394
Overview	394
Type Assertions	394
Type Switches	394
The reflect Package	395
Struct Reflection	395
When to Use Reflection	395
Performance Cost	395
Prefer Generics Over Reflection	396
Summary	396
Related Topics	396
More examples	396
Example: safe comma-ok assertion	396

Example: type switch	397
Runnable example	397
Why Generics Matter	400
Overview	400
The Problem Before Generics	400
With Generics	400
Type Parameters	401
Constraints	401
Custom Constraints	401
Generic Type Aliases (Go 1.24+)	401
Self-Referential Generics (Go 1.26+)	402
Generic Methods (Go 1.27+)	402
Benefits	403
Summary	403
Related Topics	403
More examples	403
Example: before generics—duplication	403
Example: one generic Max	404
Runnable example	404
Generic Functions	407
Overview	407
Basic Syntax	407
Examples	407
Map	407
Filter	407
Find	408
Contains	408
Keys/Values	408
Type Inference	409
Assignment contexts (Go 1.27+)	409
Summary	409
Related Topics	410
More examples	410
Example: Map slice transform	410
Example: Filter with predicate	410
Runnable example	411
Generic Types	414
Overview	414
Basic Syntax	414
Stack	414
Set	415
Pair	415
Result (Option Pattern)	416
LinkedList	416
Summary	416

Related Topics	417
More examples	417
Example: generic Stack	417
Example: generic Pair	418
Runnable example	418
Idiomatic Generics	421
Overview	421
When to Use Generics	421
Generics vs Interfaces	421
Use Interface When:	421
Use Generics When:	421
Guidelines	422
1. Start with Concrete Types	422
2. Use Meaningful Constraints	422
3. Don't Over-Constrain	422
4. Type Inference is Your Friend	423
Common Patterns	423
slices Package (stdlib)	423
maps Package (stdlib)	423
Summary	423
Related Topics	423
More examples	424
Example: prefer clear non-generic API when one type	424
Example: constrain only what you need	424
Runnable example	425
Errors	427
Errors as Values	428
Overview	428
The error Interface	428
Returning Errors	428
Creating Errors	428
Error Handling Patterns	429
Check Immediately	429
Early Return	429
Add Context	429
Custom Error Types	430
Sentinel Errors	430
Summary	430
Related Topics	431
Worked example	431
More examples	432
Runnable example	433

Wrapping and Inspecting Errors	436
Overview	436
Wrapping Errors	436
Unwrapping	436
errors.Is	436
errors.As	437
Custom Wrappers	437
Best Practices	437
Summary	438
Related Topics	438
Worked example	438
More examples	440
Runnable example	441
panic, recover, and Failure Modes	443
Overview	443
When to Panic	443
Panic	443
Common Panic Sources	443
Recover	444
HTTP Server Pattern	444
Panic vs Error	444
Summary	445
Related Topics	445
Worked example	445
More examples	447
Runnable example	448
The Must Pattern & No-GC Handling	450
Advanced Error Patterns: The “Must” & The “Odin” Way	451
1. The “Must” Pattern	451
When to use it?	451
Implementation	451
2. Linear Error Handling (The Odin/Zig Influence)	452
The Happy Path must remain left-aligned	452
No-GC Thinking: Errors as Resource Cleanup Triggers	452
3. Errors in 2026: “Join” and Structure	453
Summary	453
Worked example	454
More examples	455
Runnable example	456
Testing	459
Testing Fundamentals	460
Overview	460
Writing Tests	460

Running Tests	460
Test Functions	461
t.Error vs t.Fatal	461
Helper Functions	461
Setup and Teardown	461
Parallel Tests	462
Summary	462
Related Topics	462
Worked example	462
More examples	464
Runnable example	464
Test Organization and Coverage	467
Overview	467
File Organization	467
Package Naming	467
Testdata Directory	467
Code Coverage	468
Coverage Modes	468
Test Groups	468
Skipping Tests	469
Summary	469
Related Topics	469
Worked example	469
More examples	471
Runnable example	472
Advanced Testing	475
Overview	475
Table-Driven Tests	475
Subtests	476
Test Helpers	476
Cleanup	476
Deterministic Concurrency (<code>testing/synctest</code>)	477
Efficient Benchmarks (<code>B.Loop</code>)	478
Go 1.27 Testing Upgrade Checklist	478
Parallel Subtests	479
Benchmarks	479
Summary	479
Related Topics	479
Worked example	480
More examples	481
Runnable example	482
Integration and System Testing	486
Overview	486
HTTP Testing	486
Test Server	486

Database Testing	487
Build Tags	487
Environment-Based Tests	488
Docker Integration	488
Go 1.27 Integration Testing Upgrades	488
Summary	489
Related Topics	489
Worked example	489
More examples	491
Runnable example	492
Fuzzing and Property-Style Tests	496
Fuzzing and Property-Style Tests	497
Overview	497
Fuzz target	497
Seed corpus	497
Good fuzz targets	498
Property ideas	498
Rules of thumb	498
Try next	498
httptest and API Testing Patterns	499
httptest and API Testing Patterns	500
Overview	500
Handler unit test	500
Full mux	500
JSON helpers	500
Auth cookies	501
httptest.Server (integration-ish)	501
Rules of thumb	501
Try next	501
Golden Files and testdata	502
Golden Files and testdata	503
Overview	503
Layout	503
Pattern	503
Tips	504
embed for fixtures	504
Rules of thumb	504
Try next	504
Project: Service Test Suite	505
Project: Service Test Suite	506
Overview	506

Layers	506
Deliverables	506
Definition of done	506
Concurrency and Parallelism	507
Concurrency Basics	508
Overview	508
Goroutines	508
Creating Goroutines	508
Waiting with WaitGroup	508
Channels	509
Basic Channel Pattern	509
Worker Pool	509
Summary	510
Worked example	510
More examples	511
Runnable example	512
Related Topics	513
Channel Patterns	514
Overview	514
Channel Directions	514
Buffered vs Unbuffered	514
Closing Channels	514
Select	515
Timeout Pattern	515
Done Channel	515
Fan-Out / Fan-In Architecture	516
Summary	517
Worked example	517
More examples	518
Runnable example	520
Related Topics	522
Synchronization	523
Overview	523
sync.Mutex	523
sync.RWMutex	523
sync.WaitGroup	524
sync.Once	524
sync.Pool	525
sync.Map	525
atomic Package	525
Summary	525
Worked example	526
More examples	527

Runnable example	528
Related Topics	530
Context and Cancellation	531
Overview	531
Creating Contexts	531
Context Hierarchy & Downward Cancellation	531
Timeout	532
Deadline	532
Passing Context	532
Context Values	533
Best Practices	533
Summary	533
Worked example	533
More examples	534
Runnable example	535
Related Topics	537
Concurrency Design Guidelines	538
Overview	538
Share by Communicating	538
Avoid Goroutine Leaks	538
Always Close Channels from Sender	539
Race Detection	539
Common Patterns	539
Worker Pool	539
Bounded Concurrency	539
Deadlock Prevention	540
Summary	540
Worked example	540
More examples	541
Runnable example	542
Related Topics	544
Worker Pools	545
Worker Pools: Controlling Chaos	546
Pattern 1: The Semaphore (Simple Limiter)	546
Pattern 2: The Worker Pool (Fixed Goroutines)	546
2026: <code>errgroup</code> with Limits	547
Summary	548
Worked example	548
More examples	550
Runnable example	551
Pipelines & Complex Patterns	554
Pipelines and <code>sync.Cond</code>	555
1. The Pipeline Pattern (Fan-Out / Fan-In)	555

2. The Overlooked <code>sync.Cond</code>	556
Example: The Race Start	556
Summary	557
Worked example	557
More examples	558
Runnable example	560
Web	563
The GOTH Stack	564
The GOTH Stack: Go, Templ, HTMX, Tailwind	565
Why GOTH?	565
1. Templ: Type-Safe HTML	565
2. HTMX: The Engine	565
3. Tailwind: The Style	566
A Simple Handler Example	566
When NOT to use GOTH	566
Worked example	566
More examples	568
Runnable example	568
Skeleton Loading & HTMX	571
Skeleton Loading with Go & HTMX	572
The Pattern	572
Implementation	572
1. The Dashboard Handler (Fast)	572
2. The Skeleton Component (Templ)	572
3. The Real Data Handler (Slow)	573
Why this wins	573
Advanced: Request Coalescing	573
Worked example	573
More examples	575
Runnable example	576
Microservices & gRPC	578
Microservices: Switching to gRPC (and Connect)	579
The Protocol Buffers (Protobuf)	579
The gRPC Complexity (and the Solution: ConnectRPC)	579
Connect Example	580
When to use Microservices?	580
Worked example	580
More examples	582
Runnable example	583
Huma & OpenAPI	586

Modern APIs: Huma & OpenAPI	587
Enter Huma	587
Huma Example	587
Why Huma?	588
Worked example	588
More examples	590
Runnable example	591
Integrations: PostgreSQL & Redis	594
Integrations: PostgreSQL & Redis	595
PostgreSQL: pgx	595
Connection Pooling	595
Type-Safe SQL: sqlc	595
Redis: go-redis	595
Patterns	596
Context Awareness	596
Worked example	596
More examples	598
Runnable example	599
HTTP and Networking	603
Overview	603
HTTP Client	603
Custom Request	603
With Context	603
HTTP Server	604
JSON API	604
Middleware	604
ServeMux	604
Static Files	605
Summary	605
Worked example	605
More examples	606
Runnable example	607
Related Topics	609
Encoding and Serialization	610
Overview	610
JSON	610
Streaming JSON	611
encoding/json/v2 (Go 1.27+)	611
XML	611
Gob (Go Binary)	612
CSV	612
Base64	612
Summary	612
Worked example	613

More examples	614
Runnable example	615
Related Topics	617
Performance and Tooling	618
Reflection in Practice	619
Overview	619
Type and Value	619
Inspecting Structs	619
Modifying Values	620
Calling Methods	620
When to Use	620
Summary	620
Worked example	620
More examples	622
Runnable example	622
Related Topics	624
Unsafe Code	625
Overview	625
unsafe.Pointer	625
Common Uses	625
Memory Layout	625
String to Bytes (Zero-Copy)	625
Accessing Unexported Fields	626
Dangers	626
Guidelines	626
Summary	626
Worked example	626
More examples	627
Runnable example	628
Related Topics	629
Cgo and Foreign Function Interfaces	630
Overview	630
Basic Cgo	630
Passing Data	630
Strings	631
Linking Libraries	631
Downsides	631
When to Use	632
Summary	632
Worked example	632
More examples	633
Runnable example	633
Related Topics	635

Performance Tuning	636
Overview	636
Benchmarks	636
CPU Profiling	636
pprof Commands	636
Memory Profiling	636
HTTP Profiling	637
Tracing	637
Optimization Tips	637
Reduce Allocations	637
Avoid Copying	637
Summary	638
Worked example	638
More examples	639
Runnable example	640
Related Topics	641
Advanced Profiling (pprof/trace)	642
Performance: Deep Dives with pprof & trace	643
1. pprof: The Sampling Profiler	643
Usage	643
Mutex Profiling	643
2. Execution Tracer (go tool trace)	644
3. Benchmarks as Profiling Inputs	644
Summary	644
Worked example	644
More examples	645
Runnable example	646
The Modern Toolkit	648
The Modern 2026 Toolkit	649
General Tools (Rust-powered, Go-essential)	649
Go-Specific Tools	649
gopsutil	649
go fix Modernizers (1.26–1.27)	649
govulncheck	650
Workflow Integration	650
Worked example	650
More examples	651
Runnable example	652
Designing for the Long Term	654
Overview	654
Package Design	654
Dependency Direction	654
Interface Segregation	654
Options Pattern	655

Error Handling	655
Testing Strategy	655
Guidelines	656
Summary	656
Worked example	656
More examples	657
Runnable example	658
Related Topics	660
Static Analysis and Security	661
Overview	661
go vet	661
staticcheck	661
golangci-lint	661
govulncheck	661
gosec	662
Common Security Issues	662
CI Integration	662
Summary	662
Worked example	663
More examples	664
Runnable example	665
Related Topics	667
Code Generation and Embedding	668
Overview	668
go generate	668
Common Generators	668
Embedding Files	668
Reading Embedded Files	669
HTTP File Server	669
Templates	669
Summary	669
Worked example	670
More examples	670
Runnable example	671
Related Topics	673
Index	674
Quick Reference Guide	674
Essential Commands	674
Common Patterns	674
Type Reference	674
Key Packages	674
Resources	675
Worked example	675
More examples	676
Runnable example	677

Documentation and APIs	679
Overview	679
Writing Doc Comments	679
Conventions	679
Examples	680
Viewing Docs	680
pkg.go.dev	680
API Design Guidelines	680
Summary	680
Worked example	681
More examples	681
Runnable example	682
Related Topics	684
 Infrastructure	 685
Containerization (Docker)	686
Containerization: Multi-Stage Builds	687
Overview	687
Why Multi-Stage Matters	687
The Production Multi-Stage Pattern	687
Static Linking Essentials	688
Scratch vs Distroless vs Alpine	689
CA Certificates and Timezones	689
Non-Root, Health, and Signals	690
Build Cache and Layer Hygiene	691
Embed Version Metadata	692
Production Checklist	692
Common Pitfalls	693
Exercises	693
More examples	693
Health probe and graceful drain (container contract)	693
Bind address from env (not localhost-only)	695
Runnable example	696
Related Topics	698
 Platform Guides: Render & Leapcell	 700
Deploying Go: Render, Leapcell, and VPS	701
Overview	701
Deployment Models Compared	701
Render (render.com)	701
Typical setup	702
Production build command	702
Environment and ports	702
Databases and private networking	703
Pros and cons	703

Leapcell (and Serverless Go)	703
Conceptual difference from Lambda	704
Config sketch	704
Design for cold starts	704
VPS Deployment with systemd	705
1. Provision a non-root user	705
2. Install the binary	705
3. Environment file	705
4. Unit file	705
5. Reverse proxy (Caddy or nginx)	706
Health, Metrics, and Zero-Downtime	706
Secrets Hygiene	707
Choosing a Path	707
Production Checklist	707
Common Pitfalls	707
Exercises	708
More examples	708
PORT injection and dual health endpoints	708
Config from env file shape (KEY=VALUE)	710
Runnable example	711
Related Topics	714
Go on NixOS & FreeBSD	715
Beyond Linux: NixOS & FreeBSD	716
Overview	716
Cross-Compilation Basics	716
Matrix build script	717
Go on NixOS	717
Pure Go: the happy path	717
Development shell with Flakes	717
Packaging a Go module with Nix	718
CGO on NixOS	719
Go on FreeBSD	719
Runtime notes	719
Build on FreeBSD	719
Jails and networking	720
rc.d service sketch	720
Portable Go Code Tips	721
Build tags for OS-specific files	721
Production Checklist	722
Common Pitfalls	722
Exercises	722
More examples	723
Portable data directory (XDG + HOME fallback)	723
Build identity without CGO surprises	724
Runnable example	725
Related Topics	726

CI/CD & Hardening	727
CI/CD Pipelines & Security Hardening	728
Overview	728
Goals of a Go Pipeline	728
Baseline GitHub Actions Workflow	728
Hardening the workflow itself	729
Release Job Sketch	730
Hardening the Binary	731
Container Image Pipeline	731
Signing with Cosign (Sigstore)	731
Go-Specific Quality Gates	732
Example test job matrix	732
Supply Chain Mini-Checklist	732
Application Snippet: Fail Build on Bad Config	733
Production Checklist	733
Common Pitfalls	734
Exercises	734
More examples	734
Race-prone counter vs fixed counter	734
Module sum verification helper	736
Runnable example	737
Related Topics	739
Building and Releasing Software	740
Overview	740
Basic Build	740
Production Build	740
Version Injection	740
Why CGO_ENABLED=0 is part of a release	741
Cross-Compilation	741
Docker	741
GitHub Actions	742
GoReleaser	742
Summary	742
More examples	743
Version and commit via ldflags	743
Build info from the binary itself	744
Runnable example	744
Related Topics	746
Kubernetes Deploy Basics for Go	747
Kubernetes Deploy Basics for Go	748
Overview	748
Deployment sketch	748
Service	749
Graceful stop	749

Config & secrets	749
Rules of thumb	749
Try next	750
GoReleaser and GitHub Actions	751
GoReleaser and GitHub Actions	752
Overview	752
Minimal <code>.goreleaser.yaml</code>	752
Workflow sketch	752
Local snapshot	753
Rules of thumb	753
Try next	753
Container Health and Signals	754
Container Health and Signals	755
Overview	755
Dockerfile healthcheck (optional)	755
Signal PID 1	755
Read-only rootfs	755
12-factor in containers	756
Rules of thumb	756
Try next	756
Security	757
Security Tools	758
Security Tools: Automating Defense	759
Overview	759
Suggested Security Pipeline	759
1. <code>govulncheck</code> — Reachability-Based Vuln Scanning	759
CI snippet	760
Interpreting results	760
2. <code>gosec</code> — AST Security Linter	760
Safer random and SQL examples	761
3. <code>capslock</code> — Capability Analysis	761
4. <code>nilaway</code> — Static Nil Flows	761
5. Modern Crypto Tooling	762
<code>crypto/hpke</code> (Hybrid Public Key Encryption)	762
<code>testing/cryptotest</code>	763
6. Experimental: <code>runtime/secret</code>	764
7. Complementary Checks	764
Wiring a Makefile / CI Target	765
Production Checklist	765
Common Pitfalls	765
Exercises	765

More examples	766
Constant-time compare (HMAC / tokens)	766
Path jail (block traversal)	767
Runnable example	768
Related Topics	770
Application Hardening	771
Application Hardening	772
Why Hardening Matters	772
Core Principles	772
Hardening the Process	773
Drop privileges early	773
Container baseline	774
Hardening the HTTP Surface	774
Timeouts and body limits	774
Separate admin and data planes	775
Security headers and TLS defaults	776
Input Validation and Injection Defense	777
SQL: parameters only	777
Paths: root-jail every user-supplied path	777
Commands: never shell out with untrusted strings	777
Randomness and tokens	778
Secrets and Configuration	778
Dependency and Supply-Chain Hardening	778
Runtime Abuse Controls	779
Rate limiting (simple token bucket sketch)	779
Panic recovery at the edge	779
Production Hardening Checklist	780
Common Pitfalls	780
Exercises	781
Further Reading	781
More examples	781
Body limit + security headers middleware	781
Fail-closed required config	783
Runnable example	783
SOLID in Go	788
SOLID Principles in Go	789
Overview	789
Quick Map	789
S — Single Responsibility	789
O — Open/Closed	790
L — Liskov Substitution	791
I — Interface Segregation	792
Bad — fat interface	792
Good — small consumer interface	792

D — Dependency Inversion	793
Composition Over Inheritance	794
Package Design Checklist (SOLID-flavored)	794
Anti-Patterns	795
When <i>Not</i> to Abstract	795
Production Checklist	795
Common Pitfalls	795
Exercises	796
More examples	796
Depend on interfaces; swap fakes	796
Functional options for constructors	797
Runnable example	798
Related Topics	801
Specialized	802
Desktop Apps	803
Desktop Apps: Wails, Fyne, and Gio	804
Overview	804
Choosing a Stack	804
Wails — Electron Alternative Without Chromium	805
Project shape	805
Binding Go to JavaScript	805
Events both ways	806
Production notes for Wails	806
Fyne — Custom Widget Toolkit	807
Form and validation sketch	807
Fyne production notes	808
Gio — Immediate Mode	808
Shared Backend Pattern	808
Packaging and Distribution	809
Security Considerations	809
Production Checklist	810
Common Pitfalls	810
Exercises	810
More examples	811
UI-free domain service (testable core)	811
Safe open under app data root	812
Runnable example	813
Related Topics	815
Machine Learning & LLMs	816
AI & LLMs in Go	817
Overview	817
Architecture Split	817

Calling Local Models (Ollama)	817
Minimal raw HTTP client	818
Streaming Tokens to Clients	819
RAG Pipeline in Go	820
Embedding + top-k stub	820
Prompt assembly	821
Tool / Function Calling Agents	821
SIMD and Local Vector Math (Experimental)	822
Reliability Patterns	822
Hosted APIs	822
Production Checklist	823
Common Pitfalls	823
Exercises	823
More examples	824
SSE token stream with cancel	824
Tool registry with JSON args	825
Runnable example	826
Related Topics	830
Systems Programming	831
Systems Programming: Low-Level Go	832
Overview	832
When to Drop Below the Stdlib	832
Syscalls with <code>golang.org/x/sys</code>	832
File locking	833
Raising file descriptor limits (process)	833
Debugging with Delve	833
Core dumps	834
Remote debugging sketch	834
eBPF Userspace in Go	834
unsafe for Zero-Copy Views	835
Slice header caution	836
mmap for Large Read-Only Data	836
cgo Boundary (Reminder)	836
Production Checklist	837
Common Pitfalls	837
Exercises	837
More examples	837
Length-prefix binary frame (encode/decode)	837
Little-endian uint64 round-trip	839
Runnable example	839
Related Topics	841
I/O and Streaming	842
I/O and Streaming	843
Overview	843

Core Interfaces	843
Reading	844
HTTP bodies	844
Writing	845
Copying Streams	845
Progress reader	846
Composition Helpers	846
bufio	846
bytes.Buffer and strings.Reader	847
Pipes Between Goroutines	847
Streaming HTTP Upload / Download	848
gzip Stream	848
Checklist	849
Common Pitfalls	849
Exercises	849
More examples	850
TeeReader: copy + checksum one pass	850
Line scanner with buffer cap	851
Runnable example	851
Related Topics	854
Logging and Observability	855
Logging and Observability	856
Overview	856
The Three Pillars	856
Standard log Package	856
Structured Logging with slog	857
Levels	857
Child loggers and attributes	857
Context-aware logging	858
Custom log.Logger (Legacy / Libraries)	858
Metrics with expvar	859
Runtime Tracing and pprof	859
Error Logging Hygiene	860
Sampling and Cardinality	860
OpenTelemetry (When You Need It)	861
Production Checklist	861
Common Pitfalls	861
Exercises	861
More examples	862
slog JSON to a buffer (testable logs)	862
expvar counter + debug vars snapshot	862
Runnable example	863
Related Topics	866

Network Systems	867
Network Systems Overview	868
Network Systems Overview	869
Why This Part Exists	869
Learning Path	869
Core Concepts Map	870
1. Streams vs messages	870
2. Deadlines are load shedding	870
3. Budgets flow downward	870
4. Proxies are control planes	871
Go-Centric Building Blocks	871
Idiom: always bound the server	871
Idiom: always bound the client	871
Production Mindset	872
Relationship to Other Parts	872
How to Study These Chapters	872
Checklist: Network Systems Literacy	873
Exercises (Part Warm-Up)	873
Failure Taxonomy (Shared Vocabulary)	873
Lab Setup Recommendation	874
Anti-Patterns to Unlearn	874
Glossary	874
More examples	874
Tour: dial timeout + httptest client budget	874
Runnable example	876
Next Chapter	877
TCP Services Deep Dive	878
TCP Services Deep Dive	879
Why Custom TCP Still Matters	879
Core Mental Model	879
Framing Strategies	879
Length-prefix (recommended baseline)	880
Deadlines and Idle Connections	881
Accept Loop with Backpressure	881
IO vs Business Logic Separation	883
Protocol Errors vs Transport Errors	884
Keepalives and Half-Open Connections	884
TLS on TCP	884
Observability	885
Production Checklist	885
Common Pitfalls	885
Complete Mini Server Skeleton	886
Exercises	886

More examples	887
Length-prefix TCP session (client + server)	887
Per-connection deadlines	889
Runnable example	889
Further Reading	892
HTTP Client and Server Resilience	893
HTTP Client and Server Resilience	894
Why / Overview	894
Server-Side Guardrails	894
Configure the server, not just the handler	894
Handler-level budgets	895
Limit concurrency of expensive handlers	895
Body limits	896
Client-Side Guardrails	896
Never use the naked default client in production	896
Context-aware requests	897
Failure Classification	897
Retries Without Amplification	898
Transport Pooling and HTTP/2	899
Graceful Shutdown	899
Observability Hooks	900
Production Checklist	900
Common Pitfalls	900
Exercises	901
More examples	901
Client with transport timeouts (not zero-value)	901
Retry only on transient status	902
Runnable example	903
Further Reading	906
Reverse Proxy and Load Balancing	907
Reverse Proxy and Load Balancing	908
Why / Overview	908
Building a Reverse Proxy in Go	908
Single upstream	908
Director customization	909
Routing Model	909
Upstream Selection and Load Balancing	910
Round-robin baseline	910
Health awareness	911
Other LB algorithms (when RR is not enough)	911
Timeout and Retry Boundaries	912
Header Hygiene and Trust	912
Buffering vs Streaming	913
Observability at the Edge	913

Production Checklist	913
Common Pitfalls	913
Minimal Multi-Upstream Example	914
Exercises	914
More examples	915
Round-robin reverse proxy (stdlib)	915
Skip unhealthy upstream	916
Runnable example	917
Further Reading	920
TLS Clients and Servers	921
TLS Clients and Servers	922
Overview	922
HTTPS server (file certs)	922
Custom tls.Config	922
Client verification	922
mTLS sketch	923
Pinning (careful)	923
Rules of thumb	923
Try next	923
DNS Resolution Patterns	924
DNS Resolution Patterns	925
Overview	925
Default resolver	925
Custom resolver	925
Dialer and Happy Eyeballs	925
Caching	926
Failure modes	926
Minimal dig-like	926
Rules of thumb	926
Try next	926
HTTP Transport Tuning	927
HTTP Transport Tuning	928
Overview	928
Sensible client	928
One client per process	928
Timeouts layers	929
Disable keep-alives (rare)	929
HTTP/2	929
Rules of thumb	929
Try next	929
Project: netcheck Toolkit	930

Project: netcheck Toolkit	931
Overview	931
Commands	931
Requirements	931
Acceptance	931
 Systems Programming	 932
Systems Programming Overview	933
 Systems Programming Overview	 934
Why Systems Programming in a Go Book?	934
Learning Path	934
Core Concepts Map	935
Process as a state machine	935
The filesystem is not a database (but you can still be careful)	935
CLI tools are network services for humans and scripts	936
Go Building Blocks	936
Suggested schedule	936
Modern signal handling skeleton	936
Production Mindset	937
Relationship to Other Parts	937
How to Study	937
Literacy Checklist	937
Part Warm-Up Exercises	938
Platform Notes (Unix-first)	938
Container and CI Reality	938
Mini Project Thread	939
Failure Stories Worth Remembering	939
Checklist Before Shipping a Systems Tool	939
Sample Directory Layout	940
How This Part Connects to Network Services	940
Glossary	940
More examples	940
Tour: signal context + atomic temp file	940
Runnable example	942
Related in this book	943
Next Chapter	943
 Go as a Systems Language	 944
 Go as a Systems Language	 945
Why this belongs in a Go book	945
How a C program depends on libc	946
The “not found” surprise	946
How a Go program reaches the kernel	946
What is actually inside	947
Linux vs other kernels	948

CGO puts libc back in the picture	948
Cross-compilation is a systems feature	948
Trade-offs: Go vs C for systems work	949
Why teams pick Go	949
What you pay	949
Choose C when	950
Choose Go when	950
Literacy checklist	950
Exercises	950
Runnable example	951
Related in this book	952
Next Chapter	952
Processes, Signals, and Supervisors	953
Processes, Signals, and Supervisors	954
Why / Overview	954
Signals You Must Handle	954
Graceful Shutdown Pattern	955
Two-phase stop	956
Exit Codes as API	956
Child Processes	957
Always use context	957
Avoid shell	957
Process groups	958
Supervisor Policies	958
Readiness vs Liveness (process view)	958
Observability for Process Lifecycle	958
PID 1 and Containers	959
Production Checklist	959
Common Pitfalls	959
Exercises	959
More examples	960
HTTP graceful shutdown on signal context	960
Exit codes for CLI contract	961
Runnable example	962
Further Reading	964
Filesystem Automation and Safe IO	965
Filesystem Automation and Safe IO	966
Why / Overview	966
Atomic Replace Pattern	966
Path Safety (Root Jail)	968
Safe Read Patterns	968
Permissions and Umask	969
Directory Creation	969
Check-Then-Act Races (TOCTOU)	969

Streaming Copy and Spool	970
Walking Trees	970
Embed for Static Assets	971
Durability vs Speed Tradeoffs	971
Production Checklist	971
Common Pitfalls	971
Exercises	972
More examples	972
Atomic write via temp + rename	972
Walk files, skip dirs and symlinks to dirs	973
Runnable example	974
Further Reading	977
Unix Pipeline Style CLI Tools	978
Unix Pipeline Style CLI Tools	979
Why / Overview	979
The Pipeline Contract	979
Input Sources	980
Streaming Filters	980
Output Modes: Human vs Machine	980
Flags and Subcommands	981
Exit Codes	982
Handling SIGPIPE	982
Context and Cancellation	982
Structure of a Solid Small Tool	983
Progress and Logging	983
Composition Examples	983
Production Checklist (CLI)	983
Common Pitfalls	984
Mini Implementation: line counter	984
Exercises	985
More examples	985
Stdin filter: prefix match	985
Exit code when no matches (grep-like)	986
Runnable example	987
Further Reading	989
Environment, User Identity, CWD, and umask	990
Environment, User Identity, CWD, and umask	991
Overview	991
Environment variables	991
Patterns	991
PATH and LookPath	992
User and group identity (Unix)	992
Current working directory	992
Binary-relative assets	993

Children and Dir	993
umask and file permissions	993
Hostname and process identity	993
Minimal example: show process context	994
Rules of thumb	994
Try next	994
File Descriptors and Redirection	995
File Descriptors and Redirection	996
Overview	996
The classic trio	996
Opening files yields FDs	996
Inheritance across exec	996
Pipes as FDs	997
Redirection patterns in Go	997
Capture child stdout	997
Stream child to parent stdout	997
Tee logs	998
Checking TTY vs pipe	998
Limits and EMFILE	998
/dev/null and discard	998
Minimal tool: <code>fdcount</code> sketch	998
Rules of thumb	999
Try next	999
Memory-Mapped Files	1000
Memory-Mapped Files	1001
Overview	1001
Mental model	1001
Read-only map (Unix sketch)	1001
Write map (careful)	1003
Anonymous maps	1003
Safety with Go	1003
When mmap loses	1003
Minimal CLI: <code>mmapsum</code>	1004
Rules of thumb	1004
Try next	1004
Unix Domain Sockets and Local IPC	1005
Unix Domain Sockets and Local IPC	1006
Overview	1006
Stream server and client	1006
Lifecycle of the socket file	1007
Abstract namespace (Linux)	1007
Datagram unixgram	1007
HTTP over Unix sockets	1007

Credentials and peer auth (advanced)	1008
Timeouts	1008
Minimal tool pair: <code>udsecho</code>	1008
Rules of thumb	1008
Try next	1009
File Locks, Pidfiles, and Single-Instance Tools	1010
File Locks, Pidfiles, and Single-Instance Tools	1011
Overview	1011
Advisory flock (Unix)	1011
Pidfiles (classic, racy)	1012
Slightly better pidfile	1013
Combining lock + pid	1013
Cross-process mutex via lock directory (portable sketch)	1013
Signal-safe release	1013
Minimal CLI: <code>once</code>	1014
Rules of thumb	1014
Try next	1014
Project: Systems Mini-Tools	1015
Project: Systems Mini-Tools	1016
Overview	1016
Scaffold	1016
1. <code>id</code> — process context	1016
2. <code>watchdog</code> — supervise a child	1017
3. <code>atomic-write</code>	1018
4. <code>lines</code> — pipeline filter	1019
5. <code>once</code> — single instance	1019
6. <code>serve-sock</code> — local HTTP	1019
7. <code>tailf</code> — follow file growth	1020
Acceptance checklist	1021
Layout recap	1021
What you practiced	1021
Time, Clocks, and Timers	1023
Time, Clocks, and Timers	1024
Overview	1024
Wall clock vs monotonic	1024
Sleep and timers	1024
Stop and drain	1025
Ticker	1025
Deadlines with context	1025
Parsing and formatting	1026
File mtimes and caching	1026
Minimal tool: <code>deadline</code>	1026
Rules of thumb	1026

Try next	1027
Filesystem Watching and Reload	1028
Filesystem Watching and Reload	1029
Overview	1029
Polling watcher (stdlib only)	1029
Debounce	1030
fsnotify sketch (common library)	1031
SIGHUP reload	1031
Safe reload checklist	1032
Minimal tool: <code>reload-demo</code>	1032
Rules of thumb	1033
Try next	1033
Pseudo-Terminals (PTY)	1034
Pseudo-Terminals (PTY)	1035
Overview	1035
TTY detection (stdlib)	1035
Why pipes are not enough	1035
Start a command under a PTY (creack/pty)	1036
Window size	1036
Scripted interaction	1037
When to avoid PTYs	1037
Minimal tool: <code>runTTY</code>	1037
Rules of thumb	1037
Try next	1037
Resource Limits, rlimit, and Go Knobs	1038
Resource Limits, rlimit, and Go Knobs	1039
Overview	1039
Discovering limits (Unix)	1039
Raising NOFILE (when allowed)	1039
Common failure modes	1039
Go runtime knobs	1040
Bound your own resource use	1040
Minimal tool: <code>limits</code>	1040
cgroups (conceptual)	1041
Rules of thumb	1041
Try next	1041
Privileges, Capabilities, and Least Privilege	1042
Privileges, Capabilities, and Least Privilege	1043
Overview	1043
Prefer external policy	1043
Check identity early	1043

Bind low ports without full root	1044
setuid binaries (avoid if possible)	1044
File permissions as privilege	1044
Secrets	1044
Minimal pattern: refuse unsafe start	1045
Rules of thumb	1045
Try next	1045
Syslog, Journal, and Ops Logging	1046
Syslog, Journal, and Ops Logging	1047
Overview	1047
Default modern pattern: stderr + supervisor	1047
When to use syslog	1047
Levels and severity	1048
Structured fields for ops	1048
Avoiding log loops	1048
PID 1 and stdout	1048
Minimal dual-sink setup	1049
Rules of thumb	1049
Try next	1049
Project: Systems Ops Toolkit	1050
Project: Systems Ops Toolkit	1051
Overview	1051
Scaffold	1051
1. deadline — run with timeout	1051
2. reload — demo config watcher	1052
3. limits — print runtime snapshot	1053
4. cronish — interval runner with flock	1053
5. healthsock — UDS health for local probes	1054
6. rotate — minimal size rotate (teaching only)	1054
7. envcheck — required environment	1055
Acceptance checklist	1055
Suggested implementation order	1055
Stretch	1056
systemd Socket Activation	1057
systemd Socket Activation	1058
Overview	1058
How FDs arrive	1058
Minimal accept (stdlib)	1058
Unit sketch	1059
Notify readiness	1059
Rules	1059
Try next	1060

Linux procfs Introspection	1061
Linux procfs Introspection	1062
Overview	1062
Useful paths	1062
Count FDs	1062
Read VmRSS	1062
Minimal tool: <code>proccself</code>	1063
Cgroup memory (containers)	1063
Rules	1063
Try next	1063
Project: Mini Process Supervisor	1064
Project: Mini Process Supervisor	1065
Overview	1065
Spec	1065
Requirements	1065
Signal fanout	1065
Log prefix	1066
Acceptance	1066
Stretch	1066
Distributed Infrastructure	1067
Distributed Infrastructure Overview	1068
Distributed Infrastructure Overview	1069
Why / Overview	1069
Learning Path	1069
Core Mental Models	1070
Partial failure	1070
Amplification	1070
Backpressure vs buffering	1070
Go Building Blocks	1071
Production Principles	1071
Relationship to Other Parts	1071
Literacy Checklist	1071
Part Warm-Up Exercises	1072
Worked Example: Amplification Math	1072
Decision Guide	1072
What This Part Will Not Cover	1073
Incident Starter Questions	1073
Reference Architecture (Mental Model)	1073
Configuration Knobs Inventory	1073
Testing Strategy	1074
Glossary	1074

More examples	1074
Tour: health gate + retry budget	1074
Runnable example	1075
Next Chapter	1078
Service Discovery and Health Checks	1079
Service Discovery and Health Checks	1080
Why / Overview	1080
Discovery Mechanisms	1080
Registration and TTL	1080
Liveness vs Readiness vs Startup	1081
Warmup	1082
Client-Side Discovery Pool	1082
Passive Health and Outlier Detection	1083
Consistency and Split-Brain	1083
Security Notes	1084
Observability	1084
Production Checklist	1084
Common Pitfalls	1085
Exercises	1085
More examples	1085
In-memory registry with health filter	1085
/healthz and /readyz with dependency flag	1087
Runnable example	1088
Further Reading	1090
Retry and Circuit Breaker Patterns	1091
Retry and Circuit Breaker Patterns	1092
Why / Overview	1092
Error Classification	1092
Retry with Exponential Backoff and Jitter	1093
Policy parameters	1094
Decorrelated jitter (alternative)	1094
Idempotency Keys	1094
Circuit Breaker Implementation (Educational)	1094
Combining Retry and Breaker	1096
Budgets and Hedging	1096
Multi-Layer Retry Coordination	1096
Observability	1097
Production Checklist	1097
Common Pitfalls	1097
Exercises	1097
More examples	1098
Classify errors before retrying	1098
Budget-aware retry (honor parent deadline)	1099
Runnable example	1100

Further Reading	1103
Queue Workers and Backpressure	1104
Queue Workers and Backpressure	1105
Why / Overview	1105
Delivery Semantics	1105
In-Process Worker Pool	1106
Blocking vs dropping vs rejecting	1107
Sizing Levers	1107
External Queue Worker Loop (Shape)	1108
Ack timing	1108
Poison Messages and DLQ	1108
Backpressure Across Tiers	1109
Graceful Shutdown	1109
Observability	1109
Production Checklist	1110
Common Pitfalls	1110
Exercises	1110
More examples	1110
Bounded queue + worker pool	1110
Drop vs block when full (load shed)	1112
Runnable example	1112
Further Reading	1115
Caching Patterns	1116
Caching Patterns	1117
Overview	1117
In-process TTL map	1117
Single-flight (stampede control)	1118
Cache-aside	1118
Negative caching	1118
Rules of thumb	1118
Try next	1118
Idempotency and Deduplication	1119
Idempotency and Deduplication	1120
Overview	1120
Idempotency-Key header	1120
DB unique constraints	1121
At-least-once consumers	1121
Rules of thumb	1121
Try next	1121
Leader Election Basics	1122

Leader Election Basics	1123
Overview	1123
Lease pattern (DB)	1123
Kubernetes	1124
Rules	1124
Try next	1124
Transactional Outbox Pattern	1125
Transactional Outbox Pattern	1126
Overview	1126
Schema sketch	1126
Write path	1126
Relay worker	1127
At-least-once	1127
Rules	1127
Try next	1127
Project: Resilient Worker	1128
Project: Resilient Worker	1129
Overview	1129
Features	1129
Interface	1129
Loop	1130
Acceptance	1130
Observability and SRE	1131
Observability and SRE Overview	1132
Observability and SRE Overview	1133
Why / Overview	1133
Learning Path	1133
The Observability Triangle	1134
SLI / SLO Primer	1134
Go Building Blocks	1134
Production Principles	1134
Relationship to Other Parts	1135
Literacy Checklist	1135
Part Warm-Up Exercises	1135
Instrumentation Order of Operations	1135
What “Good” Looks Like in an Incident	1136
Cardinality and Cost Budget	1136
Anti-Patterns	1136
Minimum Dashboard Set	1136
Alert Quality Bar	1137
Library Choices (2024–2026 pragmatic)	1137

Glossary	1137
More examples	1137
Tour: slog + counter + request id	1137
Runnable example	1139
Next Chapter	1140
Structured Logging and Correlation	1141
Structured Logging and Correlation	1142
Why / Overview	1142
slog Fundamentals	1142
Loggers with attributes	1143
Correlation IDs	1143
Context-aware logging helpers	1144
Stable Schema	1144
What to Log	1144
Redaction and Security	1145
Sampling and Volume	1145
Goroutines and Lost Context	1145
slog Handlers and Production	1146
Testing Logs	1146
Production Checklist	1146
Common Pitfalls	1146
Exercises	1147
More examples	1147
slog groups and default attrs	1147
Child logger from context request id	1148
Runnable example	1149
Further Reading	1151
Metrics and Prometheus Design	1152
Metrics and Prometheus Design	1153
Why / Overview	1153
RED and USE	1153
Metric Types	1154
Instrumentation Example	1154
Label Strategy (Cardinality)	1155
Histogram Buckets	1155
Naming Conventions	1156
Outbound Client Metrics	1156
Exemplars and Trace Links	1156
Alerting from Metrics	1156
Runtime Metrics	1157
Production Checklist	1157
Common Pitfalls	1157
Exercises	1158

More examples	1158
Prometheus text exposition (stdlib)	1158
Labeled counter map (cardinality caution)	1159
Runnable example	1160
Further Reading	1164
Tracing and Context Propagation	1165
Tracing and Context Propagation	1166
Why / Overview	1166
Core Concepts	1166
Context Propagation Rules	1166
Minimal HTTP Propagation (Conceptual)	1167
Span Design	1167
Sampling	1168
Common Failure Modes	1168
Queue propagation	1168
Correlation with Logs and Metrics	1168
In-Process Performance Note	1169
SDK Lifecycle (Shape)	1169
HTTP middleware and transport (conceptual)	1169
Baggage vs Attributes	1170
Reading a Trace During an Incident	1170
Production Checklist	1170
Common Pitfalls	1170
Exercises	1171
End-to-End Lab Outline	1171
More examples	1171
Trace/span ids on context and outbound headers	1171
Nested span timing (in-process)	1173
Runnable example	1174
Further Reading	1176
SLIs, SLOs, and Error Budgets	1177
SLIs, SLOs, and Error Budgets	1178
Overview	1178
Definitions	1178
Good SLIs for HTTP APIs	1178
Expressing latency as an SLI	1178
Error budget policy (lightweight)	1179
Minimal Go: count success/fail	1179
Rules of thumb	1179
Try next	1179
Health, Liveness, and Readiness Probes	1180
Health, Liveness, and Readiness Probes	1181
Overview	1181

Minimal endpoints	1181
Dependency checks	1181
Drain on shutdown	1182
Startup probe (K8s)	1182
Rules of thumb	1182
Try next	1182
Alerting and Runbooks	1183
Alerting and Runbooks	1184
Overview	1184
Alert design	1184
Severity	1184
Runbook template	1185
Linking code to ops	1185
Rules of thumb	1185
Try next	1185
Project: Instrumented Mini Service	1187
Project: Instrumented Mini Service	1188
Overview	1188
Checklist	1188
Stretch	1188
Acceptance	1188
Security Hardening	1190
Security Hardening Overview	1191
Security Hardening Overview	1192
Why / Overview	1192
Learning Path	1192
Core Principles	1193
Go Building Blocks	1193
Relationship to Other Parts	1193
Literacy Checklist	1193
Part Warm-Up Exercises	1194
Threat Model Snapshot (Service-Centric)	1194
Layering with Part 11	1194
Minimal Secure Service Skeleton	1195
Compliance-Adjacent Habits (Not Legal Advice)	1195
Secure Defaults Checklist (Part Preview)	1195
Testing Security Logic	1195
Where Identity Lives	1196
Mapping Incidents → Chapter	1196
Non-Goals	1196
Glossary	1197

More examples	1197
Tour: bearer check + TLS client roots sketch	1197
Runnable example	1198
Next Chapter	1200
TLS and mTLS Deep Dive	1201
TLS and mTLS Deep Dive	1202
Why / Overview	1202
Go Server TLS	1202
Cipher suites	1203
Dynamic Certificate Reload	1203
Client TLS	1204
mTLS	1204
Identity from client cert	1205
Trust Bundles and Rotation	1205
HTTP/2 and ALPN	1205
Proxies and TLS	1206
Production Checklist	1206
Common Pitfalls	1206
Exercises	1206
Certificate File Permissions Checklist	1207
Handshake Failure Debugging Order	1207
More examples	1207
TLS server + RootCAs client (httptest)	1207
Inspect peer cert fields in handler	1208
Runnable example	1209
Further Reading	1211
Authentication and Authorization Patterns	1212
Authentication and Authorization Patterns	1213
Why / Overview	1213
Authentication Patterns	1213
1. Session cookies (first-party browsers)	1213
2. Bearer tokens (APIs)	1214
3. mTLS identity	1214
4. API keys	1214
Principal Model	1214
Authorization Patterns	1215
RBAC (role-based)	1215
Resource ownership (critical for IDOR)	1215
Policy as pure function	1216
Middleware composition	1216
Deny by Default	1217
Multi-Tenant Awareness	1217
Audit Logging	1217
Testing Matrix	1217

Production Checklist	1218
Common Pitfalls	1218
Exercises	1218
More examples	1219
Bearer authn middleware	1219
Role-based authz gate	1220
Runnable example	1221
Further Reading	1224
Secrets Management and Rotation	1225
Secrets Management and Rotation	1226
Why / Overview	1226
Never Do This	1226
Loading Secrets at Runtime	1226
Environment variables	1226
Files (Kubernetes secrets mounts)	1227
Secret managers	1227
Fail Closed at Startup	1227
Dual-Read Rotation (Passwords / HMAC)	1228
Runtime Refresh Without Restart	1229
Encryption Keys vs Passwords	1229
CI/CD and Build-Time Leaks	1230
Redaction Everywhere	1230
Production Checklist	1230
Common Pitfalls	1230
Exercises	1231
More examples	1231
Dual-secret accept during rotation	1231
Load secret from file with restrictive mode check	1232
Runnable example	1233
Further Reading	1236
JWT and Token Validation	1237
JWT and Token Validation	1238
Overview	1238
Minimal validation checklist	1238
Access vs refresh	1238
Service-to-service	1239
Rules of thumb	1239
Try next	1239
Input Validation and SSRF	1240
Input Validation and SSRF	1241
Overview	1241
Validation basics	1241
Path traversal	1241

SSRF defenses	1242
Header injection	1242
Rules of thumb	1242
Try next	1243
Supply Chain and Dependencies	1244
Supply Chain and Dependencies	1245
Overview	1245
Minimal practices	1245
go.mod hygiene	1245
Vendoring (optional)	1245
Reproducible builds	1246
Private modules	1246
Rules of thumb	1246
Try next	1246
Project: Secure Service Checklist	1247
Project: Secure Service Checklist	1248
Overview	1248
Transport & process	1248
Authn / authz	1248
Input / output	1248
Secrets & deps	1249
Deliverable	1249
Performance Engineering	1250
Performance Engineering Overview	1251
Performance Engineering Overview	1252
Why / Overview	1252
Learning Path	1252
Core Principles	1252
Go Tooling Map	1253
Relationship to Other Parts	1253
Literacy Checklist	1253
Part Warm-Up Exercises	1253
Performance vs Correctness vs Cost	1254
When Not to Optimize	1254
Environment Parity	1254
Suggested Lab Sequence	1254
Regression Guardrails	1255
Interaction with Timeouts	1255
Team Norms Worth Adopting	1255
Mapping Symptoms → Chapter	1255
Definition of Done for a Perf Change	1256

Quick Command Card	1256
Glossary	1256
More examples	1257
Tour: timed work + allocation awareness	1257
Runnable example	1257
Next Chapter	1259
Benchmarking and Profiling Workflow	1260
Benchmarking and Profiling Workflow	1261
Why / Overview	1261
Writing Good Benchmarks	1261
Parallel benchmarks	1262
Running and Comparing	1262
Profiling from Benchmarks	1262
Profiling a Running Service	1263
Mutex and block profiles	1263
Execution Tracer	1263
Load Testing Complements Microbenches	1263
Workflow Playbook	1264
CI Integration	1264
Production Checklist	1264
Common Pitfalls	1264
Exercises	1265
Interpreting Flame Graphs Quickly	1265
Production Capture Safety	1265
More examples	1265
Mini benchmark harness (stdlib testing style)	1265
pprof HTTP handler registration (httptest)	1267
Runnable example	1267
Further Reading	1269
Memory, Allocations, and GC	1270
Memory, Allocations, and GC	1271
Why / Overview	1271
Stack vs Heap (Practical View)	1271
Reading Allocation Metrics	1272
Hot-Path Techniques	1272
1. Preallocate slices	1272
2. Reuse buffers	1272
3. Avoid <code>fmt</code> in hot paths	1273
4. Beware of <code>[]byte(string)</code> / <code>string([]byte)</code> conversions	1273
5. Interfaces and methods	1273
<code>sync.Pool</code>	1273
String Builders	1274
Maps and Reuse	1274

GC Tuning Knobs	1274
GOGC	1274
GOMEMLIMIT (Go 1.19+)	1274
Ballast (historical)	1275
Symptoms of Allocation Problems	1275
Production Checklist	1275
Common Pitfalls	1275
Exercises	1275
Leak Patterns Specific to Go Services	1276
More examples	1276
Preallocate slice capacity	1276
sync.Pool reuse	1277
Runnable example	1278
Further Reading	1280
Contention and Concurrency Tuning	1281
Contention and Concurrency Tuning	1282
Why / Overview	1282
Sources of Contention	1282
Diagnostic Workflow	1282
Critical Section Hygiene	1283
RWMutex misuse	1283
Sharding	1283
Atomically Updated Stats	1284
Channel Contension and Pipelines	1284
Bound Fan-Out	1284
Connection Pools	1285
GOMAXPROCS and Containers	1285
False Sharing and Cache Lines (Advanced)	1285
Work Stealing and Fairness	1285
Production Checklist	1286
Common Pitfalls	1286
Exercises	1286
Throughput Curve Lab (Do This Once)	1286
Combining with Timeouts and Breakers	1287
More examples	1287
Hot mutex vs sharded counters	1287
Admission semaphore (shed load)	1288
Runnable example	1290
Further Reading	1292
Load Testing and Baselines	1293
Load Testing and Baselines	1294
Overview	1294
Tools	1294
Baseline template	1294

SLO-oriented load	1294
Avoiding test lies	1295
Rules of thumb	1295
Try next	1295
Allocation Hot-Path Recipes	1296
Allocation Hot-Path Recipes	1297
Overview	1297
Measure first	1297
Recipes	1297
1. Avoid <code>fmt</code> in hot loops	1297
2. Grow slices with known cap	1297
3. Reuse buffers	1297
4. <code>strings.Builder</code>	1298
5. Interface boxing	1298
6. JSON	1298
7. <code>io.Copy</code> buffer	1298
Pool discipline	1298
Rules of thumb	1298
Try next	1299
PGO and Production Build Flags	1300
PGO and Production Build Flags	1301
Overview	1301
Production build baseline	1301
PGO workflow (Go 1.20+)	1301
When PGO helps	1302
Rules of thumb	1302
Try next	1302
Project: Optimize a Hot Endpoint	1303
Project: Optimize a Hot Endpoint	1304
Overview	1304
Setup	1304
Steps	1304
Acceptance	1304
Stretch	1305
Modern Go Synthesis	1306
190 Modern Go Books (2024-2025) Index	1307
190 Modern Go Books (2024-2025) Index	1308
Selection Criteria	1308
Source-Derived Synthesis Chapters	1308

Cross-Book Theme Map	1308
More examples	1309
Tour: options + httptest handler + typed error	1309
Runnable example	1310
191 Patterns from Go Programming (2nd ed., 2024)	1312
191 Patterns from Go Programming (2nd ed., 2024)	1313
What This Adds to Our Book	1313
Learning Architecture	1313
Deep Integration Example: API + CLI + DB Boundary	1313
Why This Matters	1315
Curriculum Upgrades Recommended	1315
More examples	1315
Nested context budgets at the boundary	1315
Typed sentinel + wrap for handlers	1316
Runnable example	1317
192 Patterns from Effective Go Recipes (2024)	1319
192 Patterns from Effective Go Recipes (2024)	1320
What This Adds to Our Book	1320
Streaming-Oriented Mental Model	1320
Deep Integration Example: Streaming JSON Pipeline with Backpressure	1320
Why This Matters	1322
Curriculum Upgrades Recommended	1322
More examples	1323
Functional options recipe	1323
Table-driven validation recipe	1324
Runnable example	1325
193 Patterns from Go in Practice (2nd ed., 2025)	1328
193 Patterns from Go in Practice (2nd ed., 2025)	1329
What This Adds to Our Book	1329
End-to-End Service Model	1329
Deep Integration Example: External Service Client with Typed Error Strategy	1329
Why This Matters	1330
Curriculum Upgrades Recommended	1331
More examples	1331
Middleware chain composition	1331
Content-type gate for JSON POST	1332
Runnable example	1333
194 Patterns from Mastering Go (4th ed., 2024)	1336
194 Patterns from Mastering Go (4th ed., 2024)	1337
What This Adds to Our Book	1337
Advanced Engineering Loop	1337

Deep Integration Example: Fuzz-Ready Parser + Profile-Aware Fast Path	1337
Why This Matters	1338
Curriculum Upgrades Recommended	1338
More examples	1339
TCP echo with deadlines (systems flavor)	1339
Worker pool fan-out / fan-in	1340
Runnable example	1341
195 Patterns from Let Us Go! (2025)	1344
195 Patterns from Let Us Go! (2025)	1345
What This Adds to Our Book	1345
Onboarding Progression	1345
Deep Integration Example: Minimal Project Lifecycle from Day 1	1345
Why This Matters	1346
Curriculum Upgrades Recommended	1346
More examples	1346
Small CLI with flags and usage exit	1346
Debugging habit: log inputs and duration	1347
Runnable example	1348
196 Patterns from Automate Your Home Using Go (2024)	1351
196 Patterns from Automate Your Home Using Go (2024)	1352
What This Adds to Our Book	1352
Edge Automation Topology	1352
Deep Integration Example: Telemetry Collector with Health and Metrics Surface	1352
Why This Matters	1354
Curriculum Upgrades Recommended	1354
More examples	1354
Device event loop with context cancel	1354
State machine for a smart switch	1355
Runnable example	1357
Go Deep Dives	1360
Go Deep Dives Overview	1361
Go Deep Dives Overview	1362
Diagram: Deep-dive stack	1362
Who This Part Is For	1362
Learning Paths	1362
A — Runtime core	1362
B — Data representation	1363
C — Execution control	1363
D — Performance & GC	1363
E — Toolchain & platforms	1363
F — Production craft	1363

Full Chapter Map	1363
Stack Diagram	1365
Diagrams in this part	1365
Study Method	1365
Relationship to Other Parts	1365
Honesty Boundary	1366
Checklist: Deep-Dive Literacy	1366
Scheduler Deep Dive (GMP)	1367
Scheduler Deep Dive (GMP)	1368
Overview	1368
Diagram: GMP at a glance	1368
Lifecycle of a G (creation → park → run)	1369
Core Loop (Mental Model)	1369
GOMAXPROCS	1370
Preemption	1370
Hand-off and Syscalls	1370
Scheduling Latency Signals	1370
Design Rules From GMP	1371
Experiment	1371
Related	1372
Memory Model and Happens-Before	1373
Memory Model and Happens-Before	1374
Overview	1374
Diagram: Happens-before edges	1374
Data Race Definition	1374
Happens-Before (Core Edges)	1374
Incorrect: Sharing Without Sync	1375
Correct Patterns	1375
Mutex	1375
Channel hand-off (share by communicating)	1376
Atomic flag	1376
Init and Package-Level	1376
Compiler and Hardware Reordering	1376
Experiment	1376
Related	1378
Channel and Select Internals	1379
Channel and Select Internals	1380
Overview	1380
Diagram: Channel park / wake	1380
Channel Structure (Conceptual)	1380
Unbuffered (<code>make(chan T)</code>)	1381
Buffered (<code>make(chan T, n)</code>)	1381
Send / Receive Paths	1381

Select Algorithm (Intuition)	1381
Close Semantics	1382
Common Failure Modes	1382
Cost Model	1382
Experiment	1382
Related	1384
Interface Representation (eface / iface)	1385
Interface Representation (eface / iface)	1386
Overview	1386
Diagram: eface vs iface	1386
Two Runtime Shapes	1386
Dynamic Type and Dynamic Value	1387
Conversion and Escape	1387
Method Calls	1387
Type Assert and Type Switch	1388
Comparable Interfaces	1388
Experiment	1388
Related	1389
Slice and Map Internals	1390
Slice and Map Internals	1391
Overview	1391
Diagram: Slice header and map grow	1391
Slice Header	1391
Full slice expression	1392
Nil vs empty	1392
Map Runtime Model	1392
Semantics that bite	1392
Growth cost	1393
Sharing and Aliasing Checklist	1393
Experiment	1393
Related	1394
Defer, Panic, and Stack Unwind	1395
Defer, Panic, and Stack Unwind	1396
Overview	1396
Diagram: Panic unwind	1396
Defer Semantics	1396
Arguments evaluated immediately	1397
Named results	1397
Cost Model	1397
Panic Unwind	1398
What recover cannot do	1398
Stack Growth	1398
Experiment	1398

Related	1400
Generics Implementation Deep Dive	1401
Generics Implementation Deep Dive	1402
Overview	1402
Diagram: Generic call shapes	1402
What the Compiler Sees	1402
Cost Dimensions	1403
Constraints Are Interfaces	1403
Instantiation	1404
Interaction With Interfaces	1404
Avoid These Footguns	1404
Experiment	1404
Related	1405
GC Internals and Pacer	1406
GC Internals and Pacer	1407
Overview	1407
Diagram: GC cycle	1407
Heap Building Blocks	1408
Tri-color intuition	1408
Mark and Sweep Phases	1408
Write barrier	1408
Mark assist	1408
Pacer Goals	1408
GOMEMLIMIT	1409
Green Tea GC (1.26+)	1409
Reading GC Telemetry	1409
Latency pattern	1409
Design Rules	1410
Experiment	1410
Related	1411
Compiler, SSA, Inlining, and PGO	1412
Compiler, SSA, Inlining, and PGO	1413
Overview	1413
Diagram: Compiler pipeline	1413
Pipeline (Simplified)	1413
Essential Flags	1413
Inlining	1414
Bounds Check Elimination (BCE)	1414
Profile-Guided Optimization (PGO)	1415
What the Compiler Will Not Do	1415
Experiment	1415
Related	1416

Linking, Build Modes, and Race ABI	1417
Linking, Build Modes, and Race ABI	1418
Overview	1418
Diagram: Build modes	1418
Default Build	1418
Build Modes	1419
Build Tags and Constraints	1419
Linker Flags	1419
Race Detector	1419
cgo Boundary (Preview)	1420
Reproducible Builds	1420
Experiment	1420
Related	1421
Reflect Internals and Cost	1422
Reflect Internals and Cost	1423
Overview	1423
Diagram: Reflect path	1423
Core Types	1423
What Metadata Lives Where	1424
Allocations and Interface Boxing	1424
Method Calls via Reflect	1424
Unsafe Adjacent	1424
When Reflect Is Justified	1425
Experiment	1425
Related	1427
Modules, MVS, and Toolchain	1428
Modules, MVS, and Toolchain	1429
Overview	1429
Diagram: MVS selection	1429
Core Files	1429
Minimal Version Selection	1429
The Build List	1430
replace and exclude	1430
Retracts	1430
Toolchain Directive	1430
Vendoring	1431
Checksums and Proxies	1431
Diagnosis Playbook	1431
Experiment	1432
Related	1432
Netpoller and Network I/O	1433

Netpoller and Network I/O	1434
Overview	1434
Diagram: Netpoll park	1434
Mental Model	1434
Deadlines vs Poller	1435
Interaction With Syscalls	1435
Zero-Copy Adjacent	1435
Diagnosis	1435
Experiment	1436
Timers, Tickers, and time.After Pitfalls	1438
Timers, Tickers, and time.After Pitfalls	1439
Overview	1439
Diagram: Timer heap cost	1439
APIs	1439
The Classic Pitfall	1439
Tickers	1440
Runtime View	1440
Context Timeouts	1440
Experiment	1441
Stack Growth and Copying	1442
Stack Growth and Copying	1443
Overview	1443
Diagram: Stack growth	1443
Growth Path	1443
Why Deep Recursion Hurts	1443
cgo and Stacks	1444
Debugging Stacks	1444
Nosplit	1444
Experiment	1444
Finalizers and runtime.AddCleanup	1446
Finalizers and runtime.AddCleanup	1447
Overview	1447
Diagram: Lifecycle preference	1447
Prefer Explicit Lifecycle	1447
SetFinalizer (Legacy Pattern)	1447
AddCleanup (Preferred Direction)	1448
Rules of Thumb	1448
Experiment	1448
weak and unique Packages	1450
weak and unique Packages	1451
Overview	1451

Diagram: Weak vs strong	1451
weak.Pointer	1451
unique.Handle	1452
When Not To	1452
Experiment	1452
Swiss Maps and maphash	1454
Swiss Maps and maphash	1455
Overview	1455
Diagram: Map lookup path	1455
What Changed Conceptually	1455
Practical Advice (Unchanged)	1455
maphash	1456
Map Iteration and Deletes	1456
Experiment	1457
sync.Pool and Zero-Allocation Patterns	1458
sync.Pool and Zero-Allocation Patterns	1459
Overview	1459
Diagram: Pool get/put	1459
sync.Pool	1459
Zero-Alloc Techniques	1460
Measuring	1460
Anti-Patterns	1460
Experiment	1461
Goroutine Leaks and Diagnosis	1462
Goroutine Leaks and Diagnosis	1463
Overview	1463
Diagram: Leak patterns	1463
Common Leak Patterns	1463
Fix Templates	1464
Diagnosis Toolkit	1464
Experiment	1465
GODEBUG and Runtime Knobs	1467
GODEBUG and Runtime Knobs	1468
Overview	1468
Diagram: Knob layers	1468
Essential Environment	1468
GODEBUG Hits Worth Knowing	1468
GOTRACEBACK	1469
Programmatic Knobs	1469
Safety Rules	1469
Experiment	1469

pprof and Execution Tracer Internals	1471
pprof and Execution Tracer Internals	1472
Overview	1472
Diagram: Profile vs trace	1472
CPU Profile	1472
Heap / Alloc Profiles	1473
Block and Mutex Profiles	1473
Goroutine Profile	1473
Execution Tracer (<code>go tool trace</code>)	1473
Cost of Profiling	1474
Experiment	1474
testing/synctest: Bubbles, Durable Blocks, Fake Time	1475
testing/synctest: Bubbles, Durable Blocks, Fake Time	1476
Overview	1476
The problem it solves	1476
Public API (small overlay)	1477
The bubble	1477
Membership	1478
Root vs main	1478
Durable blocking = wait reason	1478
Two counters, one invariant	1479
Root event loop (fake time)	1480
time.Now inside a bubble	1481
Fake timers	1481
Channels: married at make	1481
Select	1481
WaitGroup association (heap special)	1482
sync.Cond	1482
Mutex: deliberately non-durable	1482
Deadlock detection (when root can prove it)	1483
Isolation cheat sheet	1483
Isolation is not a sandbox	1484
End-to-end narrative	1484
synctest.Wait and the race detector	1484
Practical rules	1485
Relation to other chapters	1485
Reading the source (same path as the article)	1485
Experiment	1486
Recap	1486
Zero-Copy I/O (sendfile, splice, Copy)	1487
Zero-Copy I/O (sendfile, splice, Copy)	1488
Overview	1488
Diagram: Copy fast path	1488
Copy Dispatch	1488

sendfile / splice (Linux)	1488
Practical Patterns	1489
Measuring	1489
Experiment	1489
HTTP Transport and Connection Pool	1491
HTTP Transport and Connection Pool	1492
Overview	1492
Diagram: Client stack	1492
Client Stack	1492
Pool Behaviors	1493
HTTP/2	1493
Server Side (Brief)	1493
Experiment	1494
Context Cancel Trees Internals	1495
Context Cancel Trees Internals	1496
Overview	1496
Diagram: Cancel flows down	1496
Tree Shape	1496
Implementation Intuition	1496
Values	1497
Error Taxonomy	1497
Experiment	1497
Mutex and Semaphore Runtime	1499
Mutex and Semaphore Runtime	1500
Overview	1500
Diagram: Lock fast/slow path	1500
Fast Path / Slow Path	1500
RWMutex	1501
Cond	1501
Atomics vs Mutex	1501
Avoid	1501
Experiment	1501
cgo and Scheduler Transitions (incl. Go 1.26)	1503
cgo and Scheduler Transitions (incl. Go 1.26)	1504
Overview	1504
Diagram: cgo call path	1504
Call Path (Conceptual)	1504
Costs	1505
Rules	1505
Debugging	1505
Relation To Non-cgo Syscalls	1505

Experiment	1505
Iterators and range-over-func Deep Dive	1507
Iterators and range-over-func Deep Dive	1508
Overview	1508
Diagram: Push iterator	1508
Types	1508
Desugaring Intuition	1508
Pull Iterators	1509
When Iterators Shine	1509
When They Don't	1509
Experiment	1509
ABI, Assembly, and go:nosplit	1511
ABI, Assembly, and go:nosplit	1512
Overview	1512
Diagram: ABI layers	1512
ABIs (High Level)	1512
go:nosplit	1512
go:uintptrescapes / go:uintptrkeepalive	1513
SIMD Experiments	1513
When To Write Asm	1513
Plan 9 listing and go tool objdump	1513
Experiment	1514
Coverage Instrumentation and Fuzz Engine	1515
Coverage Instrumentation and Fuzz Engine	1516
Overview	1516
Diagram: Fuzz loop	1516
Coverage	1516
Fuzz Engine Model	1517
Writing Good Fuzz Targets	1517
Coordination With Race	1517
Experiment	1517
Build Cache and Reproducibility	1519
Build Cache and Reproducibility	1520
Overview	1520
Diagram: Reproducible build	1520
Cache Location	1520
What Busts Cache	1520
Reproducible Flags	1521
Module Reproducibility	1521
Experiment	1521

GOEXPERIMENT Catalog	1522
GOEXPERIMENT Catalog	1523
Overview	1523
Diagram: Experiment lifecycle	1523
Rules	1524
JSON v2 and Friends	1524
Experiment	1524
WebAssembly and wasip1	1525
WebAssembly and wasip1	1526
Overview	1526
Diagram: Wasm targets	1526
Targets	1526
Constraints	1526
Use Cases	1526
Experiment	1527
String and []byte Internals	1528
String and []byte Internals	1529
Overview	1529
Diagram: Headers	1529
Headers	1529
Range and Indexing	1529
unsafe Conversions (Advanced)	1530
Builder and Buffer	1530
Experiment	1530
Error Interface Cost and Wrapping	1531
Error Interface Cost and Wrapping	1532
Overview	1532
Diagram: Error interface	1532
Representation	1532
Wrapping	1532
Performance Notes	1532
Sentinel vs Types	1533
Experiment	1533
Production Mistakes Deep Dive	1535
Production Mistakes Deep Dive	1536
Overview	1536
Diagram: Incident loop	1536
The List (Mapped)	1536
Incident Playbook	1537
Engineering Questions (From “knowing Go”)	1537

Mini Lab	1537
Production Debugging Tooling	1538
Production Debugging Tooling	1539
Overview	1539
Diagram: Debug surfaces	1539
Always-On Endpoints (Carefully)	1539
Signals	1539
Delve (Dev / Staging)	1540
Core / Crash	1540
Continuous Profiling	1540
Checklist	1540
Experiment	1540
sysmon, Scavenger, and Background Runtime	1542
sysmon, Scavenger, and Background Runtime	1543
Overview	1543
Diagram: Background runtime	1543
sysmon (System Monitor)	1543
Memory Scavenging	1543
GC Workers	1544
What You Control	1544
Experiment	1544
Reading the Go Runtime Source	1545
Reading the Go Runtime Source	1546
Overview	1546
Diagram: Source map	1546
Entry Points	1546
How To Study	1547
go tool objdump / nm	1547
Experiments Without Patching	1547
Experiments With Patching (Optional)	1547
Field Guide: This Book → Source	1547
Experiment	1548
crypto/tls Handshake Path	1549
crypto/tls Handshake Path	1550
Overview	1550
Diagram: TLS then HTTP	1550
Path	1550
Cost Drivers	1550
Client Integration	1551
Security Notes	1551
Experiment	1551

slog Handler Design Deep Dive	1553
slog Handler Design Deep Dive	1554
Overview	1554
Diagram: Logger vs Handler	1554
Interfaces	1554
Patterns	1555
Performance	1555
Experiment	1555
GOMAXPROCS, cgroups, and Containers	1557
GOMAXPROCS, cgroups, and Containers	1558
Overview	1558
Diagram: Quota mismatch	1558
Failure Mode	1558
Practices	1558
Memory Side	1559
Experiment	1559
Plugins and buildmode Pitfalls	1560
Plugins and buildmode Pitfalls	1561
Overview	1561
Diagram: Prefer process plugins	1561
Reality Check	1561
Prefer Alternatives	1561
If You Must	1562
Experiment	1562
Topic Radar (X + Ecosystem Signals)	1563
Topic Radar (X + Ecosystem Signals)	1564
Overview	1564
Diagram: Radar loop	1564
Hot Themes Observed	1564
Runtime & scheduler	1564
Correctness under concurrency	1564
Performance craft	1565
Production engineering	1565
Learning meta	1565
Mapped Into This Part	1565
Still-Fertile Backlog (Future)	1565
How To Extend This Book	1566
Experiment	1566
Article Radar: Internals for Interns + Kin	1567

Article Radar: Internals for Interns + Kin	1568
Overview	1568
Diagram: map sources → book chapters	1568
Internals for Interns — Runtime series	1568
Internals for Interns — Compiler series	1569
Kindred sources (same “diagram + source” energy)	1569
How we absorb articles	1570
Experiment	1570
Memory Allocator Deep Dive	1571
Memory Allocator Deep Dive	1572
Overview	1572
Diagram: three-level hierarchy	1572
Size classes and spans	1573
Arenas and address space	1573
Interaction with GC and P	1573
Escape analysis still wins	1573
Diagnosis	1573
Experiment	1574
Runtime Bootstrap	1575
Runtime Bootstrap	1576
Overview	1576
Diagram: from exec to main	1576
What gets set up	1576
Package init order	1577
Main goroutine	1577
Experiment	1577
select: Compiler and selectgo	1578
select: Compiler and selectgo	1579
Overview	1579
Diagram: two paths	1579
Rules that matter	1579
Pseudo-algorithm (teaching)	1579
Synctest note	1580
Experiment	1580
Stacktraces and Runtime Metadata	1581
Stacktraces and Runtime Metadata	1582
Overview	1582
Diagram: build time vs run time	1582
Who prints stacks	1582
Inlining and wrappers	1583
Relation to profiling	1583

Experiment	1583
mmap vs pread in Go Storage	1584
mmap vs pread in Go Storage	1585
Overview	1585
Diagram: two I/O styles	1585
Tradeoffs	1585
Go-shaped recommendation	1586
Experiment	1586
Compiler Frontend: Scan, Parse, Typecheck	1587
Compiler Frontend: Scan, Parse, Typecheck	1588
Overview	1588
Diagram: frontend stages	1588
Scanner (lexer)	1588
Parser	1588
Type checker	1589
Unified IR / export data (bridge)	1589
What the frontend will not do	1589
Experiment	1589
Compiler Mid/Back: IR, SSA, Machine Code, Link	1590
Compiler Mid/Back: IR, SSA, Machine Code, Link	1591
Overview	1591
Diagram: mid and back end	1591
IR responsibilities	1591
SSA	1592
Machine code + object files	1592
Linker	1592
Experiment	1592
Race Detector Internals	1593
Race Detector Internals	1594
Overview	1594
Diagram: instrumentation model	1594
What creates happens-before (reminder)	1594
Cost model	1595
False confidence	1595
Experiment	1595
Write Barriers and Mark Assist	1596
Write Barriers and Mark Assist	1597
Overview	1597
Diagram: barrier keeps tri-color safe	1597
When barriers are on	1597

Mark assist	1597
Tuning angle	1598
Experiment	1598
Work Stealing and findrunnable	1599
Work Stealing and findrunnable	1600
Overview	1600
Diagram: where is the next G?	1600
Local vs global queues	1600
Work stealing	1601
Netpoll integration	1601
Why you care	1601
Experiment	1601
Type Assert and Convert at Runtime	1602
Type Assert and Convert at Runtime	1603
Overview	1603
Diagram: assert path	1603
Convert vs assert	1603
Interface → interface	1603
Performance notes	1604
Experiment	1604
OS Signals and the Runtime	1605
OS Signals and the Runtime	1606
Overview	1606
Diagram: signal path (teaching)	1606
os/signal API	1606
Interactions	1606
Graceful shutdown pattern	1607
Experiment	1607
HTTP/2 in Go	1609
HTTP/2 in Go	1610
Overview	1610
Diagram: multiplex	1610
Client knobs	1610
Practical differences vs HTTP/1.1	1610
Server	1611
Experiment	1611
database/sql Pool and Context	1612
database/sql Pool and Context	1613
Overview	1613
Diagram: pool	1613

Essential settings	1613
Context everywhere	1614
Rows and transactions	1614
pgx note	1614
Experiment	1614
JSON Performance Patterns	1615
JSON Performance Patterns	1616
Overview	1616
Diagram: cost centers	1616
Patterns	1616
Encoder reuse caveats	1616
json/v2 (GA in Go 1.27)	1617
Experiment	1617
errgroup Patterns	1618
errgroup Patterns	1619
Overview	1619
Diagram: cancel on first error	1619
Basic API	1619
Limits	1620
Rules	1620
vs WaitGroup	1620
Experiment	1620
Assets	1621
Concurrency Ground Up	1622
Concurrency From the Ground Up	1623
Concurrency From the Ground Up	1624
Learning Outcomes	1624
Contents Map	1624
Suggested path	1625
Core Mental Models (Preview)	1626
Three layers of concurrency	1626
Channel as data + synchronization	1626
Cancel vs done	1626
Data race vs race condition	1626
How to Study	1626
Lab module template	1627
Relationship to Other Parts	1627
Attribution	1627

Goroutines Fundamentals	1628
Goroutines Fundamentals	1629
Overview	1629
Sequential vs concurrent	1629
Main is a goroutine	1630
WaitGroup: correct waiting	1630
WaitGroup.Go (Go 1.25+)	1630
First channel: producer and consumer	1631
Producer-consumer skeleton	1631
Rules of thumb	1632
Runnable example	1632
Channels Complete	1633
Channels Complete	1634
Overview	1634
End-of-stream: why close exists	1634
Closing a channel	1634
Rules	1635
Range over channel	1635
Directional channels	1635
Done channel	1636
Deadlocks	1636
Nil channels	1636
Buffered channels	1637
Runnable example	1637
Pipelines, Select, and Cancel	1638
Pipelines, Select, and Cancel	1639
Overview	1639
Leak: early exit without cancel	1639
Cancel channel + select	1640
Cancel vs done	1640
Merge sequentially (slow)	1641
Merge concurrently (WaitGroup)	1641
Merge with select + nil disable	1641
Fan-out / fan-in pipeline sketch	1642
Runnable example	1642
Time: Throttle, Backpressure, Timeouts	1644
Time: Throttle, Backpressure, Timeouts	1645
Overview	1645
Throttle (N concurrent)	1645
Backpressure (fail fast)	1646
Operation timeout with select	1646
Timers vs time.After	1647

Runnable example	1648
Context Complete	1649
Context Complete	1650
Overview	1650
From cancel channel to context	1650
Context is a tree	1651
Timeout and deadline	1651
Values	1651
Propagate into work	1652
Runnable example	1652
Wait Groups and Encapsulation	1654
Wait Groups and Encapsulation	1655
Overview	1655
Done channel vs WaitGroup	1655
Mental model (naive)	1656
Pass by pointer	1656
Encapsulation	1657
Hide inside a function	1657
Hide inside a type	1657
Add after Wait (advanced)	1658
Runnable example	1658
Data Races and Mutexes	1659
Data Races and Mutexes	1660
Overview	1660
Concurrent map writes	1660
Race detector	1660
Fix 1: avoid shared mutation	1660
Fix 2: mutex	1661
Non-reentrant	1661
Pass by pointer	1661
RWMutex	1662
Critical section hygiene	1662
Runnable example	1662
Semaphores, Rendezvous, Barriers	1664
Semaphores, Rendezvous, Barriers	1665
Overview	1665
Mutex is N=1	1665
Semaphore model	1665
Channel implementation	1665
Where to acquire	1666
Rendezvous	1666

Barrier (N parties)	1667
Runnable example	1668
Atomics Complete	1669
Atomics Complete	1670
Overview	1670
Non-atomic increment	1670
Atomic types and ops	1670
Composition traps	1671
CAS gate (mutex alternative for once)	1671
When to use atomics	1671
Runnable example	1672
Testing Concurrent Code	1673
Testing Concurrent Code	1674
Overview	1674
Diagram: Assert after barrier	1674
Principles	1674
Pattern: wait for completion	1674
Pattern: collect results	1675
Pattern: assert no leak	1675
Avoid sleeps as correctness	1675
Testing cancel paths	1676
Table tests still work	1676
Runnable example	1676
Scheduler Model with Diagrams	1678
Scheduler Model with Diagrams	1679
Overview	1679
Three layers	1679
Simplified scheduler loop	1680
Starvation and preemption	1680
System calls	1680
GOMAXPROCS	1680
Concurrency primitives vs scheduler	1681
Metrics, profiles, traces	1681
Runnable experiment	1681
Keep going	1682
Race Conditions vs Data Races	1683
Race Conditions vs Data Races	1684
Overview	1684
Teaching rule	1684
Data race: classic counter	1685
Fixes (any one is enough for the race)	1685

Race condition without data race: atomic composition	1686
Correct patterns	1687
Check-then-act	1687
LoadOrStore sketch	1688
Diagram: three layers of “correct”	1688
Runnable lab: compare three versions	1689
Pitfalls	1691
Checklist	1691
Keep going	1691
Signaling: Cond and Broadcast	1692
Signaling: Cond and Broadcast	1693
Overview	1693
Diagram: wait / signal	1693
Signal vs Broadcast	1693
vs channel	1694
Runnable: ready flag + Broadcast	1694
Runnable: bounded buffer with Cond	1695
Channel equivalent (usually clearer)	1697
Spurious and broad wakes	1697
Signal vs Broadcast lab	1698
Pitfalls	1699
Cancellation and Cond	1699
Checklist	1699
Keep going	1699
Web Development in Go	1701
Web Development in Go	1702
Web Development in Go	1703
Learning outcomes	1703
Key features	1704
Contents map	1704
Suggested path	1705
Running example: Bookstore	1705
Introduction to Web Development in Go	1707
Introduction to Web Development in Go	1708
Overview	1708
Why Go for web services	1708
Request lifecycle	1708
First server	1709
What to notice	1710
Status codes you will use constantly	1710
Logging from day one	1710

Error shape for APIs	1711
Bookstore north star	1711
Rules of thumb	1711
Try next	1712
Structuring Go Web Applications	1713
Structuring Go Web Applications	1714
Overview	1714
Goals of structure	1714
Recommended layout	1714
Dependency direction	1715
Config from environment	1715
Domain types (Bookstore)	1716
Repository interface	1717
Wiring in main	1717
Package naming habits	1718
Modular design without over-engineering	1718
Graceful shutdown sketch	1718
Rules of thumb	1719
Try next	1719
Handling HTTP Requests and Routing	1720
Handling HTTP Requests and Routing	1721
Overview	1721
ServeMux (Go 1.22+)	1721
Handler shape	1721
List and get (JSON)	1722
Create with validation	1722
Response helpers	1723
Middleware	1723
Query parameters and headers	1724
Gorilla Mux (when you see it)	1724
Method not allowed vs not found	1724
REST habits for Bookstore	1724
Rules of thumb	1725
Try next	1725
Templating and Rendering Content	1726
Templating and Rendering Content	1727
Overview	1727
html/template vs text/template	1727
Minimal page	1727
Parse once, execute many	1728
Static files	1729
Forms (create book)	1729
Escaping and safety	1730

Sharing data with JSON APIs	1730
Partial rendering / HTMX (optional)	1730
Rules of thumb	1730
Try next	1731
Interaction with Databases	1732
Interaction with Databases	1733
Overview	1733
Principles	1733
Schema (Postgres-flavored)	1733
Open a pool	1734
Postgres repository	1734
In-memory repository (demo + tests)	1736
Transactions	1737
N+1 and simple query design	1737
Connection errors vs not found	1738
Rules of thumb	1738
Try next	1738
Concurrency in Go Web Services	1739
Concurrency in Go Web Services	1740
Overview	1740
What the server already gives you	1740
Parallel independent fetches	1740
With cancel on first error	1741
Timeouts and context	1741
Worker pool for bulk work	1742
Shared state in handlers	1742
Deadlock patterns to avoid	1743
HTTP client reuse	1743
Fire-and-forget (careful)	1744
Rules of thumb	1744
Try next	1744
Sessions, Authentication, and Authorization	1745
Sessions, Authentication, and Authorization	1746
Overview	1746
Concepts	1746
Password hashing	1746
User model	1747
Session store (server-side)	1747
Secure cookie	1748
Login handler	1748
Auth middleware	1749
Role-based access	1750
Logout	1750

CSRF note (HTML forms)	1750
Rules of thumb	1750
Try next	1751
Frontend and Backend Communication	1752
Frontend and Backend Communication	1753
Overview	1753
API contract basics	1753
JSON tags and stability	1754
CORS (browser frontends on another origin)	1754
Fetch from a browser (sketch)	1755
Go HTTP client patterns	1755
Client checklist	1756
Retries (simple, careful)	1756
Content negotiation	1757
Rules of thumb	1757
Try next	1757
Testing and Debugging	1759
Testing and Debugging	1760
Overview	1760
What to test	1760
Table-driven domain tests	1760
Mock repository	1761
httptest handler test	1761
Full mux test	1762
Testing auth	1762
Mocking external APIs	1762
Logging and tracing events	1763
Request ID middleware	1763
Error messages and incident response	1763
Light performance checks	1764
Debug checklist	1764
Rules of thumb	1764
Try next	1764
Where to go next	1765
Middleware Patterns	1766
Middleware Patterns	1767
Overview	1767
The shape	1767
Recover	1767
Request ID	1768
Access log with status	1768
Max body	1769
Timeout (handler budget)	1769

Rules of thumb	1769
Try next	1769
Graceful Shutdown for Web Servers	1770
Graceful Shutdown for Web Servers	1771
Overview	1771
Pattern	1771
Kubernetes notes	1772
In-flight work beyond HTTP	1772
Health endpoints during drain	1772
Rules of thumb	1772
Try next	1773
Rate Limiting and Concurrency Caps	1774
Rate Limiting and Concurrency Caps	1775
Overview	1775
Concurrency semaphore	1775
Token bucket (x/time/rate)	1775
Per-IP limit (minimal)	1776
Client-side (outbound)	1776
Rules of thumb	1776
Try next	1777
WebSockets Basics	1778
WebSockets Basics	1779
Overview	1779
Upgrade sketch (gorilla)	1779
Heartbeats	1780
Concurrency	1780
Auth	1780
When not to use WebSockets	1780
Rules of thumb	1780
Try next	1781
File Uploads and Downloads	1782
File Uploads and Downloads	1783
Overview	1783
Upload (multipart)	1783
Download	1784
Static files	1784
Rules of thumb	1784
Try next	1784
Project: Bookstore API Capstone	1785

Project: Bookstore API Capstone	1786
Overview	1786
Scope	1786
Checklist	1786
Milestone order	1787
Stretch	1787
Done when	1787
 CLI Tools in Go	 1788
Building CLI Tools in Go	1789
Building CLI Tools in Go	1790
Learning outcomes	1790
Path map	1790
Suggested schedule	1791
Mental model: a CLI is a function	1792
Stdlib first, libraries later	1792
Running examples	1792
 First CLI Tools with os.Args	 1793
First CLI Tools with os.Args	1794
Overview	1794
Hello CLI	1794
Separate main from logic	1794
Positional args patterns	1795
Required path	1795
Variadic files (like <code>cat</code>)	1795
Options before flags package (<code>-n</code> style DIY)	1796
Program name for usage	1796
Environment as “args”	1797
Example: <code>add</code> calculator	1797
Example: <code>whichdir</code> — print working directory variants	1798
Rules of thumb	1798
Try next	1798
 flag Package in Depth	 1799
flag Package in Depth	1800
Overview	1800
Minimal example	1800
Var forms (bind to your own vars)	1801
Custom Usage	1801
Error handling modes	1801
Custom flag types (<code>flag.Value</code>)	1802
Enum flag	1802
Bool flags	1803

Positional args after flags	1803
Example: <code>httpget</code> (stdlib-only client)	1803
Example: <code>sleepfor</code>	1804
Example: <code>lines</code> — count lines in files	1804
Rules of thumb	1804
Try next	1804
Subcommands with <code>flag.FlagSet</code>	1806
Subcommands with <code>flag.FlagSet</code>	1807
Overview	1807
Dispatch pattern	1807
Per-command <code>FlagSet</code>	1808
Shared “global” flags	1809
Globals before verb (manual)	1809
Globals on every <code>FlagSet</code> (duplicate)	1809
Nested subcommands	1809
Example: mini <code>todo</code> CLI	1810
Help that lists commands	1810
Alias map	1811
When <code>FlagSet</code> is enough	1811
Rules of thumb	1811
Try next	1811
Exit Codes, Stdout, and Stderr	1812
Exit Codes, Stdout, and Stderr	1813
Overview	1813
The contract	1813
Stdout vs stderr	1813
Quiet / verbose / JSON modes	1814
Is stdout a TTY?	1814
Don’t call <code>os.Exit</code> in libraries	1815
Example: correct <code>grep</code> -shaped output	1815
Machine-readable errors	1816
Rules of thumb	1816
Try next	1816
Environment Variables and Config Files	1817
Environment Variables and Config Files	1818
Overview	1818
Precedence (recommended)	1818
Environment variables	1818
12-factor style names	1819
Parse duration from env	1819
Booleans	1819
Config file locations	1819

JSON config (stdlib)	1820
Merge with flags	1820
Dotenv? Not in stdlib	1822
Secrets	1822
Rules of thumb	1823
Try next	1823
Stdin, Files, and Pipes	1824
Stdin, Files, and Pipes	1825
Overview	1825
Input priority (common UX)	1825
Stream line by line	1826
Example: <code>grep1</code> (stdlib)	1826
Example: <code>wc1</code> lines/words/bytes	1827
Example: JSON filter from stdin	1828
Detect pipe vs TTY for stdin	1828
Atomic write of output files	1829
Progress on stderr only	1829
Rules of thumb	1829
Try next	1830
os/exec Orchestration	1831
os/exec Orchestration	1832
Overview	1832
Basics	1832
Stream to the user	1832
Capture and inspect exit code	1832
Pipes between processes	1833
Never shell untrusted input	1834
LookPath	1834
Example: <code>gofmt-write</code> wrapper	1834
Example: parallel commands with <code>errgroup</code>	1834
Example: <code>runjson</code> — exec and parse JSON stdout	1835
Timeouts	1835
Rules of thumb	1835
Try next	1836
Signals and Context in CLIs	1837
Signals and Context in CLIs	1838
Overview	1838
Minimal graceful cancel	1838
Pass ctx everywhere	1838
Long-running workers	1839
Ignore SIGPIPE? (advanced)	1840
Example: cancellable download CLI	1840
Double Ctrl-C	1840

Rules of thumb	1841
Try next	1841
Testing CLI Tools	1842
Testing CLI Tools	1843
Overview	1843
Golden structure	1843
FlagSet in run()	1844
Table-driven cases	1844
Temp files and env	1845
Subprocess tests (optional)	1845
Golden files	1845
Race and parallel	1846
Rules of thumb	1846
Try next	1846
Structuring Larger CLI Apps	1847
Structuring Larger CLI Apps	1848
Overview	1848
Recommended layout	1848
App struct	1848
Dependency injection for domain	1850
Cobra migration path	1850
Version and commit via ldflags	1850
Multi-binary monorepo	1851
Example: split packages sketch	1851
Config package	1851
Rules of thumb	1852
Try next	1852
Cobra Basics	1853
Cobra Basics	1854
Overview	1854
Install	1854
Minimal root + subcommand	1854
Run vs RunE	1855
Args validators	1856
Local flags	1856
Project layout with Cobra	1856
Example: kv with Cobra	1857
Silencing usage on errors	1858
Mapping to stdlib skills	1858
Rules of thumb	1858
Try next	1858
Cobra Advanced: Persistent Flags, Hooks, Completion	1859

Cobra Advanced: Persistent Flags, Hooks, Completion	1860
Overview	1860
Persistent flags	1860
Required flags	1860
PreRun / PostRun chain	1861
Bind flags to structs cleanly	1861
Context on commands	1862
Command groups (Cobra 1.6+)	1862
Custom usage / version templates	1862
Shell completion	1862
Traverse children for help	1863
Example: parent <code>remote</code> with children	1863
Testing Cobra commands	1863
Rules of thumb	1864
Try next	1864
Viper and Config Layering	1865
Viper and Config Layering	1866
Overview	1866
When Viper helps	1866
Install	1866
Minimal Viper + Cobra	1866
Precedence (Viper)	1868
Example config.yaml	1868
Unmarshal into struct	1868
Bind env explicitly	1868
Pitfalls	1868
Isolated Viper for tests	1869
Lighter alternatives	1869
Stdlib-first checklist before Viper	1869
Rules of thumb	1869
Try next	1869
Other CLI Libraries: urfave/cli, pflag, and Friends	1870
Other CLI Libraries: urfave/cli, pflag, and Friends	1871
Overview	1871
Comparison matrix	1871
pflag alone	1871
urfave/cli v2/v3 sketch	1872
Kong (struct-first)	1873
Charm ecosystem (when CLI becomes TUI)	1873
mow.cli, kingpin, ff	1873
Choosing	1874
Hybrid pattern	1874
Rules of thumb	1874
Try next	1874

Shipping: Version, Install, Completion, Release	1875
Shipping: Version, Install, Completion, Release	1876
Overview	1876
Version command / flag	1876
From build info (no ldflags)	1876
Install paths	1877
Man pages and docs	1877
Completions (install notes)	1877
Cross-compile	1877
Makefile sketch	1878
GoReleaser (overview)	1878
Docker (optional for CLIs)	1878
Checksums and supply chain	1879
Rules of thumb	1879
Try next	1879
CLI Cookbook: Stdlib and Cobra Examples	1880
CLI Cookbook: Stdlib and Cobra Examples	1881
Overview	1881
A. hello — flags + exit codes (stdlib)	1881
B. catn — files or stdin (stdlib)	1882
C. ffind — walk files by suffix (stdlib)	1883
D. jqc — extract JSON field from NDJSON stdin (stdlib)	1884
E. httpc — GET with timeout (stdlib)	1885
F. kv — subcommands with FlagSet (stdlib)	1886
G. Cobra todo — add / list (library)	1889
H. Cobra parent + persistent verbose	1891
I. Ideas to implement yourself	1891
J. Full project: network diagnostics CLI	1892
K. Full project: network security mini-tools	1892
Cookbook checklist	1892
Project: Network CLI Tools (Ping, DNS, Scan, Probe)	1893
Project: Network CLI Tools (Ping, DNS, Scan, Probe)	1894
Overview	1894
Goals	1894
Scaffold	1895
main.go	1895
app.go (dispatch)	1896
Tool 1: ping — TCP connect RTT	1897
netutil helpers	1897
cmdPing	1899
Tool 2: dns — lookup	1901
Tool 3: scan — concurrent TCP ports	1902
Tool 4: probe — HTTP status + latency	1904
Tool 5: whoami — local identity	1906

Optional: Cobra wrapper	1906
Tests (no live network required)	1907
Acceptance checklist	1907
Stretch goals	1908
What you practiced	1908
Project: Simple Network Security Mini-Tools	1909
Project: Simple Network Security Mini-Tools	1910
Overview	1910
Goals	1910
Scaffold	1911
main + dispatch (same pattern as netkit)	1911
1. tls — certificate peek	1912
2. headers — security header checklist	1913
3. banner — first bytes (service fingerprint sketch)	1915
4. redirects — chain walk	1916
5. cidr — membership check	1917
6. cookies — flag hygiene	1918
7. hash — file integrity	1919
8. secret — naive leak hunt (local files)	1920
9. rand — tokens with crypto/rand	1921
10. listen — bind check / tiny sink	1922
Wire the switch	1922
Minimal tests	1923
Suggested lab order	1923
Ethics & scope	1924
Stretch (still minimal)	1924
Acceptance checklist	1924
Project: Text and Log CLI Tools	1925
Project: Text and Log CLI Tools	1926
Overview	1926
Contracts	1926
freq core	1926
ts	1927
Acceptance	1927
Project: HTTP Debug CLI	1928
Project: HTTP Debug CLI	1929
Overview	1929
Features	1929
Timings sketch	1929
Requirements	1929
Acceptance	1930
Cross-Platform CLI Notes	1931

Cross-Platform CLI Notes	1932
Overview	1932
Paths	1932
Line endings	1932
Signals	1932
Shells	1932
Color	1933
Build tags	1933
File locking	1933
Rules	1933
Try next	1933
 Stdlib	 1934
Standard Library Tour Overview	1935
Standard Library Tour Overview	1936
Why This Part Exists	1936
Learning Path	1936
Mental Model	1937
Package Selection Cheatsheet	1937
Relationship to Other Parts	1938
How to Study	1938
Checklist: Stdlib Literacy	1939
Suggested extensions path	1939
 fmt, log, and log/slog	 1940
fmt, log, and log/slog	1941
Overview	1941
fmt Essentials	1941
Verbs you actually need	1941
Strings and builders	1942
log (legacy)	1942
log/slog (preferred)	1942
Handlers	1942
Levels and groups	1942
Context-aware logging	1943
Anti-patterns	1943
Summary	1943
Runnable example	1943
 bytes, strings, strconv, unicode	 1946
bytes, strings, strconv, unicode	1947
Overview	1947
Package map	1947

strings cookbook	1947
Builder vs buffer	1948
bytes cookbook	1948
strconv: explicit conversions	1948
unicode and utf8	1948
Validation pattern	1949
Summary	1949
Runnable example	1949
io and bufio Composition	1952
io and bufio Composition	1953
Overview	1953
Core contracts	1953
Helpers worth memorizing	1953
bufio: amortize syscalls	1954
Composition examples	1955
Checksum while uploading	1955
Bound + discard remainder	1955
Pitfalls	1955
Runnable example	1955
os, filepath, io/fs, and embed	1958
os, filepath, io/fs, and embed	1959
Overview	1959
os basics	1959
Env and args	1959
filepath: always OS-aware	1960
Prevent path traversal	1960
io/fs abstraction	1960
embed	1961
Runnable example	1961
encoding: JSON, CSV, XML, base64	1963
encoding: JSON, CSV, XML, base64	1964
Overview	1964
JSON	1964
Struct tags	1964
Encode / decode	1964
Useful options	1965
Dynamic JSON	1965
Custom marshalers	1965
encoding/json/v2 (Go 1.27+)	1965
CSV	1966
XML (when you must)	1966
base64 and hex	1966
Error handling pattern	1966

Runnable example	1967
net/http Standard Library	1969
net/http Standard Library	1970
Overview	1970
Server (Go 1.22+ patterns)	1970
Middleware pattern	1971
Client	1971
JSON helpers	1972
net/url	1972
httptest (for unit tests)	1972
Runnable example	1972
flag, os/exec, and os/signal	1975
flag, os/exec, and os/signal	1976
Overview	1976
flag	1976
Custom FlagSet (subcommands)	1976
Good CLI hygiene	1976
os/exec	1977
Stream pipes	1977
Rules	1977
os/signal and graceful shutdown	1978
Runnable example	1978
time and context Recipes	1980
time and context Recipes	1981
Overview	1981
time essentials	1981
Format and parse	1981
Timers and tickers	1981
context essentials	1982
Values (use sparingly)	1982
Waiting on cancel	1982
Recipes	1983
Bound a function	1983
Remaining budget	1983
Sleep that respects cancel	1983
Runnable example	1983
sync and sync/atomic	1986
sync and sync/atomic	1987
Overview	1987
WaitGroup	1987
Mutex and RWMutex	1987

Once	1988
Pool	1988
Map	1988
Cond (advanced)	1989
atomic	1989
Compare-and-swap	1989
Race detector	1989
Runnable example	1990
slices, maps, cmp, and iter	1992
slices, maps, cmp, and iter	1993
Overview	1993
slices	1993
Useful helpers	1993
maps	1994
cmp	1994
iter and range-over-func	1994
When not to use them	1995
Runnable example	1995
testing Package: Examples, Fuzz, Bench	1998
testing Package: Examples, Fuzz, Bench	1999
Overview	1999
Test functions	1999
Table tests	1999
Examples	2000
Benchmarks	2000
Fuzzing	2000
fstest and httptest	2001
Commands	2001
Runnable example	2001
crypto, hash, rand, and regexp	2004
crypto, hash, rand, and regexp	2005
Overview	2005
Hashing	2005
HMAC	2006
Randomness	2006
uuid (Go 1.27+)	2006
regexp	2007
Performance tips	2007
Password hashing note	2007
Runnable example	2007
Stdlib tour wrap-up	2009
database/sql	2010

database/sql	2011
Overview	2011
Mental model	2011
Open, configure, ping	2011
Query patterns	2012
Many rows	2012
One row	2013
Exec (insert/update/delete)	2013
Nullables	2013
Transactions	2013
Prepared statements	2014
Errors worth mapping	2014
Rules of thumb	2014
Try next	2015
net: Dial, Listen, and UDP	2016
net: Dial, Listen, and UDP	2017
Overview	2017
Addresses and resolution	2017
TCP server (line protocol sketch)	2017
TCP client	2018
UDP	2018
Unix domain sockets	2019
net.Conn as io.Reader / Writer	2019
Errors and temporary failures	2019
ip and ipnet helpers	2019
Rules of thumb	2019
Try next	2020
text/template and html/template	2021
text/template and html/template	2022
Overview	2022
When to use which	2022
Core loop: parse once, execute many	2022
Actions you will use constantly	2023
Define and execute named templates	2023
FuncMap	2023
html/template safety	2024
Whitespace and formatting	2024
text/template for codegen	2024
Common errors	2025
Rules of thumb	2025
Try next	2025
compress and archive	2026

compress and archive	2027
Overview	2027
Package map	2027
Gzip: compress a file	2027
Decompress	2028
HTTP Content-Encoding sketch	2028
tar + gzip (<code>.tar.gz</code>)	2029
ZIP	2030
Streaming vs memory	2030
Rules of thumb	2030
Try next	2030
encoding/binary, gob, and pem	2031
encoding/binary, gob, and pem	2032
Overview	2032
encoding/binary	2032
Fixed size helpers	2033
Length-prefixed message	2033
encoding/gob	2033
Interfaces	2034
Caveats	2034
encoding/pem	2034
encoding/hex and base64 (recap)	2035
Rules of thumb	2035
Try next	2035
math, math/bits, and math/big	2036
math, math/bits, and math/big	2037
Overview	2037
math: floats with eyes open	2037
NaN and Inf	2037
Float equality	2037
Integer-ish helpers	2038
math/bits: integer bit tricks	2038
math/big: big integers	2038
Methods mutate and return receiver	2039
Modular exponent (crypto-shaped)	2039
Rationals and floats	2039
Money warning	2039
Performance notes	2039
Rules of thumb	2040
Try next	2040
mime, multipart, expvar, and runtime/debug	2041
mime, multipart, expvar, and runtime/debug	2042
Overview	2042

mime: content types	2042
multipart: form uploads	2042
Writing multipart (client)	2043
expvar: export process vars	2044
net/http/pprof	2044
runtime/debug	2044
debug/buildinfo (binaries on disk)	2045
Rules of thumb	2045
Try next	2045
container/heap, list, and ring	2046
container/heap, list, and ring	2047
Overview	2047
container/heap	2047
container/list	2047
container/ring	2048
Rules	2048
Try next	2048
path and path/filepath Deep Dive	2049
path and path/filepath Deep Dive	2050
Overview	2050
Quick rules	2050
Safe join (jail)	2050
Walk	2051
Match / Glob	2051
Rules	2051
Try next	2051
bufio Scanner and Readers Advanced	2052
bufio Scanner and Readers Advanced	2053
Overview	2053
Scanner limits	2053
Custom split	2053
Reader Peek / Discard	2054
Writer	2054
Rules	2054
Try next	2054
Projects	2055
027 Project 27: Kubernetes Event Watcher	2056
027 Build a Kubernetes Event Watcher	2057
Problem statement	2057

Acceptance criteria	2057
Setup	2057
Full <code>main.go</code>	2058
Step-by-step build path	2060
Run and verification	2060
Stretch goals	2061
Pitfalls	2061
Learning goals	2061
028 Project 28: Kubernetes Rollout Checker	2062
028 Build a Kubernetes Rollout Checker	2063
Problem statement	2063
Acceptance criteria	2063
Setup	2063
Full <code>main.go</code>	2064
Run and verification	2066
Tests	2066
Stretch goals	2066
Pitfalls	2067
Learning goals	2067
029 Project 29: Terraform Plan Parser	2068
029 Build a Terraform Plan Parser	2069
Problem statement	2069
Acceptance criteria	2069
Setup	2069
Full <code>main.go</code>	2070
Run and verification	2072
Tests	2072
Stretch goals	2073
Pitfalls	2073
Learning goals	2073
030 Project 30: Terraform Risk Reporter	2074
030 Build a Terraform Risk Reporter	2075
Problem statement	2075
Acceptance criteria	2075
Setup	2075
Full <code>main.go</code>	2076
Policy modes	2078
Run and verification	2078
Tests	2078
Stretch goals	2079
Pitfalls	2079
Learning goals	2079

031 Project 31: Prometheus Exporter	2080
031 Build a Prometheus Exporter	2081
Problem statement	2081
Acceptance criteria	2081
Setup	2081
Full <code>main.go</code>	2082
Run and verification	2083
Step-by-step build path	2083
Label cardinality warning	2083
Stretch goals	2083
Pitfalls	2084
Learning goals	2084
032 Project 32: Prometheus Probe Service	2085
032 Build a Prometheus Probe Service	2086
Problem statement	2086
Acceptance criteria	2086
Setup	2086
Full <code>main.go</code>	2087
Run and verification	2088
Stretch goals	2088
Pitfalls	2088
Learning goals	2089
033 Project 33: eBPF Tracepoint Basics	2090
033 Build eBPF Tracepoint Basics	2091
Problem statement	2091
Acceptance criteria	2091
Setup	2091
Kernel program <code>exec.bpf.c</code> (sketch)	2092
Userspace <code>main.go</code>	2092
Step-by-step build path	2094
Run and verification	2094
Pitfalls	2094
Stretch goals	2095
Learning goals	2095
034 Project 34: eBPF XDP Packet Counter	2096
034 Build an eBPF XDP Packet Counter	2097
Problem statement	2097
Acceptance criteria	2097
Setup	2097
Kernel sketch <code>count.bpf.c</code>	2098
Userspace <code>main.go</code>	2098
Run and verification	2100

Safety	2100
Stretch goals	2100
Pitfalls	2101
Learning goals	2101
035 Project 35: Proxmox Multi-Node Scheduler	2102
035 Build a Proxmox Multi-Node Scheduler	2103
Problem statement	2103
Acceptance criteria	2103
Setup	2103
Full <code>main.go</code>	2104
Run and verification	2106
Stretch goals	2106
Pitfalls	2106
Learning goals	2107
036 Project 36: Proxmox Capacity Balancer	2108
036 Build a Proxmox Capacity Balancer (Dry-Run)	2109
Problem statement	2109
Acceptance criteria	2109
Setup	2109
Full <code>main.go</code>	2110
Run and verification	2112
Tests	2112
Stretch goals	2112
Pitfalls	2112
Learning goals	2113
037 Project 37: Kubernetes HPA Simulator	2114
037 Build a Kubernetes HPA Simulator	2115
Problem statement	2115
Acceptance criteria	2115
Setup	2115
Full <code>main.go</code>	2116
Run and verification	2118
Tests	2118
Stretch goals	2119
Pitfalls	2119
Learning goals	2119
038 Project 38: Terraform Cost Estimator	2120
038 Build a Terraform Cost Estimator (Simple)	2121
Problem statement	2121
Acceptance criteria	2121
Setup	2121

Full <code>main.go</code>	2122
Run and verification	2124
Tests	2124
Stretch goals	2125
Pitfalls	2125
Learning goals	2125
039 Project 39: Prometheus Alert Rule Tester	2126
039 Build a Prometheus Alert Rule Tester	2127
Problem statement	2127
Acceptance criteria	2127
Setup	2127
Full <code>main.go</code>	2128
Run and verification	2129
Tests	2130
Stretch goals	2130
Pitfalls	2130
Learning goals	2131
040 Project 40: Infra Reconciler Daemon	2132
040 Build an Infra Reconciler Daemon	2133
Problem statement	2133
Acceptance criteria	2133
Setup	2133
Full <code>main.go</code>	2134
Architecture	2136
Run and verification	2136
Tests	2136
Stretch goals	2137
Pitfalls	2137
Learning goals	2137
041 Project 41: TinyGo Blink & Toolchain	2138
041 TinyGo: Toolchain, Targets, and Blink	2139
Why TinyGo	2139
Step 1: Install TinyGo	2139
Step 2: Module + blink (board-agnostic pattern)	2140
No hardware? Use WASM smoke build	2140
Step 3: Flash (hardware)	2141
Step 4: Size and optimization awareness	2141
Learning Goals	2141
Extensions	2142
Common gotchas	2142
Checkpoint	2142
042 Project 42: TinyGo Button + Interrupt	2143

042 TinyGo: Button Input and Interrupt-Driven Events	2144
Realtime framing	2144
Step 1: Polling baseline (all targets)	2144
Step 2: Interrupt pattern (when supported)	2145
Step 3: Debounce theory	2146
Step 4: Measure soft-realtime behavior	2146
Learning Goals	2147
Extensions	2147
Checkpoint	2147
043 Project 43: TinyGo UART Sensor Loop	2148
043 TinyGo: UART Console + Timed Sensor Loop	2149
Architecture	2149
Step 1: Serial hello	2149
Step 2: Fixed-rate loop with miss detection	2150
Step 3: Real ADC (when target supports it)	2151
Step 4: Soft realtime checklist	2151
Learning Goals	2151
Extensions	2152
Checkpoint	2152
044 Project 44: TinyGo Cooperative Tasks (Soft RT)	2153
044 TinyGo: Cooperative Multi-Tasking Without a Full RTOS	2154
Theory (short)	2154
Step 1: Task table	2154
Step 2: Inject a “bad” task	2156
Step 3: Metrics	2156
Step 4: Design write-up	2156
Learning Goals	2156
Extensions	2157
Checkpoint	2157
045 Project 45: TinyGo I2C Realtime Monitor	2158
045 TinyGo: I2C Device + Realtime Status Monitor	2159
Hardware options	2159
Step 1: Bus init sketch	2159
Step 2: Error budget (soft RT)	2160
Step 3: Status LED semantics	2161
Step 4: Host-side plot (optional)	2161
Learning Goals	2161
Extensions	2161
Safety	2161
Checkpoint	2162
000 Projects Index	2163

000 Projects: Build Real Software	2164
Project tiers	2164
How To Use These Projects	2164
Project Path (Beginner -> Advanced)	2164
TinyGo & soft realtime (embedded)	2165
What You Will Practice	2166
Step-by-Step Explanation	2166
Learning Goals	2166
001 Project 1: CLI Ping Tool	2167
001 Build a CLI Ping Tool (TCP Ping)	2168
Why TCP Ping?	2168
Step 1: Create the Module	2168
Step 2: Full <code>main.go</code>	2168
Step 3: Run	2170
What to Extend	2170
Step-by-Step Explanation	2171
Code Anatomy	2171
Learning Goals	2171
002 Project 2: URL Shortener API	2172
002 Build a URL Shortener API	2173
Full <code>main.go</code>	2173
Run and Test	2175
What to Extend	2175
Step-by-Step Explanation	2175
Code Anatomy	2175
Learning Goals	2176
003 Project 3: Concurrent Log Analyzer	2177
003 Build a Concurrent Log Analyzer	2178
Problem statement	2178
Acceptance criteria	2178
Setup	2178
Full <code>main.go</code>	2179
Step-by-step build path	2181
Run and verification	2181
Tests	2181
Stretch goals	2182
Pitfalls	2182
Learning goals	2182
004 Project 4: File Integrity Checker	2183
004 Build a File Integrity Checker	2184
Full <code>main.go</code>	2184

Run	2186
What to Extend	2186
Step-by-Step Explanation	2187
Code Anatomy	2187
Learning Goals	2187
005 Project 5: Concurrent Port Scanner	2188
005 Build a Concurrent Port Scanner	2189
Problem statement	2189
Acceptance criteria	2189
Setup	2189
Full <code>main.go</code>	2190
Step-by-step build path	2191
Run and verification	2192
Tests (optional unit)	2192
Stretch goals	2193
Pitfalls	2193
Code anatomy	2193
Learning goals	2194
006 Project 6: Mini Job Runner	2195
006 Build a Mini Job Runner with Retries	2196
Full <code>main.go</code>	2196
Run	2198
What to Extend	2198
Step-by-Step Explanation	2198
Code Anatomy	2199
Learning Goals	2199
007 Project 7: Build an ls Clone	2200
007 Build an ls Clone	2201
Problem statement	2201
Acceptance criteria	2201
Setup	2201
Full <code>main.go</code>	2202
Step-by-step build path	2204
Run and verification	2204
Tests	2204
Stretch goals	2205
Pitfalls	2205
Learning goals	2205
008 Project 8: Build a cat Clone	2206
008 Build a cat Clone	2207
Problem statement	2207

Acceptance criteria	2207
Setup	2207
Full <code>main.go</code>	2208
Step-by-step build path	2209
Run and verification	2209
Tests	2210
Stretch goals	2210
Pitfalls	2211
Learning goals	2211
009 Project 9: Build a grep Clone	2212
009 Build a grep Clone	2213
Problem statement	2213
Acceptance criteria	2213
Setup	2214
Full <code>main.go</code>	2214
Step-by-step build path	2216
Run and verification	2216
Tests	2216
Stretch goals	2217
Pitfalls	2217
Learning goals	2217
010 Project 10: Build a tail -f Clone	2218
010 Build a tail -f Clone	2219
Problem statement	2219
Acceptance criteria	2219
Setup	2219
Full <code>main.go</code>	2220
Step-by-step build path	2222
Run and verification	2222
Tests	2223
Stretch goals	2223
Pitfalls	2223
Learning goals	2224
011 Project 11: Build a du Clone	2225
011 Build a du Clone	2226
Problem statement	2226
Acceptance criteria	2226
Setup	2226
Full <code>main.go</code>	2227
Step-by-step build path	2228
Run and verification	2229
Tests	2229
Stretch goals	2229

Pitfalls	2230
Learning goals	2230
012 Project 12: Build a find Clone	2231
012 Build a find Clone	2232
Problem statement	2232
Acceptance criteria	2232
Setup	2232
Full <code>main.go</code>	2233
Step-by-step build path	2234
Run and verification	2235
Tests	2235
Stretch goals	2236
Pitfalls	2236
Learning goals	2236
013 Project 13: Build a wc Clone	2237
013 Build a wc Clone	2238
Problem statement	2238
Acceptance criteria	2238
Setup	2238
Full <code>main.go</code>	2239
Step-by-step build path	2241
Run and verification	2241
Tests	2241
Stretch goals	2242
Pitfalls	2242
Learning goals	2242
014 Project 14: HTTP Load Tester	2243
014 Build an HTTP Load Tester	2244
Full <code>main.go</code>	2244
Run	2246
Step-by-Step Explanation	2246
Code Anatomy	2246
Learning Goals	2246
015 Project 15: TUI System Monitor	2247
015 Build a TUI System Monitor	2248
Setup	2248
Full <code>main.go</code>	2248
Run	2250
Step-by-Step Explanation	2250
Code Anatomy	2250
Learning Goals	2250

016 Project 16: Proxmox TUI Manager	2251
016 Build a Proxmox TUI Manager	2252
Setup	2252
Full <code>main.go</code>	2252
Run	2257
Notes	2257
Step-by-Step Explanation	2257
Code Anatomy	2257
Learning Goals	2257
017 Project 17: SSH Remote Orchestrator	2258
017 Build an SSH Remote Orchestrator	2259
Setup	2259
Full <code>main.go</code>	2259
Run	2261
Security Notes	2261
Step-by-Step Explanation	2262
Code Anatomy	2262
Learning Goals	2262
018 Project 18: Go Workout - 200 Ten-Minute Exercises	2263
018 Go Workout: 200 Ten-Minute Exercises	2264
How to Use	2264
Foundation Warmup (1-20)	2264
Files and CLI Tools (21-40)	2265
Structs, Methods, Interfaces (41-60)	2265
Errors and Reliability (61-80)	2266
Concurrency Basics (81-100)	2266
Networking and HTTP (101-120)	2267
Data and Persistence (121-140)	2267
Testing Workout (141-160)	2268
Linux Tooling Track (161-180)	2268
Advanced TUI and Infra Track (181-200)	2269
Chapter References in This Book	2269
Step-by-Step Explanation	2269
Learning Goals	2269
019 Project 19: Linux ps-lite	2271
019 Build a Linux ps-Lite Tool	2272
Problem statement	2272
Acceptance criteria	2272
Setup	2272
Full <code>main.go</code>	2273
Step-by-step build path	2274
Run and verification	2275

Tests	2275
Stretch goals	2275
Pitfalls	2276
Learning goals	2276
020 Project 20: KV HTTP Store	2277
020 Build a Key-Value HTTP Store	2278
Full <code>main.go</code>	2278
Run	2280
Step-by-Step Explanation	2280
Code Anatomy	2280
Learning Goals	2280
021 Project 21: WebSocket Chat	2281
021 Build a WebSocket Chat Server	2282
Problem statement	2282
Acceptance criteria	2282
Setup	2282
Full <code>main.go</code>	2283
Run and verification	2285
Tests	2285
Step-by-step build path	2285
Stretch goals	2285
Pitfalls	2286
Learning goals	2286
022 Project 22: Log Shipper CLI	2287
022 Build a Log Shipper CLI	2288
Problem statement	2288
Acceptance criteria	2288
Setup	2288
Full <code>main.go</code>	2289
Minimal ingest receiver (for local test)	2291
Step-by-step build path	2292
Batching stretch (sketch)	2292
Verification	2292
Pitfalls	2292
Learning goals	2293
023 Project 23: Proxmox Batch Operations CLI	2294
023 Build a Proxmox Batch CLI	2295
Problem statement	2295
Acceptance criteria	2295
Setup	2295
Full <code>main.go</code>	2296

Step-by-step build path	2298
Run and verification	2298
Tests	2298
Stretch goals	2299
Pitfalls	2299
Learning goals	2299
024 Project 24: Controller Pattern Simulator	2300
024 Build a Controller/Reconciler Simulator	2301
Problem statement	2301
Acceptance criteria	2301
Setup	2301
Full <code>main.go</code>	2302
Step-by-step build path	2304
Architecture	2304
Run and verification	2304
Tests	2304
Stretch goals	2305
Pitfalls	2305
Learning goals	2305
025 Project 25: Resource Index + 150 More Ideas	2306
025 Resource Index and Project Idea Backlog	2307
Resource Anchors	2307
150 Additional Project Ideas	2307
Linux CLI (1-30)	2307
Networking/Web (31-60)	2308
Data/Storage (61-90)	2308
Concurrency/Systems (91-120)	2309
TUI/Infra/Platform (121-150)	2310
Suggested Build Order	2311
Step-by-Step Explanation	2311
Learning Goals	2311
026 Project 26: Kubernetes Pod Lister	2312
026 Build a Kubernetes Pod Lister	2313
Problem statement	2313
Acceptance criteria	2313
Setup	2313
Full <code>main.go</code>	2314
Step-by-step build path	2315
Run and verification	2316
Pitfalls	2316
Stretch goals	2316
Learning goals	2316

Everything Go - Comprehensive Syllabus

A deep library, not a single linear course. Use a path below so the sidebar does not feel endless.

How to read this book (pick a path)

Path A - Productive Go (2-4 weeks)

Foundations → Core → Errors → Testing → Concurrency basics
→ Stdlib tour (http, json, slog) → small projects

Path B - Concurrent systems

Concurrency ground-up (part 21) → Context / channels deep dives
→ Network systems → Observability

Path C - Runtime internals

Deep dives: GMP → memory model → channels → GC → netpoll
→ sync/atomic → reading src/runtime

Path D - Ship production services

Web + DB → Infrastructure → Security → SRE/metrics
→ Performance engineering → projects (API / k8s / probes)

Path E - Hands-on web development (Bookstore track)

Part 22 Web Development in Go: structure → routing → templates
→ DB → concurrency at the edge → sessions/auth → API contracts → tests
Then 08-web (GOTH/gRPC/Huma) when you want modern stack recipes

Path F - CLI tools (stdlib → Cobra)

Part 23 CLI Tools in Go: os.Args → flag → subcommands → pipes/exec/signals
→ testing & layout → Cobra/Viper → cookbook
→ project: netkit (TCP ping, DNS, port scan, HTTP probe)
→ project: netsec (TLS cert, headers, banner, secrets hygiene, ...)
Pair with stdlib 987/994 and projects (goping, linux clones)

Path G - depth on one topic

Pick a single part (e.g. concurrency, web, CLI, stdlib) and work it end-to-end rather than skimming every path above.

Path H - Systems programming (userspace)

14-systems: Go without libc → processes → files → pipelines
 → ops toolkit; pair with specialized 92 (x/sys, eBPF, Delve)
 → infrastructure (static binaries, scratch / distroless)

Map of overlapping parts (twins)

Some topics appear twice on purpose: a **compact survey** early, then a **deep track** later. Use one path; do not read both end-to-end unless you want reinforcement.

Topic	Survey / recipes	Deep track	Prefer when...
Concurrency	07-concurrency-parallelism	21-concurrency-ground	Learning from zero → 21 ; quick patterns → 07
Web	08-web (GOTH, gRPC, Huma)	22-web-development-inf (Bookstore)	First web app → 22 ; modern stack recipes → 08
Security	11-security	17-security-hardening	Overview → 11 ; ship hardened service → 17
Performance	09-performance-tooling	18-performance-engineering	Profiling/tooling intro → 09 ; production tuning → 18

Also: 20-go-deep-dives (runtime/internals) pairs with Path C; 98-stdlib is a reference tour; 99-projects is build practice (and some checklist drills).

Related books

Book	When to open it
Linux	CLI clones, systemd, SSH, and operator tooling projects
VCS	Git, GitHub Actions, collaboration around Go modules
Networking	TCP/HTTP labs behind net/http and load-test projects
C	cgo / systems adjacency

Diagrams are **ASCII** (all formats) with optional **SVG** on a few runtime chapters (HTML).

1. Foundations & Mindset

- **The Go Philosophy:** Why “boring” is a superpower.
- **2026 Learning Roadmap:** Focusing on essentials vs. hype.
- **The 3-Layer Practice System:** Read, Write, Ship.
- **Transitioning to Go:** Insights for Java and Node.js developers.

2. Core Language Essentials

- **Language Building Blocks:** Variables, types, and functions. 1.26: `new(expr)` 1.27: generic methods, promoted field keys
- **Control Flow:** Effective use of `if`, `for`, and `switch`.
- **Data Structures:**
 - Structs and Interfaces (The “Go Way”).
 - Arrays, Slices, and Maps internals.
 - Working with Time in Go.
- **Project Organization:** Organizing projects and defining names; standard project layout.

3. Pointers & Memory Management

- **The Mechanics:** Understanding Stack vs. Heap.
- **Memory Optimization:**
 - Escape Analysis: Where did my memory go?
 - Memory Alignment and Padding.
 - Garbage Collection (GC) tuning and `GOMEMLIMIT`. Go 1.26: Green Tea GC default
- **Pass by Value vs. Pass by Reference:** Performance tradeoffs.

4. Advanced Error Handling

- **Beyond `if err != nil`:** Senior techniques for robust handling.
- **Patterns:** Sentinel errors, custom error types, and error wrapping.
- **Techniques:** Error handling without a garbage collector (Odin comparison); the “Must” pattern.

5. Concurrency & Parallelism

- **Goroutines & Channels:** The heart of Go’s concurrency model.
- **Sync Mechanisms:** `WaitGroups`, `Mutexes`, `sync.Map`, and the overlooked `sync.Cond`.
- **Execution Control:**
 - Context Package: Lifecycles, cancellation, and timeouts.
 - Worker Pools: Avoiding the chaos of uncontrolled concurrency.
 - Pipelines and complex concurrency patterns.

6. Web Development & Modern Stacks

- **Hands-on web path (part 22):** Bookstore curriculum — project structure, `net/http` routing, templates, databases, sessions/auth, frontend contracts, testing/logging. Simple language, real examples, modern Go without niche edge cases.
- **The GOTH Stack:** Go + Templ + HTMX + Tailwind.
- **Frontend Interaction:**
 - Skeleton loading with Go and HTMX.
 - Live reloading with Air.
- **API Design:**
 - Building REST APIs with routing and middleware.
 - Microservices: Switching to gRPC.
 - Using Huma for OpenAPI-backed APIs.
- **Integrations:** Working with PostgreSQL and Redis.

7. Performance, Profiling & Tooling

- **Benchmarking:** Proof-based optimization using `go test -bench`.
- **Profiling:** Deep dives with `pprof` and `trace`.
- **Code Generation:** Using `sqlc` for type-safe SQL; `stringer` for enums.
- **The Modern Toolkit:**
 - `ripgrep`, `fd`, `fzf`, `zoxide`, `bat`, and `jq`.
 - `go fix` modernizers + `//go:fix inline`. 1.26–1.27
 - `goroutineleak` `pprof` profile (GA). Go 1.27
 - `gopsutil` for system and hardware stats.
- **Advanced Refactoring:** `gofmt`, `gopatch`, and `goimports`.

8. Infrastructure & Deployment

- **Containerization:** Multi-stage Docker builds for lean binaries; health/signals; non-root.
- **Platform Guides:**
 - Deploying to Render and Leapcell.
 - Go on NixOS and FreeBSD.
 - Running Go on Mini-PCs for local AI (Llama 3).
- **CI/CD:** Automated vulnerability checking and hardening.
- **Kubernetes basics:** Deployment/Service, probes, resources, secrets.
- **Release:** GoReleaser + tag-driven GitHub Actions.

9. Security & Hardening

- **Security Tools:** govulncheck, gosec, capslock, and nilaway.
 - crypto/hpke, testing/cryptotest, crypto/mldsa, and experimental runtime/secret. 1.26–1.27
- **Hardening:** Production-grade Ubuntu hardening for Go apps.
- **Design Principles:** S.O.L.I.D principles in the context of Go.

10. Specialized Applications

- **Desktop Apps:** Cross-platform development with LCL, CEF, and Webview.
- **Observability:** Integrating OpenTelemetry.
- **Machine Learning:** Building LLM-powered applications in Go.
 - Portable simd + simd/archsimd (experimental via GOEXPERIMENT=simd). 1.26–1.27
- **Systems Programming:** How Go talks to the kernel without libc; debuggers, syscalls, and low-level internals.

11. Production Deep Dives

- **Network systems:** TCP services, HTTP resilience, reverse proxies; TLS; DNS; Transport tuning; **netcheck** project.
- **Systems programming:** How Go reaches the kernel without libc (static Linux binaries, CGO_ENABLED=0, Go vs C); processes, signals, safe filesystem I/O, Unix pipeline CLIs; env/CWD/umask; file descriptors; mmap; Unix domain sockets; flock/pidfiles; time/timers; config watch/reload; PTYs; rlimit/GOMEMLIMIT; least privilege; ops logging; socket activation; procs; projects **systool**, **sysops**, **minisup**.
- **Distributed infra:** Service discovery, retries/circuit breakers, queue backpressure; caching; idempotency; leader leases; outbox; **resilient worker** project.
- **Observability & SRE:** slog correlation, Prometheus metrics design, tracing propagation; SLI/SLO/error budgets; livez/readyz; alerting/runbooks; instrumented service project.
- **Security hardening:** TLS/mTLS, authn/authz, secrets rotation; JWT validation; input validation/SSRF; supply chain; secure service checklist.
- **Performance engineering:** Benchmark/profile workflows, allocations/GC, contention tuning; load-test baselines; allocation recipes; PGO/build flags; optimize-endpoint project.

12. Modern Go Book Synthesis

- Pattern distillation from recent (2024–2025) Go books — production-oriented themes mapped into this curriculum.

13. Go Deep Dives (Runtime, Toolchain & Production)

- **Runtime core:** GMP scheduler (incl. Go 1.26 syscall/P simplification), memory model, channels/select, netpoller, timers, mutex/semaphores, sysmon/scavenger.
- **Representation:** Interfaces (eface/iface), slice/map/Swiss tables, strings/bytes, **weak/unique**, error boxing cost.
- **Control & safety:** Defer/panic stacks, stack growth, context trees, goroutine leaks, finalizers/cleanup, synctest bubbles.
- **Performance:** GC pacer/GTGC, `sync.Pool`/zero-alloc, pprof/trace internals, zero-copy I/O, escape/PGO/compiler SSA.
- **Net & crypto path:** HTTP Transport pools, TLS handshake costs, slog handler design.
- **Toolchain:** Modules/MVS, link/race/buildmodes, cgo transitions, ABI/asm, coverage/fuzz engines, build cache, GOEXPERIMENT, Wasm/wasip1, plugin pitfalls.
- **Ops craft:** GOMAXPROCS vs cgroups, production mistake map, debugging tooling, reading `src/runtime`, X/ecosystem topic radar.
- **Article radar + more runtime:** Internals-for-Interns series map; memory allocator; bootstrap; selectgo; stacktraces; mmap vs pread.
- **Compiler & advanced runtime:** Frontend + IR/SSA/link; race detector; write barriers/assist; work stealing; type asserts; signals; HTTP/2; sql pool; JSON perf; errgroup.

14. Concurrency From the Ground Up

- Diagram-first path through goroutines, channels, pipelines, time, context, wait groups, races/mutexes, semaphores, atomics, testing, and a teaching scheduler model.
- Pedagogical map aligned with interactive concurrency curricula (e.g. antonzo.org *Gist of Go: Concurrency*); original prose and **ASCII diagrams** for this library.

15. Web Development in Go (Bookstore track)

- Separate hands-on section: intro → structure → HTTP/routing → templates → databases → web concurrency → sessions/auth/roles → frontend-backend contracts → testing/debugging.
- Key skills: modular design, REST with `net/http` (and Mux where relevant), secure cookies, JSON contracts, mocks, logging, external API clients—clarity over complex edge cases.

16. Golang CLI (stdlib → Cobra)

- **Stdlib-first:** `os.Args`, `flag` / `FlagSet` subcommands, exit codes, stdout vs stderr, `env` + JSON config, stdin pipes, `os/exec`, signals/context, testing, app layout.
- **Libraries:** Cobra basics & advanced (persistent flags, hooks, completion), Viper config layering, `pflag`/`urfave/cli`/`kong` notes, shipping (version ldflags, cross-compile, `GoReleaser`).
- **Cookbook:** hello, cat, find, NDJSON jq-lite, HTTP client, kv store, Cobra todo, and more ideas.

- **Project — network tools:** multi-command `netkit` (TCP ping like classic ping, DNS lookup, concurrent port scan, HTTP probe, whoami) with timeouts, Ctrl-C, and tests.
- **Project — network security mini-tools:** multi-command `netsec` (TLS cert expiry/SANs, security headers, TCP banner, redirects, CIDR check, cookie flags, file hash, naive secret scan, crypto/rand tokens, listen) — simple and minimal, stdlib only.
- **More CLI projects:** `textkit` (freq/cut/ts), `httpdbg` (curl-like timings), cross-platform notes.
- **Web production extras:** middleware chains, graceful shutdown, rate limits, WebSockets, uploads/downloads, Bookstore API capstone.

17. Standard Library Tour

- **Map of the stdlib:** When to reach for which package before any framework.
- **I/O & text:** `fmt/slog`, `bytes/strings/strconv`, `io/bufio` composition.
- **Files & embed:** `os`, `filepath`, `io/fs`, `//go:embed`.
- **Wire formats:** JSON (`encoding/json` and `encoding/json/v2`), CSV, XML, base64/hex; binary frames, gob, PEM. Stdlib uuid.
- **HTTP & tools:** Modern `ServeMux`, clients with timeouts, `flag/os/exec/signal`.
- **Budgets & concurrency:** `time` + context recipes; `sync` / `atomic`.
- **Modern collections:** `slices`, `maps`, `cmp`, `iter` (range-over-func).
- **Quality:** testing examples, fuzz, benches; `crypto/hash/rand/regexp` hygiene.
- **Data & sockets:** `database/sql` pools/queries/tx; `net` dial/listen/UDP/Unix.
- **Templates & archives:** `html/template` + `text/template`; `gzip/zip/tar`.
- **Math & ops edge:** `math/bits/big`; `mime/multipart` uploads; `expvar`, `pprof`, `runtime/debug`, `buildinfo`.

18. Projects

- Progressive labs from CLI utilities through Kubernetes, Terraform, Prometheus, eBPF, Proxmox, and TinyGo — plus a large idea backlog.

Foundations

Why Go Exists

Overview

Go (also known as Golang) was created at Google in 2007 by Robert Griesemer, Rob Pike, and Ken Thompson. It was designed to address the challenges of building large-scale, concurrent software systems while maintaining simplicity and fast compilation times.

The Problems Go Was Designed to Solve

1. Slow Compilation Times

In large codebases (like those at Google), C++ compilation could take hours. Go was designed with fast compilation as a primary goal, enabling rapid development cycles even in massive projects.

2. Complexity in Modern Languages

Languages like C++ and Java had become increasingly complex with years of feature additions. Go aimed to be simple enough that a programmer could hold the entire language specification in their head.

3. Difficulty with Concurrency

As multi-core processors became standard, writing concurrent code with traditional threading models proved error-prone and difficult. Go introduced goroutines and channels as first-class citizens.

4. Dependency Management Chaos

“Dependency hell” plagued many ecosystems. Go’s import system and later the module system were designed to solve versioning and dependency problems.

Go's Design Philosophy

```
// Go favors clarity over cleverness
func processItems(items []string) error {
    for _, item := range items {
        if err := process(item); err != nil {
            return fmt.Errorf("processing %s: %w", item, err)
        }
    }
    return nil
}
```

Key Principles

Principle	Description
Simplicity	Less is more—remove features rather than add them
Readability	Code is read more often than written
Orthogonality	Features should work well together without special cases
Composition	Prefer composition over inheritance
Explicit over implicit	Make behavior obvious rather than magical

What Makes Go Different

Compiled, Statically Typed, Fast

```
// Type safety caught at compile time, not runtime
var count int = 42
count = "hello" // Compile error: cannot use "hello" as int
```

Built-in Concurrency

```
// Goroutines are lightweight and easy to use
go func() {
    // This runs concurrently
    processData()
}()
```

Single Binary Output

```
# Go compiles to a single binary with no dependencies
$ CGO_ENABLED=0 go build -o myapp main.go
$ ./myapp # Just works, no language runtime to install
```

On Linux that binary is typically **fully static**: it does not need libc on the target. The Go runtime issues system calls itself, so the same file runs on Debian, Alpine, NixOS, or a **scratch** container. (Darwin still links **libSystem**; the famous “no libc” story is Linux-specific.) See [Go as a Systems Language](#) for file / ldd, glibc vs musl, and when cgo puts a C library back.

Standard Formatting

```
# One true style, enforced automatically
$ gofmt -w main.go
```

Who Uses Go

Go powers critical infrastructure across the industry:

- **Docker** - Container runtime
- **Kubernetes** - Container orchestration
- **Terraform** - Infrastructure as code
- **CockroachDB** - Distributed database
- **Hugo** - Static site generator
- **Prometheus** - Monitoring system

When to Choose Go

Good fit for: - Microservices and APIs - CLI tools and DevOps tooling - Concurrent/parallel processing - Networked services - Cloud-native applications - Userspace systems software (supervisors, agents, portable Linux binaries)

Consider alternatives for: - GUI desktop applications - Mobile development - Kernel modules, drivers, firmware, and hard real-time (C or Rust) - Data science/ML (Python ecosystem is stronger)

Summary

Go exists because the software industry needed a language that could:

1. **Compile quickly** at any scale
2. **Handle concurrency** naturally and safely

3. **Remain simple** despite solving complex problems
4. **Produce efficient binaries** that deploy easily — on Linux, without a libc dependency

Understanding these origins helps you write more idiomatic Go code that embraces the language's philosophy.

Related Topics

- [Learning Go in 2026](#)
- [Installing and Running Go](#)
- [How Go Code Is Organized](#)
- [Go as a Systems Language](#)

More examples

Example: clarity over cleverness

Save as `main.go` and `go run .` (with `go mod init example` if needed).

```
package main

import (
    "fmt"
    "strings"
)

func process(item string) error {
    if strings.TrimSpace(item) == "" {
        return fmt.Errorf("empty item")
    }
    fmt.Println("ok:", item)
    return nil
}

func processItems(items []string) error {
    for _, item := range items {
        if err := process(item); err != nil {
            return fmt.Errorf("processing %q: %w", item, err)
        }
    }
    return nil
}

func main() {
    if err := processItems([]string{"docker", "k8s", " "}); err != nil {
```

```

        fmt.Println("stopped:", err)
    }
}

```

Expected:

```

ok: docker
ok: k8s
stopped: processing " ": empty item

```

Example: static types catch mistakes early

Save as `main.go` and `go run .` (with `go mod init example` if needed).

```

package main

import "fmt"

func main() {
    var count int = 42
    // Uncommenting the next line fails at compile time:
    // count = "hello"
    fmt.Println("count:", count)

    // Conversions are explicit when you need them.
    label := fmt.Sprintf("n=%d", count)
    fmt.Println(label)
}

```

Expected:

```

count: 42
n=42

```

Runnable example

Save as `main.go`. From an empty directory:

```

go mod init example
go run .

```

```

package main

import (

```

```

    "fmt"
    "sync"
    "time"
)

// process is intentionally simple and explicit: no inheritance, no magic.
func process(item string) error {
    if item == "" {
        return fmt.Errorf("empty item")
    }
    // Simulate a small unit of work.
    time.Sleep(20 * time.Millisecond)
    fmt.Printf("processed: %s\n", item)
    return nil
}

func main() {
    items := []string{"api", "worker", "cli", ""}
    var wg sync.WaitGroup
    var mu sync.Mutex
    var failures int

    // Goroutines are first-class: cheap concurrency without a thread pool API.
    for _, item := range items {
        wg.Add(1)
        go func(item string) {
            defer wg.Done()
            if err := process(item); err != nil {
                mu.Lock()
                failures++
                mu.Unlock()
                fmt.Printf("error: %v\n", err)
            }
        }(item)
    }

    wg.Wait()
    fmt.Printf("done; failures=%d\n", failures)
}

```

Expected output (illustrative; order of lines may vary):

```

processed: api
processed: worker
processed: cli
error: empty item
done; failures=1

```


What to notice: - Concurrency is a few lines: `go func()` plus `WaitGroup`—no framework. - Errors are values checked with `if err != nil`, not thrown exceptions. - Empty-item handling is visible in the control flow; nothing is “magical.” - The program still finishes cleanly after concurrent work.

Try next: Change `time.Sleep` to zero and re-run a few times. Does line order stay the same?

Learning Go in 2026

Why Go is Worth Learning in 2026

Go (Golang) has moved beyond hype into the backbone of modern engineering. In 2026, it remains the language of choice for cloud-native systems, DevOps tooling, and scalable backend infrastructure.

Companies love Go for:

- **Simplicity:** Minimalist syntax that is easy to read and maintain.
- **Performance:** Compiled speed that rivals C++ and Java.
- **Built-in Concurrency:** First-class support for parallel processing.
- **Easy Deployment:** Compiles to a single static binary with no external dependencies. On Linux that includes **no libc** — the same file runs on Debian, Alpine, and **scratch** containers. See [Go as a Systems Language](#).

The Go Mindset (Most People Get This Wrong)

Go is “boring” by design, and that is its greatest superpower. To succeed in Go, you must shift your mindset:

Instead of...	Go Prefers...
Clever Code	Clear Code
Deep Inheritance	Simple Composition
Complex Abstractions	Concrete Implementations
Magic/Implicit Behavior	Explicit Logic

Pro Tip: Once you stop fighting Go’s simplicity and embrace its explicitness, you become shockingly productive.

The Fastest Roadmap to Learn Go

1. Learn Only the Essentials First

Don’t get bogged down in advanced features immediately. Focus on the core:

- Variables & Types
- Functions & Error Handling
- Control Flow (**if**, **for**, **switch**)
- Structs & Interfaces (The heart of Go’s type system)
- Packages & Modules

2. Learn Concurrency Early

Go without concurrency is like a ship without a sail. Don't delay learning: - **Goroutines**: Lightweight threads. - **Channels**: How goroutines communicate. - **Select**: Managing multiple channel operations. - **Sync Package**: WaitGroups and Mutexes for state management.

3. Build Tiny Programs Every Day

Forget massive projects at the start. Build real-world utility tools: - CLI Calculator - Log File Analyzer - API Health Checker - Simple REST API

The 3-Layer Practice System

This system ensures you don't just consume content but actually learn to build:

1. **Layer 1: Read**: Read clean, idiomatic Go code (like the standard library). Observe naming conventions and error handling patterns.
2. **Layer 2: Write**: Rewrite examples from memory. Don't copy-paste. Intentionally break things to see how the compiler and runtime react.
3. **Layer 3: Ship**: Push your code to GitHub. Use Go to solve your own small problems, like automating a repetitive task.

Go Projects That Make You Job-Ready

Build these in order to gain real-world experience: 1. **REST API**: Use the standard library or a lightweight router with middleware and JSON handling. 2. **CLI Tool**: Use flags and arguments to create a useful command-line utility. 3. **Concurrent Worker Pool**: Process data in parallel using goroutines and channels. 4. **Log Monitor**: A tool that watches files or streams and alerts on patterns. 5. **Microservice**: A simple service with environment-based configuration and health checks.

Why Go is a DevOps Superpower

If you are in DevOps or SRE, Go is a force multiplier. It allows you to: - Build internal CLI tools that are easily shared. - Write robust automation scripts that replace fragile shell scripts. - Extend CI/CD pipelines with custom logic. - Create Kubernetes Operators or Custom Resource Definitions (CRDs).

Go turns “glue code” into reliable software engineering.

Memory & Consistency

- **Practice daily:** 30–60 minutes is better than a 5-hour marathon.
- **Explain concepts:** Write short blog posts or notes to solidify your understanding.
- **Consistency beats intensity:** Every single time.

Go 1.27 Migration Checklist (Shared)

Use this checklist when upgrading existing projects to Go 1.27 (from 1.26 or earlier). Go 1.25 is out of support as of the 1.27 release.

```
upgrade path

audit current toolchain
    |
    v
update go.mod/toolchain (go 1.27)
    |
    v
run modernizers + full tests
    |
    v
opt into json/v2 / uuid / leak profile as needed
    |
    v
ship + monitor memory/latency regressions
```

1. Pin Toolchain and Module Versions

```
go version
go mod tidy
```

If you use a `toolchain` directive, set it explicitly (`toolchain go1.27.1` at this writing) and keep CI aligned. `go mod tidy` on modules with go 1.27 consolidates `require` blocks into the standard direct + indirect layout.

2. Run Modernization and Safety Baseline

```
go fix -modernize ./...
go vet ./...
govulncheck ./...
```

Go 1.27 adds `atomictypes`, `embedlit`, `slicesbackward`, and `unsafefuncs` modernizers. The `waitgroup` analyzer is now named `waitgroupgo`.

3. Re-Baseline Tests and Benchmarks

```
go test ./... -race -shuffle=on -count=1
go test ./... -bench=. -benchmem
```

Adopt `B.Loop`, `testing/synctest` (including `synctest.Sleep`), and `httptest.NewTestServer` for HTTP tests that should run inside a `synctest` bubble.

4. Language and stdlib opt-ins

- **Generic methods** and promoted field keys in struct literals compile on 1.27+; they are not back-portable.
- **encoding/json** is now backed by the v2 implementation. Behavior is preserved; error text may differ. New code can import `encoding/json/v2` for stricter defaults. Rollback: `GOEXPERIMENT=nojsonv2` (temporary).
- **uuid** is in the standard library; prefer it over `github.com/google/uuid` for new IDs.
- **goroutineleak** pprof profile is generally available (`/debug/pprof/goroutineleak`).

5. Enable Experiments Deliberately

Only opt into experiments if there is a clear payoff and rollback plan:

```
GOEXPERIMENT=simd go test ./...
GOEXPERIMENT=nogreenteagc go test ./...
GOEXPERIMENT=nosizespecializedmalloc go test ./...
```

Document where each experiment is used and how to disable it quickly.

6. Observe Production After Rollout

Track: - P95/P99 latency - GC pause and heap goal trend - Error rates and timeout rates - Goroutine count (and the leak profile if you enable pprof)

Run canary first, then broad rollout.

Summary

Don't try to "finish" Go. Use Go to solve real problems, and the language will teach itself to you along the way. Stay curious, keep building, and embrace the simplicity.

Related Topics

- [Why Go Exists](#)
- [Installing and Running Go](#)
- [Core Go Commands](#)
- [Go as a Systems Language](#)

More examples

Example: API health-check style probe (no network)

Save as `main.go` and `go run .` (with `go mod init example` if needed).

```
package main

import "fmt"

// probe models a tiny health checker without opening sockets-practice the shape first.
func probe(status int) (healthy bool, detail string) {
    if status >= 200 && status < 400 {
        return true, fmt.Sprintf("status %d ok", status)
    }
    return false, fmt.Sprintf("status %d unhealthy", status)
}

func main() {
    for _, code := range []int{200, 503} {
        ok, detail := probe(code)
        fmt.Printf("code=%d healthy=%v (%s)\n", code, ok, detail)
    }
}
```

Expected:

```
code=200 healthy=true (status 200 ok)
code=503 healthy=false (status 503 unhealthy)
```

Example: composition over inheritance

Save as `main.go` and `go run .` (with `go mod init example` if needed).

```
package main

import "fmt"
```

```

type Logger struct {
    Prefix string
}

func (l Logger) Info(msg string) {
    fmt.Printf("%s %s\n", l.Prefix, msg)
}

// Service composes a Logger instead of extending a base class.
type Service struct {
    Logger
    Name string
}

func main() {
    svc := Service{
        Logger: Logger{Prefix: "[svc]"},
        Name:   "billing",
    }
    svc.Info("started " + svc.Name)
    svc.Info("ready")
}

```

Expected:

```

[svc] started billing
[svc] ready

```

Runnable example

A tiny CLI calculator—the kind of daily practice project from this chapter’s roadmap.

Save as `main.go`. From an empty directory:

```

go mod init example
go run . add 3 9
go run . mul 4 5
go run . div 10 0

```

```

package main

```

```

import (
    "fmt"
    "os"
    "strconv"

```

```

)

func main() {
    if len(os.Args) != 4 {
        fmt.Fprintln(os.Stderr, "usage: go run . <add|sub|mul|div> <a> <b>")
        os.Exit(2)
    }

    op := os.Args[1]
    a, errA := strconv.ParseFloat(os.Args[2], 64)
    b, errB := strconv.ParseFloat(os.Args[3], 64)
    if errA != nil || errB != nil {
        fmt.Fprintln(os.Stderr, "operands must be numbers")
        os.Exit(1)
    }

    var result float64
    switch op {
    case "add":
        result = a + b
    case "sub":
        result = a - b
    case "mul":
        result = a * b
    case "div":
        if b == 0 {
            fmt.Fprintln(os.Stderr, "division by zero")
            os.Exit(1)
        }
        result = a / b
    default:
        fmt.Fprintf(os.Stderr, "unknown op %q\n", op)
        os.Exit(1)
    }

    fmt.Printf("%s(%.2f, %.2f) = %.2f\n", op, a, b, result)
}

```

Expected output (illustrative):

```

add(3.00, 9.00) = 12.00
mul(4.00, 5.00) = 20.00
division by zero

```

What to notice: - Core essentials only: args, types, **switch**, errors—matching the “learn essentials first” path. - Invalid input exits with a clear message; no exceptions. - Floats come from **strconv** so type conversion is explicit. - One file, no framework: ship-ready practice.

Try next: Add a `mod` (remainder) operation and support integer-only mode.

The Go Philosophy

The Go Philosophy: Why “Boring” is a Superpower

In a software landscape obsessed with the “next big thing,” Go stands out by explicitly trying *not* to be clever. Since its inception at Google in 2007 (and release in 2009), Go has adhered to a philosophy of simplicity, stability, and readability.

As we move through 2026, this philosophy hasn’t just survived; it has proven to be a competitive advantage.

Boring Software is Reliable Software

The core tenet of Go’s philosophy is often summarized as: **“Clear is better than clever.”**

1. Readability Over Conciseness

Go is verbose. This is intentional. When you read Go code, you don’t need to hold a complex mental model of hidden states, monkey-patched methods, or magical frameworks in your head. The control flow is visible. * **No inheritance:** You cannot hide logic in a base class three layers up. * **No method overloading:** `DoWork` and `DoWorkWithContext` are distinct. You know exactly which one is called. * **No circular dependencies:** The compiler forbids it, forcing you to keep your architecture clean.

2. One Way To Do It

While languages like Ruby or Scala offer ten ways to iterate over a list, Go gives you one: the `for` loop. This reduced cognitive load means you spend less time debating *style* and more time solving *problems*.

3. Stability is a Feature

Go’s [Go 1 compatibility promise](#) is legendary. Code written in 2012 will likely compile and run today with little to no modification. In 2026, where JavaScript frameworks churn every six months, this stability allows organizations to build for the decade, not the demo.

“Software engineering is what happens to programming when you add time and other programmers.” — Russ Cox

The 2026 Context: AI and Complexity

With the rise of AI-generated code (Copilot, Gemini, etc.), simplicity is more vital than ever. * **AI generates bugs:** Complex languages with hidden magic allow AI to hallucinate plausible but broken code. Go's explicit error handling and type system catch these issues early. * **Reviewability:** Humans still need to review AI code. Go's simplicity makes it easier for a human to verify that the code does what the LLM claims it does.

Key Principles Summarized

Principle	Description
Simplicity	The language spec is small enough to hold in your head.
Orthogonality	Features interact in predictable ways (e.g., Goroutines work the same in <code>main</code> or a <code>handler</code>).
Punt Complexity	If a feature is complex to implement in the compiler <i>and</i> hard to understand (e.g., complex metaprogramming), Go leaves it out.

Conclusion

Go is designed for **engineering**, not just programming. It accepts that software is written once but read hundreds of times. By choosing “boring,” Go enables you to build exciting, scalable, and reliable systems.

More examples

Example: one obvious loop

Save as `main.go` and `go run .` (with `go mod init example` if needed).

```
package main

import "fmt"

// One way to walk a list: a plain for-range. No hidden iterator magic.
func sum(nums []int) int {
    total := 0
    for _, n := range nums {
        total += n
    }
}
```

```

    return total
}

func main() {
    values := []int{1, 2, 3, 4}
    fmt.Println("sum:", sum(values))

    // Explicit empty-slice path-readable without special syntax.
    var empty []int
    fmt.Println("empty sum:", sum(empty))
}

```

Expected:

```

sum: 10
empty sum: 0

```

Example: no overloading—distinct names

Save as `main.go` and `go run .` (with `go mod init example` if needed).

```

package main

import (
    "fmt"
    "time"
)

// Go forbids method overloading. Names say what differs.
func greet(name string) string {
    return "hello, " + name
}

func greetWithTimeout(name string, d time.Duration) string {
    // Duration is part of the API surface, not a hidden default.
    _ = d
    return fmt.Sprintf("hello, %s (timeout %s)", name, d)
}

func main() {
    fmt.Println(greet("gopher"))
    fmt.Println(greetWithTimeout("gopher", 2*time.Second))
}

```

Expected:

```
hello, gopher
hello, gopher (timeout 2s)
```

Runnable example

“Boring Go”: zero values, explicit errors, one obvious control flow—nothing clever.

Save as `main.go`. From an empty directory:

```
go mod init example
go run .

package main

import (
    "errors"
    "fmt"
)

// Config is usable at its zero value: no constructor magic required.
type Config struct {
    Host    string
    Port    int
    Verbose bool
}

func (c Config) addr() string {
    host := c.Host
    if host == "" {
        host = "localhost"
    }
    port := c.Port
    if port == 0 {
        port = 8080
    }
    return fmt.Sprintf("%s:%d", host, port)
}

func connect(cfg Config) error {
    if cfg.Port < 0 {
        return errors.New("port cannot be negative")
    }
    if cfg.Verbose {
        fmt.Println("connecting with verbose logging")
    }
}
```

```

        fmt.Println("listening on", cfg.addr())
        return nil
    }

func main() {
    // Zero-value config is intentional and readable.
    var cfg Config
    if err := connect(cfg); err != nil {
        fmt.Println("error:", err)
        return
    }

    // Override only what differs from defaults.
    cfg.Host = "0.0.0.0"
    cfg.Port = 9090
    cfg.Verbose = true
    if err := connect(cfg); err != nil {
        fmt.Println("error:", err)
        return
    }

    // Explicit failure path-clear is better than clever.
    cfg.Port = -1
    if err := connect(cfg); err != nil {
        fmt.Println("error:", err)
    }
}

```

Expected output (illustrative):

```

listening on localhost:8080
connecting with verbose logging
listening on 0.0.0.0:9090
error: port cannot be negative

```

What to notice: - Zero values are defaults you can rely on ("", 0, false). - Error handling is visible: every call site checks `err`. - No inheritance, no DI container—just a struct and functions. - Control flow is linear and reviewable (including by AI reviewers).

Try next: Add a `Timeout` field with a zero-value default of 30s and print it in `connect`.

The 3-Layer Practice System

The 3-Layer Practice System: Read, Write, Ship

Learning a language like Go isn't about memorizing syntax; it's about building muscle memory. In 2026, the most effective way to master Go is through a cyclic system we call **Read, Write, Ship**.

Layer 1: Read (Input)

You cannot write idiomatic Go if you haven't seen idiomatic Go. Good writers are always avid readers.

What to Read

1. **The Standard Library:** The Go standard library is the gold standard.

- Read `net/http` to see how interfaces are used.
- Read `encoding/json` to understand reflection and tags.
- Read `time` to see efficient data structures.

2. **High-Quality Open Source:**

- [Caddy Server](#): For modular architecture and interfaces.
- [HashiCorp Vault/Terraform](#): For CLI patterns and plugin systems.
- [NATS Server](#): For high-performance concurrency.

How to Read

Don't just skim. **Trace.** Pick a function call (like `http.ListenAndServe`) and follow it down the rabbit hole until you hit a syscall or assembly.

Layer 2: Write (Output)

Tutorials give you a false sense of competence. You must build things that break.

The “Clone” Technique

Don't wait for a unique idea. Clone existing tools to understand how they work. * **Beginner:** Write a CLI tool (clone `ls` or `cat`). * **Intermediate:** Write a load balancer (clone basic Nginx features). * **Advanced:** Write a distributed key-value store (clone a tiny Redis).

Constraints

Force yourself out of comfort zones: * Write a program without using `struct` tags. * Write a concurrent program without `sync.Mutex` (only channels). * Write a web server without a framework (only `net/http`).

Layer 3: Ship (Feedback)

Code on your laptop doesn't count. "Done" means deployed.

The Feedback Loop

1. **Linting:** `golangci-lint` is your first critic. Configure it to be strict.
2. **Code Review:** Even if solo, use PRs. Review your own diffs. AI tools can effectively review PRs now—use them to spot potential bugs.
3. **Production:** functionality implies running in a Linux container, on a cloud provider, with real traffic.
 - Deploy to **Render**, **Leapcell**, or a **VPS**.
 - Set up **Observability** (logs/metrics). You don't know your code until you see it fail in prod.

Summary

- **Read** to load the patterns into your brain.
- **Write** to test your understanding of those patterns.
- **Ship** to validate that your code actually solves the problem in the real world.

"Amateurs practice until they get it right. Professionals practice until they can't get it wrong."

More examples

Example: clone cat (Layer 2 write)

Save as `main.go` and `go run .` (with `go mod init example` if needed).

```
package main

import (
    "fmt"
    "io"
    "os"
```

```

    "strings"
)

func cat(r io.Reader, w io.Writer) error {
    _, err := io.Copy(w, r)
    return err
}

func main() {
    // Demo when no args: practice without creating files.
    if len(os.Args) < 2 {
        r := strings.NewReader("line one\nline two\n")
        fmt.Print("(demo)\n")
        if err := cat(r, os.Stdout); err != nil {
            fmt.Fprintln(os.Stderr, err)
            os.Exit(1)
        }
        return
    }

    for _, path := range os.Args[1:] {
        f, err := os.Open(path)
        if err != nil {
            fmt.Fprintln(os.Stderr, err)
            os.Exit(1)
        }
        err = cat(f, os.Stdout)
        f.Close()
        if err != nil {
            fmt.Fprintln(os.Stderr, err)
            os.Exit(1)
        }
    }
}

```

Expected:

```

(demo)
line one
line two

```

Example: intentional breakage (Layer 2 write)

Save as `main.go` and `go run .` (with `go mod init example` if needed).

```

package main

import (
    "fmt"
    "strconv"
)

// parsePositive demonstrates checking the compiler/runtime reaction to bad input.
func parsePositive(s string) (int, error) {
    n, err := strconv.Atoi(s)
    if err != nil {
        return 0, fmt.Errorf("not an int %q: %w", s, err)
    }
    if n <= 0 {
        return 0, fmt.Errorf("need positive, got %d", n)
    }
    return n, nil
}

func main() {
    for _, s := range []string{"42", "0", "nope"} {
        n, err := parsePositive(s)
        if err != nil {
            fmt.Printf("%q -> error: %v\n", s, err)
            continue
        }
        fmt.Printf("%q -> %d\n", s, n)
    }
}

```

Expected:

```

"42" -> 42
"0" -> error: need positive, got 0
"nope" -> error: not an int "nope": strconv.Atoi: parsing "nope": invalid syntax

```

Runnable example

Layer 2 practice: a tiny clone of `wc -l / wc -w` (read stdin or a string, write counts).

Save as `main.go`. From an empty directory:

```

go mod init example
go run .
printf 'hello world\nfoo bar baz\n' | go run .

```

```

package main

import (
    "bufio"
    "fmt"
    "io"
    "os"
    "strings"
)

func count(r io.Reader) (lines, words, bytes int, err error) {
    br := bufio.NewReader(r)
    for {
        line, e := br.ReadString('\n')
        if len(line) > 0 {
            bytes += len(line)
            // Count a line if it ends with \n or is a final partial line.
            if strings.HasSuffix(line, "\n") || e == io.EOF {
                lines++
            }
            words += len(strings.Fields(line))
        }
        if e == io.EOF {
            break
        }
        if e != nil {
            return lines, words, bytes, e
        }
    }
    return lines, words, bytes, nil
}

func main() {
    // Demo mode when stdin is a terminal: practice without piping.
    info, _ := os.Stdin.Stat()
    var r io.Reader = os.Stdin
    if (info.Mode() & os.ModeCharDevice) != 0 {
        r = strings.NewReader("hello world\nfoo bar\n")
        fmt.Println("(demo input)")
    }

    lines, words, bytes, err := count(r)
    if err != nil {
        fmt.Fprintln(os.Stderr, "count:", err)
        os.Exit(1)
    }
    fmt.Printf("lines=%d words=%d bytes=%d\n", lines, words, bytes)
}

```

```
}
```

Expected output (illustrative):

```
(demo input)
lines=2 words=4 bytes=20
```

With a pipe:

```
lines=2 words=5 bytes=24
```

What to notice: - You wrote a real tool that reads input and prints observables—not a syntax drill. - Stdlib only: `bufio`, `io`, `strings`—good “read the standard library” practice. - Errors are checked; failure exits non-zero (ship-ready habit). - Demo mode lets you re-run without pipes while learning.

Try next: Accept a filename as `os.Args[1]` and fall back to `stdin` when missing.

Transitioning to Go

Transitioning to Go: For Java and Node.js Developers

If you are coming from an Object-Oriented (Java/C#) or Event-Driven (Node.js/JS) background, Go will feel familiar yet frustratingly different. This chapter maps your existing mental models to Go's reality.

For the Java Developer

Java Concept	Go Equivalent	The Shift in Thinking
Class	Struct	Data and behavior are separate. Classes bundle them; Go defines a type (data) and func (t MyType) (behavior).
Interface	Interface	Implicit. You don't implement interfaces. If you have the methods, you satisfy the interface. This decouples packages.
Exception	Error	Errors are values , not control flow events. You check them (if err != nil), you don't catch them.
ThreadPool	Goroutines	Threads are expensive (MBs); Goroutines are cheap (KBs). You can spawn 100k goroutines without blinking.
Annotation	Struct Tag	Used strictly for metadata (JSON, DB), not for behavior injection (like Spring Magic).
Maven/Gradle	Go Modules	Simplistic dependency graph. No mvn install . Just go mod tidy .

The “Spring” Trap

Don’t try to build “Spring in Go”. Dependency Injection containers are largely unnecessary in Go. Pass dependencies explicitly in constructors (struct factories).

```
// Java: @Autowired Service service;
// Go:
func NewServer(db *sql.DB, logger *Logger) *Server {
    return &Server{db: db, logger: logger}
}
```

For the Node.js Developer

Node.js Concept	Go Equivalent	The Shift in Thinking
Promise / Async Await	Blocking Code	Go code <i>looks</i> synchronous but runs concurrently. The runtime handles the I/O scheduling. No “Callback Hell” or unwieldy <code>await</code> chains.
npm	Go Modules	No <code>node_modules</code> black hole. Dependencies are compiled into a single binary.
Event Loop	Go Scheduler	Node has one thread; block it and you die. Go has M:N scheduling; if one goroutine blocks, others keep running on other OS threads.
Dynamic Types	Static Types	You catch typos at compile time, not runtime. <code>interface{}</code> (or <code>any</code>) exists but use it sparingly.
Express/NestJS	net/http	The stdlib is production-ready. You often don’t need a framework. <code>Chi</code> or <code>Echo</code> are light routers, not heavy frameworks.

The “Concurrency” Trap

In Node, you rely on `Promise.all` for concurrency. In Go, you use **Channels** and **WaitGroups**.

```
// Node
await Promise.all([task1(), task2()]);

// Go
var wg sync.WaitGroup
wg.Add(2)
go func() { defer wg.Done(); task1() }()
go func() { defer wg.Done(); task2() }()
wg.Wait()
```

Universal Truths in Go

1. **Composition over Inheritance:** You don't extend classes. You embed structs.
2. **Explicit is better than Implicit:** No magic checking. No global state if avoidable.
3. **Values matter:** Learn distinction between passing a copy (T) vs passing a pointer (*T). In JS/Java, object references are implicit; in Go, pointers are explicit.

Summary

- **Java Docs:** Drop the AbstractFactoryPatterns. Build simple structs.
- **Node Devs:** Embrace the type system and true parallelism (multi-core).
- **Everyone:** Respect the error. `if err != nil` is the heartbeat of a Go program.

More examples

Example: explicit constructor DI (not Spring)

Save as `main.go` and `go run .` (with `go mod init example` if needed).

```
package main

import "fmt"

type Clock interface {
    NowLabel() string
}

type fixedClock struct{ label string }

func (c fixedClock) NowLabel() string { return c.label }

type Server struct {
```

```

    dbName string
    clock  Clock
}

// NewServer takes dependencies as parameters-no container, no annotations.
func NewServer(dbName string, clock Clock) *Server {
    return &Server{dbName: dbName, clock: clock}
}

func (s *Server) Handle() string {
    return fmt.Sprintf("%s @ %s", s.dbName, s.clock.NowLabel())
}

func main() {
    srv := NewServer("users", fixedClock{label: "t0"})
    fmt.Println(srv.Handle())
    srv2 := NewServer("orders", fixedClock{label: "t1"})
    fmt.Println(srv2.Handle())
}

```

Expected:

```

users @ t0
orders @ t1

```

Example: value vs pointer is explicit

Save as main.go and go run . (with go mod init example if needed).

```

package main

import "fmt"

type Counter struct{ N int }

func bumpValue(c Counter) {
    c.N++
}

func bumpPointer(c *Counter) {
    c.N++
}

func main() {
    c := Counter{N: 1}
    bumpValue(c)
}

```

```

    fmt.Println("after value bump:", c.N) // still 1 - copy

    bumpPointer(&c)
    fmt.Println("after pointer bump:", c.N) // 2 - shared
}

```

Expected:

```

after value bump: 1
after pointer bump: 2

```

Runnable example

Java → errors as values; Node → concurrency without `Promise.all` ceremony.

Save as `main.go`. From an empty directory:

```

go mod init example
go run .

package main

import (
    "errors"
    "fmt"
    "sync"
    "time"
)

// Service is a plain struct: data + methods, no class hierarchy.
type Service struct {
    Name string
}

func NewService(name string) *Service {
    // Explicit constructor-style factory-no DI container.
    return &Service{Name: name}
}

func (s *Service) Fetch(id int) (string, error) {
    if id <= 0 {
        return "", errors.New("id must be positive")
    }
    time.Sleep(30 * time.Millisecond)
    return fmt.Sprintf("%s-item-%d", s.Name, id), nil
}

```

```

}

func main() {
    svc := NewService("catalog")

    // Errors are values, not exceptions.
    if _, err := svc.Fetch(0); err != nil {
        fmt.Println("expected failure:", err)
    }

    // Concurrent work looks sequential in each goroutine.
    ids := []int{1, 2, 3}
    results := make([]string, len(ids))
    errs := make([]error, len(ids))

    var wg sync.WaitGroup
    for i, id := range ids {
        wg.Add(1)
        go func(i, id int) {
            defer wg.Done()
            val, err := svc.Fetch(id)
            results[i], errs[i] = val, err
        }(i, id)
    }
    wg.Wait()

    for i := range ids {
        if errs[i] != nil {
            fmt.Printf("id %d error: %v\n", ids[i], errs[i])
            continue
        }
        fmt.Printf("id %d -> %s\n", ids[i], results[i])
    }
}

```

Expected output (illustrative):

```

expected failure: id must be positive
id 1 -> catalog-item-1
id 2 -> catalog-item-2
id 3 -> catalog-item-3

```

What to notice: - No `@Autowired`: dependencies would be constructor parameters (here, none).
 - if `err != nil` replaces try/catch as the normal control path. - `WaitGroup` + goroutines replace `Promise.all` for parallel work. - Struct methods attach behavior without inheritance.

Try next: Pass a `*log.Logger` into `NewService` and log each fetch—explicit DI.

Installing and Running Go

Overview

Setting up a Go development environment is straightforward. This chapter covers installation, verification, and the fundamental `go run` and `go build` workflow that you'll use throughout your Go journey.

Installation

macOS

Using Homebrew (recommended):

```
brew install go
```

Or download from go.dev/dl and run the installer package.

Linux

Download and extract:

```
# Download the latest version (check go.dev/dl for current version)
wget https://go.dev/dl/go1.27.1.linux-amd64.tar.gz

# Remove any previous installation and extract
sudo rm -rf /usr/local/go
sudo tar -C /usr/local -xzf go1.27.1.linux-amd64.tar.gz

# Add to PATH (add to ~/.bashrc or ~/.zshrc)
export PATH=$PATH:/usr/local/go/bin
```

Windows

Download the MSI installer from go.dev/dl and run it. The installer adds Go to your PATH automatically.

Verify Installation

```
$ go version
go version go1.27.1 darwin/arm64
```

Go 1.27 requires **macOS 13 Ventura or later**. Linux and Windows binaries from go.dev/dl remain the supported install path; always pick the current patch (1.27.1 at this writing).

Environment Variables

Variable	Purpose	Default
GOROOT	Go installation directory	Auto-detected
GOPATH	Workspace directory	\$HOME/go
GOBIN	Binary installation directory	\$GOPATH/bin

Check your environment:

```
$ go env
# Shows all Go environment variables

$ go env GOPATH
/Users/yourname/go
```

Your First Go Program

Create a file named `hello.go`:

```
package main

import "fmt"

func main() {
    fmt.Println("Hello, Go!")
}
```

The `go run` Command

`go run` compiles and executes in one step—perfect for development:

```
$ go run hello.go
Hello, Go!
```

How It Works

1. Compiles the source to a temporary binary
2. Executes that binary
3. Cleans up the temporary file

Running Multiple Files

```
# Run multiple files
$ go run main.go utils.go

# Run all Go files in current directory
$ go run .
```

The go build Command

go build creates a permanent executable:

```
$ go build hello.go
$ ls
hello    hello.go

$ ./hello
Hello, Go!
```

Common Build Options

```
# Specify output name
$ go build -o myapp hello.go

# Build for different OS/architecture
$ GOOS=linux GOARCH=amd64 go build -o myapp-linux

# Build with optimizations (strip debug info)
$ go build -ldflags="-s -w" -o myapp
```


Cross-Compilation

Go makes cross-compilation trivial:

```
# Build for Linux from macOS
$ GOOS=linux GOARCH=amd64 go build -o app-linux

# Build for Windows
$ GOOS=windows GOARCH=amd64 go build -o app.exe

# Build for ARM (Raspberry Pi)
$ GOOS=linux GOARCH=arm GOARM=7 go build -o app-arm
```

The Build Lifecycle

Source Code (.go)	Compiler	Binary (executable)
----------------------	----------	------------------------

<code>go run</code> (temp bin)	<code>go build</code> (perm. bin)
-----------------------------------	--------------------------------------

IDE and Editor Setup

Visual Studio Code

1. Install the [Go extension](#)
2. Open any .go file
3. Accept prompts to install Go tools

Recommended Tools (installed automatically)

- `gopls` - Language server
- `dlv` - Debugger
- `staticcheck` - Linter

GoLand

JetBrains GoLand is a commercial IDE with excellent Go support out of the box.

Neovim/Vim

Use `gopls` with your LSP client of choice (`nvim-lspconfig`, `coc.nvim`, etc.).

Project Structure Basics

```
myproject/
  go.mod          # Module definition
  go.sum          # Dependency checksums
  main.go         # Entry point
  internal/       # Private packages
    config/
      config.go
  pkg/            # Public packages (optional)
    utils/
      utils.go
```

Common Pitfalls

GOPATH vs Modules

Modern Go uses **modules** (introduced in Go 1.11). You don't need to work inside `$GOPATH/src` anymore:

```
# Initialize a new module anywhere
$ mkdir myproject && cd myproject
$ go mod init github.com/username/myproject
```

Executable vs Library

Only package `main` with a `main()` function creates executables:

```
// This creates a binary
package main

func main() { }
```

```
// This is a library (cannot run directly)
package mylib

func Helper() { }
```

Summary

Command	Purpose
<code>go run</code>	Compile and run (temporary)
<code>go build</code>	Compile to binary
<code>go install</code>	Compile and install to \$GOBIN
<code>go env</code>	Show environment variables
<code>go version</code>	Show Go version

Exercises

1. Install Go and verify with `go version`
2. Create a “Hello, World!” program and run it with `go run`
3. Build the same program with `go build` and execute the binary
4. Cross-compile for a different operating system

Related Topics

- [Why Go Exists](#)
- [Learning Go in 2026](#)
- [How Go Code Is Organized](#)
- [Core Go Commands](#)

More examples

Example: classic Hello, World

Save as `main.go` and `go run .` (with `go mod init example` if needed).

```
package main

import "fmt"

func main() {
    fmt.Println("Hello, World!")
}
```

```
    fmt.Println("from go run")
}
```

Expected:

```
Hello, World!
from go run
```

Example: exit codes and stderr

Save as `main.go` and `go run .` (with `go mod init example` if needed).

```
package main

import (
    "fmt"
    "os"
)

func main() {
    if len(os.Args) < 2 {
        fmt.Fprintln(os.Stderr, "usage: go run . <name>")
        os.Exit(2)
    }
    fmt.Println("hello,", os.Args[1])
}
```

Expected (with `go run . gopher`):

```
hello, gopher
```

Expected (with bare `go run .`):

```
usage: go run . <name>
```

Runnable example

Print Go runtime and build info from inside a program (complements `go version` / `go env` on the CLI).

Save as `main.go`. From an empty directory:

```
go mod init example
go run .
go build -o hello .
```

```

./hello

package main

import (
    "fmt"
    "runtime"
    "runtime/debug"
)

func main() {
    fmt.Printf("Go version (runtime): %s\n", runtime.Version())
    fmt.Printf("OS/Arch: %s/%s\n", runtime.GOOS, runtime.GOARCH)
    fmt.Printf("NumCPU: %d\n", runtime.NumCPU())

    info, ok := debug.ReadBuildInfo()
    if !ok {
        fmt.Println("build info: unavailable")
        return
    }
    fmt.Printf("module path: %s\n", info.Main.Path)
    fmt.Printf("go version (build): %s\n", info.GoVersion)
    for _, s := range info.Settings {
        switch s.Key {
            case "GOOS", "GOARCH", "vcs.revision", "vcs.modified", "-compiler":
                fmt.Printf("setting %s=%s\n", s.Key, s.Value)
        }
    }
    fmt.Println("Hello from a complete Go program")
}

```

Expected output (illustrative; values depend on your machine):

```

Go version (runtime): go1.22.5
OS/Arch: darwin/arm64
NumCPU: 8
module path: example
go version (build): go1.22.5
setting GOOS=darwin
setting GOARCH=arm64
setting -compiler=gc
Hello from a complete Go program

```

What to notice: - `runtime.Version()` matches the idea of `go version` from inside your app.
 - `debug.ReadBuildInfo()` reports the module path from `go mod init`. - The same source works with `go run` and `go build`. - Cross-compile targets show up as `GOOS/GOARCH` in build settings when set.

Try next: Build with `GOOS=linux GOARCH=amd64 go build -o hello-linux .` and inspect the binary name; run the program natively again and compare OS/Arch.

How Go Code Is Organized

Overview

Go has strong opinions about code organization. Understanding packages, imports, and project structure is essential for writing maintainable Go programs and for your code to work correctly with the Go toolchain.

Packages: The Building Blocks

Every Go file belongs to a package. Packages group related code and control visibility.

```
// File: greet.go
package greeting // Package declaration (first line)

import "fmt"

func Hello(name string) string {
    return fmt.Sprintf("Hello, %s!", name)
}
```

Package Rules

Rule	Description
All files in a directory belong to the same package	You can't have <code>package foo</code> and <code>package bar</code> in the same folder
Package name matches directory name (by convention)	Directory <code>server/</code> contains <code>package server</code>
<code>package main</code> is special	It defines an executable, not a library

The main Package

Only `package main` can produce an executable. It must have a `main()` function:

```

package main

import "fmt"

func main() {
    fmt.Println("This is the entry point")
}

```

Import System

Basic Imports

```

import "fmt"           // Standard library
import "net/http"      // Nested standard library package
import "github.com/user/repo" // External package

```

Import Block (Preferred Style)

```

import (
    // Standard library (grouped first)
    "fmt"
    "net/http"

    // Third-party packages (blank line separator)
    "github.com/gin-gonic/gin"

    // Internal packages (blank line separator)
    "myproject/internal/config"
)

```

Import Aliases

```

import (
    "crypto/rand"
    mrand "math/rand" // Alias to avoid collision
)

// Usage
cryptoBytes, _ := rand.Read(buf)
randomInt := mrand.Intn(100)

```


Blank Imports (Side Effects Only)

```
import (  
    "database/sql"  
    _ "github.com/lib/pq" // Import for side effects (driver registration)  
)
```

Dot Imports (Avoid in Production)

```
import . "fmt"  
  
// Now you can use Println directly  
Println("Hello") // Instead of fmt.Println
```

Visibility: Exported vs Unexported

Go uses capitalization to control visibility:

```
package user  
  
// Exported (public) - starts with uppercase  
type User struct {  
    Name string // Exported field  
    Email string // Exported field  
    age int // unexported field (private)  
}  
  
// Exported function  
func CreateUser(name string) *User {  
    return &User{Name: name}  
}  
  
// unexported function (private)  
func validate(u *User) bool {  
    return len(u.Name) > 0  
}
```

Name	Visibility
User	Exported (accessible from other packages)
Name	Exported field
age	Unexported (only accessible within <code>user</code> package)
CreateUser	Exported function
validate	Unexported function

Modules: Modern Dependency Management

A **module** is a collection of packages with a `go.mod` file at the root.

Creating a Module

```
$ mkdir myproject && cd myproject
$ go mod init github.com/username/myproject
```

This creates `go.mod`:

```
module github.com/username/myproject
```

```
go 1.27
```

Module Structure

```
github.com/username/myproject/
  go.mod
  go.sum
  main.go
  config/
    config.go      # package config
  internal/
    database/
      db.go        # package database (private)
  pkg/
    api/
      api.go       # package api (public)
```

Internal Packages

The `internal/` directory has special meaning: packages inside it can only be imported by code within the same module tree.

```
myproject/
  internal/
    secret/
      secret.go    # Only myproject can import this
  cmd/
    app/
      main.go      # Can import internal/secret
```

```
// This works (same module)
import "github.com/username/myproject/internal/secret"

// This fails (different module trying to import internal)
// Error: use of internal package not allowed
```

Standard Project Layout

While Go doesn't mandate a structure, this layout is widely adopted:

```
myproject/
  cmd/                # Main applications
    myapp/
      main.go
    worker/
      main.go
  internal/           # Private packages
    config/
    database/
    service/
  pkg/                # Public libraries (optional)
    api/
  api/                # OpenAPI/Swagger specs
  web/                # Web assets
  scripts/            # Build/install scripts
  docs/               # Documentation
  go.mod
  go.sum
  README.md
```

cmd/ Directory

Each subdirectory in `cmd/` becomes a separate binary:

```
$ go build ./cmd/myapp
$ go build ./cmd/worker
```

Package Naming Conventions

Good	Bad	Why
user	userPackage	Avoid redundant suffixes
http	httputil	Only add suffix if necessary

Good	Bad	Why
<code>strconv</code>	<code>stringconversion</code>	Short but clear
<code>io</code>	<code>inputoutput</code>	Abbreviations OK if common

Avoid Meaningless Names

- `util` → Use specific names like `stringutil`, `timeutil`
- `common` → Be specific about what's common
- `misc` → Break into focused packages

Circular Import Prevention

Go prohibits circular imports. If you have:

```
package a → imports → package b
package b → imports → package a // Error!
```

Solutions: 1. **Extract common code** to a third package 2. **Use interfaces** to break the dependency 3. **Restructure** your packages

Summary

Concept	Purpose
Package	Groups related code, controls visibility
Module	Collection of packages with version info
<code>internal/</code>	Private packages within a module
Uppercase	Exported (public)
Lowercase	Unexported (private)

Exercises

1. Create a module with two packages and have one import the other
2. Create an `internal/` package and verify external code can't import it
3. Create a project with `cmd/` containing multiple applications

Related Topics

- [Core Go Commands](#)
- [Dependency Management](#)

More examples

Example: exported vs unexported names

Save as `main.go` and `go run .` (with `go mod init example` if needed).

```
package main

import "fmt"

// User is exported (other packages could import and use it).
type User struct {
    Name string // exported field
    id   int    // unexported field - same package only
}

func NewUser(name string, id int) User {
    return User{Name: name, id: id}
}

func (u User) ID() int { return u.id }

func main() {
    u := NewUser("ada", 7)
    fmt.Println("name:", u.Name)
    fmt.Println("id:", u.ID())
    // fmt.Println(u.id) // works only inside this package
}
```

Expected:

```
name: ada
id: 7
```

Example: package main entry point

Save as `main.go` and `go run .` (with `go mod init example` if needed).

```
package main

import "fmt"

// helpers in the same package share unexported names.
func banner() string { return "app" }

func main() {
```

```

    fmt.Println(banner(), "starting")
    fmt.Println(banner(), "ready")
}

```

Expected:

```

app starting
app ready

```

Runnable example

Single-file demo of packages' visibility rules and `package main` as the executable entry point. (In real projects, exported helpers live in other packages; capitalization still decides export.)

Save as `main.go`. From an empty directory:

```

go mod init example
go run .

package main

import (
    "fmt"
    "strings"
)

// Exported-style names start with uppercase; unexported stay lowercase.
// In one package both are visible; other packages would only see uppercase.

type User struct {
    Name string // exported field
    age  int      // unexported field
}

func NewUser(name string, age int) (User, error) {
    if !validName(name) {
        return User{}, fmt.Errorf("invalid name %q", name)
    }
    return User{Name: name, age: age}, nil
}

func validName(name string) bool {
    return strings.TrimSpace(name) != ""
}

```

```

func (u User) Greeting() string {
    return fmt.Sprintf("Hello, %s (age %d)", u.Name, u.age)
}

func main() {
    u, err := NewUser("Ada", 36)
    if err != nil {
        fmt.Println("error:", err)
        return
    }
    fmt.Println(u.Greeting())
    fmt.Printf("type=%T name=%q age_via_method_only\n", u, u.Name)

    if _, err := NewUser(" ", 1); err != nil {
        fmt.Println("rejected:", err)
    }
}

```

Expected output (illustrative):

```

Hello, Ada (age 36)
type=main.User name="Ada" age_via_method_only
rejected: invalid name " "

```

What to notice: - package main + func main is what go run executes. - Name is uppercase (exportable); age and validName are package-private style. - Factory NewUser validates before constructing—common package API shape. - Imports use the standard library grouping style.

Try next: Split User into a second file in the same directory still declaring package main and run go run . again.

Core Go Commands

Overview

The `go` command is your Swiss Army knife for Go development. This chapter provides a deep dive into the essential commands you'll use daily, from building and testing to analyzing and maintaining your code.

Command Summary

Command	Purpose
<code>go build</code>	Compile packages and dependencies
<code>go run</code>	Compile and run Go program
<code>go test</code>	Run tests
<code>go get</code>	Add dependencies to module
<code>go mod</code>	Module maintenance
<code>go fmt</code>	Format source code
<code>go vet</code>	Report suspicious constructs
<code>go doc</code>	Show documentation
<code>go install</code>	Compile and install packages
<code>go clean</code>	Remove object files

`go build` - Compiling Code

Basic Usage

```
# Build current package
$ go build

# Build specific file
$ go build main.go

# Build with custom output name
$ go build -o myapp

# Build specific package
```



```
$ go build ./cmd/server
```

Build Flags

```
# Verbose output (show packages being compiled)
$ go build -v ./...

# Rebuild all packages (ignore cache)
$ go build -a ./...

# Print commands but don't run them
$ go build -n

# Show build work directory (don't delete)
$ go build -work
```

Cross-Compilation

The toolchain ships the runtime for each target. Add `CGO_ENABLED=0` so you do not accidentally link the *host* libc (a Linux AMD64 binary with glibc will not run on Alpine).

```
# Build for Linux (AMD64), fully static
$ GOOS=linux GOARCH=amd64 CGO_ENABLED=0 go build -o app-linux

# Build for Windows
$ GOOS=windows GOARCH=amd64 CGO_ENABLED=0 go build -o app.exe

# Build for macOS ARM (Apple Silicon)
$ GOOS=darwin GOARCH=arm64 CGO_ENABLED=0 go build -o app-macos

# Build for Raspberry Pi
$ GOOS=linux GOARCH=arm GOARM=7 CGO_ENABLED=0 go build -o app-arm
```

On Linux, file `app-linux` should say **statically linked**. Why that works — Go talks to the kernel without libc — is in [Go as a Systems Language](#).

Linker Flags

```
# Strip debug info (smaller binary)
$ go build -ldflags="-s -w" -o app

# Embed version information
$ go build -ldflags="-X main.version=1.0.0" -o app
```

```
// In your code
var version = "dev" // Overwritten by -ldflags

func main() {
    fmt.Println("Version:", version)
}
```

go run - Quick Execution

```
# Run a single file
$ go run main.go

# Run multiple files
$ go run main.go helper.go

# Run all files in directory
$ go run .

# Run with arguments
$ go run main.go --port=8080 --debug
```

go test - Testing

```
# Run all tests in current package
$ go test

# Run all tests recursively
$ go test ./...

# Verbose output
$ go test -v

# Run specific test
$ go test -run TestFunctionName

# Run with coverage
$ go test -cover

# Generate coverage report
$ go test -coverprofile=coverage.out
$ go tool cover -html=coverage.out
```

```
# Run benchmarks
$ go test -bench=.

# Test with race detector
$ go test -race ./...
```

go get - Dependency Management

```
# Add a dependency (latest version)
$ go get github.com/gin-gonic/gin

# Add specific version
$ go get github.com/gin-gonic/gin@v1.9.0

# Add specific commit
$ go get github.com/gin-gonic/gin@a1b2c3d

# Update a dependency
$ go get -u github.com/gin-gonic/gin

# Update all dependencies
$ go get -u ./...

# Update only patch versions
$ go get -u=patch ./...
```

go mod - Module Operations

```
# Initialize a new module
$ go mod init github.com/username/project

# Download dependencies
$ go mod download

# Tidy up go.mod (add missing, remove unused)
$ go mod tidy

# Verify dependencies
$ go mod verify

# Create vendor directory
$ go mod vendor
```

```
# Show dependency graph
$ go mod graph

# Explain why a dependency is needed
$ go mod why github.com/some/package

# Edit go.mod programmatically
$ go mod edit -require github.com/pkg/errors@v0.9.1
```

go fmt / gofmt - Formatting

```
# Format all Go files in current directory
$ go fmt ./...

# Use gofmt directly with options
$ gofmt -w main.go      # Write changes to file
$ gofmt -d main.go      # Show diff
$ gofmt -l .            # List files that need formatting
$ gofmt -s main.go      # Simplify code
```

go vet - Static Analysis

```
# Vet current package
$ go vet

# Vet all packages
$ go vet ./...

# Run specific analyzers
$ go vet -composites=false ./...
```

Common issues go vet catches: - Printf format string mismatches - Unreachable code - Suspicious mutex usage - Struct field tag issues

go doc - Documentation

```
# Show package documentation
$ go doc fmt

# Show function documentation
```

```
$ go doc fmt.Println

# Show type documentation
$ go doc http.Client

# Show all documentation
$ go doc -all fmt

# Start documentation server
$ godoc -http=:6060
# Then visit http://localhost:6060
```

go install - Installing Binaries

```
# Install command to $GOBIN
$ go install

# Install specific version of a tool
$ go install golang.org/x/tools/gopls@latest

# Install from specific package
$ go install ./cmd/myapp
```

go clean - Cleanup

```
# Remove object files
$ go clean

# Remove cached build files
$ go clean -cache

# Remove test cache
$ go clean -testcache

# Remove all cached data
$ go clean -cache -testcache -modcache

# Show what would be removed
$ go clean -n
```

go env - Environment

```
# Show all environment variables
$ go env

# Show specific variable
$ go env GOPATH

# Set environment variable
$ go env -w GOBIN=/usr/local/bin

# Unset environment variable
$ go env -u GOBIN
```

go list - Package Information

```
# List current package
$ go list

# List all packages
$ go list ./...

# JSON output
$ go list -json ./...

# List dependencies
$ go list -m all

# List with template
$ go list -f '{{.ImportPath}} -> {{.Deps}}' ./...
```

go generate - Code Generation

```
# Run all generate directives
$ go generate ./...

# Run with verbose output
$ go generate -v ./...

// In source file
//go:generate stringer -type=Status
```

```
type Status int
```

Tool Installation Best Practices

```
# Install development tools
$ go install golang.org/x/tools/gopls@latest           # Language server
$ go install github.com/go-delve/delve/cmd/dlv@latest # Debugger
$ go install honnef.co/go/tools/cmd/staticcheck@latest # Linter

# Verify installation
$ which gopls
$ which dlv
```

Common Workflows

Development Cycle

```
# 1. Write code
# 2. Format
$ go fmt ./...
# 3. Vet
$ go vet ./...
# 4. Test
$ go test ./...
# 5. Build
$ go build -o app ./cmd/server
```

Release Build

```
# Production build with version
$ go build \
  -ldflags="-s -w -X main.version=$(git describe --tags)" \
  -o bin/app \
  ./cmd/server
```

Summary

Task	Command
Quick run	go run main.go

Task	Command
Build binary	<code>go build -o app</code>
Run tests	<code>go test ./...</code>
Format code	<code>go fmt ./...</code>
Check issues	<code>go vet ./...</code>
Add dependency	<code>go get github.com/pkg@version</code>
Clean modules	<code>go mod tidy</code>
View docs	<code>go doc package.Function</code>

Related Topics

- [Formatting, Vetting, and Documentation](#)
- [Dependency Management](#)

More examples

Example: flags for go run experiments

Save as `main.go` and `go run .` (with `go mod init example` if needed).

```
package main

import (
    "flag"
    "fmt"
)

func main() {
    name := flag.String("name", "world", "who to greet")
    n := flag.Int("n", 1, "repeat count")
    flag.Parse()

    for i := 0; i < *n; i++ {
        fmt.Printf("hello, %s\n", *name)
    }
}
```

Expected (`go run . -name gopher -n 2`):

```
hello, gopher
hello, gopher
```


Example: build-time GOOS/GOARCH report

Save as `main.go` and `go run .` (with `go mod init example` if needed).

```
package main

import (
    "fmt"
    "runtime"
)

func main() {
    // Same values you target with GOOS/GOARCH when cross-compiling.
    fmt.Println("GOOS:", runtime.GOOS)
    fmt.Println("GOARCH:", runtime.GOARCH)
    fmt.Println("compiler:", runtime.Compiler)
}
```

Expected (illustrative; machine-specific):

```
GOOS: darwin
GOARCH: arm64
compiler: gc
```

Runnable example

A small app that prints a version string—ready for `go run`, `go build -ldflags`, and `go test` workflows.

Save as `main.go`. From an empty directory:

```
go mod init example
go run .
go build -ldflags="-X main.version=1.2.3" -o app .
./app
```

```
package main

import (
    "fmt"
    "os"
    "runtime"
)

// Overwritten at link time: go build -ldflags="-X main.version=1.2.3"
var version = "dev"
```

```

func main() {
    fmt.Printf("app version=%s go=%s %s/%s\n",
        version, runtime.Version(), runtime.GOOS, runtime.GOARCH)

    if len(os.Args) > 1 && os.Args[1] == "-v" {
        fmt.Println("verbose: argv=", os.Args)
    }
}

```

Expected output (illustrative):

```

# go run .
app version=dev go=go1.22.5 darwin/arm64

# after go build -ldflags="-X main.version=1.2.3" -o app && ./app
app version=1.2.3 go=go1.22.5 darwin/arm64

```

What to notice: - `go run` uses the source default (`dev`); `go build -ldflags` rewrites the variable. - `os.Args` is how CLI flags start before you reach `flag` package patterns. - Same file is the target of `go build`, `go run`, and (later) tests of pure helpers. - Runtime metadata is available without shelling out to `go env`.

Try next: Run `go fmt .` and `go vet .` on this directory, then add a `main_test.go` that checks `version` is non-empty.

Formatting, Vetting, and Documentation

Overview

Go has a unique culture: there's one blessed way to format code, a built-in static analyzer for common mistakes, and a documentation system that extracts docs from comments. This chapter covers `gofmt`, `go vet`, and `godoc` standards.

Code Formatting with `gofmt`

Why Gofmt Exists

Unlike other languages where style guides vary (tabs vs. spaces, brace placement), Go has a single canonical format enforced by `gofmt`. This eliminates style debates and ensures all Go code looks the same.

“Gofmt’s style is no one’s favorite, yet `gofmt` is everyone’s favorite.” — Rob Pike

Basic Usage

```
# Format a file (print to stdout)
$ gofmt main.go

# Format and overwrite file
$ gofmt -w main.go

# Format all files recursively
$ gofmt -w .

# Using go fmt (recommended)
$ go fmt ./...
```

Formatting Rules

`gofmt` enforces:

```
// BEFORE (your style)
func foo(x int,y string){
if x>0{
return
}
}

// AFTER (gofmt style)
func foo(x int, y string) {
    if x > 0 {
        return
    }
}
```

Rule	Gofmt Standard
Indentation	Tabs
Spacing	Spaces around operators
Braces	Opening brace on same line
Line length	No limit (but keep reasonable)
Blank lines	Strategic for readability

Code Simplification

Use `-s` to simplify code:

```
$ gofmt -s -w main.go
```

```
// Before
s[a:len(s)]
for x, _ := range v {}
[]int{1, 2, 3}[0:2]

// After -s
s[a:]
for x := range v {}
[]int{1, 2, 3}[:2]
```

Import Formatting with goimports

`goimports` does everything `gofmt` does, plus it manages imports:

```
# Install
$ go install golang.org/x/tools/cmd/goimports@latest
```

```

# Format and fix imports
$ goimports -w main.go

// Before (missing import, unused import)
package main
import (
    "unused"
)
func main() {
    fmt.Println("Hello")
}

// After goimports
package main

import "fmt"

func main() {
    fmt.Println("Hello")
}

```

Static Analysis with go vet

What Vet Catches

go vet detects code that compiles but is probably wrong:

```
$ go vet ./...
```

Common Issues Detected

Printf Format Errors

```

// go vet catches this
fmt.Printf("%d", "string") // wrong type
fmt.Printf("%s %s", name)  // wrong number of args

```

Unreachable Code

```
func example() int {  
    return 42  
    fmt.Println("never runs") // go vet: unreachable code  
}
```

Suspicious Loop Variables

```
// go vet warns about this common bug  
for i, v := range values {  
    go func() {  
        fmt.Println(i, v) // captures loop variable  
    }()  
}
```

Useless Assignments

```
x := 5  
x = x // go vet: self-assignment
```

Invalid Struct Tags

```
type User struct {  
    Name string `json:name` // Missing quotes around "name"  
}
```

Running Specific Checks

```
# List available analyzers  
$ go tool vet help  
  
# Run specific analyzer  
$ go vet -composites=false ./...  
$ go vet -printf=true ./...
```

Enhanced Linting with staticcheck

staticcheck provides additional checks beyond go vet:

```
# Install
$ go install honnef.co/go/tools/cmd/staticcheck@latest

# Run
$ staticcheck ./...
```

Checks include: - Unused code - Deprecated function usage - Simplification suggestions - Performance improvements - Common bugs

Documentation with godoc

Writing Documentation

Go extracts documentation from comments directly before declarations:

```
// Package math provides basic mathematical operations.
// It includes functions for arithmetic, trigonometry, and more.
package math

// Pi represents the mathematical constant .
const Pi = 3.14159

// Add returns the sum of two integers.
// It handles overflow by wrapping around.
func Add(a, b int) int {
    return a + b
}

// Calculator provides stateful mathematical operations.
type Calculator struct {
    // Result holds the current calculation result.
    Result float64
}

// Add adds n to the current result.
func (c *Calculator) Add(n float64) {
    c.Result += n
}
```

Documentation Conventions

Rule	Example
Start with name	// Add returns the sum...
Complete sentences	End with period

Rule	Example
First sentence is summary	Shown in package lists
Blank line for paragraphs	Separate blocks

Code Examples in Docs

Create examples in `*_test.go` files:

```
// example_test.go
package math_test

import (
    "fmt"
    "myproject/math"
)

func ExampleAdd() {
    result := math.Add(2, 3)
    fmt.Println(result)
    // Output: 5
}

func ExampleCalculator_Add() {
    c := &math.Calculator{}
    c.Add(10)
    c.Add(5)
    fmt.Println(c.Result)
    // Output: 15
}
```

These examples: - Appear in documentation - Are tested by `go test` - Show real usage

Viewing Documentation

```
# Command line
$ go doc fmt
$ go doc fmt.Println
$ go doc -all fmt
$ go doc -ex bytes.Buffer           # executable examples (Go 1.27+)
$ go doc example.com/pkg@v1.2.3     # docs for a module version (Go 1.27+)

# Local web server
$ go install golang.org/x/tools/cmd/godoc@latest
$ godoc -http=:6060
```



```
# Visit http://localhost:6060
```

Package Comments

For larger packages, use a `doc.go` file:

```
// doc.go

/*
Package server implements an HTTP server with middleware support.

# Getting Started

Create a new server and add routes:

    srv := server.New()
    srv.Get("/", handleHome)
    srv.Listen(":8080")

# Middleware

Add middleware to process all requests:

    srv.Use(server.Logger())
    srv.Use(server.Recovery())

# Configuration

Configure the server using options:

    srv := server.New(
        server.WithTimeout(30 * time.Second),
        server.WithMaxBodySize(1 << 20),
    )
*/
package server
```

Combining Tools in Workflow

Pre-commit Hook

```
#!/bin/bash
# .git/hooks/pre-commit

# Format
gofmt -l -w .

# Vet
go vet ./...
if [ $? -ne 0 ]; then
    echo "go vet failed"
    exit 1
fi

# Staticcheck
staticcheck ./...
if [ $? -ne 0 ]; then
    echo "staticcheck failed"
    exit 1
fi
```

Makefile Integration

```
.PHONY: check fmt vet lint

fmt:
    gofmt -w .

vet:
    go vet ./...

lint:
    staticcheck ./...

check: fmt vet lint
    @echo "All checks passed"
```

CI Pipeline (GitHub Actions)

```
- name: Check formatting
  run: |
    if [ "$(gofmt -l . | wc -l)" -gt 0 ]; then
      echo "Code is not formatted"
      gofmt -d .
      exit 1
    fi

- name: Vet
  run: go vet ./...

- name: Staticcheck
  run: |
    go install honnef.co/go/tools/cmd/staticcheck@latest
    staticcheck ./...
```

Summary

Tool	Purpose	Usage
gofmt	Format code	<code>gofmt -w .</code>
goimports	Format + manage imports	<code>goimports -w .</code>
go vet	Catch common mistakes	<code>go vet ./...</code>
staticcheck	Enhanced linting	<code>staticcheck ./...</code>
godoc	Generate documentation	<code>godoc -http=:6060</code>

Related Topics

- [Core Go Commands](#)
- [Testing Fundamentals](#)

More examples

Example: godoc-style exported comments

Save as `main.go` and `go run .` (with `go mod init example` if needed).

```
package main

import (
```

```

    "fmt"
    "strings"
)

// Title returns s with the first letter of each space-separated word uppercased.
// Empty input yields an empty string.
func Title(s string) string {
    parts := strings.Fields(strings.ToLower(s))
    for i, p := range parts {
        if p == "" {
            continue
        }
        parts[i] = strings.ToUpper(p[:1]) + p[1:]
    }
    return strings.Join(parts, " ")
}

// JoinNonEmpty joins non-empty parts with sep.
func JoinNonEmpty(sep string, parts ...string) string {
    var keep []string
    for _, p := range parts {
        if p != "" {
            keep = append(keep, p)
        }
    }
    return strings.Join(keep, sep)
}

func main() {
    fmt.Println(Title("hello go"))
    fmt.Println(JoinNonEmpty("/", "a", "", "b"))
}

```

Expected:

```

Hello Go
a/b

```

Example: a construct go vet cares about

Save as `main.go` and `go run .` (with `go mod init example` if needed).

```

package main

import "fmt"

```

```

func main() {
    // Lock-free counter demo is fine; the classic vet catch is printf args.
    name := "gopher"
    // Correct: verbs match arguments.
    fmt.Printf("hello %s count=%d\n", name, 3)

    // Intentionally correct form of a common mistake:
    // fmt.Printf("hello %s\n", name, 3) // vet: Printf format %s reads arg but has 2 extra
    fmt.Println("vet clean: format verbs match args")
}

```

Expected:

```

hello gopher count=3
vet clean: format verbs match args

```

Runnable example

Documented helpers in gofmt style—run the program, then try `go doc` / `go vet` on the same files.

Save as `main.go`. From an empty directory:

```

go mod init example
go fmt .
go vet .
go run .
go doc -all .

```

```

package main

import (
    "fmt"
    "strings"
)

// TitleCase returns s with the first letter of each word uppercased.
// Words are split on Unicode whitespace.
func TitleCase(s string) string {
    fields := strings.Fields(s)
    for i, w := range fields {
        if w == "" {
            continue
        }
        // Manual title for a tiny demo; prefer cases.Title in real code.
        r := []rune(w)

```

```

    r[0] = []rune(strings.ToUpper(string(r[0])))[0]
    if len(r) > 1 {
        fields[i] = string(r[0]) + strings.ToLower(string(r[1:]))
    } else {
        fields[i] = string(r[0])
    }
}
return strings.Join(fields, " ")
}

// Sum returns the sum of nums. An empty slice sums to 0.
func Sum(nums []int) int {
    total := 0
    for _, n := range nums {
        total += n
    }
    return total
}

func main() {
    fmt.Println(TitleCase("hello gofmt and godoc"))
    fmt.Printf("sum=%d\n", Sum([]int{1, 2, 3, 4}))
    // Intentional-looking pattern that go vet is happy with:
    fmt.Printf("count=%d name=%s\n", 2, "tools")
}

```

Expected output (illustrative):

```

Hello Gofmt And Godoc
sum=10
count=2 name=tools

```

What to notice: - Comments sit directly above `TitleCase` and `Sum`—`go doc` surfaces them. - `go fmt` will rewrite spacing/imports if you mess them up on purpose. - `go vet` checks `printf` verbs match argument types (try changing `%d` to `%s`). - Zero-value-friendly API: empty slice → sum 0, no panic.

Try next: Break the `Printf` format string (`fmt.Printf("%d", "x")`) and run `go vet` to see the diagnostic.

Dependency Management

Overview

Go modules, introduced in Go 1.11 and default since Go 1.16, provide built-in dependency management. This chapter covers everything about `go.mod`, `go.sum`, versioning, and managing external libraries.

Understanding Go Modules

A module is a collection of packages with a `go.mod` file that defines: - Module path (import path) - Go version - Dependencies and their versions

Creating a Module

```
$ mkdir myproject && cd myproject
$ go mod init github.com/username/myproject
```

This creates `go.mod`:

```
module github.com/username/myproject

go 1.27
```

The `go.mod` File

Anatomy of `go.mod`

```
module github.com/username/myproject

go 1.27

require (
    github.com/gin-gonic/gin v1.9.1
    github.com/stretchr/testify v1.8.4
)
```

```

require (
    // indirect dependencies
    github.com/bytedance/sonic v1.9.1 // indirect
    golang.org/x/net v0.10.0 // indirect
)

exclude (
    github.com/broken/pkg v1.0.0
)

replace (
    github.com/old/module => github.com/new/module v2.0.0
    github.com/local/dev => ../local-dev
)

retract (
    v1.0.0 // Contains security vulnerability
    [v1.1.0, v1.2.0] // Accidental release
)

```

Directives Explained

Directive	Purpose
<code>module</code>	Module path (required)
<code>go</code>	Minimum Go version
<code>require</code>	Dependencies with versions
<code>exclude</code>	Versions to ignore
<code>replace</code>	Substitute modules
<code>retract</code>	Mark versions as broken

The go.sum File

`go.sum` contains cryptographic checksums representing module dependencies:

```

github.com/gin-gonic/gin v1.9.1 h1:4idEAncQnU5cB7Be0kPtxjfCSye0AAm1RORVIqJ+Jmg=
github.com/gin-gonic/gin v1.9.1/go.mod h1:hPrL7YrpYKXt5YId3A/Tn+7r7IyQ4PGNmP1c42GGpvQ=

```

Each entry has: - Module path and version - Hash of module contents (`h1:`) - Hash of `go.mod` file

Never edit `go.sum` manually — it's managed by Go tools.

Adding Dependencies

Using go get

```
# Add latest version
$ go get github.com/gin-gonic/gin

# Add specific version
$ go get github.com/gin-gonic/gin@v1.9.1

# Add latest minor version
$ go get github.com/gin-gonic/gin@v1.9

# Add specific commit
$ go get github.com/gin-gonic/gin@a1b2c3d

# Add from branch
$ go get github.com/gin-gonic/gin@main
```

Automatic Detection

Simply import and run:

```
import "github.com/gin-gonic/gin"

$ go mod tidy # Downloads and records dependency
```

Updating Dependencies

```
# Update specific package (latest minor/patch)
$ go get -u github.com/gin-gonic/gin

# Update to specific version
$ go get github.com/gin-gonic/gin@v1.10.0

# Update all dependencies
$ go get -u ./...

# Update only patch versions (safer)
$ go get -u=patch ./...
```

Semantic Versioning

Go modules expect [Semantic Versioning](#):

v1.2.3

Patch: bug fixes (backward compatible)
Minor: new features (backward compatible)
Major: breaking changes

Major Version Suffixes

For v2+, the module path includes the major version:

```
// go.mod
require github.com/user/project/v2 v2.1.0

// import
import "github.com/user/project/v2"
```

Managing go.mod

go mod tidy

The most important maintenance command:

```
$ go mod tidy
```

This: - Adds missing dependencies - Removes unused dependencies - Updates `go.sum` - For modules with `go 1.27` or later, merges extra `require` blocks into the standard two-block layout (direct, then indirect). Comments on mixed blocks move onto the direct block.

Run this frequently, especially before commits.

go mod download

Pre-download all dependencies:

```
$ go mod download
```

Useful for CI caching or air-gapped environments.

go mod verify

Check that dependencies haven't been modified:

```
$ go mod verify
all modules verified
```

go mod why

Explain why a dependency is needed:

```
$ go mod why golang.org/x/net

# github.com/username/myproject
github.com/username/myproject
github.com/gin-gonic/gin
golang.org/x/net/html
```

go mod graph

Show dependency graph:

```
$ go mod graph
github.com/username/myproject github.com/gin-gonic/gin@v1.9.1
github.com/gin-gonic/gin@v1.9.1 github.com/bytedance/sonic@v1.9.1
...
```

Vendoring

Copy dependencies into your repository:

```
# Create vendor directory
$ go mod vendor

# Build using vendor
$ go build -mod=vendor
```

Vendoring pros: - Reproducible builds - Works offline - Audit dependencies

Vendoring cons: - Bloated repository - Manual updates

Replace Directive

Local Development

Replace a module with a local path:

```
replace github.com/some/package => ../local-package
```

Fork Substitution

Use your fork instead of original:

```
replace github.com/original/pkg => github.com/yourfork/pkg v1.2.3
```

Version Override

Force a specific version:

```
replace github.com/vulnerable/pkg v1.0.0 => github.com/vulnerable/pkg v1.0.1
```

Private Modules

GOPRIVATE

Configure private repository patterns:

```
$ go env -w GOPRIVATE=github.com/mycompany/*,gitlab.internal.com/*
```

Git Credentials

For private repos, configure Git:

```
# Use SSH
$ git config --global url."git@github.com:".insteadOf "https://github.com/"

# Or use access token
$ git config --global url."https://token:${TOKEN}@github.com:".insteadOf "https://github.com/"
```

Dependency Management Best Practices

1. Pin Dependencies

Always use exact versions in production:

```
# Good: exact version
$ go get github.com/pkg/errors@v0.9.1

# Risky: floating version
$ go get github.com/pkg/errors@latest
```

2. Regular Updates

Check for updates periodically:

```
$ go list -m -u all
```

3. Minimal Dependencies

Fewer dependencies = fewer problems: - Check if you really need that package - Consider stdlib alternatives - Watch for transitive dependencies

4. Security Scanning

Use vulnerability scanning:

```
# Built-in (Go 1.18+)
$ go install golang.org/x/vuln/cmd/govulncheck@latest
$ govulncheck ./...
```

Common Issues

Module Not Found

```
# Ensure GOPROXY is set
$ go env GOPROXY
https://proxy.golang.org,direct

# Try direct fetch
$ GOPROXY=direct go get github.com/some/package
```

Version Conflicts

```
# See what's required
$ go mod graph | grep conflicting-package

# Force specific version
$ go get conflicting-package@v1.2.3
```

Checksum Mismatch

```
# Clear cache and retry
$ go clean -modcache
$ go mod download
```

Summary

Command	Purpose
<code>go mod init</code>	Create new module
<code>go mod tidy</code>	Sync dependencies
<code>go get pkg@version</code>	Add/update dependency
<code>go mod download</code>	Pre-fetch dependencies
<code>go mod verify</code>	Verify checksums
<code>go mod vendor</code>	Create vendor directory

Related Topics

- [Workspaces and Multi-Module Development](#)
- [How Go Code Is Organized](#)

More examples

Example: prefer stdlib before go get

Save as `main.go` and `go run .` (with `go mod init example` if needed).

```
package main

import (
    "encoding/json"
    "fmt"
)
```

```

    "os"
)

type Config struct {
    Service string `json:"service"`
    Port    int    `json:"port"`
}

func main() {
    // Many needs are covered by encoding/json, net/http, etc.-no third party.
    raw := []byte(`{"service":"api","port":8080}`)
    var cfg Config
    if err := json.Unmarshal(raw, &cfg); err != nil {
        fmt.Fprintln(os.Stderr, err)
        os.Exit(1)
    }
    fmt.Printf("%s listens on %d\n", cfg.Service, cfg.Port)

    out, _ := json.MarshalIndent(cfg, "", " ")
    fmt.Println(string(out))
}

```

Expected:

```

api listens on 8080
{
  "service": "api",
  "port": 8080
}

```

Example: semantic version labels as data

Save as `main.go` and `go run .` (with `go mod init example` if needed).

```

package main

import (
    "fmt"
    "strconv"
    "strings"
)

// parseSemver is a teaching toy-not a full semver library.
func parseSemver(v string) (major, minor, patch int, err error) {
    v = strings.TrimPrefix(v, "v")
    parts := strings.Split(v, ".")

```

```

    if len(parts) != 3 {
        return 0, 0, 0, fmt.Errorf("want major.minor.patch, got %q", v)
    }
    major, err = strconv.Atoi(parts[0])
    if err != nil {
        return
    }
    minor, err = strconv.Atoi(parts[1])
    if err != nil {
        return
    }
    patch, err = strconv.Atoi(parts[2])
    return
}

func main() {
    for _, v := range []string{"v1.2.3", "2.0.0", "1.2"} {
        maj, min, pat, err := parseSemver(v)
        if err != nil {
            fmt.Printf("%s -> %v\n", v, err)
            continue
        }
        fmt.Printf("%s -> major=%d minor=%d patch=%d\n", v, maj, min, pat)
    }
}

```

Expected:

```

v1.2.3 -> major=1 minor=2 patch=3
2.0.0 -> major=2 minor=0 patch=0
1.2 -> want major.minor.patch, got "1.2"

```

Runnable example

Stdlib-only module demo: after `go mod init`, the program reports module path and dependency list via build info (empty deps until you `go get` something).

Save as `main.go`. From an empty directory:

```

go mod init example.com/demo
go mod tidy
go run .
cat go.mod

```



```

package main

import (
    "fmt"
    "runtime/debug"
)

func main() {
    info, ok := debug.ReadBuildInfo()
    if !ok {
        fmt.Println("no build info (unusual for go run/build)")
        return
    }

    fmt.Println("module:", info.Main.Path)
    fmt.Println("go:", info.GoVersion)
    if info.Main.Version != "" {
        fmt.Println("main version:", info.Main.Version)
    }

    if len(info.Deps) == 0 {
        fmt.Println("deps: (none - stdlib only)")
    } else {
        fmt.Println("deps:")
        for _, d := range info.Deps {
            fmt.Printf("  %s %s\n", d.Path, d.Version)
        }
    }

    // Prefer stdlib until you truly need a third-party module.
    fmt.Println("tip: run `go get github.com/some/pkg@vX.Y.Z` then `go mod tidy`")
}

```

Expected output (illustrative):

```

module: example.com/demo
go: go1.22.5
main version: (devel)
deps: (none - stdlib only)
tip: run `go get github.com/some/pkg@vX.Y.Z` then `go mod tidy`

```

What to notice: - `go mod init` sets the module path that appears in `ReadBuildInfo`. - With only `stdlib` imports, `go.mod` stays lean—minimal dependencies by design. - `go mod tidy` is safe to run anytime; it syncs `go.mod/go.sum` with imports. - Third-party modules only appear under `deps` after you import and tidy them.

Try next: Add `import _ "golang.org/x/time/rate"`, run `go get golang.org/x/time@latest` and `go mod tidy`, then re-run and inspect `go.mod` / printed `deps`.

Workspaces and Multi-Module Development

Overview

Go workspaces, introduced in Go 1.18, solve the challenge of developing multiple related modules simultaneously. With `go.work`, you can work on a main project and its dependencies together without modifying `go.mod` files.

The Problem Workspaces Solve

Without Workspaces

When developing multiple modules together, you'd need `replace` directives:

```
// main-app/go.mod
module github.com/company/main-app

replace github.com/company/shared-lib => ../shared-lib
```

Problems: - Must edit `go.mod` (can accidentally commit) - Each developer has different paths - CI/CD requires cleanup

With Workspaces

```
$ go work init main-app shared-lib
```

```
// go.work
go 1.27

use (
    ./main-app
    ./shared-lib
)
```

Benefits: - No `go.mod` modifications - Works for entire team - Automatically ignored by Git

Creating a Workspace

Initial Setup

```
# Directory structure
workspace/
  go.work          # Workspace file
  main-app/        # Main application
    go.mod
    main.go
  shared-lib/      # Library module
    go.mod
    lib.go

# Create workspace
$ cd workspace
$ go work init ./main-app ./shared-lib
```

The go.work File

```
go 1.27
```

```
use (
  ./main-app
  ./shared-lib
)
```

All commands (`go build`, `go test`, etc.) now use local modules.

Workspace Commands

```
go work init
```

Create a new workspace:

```
# Initialize with modules
$ go work init ./module1 ./module2

# Initialize empty workspace
$ go work init
```

go work use

Add modules to workspace:

```
# Add single module
$ go work use ./new-module

# Add multiple modules
$ go work use ./mod1 ./mod2

# Recursively find and add all modules
$ go work use -r .
```

go work edit

Programmatically modify workspace:

```
# Add a use directive
$ go work edit -use ./new-module

# Remove a use directive
$ go work edit -dropuse ./old-module

# Set Go version
$ go work edit -go 1.27
```

go work sync

Sync workspace modules with their dependencies:

```
$ go work sync
```

This updates each module's `go.mod` to match workspace requirements.

Multi-Module Development Workflow

Project Structure

```
company-workspace/
  go.work
  api/
    go.mod          # module github.com/company/api
    api.go
    api_test.go
```

```

core/
  go.mod          # module github.com/company/core
  core.go
cli/
  go.mod          # module github.com/company/cli
  main.go
internal-tools/
  go.mod          # module github.com/company/tools
  tools.go

```

Setting Up

```

$ cd company-workspace
$ go work init ./api ./core ./cli ./internal-tools

```

Development Flow

```

# Changes in core/ are immediately available to cli/
$ cd cli
$ go build # Uses local core package

# Run tests across all modules
$ cd ..
$ go test ./...

# Build all binaries
$ go build ./cli/...

```

Replace vs Workspace

When to Use replace

Use Case	Approach
Permanent fork	replace in go.mod
CI workaround	replace in go.mod
Single dev experiment	replace in go.mod

When to Use Workspace

Use Case	Approach
Multi-module development	<code>go.work</code>
Team development	<code>go.work</code>
Monorepo-style work	<code>go.work</code>

Workspace with Replace

You can combine both. `go.work` supports `replace`:

```
go 1.27

use (
    ./main-app
    ./shared-lib
)

replace github.com/external/dep => ./custom-fork
```

Best Practices

1. Git Ignore `go.work`

```
# .gitignore
go.work
go.work.sum
```

Each developer creates their own workspace.

2. Document Setup

Include setup instructions in README:

```
## Development Setup

1. Clone all repositories:
```bash
git clone github.com/company/api
git clone github.com/company/core
git clone github.com/company/cli
```

2. Create workspace:

```
go work init ./api ./core ./cli
```

““

### 3. CI Without Workspaces

In CI, build each module independently:

```
GitHub Actions
jobs:
 test:
 strategy:
 matrix:
 module: [api, core, cli]
 steps:
 - uses: actions/checkout@v4
 - name: Test
 working-directory: ${ matrix.module }
 run: go test ./...
```

### 4. Shared Workspace for Monorepos

For monorepos, commit go.work:

```
monorepo/
 go.work # Committed
 services/
 api/
 worker/
 gateway/
 packages/
 auth/
 database/
```

## Common Patterns

### Microservices Development

```
workspace/
 go.work
 services/
 user-service/
 order-service/
 payment-service/
```

```
shared/
 proto/
 middleware/
 database/
```

```
$ go work init
$ go work use -r .
```

## Library Development

Testing a library change across consumers:

```
workspace/
 go.work
 my-library/ # The library
 app-using-library/ # Consumer 1
 another-consumer/ # Consumer 2
```

## Troubleshooting

### Module Not Found in Workspace

```
Verify workspace includes module
$ cat go.work

Re-add module
$ go work use ./missing-module
```

### Wrong Version Used

Workspace modules take priority. To use published version:

```
Temporarily disable workspace
$ GOWORK=off go build
```

### Workspace Mode Detection

Check if workspace is active:

```
$ go env GOWORK
/path/to/go.work
```



## Summary

Command	Purpose
<code>go work init</code>	Create workspace
<code>go work use</code>	Add modules
<code>go work edit</code>	Modify workspace
<code>go work sync</code>	Sync dependencies
<code>GOWORK=off</code>	Disable workspace

## Exercises

1. Create a workspace with two modules where one depends on the other
2. Make changes to the dependency and verify the main module sees them
3. Add a third module to an existing workspace

## Related Topics

- [How Go Code Is Organized](#)
- [Dependency Management](#)

## More examples

### Example: local “library” function in one module

Save as `main.go` and `go run .` (with `go mod init example.com/app` if needed).

```
package main

import "fmt"

// In a real workspace this would live in a sibling module;
// here we simulate a library API in the same package for a quick run.
func Greet(name string) string {
 if name == "" {
 return "hello, world"
 }
 return "hello, " + name
}

func main() {
 fmt.Println(Greet(""))
 fmt.Println(Greet("workspace"))
}
```

```
}
```

Expected:

```
hello, world
hello, workspace
```

## Example: replace-path thinking with build info

Save as `main.go` and `go run .` (with `go mod init example.com/consumer` if needed).

```
package main

import (
 "fmt"
 "runtime/debug"
)

func main() {
 info, ok := debug.ReadBuildInfo()
 if !ok {
 fmt.Println("no build info")
 return
 }
 fmt.Println("this binary was built from module:", info.Main.Path)
 // Workspace / replace: your local tree is what you just compiled.
 fmt.Println("workspace tip: go work use ./lib makes local edits visible without publishing")
 fmt.Println("escape hatch: GOWORK=off uses only go.mod require versions")
}
```

Expected:

```
this binary was built from module: example.com/consumer
workspace tip: go work use ./lib makes local edits visible without publishing
escape hatch: GOWORK=off uses only go.mod require versions
```

## Runnable example

Single-module stand-in that prints whether workspace mode is active (`GOWORK`) and the main module path. Use it while experimenting with `go work init`.

Save as `main.go`. From an empty directory:

```

go mod init example.com/app
go run .
Optional workspace experiment (sibling modules not required for this printout):
go work init .
go env GOWORK
go run .
GOWORK=off go run .

package main

import (
 "fmt"
 "os"
 "runtime/debug"
)

func main() {
 gowork := os.Getenv("GOWORK")
 if gowork == "" {
 // go env GOWORK reports the path when a go.work is discovered;
 // the process env may be empty even in workspace mode.
 fmt.Println("GOWORK env: (empty - use `go env GOWORK` for discovery path)")
 } else {
 fmt.Println("GOWORK env:", gowork)
 }

 info, ok := debug.ReadBuildInfo()
 if !ok {
 fmt.Println("build info unavailable")
 return
 }
 fmt.Println("main module:", info.Main.Path)
 fmt.Println("go version:", info.GoVersion)

 // Illustrate the replace/workspace idea: local code always wins in a workspace.
 fmt.Println("local module builds use your tree; published versions need GOWORK=off")
}

```

**Expected output** (illustrative):

```

GOWORK env: (empty - use `go env GOWORK` for discovery path)
main module: example.com/app
go version: go1.22.5
local module builds use your tree; published versions need GOWORK=off

```

**What to notice:** - Workspaces change how the toolchain resolves modules, not your program's import syntax. - `go env GOWORK` is the reliable way to see if a `go.work` is active. - `GOWORK=off`

forces published module versions for a single command. - Multi-module work is about directories + `use` directives; the binary still starts at `main`.

**Try next:** Create two sibling dirs with their own `go.mod`, `go work init` both, import one from the other, change a function, and rebuild without publishing.

**Core**

# Predeclared Types

## Overview

Go provides a set of built-in types that form the foundation of all programs. Understanding these predeclared types—booleans, numerics, strings, and the error interface—is essential for writing effective Go code.

## Boolean Type

The `bool` type represents true/false values.

```
var enabled bool // Zero value: false
active := true
disabled := false

// Boolean operators
result := active && disabled // AND: false
result = active || disabled // OR: true
result = !active // NOT: false
```

## Boolean Expressions

```
x := 10
y := 20

greater := x > y // false
equal := x == y // false
notEqual := x != y // true
lessEqual := x <= y // true
```

## Numeric Types

### Integer Types

Type	Size	Range
int8	8 bits	-128 to 127
int16	16 bits	-32,768 to 32,767
int32	32 bits	-2B to 2B
int64	64 bits	±9 quintillion
int	Platform	32 or 64 bits

```
var small int8 = 127
var medium int32 = 2_147_483_647 // Underscores for readability
var large int64 = 9_223_372_036_854_775_807
var auto int = 42 // Platform-dependent size
```

## Unsigned Integers

Type	Size	Range
uint8	8 bits	0 to 255
uint16	16 bits	0 to 65,535
uint32	32 bits	0 to 4B
uint64	64 bits	0 to 18 quintillion
uint	Platform	32 or 64 bits

```
var age uint8 = 30
var count uint = 1000
```

## Type Aliases

```
type byte = uint8 // Alias for byte data
type rune = int32 // Alias for Unicode code points
```

## Floating-Point Types

Type	Size	Precision
float32	32 bits	~7 decimal digits
float64	64 bits	~15 decimal digits

```
var pi float32 = 3.14159
var precise float64 = 3.141592653589793

// Scientific notation
```

```
avogadro := 6.022e23
planck := 6.626e-34
```

## Complex Numbers

```
var c1 complex64 = 3 + 4i
c2 := complex(5, 12) // 5 + 12i

real := real(c2) // 5
imag := imag(c2) // 12
```

## String Type

Strings are immutable sequences of bytes (typically UTF-8).

```
var greeting string = "Hello, World!"
name := "Go"

// String concatenation
message := greeting + " Welcome to " + name

// Multi-line strings (raw string literals)
poem := `Roses are red,
Violets are blue,
Go is awesome,
And so are you!`

// String length (bytes, not characters)
len(greeting) // 13
```

## Strings vs Runes

```
s := "Hello, "

len(s) // 13 bytes
len([]rune(s)) // 9 characters

// Iterate by byte
for i := 0; i < len(s); i++ {
 fmt.Printf("%x ", s[i])
}
```



```
// Iterate by rune (character)
for i, r := range s {
 fmt.Printf("%d: %c\n", i, r)
}
```

## The Error Type

error is a built-in interface type:

```
type error interface {
 Error() string
}

import "errors"

func divide(a, b float64) (float64, error) {
 if b == 0 {
 return 0, errors.New("division by zero")
 }
 return a / b, nil
}

result, err := divide(10, 0)
if err != nil {
 fmt.Println("Error:", err)
}
```

## Type Conversions

Go requires explicit type conversions:

```
var i int = 42
var f float64 = float64(i) // int to float64
var u uint = uint(i) // int to uint

// String conversions
import "strconv"

s := strconv.Itoa(42) // "42"
n, _ := strconv.Atoi("42") // 42

fs := strconv.FormatFloat(3.14, 'f', 2, 64) // "3.14"
```

## Common Conversion Pitfalls

```
// Overflow (no error, just wraps)
var big int64 = 1000
var small int8 = int8(big) // Overflow! Result: -24

// Float to int truncates
var f float64 = 3.9
var i int = int(f) // 3 (not 4)
```

## Type Information

```
import "fmt"

var x int = 42
fmt.Printf("Type: %T, Value: %v\n", x, x)
// Output: Type: int, Value: 42
```

## Numeric Literals

```
// Decimal
decimal := 42

// Binary (0b prefix)
binary := 0b101010 // 42

// Octal (0o prefix)
octal := 0o52 // 42

// Hexadecimal (0x prefix)
hex := 0x2A // 42

// With underscores for readability
billion := 1_000_000_000
```

## Summary

Category	Types
Boolean	bool
Signed Integers	int, int8, int16, int32, int64

Category	Types
Unsigned Integers	uint, uint8, uint16, uint32, uint64
Floating Point	float32, float64
Complex	complex64, complex128
String	string
Aliases	byte (uint8), rune (int32)
Error	error

## Related Topics

- [Zero Values and Initialization](#)
- [Constants and Literals](#)

## More examples

### Example: integer and float basics

Save as `main.go` and `go run .` (with `go mod init example` if needed).

```
package main

import "fmt"

func main() {
 var i int = 42
 var f float64 = 3.5
 fmt.Println("int:", i)
 fmt.Println("float:", f)
 fmt.Println("int(f):", int(f))
 fmt.Println("float64(i):", float64(i))
}
```

Expected:

```
int: 42
float: 3.5
int(f): 3
float64(i): 42
```

### Example: bool and rune

Save as `main.go` and `go run .` (with `go mod init example` if needed).

```

package main

import "fmt"

func main() {
 ok := true
 r := 'G'
 fmt.Println("ok:", ok)
 fmt.Printf("rune %q codepoint %d\n", r, r)
 fmt.Println("not ok:", !ok)
}

```

Expected:

```

ok: true
rune 'G' codepoint 71
not ok: false

```

## Runnable example

Save as `main.go`. From an empty directory:

```

go mod init example
go run .

```

```

package main

import (
 "errors"
 "fmt"
 "strconv"
)

func divide(a, b float64) (float64, error) {
 if b == 0 {
 return 0, errors.New("division by zero")
 }
 return a / b, nil
}

func main() {
 var enabled bool = true
 var small int8 = 127
 var count int = 42
 var pi float64 = 3.141592653589793
}

```

```

var c complex128 = complex(3, 4)
s := "Hello, "
b := byte('A')
r := rune(' ')

fmt.Printf("bool=%t int8=%d int=%d float64=%g\n", enabled, small, count, pi)
fmt.Printf("complex=%v real=%g imag=%g\n", c, real(c), imag(c))
fmt.Printf("string=%q bytes=%d runes=%d\n", s, len(s), len([]rune(s)))
fmt.Printf("byte=%d rune=%c (%U)\n", b, r, r)

// Numeric bases and conversion
fmt.Printf("0b101010=%d 0x2A=%d\n", 0b101010, 0x2A)
n, err := strconv.Atoi("42")
fmt.Printf("Atoi=%d err=%v Itoa=%s\n", n, err, strconv.Itoa(n))

// Overflow wraps; float→int truncates (use variables so the conversions are runtime)
big := int64(1000)
fl := 3.9
fmt.Printf("int8(1000)=%d int(3.9)=%d\n", int8(big), int(fl))

if _, err := divide(10, 0); err != nil {
 fmt.Println("error:", err)
}
q, err := divide(10, 4)
fmt.Printf("10/4=%g err=%v types: q is %T\n", q, err, q)
}

```

**Expected output** (illustrative):

```

bool=true int8=127 int=42 float64=3.141592653589793
complex=(3+4i) real=3 imag=4
string="Hello, " bytes=13 runes=9
byte=65 rune= (U+4E16)
0b101010=42 0x2A=42
Atoi=42 err=<nil> Itoa=42
int8(1000)=-24 int(3.9)=3
error: division by zero
10/4=2.5 err=<nil> types: q is float64

```

**What to notice:** - %T / %v make types and values observable. - len(string) is bytes; len([]rune(s)) is characters. - Conversions are explicit; overflow wraps without an error. - error is a value returned and checked like any other type.

**Try next:** Change int8(1000) to a value you pick and predict the wrapped result.

# Zero Values and Initialization

## Overview

Every variable in Go has a value from the moment it's declared. If you don't explicitly initialize a variable, it gets a **zero value**—a sensible default that prevents undefined behavior.

## Zero Values by Type

Type	Zero Value
bool	false
int, int8, ...	0
uint, uint8, ...	0
float32, float64	0.0
complex64, complex128	(0+0i)
string	"" (empty string)
pointer	nil
slice	nil
map	nil
channel	nil
function	nil
interface	nil

## Demonstrating Zero Values

```
package main

import "fmt"

func main() {
 var b bool
 var i int
 var f float64
 var s string
 var p *int
 var sl []int
```

```

var m map[string]int
var ch chan int
var fn func()
var iface interface{}

fmt.Printf("bool: %v\n", b) // false
fmt.Printf("int: %v\n", i) // 0
fmt.Printf("float64: %v\n", f) // 0
fmt.Printf("string: %q\n", s) // ""
fmt.Printf("pointer: %v\n", p) // <nil>
fmt.Printf("slice: %v\n", sl) // []
fmt.Printf("map: %v\n", m) // map[]
fmt.Printf("channel: %v\n", ch) // <nil>
fmt.Printf("func: %v\n", fn) // <nil>
fmt.Printf("interface: %v\n", iface) // <nil>
}

```

## Struct Zero Values

Struct fields get their respective zero values:

```

type User struct {
 Name string
 Age int
 Active bool
 Balance float64
}

var user User
// user.Name = ""
// user.Age = 0
// user.Active = false
// user.Balance = 0.0

```

## Nested Structs

```

type Address struct {
 Street string
 City string
}

type Person struct {
 Name string
 Address Address // Embedded struct gets zero values too
}

```

```

}

var p Person
// p.Name = ""
// p.Address.Street = ""
// p.Address.City = ""

```

## Array Zero Values

Arrays are initialized with zero values for each element:

```

var arr [5]int // [0, 0, 0, 0, 0]
var strs [3]string // ["", "", ""]
var bools [2]bool // [false, false]

```

## Why Zero Values Matter

### 1. Eliminates Undefined Behavior

```

// In Go, this is safe and predictable
var count int
count++ // count is now 1

// In C, this would be undefined behavior
// int count;
// count++; // Undefined! Could be anything

```

### 2. Reduces Boilerplate

```

// No need for explicit initialization
type Config struct {
 Debug bool
 Timeout int
 Host string
}

cfg := Config{} // All zeros, ready to use selectively
cfg.Host = "localhost" // Only set what differs from zero

```



### 3. Enables “Usable Zero Value” Pattern

Well-designed types work correctly with zero values:

```
import "bytes"

// bytes.Buffer works without initialization
var buf bytes.Buffer
buf.WriteString("Hello, ")
buf.WriteString("World!")
fmt.Println(buf.String()) // "Hello, World!"

// sync.Mutex works without initialization
var mu sync.Mutex
mu.Lock()
mu.Unlock()
```

## Initialization Methods

### Short Variable Declaration

```
name := "Alice" // Inferred type: string
count := 42 // Inferred type: int
price := 19.99 // Inferred type: float64
```

### Explicit Type Declaration

```
var name string = "Alice"
var count int = 42
var active bool = true
```

### Type Inference (var)

```
var name = "Alice" // Type inferred from value
var count = 42 // int
```

## Multiple Variable Declaration

```
var (
 name = "Alice"
 age = 30
 active = true
)

// Or inline
x, y, z := 1, 2, 3
```

## Composite Literal Initialization

```
// Struct
user := User{
 Name: "Alice",
 Age: 30,
 Active: true,
}

// Partially initialized (rest are zero values)
user := User{Name: "Bob"} // Age: 0, Active: false

// Array
arr := [3]int{1, 2, 3}

// Slice
sl := []string{"a", "b", "c"}

// Map
m := map[string]int{"one": 1, "two": 2}
```

## nil vs Zero Value

`nil` is the zero value for pointers, slices, maps, channels, functions, and interfaces.

## nil Slice vs Empty Slice

```
var nilSlice []int // nil slice
emptySlice := []int{} // empty slice (not nil)

nilSlice == nil // true
```

```

emptySlice == nil // false

len(nilSlice) // 0
len(emptySlice) // 0

// Both work with append
nilSlice = append(nilSlice, 1) // [1]
emptySlice = append(emptySlice, 1) // [1]

```

## nil Map Danger

```

var m map[string]int // nil map

// Reading is safe
val := m["key"] // Returns 0 (zero value)

// Writing panics!
m["key"] = 42 // panic: assignment to nil map

// Always initialize before writing
m = make(map[string]int)
m["key"] = 42 // Now safe

```

## The new vs make Functions

### new(T)

Allocates zeroed memory and returns a pointer:

```

ptr := new(int) // *int pointing to 0
*ptr = 42

user := new(User) // *User with zero-valued fields
user.Name = "Alice"

```

### make(T, args)

Creates and initializes slices, maps, and channels:

```

// Slice with length and capacity
sl := make([]int, 5) // [0, 0, 0, 0, 0]
sl := make([]int, 0, 10) // [] with capacity 10

```

```
// Map
m := make(map[string]int) // Empty, ready to use

// Channel
ch := make(chan int) // Unbuffered channel
ch := make(chan int, 10) // Buffered channel
```

## Summary

Concept	Description
Zero Value	Default value assigned to uninitialized variables
Purpose	Eliminates undefined behavior, reduces boilerplate
<code>new(T)</code>	Allocates zeroed memory, returns <code>*T</code>
<code>make(T)</code>	Creates initialized slice, map, or channel
<code>nil</code>	Zero value for pointers, slices, maps, channels, funcs, interfaces

## Related Topics

- [Predeclared Types](#)
- [Variables and Scope](#)

## More examples

### Example: zero values of common types

Save as `main.go` and `go run .` (with `go mod init example` if needed).

```
package main

import "fmt"

func main() {
 var n int
 var s string
 var b bool
 var p *int
 fmt.Printf("int=%d string=%q bool=%v ptr=%v\n", n, s, b, p)
}
```

Expected:

```
int=0 string="" bool=false ptr=<nil>
```

## Example: nil map vs empty map

Save as `main.go` and `go run .` (with `go mod init example` if needed).

```
package main

import "fmt"

func main() {
 var nilMap map[string]int
 empty := map[string]int{}
 fmt.Println("nil len:", len(nilMap), "read:", nilMap["x"])
 fmt.Println("empty len:", len(empty), "read:", empty["x"])
 empty["x"] = 1
 fmt.Println("after write empty:", empty)
 // nilMap["x"] = 1 // would panic
 fmt.Println("nil map needs make before write")
}
```

Expected:

```
nil len: 0 read: 0
empty len: 0 read: 0
after write empty: map[x:1]
nil map needs make before write
```

## Runnable example

Complete program that prints zero values, shows a usable zero `bytes.Buffer`, and panics safely-avoided on a nil map.

Save as `main.go`. From an empty directory:

```
go mod init example
go run .
```

```
package main

import (
 "bytes"
 "fmt"
)
```

```

type User struct {
 Name string
 Age int
 Active bool
}

func main() {
 var b bool
 var i int
 var f float64
 var s string
 var p *int
 var sl []int
 var m map[string]int
 var ch chan int
 var fn func()
 var iface any

 fmt.Printf("bool=%v int=%v float64=%v string=%q\n", b, i, f, s)
 fmt.Printf("pointer=%v slice=%v map=%v chan=%v func=%v iface=%v\n", p, sl, m, ch, fn, if

 var user User
 fmt.Printf("user zero: %+v\n", user)

 var arr [3]int
 fmt.Printf("array zero: %v\n", arr)

 // Usable zero value: no constructor required.
 var buf bytes.Buffer
 buf.WriteString("Hello, ")
 buf.WriteString("zero values!")
 fmt.Println(buf.String())

 // nil map: read OK, write needs make
 fmt.Println("nil map read:", m["missing"])
 m = make(map[string]int)
 m["ok"] = 1
 fmt.Println("after make:", m)

 // nil vs empty slice
 var nilSlice []int
 empty := []int{}
 fmt.Printf("nilSlice==nil? %v empty==nil? %v\n", nilSlice == nil, empty == nil)
 nilSlice = append(nilSlice, 7)
 fmt.Println("append on nil slice:", nilSlice)
}

```

**Expected output** (illustrative):

```
bool=false int=0 float64=0 string=""
pointer=<nil> slice=[] map=map[] chan=<nil> func=<nil> iface=<nil>
user zero: {Name: Age:0 Active:false}
array zero: [0 0 0]
Hello, zero values!
nil map read: 0
after make: map[ok:1]
nilSlice==nil? true empty==nil? false
append on nil slice: [7]
```

**What to notice:** - Every type has a defined zero—no “undefined” locals. - Struct fields zero independently; arrays zero every element. - `bytes.Buffer` works from a zero value (idiomatic design). - Writing a nil map would panic; `make` first is required.

**Try next:** Comment out `m = make(...)` and assign `m["ok"] = 1` to observe the panic, then restore it.

# Constants and Literals

## Overview

Constants in Go are values determined at compile time that cannot be changed during program execution. Go distinguishes between typed and untyped constants, with untyped constants having special flexibility.

## Declaring Constants

### Basic Declaration

```
const Pi = 3.14159
const MaxSize = 1024
const Greeting = "Hello, World!"
```

### Constant Block

```
const (
 StatusOK = 200
 StatusNotFound = 404
 StatusError = 500
)
```

### Typed vs Untyped Constants

```
// Untyped constant (more flexible)
const untypedPi = 3.14159

// Typed constant (explicit type)
const typedPi float64 = 3.14159
```



## Untyped Constants

Untyped constants have a **kind** but not a fixed type, allowing them to adapt:

```
const answer = 42 // Untyped integer constant

var i int = answer // Works: becomes int
var f float64 = answer // Works: becomes float64
var c complex128 = answer // Works: becomes complex128
```

## Why This Matters

```
// Untyped constant can be used with any compatible type
const billion = 1e9

var i int64 = billion // Works
var f float32 = billion // Works

// Typed constant is locked to its type
const typedBillion int64 = 1e9
var f float64 = typedBillion // Error: cannot use int64 as float64
```

## Constant Kinds

Kind	Examples
Boolean	true, false
Integer	42, 0x2A, 0b101010
Floating-point	3.14, 6.022e23
Complex	1 + 2i
String	"hello", `raw string`
Rune	'A', ' '

## The iota Constant Generator

**iota** generates sequential integer constants, starting from 0 and incrementing by 1 for each constant in the block.

## Basic Usage

```
const (
 Sunday = iota // 0
 Monday // 1
 Tuesday // 2
 Wednesday // 3
 Thursday // 4
 Friday // 5
 Saturday // 6
)
```

## Skip Values

```
const (
 _ = iota // 0 (skipped)
 KB = 1 << (10 * iota) // 1 << 10 = 1024
 MB // 1 << 20 = 1,048,576
 GB // 1 << 30 = 1,073,741,824
 TB // 1 << 40
)
```

## Bit Flags with iota

```
const (
 FlagRead = 1 << iota // 1 (0001)
 FlagWrite // 2 (0010)
 FlagExecute // 4 (0100)
)

permissions := FlagRead | FlagWrite // 3 (0011)

hasRead := permissions & FlagRead != 0 // true
hasExec := permissions & FlagExecute != 0 // false
```

## Multiple iota in One Line

```
const (
 A, B = iota, iota + 10 // A=0, B=10
 C, D // C=1, D=11
 E, F // E=2, F=12
)
```

## Reset iota

Each `const` block resets `iota` to 0:

```
const (
 First = iota // 0
 Second // 1
)

const (
 Third = iota // 0 (reset!)
 Fourth // 1
)
```

## Constant Expressions

Constants can be computed from other constants at compile time:

```
const (
 SecondsPerMinute = 60
 MinutesPerHour = 60
 SecondsPerHour = SecondsPerMinute * MinutesPerHour
 SecondsPerDay = SecondsPerHour * 24
)
```

## Allowed in Constant Expressions

- Arithmetic operators: `+`, `-`, `*`, `/`, `%`
- Comparison operators: `==`, `!=`, `<`, `>`, `<=`, `>=`
- Logical operators: `&&`, `||`, `!`
- Bit operators: `&`, `|`, `^`, `<<`, `>>`, `&^`
- Built-in functions: `len()`, `cap()` (for some values), `real()`, `imag()`, `complex()`

## Not Allowed

```
const x = math.Sin(0) // Error: function call not allowed
const y = len(someVar) // Error: variable not allowed
```

## Numeric Literals

### Integer Literals

```
decimal := 42 // Base 10
binary := 0b101010 // Base 2 (Go 1.13+)
octal := 0o52 // Base 8 (Go 1.13+)
oldOctal := 052 // Base 8 (legacy)
hex := 0x2A // Base 16
```

### Floating-Point Literals

```
f1 := 3.14
f2 := .5 // 0.5
f3 := 1. // 1.0
f4 := 6.022e23 // Scientific notation
f5 := 6.022E23 // Same
f6 := 1.5e-10 // 0.00000000015
```

### Hexadecimal Floats (Go 1.13+)

```
f := 0x1.fp+2 // (1 + 15/16) * 2^2 = 7.75
```

### Imaginary Literals

```
i := 3i
c := 2 + 3i
```

### Digit Separators

Go 1.13+ allows underscores in numeric literals for readability:

```
billion := 1_000_000_000
binary := 0b_1010_1010
hex := 0xFF_FF_FF_FF
```

# String and Rune Literals

## Interpreted Strings

```
s := "Hello, World!\n" // Escape sequences processed
s := "Tab:\tNewline:\n"
s := "Quote: \"Go\""
```

## Escape Sequences

Escape	Meaning
\n	Newline
\t	Tab
\\	Backslash
\"	Double quote
\r	Carriage return
\xNN	Hex byte
\uNNNN	Unicode code point (4 hex)
\UNNNNNNNN	Unicode code point (8 hex)

## Raw String Literals

```
raw := `No escape sequences: \n \t \\
Multiple lines work!
Useful for regex: \d+\.\d+`
```

## Rune Literals

```
r1 := 'A' // 65
r2 := ' ' // 19990
r3 := '\n' // 10
r4 := '\x41' // 65 (hex)
r5 := '\u4e16' // 19990 (unicode)
```

## Common Patterns

### Enum Pattern

```
type Status int

const (
 StatusPending Status = iota
 StatusApproved
 StatusRejected
)

func (s Status) String() string {
 switch s {
 case StatusPending:
 return "Pending"
 case StatusApproved:
 return "Approved"
 case StatusRejected:
 return "Rejected"
 default:
 return "Unknown"
 }
}
```

### Configuration Constants

```
const (
 DefaultTimeout = 30 * time.Second
 MaxRetries = 3
 BufferSize = 4096
)
```

## Summary

Concept	Description
<code>const</code>	Compile-time immutable value
Untyped	Flexible, adapts to context
Typed	Fixed to a specific type
<code>iota</code>	Auto-incrementing generator
Literal	Numeric, string, or rune value in source

## Related Topics

- [Predeclared Types](#)
- [Variables and Scope](#)

## More examples

### Example: iota enums

Save as `main.go` and `go run .` (with `go mod init example` if needed).

```
package main

import "fmt"

const (
 StatusDraft = iota
 StatusOpen
 StatusClosed
)

func main() {
 fmt.Println("Draft:", StatusDraft)
 fmt.Println("Open:", StatusOpen)
 fmt.Println("Closed:", StatusClosed)
}
```

Expected:

```
Draft: 0
Open: 1
Closed: 2
```

### Example: untyped constant flexibility

Save as `main.go` and `go run .` (with `go mod init example` if needed).

```
package main

import "fmt"

const Answer = 42 // untyped integer constant

func main() {
 var i int = Answer
}
```

```

 var i64 int64 = Answer
 var f float64 = Answer
 fmt.Println(i, i64, f)
}

```

Expected:

```

42 42 42

```

## Runnable example

Save as `main.go`. From an empty directory:

```

go mod init example
go run .

package main

import "fmt"

type Status int

const (
 StatusPending Status = iota
 StatusApproved
 StatusRejected
)

func (s Status) String() string {
 switch s {
 case StatusPending:
 return "Pending"
 case StatusApproved:
 return "Approved"
 case StatusRejected:
 return "Rejected"
 default:
 return "Unknown"
 }
}

const (
 _ = iota
 KB = 1 << (10 * iota) // 1 << 10
 MB

```



```

 GB
)

 const (
 FlagRead = 1 << iota // 1
 FlagWrite // 2
 FlagExecute // 4
)

 func main() {
 // Untyped constant adapts to destination types.
 const answer = 42
 var i int = answer
 var f float64 = answer
 fmt.Printf("untyped answer as int=%d float64=%g\n", i, f)

 fmt.Printf("status: %s=%d %s=%d\n", StatusPending, StatusPending, StatusApproved, StatusApproved)
 fmt.Printf("sizes: KB=%d MB=%d GB=%d\n", KB, MB, GB)

 perms := FlagRead | FlagWrite
 fmt.Printf("perms=%04b read=%v write=%v exec=%v\n",
 perms,
 perms&FlagRead != 0,
 perms&FlagWrite != 0,
 perms&FlagExecute != 0,
)

 // Literals
 fmt.Printf("bases: %d %d %d %d\n", 42, 0b101010, 0o52, 0x2A)
 fmt.Printf("rune %q is %d; raw=%s\n", ' ', ' ', `line\nnot escaped`)
 }

```

**Expected output** (illustrative):

```

untyped answer as int=42 float64=42
status: Pending=0 Approved=1
sizes: KB=1024 MB=1048576 GB=1073741824
perms=0011 read=true write=true exec=false
bases: 42 42 42 42
rune ' ' is 19990; raw=`line\nnot escaped`

```

**What to notice:** - `iota` auto-numbers enums and builds power-of-two sizes/flags. - Untyped constants convert cleanly into both `int` and `float64`. - Bit flags combine with `|` and test with `&`. - Raw string literals do not interpret `\n`.

**Try next:** Add TB after GB in the size block and print it; confirm `1 << 40`.

# Variables and Scope

## Overview

Variables in Go store values that can change during program execution. Understanding declaration patterns and visibility rules (scope) is fundamental to writing correct Go programs.

## Variable Declaration

### Using var

```
// Explicit type
var name string
var age int

// With initialization
var name string = "Alice"
var age int = 30

// Type inference
var name = "Alice" // Inferred as string
var age = 30 // Inferred as int
```

### Short Variable Declaration (:=)

Inside functions, use := for concise declaration:

```
func main() {
 name := "Alice" // var name = "Alice"
 age := 30 // var age = 30
 active := true // var active = true
}
```

### When to Use var vs :=

Use var	Use :=
Package-level variables	Inside functions
Explicit type needed	Type can be inferred
Zero value is desired	Initialization value provided

```
// Package level (var only)
var globalConfig Config

func main() {
 // Function level (either works, := preferred)
 count := 0

 // Explicit type needed
 var ratio float64 = 0 // := 0 would be int
}
```

## Multiple Variables

```
// Same type
var x, y, z int

// Different values
var a, b, c = 1, "hello", true

// Short declaration
name, age, active := "Alice", 30, true
```

## Variable Block

```
var (
 name string = "Alice"
 age int = 30
 active bool = true
)
```

## Redeclaration and Shadowing

### Redeclaration with :=

:= can redeclare if at least one variable is new:

```

x := 1
x, y := 2, 3 // x redeclared, y is new (allowed)

// This fails:
// x := 4 // Error: no new variables on left side

```

## Variable Shadowing

An inner scope can declare a variable with the same name as an outer scope:

```

x := 1
fmt.Println(x) // 1

if true {
 x := 2 // New x, shadows outer x
 fmt.Println(x) // 2
}

fmt.Println(x) // 1 (outer x unchanged)

```

**Warning:** Shadowing often creates bugs:

```

var err error

if condition {
 result, err := someFunc() // err shadows! Outer err unchanged
 // ...
}

if err != nil { // This checks outer err, always nil!
 // Never reached
}

```

**Fix:**

```

var err error
var result Result

if condition {
 result, err = someFunc() // No :=, uses outer err
}

```

## Scope Levels

### Package Scope

Variables declared outside functions are visible to the entire package:

```
package mypackage

var packageVar = "visible to all files in mypackage"

func init() {
 packageVar = "can modify here"
}

func SomeFunc() {
 fmt.Println(packageVar) // Accessible
}
```

### File Scope (Imports)

Import names are scoped to the file:

```
// file1.go
import "fmt" // fmt available only in this file

// file2.go
import "fmt" // Must import again
```

### Function Scope

Parameters and local variables are visible within the function:

```
func greet(name string) {
 message := "Hello, " + name // Local to greet
 fmt.Println(message)
}

// name and message are not accessible here
```

### Block Scope

Variables declared in blocks ({} ) are visible only within that block:

```

if x > 0 {
 y := x * 2 // Only visible inside if block
 fmt.Println(y)
}
// y is not accessible here

for i := 0; i < 10; i++ {
 // i is visible only in the loop
}
// i is not accessible here

switch n := getValue(); n {
case 1:
 // n is visible in switch block
}
// n is not accessible here

```

## Visibility (Exported vs Unexported)

Go uses capitalization for visibility across packages:

```

package user

var PublicVar = "accessible from other packages" // Exported
var privateVar = "only in user package" // Unexported

func PublicFunc() {} // Exported
func privateFunc() {} // Unexported

type PublicType struct {
 PublicField string // Exported
 privateField string // Unexported
}

```

## The Blank Identifier (\_)

Use `_` to discard unwanted values:

```

// Discard second return value
value, _ := someFunc()

// Discard loop index
for _, item := range items {

```

```

 process(item)
}

// Import for side effects only
import _ "github.com/lib/pq"

```

## Variable Lifetime

### Stack vs Heap

Go decides where to allocate based on escape analysis:

```

func createValue() int {
 x := 42 // Likely on stack
 return x // Copy returned, x can be reclaimed
}

func createPointer() *int {
 x := 42 // Must go on heap
 return &x // Pointer escapes function
}

```

### Garbage Collection

You don't need to free memory—Go's garbage collector handles it:

```

func process() {
 data := make([]byte, 1024*1024) // 1MB allocated
 // Use data...
} // data becomes garbage, will be collected

```

## Common Patterns

### Zero Value Initialization

```

var config Config // All fields are zero values
config.Timeout = 30 * time.Second // Set only what differs

```

## Conditional Initialization

```
var s string
if condition {
 s = "yes"
} else {
 s = "no"
}

// Or with short declaration
s := "default"
if condition {
 s = "override"
}
```

## Multiple Assignment

```
// Swap without temp variable
a, b = b, a

// Multiple returns
x, y, z := multiReturn()
```

## Common Pitfalls

### Accidental Shadowing

```
// Bug: err is shadowed
var err error
if x > 0 {
 y, err := compute() // New err!
 _ = y
}
// Original err still nil

// Fix: declare y separately
var err error
var y int
if x > 0 {
 y, err = compute() // Uses outer err
}
```



## Loop Variable Capture

```
// Bug: all goroutines see last value
for _, v := range values {
 go func() {
 fmt.Println(v) // v is shared!
 }()
}

// Fix: pass as parameter
for _, v := range values {
 go func(v int) {
 fmt.Println(v)
 }(v)
}

// Go 1.22+ fixes this by default
```

## Summary

Declaration	Usage
<code>var x T</code>	Zero value, explicit type
<code>var x = value</code>	Type inference
<code>x := value</code>	Short declaration (functions only)
<code>var x, y T</code>	Multiple same type
<code>x, y := a, b</code>	Multiple with inference

Scope	Visibility
Package	All files in package
File	Imports
Function	Function body
Block	{ } boundaries

## Related Topics

- [Constants and Literals](#)
- [Functions in Depth](#)

## More examples

### Example: short declaration scope

Save as `main.go` and `go run .` (with `go mod init example` if needed).

```
package main

import "fmt"

func main() {
 x := 1
 {
 x := 2 // new x shadows outer
 fmt.Println("inner:", x)
 }
 fmt.Println("outer:", x)
}
```

Expected:

```
inner: 2
outer: 1
```

### Example: if with short statement

Save as `main.go` and `go run .` (with `go mod init example` if needed).

```
package main

import (
 "fmt"
 "strconv"
)

func main() {
 if n, err := strconv.Atoi("42"); err != nil {
 fmt.Println("err:", err)
 } else {
 fmt.Println("n:", n)
 }
 // n is not in scope here
 fmt.Println("done")
}
```

Expected:

```
n: 42
done
```

## Runnable example

Save as `main.go`. From an empty directory:

```
go mod init example
go run .

package main

import "fmt"

var packageLevel = "package"

func compute(ok bool) (int, error) {
 if !ok {
 return 0, fmt.Errorf("not ok")
 }
 return 42, nil
}

func main() {
 // Short declaration inside functions
 name := "Alice"
 var ratio float64 // zero value; := 0 would be int
 fmt.Printf("name=%s ratio=%g package=%s\n", name, ratio, packageLevel)

 // Shadowing demo
 x := 1
 fmt.Println("outer x:", x)
 if true {
 x := 2 // new x in block scope
 fmt.Println("inner x:", x)
 }
 fmt.Println("outer x after block:", x)

 // Redeclare with := when at least one name is new
 x, y := 3, 4
 fmt.Printf("redecl x=%d new y=%d\n", x, y)

 // Correct multi-return assignment (no accidental err shadow)
 var err error
 var n int
```

```

n, err = compute(true)
fmt.Printf("compute ok: n=%d err=%v\n", n, err)
n, err = compute(false)
fmt.Printf("compute fail: n=%d err=%v\n", n, err)

// Blank identifier discards values
_, _ = compute(true)
a, b := 1, 2
a, b = b, a
fmt.Printf("swapped a=%d b=%d\n", a, b)
}

```

**Expected output** (illustrative):

```

name=Alice ratio=0 package=package
outer x: 1
inner x: 2
outer x after block: 1
redecl x=3 new y=4
compute ok: n=42 err=<nil>
compute fail: n=0 err=not ok
swapped a=2 b=1

```

**What to notice:** - Block scope shadows do not update the outer variable. - `:=` redeclares only when a new variable appears on the left. - Assigning with `=` keeps a single `err` in outer scope—critical for real code. - Package-level vars are visible to all functions in the package.

**Try next:** Inside the `if true` block, change `x := 2` to `x = 2` and observe the outer value after the block.

# Conditional Logic

## Overview

Go provides `if/else` statements and `switch` statements for conditional logic. Both are straightforward but have unique features compared to other languages.

## The if Statement

### Basic Syntax

```
if condition {
 // code
}
```

### With else

```
if x > 0 {
 fmt.Println("Positive")
} else {
 fmt.Println("Non-positive")
}
```

### With else if

```
if x > 0 {
 fmt.Println("Positive")
} else if x < 0 {
 fmt.Println("Negative")
} else {
 fmt.Println("Zero")
}
```

## Initialization Statement

Go allows a statement before the condition:

```
if err := doSomething(); err != nil {
 // Handle error
 // err is only visible in this if/else block
 return err
}
// err is not accessible here
```

This is idiomatic for error handling:

```
if file, err := os.Open("data.txt"); err != nil {
 log.Fatal(err)
} else {
 defer file.Close()
 // Use file
}
```

## No Parentheses Required

Unlike C-style languages, conditions don't need parentheses:

```
// Go style
if x > 0 {
}

// Not idiomatic (but valid)
if (x > 0) {
}
```

## Braces Required

Braces are mandatory, even for single statements:

```
// Error: missing braces
if x > 0
 fmt.Println("yes")

// Correct
if x > 0 {
 fmt.Println("yes")
}
```

## Boolean Expressions

### Comparison Operators

Operator	Meaning
==	Equal
!=	Not equal
<	Less than
>	Greater than
<=	Less than or equal
>=	Greater than or equal

### Logical Operators

Operator	Meaning
&&	AND (short-circuit)
	OR (short-circuit)
!	NOT

```
// Short-circuit evaluation
if x != 0 && y/x > 1 {
 // y/x only evaluated if x != 0
}

if ptr != nil && ptr.Value > 0 {
 // Ptr.Value only accessed if ptr != nil
}
```

## The switch Statement

### Basic Switch

```
switch day {
case "Monday":
 fmt.Println("Start of week")
case "Friday":
 fmt.Println("Almost weekend!")
case "Saturday", "Sunday":
 fmt.Println("Weekend!")
default:
 fmt.Println("Midweek")
}
```

```
}
```

## No Fall-Through by Default

Unlike C, cases don't fall through:

```
switch n {
case 1:
 fmt.Println("One")
 // No break needed, stops here
case 2:
 fmt.Println("Two")
}
```

## Explicit Fallthrough

Use `fallthrough` keyword when needed:

```
switch n {
case 1:
 fmt.Println("One")
 fallthrough
case 2:
 fmt.Println("Two")
}
// Input: 1
// Output:
// One
// Two
```

## Switch with Initialization

```
switch os := runtime.GOOS; os {
case "darwin":
 fmt.Println("macOS")
case "linux":
 fmt.Println("Linux")
default:
 fmt.Println(os)
}
```



## Expression-less Switch

A switch without an expression is equivalent to `switch true`:

```
t := time.Now()

switch {
case t.Hour() < 12:
 fmt.Println("Good morning!")
case t.Hour() < 17:
 fmt.Println("Good afternoon!")
default:
 fmt.Println("Good evening!")
}
```

This is often cleaner than chained `if/else if`:

```
switch {
case x < 0:
 return "negative"
case x == 0:
 return "zero"
case x > 0:
 return "positive"
}
```

## Type Switch

Special form for interface type assertions:

```
func describe(i interface{}) {
 switch v := i.(type) {
 case int:
 fmt.Printf("Integer: %d\n", v)
 case string:
 fmt.Printf("String: %s\n", v)
 case bool:
 fmt.Printf("Boolean: %t\n", v)
 default:
 fmt.Printf("Unknown type: %T\n", v)
 }
}

describe(42) // Integer: 42
describe("hello") // String: hello
describe(true) // Boolean: true
```

```
describe(3.14) // Unknown type: float64
```

## Common Patterns

### Error Checking

```
result, err := someOperation()
if err != nil {
 return fmt.Errorf("operation failed: %w", err)
}
// Use result
```

### Boolean Assignment

```
var status string
if success {
 status = "completed"
} else {
 status = "failed"
}

// Note: Go doesn't have a ternary operator
// This doesn't work: status := success ? "completed" : "failed"
```

### Guard Clauses

```
func processUser(user *User) error {
 if user == nil {
 return errors.New("user cannot be nil")
 }
 if user.Name == "" {
 return errors.New("user name required")
 }
 if user.Age < 0 {
 return errors.New("invalid age")
 }

 // Main logic here
 return nil
}
```

## ok-idiom

```
// Map lookup
if val, ok := myMap[key]; ok {
 fmt.Println("Found:", val)
} else {
 fmt.Println("Not found")
}

// Type assertion
if str, ok := value.(string); ok {
 fmt.Println("String:", str)
}

// Channel receive
if val, ok := <-ch; ok {
 fmt.Println("Received:", val)
} else {
 fmt.Println("Channel closed")
}
```

## Comma-ok Pattern

```
// Check map existence
if _, exists := users[id]; exists {
 // User exists
}

// Check channel state
if _, open := <-ch; !open {
 // Channel is closed
}
```

## Switch vs If/Else

### Prefer Switch When:

```
// Multiple discrete values
switch status {
case "pending", "processing":
 queue()
case "done":
 complete()
}
```

```
case "error":
 retry()
}
```

## Prefer If/Else When:

```
// Complex conditions
if x > 0 && y > 0 && x*y < 100 {
 // ...
} else if x < 0 || y < 0 {
 // ...
}
```

## Summary

Feature	Description
<code>if</code>	Basic conditional
<code>if x := val; condition</code>	If with initialization
<code>switch expr</code>	Multi-way branching
<code>switch { case cond: }</code>	Expression-less switch
<code>switch v := x.(type)</code>	Type switch
<code>fallthrough</code>	Explicit fall-through in switch

## Related Topics

- [Loops and Iteration](#)
- [Errors as Values](#)

## More examples

### Example: if / else if ladder

Save as `main.go` and `go run .` (with `go mod init example` if needed).

```
package main

import "fmt"

func grade(score int) string {
 if score >= 90 {
```

```

 return "A"
 } else if score >= 80 {
 return "B"
 } else if score >= 70 {
 return "C"
 }
 return "F"
}

func main() {
 fmt.Println(grade(95))
 fmt.Println(grade(82))
 fmt.Println(grade(60))
}

```

Expected:

```

A
B
F

```

### Example: switch on strings

Save as main.go and go run . (with go mod init example if needed).

```

package main

import "fmt"

func main() {
 for _, cmd := range []string{"start", "stop", "wat"} {
 switch cmd {
 case "start":
 fmt.Println(cmd, "-> running")
 case "stop":
 fmt.Println(cmd, "-> halted")
 default:
 fmt.Println(cmd, "-> unknown")
 }
 }
}

```

Expected:

```

start -> running
stop -> halted

```

```
wat -> unknown
```

## Runnable example

Save as `main.go`. From an empty directory:

```
go mod init example
go run .
```

```
package main

import (
 "fmt"
 "runtime"
)

func sign(x int) string {
 switch {
 case x < 0:
 return "negative"
 case x == 0:
 return "zero"
 default:
 return "positive"
 }
}

func describe(i any) {
 switch v := i.(type) {
 case int:
 fmt.Printf("Integer: %d\n", v)
 case string:
 fmt.Printf("String: %s\n", v)
 case bool:
 fmt.Printf("Boolean: %t\n", v)
 default:
 fmt.Printf("Unknown type: %T value=%v\n", v, v)
 }
}

func main() {
 // if with short initialization (err scoped to if/else)
 if n := 10; n > 0 {
 fmt.Println("positive init:", n)
 }
}
```

```

x := -3
if x > 0 {
 fmt.Println("Positive")
} else if x < 0 {
 fmt.Println("Negative")
} else {
 fmt.Println("Zero")
}

// switch: no fall-through by default; multi-case lists
day := "Saturday"
switch day {
case "Monday":
 fmt.Println("Start of week")
case "Saturday", "Sunday":
 fmt.Println("Weekend!")
default:
 fmt.Println("Weekday")
}

switch os := runtime.GOOS; os {
case "darwin":
 fmt.Println("platform: macOS")
case "linux":
 fmt.Println("platform: Linux")
default:
 fmt.Println("platform:", os)
}

for _, v := range []int{-1, 0, 2} {
 fmt.Printf("sign(%d)=%s\n", v, sign(v))
}

// ok-idiom on a map
m := map[string]int{"alice": 30}
if age, ok := m["alice"]; ok {
 fmt.Println("found alice:", age)
}
if _, ok := m["bob"]; !ok {
 fmt.Println("bob missing")
}

describe(42)
describe("hello")
describe(3.14)
}

```

**Expected output** (illustrative; platform line varies):

```
positive init: 10
Negative
Weekend!
platform: macOS
sign(-1)=negative
sign(0)=zero
sign(2)=positive
found alice: 30
bob missing
Integer: 42
String: hello
Unknown type: float64 value=3.14
```

**What to notice:** - `if init; cond` limits temporary variables to the `if` statement. - Expression-less `switch` replaces long `if/else` chains. - Type switches branch on dynamic type of **any**. - `Map` lookup uses the comma-ok idiom, not sentinel ages alone.

**Try next:** Add `fallthrough` under `case "Monday"` and switch `day` to `"Monday"` to see both cases print.



# Loops and Iteration

## Overview

Go has only one looping construct: the `for` loop. This single keyword handles all looping patterns.

## The Basic for Loop

```
for i := 0; i < 10; i++ {
 fmt.Println(i)
}
```

## The while Loop (Condition Only)

```
n := 1
for n < 100 {
 n *= 2
}
```

## The Infinite Loop

```
for {
 if done {
 break
 }
}
```

## The range Loop

### Slices and Arrays

```
nums := []int{10, 20, 30}

for i, v := range nums {
 fmt.Printf("%d: %d\n", i, v)
}

for _, v := range nums { // Value only
 fmt.Println(v)
}
```

### Maps

```
ages := map[string]int{"Alice": 30, "Bob": 25}

for key, value := range ages {
 fmt.Printf("%s: %d\n", key, value)
}
```

### Strings

```
for i, r := range "Hello" {
 fmt.Printf("%d: %c\n", i, r)
}
```

### Channels

```
for v := range ch { // Exits when channel closes
 fmt.Println(v)
}
```

## Iterators (Go 1.23+)

Go 1.23 introduced “range-over-func,” allowing you to use `range` with custom iterator functions.

## Sequence Iterators

A sequence iterator is a function that takes a yield function: `func(yield func(V) bool)` (single value) or `func(yield func(K, V) bool)` (key-value).

```
func All[T any](s []T) iter.Seq[T] {
 return func(yield func(T) bool) {
 for _, v := range s {
 if !yield(v) {
 return
 }
 }
 }
}

// Usage
for v := range All(nums) {
 fmt.Println(v)
}
```

## Pull Iterators

For more control, you can use pull iterators:

```
next, stop := iter.Pull(All(nums))
defer stop()

for {
 v, ok := next()
 if !ok {
 break
 }
 fmt.Println(v)
}
```

## Loop Control

### break and continue

```
for i := 0; i < 10; i++ {
 if i == 5 {
 break // Exit loop
 }
 if i%2 == 0 {
```

```

 continue // Skip to next iteration
 }
 fmt.Println(i)
}

```

## Labels for Nested Loops

```

outer:
for i := 0; i < 3; i++ {
 for j := 0; j < 3; j++ {
 if i == 1 && j == 1 {
 break outer // Break both loops
 }
 }
}

```

## Common Patterns

```

// Reverse iteration
for i := len(items) - 1; i >= 0; i-- {
 process(items[i])
}

// Step by N
for i := 0; i < 100; i += 10 {
 fmt.Println(i)
}

```

## Summary

Pattern	Syntax
Classic for	for i := 0; i < n; i++ {}
While loop	for condition {}
Infinite loop	for {}
Range	for i, v := range slice {}
Iterators	for v := range myIter() {}

## Related Topics

- [Arrays and Slices](#)

- [Maps](#)

## More examples

### Example: classic for and range

Save as `main.go` and `go run .` (with `go mod init example` if needed).

```
package main

import "fmt"

func main() {
 sum := 0
 for i := 1; i <= 4; i++ {
 sum += i
 }
 fmt.Println("sum:", sum)

 for i, v := range []string{"a", "b", "c"} {
 fmt.Printf("%d:%s ", i, v)
 }
 fmt.Println()
}
```

Expected:

```
sum: 10
0:a 1:b 2:c
```

### Example: break and continue

Save as `main.go` and `go run .` (with `go mod init example` if needed).

```
package main

import "fmt"

func main() {
 for i := 0; i < 6; i++ {
 if i%2 == 0 {
 continue
 }
 if i > 3 {
 break
 }
 }
}
```

```

 }
 fmt.Println("odd:", i)
}
}

```

Expected:

```

odd: 1
odd: 3

```

## Runnable example

Save as `main.go`. From an empty directory:

```

go mod init example
go run .

```

```

package main

import "fmt"

func main() {
 // Classic for
 sum := 0
 for i := 1; i <= 5; i++ {
 sum += i
 }
 fmt.Println("sum 1..5:", sum)

 // while-style
 n := 1
 for n < 100 {
 n *= 2
 }
 fmt.Println("first power of two >= 100:", n)

 // range over slice
 nums := []int{10, 20, 30}
 for i, v := range nums {
 fmt.Printf("slice[%d]=%d\n", i, v)
 }

 // range over map (order not guaranteed)
 ages := map[string]int{"Ada": 36, "Grace": 85}
 for name, age := range ages {

```

```

 fmt.Printf("%s is %d\n", name, age)
}

// range over string yields runes
for i, r := range "Go " {
 fmt.Printf("string index %d rune %c\n", i, r)
}

// break / continue / labeled break
fmt.Print("odds < 5: ")
for i := 0; i < 10; i++ {
 if i >= 5 {
 break
 }
 if i%2 == 0 {
 continue
 }
 fmt.Print(i, " ")
}
fmt.Println()

outer:
for i := 0; i < 3; i++ {
 for j := 0; j < 3; j++ {
 if i == 1 && j == 1 {
 fmt.Printf("breaking outer at i=%d j=%d\n", i, j)
 break outer
 }
 }
}

// reverse step
for i := 4; i >= 0; i -= 2 {
 fmt.Println("reverse step:", i)
}
}

```

**Expected output** (illustrative; map line order may vary):

```

sum 1..5: 15
first power of two >= 100: 128
slice[0]=10
slice[1]=20
slice[2]=30
Ada is 36
Grace is 85
string index 0 rune G

```

```
string index 1 rune o
string index 2 rune
odds < 5: 1 3
breaking outer at i=1 j=1
reverse step: 4
reverse step: 2
reverse step: 0
```

**What to notice:** - One keyword (**for**) covers C-style, while-style, and infinite loops. - **range** adapts to slices, maps, and strings (byte index + rune). - **continue** / **break** and labels control nested loops cleanly. - Map iteration order is randomized—never rely on it.

**Try next:** Change the string to include combining characters or emoji and print both **i** and **len("...")**.



# Arrays and Slices

## Overview

Arrays and slices are Go's primary sequence types. Arrays are fixed-size, while slices are dynamic views into arrays.

## Arrays

### Declaration

```
var arr [5]int // Zero values: [0 0 0 0 0]
arr := [5]int{1, 2, 3, 4, 5} // Literal
arr := [...]int{1, 2, 3} // Size inferred: [3]int
```

### Properties

- Fixed size (part of type): `[5]int` `[6]int`
- Value semantics: copying creates independent copy
- Zero value: array of zero values

```
a := [3]int{1, 2, 3}
b := a // Copy!
b[0] = 99
fmt.Println(a) // [1 2 3] (unchanged)
```

## Slices

### Creation

```
var s []int // nil slice
s := []int{1, 2, 3} // Literal
s := make([]int, 5) // Length 5, capacity 5
s := make([]int, 0, 10) // Length 0, capacity 10
```

## From Array

```
arr := [5]int{1, 2, 3, 4, 5}
s := arr[1:4] // [2 3 4] - shares memory with arr
```

## Slice Internals & Reallocation

A slice header consists of three words: a pointer to the backing array, a length (`len`), and a capacity (`cap`).

```
flow:
 "ptr: 0x7FFF00" -> E0
```

When `append()` exceeds `cap`, Go allocates a new, larger backing array and copies existing elements:

```
flow:
 OldSlice -> Realloc (cap full)
```

## Length and Capacity

```
s := make([]int, 3, 5)
len(s) // 3 (elements accessible)
cap(s) // 5 (space available)
```

## Slice Operations

### Append

```
s := []int{1, 2, 3}
s = append(s, 4) // [1 2 3 4]
s = append(s, 5, 6, 7) // [1 2 3 4 5 6 7]
s = append(s, other...) // Append another slice
```

### Slicing

```
s := []int{0, 1, 2, 3, 4, 5}
s[1:4] // [1 2 3]
s[:3] // [0 1 2]
s[3:] // [3 4 5]
```

```
s[:] // [0 1 2 3 4 5] (copy of slice header)
```

## Copy

```
src := []int{1, 2, 3}
dst := make([]int, len(src))
copy(dst, src)
```

## The slices Package (Go 1.21+)

The standard library `slices` package provides generic utilities for common operations.

```
import "slices"

slices.Sort(nums) // Sort in place
slices.Reverse(nums) // Reverse in place
slices.Contains(nums, 42) // Search
slices.Index(nums, 42) // Find index
slices.Delete(nums, i, j) // Remove range [i, j)
slices.Clone(nums) // Shallow copy
slices.Equal(s1, s2) // Compare
```

## Modern Loop Pattern (Go 1.23+)

Using iterators with slices:

```
for i, v := range slices.All(nums) { /* ... */ }
for v := range slices.Values(nums) { /* ... */ }
```

## nil vs Empty Slice

```
var nilSlice []int // nil
emptySlice := []int{} // not nil, but empty

nilSlice == nil // true
emptySlice == nil // false
len(nilSlice) // 0
len(emptySlice) // 0
```

## Common Patterns

### Remove Element

```
s = append(s[:i], s[i+1:]...) // Remove element at index i
```

### Insert Element

```
s = append(s[:i], append([]int{v}, s[i:]...)...)
```

### Stack

```
stack = append(stack, v) // Push
v, stack = stack[len(stack)-1], stack[:len(stack)-1] // Pop
```

## Summary

Feature	Array	Slice
Size	Fixed	Dynamic
Type	[n]T	[]T
Zero value	Array of zeros	nil
Comparison	== works	Not comparable

## Related Topics

- [Working with Strings](#)
- [Maps](#)

## More examples

### Example: array vs slice

Save as `main.go` and go `run .` (with `go mod init example` if needed).

```
package main

import "fmt"
```

```

func main() {
 var arr [3]int = [3]int{1, 2, 3}
 s := arr[:]
 s[0] = 9
 fmt.Println("arr:", arr)
 fmt.Println("slice:", s, "len", len(s), "cap", cap(s))
}

```

Expected:

```

arr: [9 2 3]
slice: [9 2 3] len 3 cap 3

```

## Example: append and capacity growth

Save as `main.go` and `go run .` (with `go mod init example` if needed).

```

package main

import "fmt"

func main() {
 s := make([]int, 0, 2)
 fmt.Printf("start len=%d cap=%d\n", len(s), cap(s))
 s = append(s, 1, 2)
 fmt.Printf("full len=%d cap=%d %v\n", len(s), cap(s), s)
 s = append(s, 3)
 fmt.Printf("grow len=%d cap=%d %v\n", len(s), cap(s), s)
}

```

Expected:

```

start len=0 cap=2
full len=2 cap=2 [1 2]
grow len=3 cap=4 [1 2 3]

```

## Runnable example

Save as `main.go`. From an empty directory:

```

go mod init example
go run .

```

```

package main

import (
 "fmt"
 "slices"
)

func main() {
 // Arrays: fixed size, value semantics
 a := [3]int{1, 2, 3}
 b := a
 b[0] = 99
 fmt.Println("array a (unchanged):", a)
 fmt.Println("array copy b:", b)

 // Slices: dynamic, share backing array until reallocation
 s := []int{10, 20, 30}
 fmt.Printf("s=%v len=%d cap=%d\n", s, len(s), cap(s))

 t := s[1:]
 t[0] = 200
 fmt.Println("after t[0]=200, s=", s, "t=", t)

 // append may reallocate when cap is exceeded
 s2 := make([]int, 3, 4)
 copy(s2, []int{1, 2, 3})
 fmt.Printf("before grow: %v len=%d cap=%d\n", s2, len(s2), cap(s2))
 s2 = append(s2, 4)
 fmt.Printf("cap full: %v len=%d cap=%d\n", s2, len(s2), cap(s2))
 s2 = append(s2, 5)
 fmt.Printf("after grow: %v len=%d cap=%d\n", s2, len(s2), cap(s2))

 // Independent copy
 src := []int{1, 2, 3}
 dst := make([]int, len(src))
 copy(dst, src)
 dst[0] = 7
 fmt.Println("src", src, "dst", dst)

 // nil vs empty
 var nilSlice []int
 empty := []int{}
 fmt.Printf("nil==nil? %v empty==nil? %v\n", nilSlice == nil, empty == nil)
 nilSlice = append(nilSlice, 1, 2)
 fmt.Println("append on nil:", nilSlice)

 // slices package (Go 1.21+)

```

```

 nums := []int{3, 1, 2}
 cloned := slices.Clone(nums)
 slices.Sort(cloned)
 fmt.Println("sorted clone:", cloned, "contains 2?", slices.Contains(cloned, 2))
}

```

**Expected output** (illustrative; exact new cap may vary by Go version):

```

array a (unchanged): [1 2 3]
array copy b: [99 2 3]
s=[10 20 30] len=3 cap=3
after t[0]=200, s= [10 200 30] t= [200 30]
before grow: [1 2 3] len=3 cap=4
cap full: [1 2 3 4] len=4 cap=4
after grow: [1 2 3 4 5] len=5 cap=8
src [1 2 3] dst [7 2 3]
nil==nil? true empty==nil? false
append on nil: [1 2]
sorted clone: [1 2 3] contains 2? true

```

**What to notice:** - Assigning an array copies elements; assigning/slicing a slice shares storage. - `append` grows length and may allocate a larger backing array. - `copy` creates an independent element sequence. - `slices.Sort` / `Clone` are stdlib helpers—no third-party deps.

**Try next:** After `t := s[1:]`, append enough elements to `t` to force reallocation, then change `t[0]` and see whether `s` still changes.

# Working with Strings

## Overview

Strings in Go are immutable sequences of bytes, typically UTF-8 encoded. Understanding the distinction between bytes and runes is essential.

## String Basics

```
s := "Hello, World!"
s := `Raw string with
multiple lines`
```

## Immutability

```
s := "hello"
// s[0] = 'H' // Error: cannot assign to s[0]
s = "Hello" // OK: reassign entire string
```

## Bytes vs Runes

```
s := "Hello, "

len(s) // 13 (bytes)
len([]rune(s)) // 9 (characters)
```

## Iterating

```
// By byte
for i := 0; i < len(s); i++ {
 fmt.Printf("%x ", s[i])
}

// By rune (character)
```



```

for i, r := range s {
 fmt.Printf("%d: %c\n", i, r)
}

```

## String Operations

### Concatenation

```

s := "Hello" + ", " + "World"

// Efficient building
var b strings.Builder
b.WriteString("Hello")
b.WriteString(", World")
result := b.String()

```

### Comparison

```

s1 == s2 // Equality
s1 < s2 // Lexicographic
strings.EqualFold(s1, s2) // Case-insensitive

```

## strings Package

```

import "strings"

strings.Contains(s, "sub") // true/false
strings.HasPrefix(s, "pre") // true/false
strings.HasSuffix(s, "suf") // true/false
strings.Index(s, "sub") // Position or -1
strings.Split(s, ",") // []string
strings.Join(parts, ",") // string
strings.ToUpper(s) // UPPERCASE
strings.ToLower(s) // lowercase
strings.TrimSpace(s) // Remove whitespace
strings.Replace(s, "old", "new", n) // Replace n times
strings.ReplaceAll(s, "old", "new") // Replace all

```

## strconv Package

```
import "strconv"

// String to int
n, err := strconv.Atoi("42")

// Int to string
s := strconv.Itoa(42)

// Parse float
f, err := strconv.ParseFloat("3.14", 64)

// Format float
s := strconv.FormatFloat(3.14, 'f', 2, 64)
```

## fmt Package

```
// Formatting
s := fmt.Sprintf("Name: %s, Age: %d", name, age)

// Parsing
var name string
var age int
fmt.Sscanf("Alice 30", "%s %d", &name, &age)
```

## Rune Operations

```
r := 'A'
unicode.IsLetter(r) // true
unicode.IsDigit(r) // false
unicode.ToUpper(r) // 'A'
unicode.ToLower(r) // 'a'
```

## Summary

Operation	Method
Length (bytes)	<code>len(s)</code>
Length (chars)	<code>len([]rune(s))</code>
Concatenate	<code>+</code> or <code>strings.Builder</code>

Operation	Method
Contains	<code>strings.Contains()</code>
Split/Join	<code>strings.Split()</code> , <code>strings.Join()</code>
Convert	<code>strconv.Atoi()</code> , <code>strconv.Itoa()</code>

## Related Topics

- [Predeclared Types](#)
- [I/O and Streaming](#)

## More examples

### Example: bytes vs runes

Save as `main.go` and `go run .` (with `go mod init example` if needed).

```
package main

import "fmt"

func main() {
 s := "Go "
 fmt.Println("bytes:", len(s))
 fmt.Println("runes:", len([]rune(s)))
 for i, r := range s {
 fmt.Printf("i=%d r=%q\n", i, r)
 }
}
```

Expected:

```
bytes: 5
runes: 3
i=0 r='G'
i=1 r='o'
i=2 r=' '
```

### Example: strings.Builder

Save as `main.go` and `go run .` (with `go mod init example` if needed).

```

package main

import (
 "fmt"
 "strings"
)

func main() {
 var b strings.Builder
 b.WriteString("hello")
 b.WriteByte(' ')
 b.WriteString("gopher")
 fmt.Println(b.String())
 fmt.Println("len:", b.Len())
}

```

Expected:

```

hello gopher
len: 12

```

## Runnable example

Save as `main.go`. From an empty directory:

```

go mod init example
go run .

```

```

package main

import (
 "fmt"
 "strconv"
 "strings"
 "unicode"
)

func main() {
 s := "Hello, "
 fmt.Printf("%q bytes=%d runes=%d\n", s, len(s), len([]rune(s)))

 fmt.Print("bytes: ")
 for i := 0; i < len(s); i++ {
 fmt.Printf("%02x ", s[i])
 }
}

```

```

fmt.Println()

fmt.Println("runes:")
for i, r := range s {
 fmt.Printf(" index %d -> %c U+%04X\n", i, r, r)
}

// Immutability: reassign whole string only
s = "Hello"
// s[0] = 'h' // compile error if uncommented
fmt.Println("reassigned:", s)

var b strings.Builder
b.WriteString("Go")
b.WriteString(" ")
b.WriteString("strings")
built := b.String()
fmt.Println("builder:", built)

fmt.Println("Contains Go?", strings.Contains(built, "Go"))
fmt.Println("fields:", strings.Fields(" a b\tc "))
fmt.Println("join:", strings.Join([]string{"a", "b", "c"}, "-"))
fmt.Println("upper:", strings.ToUpper("gopher"))

n, err := strconv.Atoi("42")
fmt.Printf("Atoi=%d err=%v Itoa=%s\n", n, err, strconv.Itoa(n+1))
f, _ := strconv.ParseFloat("3.14", 64)
fmt.Println("float:", strconv.FormatFloat(f, 'f', 2, 64))

r := 'g'
fmt.Printf("rune %q letter=%v upper=%q\n", r, unicode.IsLetter(r), unicode.ToUpper(r))

fmt.Println(fmt.Sprintf("Name: %s Age: %d", "Ada", 36))
}

```

**Expected output** (illustrative):

```

"Hello, " bytes=13 runes=9
bytes: 48 65 6c 6c 6f 20 e4 b8 96 e7 95 8c
runes:
 index 0 -> H U+0048
 index 1 -> e U+0065
 index 2 -> l U+006C
 index 3 -> l U+006C
 index 4 -> o U+006F
 index 5 -> , U+002C
 index 6 -> U+0020

```

```
index 7 -> U+4E16
index 10 -> U+754C
reassigned: Hello
builder: Go strings
Contains Go? true
fields: [a b c]
join: a-b-c
upper: GOPHER
Atoi=42 err=<nil> Itoa=43
float: 3.14
rune 'g' letter=true upper='G'
Name: Ada Age: 36
```

**What to notice:** - Multi-byte UTF-8 characters make `len` (bytes) larger than rune count. - `range` over a string steps by runes but indexes are byte offsets. - `strings.Builder` avoids quadratic + chains for assembly. - `strconv` is the explicit bridge between text and numbers.

**Try next:** Use `strings.ReplaceAll` to censor a word in a sentence and print before/after.

# Maps

## Overview

Maps are Go's built-in hash table type, providing  $O(1)$  average-time lookups, insertions, and deletions.

## Creating Maps

```
var m map[string]int // nil map (read-only!)
m := make(map[string]int) // Empty, ready to use
m := make(map[string]int, 100) // With capacity hint
m := map[string]int{ // Literal
 "one": 1,
 "two": 2,
}
```

## Basic Operations

### Set

```
m["key"] = value
```

### Get

```
val := m["key"] // Returns zero value if missing
```

### Delete

```
delete(m, "key") // No-op if key doesn't exist
```

## Check Existence

```
val, ok := m["key"]
if ok {
 // Key exists
}

if _, exists := m["key"]; exists {
 // Key exists
}
```

## nil Map Behavior

```
var m map[string]int // nil

val := m["key"] // OK: returns 0
m["key"] = 1 // PANIC! Cannot write to nil map
```

Always initialize before writing:

```
m := make(map[string]int)
m["key"] = 1 // OK
```

## Iteration

```
for key, value := range m {
 fmt.Printf("%s: %d\n", key, value)
}

for key := range m { // Keys only
 fmt.Println(key)
}
```

**Warning:** Iteration order is randomized!



## Common Patterns

### Set (Unique Values)

```
set := make(map[string]struct{})
set["item"] = struct{}{}
if _, exists := set["item"]; exists {
 // Item is in set
}
delete(set, "item")
```

### Counter

```
counter := make(map[string]int)
for _, word := range words {
 counter[word]++ // Zero value works!
}
```

### Grouping

```
groups := make(map[string][]User)
for _, user := range users {
 groups[user.Country] = append(groups[user.Country], user)
}
```

### Default Value

```
val := m["key"]
if val == 0 {
 val = defaultValue
}
```

## Concurrency Warning

Maps are **not** goroutine-safe:

```
// Unsafe: concurrent read/write
go func() { m["key"] = 1 }()
go func() { _ = m["key"] }()
```

```
// Use sync.Map or mutex
var mu sync.Mutex
mu.Lock()
m["key"] = 1
mu.Unlock()
```

## The maps Package (Go 1.21+)

The standard library `maps` package provides generic utilities for common operations.

```
import "maps"

maps.Clone(m) // Shallow copy
maps.Equal(m1, m2) // Compare
maps.DeleteFunc(m, func(k K, v V) bool {
 return v < 10 // Conditional delete
})
```

## Modern Loop Pattern (Go 1.23+)

Using iterators with maps:

```
for k, v := range maps.All(m) { /* ... */ }
for k := range maps.Keys(m) { /* ... */ }
for v := range maps.Values(m) { /* ... */ }
```

## Map Internals

- Keys must be comparable (`==` must work)
- Valid key types: `bool`, `numeric`, `string`, `pointer`, `channel`, `interface`, `structs/arrays` of comparable types
- Invalid: `slices`, `maps`, `functions`

## Summary

Operation	Syntax
Create	<code>make(map[K]V)</code>
Set	<code>m[key] = value</code>
Get	<code>val := m[key]</code>
Check	<code>val, ok := m[key]</code>

Operation	Syntax
Delete	<code>delete(m, key)</code>
Length	<code>len(m)</code>

## Related Topics

- [Arrays and Slices](#)
- [Synchronization](#)

## More examples

### Example: comma-ok lookup

Save as `main.go` and `go run .` (with `go mod init example` if needed).

```
package main

import "fmt"

func main() {
 m := map[string]int{"a": 1}
 if v, ok := m["a"]; ok {
 fmt.Println("a:", v)
 }
 if _, ok := m["missing"]; !ok {
 fmt.Println("missing: not present")
 }
}
```

Expected:

```
a: 1
missing: not present
```

### Example: delete and range

Save as `main.go` and `go run .` (with `go mod init example` if needed).

```
package main

import "fmt"

func main() {
```

```

 m := map[string]int{"x": 10, "y": 20}
 delete(m, "x")
 fmt.Println("len:", len(m))
 for k, v := range m {
 fmt.Printf("%s=%d\n", k, v)
 }
}

```

Expected:

```

len: 1
y=20

```

## Runnable example

Save as main.go. From an empty directory:

```

go mod init example
go run .

```

```

package main

import (
 "fmt"
 "maps"
)

func main() {
 // nil map: read ok, write panics - initialize first
 var nilMap map[string]int
 fmt.Println("nil read:", nilMap["x"])

 m := make(map[string]int)
 m["one"] = 1
 m["two"] = 2
 fmt.Println("m:", m, "len:", len(m))

 if v, ok := m["two"]; ok {
 fmt.Println("found two:", v)
 }
 if _, ok := m["three"]; !ok {
 fmt.Println("three missing")
 }

 delete(m, "one")
}

```

```

fmt.Println("after delete one:", m)

// Word counter uses zero value on missing keys
counter := make(map[string]int)
for _, w := range []string{"go", "is", "go", "fun", "go"} {
 counter[w]++
}
fmt.Println("counts:", counter)

// Set via map[T]struct{}
set := make(map[string]struct{})
set["alpha"] = struct{}{}
set["beta"] = struct{}{}
if _, ok := set["alpha"]; ok {
 fmt.Println("set has alpha")
}

// Grouping
type User struct{ Name, Country string }
users := []User{
 {"Ada", "UK"},
 {"Linus", "FI"},
 {"Alan", "UK"},
}
groups := make(map[string][]string)
for _, u := range users {
 groups[u.Country] = append(groups[u.Country], u.Name)
}
fmt.Println("groups:", groups)

// maps package (Go 1.21+)
clone := maps.Clone(counter)
clone["fun"] = 99
fmt.Println("original fun:", counter["fun"], "clone fun:", clone["fun"])
fmt.Println("equal?", maps.Equal(counter, clone))

fmt.Println("keys (order random):")
for k, v := range counter {
 fmt.Printf(" %s=%d\n", k, v)
}
}

```

**Expected output** (illustrative; key order varies):

```

nil read: 0
m: map[one:1 two:2] len: 2
found two: 2

```

```
three missing
after delete one: map[two:2]
counts: map[fun:1 go:3 is:1]
set has alpha
groups: map[FI:[Linus] UK:[Ada Alan]]
original fun: 1 clone fun: 99
equal? false
keys (order random):
 go=3
 is=1
 fun=1
```

**What to notice:** - Comma-ok distinguishes missing keys from zero values. - `counter[w]++` works because missing keys read as 0. - `map[T]struct{}` is an idiomatic set with near-zero value size. - Iteration order is deliberately random.

**Try next:** Uncomment a write to `nilMap["x"] = 1` once to see the panic, then remove it.

# Structs and Data Modeling

## Overview

Structs are Go's primary mechanism for creating custom composite types. They group related fields together.

## Defining Structs

```
type Person struct {
 Name string
 Age int
 Email string
 Active bool
}
```

## Creating Instances

```
// Zero value
var p Person // All fields get zero values

// Literal (named fields - preferred)
p := Person{
 Name: "Alice",
 Age: 30,
 Email: "alice@example.com",
}

// Literal (positional - fragile)
p := Person{"Alice", 30, "alice@example.com", true}

// Using new
p := new(Person) // *Person with zero values

// Expression-based new (Go 1.26+)
p := new(Person{Name: "Alice"}) // *Person initialized with values
```

```
Embedded fields (composition)
```

```
Employee
|-- Person
| |-- Name
| |-- Age
|-- Address
| |-- City
|-- Title
```

## Accessing Fields

```
p.Name = "Bob"
fmt.Println(p.Age)

// Pointer access (automatic dereference)
ptr := &p
ptr.Name = "Carol" // Same as (*ptr).Name
```

## Anonymous Structs

```
point := struct {
 X, Y int
}{10, 20}

// Useful for one-off data
data := struct {
 ID int
 Value string
}{1, "test"}
```

## Embedding (Composition)

```
type Address struct {
 Street string
 City string
}

type Employee struct {
 Person // Embedded
```



```

 Address // Embedded
 Title string
}

e := Employee{
 Person: Person{Name: "Alice", Age: 30},
 Address: Address{City: "NYC"},
 Title: "Engineer",
}

// Promoted fields
e.Name // Same as e.Person.Name
e.City // Same as e.Address.City

```

## Promoted field keys in literals (Go 1.27+)

A key in a struct literal may be any valid field selector, not only a top-level field name. You can initialize a promoted embedded field without nesting another literal:

```

e := Employee{
 Name: "Alice", // promoted from Person
 Age: 30,
 City: "NYC", // promoted from Address
 Title: "Engineer",
}

```

The nested form still works and is clearer when several fields of the same embedded type are set together. Ambiguous names (two embeddings that both promote `Name`) still require the nested form.

## Struct Tags

Metadata for serialization and other tools:

```

type User struct {
 ID int `json:"id" db:"user_id"`
 Name string `json:"name,omitempty"`
 Password string `json:"- "` // Omit from JSON
 CreatedAt time.Time `json:"created_at"`
}

```

## Reading Tags

```
import "reflect"

t := reflect.TypeOf(User{})
field, _ := t.FieldByName("Name")
tag := field.Tag.Get("json") // "name,omitempty"
```

## Comparison

Structs are comparable if all fields are comparable:

```
p1 := Person{Name: "Alice"}
p2 := Person{Name: "Alice"}
p1 == p2 // true
```

## Constructors

Go uses factory functions:

```
func NewPerson(name string, age int) *Person {
 return &Person{
 Name: name,
 Age: age,
 Active: true, // Default
 }
}

func NewPersonWithDefaults() *Person {
 return &Person{
 Active: true,
 }
}
```

## Summary

Pattern	Usage
Define	<code>type Name struct { fields }</code>
Create	<code>Name{Field: value}</code>
Embed	Include type name without field name

Pattern	Usage
Tags	Field Type <code>\key:"value"   </code> Constructor <code> func NewType() *Type'</code>

## Related Topics

- [Methods and Receivers](#)
- [Designing with Interfaces](#)

## More examples

### Example: literal and field access

Save as `main.go` and `go run .` (with `go mod init example` if needed).

```
package main

import "fmt"

type User struct {
 Name string
 Age int
}

func main() {
 u := User{Name: "ada", Age: 36}
 fmt.Printf("%+v\n", u)
 u.Age++
 fmt.Println("age:", u.Age)
}
```

Expected:

```
{Name:ada Age:36}
age: 37
```

### Example: embedding promotes fields

Save as `main.go` and `go run .` (with `go mod init example` if needed).

```
package main
```

```

import "fmt"

type Address struct {
 City string
}

type Person struct {
 Name string
 Address
}

func main() {
 p := Person{Name: "lin", Address: Address{City: "Berlin"}}
 fmt.Println(p.Name, "in", p.City) // promoted field
}

```

Expected:

```
lin in Berlin
```

## Runnable example

Save as `main.go`. From an empty directory:

```

go mod init example
go run .

package main

import (
 "encoding/json"
 "fmt"
 "reflect"
)

type Address struct {
 Street string
 City string
}

type Person struct {
 Name string
 Age int
}

```

```

type Employee struct {
 Person // embedded
 Address // embedded
 Title string
}

type User struct {
 ID int `json:"id"`
 Name string `json:"name,omitempty"`
 Password string `json:"- "`
}

func NewEmployee(name, title, city string) *Employee {
 return &Employee{
 Person: Person{Name: name, Age: 0},
 Address: Address{City: city},
 Title: title,
 }
}

func main() {
 var zero Person
 fmt.Printf("zero person: %+v\n", zero)

 e := NewEmployee("Ada", "Engineer", "London")
 e.Age = 36 // promoted field
 fmt.Printf("employee: name=%s title=%s city=%s age=%d\n", e.Name, e.Title, e.City, e.Age)

 // Pointer field access auto-dereferences
 p := &Person{Name: "Grace", Age: 85}
 p.Age++
 fmt.Printf("pointer person: %+v\n", p)

 // Anonymous struct for one-off data
 point := struct{ X, Y int }{10, 20}
 fmt.Printf("point=(%d,%d)\n", point.X, point.Y)

 // Comparable when all fields are comparable
 a := Person{Name: "Ada", Age: 36}
 b := Person{Name: "Ada", Age: 36}
 fmt.Println("a==b?", a == b)

 // Tags for JSON + reflection
 u := User{ID: 7, Name: "Ada", Password: "secret"}
 raw, err := json.Marshal(u)
 if err != nil {

```

```

 fmt.Println("json error:", err)
 return
 }
 fmt.Println("json:", string(raw))

 t := reflect.TypeOf(User{})
 if f, ok := t.FieldByName("Name"); ok {
 fmt.Println("Name tag:", f.Tag.Get("json"))
 }
 if f, ok := t.FieldByName("Password"); ok {
 fmt.Println("Password tag:", f.Tag.Get("json"))
 }
}

```

**Expected output** (illustrative):

```

zero person: {Name: Age:0}
employee: name=Ada title=Engineer city=London age=36
pointer person: &{Name:Grace Age:86}
point=(10,20)
a==b? true
json: {"id":7,"name":"Ada"}
Name tag: name,omitempty
Password tag: -

```

**What to notice:** - Embedding promotes fields (`e.Name`, `e.City`) without inheritance. - Factory `NewEmployee` sets defaults; zero `Person` is still valid. - JSON tags control names, `omitempty`, and secrets (`json:"-"`). - Struct equality works when every field is comparable.

**Try next:** Set `Name` to `""` and remarshal—observe `omitempty` dropping the field.

# Time and Scheduling

## Overview

The `time` package provides functionality for measuring and displaying time.

## Current Time

```
now := time.Now()
fmt.Println(now) // 2024-01-15 10:30:00 -0500 EST
```

## Duration

```
d := 5 * time.Second
d := time.Minute + 30*time.Second
d := time.ParseDuration("1h30m")
```

## Sleeping

```
time.Sleep(2 * time.Second)
```

## Timers

```
// One-shot timer
timer := time.NewTimer(5 * time.Second)
<-timer.C // Blocks until timer fires

// Cancel timer
if !timer.Stop() {
 <-timer.C
}
```

## Tickers

```
ticker := time.NewTicker(1 * time.Second)
defer ticker.Stop()

for t := range ticker.C {
 fmt.Println("Tick at", t)
}
```

## Formatting

```
// Format uses reference time: Mon Jan 2 15:04:05 MST 2006
now := time.Now()
fmt.Println(now.Format("2006-01-02")) // 2024-01-15
fmt.Println(now.Format("15:04:05")) // 10:30:00
fmt.Println(now.Format(time.RFC3339)) // 2024-01-15T10:30:00-05:00
```

## Parsing

```
t, err := time.Parse("2006-01-02", "2024-01-15")
t, err := time.Parse(time.RFC3339, "2024-01-15T10:30:00Z")
```

## Time Zones

```
loc, _ := time.LoadLocation("America/New_York")
t := time.Now().In(loc)

utc := time.Now().UTC()
```

## Comparisons

```
t1.Before(t2)
t1.After(t2)
t1.Equal(t2)
t1.Sub(t2) // Duration between
t1.Add(d) // Add duration
```



## Summary

Type/Function	Purpose
<code>time.Now()</code>	Current time
<code>time.Duration</code>	Time interval
<code>time.Timer</code>	One-shot delay
<code>time.Ticker</code>	Repeated intervals
<code>time.Format()</code>	Time to string
<code>time.Parse()</code>	String to time

## Related Topics

- [Context and Cancellation](#)
- [I/O and Streaming](#)

## More examples

### Example: duration arithmetic

Save as `main.go` and `go run .` (with `go mod init example` if needed).

```
package main

import (
 "fmt"
 "time"
)

func main() {
 d := 90 * time.Second
 fmt.Println("seconds:", d.Seconds())
 fmt.Println("plus minute:", d+time.Minute)
 fmt.Println("truncated:", d.Truncate(30*time.Second))
}
```

Expected:

```
seconds: 90
plus minute: 2m30s
truncated: 1m30s
```

## Example: parse and format

Save as `main.go` and `go run .` (with `go mod init example` if needed).

```
package main

import (
 "fmt"
 "time"
)

func main() {
 const layout = "2006-01-02"
 t, err := time.Parse(layout, "2026-07-28")
 if err != nil {
 fmt.Println("err:", err)
 return
 }
 fmt.Println("parsed:", t.Format(layout))
 fmt.Println("weekday:", t.Weekday())
}
```

Expected:

```
parsed: 2026-07-28
weekday: Tuesday
```

## Runnable example

Save as `main.go`. From an empty directory:

```
go mod init example
go run .
```

```
package main

import (
 "fmt"
 "time"
)

func main() {
 now := time.Now()
 fmt.Println("now:", now.Format(time.RFC3339))
 fmt.Println("date:", now.Format("2006-01-02"))
}
```

```

fmt.Println("clock:", now.Format("15:04:05"))

d := 90*time.Second + 500*time.Millisecond
parsed, err := time.ParseDuration("1h30m")
if err != nil {
 fmt.Println("ParseDuration:", err)
 return
}
fmt.Println("90.5s:", d, "parsed 1h30m:", parsed)

start := time.Now()
time.Sleep(50 * time.Millisecond)
elapsed := time.Since(start)
fmt.Printf("slept about %v (elapsed=%v)\n", 50*time.Millisecond, elapsed)

// Parse / compare / arithmetic
t1, err := time.Parse("2006-01-02", "2024-01-15")
if err != nil {
 fmt.Println(err)
 return
}
t2 := t1.Add(48 * time.Hour)
fmt.Println("t1:", t1.Format("2006-01-02"))
fmt.Println("t2:", t2.Format("2006-01-02"))
fmt.Println("t1 before t2?", t1.Before(t2))
fmt.Println("sub:", t2.Sub(t1))

utc := now.UTC()
fmt.Println("utc:", utc.Format(time.RFC3339))

// One-shot timer (short so the example finishes quickly)
timer := time.NewTimer(30 * time.Millisecond)
fire := <-timer.C
fmt.Println("timer fired at:", fire.Format("15:04:05.000"))

// Ticker: two ticks then stop
ticker := time.NewTicker(20 * time.Millisecond)
defer ticker.Stop()
for i := 0; i < 2; i++ {
 t := <-ticker.C
 fmt.Printf("tick %d at %s\n", i+1, t.Format("15:04:05.000"))
}
}

```

**Expected output** (illustrative; timestamps vary):

```
now: 2026-07-28T12:00:00-07:00
date: 2026-07-28
clock: 12:00:00
90.5s: 1m30.5s parsed 1h30m: 1h30m0s
slept about 50ms (elapsed=51.2ms)
t1: 2024-01-15
t2: 2024-01-17
t1 before t2? true
sub: 48h0m0s
utc: 2026-07-28T19:00:00Z
timer fired at: 12:00:00.031
tick 1 at 12:00:00.051
tick 2 at 12:00:00.071
```

**What to notice:** - Format layouts use the reference time Mon Jan 2 15:04:05 MST 2006. - Duration values multiply cleanly ( $90 * \text{time.Second}$ ). - Before / Add / Sub are the comparison and arithmetic API. - Always Stop tickers (and unused timers) to avoid leaks.

**Try next:** Load `time.LoadLocation("America/New_York")` and print `now.In(loc)`.

# Memory

# Functions in Depth

## Overview

Functions are fundamental building blocks in Go. They support multiple return values, named returns, variadic parameters, and closures.

## Function Declaration

```
func name(params) returnType {
 // body
}

func add(a, b int) int {
 return a + b
}

func greet(name string) {
 fmt.Println("Hello,", name)
}
```

## Multiple Return Values

```
func divide(a, b float64) (float64, error) {
 if b == 0 {
 return 0, errors.New("division by zero")
 }
 return a / b, nil
}

result, err := divide(10, 2)
```

## Named Return Values

```
func split(sum int) (x, y int) {
 x = sum * 4 / 9
 y = sum - x
 return // Naked return
}
```

## Variadic Functions

```
func sum(nums ...int) int {
 total := 0
 for _, n := range nums {
 total += n
 }
 return total
}

sum(1, 2, 3) // 6
sum([]int{1, 2, 3}...) // Spread slice
```

## Closures

Functions can capture variables from their enclosing scope:

```
func counter() func() int {
 count := 0
 return func() int {
 count++
 return count
 }
}

c := counter()
c() // 1
c() // 2
c() // 3
```

## Defer

Defer postpones execution until the surrounding function returns:

```
func readFile(name string) error {
 f, err := os.Open(name)
 if err != nil {
 return err
 }
 defer f.Close() // Executes when function returns

 // Use file...
 return nil
}
```

## Defer Order (LIFO)

```
defer fmt.Println("1")
defer fmt.Println("2")
defer fmt.Println("3")
// Output: 3, 2, 1
```

## Function Signatures

```
// Function type
type Operation func(int, int) int

func apply(op Operation, a, b int) int {
 return op(a, b)
}

add := func(a, b int) int { return a + b }
result := apply(add, 2, 3) // 5
```

## Summary

Feature	Syntax
Multiple returns	<code>func f() (T1, T2)</code>
Named returns	<code>func f() (x T1, y T2)</code>
Variadic	<code>func f(args ...T)</code>
Closure	<code>func() { /* capture vars */ }</code>
Defer	<code>defer cleanup()</code>



## Related Topics

- [Functions as Values](#)
- [Methods and Receivers](#)

## More examples

### Example: multiple returns and named results

Save as `main.go` and `go run .` (with `go mod init example` if needed).

```
package main

import (
 "errors"
 "fmt"
)

func div(a, b int) (quot int, err error) {
 if b == 0 {
 err = errors.New("divide by zero")
 return
 }
 quot = a / b
 return
}

func main() {
 q, err := div(10, 2)
 fmt.Println("10/2:", q, err)
 q, err = div(10, 0)
 fmt.Println("10/0:", q, err)
}
```

Expected:

```
10/2: 5 <nil>
10/0: 0 divide by zero
```

### Example: defer runs LIFO

Save as `main.go` and `go run .` (with `go mod init example` if needed).

```

package main

import "fmt"

func main() {
 fmt.Println("start")
 defer fmt.Println("first defer")
 defer fmt.Println("second defer")
 fmt.Println("end")
}

```

Expected:

```

start
end
second defer
first defer

```

## Runnable example

Save as `main.go`. Then:

```

go mod init example
go run .

package main

import (
 "errors"
 "fmt"
)

func add(a, b int) int { return a + b }

func divide(a, b float64) (float64, error) {
 if b == 0 {
 return 0, errors.New("division by zero")
 }
 return a / b, nil
}

func sum(nums ...int) int {
 total := 0
 for _, n := range nums {
 total += n
 }
}

```

```

 }
 return total
}

func counter() func() int {
 count := 0
 return func() int {
 count++
 return count
 }
}

func main() {
 fmt.Println("add:", add(2, 3))

 q, err := divide(10, 2)
 fmt.Println("divide ok:", q, err)
 _, err = divide(1, 0)
 fmt.Println("divide err:", err)

 fmt.Println("sum:", sum(1, 2, 3, 4))

 c := counter()
 fmt.Println("counter:", c(), c(), c())

 defer fmt.Println("defer: first (runs last)")
 defer fmt.Println("defer: second")
 fmt.Println("before return")
}

```

### Expected output:

```

add: 5
divide ok: 5 <nil>
divide err: division by zero
sum: 10
counter: 1 2 3
before return
defer: second
defer: first (runs last)

```

**What to notice:** Multiple returns make success and failure explicit; the closure keeps private count state; deferred calls run LIFO after the rest of `main`.

**Try next:** Add a named-return `split(sum int) (x, y int)` and print both values; wrap `divide` so callers always get a non-nil error message with the operands.

# Functions as Values

## Overview

In Go, functions are first-class citizens—they can be assigned to variables, passed as arguments, and returned from other functions.

## Function Variables

```
// Assign function to variable
add := func(a, b int) int {
 return a + b
}

result := add(2, 3) // 5

// Reassign
add = func(a, b int) int {
 return a + b + 1 // Different implementation
}
```

## Function Types

```
type BinaryOp func(int, int) int

var op BinaryOp = func(a, b int) int {
 return a + b
}
```

# Higher-Order Functions

## Functions as Parameters

```
func apply(nums []int, fn func(int) int) []int {
 result := make([]int, len(nums))
 for i, n := range nums {
 result[i] = fn(n)
 }
 return result
}

double := func(n int) int { return n * 2 }
result := apply([]int{1, 2, 3}, double) // [2, 4, 6]
```

## Functions as Return Values

```
func multiplier(factor int) func(int) int {
 return func(n int) int {
 return n * factor
 }
}

double := multiplier(2)
triple := multiplier(3)

double(5) // 10
triple(5) // 15
```

# Common Patterns

## Callback Pattern

```
func fetchData(url string, callback func(data []byte, err error)) {
 // Async operation
 go func() {
 data, err := http.Get(url)
 callback(data, err)
 }()
}
```

## Option Pattern

```
type Server struct {
 port int
 timeout time.Duration
}

type Option func(*Server)

func WithPort(p int) Option {
 return func(s *Server) { s.port = p }
}

func WithTimeout(t time.Duration) Option {
 return func(s *Server) { s.timeout = t }
}

func NewServer(opts ...Option) *Server {
 s := &Server{port: 8080, timeout: 30 * time.Second}
 for _, opt := range opts {
 opt(s)
 }
 return s
}

srv := NewServer(WithPort(9000), WithTimeout(time.Minute))
```

## Middleware Pattern

```
type Handler func(http.ResponseWriter, *http.Request)

func WithLogging(h Handler) Handler {
 return func(w http.ResponseWriter, r *http.Request) {
 log.Printf("%s %s", r.Method, r.URL)
 h(w, r)
 }
}
```

## Anonymous Functions

```
// Immediate invocation (IIFE)
result := func(x int) int {
 return x * x
}(5) // 25

// In goroutines
go func() {
 fmt.Println("async")
}()
```

## Summary

Pattern	Use Case
Function variable	Store/swap implementations
Higher-order	Transform, filter, reduce
Closures	Capture state
Options	Flexible configuration

## Related Topics

- [Functions in Depth](#)
- [Designing with Interfaces](#)

## More examples

### Example: swap function variables

Save as `main.go` and `go run .` (with `go mod init example` if needed).

```
package main

import "fmt"

func add(a, b int) int { return a + b }
func mul(a, b int) int { return a * b }

func main() {
 op := add
 fmt.Println("add:", op(3, 4))
}
```

```

 op = mul
 fmt.Println("mul:", op(3, 4))
}

```

Expected:

```

add: 7
mul: 12

```

## Example: closure captures state

Save as `main.go` and `go run .` (with `go mod init example` if needed).

```

package main

import "fmt"

func counter(start int) func() int {
 n := start
 return func() int {
 n++
 return n
 }
}

func main() {
 next := counter(10)
 fmt.Println(next())
 fmt.Println(next())
 fmt.Println(next())
}

```

Expected:

```

11
12
13

```

## Runnable example

Save as `main.go`. Then:

```

go mod init example
go run .

```



```

package main

import "fmt"

type BinaryOp func(int, int) int

func apply(nums []int, fn func(int) int) []int {
 out := make([]int, len(nums))
 for i, n := range nums {
 out[i] = fn(n)
 }
 return out
}

func multiplier(factor int) func(int) int {
 return func(n int) int { return n * factor }
}

type Server struct {
 port int
 timeout int // seconds, kept simple
}

type Option func(*Server)

func WithPort(p int) Option {
 return func(s *Server) { s.port = p }
}

func WithTimeout(sec int) Option {
 return func(s *Server) { s.timeout = sec }
}

func NewServer(opts ...Option) *Server {
 s := &Server{port: 8080, timeout: 30}
 for _, opt := range opts {
 opt(s)
 }
 return s
}

func main() {
 add := func(a, b int) int { return a + b }
 var op BinaryOp = add
 fmt.Println("BinaryOp:", op(2, 3))

 doubled := apply([]int{1, 2, 3}, multiplier(2))
}

```

```
fmt.Println("map-like apply:", doubled)

srv := NewServer(WithPort(9000), WithTimeout(60))
fmt.Printf("server port=%d timeout=%ds\n", srv.port, srv.timeout)

square := func(x int) int { return x * x }(5)
fmt.Println("IIFE square:", square)
}
```

**Expected output:**

```
BinaryOp: 5
map-like apply: [2 4 6]
server port=9000 timeout=60s
IIFE square: 25
```

**What to notice:** Functions are values you can store, pass, and return; the option pattern is just a slice of `func(*Server)` applied at construction.

**Try next:** Add a `WithDefaults` option that resets port and timeout, or a `compose(f, g)` helper that returns `func(int) int` applying both.

# Methods and Receivers

## Overview

Methods are functions with a receiver argument, allowing you to define behavior on types.

## Method Syntax

```
type Rectangle struct {
 Width, Height float64
}

func (r Rectangle) Area() float64 {
 return r.Width * r.Height
}

rect := Rectangle{10, 5}
area := rect.Area() // 50
```

## Value vs Pointer Receivers

### Value Receiver

```
func (r Rectangle) Scale(factor float64) Rectangle {
 return Rectangle{r.Width * factor, r.Height * factor}
}

// Original unchanged
r := Rectangle{10, 5}
scaled := r.Scale(2) // New rectangle
```

## Pointer Receiver

```
func (r *Rectangle) ScaleInPlace(factor float64) {
 r.Width *= factor
 r.Height *= factor
}

// Original modified
r := Rectangle{10, 5}
r.ScaleInPlace(2) // r is now {20, 10}
```

## When to Use Pointer Receivers

Use pointer receiver when: - Method needs to modify the receiver - Receiver is a large struct (avoid copying) - Consistency: if any method needs pointer, use pointer for all

```
type Buffer struct {
 data []byte
}

func (b *Buffer) Write(p []byte) {
 b.data = append(b.data, p...)
}

func (b *Buffer) String() string {
 return string(b.data) // Even though it doesn't modify, use * for consistency
}
```

## Methods on Any Type

```
type MyInt int

func (m MyInt) Double() MyInt {
 return m * 2
}

n := MyInt(5)
n.Double() // 10
```

## Automatic Dereferencing

Go automatically handles \* and & for method calls:

```

r := Rectangle{10, 5}
(&r).ScaleInPlace(2) // Explicit pointer
r.ScaleInPlace(2) // Go handles it automatically

ptr := &Rectangle{10, 5}
(*ptr).Area() // Explicit dereference
ptr.Area() // Go handles it automatically

```

## Embedding and Method Promotion

```

type Animal struct {
 Name string
}

func (a Animal) Speak() string {
 return "..."
}

type Dog struct {
 Animal // Embedded
 Breed string
}

d := Dog{Animal: Animal{Name: "Rex"}, Breed: "Labrador"}
d.Speak() // Promoted method: "..."
d.Name // Promoted field: "Rex"

```

## Method Override

```

func (d Dog) Speak() string {
 return "Woof!"
}

d.Speak() // "Woof!" (Dog's method)
d.Animal.Speak() // "..." (Animal's method)

```

## Generic Methods (Go 1.27+)

A method may declare **its own** type parameters. That is different from a method on a generic type, which already existed (`func (s *Stack[T]) Push(T)`):

```

type Store struct {
 items []any
}

// Method-level type parameter: one Put works for any T.
func (s *Store) Put[T any](v T) {
 s.items = append(s.items, v)
}

func (s *Store) Get[T any](i int) (T, bool) {
 var zero T
 if i < 0 || i >= len(s.items) {
 return zero, false
 }
 v, ok := s.items[i].(T)
 return v, ok
}

```

The standard library uses this in `math/rand/v2`: `(*Rand).N[Int intType](n Int) Int` replaces a family of `IntN` / `Int32N` / `Int64N` methods.

**Limits:** - Interface methods **cannot** declare type parameters. - A generic method **cannot** implement an interface method (the method set is still non-generic). - Prefer a package-level generic function when the operation is not naturally namespaced on a type.

## Summary

Receiver	Use Case
<code>(t T)</code>	Read-only, small types
<code>(t *T)</code>	Modify state, large types

Feature	Description
Auto-deref	Go handles <code>*</code> and <code>&amp;</code>
Embedding	Methods are promoted
Override	Inner type's method shadows embedded
Generic methods (1.27+)	Method declares its own type parameters

## Related Topics

- [Pointers Explained](#)
- [Designing with Interfaces](#)
- [Why Generics Matter](#)

## More examples

### Example: value receiver cannot mutate

Save as `main.go` and `go run .` (with `go mod init example` if needed).

```
package main

import "fmt"

type Point struct{ X, Y int }

func (p Point) Move(dx, dy int) {
 p.X += dx
 p.Y += dy
}

func main() {
 p := Point{1, 2}
 p.Move(10, 10)
 fmt.Printf("after value Move: %+v\n", p)
}
```

Expected:

```
after value Move: {X:1 Y:2}
```

### Example: pointer receiver mutates

Save as `main.go` and `go run .` (with `go mod init example` if needed).

```
package main

import "fmt"

type Point struct{ X, Y int }

func (p *Point) Move(dx, dy int) {
 p.X += dx
 p.Y += dy
}

func main() {
 p := Point{1, 2}
 p.Move(10, 10)
 fmt.Printf("after pointer Move: %+v\n", p)
}
```

```
}
```

Expected:

```
after pointer Move: {X:11 Y:12}
```

## Runnable example

Save as `main.go`. Then:

```
go mod init example
go run .
```

```
package main
```

```
import "fmt"
```

```
type Rectangle struct {
 Width, Height float64
}
```

```
func (r Rectangle) Area() float64 {
 return r.Width * r.Height
}
```

```
// Value receiver: returns a new value; original unchanged.
func (r Rectangle) Scale(factor float64) Rectangle {
 return Rectangle{r.Width * factor, r.Height * factor}
}
```

```
// Pointer receiver: mutates in place.
func (r *Rectangle) ScaleInPlace(factor float64) {
 r.Width *= factor
 r.Height *= factor
}
```

```
type Animal struct{ Name string }
```

```
func (a Animal) Speak() string { return "... " }
```

```
type Dog struct {
 Animal
 Breed string
}
```



```

func (d Dog) Speak() string { return "Woof!" }

func main() {
 r := Rectangle{Width: 10, Height: 5}
 fmt.Println("area:", r.Area())

 scaled := r.Scale(2)
 fmt.Printf("after Scale: orig=%v scaled=%v\n", r, scaled)

 r.ScaleInPlace(2) // Go passes &r automatically
 fmt.Printf("after ScaleInPlace: %v\n", r)

 d := Dog{Animal: Animal{Name: "Rex"}, Breed: "Labrador"}
 fmt.Println("promoted Name:", d.Name)
 fmt.Println("Dog.Speak:", d.Speak())
 fmt.Println("Animal.Speak:", d.Animal.Speak())
}

```

**Expected output:**

```

area: 50
after Scale: orig={10 5} scaled={20 10}
after ScaleInPlace: {20 10}
promoted Name: Rex
Dog.Speak: Woof!
Animal.Speak: ...

```

**What to notice:** Value receivers copy; pointer receivers share the same `Rectangle`. Embedding promotes `Name` and `Speak`, and `Dog.Speak` shadows the embedded method.

**Try next:** Give `Rectangle` a `String()` `string` method and print with `%v` vs `%s`, or add a value-receiver `ScaleInPlace` (wrongly) and observe that the caller's fields no longer change.

# Understanding Memory in Go

## Overview

Go manages memory automatically through its runtime, but understanding stack vs heap allocation helps write more efficient code.

## Stack vs Heap

### Stack

- Fast allocation/deallocation
- Function-local variables
- Fixed size per goroutine (~2KB initial)
- Automatically cleaned up when function returns

### Heap

- Dynamic allocation
- Survives function scope
- Managed by garbage collector
- More expensive than stack

## Escape Analysis

Go's compiler decides where to allocate based on **escape analysis**:

```
func stackAlloc() int {
 x := 42 // Stays on stack
 return x // Value copied
}

func heapAlloc() *int {
 x := 42 // Escapes to heap
 return &x // Pointer returned
}
```

## Check Escape Analysis

```
go build -gcflags="-m" main.go
Shows: "moved to heap" for escaped variables
```

## What Causes Escape

```
// 1. Returning pointer to local
func escape1() *int {
 n := 1
 return &n // Escapes
}

// 2. Storing in interface
func escape2() {
 n := 1
 fmt.Println(n) // Escapes (interface{} argument)
}

// 3. Closure capturing variable
func escape3() func() int {
 n := 1
 return func() int { return n } // Escapes
}

// 4. Size too large for stack
func escape4() {
 _ = make([]byte, 10<<20) // 10MB, escapes
}
```

## Memory Layout

### Value Types

```
type Point struct { X, Y int }
p := Point{1, 2} // 16 bytes inline
```

## Reference Types

```
s := make([]int, 10) // Slice header on stack, data on heap
m := make(map[string]int) // Map structure on heap
```

## Best Practices

### Reduce Allocations

```
// Bad: allocates each iteration
for i := 0; i < n; i++ {
 data := make([]byte, 1024)
 process(data)
}

// Good: reuse allocation
data := make([]byte, 1024)
for i := 0; i < n; i++ {
 process(data)
}
```

### Preallocate Slices

```
// Bad: multiple reallocations
var result []int
for _, v := range data {
 result = append(result, v)
}

// Good: preallocate
result := make([]int, 0, len(data))
for _, v := range data {
 result = append(result, v)
}
```

### Use Value Semantics When Possible

```
// May allocate
func process(p *Point) { }

// Stays on stack
func process(p Point) { }
```

## Summary

Location	Characteristics
Stack	Fast, automatic, limited size
Heap	Dynamic, GC-managed, survives scope

Causes Escape	Example
Return pointer	<code>return &amp;x</code>
Interface	<code>fmt.Println(x)</code>
Closure	<code>func() { use(x) }</code>
Large size	<code>make([]byte, 1MB)</code>

## Related Topics

- [Pointers Explained](#)
- [Garbage Collection](#)

## More examples

### Example: value stays local

Save as `main.go` and `go run .` (with `go mod init example` if needed).

```
package main

import "fmt"

func double(x int) int {
 return x * 2
}

func main() {
 n := 21
 fmt.Println("double:", double(n))
}
```

```
 fmt.Println("original:", n)
}
```

Expected:

```
double: 42
original: 21
```

## Example: interface argument often forces heap

Save as `main.go` and `go run .` (with `go mod init example` if needed).

```
package main

import "fmt"

func show(v any) {
 fmt.Printf("value=%v type=%T\n", v, v)
}

func main() {
 x := 99
 show(x) // boxing into any is a common escape trigger
 show("ok")
}
```

Expected:

```
value=99 type=int
value=ok type=string
```

## Runnable example

Save as `main.go`. Then:

```
go mod init example
go run .
Optional: see escape decisions
go build -gcflags="-m" .

package main

import "fmt"
```

```

type Point struct{ X, Y int }

// Returns a value: caller gets a copy; x can stay on the stack.
func stackAlloc() int {
 x := 42
 return x
}

// Returns a pointer: x must live past the call, so it escapes to the heap.
func heapAlloc() *int {
 x := 42
 return &x
}

// Reuse one buffer instead of allocating per iteration.
func processReuse(n int) int {
 buf := make([]byte, 64)
 sum := 0
 for i := 0; i < n; i++ {
 buf[0] = byte(i)
 sum += int(buf[0])
 }
 return sum
}

func main() {
 fmt.Println("stackAlloc:", stackAlloc())

 p := heapAlloc()
 fmt.Println("heapAlloc value:", *p)
 // Do not print the address itself - it changes every run.

 // Slice header can be on the stack; backing array is typically on the heap.
 pts := make([]Point, 0, 3)
 pts = append(pts, Point{1, 2}, Point{3, 4})
 fmt.Printf("points len=%d cap=%d first=%v\n", len(pts), cap(pts), pts[0])

 fmt.Println("processReuse:", processReuse(5))
}

```

### Expected output:

```

stackAlloc: 42
heapAlloc value: 42
points len=2 cap=3 first={1 2}
processReuse: 10

```

**What to notice:** Returning `*int` forces heap allocation; reusing one slice buffer avoids per-loop allocs. Run with `-gcflags="-m"` to confirm which locals the compiler marks as escaping.

**Try next:** Change `heapAlloc` to return `int` by value and re-run escape analysis; preallocate `pts` with `make([]Point, 0, 100)` and watch capacity stay put under many `appends`.



# Pointers Explained

## Overview

Pointers hold memory addresses. In Go, they're simple and safe—no pointer arithmetic allowed.

## Pointer Basics

```
x := 42
p := &x // p is *int, holds address of x
fmt.Println(*p) // 42 (dereference)
*p = 100 // Modify x through pointer
fmt.Println(x) // 100
```

## Declaration

```
var p *int // nil pointer
p = &x // Point to x

// Classic new (zero value)
p := new(int) // *int to 0

// Expression-based new (Go 1.26+)
p := new(42) // *int to 42
```

Stack/heap view (simplified)

```
x: 42 p: 0xABC
| |
+----- addr ----+
|
[42]
```

# Why Use Pointers

## 1. Modify Variables in Functions

```
func increment(n *int) {
 *n++
}

x := 1
increment(&x)
fmt.Println(x) // 2
```

## 2. Avoid Copying Large Structs

```
type BigStruct struct {
 Data [1000]int
}

// Bad: copies entire struct
func process(b BigStruct) { }

// Good: passes pointer (8 bytes)
func process(b *BigStruct) { }
```

## 3. Share Data

```
type Config struct {
 Debug bool
}

func loadConfig() *Config {
 return &Config{Debug: true}
}

cfg := loadConfig() // All code shares same Config
```

## nil Pointers

```
var p *int // nil

if p != nil {
 fmt.Println(*p) // Safe
}

// Dereferencing nil panics!
// *p = 1 // panic: runtime error
```

Safe pattern for optional fields in APIs:

```
type UpdateUserRequest struct {
 Name *string `json:"name,omitempty"`
 Email *string `json:"email,omitempty"`
}
```

## Pointer to Pointer

```
x := 42
p := &x
pp := &p // **int

**pp = 100
fmt.Println(x) // 100
```

## Pointers and Slices/Maps

Slices and maps are already reference types:

```
func modify(s []int) {
 s[0] = 100 // Modifies original
}

nums := []int{1, 2, 3}
modify(nums)
fmt.Println(nums[0]) // 100

// But reassigning needs pointer
func replace(s *[]int) {
 *s = []int{4, 5, 6}
}
```

## Common Patterns

### Return Pointer for “No Result”

```
func find(id int) *User {
 if found {
 return &user
 }
 return nil // Not found
}

if user := find(1); user != nil {
 // Use user
}
```

### Constructor Pattern

```
func NewUser(name string) *User {
 return &User{
 Name: name,
 CreatedAt: time.Now(),
 }
}
```

## Summary

Operation	Syntax
Get address	<code>&amp;x</code>
Dereference	<code>*p</code>
Allocate	<code>new(T)</code>
Check nil	<code>p != nil</code>

## Related Topics

- [Methods and Receivers](#)
- [Mutability and Data Sharing](#)

## More examples

### Example: address and dereference

Save as `main.go` and `go run .` (with `go mod init example` if needed).

```
package main

import "fmt"

func main() {
 x := 10
 p := &x
 fmt.Println("x:", x)
 fmt.Println("*p:", *p)
 *p = 20
 fmt.Println("x after *p=20:", x)
}
```

Expected:

```
x: 10
*p: 10
x after *p=20: 20
```

### Example: nil pointer check

Save as `main.go` and `go run .` (with `go mod init example` if needed).

```
package main

import "fmt"

func safeLen(p *string) int {
 if p == nil {
 return 0
 }
 return len(*p)
}

func main() {
 var missing *string
 s := "gopher"
 fmt.Println("nil:", safeLen(missing))
 fmt.Println("set:", safeLen(&s))
}
```

Expected:

```
nil: 0
set: 6
```

## Runnable example

Save as main.go. Then:

```
go mod init example
go run .
```

```
package main

import "fmt"

func increment(n *int) {
 *n++
}

func modifySlice(s []int) {
 if len(s) > 0 {
 s[0] = 100 // mutates shared backing array
 }
}

func replaceSlice(s *[]int) {
 *s = []int{4, 5, 6} // replaces the caller's slice header
}

func main() {
 x := 42
 p := &x
 fmt.Println("via pointer:", *p)
 *p = 100
 fmt.Println("after *p=100, x:", x)

 increment(&x)
 fmt.Println("after increment:", x)

 var nilP *int
 fmt.Println("nil pointer is nil:", nilP == nil)
 // *nilP would panic - always check first.

 // new allocates a zero value and returns a pointer.
```

```

z := new(int)
fmt.Println("new(int) value:", *z)
*z = 7
fmt.Println("new(int) after set:", *z)

nums := []int{1, 2, 3}
modifySlice(nums)
fmt.Println("after modifySlice:", nums)

replaceSlice(&nums)
fmt.Println("after replaceSlice:", nums)
}

```

### Expected output:

```

via pointer: 42
after *p=100, x: 100
after increment: 101
nil pointer is nil: true
new(int) value: 0
new(int) after set: 7
after modifySlice: [100 2 3]
after replaceSlice: [4 5 6]

```

**What to notice:** `*p` reads/writes the same storage as `x`. Slice element writes share the backing array without a pointer; replacing the slice header itself needs `*[]int`.

**Try next:** Pass a large struct by value vs pointer and compare with `unsafe.Sizeof` (or just reason about copy cost); write a safe helper that returns zero when given a nil `*int`.

# Mutability and Data Sharing

## Overview

Understanding when data is copied vs shared is crucial for avoiding bugs and writing efficient Go code.

## Value vs Reference Semantics

### Value Types (Copied)

```
// Integers, floats, bools, strings, arrays, structs
a := 1
b := a
b = 2
fmt.Println(a) // 1 (unchanged)

arr := [3]int{1, 2, 3}
arr2 := arr
arr2[0] = 100
fmt.Println(arr[0]) // 1 (unchanged)
```

### Reference Types (Shared)

```
// Slices, maps, channels, pointers
s := []int{1, 2, 3}
s2 := s
s2[0] = 100
fmt.Println(s[0]) // 100 (changed!)

m := map[string]int{"a": 1}
m2 := m
m2["a"] = 100
fmt.Println(m["a"]) // 100 (changed!)
```



## Controlling Mutability

### Immutable by Design

```
// Return new instead of modifying
func addItem(items []string, item string) []string {
 return append(items, item) // May return new slice
}
```

### Defensive Copy

```
func process(s []int) {
 local := make([]int, len(s))
 copy(local, s) // Safe to modify local
}
```

### Deep Copy

```
type User struct {
 Name string
 Friends []string
}

func (u User) Clone() User {
 friends := make([]string, len(u.Friends))
 copy(friends, u.Friends)
 return User{Name: u.Name, Friends: friends}
}
```

## Function Parameters

```
// Value: caller's data safe
func process(u User) {
 u.Name = "modified" // Doesn't affect caller
}

// Pointer: can modify caller's data
func process(u *User) {
 u.Name = "modified" // Affects caller
}
```

## Struct Field Mutability

```
type Counter struct {
 count int
}

// Value receiver: cannot modify
func (c Counter) Increment() {
 c.count++ // Modifies copy, original unchanged
}

// Pointer receiver: modifies original
func (c *Counter) Increment() {
 c.count++ // Original modified
}
```

## Slice Gotchas

```
// Append may or may not share backing array
s := []int{1, 2, 3}
s2 := s[:2] // Shares backing array
s2 = append(s2, 4) // Might overwrite s[2]!

// Safer: force new allocation
s2 := append([]int(nil), s[:2]...)
```

## Summary

Type	Behavior	Copy When...
int, bool, string	Value	Always copied
struct	Value	Always copied
array	Value	Always copied
slice	Reference	Use <code>copy()</code>
map	Reference	Rebuild manually
pointer	Reference	-

## Related Topics

- [Pointers Explained](#)
- [Concurrency Design Guidelines](#)

## More examples

### Example: slice header shares backing array

Save as `main.go` and `go run .` (with `go mod init example` if needed).

```
package main

import "fmt"

func main() {
 a := []int{1, 2, 3}
 b := a
 b[0] = 99
 fmt.Println("a:", a)
 fmt.Println("b:", b)
}
```

Expected:

```
a: [99 2 3]
b: [99 2 3]
```

### Example: copy to isolate mutation

Save as `main.go` and `go run .` (with `go mod init example` if needed).

```
package main

import "fmt"

func main() {
 a := []int{1, 2, 3}
 b := make([]int, len(a))
 copy(b, a)
 b[0] = 99
 fmt.Println("a:", a)
 fmt.Println("b:", b)
}
```

Expected:

```
a: [1 2 3]
b: [99 2 3]
```

## Runnable example

Save as `main.go`. Then:

```
go mod init example
go run .

package main

import "fmt"

type User struct {
 Name string
 Friends []string
}

func (u User) Clone() User {
 friends := make([]string, len(u.Friends))
 copy(friends, u.Friends)
 return User{Name: u.Name, Friends: friends}
}

type Counter struct{ count int }

func (c Counter) IncValue() {
 c.count++ // mutates a copy only
}

func (c *Counter) IncPointer() {
 c.count++
}

func main() {
 // Value types: assignment copies.
 a := 1
 b := a
 b = 2
 fmt.Println("ints a,b:", a, b)

 arr := [3]int{1, 2, 3}
 arr2 := arr
 arr2[0] = 99
 fmt.Println("arrays arr,arr2:", arr, arr2)

 // Reference-like headers: slice assignment shares data.
 s := []int{1, 2, 3}
```

```

s2 := s
s2[0] = 100
fmt.Println("shared slice s:", s)

// Defensive copy of the backing array.
safe := make([]int, len(s))
copy(safe, s)
safe[0] = 1
fmt.Println("after defensive copy, s still:", s, "safe:", safe)

// append may or may not reallocate - force a fresh backing array.
base := []int{1, 2, 3}
alias := base[:2]
alias = append(alias, 9) // often overwrites base[2]
fmt.Println("append into shared capacity base:", base)

fresh := append([]int(nil), base[:2]...)
fresh = append(fresh, 9)
fmt.Println("append into fresh copy base:", base, "fresh:", fresh)

u := User{Name: "Ada", Friends: []string{"Bob"}}
u2 := u.Clone()
u2.Friends[0] = "Carol"
fmt.Println("clone independent:", u.Friends, u2.Friends)

c := Counter{}
c.IncValue()
fmt.Println("after value Inc:", c.count)
c.IncPointer()
fmt.Println("after pointer Inc:", c.count)
}

```

### Expected output:

```

ints a,b: 1 2
arrays arr,arr2: [1 2 3] [99 2 3]
shared slice s: [100 2 3]
after defensive copy, s still: [100 2 3] safe: [1 2 3]
append into shared capacity base: [1 2 9]
append into fresh copy base: [1 2 9] fresh: [1 2 9]
clone independent: [Bob] [Carol]
after value Inc: 0
after pointer Inc: 1

```

**What to notice:** Arrays and structs copy deeply for their own fields, but nested slices still share unless you copy. Value receivers never update the caller's **Counter**.

**Try next:** Map the same experiment with `map[string]int` (assignment shares); fix the shared-

append case by capping first: `base[:2:2]`.

# Garbage Collection

## Overview

Go uses automatic garbage collection (GC) to manage memory. Understanding how it works helps you write GC-friendly code.

## Modern GC Architecture

### Concurrent Mark and Sweep

Go's GC is concurrent and non-moving. At a high level: 1. **Mark Setup:** (STW) Prepare for marking. 2. **Marking:** (Concurrent) Mark reachable objects. 3. **Mark Termination:** (STW) Finalize marking. 4. **Sweeping:** (Concurrent) Reclaim memory.

```
flow:
] -> Mark[
 Mark -> STW2[
 STW2 -> Sweep[
```

### Go 1.26: Green Tea GC (GTGC)

Go 1.26 switched the default collector to **Green Tea GC (GTGC)** for lower pause times and better cache locality in pointer-heavy workloads.

```
flow:
] -> LObj1[(Random Heap Jumps)
 LObj1 -> LObj2[(Cache Miss)
] -> GSpan1[(Span-Aware Local Scan)
 GSpan1 -> GSpan2[(Vectorized Bitmaps)
```

- **Span-Aware Locality:** Groups pointer scanning by physical memory spans to maximize CPU L1/L2 cache hits.
- **Lower CPU Overhead:** Reduces write-barrier costs in concurrent mutator workers.
- **Opt-Out:** If needed for debugging or legacy performance comparisons:

```
GOEXPERIMENT=nogreenteagc go test ./...
```

## Go 1.27: size-specialized small allocations

The compiler emits calls to size-specialized malloc for some objects under 80 bytes, cutting those allocation costs by up to ~30% (~1% in allocation-heavy programs; ~60 KB extra binary). Opt out with `GOEXPERIMENT=nosizespecializedmalloc` if you hit a regression (expected to be removed in Go 1.28). See [Memory Allocator](#).

## GOGC and Memory Limits

Think of GOGC and GOMEMLIMIT as two controls:

- GOGC controls *when* to trigger based on heap growth.
- GOMEMLIMIT puts a soft cap on total Go-managed memory.

Heap growth trigger	Memory budget trigger
<code>live heap * (1 + GOGC/100)</code>	<code>total Go memory near GOMEMLIMIT</code>

## Tuning with GOGC

```
Default: GC when heap doubles (100% growth)
GOGC=100 ./myapp

More aggressive: GC at 50% growth (uses less memory)
GOGC=50 ./myapp

Less frequent: GC at 200% growth (faster, more memory)
GOGC=200 ./myapp

Disable GC (not recommended)
GOGC=off ./myapp
```

## Tuning with GOMEMLIMIT

```
Keep process around 2 GiB Go-managed memory
GOMEMLIMIT=2GiB ./myapp
```

Programmatic control:

```
import "runtime/debug"

func init() {
```



```

 debug.SetMemoryLimit(2 << 30) // 2 GiB
}

```

## Memory Stats

```

var m runtime.MemStats
runtime.ReadMemStats(&m)

fmt.Printf("Alloc: %d MB\n", m.Alloc/1024/1024)
fmt.Printf("Total Alloc: %d MB\n", m.TotalAlloc/1024/1024)
fmt.Printf("Heap Objects: %d\n", m.HeapObjects)
fmt.Printf("GC Cycles: %d\n", m.NumGC)

```

A low-overhead option for production telemetry:

```

import "runtime/metrics"

samples := []metrics.Sample{
 {Name: "/gc/heap/live:bytes"},
 {Name: "/gc/heap/goal:bytes"},
}
metrics.Read(samples)

```

## Reducing GC Pressure

### 1. Reduce Allocations

```

// Bad: allocates each call
func getBuffer() []byte {
 return make([]byte, 1024)
}

// Good: reuse with sync.Pool
var bufPool = sync.Pool{
 New: func() any { return make([]byte, 1024) },
}

func getBuffer() []byte {
 return bufPool.Get().([]byte)
}

func putBuffer(b []byte) {

```

```
 bufPool.Put(b)
}
```

## 2. Preallocate

```
result := make([]int, 0, expectedSize)
```

## 3. Use Value Types

```
// More allocations
type Points []*Point

// Fewer allocations
type Points []Point
```

## 4. Avoid String Concatenation in Loops

```
// Bad: allocates each iteration
s := ""
for _, part := range parts {
 s += part
}

// Good: single allocation
var b strings.Builder
for _, part := range parts {
 b.WriteString(part)
}
s := b.String()
```

## Profiling

```
CPU profile
go test -cpuprofile=cpu.out
go tool pprof cpu.out

Memory profile
go test -memprofile=mem.out
go tool pprof mem.out
```

```
View allocations
go tool pprof -alloc_space mem.out
```

## Summary

Optimization	Technique
Reuse memory	<code>sync.Pool</code>
Preallocate	<code>make([]T, 0, cap)</code>
Values vs pointers	Use values for small types
String building	<code>strings.Builder</code>

## Related Topics

- [Understanding Memory in Go](#)
- [Performance Tuning](#)

## More examples

### Example: preallocate slice capacity

Save as `main.go` and `go run .` (with `go mod init example` if needed).

```
package main

import "fmt"

func main() {
 // Growing without capacity reallocates more often.
 var grow []int
 for i := 0; i < 5; i++ {
 grow = append(grow, i)
 }

 // Preallocated capacity reduces realloc churn.
 ready := make([]int, 0, 5)
 for i := 0; i < 5; i++ {
 ready = append(ready, i)
 }
 fmt.Println("grow:", grow, "len", len(grow), "cap", cap(grow))
 fmt.Println("ready:", ready, "len", len(ready), "cap", cap(ready))
}
```

Expected:

```
grow: [0 1 2 3 4] len 5 cap 8
ready: [0 1 2 3 4] len 5 cap 5
```

## Example: strings.Builder vs naive concat

Save as `main.go` and `go run .` (with `go mod init example` if needed).

```
package main

import (
 "fmt"
 "strings"
)

func main() {
 var b strings.Builder
 for i := 0; i < 4; i++ {
 fmt.Fprintf(&b, "%d,", i)
 }
 fmt.Println("builder:", b.String())

 // Educational contrast: repeated + creates more temporary strings.
 s := ""
 for i := 0; i < 4; i++ {
 s += fmt.Sprintf("%d,", i)
 }
 fmt.Println("concat:", s)
}
```

Expected:

```
builder: 0,1,2,3,
concat: 0,1,2,3,
```

## Runnable example

Save as `main.go`. Then:

```
go mod init example
go run .
```

```

package main

import (
 "fmt"
 "runtime"
)

func readStats() runtime.MemStats {
 var m runtime.MemStats
 runtime.ReadMemStats(&m)
 return m
}

func printStats(label string, m runtime.MemStats) {
 // HeapAlloc / HeapObjects are more stable educational signals than Alloc alone.
 fmt.Printf("%s: HeapAlloc=%d bytes HeapObjects=%d NumGC=%d\n",
 label, m.HeapAlloc, m.HeapObjects, m.NumGC)
}

func main() {
 runtime.GC()
 before := readStats()
 printStats("before alloc", before)

 // Keep a live reference so the GC cannot free these yet.
 keep := make([][]byte, 0, 100)
 for i := 0; i < 100; i++ {
 keep = append(keep, make([]byte, 64*1024)) // 64 KiB each
 }
 afterAlloc := readStats()
 printStats("after alloc", afterAlloc)

 // Drop references, then force a collection.
 keep = nil
 runtime.GC()
 afterGC := readStats()
 printStats("after GC", afterGC)

 fmt.Printf("objects grew by ~%d then GC cycles advanced by %d\n",
 int64(afterAlloc.HeapObjects)-int64(before.HeapObjects),
 afterGC.NumGC-before.NumGC)
}

```

**Expected output:** (numbers vary by platform and Go version; shape is stable)

```
before alloc: HeapAlloc=... bytes HeapObjects=... NumGC=...
after alloc: HeapAlloc=... bytes HeapObjects=... NumGC=...
after GC: HeapAlloc=... bytes HeapObjects=... NumGC=...
objects grew by ~100 then GC cycles advanced by 1
```

Typically `HeapAlloc` and `HeapObjects` rise after the loop, then fall after `keep = nil + runtime.GC()`, while `NumGC` increases.

**What to notice:** Live pointers keep heap memory reachable; clearing the root and forcing GC lets the collector reclaim it. Absolute byte counts are not portable — look at direction of change.

**Try next:** Run with `GOGC=50` vs `GOGC=200` and compare how soon `NumGC` climbs during a longer allocation loop; try `sync.Pool` to reuse the 64 KiB buffers.

## **Stack Mechanics & Optimizations**

# Understanding Memory: Stack vs. Heap in Go

To write high-performance Go, you must understand where your variables live. Go abstracts memory management, but it doesn't eliminate the physics of hardware.

## The Two Worlds

### 1. The Stack

- **What it is:** A pre-allocated, contiguous block of memory for each goroutine (starts at 2KB).
- **Allocation:** Instant (move a pointer).
- **Deallocation:** Instant (move a pointer back).
- **GC Cost: Zero.** The Garbage Collector (GC) does not scan the stack.
- **Cache Locality:** Excellent.

### 2. The Heap

- **What it is:** A global pool of memory for dynamic allocations.
- **Allocation:** Slow (search for free block, possibly lock).
- **Deallocation:** Garbage Collected (expensive scan).
- **GC Cost:** High. Pointers on the heap must be traced.
- **Cache Locality:** Poor (scattered).

## Go's Optimization Goal

**Keep everything on the stack.**

If the compiler can prove a variable's life cycle is contained within a function (it doesn't "escape"), it allocates it on the stack.

### Stack Growth (Contiguous Stacks)

Goroutines start with 2KB stacks. If a function call needs more space, the runtime: 1. Allocates a larger stack (e.g., 4KB). 2. **Copies** everything from old stack to new stack. 3. Updates pointers to the new stack. 4. Frees the old stack.

*Note: This "copying" is why pointers to stack variables are safe only if they don't escape. If they escaped to the heap, moving the stack would invalidate external pointers.*



## 2026: The “Mid-Stack” Inlining

Recent Go versions have become aggressive about inlining function calls mid-stack. \* If `UserFunc` calls `AllocFunc`, and `AllocFunc` is small, the compiler copies `AllocFunc`’s body into `UserFunc`. \* This removes the function call overhead *and* allows variables that might have escaped (due to being return values) to stay on the stack.

### Visualization

Variable Scenario	Destination	Why?
<code>x := 42</code>	<b>Stack</b>	Never leaves function references.
<code>y := &amp;x</code> (used locally)	<b>Stack</b>	Address taken, but scope is local.
<code>return &amp;x</code>	<b>Heap</b>	Escapes to caller; stack frame dies on return.
<code>fmt.Println(x)</code>	<b>Heap</b> (usually)	<code>fmt.Println</code> takes <code>interface{}</code> , which often causes escape.
<code>make([]byte, 1024)</code>	<b>Stack</b>	Size known at compile time, fits in stack.
<code>make([]byte, n)</code>	<b>Heap</b>	Size unknown at compile time.

### Practical Rule

Don’t fear the Heap, but respect the Stack. If you are writing a hot loop (a game loop, a high-frequency trading handler), ensure your temporary variables do not escape to the heap.

Use detailed analysis tools to verify this (covered in next chapter).

### More examples

#### Example: stack-friendly local work

Save as `main.go` and go `run .` (with `go mod init example` if needed).

```
package main

import "fmt"

func fold(nums []int) int {
 // Locals and the slice header stay short-lived; good stack candidates.
```

```

 total := 0
 for _, n := range nums {
 total += n
 }
 return total
}

func main() {
 data := [4]int{10, 20, 30, 40} // fixed array: stack-friendly storage
 fmt.Println("fold:", fold(data[:]))
 fmt.Println("fold empty:", fold(nil))
}

```

Expected:

```

fold: 100
fold empty: 0

```

### Example: escape when a pointer outlives the frame

Save as main.go and go run . (with go mod init example if needed).

```

package main

import "fmt"

type Box struct{ Label string }

func newBox(label string) *Box {
 b := Box{Label: label} // typically moves to heap: returned address
 return &b
}

func localBox() string {
 b := Box{Label: "stack-ish"}
 p := &b // address taken but still local
 return p.Label
}

func main() {
 fmt.Println("local:", localBox())
 b := newBox("heap-ish")
 fmt.Println("returned:", b.Label)
}

```

Expected:

```
local: stack-ish
returned: heap-ish
```

## Runnable example

Save as `main.go`. Then:

```
go mod init example
go run .
Escape report (educational; messages depend on compiler version):
go build -gcflags="-m -m" .

package main

import "fmt"

type Point struct{ X, Y int }

// Local use only - good stack candidate.
func useLocally() int {
 p := Point{X: 1, Y: 2}
 q := &p // address taken, but still only used here
 return q.X + q.Y
}

// Returned pointer must outlive this frame → heap.
func returnPointer() *Point {
 p := Point{X: 3, Y: 4}
 return &p
}

// Fixed-size array often stays stack-friendly; dynamic make usually does not.
func fixedBufSum() int {
 var buf [16]byte
 for i := range buf {
 buf[i] = byte(i)
 }
 sum := 0
 for _, b := range buf {
 sum += int(b)
 }
 return sum
}
```

```

func dynamicBufSum(n int) int {
 buf := make([]byte, n)
 for i := range buf {
 buf[i] = byte(i)
 }
 sum := 0
 for _, b := range buf {
 sum += int(b)
 }
 return sum
}

func main() {
 fmt.Println("local pointer use:", useLocally())
 hp := returnPointer()
 fmt.Printf("heap point: {%d %d}\n", hp.X, hp.Y)
 fmt.Println("fixedBufSum:", fixedBufSum())
 fmt.Println("dynamicBufSum:", dynamicBufSum(16))
}

```

#### Expected output:

```

local pointer use: 3
heap point: {3 4}
fixedBufSum: 120
dynamicBufSum: 120

```

**What to notice:** Taking an address does not automatically mean “heap” — escaping past the function does. Fixed-size `[16]byte` and `make([]byte, n)` can behave differently under escape analysis even when they compute the same sum.

**Try next:** Re-run with `go build -gcflags="-m"` and find which locals “moved to heap”; inline a tiny helper into `main` and see if the compiler still reports an escape for a returned pointer.

## **Escape Analysis & Alignment**

# Escape Analysis & Memory Alignment

Memory optimization usually comes down to two questions: 1. Did it hit the heap when it didn't need to? (Excessive allocation) 2. Is it wasting space? (Poor alignment)

## Part 1: Escape Analysis

“Escape Analysis” is the compiler phase that decides Stack vs. Heap.

### Asking the Compiler

You don't need to guess. Ask the compiler what it decided:

```
go build -gcflags="-m" main.go
```

**Output Interpretation:** \* can inline main: Good. \* x does not escape: Great (Stack). \* moved to heap: x: Bad (Heap).

### Common Escape Triggers

1. **Returning Pointers:** `func New() *T { return &T{} } -> Escapes.`
2. **Interfaces:** `func Log(v interface{})`. The concrete type inside the interface box often escapes.
3. **Closures:** Variables captured by a closure *might* escape if the closure outlives the function.
4. **Unknown Size:** `make([]int, n)` always escapes if `n` is dynamic.

### The “Mid-Stack” Trick

If you need a large buffer inside a hot loop, allocate it *once* outside the loop or reuse a `sync.Pool`.  
\* **Bad:** `for { b := make([]byte, 1024); process(b) }` (Trash on heap every loop) \* **Better:** `b := make([]byte, 1024); for { process(b) }` (One alloc) \* **Best (Stack):** `b := [1024]byte{}; for { process(b[:]) }` (Stack alloc if small enough)

## Part 2: Memory Alignment (Padding)

CPU reads memory in words (e.g., 64-bit / 8 bytes). If a field doesn't align with a word boundary, the compiler adds **Padding** (wasted bytes).

### The Struct Layout Game

Consider this struct:

```
type BadStruct struct {
 Flag bool // 1 byte
 Counter int64 // 8 bytes
 Active bool // 1 byte
}
```

**Total Size:** \* bool (1) + padding (7) -> to align int64 \* int64 (8) \* bool (1) + padding (7) -> to align struct size to 8 \* **Total:** 24 bytes

**Reshuffled:**

```
type GoodStruct struct {
 Counter int64 // 8 bytes (0-7)
 Flag bool // 1 byte (8)
 Active bool // 1 byte (9)
 // padding (6) -> to align struct size to multiple of 8 (16)
}
```

**Total:** 16 bytes. **33% savings just by reordering fields.**

### Tools used in 2026

Do not optimize manually unless you are bored. Use tools.

- **fieldalignment:** `bash go install golang.org/x/tools/go/analysis/passes/fieldalignment/cmd/fieldalignment ./...` It will report structs that use too much memory and suggest a fix.
- **betteralign:** A more modern wrapper that can automatically apply changes (-apply).

### When does this matter?

- **Single struct:** Who cares? 8 bytes is nothing.
- **Slice of 100 million structs:** 8 bytes \* 100M = 800MB of RAM wasted. This triggers GC more often, costs money on cloud bills, and slows down cache.

**Rule:** Optimize alignment for “Data Types” (things you store in DB/Arrays). Ignore it for “Service Types” (singletons, handlers).

## More examples

### Example: measure padding with `unsafe.Sizeof`

Save as `main.go` and `go run .` (with `go mod init example` if needed).

```
package main

import (
 "fmt"
 "unsafe"
)

type Wide struct {
 A bool
 B int64
 C bool
 D int32
}

type Tight struct {
 B int64
 D int32
 A bool
 C bool
}

func main() {
 fmt.Println("Wide:", unsafe.Sizeof(Wide{}))
 fmt.Println("Tight:", unsafe.Sizeof(Tight{}))
 fmt.Println("saved bytes:", int(unsafe.Sizeof(Wide{})-unsafe.Sizeof(Tight{})))
}
```

Expected (64-bit; illustrative):

```
Wide: 24
Tight: 16
saved bytes: 8
```

### Example: reuse buffer vs allocate every loop

Save as `main.go` and `go run .` (with `go mod init example` if needed).

```
package main

import "fmt"
```



```

func process(b []byte) int {
 sum := 0
 for _, x := range b {
 sum += int(x)
 }
 return sum
}

func main() {
 // Better: one allocation reused (or stack array as view).
 var stack [8]byte
 buf := stack[:]
 total := 0
 for i := 0; i < 3; i++ {
 for j := range buf {
 buf[j] = byte(i + j)
 }
 total += process(buf)
 }
 fmt.Println("reused total:", total)

 // Contrasting pattern (educational): allocate each iteration.
 total = 0
 for i := 0; i < 3; i++ {
 b := make([]byte, 8)
 for j := range b {
 b[j] = byte(i + j)
 }
 total += process(b)
 }
 fmt.Println("alloc-each total:", total)
}

```

Expected:

```

reused total: 108
alloc-each total: 108

```

## Runnable example

Save as `main.go`. Then:

```

go mod init example
go run .
go build -gcflags="-m" .

```

```

package main

import (
 "fmt"
 "unsafe"
)

// Bad order: bools separated by int64 force padding.
type BadStruct struct {
 Flag bool
 Counter int64
 Active bool
}

// Better order: large fields first, bools packed together.
type GoodStruct struct {
 Counter int64
 Flag bool
 Active bool
}

// Escape triggers: return pointer, interface boxing, open-ended make.
func newCounter() *int {
 n := 1
 return &n // escapes
}

func boxAndPrint(v any) {
 // Interface conversion often forces heap allocation of the concrete value.
 _ = fmt.Sprintf("%v", v)
}

func main() {
 fmt.Println("sizeof BadStruct:", unsafe.Sizeof(BadStruct{}))
 fmt.Println("sizeof GoodStruct:", unsafe.Sizeof(GoodStruct{}))

 // Live use of escaped values so the compiler cannot drop them entirely.
 c := newCounter()
 fmt.Println("escaped counter:", *c)

 x := 42
 boxAndPrint(x)

 // Unknown size → typically heap.
 n := 32
 buf := make([]byte, n)
 buf[0] = 7

```

```

 fmt.Println("dynamic buffer first byte:", buf[0])

 // Stack-friendly pattern: fixed array reused as a slice view.
 var stackBuf [32]byte
 view := stackBuf[:]
 view[0] = 9
 fmt.Println("stack-ish view first byte:", view[0])
}

```

**Expected output:** (sizes are for 64-bit Go; 32-bit differs)

```

sizeof BadStruct: 24
sizeof GoodStruct: 16
escaped counter: 1
dynamic buffer first byte: 7
stack-ish view first byte: 9

```

**What to notice:** Field order changes padding and total size without changing semantics. Returning `&n` and boxing into `any` are classic escape triggers; a fixed `[N]byte` reused as `[:]` is a common stack-friendly hot-path pattern.

**Try next:** Reorder another struct (`byte`, `int64`, `byte`, `int32`) and recompute sizes; run `-gcflags="-m"` and match each “moved to heap” line to a trigger above.

# Generics

# Designing with Interfaces

## Overview

Go interfaces enable polymorphism through implicit satisfaction—types implement interfaces by having matching methods, without explicit declarations.

## Interface Basics

```
type Reader interface {
 Read(p []byte) (n int, err error)
}

// Any type with Read method satisfies Reader
type File struct { /* ... */ }
func (f *File) Read(p []byte) (int, error) { /* ... */ }

type Buffer struct { /* ... */ }
func (b *Buffer) Read(p []byte) (int, error) { /* ... */ }
```

## Implicit Satisfaction

```
type Stringer interface {
 String() string
}

type Person struct {
 Name string
}

// Person implicitly implements Stringer
func (p Person) String() string {
 return p.Name
}

var s Stringer = Person{Name: "Alice"} // Works!
```

## The Empty Interface

```
interface{} // Matches any type
any // Alias (Go 1.18+)

func print(v any) {
 fmt.Println(v)
}

print(42)
print("hello")
print(true)
```

## Interface Values

An interface value holds a (type, value) pair:

```
var r io.Reader
r = os.Stdin // (type: *os.File, value: stdin)
r = &bytes.Buffer{} // (type: *bytes.Buffer, value: buf)
```

### nil Interface vs nil Value

```
var r io.Reader // nil interface (no type, no value)
var f *os.File // nil pointer

r = f // Interface holds (*os.File, nil)
r == nil // false! Has type, value is nil
```

## Common Patterns

### Accept Interface, Return Concrete

```
// Accept interface
func Process(r io.Reader) error { }

// Return concrete type
func NewBuffer() *bytes.Buffer {
 return &bytes.Buffer{}
}
```

## Small Interfaces

```
type Reader interface {
 Read([]byte) (int, error)
}

type Writer interface {
 Write([]byte) (int, error)
}

type ReadWriter interface {
 Reader
 Writer
}
```

## Assert Interface Compliance

```
var _ io.Reader = (*MyReader)(nil) // Compile-time check
```

## Summary

Concept	Description
Implicit	No <code>implements</code> keyword
<code>any/interface{}</code>	Matches all types
Small interfaces	Prefer 1-3 methods
Accept interface	Flexible function parameters

## Related Topics

- [Interface Best Practices](#)
- [Type Assertions](#)

## More examples

### Example: implicit satisfaction

Save as `main.go` and `go run .` (with `go mod init example` if needed).

```

package main

import "fmt"

type Speaker interface {
 Speak() string
}

type Dog struct{ Name string }

func (d Dog) Speak() string { return d.Name + " says woof" }

func greet(s Speaker) {
 fmt.Println(s.Speak())
}

func main() {
 // Dog never writes "implements Speaker".
 greet(Dog{Name: "Rex"})
}

```

Expected:

```
Rex says woof
```

### Example: empty interface / any

Save as main.go and go run . (with go mod init example if needed).

```

package main

import "fmt"

func describe(v any) {
 fmt.Printf("%T -> %v\n", v, v)
}

func main() {
 describe(42)
 describe("hi")
 describe(true)
}

```

Expected:



```
int -> 42
string -> hi
bool -> true
```

## Runnable example

Save as `main.go`. Then:

```
go mod init example
go run .

package main

import (
 "fmt"
 "io"
 "strings"
)

type Stringer interface {
 String() string
}

type Person struct{ Name string }

func (p Person) String() string { return p.Name }

// Implicit satisfaction: no "implements" keyword.
var _ Stringer = Person{}

type ByteCounter int

func (c *ByteCounter) Write(p []byte) (int, error) {
 *c += ByteCounter(len(p))
 return len(p), nil
}

func dump(r io.Reader) (string, error) {
 b, err := io.ReadAll(r)
 return string(b), err
}

func main() {
 var s Stringer = Person{Name: "Alice"}
 fmt.Println("Stringer:", s.String())
}
```

```

// Interface value is a (type, value) pair.
var r io.Reader = strings.NewReader("hello interfaces")
text, err := dump(r)
fmt.Println("from Reader:", text, err)

var c ByteCounter
// *ByteCounter satisfies io.Writer.
_, _ = c.Write([]byte("abcd"))
_, _ = fmt.Fprintf(&c, "ef")
fmt.Println("bytes written:", int(c))

// nil interface vs typed-nil inside interface
var w io.Writer
fmt.Println("nil interface:", w == nil)
var bc *ByteCounter
w = bc
fmt.Println("typed-nil interface equals nil?", w == nil)
}

```

#### Expected output:

```

Stringer: Alice
from Reader: hello interfaces <nil>
bytes written: 6
nil interface: true
typed-nil interface equals nil? false

```

**What to notice:** Types satisfy interfaces by method set alone. An interface holding a typed nil is not equal to a true nil interface — a common source of surprising error checks.

**Try next:** Assert compile-time compliance with `var _ io.Writer = (*ByteCounter)(nil)`; pass both `strings.Reader` and `*ByteCounter` into one function that only accepts small interfaces.

# Interface Best Practices

## Overview

Well-designed interfaces make Go code flexible, testable, and maintainable.

## Keep Interfaces Small

```
// Good: small, focused
type Reader interface {
 Read([]byte) (int, error)
}

// Avoid: too many methods
type DataProcessor interface {
 Read([]byte) (int, error)
 Write([]byte) (int, error)
 Close() error
 Flush() error
 Seek(int64, int) (int64, error)
 // ...10 more methods
}
```

## Define Interfaces at Consumer

```
// In the package that USES the interface
package myapp

type Storage interface {
 Save(data []byte) error
}

func Process(s Storage) {
 s.Save([]byte("data"))
}

// NOT in the package that implements it
```

## Accept Interfaces, Return Structs

```
// Accept interface (flexible)
func ParseJSON(r io.Reader) (*Config, error)

// Return concrete type (clear)
func NewFileReader(path string) *FileReader
```

## Interface Composition

```
type Reader interface {
 Read([]byte) (int, error)
}

type Closer interface {
 Close() error
}

type ReadCloser interface {
 Reader
 Closer
}
```

## Avoid Interface Pollution

```
// Don't create interfaces for single implementations
// Just use the concrete type

// Unnecessary
type UserService interface {
 GetUser(id int) *User
}
type userServiceImpl struct{}

// Better: just use the struct directly
type UserService struct{}
func (s *UserService) GetUser(id int) *User
```

## Testing with Interfaces

```
type EmailSender interface {
 Send(to, subject, body string) error
}

// Production
type SMTPSender struct{}
func (s *SMTPSender) Send(to, subject, body string) error

// Test mock
type MockSender struct {
 SentEmails []string
}
func (m *MockSender) Send(to, subject, body string) error {
 m.SentEmails = append(m.SentEmails, to)
 return nil
}
```

## Summary

Practice	Reason
Small interfaces	Easy to implement and mock
Consumer-defined	Loose coupling
Accept interfaces	Flexibility
Return structs	Clarity
Compose	Build from small pieces

## Related Topics

- [Designing with Interfaces](#)
- [Testing Fundamentals](#)

## More examples

### Example: accept interfaces, return structs

Save as `main.go` and `go run .` (with `go mod init example` if needed).

```

package main

import "fmt"

type Writer interface {
 Write([]byte) (int, error)
}

type memWriter struct{ n int }

func (m *memWriter) Write(p []byte) (int, error) {
 m.n += len(p)
 return len(p), nil
}

// NewMemWriter returns a concrete type; callers depend on Writer where needed.
func NewMemWriter() *memWriter { return &memWriter{} }

func writeAll(w Writer, msg string) error {
 _, err := w.Write([]byte(msg))
 return err
}

func main() {
 mw := NewMemWriter()
 _ = writeAll(mw, "hello")
 _ = writeAll(mw, "!")
 fmt.Println("bytes written:", mw.n)
}

```

Expected:

```
bytes written: 6
```

## Example: small interfaces compose

Save as `main.go` and go `run .` (with `go mod init example` if needed).

```

package main

import "fmt"

type Reader interface{ Read() string }
type Closer interface{ Close() error }
type ReadCloser interface {
 Reader

```

```

 Closer
}

type src struct{ closed bool }

func (s *src) Read() string {
 if s.closed {
 return ""
 }
 return "data"
}

func (s *src) Close() error {
 s.closed = true
 return nil
}

func drain(rc ReadCloser) {
 fmt.Println("read:", rc.Read())
 _ = rc.Close()
 fmt.Println("after close:", rc.Read())
}

func main() {
 drain(&src{})
}

```

Expected:

```

read: data
after close:

```

## Runnable example

Save as `main.go`. Then:

```

go mod init example
go run .

```

```

package main

import (
 "fmt"
 "io"
 "strings"
)

```

```

// Small, consumer-defined interfaces.
type Storage interface {
 Save(data []byte) error
}

type EmailSender interface {
 Send(to, subject, body string) error
}

// Concrete production types - no interface required at definition site.
type MemoryStorage struct {
 items []string
}

func (m *MemoryStorage) Save(data []byte) error {
 m.items = append(m.items, string(data))
 return nil
}

type MockSender struct {
 Sent []string
}

func (m *MockSender) Send(to, subject, body string) error {
 m.Sent = append(m.Sent, to+":"+subject)
 return nil
}

// Accept interfaces, return concrete values.
func ParseConfig(r io.Reader) (string, error) {
 b, err := io.ReadAll(r)
 if err != nil {
 return "", err
 }
 return strings.TrimSpace(string(b)), nil
}

func Process(s Storage, msg string) error {
 return s.Save([]byte(msg))
}

func Notify(sender EmailSender, user string) error {
 return sender.Send(user, "welcome", "hello")
}

func main() {

```



```

cfg, err := ParseConfig(strings.NewReader(" port=8080 "))
fmt.Println("config:", cfg, err)

store := &MemoryStorage{}
_ = Process(store, "event-1")
_ = Process(store, "event-2")
fmt.Println("stored:", store.items)

mock := &MockSender{}
_ = Notify(mock, "ada@example.com")
fmt.Println("sent:", mock.Sent)
}

```

### Expected output:

```

config: port=8080 <nil>
stored: [event-1 event-2]
sent: [ada@example.com:welcome]

```

**What to notice:** Call sites depend on tiny interfaces (`Storage`, `EmailSender`, `io.Reader`); implementations stay concrete structs you can construct and inspect in tests without a framework.

**Try next:** Compose type `ReadSaver` interface `{ io.Reader; Storage }` or inject a failing `EmailSender` that returns an error and handle it in `Notify`.

# Type Assertions and Reflection Boundaries

## Overview

Type assertions extract concrete types from interfaces. Reflection provides runtime type inspection but should be used sparingly.

## Type Assertions

```
var i interface{} = "hello"

s := i.(string) // Panic if wrong type
s, ok := i.(string) // Safe: ok is false if wrong type

if s, ok := i.(string); ok {
 fmt.Println(s)
}
```

## Type Switches

```
func describe(i interface{}) {
 switch v := i.(type) {
 case int:
 fmt.Println("int:", v*2)
 case string:
 fmt.Println("string:", len(v))
 case bool:
 fmt.Println("bool:", !v)
 default:
 fmt.Printf("unknown: %T\n", v)
 }
}
```

## The reflect Package

```
import "reflect"

v := 42
t := reflect.TypeOf(v) // int
val := reflect.ValueOf(v) // 42

fmt.Println(t.Name()) // "int"
fmt.Println(t.Kind()) // int
fmt.Println(val.Int()) // 42
```

## Struct Reflection

```
type User struct {
 Name string `json:"name"`
 Age int `json:"age"`
}

u := User{Name: "Alice", Age: 30}
t := reflect.TypeOf(u)

for i := 0; i < t.NumField(); i++ {
 field := t.Field(i)
 fmt.Printf("%s: %s\n", field.Name, field.Tag.Get("json"))
}
```

## When to Use Reflection

**Good uses:** - Serialization (JSON, XML) - ORM field mapping - Generic testing utilities - Dependency injection

**Avoid for:** - Performance-critical code - Simple type checking - Normal business logic

## Performance Cost

```
// Direct access: ~1ns
user.Name

// Reflection: ~100ns+
reflect.ValueOf(user).FieldByName("Name").String()
```

## Prefer Generics Over Reflection

```
// Go 1.18+: Use generics instead of reflect
func PrintSlice[T any](s []T) {
 for _, v := range s {
 fmt.Println(v)
 }
}
```

## Summary

Approach	Use Case
Type assertion	Extract known concrete type
Type switch	Handle multiple types
reflect	Runtime introspection (last resort)
Generics	Compile-time type safety

## Related Topics

- [Why Generics Matter](#)
- [Encoding and Serialization](#)

## More examples

### Example: safe comma-ok assertion

Save as `main.go` and `go run .` (with `go mod init example` if needed).

```
package main

import "fmt"

func asInt(v any) {
 if n, ok := v.(int); ok {
 fmt.Println("int:", n)
 return
 }
 fmt.Printf("not int: %T\n", v)
}

func main() {
```

```
 asInt(7)
 asInt("nope")
}
```

Expected:

```
int: 7
not int: string
```

## Example: type switch

Save as `main.go` and `go run .` (with `go mod init example` if needed).

```
package main

import "fmt"

func classify(v any) {
 switch x := v.(type) {
 case int:
 fmt.Println("int", x*2)
 case string:
 fmt.Println("string len", len(x))
 default:
 fmt.Printf("other %T\n", x)
 }
}

func main() {
 classify(3)
 classify("go")
 classify(true)
}
```

Expected:

```
int 6
string len 2
other bool
```

## Runnable example

Save as `main.go`. Then:

```
go mod init example
go run .
```

```
package main
```

```
import (
 "fmt"
 "reflect"
)
```

```
func describe(i any) {
 switch v := i.(type) {
 case int:
 fmt.Println("int:", v*2)
 case string:
 fmt.Println("string len:", len(v))
 case bool:
 fmt.Println("bool not:", !v)
 default:
 fmt.Printf("unknown: %T\n", v)
 }
}
```

```
type User struct {
 Name string `json:"name"`
 Age int `json:"age"`
}
```

```
func main() {
 var i any = "hello"
 s, ok := i.(string)
 fmt.Println("assert string:", s, ok)
 n, ok := i.(int)
 fmt.Println("assert int:", n, ok)
```

```
 describe(21)
 describe("go")
 describe(true)
 describe(3.14)
```

```
 // Reflection: useful for tags/serialization; prefer assertions/generics elsewhere.
 u := User{Name: "Ada", Age: 36}
 t := reflect.TypeOf(u)
 for j := 0; j < t.NumField(); j++ {
 f := t.Field(j)
 fmt.Printf("field %s json=%q\n", f.Name, f.Tag.Get("json"))
 }
}
```

```
}
}
```

**Expected output:**

```
assert string: hello true
assert int: 0 false
int: 42
string len: 2
bool not: false
unknown: float64
field Name json="name"
field Age json="age"
```

**What to notice:** The comma-ok form never panics; a type switch is clearer than a chain of assertions. Reflection can read struct tags but costs more and loses static checking.

**Try next:** Replace the default branch with a generic `PrintSlice[T any]` for slice values, or make a failing assertion `i.(int)` without `ok` and observe the panic.

# Why Generics Matter

## Overview

Generics (type parameters), introduced in Go 1.18, allow writing functions and types that work with multiple types while maintaining type safety.

## The Problem Before Generics

```
// Without generics: duplicate code or use interface{}
func MinInt(a, b int) int {
 if a < b { return a }
 return b
}

func MinFloat(a, b float64) float64 {
 if a < b { return a }
 return b
}

// Or lose type safety
func Min(a, b interface{}) interface{} {
 // Requires type assertions, runtime checks
}
```

## With Generics

```
func Min[T constraints.Ordered](a, b T) T {
 if a < b {
 return a
 }
 return b
}

Min(1, 2) // int
Min(1.5, 2.5) // float64
Min("a", "b") // string
```



## Type Parameters

```
func Print[T any](v T) {
 fmt.Println(v)
}

// Multiple type parameters
func Pair[K, V any](k K, v V) (K, V) {
 return k, v
}
```

## Constraints

```
import "golang.org/x/exp/constraints"

// Built-in constraints
any // No requirements
comparable // Supports == and !=

// From constraints package
constraints.Ordered // Supports < > <= >=
constraints.Integer // Int types
constraints.Float // Float types
```

## Custom Constraints

```
type Number interface {
 int | int32 | int64 | float32 | float64
}

func Sum[T Number](nums []T) T {
 var total T
 for _, n := range nums {
 total += n
 }
 return total
}
```

## Generic Type Aliases (Go 1.24+)

Go 1.24 introduced support for generic type aliases, allowing you to create aliases for generic types.

```

type Set[T comparable] map[T]struct{}

// Type alias (Go 1.24+)
type IntSet = Set[int]
type AnySet[T comparable] = Set[T]

```

This is particularly useful for refactoring and maintaining compatibility while migrating to generic implementations.

## Self-Referential Generics (Go 1.26+)

Go 1.26 lifted the restriction that prevented generic type parameters from referring to their enclosing type definition in constraint declarations. This allows fluent builder interfaces and self-referential tree nodes with static type safety:

```

// Self-referential constraint allowed in Go 1.26+
type Node[T Node[T]] interface {
 Value() string
 Next() T
}

type MyNode struct {
 val string
 next *MyNode
}

func (m *MyNode) Value() string { return m.val }
func (m *MyNode) Next() *MyNode { return m.next }

```

## Generic Methods (Go 1.27+)

A method declaration may declare its own type parameters. Use this when the operation belongs on a concrete type but the argument/result type varies:

```

func (r *rand.Rand) N[Int intType](n Int) Int

```

That is **not** the same as a method on a generic type (`func (s *Stack[T]) Push(T)`), which has been valid since Go 1.18. Interface methods still cannot be generic, and a generic method cannot satisfy an interface method.

Function type inference is also broader in 1.27: a generic function can be assigned, converted, sent on a channel, or placed in a composite literal without spelling the type arguments when the destination type is a matching function type. See [Generic Functions](#).

## Benefits

1. **Type safety:** Errors caught at compile time
2. **No code duplication:** One implementation for many types
3. **Performance:** No interface boxing/unboxing
4. **Better documentation:** Types are explicit

## Summary

Before	After
Code duplication	Single generic function
<code>interface{}</code>	Type parameters
Runtime errors	Compile-time errors
Type assertions	Direct usage

## Related Topics

- [Generic Functions](#)
- [Generic Types](#)

## More examples

### Example: before generics—duplication

Save as `main.go` and `go run .` (with `go mod init example` if needed).

```
package main

import "fmt"

func maxInt(a, b int) int {
 if a > b {
 return a
 }
 return b
}

func maxFloat(a, b float64) float64 {
 if a > b {
 return a
 }
 return b
}
```

```

}

func main() {
 fmt.Println(maxInt(3, 7))
 fmt.Println(maxFloat(3.5, 2.1))
}

```

Expected:

```

7
3.5

```

### Example: one generic Max

Save as `main.go` and `go run .` (with `go mod init example` if needed).

```

package main

import (
 "cmp"
 "fmt"
)

func Max[T cmp.Ordered](a, b T) T {
 if a > b {
 return a
 }
 return b
}

func main() {
 fmt.Println(Max(3, 7))
 fmt.Println(Max(3.5, 2.1))
 fmt.Println(Max("a", "z"))
}

```

Expected:

```

7
3.5
z

```

### Runnable example

Save as `main.go`. Then:

```

go mod init example
go run .

package main

import (
 "cmp"
 "fmt"
)

// cmp.Ordered covers types that support < (stdlib; no x/exp needed).
func Min[T cmp.Ordered](a, b T) T {
 if a < b {
 return a
 }
 return b
}

func Pair[K, V any](k K, v V) (K, V) {
 return k, v
}

// Custom constraint with a type set (stdlib only).
type Number interface {
 ~int | ~int64 | ~float64
}

func Sum[T Number](nums []T) T {
 var total T
 for _, n := range nums {
 total += n
 }
 return total
}

func main() {
 fmt.Println("Min int:", Min(3, 1))
 fmt.Println("Min float:", Min(2.5, 2.7))
 fmt.Println("Min string:", Min("go", "ai"))

 k, v := Pair("age", 30)
 fmt.Printf("Pair: %s=%d\n", k, v)

 fmt.Println("Sum ints:", Sum([]int{1, 2, 3}))
 fmt.Println("Sum floats:", Sum([]float64{0.5, 1.5}))
}

```

### Expected output:

```
Min int: 1
Min float: 2.5
Min string: ai
Pair: age=30
Sum ints: 6
Sum floats: 2
```

**What to notice:** One `Min` works for any ordered type with compile-time safety; `cmp.Ordered` lives in the standard library. Type sets let you restrict `Sum` to numeric-ish types without `interface{}` assertions.

**Try next:** Call `Min` on a struct (should fail to compile); add `~string` to a constraint and write `Join[T ~string](parts []T) T`.

# Generic Functions

## Overview

Generic functions use type parameters to work with multiple types while maintaining type safety.

## Basic Syntax

```
func FunctionName[T constraint](params) returnType {
 // body
}
```

## Examples

### Map

```
func Map[T, U any](slice []T, fn func(T) U) []U {
 result := make([]U, len(slice))
 for i, v := range slice {
 result[i] = fn(v)
 }
 return result
}

doubled := Map([]int{1, 2, 3}, func(n int) int { return n * 2 })
// [2, 4, 6]
```

### Filter

```
func Filter[T any](slice []T, fn func(T) bool) []T {
 var result []T
 for _, v := range slice {
 if fn(v) {
 result = append(result, v)
 }
 }
}
```

```

 }
 return result
}

evens := Filter([]int{1, 2, 3, 4}, func(n int) bool { return n%2 == 0 })
// [2, 4]

```

## Find

```

func Find[T any](slice []T, fn func(T) bool) (T, bool) {
 for _, v := range slice {
 if fn(v) {
 return v, true
 }
 }
 var zero T
 return zero, false
}

```

## Contains

```

func Contains[T comparable](slice []T, target T) bool {
 for _, v := range slice {
 if v == target {
 return true
 }
 }
 return false
}

```

## Keys/Values

```

func Keys[K comparable, V any](m map[K]V) []K {
 keys := make([]K, 0, len(m))
 for k := range m {
 keys = append(keys, k)
 }
 return keys
}

func Values[K comparable, V any](m map[K]V) []V {
 vals := make([]V, 0, len(m))

```



```

 for _, v := range m {
 vals = append(vals, v)
 }
 return vals
}

```

## Type Inference

Go infers type arguments when possible:

```

// Explicit
result := Map[int, int](nums, double)

// Inferred (preferred)
result := Map(nums, double)

```

## Assignment contexts (Go 1.27+)

Inference now applies in every assignment-shaped context: composite literals, conversions, and channel sends, not only direct calls:

```

func GenericFormatter[T any](v T) string {
 return fmt.Sprintf("value: %v", v)
}

type IntFormatter func(int) string

formatters := []IntFormatter{GenericFormatter} // T = int
fn := IntFormatter(GenericFormatter)
ch := make(chan IntFormatter, 1)
ch <- GenericFormatter

```

## Summary

Pattern	Signature
Transform	<code>func Map[T, U any] ([]T, func(T) U) []U</code>
Filter	<code>func Filter[T any] ([]T, func(T) bool) []T</code>
Find	<code>func Find[T any] ([]T, func(T) bool) (T, bool)</code>
Contains	<code>func Contains[T comparable] ([]T, T) bool</code>

## Related Topics

- [Generic Types](#)
- [Idiomatic Generics](#)

## More examples

### Example: Map slice transform

Save as `main.go` and `go run .` (with `go mod init example` if needed).

```
package main

import "fmt"

func Map[T, U any](in []T, f func(T) U) []U {
 out := make([]U, len(in))
 for i, v := range in {
 out[i] = f(v)
 }
 return out
}

func main() {
 nums := []int{1, 2, 3}
 strs := Map(nums, func(n int) string {
 return fmt.Sprintf("#%d", n)
 })
 fmt.Println(strs)
}
```

Expected:

```
[#1 #2 #3]
```

### Example: Filter with predicate

Save as `main.go` and `go run .` (with `go mod init example` if needed).

```
package main

import "fmt"

func Filter[T any](in []T, keep func(T) bool) []T {
 var out []T
```

```

 for _, v := range in {
 if keep(v) {
 out = append(out, v)
 }
 }
 return out
}

func main() {
 nums := []int{1, 2, 3, 4, 5}
 evens := Filter(nums, func(n int) bool { return n%2 == 0 })
 fmt.Println(evens)
}

```

Expected:

```
[2 4]
```

## Runnable example

Save as `main.go`. Then:

```

go mod init example
go run .

package main

import (
 "fmt"
 "slices"
)

func Map[T, U any](in []T, fn func(T) U) []U {
 out := make([]U, len(in))
 for i, v := range in {
 out[i] = fn(v)
 }
 return out
}

func Filter[T any](in []T, keep func(T) bool) []T {
 var out []T
 for _, v := range in {
 if keep(v) {
 out = append(out, v)
 }
 }
 return out
}

```

```

 }
}
return out
}

func Contains[T comparable](in []T, target T) bool {
 for _, v := range in {
 if v == target {
 return true
 }
 }
 return false
}

func Keys[K comparable, V any](m map[K]V) []K {
 keys := make([]K, 0, len(m))
 for k := range m {
 keys = append(keys, k)
 }
 return keys
}

func main() {
 nums := []int{1, 2, 3, 4}
 doubled := Map(nums, func(n int) int { return n * 2 })
 fmt.Println("Map:", doubled)

 evens := Filter(nums, func(n int) bool { return n%2 == 0 })
 fmt.Println("Filter:", evens)

 fmt.Println("Contains 3:", Contains(nums, 3))
 fmt.Println("Contains 9:", Contains(nums, 9))

 // Prefer stdlib helpers when they already exist.
 fmt.Println("slices.Contains 3:", slices.Contains(nums, 3))
 fmt.Println("slices.Index 4:", slices.Index(nums, 4))

 m := map[string]int{"a": 1, "b": 2}
 keys := Keys(m)
 slices.Sort(keys) // map iteration order is random
 fmt.Println("Keys sorted:", keys)
}

```

**Expected output:**

```
Map: [2 4 6 8]
Filter: [2 4]
Contains 3: true
Contains 9: false
slices.Contains 3: true
slices.Index 4: 3
Keys sorted: [a b]
```

**What to notice:** Type parameters preserve element types end-to-end (`[]int` in, `[]int` out). The stdlib `slices` package already covers many of these helpers — write your own when the transform is domain-specific.

**Try next:** Add `Reduce[T any](in []T, init T, fn func(T, T) T) T`; map `[]string` to `[]int` lengths with `Map`.

# Generic Types

## Overview

Generic types let you create data structures that work with any type while maintaining type safety.

## Basic Syntax

```
type TypeName[T constraint] struct {
 // fields using T
}
```

## Stack

```
type Stack[T any] struct {
 items []T
}

func (s *Stack[T]) Push(item T) {
 s.items = append(s.items, item)
}

func (s *Stack[T]) Pop() (T, bool) {
 if len(s.items) == 0 {
 var zero T
 return zero, false
 }
 item := s.items[len(s.items)-1]
 s.items = s.items[:len(s.items)-1]
 return item, true
}

// Usage
stack := &Stack[int]{}
stack.Push(1)
stack.Push(2)
v, _ := stack.Pop() // 2
```

## Set

```
type Set[T comparable] map[T]struct{}

func NewSet[T comparable]() Set[T] {
 return make(Set[T])
}

func (s Set[T]) Add(item T) {
 s[item] = struct{}{}
}

func (s Set[T]) Contains(item T) bool {
 _, ok := s[item]
 return ok
}

func (s Set[T]) Remove(item T) {
 delete(s, item)
}

// Usage
set := NewSet[string]()
set.Add("a")
set.Contains("a") // true
```

## Pair

```
type Pair[T, U any] struct {
 First T
 Second U
}

func NewPair[T, U any](first T, second U) Pair[T, U] {
 return Pair[T, U]{first, second}
}

p := NewPair("age", 30)
// Pair[string, int]{First: "age", Second: 30}
```

## Result (Option Pattern)

```
type Result[T any] struct {
 value T
 err error
}

func Ok[T any](v T) Result[T] {
 return Result[T]{value: v}
}

func Err[T any](err error) Result[T] {
 return Result[T]{err: err}
}

func (r Result[T]) Unwrap() (T, error) {
 return r.value, r.err
}
```

## LinkedList

```
type Node[T any] struct {
 Value T
 Next *Node[T]
}

type LinkedList[T any] struct {
 Head *Node[T]
}

func (l *LinkedList[T]) Add(v T) {
 node := &Node[T]{Value: v, Next: l.Head}
 l.Head = node
}
```

These methods use the **type**'s type parameter T. Go 1.27 also allows a method to declare **additional** type parameters of its own (`func (s *Store) Put[U any](v U)`). See [Methods and Receivers](#).

## Summary

Type	Purpose
Stack[T]	LIFO collection



Type	Purpose
<code>Set[T]</code>	Unique elements
<code>Pair[T, U]</code>	Key-value tuple
<code>Result[T]</code>	Error handling

## Related Topics

- [Generic Functions](#)
- [Idiomatic Generics](#)

## More examples

### Example: generic Stack

Save as `main.go` and go `run .` (with `go mod init example` if needed).

```
package main

import "fmt"

type Stack[T any] struct {
 items []T
}

func (s *Stack[T]) Push(v T) { s.items = append(s.items, v) }

func (s *Stack[T]) Pop() (T, bool) {
 var zero T
 if len(s.items) == 0 {
 return zero, false
 }
 i := len(s.items) - 1
 v := s.items[i]
 s.items = s.items[:i]
 return v, true
}

func main() {
 var s Stack[string]
 s.Push("a")
 s.Push("b")
 v, ok := s.Pop()
 fmt.Println(v, ok)
```

```

 v, ok = s.Pop()
 fmt.Println(v, ok)
 v, ok = s.Pop()
 fmt.Println(v, ok)
}

```

Expected:

```

b true
a true
false

```

## Example: generic Pair

Save as `main.go` and go `run .` (with `go mod init example` if needed).

```

package main

import "fmt"

type Pair[A, B any] struct {
 First A
 Second B
}

func main() {
 p := Pair[string, int]{First: "age", Second: 42}
 fmt.Printf("%s=%d\n", p.First, p.Second)
 q := Pair[int, bool]{First: 1, Second: true}
 fmt.Println(q.First, q.Second)
}

```

Expected:

```

age=42
1 true

```

## Runnable example

Save as `main.go`. Then:

```

go mod init example
go run .

```

```

package main

import "fmt"

type Stack[T any] struct {
 items []T
}

func (s *Stack[T]) Push(item T) {
 s.items = append(s.items, item)
}

func (s *Stack[T]) Pop() (T, bool) {
 if len(s.items) == 0 {
 var zero T
 return zero, false
 }
 i := len(s.items) - 1
 item := s.items[i]
 s.items = s.items[:i]
 return item, true
}

type Set[T comparable] map[T]struct{}

func NewSet[T comparable]() Set[T] {
 return make(Set[T])
}

func (s Set[T]) Add(item T) { s[item] = struct{}{} }
func (s Set[T]) Contains(item T) bool {
 _, ok := s[item]
 return ok
}

type Pair[T, U any] struct {
 First T
 Second U
}

func main() {
 st := &Stack[string]{}
 st.Push("a")
 st.Push("b")
 top, ok := st.Pop()
 fmt.Println("stack pop:", top, ok)
}

```

```

 set := NewSet[int]()
 set.Add(1)
 set.Add(1)
 set.Add(2)
 fmt.Println("set has 1:", set.Contains(1))
 fmt.Println("set has 3:", set.Contains(3))
 fmt.Println("set size:", len(set))

 p := Pair[string, int]{First: "age", Second: 30}
 fmt.Printf("pair: %s=%d\n", p.First, p.Second)
}

```

**Expected output:**

```

stack pop: b true
set has 1: true
set has 3: false
set size: 2
pair: age=30

```

**What to notice:** `Stack[T]` and `Set[T]` keep a single implementation while the compiler enforces element types. Empty `Pop` returns the type's zero value plus `false` — a common generic API pattern.

**Try next:** Add `Peek()` (`T`, `bool`) and `Size()` `int` to `Stack`; build a `Queue[T]` with head/tail indices.

# Idiomatic Generics

## Overview

Generics are powerful but should be used judiciously. This chapter covers when to use generics vs interfaces, and best practices.

## When to Use Generics

**Good uses:** - Container types (Stack, Set, Queue) - Utility functions (Map, Filter, Reduce) - Type-safe concurrent constructs - Avoiding reflection

**Don't use when:** - Interface works fine - Only one type will ever be used - Adding complexity without benefit

Go 1.27 generic **methods** (`func (s *Store) Put[T any](v T)`) are for operations namespaced on a type. They cannot implement interface methods—keep those non-generic.

## Generics vs Interfaces

### Use Interface When:

```
// Behavior-based polymorphism
type Writer interface {
 Write([]byte) (int, error)
}

func Save(w Writer, data []byte) error {
 _, err := w.Write(data)
 return err
}
```

### Use Generics When:

```
// Type preservation needed
func Reverse[T any](s []T) []T {
 result := make([]T, len(s))
```

```

 for i, v := range s {
 result[len(s)-1-i] = v
 }
 return result
}

```

## Guidelines

### 1. Start with Concrete Types

```

// Start here
func ReverseInts(s []int) []int { }

// Generalize only when needed
func Reverse[T any](s []T) []T { }

```

### 2. Use Meaningful Constraints

```

// Too broad
func Process[T any](v T) { }

// Better: documents requirements
func Process[T constraints.Ordered](v T) { }

```

### 3. Don't Over-Constrain

```

// Over-constrained
type Number interface {
 int | int8 | int16 | int32 | int64 |
 uint | uint8 | uint16 | uint32 | uint64 |
 float32 | float64
}

// Better: use existing constraint
func Sum[T constraints.Integer | constraints.Float](nums []T) T

```

## 4. Type Inference is Your Friend

```
// Let Go infer when obvious
result := Map(nums, double)

// Explicit only when needed
result := Convert[int, float64](nums)
```

## Common Patterns

### slices Package (stdlib)

```
import "slices"

slices.Sort(nums)
slices.Reverse(nums)
slices.Contains(nums, 5)
slices.Index(nums, 5)
```

### maps Package (stdlib)

```
import "maps"

maps.Clone(m)
maps.Keys(m)
maps.Values(m)
```

## Summary

Decision	Choice
Need type preservation	Generics
Need polymorphic behavior	Interface
Operating on containers	Generics
Method set requirements	Interface

## Related Topics

- [Why Generics Matter](#)
- [Interface Best Practices](#)

## More examples

### Example: prefer clear non-generic API when one type

Save as `main.go` and `go run .` (with `go mod init example` if needed).

```
package main

import "fmt"

// SumInts is clearer than Sum[int] when you only ever sum ints.
func SumInts(nums []int) int {
 total := 0
 for _, n := range nums {
 total += n
 }
 return total
}

func main() {
 fmt.Println(SumInts([]int{1, 2, 3}))
 fmt.Println(SumInts(nil))
}
```

Expected:

```
6
0
```

### Example: constrain only what you need

Save as `main.go` and `go run .` (with `go mod init example` if needed).

```
package main

import (
 "fmt"
 "strconv"
)

// Stringer-like constraint without importing fmt for the type set.
type stringer interface {
 String() string
}

type ID int
```



```

func (id ID) String() string { return "id=" + strconv.Itoa(int(id)) }

func PrintAll[T stringer](items []T) {
 for _, it := range items {
 fmt.Println(it.String())
 }
}

func main() {
 PrintAll([]ID{7, 8})
}

```

Expected:

```

id=7
id=8

```

## Runnable example

Save as `main.go`. Then:

```

go mod init example
go run .

package main

import (
 "fmt"
 "io"
 "maps"
 "slices"
 "strings"
)

// Behavior polymorphism → interface (not generics).
func Save(w io.Writer, data []byte) error {
 _, err := w.Write(data)
 return err
}

// Type preservation on containers → generics.
func Reverse[T any](s []T) []T {
 out := make([]T, len(s))
 for i, v := range s {

```

```

 out[len(s)-1-i] = v
 }
 return out
}

func main() {
 var b strings.Builder
 _ = Save(&b, []byte("hello"))
 fmt.Println("interface writer:", b.String())

 nums := []int{1, 2, 3}
 fmt.Println("Reverse ints:", Reverse(nums))
 fmt.Println("original unchanged:", nums)

 words := []string{"go", "is", "fun"}
 fmt.Println("Reverse strings:", Reverse(words))

 // Prefer stdlib generic helpers when they fit.
 cloned := slices.Clone(nums)
 slices.Sort(cloned)
 fmt.Println("slices.Sort clone:", cloned)
 fmt.Println("slices.Contains:", slices.Contains(words, "is"))

 m := map[string]int{"b": 2, "a": 1}
 cp := maps.Clone(m)
 cp["c"] = 3
 fmt.Println("maps.Clone size:", len(m), "->", len(cp))
}

```

### Expected output:

```

interface writer: hello
Reverse ints: [3 2 1]
original unchanged: [1 2 3]
Reverse strings: [fun is go]
slices.Sort clone: [1 2 3]
slices.Contains: true
maps.Clone size: 2 -> 3

```

**What to notice:** `Save` cares about behavior (`Write`), so an interface is enough. `Reverse` must preserve element type across many containers, so generics win. Reach for `slices` / `maps` before inventing another utility package.

**Try next:** Start from a concrete `ReverseInts` and only generalize when a second type appears; replace a hand-rolled `Contains` with `slices.Contains`.

# Errors

# Errors as Values

## Overview

Go treats errors as values, not exceptions. Functions return errors alongside results, making error handling explicit and visible.

## The error Interface

```
type error interface {
 Error() string
}
```

## Returning Errors

```
func divide(a, b float64) (float64, error) {
 if b == 0 {
 return 0, errors.New("division by zero")
 }
 return a / b, nil
}

result, err := divide(10, 0)
if err != nil {
 log.Fatal(err)
}
```

## Creating Errors

```
import "errors"

// Simple error
err := errors.New("something went wrong")

// Formatted error
```

```
err := fmt.Errorf("failed to open %s: %v", filename, originalErr)
```

## Error Handling Patterns

### Check Immediately

```
result, err := doSomething()
if err != nil {
 return err
}
// Use result
```

### Early Return

```
func process() error {
 data, err := fetch()
 if err != nil {
 return err
 }

 result, err := transform(data)
 if err != nil {
 return err
 }

 return save(result)
}
```

### Add Context

```
data, err := readFile(path)
if err != nil {
 return fmt.Errorf("loading config: %w", err)
}
```

## Custom Error Types

```
type ValidationError struct {
 Field string
 Message string
}

func (e *ValidationError) Error() string {
 return fmt.Sprintf("%s: %s", e.Field, e.Message)
}

func validate(u User) error {
 if u.Name == "" {
 return &ValidationError{Field: "name", Message: "required"}
 }
 return nil
}
```

## Sentinel Errors

```
var ErrNotFound = errors.New("not found")
var ErrPermission = errors.New("permission denied")

func find(id int) (*User, error) {
 if !exists(id) {
 return nil, ErrNotFound
 }
 // ...
}

// Check
if err == ErrNotFound {
 // Handle not found
}
```

## Summary

Pattern	Usage
<code>errors.New()</code>	Simple error
<code>fmt.Errorf()</code>	Formatted error
Custom type	Structured error data
Sentinel	Known error values

## Related Topics

- [Wrapping and Inspecting Errors](#)
- [panic, recover, and Failure Modes](#)

## Worked example

Multi-step pipeline that returns, wraps, and classifies errors without panicking.

Save as `main.go`. Then:

```
go mod init example
go run .

package main

import (
 "errors"
 "fmt"
)

var ErrEmpty = errors.New("empty input")

type ParseError struct {
 Token string
 Why string
}

func (e *ParseError) Error() string {
 return fmt.Sprintf("parse %q: %s", e.Token, e.Why)
}

func parseIntToken(s string) (int, error) {
 if s == "" {
 return 0, ErrEmpty
 }
 n := 0
 for _, r := range s {
 if r < '0' || r > '9' {
 return 0, &ParseError{Token: s, Why: "not digits"}
 }
 n = n*10 + int(r-'0')
 }
 return n, nil
}
```

```

func sumTokens(tokens []string) (int, error) {
 total := 0
 for i, t := range tokens {
 n, err := parseIntToken(t)
 if err != nil {
 return 0, fmt.Errorf("sumTokens[%d]: %w", i, err)
 }
 total += n
 }
 return total, nil
}

func main() {
 got, err := sumTokens([]string{"10", "20", "12"})
 fmt.Println("ok sum:", got, err)

 _, err = sumTokens([]string{"10", "", "3"})
 fmt.Println("empty err:", err)
 fmt.Println("Is ErrEmpty:", errors.Is(err, ErrEmpty))

 _, err = sumTokens([]string{"10", "x9"})
 var pe *ParseError
 if errors.As(err, &pe) {
 fmt.Println("As ParseError token:", pe.Token, "why:", pe.Why)
 }
}

```

### Expected output:

```

ok sum: 42 <nil>
empty err: sumTokens[1]: empty input
Is ErrEmpty: true
As ParseError token: x9 why: not digits

```

## More examples

Sentinel comparison vs string equality—only the value works reliably after wrapping.

```

package main

import (
 "errors"
 "fmt"
)

```



```

var ErrDenied = errors.New("denied")

func authorize(role string) error {
 if role != "admin" {
 return fmt.Errorf("authorize(%s): %w", role, ErrDenied)
 }
 return nil
}

func main() {
 err := authorize("guest")
 // Correct: compare the sentinel through the chain.
 fmt.Println("Is denied:", errors.Is(err, ErrDenied))
 // Fragile: string form changes when you add context.
 fmt.Println("string equal ErrDenied:", err.Error() == ErrDenied.Error())
 fmt.Println("err text:", err)
}

```

Expected output:

```

Is denied: true
string equal ErrDenied: false
err text: authorize(guest): denied

```

## Runnable example

Save as `main.go`. Then:

```

go mod init example
go run .

package main

import (
 "errors"
 "fmt"
)

var ErrNotFound = errors.New("not found")

type ValidationError struct {
 Field string
 Message string
}

```

```

func (e *ValidationError) Error() string {
 return fmt.Sprintf("%s: %s", e.Field, e.Message)
}

func divide(a, b float64) (float64, error) {
 if b == 0 {
 return 0, errors.New("division by zero")
 }
 return a / b, nil
}

func findUser(id int) (string, error) {
 if id <= 0 {
 return "", ErrNotFound
 }
 return fmt.Sprintf("user-%d", id), nil
}

func validateName(name string) error {
 if name == "" {
 return &ValidationError{Field: "name", Message: "required"}
 }
 return nil
}

func main() {
 q, err := divide(10, 2)
 fmt.Println("divide ok:", q, err)
 _, err = divide(1, 0)
 fmt.Println("divide err:", err)

 name, err := findUser(7)
 fmt.Println("find:", name, err)
 _, err = findUser(0)
 if errors.Is(err, ErrNotFound) {
 fmt.Println("sentinel: not found")
 }

 if err := validateName(""); err != nil {
 fmt.Println("validation:", err)
 }
 if err := validateName("Ada"); err != nil {
 fmt.Println("unexpected:", err)
 } else {
 fmt.Println("validation: ok")
 }
}

```

**Expected output:**

```
divide ok: 5 <nil>
divide err: division by zero
find: user-7 <nil>
sentinel: not found
validation: name: required
validation: ok
```

**What to notice:** Errors are ordinary values you return and inspect. Sentinel errors (`ErrNotFound`) and custom types (`ValidationError`) both implement the same `error` interface.

**Try next:** Use `fmt.Errorf("findUser(%d): %w", id, ErrNotFound)` and compare string form vs `errors.Is`; add a second field to `ValidationError` for a code.

# Wrapping and Inspecting Errors

## Overview

Go 1.13 introduced error wrapping, allowing you to add context while preserving the original error for inspection.

## Wrapping Errors

```
originalErr := errors.New("file not found")

// %w wraps the error
wrappedErr := fmt.Errorf("loading config: %w", originalErr)

// The error chain: wrappedErr -> originalErr
```

## Unwrapping

```
err := fmt.Errorf("outer: %w",
 fmt.Errorf("middle: %w",
 errors.New("inner")))

inner := errors.Unwrap(err) // middle: inner
```

## errors.Is

Check if any error in the chain matches:

```
var ErrNotFound = errors.New("not found")

err := fmt.Errorf("user lookup: %w", ErrNotFound)

if errors.Is(err, ErrNotFound) {
 // Handle not found
}
```

## errors.As

Extract specific error types:

```
type ValidationError struct {
 Field string
}

func (e *ValidationError) Error() string {
 return "validation: " + e.Field
}

err := fmt.Errorf("processing: %w", &ValidationError{Field: "email"})

var valErr *ValidationError
if errors.As(err, &valErr) {
 fmt.Println("Invalid field:", valErr.Field)
}
```

## Custom Wrappers

```
type WrappedError struct {
 Context string
 Err error
}

func (e *WrappedError) Error() string {
 return fmt.Sprintf("%s: %v", e.Context, e.Err)
}

func (e *WrappedError) Unwrap() error {
 return e.Err
}
```

## Best Practices

```
// Add context when crossing boundaries
func LoadUser(id int) (*User, error) {
 data, err := db.Query(id)
 if err != nil {
 return nil, fmt.Errorf("LoadUser(%d): %w", id, err)
 }
}
```

```
 return parse(data)
}
```

## Summary

Function	Purpose
<code>fmt.Errorf("%w", err)</code>	Wrap error
<code>errors.Unwrap(err)</code>	Get wrapped error
<code>errors.Is(err, target)</code>	Check chain for match
<code>errors.As(err, &amp;target)</code>	Extract typed error

## Related Topics

- [Errors as Values](#)
- [panic, recover, and Failure Modes](#)

## Worked example

Deep wrap chain with `Is`, `As`, and manual `Unwrap` walking.

Save as `main.go`. Then:

```
go mod init example
go run .

package main

import (
 "errors"
 "fmt"
)

var ErrNotFound = errors.New("not found")

type HTTPError struct {
 Code int
 Err error
}

func (e *HTTPError) Error() string {
 return fmt.Sprintf("http %d: %v", e.Code, e.Err)
```

```

}

func (e *HTTPError) Unwrap() error { return e.Err }

func storeGet(key string) error {
 if key == "" {
 return ErrNotFound
 }
 return nil
}

func serviceGet(key string) error {
 if err := storeGet(key); err != nil {
 return fmt.Errorf("service.get(%q): %w", key, err)
 }
 return nil
}

func handlerGet(key string) error {
 if err := serviceGet(key); err != nil {
 return &HTTPError{Code: 404, Err: err}
 }
 return nil
}

func main() {
 err := handlerGet("")
 fmt.Println("chain:", err)
 fmt.Println("Is NotFound:", errors.Is(err, ErrNotFound))

 var he *HTTPError
 if errors.As(err, &he) {
 fmt.Println("As HTTPError code:", he.Code)
 }

 fmt.Print("unwrap walk:")
 for e := err; e != nil; e = errors.Unwrap(e) {
 fmt.Printf(" -> %v", e)
 }
 fmt.Println()
}

```

#### Expected output:

```

chain: http 404: service.get(""): not found
Is NotFound: true
As HTTPError code: 404

```

```
unwrap walk: -> http 404: service.get(""): not found -> service.get(""): not found -> not fo
```

## More examples

errors.Join + Is: each joined error stays inspectable.

```
package main

import (
 "errors"
 "fmt"
)

var (
 ErrName = errors.New("missing name")
 ErrAge = errors.New("invalid age")
)

func validate(name string, age int) error {
 var errs error
 if name == "" {
 errs = errors.Join(errs, ErrName)
 }
 if age < 0 {
 errs = errors.Join(errs, ErrAge)
 }
 return errs
}

func main() {
 err := fmt.Errorf("signup: %w", validate("", -3))
 fmt.Println(err)
 fmt.Println("Is name:", errors.Is(err, ErrName))
 fmt.Println("Is age:", errors.Is(err, ErrAge))
 fmt.Println("Is notfound:", errors.Is(err, errors.New("not found")))
}
```

Expected output:

```
signup: missing name
invalid age
Is name: true
Is age: true
Is notfound: false
```



## Runnable example

Save as `main.go`. Then:

```
go mod init example
go run .

package main

import (
 "errors"
 "fmt"
)

var ErrNotFound = errors.New("not found")

type ValidationError struct {
 Field string
}

func (e *ValidationError) Error() string {
 return "validation: " + e.Field
}

type WrappedError struct {
 Context string
 Err error
}

func (e *WrappedError) Error() string {
 return fmt.Sprintf("%s: %v", e.Context, e.Err)
}

func (e *WrappedError) Unwrap() error { return e.Err }

func loadUser(id int) error {
 if id == 0 {
 return fmt.Errorf("user lookup: %w", ErrNotFound)
 }
 if id < 0 {
 return fmt.Errorf("processing: %w", &ValidationError{Field: "id"})
 }
 return nil
}

func main() {
```

```

err := loadUser(0)
fmt.Println("wrapped:", err)
fmt.Println("Is NotFound:", errors.Is(err, ErrNotFound))
fmt.Println("Unwrap once:", errors.Unwrap(err))

err = loadUser(-1)
var ve *ValidationError
if errors.As(err, &ve) {
 fmt.Println("As ValidationError field:", ve.Field)
}

// Custom Unwrap still participates in Is/As chains.
chain := &WrappedError{
 Context: "handler",
 Err: fmt.Errorf("service: %w", ErrNotFound),
}
fmt.Println("custom wrap:", chain)
fmt.Println("Is through custom wrap:", errors.Is(chain, ErrNotFound))
}

```

#### Expected output:

```

wrapped: user lookup: not found
Is NotFound: true
Unwrap once: not found
As ValidationError field: id
custom wrap: handler: service: not found
Is through custom wrap: true

```

**What to notice:** `%w` builds a chain; `errors.Is` walks that chain for sentinel equality, `errors.As` extracts a concrete type. Implementing `Unwrap()` error opts your type into the same machinery.

**Try next:** Nest three layers of `fmt.Errorf("%w")` and walk with a loop on `errors.Unwrap`; try `errors.Join` of two validation failures and inspect with `errors.Is`.

# panic, recover, and Failure Modes

## Overview

`panic` and `recover` handle unrecoverable errors. Unlike normal errors, panics crash the program unless recovered.

## When to Panic

**Appropriate:** - Programmer errors (bugs) - Impossible conditions - Failed invariants - During initialization

**Avoid for:** - Expected errors (file not found, network timeout) - User input validation - Anything recoverable

## Panic

```
func divide(a, b int) int {
 if b == 0 {
 panic("division by zero") // Terminates program
 }
 return a / b
}
```

## Common Panic Sources

```
// nil pointer dereference
var p *int
*p = 1 // panic

// Index out of range
arr := []int{1, 2}
arr[10] // panic

// Invalid type assertion
var i interface{} = "string"
i.(int) // panic
```

## Recover

Recover catches panics in deferred functions:

```
func safeCall() (err error) {
 defer func() {
 if r := recover(); r != nil {
 err = fmt.Errorf("panic recovered: %v", r)
 }
 }()

 dangerousOperation()
 return nil
}
```

## HTTP Server Pattern

```
func handler(w http.ResponseWriter, r *http.Request) {
 defer func() {
 if err := recover(); err != nil {
 log.Printf("panic: %v\n%s", err, debug.Stack())
 http.Error(w, "Internal Server Error", 500)
 }
 }()

 // Handle request
}
```

## Panic vs Error

```
// Return error for expected failures
func ReadFile(name string) ([]byte, error) {
 if !exists(name) {
 return nil, ErrNotFound // Expected case
 }
 // ...
}

// Panic for bugs
func MustCompile(pattern string) *Regexp {
 r, err := Compile(pattern)
 if err != nil {
 panic(err) // Invalid pattern is programmer error
 }
}
```

```

 }
 return r
}

```

## Summary

Keyword	Purpose
<b>panic</b>	Unrecoverable error (crash)
<b>recover</b>	Catch panic in defer
<b>error</b>	Expected, recoverable failures

Use panic for	Use error for
Bugs	Expected failures
Invariant violations	User/external input
Initialization failures	I/O, network errors

## Related Topics

- [Errors as Values](#)
- [Testing Fundamentals](#)

## Worked example

Boundary recover that turns a panic into an **error** (HTTP/worker style).

Save as `main.go`. Then:

```

go mod init example
go run .

package main

import (
 "fmt"
 "runtime/debug"
)

func protect(label string, fn func() error) (err error) {
 defer func() {
 if r := recover(); r != nil {

```

```

 // In real servers, log debug.Stack() and return a generic 500.
 _ = debug.Stack()
 err = fmt.Errorf("%s: panic: %v", label, r)
 }
}()
return fn()
}

func parseAge(s string) (int, error) {
 if s == "" {
 return 0, fmt.Errorf("empty age")
 }
 // Bug-style panic: index out of range on bad internal invariant.
 if s[0] == '!' {
 _ = []int{}[1]
 }
 n := 0
 for _, r := range s {
 if r < '0' || r > '9' {
 return 0, fmt.Errorf("bad digit in %q", s)
 }
 n = n*10 + int(r-'0')
 }
 return n, nil
}

func main() {
 err := protect("handler", func() error {
 age, err := parseAge("34")
 if err != nil {
 return err
 }
 fmt.Println("age:", age)
 return nil
 })
 fmt.Println("ok path err:", err)

 err = protect("handler", func() error {
 _, err := parseAge("")
 return err
 })
 fmt.Println("expected error:", err)

 err = protect("handler", func() error {
 _, err := parseAge("!boom")
 return err
 })
}

```

```
 fmt.Println("recovered panic:", err)
}
```

### Expected output:

```
age: 34
ok path err: <nil>
expected error: empty age
recovered panic: handler: panic: runtime error: index out of range [1] with length 0
```

## More examples

recover only works on the **same** goroutine that panics.

```
package main

import (
 "fmt"
 "sync"
)

func main() {
 var wg sync.WaitGroup

 // Wrong: recover in parent does not catch child panic.
 // We catch inside the child instead.
 wg.Add(1)
 go func() {
 defer wg.Done()
 defer func() {
 if r := recover(); r != nil {
 fmt.Println("child recovered:", r)
 }
 }()
 panic("child boom")
 }()
 wg.Wait()
 fmt.Println("parent still running")
}
```

### Expected output:

```
child recovered: child boom
parent still running
```

## Runnable example

Save as `main.go`. Then:

```
go mod init example
go run .

package main

import (
 "fmt"
 "regexp"
)

func mustCompile(pattern string) *regexp.Regexp {
 r, err := regexp.Compile(pattern)
 if err != nil {
 panic(err) // invalid pattern is a programmer error at init
 }
 return r
}

func safeCall(fn func()) (err error) {
 defer func() {
 if r := recover(); r != nil {
 err = fmt.Errorf("panic recovered: %v", r)
 }
 }()
 fn()
 return nil
}

func divide(a, b int) int {
 if b == 0 {
 panic("division by zero")
 }
 return a / b
}

func main() {
 re := mustCompile(`^\d+$`)
 fmt.Println("regex match 42:", re.MatchString("42"))

 err := safeCall(func() {
 fmt.Println("divide:", divide(10, 2))
 })
}
```



```

 fmt.Println("safe ok:", err)

 err = safeCall(func() {
 _ = divide(1, 0)
 })
 fmt.Println("safe recovered:", err)

 // Prefer error returns for expected failure modes.
 if !re.MatchString("xx") {
 fmt.Println("expected validation failure handled without panic")
 }
}

```

### Expected output:

```

regex match 42: true
divide: 5
safe ok: <nil>
safe recovered: panic recovered: division by zero
expected validation failure handled without panic

```

**What to notice:** `recover` only works inside deferred functions on the panicking goroutine. Convert panics to errors at boundaries (workers, HTTP handlers); keep normal validation on the **error** path.

**Try next:** Panic inside a new goroutine without `recover` and observe process exit; wrap a handler-style function that always recovers and logs.

## **The Must Pattern & No-GC Handling**

# Advanced Error Patterns: The “Must” & The “Odin” Way

Go’s error handling (`if err != nil`) is explicit, but sometimes you need different strategies for initialization vs. runtime, or you want to adopt patterns from systems languages like Odin/Zig.

## 1. The “Must” Pattern

**Rule:** Return errors to callers; Panic only on programmer error or unrecoverable startup failure.

The “Must” pattern is a convention for functions that wrap a standard error-returning function but `panic` instead of returning the error.

### When to use it?

- **Global/Package Initialization:** `var t = template.Must(template.New(...))`
- **Startup Configuration:** If your compiled regex is invalid, the app is broken. Don’t start.

### Implementation

```
func Must[T any](obj T, err error) T {
 if err != nil {
 panic(err)
 }
 return obj
}

// Usage
var db = Must(sql.Open("postgres", "..."))
var regex = Must(regexp.Compile(`^[a-z]+$`))
```

With Generics (Go 1.18+), you can write a universal `Must` wrapper one time and use it everywhere.

## 2. Linear Error Handling (The Odin/Zig Influence)

Languages like **Odin** (a data-oriented C alternative) handle errors by treating them as distinct return values, much like Go, but often with syntactic sugar (`or_return`) or strict definitions.

While Go doesn't have `try` or `?` operators, we can adopt the “**Guard Clause**” mentality.

### The Happy Path must remain left-aligned

Bad (Nested):

```
func process() error {
 err := step1()
 if err == nil {
 err = step2()
 if err == nil {
 return step3()
 }
 }
 return err
}
```

Good (Guard Clauses / “Odin Style”):

```
func process() error {
 if err := step1(); err != nil {
 return fmt.Errorf("step1: %w", err)
 }

 // The "Happy Path" stays at indentation 0
 if err := step2(); err != nil {
 return fmt.Errorf("step2: %w", err)
 }

 return step3()
}
```

### No-GC Thinking: Errors as Resource Cleanup Triggers

In non-GC languages, an error often implies manual resource cleanup (`defer` in Swift/Zig). Go automates memory, but not *resources* (Files, Sockets, DB Connections).

The `defer` Trap in Loops:

```
for _, file := range files {
 f, err := os.Open(file)
```

```

 if err != nil { return err }
 defer f.Close() // DANGEROUS: Closes only at end of function, not loop!
 // FD exhaustion possible.
}

```

### The Fix (Anonymous Function):

```

for _, file := range files {
 err := func() error {
 f, err := os.Open(file)
 if err != nil { return err }
 defer f.Close() // Closes at end of this anonymous func
 return process(f)
 }()
 if err != nil { return err }
}

```

## 3. Errors in 2026: “Join” and Structure

Since Go 1.20+, `errors.Join` allows returning *multiple* errors at once (e.g., from parallel validation).

```

func validate(u User) error {
 var errs error
 if u.Name == "" {
 errs = errors.Join(errs, errors.New("missing name"))
 }
 if u.Age < 0 {
 errs = errors.Join(errs, errors.New("invalid age"))
 }
 return errs // Returns nil if no errors joined
}

```

This is cleaner than older `multierror` libraries.

## Summary

1. **Must:** Use for hard startup dependencies. Crash early.
2. **Left-Align:** Keep your happy path on the left edge.
3. **Defer scope:** Remember `defer` is function-scoped, not block-scoped.

## Worked example

Guard-clause pipeline with `errors.Join` validation and scoped `defer` cleanup.

Save as `main.go`. Then:

```
go mod init example
go run .

package main

import (
 "errors"
 "fmt"
)

type Resource struct {
 Name string
 closed bool
}

func open(name string) (*Resource, error) {
 if name == "" {
 return nil, errors.New("empty name")
 }
 return &Resource{Name: name}, nil
}

func (r *Resource) Close() {
 r.closed = true
 fmt.Println("closed:", r.Name)
}

func (r *Resource) Work() error {
 if r.closed {
 return errors.New("use after close")
 }
 fmt.Println("work:", r.Name)
 return nil
}

func processAll(names []string) error {
 for _, name := range names {
 // Inner func scopes defer so each resource closes per iteration.
 if err := func() error {
 r, err := open(name)
 if err != nil {
```

```

 return fmt.Errorf("open %q: %w", name, err)
 }
 defer r.Close()
 return r.Work()
}(); err != nil {
 return err
}
}
return nil
}

func main() {
 if err := processAll([]string{"a.txt", "b.txt"}); err != nil {
 fmt.Println("process:", err)
 }

 var errs error
 errs = errors.Join(errs, errors.New("missing name"))
 errs = errors.Join(errs, errors.New("invalid age"))
 fmt.Println("joined:", errs)
}

```

Expected output:

```

work: a.txt
closed: a.txt
work: b.txt
closed: b.txt
joined: missing name
invalid age

```

## More examples

Generic Must for compile-time-valid startup data only.

```

package main

import (
 "fmt"
 "net/url"
 "regexp"
)

func Must[T any](v T, err error) T {
 if err != nil {

```

```

 panic(err)
 }
 return v
}

func main() {
 re := Must(regex.Compile(`^[a-z]+$`))
 u := Must(url.Parse("https://example.com/path"))
 fmt.Println("regex:", re.MatchString("gopher"))
 fmt.Println("host:", u.Host, "path:", u.Path)
}

```

**Expected output:**

```

regex: true
host: example.com path: /path

```

## Runnable example

Save as main.go. Then:

```

go mod init example
go run .

```

```

package main

import (
 "errors"
 "fmt"
 "regexp"
)

func Must[T any](v T, err error) T {
 if err != nil {
 panic(err)
 }
 return v
}

func openConfig(path string) (string, error) {
 if path == "" {
 return "", errors.New("empty path")
 }
 return "config:" + path, nil
}

```



```

func validate(name string, age int) error {
 var errs error
 if name == "" {
 errs = errors.Join(errs, errors.New("missing name"))
 }
 if age < 0 {
 errs = errors.Join(errs, errors.New("invalid age"))
 }
 return errs
}

func process() error {
 // Guard clauses keep the happy path left-aligned.
 cfg, err := openConfig("app.toml")
 if err != nil {
 return fmt.Errorf("step open: %w", err)
 }

 if err := validate("Ada", 36); err != nil {
 return fmt.Errorf("step validate: %w", err)
 }

 fmt.Println("happy path with", cfg)
 return nil
}

func main() {
 // Must: fine for static startup data that must be correct.
 re := Must(regex.Compile(`^[a-z]+$`))
 fmt.Println("must regex:", re.MatchString("gopher"))

 if err := process(); err != nil {
 fmt.Println("process:", err)
 }

 if err := validate("", -1); err != nil {
 fmt.Println("joined:", err)
 }

 // Recover a deliberate Must failure for demo purposes only.
 func() {
 defer func() {
 if r := recover(); r != nil {
 fmt.Println("must panicked as expected:", r)
 }
 }()
 }()
}

```

```

 _ = Must(openConfig(""))
 }()
}

```

### Expected output:

```

must regex: true
happy path with config:app.toml
joined: missing name
invalid age
must panicked as expected: empty path

```

**What to notice:** Generic `Must` collapses `(T, error)` for irrecoverable init. `errors.Join` reports multiple validation problems at once. Guard clauses avoid nested `if err == nil` pyramids.

**Try next:** Move `Must(regex.Compile(...))` to package level `var`; add a loop that opens several pseudo-files with an inner `func() error { defer close... }` so `defer` runs per iteration.

# Testing

# Testing Fundamentals

## Overview

Go has built-in testing support. Test files end with `_test.go` and use the `testing` package.

## Writing Tests

```
// math.go
package math

func Add(a, b int) int {
 return a + b
}

// math_test.go
package math

import "testing"

func TestAdd(t *testing.T) {
 result := Add(2, 3)
 if result != 5 {
 t.Errorf("Add(2, 3) = %d; want 5", result)
 }
}
```

## Running Tests

```
go test # Current package
go test ./... # All packages
go test -v # Verbose
go test -run TestAdd # Specific test
```

## Test Functions

```
func TestXxx(t *testing.T) // Test
func BenchmarkXxx(b *testing.B) // Benchmark
func ExampleXxx() // Example
func FuzzXxx(f *testing.F) // Fuzz test
```

## t.Error vs t.Fatal

```
func TestExample(t *testing.T) {
 // Continues after failure
 t.Error("failed but continuing")

 // Stops test immediately
 t.Fatal("failed, stopping now")
}
```

## Helper Functions

```
func assertEqual(t *testing.T, got, want int) {
 t.Helper() // Marks this as helper
 if got != want {
 t.Errorf("got %d, want %d", got, want)
 }
}

func TestMath(t *testing.T) {
 assertEqual(t, Add(1, 2), 3)
}
```

## Setup and Teardown

```
func TestMain(m *testing.M) {
 // Setup
 setup()

 code := m.Run() // Run tests

 // Teardown
 teardown()
}
```

```
 os.Exit(code)
}
```

## Parallel Tests

```
func TestParallel(t *testing.T) {
 t.Parallel() // Run in parallel
 // Test code
}
```

## Summary

Command	Purpose
<code>go test</code>	Run tests
<code>go test -v</code>	Verbose output
<code>go test -cover</code>	Show coverage
<code>go test -race</code>	Race detection

## Related Topics

- [Test Organization and Coverage](#)
- [Advanced Testing](#)

## Worked example

`t.Helper`, `Error` vs `Fatal`, and `TestMain` setup/teardown.

Save as `strutil.go` and `strutil_test.go`. Then:

```
go mod init example
go test -v

// strutil.go
package main

import "strings"

func TitleWords(s string) string {
 parts := strings.Fields(s)
```

```

 for i, p := range parts {
 if p == "" {
 continue
 }
 parts[i] = strings.ToUpper(p[:1]) + strings.ToLower(p[1:])
 }
 return strings.Join(parts, " ")
}

// strutil_test.go
package main

import (
 "fmt"
 "os"
 "testing"
)

func TestMain(m *testing.M) {
 fmt.Println("setup: suite start")
 code := m.Run()
 fmt.Println("teardown: suite end")
 os.Exit(code)
}

func assertEq(t *testing.T, got, want string) {
 t.Helper()
 if got != want {
 t.Errorf("got %q want %q", got, want)
 }
}

func TestTitleWords(t *testing.T) {
 assertEq(t, TitleWords("hello world"), "Hello World")
 assertEq(t, TitleWords("gO"), "Go")
}

func TestTitleWordsFatalStops(t *testing.T) {
 if TitleWords("") != "" {
 t.Fatal("empty should stay empty") // would stop this test
 }
 // Continues only if Fatal did not fire.
 assertEq(t, TitleWords("a b"), "A B")
}

```

**Expected output:**

```

setup: suite start
=== RUN TestTitleWords
--- PASS: TestTitleWords (0.00s)
=== RUN TestTitleWordsFatalStops
--- PASS: TestTitleWordsFatalStops (0.00s)
PASS
teardown: suite end

```

## More examples

Parallel top-level tests (`t.Parallel`) on independent pure functions.

```

// parallel_test.go
package main

import "testing"

func TestTitleHello(t *testing.T) {
 t.Parallel()
 if TitleWords("hello") != "Hello" {
 t.Fatal("bad title")
 }
}

func TestTitleWorld(t *testing.T) {
 t.Parallel()
 if TitleWords("world") != "World" {
 t.Fatal("bad title")
 }
}

go test -v -parallel 4

```

## Runnable example

Save these two files in a directory (for example `mathx/`). Then:

```

cd mathx
go mod init example/mathx
go test -v
go test -cover

```

`mathx.go`:



```

package mathx

func Add(a, b int) int {
 return a + b
}

func Div(a, b int) (int, error) {
 if b == 0 {
 return 0, errDivZero
 }
 return a / b, nil
}

var errDivZero = errString("division by zero")

type errString string

func (e errString) Error() string { return string(e) }

```

mathx\_test.go:

```

package mathx

import "testing"

func assertEqual(t *testing.T, got, want int) {
 t.Helper()
 if got != want {
 t.Errorf("got %d, want %d", got, want)
 }
}

func TestAdd(t *testing.T) {
 assertEqual(t, Add(2, 3), 5)
 assertEqual(t, Add(-1, 1), 0)
}

func TestDiv(t *testing.T) {
 got, err := Div(10, 2)
 if err != nil {
 t.Fatalf("unexpected err: %v", err)
 }
 assertEqual(t, got, 5)

 _, err = Div(1, 0)
 if err == nil {
 t.Fatal("expected error for divide by zero")
 }
}

```

```

 }
}

func TestAddParallel(t *testing.T) {
 t.Parallel()
 assertEquals(t, Add(10, 5), 15)
}

```

**Expected output:** (verbose)

```

=== RUN TestAdd
--- PASS: TestAdd (0.00s)
=== RUN TestDiv
--- PASS: TestDiv (0.00s)
=== RUN TestAddParallel
--- PASS: TestAddParallel (0.00s)
PASS
ok example/mathx 0.00xs

```

**What to notice:** Tests live in `*_test.go`, use `TestXxx(*testing.T)`, and fail with `t.Error` / `t.Fatal`. `t.Helper()` makes failure lines point at the call site, not the helper body.

**Try next:** Add `go test -run TestDiv -v`; introduce a deliberate bug in `Add` and watch the assertion fail; try `t.Error` vs `t.Fatal` after a failed check.

# Test Organization and Coverage

## Overview

Well-organized tests and good coverage help maintain code quality over time.

## File Organization

```
mypackage/
 user.go # Implementation
 user_test.go # Tests
 user_internal_test.go # Internal tests
 testdata/ # Test fixtures
 users.json
```

## Package Naming

```
// Same package (white-box testing)
package mypackage

// External package (black-box testing)
package mypackage_test
```

## Testdata Directory

```
func TestParseConfig(t *testing.T) {
 data, err := os.ReadFile("testdata/config.json")
 if err != nil {
 t.Fatal(err)
 }
 // Use data
}
```

## Code Coverage

```
Show coverage percentage
go test -cover

Generate coverage profile
go test -coverprofile=coverage.out

View in browser
go tool cover -html=coverage.out

Show coverage by function
go tool cover -func=coverage.out
```

## Coverage Modes

```
Statement coverage (default)
go test -covermode=set

Count mode (how many times)
go test -covermode=count

Atomic (for concurrent tests)
go test -covermode=atomic
```

## Test Groups

```
func TestUser(t *testing.T) {
 t.Run("Create", func(t *testing.T) {
 // Test creation
 })

 t.Run("Update", func(t *testing.T) {
 // Test update
 })

 t.Run("Delete", func(t *testing.T) {
 // Test deletion
 })
}
```

## Skipping Tests

```
func TestIntegration(t *testing.T) {
 if testing.Short() {
 t.Skip("skipping in short mode")
 }
 // Long test
}
```

```
go test -short # Skip long tests
```

## Summary

Flag	Purpose
-cover	Show coverage %
-coverprofile=file	Generate profile
-short	Skip long tests
-run=Pattern	Run matching tests

## Related Topics

- [Testing Fundamentals](#)
- [Advanced Testing](#)

## Worked example

White-box vs black-box packages and `testdata` fixtures side by side.

Save under `greet/`. Then:

```
cd greet
go mod init example/greet
go test -v ./...
go test -cover
```

```
// greet.go
package greet

import "fmt"
```

```

func Hello(name string) string {
 if name == "" {
 name = defaultName()
 }
 return fmt.Sprintf("Hello, %s!", name)
}

func defaultName() string { return "world" }

// greet_internal_test.go - white-box (same package)
package greet

import "testing"

func TestDefaultName(t *testing.T) {
 if defaultName() != "world" {
 t.Fatalf("got %q", defaultName())
 }
}

// greet_test.go - black-box (external test package)
package greet_test

import (
 "os"
 "strings"
 "testing"

 "example/greet"
)

func TestHelloFromFixture(t *testing.T) {
 data, err := os.ReadFile("testdata/names.txt")
 if err != nil {
 t.Fatal(err)
 }
 for _, name := range strings.Split(strings.TrimSpace(string(data)), "\n") {
 got := greet.Hello(name)
 if !strings.Contains(got, name) {
 t.Errorf("Hello(%q)=%q", name, got)
 }
 }
}

func TestHelloEmpty(t *testing.T) {
 if greet.Hello("") != "Hello, world!" {

```

```

 t.Fatal(greet.Hello(""))
 }
}

```

testdata/names.txt:

```

Ada
Grace

```

**Expected output:** (go test -v)

```

=== RUN TestDefaultName
--- PASS: TestDefaultName (0.00s)
=== RUN TestHelloFromFixture
--- PASS: TestHelloFromFixture (0.00s)
=== RUN TestHelloEmpty
--- PASS: TestHelloEmpty (0.00s)
PASS

```

## More examples

Grouped subtests + `-short` skip for slow suites.

```

// suite_test.go
package greet_test

import (
 "testing"

 "example/greet"
)

func TestHelloSuite(t *testing.T) {
 t.Run("named", func(t *testing.T) {
 if greet.Hello("Lin") != "Hello, Lin!" {
 t.Fatal("named")
 }
 })
 t.Run("default", func(t *testing.T) {
 if greet.Hello("") != "Hello, world!" {
 t.Fatal("default")
 }
 })
 t.Run("slow", func(t *testing.T) {
 if testing.Short() {

```

```

 t.Skip("skipping slow case")
 }
 // pretend expensive integration work
 _ = greet.Hello("slow")
})
}

```

```
go test -short -v -run TestHelloSuite
```

## Runnable example

Save these files under `mathx/`. Create `testdata/cases.txt` as shown. Then:

```

cd mathx
go mod init example/mathx
go test -v
go test -cover
go test -short -v
go test -coverprofile=coverage.out && go tool cover -func=coverage.out

```

`mathx.go`:

```

package mathx

import (
 "strconv"
 "strings"
)

func Add(a, b int) int { return a + b }

// ParseSum reads lines of "a+b" and returns the sums.
func ParseSum(data string) ([]int, error) {
 lines := strings.Split(strings.TrimSpace(data), "\n")
 out := make([]int, 0, len(lines))
 for _, line := range lines {
 if line == "" {
 continue
 }
 parts := strings.Split(line, "+")
 if len(parts) != 2 {
 return nil, errString("bad line: " + line)
 }
 a, err := strconv.Atoi(strings.TrimSpace(parts[0]))
 if err != nil {

```



```

 return nil, err
 }
 b, err := strconv.Atoi(strings.TrimSpace(parts[1]))
 if err != nil {
 return nil, err
 }
 out = append(out, Add(a, b))
}
return out, nil
}

type errString string

func (e errString) Error() string { return string(e) }

```

mathx\_test.go (black-box: external test package):

```

package mathx_test

import (
 "os"
 "testing"

 "example/mathx"
)

func TestAdd(t *testing.T) {
 t.Run("positive", func(t *testing.T) {
 if got := mathx.Add(2, 3); got != 5 {
 t.Fatalf("got %d", got)
 }
 })
 t.Run("negative", func(t *testing.T) {
 if got := mathx.Add(-2, -3); got != -5 {
 t.Fatalf("got %d", got)
 }
 })
}

func TestParseSumFromTestdata(t *testing.T) {
 data, err := os.ReadFile("testdata/cases.txt")
 if err != nil {
 t.Fatal(err)
 }
 sums, err := mathx.ParseSum(string(data))
 if err != nil {
 t.Fatal(err)
 }
}

```

```

 }
 want := []int{5, 0, 9}
 if len(sums) != len(want) {
 t.Fatalf("len=%d want %d", len(sums), len(want))
 }
 for i := range want {
 if sums[i] != want[i] {
 t.Errorf("sums[%d]=%d want %d", i, sums[i], want[i])
 }
 }
}

func TestSlow(t *testing.T) {
 if testing.Short() {
 t.Skip("skipping slow test in short mode")
 }
 // Pretend this is an expensive integration-style check.
 if mathx.Add(100, 1) != 101 {
 t.Fatal("unexpected")
 }
}

```

testdata/cases.txt:

```

2+3
-1+1
4+5

```

**Expected output:** (go test -v without -short)

```

=== RUN TestAdd
=== RUN TestAdd/positive
=== RUN TestAdd/negative
--- PASS: TestAdd (0.00s)
=== RUN TestParseSumFromTestdata
--- PASS: TestParseSumFromTestdata (0.00s)
=== RUN TestSlow
--- PASS: TestSlow (0.00s)
PASS

```

With go test -short -v, TestSlow reports SKIP.

**What to notice:** package mathx\_test only sees exported API (black-box). testdata/ is the conventional place for fixtures. Subtests (t.Run) organize cases; -short skips long work.

**Try next:** Generate an HTML coverage report with go tool cover -html=coverage.out; add an internal white-box test file with package mathx that exercises an unexported helper.

# Advanced Testing

## Overview

Table-driven tests, subtests, and test helpers make Go tests maintainable and expressive.

Fast feedback test loop

unit tests ----->	deterministic concurrency tests ----->	benchmarks
<1s target	race-safe	track regressions

## Table-Driven Tests

```
func TestAdd(t *testing.T) {
 tests := []struct {
 name string
 a, b int
 expected int
 }{
 {"positive", 2, 3, 5},
 {"negative", -1, -2, -3},
 {"zero", 0, 0, 0},
 {"mixed", -1, 5, 4},
 }

 for _, tt := range tests {
 t.Run(tt.name, func(t *testing.T) {
 result := Add(tt.a, tt.b)
 if result != tt.expected {
 t.Errorf("Add(%d, %d) = %d; want %d",
 tt.a, tt.b, result, tt.expected)
 }
 })
 }
}
```

## Subtests

```
func TestAPI(t *testing.T) {
 t.Run("GET", func(t *testing.T) {
 t.Run("success", func(t *testing.T) { })
 t.Run("not_found", func(t *testing.T) { })
 })

 t.Run("POST", func(t *testing.T) {
 t.Run("valid", func(t *testing.T) { })
 t.Run("invalid", func(t *testing.T) { })
 })
}
```

## Test Helpers

```
func newTestServer(t *testing.T) *httptest.Server {
 t.Helper()
 return httptest.NewServer(http.HandlerFunc(func(w http.ResponseWriter, r *http.Request)
 w.Write([]byte("OK"))
)))
}

func TestClient(t *testing.T) {
 server := newTestServer(t)
 defer server.Close()
 // Use server.URL
}
```

## Cleanup

```
func TestWithCleanup(t *testing.T) {
 tmpDir := t.TempDir() // Auto-removed

 f, _ := os.CreateTemp(tmpDir, "test")
 t.Cleanup(func() {
 f.Close()
 })
}
```

## Deterministic Concurrency (testing/synctest)

Since Go 1.25, `testing/synctest` provides virtualized time and goroutine bubble scheduling. Time advances instantly when all goroutines in the `synctest.Run` bubble are blocked on timers or channel operations, eliminating artificial `time.Sleep` in tests and eliminating flaky timing race conditions.

```
flow:
 T -> G (Spawn Worker with 10s Timer)
 G -> B (Sleep 10s (Goroutine Blocks))
 B -> G (Wake up immediately (0 wall-clock ms elapsed))
 G -> T (Send completion signal)
 T -> B (synctest.Wait() (Verifies all goroutines settled))

import (
 "testing"
 "testing/synctest"
 "time"
)

func TestRateLimiter(t *testing.T) {
 synctest.Run(func() {
 rl := NewRateLimiter(1 * time.Second)
 // Time is virtual and controlled within synctest.Run
 if !rl.Allow() { t.Error("should allow first request") }
 if rl.Allow() { t.Error("should block immediate second request") }

 // Advance virtual time instantly without waiting real seconds
 time.Sleep(1 * time.Second)

 if !rl.Allow() { t.Error("should allow request after virtual 1s window") }
 synctest.Wait() // Wait for all bubble goroutines to settle
 })
}
```

Go 1.27 adds `synctest.Sleep(d)`, which is `time.Sleep(d)` plus `synctest.Wait()`: advance the fake clock, then wait until the bubble is settled.

```
synctest.Test(t, func(t *testing.T) {
 go worker()
 synctest.Sleep(time.Second) // jump clock + wait for reactions
})
```

Tip: keep business logic outside goroutine setup so `synctest.Run` / `synctest.Test` stays small and focused.

## Efficient Benchmarks (B.Loop)

Prefer B.Loop in modern toolchains for cleaner benchmark loops and fewer loop-control mistakes.

```
func BenchmarkAdd(b *testing.B) {
 for b.Loop() {
 Add(1, 2)
 }
}
```

Migration pattern:

```
// old
func BenchmarkParseOld(b *testing.B) {
 for i := 0; i < b.N; i++ {
 Parse(input)
 }
}

// new
func BenchmarkParse(b *testing.B) {
 for b.Loop() {
 Parse(input)
 }
}
```

## Go 1.27 Testing Upgrade Checklist

1. Use `t.Cleanup`, `t.TempDir`, and `t.Setenv` instead of manual teardown.
2. Move flaky concurrency tests into `testing/synctest`; prefer `synctest.Sleep` over wall-clock sleeps.
3. Standardize benchmark style on `B.Loop`.
4. For HTTP tests that need a fake network (especially inside `synctest`), use `httptest.NewTestServer(t, handler)` instead of `NewServer`.
5. Run strict CI test command:

```
go test ./... -race -shuffle=on -count=1
```

## Parallel Subtests

```
func TestParallel(t *testing.T) {
 tests := []struct{ name string }{"a"}, {"b"}, {"c"}}

 for _, tt := range tests {
 t.Run(tt.name, func(t *testing.T) {
 t.Parallel()
 // Runs concurrently
 })
 }
}
```

## Benchmarks

```
func BenchmarkAdd(b *testing.B) {
 for i := 0; i < b.N; i++ {
 Add(1, 2)
 }
}
```

```
go test -bench=.
go test -bench=. -benchmem # Include memory
```

## Summary

Technique	Purpose
Table-driven	Test multiple cases
Subtests	Organize and filter
<code>t.Helper()</code>	Clean error reporting
<code>t.Cleanup()</code>	Guaranteed cleanup
<code>t.Parallel()</code>	Concurrent tests

## Related Topics

- [Testing Fundamentals](#)
- [Integration Testing](#)

## Worked example

Table-driven tests with nested subtests and shared helpers.

Save as `clamp.go` and `clamp_test.go`. Then:

```
go mod init example
go test -v
go test -run 'TestClamp/high' -v

// clamp.go
package main

func Clamp(v, lo, hi int) int {
 if lo > hi {
 lo, hi = hi, lo
 }
 if v < lo {
 return lo
 }
 if v > hi {
 return hi
 }
 return v
}

func InRange(v, lo, hi int) bool {
 return Clamp(v, lo, hi) == v
}

// clamp_test.go
package main

import "testing"

func TestClamp(t *testing.T) {
 tests := []struct {
 name string
 v, lo, hi int
 want int
 }{
 {"inside", 5, 0, 10, 5},
 {"low", -1, 0, 10, 0},
 {"high", 99, 0, 10, 10},
 {"swapped bounds", 5, 10, 0, 5},
 }
}
```



```

 for _, tt := range tests {
 t.Run(tt.name, func(t *testing.T) {
 if got := Clamp(tt.v, tt.lo, tt.hi); got != tt.want {
 t.Fatalf("Clamp(%d,%d,%d)=%d want %d", tt.v, tt.lo, tt.hi, got, tt.want)
 }
 })
 }
}

func TestInRangeTable(t *testing.T) {
 t.Run("true", func(t *testing.T) {
 cases := []int{0, 5, 10}
 for _, v := range cases {
 if !InRange(v, 0, 10) {
 t.Fatalf("%d should be in range", v)
 }
 }
 })
 t.Run("false", func(t *testing.T) {
 if InRange(-1, 0, 10) || InRange(11, 0, 10) {
 t.Fatal("out of range reported true")
 }
 })
}

```

### Expected output:

```

=== RUN TestClamp
=== RUN TestClamp/inside
=== RUN TestClamp/low
=== RUN TestClamp/high
=== RUN TestClamp/swapped_bounds
--- PASS: TestClamp (0.00s)
=== RUN TestInRangeTable
=== RUN TestInRangeTable/true
=== RUN TestInRangeTable/false
--- PASS: TestInRangeTable (0.00s)
PASS

```

## More examples

Benchmarks comparing algorithms (`-benchmem`).

```

// clamp_bench_test.go
package main

```

```

import "testing"

func minMaxClamp(v, lo, hi int) int {
 if v < lo {
 return lo
 }
 if v > hi {
 return hi
 }
 return v
}

func BenchmarkClamp(b *testing.B) {
 for i := 0; i < b.N; i++ {
 _ = Clamp(i%20, 0, 10)
 }
}

func BenchmarkMinMaxClamp(b *testing.B) {
 for i := 0; i < b.N; i++ {
 _ = minMaxClamp(i%20, 0, 10)
 }
}

go test -bench=. -benchmem

```

## Runnable example

Save these two files in `mathx/`. Then:

```

cd mathx
go mod init example/mathx
go test -v
go test -bench='BenchmarkAdd$' -benchmem

```

`mathx.go`:

```

package mathx

func Add(a, b int) int { return a + b }

func Max(a, b int) int {
 if a > b {
 return a
 }
}

```

```

 }
 return b
}

```

mathx\_test.go:

```

package mathx

import (
 "os"
 "path/filepath"
 "testing"
)

func TestAddTable(t *testing.T) {
 tests := []struct {
 name string
 a, b int
 expected int
 }{
 {"positive", 2, 3, 5},
 {"negative", -1, -2, -3},
 {"zero", 0, 0, 0},
 {"mixed", -1, 5, 4},
 }
 for _, tt := range tests {
 t.Run(tt.name, func(t *testing.T) {
 if got := Add(tt.a, tt.b); got != tt.expected {
 t.Errorf("Add(%d,%d)=%d; want %d", tt.a, tt.b, got, tt.expected)
 }
 })
 }
}

func TestMaxParallel(t *testing.T) {
 cases := []struct {
 name string
 a, b, want int
 }{
 {"a", 1, 2, 2},
 {"b", 5, 5, 5},
 {"c", -3, -1, -1},
 }
 for _, tc := range cases {
 tc := tc // capture (harmless; required before Go 1.22 loop scoping)
 t.Run(tc.name, func(t *testing.T) {
 t.Parallel()

```

```

 if got := Max(tc.a, tc.b); got != tc.want {
 t.Fatalf("got %d want %d", got, tc.want)
 }
 })
}

func TestWithCleanup(t *testing.T) {
 dir := t.TempDir() // removed automatically after the test
 path := filepath.Join(dir, "note.txt")
 if err := writeFile(t, path, "ok"); err != nil {
 t.Fatal(err)
 }
 t.Cleanup(func() {
 // Extra hooks run LIFO before TempDir removal.
 })
 data, err := os.ReadFile(path)
 if err != nil {
 t.Fatal(err)
 }
 if string(data) != "ok" {
 t.Fatalf("got %q", data)
 }
}

func writeFile(t *testing.T, path, body string) error {
 t.Helper()
 return os.WriteFile(path, []byte(body), 0o600)
}

func BenchmarkAdd(b *testing.B) {
 for i := 0; i < b.N; i++ {
 Add(1, 2)
 }
}

```

**Expected output:** (go test -v)

```

=== RUN TestAddTable
=== RUN TestAddTable/positive
=== RUN TestAddTable/negative
=== RUN TestAddTable/zero
=== RUN TestAddTable/mixed
--- PASS: TestAddTable (0.00s)
=== RUN TestMaxParallel
=== RUN TestMaxParallel/a
=== RUN TestMaxParallel/b

```

```
=== RUN TestMaxParallel/c
--- PASS: TestMaxParallel (0.00s)
=== RUN TestWithCleanup
--- PASS: TestWithCleanup (0.00s)
PASS
```

Benchmark numbers vary by machine:

```
BenchmarkAdd-8 ns/op 0 B/op 0 allocs/op
```

**What to notice:** Table-driven `t.Run` names show up in failures and `-run` filters. `t.Parallel` runs subtests concurrently. `t.TempDir` / `t.Cleanup` replace manual teardown. Classic `b.N` benchmarks work everywhere; prefer `b.Loop()` on newer toolchains when available.

**Try next:** Filter with `go test -run 'TestAddTable/mixed' -v`; break one table case and read the subtest name in the failure; try `go test -count=1 -shuffle=on`.

# Integration and System Testing

## Overview

Integration tests verify components work together, testing against real databases, APIs, and external services.

Test layers

```
unit (many, fast)
|
integration (fewer, realistic deps)
|
system/e2e (fewest, highest confidence)
```

## HTTP Testing

```
import "net/http/httptest"

func TestHandler(t *testing.T) {
 req := httptest.NewRequest("GET", "/users", nil)
 w := httptest.NewRecorder()

 handler(w, req)

 if w.Code != http.StatusOK {
 t.Errorf("status = %d; want 200", w.Code)
 }
}
```

## Test Server

```
func TestAPIClient(t *testing.T) {
 // Go 1.27+: in-memory network, auto-cleanup, synctest-safe.
 server := httptest.NewTestServer(t, http.HandlerFunc(
 func(w http.ResponseWriter, r *http.Request) {
```

```

 w.Write([]byte(`{"id": 1}`))
)))

 resp, err := server.Client().Get("http://example.com/users/1")
 // Assert...
}

```

`NewTestServer` uses an in-memory fake network (no loopback port), registers `t.Cleanup` to shut the server down, and fails the test if the handler panics. Prefer it over `httptest.NewServer` for new tests. `NewServer` still listens on loopback when you need a real TCP address.

## Database Testing

```

func TestUserRepository(t *testing.T) {
 if testing.Short() {
 t.Skip("skipping integration test")
 }

 db := setupTestDB(t)
 t.Cleanup(func() { db.Close() })

 repo := NewUserRepository(db)

 // Test operations
 err := repo.Create(&User{Name: "test"})
 if err != nil {
 t.Fatal(err)
 }
}

```

## Build Tags

```

//go:build integration
// +build integration

package myapp_test

// Only runs with: go test -tags=integration

```

## Environment-Based Tests

```
func TestWithEnv(t *testing.T) {
 url := os.Getenv("API_URL")
 if url == "" {
 t.Skip("API_URL not set")
 }
 // Test against real API
}
```

## Docker Integration

```
func TestWithDocker(t *testing.T) {
 if testing.Short() {
 t.Skip("skipping docker test")
 }

 // Use testcontainers-go or similar
 container, err := startPostgres()
 if err != nil {
 t.Fatal(err)
 }
 t.Cleanup(func() { container.Terminate(ctx) })

 // Run tests against container
}
```

## Go 1.27 Integration Testing Upgrades

Use a two-lane CI strategy:

1. Default lane (every push): fast unit + short integration.
2. Full lane (main/nightly): DB containers, external API contract checks, and race detector.

```
default
go test ./... -short -shuffle=on

full
go test ./... -tags=integration -race -count=1
```

For external APIs, always assert status + schema shape, not only status codes:



```

func assertUserShape(t *testing.T, body []byte) {
 t.Helper()
 var got map[string]any
 if err := json.Unmarshal(body, &got); err != nil {
 t.Fatal(err)
 }
 if _, ok := got["id"]; !ok {
 t.Fatal("missing id")
 }
}

```

## Summary

Tool	Use Case
<code>httptest.NewTestServer</code>	In-memory HTTP (Go 1.27+; default for new tests)
<code>httptest.NewServer</code>	Loopback TCP when you need a real URL
Build tags	Separate test suites
Env vars	External dependencies
<code>testing.Short()</code>	Skip slow tests

## Related Topics

- [Testing Fundamentals](#)
- [Advanced Testing](#)

## Worked example

Handler unit test with `httptest.NewRecorder` plus client test against `httptest.NewServer`.

Save as `api.go` and `api_test.go`. Then:

```

go mod init example
go test -v

```

```

// api.go
package main

import (
 "encoding/json"
 "net/http"
)

```

```

type Item struct {
 ID string `json:"id"`
 Name string `json:"name"`
}

func ItemHandler(w http.ResponseWriter, r *http.Request) {
 if r.Method != http.MethodGet {
 http.Error(w, "method not allowed", http.StatusMethodNotAllowed)
 return
 }
 id := r.PathValue("id")
 if id == "" {
 http.Error(w, "missing id", http.StatusBadRequest)
 return
 }
 w.Header().Set("Content-Type", "application/json")
 _ = json.NewEncoder(w).Encode(Item{ID: id, Name: "widget-" + id})
}

func NewAPI() http.Handler {
 mux := http.NewServeMux()
 mux.HandleFunc("GET /items/{id}", ItemHandler)
 return mux
}

// api_test.go
package main

import (
 "encoding/json"
 "io"
 "net/http"
 "net/http/httptest"
 "testing"
)

func TestItemHandlerRecorder(t *testing.T) {
 req := httptest.NewRequest(http.MethodGet, "/items/7", nil)
 req.SetPathValue("id", "7")
 rec := httptest.NewRecorder()
 ItemHandler(rec, req)

 if rec.Code != http.StatusOK {
 t.Fatalf("status=%d", rec.Code)
 }
 var item Item

```

```

 if err := json.Unmarshal(rec.Body.Bytes(), &item); err != nil {
 t.Fatal(err)
 }
 if item.ID != "7" || item.Name == "" {
 t.Fatalf("bad body: %+v", item)
 }
}

func TestItemClientServer(t *testing.T) {
 srv := httptest.NewServer(NewAPI())
 t.Cleanup(srv.Close)

 res, err := http.Get(srv.URL + "/items/9")
 if err != nil {
 t.Fatal(err)
 }
 defer res.Body.Close()
 body, _ := io.ReadAll(res.Body)
 if res.StatusCode != 200 {
 t.Fatalf("status=%d body=%s", res.StatusCode, body)
 }
 var item Item
 if err := json.Unmarshal(body, &item); err != nil {
 t.Fatal(err)
 }
 if item.ID != "9" {
 t.Fatalf("id=%s", item.ID)
 }
}
}

```

### Expected output:

```

=== RUN TestItemHandlerRecorder
--- PASS: TestItemHandlerRecorder (0.00s)
=== RUN TestItemClientServer
--- PASS: TestItemClientServer (0.00s)
PASS

```

## More examples

Env-gated “external” test and method-not-allowed check.

```

// api_extra_test.go
package main

```

```

import (
 "net/http"
 "net/http/httptest"
 "os"
 "testing"
)

func TestPostRejected(t *testing.T) {
 req := httptest.NewRequest(http.MethodPost, "/items/1", nil)
 req.SetPathValue("id", "1")
 rec := httptest.NewRecorder()
 ItemHandler(rec, req)
 if rec.Code != http.StatusMethodNotAllowed {
 t.Fatalf("status=%d", rec.Code)
 }
}

func TestExternalURL_SkipIfUnset(t *testing.T) {
 if os.Getenv("API_URL") == "" {
 t.Skip("API_URL not set")
 }
 // Would hit a real service when configured.
 t.Log("API_URL=", os.Getenv("API_URL"))
}

go test -v
API_URL=http://example.invalid go test -v -run External

```

## Runnable example

Save these two files in `usersvc/`. Then:

```

cd usersvc
go mod init example/usersvc
go test -v
go test -short -v

```

`handler.go`:

```

package usersvc

import (
 "encoding/json"
 "net/http"
 "strings"

```

```

)

type User struct {
 ID int `json:"id"`
 Name string `json:"name"`
}

func UserHandler(w http.ResponseWriter, r *http.Request) {
 if r.Method != http.MethodGet {
 http.Error(w, "method not allowed", http.StatusMethodNotAllowed)
 return
 }
 id := strings.TrimPrefix(r.URL.Path, "/users/")
 if id == "" || id == r.URL.Path {
 http.Error(w, "missing id", http.StatusBadRequest)
 return
 }
 w.Header().Set("Content-Type", "application/json")
 _ = json.NewEncoder(w).Encode(User{ID: 1, Name: "Ada"})
}

func NewMux() http.Handler {
 mux := http.NewServeMux()
 mux.HandleFunc("/users/", UserHandler)
 return mux
}

```

handler\_test.go:

```

package usersvc_test

import (
 "encoding/json"
 "io"
 "net/http"
 "net/http/httptest"
 "testing"

 "example/usersvc"
)

func TestUserHandler(t *testing.T) {
 req := httptest.NewRequest(http.MethodGet, "/users/1", nil)
 rec := httptest.NewRecorder()

 usersvc.UserHandler(rec, req)
}

```

```

 res := rec.Result()
 defer res.Body.Close()
 if res.StatusCode != http.StatusOK {
 t.Fatalf("status=%d", res.StatusCode)
 }
 body, _ := io.ReadAll(res.Body)
 assertUserShape(t, body)
}

func TestAPIClientAgainstTestServer(t *testing.T) {
 srv := httptest.NewServer(usersvc.NewMux())
 t.Cleanup(srv.Close)

 res, err := http.Get(srv.URL + "/users/1")
 if err != nil {
 t.Fatal(err)
 }
 defer res.Body.Close()
 if res.StatusCode != http.StatusOK {
 t.Fatalf("status=%d", res.StatusCode)
 }
 body, err := io.ReadAll(res.Body)
 if err != nil {
 t.Fatal(err)
 }
 assertUserShape(t, body)
}

func TestExternalAPI_SkipUnlessConfigured(t *testing.T) {
 if testing.Short() {
 t.Skip("skipping external-style check in short mode")
 }
 // In real suites you might also require API_URL; here we keep it offline.
 req := httptest.NewRequest(http.MethodGet, "/users/1", nil)
 rec := httptest.NewRecorder()
 usersvc.UserHandler(rec, req)
 if rec.Code != http.StatusOK {
 t.Fatalf("status=%d", rec.Code)
 }
}

func assertUserShape(t *testing.T, body []byte) {
 t.Helper()
 var got map[string]any
 if err := json.Unmarshal(body, &got); err != nil {
 t.Fatal(err)
 }
}

```

```

 if _, ok := got["id"]; !ok {
 t.Fatal("missing id")
 }
 if _, ok := got["name"]; !ok {
 t.Fatal("missing name")
 }
}

```

### Expected output:

```

=== RUN TestUserHandler
--- PASS: TestUserHandler (0.00s)
=== RUN TestAPIClientAgainstTestServer
--- PASS: TestAPIClientAgainstTestServer (0.00s)
=== RUN TestExternalAPI_SkipUnlessConfigured
--- PASS: TestExternalAPI_SkipUnlessConfigured (0.00s)
PASS

```

With `go test -short -v`, the last test is `SKIP`.

**What to notice:** `httptest` exercises handlers without a listening port; `httptest.NewServer` gives a real URL for client code. Assert JSON **shape** (keys/types), not only status codes. Use `-short` / `env` gates for slower dependency lanes.

**Try next:** Add a `POST` path that returns `405` and test it; introduce a build-tagged file `//go:build integration` that only runs under `go test -tags=integration`.

## **Fuzzing and Property-Style Tests**



# Fuzzing and Property-Style Tests

## Overview

Fuzzing feeds random inputs to find panics and logic bugs. Go's built-in fuzzing (`go test -fuzz`) is ideal for parsers, validators, and codecs.

## Fuzz target

```
func FuzzParsePort(f *testing.F) {
 f.Add("8080")
 f.Add("0")
 f.Add("65535")
 f.Fuzz(func(t *testing.T, s string) {
 n, err := strconv.Atoi(s)
 if err != nil {
 return
 }
 if n < 0 {
 t.Fatalf("negative: %d", n)
 }
 // property: round-trip for valid ports
 if n >= 0 && n <= 65535 {
 if strconv.Itoa(n) != s && !strings.HasPrefix(s, "0") && s != "0" {
 // allow leading zeros mismatch - define your contract
 }
 }
 })
}
```

```
go test -fuzz=FuzzParsePort -fuzztime=10s
```

## Seed corpus

`f.Add` seeds interesting cases; failing inputs land in `testdata/fuzz/`.

## Good fuzz targets

Good	Poor
JSON/decode, path clean, URL parse wrappers	Tests needing full DB
Pure functions	Non-deterministic time without injection

## Property ideas

- Round-trip: `decode(encode(x)) == x`
- Idempotent clean: `clean(clean(p)) == clean(p)`
- Never panics on any byte slice

## Rules of thumb

Do	Don't
Fix corpus failures as tests	Delete corpus to “go green”
Bound work in fuzz body	Sleep/network in fuzz
Run in CI with <code>-fuzztime</code> short	Only fuzz manually once

## Try next

1. Fuzz your `safeJoin` — should never escape root.
2. Fuzz JSON validator.
3. Commit a minimized failing corpus once fixed.

## **httptest and API Testing Patterns**

# httptest and API Testing Patterns

## Overview

`net/http/httptest` tests handlers without a real network. Combine with table tests and fake dependencies for fast API coverage.

## Handler unit test

```
func TestHealth(t *testing.T) {
 req := httptest.NewRequest(http.MethodGet, "/healthz", nil)
 rec := httptest.NewRecorder()
 healthz(rec, req)
 if rec.Code != 200 {
 t.Fatalf("code %d", rec.Code)
 }
}
```

## Full mux

```
srv := NewServer(fakeStore)
req := httptest.NewRequest(http.MethodGet, "/api/books", nil)
rec := httptest.NewRecorder()
srv.Handler().ServeHTTP(rec, req)
```

## JSON helpers

```
func decodeJSON[T any](t *testing.T, rec *httptest.ResponseRecorder) T {
 t.Helper()
 var v T
 if err := json.NewDecoder(rec.Body).Decode(&v); err != nil {
 t.Fatal(err)
 }
 return v
}
```

## Auth cookies

```
req.AddCookie(&http.Cookie{Name: "session_id", Value: sid})
```

## httptest.Server (integration-ish)

```
// Go 1.27+: in-memory network, t.Cleanup, syncctest-safe.
ts := httptest.NewTestServer(t, handler)
res, err := ts.Client().Get("http://example.com/api/books")
```

Prefer `NewTestServer` for new tests. `httptest.NewServer` still listens on loopback when you need a real TCP URL:

```
ts := httptest.NewServer(handler)
t.Cleanup(ts.Close)
res, err := http.Get(ts.URL + "/api/books")
```

Still in-process; good middle ground before Testcontainers.

## Rules of thumb

Do	Don't
Fake stores at interfaces	Real network in unit tests
Table-drive status codes	One mega test for all routes
<code>t.Helper</code> on asserts	Ignore body on 500s

## Try next

1. Table test 200/400/401 for create book.
2. Cookie session test.
3. Assert `Content-Type` on JSON APIs.

## Golden Files and testdata

# Golden Files and testdata

## Overview

**Golden** files store expected output (JSON, text, CLI stdout). Update deliberately when format changes.

## Layout

```
mypkg/
 format.go
 format_test.go
 testdata/
 report.golden
 cases/
 in.json
```

## Pattern

```
func TestReport(t *testing.T) {
 got := RenderReport(sample)
 path := filepath.Join("testdata", "report.golden")
 if os.Getenv("UPDATE_GOLDEN") == "1" {
 if err := os.WriteFile(path, got, 0o644); err != nil {
 t.Fatal(err)
 }
 }
 want, err := os.ReadFile(path)
 if err != nil {
 t.Fatal(err)
 }
 if !bytes.Equal(got, want) {
 t.Fatalf("mismatch (-want +got)\n%s", diff(want, got))
 }
}
```

```
UPDATE_GOLDEN=1 go test ./...
go test ./...
```

## Tips

- Normalize time/randomness (inject clock)
- Prefer stable JSON key order (`json.Encoder` / sorted maps)
- Review golden diffs in PRs carefully

## embed for fixtures

```
//go:embed testdata/*.json
var fixtures embed.FS
```

## Rules of thumb

Do	Don't
Commit goldens	Regenerate without reading diff
Keep fixtures small	50MB binaries in git
Document <code>UPDATE_GOLDEN</code>	Hidden magic env names only in CI lore

## Try next

1. Golden-test a CLI help string.
2. Golden-test pretty JSON error envelope.
3. Break format; see test fail; update consciously.



## **Project: Service Test Suite**

# Project: Service Test Suite

## Overview

For Bookstore (or any API), build a layered suite.

## Layers

Layer	Tools	Speed
Domain unit	table tests	ms
Handler	httptest + fakes	ms
Fuzz	validators/parsers	CI timed
Golden	error JSON / reports	ms
Integration	optional Testcontainers	slower

## Deliverables

- ☐ `internal/domain/*_test.go`
- ☐ `internal/httpapi/*_test.go` with auth cases
- ☐ One fuzz target
- ☐ One golden file
- ☐ `make test` → `go test ./...`
- ☐ `make test-race` → `go test -race ./...`

## Definition of done

`go test ./...` green; race clean; README documents how to run fuzz for 10s.

# **Concurrency and Parallelism**

# Concurrency Basics

## Overview

Go's concurrency model uses goroutines (lightweight threads) and channels for communication.

## Goroutines

```
go func() {
 // Runs concurrently
 fmt.Println("Hello from goroutine")
}()

go processData() // Named function
```

Goroutines are cheap (~2KB stack, can have millions).

## Creating Goroutines

```
func main() {
 go sayHello()
 go sayWorld()
 time.Sleep(100 * time.Millisecond) // Wait (not ideal)
}

func sayHello() { fmt.Println("Hello") }
func sayWorld() { fmt.Println("World") }
```

## Waiting with WaitGroup

```
var wg sync.WaitGroup

for i := 0; i < 5; i++ {
 wg.Add(1)
 go func(n int) {
```

```

 defer wg.Done()
 fmt.Println(n)
 }(i)
}

wg.Wait() // Block until all done

```

## Channels

```

ch := make(chan int) // Unbuffered
ch := make(chan int, 10) // Buffered

ch <- 42 // Send
value := <-ch // Receive

```

## Basic Channel Pattern

```

func main() {
 ch := make(chan string)

 go func() {
 ch <- "Hello"
 }()

 msg := <-ch
 fmt.Println(msg)
}

```

## Worker Pool

```

func worker(id int, jobs <-chan int, results chan<- int) {
 for job := range jobs {
 results <- job * 2
 }
}

func main() {
 jobs := make(chan int, 100)
 results := make(chan int, 100)

 // Start workers

```

```

 for w := 0; w < 3; w++ {
 go worker(w, jobs, results)
 }

 // Send jobs
 for j := 0; j < 10; j++ {
 jobs <- j
 }
 close(jobs)

 // Collect results
 for r := 0; r < 10; r++ {
 fmt.Println(<-results)
 }
}

```

## Summary

Concept	Purpose
<code>go func()</code>	Start goroutine
<code>sync.WaitGroup</code>	Wait for completion
<code>make(chan T)</code>	Create channel
<code>ch &lt;- / &lt;-ch</code>	Send/receive

## Worked example

Fan of goroutines joined with `WaitGroup` and a results channel (no `Sleep`).

Save as `main.go`. Then:

```

go mod init example
go run .

package main

import (
 "fmt"
 "sync"
)

func main() {
 const n = 5
 results := make(chan int, n)

```

```

var wg sync.WaitGroup

for i := 1; i <= n; i++ {
 wg.Add(1)
 go func(x int) {
 defer wg.Done()
 results <- x * x
 }(i)
}

go func() {
 wg.Wait()
 close(results)
}()

sum := 0
for v := range results {
 sum += v
}
fmt.Println("sum of squares 1..5:", sum)
}

```

**Expected output:**

```
sum of squares 1..5: 55
```

## More examples

Buffered vs unbuffered: buffer decouples a single send from receive.

```

package main

import "fmt"

func main() {
 // Unbuffered needs a receiver ready (other goroutine).
 u := make(chan string)
 go func() { u <- "unbuffered" }()
 fmt.Println(<-u)

 // Buffered send can complete without a concurrent receiver.
 b := make(chan string, 1)
 b <- "buffered"
 fmt.Println(<-b)
}

```

## Expected output:

```
unbuffered
buffered
```

## Runnable example

Save as main.go. Then:

```
go mod init example
go run .

package main

import (
 "fmt"
 "sync"
)

func main() {
 // 1) Goroutines + WaitGroup (no Sleep)
 var wg sync.WaitGroup
 for i := 1; i <= 3; i++ {
 wg.Add(1)
 go func(n int) {
 defer wg.Done()
 fmt.Printf("goroutine %d done\n", n)
 }(i)
 }
 wg.Wait()
 fmt.Println("all goroutines finished")

 // 2) Unbuffered channel: send happens in another goroutine
 ch := make(chan string)
 go func() {
 ch <- "hello from channel"
 }()
 fmt.Println(<-ch)

 // 3) Tiny worker pool: jobs → workers → results
 jobs := make(chan int, 5)
 results := make(chan int, 5)

 const workers = 2
 var pool sync.WaitGroup
```



```

 for w := 1; w <= workers; w++ {
 pool.Add(1)
 go func(id int) {
 defer pool.Done()
 for job := range jobs {
 results <- job * 2
 }
 }(w)
 }

 for j := 1; j <= 5; j++ {
 jobs <- j
 }
 close(jobs)

 go func() {
 pool.Wait()
 close(results)
 }()

 sum := 0
 for r := range results {
 sum += r
 }
 fmt.Println("worker pool sum:", sum)
}

```

**Expected output:** (goroutine lines may appear in any order)

```

goroutine 1 done
goroutine 2 done
goroutine 3 done
all goroutines finished
hello from channel
worker pool sum: 30

```

**What to notice:** `WaitGroup` replaces `time.Sleep` for joining. Closing `jobs` lets workers exit range; closing `results` after `pool.Wait()` ends the collector cleanly.

**Try next:** Change the channel buffer sizes and watch when sends block; run with `go run -race .` to confirm the example is race-free.

## Related Topics

- [Channel Patterns](#)
- [Synchronization](#)

# Channel Patterns

## Overview

Channels enable safe communication between goroutines. This chapter covers essential channel patterns.

## Channel Directions

```
chan T // Bidirectional
chan<- T // Send-only
<-chan T // Receive-only

func producer(out chan<- int) { }
func consumer(in <-chan int) { }
```

## Buffered vs Unbuffered

```
// Unbuffered: send blocks until receive
ch := make(chan int)

// Buffered: send blocks when full
ch := make(chan int, 10)
```

## Closing Channels

```
close(ch)

// Check if closed
v, ok := <-ch
if !ok {
 // Channel closed
}

// Range over channel
```

```

for v := range ch {
 // Receives until closed
}

```

## Select

```

select {
case v := <-ch1:
 fmt.Println("from ch1:", v)
case v := <-ch2:
 fmt.Println("from ch2:", v)
case ch3 <- x:
 fmt.Println("sent to ch3")
default:
 fmt.Println("no channel ready")
}

```

## Timeout Pattern

```

select {
case result := <-ch:
 fmt.Println(result)
case <-time.After(1 * time.Second):
 fmt.Println("timeout")
}

```

## Done Channel

```

done := make(chan struct{})

go func() {
 // Work...
 close(done) // Signal completion
}()

<-done // Wait for signal

```

## Fan-Out / Fan-In Architecture

The **Fan-Out** pattern distributes work across multiple worker goroutines reading from a single input channel. The **Fan-In** pattern consolidates the results from multiple worker channels back into a single output stream.

```
flow:
] -> JobsCh[
 JobsCh -> Worker1[
 JobsCh -> Worker2[
 JobsCh -> Worker3[
 Worker1 -> Out1[
 Worker2 -> Out2[
 Worker3 -> Out3[
 Out1 -> Multiplexer[
 Out2 -> Multiplexer
 Out3 -> Multiplexer
 Multiplexer -> FinalCh[

// Worker function processing jobs from input channel
func worker(id int, jobs <-chan int) <-chan int {
 out := make(chan int)
 go func() {
 defer close(out)
 for j := range jobs {
 // Process item (e.g. compute square)
 out <- j * j
 }
 }()
 return out
}

// Fan-in: merge multiple worker output channels into one aggregated channel
func fanIn(channels ...<-chan int) <-chan int {
 out := make(chan int)
 var wg sync.WaitGroup

 for _, ch := range channels {
 wg.Add(1)
 go func(c <-chan int) {
 defer wg.Done()
 for v := range c {
 out <- v
 }
 }(ch)
 }
}
```

```

 go func() {
 wg.Wait()
 close(out)
 }()
 return out
}

```

## Summary

Pattern	Use Case
Unbuffered	Synchronization
Buffered	Decouple speed
Select	Multiple channels
Done channel	Cancellation signal
Fan-out/in	Parallel processing

## Worked example

select with default (non-blocking try) and a multi-channel race.

Save as `main.go`. Then:

```

go mod init example
go run .

package main

import (
 "fmt"
 "time"
)

func main() {
 ch := make(chan int, 1)

 // Non-blocking receive
 select {
 case v := <-ch:
 fmt.Println("got", v)
 default:
 fmt.Println("empty: default branch")
 }
}

```

```

ch <- 7
select {
case v := <-ch:
 fmt.Println("got", v)
default:
 fmt.Println("unexpected default")
}

// First ready wins
a := make(chan string, 1)
b := make(chan string, 1)
a <- "from-a"
select {
case msg := <-a:
 fmt.Println("select:", msg)
case msg := <-b:
 fmt.Println("select:", msg)
case <-time.After(50 * time.Millisecond):
 fmt.Println("select: timeout")
}
}

```

**Expected output:**

```

empty: default branch
got 7
select: from-a

```

## More examples

Or-done pattern: stop ranging when an external `done` closes. Producer also selects on `done` so nothing hangs.

```

package main

import "fmt"

func orDone(done <-chan struct{}, in <-chan int) <-chan int {
 out := make(chan int)
 go func() {
 defer close(out)
 for {
 select {
 case <-done:
 return

```

```

 case v, ok := <-in:
 if !ok {
 return
 }
 select {
 case out <- v:
 case <-done:
 return
 }
 }
 }()
 return out
}

func main() {
 done := make(chan struct{})
 in := make(chan int)

 go func() {
 defer close(in)
 for i := 1; i <= 100; i++ {
 select {
 case <-done:
 return
 case in <- i:
 }
 }
 }()

 out := orDone(done, in)
 count := 0
 for v := range out {
 fmt.Println("v:", v)
 count++
 if count == 3 {
 close(done)
 }
 }
 fmt.Println("stopped early at", count)
}

```

**Expected output:**

```

v: 1
v: 2
v: 3

```

stopped early at 3

## Runnable example

Save as `main.go`. Then:

```
go mod init example
go run .
```

```
package main

import (
 "fmt"
 "sync"
 "time"
)

func worker(id int, jobs <-chan int) <-chan int {
 out := make(chan int)
 go func() {
 defer close(out)
 for j := range jobs {
 // Simulate work; square the job id.
 out <- j * j
 }
 }()
 return out
}

func fanIn(channels ...<-chan int) <-chan int {
 out := make(chan int)
 var wg sync.WaitGroup
 for _, ch := range channels {
 wg.Add(1)
 go func(c <-chan int) {
 defer wg.Done()
 for v := range c {
 out <- v
 }
 }(ch)
 }
 go func() {
 wg.Wait()
 close(out)
 }()
}
```



```

 return out
}

func main() {
 // Timeout via select
 slow := make(chan string)
 go func() {
 time.Sleep(50 * time.Millisecond)
 slow <- "late result"
 }()
 select {
 case msg := <-slow:
 fmt.Println("got:", msg)
 case <-time.After(10 * time.Millisecond):
 fmt.Println("timeout: waited 10ms")
 }

 // Done channel signal
 done := make(chan struct{})
 go func() {
 // pretend work finished
 close(done)
 }()
 <-done
 fmt.Println("done channel closed")

 // Fan-out / fan-in
 jobs := make(chan int)
 go func() {
 for i := 1; i <= 4; i++ {
 jobs <- i
 }
 close(jobs)
 }()

 out1 := worker(1, jobs)
 out2 := worker(2, jobs)
 merged := fanIn(out1, out2)

 sum := 0
 for v := range merged {
 sum += v
 }
 // 12+22+32+42 = 30
 fmt.Println("fan-in sum of squares:", sum)
}

```

Expected output:

```
timeout: waited 10ms
done channel closed
fan-in sum of squares: 30
```

**What to notice:** `select` with `time.After` implements a one-shot timeout. Fan-out shares one jobs channel across workers; fan-in merges their outputs and closes only after every input channel is drained.

**Try next:** Make the timeout larger than 50ms so the slow send wins the `select`. Add a third worker and re-run.

## Related Topics

- [Concurrency Basics](#)
- [Context and Cancellation](#)

# Synchronization

## Overview

The `sync` package provides low-level synchronization primitives for coordinating goroutines.

### `sync.Mutex`

```
var (
 mu sync.Mutex
 count int
)

func increment() {
 mu.Lock()
 count++
 mu.Unlock()
}

// With defer
func safe() {
 mu.Lock()
 defer mu.Unlock()
 // Critical section
}
```

### `sync.RWMutex`

```
var (
 mu sync.RWMutex
 data map[string]string
)

func read(key string) string {
 mu.RLock()
 defer mu.RUnlock()
 return data[key]
}
```

```

}

func write(key, value string) {
 mu.Lock()
 defer mu.Unlock()
 data[key] = value
}

```

## sync.WaitGroup

```

var wg sync.WaitGroup

for i := 0; i < 5; i++ {
 wg.Add(1)
 go func(n int) {
 defer wg.Done()
 work(n)
 }(i)
}

wg.Wait()

```

## sync.Once

```

var (
 once sync.Once
 config *Config
)

func getConfig() *Config {
 once.Do(func() {
 config = loadConfig() // Runs exactly once
 })
 return config
}

```

## sync.Pool

```
var pool = sync.Pool{
 New: func() any {
 return make([]byte, 1024)
 },
}

func process() {
 buf := pool.Get().([]byte)
 defer pool.Put(buf)
 // Use buf
}
```

## sync.Map

```
var m sync.Map

m.Store("key", "value")
v, ok := m.Load("key")
m.Delete("key")
m.Range(func(k, v any) bool {
 fmt.Println(k, v)
 return true // Continue iteration
})
```

## atomic Package

```
import "sync/atomic"

var counter int64

atomic.AddInt64(&counter, 1)
value := atomic.LoadInt64(&counter)
atomic.StoreInt64(&counter, 0)
```

## Summary

Type	Purpose
Mutex	Exclusive lock
RWMutex	Reader/writer lock
WaitGroup	Wait for goroutines
Once	Single execution
Pool	Object reuse
Map	Concurrent map

## Worked example

RWMutex cache: many readers, occasional writer.

Save as main.go. Then:

```

go mod init example
go run .

package main

import (
 "fmt"
 "sync"
)

type Cache struct {
 mu sync.RWMutex
 data map[string]int
}

func (c *Cache) Get(k string) (int, bool) {
 c.mu.RLock()
 defer c.mu.RUnlock()
 v, ok := c.data[k]
 return v, ok
}

func (c *Cache) Set(k string, v int) {
 c.mu.Lock()
 defer c.mu.Unlock()
 c.data[k] = v
}

func main() {
 c := &Cache{data: map[string]int{}}
 var wg sync.WaitGroup

```

```

// Writers
for i := 0; i < 10; i++ {
 wg.Add(1)
 go func(n int) {
 defer wg.Done()
 c.Set(fmt.Sprintf("k%d", n%3), n)
 }(i)
}
wg.Wait()

// Readers
for i := 0; i < 3; i++ {
 wg.Add(1)
 go func(n int) {
 defer wg.Done()
 k := fmt.Sprintf("k%d", n)
 v, ok := c.Get(k)
 fmt.Printf("get %s -> %d ok=%v\n", k, v, ok)
 }(i)
}
wg.Wait()
}

```

**Expected output:** (values vary; keys present)

```

get k0 -> ... ok=true
get k1 -> ... ok=true
get k2 -> ... ok=true

```

## More examples

`sync.Map` for disjoint keys; `atomic` for a simple counter.

```

package main

import (
 "fmt"
 "sync"
 "sync/atomic"
)

func main() {
 var m sync.Map
 var hits atomic.Int64
 var wg sync.WaitGroup

```

```

 for i := 0; i < 100; i++ {
 wg.Add(1)
 go func(n int) {
 defer wg.Done()
 m.Store(n%5, n)
 hits.Add(1)
 }(i)
 }
 wg.Wait()

 count := 0
 m.Range(func(_, _ any) bool {
 count++
 return true
 })
 fmt.Println("keys:", count, "hits:", hits.Load())
}

```

Expected output:

```
keys: 5 hits: 100
```

## Runnable example

Save as `main.go`. Then:

```

go mod init example
go run .

package main

import (
 "fmt"
 "sync"
 "sync/atomic"
)

func main() {
 // Mutex-protected counter
 var (
 mu sync.Mutex
 count int
 wg sync.WaitGroup
)

```



```

for i := 0; i < 1000; i++ {
 wg.Add(1)
 go func() {
 defer wg.Done()
 mu.Lock()
 count++
 mu.Unlock()
 }()
}
wg.Wait()
fmt.Println("mutex count:", count)

// Once: initialization runs exactly once
var (
 once sync.Once
 inited int
)
for i := 0; i < 5; i++ {
 wg.Add(1)
 go func() {
 defer wg.Done()
 once.Do(func() {
 inited++
 fmt.Println("once.Do ran")
 })
 }()
}
wg.Wait()
fmt.Println("init count:", inited)

// atomic counter (no mutex)
var atomicCount int64
for i := 0; i < 1000; i++ {
 wg.Add(1)
 go func() {
 defer wg.Done()
 atomic.AddInt64(&atomicCount, 1)
 }()
}
wg.Wait()
fmt.Println("atomic count:", atomic.LoadInt64(&atomicCount))

// Pool: reuse a buffer
pool := sync.Pool{
 New: func() any {
 return make([]byte, 8)
 },
}

```

```
}
buf := pool.Get().([]byte)
copy(buf, []byte("go-pool!"))
fmt.Println("pool buffer:", string(buf))
pool.Put(buf)
}
```

#### Expected output:

```
mutex count: 1000
once.Do ran
init count: 1
atomic count: 1000
pool buffer: go-pool!
```

**What to notice:** Without `mu.Lock` / `atomic`, the counters would race. `sync.Once` guarantees single execution even under concurrent callers. `sync.Pool` is for short-lived reuse—never assume a put buffer is still yours after `Put`.

**Try next:** Remove the mutex around `count++` and run `go run -race .` to watch the race detector fire.

## Related Topics

- [Concurrency Basics](#)
- [Concurrency Design Guidelines](#)

# Context and Cancellation

## Overview

The `context` package manages deadlines, cancellation, and request-scoped values across API boundaries.

## Creating Contexts

```
ctx := context.Background() // Root context
ctx := context.TODO() // Placeholder
```

## Context Hierarchy & Downward Cancellation

Contexts form an immutable tree. When a parent context is canceled or times out, all child contexts derived from it are automatically canceled. However, canceling a child context does **not** affect the parent or siblings.

```
flow:
] -> ReqCtx[
ReqCtx -> DBTimeout[
ReqCtx -> RPCCancel[
RPCCancel -> SubWorker1[
RPCCancel -> SubWorker2[

ctx, cancel := context.WithCancel(context.Background())
defer cancel()

go func() {
 select {
 case <-ctx.Done():
 fmt.Println("cancelled:", ctx.Err())
 return
 case <-time.After(time.Hour):
 fmt.Println("completed")
 }
}()
```

```
cancel() // Signal cancellation downwards to all derived contexts
```

## Timeout

```
ctx, cancel := context.WithTimeout(context.Background(), 5*time.Second)
defer cancel()

select {
case result := <-doWork(ctx):
 fmt.Println(result)
case <-ctx.Done():
 fmt.Println("timeout:", ctx.Err())
}
```

## Deadline

```
deadline := time.Now().Add(10 * time.Second)
ctx, cancel := context.WithDeadline(context.Background(), deadline)
defer cancel()
```

## Passing Context

```
func handleRequest(ctx context.Context) {
 result, err := fetchData(ctx)
 if err != nil {
 return
 }
 processData(ctx, result)
}

func fetchData(ctx context.Context) (Data, error) {
 req, _ := http.NewRequestWithContext(ctx, "GET", url, nil)
 return http.DefaultClient.Do(req)
}
```

## Context Values

```
type key string
const userKey key = "user"

ctx := context.WithValue(ctx, userKey, "alice")

user := ctx.Value(userKey).(string)
```

## Best Practices

- Pass context as first parameter
- Never store context in structs
- Always call cancel() (use defer)
- Only use context values for request-scoped data

## Summary

Function	Purpose
WithCancel	Manual cancellation
WithTimeout	Duration-based timeout
WithDeadline	Absolute time limit
WithValue	Request-scoped data

## Worked example

Context-aware worker that respects cancel and timeout.

Save as `main.go`. Then:

```
go mod init example
go run .
```

```
package main
```

```
import (
 "context"
 "fmt"
 "sync"
 "time"
)
```

```

func worker(ctx context.Context, id int, out chan<- string) {
 select {
 case <-time.After(30 * time.Millisecond):
 out <- fmt.Sprintf("worker-%d done", id)
 case <-ctx.Done():
 out <- fmt.Sprintf("worker-%d %v", id, ctx.Err())
 }
}

func main() {
 ctx, cancel := context.WithTimeout(context.Background(), 10*time.Millisecond)
 defer cancel()

 out := make(chan string, 2)
 var wg sync.WaitGroup
 for i := 1; i <= 2; i++ {
 wg.Add(1)
 go func(id int) {
 defer wg.Done()
 worker(ctx, id, out)
 }(i)
 }
 wg.Wait()
 close(out)
 for msg := range out {
 fmt.Println(msg)
 }
}

```

**Expected output:** (both lines show deadline exceeded)

```

worker-1 context deadline exceeded
worker-2 context deadline exceeded

```

## More examples

Propagate request IDs with typed `WithValue` keys.

```

package main

import (
 "context"
 "fmt"
)

```

```

type key int

const requestID key = 1

func handle(ctx context.Context) {
 fmt.Println("request_id:", ctx.Value(requestID))
}

func main() {
 ctx := context.WithValue(context.Background(), requestID, "req-100")
 // Child inherits values.
 child, cancel := context.WithCancel(ctx)
 defer cancel()
 handle(child)
}

```

**Expected output:**

```
request_id: req-100
```

## Runnable example

Save as main.go. Then:

```

go mod init example
go run .

```

```

package main

import (
 "context"
 "fmt"
 "time"
)

type ctxKey string

const requestIDKey ctxKey = "request_id"

func work(ctx context.Context, name string, d time.Duration) error {
 select {
 case <-time.After(d):
 fmt.Printf("%s finished (request_id=%v)\n", name, ctx.Value(requestIDKey))
 return nil
 case <-ctx.Done():

```

```

 fmt.Printf("%s cancelled: %v\n", name, ctx.Err())
 return ctx.Err()
 }
}

func main() {
 // WithValue: request-scoped data flows downward
 root := context.WithValue(context.Background(), requestIDKey, "req-42")

 // Manual cancel
 ctx, cancel := context.WithCancel(root)
 go func() {
 time.Sleep(20 * time.Millisecond)
 cancel()
 }()
 _ = work(ctx, "cancel-demo", 200*time.Millisecond)

 // Timeout: child inherits parent values
 tctx, tcancel := context.WithTimeout(root, 15*time.Millisecond)
 defer tcancel()
 _ = work(tctx, "timeout-demo", 200*time.Millisecond)

 // Success path: finishes before deadline
 okCtx, okCancel := context.WithTimeout(root, 100*time.Millisecond)
 defer okCancel()
 _ = work(okCtx, "ok-demo", 5*time.Millisecond)

 // Parent cancel cancels children
 parent, pCancel := context.WithCancel(root)
 child, childCancel := context.WithCancel(parent)
 defer childCancel()
 pCancel()
 select {
 case <-child.Done():
 fmt.Println("child saw parent cancel:", child.Err())
 case <-time.After(time.Second):
 fmt.Println("child did not cancel (unexpected)")
 }
}

```

### Expected output:

```

cancel-demo cancelled: context canceled
timeout-demo cancelled: context deadline exceeded
ok-demo finished (request_id=req-42)
child saw parent cancel: context canceled

```



**What to notice:** Always `defer cancel()` (or call it when done) so timers and resources are released. Cancellation propagates **down** the tree; canceling a child never cancels its parent. Prefer typed keys for `WithValue`, not bare strings.

**Try next:** Swap `WithTimeout` for `WithDeadline(time.Now().Add(...))`. Pass the cancelled context into a second goroutine and confirm both exit.

## Related Topics

- [Channel Patterns](#)
- [HTTP and Networking](#)

# Concurrency Design Guidelines

## Overview

Concurrency can introduce subtle bugs. Follow these guidelines to write safe concurrent code.

## Share by Communicating

```
// Prefer: communicate over channels
result := <-worker.Results()

// Avoid: shared memory with locks
mu.Lock()
result := sharedData
mu.Unlock()
```

## Avoid Goroutine Leaks

```
// Bad: goroutine leaks if nobody receives
go func() {
 ch <- result // Blocks forever
}()

// Good: use context cancellation
go func() {
 select {
 case ch <- result:
 case <-ctx.Done():
 }
}()
```

## Always Close Channels from Sender

```
// Producer closes
func producer(out chan<- int) {
 for i := 0; i < 10; i++ {
 out <- i
 }
 close(out) // Only sender closes
}

// Consumer ranges
for v := range in {
 process(v)
}
```

## Race Detection

```
go test -race ./...
go run -race main.go
```

## Common Patterns

### Worker Pool

```
jobs := make(chan Job)
for i := 0; i < workers; i++ {
 go worker(jobs)
}
```

### Bounded Concurrency

```
sem := make(chan struct{}, maxConcurrent)

for _, task := range tasks {
 sem <- struct{}{}
 go func(t Task) {
 defer func() { <-sem }()
 process(t)
 }(task)
}
```

## Deadlock Prevention

```
// Deadlock: circular wait
ch := make(chan int)
ch <- 1 // Blocks: no receiver
<-ch // Never reached

// Fix: buffer or separate goroutine
ch := make(chan int, 1)
ch <- 1
<-ch
```

## Summary

Guideline	Reason
Share by communicating	Clearer ownership
Use context for cancellation	Prevent leaks
Only sender closes	Avoid panic
Run race detector	Catch data races

## Worked example

Ownership: only the sender closes; consumers range safely.

Save as `main.go`. Then:

```
go mod init example
go run .
go run -race .

package main

import (
 "fmt"
 "sync"
)

func produce(n int) <-chan int {
 out := make(chan int)
 go func() {
 defer close(out) // sender closes
 for i := 1; i <= n; i++ {
```

```

 out <- i
 }
}()
return out
}

func main() {
 in := produce(5)
 var wg sync.WaitGroup
 sum := 0
 var mu sync.Mutex

 // Fan-out consumers; channel close ends all ranges.
 for c := 0; c < 2; c++ {
 wg.Add(1)
 go func() {
 defer wg.Done()
 for v := range in {
 mu.Lock()
 sum += v
 mu.Unlock()
 }
 }()
 }
 wg.Wait()
 fmt.Println("sum 1..5:", sum)
}

```

**Expected output:**

```
sum 1..5: 15
```

## More examples

Detect a deliberate data race (educational—expect FAIL under `-race`).

```

package main

import (
 "fmt"
 "sync"
)

func main() {
 // Correct version first.

```

```

var (
 mu sync.Mutex
 count int
 wg sync.WaitGroup
)
for i := 0; i < 100; i++ {
 wg.Add(1)
 go func() {
 defer wg.Done()
 mu.Lock()
 count++
 mu.Unlock()
 }()
}
wg.Wait()
fmt.Println("safe count:", count)
}

```

Expected output:

```
safe count: 100
```

## Runnable example

Save as main.go. Then:

```

go mod init example
go run .

```

```
package main
```

```

import (
 "context"
 "fmt"
 "sync"
 "time"
)

```

```

// safeSend never leaks a goroutine when the consumer walks away.
func safeSend(ctx context.Context, ch chan<- int, v int) {
 select {
 case ch <- v:
 case <-ctx.Done():
 // drop result; exit without blocking forever
 }
}

```

```

}

func main() {
 // Bounded concurrency with a semaphore channel
 const maxConcurrent = 2
 sem := make(chan struct{}, maxConcurrent)
 var wg sync.WaitGroup

 tasks := []string{"a", "b", "c", "d", "e"}
 var mu sync.Mutex
 var order []string

 for _, t := range tasks {
 wg.Add(1)
 go func(task string) {
 defer wg.Done()
 sem <- struct{}{} // acquire
 defer func() { <-sem }() // release

 // critical/limited work
 mu.Lock()
 order = append(order, task)
 mu.Unlock()
 }(t)
 }
 wg.Wait()
 fmt.Println("processed tasks:", len(order))

 // Avoid goroutine leak: context cancels a blocked send
 ctx, cancel := context.WithTimeout(context.Background(), 30*time.Millisecond)
 defer cancel()

 ch := make(chan int) // unbuffered, no receiver → would block
 var senderWG sync.WaitGroup
 senderWG.Add(1)
 go func() {
 defer senderWG.Done()
 safeSend(ctx, ch, 99)
 fmt.Println("sender exited without leak")
 }()
 senderWG.Wait()

 // Only the sender closes; consumer ranges until closed
 out := make(chan int, 3)
 go func() {
 for i := 1; i <= 3; i++ {
 out <- i
 }
 }()
}

```

```

 }
 close(out) // sender closes
}()
sum := 0
for v := range out {
 sum += v
}
fmt.Println("ranged sum:", sum)
}

```

### Expected output:

```

processed tasks: 5
sender exited without leak
ranged sum: 6

```

**What to notice:** The semaphore bounds *execution*, not just intention. `select` on `ctx.Done()` is the standard way to stop a blocked send/receive. Closing from the receiver (or twice) panics—ownership stays with the sender.

**Try next:** Run `go run -race ..` Temporarily remove the `ctx.Done()` case and see the program hang until you kill it.

## Related Topics

- [Synchronization](#)
- [Context and Cancellation](#)



# Worker Pools

# Worker Pools: Controlling Chaos

Go makes it easy to spawn 100,000 goroutines. The OS makes it easy to crash when you open 100,000 file descriptors.

**Unbounded concurrency is a bug.** You must limit parallelism to match your resource limits (CPU, Memory, Network/DB Connections).

## Pattern 1: The Semaphore (Simple Limiter)

The easiest way to limit concurrency is a buffered channel (semaphore).

```
func ProcessItems(items []string) {
 sem := make(chan struct{}, 10) // Limit to 10 concurrent
 var wg sync.WaitGroup

 for _, item := range items {
 wg.Add(1)
 go func(val string) {
 defer wg.Done()

 sem <- struct{}{} // Acquire token (blocks if full)
 defer func() { <-sem }() // Release token

 DoWork(val)
 }(item)
 }
 wg.Wait()
}
```

**Pros:** Trivial to implement. **Cons:** Still spawns N goroutines (memory cost), just blocks them from *executing* the heavy work.

## Pattern 2: The Worker Pool (Fixed Goroutines)

Instead of spawning a goroutine per item, spawn a fixed number of workers (e.g., `runtime.NumCPU()`) and feed them work.

```

func WorkerPool(jobs <-chan Job, results chan<- Result, workers int) {
 var wg sync.WaitGroup

 // Spawn fixed workers
 for i := 0; i < workers; i++ {
 wg.Add(1)
 go func(id int) {
 defer wg.Done()
 for job := range jobs {
 results <- process(job)
 }
 }(i)
 }

 wg.Wait()
 close(results) // Close results when all workers satisfy
}

func main() {
 jobs := make(chan Job, 100)
 results := make(chan Result, 100)

 go func() {
 // Enqueue jobs
 jobs <- Job{...}
 close(jobs) // Important: Tell workers no more jobs coming
 }()

 // Start pool
 WorkerPool(jobs, results, 5)

 // Consume results
 for res := range results {
 fmt.Println(res)
 }
}

```

**Pros:** Fixed memory footprint. Zero waste. **Cons:** Slightly more code.

## 2026: errgroup with Limits

The [golang.org/x/sync/errgroup](https://golang.org/x/sync/errgroup) package handles error propagation and limits.

```

g := new(errgroup.Group)
g.SetLimit(10) // New in recent versions

```

```

for _, item := range items {
 item := item
 g.Go(func() error {
 return process(item)
 })
}

if err := g.Wait(); err != nil {
 return err
}

```

**This is the preferred modern way.** It handles wait groups, error propagation (first error cancels context), and concurrency limits in one standard package.

## Summary

- Never use `go func()` inside a loop without a limit mechanism.
- Use `errgroup.SetLimit` for 90% of cases.
- Use manual Worker Patterns for complex, long-lived background processing queues.

## Worked example

Fixed pool with context cancel mid-flight (workers and producer both exit cleanly).

Save as `main.go`. Then:

```

go mod init example
go run .

package main

import (
 "context"
 "fmt"
 "sync"
)

func pool(ctx context.Context, jobs <-chan int, workers int) <-chan int {
 out := make(chan int)
 var wg sync.WaitGroup
 for w := 0; w < workers; w++ {
 wg.Add(1)
 go func() {
 defer wg.Done()

```

```

 for {
 select {
 case <-ctx.Done():
 return
 case j, ok := <-jobs:
 if !ok {
 return
 }
 select {
 case out <- j * 2:
 case <-ctx.Done():
 return
 }
 }
 }
 }()
}

go func() {
 wg.Wait()
 close(out)
}()

return out
}

func main() {
 ctx, cancel := context.WithCancel(context.Background())
 defer cancel()

 jobs := make(chan int)
 results := pool(ctx, jobs, 3)

 go func() {
 defer close(jobs)
 for i := 1; i <= 20; i++ {
 select {
 case <-ctx.Done():
 return
 case jobs <- i:
 }
 }
 }()

 count := 0
 for r := range results {
 _ = r
 count++
 if count == 5 {

```

```

 cancel() // stop early; remaining workers observe ctx.Done()
 }
}
fmt.Println("stopped after:", count)
}

```

**Expected output:** (may be 5 or a few more if in-flight sends complete)

```
stopped after: 5
```

## More examples

Stdlib-only errgroup-style first-error cancel.

```

package main

import (
 "context"
 "errors"
 "fmt"
 "sync"
)

func main() {
 ctx, cancel := context.WithCancel(context.Background())
 defer cancel()

 var (
 wg sync.WaitGroup
 once sync.Once
 err error
)
 tasks := []func(context.Context) error{
 func(ctx context.Context) error { return nil },
 func(ctx context.Context) error { return errors.New("boom") },
 func(ctx context.Context) error {
 <-ctx.Done()
 return ctx.Err()
 },
 }
 for _, t := range tasks {
 t := t
 wg.Add(1)
 go func() {
 defer wg.Done()

```

```

 if e := t(ctx); e != nil {
 once.Do(func() {
 err = e
 cancel()
 })
 }
 }()
}
wg.Wait()
fmt.Println("first error:", err)
}

```

Expected output:

```
first error: boom
```

## Runnable example

Save as `main.go`. Then:

```

go mod init example
go run .

```

```

package main

import (
 "fmt"
 "sync"
)

type Job struct {
 ID int
 Val int
}

type Result struct {
 JobID int
 Worker int
 Out int
}

func runPool(jobs <-chan Job, workers int) <-chan Result {
 results := make(chan Result)
 var wg sync.WaitGroup

```

```

for w := 1; w <= workers; w++ {
 wg.Add(1)
 go func(id int) {
 defer wg.Done()
 for job := range jobs {
 results <- Result{
 JobID: job.ID,
 Worker: id,
 Out: job.Val * job.Val,
 }
 }
 }(w)
}

go func() {
 wg.Wait()
 close(results)
}()
return results
}

func main() {
 const nJobs = 8
 const nWorkers = 3

 jobs := make(chan Job, nJobs)
 results := runPool(jobs, nWorkers)

 // Enqueue, then close so workers drain and exit.
 for i := 1; i <= nJobs; i++ {
 jobs <- Job{ID: i, Val: i}
 }
 close(jobs)

 // Semaphore-style limiter (spawn per item, cap concurrency)
 sem := make(chan struct{}, 2)
 var limWG sync.WaitGroup
 limitedSum := 0
 var mu sync.Mutex
 for i := 1; i <= 4; i++ {
 limWG.Add(1)
 go func(n int) {
 defer limWG.Done()
 sem <- struct{}{}
 defer func() { <-sem }()
 mu.Lock()
 limitedSum += n
 }(i)
 }
 limWG.Wait()
 return limitedSum
}

```



```

 mu.Unlock()
 }(i)
}
limWG.Wait()

poolSum := 0
seenWorkers := map[int]bool{}
for r := range results {
 poolSum += r.Out
 seenWorkers[r.Worker] = true
}

fmt.Println("pool workers used:", len(seenWorkers))
fmt.Println("pool sum of squares:", poolSum) // 1+4+9+16+25+36+49+64 = 204
fmt.Println("semaphore sum:", limitedSum) // 10
}

```

### Expected output:

```

pool workers used: 3
pool sum of squares: 204
semaphore sum: 10

```

**What to notice:** Fixed workers + closed job channel give a hard cap on goroutines and a clean shutdown. Closing `results` only after `wg.Wait()` prevents “send on closed channel.” The semaphore still spawns N goroutines but caps concurrent critical sections—cheaper to write, less ideal under huge N.

**Try next:** Set `nWorkers` to 1 and confirm the sum is unchanged. Replace the mutex around `limitedSum` with `atomic.AddInt64`. For production error propagation + limits, prefer [golang.org/x/sync/errgroup](https://golang.org/x/sync/errgroup) (not shown here so this stays `stdlib`-only).

## Pipelines & Complex Patterns

# Pipelines and sync.Cond

Beyond simple worker pools, Go concurrency shines in **Data Pipelines** (streaming data through stages) and **Signaling** (complex coordination).

## 1. The Pipeline Pattern (Fan-Out / Fan-In)

Pipelines allow you to process streams of data where each stage runs concurrently.

**Stages:** 1. **Generator:** Converts data (file lines, DB rows) into a channel. 2. **Transformer:** Reads one channel, modifies/filters, writes to another. 3. **Sink:** Consumes final output.

```
func Generator(nums ...int) <-chan int {
 out := make(chan int)
 go func() {
 for _, n := range nums { out <- n }
 close(out)
 }()
 return out
}

func Square(in <-chan int) <-chan int {
 out := make(chan int)
 go func() {
 for n := range in { out <- n * n }
 close(out)
 }()
 return out
}

func Merge(channels ...<-chan int) <-chan int {
 // Advanced: Fan-In multiple channels to one
 var wg sync.WaitGroup
 out := make(chan int)

 output := func(c <-chan int) {
 defer wg.Done()
 for n := range c { out <- n }
 }
```

```

 wg.Add(len(channels))
 for _, c := range channels {
 go output(c)
 }

 go func() {
 wg.Wait()
 close(out)
 }()
 return out
}

```

**Cancellation:** Real pipelines must pass `context.Context` to every stage to support early cancellation (e.g., stop processing if the user disconnects).

## 2. The Overlooked `sync.Cond`

Channels are for passing data. Mutexes are for locking data. **`sync.Cond`** is for **broadcasting signals**.

**Use Case:** You have 100 goroutines waiting for a specific event (e.g., “Configuration Loaded” or “Queue is not empty”). \* **Channels:** Sending 100 messages is  $O(N)$ . \* **Channels (Close):** Closing a channel broadcasts to all, but you can only do it *once*. \* **`sync.Cond`:** Can `Broadcast()` multiple times.

### Example: The Race Start

```

var mu sync.Mutex
cond := sync.NewCond(&mu)
ready := false

// Workers
for i := 0; i < 10; i++ {
 go func(id int) {
 mu.Lock()
 for !ready {
 cond.Wait() // Atomically unlocks mu and suspends execution
 }
 mu.Unlock()
 fmt.Println("Worker", id, "started")
 }(i)
}

// Coordinator
time.Sleep(1 * time.Second)

```

```
mu.Lock()
ready = true
mu.Unlock()
cond.Broadcast() // Wakes all 10 workers simultaneously
```

**Warning:** `cond.Wait()` *must* be in a loop (spurious wakeups are possible, though rare in Go, logic dictates checking the condition again).

## Summary

- **Pipelines:** Composable, streaming architecture. Great for ETL jobs.
- **Fan-In:** Merging multiple concurrent streams.
- **sync.Cond:** For “One-to-Many” signaling where the event can happen multiple times (unlike `close(ch)` which is one-off).

## Worked example

Cancellable pipeline stages with `context`.

Save as `main.go`. Then:

```
go mod init example
go run .

package main

import (
 "context"
 "fmt"
)

func gen(ctx context.Context, nums ...int) <-chan int {
 out := make(chan int)
 go func() {
 defer close(out)
 for _, n := range nums {
 select {
 case <-ctx.Done():
 return
 case out <- n:
 }
 }
 }()
 return out
}
```

```

}

func mul(ctx context.Context, in <-chan int, factor int) <-chan int {
 out := make(chan int)
 go func() {
 defer close(out)
 for n := range in {
 select {
 case <-ctx.Done():
 return
 case out <- n * factor:
 }
 }
 }()
 return out
}

func main() {
 ctx, cancel := context.WithCancel(context.Background())
 defer cancel()

 // 1,2,3,4 → *10 → take first two then cancel
 stream := mul(ctx, gen(ctx, 1, 2, 3, 4), 10)
 var got []int
 for v := range stream {
 got = append(got, v)
 if len(got) == 2 {
 cancel()
 }
 }
 fmt.Println("got:", got)
}

```

**Expected output:** (length 2; may include a couple extra depending on timing)

```
got: [10 20]
```

## More examples

sync.Cond broadcast twice (reload signal).

```

package main

import (
 "fmt"

```

```

 "sync"
)

func main() {
 var mu sync.Mutex
 cond := sync.NewCond(&mu)
 generation := 0

 const workers = 3
 var wg sync.WaitGroup
 seen := make(chan int, workers*2)

 for i := 0; i < workers; i++ {
 wg.Add(1)
 go func() {
 defer wg.Done()
 local := 0
 for local < 2 {
 mu.Lock()
 for local == generation {
 cond.Wait()
 }
 local = generation
 mu.Unlock()
 seen <- local
 }
 }()
 }

 for g := 1; g <= 2; g++ {
 mu.Lock()
 generation = g
 mu.Unlock()
 cond.Broadcast()
 }

 wg.Wait()
 close(seen)
 count := 0
 for range seen {
 count++
 }
 fmt.Println("signals delivered:", count) // 3 workers * 2 gens
}

```

**Expected output:**

```
signals delivered: 6
```

## Runnable example

Save as `main.go`. Then:

```
go mod init example
go run .
```

```
package main

import (
 "fmt"
 "sync"
 "time"
)

func generator(nums ...int) <-chan int {
 out := make(chan int)
 go func() {
 for _, n := range nums {
 out <- n
 }
 close(out)
 }()
 return out
}

func square(in <-chan int) <-chan int {
 out := make(chan int)
 go func() {
 defer close(out)
 for n := range in {
 out <- n * n
 }
 }()
 return out
}

func filterEven(in <-chan int) <-chan int {
 out := make(chan int)
 go func() {
 defer close(out)
 for n := range in {
 if n%2 == 0 {
```



```

 out <- n
 }
}
}()
return out
}

func main() {
 // Pipeline: generate → square → keep even squares
 // inputs 1..5 → squares 1,4,9,16,25 → even 4,16
 stage1 := generator(1, 2, 3, 4, 5)
 stage2 := square(stage1)
 stage3 := filterEven(stage2)

 var evenSquares []int
 for v := range stage3 {
 evenSquares = append(evenSquares, v)
 }
 fmt.Println("even squares:", evenSquares)

 // sync.Cond: one-to-many start signal
 var mu sync.Mutex
 cond := sync.NewCond(&mu)
 ready := false

 const n = 4
 var wg sync.WaitGroup
 started := make(chan int, n)

 for i := 1; i <= n; i++ {
 wg.Add(1)
 go func(id int) {
 defer wg.Done()
 mu.Lock()
 for !ready {
 cond.Wait() // unlocks mu while waiting
 }
 mu.Unlock()
 started <- id
 }(i)
 }

 time.Sleep(20 * time.Millisecond) // workers park on Wait
 mu.Lock()
 ready = true
 mu.Unlock()
 cond.Broadcast()
}

```

```
 wg.Wait()
 close(started)
 count := 0
 for range started {
 count++
 }
 fmt.Println("workers started after broadcast:", count)
}
```

**Expected output:**

```
even squares: [4 16]
workers started after broadcast: 4
```

**What to notice:** Each pipeline stage owns its output channel and closes it when done, so **range** terminates. `cond.Wait()` lives in a `for !ready` loop (predicate re-check). **Broadcast** wakes everyone; **Signal** would wake only one.

**Try next:** Insert a third pipeline stage that sums values. Change **Broadcast** to **Signal** in a loop and observe staggered starts.

**Web**

## The GOTH Stack

# The GOTH Stack: Go, Templ, HTMX, Tailwind

In 2026, the pendulum has swung back. The complexity of React/Next.js/Hydration/ServerComponents has pushed many developers toward simpler, server-driven architectures.

Enter the **GOTH Stack**: \* **G**o: Backend logic and router. \* **O** (Templ?): Usually **Templ**, a type-safe templating language for Go. \* **T**ailwind: Utility-first CSS. \* **H**TMX: High-power tools for HTML.

## Why GOTH?

It combines the type safety and speed of Go with the interactivity of an SPA, without the JSON-serialization overhead or state synchronization nightmares (Redux/Zustand).

## 1. Templ: Type-Safe HTML

Standard Go `html/template` is loosely typed. If you misspell a variable, it renders nothing. **Templ** compiles `.templ` files into Go code.

```
// hello.templ
package main

templ Hello(name string) {
 <div class="p-4 bg-blue-100 text-blue-800 rounded">
 <h1>Hello, { name }!</h1>
 </div>
}
```

If you pass an `int` to `Hello`, the Go compiler screams. This is revolutionary for backend rendering.

## 2. HTMX: The Engine

HTMX allows you to swap parts of the page via AJAX attributes directly in HTML.

```
<!-- When clicked, POST to /clicked, and replace this button with the response -->
<button hx-post="/clicked" hx-swap="outerHTML">
 Click Me
</button>
```

No JavaScript required. The server returns the *new HTML* (rendered via Templ), and HTMX injects it.

### 3. Tailwind: The Style

Since HTMX swaps HTML chunks, scoping CSS classes is hard. Tailwind solves this by embedding styles *in* the HTML. When HTMX injects a new `<div>`, it brings its styles with it.

#### A Simple Handler Example

```
func Handler(w http.ResponseWriter, r *http.Request) {
 // 1. Logic
 user := db.GetUser()

 // 2. Render Component
 component := components.UserProfile(user)

 // 3. Serve
 component.Render(r.Context(), w)
}
```

#### When NOT to use GOTH

- **Offline-first Apps:** If you need it to work in a tunnel, you need a heavy client (React/Flutter).
- **Complex Canvas/Games:** WebGL implementations.

For 95% of CRUD apps, Dashboards, and SaaS tools, the GOTH stack is faster to build and orders of magnitude faster to run.

#### Worked example

Server-rendered form post that returns an HTML fragment (HTMX-style).

Save as `main.go`. Then:

```
go mod init example
go run .
```

```
package main
```

```

import (
 "fmt"
 "html/template"
 "io"
 "net/http"
 "net/http/httptest"
 "strings"
)

var formTpl = template.Must(template.New("f").Parse(`
{{define "form"}}<form hx-post="/greet" hx-target="#out">
 <input name="name" value="{{.}}">
 <button type="submit">Greet</button>
</form>{{end}}
{{define "result"}}<p id="out">Hello, {{.}}!</p>{{end}}
`))

func main() {
 mux := http.NewServeMux()
 mux.HandleFunc("GET /${}", func(w http.ResponseWriter, r *http.Request) {
 w.Header().Set("Content-Type", "text/html; charset=utf-8")
 _ = formTpl.ExecuteTemplate(w, "form", "world")
 })
 mux.HandleFunc("POST /greet", func(w http.ResponseWriter, r *http.Request) {
 _ = r.ParseForm()
 name := r.FormValue("name")
 if name == "" {
 name = "friend"
 }
 w.Header().Set("Content-Type", "text/html; charset=utf-8")
 _ = formTpl.ExecuteTemplate(w, "result", name)
 })

 srv := httptest.NewServer(mux)
 defer srv.Close()

 res, _ := http.PostForm(srv.URL+"/greet", map[string][]string{"name": {"Ada"}})
 body, _ := io.ReadAll(res.Body)
 res.Body.Close()
 fmt.Println(strings.TrimSpace(string(body)))
}

```

**Expected output:**

```
<p id="out">Hello, Ada!</p>
```

## More examples

Compose page + partial templates without external Templ.

```
package main

import (
 "fmt"
 "html/template"
 "strings"
)

func main() {
 t := template.Must(template.New("root").Parse(`
{{define "card"}}<div class="card">{{.}}</div>{{end}}
{{define "page"}}<main>{{template "card" .}}</main>{{end}}
`))
 var b strings.Builder
 _ = t.ExecuteTemplate(&b, "page", "Revenue OK")
 fmt.Println(strings.TrimSpace(b.String()))
}
```

Expected output:

```
<main><div class="card">Revenue OK</div></main>
```

## Runnable example

**Note:** Production GOTH uses **Templ**, **HTMX**, and **Tailwind**. This stdlib-only stand-in shows the same idea: the server returns HTML fragments, and a client can request partial updates.

Save as `main.go`. Then:

```
go mod init example
go run .

package main

import (
 "fmt"
 "html/template"
 "io"
 "net/http"
 "net/http/httptest"
```



```

 "strings"
 "sync/atomic"
)

var clicks atomic.Int64

// One template set: page embeds the "counter" partial (HTMX-style fragment).
var tpls = template.Must(template.New("root").Parse(`
{{define "counter"}}<button hx-post="/clicked">Clicks: {{.}}</button>{{end}}
{{define "page"}}<!doctype html>
<html><body>
 <h1>GOTH-style (stdlib)</h1>
 <div id="counter">{{template "counter" .}}</div>
</body></html>{{end}}
`))

func main() {
 mux := http.NewServeMux()

 // Full page (initial load)
 mux.HandleFunc("GET /${}", func(w http.ResponseWriter, r *http.Request) {
 w.Header().Set("Content-Type", "text/html; charset=utf-8")
 _ = tpls.ExecuteTemplate(w, "page", clicks.Load())
 })

 // HTMX-style partial: return only the button HTML
 mux.HandleFunc("POST /clicked", func(w http.ResponseWriter, r *http.Request) {
 clicks.Add(1)
 w.Header().Set("Content-Type", "text/html; charset=utf-8")
 _ = tpls.ExecuteTemplate(w, "counter", clicks.Load())
 })

 // Drive requests without binding a port forever
 srv := httptest.NewServer(mux)
 defer srv.Close()

 resp, err := http.Get(srv.URL + "/")
 if err != nil {
 panic(err)
 }
 body, _ := io.ReadAll(resp.Body)
 resp.Body.Close()
 fmt.Println("status:", resp.StatusCode)
 fmt.Println("page contains counter:", strings.Contains(string(body), "Clicks: 0"))

 post, err := http.Post(srv.URL+"/clicked", "text/plain", nil)
 if err != nil {

```

```
 panic(err)
 }
 frag, _ := io.ReadAll(post.Body)
 post.Body.Close()
 fmt.Println("fragment:", strings.TrimSpace(string(frag)))
}
```

#### Expected output:

```
status: 200
page contains counter: true
fragment: <button hx-post="/clicked">Clicks: 1</button>
```

**What to notice:** Full page vs fragment is just different handlers. `httptest.NewServer` exercises real HTTP without leaving a process listening. Production Templ/HTMX would swap `#counter` automatically; here we print the fragment.

**Try next:** Add a GET `/clicked` that returns the same fragment, and chain three POSTs to watch the count rise.

## **Skeleton Loading & HTMX**

# Skeleton Loading with Go & HTMX

Users hate waiting. If your dashboard takes 2 seconds to calculate “Total Revenue,” the entire page shouldn’t hang for 2 seconds.

In standard MPAs (Multi-Page Apps), the browser waits for the full HTML response. In the GOTH stack, we use **Lazy Loading** with Skeleton screens.

## The Pattern

1. **Initial Load:** Return the page structure immediately (Header, Sidebar, Footer) but fill the “Slow Widgets” with **Skeleton Loaders** (grey pulsing boxes).
2. **Trigger:** The Skeleton HTML includes `hx-trigger="load"` to immediately ask the server for the real content.
3. **Swap:** The server calculates the expensive data and returns the real HTML, swapping out the skeleton.

## Implementation

### 1. The Dashboard Handler (Fast)

```
func DashboardHandler(w http.ResponseWriter, r *http.Request) {
 // Render the layout immediately.
 // Notice we don't fetch revenue here.
 Layout(
 // Inject Skeletons
 components.RevenueSkeleton(),
 components.UsersSkeleton(),
).Render(r.Context(), w)
}
```

### 2. The Skeleton Component (Templ)

```
templ RevenueSkeleton() {
 <div hx-get="/api/revenue"
 hx-trigger="load"
 hx-swap="outerHTML"
```

```

 class="animate-pulse bg-gray-200 h-32 rounded">
 <!-- Pulsing Grey Box -->
 </div>
}

```

### 3. The Real Data Handler (Slow)

```

func RevenueHandler(w http.ResponseWriter, r *http.Request) {
 // Extensive DB calculation...
 time.Sleep(1 * time.Second)
 amount := db.CalculateRevenue()

 // Return real component
 components.RevenueCard(amount).Render(r.Context(), w)
}

```

### Why this wins

- **TTFB (Time To First Byte):** 10-20ms. The user sees the UI instantly.
- **Perceived Performance:** The app feels alive while data loads in parallel.
- **Simplicity:** No `useEffect`, no `isLoading` state management, no JSON parsing. Just HTML swapping.

### Advanced: Request Coalescing

If 10 widgets all fire `hx-get` at once, you might hit browser connection limits (HTTP/1.1 limit is 6). \* **HTTP/2 or HTTP/3:** Go's `net/http` supports these automatically if you use TLS. They multiplex requests, so 10 requests cost nearly the same connection overhead as 1. \* Ensure your server supports TLS (Let's Encrypt) to enable H2/H3.

### Worked example

Fast shell + parallel widget endpoints (stdlib model of skeleton fill).

Save as `main.go`. Then:

```

go mod init example
go run .

```

```

package main

```

```

import (
 "fmt"
 "io"
 "net/http"
 "net/http/httptest"
 "strings"
 "sync"
 "time"
)

func main() {
 mux := http.NewServeMux()
 mux.HandleFunc("GET /dash", func(w http.ResponseWriter, r *http.Request) {
 _, _ = io.WriteString(w, `<div class="skel" data-src="/w/a"></div><div class="skel"
 })
 mux.HandleFunc("GET /w/{id}", func(w http.ResponseWriter, r *http.Request) {
 time.Sleep(15 * time.Millisecond)
 fmt.Fprintf(w, `<div class="card">widget %s</div>`, r.PathValue("id"))
 })

 srv := httptest.NewServer(mux)
 defer srv.Close()

 shell, _ := http.Get(srv.URL + "/dash")
 sb, _ := io.ReadAll(shell.Body)
 shell.Body.Close()
 fmt.Println("shell has skeleton:", strings.Contains(string(sb), `class="skel"``))

 // Parallel widget fetch (what the browser/HTMX would do)
 var wg sync.WaitGroup
 for _, id := range []string{"a", "b"} {
 wg.Add(1)
 go func(id string) {
 defer wg.Done()
 res, err := http.Get(srv.URL + "/w/" + id)
 if err != nil {
 panic(err)
 }
 b, _ := io.ReadAll(res.Body)
 res.Body.Close()
 fmt.Println(strings.TrimSpace(string(b)))
 }(id)
 }
 wg.Wait()
}

```

**Expected output:** (widget lines may reorder)

```
shell has skeleton: true
<div class="card">widget a</div>
<div class="card">widget b</div>
```

## More examples

Measure shell TTFB vs sequential widgets.

```
package main

import (
 "fmt"
 "net/http"
 "net/http/httpptest"
 "time"
)

func main() {
 mux := http.NewServeMux()
 mux.HandleFunc("GET /fast", func(w http.ResponseWriter, r *http.Request) { w.Write([]byte("fast")) })
 mux.HandleFunc("GET /slow", func(w http.ResponseWriter, r *http.Request) {
 time.Sleep(30 * time.Millisecond)
 w.Write([]byte("data"))
 })
 srv := httpptest.NewServer(mux)
 defer srv.Close()

 t0 := time.Now()
 http.Get(srv.URL + "/fast")
 fast := time.Since(t0)

 t1 := time.Now()
 http.Get(srv.URL + "/slow")
 slow := time.Since(t1)

 fmt.Println("fast < slow:", fast < slow)
}
```

**Expected output:**

```
fast < slow: true
```

## Runnable example

**Note:** Real skeleton UIs use HTMX (`hx-trigger="load"`) + Templ. This stdlib demo models the same flow: fast shell HTML, then a slow partial.

Save as `main.go`. Then:

```
go mod init example
go run .
```

```
package main
```

```
import (
 "fmt"
 "io"
 "net/http"
 "net/http/httptest"
 "strings"
 "time"
)
```

```
func main() {
 mux := http.NewServeMux()
```

```
 // Fast shell: structure + skeleton placeholder
 mux.HandleFunc("GET /dashboard", func(w http.ResponseWriter, r *http.Request) {
 w.Header().Set("Content-Type", "text/html; charset=utf-8")
 _, _ = io.WriteString(w, `<!doctype html>
<html><body>
 <h1>Dashboard</h1>
 <div id="revenue"
 hx-get="/api/revenue"
 hx-trigger="load"
 class="skeleton">Loading revenue...</div>
</body></html>`)
 })

 // Slow widget: pretend DB work, return final HTML card
 mux.HandleFunc("GET /api/revenue", func(w http.ResponseWriter, r *http.Request) {
 time.Sleep(25 * time.Millisecond)
 w.Header().Set("Content-Type", "text/html; charset=utf-8")
 _, _ = io.WriteString(w, `<div id="revenue" class="card">Total revenue: $42,000</div>`)
 })

 srv := httptest.NewServer(mux)
 defer srv.Close()
```



```

t0 := time.Now()
shell, err := http.Get(srv.URL + "/dashboard")
if err != nil {
 panic(err)
}
shellBody, _ := io.ReadAll(shell.Body)
shell.Body.Close()
shellMS := time.Since(t0).Milliseconds()

t1 := time.Now()
widget, err := http.Get(srv.URL + "/api/revenue")
if err != nil {
 panic(err)
}
widgetBody, _ := io.ReadAll(widget.Body)
widget.Body.Close()
widgetMS := time.Since(t1).Milliseconds()

fmt.Println("shell status:", shell.StatusCode)
fmt.Println("shell has skeleton:", strings.Contains(string(shellBody), "class=\"skeleton"))
fmt.Println("shell faster than widget:", shellMS < widgetMS)
fmt.Println("widget:", strings.TrimSpace(string(widgetBody)))
fmt.Println("widget latency ms >= 20:", widgetMS >= 20)
}

```

### Expected output:

```

shell status: 200
shell has skeleton: true
shell faster than widget: true
widget: <div id="revenue" class="card">Total revenue: $42,000</div>
widget latency ms >= 20: true

```

**What to notice:** Time-to-first-byte for the shell stays low because expensive work lives on a separate endpoint. The browser (or HTMX) fills the skeleton after load; the user sees structure immediately.

**Try next:** Fire three parallel GET `/api/revenue` requests with goroutines and measure wall time vs sequential.

## Microservices & gRPC

# Microservices: Switching to gRPC (and Connect)

REST is fine for public APIs. But for internal communication between microservices, JSON over HTTP/1.1 is inefficient (text parsing, no types, high overhead).

**gRPC** (Google Remote Procedure Call) is the standard for high-performance internal traffic.

## The Protocol Buffers (Protobuf)

You define your data and service in a `.proto` file.

```
syntax = "proto3";

service UserService {
 rpc GetUser (UserRequest) returns (UserResponse);
}

message UserRequest {
 string id = 1;
}

message UserResponse {
 string name = 1;
 int32 age = 2;
}
```

Then, you compile this using `protoc` to generate Go code. \* **Strict Types:** The generated code guarantees the wire format matches the struct. \* **Binary:** Messages are binary (smaller, faster to parse than JSON).

## The gRPC Complexity (and the Solution: ConnectRPC)

Classic gRPC requires special load balancers (because it breaks HTTP/1.1 framing) and is hard to debug with `curl`.

In 2026, we prefer **ConnectRPC** (by Buf). \* **It's just HTTP:** It supports gRPC but *also* standard HTTP/JSON. \* You can `curl` a Connect service with JSON, but talk to it via gRPC from other Go services.

## Connect Example

```
package main

import (
 "context"
 "net/http"
 "connectrpc.com/connect"
 user "example/gen/user/v1" // Generated code
 "example/gen/user/v1/userv1connect"
)

type UserServer struct {}

func (s *UserServer) GetUser(
 ctx context.Context,
 req *connect.Request[user.UserRequest],
) (*connect.Response[user.UserResponse], error) {
 return connect.NewResponse(&user.UserResponse{
 Name: "Alice",
 Age: 30,
 }), nil
}

func main() {
 mux := http.NewServeMux()
 path, handler := userv1connect.NewUserServiceHandler(&UserServer{})
 mux.Handle(path, handler)
 http.ListenAndServe(":8080", mux)
}
```

## When to use Microservices?

**Default to Monolith.** Microservices introduce: 1. Network Latency. 2. Distributed Tracing requirements. 3. Deployment complexity.

Only switch to Microservices (and gRPC) when: \* You have distinct teams who need to deploy independently. \* You have distinct scaling requirements (e.g., the Video Encoder needs 100 GPUs, but the Login server needs 1 CPU).

## Worked example

Unary JSON-RPC client/server with typed request envelopes.

Save as `main.go`. Then:

```
go mod init example
go run .
```

```
package main
```

```
import (
 "bytes"
 "encoding/json"
 "fmt"
 "net/http"
 "net/http/httptest"
)
```

```
type Req[T any] struct {
 Payload T `json:"payload"`
}
```

```
type Resp[T any] struct {
 Payload T `json:"payload"`
}
```

```
type AddIn struct {
 A, B int `json:"a"`
}
```

```
type AddOut struct {
 Sum int `json:"sum"`
}
```

```
func main() {
 mux := http.NewServeMux()
 mux.HandleFunc("POST /calc.v1/Add", func(w http.ResponseWriter, r *http.Request) {
 var in Req[AddIn]
 if err := json.NewDecoder(r.Body).Decode(&in); err != nil {
 http.Error(w, "bad json", 400)
 return
 }
 _ = json.NewEncoder(w).Encode(Resp[AddOut]{Payload: AddOut{Sum: in.Payload.A + in.Payload.B}})
 })
 srv := httptest.NewServer(mux)
 defer srv.Close()

 body, _ := json.Marshal(Req[AddIn]{Payload: AddIn{A: 2, B: 40}})
 res, err := http.Post(srv.URL+"/calc.v1/Add", "application/json", bytes.NewReader(body))
 if err != nil {
 panic(err)
 }
}
```

```

defer res.Body.Close()
var out Resp[AddOut]
_ = json.NewDecoder(res.Body).Decode(&out)
fmt.Println("sum:", out.Payload.Sum)
}

```

Expected output:

```
sum: 42
```

## More examples

RPC error envelope + status mapping.

```

package main

import (
 "bytes"
 "encoding/json"
 "fmt"
 "net/http"
 "net/http/httptest"
)

func main() {
 mux := http.NewServeMux()
 mux.HandleFunc("POST /user.v1/Get", func(w http.ResponseWriter, r *http.Request) {
 var req struct {
 ID string `json:"id"`
 }
 _ = json.NewDecoder(r.Body).Decode(&req)
 if req.ID == "" {
 w.WriteHeader(http.StatusBadRequest)
 _ = json.NewEncoder(w).Encode(map[string]string{"error": "id required"})
 return
 }
 _ = json.NewEncoder(w).Encode(map[string]string{"name": "Ada"})
 })
 srv := httptest.NewServer(mux)
 defer srv.Close()

 res, _ := http.Post(srv.URL+"/user.v1/Get", "application/json", bytes.NewReader([]byte(`
 fmt.Println("status:", res.StatusCode)
 res.Body.Close()
 }
 `)))
}

```

## Expected output:

```
status: 400
```

## Runnable example

**Note:** Production internal APIs often use **gRPC** or **ConnectRPC** with protobuf. This stdlib stand-in is a typed JSON “RPC over HTTP” service—same request/response shape idea, no codegen.

Save as `main.go`. Then:

```
go mod init example
go run .

package main

import (
 "bytes"
 "encoding/json"
 "fmt"
 "net/http"
 "net/http/httptest"
)

type UserRequest struct {
 ID string `json:"id"`
}

type UserResponse struct {
 Name string `json:"name"`
 Age int `json:"age"`
}

type ErrorBody struct {
 Error string `json:"error"`
}

func getUser(id string) (UserResponse, bool) {
 db := map[string]UserResponse{
 "u1": {Name: "Alice", Age: 30},
 "u2": {Name: "Bob", Age: 25},
 }
 u, ok := db[id]
 return u, ok
}
```

```

}

func main() {
 mux := http.NewServeMux()
 // Connect/gRPC-style unary RPC as JSON POST
 mux.HandleFunc("POST /user.v1.UserService/GetUser", func(w http.ResponseWriter, r *http.
 var req UserRequest
 if err := json.NewDecoder(r.Body).Decode(&req); err != nil {
 w.WriteHeader(http.StatusBadRequest)
 _ = json.NewEncoder(w).Encode(ErrorBody{Error: "invalid json"})
 return
 }
 user, ok := getUser(req.ID)
 if !ok {
 w.WriteHeader(http.StatusNotFound)
 _ = json.NewEncoder(w).Encode(ErrorBody{Error: "user not found"})
 return
 }
 w.Header().Set("Content-Type", "application/json")
 _ = json.NewEncoder(w).Encode(user)
 })

 srv := httptest.NewServer(mux)
 defer srv.Close()

 call := func(id string) {
 body, _ := json.Marshal(UserRequest{ID: id})
 resp, err := http.Post(srv.URL+"/user.v1.UserService/GetUser", "application/json", b
 if err != nil {
 panic(err)
 }
 defer resp.Body.Close()
 var raw map[string]any
 _ = json.NewDecoder(resp.Body).Decode(&raw)
 fmt.Printf("id=%s status=%d body=%v\n", id, resp.StatusCode, raw)
 }

 call("u1")
 call("missing")
}

```

### Expected output:

```

id=u1 status=200 body=map[age:30 name:Alice]
id=missing status=404 body=map[error:user not found]

```

**What to notice:** Method path + typed request/response mirrors an RPC surface even over



JSON/HTTP. `httptest` keeps the demo offline-friendly. Real gRPC adds binary protobuf, streaming, and code-generated stubs.

**Try next:** Add a `ListUsers` method. Wrap the handler with a middleware that logs RPC name and status code.

## Huma & OpenAPI

# Modern APIs: Huma & OpenAPI

Writing REST APIs involves two pains: 1. Validation (Checking JSON inputs). 2. Documentation (Keeping Swagger/OpenAPI YAML in sync with code).

In the past, we wrote code, then manually wrote YAML. The YAML effectively lied because it drifted from the code.

## Enter Huma

**Huma** (huma.rocks) is the modern (2025/2026) framework of choice for building APIs. \* **Code-First OpenAPI**: It generates the OpenAPI 3.1 Spec *from* your Go structs. \* **Automatic Validation**: It validates inputs based on struct tags.

## Huma Example

```
package main

import (
 "context"
 "net/http"
 "github.com/danielgtaylor/huma/v2"
 "github.com/danielgtaylor/huma/v2/adapters/humachi"
 "github.com/go-chi/chi/v5"
)

// 1. Define Request/Response
type GreetingInput struct {
 Name string `query:"name" maxLength:"30" doc:"Name to greet"`
}

type GreetingOutput struct {
 Body struct {
 Message string `json:"message" example:"Hello, World!"`
 }
}

func main() {
 router := chi.NewMux()
```

```

api := humachi.New(router, huma.DefaultConfig("My API", "1.0.0"))

// 2. Register Operation
huma.Register(api, huma.Operation{
 OperationID: "get-greeting",
 Method: http.MethodGet,
 Path: "/greeting",
 Summary: "Get a welcome message",
}, func(ctx context.Context, input *GreetingInput) (*GreetingOutput, error) {
 // Only executes if validation passes!
 resp := &GreetingOutput{}
 resp.Body.Message = "Hello, " + input.Name
 return resp, nil
})

http.ListenAndServe(":8888", router)
}

```

## Why Huma?

1. **Documentation:** Visit `/docs` (or similar) and you get a beautiful, interactive Scalar or Swagger UI automatically.
2. **Safety:** Meaningful error messages for users (“Field ‘name’ is too long”).
3. **Standard Library Compatible:** It works on top of `http.ServeMux`, `Chi`, or `Gin`. It doesn’t lock you into a monolithic ecosystem.

This replaces the older `swag` comment-based generation which was error-prone and brittle.

## Worked example

Struct-driven validation + JSON error/success responses (Huma-shaped, `stdlib`).

Save as `main.go`. Then:

```

go mod init example
go run .

package main

import (
 "encoding/json"
 "fmt"
 "net/http"
 "net/http/httptest"

```

```

 "strings"
 "unicode/utf8"
)

type CreateUser struct {
 Name string `json:"name"`
 Age int `json:"age"`
}

func validate(u CreateUser) []string {
 var errs []string
 if u.Name == "" {
 errs = append(errs, "name is required")
 }
 if utf8.RuneCountInString(u.Name) > 30 {
 errs = append(errs, "name maxLength 30")
 }
 if u.Age < 0 || u.Age > 150 {
 errs = append(errs, "age out of range")
 }
 return errs
}

func main() {
 mux := http.NewServeMux()
 mux.HandleFunc("POST /users", func(w http.ResponseWriter, r *http.Request) {
 w.Header().Set("Content-Type", "application/json")
 var u CreateUser
 if err := json.NewDecoder(r.Body).Decode(&u); err != nil {
 w.WriteHeader(http.StatusBadRequest)
 _ = json.NewEncoder(w).Encode(map[string]string{"error": "invalid json"})
 return
 }
 if errs := validate(u); len(errs) > 0 {
 w.WriteHeader(http.StatusUnprocessableEntity)
 _ = json.NewEncoder(w).Encode(map[string]any{"errors": errs})
 return
 }
 _ = json.NewEncoder(w).Encode(map[string]string{"status": "created", "name": u.Name})
 })

 srv := httptest.NewServer(mux)
 defer srv.Close()

 res, _ := http.Post(srv.URL+"/users", "application/json", strings.NewReader(`{"name": ""}`,
 fmt.Println("invalid status:", res.StatusCode)
 res.Body.Close()

```

```

res, _ = http.Post(srv.URL+"/users", "application/json", strings.NewReader(`{"name":"Ada"}`))
var ok map[string]string
_ = json.NewDecoder(res.Body).Decode(&ok)
res.Body.Close()
fmt.Println("valid:", res.StatusCode, ok["status"], ok["name"])
}

```

Expected output:

```

invalid status: 422
valid: 200 created Ada

```

## More examples

Serve a tiny OpenAPI document next to the handler (docs stay near code).

```

package main

import (
 "encoding/json"
 "fmt"
 "net/http"
 "net/http/httptest"
)

func main() {
 spec := map[string]any{
 "openapi": "3.1.0",
 "info": map[string]string{"title": "Users", "version": "1.0.0"},
 "paths": map[string]any{
 "/users": map[string]any{"post": map[string]string{"operationId": "createUser"}}
 },
 }

 mux := http.NewServeMux()
 mux.HandleFunc("GET /openapi.json", func(w http.ResponseWriter, r *http.Request) {
 _ = json.NewEncoder(w).Encode(spec)
 })

 srv := httptest.NewServer(mux)
 defer srv.Close()
 res, _ := http.Get(srv.URL + "/openapi.json")
 var doc map[string]any
 _ = json.NewDecoder(res.Body).Decode(&doc)
 res.Body.Close()
 fmt.Println("openapi:", doc["openapi"])
}

```

## Expected output:

```
openapi: 3.1.0
```

## Runnable example

**Note:** **Huma** generates OpenAPI 3.1 and validates from structs automatically. This stdlib stand-in shows the same product idea: validate input, return JSON, and expose a tiny machine-readable “spec” document.

Save as `main.go`. Then:

```
go mod init example
go run .

package main

import (
 "encoding/json"
 "fmt"
 "net/http"
 "net/http/httptest"
 "unicode/utf8"
)

type GreetingOutput struct {
 Message string `json:"message"`
}

type APIError struct {
 Error string `json:"error"`
}

// Mini OpenAPI-ish document (hand-written for the demo).
var openAPI = map[string]any{
 "openapi": "3.1.0",
 "info": map[string]string{"title": "Greeting API", "version": "1.0.0"},
 "paths": map[string]any{
 "/greeting": map[string]any{
 "get": map[string]any{
 "operationId": "get-greeting",
 "summary": "Get a welcome message",
 "parameters": []map[string]any{
 {
 "name": "name", "in": "query", "required": true,
```

```

 "schema": map[string]any{"type": "string", "maxLength": 30},
 },
},
},
},
}

func greetingHandler(w http.ResponseWriter, r *http.Request) {
 name := r.URL.Query().Get("name")
 w.Header().Set("Content-Type", "application/json")

 if name == "" {
 w.WriteHeader(http.StatusBadRequest)
 _ = json.NewEncoder(w).Encode(APIError{Error: "query param 'name' is required"})
 return
 }
 if utf8.RuneCountInString(name) > 30 {
 w.WriteHeader(http.StatusUnprocessableEntity)
 _ = json.NewEncoder(w).Encode(APIError{Error: "field 'name' is too long (max 30)"})
 return
 }

 _ = json.NewEncoder(w).Encode(GreetingOutput{Message: "Hello, " + name})
}

func main() {
 mux := http.NewServeMux()
 mux.HandleFunc("GET /greeting", greetingHandler)
 mux.HandleFunc("GET /openapi.json", func(w http.ResponseWriter, r *http.Request) {
 w.Header().Set("Content-Type", "application/json")
 _ = json.NewEncoder(w).Encode(openAPI)
 })

 srv := httpptest.NewServer(mux)
 defer srv.Close()

 // Missing name → validation error
 r1, _ := http.Get(srv.URL + "/greeting")
 var e1 APIError
 _ = json.NewDecoder(r1.Body).Decode(&e1)
 r1.Body.Close()
 fmt.Println("missing:", r1.StatusCode, e1.Error)

 // Valid
 r2, _ := http.Get(srv.URL + "/greeting?name=World")

```



```

var ok GreetingOutput
_ = json.NewDecoder(r2.Body).Decode(&ok)
r2.Body.Close()
fmt.Println("ok:", r2.StatusCode, ok.Message)

// Spec exists
r3, _ := http.Get(srv.URL + "/openapi.json")
var spec map[string]any
_ = json.NewDecoder(r3.Body).Decode(&spec)
r3.Body.Close()
info := spec["info"].(map[string]any)
fmt.Println("spec title:", info["title"])
}

```

### Expected output:

```

missing: 400 query param 'name' is required
ok: 200 Hello, World
spec title: Greeting API

```

**What to notice:** Validation lives next to the handler (Huma derives it from tags). Shipping `/openapi.json` keeps docs honest—Huma generates that document from the same structs that validate.

**Try next:** Reject names containing digits. Add POST `/greeting` with a JSON body and validate a required `name` field.

## **Integrations: PostgreSQL & Redis**

# Integrations: PostgreSQL & Redis

Most Go backends in 2026 rely on this “boring” but powerful trio: Go + Postgres + Redis.

## PostgreSQL: pgx

Don’t use `lib/pq`. It is in maintenance mode. Use `jackc/pgx`. It is faster, safer, and supports more features (COPY, Listen/Notify).

### Connection Pooling

Always use `pgxpool`, not a single connection.

```
import "github.com/jackc/pgx/v5/pgxpool"

func NewDB(ctx context.Context, connString string) (*pgxpool.Pool, error) {
 config, err := pgxpool.ParseConfig(connString)
 // Tuning
 config.MaxConns = 25
 config.MinConns = 5
 config.MaxConnLifetime = 1 * time.Hour

 return pgxpool.NewWithConfig(ctx, config)
}
```

## Type-Safe SQL: sqlc

Don’t write ORM code (GORM) unless you love runtime crashes and slow queries. Use `sqlc`. You write SQL, it generates type-safe Go structs and interfaces. (See Chapter 59 “Code Generation” for details).

---

## Redis: go-redis

Use `github.com/redis/go-redis/v9`.

## Patterns

1. **Caching (Look-aside):**

```
go val, err := rdb.Get(ctx, key).Result() if
err == redis.Nil { // Miss: Fetch DB, Set Redis val = fetchFromDB()
rdb.Set(ctx, key, val, 10*time.Minute) } else if err != nil { return
err // Real error } // Hit: use val
```
2. **Rate Limiting:** Redis is atomic. INCR and EXPIRE are perfect for API limits.
3. **Queues:** Redis Streams or simple Lists (LPUSH/BRPOP) make for excellent lightweight worker queues before you upgrade to Kafka or NATS.

## Context Awareness

Both `pgx` and `go-redis` require `context.Context` in every method call. **Always pass the context.** This allows your database queries to automatically timeout if the user cancels the HTTP request, preventing zombie queries from eating your database CPU.

```
// Good
row := db.QueryRow(ctx, "SELECT ...")

// Bad (cancelling request won't stop query)
row := db.QueryRow(context.Background(), "SELECT ...")
```

## Worked example

Look-aside cache with TTL expiry (stdlib Redis/DB stand-in).

Save as `main.go`. Then:

```
go mod init example
go run .

package main

import (
 "context"
 "fmt"
 "sync"
 "time"
)

type cache struct {
 mu sync.Mutex
 items map[string]entry
```

```

}

type entry struct {
 val string
 exp time.Time
}

func (c *cache) Get(k string) (string, bool) {
 c.mu.Lock()
 defer c.mu.Unlock()
 e, ok := c.items[k]
 if !ok || time.Now().After(e.exp) {
 delete(c.items, k)
 return "", false
 }
 return e.val, true
}

func (c *cache) Set(k, v string, ttl time.Duration) {
 c.mu.Lock()
 defer c.mu.Unlock()
 c.items[k] = entry{val: v, exp: time.Now().Add(ttl)}
}

func main() {
 c := &cache{items: map[string]entry{}}
 c.Set("u1", "alice", 20*time.Millisecond)
 if v, ok := c.Get("u1"); ok {
 fmt.Println("hit:", v)
 }
 time.Sleep(25 * time.Millisecond)
 _, ok := c.Get("u1")
 fmt.Println("after ttl hit:", ok)

 // context cancel around a "query"
 ctx, cancel := context.WithCancel(context.Background())
 cancel()
 select {
 case <-ctx.Done():
 fmt.Println("query aborted:", ctx.Err())
 default:
 }
}

```

**Expected output:**

```
hit: alice
after ttl hit: false
query aborted: context canceled
```

## More examples

Simple atomic rate limiter (Redis INCR window analogue).

```
package main

import (
 "fmt"
 "sync"
 "sync/atomic"
 "time"
)

type limiter struct {
 mu sync.Mutex
 count int
 reset time.Time
 limit int
 window time.Duration
}

func (l *limiter) Allow() bool {
 l.mu.Lock()
 defer l.mu.Unlock()
 now := time.Now()
 if now.After(l.reset) {
 l.count = 0
 l.reset = now.Add(l.window)
 }
 if l.count >= l.limit {
 return false
 }
 l.count++
 return true
}

func main() {
 l := &limiter{limit: 3, window: time.Second, reset: time.Now().Add(time.Second)}
 var allowed atomic.Int64
 var wg sync.WaitGroup
 for i := 0; i < 10; i++ {
```

```

 wg.Add(1)
 go func() {
 defer wg.Done()
 if l.Allow() {
 allowed.Add(1)
 }
 }()
 }
 wg.Wait()
 fmt.Println("allowed:", allowed.Load()) // 3
}

```

Expected output:

```
allowed: 3
```

## Runnable example

**Note:** Production uses **pgx** + **Redis**. This stdlib stand-in models a look-aside cache and context-aware “query” against in-memory stores so the pattern is runnable offline.

Save as `main.go`. Then:

```

go mod init example
go run .

package main

import (
 "context"
 "errors"
 "fmt"
 "sync"
 "time"
)

// FakeDB simulates Postgres latency + storage.
type FakeDB struct {
 mu sync.Mutex
 rows map[string]string
 hits int
}

func (db *FakeDB) QueryRow(ctx context.Context, key string) (string, error) {
 select {

```

```

 case <-ctx.Done():
 return "", ctx.Err()
 case <-time.After(20 * time.Millisecond): // pretend network/DB
 }
 db.mu.Lock()
 defer db.mu.Unlock()
 db.hits++
 v, ok := db.rows[key]
 if !ok {
 return "", errors.New("not found")
 }
 return v, nil
}

// FakeRedis is a tiny TTL cache.
type FakeRedis struct {
 mu sync.Mutex
 items map[string]cacheItem
}

type cacheItem struct {
 val string
 exp time.Time
}

func NewRedis() *FakeRedis {
 return &FakeRedis{items: map[string]cacheItem{}}
}

func (r *FakeRedis) Get(_ context.Context, key string) (string, bool) {
 r.mu.Lock()
 defer r.mu.Unlock()
 it, ok := r.items[key]
 if !ok || time.Now().After(it.exp) {
 delete(r.items, key)
 return "", false
 }
 return it.val, true
}

func (r *FakeRedis) Set(_ context.Context, key, val string, ttl time.Duration) {
 r.mu.Lock()
 defer r.mu.Unlock()
 r.items[key] = cacheItem{val: val, exp: time.Now().Add(ttl)}
}

```



```

func getUser(ctx context.Context, db *FakeDB, cache *FakeRedis, id string) (string, string,
 if v, ok := cache.Get(ctx, id); ok {
 return v, "cache", nil
 }
 v, err := db.QueryRow(ctx, id)
 if err != nil {
 return "", "", err
 }
 cache.Set(ctx, id, v, time.Minute)
 return v, "db", nil
}

func main() {
 db := &FakeDB{rows: map[string]string{"u1": "alice"}}
 cache := NewRedis()
 ctx := context.Background()

 v1, src1, err := getUser(ctx, db, cache, "u1")
 if err != nil {
 panic(err)
 }
 v2, src2, err := getUser(ctx, db, cache, "u1")
 if err != nil {
 panic(err)
 }

 // Cancelled context aborts the "query"
 cctx, cancel := context.WithCancel(context.Background())
 cancel()
 _, _, cerr := db.QueryRow(cctx, "u1")

 fmt.Println("first:", v1, "via", src1)
 fmt.Println("second:", v2, "via", src2)
 fmt.Println("db hits:", db.hits)
 fmt.Println("cancelled err:", cerr)
}

```

### Expected output:

```

first: alice via db
second: alice via cache
db hits: 1
cancelled err: context canceled

```

**What to notice:** Look-aside cache protects the DB on hot keys (`db.hits` stays 1). Passing a cancelled `context` stops work early—the same discipline you need with `pgx` / `go-redis`.

**Try next:** Expire the cache entry with a 1ms TTL and sleep before the second read. Add a simple

rate limiter using an in-memory counter + window (Redis `INCR/EXPIRE` analogue).

# HTTP and Networking

## Overview

Go's `net/http` package provides a complete HTTP client and server implementation.

## HTTP Client

```
// Simple GET
resp, err := http.Get("https://api.example.com/data")
if err != nil {
 log.Fatal(err)
}
defer resp.Body.Close()

body, _ := io.ReadAll(resp.Body)
```

## Custom Request

```
req, _ := http.NewRequest("POST", url, bytes.NewBuffer(data))
req.Header.Set("Content-Type", "application/json")
req.Header.Set("Authorization", "Bearer "+token)

client := &http.Client{Timeout: 10 * time.Second}
resp, err := client.Do(req)
```

## With Context

```
ctx, cancel := context.WithTimeout(context.Background(), 5*time.Second)
defer cancel()

req, _ := http.NewRequestWithContext(ctx, "GET", url, nil)
resp, err := http.DefaultClient.Do(req)
```

## HTTP Server

```
func handler(w http.ResponseWriter, r *http.Request) {
 fmt.Fprintf(w, "Hello, %s!", r.URL.Path[1:])
}

func main() {
 http.HandleFunc("/", handler)
 log.Fatal(http.ListenAndServe(":8080", nil))
}
```

## JSON API

```
func userHandler(w http.ResponseWriter, r *http.Request) {
 user := User{Name: "Alice"}

 w.Header().Set("Content-Type", "application/json")
 json.NewEncoder(w).Encode(user)
}
```

## Middleware

```
func logging(next http.Handler) http.Handler {
 return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
 log.Printf("%s %s", r.Method, r.URL)
 next.ServeHTTP(w, r)
 })
}

http.Handle("/", logging(http.HandlerFunc(handler)))
```

## ServeMux

```
mux := http.NewServeMux()
mux.HandleFunc("/users", usersHandler)
mux.HandleFunc("/posts", postsHandler)

http.ListenAndServe(":8080", mux)
```

## Static Files

```
fs := http.FileServer(http.Dir("./static"))
http.Handle("/static/", http.StripPrefix("/static/", fs))
```

## Summary

Function	Purpose
<code>http.Get()</code>	Simple GET request
<code>http.Post()</code>	Simple POST request
<code>http.NewRequest()</code>	Custom request
<code>http.HandleFunc()</code>	Register handler
<code>http.ListenAndServe()</code>	Start server

## Worked example

JSON POST API with middleware, tested via `httpptest.NewServer`.

Save as `main.go`. Then:

```
go mod init example
go run .
```

```
package main

import (
 "encoding/json"
 "fmt"
 "io"
 "net/http"
 "net/http/httpptest"
 "strings"
)

type echoReq struct {
 Msg string `json:"msg"`
}

type echoResp struct {
 Echo string `json:"echo"`
}
```

```

func withJSON(next http.Handler) http.Handler {
 return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
 w.Header().Set("Content-Type", "application/json")
 next.ServeHTTP(w, r)
 })
}

func main() {
 mux := http.NewServeMux()
 mux.Handle("POST /echo", withJSON(http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
 var req echoReq
 if err := json.NewDecoder(r.Body).Decode(&req); err != nil || req.Msg == "" {
 w.WriteHeader(http.StatusBadRequest)
 _ = json.NewEncoder(w).Encode(map[string]string{"error": "bad json"})
 return
 }
 _ = json.NewEncoder(w).Encode(echoResp{Echo: strings.ToUpper(req.Msg)})
 })))

 srv := httptest.NewServer(mux)
 defer srv.Close()

 res, err := http.Post(srv.URL+"/echo", "application/json", strings.NewReader(`{"msg":"hi"}`))
 if err != nil {
 panic(err)
 }
 body, _ := io.ReadAll(res.Body)
 res.Body.Close()
 fmt.Println("status:", res.StatusCode)
 fmt.Println("body:", strings.TrimSpace(string(body)))
}

```

### Expected output:

```

status: 200
body: {"echo":"HI"}

```

## More examples

`httptest.NewRecorder` unit test of a pure handler (no listening socket).

```

package main

import (
 "fmt"

```

```

 "net/http"
 "net/http/httptest"
)

func health(w http.ResponseWriter, r *http.Request) {
 w.WriteHeader(http.StatusOK)
 _, _ = w.Write([]byte("ok"))
}

func main() {
 rec := httptest.NewRecorder()
 req := httptest.NewRequest(http.MethodGet, "/health", nil)
 health(rec, req)
 fmt.Println(rec.Code, rec.Body.String())
}

```

Expected output:

```
200 ok
```

## Runnable example

Save as `main.go`. Then:

```

go mod init example
go run .

package main

import (
 "context"
 "encoding/json"
 "fmt"
 "io"
 "log"
 "net/http"
 "net/http/httptest"
 "strings"
 "time"
)

type User struct {
 Name string `json:"name"`
}

```

```

func logging(next http.Handler) http.Handler {
 return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
 start := time.Now()
 next.ServeHTTP(w, r)
 log.Printf("%s %s %s", r.Method, r.URL.Path, time.Since(start))
 })
}

func userHandler(w http.ResponseWriter, r *http.Request) {
 w.Header().Set("Content-Type", "application/json")
 _ = json.NewEncoder(w).Encode(User{Name: "Alice"})
}

func main() {
 mux := http.NewServeMux()
 mux.HandleFunc("GET /users/{id}", userHandler)
 mux.HandleFunc("GET /health", func(w http.ResponseWriter, r *http.Request) {
 _, _ = io.WriteString(w, "ok")
 })

 // Middleware chain + real HTTP without ListenAndServe forever
 srv := httptest.NewServer(logging(mux))
 defer srv.Close()

 // Client with timeout + context
 client := &http.Client{Timeout: 2 * time.Second}
 ctx, cancel := context.WithTimeout(context.Background(), time.Second)
 defer cancel()

 req, err := http.NewRequestWithContext(ctx, http.MethodGet, srv.URL+"/users/42", nil)
 if err != nil {
 panic(err)
 }
 req.Header.Set("Accept", "application/json")

 resp, err := client.Do(req)
 if err != nil {
 panic(err)
 }
 defer resp.Body.Close()

 body, _ := io.ReadAll(resp.Body)
 fmt.Println("status:", resp.StatusCode)
 fmt.Println("content-type:", resp.Header.Get("Content-Type"))
 fmt.Println("body:", strings.TrimSpace(string(body)))

 // httptest.NewRecorder for pure handler unit tests (no server)

```



```
rec := httptest.NewRecorder()
req2 := httptest.NewRequest(http.MethodGet, "/health", nil)
mux.ServeHTTP(rec, req2)
fmt.Println("health:", rec.Code, strings.TrimSpace(rec.Body.String()))
}
```

**Expected output:** (log line may interleave; body lines are stable)

```
status: 200
content-type: application/json
body: {"name":"Alice"}
health: 200 ok
```

**What to notice:** `httptest.NewServer` is ideal for client+server integration examples; `httptest.NewRecorder` unit-tests handlers with zero network. Always set client timeouts and prefer `NewRequestWithContext`.

**Try next:** Add a `POST /users` that decodes JSON. Chain a second middleware that rejects missing Accept headers.

## Related Topics

- [Context and Cancellation](#)
- [Encoding and Serialization](#)

# Encoding and Serialization

## Overview

Go provides standard encoding packages for JSON, XML, and binary data.

## JSON

`encoding/json` remains the compatibility API. Since Go 1.27 it is **backed by the v2 implementation**: `marshal/unmarshal` behavior is preserved, error text may differ, and `unmarshal` is faster. New code that wants stricter defaults (reject invalid UTF-8, reject duplicate names, case-sensitive field match) should import `encoding/json/v2`. Rollback for a v1-compat emergency: `GOEXPERIMENT=nojsonv2` at build time (temporary; expected to go away).

```
import "encoding/json"

type User struct {
 Name string `json:"name"`
 Email string `json:"email,omitempty"`
 Age int `json:"-"` // Ignored
}

// Encode
user := User{Name: "Alice", Email: "a@b.com"}
data, _ := json.Marshal(user)
// {"name":"Alice","email":"a@b.com"}

// Decode
var u User
json.Unmarshal(data, &u)

// Pretty print
data, _ := json.MarshalIndent(user, "", " ")
```

## Streaming JSON

```
// Encode to writer
encoder := json.NewEncoder(w)
encoder.Encode(user)

// Decode from reader
decoder := json.NewDecoder(r)
decoder.Decode(&user)
```

## encoding/json/v2 (Go 1.27+)

```
import jsonv2 "encoding/json/v2"

b, err := jsonv2.Marshal(user)
err = jsonv2.Unmarshal(b, &user)
err = jsonv2.MarshalWrite(w, user)
err = jsonv2.UnmarshalRead(r.Body, &user)
```

Options configure semantics without a new Encoder type:

```
b, err := jsonv2.Marshal(user, jsonv2.RejectUnknownMembers(true))
```

Low-level token streaming lives in `encoding/json/jsontext`. Keep v1 (`encoding/json`) for existing code; migrate hot or strict APIs to v2 when you can test the stricter defaults.

## XML

```
import "encoding/xml"

type Person struct {
 XMLName xml.Name `xml:"person"`
 Name string `xml:"name"`
 Age int `xml:"age,attr"`
}

data, _ := xml.MarshalIndent(person, "", " ")
xml.Unmarshal(data, &p)
```

## Gob (Go Binary)

```
import "encoding/gob"

// Encode
var buf bytes.Buffer
enc := gob.NewEncoder(&buf)
enc.Encode(data)

// Decode
dec := gob.NewDecoder(&buf)
dec.Decode(&result)
```

## CSV

```
import "encoding/csv"

// Write
w := csv.NewWriter(file)
w.Write([]string{"a", "b", "c"})
w.Flush()

// Read
r := csv.NewReader(file)
records, _ := r.ReadAll()
```

## Base64

```
import "encoding/base64"

encoded := base64.StdEncoding.EncodeToString(data)
decoded, _ := base64.StdEncoding.DecodeString(encoded)
```

## Summary

Package	Format
encoding/json	JSON (v1 API; v2 engine in Go 1.27+)
encoding/json/v2	JSON with options and stricter defaults
encoding/xml	XML
encoding/gob	Go binary

Package	Format
encoding/csv	CSV
encoding/base64	Base64

## Worked example

Streaming JSON encode/decode round-trip over an in-memory buffer.

Save as `main.go`. Then:

```
go mod init example
go run .

package main

import (
 "bytes"
 "encoding/json"
 "fmt"
)

type Event struct {
 Type string `json:"type"`
 N int `json:"n"`
}

func main() {
 var buf bytes.Buffer
 enc := json.NewEncoder(&buf)
 for i := 1; i <= 3; i++ {
 if err := enc.Encode(Event{Type: "tick", N: i}); err != nil {
 panic(err)
 }
 }

 dec := json.NewDecoder(&buf)
 sum := 0
 for dec.More() {
 var e Event
 if err := dec.Decode(&e); err != nil {
 panic(err)
 }
 sum += e.N
 fmt.Printf("event type=%s n=%d\n", e.Type, e.N)
 }
}
```

```
 fmt.Println("sum:", sum)
}
```

Expected output:

```
event type=tick n=1
event type=tick n=2
event type=tick n=3
sum: 6
```

## More examples

Tag-driven JSON `omitempty` + gob preserving unexported-unrelated fields.

```
package main

import (
 "bytes"
 "encoding/gob"
 "encoding/json"
 "fmt"
)

type User struct {
 Name string `json:"name"`
 Email string `json:"email,omitempty"`
 Score int `json:"score,omitempty"`
}

func main() {
 u := User{Name: "Ada", Score: 0} // Score omitted when 0
 j, _ := json.Marshal(u)
 fmt.Println("json:", string(j))

 var buf bytes.Buffer
 _ = gob.NewEncoder(&buf).Encode(User{Name: "Ada", Score: 10})
 var out User
 _ = gob.NewDecoder(&buf).Decode(&out)
 fmt.Println("gob score:", out.Score)
}
```

Expected output:

```
json: {"name":"Ada"}
gob score: 10
```

## Runnable example

Save as `main.go`. Then:

```
go mod init example
go run .

package main

import (
 "bytes"
 "encoding/base64"
 "encoding/csv"
 "encoding/gob"
 "encoding/json"
 "encoding/xml"
 "fmt"
 "strings"
)

type User struct {
 Name string `json:"name" xml:"name"`
 Email string `json:"email,omitempty" xml:"email"`
 Age int `json:"- " xml:"- " // ignored by JSON/XML tags in this demo`
}

type PersonXML struct {
 XMLName xml.Name `xml:"person"`
 Name string `xml:"name"`
 Age int `xml:"age,attr"`
}

func main() {
 u := User{Name: "Alice", Email: "a@b.com", Age: 30}

 // JSON round-trip
 j, err := json.Marshal(u)
 if err != nil {
 panic(err)
 }
 var u2 User
 if err := json.Unmarshal(j, &u2); err != nil {
 panic(err)
 }
 fmt.Println("json:", string(j))
 fmt.Println("json name:", u2.Name, "age zeroed:", u2.Age == 0)
```

```

// XML round-trip
p := PersonXML{Name: "Bob", Age: 25}
x, err := xml.Marshal(p)
if err != nil {
 panic(err)
}
var p2 PersonXML
if err := xml.Unmarshal(x, &p2); err != nil {
 panic(err)
}
fmt.Println("xml:", string(x))
fmt.Println("xml age:", p2.Age)

// Gob round-trip (Go-native binary)
var buf bytes.Buffer
if err := gob.NewEncoder(&buf).Encode(u); err != nil {
 panic(err)
}
var u3 User
if err := gob.NewDecoder(&buf).Decode(&u3); err != nil {
 panic(err)
}
fmt.Println("gob age preserved:", u3.Age)

// CSV
var csvBuf strings.Builder
w := csv.NewWriter(&csvBuf)
_ = w.Write([]string{"name", "email"})
_ = w.Write([]string{u.Name, u.Email})
w.Flush()
records, err := csv.NewReader(strings.NewReader(csvBuf.String())).ReadAll()
if err != nil {
 panic(err)
}
fmt.Println("csv rows:", len(records), "data:", records[1])

// Base64
enc := base64.StdEncoding.EncodeToString([]byte("hi"))
dec, _ := base64.StdEncoding.DecodeString(enc)
fmt.Println("b64:", enc, "->", string(dec))
}

```

### Expected output:

```

json: {"name":"Alice","email":"a@b.com"}
json name: Alice age zeroed: true

```



```
xml: <person age="25"><name>Bob</name></person>
xml age: 25
gob age preserved: 30
csv rows: 2 data: [Alice a@b.com]
b64: aGk= -> hi
```

**What to notice:** `json:"-"` drops `Age` from JSON, so it comes back as zero—`gob` has no tags and preserves the full struct. XML attributes use `,attr`. Prefer streaming `Encoder/Decoder` for large payloads and HTTP bodies.

**Try next:** Use `json.MarshalIndent` for pretty output. Stream-decode a JSON array with `decoder.Token` / `decoder.More`.

## Related Topics

- [I/O and Streaming](#)
- [HTTP and Networking](#)

# **Performance and Tooling**

# Reflection in Practice

## Overview

The `reflect` package provides runtime type inspection. Use sparingly—prefer generics or interfaces when possible.

## Type and Value

```
import "reflect"

x := 42
t := reflect.TypeOf(x) // Type information
v := reflect.ValueOf(x) // Value information

fmt.Println(t.Name()) // int
fmt.Println(t.Kind()) // int
fmt.Println(v.Int()) // 42
```

## Inspecting Structs

```
type User struct {
 Name string `json:"name"`
 Email string `json:"email"`
}

u := User{Name: "Alice", Email: "a@b.com"}
t := reflect.TypeOf(u)
v := reflect.ValueOf(u)

for i := 0; i < t.NumField(); i++ {
 field := t.Field(i)
 value := v.Field(i)
 tag := field.Tag.Get("json")
 fmt.Printf("%s: %v (tag: %s)\n", field.Name, value, tag)
}
```

## Modifying Values

```
x := 42
v := reflect.ValueOf(&x).Elem() // Need pointer for modification
v.SetInt(100)
fmt.Println(x) // 100
```

## Calling Methods

```
type Calculator struct{}
func (c Calculator) Add(a, b int) int { return a + b }

c := Calculator{}
v := reflect.ValueOf(c)
method := v.MethodByName("Add")
args := []reflect.Value{reflect.ValueOf(2), reflect.ValueOf(3)}
result := method.Call(args)
fmt.Println(result[0].Int()) // 5
```

## When to Use

**Appropriate:** - Serialization/deserialization - ORM mapping - Dependency injection - Test utilities

**Avoid:** - Performance-critical paths - Simple type conversions - When generics suffice

## Summary

Function	Purpose
TypeOf()	Get type info
ValueOf()	Get value wrapper
Kind()	Base type category
Field()	Access struct field
MethodByName()	Access method

## Worked example

Copy exported fields from `map[string]any` into a struct via reflection.

Save as `main.go`. Then:

```

go mod init example
go run .

package main

import (
 "fmt"
 "reflect"
)

type User struct {
 Name string
 Age int
}

func mapToStruct(m map[string]any, dest any) error {
 v := reflect.ValueOf(dest)
 if v.Kind() != reflect.Pointer || v.Elem().Kind() != reflect.Struct {
 return fmt.Errorf("dest must be *struct")
 }
 v = v.Elem()
 t := v.Type()
 for i := 0; i < t.NumField(); i++ {
 f := t.Field(i)
 if !f.IsExported() {
 continue
 }
 raw, ok := m[f.Name]
 if !ok {
 continue
 }
 fv := v.Field(i)
 rv := reflect.ValueOf(raw)
 if !rv.Type().AssignableTo(fv.Type()) {
 return fmt.Errorf("field %s: cannot assign %T", f.Name, raw)
 }
 fv.Set(rv)
 }
 return nil
}

func main() {
 var u User
 err := mapToStruct(map[string]any{"Name": "Ada", "Age": 36}, &u)
 fmt.Println("user:", u, "err:", err)
}

```

### Expected output:

```
user: {Ada 36} err: <nil>
```

## More examples

Read struct tags (serializer-style).

```
package main

import (
 "fmt"
 "reflect"
)

type Row struct {
 ID int `json:"id"`
 Email string `json:"email,omitempty"`
 Skip string `json:"- "`
}

func main() {
 t := reflect.TypeOf(Row{})
 for i := 0; i < t.NumField(); i++ {
 f := t.Field(i)
 fmt.Printf("%s tag=%q\n", f.Name, f.Tag.Get("json"))
 }
}
```

### Expected output:

```
ID tag="id"
Email tag="email,omitempty"
Skip tag="- "
```

## Runnable example

Save as `main.go`. Then:

```
go mod init example
go run .
```

```

package main

import (
 "fmt"
 "reflect"
)

type User struct {
 Name string `json:"name"`
 Email string `json:"email"`
 age int // unexported: visible to reflect only within this package
}

type Calculator struct{}

func (Calculator) Add(a, b int) int { return a + b }

func main() {
 u := User{Name: "Alice", Email: "a@b.com", age: 30}
 t := reflect.TypeOf(u)
 v := reflect.ValueOf(u)

 fmt.Println("type:", t.Name(), "kind:", t.Kind())
 for i := 0; i < t.NumField(); i++ {
 f := t.Field(i)
 fmt.Printf("field %s tag=%q value=%v\n", f.Name, f.Tag.Get("json"), v.Field(i).Interface())
 }

 // Modify via pointer + Elem
 x := 42
 rv := reflect.ValueOf(&x).Elem()
 rv.SetInt(100)
 fmt.Println("modified x:", x)

 // Call method by name
 c := Calculator{}
 m := reflect.ValueOf(c).MethodByName("Add")
 out := m.Call([]reflect.Value{reflect.ValueOf(2), reflect.ValueOf(3)})
 fmt.Println("Add via reflect:", out[0].Int())
}

```

### Expected output:

```

type: User kind: struct
field Name tag="name" value=Alice
field Email tag="email" value=a@b.com
field age tag="" value=30

```

```
modified x: 100
Add via reflect: 5
```

**What to notice:** `ValueOf(x)` is not settable—you need a pointer and `Elem()`. Tags power serializers (JSON, ORMs). Prefer generics/interfaces when you know the types at compile time; reflection costs CPU and clarity.

**Try next:** Build a tiny `MapToStruct` helper that fills exported fields from `map[string]any`. Compare it to a typed constructor.

## Related Topics

- [Type Assertions](#)
- [Performance Tuning](#)



# Unsafe Code

## Overview

The `unsafe` package bypasses Go's type safety. Use only when absolutely necessary for performance or interoperability.

## `unsafe.Pointer`

```
import "unsafe"

// Convert between pointer types
var x int = 42
p := unsafe.Pointer(&x)
fp := (*float64)(p) // Reinterpret as float64 pointer
```

## Common Uses

### Memory Layout

```
type Data struct {
 a int32
 b int64
}

fmt.Println(unsafe.Sizeof(Data{})) // Size in bytes
fmt.Println(unsafe.Alignof(Data{})) // Alignment
fmt.Println(unsafe.Offsetof(Data{}.b)) // Field offset
```

### String to Bytes (Zero-Copy)

```
func stringToBytes(s string) []byte {
 return unsafe.Slice(unsafe.StringData(s), len(s))
}
```

## Accessing Unexported Fields

```
// Not recommended, but possible
type hidden struct {
 secret int
}

h := hidden{secret: 42}
p := unsafe.Pointer(&h)
secretPtr := (*int)(p)
fmt.Println(*secretPtr)
```

## Dangers

- Not portable across platforms
- May break with Go versions
- Undefined behavior if misused
- Bypasses garbage collector

## Guidelines

1. **Avoid if possible** - Use safer alternatives first
2. **Document thoroughly** - Explain why unsafe is needed
3. **Isolate usage** - Keep in small, well-tested functions
4. **Test extensively** - Including on target platforms

## Summary

Function	Purpose
<code>unsafe.Pointer</code>	Generic pointer type
<code>unsafe.Sizeof</code>	Size in bytes
<code>unsafe.Alignof</code>	Alignment requirement
<code>unsafe.Offsetof</code>	Field offset

## Worked example

Layout inspection: padding vs field reordering.

Save as `main.go`. Then:

```
go mod init example
go run .
```

```
package main
```

```
import (
 "fmt"
 "unsafe"
)
```

```
type Bad struct {
 A bool
 B int64
 C bool
}
```

```
type Good struct {
 B int64
 A bool
 C bool
}
```

```
func main() {
 fmt.Println("sizeof Bad:", unsafe.Sizeof(Bad{}))
 fmt.Println("sizeof Good:", unsafe.Sizeof(Good{}))
 fmt.Println("offset Bad.B:", unsafe.Offsetof(Bad{}.B))
 fmt.Println("offset Good.B:", unsafe.Offsetof(Good{}.B))
}
```

**Expected output:** (typical amd64/arm64)

```
sizeof Bad: 24
sizeof Good: 16
offset Bad.B: 8
offset Good.B: 0
```

## More examples

Safe-ish string data inspection (do not mutate the slice).

```
package main
```

```
import (
 "fmt"
 "unsafe"
)
```

```

)

func main() {
 s := "gopher"
 b := unsafe.Slice(unsafe.StringData(s), len(s))
 fmt.Println("len:", len(b), "as string:", string(b))
 // b[0] = 'x' // NEVER: would corrupt immutable string data
}

```

**Expected output:**

```
len: 6 as string: gopher
```

## Runnable example

Save as main.go. Then:

```

go mod init example
go run .

```

```

package main

import (
 "fmt"
 "unsafe"
)

type Data struct {
 A int32
 B int64
}

func main() {
 d := Data{A: 1, B: 2}
 fmt.Println("sizeof Data:", unsafe.Sizeof(d))
 fmt.Println("alignof Data:", unsafe.Alignof(d))
 fmt.Println("offsetof B:", unsafe.Offsetof(d.B))

 // Safe-ish modern helpers: inspect string backing without copy.
 // The returned slice must not be mutated; string data is immutable.
 s := "hello"
 ptr := unsafe.StringData(s)
 b := unsafe.Slice(ptr, len(s))
 fmt.Println("string bytes len:", len(b), "first:", string(b[0]))
}

```

```

// Reinterpret an int64's bits as float64 (IEEE754 bit pattern demo)
var n uint64 = 0x3FF0000000000000 // 1.0 in float64
f := *(*float64)(unsafe.Pointer(&n))
fmt.Println("bits as float64:", f)
}

```

**Expected output:** (sizes may vary by arch; on typical amd64/arm64)

```

sizeof Data: 16
alignof Data: 8
offsetof B: 8
string bytes len: 5 first: h
bits as float64: 1

```

**What to notice:** Padding makes `Data` larger than 4+8 raw bytes. `unsafe` skips type checks—misaligned or out-of-lifetime pointers are undefined behavior. Prefer `binary`, `math`, and normal conversions unless you measure a need.

**Try next:** Print `Sizeof` for `struct { a bool; b int64 }` vs reordered fields. Never store `unsafe.Pointer` derived from a stack variable past its lifetime.

## Related Topics

- [Understanding Memory](#)
- [Cgo](#)

# Cgo and Foreign Function Interfaces

## Overview

Cgo enables calling C code from Go. It's useful for interfacing with system libraries but adds complexity.

## Basic Cgo

```
package main

/*
#include <stdio.h>

void hello() {
 printf("Hello from C!\n");
}
*/
import "C"

func main() {
 C.hello()
}
```

## Passing Data

```
/*
int add(int a, int b) {
 return a + b;
}
*/
import "C"

func main() {
 result := C.add(C.int(2), C.int(3))
 fmt.Println(int(result)) // 5
}
```

## Strings

```
/*
#include <stdlib.h>
#include <string.h>

char* greet(const char* name) {
 char* buf = malloc(100);
 sprintf(buf, "Hello, %s!", name);
 return buf;
}
*/
import "C"
import "unsafe"

func main() {
 name := C.CString("World")
 defer C.free(unsafe.Pointer(name))

 result := C.greet(name)
 defer C.free(unsafe.Pointer(result))

 fmt.Println(C.GoString(result))
}
```

## Linking Libraries

```
// #cgo LDFLAGS: -lssl -lcrypto
// #include <openssl/sha.h>
import "C"
```

## Downsides

- **No cross-compilation** without a C cross-toolchain
- **CGO\_ENABLED=1** required (and that is often the *default* if a C compiler is installed)
- **Performance overhead** at the Go C boundary (stack switch, scheduler)
- **Complicates deployment** — libc comes back

That last point is the systems one. A `CGO_ENABLED=0` Linux binary talks to the kernel itself and needs no `libc.so`. Enable cgo and the linker may pull in **glibc or musl**. A glibc-linked binary then fails on Alpine with a confusing **not found** (the missing file is the dynamic linker `/lib64/ld-linux-x86-64.so.2`, not your program). See [Go as a Systems Language](#).

```
Fail the build if anything in the graph needs cgo:
CGO_ENABLED=0 go build ./...

See whether the artifact grew a libc dependency:
ldd ./myapp # Linux; "not a dynamic executable" is the happy line
```

## When to Use

Use for: - System libraries (OpenSSL, SQLite) - Hardware interfaces - Legacy code integration

Avoid for: - Simple functionality - When pure Go libraries exist - Cross-platform tools

## Summary

Task	Approach
Call C	Import pseudo-package “C”
Pass string	C.CString() / C.GoString()
Free memory	C.free(unsafe.Pointer(...))
Link library	#cgo LDFLAGS: -lname

## Worked example

Pure-Go FFI boundary stand-in: keep a thin wrapper package around “foreign” logic.

Save as `main.go`. Then:

```
go mod init example
go run .

package main

import "fmt"

// Imagine this lives behind cgo; callers only see Go types.
type Lib struct{}

func (Lib) Add(a, b int) int { return a + b }

func (Lib) Version() string { return "pure-go-shim/1.0" }

func main() {
 var lib Lib
```



```

 fmt.Println("version:", lib.Version())
 fmt.Println("add:", lib.Add(20, 22))
}

```

**Expected output:**

```

version: pure-go-shim/1.0
add: 42

```

## More examples

Document why you would enable cgo (feature gate).

```

package main

import (
 "fmt"
 "runtime"
)

func main() {
 fmt.Println("GOOS:", runtime.GOOS)
 fmt.Println("cgo available at compile time only; this binary is pure Go")
}

```

**Expected output:** (GOOS varies)

```

GOOS: darwin
cgo available at compile time only; this binary is pure Go

```

## Runnable example

**Default path is pure Go** so `go run .` works with `CGO_ENABLED=0`. The optional block below is a separate cgo program (needs a C toolchain).

Save as `main.go`. Then:

```

go mod init example
go run .

```

```

package main

import "fmt"

```

```
// Pure-Go stand-in for a tiny C helper.
func add(a, b int) int { return a + b }

func greet(name string) string {
 return "Hello, " + name + "!"
}

func main() {
 fmt.Println("add:", add(2, 3))
 fmt.Println(greet("World"))
 fmt.Println("pure Go: no cgo required")
}
```

### Expected output:

```
add: 5
Hello, World!
pure Go: no cgo required
```

Optional cgo twin (only if you have a C compiler; do **not** mix into the pure-Go file):

```
package main

/*
#include <stdlib.h>
int add(int a, int b) { return a + b; }
*/
import "C"
import (
 "fmt"
 "unsafe"
)

func main() {
 sum := int(C.add(2, 3))
 fmt.Println("add:", sum)
 cname := C.CString("World")
 defer C.free(unsafe.Pointer(cname))
 fmt.Println("c string bytes:", C.GoString(cname))
}
```

CGO\_ENABLED=1 go run .

**What to notice:** Crossing the C boundary has call overhead, complicates cross-compilation, and forces library/toolchain concerns. Prefer pure Go unless a C library is the product.

**Try next:** Compare `CGO_ENABLED=0 go build` vs `CGO_ENABLED=1 go build` binary sizes on your machine.

## Related Topics

- [Unsafe Code](#)
- [Building and Releasing](#)
- [Go as a Systems Language](#)
- [cgo and scheduler transitions](#)

# Performance Tuning

## Overview

Go provides built-in profiling tools for CPU, memory, and execution analysis.

## Benchmarks

```
func BenchmarkFoo(b *testing.B) {
 for i := 0; i < b.N; i++ {
 Foo()
 }
}
```

```
go test -bench=. -benchmem
```

## CPU Profiling

```
go test -cpuprofile=cpu.out -bench=.
go tool pprof cpu.out
```

### pprof Commands

```
(pprof) top10 # Top functions
(pprof) list FuncName # Annotated source
(pprof) web # Visual graph
```

## Memory Profiling

```
go test -memprofile=mem.out -bench=.
go tool pprof -alloc_space mem.out
```

## HTTP Profiling

```
import _ "net/http/pprof"

// Endpoints:
// /debug/pprof/
// /debug/pprof/heap
// /debug/pprof/goroutine
// /debug/pprof/goroutineleak (Go 1.27+; permanently blocked Gs)
// /debug/pprof/profile
```

## Tracing

```
go test -trace=trace.out
go tool trace trace.out
```

## Optimization Tips

### Reduce Allocations

```
// Preallocate
s := make([]int, 0, expectedSize)

// Reuse with sync.Pool
var pool = sync.Pool{New: func() any { return &Buffer{} }}

// strings.Builder for concatenation
var b strings.Builder
```

### Avoid Copying

```
// Pass pointer for large structs
func process(data *LargeStruct)

// Use slices instead of arrays
func process(data []byte)
```

## Summary

Tool	Purpose
<code>go test -bench</code>	Benchmarking
<code>go tool pprof</code>	Profile analysis
<code>go tool trace</code>	Execution tracing
<code>-gcflags=-m</code>	Escape analysis

## Worked example

Slice preallocation vs grow-from-zero (benchmark pair).

Save as `slice_test.go`. Then:

```
go mod init example
go test -bench=. -benchmem

package main

import "testing"

func grow(n int) []int {
 var s []int
 for i := 0; i < n; i++ {
 s = append(s, i)
 }
 return s
}

func prealloc(n int) []int {
 s := make([]int, 0, n)
 for i := 0; i < n; i++ {
 s = append(s, i)
 }
 return s
}

func BenchmarkGrow(b *testing.B) {
 for i := 0; i < b.N; i++ {
 _ = grow(1024)
 }
}

func BenchmarkPrealloc(b *testing.B) {
```

```

 for i := 0; i < b.N; i++ {
 _ = prealloc(1024)
 }
}

```

**Expected output:** (Prealloc far fewer allocs)

BenchmarkGrow-8	...	... ns/op	... B/op	~10 allocs/op
BenchmarkPrealloc-8	...	... ns/op	8192 B/op	1 allocs/op

## More examples

sync.Pool buffer reuse microbench.

```

package main

import (
 "sync"
 "testing"
)

var pool = sync.Pool{New: func() any { return make([]byte, 0, 256) }}

func BenchmarkPool(b *testing.B) {
 for i := 0; i < b.N; i++ {
 buf := pool.Get().([]byte)
 buf = buf[:0]
 buf = append(buf, "hello"...)
 pool.Put(buf)
 }
}

func BenchmarkAlloc(b *testing.B) {
 for i := 0; i < b.N; i++ {
 buf := make([]byte, 0, 256)
 buf = append(buf, "hello"...)
 _ = buf
 }
}

go test -bench='Benchmark(Pool|Alloc)' -benchmem

```

## Runnable example

Save as `concat_test.go` (benchmarks live in `_test.go` files). Then:

```
go mod init example
go test -bench=. -benchmem

package main

import (
 "strings"
 "testing"
)

func concatPlus(n int) string {
 s := ""
 for i := 0; i < n; i++ {
 s += "x"
 }
 return s
}

func concatBuilder(n int) string {
 var b strings.Builder
 b.Grow(n)
 for i := 0; i < n; i++ {
 b.WriteByte('x')
 }
 return b.String()
}

func BenchmarkConcatPlus(b *testing.B) {
 for i := 0; i < b.N; i++ {
 _ = concatPlus(64)
 }
}

func BenchmarkConcatBuilder(b *testing.B) {
 for i := 0; i < b.N; i++ {
 _ = concatBuilder(64)
 }
}
```

**Expected output:** (numbers vary by machine; Builder should allocate far less)



```

goos: ...
goarch: ...
pkg: example
BenchmarkConcatPlus-8 200000 6000 ns/op 2000 B/op 60 all
BenchmarkConcatBuilder-8 10000000 100 ns/op 64 B/op 1 all
PASS

```

**What to notice:** `-benchmem` reports `B/op` and `allocs/op`—often more actionable than raw ns. Pre-sizing (`Grow`) and builders beat naive `+=` on hot paths. Always re-benchmark after a “fix.”

**Try next:** Add `BenchmarkConcatPrealloc` using `make([]byte, 0, n)`. Profile with `go test -bench=Builder -cpuprofile=cpu.out` and open `go tool pprof cpu.out`.

## Related Topics

- [Garbage Collection](#)
- [Understanding Memory](#)

## Advanced Profiling (pprof/trace)

# Performance: Deep Dives with pprof & trace

Optimization without measurement is just guesswork. Go has arguably the best built-in profiling tools of any compiled language.

## 1. pprof: The Sampling Profiler

pprof creates a statistical profile of your program.

### Usage

Add one line to your main:

```
import _ "net/http/pprof"

func main() {
 go func() {
 http.ListenAndServe("localhost:6060", nil)
 }()
 // ... app code ...
}
```

Now, while your app runs under load, grab a profile:

```
go tool pprof -http=:8081 http://localhost:6060/debug/pprof/profile?seconds=30
```

This opens a web UI. Look at: \* **Flame Graph:** The widest bars are using the most CPU. \* **Alloc Space:** Who allocated the most memory (Heap). \* **Alloc Objects:** Who created the most *count* of objects (high GC pressure). \* **Goroutine leak profile (Go 1.27+):** /debug/pprof/goroutineleak lists Gs blocked on unreachable concurrency primitives.

### Mutex Profiling

If your CPU usage is low but performance sucks, you have contention. Enable mutex profiling in code: `runtime.SetMutexProfileFraction(1)` Then look at /debug/pprof/mutex to see who is waiting for locks.

## 2. Execution Tracer (go tool trace)

While pprof aggregates data, **Trace** shows you a timeline of *every event*. \* When did a Goroutine start? \* When did it block on a channel? \* When did GC pause execution?

Capture a trace:

```
curl -o trace.out http://localhost:6060/debug/pprof/trace?seconds=5
go tool trace trace.out
```

**Use Case:** Debugging latency spikes. You might see a 100ms gap where *nothing* ran. Zooming in, you might find a Stop-The-World GC event or a single mutex holding up 50 goroutines.

## 3. Benchmarks as Profiling Inputs

You can profile specific functions using benchmarks:

```
go test -bench=. -cpuprofile=cpu.out
go tool pprof -http=:8080 cpu.out
```

## Summary

- **pprof:** For “Where is my CPU/RAM going?”
- **trace:** For “Why is there latency?” and “What is the scheduler doing?”
- Optimization loop: Measure -> Fix -> Verify (Benchmark).

## Worked example

Two algorithms, one clear allocation winner—profile-friendly benchmarks.

Save as `join_test.go`. Then:

```
go mod init example
go test -bench=. -benchmem
go test -bench=Builder -cpuprofile=cpu.out -benchmem
```

```
package main
```

```
import (
 "strings"
 "testing"
)
```

```

func joinPlus(parts []string) string {
 s := ""
 for _, p := range parts {
 s += p
 }
 return s
}

func joinBuilder(parts []string) string {
 var b strings.Builder
 for _, p := range parts {
 b.WriteString(p)
 }
 return b.String()
}

var parts = []string{"go", "is", "fast", "enough", "when", "measured"}

func BenchmarkPlus(b *testing.B) {
 for i := 0; i < b.N; i++ {
 _ = joinPlus(parts)
 }
}

func BenchmarkBuilder(b *testing.B) {
 for i := 0; i < b.N; i++ {
 _ = joinBuilder(parts)
 }
}

```

**Expected output:** (Builder lower B/op and allocs/op)

```

BenchmarkPlus-8 ns/op ... B/op ... allocs/op
BenchmarkBuilder-8 ns/op ... B/op ~1 allocs/op

```

## More examples

Emit a short execution trace from a benchmark.

```

go test -bench=Builder -trace=trace.out
go tool trace trace.out

```

```

// same package as above; no extra code required for -trace

```

## Runnable example

Save as `alloc_test.go`. Then:

```
go mod init example
go test -bench=. -benchmem
optional: write a CPU profile for go tool pprof
go test -bench=SumPrealloc -cpuprofile=cpu.out -benchmem
```

```
package main

import "testing"

func sumAlloc(n int) int {
 // allocates a new backing array every call
 s := make([]int, 0)
 for i := 0; i < n; i++ {
 s = append(s, i)
 }
 total := 0
 for _, v := range s {
 total += v
 }
 return total
}

func sumPrealloc(n int) int {
 s := make([]int, 0, n)
 for i := 0; i < n; i++ {
 s = append(s, i)
 }
 total := 0
 for _, v := range s {
 total += v
 }
 return total
}

func BenchmarkSumAlloc(b *testing.B) {
 for i := 0; i < b.N; i++ {
 _ = sumAlloc(1024)
 }
}

func BenchmarkSumPrealloc(b *testing.B) {
 for i := 0; i < b.N; i++ {
```

```

 _ = sumPrealloc(1024)
 }
}

```

**Expected output:** (SumPrealloc should show ~1 alloc/op vs many for SumAlloc)

BenchmarkSumAlloc-8	100000	12000 ns/op	20000 B/op	10 allocs/op
BenchmarkSumPrealloc-8	200000	6000 ns/op	8192 B/op	1 allocs/op
PASS				

**What to notice:** Benchmarks are excellent profiling inputs—no long-running server required. Allocation differences show up clearly with `-benchmem` before you open a flame graph. For live services, the chapter’s `net/http/pprof` endpoints are the online equivalent.

**Try next:** `go tool pprof -http=:8080 cpu.out` after the profile command. Capture a short trace with `go test -trace=trace.out -bench=SumPrealloc` then `go tool trace trace.out`.

## **The Modern Toolkit**



# The Modern 2026 Toolkit

A senior Go engineer is defined by their tools. Using standard `grep`/`find` is slow. Upgrade your terminal.

## General Tools (Rust-powered, Go-essential)

These aren't written in Go, but every Go dev uses them. 1. **ripgrep** (**rg**): The fastest text search. \* `rg "func main" -t go`: Find main in Go files only. 2. **fd**: Faster `find`. \* `fd -e go`: Find all Go files. 3. **fzf**: Fuzzy finder. Pipe `fd` into `fzf` to open files instantly. 4. **jq**: JSON processor. Crucial for debugging API responses or logs (`zap/slog` JSON output).

## Go-Specific Tools

### `gopsutil`

If you are building system tools (agents, monitors), [github.com/shirou/gopsutil](https://github.com/shirou/gopsutil) is the standard for reading CPU/Memory/Disk stats cross-platform.

### `go fix` Modernizers (1.26–1.27)

Go 1.26 rebuilt `go fix` around modernizers and `//go:fix inline`. Go 1.27 adds `atomictypes`, `embedlit`, `slicesbackward`, and `unsafefuncs`; the old `waitgroup` analyzer is now `waitgroupgo`; `fmtappendf` was removed.

```
go fix -modernize ./...
```

Examples of what it fixes: \* Replaces manual loops with `slices.Contains`. \* Updates older `interface{}` to `any`. \* Updates deprecated package usage. \* Atomic type aliases, embed composite literals, `slices.Backward`, unsafe helpers.

Inline fix for one location:

```
//go:fix inline
func has(xs []string, x string) bool {
 for _, v := range xs {
 if v == x {
 return true
 }
 }
}
```

```
 }
 return false
}
```

## govulncheck

The official vulnerability scanner.

```
go install golang.org/x/vuln/cmd/govulncheck@latest
govulncheck ./...
```

It tells you if *your code actually calls* the vulnerable function in a dependency. This drastically reduces false positives compared to generic scanners (Dependabot).

## Workflow Integration

Don't run these manually. Put them in a **Makefile** or **Taskfile** (Task is a modern Make alternative written in Go).

```
Taskfile.yaml
tasks:
 audit:
 cmds:
 - go vet ./...
 - staticcheck ./...
 - govulncheck ./...
```

## Worked example

slices / maps modern stdlib helpers vs hand-rolled loops.

Save as util.go and util\_test.go. Then:

```
go mod init example
go test -v
```

```
// util.go
package main

import (
 "maps"
 "slices"
)
```

```

func has(xs []string, x string) bool {
 return slices.Contains(xs, x)
}

func cloneMap(m map[string]int) map[string]int {
 return maps.Clone(m)
}

// util_test.go
package main

import "testing"

func TestHas(t *testing.T) {
 xs := []string{"a", "b", "c"}
 if !has(xs, "b") || has(xs, "z") {
 t.Fatal("contains mismatch")
 }
}

func TestCloneMap(t *testing.T) {
 m := map[string]int{"x": 1}
 c := cloneMap(m)
 c["x"] = 9
 if m["x"] != 1 {
 t.Fatal("clone should be independent")
 }
}

```

### Expected output:

```

=== RUN TestHas
--- PASS: TestHas (0.00s)
=== RUN TestCloneMap
--- PASS: TestCloneMap (0.00s)
PASS

```

## More examples

Table of “grep-friendly” symbols your toolkit will find.

```

package main

import "fmt"

```

```
// AuditTarget is intentionally distinctive for rg demos.
func AuditTarget() string { return "ok" }

func main() {
 fmt.Println(AuditTarget())
}

in a shell with ripgrep installed:
rg 'func AuditTarget' -t go
go run .
```

## Runnable example

**Note:** Tools like `rg`, `staticcheck`, and `govulncheck` are external CLIs. This stdlib demo captures the *modernizer* idea: replace a hand-rolled loop with `slices.Contains`, and run `go vet`-friendly code under `go test`.

Save as `modern.go` and `modern_test.go`. Then:

```
go mod init example
go test -v
go vet ./...

// modern.go
package main

import "slices"

// legacyHas is the pre-slices style many codebases still have.
func legacyHas(xs []string, x string) bool {
 for _, v := range xs {
 if v == x {
 return true
 }
 }
 return false
}

// modernHas is the 1.21+ equivalent (what go fix -modernize targets).
func modernHas(xs []string, x string) bool {
 return slices.Contains(xs, x)
}
```

```
// modern_test.go
package main

import "testing"

func TestHas(t *testing.T) {
 xs := []string{"go", "rust", "zig"}
 if !legacyHas(xs, "go") || !modernHas(xs, "go") {
 t.Fatal("expected to find go")
 }
 if legacyHas(xs, "c") || modernHas(xs, "c") {
 t.Fatal("did not expect c")
 }
 if legacyHas(xs, "rust") != modernHas(xs, "rust") {
 t.Fatal("implementations diverged")
 }
}
```

**Expected output:**

```
=== RUN TestHas
--- PASS: TestHas (0.00s)
PASS
```

**What to notice:** Stdlib `slices` / `maps` replace a lot of utility code. Keep a small `go test + go vet` loop before pulling heavier linters. `govulncheck` still needs a separate install—run it in CI on real modules.

**Try next:** Run `go fix -modernize ./...` on a module that still uses `interface{}` and manual contains loops. Pipe JSON logs through `jq` in your shell.

# Designing for the Long Term

## Overview

Maintainable Go code follows consistent patterns and architectural principles.

## Package Design

```
myapp/
 cmd/ # Entry points
 server/
 internal/ # Private packages
 auth/
 database/
 handlers/
 pkg/ # Public libraries
 api/ # API definitions
```

## Dependency Direction

```
cmd → internal → pkg
 ↓
 external
```

Lower layers should not import higher layers.

## Interface Segregation

```
// Small, focused interfaces
type Reader interface { Read([]byte) (int, error) }
type Writer interface { Write([]byte) (int, error) }

// Compose when needed
type ReadWriter interface {
 Reader
 Writer
```

```
}
```

## Options Pattern

```
type Server struct {
 host string
 port int
 timeout time.Duration
}

type Option func(*Server)

func WithPort(p int) Option {
 return func(s *Server) { s.port = p }
}

func NewServer(opts ...Option) *Server {
 s := &Server{host: "localhost", port: 8080}
 for _, opt := range opts {
 opt(s)
 }
 return s
}
```

## Error Handling

```
// Wrap with context
return fmt.Errorf("repository.GetUser: %w", err)

// Custom error types for inspection
type NotFoundError struct{ ID int }
func (e NotFoundError) Error() string { ... }
```

## Testing Strategy

- Unit tests for business logic
- Integration tests for boundaries
- Table-driven tests for coverage
- Mocks for external dependencies

## Guidelines

1. **Keep packages small** - Single responsibility
2. **Accept interfaces** - Flexible dependencies
3. **Return structs** - Clear types
4. **Handle errors early** - Fail fast
5. **Document exported APIs** - Clear comments

## Summary

Principle	Implementation
Encapsulation	<code>internal/</code> packages
Abstraction	Small interfaces
Flexibility	Options pattern
Testability	Dependency injection

## Worked example

Dependency injection with a tiny interface + table-driven unit tests.

Save as `greeter.go` and `greeter_test.go`. Then:

```
go mod init example
go test -v

// greeter.go
package main

type Clock interface {
 Hour() int
}

type fixedClock struct{ h int }

func (f fixedClock) Hour() int { return f.h }

func Greeting(c Clock, name string) string {
 if name == "" {
 name = "friend"
 }
 if c.Hour() < 12 {
 return "Good morning, " + name
 }
}
```



```

 return "Hello, " + name
}

// greeter_test.go
package main

import "testing"

func TestGreeting(t *testing.T) {
 tests := []struct {
 hour int
 name string
 want string
 }{
 {9, "Ada", "Good morning, Ada"},
 {15, "Ada", "Hello, Ada"},
 {8, "", "Good morning, friend"},
 }
 for _, tt := range tests {
 got := Greeting(fixedClock{tt.hour}, tt.name)
 if got != tt.want {
 t.Fatalf("hour=%d name=%q got %q want %q", tt.hour, tt.name, got, tt.want)
 }
 }
}

```

### Expected output:

```

=== RUN TestGreeting
--- PASS: TestGreeting (0.00s)
PASS

```

## More examples

Functional options with defaults.

```

package main

import "fmt"

type Cfg struct {
 Addr string
 N int
}

```

```

type Opt func(*Cfg)

func WithAddr(a string) Opt { return func(c *Cfg) { c.Addr = a } }
func WithN(n int) Opt { return func(c *Cfg) { c.N = n } }

func New(opts ...Opt) Cfg {
 c := Cfg{Addr: ":8080", N: 1}
 for _, o := range opts {
 o(&c)
 }
 return c
}

func main() {
 fmt.Printf("%+v\n", New(WithN(3)))
}

```

Expected output:

```
{Addr::8080 N:3}
```

## Runnable example

Save as main.go. Then:

```

go mod init example
go run .

package main

import (
 "fmt"
 "io"
 "strings"
 "time"
)

// Small interfaces at the boundary
type Greeter interface {
 Greet(name string) string
}

type formalGreeter struct{}

func (formalGreeter) Greet(name string) string {

```

```

 return "Hello, " + name
}

// Options pattern for construction
type Server struct {
 host string
 port int
 timeout time.Duration
 greeter Greeter
}

type Option func(*Server)

func WithPort(p int) Option {
 return func(s *Server) { s.port = p }
}

func WithTimeout(d time.Duration) Option {
 return func(s *Server) { s.timeout = d }
}

func WithGreeter(g Greeter) Option {
 return func(s *Server) { s.greeter = g }
}

func NewServer(opts ...Option) *Server {
 s := &Server{
 host: "localhost",
 port: 8080,
 timeout: time.Second,
 greeter: formalGreeter{},
 }
 for _, opt := range opts {
 opt(s)
 }
 return s
}

func (s *Server) Handle(r io.Reader) (string, error) {
 b, err := io.ReadAll(r)
 if err != nil {
 return "", fmt.Errorf("server.Handle: %w", err)
 }
 name := strings.TrimSpace(string(b))
 if name == "" {
 return "", fmt.Errorf("server.Handle: empty name")
 }
}

```

```

 return s.greeter.Greet(name), nil
}

func main() {
 srv := NewServer(WithPort(9090), WithTimeout(2*time.Second))
 fmt.Printf("listen %s:%d timeout=%s\n", srv.host, srv.port, srv.timeout)

 msg, err := srv.Handle(strings.NewReader("Ada"))
 if err != nil {
 panic(err)
 }
 fmt.Println(msg)

 _, err = srv.Handle(strings.NewReader(" "))
 fmt.Println("empty err:", err)
}

```

### Expected output:

```

listen localhost:9090 timeout=2s
Hello, Ada
empty err: server.Handle: empty name

```

**What to notice:** Defaults live in `NewServer`; options only override what callers care about. Accepting `io.Reader` and a small `Greeter` interface keeps the core testable without a network. Errors wrap with `%w` and a stable prefix.

**Try next:** Inject a mock `Greeter` that returns `"yo " + name`. Add `WithHost` and a table-driven test for validation.

## Related Topics

- [Interface Best Practices](#)
- [How Go Code Is Organized](#)

# Static Analysis and Security

## Overview

Go provides built-in and third-party tools for code quality and security analysis.

### go vet

```
go vet ./...
```

Catches: - Printf format errors - Unreachable code - Suspicious constructs

### staticcheck

```
go install honnef.co/go/tools/cmd/staticcheck@latest
staticcheck ./...
```

Catches: - Deprecated APIs - Simplifications - Performance issues

### golangci-lint

```
Install
go install github.com/golangci/golangci-lint/cmd/golangci-lint@latest

Run all linters
golangci-lint run
```

### govulncheck

```
go install golang.org/x/vuln/cmd/govulncheck@latest
govulncheck ./...
```

Checks dependencies for known vulnerabilities.

## gosec

```
go install github.com/securego/gosec/v2/cmd/gosec@latest
gosec ./...
```

Security-focused linter: - SQL injection - Hardcoded credentials - Weak crypto

## Common Security Issues

```
// SQL injection - BAD
query := "SELECT * FROM users WHERE id = " + id

// Use parameters - GOOD
db.Query("SELECT * FROM users WHERE id = ?", id)

// Path traversal - BAD
path := filepath.Join(baseDir, userInput)

// Sanitize - GOOD
if strings.Contains(userInput, "..") {
 return errors.New("invalid path")
}
```

## CI Integration

```
.github/workflows/lint.yml
- name: Lint
 run: golangci-lint run

- name: Security scan
 run: |
 govulncheck ./...
 gosec ./...
```

## Summary

Tool	Focus
go vet	Basic correctness
staticcheck	Advanced analysis
golangci-lint	Multiple linters

Tool	Focus
govulncheck	Vulnerability scan
gosec	Security issues

## Worked example

Path traversal guards unit-tested (the boring fix scanners want).

Save as `pathx.go` and `pathx_test.go`. Then:

```
go mod init example
go test -v
go vet ./...
```

```
// pathx.go
package main
```

```
import (
 "errors"
 "path/filepath"
 "strings"
)
```

```
func UnderRoot(root, user string) (string, error) {
 if user == "" || strings.Contains(user, "..") {
 return "", errors.New("invalid path")
 }
 full := filepath.Clean(filepath.Join(root, user))
 root = filepath.Clean(root)
 sep := string(filepath.Separator)
 if full != root && !strings.HasPrefix(full, root+sep) {
 return "", errors.New("escape")
 }
 return full, nil
}
```

```
// pathx_test.go
package main
```

```
import (
 "path/filepath"
 "testing"
)
```

```

func TestUnderRoot(t *testing.T) {
 root := filepath.Join("var", "data")
 ok, err := UnderRoot(root, "a.txt")
 if err != nil || ok != filepath.Join(root, "a.txt") {
 t.Fatalf("got %q err=%v", ok, err)
 }
 if _, err := UnderRoot(root, "../etc/passwd"); err == nil {
 t.Fatal("expected error")
 }
}

```

**Expected output:**

```

=== RUN TestUnderRoot
--- PASS: TestUnderRoot (0.00s)
PASS

```

## More examples

SQL placeholder style vs string concat (pattern comparison only).

```

package main

import (
 "fmt"
 "strings"
)

func main() {
 q := "SELECT * FROM t WHERE id=?"
 args := []any{"42"}
 fmt.Println("placeholder query:", q, "args:", args)
 bad := "SELECT * FROM t WHERE id='" + "x" + "' OR '1'='1'"
 fmt.Println("concat is dangerous:", strings.Contains(bad, "OR"))
}

```

**Expected output:**

```

placeholder query: SELECT * FROM t WHERE id=? args: [42]
concat is dangerous: true

```



## Runnable example

External scanners (`staticcheck`, `gosec`, `govulncheck`) need separate installs. This stdlib program demonstrates **safe vs unsafe** patterns those tools flag, and runs cleanly under `go vet` / `go test`.

Save as `safe.go` and `safe_test.go`. Then:

```
go mod init example
go test -v
go vet ./...

// safe.go
package main

import (
 "errors"
 "path/filepath"
 "strings"
)

// badQuery shows string-built SQL (do not use with real drivers).
func badQuery(id string) string {
 return "SELECT * FROM users WHERE id = '" + id + "'"
}

// goodQuery keeps user data out of the SQL string (placeholder style).
func goodQuery() (query string, args []any) {
 return "SELECT * FROM users WHERE id = ?", []any{"42"}
}

// safeJoin cleans user input and rejects paths that escape base.
func safeJoin(base, userInput string) (string, error) {
 if userInput == "" || strings.Contains(userInput, "..") {
 return "", errors.New("invalid path")
 }
 // Build under base, then verify the cleaned result still has base as prefix.
 full := filepath.Clean(filepath.Join(base, userInput))
 baseClean := filepath.Clean(base)
 sep := string(filepath.Separator)
 if full != baseClean && !strings.HasPrefix(full, baseClean+sep) {
 return "", errors.New("invalid path")
 }
 return full, nil
}
```

```
// safe_test.go
package main

import (
 "path/filepath"
 "testing"
)

func TestSafeJoin(t *testing.T) {
 base := filepath.Join("var", "data")
 ok, err := safeJoin(base, "notes.txt")
 if err != nil {
 t.Fatal(err)
 }
 if ok != filepath.Join(base, "notes.txt") {
 t.Fatalf("got %q", ok)
 }
 if _, err := safeJoin(base, "../etc/passwd"); err == nil {
 t.Fatal("expected rejection")
 }
}

func TestGoodQuery(t *testing.T) {
 q, args := goodQuery()
 if q == badQuery("x") {
 t.Fatal("good query should not embed user input")
 }
 if len(args) != 1 {
 t.Fatal("expected bound args")
 }
}
}
```

### Expected output:

```
=== RUN TestSafeJoin
--- PASS: TestSafeJoin (0.00s)
=== RUN TestGoodQuery
--- PASS: TestGoodQuery (0.00s)
PASS
```

**What to notice:** Parameterized queries and path containment are the boring fixes **gosec** pushes for. **go vet** is free CI; layer **staticcheck** / **golangci-lint** / **govulncheck** once the module is real.

**Try next:** Install **staticcheck** and run it on this package. Introduce an unused write and watch the tool complain.

## Related Topics

- [Formatting, Vetting, Docs](#)
- [Building and Releasing](#)

# Code Generation and Embedding

## Overview

`go generate` runs commands to generate code, and `//go:embed` embeds files into binaries.

## `go generate`

```
//go:generate stringer -type=Status

type Status int

const (
 Pending Status = iota
 Approved
 Rejected
)

go generate ./...
```

## Common Generators

- `stringer` - String methods for enums
- `mockgen` - Mock interfaces for testing
- `protoc` - Protocol buffer code

## Embedding Files

```
import "embed"

//go:embed config.json
var configData []byte

//go:embed templates/*
var templates embed.FS
```

```
//go:embed static
var staticFiles embed.FS
```

## Reading Embedded Files

```
//go:embed data.txt
var data string // As string

//go:embed data.bin
var data []byte // As bytes

//go:embed files/*
var fs embed.FS // As filesystem

content, _ := fs.ReadFile("files/config.json")
```

## HTTP File Server

```
//go:embed static/*
var static embed.FS

func main() {
 fs := http.FileServer(http.FS(static))
 http.Handle("/static/", fs)
 http.ListenAndServe(":8080", nil)
}
```

## Templates

```
//go:embed templates/*.html
var templates embed.FS

tmpl := template.Must(template.ParseFS(templates, "templates/*.html"))
```

## Summary

Directive	Purpose
<code>//go:generate cmd</code>	Run code generator
<code>//go:embed file</code>	Embed as string/bytes
<code>//go:embed dir/*</code>	Embed as filesystem

## Worked example

Generate a tiny constants file at runtime (stand-in for `go generate` output), then use it.

Save as `main.go`. Then:

```
go mod init example
go run .
```

```
package main
```

```
import (
 "fmt"
 "strings"
)
```

```
func generateVersionGo(version string) string {
 return fmt.Sprintf("// Code generated by demo; DO NOT EDIT.\npackage main\n\nconst Gener
}
```

```
func main() {
 src := generateVersionGo("1.2.3")
 // Real workflows write this via //go:generate (stringer, sqlc, mockgen, ...).
 fmt.Println(strings.TrimSpace(src))
 fmt.Println("lines:", strings.Count(src, "\n"))
}
```

Expected output:

```
// Code generated by demo; DO NOT EDIT.
package main

const GeneratedVersion = "1.2.3"
lines: 4
```

## More examples

`//go:embed` string + FS in one program (needs a local file).

```
printf 'hi' > note.txt
```

```
package main
```

```
import (
 "embed"
```

```

 "fmt"
)

//go:embed note.txt
var note string

//go:embed note.txt
var fs embed.FS

func main() {
 fmt.Println("string:", note)
 b, _ := fs.ReadFile("note.txt")
 fmt.Println("fs:", string(b))
}

```

Expected output:

```

string: hi
fs: hi

```

## Runnable example

Save as `main.go` and create a sibling `hello.txt` with the text embedded `hello`. Then:

```

go mod init example
printf 'embedded hello' > hello.txt
go run .

```

```

package main

import (
 "embed"
 "fmt"
 "io/fs"
 "strings"
)

//go:embed hello.txt
var hello string

//go:embed hello.txt
var helloBytes []byte

//go:embed hello.txt
var content embed.FS

```

```
// version is normally produced by go:generate; pure-Go fallback for this demo.
var version = "dev"

func main() {
 fmt.Println("embed string:", hello)
 fmt.Println("embed bytes:", string(helloBytes))

 b, err := content.ReadFile("hello.txt")
 if err != nil {
 panic(err)
 }
 fmt.Println("embed FS:", string(b))

 // Walk the embedded FS (handy for static assets)
 _ = fs.WalkDir(content, ".", func(path string, d fs.DirEntry, err error) error {
 if err != nil || d.IsDir() {
 return err
 }
 fmt.Println("walk:", path)
 return nil
 })

 // Stand-in for generated code: a tiny template expansion
 generated := strings.ReplaceAll(
 "// Code generated by demo; DO NOT EDIT.\nconst Version = \"{{V}}\"\n",
 "{{V}}",
 version,
)
 fmt.Println(strings.TrimSpace(generated))
}
```

### Expected output:

```
embed string: embedded hello
embed bytes: embedded hello
embed FS: embedded hello
walk: hello.txt
// Code generated by demo; DO NOT EDIT.
const Version = "dev"
```

**What to notice:** `//go:embed` paths are relative to the source file and bake content into the binary—no runtime file needed after build. Real `go generate` tools (`stringer`, `sqlc`, `mockgen`) write Go you commit or produce in CI; keep generators deterministic.

**Try next:** Embed a `static/*` directory and serve it with `http.FileServer(http.FS(static))` via `httptest`. Add a `//go:generate echo package main > zzz_gen.go` and run `go generate`.



## Related Topics

- [Core Go Commands](#)
- [Building and Releasing](#)

# Index

## Quick Reference Guide

### Essential Commands

Command	Purpose
<code>go run main.go</code>	Compile and run
<code>go build</code>	Build binary
<code>go test ./...</code>	Run tests
<code>go mod tidy</code>	Sync dependencies
<code>go fmt ./...</code>	Format code
<code>go vet ./...</code>	Check for issues

### Common Patterns

Pattern	Chapter
Error handling	<a href="#">Ch 33</a>
Table-driven tests	<a href="#">Ch 38</a>
Options pattern	<a href="#">Ch 58</a>
Worker pool	<a href="#">Ch 40</a>
Context cancellation	<a href="#">Ch 43</a>

### Type Reference

Type	Zero Value
<code>bool</code>	<code>false</code>
<code>int</code>	<code>0</code>
<code>string</code>	<code>""</code>
<code>pointer</code>	<code>nil</code>
<code>slice</code>	<code>nil</code>
<code>map</code>	<code>nil</code>

### Key Packages

Package	Purpose
<code>fmt</code>	Formatted I/O
<code>io</code>	I/O interfaces
<code>os</code>	OS operations
<code>net/http</code>	HTTP client/server
<code>encoding/json</code>	JSON encoding
<code>sync</code>	Synchronization
<code>context</code>	Cancellation
<code>testing</code>	Test framework

## Resources

- [go.dev](https://golang.org/) - Official site
- [pkg.go.dev](https://pkg.go.dev/) - Package docs
- [Go by Example](#)
- [Effective Go](#)

## Worked example

Mini checklist program: type switch + options + timed work.

Save as `main.go`. Then:

```
go mod init example
go run .

package main

import (
 "fmt"
 "time"
)

type Opt func(*cfg)
type cfg struct{ n int }

func withN(n int) Opt { return func(c *cfg) { c.n = n } }

func describe(v any) string {
 switch x := v.(type) {
 case string:
 return fmt.Sprintf("string(%d)", len(x))
 case int:
 return fmt.Sprintf("int(%d)", x)
 }
```

```

 default:
 return fmt.Sprintf("%T", v)
 }
}

func main() {
 c := cfg{n: 2}
 for _, o := range []Opt{withN(4)} {
 o(&c)
 }
 start := time.Now()
 sum := 0
 for i := 0; i < c.n*100; i++ {
 sum += i
 }
 fmt.Println("describe:", describe("go"))
 fmt.Println("sum:", sum, "took:", time.Since(start) < time.Second)
}

```

Expected output:

```

describe: string(2)
sum: 79800 took: true

```

## More examples

Point yourself at the right deep-dive with a tiny table printer.

```

package main

import "fmt"

func main() {
 rows := [][]string{
 {"errors.Is/As", "05-errors"},
 {"table tests", "06-testing"},
 {"WaitGroup/select", "07-concurrency"},
 {"httptest", "08-web"},
 {"bench/pprof", "09-performance"},
 }
 for _, r := range rows {
 fmt.Printf("%-18s → %s\n", r[0], r[1])
 }
}

```

Expected output:

```
errors.Is/As → 05-errors
table tests → 06-testing
WaitGroup/select → 07-concurrency
httptest → 08-web
bench/pprof → 09-performance
```

## Runnable example

Tour of this part in one small program: reflection-ish type switch, a micro “options” knob, and a timed loop you could later benchmark.

Save as `main.go`. Then:

```
go mod init example
go run .

package main

import (
 "fmt"
 "time"
 "unsafe"
)

type Config struct {
 Label string
 N int
}

type Option func(*Config)

func WithN(n int) Option {
 return func(c *Config) { c.N = n }
}

func NewConfig(opts ...Option) Config {
 c := Config{Label: "perf-tour", N: 3}
 for _, o := range opts {
 o(&c)
 }
 return c
}

func describe(v any) string {
 switch x := v.(type) {
```

```

 case string:
 return fmt.Sprintf("string len=%d", len(x))
 case int:
 return fmt.Sprintf("int %d", x)
 default:
 return fmt.Sprintf("%T", v)
}

}

func main() {
 cfg := NewConfig(WithN(5))
 fmt.Println("config:", cfg.Label, cfg.N)
 fmt.Println("describe:", describe(cfg.Label))

 // Layout peek (see unsafe chapter for caveats)
 fmt.Println("sizeof Config:", unsafe.Sizeof(cfg))

 // Tiny workload worth timing / benchmarking later
 start := time.Now()
 sum := 0
 for i := 0; i < cfg.N*1000; i++ {
 sum += i
 }
 fmt.Println("sum:", sum, "took:", time.Since(start).Truncate(time.Microsecond))
}

```

**Expected output:** (sizeof and timing vary by arch/machine)

```

config: perf-tour 5
describe: string len=9
sizeof Config: 24
sum: 12497500 took: 0s

```

**What to notice:** This part mixes *language escape hatches* (reflect/unsafe/cgo), *measurement* (bench/pprof), and *engineering hygiene* (design, static analysis, generate, docs). Measure before micro-optimizing.

**Try next:** Move the loop into `sum_test.go` as a benchmark with `-benchmem`. Open related chapters from the table above as you hit each tool.

# Documentation and APIs

## Overview

Good documentation makes code usable. Go has built-in tools for extracting docs from source comments.

## Writing Doc Comments

```
// Package math provides mathematical utilities.
//
// It includes functions for basic arithmetic and
// statistical calculations.
package math

// Pi represents the mathematical constant .
const Pi = 3.14159

// Add returns the sum of a and b.
func Add(a, b int) int {
 return a + b
}

// Calculator provides stateful calculations.
type Calculator struct {
 // Result holds the current value.
 Result float64
}
```

## Conventions

- Start with the name being documented
- Use complete sentences
- First sentence is the summary

## Examples

```
func ExampleAdd() {
 result := Add(2, 3)
 fmt.Println(result)
 // Output: 5
}
```

## Viewing Docs

```
go doc fmt
go doc fmt.Println
go doc -all mypackage
```

## pkg.go.dev

Public packages are automatically documented at:

<https://pkg.go.dev/github.com/user/package>

## API Design Guidelines

```
// Accept interfaces, return structs
func Process(r io.Reader) (*Result, error)

// Use functional options
func NewServer(opts ...Option) *Server

// Clear error messages
return fmt.Errorf("user %d: %w", id, err)
```

## Summary

Tool	Purpose
go doc	View documentation
godoc	Local doc server
pkg.go.dev	Public package docs
Examples	Testable documentation



## Worked example

Example tests as executable docs (`// Output: checked by go test`).

Save as `mathdoc.go` and `mathdoc_test.go`. Then:

```
go mod init example
go test -v -run Example

// mathdoc.go
package main

// Mul returns a * b.
func Mul(a, b int) int { return a * b }

// mathdoc_test.go
package main

import "fmt"

func ExampleMul() {
 fmt.Println(Mul(6, 7))
 // Output: 42
}

func ExampleMul_zero() {
 fmt.Println(Mul(0, 99))
 // Output: 0
}
```

### Expected output:

```
=== RUN ExampleMul
--- PASS: ExampleMul (0.00s)
=== RUN ExampleMul_zero
--- PASS: ExampleMul_zero (0.00s)
PASS
```

## More examples

Godoc-shaped comments you can print with `go doc`.

```
// Package main demonstrates documentation comments.
//
// Start package comments with "Package <name>".
```

```

package main

// Answer is the answer to life, the universe, and everything.
const Answer = 42

// Double returns 2 * n.
func Double(n int) int { return n * 2 }

func main() {}

go doc Double

```

## Runnable example

Save as docdemo.go and docdemo\_test.go. Then:

```

go mod init example
go test -v
go doc -all .

// docdemo.go
// Package docdemo shows godoc-friendly comments and example tests.
//
// Doc comments become the package page on pkg.go.dev.
package main

import "fmt"

// Pi is a truncated mathematical constant for demos.
const Pi = 3.14159

// Add returns the sum of a and b.
//
// It does not check for overflow.
func Add(a, b int) int {
 return a + b
}

// Calculator keeps a running total.
type Calculator struct {
 // Result is the current accumulator value.
 Result int
}

```

```
// AddTo adds n to the calculator result and returns the new total.
func (c *Calculator) AddTo(n int) int {
 c.Result += n
 return c.Result
}

func main() {
 fmt.Println(Add(2, 3))
}

// docdemo_test.go
package main

import "fmt"

func ExampleAdd() {
 fmt.Println(Add(2, 3))
 // Output: 5
}

func ExampleCalculator_AddTo() {
 c := &Calculator{}
 fmt.Println(c.AddTo(4))
 fmt.Println(c.AddTo(6))
 // Output:
 // 4
 // 10
}
```

### Expected output:

```
=== RUN ExampleAdd
--- PASS: ExampleAdd (0.00s)
=== RUN ExampleCalculator_AddTo
--- PASS: ExampleCalculator_AddTo (0.00s)
PASS
```

go doc Add prints something like:

```
package main // import "example"

func Add(a, b int) int
 Add returns the sum of a and b.

 It does not check for overflow.
```

**What to notice:** Example functions are tests—the `// Output:` block is checked by `go test`. The

first sentence of a doc comment is the summary shown in indexes. Start comments with the name (Add returns...).

**Try next:** Run `go test -run Example`. Add a package comment and view it with `go doc package`.

## Related Topics

- [Formatting, Vetting, Docs](#)
- [Interface Best Practices](#)

# Infrastructure

## Containerization (Docker)

# Containerization: Multi-Stage Builds

## Overview

Go compiles to a static binary. That is the deployment superpower: production does not need the Go toolchain, a language runtime, or a package manager. Multi-stage Docker builds turn that property into tiny, hard-to-attack images—typically tens of megabytes instead of hundreds.

This chapter covers the production multi-stage pattern, static linking, distroless vs scratch, CA certs and timezones, non-root runtime, healthchecks, and a release checklist.

## Why Multi-Stage Matters

A naive Dockerfile copies the whole module tree into a fat `golang` image and runs the binary there. That ships the compiler, shell, package manager, and build tools into production—large attack surface, slow pulls, noisy CVEs.

Multi-stage splits the world in two:

1. **Builder** — full Go toolchain; compile once.
2. **Runtime** — only the binary (and maybe CA certs / tzdata). No shell, no package manager.

```
source + go.mod
 |
 v
[builder stage] go build --> static binary
 |
 v
[runtime stage] copy binary only --> ~10-20 MB image
```

## The Production Multi-Stage Pattern

```
syntax=docker/dockerfile:1

----- Stage 1: Build -----
FROM golang:1.27-bookworm AS builder

WORKDIR /src
```

```

Cache modules separately from source for faster rebuilds.
COPY go.mod go.sum ./
RUN go mod download

COPY . .

CGO_ENABLED=0 => pure static binary (no libc dependency)
-trimpath => strip local paths from the binary
-ldflags="-s -w" => strip symbol table and DWARF
ARG TARGETOS=linux
ARG TARGETARCH=amd64
RUN CGO_ENABLED=0 GOOS=${TARGETOS} GOARCH=${TARGETARCH} \
 go build -trimpath -ldflags="-s -w" -o /out/server ./cmd/server

----- Stage 2: Runtime -----
distroless/static includes CA certs + timezone data; no shell.
FROM gcr.io/distroless/static-debian12:nonroot

COPY --from=builder /out/server /server

USER nonroot:nonroot
EXPOSE 8080
ENTRYPOINT ["/server"]

```

Build and inspect:

```

docker build -t myapp:local .
docker images myapp:local
docker run --rm -p 8080:8080 myapp:local

```

## Static Linking Essentials

Flag / env	Effect
CGO_ENABLED=0	Pure Go binary; runs without glibc/musl
-trimpath	Removes host filesystem paths from the binary
-ldflags="-s -w"	Smaller binary; harder casual reverse engineering
-buildmode=pie	Position-independent executable (ASLR-friendly)

If you **must** use CGO (SQLite via C, certain crypto, etc.):



```
FROM golang:1.27-bookworm AS builder
RUN apt-get update && apt-get install -y --no-install-recommends gcc libc6-dev
...
RUN CGO_ENABLED=1 go build -o /out/server ./cmd/server

FROM gcr.io/distroless/base-debian12:nonroot
COPY --from=builder /out/server /server
```

Prefer `CGO_ENABLED=0` whenever the dependency graph allows it.

A glibc-linked binary copied onto Alpine fails with `not found` because the ELF interpreter (`/lib64/ld-linux-x86-64.so.2`) is missing — not because your file vanished. That is the same trap as shipping a Ubuntu C tool onto Alpine. Pure Go sidesteps it: on Linux the runtime issues syscalls itself, so `scratch` and `distroless/static` have no libc to provide. See [Go as a Systems Language](#).

## Scratch vs Distroless vs Alpine

Base	Size	Shell	CA certs	Best for
<code>scratch</code>	~0 + binary	No	No	Fully static apps that never call HTTPS / <code>LoadLocation</code>
<code>distroless/static</code>	~2 MB + binary	No	Yes	Most production Go services
<code>distroless/base</code>	larger	No	Yes	CGO / dynamic glibc needs
<code>alpine</code>	~5–8 MB	Yes	Optional	Debugging, ops scripts (larger attack surface)

**Rule of thumb:** start with `distroless/static-debian12:nonroot`. Drop to `scratch` only when you have proven CA/tz needs are handled and you want absolute minimalism.

## CA Certificates and Timezones

`scratch` fails the moment your app dials `https://...` or calls `time.LoadLocation("America/New_York")`.

Manual copy if you insist on `scratch`:

```
FROM golang:1.27-bookworm AS builder
... build ...

FROM scratch
COPY --from=builder /etc/ssl/certs/ca-certificates.crt /etc/ssl/certs/
COPY --from=builder /usr/share/zoneinfo /usr/share/zoneinfo
```

```
COPY --from=builder /out/server /server
ENTRYPOINT ["/server"]
```

In application code, prefer UTC in storage and convert only at the edge:

```
func main() {
 // Prefer UTC in process; convert at presentation boundaries.
 time.Local = time.UTC

 loc, err := time.LoadLocation("America/Los_Angeles")
 if err != nil {
 log.Fatal(err) // fail closed if tzdata missing in image
 }
 _ = loc
}
```

## Non-Root, Health, and Signals

Distroless :nonroot images run as UID 65532. Match that in compose/k8s security contexts.

```
docker-compose snippet
services:
 api:
 image: myapp:local
 read_only: true
 security_opt:
 - no-new-privileges:true
 cap_drop:
 - ALL
 ports:
 - "8080:8080"
```

Handle SIGTERM so orchestrators get clean shutdown:

```
package main

import (
 "context"
 "log"
 "net/http"
 "os"
 "os/signal"
 "syscall"
 "time"
)
```

```

func main() {
 mux := http.NewServeMux()
 mux.HandleFunc("/healthz", func(w http.ResponseWriter, _ *http.Request) {
 w.WriteHeader(http.StatusOK)
 _, _ = w.Write([]byte("ok"))
 })
 mux.HandleFunc("/readyz", func(w http.ResponseWriter, _ *http.Request) {
 // Check DB/cache if needed; fail closed when not ready.
 w.WriteHeader(http.StatusOK)
 })

 srv := &http.Server{
 Addr: ":8080",
 Handler: mux,
 ReadHeaderTimeout: 5 * time.Second,
 }

 go func() {
 log.Printf("listening on %s", srv.Addr)
 if err := srv.ListenAndServe(); err != nil && err != http.ErrServerClosed {
 log.Fatal(err)
 }
 }()

 stop := make(chan os.Signal, 1)
 signal.Notify(stop, syscall.SIGINT, syscall.SIGTERM)
 <-stop

 ctx, cancel := context.WithTimeout(context.Background(), 15*time.Second)
 defer cancel()
 if err := srv.Shutdown(ctx); err != nil {
 log.Printf("shutdown: %v", err)
 }
}

```

## Build Cache and Layer Hygiene

- Copy go.mod / go.sum before source so module download caches across code-only changes.
- Use .dockerignore aggressively:

```

.git
_book
**/*_test.go
**/.DS_Store
*.md

```

```
.env
tmp/
```

- Multi-arch (Buildx):

```
docker buildx build \
 --platform linux/amd64,linux/arm64 \
 -t ghcr.io/example/myapp:1.2.3 \
 --push .
```

Pass `TARGETOS` / `TARGETARCH` (BuildKit sets them automatically when using `ARG TARGETOS` / `ARG TARGETARCH`).

## Embed Version Metadata

```
// cmd/server/main.go
var (
 version = "dev"
 commit = "none"
)

func main() {
 log.Printf("starting version=%s commit=%s", version, commit)
}

ARG VERSION=dev
ARG COMMIT=none
RUN CGO_ENABLED=0 go build -trimpath \
 -ldflags="-s -w -X main.version=${VERSION} -X main.commit=${COMMIT}" \
 -o /out/server ./cmd/server
```

## Production Checklist

- ☐ Multi-stage: builder + minimal runtime (`distroless` or carefully prepared `scratch`)
- ☐ `CGO_ENABLED=0` unless CGO is required and documented
- ☐ `-trimpath` and `-ldflags="-s -w"` on release builds
- ☐ Non-root user (`:nonroot` or explicit `USER`)
- ☐ CA certs present if the app makes HTTPS calls
- ☐ Timezone data present if `LoadLocation` is used
- ☐ `.dockerignore` excludes secrets, tests, and junk
- ☐ `/healthz` and `/readyz` (or platform equivalents)
- ☐ Graceful shutdown on `SIGTERM`
- ☐ Image scanned in CI (`trivy`, `grype`, or registry scanning)
- ☐ Tags immutable (`1.2.3`, `digest`)—avoid relying only on `latest`

## Common Pitfalls

1. **Shell debugging in distroless** — there is no `/bin/sh`. Debug with a sidecar, ephemeral debug image, or temporary Alpine stage—not by shipping a shell to prod.
2. **CGO by accident** — a transitive dependency may force CGO; verify with `go build -x` or `ldd` (static binary should not need dynamic libs).
3. **Wrong GOOS/GOARCH** — building on Apple Silicon without setting `GOARCH` can produce arm64 images that fail on amd64 clusters.
4. **Huge context** — missing `.dockerignore` sends `node_modules`, `.git`, and secrets into the daemon.
5. **Listening on localhost only** — inside a container, bind `0.0.0.0:8080` (or `:8080`), not `127.0.0.1`.
6. **Root for convenience** — root + writable filesystem + shell is the classic container escape path.

## Exercises

1. **Minimal image** — Take any small `net/http` server and produce a multi-stage Dockerfile using `distroless/static-debian12:nonroot`. Measure image size with `docker images`.
2. **HTTPS smoke test** — Call an external HTTPS API from inside the container. Confirm it works on `distroless` and fails on bare `scratch` without CA certs.
3. **Multi-arch** — Build amd64 and arm64 images with `Buildx`. Run the matching arch on your machine or in CI.
4. **Signal drill** — Send `docker stop` and confirm your server logs a clean `Shutdown` within the grace period (not a hard kill after timeout).
5. **Scan** — Run `trivy image myapp:local` (or equivalent) and fix or document any HIGH/CRITICAL findings in base layers.

## More examples

### Health probe and graceful drain (container contract)

Containers need a cheap liveness probe and a clean `SIGTERM` path. This demo uses `httptest` so it finishes without a forever-listen loop, and models drain with a cancellable context.

```
mkdir -p /tmp/go-docker-health && cd /tmp/go-docker-health
go mod init example.com/docker-health
```

Save as `main.go`:

```
package main

import (
 "context"
```

```

"encoding/json"
"fmt"
"net/http"
"net/http/httptest"
"sync/atomic"
"time"
)

func main() {
 var ready atomic.Bool
 ready.Set(true)

 mux := http.NewServeMux()
 mux.HandleFunc("GET /healthz", func(w http.ResponseWriter, _ *http.Request) {
 w.WriteHeader(http.StatusOK)
 fmt.Fprintln(w, "ok")
 })
 mux.HandleFunc("GET /readyz", func(w http.ResponseWriter, _ *http.Request) {
 if !ready.Load() {
 http.Error(w, "not ready", http.StatusServiceUnavailable)
 return
 }
 _ = json.NewEncoder(w).Encode(map[string]string{"status": "ready"})
 })
 mux.HandleFunc("GET /work", func(w http.ResponseWriter, r *http.Request) {
 select {
 case <-r.Context().Done():
 return
 case <-time.After(20 * time.Millisecond):
 fmt.Fprintln(w, "done")
 }
 })

 // Simulate drain: mark not-ready, then cancel in-flight work.
 srv := httptest.NewServer(mux)
 defer srv.Close()

 resp, err := http.Get(srv.URL + "/healthz")
 if err != nil {
 panic(err)
 }
 resp.Body.Close()
 fmt.Println("healthz:", resp.StatusCode)

 resp, err = http.Get(srv.URL + "/readyz")
 if err != nil {
 panic(err)
 }

```

```

 }
 resp.Body.Close()
 fmt.Println("readyz before drain:", resp.StatusCode)

 ready.Store(false)
 resp, err = http.Get(srv.URL + "/readyz")
 if err != nil {
 panic(err)
 }
 resp.Body.Close()
 fmt.Println("readyz during drain:", resp.StatusCode)

 ctx, cancel := context.WithTimeout(context.Background(), 5*time.Millisecond)
 defer cancel()
 req, _ := http.NewRequestWithContext(ctx, http.MethodGet, srv.URL+"/work", nil)
 _, err = http.DefaultClient.Do(req)
 fmt.Println("in-flight cancelled:", err != nil)
}

go run .

```

### Expected output:

```

healthz: 200
readyz before drain: 200
readyz during drain: 503
in-flight cancelled: true

```

### Bind address from env (not localhost-only)

```

mkdir -p /tmp/go-docker-bind && cd /tmp/go-docker-bind
go mod init example.com/docker-bind

```

Save as main.go:

```

package main

import (
 "fmt"
 "net"
 "os"
)

func listenAddr() string {
 if p := os.Getenv("PORT"); p != "" {

```

```

 return ":" + p
 }
 return ":8080"
}

func main() {
 // Resolve without accepting forever: prove the address form containers expect.
 addr := listenAddr()
 ln, err := net.Listen("tcp", "127.0.0.1"+addr) // local only for the demo
 if err != nil {
 panic(err)
 }
 fmt.Println("configured bind form:", addr)
 fmt.Println("listening:", ln.Addr().String())
 _ = ln.Close()

 os.Setenv("PORT", "9090")
 fmt.Println("with PORT=9090:", listenAddr())
}

go run .

```

### Expected output:

```

configured bind form: :8080
listening: 127.0.0.1:8080
with PORT=9090: :9090

```

## Runnable example

Multi-stage Docker is prose and Dockerfiles; the Go side is a release-shaped binary with version ldflags, a health endpoint, and graceful shutdown—the process you copy into distroless.

```

mkdir -p /tmp/go-docker-app && cd /tmp/go-docker-app
go mod init example.com/docker-app

```

Save as `main.go`:

```

package main

import (
 "context"
 "encoding/json"
 "fmt"
 "log"

```



```

 "net"
 "net/http"
 "time"
)

// Injected at link time:
// go build -ldflags="-X main.version=1.2.3 -X main.commit=deadbeef"
var (
 version = "dev"
 commit = "none"
)

func main() {
 mux := http.NewServeMux()
 mux.HandleFunc("GET /healthz", func(w http.ResponseWriter, _ *http.Request) {
 w.WriteHeader(http.StatusOK)
 _, _ = w.Write([]byte("ok"))
 })
 mux.HandleFunc("GET /version", func(w http.ResponseWriter, _ *http.Request) {
 w.Header().Set("Content-Type", "application/json")
 _ = json.NewEncoder(w).Encode(map[string]string{
 "version": version,
 "commit": commit,
 })
 })
}

ln, err := net.Listen("tcp", "127.0.0.1:0")
if err != nil {
 log.Fatal(err)
}
srv := &http.Server{Handler: mux, ReadHeaderTimeout: 5 * time.Second}

go func() {
 if err := srv.Serve(ln); err != nil && err != http.ErrServerClosed {
 log.Fatal(err)
 }
}()

base := "http://" + ln.Addr().String()
fmt.Printf("listening on %s version=%s commit=%s\n", base, version, commit)

resp, err := http.Get(base + "/healthz")
if err != nil {
 log.Fatal(err)
}
resp.Body.Close()
fmt.Println("healthz:", resp.StatusCode)

```

```

 resp, err = http.Get(base + "/version")
 if err != nil {
 log.Fatal(err)
 }
 var v map[string]string
 _ = json.NewDecoder(resp.Body).Decode(&v)
 resp.Body.Close()
 fmt.Printf("version payload: %v\n", v)

 shutdownCtx, cancel := context.WithTimeout(context.Background(), 2*time.Second)
 defer cancel()
 if err := srv.Shutdown(shutdownCtx); err != nil {
 log.Fatal(err)
 }
 fmt.Println("shutdown complete")
}

go run -ldflags="-X main.version=1.2.3 -X main.commit=deadbeef" .
release-shaped binary (what multi-stage copies):
CGO_ENABLED=0 go build -trimpath -ldflags="-s -w -X main.version=1.2.3 -X main.commit=dead

```

**Expected output** (illustrative):

```

listening on http://127.0.0.1:54321 version=1.2.3 commit=deadbeef
healthz: 200
version payload: map[commit:deadbeef version:1.2.3]
shutdown complete

```

## What to notice

- Version injection is link-time (-X); the runtime image never needs the Go toolchain.
- /healthz is what Docker HEALTHCHECK and orchestrators probe—keep it cheap and dependency-light.
- Shutdown is the process side of multi-stage ops: `docker stop` / Kubernetes send `SIGTERM` and expect a drain.

## Try next

- Wire `signal.NotifyContext` and run under `docker stop` to confirm clean exit within the grace period.
- Wrap this binary in the multi-stage Dockerfile from this chapter; compare size to a single-stage `golang` image.

## Related Topics

- [Building and Releasing](#)

- CI/CD & Hardening
- Application Hardening

## **Platform Guides: Render & Leapcell**

# Deploying Go: Render, Leapcell, and VPS

## Overview

Go's static binary makes PaaS and VPS deployment straightforward: build once, run the file. In 2026 the common paths are managed platforms (Render, Leapcell, Fly, Railway) for speed, and a small VPS with systemd when you want full control and predictable cost.

This chapter walks through Render, Leapcell-style serverless Go, a production systemd unit, environment and secrets, health checks, and when to choose each model.

## Deployment Models Compared

Model	Cold start	Cost shape	Ops burden	Good for
PaaS (Render web service)	Warm (always-on or min instances)	Monthly / usage	Low	APIs, webhooks, workers
Serverless binary (Leapcell, similar)	Milliseconds–low seconds	Scale-to-zero	Very low	Spiky or low-traffic APIs
VPS + systemd	None (always running)	Flat \$	Medium	Homelab, steady load, custom networking
Containers on K8s	Depends	Cluster fixed + usage	High	Multi-service platforms

Go fits all four: same binary shape, different process supervisors.

## Render ([render.com](https://render.com))

Render detects Go repositories and builds with the Go toolchain on their builders.

## Typical setup

1. Push the repo to GitHub/GitLab.
2. New **Web Service** → connect the repo.
3. Configure:

Field	Example
Runtime	Go
Build Command	<code>go build -o bin/server ./cmd/server</code>
Start Command	<code>./bin/server</code>
Health Check Path	<code>/healthz</code>

## Production build command

Prefer flags you would use locally for release:

```
CGO_ENABLED=0 go build -trimpath -ldflags="-s -w" -o bin/server ./cmd/server
```

If you need a specific module path:

```
go build -o bin/server ./cmd/api
```

## Environment and ports

Render injects `PORT`. **Always bind to `PORT`**, not a hard-coded 8080:

```
package main

import (
 "log"
 "net/http"
 "os"
 "time"
)

func main() {
 port := os.Getenv("PORT")
 if port == "" {
 port = "8080"
 }

 mux := http.NewServeMux()
 mux.HandleFunc("/healthz", func(w http.ResponseWriter, _ *http.Request) {
 w.WriteHeader(http.StatusOK)
 })
}
```

```

 _, _ = w.Write([]byte("ok"))
})
mux.HandleFunc("/", func(w http.ResponseWriter, r *http.Request) {
 _, _ = w.Write([]byte("hello from render"))
})

srv := &http.Server{
 Addr: ":",
 Handler: mux,
 ReadHeaderTimeout: 5 * time.Second,
}
log.Printf("listening on %s", srv.Addr)
log.Fatal(srv.ListenAndServe())
}

```

## Databases and private networking

- Managed Postgres/Redis: set DATABASE\_URL / REDIS\_URL as env vars (never commit them).
- Private services: talk over the internal hostname Render provides; do not expose internal ports publicly.
- Migrations: run as a **one-off job** or release command, not on every web dyno boot if multiple instances race.

```

Example release / pre-deploy idea (platform-specific)
./bin/migrate up
./bin/server

```

## Pros and cons

**Pros:** Free/hobby tiers, managed TLS, Git-driven deploys, managed data stores, low ops.

**Cons:** Less control than a VPS; cold starts on free tiers; build minutes and instance sizes are the cost levers.

## Leapcell (and Serverless Go)

Leapcell targets **serverless Go**: scale to zero when idle, spin up quickly (often Firecracker-style microVMs). Unlike AWS Lambda's custom handler contract, many of these platforms run a normal **net/http** server binary.

## Conceptual difference from Lambda

```
AWS Lambda: event JSON --> handler(ctx, event) --> response JSON
Leapcell-like: cold start binary --> listen :PORT --> ordinary HTTP
```

You keep idiomatic Go HTTP code, middleware, and routers.

## Config sketch

```
leapcell.yaml (illustrative)
runtime: go
entrypoint: ./server
memory / region set in dashboard or CI
```

Build locally the same way:

```
CGO_ENABLED=0 go build -trimpath -ldflags="-s -w" -o server ./cmd/server
```

## Design for cold starts

1. **Defer heavy work** — open DB pools on first request or in a short `init` with timeouts; avoid multi-second global init.
2. **Small binary** — strip symbols; avoid huge embedded assets unless needed.
3. **Idempotent migrations** — do not migrate schema on every cold start.
4. **Stateless handlers** — session state in Redis/DB/cookies, not process memory alone.

```
var dbOnce sync.Once
var db *sql.DB
var dbErr error

func getDB(ctx context.Context) (*sql.DB, error) {
 dbOnce.Do(func() {
 dsn := os.Getenv("DATABASE_URL")
 db, dbErr = sql.Open("pgx", dsn)
 if dbErr != nil {
 return
 }
 db.SetMaxOpenConns(5)
 db.SetConnMaxLifetime(5 * time.Minute)
 pingCtx, cancel := context.WithTimeout(ctx, 3*time.Second)
 defer cancel()
 dbErr = db.PingContext(pingCtx)
 })
 return db, dbErr
}
```



```
}
```

## VPS Deployment with systemd

For a \$5–15/mo Linux VPS (Hetzner, DigitalOcean, Linode):

### 1. Provision a non-root user

```
sudo useradd --system --home /var/lib/myapp --shell /usr/sbin/nologin myapp
sudo mkdir -p /var/lib/myapp /etc/myapp
sudo chown myapp:myapp /var/lib/myapp
```

### 2. Install the binary

```
CGO_ENABLED=0 GOOS=linux GOARCH=amd64 go build -trimpath -ldflags="-s -w" -o myapp ./cmd/server
scp myapp deploy@host:/tmp/myapp
ssh deploy@host 'sudo install -o myapp -g myapp -m 0755 /tmp/myapp /usr/local/bin/myapp'
```

### 3. Environment file

```
/etc/myapp.env (mode 0640, root:myapp)
PORT=8080
DATABASE_URL=postgres://...
LOG_LEVEL=info
```

### 4. Unit file

```
/etc/systemd/system/myapp.service
[Unit]
Description=My Go App
After=network-online.target
Wants=network-online.target

[Service]
Type=simple
User=myapp
Group=myapp
EnvironmentFile=/etc/myapp.env
WorkingDirectory=/var/lib/myapp
```

```

ExecStart=/usr/local/bin/myapp
Restart=on-failure
RestartSec=2
Hardening
NoNewPrivileges=true
ProtectSystem=strict
ProtectHome=true
PrivateTmp=true
ReadWritePaths=/var/lib/myapp
AmbientCapabilities=
CapabilityBoundingSet=
LimitNOFILE=65536

[Install]
WantedBy=multi-user.target

```

```

sudo systemctl daemon-reload
sudo systemctl enable --now myapp
sudo journalctl -u myapp -f

```

## 5. Reverse proxy (Caddy or nginx)

Terminate TLS at the edge; proxy to 127.0.0.1:8080:

```

api.example.com {
 reverse_proxy 127.0.0.1:8080
}

```

## Health, Metrics, and Zero-Downtime

Regardless of platform:

```

mux.HandleFunc("GET /healthz", func(w http.ResponseWriter, _ *http.Request) {
 w.WriteHeader(http.StatusOK)
})

mux.HandleFunc("GET /readyz", func(w http.ResponseWriter, r *http.Request) {
 if err := pingDependencies(r.Context()); err != nil {
 http.Error(w, "not ready", http.StatusServiceUnavailable)
 return
 }
 w.WriteHeader(http.StatusOK)
})

```

- **Liveness** (/healthz): process is up.
- **Readiness** (/readyz): safe to receive traffic (DB reachable, migrations done).

For rolling deploys on a VPS, run two units behind the proxy or use a brief drain period with `systemctl reload patterns` and connection draining.

## Secrets Hygiene

Do	Don't
Platform secret store / <code>EnvironmentFile</code> with tight permissions	Commit <code>.env</code> with production credentials
Rotate DB passwords and API keys on a schedule	Share one root DB user across apps
Inject secrets at runtime	Bake secrets into the binary or image layers
Scope tokens to least privilege	Put admin tokens in frontend env

## Choosing a Path

```
Need scale-to-zero + almost no ops? --> Leapcell-style serverless
Need managed Postgres + Git deploys? --> Render / similar PaaS
Need full control / homelab / cost? --> VPS + systemd + Caddy
Multi-team multi-service mesh? --> Containers / K8s (separate chapter)
```

## Production Checklist

- ☐ Bind to PORT (or platform-provided address)
- ☐ /healthz and /readyz implemented and wired in the platform
- ☐ Release build flags (`CGO_ENABLED=0`, `-trimpath`, `-s -w`)
- ☐ Secrets only via platform env / secret store / `EnvironmentFile`
- ☐ TLS at the edge (platform or Caddy/nginx)
- ☐ Graceful shutdown on `SIGTERM` (PaaS and systemd both send it)
- ☐ Structured logs to stdout/stderr (platforms capture them)
- ☐ Database connection limits sized for instance count
- ☐ Document rollback: previous binary tag or image digest

## Common Pitfalls

1. **Hard-coded port** — works locally, fails on Render/Leapcell.

2. **Listening on 127.0.0.1 only** — fine behind local Caddy; wrong if the platform expects 0.0.0.0.
3. **Migrate on every boot with N replicas** — racey schema changes; use one-shot jobs.
4. **Huge cold start init** — loading models, syncing caches, or compiling templates at package init time.
5. **Root systemd service** — unnecessary privilege; use a dedicated user.
6. **No disk quota awareness** — writing unbounded logs/uploads under `/var/lib` fills the VPS.

## Exercises

1. **PORT wiring** — Deploy a tiny handler that prints PORT and the listen address; verify on a PaaS free tier or with `PORT=9090 ./server` locally.
2. **systemd unit** — Install a binary as a non-root service with `ProtectSystem=strict` and confirm `journalctl` shows clean restarts after `kill -TERM`.
3. **Readiness gate** — Make `/readyz` fail until a file `/var/lib/myapp/ready` exists; flip it and watch a reverse proxy start routing.
4. **Cold-start budget** — Measure time from process start to first successful `/healthz` with artificial 2s DB connect delay; refactor init to stay under 500ms when DB is up.
5. **Secret file perms** — Create `/etc/myapp.env` as `0640 root:myapp` and prove the service user can read it while an unprivileged login cannot.

## More examples

### PORT injection and dual health endpoints

PaaS injects PORT and probes liveness separately from readiness. This program prints the resolved listen port and exercises both endpoints via `httpptest`.

```
mkdir -p /tmp/go-paas-port && cd /tmp/go-paas-port
go mod init example.com/paas-port
```

Save as `main.go`:

```
package main

import (
 "fmt"
 "net/http"
 "net/http/httpptest"
 "os"
 "sync/atomic"
)
```

```

func port() string {
 if p := os.Getenv("PORT"); p != "" {
 return p
 }
 return "8080"
}

func main() {
 os.Setenv("PORT", "10000")
 fmt.Println("listen :"+port())

 var dbOK atomic.Bool
 dbOK.Store(false)

 mux := http.NewServeMux()
 mux.HandleFunc("GET /healthz", func(w http.ResponseWriter, _ *http.Request) {
 fmt.Fprintln(w, "alive")
 })
 mux.HandleFunc("GET /readyz", func(w http.ResponseWriter, _ *http.Request) {
 if !dbOK.Load() {
 http.Error(w, "db warming", http.StatusServiceUnavailable)
 return
 }
 fmt.Fprintln(w, "ready")
 })

 ts := httptest.NewServer(mux)
 defer ts.Close()

 r, _ := http.Get(ts.URL + "/healthz")
 r.Body.Close()
 fmt.Println("healthz:", r.StatusCode)

 r, _ = http.Get(ts.URL + "/readyz")
 r.Body.Close()
 fmt.Println("readyz cold:", r.StatusCode)

 dbOK.Store(true)
 r, _ = http.Get(ts.URL + "/readyz")
 r.Body.Close()
 fmt.Println("readyz warm:", r.StatusCode)
}

go run .

```

**Expected output:**

```
listen :10000
healthz: 200
readyz cold: 503
readyz warm: 200
```

## Config from env file shape (KEY=VALUE)

```
mkdir -p /tmp/go-paas-env && cd /tmp/go-paas-env
go mod init example.com/paas-env
```

Save as main.go:

```
package main

import (
 "bufio"
 "fmt"
 "os"
 "strings"
)

func loadEnvFile(path string) (map[string]string, error) {
 f, err := os.Open(path)
 if err != nil {
 return nil, err
 }
 defer f.Close()
 out := map[string]string{}
 sc := bufio.NewScanner(f)
 for sc.Scan() {
 line := strings.TrimSpace(sc.Text())
 if line == "" || strings.HasPrefix(line, "#") {
 continue
 }
 k, v, ok := strings.Cut(line, "=")
 if !ok {
 continue
 }
 out[strings.TrimSpace(k)] = strings.TrimSpace(v)
 }
 return out, sc.Err()
}

func main() {
 path := "app.env"
 _ = os.WriteFile(path, []byte("# demo\nPORT=7777\nAPP_ENV=staging\n"), 0o600)
```

```

 defer os.Remove(path)

 cfg, err := loadEnvFile(path)
 if err != nil {
 panic(err)
 }
 fmt.Println("PORT:", cfg["PORT"])
 fmt.Println("APP_ENV:", cfg["APP_ENV"])
 fmt.Println("keys:", len(cfg))
}

go run .

```

### Expected output:

```

PORT: 7777
APP_ENV: staging
keys: 2

```

## Runnable example

PaaS platforms inject `PORT` and probe health. This program binds to `PORT`, exposes `/healthz` and `/readyz`, and demonstrates readiness gating—the same contract Render, Leapcell-style hosts, and systemd expect.

```

mkdir -p /tmp/go-paas-app && cd /tmp/go-paas-app
go mod init example.com/paas-app

```

Save as `main.go`:

```

package main

import (
 "context"
 "fmt"
 "log"
 "net"
 "net/http"
 "os"
 "sync/atomic"
 "time"
)

func main() {
 port := os.Getenv("PORT")

```

```

if port == "" {
 port = "0" // ephemeral for local demo
}

var ready atomic.Bool
ready.Store(false)

mux := http.NewServeMux()
mux.HandleFunc("GET /healthz", func(w http.ResponseWriter, _ *http.Request) {
 w.WriteHeader(http.StatusOK)
 _, _ = w.Write([]byte("ok"))
})
mux.HandleFunc("GET /readyz", func(w http.ResponseWriter, _ *http.Request) {
 if !ready.Load() {
 http.Error(w, "not ready", http.StatusServiceUnavailable)
 return
 }
 w.WriteHeader(http.StatusOK)
 _, _ = w.Write([]byte("ready"))
})
mux.HandleFunc("GET /", func(w http.ResponseWriter, _ *http.Request) {
 fmt.Fprintln(w, "hello from paas-shaped go")
})

ln, err := net.Listen("tcp", "127.0.0.1:"+port)
if err != nil {
 log.Fatal(err)
}

srv := &http.Server{Handler: mux, ReadHeaderTimeout: 5 * time.Second}

go func() {
 if err := srv.Serve(ln); err != nil && err != http.ErrServerClosed {
 log.Fatal(err)
 }
}()

base := "http://" + ln.Addr().String()
fmt.Println("listen_addr:", ln.Addr().String())
fmt.Println("PORT env:", os.Getenv("PORT"))

// Simulate warmup: live before ready.
check(base+"/healthz", "healthz-before-ready")
check(base+"/readyz", "readyz-before-ready")

ready.Store(true)
fmt.Println("warmup complete; marking ready")

```



```

 check(base+"/readyz", "readyz-after-ready")
 check(base+"/", "root")

 ctx, cancel := context.WithTimeout(context.Background(), 2*time.Second)
 defer cancel()
 _ = srv.Shutdown(ctx)
 fmt.Println("shutdown complete")
}

func check(url, label string) {
 resp, err := http.Get(url)
 if err != nil {
 fmt.Printf("%s: error %v\n", label, err)
 return
 }
 defer resp.Body.Close()
 fmt.Printf("%s: %d\n", label, resp.StatusCode)
}

go run .
PORT=9090 go run .

```

**Expected output** (illustrative):

```

listen_addr: 127.0.0.1:54321
PORT env:
healthz-before-ready: 200
readyz-before-ready: 503
warmup complete; marking ready
readyz-after-ready: 200
root: 200
shutdown complete

```

### What to notice

- Always prefer PORT from the environment; hard-coded 8080 fails on managed platforms.
- Liveness (/healthz) can succeed while readiness (/readyz) is 503 during migrations or dependency warmup.
- Listen on all interfaces in real deploys (:PORT); this demo uses 127.0.0.1 for a self-contained local run.

### Try next

- Run with PORT=9090 and curl from another terminal.
- On SIGTERM, flip ready to false, sleep briefly (connection drain), then Shutdown.

## Related Topics

- [Containerization \(Docker\)](#)
- [CI/CD & Hardening](#)
- [Building and Releasing](#)

**Go on NixOS & FreeBSD**

# Beyond Linux: NixOS & FreeBSD

## Overview

Go treats cross-compilation as a first-class feature: set `GOOS` and `GOARCH`, get a binary. That matters when you target NixOS (purely functional paths, no global `/lib`) or FreeBSD (jails, ZFS, `kqueue`). Both platforms reward pure-Go static binaries and punish casual CGO.

This chapter covers cross-compilation matrices, Nix flakes for reproducible toolchains, FreeBSD notes (`kqueue`, `jails`), and production pitfalls.

## Cross-Compilation Basics

From any host (macOS, Linux, Windows):

```
Linux amd64 (most VPS / cloud)
GOOS=linux GOARCH=amd64 CGO_ENABLED=0 go build -o bin/app-linux-amd64 ./cmd/server

Linux arm64 (Graviton, Raspberry Pi OS 64-bit)
GOOS=linux GOARCH=arm64 CGO_ENABLED=0 go build -o bin/app-linux-arm64 ./cmd/server

FreeBSD amd64
GOOS=freebsd GOARCH=amd64 CGO_ENABLED=0 go build -o bin/app-freebsd-amd64 ./cmd/server

FreeBSD arm64
GOOS=freebsd GOARCH=arm64 CGO_ENABLED=0 go build -o bin/app-freebsd-arm64 ./cmd/server

Windows
GOOS=windows GOARCH=amd64 CGO_ENABLED=0 go build -o bin/app.exe ./cmd/server

macOS Apple Silicon
GOOS=darwin GOARCH=arm64 CGO_ENABLED=0 go build -o bin/app-darwin-arm64 ./cmd/server
```

List supported pairs:

```
go tool dist list
```

## Matrix build script

```
#!/usr/bin/env bash
set -euo pipefail
mkdir -p bin
for goos in linux freebsd darwin windows; do
 for goarch in amd64 arm64; do
 ext=""
 [["$goos" == "windows"]] && ext=".exe"
 out="bin/app-${goos}-${goarch}${ext}"
 echo "building $out"
 CGO_ENABLED=0 GOOS="$goos" GOARCH="$goarch" \
 go build -trimpath -ldflags="-s -w" -o "$out" ./cmd/server
 done
done
```

**CGO breaks easy cross-compilation.** Cross-CGO needs a matching cross-compiler and libraries. Prefer pure Go drivers (e.g. jackc/pgx pure Go, modernc.org/sqlite) when you ship multi-OS binaries.

## Go on NixOS

NixOS has no FHS layout by default. Dynamically linked binaries that expect `/lib64/ld-linux-x86-64.so.2` often fail unless you use `steam-run`, `buildFHSUserEnv`, or `patchelf`. **Static pure-Go binaries just run.**

### Pure Go: the happy path

```
CGO_ENABLED=0 go build -o app ./cmd/server
./app # works on NixOS without wrapping
```

Verify linkage (on a system with `ldd`):

```
ldd ./app
statically linked (or "not a dynamic executable")
```

## Development shell with Flakes

Pin the exact Go toolchain for the whole team:

```
flake.nix
{
 description = "Go service dev shell";
```

```
inputs.nixpkgs.url = "github:NixOS/nixpkgs/nixos-unstable";

outputs = { self, nixpkgs }:
 let
 systems = ["x86_64-linux" "aarch64-linux" "x86_64-darwin" "aarch64-darwin"];
 forAllSystems = f: nixpkgs.lib.genAttrs systems (system: f nixpkgs.legacyPackages.${system});
 in {
 devShells = forAllSystems (pkgs: {
 default = pkgs.mkShell {
 packages = with pkgs; [
 go_1_26
 gopls
 gotools
 golangci-lint
 govulncheck
 delv
];
 shellHook = ''
 export CGO_ENABLED=0
 echo "Go $(go version)"
 '';
 };
 });
 };

nix develop
go test ./...
```

```

meta = {
 description = "Example Go service";
 mainProgram = "myapp";
};
}

```

Run `nix build` and let Nix tell you the correct `vendorHash` on first failure, then pin it.

## CGO on NixOS

If you need CGO, build inside Nix so headers and libs are on `NIX_CFLAGS_COMPILE` / `NIX_LDFLAGS`:

```

pkgs.mkShell {
 packages = [pkgs.go pkgs.pkg-config pkgs.sqlite];
 # CGO_ENABLED=1 only inside this shell when required
}

```

Never assume `/usr/include` exists.

## Go on FreeBSD

FreeBSD is a first-class Go port. The runtime uses **kqueue** for network and file notifications (instead of Linux `epoll`/`inotify`).

### Runtime notes

Area	FreeBSD behavior
<code>net</code> poller	kqueue-backed
<code>fsnotify</code> and similar	kqueue (not <code>inotify</code> )
<code>os/signal</code>	Works; jail signal policies may differ
File paths	Case-sensitive UFS/ZFS; no drive letters
<code>syscall</code> / <code>x/sys/unix</code>	FreeBSD-specific constants and wrappers

## Build on FreeBSD

```

pkg install go git
git clone https://example.com/myapp.git
cd myapp
CGO_ENABLED=0 go build -o myapp ./cmd/server

```

Cross-build from Linux/macOS as shown above; `scp` into a jail or host.

## Jails and networking

Run the service inside a jail with a dedicated IP or VNET:

```
host
 jail: myapp
 /usr/local/bin/myapp
 env: DATABASE_URL=...
 listen: 10.0.0.20:8080
```

Prefer binding explicitly:

```
addr := os.Getenv("LISTEN_ADDR")
if addr == "" {
 addr = "0.0.0.0:8080"
}
log.Fatal(http.ListenAndServe(addr, mux))
```

## rc.d service sketch

```
#!/bin/sh
/usr/local/etc/rc.d/myapp
PROVIDE: myapp
REQUIRE: NETWORKING
KEYWORD: shutdown

. /etc/rc.subr

name="myapp"
rcvar="myapp_enable"
command="/usr/local/bin/myapp"
myapp_user="myapp"
pidfile="/var/run/${name}.pid"
command_args=""

load_rc_config $name
run_rc_command "$1"

sysrc myapp_enable=YES
service myapp start
```



## Portable Go Code Tips

```
package platform

import "runtime"

func DataDir() string {
 switch runtime.GOOS {
 case "windows":
 return filepath.Join(os.Getenv("ProgramData"), "MyApp")
 case "darwin":
 return filepath.Join(os.Getenv("HOME"), "Library", "Application Support", "MyApp")
 default: // linux, freebsd, ...
 if xdg := os.Getenv("XDG_DATA_HOME"); xdg != "" {
 return filepath.Join(xdg, "myapp")
 }
 return filepath.Join(os.Getenv("HOME"), ".local", "share", "myapp")
 }
}
```

Use `path/filepath`, not string concat with `/`. Use `runtime.GOOS` only at boundaries (paths, notifications), not scattered through business logic.

## Build tags for OS-specific files

```
// notify_linux.go
//go:build linux

package notify

func watch(path string) error { /* inotify path via library */ return nil }

// notify_freebsd.go
//go:build freebsd

package notify

func watch(path string) error { /* kqueue path */ return nil }

// notify_stub.go
//go:build !linux && !freebsd

package notify
```

```
func watch(path string) error { return errUnsupported }
```

## Production Checklist

- ☐ Default builds use `CGO_ENABLED=0` for portable artifacts
- ☐ CI builds at least `linux/amd64` and any real target (`freebsd/amd64`, `linux/arm64`)
- ☐ Nix users have a `flake/shell.nix` with pinned Go
- ☐ No hard-coded Linux-only paths (`/proc` scraped only behind `linux` tags)
- ☐ File watching libraries verified on FreeBSD if used
- ☐ Release binaries named with `GOOS-GOARCH`
- ☐ Document CGO exceptions (and required C toolchains) explicitly

## Common Pitfalls

1. **Dynamic binary on NixOS** — “No such file or directory” often means the dynamic loader path is wrong, not that the file is missing. Same class of bug as a glibc binary on Alpine (`/lib64/ld-linux-x86-64.so.2` absent). Pure Go (`CGO_ENABLED=0`) never asks for that loader; see [Go as a Systems Language](#).
2. **CGO cross-compile surprise** — `go build` silently uses CGO when `CGO_ENABLED` is unset and a C compiler exists; set `CGO_ENABLED=0` in CI.
3. **/proc and cgroup assumptions** — code that reads Linux cgroup files breaks on FreeBSD; guard with build tags.
4. **Case-insensitive assumptions** — macOS default FS is case-insensitive; FreeBSD/Linux are not. Tests that rely on case folding will fail.
5. **Firewall / jail** — binding succeeds but traffic never arrives because the jail has no IP or pf blocks the port.
6. **Old FreeBSD + new Go** — use a FreeBSD version supported by your Go release; check Go’s FreeBSD port notes when upgrading.

## Exercises

1. **Cross matrix** — Build the same `./cmd/server` for `linux/amd64`, `linux/arm64`, and `freebsd/amd64`. Record binary sizes.
2. **Static proof** — On Linux, run `file` and `ldd` on the artifact; ensure it is statically linked with `CGO_ENABLED=0`.
3. **Flake shell** — Add a minimal `flake.nix` with `go` and `gopls`; run `nix develop -c go test ./...`
4. **GOOS-specific file** — Split a helper with `//go:build linux` and `//go:build freebsd` stubs; `go test` on your host and `GOOS=freebsd go test` (compile-only is fine).
5. **Path portability** — Implement `DataDir()` above with tests using `t.Setenv` for `XDG_DATA_HOME` and `HOME`.

## More examples

### Portable data directory (XDG + HOME fallback)

```
mkdir -p /tmp/go-xdg && cd /tmp/go-xdg
go mod init example.com/xdg
```

Save as main.go:

```
package main

import (
 "fmt"
 "os"
 "path/filepath"
 "runtime"
)

func dataDir(app string) string {
 if xdg := os.Getenv("XDG_DATA_HOME"); xdg != "" {
 return filepath.Join(xdg, app)
 }
 home, err := os.UserHomeDir()
 if err != nil {
 return filepath.Join(".", app)
 }
 // FreeBSD/Linux/macOS-friendly default under home.
 return filepath.Join(home, ".local", "share", app)
}

func main() {
 fmt.Println("GOOS/GOARCH:", runtime.GOOS+"/"+runtime.GOARCH)
 os.Setenv("XDG_DATA_HOME", "/var/lib/custom")
 fmt.Println("with XDG:", dataDir("myapp"))
 os.Unsetenv("XDG_DATA_HOME")
 // Force HOME for deterministic demo output.
 os.Setenv("HOME", "/home/demo")
 fmt.Println("fallback:", dataDir("myapp"))
}

go run .
```

Expected output (GOOS/GOARCH varies by host):

```
GOOS/GOARCH: darwin/arm64
with XDG: /var/lib/custom/myapp
fallback: /home/demo/.local/share/myapp
```

## Build identity without CGO surprises

```
mkdir -p /tmp/go-static-id && cd /tmp/go-static-id
go mod init example.com/static-id
```

Save as main.go:

```
package main

import (
 "fmt"
 "runtime"
 "runtime/debug"
)

func main() {
 fmt.Println("compiler:", runtime.Compiler)
 fmt.Println("version:", runtime.Version())
 fmt.Println("cgo:", cgoEnabled())
 if bi, ok := debug.ReadBuildInfo(); ok {
 fmt.Println("main path:", bi.Path)
 fmt.Println("go version (mod):", bi.GoVersion)
 }
}

func cgoEnabled() string {
 // Pure-Go programs can still report whether the toolchain had CGO available
 // at build time via the "CGO_ENABLED" setting in build info when present.
 if bi, ok := debug.ReadBuildInfo(); ok {
 for _, s := range bi.Settings {
 if s.Key == "CGO_ENABLED" {
 return s.Value
 }
 }
 }
 return "unknown (run: CGO_ENABLED=0 go build)"
}
```

```
CGO_ENABLED=0 go run .
```

Expected output (illustrative):

```
compiler: gc
version: go1.22.x
cgo: 0
main path: example.com/static-id
go version (mod): go1.22.x
```

## Runnable example

Cross-compilation and portable paths matter more than platform-specific syscalls for most services. This program prints build identity (GOOS/GOARCH), uses XDG-style data dirs, and stays pure Go (static-friendly).

```
mkdir -p /tmp/go-portable && cd /tmp/go-portable
go mod init example.com/portable
```

Save as `main.go`:

```
package main

import (
 "fmt"
 "os"
 "path/filepath"
 "runtime"
)

func dataDir(app string) string {
 if xdg := os.Getenv("XDG_DATA_HOME"); xdg != "" {
 return filepath.Join(xdg, app)
 }
 home, err := os.UserHomeDir()
 if err != nil {
 return filepath.Join(".", app+"-data")
 }
 // Portable default; FreeBSD/Linux/macOS all understand home-relative paths.
 return filepath.Join(home, ".local", "share", app)
}

func main() {
 fmt.Printf("goos=%s goarch=%s compiler=%s\n", runtime.GOOS, runtime.GOARCH, runtime.Compiler)
 fmt.Printf("version=%s\n", runtime.Version())
 fmt.Printf("num_cpu=%d\n", runtime.NumCPU())

 dir := dataDir("myapp")
 fmt.Println("data_dir:", dir)

 if err := os.MkdirAll(dir, 0o750); err != nil {
 fmt.Println("mkdir error:", err)
 os.Exit(1)
 }
 marker := filepath.Join(dir, "hello.txt")
 if err := os.WriteFile(marker, []byte("portable pure-go write\n"), 0o640); err != nil {
 fmt.Println("write error:", err)
 }
}
```

```

 os.Exit(1)
 }
 b, err := os.ReadFile(marker)
 if err != nil {
 fmt.Println("read error:", err)
 os.Exit(1)
 }
 fmt.Printf("read_back: %q\n", string(b))
 _ = os.Remove(marker)
 fmt.Println("ok: pure Go paths work without CGO")
}

go run .
Cross-compile without running foreign binaries:
GOOS=linux GOARCH=amd64 CGO_ENABLED=0 go build -o /tmp/app-linux-amd64 .
GOOS=freebsd GOARCH=amd64 CGO_ENABLED=0 go build -o /tmp/app-freebsd-amd64 .
file /tmp/app-linux-amd64 /tmp/app-freebsd-amd64 2>/dev/null || ls -la /tmp/app-*

```

**Expected output** (illustrative, host-dependent):

```

goos=darwin goarch=arm64 compiler=gc
version=go1.22.x
num_cpu=8
data_dir: /Users/you/.local/share/myapp
read_back: "portable pure-go write\n"
ok: pure Go paths work without CGO

```

## What to notice

- `runtime.GOOS` / `GOARCH` report the *target* you built for—set env vars at build time for NixOS/FreeBSD artifacts.
- `CGO_ENABLED=0` keeps binaries static so NixOS does not need an FHS loader path.
- Prefer filepath + env (`XDG_DATA_HOME`) over hard-coded `/var` Linux paths.

## Try next

- Add a `//go:build linux` file and a stub for other OS; compile with `GOOS=freebsd go build`.
- On Linux, run `ldd / file` on a `CGO_ENABLED=0` binary and confirm it is statically linked.

## Related Topics

- [Containerization \(Docker\)](#)
- [Building and Releasing](#)
- [Go as a Systems Language](#)
- [Systems Programming](#)

## CI/CD & Hardening

# CI/CD Pipelines & Security Hardening

## Overview

CI/CD should make the boring path automatic: format, lint, test, vuln scan, build, sign, and ship. For Go, the toolchain is friendly—fast tests, excellent race detector, first-party **govulncheck**—but pipelines still get compromised through unpinned actions, over-privileged tokens, and unsigned artifacts.

This chapter provides a 2026-oriented GitHub Actions workflow, binary hardening flags, image signing with Cosign, and a release checklist.

## Goals of a Go Pipeline

```
push / PR
--> checkout (pinned)
--> setup-go (cache modules)
--> vet / lint
--> govulncheck
--> test -race -cover
--> build (reproducible flags)
--> (main/tag) sign + publish
```

Every PR should fail closed on lint, tests, or reachable vulns. Releases should produce **signed**, **versioned** artifacts.

## Baseline GitHub Actions Workflow

```
.github/workflows/go.yml
name: Go Build & Test

on:
 push:
 branches: [main]
 pull_request:
 branches: [main]

permissions:
```



```

 contents: read

jobs:
 build:
 runs-on: ubuntu-latest
 steps:
 - name: Checkout
 uses: actions/checkout@v4

 - name: Setup Go
 uses: actions/setup-go@v5
 with:
 go-version: "1.27.x"
 cache: true

 - name: Verify modules
 run: go mod verify

 - name: Vet
 run: go vet ./...

 - name: Lint
 uses: golangci/golangci-lint-action@v6
 with:
 version: v1.64.0

 - name: Vulnerability check
 run: |
 go install golang.org/x/vuln/cmd/govulncheck@latest
 govulncheck ./...

 - name: Test
 run: go test -race -count=1 -coverprofile=coverage.out ./...

 - name: Coverage report
 run: go tool cover -func=coverage.out | tail -n 1

 - name: Build
 run: |
 CGO_ENABLED=0 go build -trimpath -ldflags="-s -w" -o bin/app ./cmd/server

```

## Hardening the workflow itself

1. **Least privilege tokens** — default permissions: `contents: read`. Grant `id-token: write` only for OIDC/signing jobs; `contents: write` only for release jobs.
2. **Pin actions by digest** in high-security repos:

```
Example shape - replace with current digest from the action repo
- uses: actions/checkout@11bd71901bbe5b1630ceea73d27597364c9af683 # v4.2.2
```

3. **No pull\_request\_target with untrusted checkout** — easy RCE pattern; avoid unless you fully understand the threat model.
4. **Cache carefully** — setup-go cache is fine for modules; do not cache world-writable custom paths without validation.
5. **Secrets** — only on main/tags, not on forks' PR workflows that expose secrets.

## Release Job Sketch

```
release:
 if: startsWith(github.ref, 'refs/tags/v')
 needs: build
 runs-on: ubuntu-latest
 permissions:
 contents: write
 id-token: write # for keyless Cosign / OIDC
 steps:
 - uses: actions/checkout@v4
 - uses: actions/setup-go@v5
 with:
 go-version: "1.27.x"
 cache: true

 - name: Build multi-arch binaries
 run: |
 mkdir -p dist
 for pair in linux/amd64 linux/arm64 darwin/arm64; do
 os=${pair%/*}; arch=${pair##*/}
 CGO_ENABLED=0 GOOS=$os GOARCH=$arch \
 go build -trimpath \
 -ldflags="-s -w -X main.version=${GITHUB_REF_NAME}" \
 -o "dist/app_${os}_${arch}" ./cmd/server
 done

 - name: Upload release assets
 uses: softprops/action-gh-release@v2
 with:
 files: dist/*
```

## Hardening the Binary

```
CGO_ENABLED=0 go build \
-trimpath \
-buildmode=pie \
-ldflags="-s -w -X main.version=${VERSION} -X main.commit=${COMMIT}" \
-o app \
./cmd/server
```

Flag	Purpose
-trimpath	Strip local paths (reproducibility + privacy)
-s -w	Strip symbol/DWARF tables (smaller; less free intel)
-buildmode=pie	Position-independent executable; works with ASLR
-X main.version=...	Embed release metadata without editing source

Optional: generate a Software Bill of Materials (SBOM) with `syft` or `govulncheck` tooling and attach it to the release.

## Container Image Pipeline

```
- name: Build image
 run: |
 docker build \
 --build-arg VERSION=${GITHUB_REF_NAME} \
 -t ghcr.io/${{ github.repository }}:${GITHUB_REF_NAME} .

- name: Scan image
 run: |
 # install trivy or use a GH action
 trivy image --exit-code 1 --severity HIGH,CRITICAL \
 ghcr.io/${{ github.repository }}:${GITHUB_REF_NAME}
```

## Signing with Cosign (Sigstore)

Signing proves the artifact came from *your* CI identity, not a random uploader.

```
Keyless (OIDC) example in CI after cosign is installed
cosign sign --yes ghcr.io/org/myapp:1.2.3

Verify elsewhere
cosign verify ghcr.io/org/myapp:1.2.3 \

```

```
--certificate-identity-regexp='https://github.com/org/myapp/.*' \
--certificate-oidc-issuer=https://token.actions.githubusercontent.com
```

Sign checksums for raw binaries:

```
(cd dist && sha256sum * > checksums.txt)
cosign sign-blob --yes --output-signature checksums.txt.sig checksums.txt
```

## Go-Specific Quality Gates

```
Race detector (linux/amd64/arm64; not all platforms)
go test -race ./...

Fuzz (kept short in CI)
go test -fuzz=FuzzParse -fuzztime=20s ./internal/parser

Module hygiene
go mod tidy
git diff --exit-code go.mod go.sum
```

## Example test job matrix

```
strategy:
 matrix:
 os: [ubuntu-latest, macos-latest]
 go: ["1.26.x", "1.27.x"]
```

Use `-race` on Linux; on macOS it works but is slower—optional for PRs, required on main.

## Supply Chain Mini-Checklist

Control	Why
go.sum committed	Pin module hashes
go mod verify	Detect sum database mismatches
govulncheck	Reachable vuln analysis (not just “depends on”)
Pinned actions digests	Action repo compromise resistance
Minimal permissions	Stolen token blast radius
Cosign / provenance	Consumer verification
Immutable image tags + digests	Prevent tag overwrite attacks

## Application Snippet: Fail Build on Bad Config

CI should also run a **config dry-run** so broken env schemas never ship:

```
package config

import (
 "fmt"
 "os"
 "strconv"
)

type Config struct {
 Addr string
 Env string
}

func Load() (Config, error) {
 addr := os.Getenv("LISTEN_ADDR")
 if addr == "" {
 addr = ":8080"
 }
 env := os.Getenv("APP_ENV")
 if env == "" {
 return Config{}, fmt.Errorf("APP_ENV required")
 }
 if _, err := strconv.Atoi(os.Getenv("READY_TIMEOUT_SEC")); err != nil && os.Getenv("READ
 return Config{}, fmt.Errorf("READY_TIMEOUT_SEC: %w", err)
 }
 return Config{Addr: addr, Env: env}, nil
}
```

```
APP_ENV=ci go test ./internal/config -run TestLoad
```

## Production Checklist

- ☐ PR pipeline: vet, lint, govulncheck, go test -race
- ☐ permissions default to read-only
- ☐ Actions pinned (version at minimum; digest for high assurance)
- ☐ Release builds use -trimpath, -s -w, version ldflags
- ☐ Artifacts checksummed; containers scanned
- ☐ Cosign (or equivalent) sign on tag release
- ☐ Secrets not available to untrusted fork PRs
- ☐ go.mod/go.sum tidy enforced
- ☐ Branch protection requires the CI check

## Common Pitfalls

1. **go test without -race** — data races ship silently under load.
2. **Latest action tags only** — `uses: foo@main` is a supply-chain footgun.
3. **Over-powered GITHUB\_TOKEN** — write access on every PR job.
4. **Skipping vuln checks “to go green”** — track ignore rules with expiry, don’t delete the step.
5. **Building with CGO in CI by accident** — non-portable binaries and missing cross-compile.
6. **Signing only containers, not binaries** — attackers swap the `.tar.gz` on the release page.
7. **No module verify** — tampered cache or mirror goes unnoticed.

## Exercises

1. **Green pipeline** — Add the baseline workflow to a sample module; break a test and confirm the PR fails.
2. **Pin an action** — Replace `actions/checkout@v4` with a full commit SHA and document the version in a comment.
3. **Race lab** — Introduce a deliberate data race; show `-race` fails and the fix (mutex or channel) passes.
4. **govulncheck** — Run against a module with an old vulnerable dependency; upgrade until clean.
5. **Cosign dry-run** — Generate a local key pair with Cosign, sign a blob, verify it (keyless optional later).
6. **Permissions audit** — Read your workflow YAML and list every permission granted; remove any unused write scope.

## More examples

### Race-prone counter vs fixed counter

CI should run `go test -race`. This program shows the bug class the race detector catches, then a mutex-fixed version.

```
mkdir -p /tmp/go-ci-race && cd /tmp/go-ci-race
go mod init example.com/ci-race
```

Save as `main.go`:

```
package main

import (
 "fmt"
 "sync"
)
```

```

func racey(n int) int {
 var c int
 var wg sync.WaitGroup
 for i := 0; i < n; i++ {
 wg.Add(1)
 go func() {
 defer wg.Done()
 c++
 }()
 }
 wg.Wait()
 return c
}

func fixed(n int) int {
 var c int
 var mu sync.Mutex
 var wg sync.WaitGroup
 for i := 0; i < n; i++ {
 wg.Add(1)
 go func() {
 defer wg.Done()
 mu.Lock()
 c++
 mu.Unlock()
 }()
 }
 wg.Wait()
 return c
}

func main() {
 // Deterministic path: only exercise the fixed counter in default runs.
 // To see the race: temporarily print racey(1000) under `go run -race .`
 got := fixed(1000)
 fmt.Println("fixed count:", got)
 fmt.Println("ok:", got == 1000)
}

go run .
optional: go test -race (after adding a _test.go that calls racey)

```

### Expected output:

```

fixed count: 1000
ok: true

```

## Module sum verification helper

```
mkdir -p /tmp/go-ci-verify && cd /tmp/go-ci-verify
go mod init example.com/ci-verify
```

Save as main.go:

```
package main

import (
 "crypto/sha256"
 "encoding/hex"
 "fmt"
 "os"
)

func fileSHA256(path string) (string, error) {
 b, err := os.ReadFile(path)
 if err != nil {
 return "", err
 }
 sum := sha256.Sum256(b)
 return hex.EncodeToString(sum[:]), nil
}

func main() {
 path := "artifact.bin"
 _ = os.WriteFile(path, []byte("release-bytes-v1"), 0o644)
 defer os.Remove(path)

 sum, err := fileSHA256(path)
 if err != nil {
 panic(err)
 }
 fmt.Println("sha256:", sum)
 // Simulate CI check against a known digest.
 want, _ := fileSHA256(path)
 fmt.Println("matches pinned digest:", sum == want)
}

go run .
```

### Expected output:

```
sha256: <64 hex chars>
matches pinned digest: true
```



## Runnable example

CI gates are YAML; the Go work they gate is tests, races, and release-shaped builds. This package includes a race-prone function, a fixed version, and a small test file—run the same commands your pipeline should run.

```
mkdir -p /tmp/go-ci-gate && cd /tmp/go-ci-gate
go mod init example.com/ci-gate
```

Save as `sum.go`:

```
package cigate

import "sync"

// BrokenAdd races on total (do not use in production).
func BrokenAdd(n int) int {
 var total int
 var wg sync.WaitGroup
 wg.Add(n)
 for i := 0; i < n; i++ {
 go func() {
 defer wg.Done()
 total++
 }()
 }
 wg.Wait()
 return total
}

// SafeAdd uses an atomic-style mutex critical section.
func SafeAdd(n int) int {
 var total int
 var mu sync.Mutex
 var wg sync.WaitGroup
 wg.Add(n)
 for i := 0; i < n; i++ {
 go func() {
 defer wg.Done()
 mu.Lock()
 total++
 mu.Unlock()
 }()
 }
 wg.Wait()
 return total
}
```

Save as `sum_test.go`:

```
package cigate

import "testing"

func TestSafeAdd(t *testing.T) {
 if got := SafeAdd(1000); got != 1000 {
 t.Fatalf("SafeAdd: got %d want 1000", got)
 }
}

func TestBrokenAdd_DocumentRace(t *testing.T) {
 // Without -race this may flakily pass; with -race it should fail.
 // Comment out after you have seen the race detector fire once.
 t.Skip("uncomment to demo: go test -race (expect FAIL on BrokenAdd)")
 _ = BrokenAdd(1000)
}
```

Save as `cmd/check/main.go`:

```
package main

import (
 "fmt"
 "runtime/debug"

 cigate "example.com/ci-gate"
)

func main() {
 fmt.Println("safe_sum:", cigate.SafeAdd(100))
 if info, ok := debug.ReadBuildInfo(); ok {
 fmt.Println("go:", info.GoVersion)
 fmt.Println("path:", info.Path)
 }
 fmt.Println("ci gate sample ok")
}

go vet ./...
go test -count=1 ./...
go test -race -count=1 ./...
CGO_ENABLED=0 go build -trimpath -ldflags="-s -w" -o /tmp/ci-check ./cmd/check
/tmp/ci-check
optional: go install golang.org/x/vuln/cmd/govulncheck@latest && govulncheck ./...
```

**Expected output** (illustrative):

```
ok example.com/ci-gate 0.012s
safe_sum: 100
go: go1.22.x
path: example.com/ci-gate
ci gate sample ok
```

To see the race detector, temporarily call `BrokenAdd` from a test without `t.Skip` and run `go test -race`.

### What to notice

- Pipelines should fail on `go test -race` for concurrent packages—silent races ship without it.
- Release flags (`-trimpath`, `-s -w`, `CGO_ENABLED=0`) belong in the same job that publishes artifacts.
- `go vet` is free correctness signal before heavier linters and `govulncheck`.

### Try next

- Unskip the race test, watch `-race` fail, then delete `BrokenAdd`.
- Add a Makefile target `sec: go vet && go test -race ./...` matching the workflow in this chapter.

## Related Topics

- [Containerization \(Docker\)](#)
- [Security Tools](#)
- [Application Hardening](#)
- [Building and Releasing](#)

# Building and Releasing Software

## Overview

Go compiles to static binaries, making deployment simple. This chapter covers building, cross-compilation, and release workflows.

## Basic Build

```
go build -o myapp ./cmd/server
```

## Production Build

```
go build \
 -ldflags="-s -w" \
 -o bin/myapp \
 ./cmd/server
```

Flags: - -s - Strip symbol table - -w - Strip DWARF debugging info

## Version Injection

```
var (
 version = "dev"
 commit = "none"
 date = "unknown"
)
```

```
go build -ldflags="-X main.version=1.0.0 -X main.commit=$(git rev-parse HEAD)"
```

## Why CGO\_ENABLED=0 is part of a release

On a machine with a C compiler, cgo is **on by default**. That can silently link **glibc** (Debian/Ubuntu) or **musl** (Alpine). The binary then needs a matching dynamic linker on the target: a glibc build dies on Alpine with **not found** even though the file is sitting in the current directory.

```
Release default: pure Go, fully static on Linux.
CGO_ENABLED=0 go build -o myapp ./cmd/server

file myapp # Linux: "statically linked"
ldd myapp # Linux: "not a dynamic executable"
```

CGO\_ENABLED=0 also selects Go's own DNS resolver and **os/user** path instead of libc. That is what you want for **scratch** / distroless images and for a CLI you copy onto someone else's laptop. Turn cgo on only when a C library is irreplaceable. The full kernel-vs-libc story is in [Go as a Systems Language](#).

## Cross-Compilation

```
Linux (static when CGO is off)
GOOS=linux GOARCH=amd64 CGO_ENABLED=0 go build -o app-linux

Windows
GOOS=windows GOARCH=amd64 CGO_ENABLED=0 go build -o app.exe

macOS ARM
GOOS=darwin GOARCH=arm64 CGO_ENABLED=0 go build -o app-darwin

All platforms
for os in linux darwin windows; do
 for arch in amd64 arm64; do
 GOOS=$os GOARCH=$arch CGO_ENABLED=0 go build -o bin/app-$os-$arch
 done
done
```

The toolchain ships the runtime for each pair, so you do not need a C cross-compiler unless cgo is enabled.

## Docker

```
Multi-stage build
FROM golang:1.27-alpine AS builder
WORKDIR /app
COPY . .
```

```
RUN go build -o /app/server ./cmd/server

FROM alpine:latest
COPY --from=builder /app/server /server
ENTRYPOINT ["/server"]
```

## GitHub Actions

```
name: Release
on:
 push:
 tags: ['v*']

jobs:
 build:
 runs-on: ubuntu-latest
 steps:
 - uses: actions/checkout@v4
 - uses: actions/setup-go@v5
 with:
 go-version: '1.26'
 - run: go build -o app
 - uses: softprops/action-gh-release@v1
 with:
 files: app
```

## GoReleaser

```
.goreleaser.yaml
builds:
 - main: ./cmd/server
 goos: [linux, darwin, windows]
 goarch: [amd64, arm64]

archives:
 - format: tar.gz

goreleaser release
```

## Summary

Task	Command
Build	<code>go build -o app</code>
Optimize	<code>-ldflags="-s -w"</code>
Cross-compile	<code>GOOS=linux GOARCH=amd64</code>
Version inject	<code>-ldflags="-X main.version=x"</code>

## More examples

### Version and commit via ldflags

```
mkdir -p /tmp/go-ldflags && cd /tmp/go-ldflags
go mod init example.com/ldflags
```

Save as `main.go`:

```
package main

import "fmt"

// Overridden at link time:
// go run -ldflags="-X main.version=1.2.3 -X main.commit=abc1234" .
var (
 version = "dev"
 commit = "none"
)

func main() {
 fmt.Printf("version=%s commit=%s\n", version, commit)
}

go run .
go run -ldflags="-X main.version=1.2.3 -X main.commit=abc1234" .
```

### Expected output:

```
version=dev commit=none
version=1.2.3 commit=abc1234
```

## Build info from the binary itself

```
mkdir -p /tmp/go-buildinfo && cd /tmp/go-buildinfo
go mod init example.com/buildinfo
```

Save as `main.go`:

```
package main

import (
 "fmt"
 "runtime/debug"
)

func main() {
 bi, ok := debug.ReadBuildInfo()
 if !ok {
 fmt.Println("no build info")
 return
 }
 fmt.Println("path:", bi.Path)
 fmt.Println("go:", bi.GoVersion)
 for _, s := range bi.Settings {
 switch s.Key {
 case "GOOS", "GOARCH", "-compiler", "CGO_ENABLED", "vcs.revision", "vcs.modified":
 fmt.Printf("%s=%s\n", s.Key, s.Value)
 }
 }
}

go run .
```

**Expected output** (keys vary by toolchain/VCS):

```
path: example.com/buildinfo
go: go1.22.x
GOOS=...
GOARCH=...
-compiler=gc
CGO_ENABLED=...
```

## Runnable example

Version injection and strip flags are the core of Go release builds. This program prints embedded metadata and reports binary-oriented build info.



```
mkdir -p /tmp/go-release-demo && cd /tmp/go-release-demo
go mod init example.com/release-demo
```

Save as main.go:

```
package main

import (
 "fmt"
 "runtime"
 "runtime/debug"
)

// Populated via: -ldflags="-X main.version=... -X main.commit=... -X main.date=..."
var (
 version = "dev"
 commit = "none"
 date = "unknown"
)

func main() {
 fmt.Printf("version=%s commit=%s date=%s\n", version, commit, date)
 fmt.Printf("runtime=%s %s/%s\n", runtime.Version(), runtime.GOOS, runtime.GOARCH)

 if info, ok := debug.ReadBuildInfo(); ok {
 fmt.Println("module:", info.Path)
 for _, s := range info.Settings {
 switch s.Key {
 case "-tags", "CGO_ENABLED", "GOOS", "GOARCH", "vcs.revision", "vcs.modified":
 fmt.Printf("build %s=%s\n", s.Key, s.Value)
 }
 }
 }
}

go run -ldflags="-X main.version=1.0.0 -X main.commit=abc1234 -X main.date=2026-07-28" .

CGO_ENABLED=0 go build -trimpath -ldflags="-s -w \
-X main.version=1.0.0 \
-X main.commit=abc1234 \
-X main.date=2026-07-28" -o app .

./app

Cross-compile matrix sample:
for pair in linux/amd64 linux/arm64 darwin/arm64 windows/amd64; do
```

```

GOOS=${pair%/*} GOARCH=${pair#*/}
ext=""; ["$GOOS" = windows] && ext=".exe"
CGO_ENABLED=0 go build -trimpath -ldflags="-s -w -X main.version=1.0.0" \
 -o "app-${GOOS}-${GOARCH}${ext}" .
echo "built app-${GOOS}-${GOARCH}${ext}"
done

```

**Expected output** (illustrative):

```

version=1.0.0 commit=abc1234 date=2026-07-28
runtime=go1.22.x darwin/arm64
module: example.com/release-demo
build GOOS=darwin
build GOARCH=arm64
build CGO_ENABLED=0

```

### What to notice

- `-X main.version=...` rewrites string vars at link time—no runtime git dependency in production.
- `-s -w` strips symbol/DWARF data; pair with `-trimpath` for more reproducible artifacts.
- `debug.ReadBuildInfo` exposes module and VCS metadata embedded by modern Go toolchains.

### Try next

- Compare `ls -la` sizes of stripped vs unstripped builds.
- Add a `-X main.commit=$(git rev-parse --short HEAD)` one-liner to a release script.

## Related Topics

- [Core Go Commands](#)
- [Documentation and APIs](#)
- [Go as a Systems Language](#)
- [Containerization](#)

# Kubernetes Deploy Basics for Go

# Kubernetes Deploy Basics for Go

## Overview

A minimal production shape: container image, Deployment, Service, probes, resources. Go's single binary fits multi-stage images well (chapter 70).

## Deployment sketch

```
apiVersion: apps/v1
kind: Deployment
metadata:
 name: bookstore
spec:
 replicas: 2
 selector:
 matchLabels: { app: bookstore }
 template:
 metadata:
 labels: { app: bookstore }
 spec:
 securityContext:
 runAsNonRoot: true
 runAsUser: 65532
 containers:
 - name: app
 image: ghcr.io/org/bookstore:1.2.3
 ports: [{ containerPort: 8080 }]
 env:
 - name: ADDR
 value: ":8080"
 readinessProbe:
 httpGet: { path: /readyz, port: 8080 }
 periodSeconds: 5
 livenessProbe:
 httpGet: { path: /livez, port: 8080 }
 periodSeconds: 10
 resources:
 requests: { cpu: "100m", memory: "128Mi" }
```

```
limits: { memory: "256Mi" }
securityContext:
 allowPrivilegeEscalation: false
 readOnlyRootFilesystem: true
```

## Service

```
apiVersion: v1
kind: Service
metadata:
 name: bookstore
spec:
 selector: { app: bookstore }
 ports: [{ port: 80, targetPort: 8080 }]
```

## Graceful stop

- `terminationGracePeriodSeconds` app shutdown timeout
- Fail readiness on SIGTERM first

## Config & secrets

```
envFrom:
 - secretRef: { name: bookstore-secrets }
```

Mount secrets as files when possible (TOKEN\_FILE pattern).

## Rules of thumb

Do	Don't
Probes + resources	Unlimited memory “to fix OOM” forever
Immutable tags (:1.2.3)	:latest in prod
Non-root	Root containers by habit

## Try next

1. Deploy locally with kind/minikube.
2. Break readiness; observe Service endpoints.
3. Rollout image; confirm zero-downtime with probes.

## GoReleaser and GitHub Actions

# GoReleaser and GitHub Actions

## Overview

Automate multi-OS binaries, checksums, and GitHub Releases with **GoReleaser** and a tag-driven workflow.

## Minimal `.goreleaser.yaml`

```
version: 2
builds:
 - id: app
 main: ./cmd/app
 env: [CGO_ENABLED=0]
 goos: [linux, darwin, windows]
 goarch: [amd64, arm64]
 ldflags:
 - -s -w -X main.version={{.Version}} -X main.commit={{.Commit}}
archives:
 - formats: [tar.gz]
 format_overrides:
 - goos: windows
 formats: [zip]
checksum:
 name_template: checksums.txt
```

## Workflow sketch

```
.github/workflows/release.yml
on:
 push:
 tags: ["v*"]
jobs:
 release:
 runs-on: ubuntu-latest
 permissions:
 contents: write
```



```
steps:
 - uses: actions/checkout@v4
 with: { fetch-depth: 0 }
 - uses: actions/setup-go@v5
 with: { go-version-file: go.mod }
 - uses: goreleaser/goreleaser-action@v6
 with:
 args: release --clean
 env:
 GITHUB_TOKEN: ${ secrets.GITHUB_TOKEN }
```

## Local snapshot

```
goreleaser release --snapshot --clean
```

## Rules of thumb

Do	Don't
Tag semver v1.2.3	Untagged mystery builds as “prod”
Pin action versions	@master for release
Verify checksums	Trust unsigned downloads blindly

## Try next

1. Snapshot release locally.
2. Add `version` subcommand reading ldflags.
3. Attach `govulncheck` job on PR.

## Container Health and Signals

# Container Health and Signals

## Overview

Containers change process semantics: PID 1, signal delivery, and healthchecks. Go apps should handle **SIGTERM**, expose probes, and avoid zombie reaping issues (use a proper init if needed).

## Dockerfile healthcheck (optional)

```
HEALTHCHECK --interval=10s --timeout=2s --retries=3 \
 CMD wget -qO- http://127.0.0.1:8080/livez || exit 1
```

Prefer orchestrator probes in K8s over relying only on Docker HEALTHCHECK.

## Signal PID 1

If your app is PID 1 and spawns children, consider **tini** or ensure children are reaped. Many Go apps don't spawn—still handle SIGTERM yourself.

```
ctx, stop := signal.NotifyContext(context.Background(), syscall.SIGTERM, syscall.SIGINT)
defer stop()
```

## Read-only rootfs

```
run as non-root; mount emptyDir for temp if needed
USER nonroot:nonroot
```

Write only to **/tmp** or mounted volumes.

Concern	Practice
---------	----------

## 12-factor in containers

Concern	Practice
Config	env + files
Logs	stdout/stderr
Graceful	SIGTERM → Shutdown
Port	ADDR=:8080

## Rules of thumb

Do	Don't
Listen on 0.0.0.0/: in container	Hardcode localhost only for in-cluster listen
Exit non-zero on fatal boot	Hang forever if DB missing (or use restart policy consciously)
Document probe paths	Silent health

## Try next

1. `docker run + docker stop`; measure drain time.
2. Read-only rootfs; fix writes to temp.
3. Align HEALTHCHECK with `/livez`.

# Security

# Security Tools

# Security Tools: Automating Defense

## Overview

Security should not depend on heroic manual review. Go has a strong toolchain for **static detection**, **reachable vulnerability analysis**, **capability inventory**, and **cryptographic testing**. Wire these into CI so regressions fail the build before they reach production.

This chapter covers `govulncheck`, `gosec`, `capslock`, `nilaway`, modern crypto helpers (`crypto/hpke`, `testing/cryptotest`), experimental secret handling, and how to assemble them into a pipeline.

## Suggested Security Pipeline

```
commit
--> go vet / golangci-lint
--> gosec (or security linters)
--> nilaway (optional, deeper nil analysis)
--> govulncheck (reachable vulns)
--> capslock (dependency capability review)
--> go test (incl. crypto vectors)
--> release / sign
```

trust boundary diagram

```
developer laptop --> CI runners --> artifact registry --> runtime
| | | |
secrets pinned actions cosign verify least privilege
```

## 1. `govulncheck` — Reachability-Based Vuln Scanning

`govulncheck` uses the Go vulnerability database and **call graph reachability**. It prefers reporting issues your code can actually hit, not every CVE in the module graph.

```
go install golang.org/x/vuln/cmd/govulncheck@latest
govulncheck ./...
```

```
JSON for CI parsing
```

```
govulncheck -json ./... > govulncheck.json
```

## CI snippet

```
- name: govulncheck
 run: |
 go install golang.org/x/vuln/cmd/govulncheck@latest
 govulncheck ./...
```

## Interpreting results

1. Upgrade the module if a fixed version exists.
2. If unused, confirm with `govulncheck` (symbol level)—don't ignore blindly.
3. Vendor or replace only with a documented exception and expiry.

```
go get example.com/module@v1.2.4
go mod tidy
govulncheck ./...
```

## 2. gosec — AST Security Linter

`gosec` walks the AST looking for dangerous patterns.

```
go install github.com/securego/gosec/v2/cmd/gosec@latest
gosec ./...

Exclude generated code
gosec -exclude-generated ./...
```

### Common findings:

Pattern	Risk
Hardcoded credentials	Secret leakage
SQL via string concat	Injection
<code>math/rand</code> for security tokens	Predictable values
<code>chmod 0777</code> / weak file perms	Local privilege issues
<code>http</code> without TLS in prod configs	Cleartext
<code>unsafe</code> / weak crypto APIs	Memory / crypto breaks



## Safer random and SQL examples

```
// Bad: math/rand for tokens
// token := fmt.Sprintf("%d", rand.Int())

// Good: crypto/rand
func sessionToken() (string, error) {
 var b [16]byte
 if _, err := rand.Read(b[:]); err != nil {
 return "", err
 }
 return hex.EncodeToString(b[:]), nil
}

// Bad
// db.Query("SELECT * FROM users WHERE name = '" + name + "'")

// Good
rows, err := db.QueryContext(ctx, `SELECT id, name FROM users WHERE name = $1`, name)
```

Tune severity in CI: fail on high/critical; track medium findings.

## 3. capslock — Capability Analysis

Google's **capslock** reports what *capabilities* a package graph uses: network, filesystem, runtime, unsafe, os exec, etc.

```
go install github.com/google/capslock/cmd/capslock@latest
capslock -packages ./...
```

Use it when reviewing dependencies:

- A pure string helper that needs **Network** is a smell.
- A config parser that needs **OsExec** deserves scrutiny.
- Diff capslock output across PRs that bump dependencies.

```
expected for a CLI file tool: FileSystem, maybe Runtime
unexpected for left-pad clone: Network, CAP_SYS_ADMIN stories, etc.
```

## 4. nilaway — Static Nil Flows

Uber's **nilaway** tracks potential nil dereferences across functions more deeply than many linters.

```
go install go.uber.org/nilaway/cmd/nilaway@latest
nilaway ./...
```

It does not turn Go into Rust, but it catches real production panics:

```
func loadUser(id string) (*User, error) {
 if id == "" {
 return nil, fmt.Errorf("empty id")
 }
 // ...
 return u, nil
}

func handler() {
 u, err := loadUser(id)
 if err != nil {
 return
 }
 // nilaway helps when err is ignored or only partially checked
 fmt.Println(u.Name)
}
```

Run as a periodic or blocking CI job if the signal-to-noise ratio works for your codebase.

## 5. Modern Crypto Tooling

### crypto/hpke (Hybrid Public Key Encryption)

HPKE (RFC 9180) combines KEM + KDF + AEAD for standardized envelope encryption—useful for config blobs, recipient-encrypted messages, and modern protocol designs.

```
import (
 "crypto/hpke"
 "crypto/rand"
)

func encryptHPKE(recipientPub hpke.PublicKey, msg []byte) (enc, ciphertext []byte, err error) {
 suite := hpke.NewSuite(
 hpke.KEM_X25519_HKDF_SHA256,
 hpke.KDF_HKDF_SHA256,
 hpke.AEAD_AES_128_GCM,
)
 sender, err := suite.NewSender(recipientPub, nil)
 if err != nil {
 return nil, nil, err
 }
}
```

```

 }
 enc, sealer, err := sender.Setup(rand.Reader)
 if err != nil {
 return nil, nil, err
 }
 ciphertext, err = sealer.Seal(msg, nil)
 return enc, ciphertext, err
}

```

```

flow:
] -> KEM[
] -> AEAD[
 KEM -> EncapsKey[
 KEM -> SharedSecret[
 EncapsKey -> Network
 AEAD -> CipherText
] -> Decaps[
 Network -> Decaps
 Decaps -> AEADDec[
 Network -> AEADDec
 AEADDec -> Plain[

```

Prefer stdlib crypto primitives over hand-rolled combinations of ECDH + HKDF + GCM.

## testing/cryptotest

Use `testing/cryptotest` (where available in your Go version) to exercise cryptographic implementations against known vectors, basic correctness, and timing-related expectations for custom code that wraps stdlib primitives.

```

func TestHMACVectors(t *testing.T) {
 // table-driven known-answer tests for your wrappers
 mac := hmac.New(sha256.New, []byte("key"))
 mac.Write([]byte("data"))
 got := mac.Sum(nil)
 want := mustHex("...") // known vector
 if !hmac.Equal(got, want) {
 t.Fatalf("mismatch")
 }
}

```

Always compare secrets with **constant-time** helpers (`subtle.ConstantTimeCompare`, `hmac.Equal`).

## 6. Experimental: runtime/secret

Experiments around tighter secret lifetimes appear behind build tags / `GOEXPERIMENT`. Treat them as **opt-in** and verify support for your Go version before relying on them in production.

```
//go:build goexperiment.runtimesecret

import "runtime/secret"

func useSecret(token []byte) {
 secret.Do(func() {
 // limited lifetime window for sensitive material
 _ = token
 })
}

GOEXPERIMENT=runtimesecret go test ./...
```

Still follow classic hygiene: minimize copies, zero buffers when practical, don't log secrets, prefer OS keychains/HSMs for long-term keys.

## 7. Complementary Checks

Tool	Role
go vet	Compiler-adjacent mistakes
golangci-lint	Aggregates many linters
staticcheck	High-signal correctness
semgrep / CodeQL	Org-wide custom rules
govulncheck	Module CVEs with reachability
Image scanners (trivy)	Container base CVEs

Example golangci config fragment:

```
linters:
 enable:
 - gosec
 - govet
 - staticcheck
 - errcheck
```

## Wiring a Makefile / CI Target

```
.PHONY: sec
sec:
 go vet ./...
 gosec -quiet ./...
 govulncheck ./...

GitHub Actions job fragment
- name: Security
 run: make sec
```

## Production Checklist

- ☐ govulncheck ./... on every PR
- ☐ gosec or equivalent security linters enabled
- ☐ Dependency bumps reviewed with capslock when capabilities jump
- ☐ No `math/rand` for security tokens
- ☐ Parameterized SQL only
- ☐ Known-answer tests for crypto wrappers
- ☐ Secrets never in git; scanner in pre-commit optional
- ☐ Container images scanned on release
- ☐ Exceptions documented with owner + expiry

## Common Pitfalls

1. **Ignoring govulncheck because “it’s transitive”** — reachability exists for a reason; verify.
2. **Disabling gosec rules globally** — narrow suppressions with comments and justification.
3. **Custom crypto** — combining primitives incorrectly; prefer HPKE/TLS/libsodium-grade designs.
4. **Logging tokens** — still the #1 operational leak.
5. **Security tools only on main** — PRs need the same gates.
6. **False confidence** — static tools don’t replace threat modeling or fuzzing of parsers.

## Exercises

1. **Install and run** — Run govulncheck and gosec on this book’s sample modules or a toy service; fix or document each finding.
2. **Token generator** — Implement `sessionToken` with `crypto/rand`; add a test that 1000 tokens are unique and correct length.
3. **SQL injection lab** — Write a deliberate vulnerable query (in a `_test` file or clearly broken sample) and show gosec flags it; then fix with parameters.

4. **Capslock review** — Run capslock on a module that imports `net/http` vs a pure algorithm package; compare capability sets.
5. **HMAC vectors** — Table-test an HMAC helper against a published RFC vector; use `hmac.Equal`.
6. **CI gate** — Add a `make sec` target and a workflow step that fails the build on findings.

## More examples

### Constant-time compare (HMAC / tokens)

```
mkdir -p /tmp/go-hmac-eq && cd /tmp/go-hmac-eq
go mod init example.com/hmac-eq
```

Save as `main.go`:

```
package main

import (
 "crypto/hmac"
 "crypto/sha256"
 "encoding/hex"
 "fmt"
)

func sign(secret, msg string) string {
 m := hmac.New(sha256.New, []byte(secret))
 _, _ = m.Write([]byte(msg))
 return hex.EncodeToString(m.Sum(nil))
}

func valid(secret, msg, gotHex string) bool {
 want := sign(secret, msg)
 // Decode both; compare with hmac.Equal (constant-time on equal-length slices).
 a, err1 := hex.DecodeString(want)
 b, err2 := hex.DecodeString(gotHex)
 if err1 != nil || err2 != nil || len(a) != len(b) {
 return false
 }
 return hmac.Equal(a, b)
}

func main() {
 secret, msg := "s3cr3t", "POST /pay|42"
 sig := sign(secret, msg)
 fmt.Println("sig ok:", valid(secret, msg, sig))
}
```

```

 fmt.Println("tampered:", valid(secret, msg, sig[:len(sig)-1]+"0"))
 fmt.Println("wrong secret:", valid("other", msg, sig))
}

go run .

```

### Expected output:

```

sig ok: true
tampered: false
wrong secret: false

```

### Path jail (block traversal)

```

mkdir -p /tmp/go-path-jail && cd /tmp/go-path-jail
go mod init example.com/path-jail

```

Save as main.go:

```

package main

import (
 "fmt"
 "path/filepath"
 "strings"
)

func underRoot(root, userPath string) (string, error) {
 clean := filepath.Clean("/") + userPath // force absolute-ish clean
 clean = strings.TrimPrefix(clean, string(filepath.Separator))
 full := filepath.Join(root, clean)
 rel, err := filepath.Rel(root, full)
 if err != nil || strings.HasPrefix(rel, "..") {
 return "", fmt.Errorf("escape: %q", userPath)
 }
 return full, nil
}

func main() {
 root := "/var/app/data"
 for _, p := range []string{"docs/a.txt", "../etc/passwd", "/etc/passwd"} {
 full, err := underRoot(root, p)
 if err != nil {
 fmt.Printf("%q -> ERR %v\n", p, err)
 continue
 }
 }
}

```

```

 }
 fmt.Printf("%q -> %s\n", p, full)
}
}

```

```
go run .
```

### Expected output:

```

"docs/a.txt" -> /var/app/data/docs/a.txt
"./etc/passwd" -> ERR escape: "../etc/passwd"
"/etc/passwd" -> ERR escape: "/etc/passwd"

```

## Runnable example

Security tools are external; the patterns they enforce are stdlib. This program generates session tokens with `crypto/rand`, signs and verifies an HMAC, and compares digests in constant time—the kind of code `gosec` expects instead of `math/rand`.

```

mkdir -p /tmp/go-sec-tools && cd /tmp/go-sec-tools
go mod init example.com/sec-tools

```

Save as `main.go`:

```

package main

import (
 "crypto/hmac"
 "crypto/rand"
 "crypto/sha256"
 "encoding/hex"
 "fmt"
 "os"
)

func sessionToken(n int) (string, error) {
 b := make([]byte, n)
 if _, err := rand.Read(b); err != nil {
 return "", err
 }
 return hex.EncodeToString(b), nil
}

func sign(key, msg []byte) []byte {
 m := hmac.New(sha256.New, key)

```



```

 _, _ = m.Write(msg)
 return m.Sum(nil)
}

func main() {
 tok, err := sessionToken(16)
 if err != nil {
 fmt.Fprintln(os.Stderr, err)
 os.Exit(1)
 }
 fmt.Println("token:", tok)
 fmt.Println("token_len_hex:", len(tok))

 key := make([]byte, 32)
 if _, err := rand.Read(key); err != nil {
 fmt.Fprintln(os.Stderr, err)
 os.Exit(1)
 }
 msg := []byte("order:42:ship")
 sig := sign(key, msg)
 fmt.Println("hmac:", hex.EncodeToString(sig))

 // Constant-time compare (never == on secret slices).
 again := sign(key, msg)
 if !hmac.Equal(sig, again) {
 fmt.Println("verify: FAIL")
 os.Exit(1)
 }
 fmt.Println("verify: ok")

 // Wrong key must fail.
 badKey := make([]byte, 32)
 _, _ = rand.Read(badKey)
 if hmac.Equal(sig, sign(badKey, msg)) {
 fmt.Println("bad key unexpectedly matched")
 os.Exit(1)
 }
 fmt.Println("bad_key_rejected: ok")

 // Uniqueness smoke test.
 seen := map[string]struct{}{}
 for i := 0; i < 1000; i++ {
 t, err := sessionToken(16)
 if err != nil {
 fmt.Fprintln(os.Stderr, err)
 os.Exit(1)
 }
 }
}

```

```

 if _, ok := seen[t]; ok {
 fmt.Println("duplicate token")
 os.Exit(1)
 }
 seen[t] = struct{}{}
 }
 fmt.Println("unique_tokens: 1000")
}

go run .

```

**Expected output** (illustrative; tokens random):

```

token: a3f1c9...
token_len_hex: 32
hmac: 9b2e...
verify: ok
bad_key_rejected: ok
unique_tokens: 1000

```

### What to notice

- `crypto/rand` is for security-sensitive values; `math/rand` is a classic `gosec` finding.
- `hmac.Equal` (or `subtle.ConstantTimeCompare`) avoids timing leaks from `bytes.Equal` on secrets.
- Known-answer tests and uniqueness checks belong in CI next to `govulncheck` / `gosec`.

### Try next

- Add a table-driven HMAC test with a fixed key/message and expected hex digest.
- Run `gosec ./...` and `govulncheck ./...` on this module once the tools are installed.

## Related Topics

- [Application Hardening](#)
- [CI/CD & Hardening](#)
- [Static Analysis](#)
- [TLS / mTLS](#)

# Application Hardening

# Application Hardening

Security tools find known problems. **Hardening** is the engineering practice of making services difficult to exploit and safe to operate when something goes wrong. This chapter is the production checklist for Go services: least privilege, attack-surface reduction, input distrust, secure defaults, and fail-closed behavior.

## Why Hardening Matters

A service can pass `govulncheck` and still be fragile:

- Debug endpoints exposed on the public interface
- Overly broad filesystem or network capabilities
- Missing request size limits (memory exhaustion)
- Secrets loaded from env forever with no rotation path
- Process running as root “because Docker made it easy”

Hardening is not a single library. It is a set of defaults you enforce until they become muscle memory.

```
attack surface
|-- network listeners
|-- auth boundaries
|-- parse/decode paths
|-- filesystem access
|-- subprocesses / cgo
|-- admin/debug tooling
+-- dependency graph
```

## Core Principles

1. **Least privilege** — the process gets only what it needs (UID, capabilities, mounts, outbound destinations).
2. **Fail closed** — ambiguous auth, bad config, or missing secrets stop the service rather than open a hole.
3. **Distrust boundaries** — every byte from the network, queue, or filesystem is hostile until validated.
4. **Defense in depth** — TLS + authn + authz + rate limits + audit logs; no single control is enough.
5. **Operational security** — logs, metrics, and admin APIs are part of the threat model.

# Hardening the Process

## Drop privileges early

Never keep root after bind if you can avoid it. Prefer binding via a privileged parent, then drop; or use non-privileged ports behind a reverse proxy.

```
package main

import (
 "log"
 "net/http"
 "os"
 "os/user"
 "strconv"
 "syscall"
 "time"
)

func dropToUser(username string) error {
 u, err := user.Lookup(username)
 if err != nil {
 return err
 }
 uid, _ := strconv.Atoi(u.Uid)
 gid, _ := strconv.Atoi(u.Gid)
 // Order matters: groups, then gid, then uid.
 if err := syscall.Setgid(gid); err != nil {
 return err
 }
 if err := syscall.Setuid(uid); err != nil {
 return err
 }
 return nil
}

func main() {
 // Prefer: start as non-root in the container/image.
 // If you must bind privileged ports, drop after Listen.
 if os.Geteuid() == 0 {
 log.Println("warning: running as root; drop privileges in production")
 }

 mux := http.NewServeMux()
 mux.HandleFunc("GET /healthz", func(w http.ResponseWriter, r *http.Request) {
 w.WriteHeader(http.StatusOK)
 })
}
```

```

 srv := &http.Server{
 Addr: ":8080",
 Handler: mux,
 ReadHeaderTimeout: 5 * time.Second,
 }
 log.Fatal(srv.ListenAndServe())
}

```

In practice use a container `USER` directive and a distroless/static image rather than runtime `setuid`. The Go code above is for understanding the principle.

## Container baseline

```

multi-stage: build as root, run as non-root
FROM golang:1.22-bookworm AS build
WORKDIR /src
COPY . .
RUN CGO_ENABLED=0 go build -trimpath -ldflags="-s -w" -o /out/app ./cmd/app

FROM gcr.io/distroless/static-debian12:nonroot
COPY --from=build /out/app /app
USER nonroot:nonroot
EXPOSE 8080
ENTRYPOINT ["/app"]

```

Production notes:

- `CGO_ENABLED=0` shrinks the binary and removes libc attack surface (when pure Go is viable).
- `-trimpath` removes local paths from binaries (less recon info).
- Read-only root filesystem + tmpfs for scratch dirs in orchestrators.
- No shell in the image reduces interactive abuse after break-in.

## Hardening the HTTP Surface

### Timeouts and body limits

Unbounded bodies and missing timeouts are DoS vectors.

```

package httpserver

import (
 "context"
 "encoding/json"
 "io"
 "net"

```

```

 "net/http"
 "time"
)

const maxBody = 1 << 20 // 1 MiB

func NewServer(addr string, h http.Handler) *http.Server {
 return &http.Server{
 Addr: addr,
 Handler: limitBody(h, maxBody),
 ReadHeaderTimeout: 5 * time.Second,
 ReadTimeout: 15 * time.Second,
 WriteTimeout: 30 * time.Second,
 IdleTimeout: 60 * time.Second,
 MaxHeaderBytes: 1 << 16, // 64 KiB
 BaseContext: func(net.Listener) context.Context {
 return context.Background()
 },
 }
}

func limitBody(next http.Handler, n int64) http.Handler {
 return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
 r.Body = http.MaxBytesReader(w, r.Body, n)
 next.ServeHTTP(w, r)
 })
}

// Safe JSON decode: size-limited + DisallowUnknownFields for strict APIs.
func DecodeJSON(w http.ResponseWriter, r *http.Request, dst any) error {
 defer r.Body.Close()
 dec := json.NewDecoder(io.LimitReader(r.Body, maxBody))
 dec.DisallowUnknownFields()
 if err := dec.Decode(dst); err != nil {
 http.Error(w, "bad request", http.StatusBadRequest)
 return err
 }
 return nil
}

```

Import `encoding/json` alongside the other packages in a real module.

## Separate admin and data planes

Never mount `pprof` or admin UIs on the public listener.

```

package main

import (
 "log"
 "net/http"
 _ "net/http/pprof"
)

func main() {
 // Public traffic only.
 public := http.NewServeMux()
 public.HandleFunc("GET /api/v1/items", listItems)
 go func() {
 log.Fatal(http.ListenAndServe(":8080", public))
 }()

 // Admin/debug on localhost or a private network interface only.
 admin := http.NewServeMux()
 admin.Handle("/debug/pprof/", http.DefaultServeMux)
 admin.HandleFunc("GET /readyz", readiness)
 log.Fatal(http.ListenAndServe("127.0.0.1:6060", admin))
}

```

## Security headers and TLS defaults

```

func securityHeaders(next http.Handler) http.Handler {
 return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
 w.Header().Set("X-Content-Type-Options", "nosniff")
 w.Header().Set("X-Frame-Options", "DENY")
 w.Header().Set("Referrer-Policy", "no-referrer")
 w.Header().Set("Content-Security-Policy", "default-src 'none'; frame-ancestors 'none'")
 // HSTS only when TLS terminates here or is guaranteed upstream.
 // w.Header().Set("Strict-Transport-Security", "max-age=63072000; includeSubDomains")
 next.ServeHTTP(w, r)
 })
}

```

Prefer TLS 1.2+ with modern cipher suites; see the dedicated TLS chapter in part 17 for mTLS and cert rotation.



## Input Validation and Injection Defense

### SQL: parameters only

```
// BAD: string concatenation → injection
// q := "SELECT * FROM users WHERE name = '" + name + "'"

// GOOD: placeholders
row := db.QueryRowContext(ctx, `SELECT id, email FROM users WHERE name = $1`, name)
```

### Paths: root-jail every user-supplied path

```
package safepath

import (
 "fmt"
 "path/filepath"
 "strings"
)

func UnderRoot(root, userPath string) (string, error) {
 cleanRoot := filepath.Clean(root)
 joined := filepath.Join(cleanRoot, userPath)
 // Resolve ".." and symlinks carefully; Clean alone is not enough against all symlink es
 abs, err := filepath.Abs(joined)
 if err != nil {
 return "", err
 }
 rel, err := filepath.Rel(cleanRoot, abs)
 if err != nil || strings.HasPrefix(rel, "..") {
 return "", fmt.Errorf("path escapes root")
 }
 return abs, nil
}
```

### Commands: never shell out with untrusted strings

```
// BAD
// exec.Command("sh", "-c", "convert "+userFile)

// GOOD: fixed binary + argv array
cmd := exec.CommandContext(ctx, "convert", userFile, outFile)
```

## Randomness and tokens

Use `crypto/rand` for session IDs, CSRF tokens, and API keys — never `math/rand`.

```
func randomToken(n int) (string, error) {
 b := make([]byte, n)
 if _, err := rand.Read(b); err != nil {
 return "", err
 }
 return hex.EncodeToString(b), nil
}
```

## Secrets and Configuration

Do	Don't
Load from env / secret manager at runtime	Commit secrets to git
Fail fast on missing required secrets	Silently use empty defaults
Support dual-read during rotation	Hard-code a single static key forever
Redact secrets in logs	Log full Authorization headers

```
func mustEnv(key string) string {
 v := os.Getenv(key)
 if v == "" {
 slog.Error("missing required secret", "key", key)
 os.Exit(1)
 }
 return v
}
```

## Dependency and Supply-Chain Hardening

1. Pin modules with `go.sum`; review unexpected upgrades in PRs.
2. Run `govulncheck ./...` in CI on every change.
3. Prefer small, well-maintained libraries over kitchen-sink frameworks for security-sensitive paths.
4. Use `capslock` (or similar) when adopting new deps that should not need network/FS capabilities.
5. Build with reproducible flags; publish checksums for release artifacts.

```
go install golang.org/x/vuln/cmd/govulncheck@latest
govulncheck ./...
gosec ./...
```

## Runtime Abuse Controls

### Rate limiting (simple token bucket sketch)

```
type limiter struct {
 mu sync.Mutex
 tokens float64
 max float64
 refill float64 // tokens per second
 last time.Time
}

func (l *limiter) Allow() bool {
 l.mu.Lock()
 defer l.mu.Unlock()
 now := time.Now()
 elapsed := now.Sub(l.last).Seconds()
 l.last = now
 l.tokens += elapsed * l.refill
 if l.tokens > l.max {
 l.tokens = l.max
 }
 if l.tokens < 1 {
 return false
 }
 l.tokens--
 return true
}
```

Place rate limits per IP / per principal *after* authentication for authenticated APIs so one user cannot exhaust another's budget.

### Panic recovery at the edge

Recover panics in HTTP handlers so one bad request does not kill the process — but log stack traces and return 500, never continue with half-written responses.

```
func recoverMiddleware(next http.Handler) http.Handler {
 return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
 defer func() {
```

```

 if rec := recover(); rec != nil {
 slog.Error("panic", "err", rec, "path", r.URL.Path)
 http.Error(w, "internal error", http.StatusInternalServerError)
 }
 }()
 next.ServeHTTP(w, r)
}
}

```

## Production Hardening Checklist

- ☐ Non-root process / distroless image
- ☐ Read-only root FS where possible
- ☐ Public listener has full timeout suite + body limits
- ☐ Admin/pprof not on public interface
- ☐ TLS 1.2+ (or terminated by a hardened proxy with the same bar)
- ☐ Authn before authz; default deny
- ☐ Parameterized queries; no shell interpolation
- ☐ Path jail for file operations
- ☐ Secrets from env/manager; fail-closed on missing
- ☐ Structured logs with redaction
- ☐ `govulncheck` + static security checks in CI
- ☐ Graceful shutdown on SIGTERM with deadline
- ☐ Resource limits (CPU/memory) set in orchestrator
- ☐ Network policy: egress allowlist when feasible

## Common Pitfalls

1. **Trusting reverse proxies blindly** — if you use `X-Forwarded-For` for rate limits or auth, validate that only trusted proxies can set it.
2. **Debug builds in prod** — race detector and extra logging are fine; open pprof on `0.0.0.0` is not.
3. **“Internal” network = secure** — flat VPC trust is outdated; use mTLS or service identity between services.
4. **Logging secrets** — JWT, cookies, and API keys leak through request dumps.
5. **Unbounded fan-out** — one authenticated request spawning N unbounded downstream calls is a DoS amplifier.
6. **Ignoring context** — operations that ignore cancellation hold resources under attack load.
7. **World-writable files** — `0666/0777` permissions in services that write state.

## Exercises

1. Take a minimal `net/http` server and add body limits, header/read/write timeouts, and a max header size. Load-test with a slow client and confirm it disconnects.
2. Split public and admin muxes; prove with `curl` that `pprof` is unreachable on `:8080`.
3. Write a `UnderRoot` function and unit-test path traversal attempts (`../../etc/passwd`, absolute paths, symlink escapes if you resolve them).
4. Introduce a deliberate SQL string concat and show `gosec` (or a review checklist) catching it; fix with placeholders.
5. Implement fail-closed startup: missing `DATABASE_URL` must exit non-zero before listening.
6. Add panic recovery middleware and a handler that panics; verify the process stays up and a 500 is returned.
7. Build a multi-stage Dockerfile that runs as non-root and contains no shell; inspect the image.
8. Implement a per-IP rate limiter middleware and verify 429 responses under burst.
9. Audit your logs: log a fake Authorization header once, then add redaction so it never appears again.
10. Run `govulncheck ./...` on a sample module and document how you would gate merges on new findings.

## Further Reading

- Go security policy and release notes for crypto changes
- OWASP ASVS (application security verification)
- Part 17 of this book: TLS/mTLS, authn/authz, secrets rotation
- Previous chapter: security tooling (`govulncheck`, `gosec`, capability analysis)

## More examples

### Body limit + security headers middleware

```
mkdir -p /tmp/go-harden-mw && cd /tmp/go-harden-mw
go mod init example.com/harden-mw
```

Save as `main.go`:

```
package main

import (
 "fmt"
 "io"
 "net/http"
 "net/http/httptest"
 "strings"
)
```

```

func secureHeaders(next http.Handler) http.Handler {
 return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
 w.Header().Set("X-Content-Type-Options", "nosniff")
 w.Header().Set("X-Frame-Options", "DENY")
 w.Header().Set("Referrer-Policy", "no-referrer")
 next.ServeHTTP(w, r)
 })
}

func maxBytes(n int64, next http.Handler) http.Handler {
 return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
 r.Body = http.MaxBytesReader(w, r.Body, n)
 next.ServeHTTP(w, r)
 })
}

func main() {
 h := secureHeaders(maxBytes(8, http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
 b, err := io.ReadAll(r.Body)
 if err != nil {
 http.Error(w, "payload too large", http.StatusRequestEntityTooLarge)
 return
 }
 fmt.Fprintf(w, "got %d bytes\n", len(b))
 })))

 // OK body
 rr := httptest.NewRecorder()
 req := httptest.NewRequest(http.MethodPost, "/", strings.NewReader("hello"))
 h.ServeHTTP(rr, req)
 fmt.Println("small:", rr.Code, strings.TrimSpace(rr.Body.String()), rr.Header().Get("X-Content-Type-Options"))

 // Oversized
 rr = httptest.NewRecorder()
 req = httptest.NewRequest(http.MethodPost, "/", strings.NewReader("0123456789"))
 h.ServeHTTP(rr, req)
 fmt.Println("large:", rr.Code)
}

go run .

```

### Expected output:

```

small: 200 got 5 bytes nosniff
large: 413

```

## Fail-closed required config

```
mkdir -p /tmp/go-fail-closed && cd /tmp/go-fail-closed
go mod init example.com/fail-closed
```

Save as main.go:

```
package main

import (
 "fmt"
 "os"
)

func requireEnv(keys ...string) error {
 for _, k := range keys {
 if os.Getenv(k) == "" {
 return fmt.Errorf("missing required env %s", k)
 }
 }
 return nil
}

func main() {
 os.Unsetenv("DATABASE_URL")
 os.Setenv("APP_SECRET", "x")
 err := requireEnv("DATABASE_URL", "APP_SECRET")
 fmt.Println("before:", err)

 os.Setenv("DATABASE_URL", "postgres://local/app")
 err = requireEnv("DATABASE_URL", "APP_SECRET")
 fmt.Println("after:", err)
}

go run .
```

## Expected output:

```
before: missing required env DATABASE_URL
after: <nil>
```

## Runnable example

Hardening is timeouts, body limits, path jails, and fail-closed config. This server rejects oversized bodies, blocks path escape, and requires a secret env var before listening.

```
mkdir -p /tmp/go-hardening && cd /tmp/go-hardening
go mod init example.com/hardening
```

Save as main.go:

```
package main

import (
 "encoding/json"
 "fmt"
 "io"
 "log"
 "net"
 "net/http"
 "os"
 "path/filepath"
 "strings"
 "time"
)

const maxBody = 1 << 10 // 1 KiB for the demo

func underRoot(root, userPath string) (string, error) {
 cleanRoot := filepath.Clean(root)
 joined := filepath.Join(cleanRoot, userPath)
 abs, err := filepath.Abs(joined)
 if err != nil {
 return "", err
 }
 rootAbs, err := filepath.Abs(cleanRoot)
 if err != nil {
 return "", err
 }
 rel, err := filepath.Rel(rootAbs, abs)
 if err != nil || strings.HasPrefix(rel, "..") {
 return "", fmt.Errorf("path escapes root")
 }
 return abs, nil
}

func limitBody(next http.Handler, n int64) http.Handler {
 return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
 r.Body = http.MaxBytesReader(w, r.Body, n)
 next.ServeHTTP(w, r)
 })
}
```



```

func securityHeaders(next http.Handler) http.Handler {
 return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
 w.Header().Set("X-Content-Type-Options", "nosniff")
 w.Header().Set("X-Frame-Options", "DENY")
 next.ServeHTTP(w, r)
 })
}

func main() {
 // Fail closed in production: if os.Getenv("API_TOKEN") == "" { log.Fatal("missing API_T
 token := os.Getenv("API_TOKEN")
 if token == "" {
 token = "dev-only-token"
 fmt.Println("warning: API_TOKEN unset; using dev-only-token")
 }
 fmt.Println("token_configured:", token != "")

 root := filepath.Join(os.TempDir(), "hardening-demo-root")
 _ = os.MkdirAll(root, 0o750)
 _ = os.WriteFile(filepath.Join(root, "note.txt"), []byte("safe\n"), 0o640)

 mux := http.NewServeMux()
 mux.HandleFunc("GET /healthz", func(w http.ResponseWriter, _ *http.Request) {
 w.WriteHeader(http.StatusOK)
 _, _ = w.Write([]byte("ok"))
 })
 mux.HandleFunc("POST /echo", func(w http.ResponseWriter, r *http.Request) {
 b, err := io.ReadAll(r.Body)
 if err != nil {
 http.Error(w, "payload too large or unreadable", http.StatusRequestEntityTooLarge)
 return
 }
 w.Header().Set("Content-Type", "application/json")
 _ = json.NewEncoder(w).Encode(map[string]any{"n": len(b)})
 })
 mux.HandleFunc("GET /file", func(w http.ResponseWriter, r *http.Request) {
 name := r.URL.Query().Get("name")
 path, err := underRoot(root, name)
 if err != nil {
 http.Error(w, "forbidden", http.StatusForbidden)
 return
 }
 b, err := os.ReadFile(path)
 if err != nil {
 http.Error(w, "not found", http.StatusNotFound)
 return
 }
 })
}

```

```

 _, _ = w.Write(b)
})

ln, err := net.Listen("tcp", "127.0.0.1:0")
if err != nil {
 log.Fatal(err)
}
h := securityHeaders(limitBody(mux, maxBody))
srv := &http.Server{
 Handler: h,
 ReadHeaderTimeout: 5 * time.Second,
 ReadTimeout: 10 * time.Second,
 WriteTimeout: 10 * time.Second,
 MaxHeaderBytes: 1 << 14,
}
go func() { _ = srv.Serve(ln) }()
base := "http://" + ln.Addr().String()

// Self-tests.
resp, _ := http.Get(base + "/healthz")
fmt.Println("healthz:", resp.StatusCode)
resp.Body.Close()

resp, _ = http.Get(base + "/file?name=note.txt")
fmt.Println("file ok:", resp.StatusCode)
resp.Body.Close()

resp, _ = http.Get(base + "/file?name=../../etc/passwd")
fmt.Println("path escape:", resp.StatusCode)
resp.Body.Close()

big := strings.Repeat("x", int(maxBody)+10)
resp, err = http.Post(base+"/echo", "text/plain", strings.NewReader(big))
if err != nil {
 fmt.Println("oversize post err:", err)
} else {
 fmt.Println("oversize body:", resp.StatusCode)
 resp.Body.Close()
}

_ = srv.Close()
fmt.Println("hardening demo done")
// Production: replace the soft default above with token := mustEnv("API_TOKEN").
}

```

```
go run .
API_TOKEN=secret go run .
```

**Expected output** (illustrative):

```
warning: API_TOKEN unset; using dev-only-token
token_configured: true
healthz: 200
file ok: 200
path escape: 403
oversize body: 413
hardening demo done
```

### What to notice

- `http.MaxBytesReader` + server timeouts are first-line DoS controls.
- Path jails need `filepath.Rel` / prefix checks—not only `Clean`.
- Production should `log.Fatal` on missing secrets *before* `Listen` (no soft defaults).

### Try next

- Mount pprof on `127.0.0.1:6060` only; prove it is unreachable on the public mux.
- Replace the demo token default with hard fail-closed exit when `API_TOKEN` is unset.

Next: once tooling and hardening defaults are in place, design patterns (SOLID-ish module boundaries) keep secure code maintainable — see `82-solid-design.qmd`.

## **SOLID in Go**

# SOLID Principles in Go

## Overview

SOLID comes from class-heavy OOP, but its goals—**cohesion**, **decoupling**, **testability**—map cleanly onto Go. You implement them with **packages**, **small interfaces**, and **constructor injection**, not inheritance trees.

This chapter translates each letter into idiomatic Go, with production examples, checklists, and common misapplications.

## Quick Map

Letter	OOP slogan	Go practice
<b>S</b>	Single responsibility	Small packages; one reason to change
<b>O</b>	Open/closed	Accept interfaces; add types without editing core
<b>L</b>	Liskov substitution	Behavioral contracts for interface implementers
<b>I</b>	Interface segregation	Consumer-defined, tiny interfaces
<b>D</b>	Dependency inversion	Depend on abstractions at boundaries

## S — Single Responsibility

“A package (or type) should have one reason to change.”

Go’s package system is the first SRP tool.

Avoid	Prefer
<code>package utils</code>	<code>package clock</code> , <code>package billhash</code> , <code>package useremail</code>
God Service struct	Separate Billing, Notifier, Store

```
// Bad: one type owns HTTP, SQL, and SMTP
type App struct {
 db *sql.DB
}

func (a *App) HandleRegister(w http.ResponseWriter, r *http.Request) { /* parse */ }
func (a *App) insertUser(...) error { /* sql */ }
func (a *App) sendWelcome(...) error { /* smtp */ }

// Better: each concern is a type with a narrow job
type UserStore interface {
 Create(ctx context.Context, u User) (UserID, error)
}

type Mailer interface {
 SendWelcome(ctx context.Context, email string) error
}

type RegisterHandler struct {
 Store UserStore
 Mailer Mailer
}

func (h RegisterHandler) ServeHTTP(w http.ResponseWriter, r *http.Request) {
 // parse → Store.Create → Mailer.SendWelcome
}
```

SRP does **not** mean “one function per file.” It means changes to email templates should not risk breaking SQL migrations in the same type.

## O — Open/Closed

“Open for extension, closed for modification.”

In Go, extension is usually a **new type** satisfying an interface—not subclassing.

```
type Shape interface {
 Area() float64
}

func TotalArea(shapes []Shape) float64 {
 var total float64
 for _, s := range shapes {
 total += s.Area()
 }
 return total
}
```

```

}

type Circle struct{ R float64 }

func (c Circle) Area() float64 { return math.Pi * c.R * c.R }

type Rectangle struct{ W, H float64 }

func (r Rectangle) Area() float64 { return r.W * r.H }

// Add Triangle later without editing TotalArea.

```

Another pattern: middleware stacks—wrap `http.Handler` without editing handlers.

```

func WithLogging(next http.Handler) http.Handler {
 return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
 start := time.Now()
 next.ServeHTTP(w, r)
 slog.Info("request", "path", r.URL.Path, "dur", time.Since(start))
 })
}

```

## L — Liskov Substitution

**“Implementations must honor the contract callers rely on.”**

Go has no subtypes, but interface satisfaction is a promise:

If `w` is an `io.Writer`, callers expect:

- `Write` either writes or returns an error
- No surprise panics on empty input (unless documented)
- No “always returns nil error but drops data” behavior

```

// Violates expectations of io.Writer users
type BrokenWriter struct{}

func (BrokenWriter) Write(p []byte) (int, error) {
 // Pretends success without writing
 return len(p), nil
}

```

Document behavioral contracts:

```

// Bucket stores objects.
// Delete is idempotent: deleting a missing key returns nil.

```

```

type Bucket interface {
 Put(ctx context.Context, key string, val []byte) error
 Delete(ctx context.Context, key string) error
}

```

When a mock in tests behaves differently from production (e.g., never returns `ErrNotFound`), tests lie—LSP applied to fakes.

## I — Interface Segregation

“Clients must not depend on methods they do not use.”

This is Go’s superpower: **define interfaces at the consumer**, keep them tiny (often 1–3 methods).

### Bad — fat interface

```

type UserService interface {
 Create(ctx context.Context, u User) error
 Delete(ctx context.Context, id UserID) error
 SendEmail(ctx context.Context, id UserID, body string) error
 Validate(u User) error
 ExportCSV(ctx context.Context) ([]byte, error)
}

func NotifyWelcome(s UserService, id UserID) error {
 return s.SendEmail(ctx, id, "welcome")
}

// NotifyWelcome is forced to mock Create/Delete/Export in tests

```

### Good — small consumer interface

```

type WelcomeMailer interface {
 SendEmail(ctx context.Context, id UserID, body string) error
}

func NotifyWelcome(m WelcomeMailer, id UserID) error {
 return m.SendEmail(ctx, id, "welcome")
}

```

Stdlib examples: `io.Reader`, `io.Writer`, `fs.FS`, `http.Handler`—tiny and ubiquitous.

**Accept interfaces, return structs** (common guideline):



```
func NewServer(store *PostgresStore) *Server // concrete for construction
// methods of Server depend on small interfaces internally if needed
```

Or inject interfaces at construction when tests require it:

```
func NewServer(store UserStore, mail WelcomeMailer) *Server
```

## D — Dependency Inversion

“High-level policy should not depend on low-level details; both depend on abstractions.”

```
// Bad: business logic locked to sql.DB
type OrderService struct {
 DB *sql.DB
}

func (s *OrderService) Place(ctx context.Context, o Order) error {
 _, err := s.DB.ExecContext(ctx, `INSERT INTO orders ...`)
 return err
}

// Good: policy depends on a port
type OrderRepository interface {
 Insert(ctx context.Context, o Order) error
}

type OrderService struct {
 Repo OrderRepository
}

func (s *OrderService) Place(ctx context.Context, o Order) error {
 if err := o.Validate(); err != nil {
 return err
 }
 return s.Repo.Insert(ctx, o)
}

// Detail implements the port
type PostgresOrders struct{ DB *sql.DB }

func (p PostgresOrders) Insert(ctx context.Context, o Order) error {
 _, err := p.DB.ExecContext(ctx, `INSERT INTO orders ...`, o.ID, o.Total)
 return err
}
```

Wiring stays in `main / cmd`:

```
func main() {
 db := mustOpenDB()
 svc := OrderService{Repo: PostgresOrders{DB: db}}
 // http handlers get svc
}
```

Tests inject fakes:

```
type fakeRepo struct{ last Order }

func (f *fakeRepo) Insert(ctx context.Context, o Order) error {
 f.last = o
 return nil
}

func TestPlace(t *testing.T) {
 f := &fakeRepo{}
 svc := OrderService{Repo: f}
 if err := svc.Place(context.Background(), Order{ID: "1", Total: 10}); err != nil {
 t.Fatal(err)
 }
 if f.last.ID != "1" {
 t.Fatalf("got %+v", f.last)
 }
}
```

## Composition Over Inheritance

Go has no inheritance. Use **embedding** carefully for data reuse, not for deep type hierarchies.

```
type Logger struct {
 *slog.Logger
}

// Prefer explicit fields and small interfaces over deep embedding graphs.
```

## Package Design Checklist (SOLID-flavored)

- ☐ Package name describes a domain concept, not `util / common`
- ☐ Interfaces live with consumers (or in a thin `port` layer), not giant `interfaces.go` with 20 methods
- ☐ Concrete adapters (`postgres`, `smtp`, `s3`) sit at the edges

- ☐ **main** wires dependencies
- ☐ Domain logic has no import of frameworks it does not need
- ☐ Tests use fakes via interfaces, not live network by default

## Anti-Patterns

1. **Interface on producer with 15 methods “for flexibility”** — freezes implementations; violates I.
2. **package interfaces shared by all** — creates import cycles and meaningless abstractions.
3. **Mocking everything** — if there is no behavior branch, don’t introduce an interface.
4. **SRP as infinite micro-packages** — 50 packages of 20 lines harm navigation; balance cohesion.
5. **Embedding \*sql.DB in every service** — hidden dependency; prefer explicit fields.
6. **Panicking implementers** — violates L for interfaces that expect errors.

## When *Not* to Abstract

```
// YAGNI: single use, no test seam needed yet
func loadConfig(path string) (Config, error) {
 b, err := os.ReadFile(path)
 // ...
}
```

Introduce an interface when you have a **second implementation** (or a real test need), not on day one for every function.

## Production Checklist

- ☐ New packages have a clear single purpose statement (README one-liner or doc comment)
- ☐ Public APIs prefer small interfaces at boundaries
- ☐ Handlers/services don’t import driver-specific packages when avoidable
- ☐ Fakes honor real contracts (LSP for tests)
- ☐ Dependency graph is acyclic: domain ← ports → adapters
- ☐ CI tests domain with fakes; integration tests cover one real adapter

## Common Pitfalls

1. **Java-style interface hierarchies in Go** — fighting the language.
2. **Returning interfaces from factories always** — hides types; return concrete, accept interfaces.
3. **Circular imports** — often a sign SRP/DIP layering is wrong.
4. **God App struct** — DIP violation magnet; split constructors.

5. **Copy-paste SOLID blogs with class diagrams** — rephrase in packages and interfaces.

## Exercises

1. **Split the god type** — Take a type with DB + HTTP + email methods; refactor into **Store**, **Mailer**, and handler with interfaces.
2. **Segregate** — From a 6-method interface, derive two consumer interfaces used by different functions; update tests.
3. **Open/closed middleware** — Add **WithAuth** and **WithLogging** without editing business handlers.
4. **Fake honesty** — Write a fake **Bucket** that violates idempotent delete; show a test that would pass with a bad fake and fix the fake.
5. **Wire main** — Build `cmd/server/main.go` that constructs Postgres + SMTP adapters and injects them into services.
6. **Package rename** — Break `internal/utils` into two named packages; update imports until `go test ./...` is green.

## More examples

### Depend on interfaces; swap fakes

```
mkdir -p /tmp/go-solid-iface && cd /tmp/go-solid-iface
go mod init example.com/solid-iface
```

Save as `main.go`:

```
package main

import "fmt"

type Mailer interface {
 Send(to, body string) error
}

type Notifier struct {
 mail Mailer
}

func (n Notifier) Welcome(to string) error {
 return n.mail.Send(to, "welcome")
}

type MemMailer struct {
 sent []string
}
```

```

}

func (m *MemMailer) Send(to, body string) error {
 m.sent = append(m.sent, to+": "+body)
 return nil
}

func main() {
 m := &MemMailer{}
 n := Notifier{mail: m}
 _ = n.Welcome("a@b.co")
 _ = n.Welcome("c@d.co")
 fmt.Println("sent:", len(m.sent), m.sent[0])
}

go run .

```

Expected output:

```
sent: 2 a@b.co:welcome
```

## Functional options for constructors

```

mkdir -p /tmp/go-solid-opts && cd /tmp/go-solid-opts
go mod init example.com/solid-opts

```

Save as main.go:

```

package main

import "fmt"

type Server struct {
 addr string
 tls bool
}

type Option func(*Server)

func WithAddr(a string) Option { return func(s *Server) { s.addr = a } }
func WithTLS(on bool) Option { return func(s *Server) { s.tls = on } }

func NewServer(opts ...Option) *Server {
 s := &Server{addr: ":8080"}
 for _, o := range opts {

```

```

 o(s)
 }
 return s
}

func main() {
 s := NewServer(WithAddr(":8443"), WithTLS(true))
 fmt.Printf("addr=%s tls=%v\n", s.addr, s.tls)
}

go run .

```

Expected output:

```
addr=:8443 tls=true
```

## Runnable example

SOLID in Go is small consumer interfaces and constructor injection. This program places an order through a service that depends on a `OrderRepository` port; `main` wires an in-memory adapter (swap for Postgres without changing policy).

```
mkdir -p /tmp/go-solid && cd /tmp/go-solid
go mod init example.com/solid
```

Save as `main.go`:

```

package main

import (
 "context"
 "fmt"
 "sync"
)

type Order struct {
 ID string
 Total int
}

func (o Order) Validate() error {
 if o.ID == "" || o.Total <= 0 {
 return fmt.Errorf("invalid order")
 }
 return nil
}

```

```

}

// Port: defined by the consumer (DIP + ISP).
type OrderRepository interface {
 Insert(ctx context.Context, o Order) error
}

type OrderService struct {
 Repo OrderRepository
}

func (s OrderService) Place(ctx context.Context, o Order) error {
 if err := o.Validate(); err != nil {
 return err
 }
 return s.Repo.Insert(ctx, o)
}

// Adapter: in-memory store (tests / demos).
type MemoryOrders struct {
 mu sync.Mutex
 byID map[string]Order
}

func NewMemoryOrders() *MemoryOrders {
 return &MemoryOrders{byID: make(map[string]Order)}
}

func (m *MemoryOrders) Insert(ctx context.Context, o Order) error {
 select {
 case <-ctx.Done():
 return ctx.Err()
 default:
 }
 m.mu.Lock()
 defer m.mu.Unlock()
 if _, ok := m.byID[o.ID]; ok {
 return fmt.Errorf("duplicate id %s", o.ID)
 }
 m.byID[o.ID] = o
 return nil
}

func (m *MemoryOrders) Get(id string) (Order, bool) {
 m.mu.Lock()
 defer m.mu.Unlock()

```

```

 o, ok := m.byID[id]
 return o, ok
}

func main() {
 repo := NewMemoryOrders()
 svc := OrderService{Repo: repo} // wire in main

 ctx := context.Background()
 if err := svc.Place(ctx, Order{ID: "o1", Total: 42}); err != nil {
 fmt.Println("place:", err)
 return
 }
 if err := svc.Place(ctx, Order{ID: "bad", Total: 0}); err != nil {
 fmt.Println("validate rejected:", err)
 }
 if err := svc.Place(ctx, Order{ID: "o1", Total: 10}); err != nil {
 fmt.Println("duplicate rejected:", err)
 }
 if o, ok := repo.Get("o1"); ok {
 fmt.Printf("stored: id=%s total=%d\n", o.ID, o.Total)
 }
 fmt.Println("solid wiring ok")
}

go run .

```

### Expected output:

```

validate rejected: invalid order
duplicate rejected: duplicate id o1
stored: id=o1 total=42
solid wiring ok

```

### What to notice

- Policy (`OrderService`) never imports a database driver—only the port.
- The interface is one method (ISP); tests inject fakes without mocking frameworks.
- Construction stays in `main` (or `cmd/`); packages export structs, accept interfaces.

### Try next

- Extract `OrderService` tests with a fake that records last `Order`.
- Add a `WelcomeMailer` interface and call it after successful `Insert` without growing a god service.



## Related Topics

- [Interfaces](#)
- [Interface practices](#)
- [Long-term design](#)
- [Testing fundamentals](#)

**Specialized**

## Desktop Apps

# Desktop Apps: Wails, Fyne, and Gio

## Overview

Go is a strong choice for cross-platform desktop tools: one language for business logic, solid concurrency, and easy shipping of a single binary (or a small bundle). The three main UI paths in 2026 are:

Stack	UI model	Look	Binary size (order of mag.)	Best for
<b>Wails</b>	Native WebView + HTML/JS	OS- native web engine	~10–30 MB	Dashboards, internal tools, SaaS companions
<b>Fyne</b>	Retained custom widgets (GL)	Consistent Material- ish	larger than pure CLI	Cross- platform + mobile, offline tools
<b>Gio</b>	Immediate- mode	Fully custom	small core	High-control UIs, specialized editors

This chapter compares the stacks, shows idiomatic wiring, and covers packaging and pitfalls.

## Choosing a Stack

```
Reuse web frontend (React/Svelte/Vue)? --> Wails
Want one UI on desktop + mobile? --> Fyne
Need game-like / fully custom drawing? --> Gio
Just a tray icon + menu? --> systray / small Fyne or Wails
```

**Rule of thumb:** If the product *is* a website in a window, Wails. If the product *is* a native-feeling tool without a web team, Fyne.

## Wails — Electron Alternative Without Chromium

Wails uses the **system WebView** (WebKit on macOS, WebView2 on Windows, WebKitGTK on Linux). Backend is Go; frontend is any SPA stack. No bundled Chrome → much smaller than Electron.

### Project shape

```
myapp/
 main.go # Wails entry, app struct
 app.go # Methods bound to JS
 frontend/ # Vite + React/Svelte/Vue
 wails.json
 go.mod
```

### Binding Go to JavaScript

```
package main

import (
 "context"
 "fmt"
)

// App holds application state for the UI bridge.
type App struct {
 ctx context.Context
}

func NewApp() *App { return &App{} }

func (a *App) startup(ctx context.Context) {
 a.ctx = ctx
}

// Greet is callable from the frontend after Wails generates bindings.
func (a *App) Greet(name string) string {
 if name == "" {
 name = "world"
 }
 return fmt.Sprintf("Hello %s from Go", name)
}

// SavePrefs demonstrates returning errors across the bridge.
```

```
func (a *App) SavePrefs(theme string) error {
 if theme != "light" && theme != "dark" {
 return fmt.Errorf("invalid theme %q", theme)
 }
 // persist...
 return nil
}
```

Frontend (generated bindings):

```
import { Greet, SavePrefs } from "../wailsjs/go/main/App";

const msg = await Greet("Ada");
await SavePrefs("dark");
```

## Events both ways

Go can emit events the UI listens for (background jobs, tray actions):

```
import "github.com/wailsapp/wails/v2/pkg/runtime"

func (a *App) startWatcher() {
 go func() {
 // ...
 runtime.EventsEmit(a.ctx, "job:done", map[string]any{
 "id": "42",
 "status": "ok",
 })
 }()
}

import { EventsOn } from "../wailsjs/runtime/runtime";
EventsOn("job:done", (payload) => console.log(payload));
```

## Production notes for Wails

- Keep **heavy work off the UI bind path**; return quickly or stream progress via events.
- Never put secrets in the frontend bundle; call Go methods that read OS keychain/env.
- Test pure Go packages with ordinary `go test`; treat UI as an integration layer.
- Build: `wails build` produces platform-native binaries; sign/notarize on macOS for Gatekeeper.

## Fyne — Custom Widget Toolkit

Fyne draws its own controls via OpenGL (and related backends). Same UI code can target desktop and mobile.

```
package main

import (
 "fyne.io/fyne/v2/app"
 "fyne.io/fyne/v2/container"
 "fyne.io/fyne/v2/widget"
)

func main() {
 a := app.New()
 w := a.NewWindow("Counter")

 n := 0
 label := widget.NewLabel("Count: 0")
 btn := widget.NewButton("Increment", func() {
 n++
 label.SetText("Count: " + itoa(n))
 })

 w.SetContent(container.NewVBox(label, btn))
 w.Resize(fyne.NewSize(320, 160))
 w.ShowAndRun()
}

func itoa(n int) string {
 return fmt.Sprintf("%d", n)
}
```

### Form and validation sketch

```
entry := widget.NewEntry()
entry.SetPlaceholder("email@example.com")

form := &widget.Form{
 Items: []*widget.FormItem{
 {Text: "Email", Widget: entry},
 },
 OnSubmit: func() {
 if !strings.Contains(entry.Text, "@") {
 // show dialog / error label
 return
 }
 },
}
```

```

 }
 // save...
 },
}

```

## Fyne production notes

- Asset bundling: use Fyne’s resource bundling for icons/fonts.
- Long tasks: run in goroutines; update widgets with `fyne.Do` / thread-safe APIs so you don’t touch UI from random goroutines incorrectly.
- Packaging: `fyne package` for `.app` / `.exe` / mobile artifacts.
- Look is **consistent**, not always native—set expectations with designers.

## Gio — Immediate Mode

Gio rebuilds the UI every frame from state (game-engine style). Maximum control, more code.

Conceptual loop:

```

for each frame:
 process input events
 layout widgets from current state
 draw

```

Use Gio when you are building something like a timeline editor, custom charting, or a specialized instrument UI—not for a settings form CRUD app.

## Shared Backend Pattern

Regardless of UI kit, keep domain logic UI-free:

```

package greeter

type Service struct {
 // deps: db, config, http client
}

func (s *Service) Greet(name string) (string, error) {
 if name == "" {
 return "", fmt.Errorf("name required")
 }
 return "Hello " + name, nil
}

```



```
// Wails adapter
func (a *App) Greet(name string) (string, error) {
 return a.svc.Greet(name)
}

// Fyne adapter inside button callback
text, err := svc.Greet(entry.Text)
```

Unit-test `greeter.Service` without starting a window.

## Packaging and Distribution

Platform	Notes
macOS	Codesign + notarize; hardened runtime; entitlements for network/files
Windows	Authenticode sign; WebView2 ever-green runtime for Wails
Linux	.tar.gz, .AppImage, or distro packages; WebKitGTK deps for Wails

CI sketch:

```
build desktop artifacts on tags (matrix os)
- run: wails build -platform darwin/arm64
- run: wails build -platform windows/amd64
```

Embed version the same way as CLI apps:

```
go build -ldflags="-X main.version=${VERSION}"
```

## Security Considerations

Desktop apps inherit a large attack surface: local files, IPC, and web content.

1. **Validate all UI input** in Go, not only in JS.
2. **Disable dangerous WebView features** you do not need (arbitrary file URLs, remote debugging in prod).
3. **Path traversal** — if the UI asks to “open file”, resolve and confine to a sandbox directory.
4. **Auto-update** — verify signatures (Cosign, Sparcle-like flows, or platform stores).
5. **Least privilege** — macOS entitlements and Windows manifests should not request admin by default.

```

func safeOpen(root, userPath string) (*os.File, error) {
 clean := filepath.Clean(userPath)
 full := filepath.Join(root, clean)
 if !strings.HasPrefix(full, filepath.Clean(root)+string(os.PathSeparator)) {
 return nil, fmt.Errorf("path escapes root")
 }
 return os.Open(full)
}

```

## Production Checklist

- ☐ Domain logic in plain Go packages with tests
- ☐ UI layer is a thin adapter (Wails methods / Fyne callbacks)
- ☐ Long work off the UI thread; progress via events or bindings
- ☐ Secrets never shipped in frontend assets
- ☐ Signed builds on macOS/Windows for distribution outside stores
- ☐ Version embedded; about dialog shows it
- ☐ Crash logging to a user-visible file or OS log
- ☐ Document runtime deps (WebView2, graphics drivers)

## Common Pitfalls

1. **Business logic in React only** — duplicates rules; desktop and future CLI drift.
2. **Blocking the UI goroutine** — multi-second HTTP inside a button callback freezes Fyne/Wails.
3. **Assuming Electron mental model** — Wails does not bundle Chromium; Linux WebKit-GTK missing breaks installs.
4. **CGO surprises** — Fyne/Gio may pull CGO; cross-compile carefully.
5. **No code signing** — macOS quarantines unsigned apps; users see scary dialogs.
6. **Shipping debug tooling** — open DevTools in production builds.

## Exercises

1. **Wails hello** — Scaffold a Wails app; bind a Go `Add(a, b int) int` and call it from the frontend.
2. **Fyne counter** — Build the counter window; add a Reset button and a keyboard shortcut.
3. **Extract service** — Move greet logic into an internal package; unit-test it without UI.
4. **Safe paths** — Implement `safeOpen` with table-driven tests for `../` escapes.
5. **Release metadata** — Inject `version` via `-ldflags` and show it in a Fyne label or Wails about dialog.
6. **Compare sizes** — Build a minimal Wails and minimal Fyne app; record binary/bundle sizes on your OS.

## More examples

### UI-free domain service (testable core)

Desktop shells should call pure logic. This is the Go core you unit-test without Fyne/Wails.

```
mkdir -p /tmp/go-desk-core && cd /tmp/go-desk-core
go mod init example.com/desk-core
```

Save as `main.go`:

```
package main

import (
 "fmt"
 "strings"
)

type Greeter struct {
 prefix string
}

func (g Greeter) Greet(name string) string {
 name = strings.TrimSpace(name)
 if name == "" {
 name = "world"
 }
 return g.prefix + name
}

func main() {
 g := Greeter{prefix: "Hello, "}
 fmt.Println(g.Greet("Ada"))
 fmt.Println(g.Greet(" "))
}

go run .
```

**Expected output:**

```
Hello, Ada
Hello, world
```

## Safe open under app data root

```
mkdir -p /tmp/go-desk-safe && cd /tmp/go-desk-safe
go mod init example.com/desk-safe
```

Save as main.go:

```
package main

import (
 "fmt"
 "os"
 "path/filepath"
 "strings"
)

func safeOpen(root, rel string) ([]byte, error) {
 full := filepath.Join(root, filepath.Clean("/"+rel))
 relOut, err := filepath.Rel(root, full)
 if err != nil || strings.HasPrefix(relOut, "..") {
 return nil, fmt.Errorf("denied: %s", rel)
 }
 return os.ReadFile(full)
}

func main() {
 root, _ := os.MkdirTemp("", "deskdata")
 defer os.RemoveAll(root)
 _ = os.WriteFile(filepath.Join(root, "note.txt"), []byte("secret-note"), 0o600)

 b, err := safeOpen(root, "note.txt")
 fmt.Println("ok:", string(b), err)
 _, err = safeOpen(root, "../note.txt")
 fmt.Println("escape err:", err != nil)
}

go run .
```

Expected output:

```
ok: secret-note <nil>
escape err: true
```

## Runnable example

Desktop frameworks (Wails/Fyne/Gio) are not stdlib; the durable part is **domain logic outside the UI**. This CLI-shaped program is the service layer you unit-test without a window toolkit—bind the same functions from Wails or call them from Fyne callbacks.

```
mkdir -p /tmp/go-desktop-core && cd /tmp/go-desktop-core
go mod init example.com/desktop-core
```

Save as `main.go`:

```
package main

import (
 "fmt"
 "path/filepath"
 "strings"
)

// App is what a Wails struct or Fyne model should call-no UI imports.
type App struct {
 version string
 count int
}

func NewApp(version string) *App { return &App{version: version} }

func (a *App) Greet(name string) string {
 name = strings.TrimSpace(name)
 if name == "" {
 name = "world"
 }
 return fmt.Sprintf("Hello, %s! (v%s)", name, a.version)
}

func (a *App) Add(delta int) int {
 a.count += delta
 return a.count
}

func (a *App) Reset() { a.count = 0 }

// safeOpen jails user-selected relative paths under root (file dialogs).
func safeOpen(root, userPath string) (string, error) {
 cleanRoot, err := filepath.Abs(filepath.Clean(root))
 if err != nil {
 return "", err
 }
}
```

```

 }
 joined := filepath.Join(cleanRoot, userPath)
 abs, err := filepath.Abs(joined)
 if err != nil {
 return "", err
 }
 rel, err := filepath.Rel(cleanRoot, abs)
 if err != nil || strings.HasPrefix(rel, "..") {
 return "", fmt.Errorf("path escapes root: %s", userPath)
 }
 return abs, nil
}

func main() {
 app := NewApp("1.0.0")
 fmt.Println(app.Greet("Ada"))
 fmt.Println("count:", app.Add(1))
 fmt.Println("count:", app.Add(2))
 app.Reset()
 fmt.Println("after reset:", app.Add(0))

 root := "/tmp/desktop-docs"
 ok, err := safeOpen(root, "notes/todo.txt")
 fmt.Println("safe path:", ok, err)
 _, err = safeOpen(root, "../../etc/passwd")
 fmt.Println("escape blocked:", err != nil)
}

go run .

```

**Expected output** (paths host-dependent):

```

Hello, Ada! (v1.0.0)
count: 1
count: 3
after reset: 0
safe path: /tmp/desktop-docs/notes/todo.txt <nil>
escape blocked: true

```

### What to notice

- UI toolkits bind methods; keep business rules in plain Go packages so tests do not need a display server.
- Path validation belongs in Go, not only in the frontend file picker.
- Inject version via ldflags the same way as CLI/server releases.

**Try next**

- Move `App` to `internal/app` and add table tests for `Greet` and `safeOpen`.
- Scaffold a Wails or Fyne shell that calls the same `Greet/Add` methods.

## Related Topics

- [Building and Releasing](#)
- [CI/CD & Hardening](#)
- [Application Hardening](#)

# Machine Learning & LLMs



# AI & LLMs in Go

## Overview

Python owns model **training**. Go owns **serving, orchestration, and glue**: APIs, RAG pipelines, agents, tool routers, WebSocket streams, and high-concurrency gateways in front of model runtimes.

In 2026 you rarely train foundation models yourself. You call hosted APIs or local runtimes (Ollama, vLLM, llama.cpp servers) and build reliable product software around them. Go's strengths—static binaries, `context` cancellation, excellent HTTP—map directly onto that job.

## Architecture Split

[Python / CUDA land]	[Go land]
research notebooks	API gateway
training / fine-tunes	RAG orchestration
experimental kernels	auth, quotas, billing
	tool/function calling
	workers + queues

Keep models and experiments where the ecosystem is; put the product boundary in Go.

## Calling Local Models (Ollama)

Ollama exposes an HTTP API. Many teams use **LangChainGo** ([github.com/tmc/langchaingo](https://github.com/tmc/langchaingo)) or a thin custom client.

```
package main

import (
 "context"
 "fmt"
 "log"
 "time"

 "github.com/tmc/langchaingo/llms"
 "github.com/tmc/langchaingo/llms/ollama"
)
```

```

)

func main() {
 llm, err := ollama.New(ollama.WithModel("llama3.2"))
 if err != nil {
 log.Fatal(err)
 }

 ctx, cancel := context.WithTimeout(context.Background(), 60*time.Second)
 defer cancel()

 out, err := llms.GenerateFromSinglePrompt(ctx, llm, "Explain Go channels in 3 bullets")
 if err != nil {
 log.Fatal(err)
 }
 fmt.Println(out)
}

```

## Minimal raw HTTP client

Prefer owning the client when you need strict control:

```

type genRequest struct {
 Model string `json:"model"`
 Prompt string `json:"prompt"`
 Stream bool `json:"stream"`
}

type genResponse struct {
 Response string `json:"response"`
 Done bool `json:"done"`
}

func generate(ctx context.Context, client *http.Client, base, model, prompt string) (string,
 body, _ := json.Marshal(genRequest{Model: model, Prompt: prompt, Stream: false})
 req, err := http.NewRequestWithContext(ctx, http.MethodPost, base+"/api/generate", bytes
 if err != nil {
 return "", err
 }
 req.Header.Set("Content-Type", "application/json")

 res, err := client.Do(req)
 if err != nil {
 return "", err
 }
 defer res.Body.Close()

```

```

 if res.StatusCode >= 300 {
 b, _ := io.ReadAll(res.Body)
 return "", fmt.Errorf("ollama %s: %s", res.Status, b)
 }
 var gr genResponse
 if err := json.NewDecoder(res.Body).Decode(&gr); err != nil {
 return "", err
 }
 return gr.Response, nil
}

```

Always set **timeouts** and honor **ctx** cancellation—model calls are slow and easy to pile up under load.

## Streaming Tokens to Clients

UX for chat means streaming. Bridge model SSE/NDJSON to your API:

```

func streamHandler(w http.ResponseWriter, r *http.Request) {
 flusher, ok := w.(http.Flusher)
 if !ok {
 http.Error(w, "streaming unsupported", http.StatusInternalServerError)
 return
 }
 w.Header().Set("Content-Type", "text/event-stream")
 w.Header().Set("Cache-Control", "no-cache")

 // pseudo: for each token from model stream
 tokens := []string{"Hello", " ", "from", " ", "Go"}
 for _, t := range tokens {
 select {
 case <-r.Context().Done():
 return
 default:
 }
 fmt.Fprintf(w, "data: %s\n\n", t)
 flusher.Flush()
 }
 fmt.Fprintf(w, "data: [DONE]\n\n")
 flusher.Flush()
}

```

## RAG Pipeline in Go

```
query
--> embed(query) -> []float32
--> vector search top-k chunks
--> build prompt (system + chunks + query)
--> LLM completion
--> cite sources + log token usage
```

### Embedding + top-k stub

```
type Chunk struct {
 ID string
 Text string
 // Vector stored in DB; not always loaded
}

type Scored struct {
 Chunk
 Score float32
}

func cosine(a, b []float32) float32 {
 var dot, na, nb float32
 for i := range a {
 dot += a[i] * b[i]
 na += a[i] * a[i]
 nb += b[i] * b[i]
 }
 if na == 0 || nb == 0 {
 return 0
 }
 return dot / (float32(math.Sqrt(float64(na))) * float32(math.Sqrt(float64(nb))))
}

func topK(query []float32, docs []struct {
 Chunk
 Vec []float32
}, k int) []Scored {
 scored := make([]Scored, 0, len(docs))
 for _, d := range docs {
 scored = append(scored, Scored{Chunk: d.Chunk, Score: cosine(query, d.Vec)})
 }
 sort.Slice(scored, func(i, j int) bool { return scored[i].Score > scored[j].Score })
 if k > len(scored) {

```

```

 k = len(scored)
 }
 return scored[:k]
}

```

Production retrieval usually lives in **pgvector**, Weaviate, Milvus, Qdrant—not in-process loops. Go is the language of several of those systems’ clients and control planes.

## Prompt assembly

```

func buildPrompt(system string, chunks []Scored, question string) string {
 var b strings.Builder
 b.WriteString(system)
 b.WriteString("\n\nContext:\n")
 for i, c := range chunks {
 fmt.Fprintf(&b, "[%d] (%s) %s\n", i+1, c.ID, c.Text)
 }
 b.WriteString("\nQuestion: ")
 b.WriteString(question)
 b.WriteString("\nAnswer using only the context. Cite chunk ids.\n")
 return b.String()
}

```

## Tool / Function Calling Agents

Agents are orchestration: model proposes a tool call → Go executes **allow-listed** tools → model continues.

```

type Tool interface {
 Name() string
 Description() string
 Run(ctx context.Context, args json.RawMessage) (string, error)
}

type Registry map[string]Tool

func (r Registry) Exec(ctx context.Context, name string, args json.RawMessage) (string, error) {
 t, ok := r[name]
 if !ok {
 return "", fmt.Errorf("unknown tool %q", name)
 }
 // enforce timeout per tool
 ctx, cancel := context.WithTimeout(ctx, 10*time.Second)
 defer cancel()

```

```
 return t.Run(ctx, args)
}
```

**Never** let the model choose arbitrary shell commands or unconstrained HTTP. Map names to vetted implementations only.

## SIMD and Local Vector Math (Experimental)

Go continues to invest in performance for numeric loops. Go 1.27's experimental portable `simd` package plus `simd/archsimd` (still `GOEXPERIMENT=simd`) is the supported experiment path. For production similarity at scale you still want a vector DB; for small in-process indexes or filters, careful float32 loops (and SIMD where justified) help.

```
When available in your Go version / experiment flags:
GOEXPERIMENT=simd go test ./internal/vec/...
```

Profile before micro-optimizing: retrieval latency is usually network + model, not dot products.

## Reliability Patterns

Concern	Go approach
Slow models	<code>context.WithTimeout</code> , client-side deadlines
Overload	worker pool / queue; reject with 429
Cost	token counters, per-tenant budgets
Safety	output filters, tool allow-lists, PII redaction logs
Observability	slog attrs: <code>request_id</code> , <code>model</code> , <code>tokens_in/out</code> , <code>latency_ms</code>

```
func withUsageLog(model string, tokensIn, tokensOut int, d time.Duration) {
 slog.Info("llm.completion",
 "model", model,
 "tokens_in", tokensIn,
 "tokens_out", tokensOut,
 "latency_ms", d.Milliseconds(),
)
}
```

## Hosted APIs

Same patterns against OpenAI-compatible endpoints:

```
req, _ := http.NewRequestWithContext(ctx, http.MethodPost, endpoint, bytes.NewReader(payload))
req.Header.Set("Authorization", "Bearer "+os.Getenv("OPENAI_API_KEY"))
req.Header.Set("Content-Type", "application/json")
```

- Store keys in env/secret manager, never in the repo.
- Prefer **one outbound client** with connection pooling and explicit timeouts.
- Implement retries only on idempotent GETs or with careful dedupe keys for jobs—not blindly on every chat POST.

## Production Checklist

- ☐ Timeouts on every model HTTP call
- ☐ Cancellation wired from client disconnect to upstream
- ☐ Streaming path tested under proxy buffering settings
- ☐ Tool calls allow-listed and argument-validated
- ☐ RAG sources logged for audit (chunk ids)
- ☐ Token/cost metrics per tenant
- ☐ Secrets only in env/secret store
- ☐ Rate limits at the edge
- ☐ Eval suite for prompt regressions (golden questions)

## Common Pitfalls

1. **No timeout** — hung Ollama/vLLM pins goroutines and exhausts the server.
2. **Loading whole corpora into RAM** — use a vector store; page chunk text by id.
3. **Prompt injection** — treat retrieved docs and user text as hostile; separate system instructions clearly; don't execute tools from doc content.
4. **Python for the API layer** — GIL and packaging pain for a concurrent gateway you could write in Go.
5. **Logging full prompts with secrets** — redaction policies for PII and API keys.
6. **Unbounded agent loops** — cap steps and total tokens per request.

## Exercises

1. **Ollama client** — Write `generate(ctx, prompt)` against local Ollama; add a 30s timeout test using a cancelled context.
2. **Cosine top-k** — Implement `topK` with table-driven tests (identical vectors, orthogonal, empty).
3. **SSE chat** — Stream fake tokens to `curl -N`; cancel mid-stream and confirm the handler exits.
4. **Tool registry** — Register `weather` and `time` tools; reject unknown names; unit-test argument JSON validation.

5. **RAG prompt** — Given three chunks and a question, build a prompt and assert citations instructions are present.
6. **Usage metrics** — Wrap a completion call and emit slog fields for tokens and latency; scrape logs in a test with `slog.NewJSONHandler` to a buffer.

## More examples

### SSE token stream with cancel

```
mkdir -p /tmp/go-llm-sse && cd /tmp/go-llm-sse
go mod init example.com/llm-sse
```

Save as `main.go`:

```
package main

import (
 "bufio"
 "context"
 "fmt"
 "net/http"
 "net/http/httptest"
 "strings"
 "time"
)

func main() {
 tokens := []string{"Hello", " ", "world", "!"}
 h := http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
 fl, ok := w.(http.Flusher)
 if !ok {
 http.Error(w, "no flush", 500)
 return
 }
 w.Header().Set("Content-Type", "text/event-stream")
 for _, t := range tokens {
 select {
 case <-r.Context().Done():
 return
 default:
 }
 fmt.Fprintf(w, "data: %s\n\n", t)
 fl.Flush()
 time.Sleep(5 * time.Millisecond)
 }
 })
}
```



```

 fmt.Fprintf(w, "data: [DONE]\n\n")
 })

 ts := httptest.NewServer(h)
 defer ts.Close()

 ctx, cancel := context.WithCancel(context.Background())
 defer cancel()
 req, _ := http.NewRequestWithContext(ctx, http.MethodGet, ts.URL, nil)
 resp, err := http.DefaultClient.Do(req)
 if err != nil {
 panic(err)
 }
 defer resp.Body.Close()

 var got []string
 sc := bufio.NewScanner(resp.Body)
 for sc.Scan() {
 line := sc.Text()
 if strings.HasPrefix(line, "data: ") {
 got = append(got, strings.TrimPrefix(line, "data: "))
 }
 }
 fmt.Println("tokens:", strings.Join(got, "|"))
}

go run .

```

Expected output:

```
tokens: Hello| |world|!|[DONE]
```

## Tool registry with JSON args

```

mkdir -p /tmp/go-llm-tools && cd /tmp/go-llm-tools
go mod init example.com/llm-tools

```

Save as main.go:

```

package main

import (
 "encoding/json"
 "fmt"
)

```

```

type ToolFunc func(args json.RawMessage) (string, error)

type Registry map[string]ToolFunc

func (r Registry) Call(name string, args json.RawMessage) (string, error) {
 fn, ok := r[name]
 if !ok {
 return "", fmt.Errorf("unknown tool %q", name)
 }
 return fn(args)
}

func main() {
 reg := Registry{
 "echo": func(args json.RawMessage) (string, error) {
 var in struct {
 Text string `json:"text"`
 }
 if err := json.Unmarshal(args, &in); err != nil {
 return "", err
 }
 return in.Text, nil
 },
 }
 out, err := reg.Call("echo", json.RawMessage(`{"text":"hi"}`))
 fmt.Println("echo:", out, err)
 _, err = reg.Call("weather", json.RawMessage(`{}`))
 fmt.Println("missing:", err != nil)
}

go run .

```

### Expected output:

```

echo: hi <nil>
missing: true

```

## Runnable example

LLM gateways need timeouts, tool registries, and retrieval helpers—not only HTTP clients to vendors. This program implements cosine top-k, a tool allowlist, and a timeout-bounded “completion” (stdlib mock).

```

mkdir -p /tmp/go-llm-core && cd /tmp/go-llm-core
go mod init example.com/llm-core

```

Save as `main.go`:

```
package main

import (
 "context"
 "encoding/json"
 "fmt"
 "math"
 "sort"
 "strings"
 "time"
)

type Doc struct {
 ID string
 Text string
 Vec []float64
}

func cosine(a, b []float64) float64 {
 var dot, na, nb float64
 for i := range a {
 dot += a[i] * b[i]
 na += a[i] * a[i]
 nb += b[i] * b[i]
 }
 if na == 0 || nb == 0 {
 return 0
 }
 return dot / (math.Sqrt(na) * math.Sqrt(nb))
}

func topK(query []float64, docs []Doc, k int) []Doc {
 type scored struct {
 d Doc
 s float64
 }
 var all []scored
 for _, d := range docs {
 all = append(all, scored{d, cosine(query, d.Vec)})
 }
 sort.Slice(all, func(i, j int) bool { return all[i].s > all[j].s })
 if k > len(all) {
 k = len(all)
 }
 out := make([]Doc, 0, k)
```

```

 for i := 0; i < k; i++ {
 out = append(out, all[i].d)
 }
 return out
}

type ToolFunc func(ctx context.Context, args json.RawMessage) (string, error)

type Registry struct {
 tools map[string]ToolFunc
}

func NewRegistry() *Registry {
 return &Registry{tools: map[string]ToolFunc{}}
}

func (r *Registry) Register(name string, fn ToolFunc) { r.tools[name] = fn }

func (r *Registry) Call(ctx context.Context, name string, args json.RawMessage) (string, error) {
 fn, ok := r.tools[name]
 if !ok {
 return "", fmt.Errorf("unknown tool %q", name)
 }
 return fn(ctx, args)
}

func complete(ctx context.Context, prompt string) (string, error) {
 // Stand-in for Ollama/OpenAI: honor context deadline.
 select {
 case <-ctx.Done():
 return "", ctx.Err()
 case <-time.After(20 * time.Millisecond):
 return "answer based on: " + truncate(prompt, 60), nil
 }
}

func truncate(s string, n int) string {
 if len(s) <= n {
 return s
 }
 return s[:n] + "..."
}

func main() {
 docs := []Doc{
 {ID: "1", Text: "Go concurrency uses goroutines", Vec: []float64{1, 0, 0}},
 }

```

```

 {ID: "2", Text: "Rust has ownership and borrowing", Vec: []float64{0, 1, 0}},
 {ID: "3", Text: "Go interfaces are satisfied implicitly", Vec: []float64{0.9, 0.1, 0}}
 }
 q := []float64{1, 0, 0}
 hits := topK(q, docs, 2)
 fmt.Println("top-k:")
 for _, h := range hits {
 fmt.Printf(" %s: %s\n", h.ID, h.Text)
 }

 var b strings.Builder
 b.WriteString("Use only these sources:\n")
 for _, h := range hits {
 fmt.Fprintf(&b, "- [%s] %s\n", h.ID, h.Text)
 }
 b.WriteString("Question: What does Go use for concurrency?\n")
 prompt := b.String()

 reg := NewRegistry()
 reg.Register("time_now", func(ctx context.Context, _ json.RawMessage) (string, error) {
 return time.Now().UTC().Format(time.RFC3339), nil
 })
 out, err := reg.Call(context.Background(), "time_now", nil)
 fmt.Println("tool time_now:", out, err)
 _, err = reg.Call(context.Background(), "shell_exec", nil)
 fmt.Println("unknown tool rejected:", err != nil)

 ctx, cancel := context.WithTimeout(context.Background(), 2*time.Second)
 defer cancel()
 ans, err := complete(ctx, prompt)
 fmt.Println("completion:", ans, err)

 // Timeout path
 ctx2, cancel2 := context.WithTimeout(context.Background(), time.Nanosecond)
 defer cancel2()
 time.Sleep(time.Millisecond)
 _, err = complete(ctx2, prompt)
 fmt.Println("timeout err:", err != nil)
}

go run .

```

**Expected output** (timestamps vary):

```

top-k:
 1: Go concurrency uses goroutines
 3: Go interfaces are satisfied implicitly

```

```
tool time_now: 2026-07-28T12:00:00Z <nil>
unknown tool rejected: true
completion: answer based on: Use only these sources:
- [1] Go concurrency uses... <nil>
timeout err: true
```

### What to notice

- Retrieval (top-k) and tool allowlists are pure Go; treat model I/O as an untrusted boundary with timeouts.
- Unknown tools must fail closed—never reflect arbitrary names into `os/exec`.
- Cancelled contexts prevent hung upstream model servers from pinning handlers forever.

### Try next

- Stream fake tokens to stdout with flushes (SSE-style) and cancel mid-stream.
- Add slog fields for `tokens` and `dur_ms` around `complete`.

## Related Topics

- [HTTP and Networking](#)
- [Context](#)
- [Logging and Observability](#)
- [I/O and Streaming](#)

# Systems Programming

# Systems Programming: Low-Level Go

## Overview

Go was designed to replace much of the C++/Java systems software at Google: servers, CLIs, and infrastructure agents. It still offers deliberate low-level power—raw syscalls via `golang.org/x/sys`, unsafe zero-copy tricks, cgo bridges, and userspace control planes for eBPF—while keeping a memory-safe default path.

On Linux the usual path never goes through `libc`. `fmt.Println`, `os.Open`, and `net.Listen` end in Go runtime assembly that executes the `syscall` instruction. That is why a `CGO_ENABLED=0` binary is fully static and runs on Alpine, Debian, or `scratch` without a C library. The portability story, glibc vs musl, and Go-versus-C trade-offs live in [Go as a Systems Language](#). This chapter is the next layer: syscalls you opt into, Delve, eBPF userspace, `unsafe`, and production guardrails.

## When to Drop Below the Stdlib

Need	Prefer	Reach for low-level when
Files	<code>os, io</code>	Special flags, <code>flock</code> , <code>sendfile</code>
Processes	<code>os/exec</code>	Fine-grained credentials, <code>ptrace</code>
Network	<code>net</code>	Raw sockets, <code>SO_REUSEPORT</code> tuning
Memory	GC + slices	<code>mlock</code> , huge pages, <code>mmap</code> shared regions
Tracing	<code>slog/pprof</code>	eBPF kernel probes

Most services never need `unsafe` or eBPF. Learn the tools so you recognize when they earn their complexity.

## Syscalls with `golang.org/x/sys`

The `syscall` package is frozen. New code should use `golang.org/x/sys/unix` (and `windows`).

```
package main

import (
 "log"
 "golang.org/x/sys/unix"
)
```



```

func main() {
 // Example: prevent a secret buffer from being swapped to disk (best-effort).
 secret := make([]byte, 4096)
 copy(secret, []byte("super-secret-token"))

 if err := unix.Mlock(secret); err != nil {
 log.Printf("mlock: %v (may need privileges)", err)
 }
 defer func() {
 // Zero then unlock
 for i := range secret {
 secret[i] = 0
 }
 _ = unix.Munlock(secret)
 }()

 // use secret...
 _ = secret
}

```

## File locking

```

func lockFile(fd int) error {
 return unix.Flock(fd, unix.LOCK_EX|unix.LOCK_NB)
}

```

## Raising file descriptor limits (process)

Often better done by systemd/ulimit, but diagnostics help:

```

var rLimit unix.Rlimit
if err := unix.Getrlimit(unix.RLIMIT_NOFILE, &rLimit); err != nil {
 log.Fatal(err)
}
log.Printf("nofile cur=%d max=%d", rLimit.Cur, rLimit.Max)

```

**Portability:** unix builds are OS-specific. Use build tags or high-level wrappers for multi-OS products.

## Debugging with Delve

Delve (dlv) understands goroutines, channels, and Go's runtime—unlike gdb alone.

```

go install github.com/go-delve/delve/cmd/dlv@latest

local
dlv debug ./cmd/server -- --config config.yaml

attach
dlv attach $(pgrep -n myapp)

tests
dlv test ./internal/store -- -test.run TestRace

```

## Core dumps

On a crash with core dumps enabled:

```

ulimit -c unlimited
... crash ...
dlv core ./myapp core.12345

```

Inside Delve: `bt`, `goroutines`, `frame`, `print varname`.

## Remote debugging sketch

```

on server
dlv exec ./myapp --headless --listen=:2345 --api-version=2 --accept-multiclient

local IDE / dlv connect :2345

```

**Never** leave headless Delve exposed on a public interface in production.

## eBPF Userspace in Go

eBPF programs run in the kernel (restricted VM). Go is the dominant language for **loaders and control planes** (Cilium, many observability agents) via [github.com/cilium/ebpf](https://github.com/cilium/ebpf).

```

[kernel eBPF program] <--- map read/write ---> [Go agent]
 packet filter / probe metrics, policy, UI

```

Conceptual load flow:

```

// Illustrative - real programs use bpf2go generated specs.
spec, err := ebpf.LoadCollectionSpec("bpf/prog.o")
if err != nil {

```

```

 log.Fatal(err)
 }
 coll, err := ebpf.NewCollection(spec)
 if err != nil {
 log.Fatal(err)
 }
 defer coll.Close()

 // Read a map of counters
 var key uint32 = 0
 var value uint64
 if err := coll.Maps["pkt_count"].Lookup(&key, &value); err == nil {
 log.Printf("packets=%d", value)
 }

```

Requirements: Linux, appropriate kernel version, often `CAP_BPF` / `CAP_SYS_ADMIN` or unprivileged eBPF policy depending on distro. This is **not** portable to FreeBSD/macOS the same way.

## unsafe for Zero-Copy Views

`unsafe` bypasses type and memory safety. Use only with invariants you can prove.

```

package endianx

import (
 "encoding/binary"
 "unsafe"
)

// BytesToFloat64LE interprets the first 8 bytes as a little-endian float64.
// Prefer encoding/binary in application code; this shows the unsafe cast pattern.
func BytesToFloat64LE(b []byte) float64 {
 if len(b) < 8 {
 panic("short buffer")
 }
 // Safer portable approach:
 u := binary.LittleEndian.Uint64(b[:8])
 return *(*float64)(unsafe.Pointer(&u))
}

```

Rules of thumb:

1. Document alignment and lifetime; the backing array must live as long as the view.
2. Prefer `encoding/binary`, `bytealg`, or generated code before `unsafe`.
3. Gate with build tags / architecture checks when needed.
4. Race detector and tests still matter—`unsafe` does not excuse data races.

## Slice header caution

```
// Dangerous if misused: sharing underlying array across ownership boundaries.
func noCopyString(b []byte) string {
 return unsafe.String(unsafe.SliceData(b), len(b))
}
// Only valid while b is not mutated; do not return if caller may recycle b.
```

Go 1.20+ `unsafe.String` / `unsafe.Slice` make intent clearer than raw pointer casts—still sharp tools.

## mmap for Large Read-Only Data

```
f, err := os.Open("large.bin")
if err != nil {
 return err
}
defer f.Close()

st, err := f.Stat()
if err != nil {
 return err
}
data, err := unix.Mmap(int(f.Fd()), 0, int(st.Size()),
 unix.PROT_READ, unix.MAP_SHARED)
if err != nil {
 return err
}
defer unix.Munmap(data)

// read-only use of data
_ = data[0]
```

Good for large static corpora; bad for small files where `os.ReadFile` is simpler.

## cgo Boundary (Reminder)

cgo lets you call C, but:

- Slows builds and complicates cross-compilation
- Shares the C world of undefined behavior
- Has a cost crossing the boundary under high call rates

Prefer pure Go or subprocess isolation unless a library is irreplaceable. See the dedicated cgo chapter for details.

## Production Checklist

- ☐ Prefer `stdlib` / high-level packages before `x/sys` and `unsafe`
- ☐ OS-specific code isolated behind interfaces + build tags
- ☐ Delve/core dump workflow documented for on-call
- ☐ eBPF agents least-privilege; never world-open debug ports
- ☐ Secrets: mlock where required + zeroization; still prefer hardware/HSM when critical
- ☐ Integration tests on the real target kernel/OS for syscall features
- ☐ `go test -race` on all packages that share memory

## Common Pitfalls

1. **Using frozen syscall for new code** — miss fixes and constants; use `x/sys`.
2. **Ignoring errors from Mlock / Flock** — silent degraded security or locking.
3. **eBPF on the wrong kernel** — load fails mysteriously; pin kernel versions in ops docs.
4. **unsafe aliasing bugs** — GC moves aren't the issue (pinned carefully) so much as **lifetime and mutation**.
5. **Privileged containers by default** — systems agents often over-request `privileged: true`.
6. **Core dumps with secrets** — cores can contain tokens; encrypt at rest and restrict access.

## Exercises

1. **Flock CLI** — Write a tiny program that takes a lock file path, `LOCK_EX`, prints “locked”, sleeps 10s; run two instances and show the second fails with `LOCK_NB`.
2. **Rlimit log** — Print `RLIMIT_NOFILE` at startup; document how `systemd LimitNOFILE` changes it.
3. **Delve session** — Break on a handler, inspect a request struct, continue.
4. **unsafe round-trip** — Convert `float64` → bytes → `float64` with `encoding/binary` and with `unsafe`; fuzz compare.
5. **mmap read** — `mmap` a file of `uint64` LE values and sum them; compare speed to `os.ReadFile` for a 100MB file.
6. **Build tags** — Provide `mlock_linux.go` and a no-op stub for other OSes; `GOOS=darwin go build` must succeed.

## More examples

### Length-prefix binary frame (encode/decode)

```
mkdir -p /tmp/go-sys-frame && cd /tmp/go-sys-frame
go mod init example.com/sys-frame
```

Save as `main.go`:

```

package main

import (
 "bytes"
 "encoding/binary"
 "fmt"
 "io"
)

func writeFrame(w io.Writer, payload []byte) error {
 var hdr [4]byte
 binary.BigEndian.PutUint32(hdr[:], uint32(len(payload)))
 if _, err := w.Write(hdr[:]); err != nil {
 return err
 }
 _, err := w.Write(payload)
 return err
}

func readFrame(r io.Reader) ([]byte, error) {
 var hdr [4]byte
 if _, err := io.ReadFull(r, hdr[:]); err != nil {
 return nil, err
 }
 n := binary.BigEndian.Uint32(hdr[:])
 if n > 1<<20 {
 return nil, fmt.Errorf("frame too large: %d", n)
 }
 buf := make([]byte, n)
 if _, err := io.ReadFull(r, buf); err != nil {
 return nil, err
 }
 return buf, nil
}

func main() {
 var buf bytes.Buffer
 _ = writeFrame(&buf, []byte("ping"))
 _ = writeFrame(&buf, []byte("pong-payload"))
 a, _ := readFrame(&buf)
 b, _ := readFrame(&buf)
 fmt.Println(string(a), string(b), "left", buf.Len())
}

go run .

```

**Expected output:**

```
ping pong-payload left 0
```

## Little-endian uint64 round-trip

```
mkdir -p /tmp/go-sys-le && cd /tmp/go-sys-le
go mod init example.com/sys-le
```

Save as main.go:

```
package main

import (
 "encoding/binary"
 "fmt"
)

func main() {
 var raw [8]byte
 binary.LittleEndian.PutUint64(raw[:], 0x0102030405060708)
 fmt.Printf("bytes: % x\n", raw)
 v := binary.LittleEndian.Uint64(raw[:])
 fmt.Printf("value: 0x%x\n", v)
}

go run .
```

Expected output:

```
bytes: 08 07 06 05 04 03 02 01
value: 0x102030405060708
```

## Runnable example

Systems programming often means process identity, file locks, and child processes. This program prints OS/runtime identity, uses an exclusive lock file, and runs a short child with `CommandContext`.

```
mkdir -p /tmp/go-systems && cd /tmp/go-systems
go mod init example.com/systems
```

Save as main.go:

```
package main

import (
```

```

"context"
"fmt"
"os"
"os/exec"
"path/filepath"
"runtime"
"time"
)

func main() {
 fmt.Printf("goos=%s pid=%d goroutines=%d\n", runtime.GOOS, os.Getpid(), runtime.NumGorou

 dir := filepath.Join(os.TempDir(), "go-systems-demo")
 _ = os.MkdirAll(dir, 0o750)
 lockPath := filepath.Join(dir, "app.lock")

 // Exclusive lock via O_EXCL lockfile (simple, portable pattern for single-instance tool
 lf, err := os.OpenFile(lockPath, os.O_CREATE|os.O_EXCL|os.O_WRONLY, 0o600)
 if err != nil {
 fmt.Println("lock busy or error:", err)
 } else {
 fmt.Fprintln(lf, os.Getpid())
 fmt.Println("acquired lock:", lockPath)
 defer func() {
 lf.Close()
 _ = os.Remove(lockPath)
 fmt.Println("released lock")
 }()
 }

 // Second acquire should fail while first holds O_EXCL file.
 _, err = os.OpenFile(lockPath, os.O_CREATE|os.O_EXCL|os.O_WRONLY, 0o600)
 fmt.Println("second lock failed:", err != nil)

 ctx, cancel := context.WithTimeout(context.Background(), 2*time.Second)
 defer cancel()
 cmd := exec.CommandContext(ctx, "echo", "child-hello")
 out, err := cmd.CombinedOutput()
 fmt.Printf("child: %q err=%v\n", string(out), err)

 // Cancelled child
 ctx2, cancel2 := context.WithTimeout(context.Background(), 50*time.Millisecond)
 defer cancel2()
 slow := exec.CommandContext(ctx2, "sleep", "2")
 err = slow.Run()
 fmt.Println("sleep cancelled:", err != nil)
}

```



```
 fmt.Println("systems demo ok")
}

go run .
```

**Expected output** (illustrative):

```
goos=darwin pid=12345 goroutines=1
acquired lock: /tmp/go-systems-demo/app.lock
second lock failed: true
child: "child-hello\n" err=<nil>
sleep cancelled: true
released lock
systems demo ok
```

### What to notice

- `CommandContext` kills children when the deadline hits—critical for supervisors and CI wrappers.
- Exclusive lock files (`O_EXCL`) are a portable single-instance pattern; Linux production often prefers `flock` via [golang.org/x/sys/unix](http://golang.org/x/sys/unix) (not `stdlib`).
- Always clean lock files in `defer` on graceful exit; crash may leave stale locks (document recovery).

### Try next

- On Linux, experiment with `unix.Flock` from [golang.org/x/sys](http://golang.org/x/sys) (outside pure `stdlib`).
- Print open file soft limit via platform APIs or document `ulimit -n` next to your service unit.

## Related Topics

- [Go as a Systems Language](#)
- [Unsafe](#)
- [cgo](#)
- [Go on NixOS & FreeBSD](#)
- [Application Hardening](#)

## I/O and Streaming

# I/O and Streaming

## Overview

Go's `io` package defines the small interfaces that power files, networks, compression, and crypto: `Reader`, `Writer`, `Closer`, and friends. Idiomatic streaming means **processing data in bounded buffers** instead of slurping multi-gigabyte payloads into memory.

This chapter covers the core interfaces, composition (`MultiReader`, `TeeReader`, `LimitReader`), `bufio`, pipes, practical streaming patterns, and production pitfalls.

## Core Interfaces

```
type Reader interface {
 Read(p []byte) (n int, err error)
}

type Writer interface {
 Write(p []byte) (n int, err error)
}

type Closer interface {
 Close() error
}

type ReadCloser interface {
 Reader
 Closer
}

type WriteCloser interface {
 Writer
 Closer
}
```

Contracts that matter in production:

- `Read` may return `n > 0` and `err == io.EOF` on the final chunk—process `n` before handling EOF.
- `Write` must report short writes (`n < len(p)`) as an error if not fully written (`io.ErrShortWrite` patterns).

- Always Close `ReadCloser` values you own (HTTP bodies, files).

## Reading

```
f, err := os.Open("file.txt")
if err != nil {
 return err
}
defer f.Close()

buf := make([]byte, 32*1024)
for {
 n, err := f.Read(buf)
 if n > 0 {
 // process buf[:n]
 }
 if err == io.EOF {
 break
 }
 if err != nil {
 return err
 }
}
```

Helpers:

```
data, err := io.ReadAll(f) // entire stream into memory
chunk, err := io.ReadAll(io.LimitReader(r, 1<<20)) // cap at 1 MiB
```

## HTTP bodies

```
resp, err := http.Get(url)
if err != nil {
 return err
}
defer resp.Body.Close()

const maxBody = 8 << 20 // 8 MiB
limited := io.LimitReader(resp.Body, maxBody+1)
b, err := io.ReadAll(limited)
if err != nil {
 return err
}
if len(b) > maxBody {
```

```
 return fmt.Errorf("response body exceeds %d bytes", maxBody)
}
```

## Writing

```
f, err := os.Create("output.txt")
if err != nil {
 return err
}
defer f.Close()

if _, err := f.Write([]byte("Hello")); err != nil {
 return err
}
if _, err := io.WriteString(f, " World"); err != nil {
 return err
}
return f.Sync() // durability when needed
```

## Copying Streams

```
// Efficient path: uses WriterTo / ReaderFrom when available
if _, err := io.Copy(dst, src); err != nil {
 return err
}

// Cap bytes
if _, err := io.Copy(dst, io.LimitReader(src, 1024)); err != nil {
 return err
}

// Fixed buffer size (Go 1.15+ has CopyBuffer)
buf := make([]byte, 64*1024)
if _, err := io.CopyBuffer(dst, src, buf); err != nil {
 return err
}
```

`io.Copy` is the default for proxying HTTP, piping files, and unpacking archives—prefer it over manual loops unless you need progress hooks.

## Progress reader

```
type countingReader struct {
 r io.Reader
 n int64
}

func (c *countingReader) Read(p []byte) (int, error) {
 n, err := c.r.Read(p)
 c.n += int64(n)
 return n, err
}
```

## Composition Helpers

Helper	Role
io.MultiReader	Concatenate readers sequentially
io.MultiWriter	Fan-out writes to multiple writers
io.TeeReader	Read while copying to a side writer (checksums, logs)
io.LimitReader	Hard cap bytes read
io.SectionReader	Window into an <code>ReaderAt</code>
io.Pipe	In-memory streaming between goroutines

```
// Checksum while uploading
h := sha256.New()
tr := io.TeeReader(file, h)
if _, err := io.Copy(dest, tr); err != nil {
 return err
}
sum := h.Sum(nil)

// Concatenate header + body
r := io.MultiReader(strings.NewReader("HDR\n"), body)
```

## bufio

Buffering cuts syscalls for small reads/writes.

```
f, err := os.Open("access.log")
if err != nil {
```

```

 return err
}
defer f.Close()

sc := bufio.NewScanner(f)
// Raise token size for long lines (default ~64K)
sc.Buffer(make([]byte, 0, 64*1024), 1024*1024)

for sc.Scan() {
 line := sc.Text()
 _ = line
}
return sc.Err()

bw := bufio.NewWriterSize(f, 64*1024)
if _, err := bw.WriteString("row\n"); err != nil {
 return err
}
return bw.Flush() // never forget Flush

```

## bytes.Buffer and strings.Reader

```

var buf bytes.Buffer
buf.WriteString("Hello")
buf.WriteByte(' ')
buf.Write([]byte("World"))
out := buf.String()

r := strings.NewReader("stream me")
io.Copy(os.Stdout, r)

```

Use `bytes.Buffer` for building in memory; switch to streaming files when size is unbounded.

## Pipes Between Goroutines

```

pr, pw := io.Pipe()

go func() {
 defer pw.Close()
 // If Write fails, CloseWithError
 if _, err := io.WriteString(pw, "payload"); err != nil {
 pw.CloseWithError(err)
 }
}()

```

```

 return
 }
}()

data, err := io.ReadAll(pr)
if err != nil {
 return err
}
_ = data

```

Pipe blocks when the buffer is full—natural **backpressure**. Always close the writer side or readers hang forever.

## Streaming HTTP Upload / Download

```

func upload(ctx context.Context, client *http.Client, url string, r io.Reader) error {
 req, err := http.NewRequestWithContext(ctx, http.MethodPut, url, r)
 if err != nil {
 return err
 }
 // If size known:
 // req.ContentLength = size
 res, err := client.Do(req)
 if err != nil {
 return err
 }
 defer res.Body.Close()
 if res.StatusCode >= 300 {
 b, _ := io.ReadAll(io.LimitReader(res.Body, 4096))
 return fmt.Errorf("upload: %s: %s", res.Status, b)
 }
 return nil
}

```

## gzip Stream

```

func compress(dst io.Writer, src io.Reader) error {
 zw := gzip.NewWriter(dst)
 if _, err := io.Copy(zw, src); err != nil {
 _ = zw.Close()
 return err
 }
 return zw.Close()
}

```



```

}

func decompress(dst io.Writer, src io.Reader) error {
 zr, err := gzip.NewReader(src)
 if err != nil {
 return err
 }
 defer zr.Close()
 _, err = io.Copy(dst, zr)
 return err
}

```

## Checklist

- ☐ Bound all untrusted readers with `LimitReader` or max bytes
- ☐ Close every `ReadCloser` you own; check `Close` errors on writers when data integrity matters
- ☐ Prefer `io.Copy` / `CopyBuffer` over ad-hoc loops
- ☐ Flush buffered writers before close/return
- ☐ Pipe writer closed on both success and error paths
- ☐ Scanner buffer sized for your longest valid line
- ☐ No `ReadAll` on unbounded network input
- ☐ Use `context` cancellation on HTTP and long copies (via `req.Context()` / wrappers)

## Common Pitfalls

1. **Ignoring `n` on EOF** — last bytes dropped.
2. **Forgetting Flush** — data stuck in `bufio.Writer`.
3. **Scanner token too long** — silent `bufio.ErrTooLong`; raise buffer max.
4. **Unclosed HTTP bodies** — connection pool stalls.
5. **Deadlocked pipes** — writer blocked because reader never reads; always schedule both sides.
6. **`ioutil` legacy** — use `io` and `os` (`ioutil` deprecated).
7. **Assuming Write is atomic for large buffers** — loop or use helpers that handle short writes.

## Exercises

1. **Chunked SHA-256** — Stream a file with 32KiB buffers; print hex digest; compare to `sha256sum`.
2. **LimitReader gate** — Reject bodies over 1MiB with a clear error; test with `strings.NewReader` of known sizes.
3. **TeeReader** — Copy a file to another path while computing CRC32C (or SHA-256) in one pass.

4. **Line scanner** — Parse a 10MB log with long lines; set buffer max; count lines matching a prefix.
5. **Pipe pipeline** — Goroutine A writes numbers; goroutine B filters even lines; main reads result.
6. **gzip round-trip** — Compress and decompress a random 1MB payload; require byte-identical output.

## More examples

### TeeReader: copy + checksum one pass

```
mkdir -p /tmp/go-io-tee && cd /tmp/go-io-tee
go mod init example.com/io-tee
```

Save as `main.go`:

```
package main

import (
 "crypto/sha256"
 "encoding/hex"
 "fmt"
 "io"
 "os"
 "strings"
)

func main() {
 src := strings.NewReader("stream-me-please")
 h := sha256.New()
 dst, _ := os.CreateTemp("", "out")
 defer os.Remove(dst.Name())
 defer dst.Close()

 n, err := io.Copy(io.MultiWriter(dst, h), src)
 if err != nil {
 panic(err)
 }
 fmt.Println("bytes:", n)
 fmt.Println("sha256:", hex.EncodeToString(h.Sum(nil))[:16]+"...")
}

go run .
```

Expected output:

bytes: 15  
sha256: <16 hex chars>...

## Line scanner with buffer cap

```
mkdir -p /tmp/go-io-scan && cd /tmp/go-io-scan
go mod init example.com/io-scan
```

Save as main.go:

```
package main

import (
 "bufio"
 "fmt"
 "strings"
)

func main() {
 in := strings.NewReader("alpha\nbeta\ngamma-long-line\n")
 sc := bufio.NewScanner(in)
 sc.Buffer(make([]byte, 64), 64)
 var n int
 for sc.Scan() {
 if strings.HasPrefix(sc.Text(), "g") {
 n++
 }
 }
 fmt.Println("g-lines:", n, "err:", sc.Err())
}

go run .
```

Expected output:

g-lines: 1 err: <nil>

## Runnable example

Streaming IO is chunked reads, limited readers, tee, and compressors—never `ReadAll` on unbounded sources. This program hashes while copying, enforces a size cap, and round-trips gzip.

```
mkdir -p /tmp/go-io-stream && cd /tmp/go-io-stream
go mod init example.com/io-stream
```

Save as `main.go`:

```
package main

import (
 "bytes"
 "compress/gzip"
 "crypto/sha256"
 "encoding/hex"
 "fmt"
 "io"
 "os"
 "path/filepath"
 "strings"
)

func main() {
 dir := filepath.Join(os.TempDir(), "go-io-stream")
 _ = os.MkdirAll(dir, 0o750)
 srcPath := filepath.Join(dir, "src.bin")
 dstPath := filepath.Join(dir, "dst.bin")

 payload := bytes.Repeat([]byte("stream-me-"), 1000) // 10_000 bytes
 if err := os.WriteFile(srcPath, payload, 0o640); err != nil {
 panic(err)
 }

 // Chunked SHA-256 + copy (one pass with TeeReader).
 in, err := os.Open(srcPath)
 if err != nil {
 panic(err)
 }
 defer in.Close()
 out, err := os.Create(dstPath)
 if err != nil {
 panic(err)
 }
 h := sha256.New()
 n, err := io.Copy(out, io.TeeReader(in, h))
 out.Close()
 if err != nil {
 panic(err)
 }
 fmt.Println("copied_bytes:", n)
 fmt.Println("sha256:", hex.EncodeToString(h.Sum(nil)))

 // LimitReader gate
}
```

```

 limited := io.LimitReader(strings.NewReader(strings.Repeat("x", 100)), 16)
 buf, _ := io.ReadAll(limited)
 fmt.Println("limited_len:", len(buf))

 // Reject oversize: read with Max-style check
 const max = 1024
 r := strings.NewReader(strings.Repeat("y", 2000))
 data, err := io.ReadAll(io.LimitReader(r, max+1))
 if err != nil {
 panic(err)
 }
 if len(data) > max {
 fmt.Println("oversize: rejected", len(data))
 }

 // gzip round-trip
 var zbuf bytes.Buffer
 zw := gzip.NewWriter(&zbuf)
 _, _ = zw.Write(payload)
 _ = zw.Close()
 zr, err := gzip.NewReader(&zbuf)
 if err != nil {
 panic(err)
 }
 round, err := io.ReadAll(zr)
 _ = zr.Close()
 if err != nil {
 panic(err)
 }
 fmt.Println("gzip_roundtrip_ok:", bytes.Equal(round, payload))
 fmt.Println("gzip_ratio:", len(zbuf.Bytes()), "/", len(payload))
}

go run .

```

**Expected output** (hash stable for this payload):

```

copied_bytes: 10000
sha256: <64 hex chars>
limited_len: 16
oversize: rejected 1025
gzip_roundtrip_ok: true
gzip_ratio: <compressed> / 10000

```

### What to notice

- `io.TeeReader` hashes and copies without buffering the whole file twice.

- `io.LimitReader` is the building block for body caps; check length after read or use `http.MaxBytesReader` on servers.
- Always `Close` gzip writers/readers or trailers/checksums can be wrong.

### Try next

- Stream a file with a 32KiB buffer loop (Read into a pooled slice) and compare to `io.Copy`.
- Build a pipe: goroutine writes lines, main `bufio.Scanner` counts them.

## Related Topics

- [Encoding and Serialization](#)
- [HTTP and Networking](#)
- [Filesystem automation](#)

## Logging and Observability

# Logging and Observability

## Overview

Production systems need three pillars: **logs** (event narrative), **metrics** (aggregates), and **traces** (request journeys). Go's standard library covers solid baselines—**log/slog** for structured logs, **expvar** for simple metrics, **runtime/trace** and **net/http/pprof** for diagnostics—and interoperates with OpenTelemetry and Prometheus when you outgrow them.

This chapter focuses on idiomatic **slog**, context correlation, metrics exposure, and practical operational patterns.

## The Three Pillars

```
request
|-- trace id / span id -----> distributed tracer
|-- counters / histograms -----> metrics backend
+-- structured log lines -----> log shipper
```

Correlate them with a **request ID** (and trace IDs when tracing is enabled).

## Standard log Package

Still fine for tiny tools:

```
import "log"

log.Println("starting")
log.Printf("user %d logged in", userID)
// log.Fatal / log.Panic for process exit / panic helpers
```

For services, prefer **log/slog**.



## Structured Logging with slog

```
package main

import (
 "log/slog"
 "os"
)

func main() {
 handler := slog.NewJSONHandler(os.Stdout, &slog.HandlerOptions{
 Level: slog.LevelInfo,
 })
 slog.SetDefault(slog.New(handler))

 slog.Info("server starting", "addr", ":8080")
 slog.Error("db ping failed", "err", err)
}
```

### Levels

Level	Use
Debug	High-volume diagnostics (off in prod by default)
Info	Normal lifecycle and business events
Warn	Recoverable anomalies
Error	Failures needing attention

```
slog.Debug("cache lookup", "key", key, "hit", hit)
slog.Warn("retrying upstream", "attempt", n)
slog.Error("checkout failed", "order_id", id, "err", err)
```

### Child loggers and attributes

```
logger := slog.With("service", "checkout", "version", version)
logger.Info("ready")

reqLog := logger.With("request_id", reqID, "path", r.URL.Path)
reqLog.Info("request started")
```

## Context-aware logging

Propagate IDs via `context.Context`:

```
type ctxKey struct{}

func WithRequestID(ctx context.Context, id string) context.Context {
 return context.WithValue(ctx, ctxKey{}, id)
}

func RequestID(ctx context.Context) string {
 if v, ok := ctx.Value(ctxKey{}).(string); ok {
 return v
 }
 return ""
}

func LogInfo(ctx context.Context, msg string, args ...any) {
 if id := RequestID(ctx); id != "" {
 args = append(args, "request_id", id)
 }
 slog.Info(msg, args...)
}
```

HTTP middleware sketch:

```
func withRequestID(next http.Handler) http.Handler {
 return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
 id := r.Header.Get("X-Request-ID")
 if id == "" {
 id = uuidNew() // your id generator
 }
 w.Header().Set("X-Request-ID", id)
 ctx := WithRequestID(r.Context(), id)
 next.ServeHTTP(w, r.WithContext(ctx))
 })
}
```

## Custom log.Logger (Legacy / Libraries)

```
logger := log.New(os.Stderr, "API ", log.Ldate|log.Ltime|log.LUTC|log.Lshortfile)
logger.Println("listening")
```

Prefer slog for new code; wrap slog if a dependency wants `*log.Logger` (`slog.NewLogLogger`).

## Metrics with expvar

Quick and dependency-free:

```
import (
 "expvar"
 "net/http"
)

var (
 requests = expvar.NewInt("requests_total")
 errors = expvar.NewInt("errors_total")
 inflight = expvar.NewInt("inflight")
)

func instrument(next http.Handler) http.Handler {
 return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
 requests.Add(1)
 inflight.Add(1)
 defer inflight.Add(-1)
 next.ServeHTTP(w, r)
 })
}

func main() {
 mux := http.NewServeMux()
 mux.Handle("/debug/vars", expvar.Handler())
 // protect /debug/* in production
 http.ListenAndServe(":8080", instrument(mux))
}
```

For Prometheus histograms/labels, use `prometheus/client_golang` and a `/metrics` endpoint—same discipline: **auth or private listener**.

## Runtime Tracing and pprof

```
import (
 "os"
 "runtime/trace"
)

f, err := os.Create("trace.out")
if err != nil {
 log.Fatal(err)
}
```

```

defer f.Close()
if err := trace.Start(f); err != nil {
 log.Fatal(err)
}
defer trace.Stop()
// run workload

```

```

go tool trace trace.out

```

HTTP pprof (import for side effects):

```

import _ "net/http/pprof"

// Serve on internal addr only:
go func() {
 log.Println(http.ListenAndServe("127.0.0.1:6060", nil))
}()

go tool pprof http://127.0.0.1:6060/debug/pprof/profile?seconds=30

```

## Error Logging Hygiene

```

// Good: err is an attribute, message is stable for grouping
slog.Error("update user", "user_id", id, "err", err)

// Bad: free-form strings that explode cardinality
slog.Error(fmt.Sprintf("update user %s failed: %v", id, err))

```

Wrap errors with context for stacks of cause (`fmt.Errorf("update user: %w", err)`) and log once at the boundary—avoid logging-and-returning at every layer (duplicate noise).

## Sampling and Cardinality

High-traffic debug logs can cost more than the product:

- Gate debug behind level config (`LOG_LEVEL=info`).
- Do not put unbounded IDs into **metric labels** (user ids, URLs).
- Logs can carry high-cardinality fields; metrics should not.

```

level := slog.LevelInfo
if os.Getenv("LOG_LEVEL") == "debug" {
 level = slog.LevelDebug
}

```

## OpenTelemetry (When You Need It)

For multi-service traces, adopt OTel exporters rather than inventing headers:

```
incoming HTTP
--> extract trace context
--> start span
--> child client span for outbound HTTP/DB
--> inject headers
--> end span
```

Keep slog as the human log stream; attach `trace_id` as a slog attr when a span is present.

## Production Checklist

- ☐ JSON logs in production (machine-parseable)
- ☐ `request_id` on every request log line
- ☐ Log level configurable via env
- ☐ Errors logged with `err` attribute once at boundary
- ☐ `/debug/pprof` and `/debug/vars` not public
- ☐ Metrics for rate, errors, latency (RED) on critical paths
- ☐ Timeouts logged with enough context to act
- ☐ PII redaction policy for logs
- ☐ Trace sampling strategy documented

## Common Pitfalls

1. **Unstructured printf logs** — hard to query in any backend.
2. **Logging secrets** — tokens in Authorization headers, raw card data.
3. **Metric label explosion** — `path` with raw IDs → millions of series.
4. **Public pprof** — remote attackers profile your heap.
5. **Log-and-return everywhere** — 5 identical errors per failure.
6. **Synchronous remote logging in request path** — prefer local shipper / async agent.
7. **Forgetting Close on trace files** — empty or truncated traces.

## Exercises

1. **JSON slog** — Configure JSON handler; log `Info` with three attributes; pretty-print with `jq`.
2. **Request middleware** — Inject `X-Request-ID`; assert it appears in a log buffer during a handler test.
3. **expvar counter** — Increment on each hit to `/`; fetch `/debug/vars` and parse JSON in a test.
4. **Error boundary** — Build a small stack of functions that wrap `%w` errors; log only in `main`.
5. **pprof bind** — Serve pprof on `127.0.0.1` only; confirm LAN clients cannot connect.
6. **Level gate** — Read `LOG_LEVEL` and verify Debug lines disappear when set to `info`.

## More examples

### slog JSON to a buffer (testable logs)

```
mkdir -p /tmp/go-slog-buf && cd /tmp/go-slog-buf
go mod init example.com/slog-buf
```

Save as main.go:

```
package main

import (
 "bytes"
 "fmt"
 "log/slog"
 "strings"
)

func main() {
 var buf bytes.Buffer
 log := slog.New(slog.NewJSONHandler(&buf, &slog.HandlerOptions{Level: slog.LevelInfo}))
 log.Debug("hidden")
 log.Info("request", "method", "GET", "path", "/x", "dur_ms", 12)
 out := strings.TrimSpace(buf.String())
 fmt.Println(out)
 fmt.Println("has debug:", strings.Contains(out, "hidden"))
}

go run .
```

#### Expected output:

```
{"time":"...","level":"INFO","msg":"request","method":"GET","path":"/x","dur_ms":12}
has debug: false
```

### expvar counter + debug vars snapshot

```
mkdir -p /tmp/go-expvar && cd /tmp/go-expvar
go mod init example.com/expvar
```

Save as main.go:

```
package main

import (
```

```

 "expvar"
 "fmt"
 "io"
 "net/http"
 "net/http/httptest"
 "strings"
)

func main() {
 hits := expvar.NewInt("hits")
 mux := http.NewServeMux()
 mux.HandleFunc("GET /", func(w http.ResponseWriter, _ *http.Request) {
 hits.Add(1)
 fmt.Fprintln(w, "ok")
 })
 mux.Handle("GET /debug/vars", expvar.Handler())

 ts := httptest.NewServer(mux)
 defer ts.Close()

 for i := 0; i < 3; i++ {
 r, _ := http.Get(ts.URL + "/")
 r.Body.Close()
 }
 r, _ := http.Get(ts.URL + "/debug/vars")
 defer r.Body.Close()
 body, _ := io.ReadAll(r.Body)
 fmt.Println("hits var:", hits.Value())
 fmt.Println("vars contains hits:", strings.Contains(string(body), "hits"))
}

go run .

```

### Expected output:

```

hits var: 3
vars contains hits: true

```

## Runnable example

Production observability starts with `log/slog` JSON, request IDs, and simple counters. This program logs structured events, correlates a request id, and prints a tiny text exposition counter (Prometheus-shaped, no client library).

```
mkdir -p /tmp/go-observ && cd /tmp/go-observ
go mod init example.com/observ
```

Save as `main.go`:

```
package main

import (
 "bytes"
 "context"
 "fmt"
 "log/slog"
 "net/http"
 "net/http/httpptest"
 "sync/atomic"
 "time"
)

type ctxKey string

const requestIDKey ctxKey = "request_id"

func withRequestID(ctx context.Context, id string) context.Context {
 return context.WithValue(ctx, requestIDKey, id)
}

func requestID(ctx context.Context) string {
 if v, ok := ctx.Value(requestIDKey).(string); ok {
 return v
 }
 return ""
}

func main() {
 var buf bytes.Buffer
 log := slog.New(slog.NewJSONHandler(&buf, &slog.HandlerOptions{Level: slog.LevelInfo}))

 var hits atomic.Int64

 mux := http.NewServeMux()
 mux.HandleFunc("GET /work", func(w http.ResponseWriter, r *http.Request) {
 id := r.Header.Get("X-Request-ID")
 if id == "" {
 id = fmt.Sprintf("req-%d", time.Now().UnixNano())
 }
 ctx := withRequestID(r.Context(), id)
 start := time.Now()
 })
}
```



```

 hits.Add(1)

 log.Info("handle",
 "msg_event", "work",
 "request_id", requestID(ctx),
 "path", r.URL.Path,
 "dur_ms", time.Since(start).Milliseconds(),
)
 w.Header().Set("X-Request-ID", id)
 fmt.Fprintln(w, "ok")
 })
 mux.HandleFunc("GET /metrics", func(w http.ResponseWriter, _ *http.Request) {
 // Manual Prometheus text exposition (stdlib only).
 fmt.Fprintf(w, "# HELP work_hits_total Demo counter\n")
 fmt.Fprintf(w, "# TYPE work_hits_total counter\n")
 fmt.Fprintf(w, "work_hits_total %d\n", hits.Load())
 })

 // Drive with httptest (no real listen needed).
 for i := 0; i < 3; i++ {
 req := httptest.NewRequest(http.MethodGet, "/work", nil)
 req.Header.Set("X-Request-ID", fmt.Sprintf("demo-%d", i+1))
 rr := httptest.NewRecorder()
 mux.ServeHTTP(rr, req)
 fmt.Println("status:", rr.Code, "id:", rr.Header().Get("X-Request-ID"))
 }
 rr := httptest.NewRecorder()
 mux.ServeHTTP(rr, httptest.NewRequest(http.MethodGet, "/metrics", nil))
 fmt.Println("--- metrics ---")
 fmt.Print(rr.Body.String())
 fmt.Println("--- logs ---")
 fmt.Print(buf.String())
}

go run .

```

**Expected output** (illustrative):

```

status: 200 id: demo-1
status: 200 id: demo-2
status: 200 id: demo-3
--- metrics ---
HELP work_hits_total Demo counter
TYPE work_hits_total counter
work_hits_total 3
--- logs ---
{"time":"...", "level":"INFO", "msg":"handle", "msg_event":"work", "request_id":"demo-1", ...}

```

...

### What to notice

- JSON logs are queryable (jq); keep stable field names (`request_id`, `dur_ms`).
- Counters need `_total` and type comments if you want Prometheus-compatible scrapes.
- Correlation ids must ride on context and outbound headers—not only access logs.

### Try next

- Redact an `Authorization` header before logging.
- Start a real listener and `curl /metrics`; open pprof on localhost only.

## Related Topics

- [Structured logging \(deep dive\)](#)
- [Metrics with Prometheus](#)
- [Tracing](#)
- [Performance / pprof](#)

# Network Systems

# Network Systems Overview

# Network Systems Overview

This part treats networking in Go as a **systems** discipline: connections, budgets, failure domains, and operability — not merely how to call `http.Get`.

If earlier chapters taught you the standard library surface, this part teaches how network software behaves under load, packet loss, slow clients, and partial dependency failure.

## Why This Part Exists

Most production outages involving Go services are not “Go is slow.” They are:

- Unbounded concurrency on accept/read paths
- Missing or inconsistent timeouts
- Retry storms that amplify a partial outage
- Proxies that hide backend health
- Protocol bugs (especially custom TCP framing)

You need a mental model that spans the stack:

```
client process
 |
 v
TCP/TLS session ---- deadlines, keepalive, congestion
 |
 v
application protocol ---- framing / HTTP semantics
 |
 v
service policy ---- auth, limits, routing, retries
 |
 v
upstream dependency ---- its own budgets and failures
```

## Learning Path

Chapter	Focus	You will be able to
131 TCP services	Byte streams, framing, conn lifecycle	Build a correct custom TCP protocol handler
132 HTTP resilience	Timeout budgets, client/server guardrails	Design end-to-end latency budgets without cascade
133 Reverse proxy	Routing, upstream selection, blast radius	Place reliability policy at the edge
134 TLS clients/servers	<code>crypto/tls</code> , mTLS sketch	Serve and dial HTTPS safely
135 DNS patterns	Resolver, timeouts, dual-stack	Debug name→dial failures
136 HTTP Transport	Pools, limits, keep-alives	Tune high fan-out clients
137 Project: netcheck	dial/dns/tls/http toolkit	Ship a multi-check ops binary

## Core Concepts Map

### 1. Streams vs messages

TCP delivers an ordered byte stream. Message boundaries are your problem. HTTP is a message protocol layered on TCP (and now HTTP/2/3 multiplex differently). Conflating stream and message is the root of many TCP bugs.

### 2. Deadlines are load shedding

Every `Read/Write` without a deadline is a potential goroutine leak under slowloris-style behavior. Deadlines are not only “correctness”; they are capacity protection.

### 3. Budgets flow downward

An inbound request with a 1.2s budget cannot safely give every dependency a fresh 1.2s. Propagate remaining time via `context.Context` and refuse work you cannot finish.

```
total budget: 1200ms
 auth 50ms
 cache 100ms
 db 400ms
 upstream 500ms (must use remaining, not full 1200)
 encode 50ms
```

```
slack 100ms
```

## 4. Proxies are control planes

A reverse proxy is not “just nginx in front.” In Go (or any edge), it is where you centralize routing, timeouts, identity headers, and failure isolation so every backend need not reimplement policy.

## Go-Centric Building Blocks

You will use these packages heavily:

Package	Role
<code>net</code>	Listeners, dialers, TCP conn deadlines
<code>net/http</code>	Servers, clients, transports, reverse proxy helpers
<code>context</code>	Cancellation and deadline propagation
<code>io / bufio</code>	Framing, limited readers, efficient IO
<code>sync / golang.org/x/sync/errgroup</code>	Bounded fan-out, lifecycle
<code>crypto/tls</code>	Secure transport (deep dive in part 17)
<code>log/slog</code>	Structured operational logs

### Idiom: always bound the server

```
srv := &http.Server{
 Addr: ":8080",
 Handler: mux,
 ReadHeaderTimeout: 5 * time.Second,
 ReadTimeout: 15 * time.Second,
 WriteTimeout: 30 * time.Second,
 IdleTimeout: 60 * time.Second,
}
```

### Idiom: always bound the client

```
client := &http.Client{
 Timeout: 10 * time.Second,
 Transport: &http.Transport{
 Proxy: http.ProxyFromEnvironment,
 DialContext: (&net.Dialer{Timeout: 3 * time.Second}).DialContext,
 MaxIdleConns: 100,
 IdleConnTimeout: 90 * time.Second,
 TLSHandshakeTimeout: 5 * time.Second,
 },
}
```

```
 ExpectContinueTimeout: 1 * time.Second,
 ForceAttemptHTTP2: true,
},
}
```

Defaults without timeouts are the most common production footgun in Go HTTP code.

## Production Mindset

Before writing a network feature, answer:

1. **What is the unit of work?** (TCP frame, HTTP request, gRPC RPC)
2. **What is the timeout budget?** (client SLA → service → dependencies)
3. **What fails?** (timeout, cancel, protocol error, overload)
4. **What do we do on failure?** (fail fast, retry once, degrade, shed load)
5. **What do we observe?** (accepts, active conns, latency histograms, error classes)

If you cannot answer these, the implementation will invent answers under fire — usually the wrong ones.

## Relationship to Other Parts

- **Part 7 (concurrency)** — worker pools, context, pipelines feed network handler design.
- **Part 8 (web)** — higher-level HTTP/API frameworks sit on the resilience patterns here.
- **Part 15 (distributed infra)** — retries, circuit breakers, and queues build on timeout budgets.
- **Part 16 (observability)** — correlation and metrics make network failures diagnosable.
- **Part 17 (security)** — TLS/mTLS protect the same sockets you harden for availability.

## How to Study These Chapters

1. Read 131 with a scratch TCP echo/framed server; break it deliberately (partial reads, huge frames).
2. Read 132 while instrumenting a real `http.Server` and `http.Client` under `hey` or `vegeta`.
3. Read 133 by building a tiny reverse proxy with health-aware round-robin and watching failover.

Prefer one solid lab over ten passive reads. Network bugs only become intuitive when you have watched a stuck `Read` hold a goroutine for minutes.



## Checklist: Network Systems Literacy

After this part you should confidently:

- ☐ Explain why TCP needs framing
- ☐ Set and reason about deadlines on both ends of a connection
- ☐ Draw a timeout budget for a multi-dependency request
- ☐ Configure `http.Server` and `http.Transport` for production
- ☐ Classify retryable vs non-retryable HTTP failures
- ☐ Describe how a reverse proxy reduces blast radius
- ☐ Name the metrics you would export for a TCP and HTTP service

## Exercises (Part Warm-Up)

1. List every network listener in a service you know; for each, write down timeouts or mark “missing.”
2. Capture a pcap or use `ss/netstat` during a local load test; correlate ESTABLISHED count with goroutine count.
3. Sketch the failure modes of “client timeout 30s, server no timeout, dependency timeout 60s.”
4. Write a one-page design note: where should retries live (client, proxy, both)? Defend one answer.
5. Identify whether your public API needs custom TCP or whether HTTP/gRPC already provides your framing.

## Failure Taxonomy (Shared Vocabulary)

Use the same words across the next three chapters so metrics and runbooks align:

Class	Examples	Typical response
timeout	deadline exceeded, slowloris	fail fast / shed; fix budgets
cancel	client gone, parent cancel	stop work; do not retry
protocol	bad frame, HTTP 400	close or 4xx; no retry
overload	semaphore full, 503	backpressure; Retry-After
dep_down	connect refused, 502/503	retry with policy; breaker
dep_slow	high latency within budget	optional degrade; not always retry

When an incident starts, classify the failure before changing code. Many “network is broken” tickets are mis-set budgets.

## Lab Setup Recommendation

A single laptop is enough for this part:

```
Terminal A: your experimental server
go run ./cmd/labserve

Terminal B: load
hey -n 5000 -c 50 http://127.0.0.1:8080/api

Terminal C: profiles (admin port)
go tool pprof -http=:18081 http://127.0.0.1:6060/debug/pprof/profile?seconds=15
```

Capture goroutine counts (`/debug/pprof/goroutine?debug=1`) before and after introducing missing timeouts — the difference is the lesson.

## Anti-Patterns to Unlearn

1. **`http.DefaultClient` in libraries** — no timeout, shared by the whole process.
2. **`go handle(conn)` without a bound** — connection floods become memory floods.
3. **Retries at every layer** — browser  $\times$  edge  $\times$  app  $\times$  client SDK.
4. **Health checks that only dial TCP** — process up, app not ready.
5. **Logging full request bodies** on the edge — cost and PII.

## Glossary

- **Deadline** — absolute time when an IO op must fail.
- **Timeout budget** — remaining time for a request tree.
- **Framing** — how messages are delimited on a byte stream.
- **Keep-alive** — reusing connections; needs idle limits.
- **Load shedding** — refusing work to protect capacity.
- **Blast radius** — how far a failure propagates.

## More examples

### Tour: dial timeout + httptest client budget

One mini-program that touches the part's themes: bounded dial, HTTP round-trip with context, and fail-fast on deadline.

```
mkdir -p /tmp/go-net-tour && cd /tmp/go-net-tour
go mod init example.com/net-tour
```

Save as `main.go`:

```
package main

import (
 "context"
 "fmt"
 "io"
 "net"
 "net/http"
 "net/http/httptest"
 "time"
)

func main() {
 // 1) Dial with timeout (refused port → quick error, not hang)
 d := net.Dialer{Timeout: 100 * time.Millisecond}
 _, err := d.Dial("tcp", "127.0.0.1:1")
 fmt.Println("dial fail fast:", err != nil)

 // 2) Server + client with request context budget
 ts := httptest.NewServer(http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
 fmt.Fprintln(w, "pong")
 }))
 defer ts.Close()

 ctx, cancel := context.WithTimeout(context.Background(), time.Second)
 defer cancel()
 req, _ := http.NewRequestWithContext(ctx, http.MethodGet, ts.URL, nil)
 resp, err := http.DefaultClient.Do(req)
 if err != nil {
 panic(err)
 }
 b, _ := io.ReadAll(resp.Body)
 resp.Body.Close()
 fmt.Print("http: ", string(b))
}

go run .
```

**Expected output:**

```
dial fail fast: true
http: pong
```

## Runnable example

Tour of this part: a tiny HTTP server and client with production-shaped timeouts—the shared foundation for TCP framing, HTTP resilience, and reverse proxies.

```
mkdir -p /tmp/go-net-tour && cd /tmp/go-net-tour
go mod init example.com/net-tour
```

Save as `main.go`:

```
package main

import (
 "fmt"
 "io"
 "log"
 "net"
 "net/http"
 "time"
)

func main() {
 mux := http.NewServeMux()
 mux.HandleFunc("GET /api", func(w http.ResponseWriter, r *http.Request) {
 fmt.Fprintln(w, "pong")
 })

 ln, err := net.Listen("tcp", "127.0.0.1:0")
 if err != nil {
 log.Fatal(err)
 }
 srv := &http.Server{
 Handler: mux,
 ReadHeaderTimeout: 5 * time.Second,
 ReadTimeout: 10 * time.Second,
 WriteTimeout: 10 * time.Second,
 IdleTimeout: 30 * time.Second,
 }
 go func() { _ = srv.Serve(ln) }()

 client := &http.Client{
 Timeout: 3 * time.Second,
 Transport: &http.Transport{
 DialContext: (&net.Dialer{Timeout: time.Second}).DialContext,
 TLSHandshakeTimeout: 2 * time.Second,
 MaxIdleConns: 10,
 IdleConnTimeout: 30 * time.Second,
 },
 }
}
```

```

 },
}

url := "http://" + ln.Addr().String() + "/api"
resp, err := client.Get(url)
if err != nil {
 log.Fatal(err)
}
body, _ := io.ReadAll(resp.Body)
resp.Body.Close()
fmt.Printf("status=%d body=%q\n", resp.StatusCode, string(body))
fmt.Println("tour: timeouts set on server and client")
_ = srv.Close()
}

go run .

```

### Expected output:

```

status=200 body="pong\n"
tour: timeouts set on server and client

```

### What to notice

- Default clients and servers without timeouts are the #1 production footgun.
- This part stacks TCP framing (131), budgeted HTTP (132), and edge policy (133) on these primitives.

### Try next

- Remove `client.Timeout` and point at a black-hole address; restore the timeout.
- Read chapter 131 and implement length-prefix framing on a raw TCP conn.

## Next Chapter

Start with **TCP Services Deep Dive** — the lowest layer of the model, and the place most custom-protocol bugs hide.

## **TCP Services Deep Dive**

# TCP Services Deep Dive

TCP is a **byte stream**, not a message protocol. Most production bugs in custom TCP systems come from treating partial reads as complete messages, or unbounded buffers as “simpler.”

This chapter builds a correct, production-shaped TCP service in Go: framing, deadlines, concurrency limits, graceful shutdown, and observability.

## Why Custom TCP Still Matters

HTTP and gRPC cover most APIs. You still touch raw TCP when:

- Building binary protocols (game servers, market data, device telemetry)
- Speaking legacy line protocols (SMTP-ish, Redis RESP-like, custom IoT)
- Implementing proxies, tunnels, or multiplexers
- Teaching yourself how higher protocols actually work

If you only ever use frameworks, this chapter still upgrades your debugging of “connection stuck” incidents.

## Core Mental Model

```
application messages: | msg A | msg B | msg C |
 _____/ _____/
TCP may deliver: [A1] [A2+B1] [B2] [C1] [C2] [C3] ...
```

Rules:

1. One **Read** can return any number of bytes  $\geq 1$  (or 0 with error/EOF).
2. One **Write** may write fewer bytes than you asked; loop until done or use helpers.
3. Message boundaries are **application-defined**.

## Framing Strategies

Strategy	Shape	Pros	Cons
Length-prefix	[u32 len] [payload]	Simple, binary-friendly	Need max-size cap

Strategy	Shape	Pros	Cons
Delimiter	lines / null-terminated	Human debuggable	Escaping, binary pain
Fixed size	always N bytes	Trivial parse	Wasteful / inflexible
Self-describing	protobuf-style	Rich	Heavier stack

## Length-prefix (recommended baseline)

```

0 4 4+N
| len | payload |
u32 BE N bytes

```

Always enforce  $N \leq \text{MaxMessage}$ .

```

package frame

import (
 "encoding/binary"
 "fmt"
 "io"
)

const MaxMessage = 1 << 20 // 1 MiB

func WriteFrame(w io.Writer, payload []byte) error {
 if len(payload) > MaxMessage {
 return fmt.Errorf("payload too large: %d", len(payload))
 }
 var hdr [4]byte
 binary.BigEndian.PutUint32(hdr[:], uint32(len(payload)))
 if _, err := w.Write(hdr[:]); err != nil {
 return err
 }
 _, err := w.Write(payload)
 return err
}

func ReadFrame(r io.Reader) ([]byte, error) {
 var hdr [4]byte
 if _, err := io.ReadFull(r, hdr[:]); err != nil {
 return nil, err
 }
 n := binary.BigEndian.Uint32(hdr[:])
 if n > MaxMessage {

```



```

 return nil, fmt.Errorf("frame %d exceeds max %d", n, MaxMessage)
 }
 buf := make([]byte, n)
 if _, err := io.ReadFull(r, buf); err != nil {
 return nil, err
 }
 return buf, nil
}

```

`io.ReadFull` is the correct primitive: it loops until the buffer is full or an error occurs.

## Deadlines and Idle Connections

Without deadlines, a client that stops sending after the length header leaves your handler blocked forever.

```

func handleConn(conn net.Conn) {
 defer conn.Close()
 for {
 // Per-read idle deadline (sliding).
 _ = conn.SetReadDeadline(time.Now().Add(30 * time.Second))
 msg, err := frame.ReadFrame(conn)
 if err != nil {
 return // timeout, EOF, protocol error
 }
 resp := process(msg)
 _ = conn.SetWriteDeadline(time.Now().Add(10 * time.Second))
 if err := frame.WriteFrame(conn, resp); err != nil {
 return
 }
 }
}

```

Patterns:

- **Idle timeout:** refresh deadline before each read (above).
- **Absolute request timeout:** set once at accept for request-response protocols.
- **Separate read/write:** long uploads vs quick control messages need different policies.

## Accept Loop with Backpressure

Unbounded `go handle(conn)` is a classic self-DoS under connection floods.

```

package tcpsvc

import (
 "context"
 "log/slog"
 "net"
 "sync"
 "time"
)

type Server struct {
 Addr string
 Sem chan struct{} // capacity = max concurrent handlers
 Log *slog.Logger
 Handler func(context.Context, net.Conn)
}

func (s *Server) ListenAndServe(ctx context.Context) error {
 ln, err := net.Listen("tcp", s.Addr)
 if err != nil {
 return err
 }
 defer ln.Close()

 go func() {
 <-ctx.Done()
 _ = ln.Close() // unblock Accept
 }()

 var wg sync.WaitGroup
 for {
 conn, err := ln.Accept()
 if err != nil {
 select {
 case <-ctx.Done():
 wg.Wait()
 return ctx.Err()
 default:
 s.Log.Error("accept", "err", err)
 continue
 }
 }

 select {
 case s.Sem <- struct{}{}:
 // acquired slot
 default:

```

```

 // overload: fail fast
 _ = conn.Close()
 s.Log.Warn("reject connection: at capacity")
 continue
 }

 wg.Add(1)
 go func(c net.Conn) {
 defer wg.Done()
 defer func() { <-s.Sem }()
 defer c.Close()
 s.Handler(ctx, c)
 }(conn)
}
}

```

Production refinements:

- Accept a connection then **read a bit** before committing a heavy worker (cheap auth / magic bytes).
- Use `ListenConfig` with `Control` to set `SO_REUSEADDR` / socket options when needed.
- Prefer closing the listener on shutdown, then waiting for in-flight handlers with a deadline.

## IO vs Business Logic Separation

Slow CPU work on the same goroutine that reads the socket prevents reading the next frames and can stall peers.

```

read loop (per conn) -> jobs channel -> worker pool -> write responses

```

For request/response with ordering requirements, keep per-connection serialization of writes (one writer goroutine or a mutex on write).

```

type session struct {
 conn net.Conn
 out chan []byte
}

func (s *session) writer(ctx context.Context) {
 for {
 select {
 case <-ctx.Done():
 return
 case msg, ok := <-s.out:
 if !ok {
 return
 }

```

```

 }
 _ = s.conn.SetWriteDeadline(time.Now().Add(10 * time.Second))
 if err := frame.WriteFrame(s.conn, msg); err != nil {
 return
 }
}
}
}
}

```

## Protocol Errors vs Transport Errors

Classify failures:

Class	Examples	Action
Transport	EOF, reset, timeout	Close conn; metric <code>conn_errors</code>
Protocol	bad length, unknown type	Close or send error frame; metric <code>proto_errors</code>
App	business rejection	Error response frame; keep conn if protocol allows

Do not retry protocol errors on the same connection without a reset of parser state — your stream may be desynchronized.

## Keepalives and Half-Open Connections

TCP keepalives detect dead peers, but OS defaults are often too long (hours). For application liveness, prefer **application pings** with your own deadlines.

```
// Application-level heartbeat every 15s; miss 2 → close.
```

Also handle half-close: peer may `CloseWrite` while you still flush responses. Know your protocol's rules for FIN.

## TLS on TCP

Wrap accepted conns:

```

tlsLn := tls.NewListener(ln, tlsConfig)
// Accept from tlsLn as usual; deadlines still apply on the underlying net.Conn.

```

Certificate lifecycle is covered in part 17; from a TCP perspective, handshake timeouts matter:

```

_ = conn.SetDeadline(time.Now().Add(5 * time.Second))
tlsConn := tls.Server(conn, cfg)
if err := tlsConn.HandshakeContext(ctx); err != nil {
 return err
}
_ = tlsConn.SetDeadline(time.Time{}) // clear; use per-op deadlines later

```

## Observability

Minimum metrics for a TCP service:

- tcp\_accepts\_total
- tcp\_active\_conns
- tcp\_rejected\_total{reason="capacity|protocol"}
- tcp\_read\_timeouts\_total
- tcp\_frame\_bytes (histogram)
- tcp\_handler\_seconds (histogram)

Log fields: remote\_addr, conn\_id, frame\_type, err\_class, duration\_ms.

## Production Checklist

- ☐ Explicit framing with max message size
- ☐ SetReadDeadline / SetWriteDeadline on every path
- ☐ Bounded concurrent handlers (semaphore / worker pool)
- ☐ Reject or shed on overload instead of unbounded go
- ☐ Graceful shutdown: stop accept → drain → force close after deadline
- ☐ Metrics for accepts, active, errors, timeouts
- ☐ Fuzz or property-test the frame parser
- ☐ Document idle timeout and max frame in the protocol spec

## Common Pitfalls

1. **ioutil.ReadAll on a conn** — unbounded memory.
2. **Assuming one Read = one message.**
3. **Forgetting Write can be partial** — use frame.Write loops or io.Copy patterns carefully.
4. **Sharing one deadline for the entire connection lifetime** — long sessions need sliding idle deadlines.
5. **Logging every byte** — CPU and PII risk.
6. **No max concurrent conns** — memory death under SYN floods / connection storms.
7. **Mixing buffered readers incorrectly** — if you bufio.Reader wrap a conn, never read the conn underneath or you desync.

## Complete Mini Server Skeleton

```
func main() {
 ctx, stop := signal.NotifyContext(context.Background(), os.Interrupt, syscall.SIGTERM)
 defer stop()

 s := &tcpsvc.Server{
 Addr: ":9000",
 Sem: make(chan struct{}, 256),
 Log: slog.Default(),
 Handler: func(ctx context.Context, c net.Conn) {
 for {
 select {
 case <-ctx.Done():
 return
 default:
 }
 _ = c.SetReadDeadline(time.Now().Add(30 * time.Second))
 msg, err := frame.ReadFrame(c)
 if err != nil {
 return
 }
 // Echo for demo; replace with real handler.
 _ = c.SetWriteDeadline(time.Now().Add(10 * time.Second))
 _ = frame.WriteFrame(c, msg)
 }
 },
 }
 if err := s.ListenAndServe(ctx); err != nil && !errors.Is(err, context.Canceled) {
 slog.Error("server exit", "err", err)
 os.Exit(1)
 }
}
```

## Exercises

1. Implement length-prefix client/server echo; verify multi-message reads with a client that writes two frames in one `Write`.
2. Send a frame claiming `len = 2GiB`; ensure the server rejects without allocating.
3. Remove read deadlines and connect with `nc` without sending data; watch goroutine count with `pprof`.
4. Cap handlers at 2; open 10 connections; confirm extras are closed immediately.
5. Add a simple opcode byte after the length; fuzz the parser with `go test -fuzz`.
6. Implement graceful shutdown: in-flight echo must finish within 5s; then force close.
7. Benchmark frames of 64B vs 64KiB; record allocs/op for `ReadFrame`.

8. Wrap the server in TLS with a self-signed cert; measure handshake timeout behavior.
9. Deliberately use `bufio.Reader` for reads and `raw conn` for a side read — observe desync, then fix.
10. Export Prometheus metrics for active conns and timeouts; load-test and graph them.

## More examples

### Length-prefix TCP session (client + server)

```
mkdir -p /tmp/go-tcp-lp && cd /tmp/go-tcp-lp
go mod init example.com/tcp-lp
```

Save as `main.go`:

```
package main

import (
 "encoding/binary"
 "fmt"
 "io"
 "net"
 "time"
)

func writeMsg(w io.Writer, s string) error {
 var h [4]byte
 binary.BigEndian.PutUint32(h[:], uint32(len(s)))
 if _, err := w.Write(h[:]); err != nil {
 return err
 }
 _, err := io.WriteString(w, s)
 return err
}

func readMsg(r io.Reader) (string, error) {
 var h [4]byte
 if _, err := io.ReadFull(r, h[:]); err != nil {
 return "", err
 }
 n := binary.BigEndian.Uint32(h[:])
 buf := make([]byte, n)
 if _, err := io.ReadFull(r, buf); err != nil {
 return "", err
 }
 return string(buf), nil
}
```

```

}

func main() {
 ln, err := net.Listen("tcp", "127.0.0.1:0")
 if err != nil {
 panic(err)
 }
 defer ln.Close()

 done := make(chan struct{})
 go func() {
 defer close(done)
 c, err := ln.Accept()
 if err != nil {
 return
 }
 defer c.Close()
 _ = c.SetDeadline(time.Now().Add(time.Second))
 msg, err := readMsg(c)
 if err != nil {
 return
 }
 _ = writeMsg(c, "echo:"+msg)
 }()

 c, err := net.DialTimeout("tcp", ln.Addr().String(), time.Second)
 if err != nil {
 panic(err)
 }
 defer c.Close()
 _ = c.SetDeadline(time.Now().Add(time.Second))
 _ = writeMsg(c, "hello")
 out, err := readMsg(c)
 if err != nil {
 panic(err)
 }
 fmt.Println(out)
 <-done
}

go run .

```

**Expected output:**

echo:hello



## Per-connection deadlines

```
mkdir -p /tmp/go-tcp-deadline && cd /tmp/go-tcp-deadline
go mod init example.com/tcp-deadline
```

Save as `main.go`:

```
package main

import (
 "fmt"
 "net"
 "time"
)

func main() {
 ln, _ := net.Listen("tcp", "127.0.0.1:0")
 defer ln.Close()
 go func() {
 c, _ := ln.Accept()
 defer c.Close()
 time.Sleep(50 * time.Millisecond) // slow peer
 }()

 c, _ := net.Dial("tcp", ln.Addr().String())
 defer c.Close()
 _ = c.SetReadDeadline(time.Now().Add(10 * time.Millisecond))
 buf := make([]byte, 8)
 _, err := c.Read(buf)
 fmt.Println("timeout:", err != nil)
}

go run .
```

### Expected output:

```
timeout: true
```

## Runnable example

Length-prefix framing with max size, deadlines, and a single-process client/server echo—the core TCP service pattern.

```
mkdir -p /tmp/go-tcp-frame && cd /tmp/go-tcp-frame
go mod init example.com/tcp-frame
```

Save as `main.go`:

```
package main

import (
 "encoding/binary"
 "fmt"
 "io"
 "log"
 "net"
 "time"
)

const maxMsg = 64 << 10 // 64 KiB

func writeFrame(w io.Writer, payload []byte) error {
 if len(payload) > maxMsg {
 return fmt.Errorf("message too large")
 }
 var hdr [4]byte
 binary.BigEndian.PutUint32(hdr[:], uint32(len(payload)))
 if _, err := w.Write(hdr[:]); err != nil {
 return err
 }
 _, err := w.Write(payload)
 return err
}

func readFrame(r io.Reader) ([]byte, error) {
 var hdr [4]byte
 if _, err := io.ReadFull(r, hdr[:]); err != nil {
 return nil, err
 }
 n := binary.BigEndian.Uint32(hdr[:])
 if n > maxMsg {
 return nil, fmt.Errorf("frame %d exceeds max %d", n, maxMsg)
 }
 buf := make([]byte, n)
 if _, err := io.ReadFull(r, buf); err != nil {
 return nil, err
 }
 return buf, nil
}

func main() {
 ln, err := net.Listen("tcp", "127.0.0.1:0")
 if err != nil {
```

```

 log.Fatal(err)
}
defer ln.Close()

errCh := make(chan error, 1)
go func() {
 c, err := ln.Accept()
 if err != nil {
 errCh <- err
 return
 }
 defer c.Close()
 _ = c.SetDeadline(time.Now().Add(5 * time.Second))
 msg, err := readFrame(c)
 if err != nil {
 errCh <- err
 return
 }
 errCh <- writeFrame(c, append([]byte("echo:"), msg...))
}()

conn, err := net.DialTimeout("tcp", ln.Addr().String(), time.Second)
if err != nil {
 log.Fatal(err)
}
defer conn.Close()
_ = conn.SetDeadline(time.Now().Add(5 * time.Second))

if err := writeFrame(conn, []byte("hello-tcp")); err != nil {
 log.Fatal(err)
}
resp, err := readFrame(conn)
if err != nil {
 log.Fatal(err)
}
fmt.Printf("response: %q\n", string(resp))

// Oversized frame rejected
conn2, err := net.Dial("tcp", ln.Addr().String())
if err == nil {
 conn2.Close()
}

// Direct unit-style check of max enforcement
var bad [4]byte
binary.BigEndian.PutUint32(bad[:], uint32(maxMsg)+1)
_, err = readFrame(&limitedReader{b: bad[:]}))

```

```

 fmt.Println("oversize rejected:", err != nil)

 if err := <-errCh; err != nil {
 // server may error after client closed second attempt; ignore if already echoed
 _ = err
 }
 fmt.Println("tcp framing ok")
}

// limitedReader feeds only the header for the oversize demo.
type limitedReader struct{ b []byte }

func (l *limitedReader) Read(p []byte) (int, error) {
 if len(l.b) == 0 {
 return 0, io.EOF
 }
 n := copy(p, l.b)
 l.b = l.b[n:]
 return n, nil
}

go run .

```

### Expected output:

```

response: "echo:hello-tcp"
oversize rejected: true
tcp framing ok

```

### What to notice

- `io.ReadFull` is required; one `Read` is never a full frame.
- Cap `n` before allocating—malicious length prefixes are a memory DoS.
- Deadlines on the conn protect against silent peers.

### Try next

- Send two frames in one `Write` and read both on the server loop.
- Cap concurrent handlers with a buffered channel semaphore.

## Further Reading

- `man 7 tcp`, `man 2 setsockopt` (keepalive, linger)
- Go blog: deadlines, netpoller behavior
- Next: **HTTP Client and Server Resilience** — budgets and policy on top of streams
- Part 17: TLS/mTLS for securing these sockets

# HTTP Client and Server Resilience

# HTTP Client and Server Resilience

Resilience in HTTP systems is **budget management**: every inbound request has limited time, and each downstream call spends part of that budget. When budgets are implicit, partial outages become full outages.

## Why / Overview

`net/http` is excellent — and dangerously easy to misuse. The zero-value `Client` has **no timeout**. The zero-value `Server` has **no read/write timeouts**. Those defaults are fine for toy programs and catastrophic under load.

This chapter makes timeouts, cancellation, retries, and transport tuning explicit.

```
client SLA (e.g. 2s)
|
v
edge / server budget (1.8s) -- ReadHeader/Read/Write/Idle
|
+--> auth (50ms)
+--> cache (100ms)
+--> db (400ms)
+--> upstream HTTP (remaining via context)
+--> encode
```

## Server-Side Guardrails

**Configure the server, not just the handler**

```
srv := &http.Server{
 Addr: ":8080",
 Handler: logging(mux),
 ReadHeaderTimeout: 5 * time.Second, // slowloris protection
 ReadTimeout: 15 * time.Second, // headers + body
 WriteTimeout: 30 * time.Second, // careful with streaming
 IdleTimeout: 60 * time.Second, // keep-alives
 MaxHeaderBytes: 1 << 16,
}
```

Field	Protects against
ReadHeaderTimeout	Slow header drip (slowloris)
ReadTimeout	Slow bodies
WriteTimeout	Stuck clients on response
IdleTimeout	Idle keep-alive resource waste
MaxHeaderBytes	Oversized header memory use

**Streaming caveat:** WriteTimeout is absolute from request start on many setups; long-lived streams (SSE, chunked) may need a different server or hijacking strategy.

## Handler-level budgets

```
func handleOrder(w http.ResponseWriter, r *http.Request) {
 ctx, cancel := context.WithTimeout(r.Context(), 1200*time.Millisecond)
 defer cancel()

 user, err := auth.User(ctx, r)
 if err != nil {
 writeErr(w, err)
 return
 }
 order, err := orders.Create(ctx, user, r.Body)
 if err != nil {
 writeErr(w, err)
 return
 }
 writeJSON(w, order)
}
```

Always derive from `r.Context()` so client disconnects cancel work.

## Limit concurrency of expensive handlers

```
var sem = make(chan struct{}, 100)

func limited(next http.Handler) http.Handler {
 return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
 select {
 case sem <- struct{}{}:
 defer func() { <-sem }()
 next.ServeHTTP(w, r)
 default:
 http.Error(w, "overloaded", http.StatusServiceUnavailable)
 }
 })
}
```

```
 })
}
```

Return **503** with `Retry-After` when shedding load so well-behaved clients back off.

## Body limits

```
r.Body = http.MaxBytesReader(w, r.Body, 1<<20)
```

Combine with JSON decoders that use `DisallowUnknownFields` for strict APIs.

## Client-Side Guardrails

### Never use the naked default client in production

```
var outbound = &http.Client{
 Timeout: 10 * time.Second, // hard cap including body read
 Transport: &http.Transport{
 Proxy: http.ProxyFromEnvironment,
 DialContext: (&net.Dialer{
 Timeout: 3 * time.Second,
 KeepAlive: 30 * time.Second,
 }).DialContext,
 ForceAttemptHTTP2: true,
 MaxIdleConns: 100,
 MaxIdleConnsPerHost: 10,
 MaxConnsPerHost: 32,
 IdleConnTimeout: 90 * time.Second,
 TLSHandshakeTimeout: 5 * time.Second,
 ExpectContinueTimeout: 1 * time.Second,
 ResponseHeaderTimeout: 5 * time.Second,
 },
}
```

Notes:

- `Client.Timeout` covers the whole request (dial → headers → body).
- Prefer **per-call context** for operation-specific budgets finer than the client timeout.
- Tune `MaxConnsPerHost` to avoid stampedes on a single dependency.



## Context-aware requests

```
func fetchUser(ctx context.Context, id string) (*User, error) {
 ctx, cancel := context.WithTimeout(ctx, 300*time.Millisecond)
 defer cancel()

 req, err := http.NewRequestWithContext(ctx, http.MethodGet, baseURL+"/users/"+id, nil)
 if err != nil {
 return nil, err
 }
 resp, err := outbound.Do(req)
 if err != nil {
 return nil, err
 }
 defer resp.Body.Close()

 if resp.StatusCode >= 500 {
 return nil, fmt.Errorf("upstream status %d", resp.StatusCode)
 }
 var u User
 if err := json.NewDecoder(io.LimitReader(resp.Body, 1<<20)).Decode(&u); err != nil {
 return nil, err
 }
 return &u, nil
}
```

Always Close bodies — leaking bodies leaks connections from the pool.

## Failure Classification

Signal	Likely meaning	Retry?
<code>context.DeadlineExceeded</code>	Budget exhausted / slow dep	Maybe once, if idempotent & budget remains
<code>context.Canceled</code>	Caller gave up	No
Connection refused / reset	Dep down or restarting	Yes with backoff
HTTP 408 / 429	Timeout / rate limit	Yes respecting <b>Retry-After</b> Conditional
HTTP 500 / 502 / 503 / 504	Upstream error	
HTTP 400 / 401 / 403 / 404	Caller / auth / missing	No
HTTP 409 / 422	Business conflict	Usually no

```

func retryable(status int, err error) bool {
 if err != nil {
 if errors.Is(err, context.Canceled) {
 return false
 }
 return true // network errors: often transient
 }
 switch status {
 case 408, 429, 500, 502, 503, 504:
 return true
 default:
 return false
 }
}

```

## Retries Without Amplification

Rules of thumb:

1. Retry only **idempotent** methods by default (GET, HEAD, PUT, DELETE with idempotency keys).
2. Cap attempts (e.g. 2–3 total).
3. Exponential backoff **with jitter**.
4. Bound **total elapsed time**, not just attempt count.
5. Propagate remaining parent context budget.
6. Prefer idempotency keys for POST when the business allows.

```

func doWithRetry(ctx context.Context, c *http.Client, req *http.Request) (*http.Response, error) {
 var last error
 backoff := 50 * time.Millisecond
 for attempt := 0; attempt < 3; attempt++ {
 if attempt > 0 {
 // Clone body if needed; for GET, req can be reused carefully.
 select {
 case <-ctx.Done():
 return nil, ctx.Err()
 case <-time.After(backoff + jitter(backoff)):
 }
 backoff *= 2
 }
 r := req.Clone(ctx)
 resp, err := c.Do(r)
 if err == nil && !retryable(resp.StatusCode, nil) {
 return resp, nil
 }
 if err == nil {

```

```

 resp.Body.Close()
 last = fmt.Errorf("status %d", resp.StatusCode)
 } else {
 last = err
 if !retryable(0, err) {
 return nil, err
 }
 }
}
return nil, last
}

```

Misconfigured retries at **browser + edge + service + client library** layers multiply load — coordinate policy.

## Transport Pooling and HTTP/2

- Reuse a single `*http.Client` / `Transport` (connection pool lives there).
- Creating a new `Transport` per request disables reuse and burns sockets.
- HTTP/2 multiplexes streams on one conn; head-of-line blocking moves to the application layer — still use deadlines per request.
- For many hosts, raise `MaxIdleConns`; for one hot host, raise `MaxIdleConnsPerHost`.

## Graceful Shutdown

```

ctx, stop := signal.NotifyContext(context.Background(), os.Interrupt, syscall.SIGTERM)
defer stop()

go func() {
 if err := srv.ListenAndServe(); err != nil && !errors.Is(err, http.ErrServerClosed) {
 slog.Error("listen", "err", err)
 os.Exit(1)
 }
}()

<-ctx.Done()
shutdownCtx, cancel := context.WithTimeout(context.Background(), 20*time.Second)
defer cancel()
if err := srv.Shutdown(shutdownCtx); err != nil {
 slog.Error("shutdown", "err", err)
 _ = srv.Close()
}

```

**Shutdown** stops accepting, waits for handlers; handlers must honor `r.Context()` to finish promptly.

## Observability Hooks

Emit at least:

- Request rate, error rate, latency histogram (RED)
- Client-side: outbound latency and error class by dependency
- Timeout counts separately from application 5xx
- In-flight requests / active connections

```
func instrument(next http.Handler) http.Handler {
 return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
 start := time.Now()
 ww := &statusWriter{ResponseWriter: w, code: 200}
 next.ServeHTTP(ww, r)
 slog.Info("request",
 "method", r.Method,
 "path", r.URL.Path,
 "status", ww.code,
 "dur_ms", time.Since(start).Milliseconds(),
)
 })
}
```

## Production Checklist

- ☐ Every `http.Server` has header/read/write/idle timeouts
- ☐ Every production `http.Client` has `Timeout` + tuned `Transport`
- ☐ Handlers derive work from `r.Context()` with sub-timeouts
- ☐ Response bodies always closed; request bodies limited
- ☐ Retries are idempotent, jittered, budget-capped
- ☐ Load shedding returns 503 under overload
- ☐ Graceful Shutdown with handler deadlines
- ☐ Metrics distinguish timeouts, cancels, and app errors
- ☐ Single shared clients for outbound deps (pooled)

## Common Pitfalls

1. `http.Get` in library code — uses default client, no timeout.
2. Ignoring `resp.Body` — connection pool starvation.
3. Retrying POST without idempotency — double charges / double writes.
4. Server `WriteTimeout` too aggressive for large downloads.
5. Context with values used for cancellation only half the stack — DB layer ignores cancel.
6. Global 30s timeout for all outbound calls — wastes budget on optional paths.
7. Fan-out without `errgroup` limits — one request opens 200 deps.

## Exercises

1. Start a server without timeouts; use a slow client that drips headers; then add `ReadHeaderTimeout` and retest.
2. Build a client with and without `Timeout`; point at a black hole IP; compare hang vs fail.
3. Implement status-aware retries with jitter; unit-test that 404 is not retried and 503 is.
4. Use `httptest.Server` to inject 100ms / 500ms / 2s latencies; verify parent 300ms budget cancels.
5. Deliberately leak `resp.Body` in a loop; watch `TIME_WAIT` / goroutines / pool behavior; fix.
6. Add a concurrency limiter middleware; load-test until you see 503s; graph in-flight.
7. Implement graceful shutdown; send `SIGTERM` under load; confirm in-flight requests complete within budget.
8. Split timeouts: dial 200ms, TLS 500ms, headers 500ms, total 2s — document where each is configured.
9. Compare HTTP/1.1 vs HTTP/2 under many parallel requests to one host; note conn counts.
10. Write a runbook entry: “p99 latency spike” — which timeout metric do you check first?

## More examples

### Client with transport timeouts (not zero-value)

```
mkdir -p /tmp/go-http-client-to && cd /tmp/go-http-client-to
go mod init example.com/http-client-to
```

Save as `main.go`:

```
package main

import (
 "fmt"
 "net"
 "net/http"
 "net/http/httptest"
 "time"
)

func main() {
 slow := httptest.NewServer(http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
 time.Sleep(80 * time.Millisecond)
 fmt.Fprintln(w, "late")
 }))
 defer slow.Close()

 client := &http.Client{
 Timeout: 20 * time.Millisecond,
```

```

 Transport: &http.Transport{
 DialContext: (&net.Dialer{Timeout: 50 * time.Millisecond}).DialContext,
 ResponseHeaderTimeout: 50 * time.Millisecond,
 },
 }
 _, err := client.Get(slow.URL)
 fmt.Println("client timeout:", err != nil)

 fast := httptest.NewServer(http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
 fmt.Fprintln(w, "ok")
 }))
 defer fast.Close()
 client.Timeout = time.Second
 resp, err := client.Get(fast.URL)
 if err != nil {
 panic(err)
 }
 resp.Body.Close()
 fmt.Println("fast status:", resp.StatusCode)
}

go run .

```

### Expected output:

```

client timeout: true
fast status: 200

```

### Retry only on transient status

```

mkdir -p /tmp/go-http-retry && cd /tmp/go-http-retry
go mod init example.com/http-retry

```

Save as main.go:

```

package main

import (
 "fmt"
 "net/http"
 "net/http/httptest"
 "sync/atomic"
)

func main() {

```

```

var n atomic.Int32
ts := httpptest.NewServer(http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
 if n.Add(1) < 3 {
 w.WriteHeader(http.StatusServiceUnavailable)
 return
 }
 fmt.Fprintln(w, "ok")
}))
defer ts.Close()

var last int
for attempt := 1; attempt <= 5; attempt++ {
 resp, err := http.Get(ts.URL)
 if err != nil {
 panic(err)
 }
 last = resp.StatusCode
 resp.Body.Close()
 if last < 500 {
 break
 }
}
fmt.Println("attempts:", n.Load(), "final:", last)
}

go run .

```

**Expected output:**

```
attempts: 3 final: 200
```

## Runnable example

httpptest backend with injected latency, a budgeted client, and retries with exponential backoff + jitter—stdlib-only resilience loop.

```

mkdir -p /tmp/go-http-resilience && cd /tmp/go-http-resilience
go mod init example.com/http-resilience

```

Save as main.go:

```

package main

import (
 "context"

```

```

 "fmt"
 "io"
 "math/rand"
 "net/http"
 "net/http/httptest"
 "time"
)

func backoff(attempt int) time.Duration {
 // Full jitter: random in [0, base*2^attempt]
 base := 20 * time.Millisecond
 max := base << attempt
 if max > 200*time.Millisecond {
 max = 200 * time.Millisecond
 }
 return time.Duration(rand.Int63n(int64(max) + 1))
}

func getWithRetry(ctx context.Context, client *http.Client, url string, attempts int) (int,
 var last error
 for i := 0; i < attempts; i++ {
 req, err := http.NewRequestWithContext(ctx, http.MethodGet, url, nil)
 if err != nil {
 return 0, err
 }
 resp, err := client.Do(req)
 if err != nil {
 last = err
 } else {
 code := resp.StatusCode
 io.Copy(io.Discard, resp.Body)
 resp.Body.Close()
 if code < 500 {
 return code, nil // do not retry 4xx
 }
 last = fmt.Errorf("status %d", code)
 }
 if i+1 == attempts {
 break
 }
 t := time.NewTimer(backoff(i))
 select {
 case <-ctx.Done():
 t.Stop()
 return 0, ctx.Err()
 case <-t.C:
 }
 }
}

```



```

 }
 return 0, last
}

func main() {
 var calls int
 ts := httptest.NewServer(http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
 calls++
 if calls < 3 {
 w.WriteHeader(http.StatusServiceUnavailable)
 return
 }
 fmt.Fprintln(w, "ok")
 }))
 defer ts.Close()

 client := &http.Client{Timeout: 500 * time.Millisecond}

 ctx, cancel := context.WithTimeout(context.Background(), 2*time.Second)
 defer cancel()
 code, err := getWithRetry(ctx, client, ts.URL, 5)
 fmt.Printf("final status=%d err=%v upstream_calls=%d\n", code, err, calls)

 // Budget cancel demo: server slower than parent context
 slow := httptest.NewServer(http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
 time.Sleep(200 * time.Millisecond)
 fmt.Fprintln(w, "slow")
 }))
 defer slow.Close()
 ctx2, cancel2 := context.WithTimeout(context.Background(), 50*time.Millisecond)
 defer cancel2()
 _, err = getWithRetry(ctx2, client, slow.URL, 3)
 fmt.Println("budget exceeded:", err != nil)
}

go run .

```

**Expected output** (illustrative):

```

final status=200 err=<nil> upstream_calls=3
budget exceeded: true

```

### What to notice

- Retry only idempotent-safe cases (here GET) and 5xx/transport errors—not 404.
- Parent context budget must bound total attempts; infinite retry is an outage amplifier.
- `httptest.Server` is the right unit-test stand-in for dependency latency and failure.

### Try next

- Skip retries on `context.Canceled` and count them separately in metrics.
- Add a concurrency semaphore middleware that returns 503 under overload.

### Further Reading

- Go `net/http` docs for `Transport` fields
- SRE books on tail latency and load shedding
- Previous: TCP framing and deadlines
- Next: **Reverse Proxy and Load Balancing** — centralizing these policies at the edge

# Reverse Proxy and Load Balancing

# Reverse Proxy and Load Balancing

A reverse proxy is a **reliability boundary**: it enforces routing policy, mediates failures, and protects backend services. In Go you can build edge-like behavior with `net/http/httputil` and a small amount of discipline — or understand nginx/Envoy better by implementing the same ideas.

## Why / Overview

Backends should focus on business logic. The proxy owns:

- Route matching and path rewrite
- Upstream selection and health
- Timeout and retry policy (carefully)
- Request identity / header hygiene
- TLS termination or pass-through
- Observability of the edge hop

```
client -> edge proxy -> route match -> pick upstream -> backend
 \-> policies: authn, rate limit, timeout, LB
```

## Building a Reverse Proxy in Go

### Single upstream

```
package main

import (
 "net/http"
 "net/http/httputil"
 "net/url"
 "time"
)

func main() {
 target, _ := url.Parse("http://127.0.0.1:8081")
 proxy := httputil.NewSingleHostReverseProxy(target)

 // Optional: custom transport with timeouts.
```

```

proxy.Transport = &http.Transport{
 Proxy: http.ProxyFromEnvironment,
 ResponseHeaderTimeout: 5 * time.Second,
 IdleConnTimeout: 90 * time.Second,
 MaxIdleConnsPerHost: 32,
}

// Error handler when upstream is down.
proxy.ErrorHandler = func(w http.ResponseWriter, r *http.Request, err error) {
 http.Error(w, "bad gateway", http.StatusBadGateway)
}

srv := &http.Server{
 Addr: ":8080",
 Handler: proxy,
 ReadHeaderTimeout: 5 * time.Second,
}
srv.ListenAndServe()
}

```

NewSingleHostReverseProxy rewrites the request URL to the target and sets X-Forwarded-For (with care about trust).

## Director customization

```

proxy := &httputil.ReverseProxy{
 Director: func(req *http.Request) {
 req.URL.Scheme = "http"
 req.URL.Host = upstream // host:port
 req.Host = upstream
 // Strip internal headers clients must not inject.
 req.Header.Del("X-Internal-Auth")
 req.Header.Set("X-Request-Id", ensureRequestID(req))
 },
 Transport: transport,
 ErrorHandler: edgeError,
 ModifyResponse: func(resp *http.Response) error {
 resp.Header.Set("X-Edge", "go-proxy")
 return nil
 },
}
}

```

## Routing Model

Keep routing deterministic and debuggable:

1. Exact path matches first
2. Method-aware routes when needed
3. Longest prefix next
4. Explicit default / 404

```
type route struct {
 prefix string
 pool *UpstreamPool
}

func (r *Router) ServeHTTP(w http.ResponseWriter, req *http.Request) {
 for _, rt := range r.routes { // ordered: longest prefix first
 if strings.HasPrefix(req.URL.Path, rt.prefix) {
 rt.pool.Proxy().ServeHTTP(w, req)
 return
 }
 }
 http.NotFound(w, req)
}
```

Document route tables in config, not only in code — operators need to see them during incidents.

## Upstream Selection and Load Balancing

### Round-robin baseline

```
type UpstreamPool struct {
 mu sync.Mutex
 addrs []string
 next uint64
 alive []bool
}

func (p *UpstreamPool) pick() (string, bool) {
 p.mu.Lock()
 defer p.mu.Unlock()
 n := len(p.addrs)
 for i := 0; i < n; i++ {
 idx := int(p.next % uint64(n))
 p.next++
 if p.alive[idx] {
 return p.addrs[idx], true
 }
 }
 return "", false
}
```

```
}
```

## Health awareness

```
active check: proxy -> GET /healthz on each upstream every N seconds
passive check: consecutive 5xx / connect errors mark unhealthy
reentry: require M successes before alive=true again
```

```
func (p *UpstreamPool) healthLoop(ctx context.Context, client *http.Client, every time.Duration) {
 t := time.NewTicker(every)
 defer t.Stop()
 for {
 select {
 case <-ctx.Done():
 return
 case <-t.C:
 for i, addr := range p.addrs {
 ok := probe(ctx, client, "http://"+addr+"/healthz")
 p.mu.Lock()
 p.alive[i] = ok
 p.mu.Unlock()
 }
 }
 }
}
```

Use **readiness**, not mere process liveness, for routing. A process can be up while its DB is down.

## Other LB algorithms (when RR is not enough)

Algorithm	Use when
Round-robin	Homogeneous backends
Least connections	Long-lived / uneven requests
Consistent hash	Cache locality / sticky sessions without central store
Weighted	Canary / heterogeneous capacity

Sticky sessions couple clients to instances and complicate deploys — prefer shared session stores when possible.

## Timeout and Retry Boundaries

Proxy retries can **multiply** load. Rules:

- Retry only safe methods by default.
- Cap attempts (often 1 retry for idempotent GET).
- Enforce a **total edge budget** (e.g. 5s) independent of backend generosity.
- Do not retry if the client already disconnected.
- Prefer failovers to a different healthy instance over hammering the same one.

```
func (p *UpstreamPool) Proxy() http.Handler {
 return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
 ctx, cancel := context.WithTimeout(r.Context(), 5*time.Second)
 defer cancel()
 r = r.WithContext(ctx)

 addr, ok := p.pick()
 if !ok {
 http.Error(w, "no healthy upstream", http.StatusServiceUnavailable)
 return
 }
 // Build reverse proxy for addr or set Director host to addr.
 p.reverseProxyTo(addr).ServeHTTP(w, r)
 })
}
```

## Header Hygiene and Trust

Critical edge concerns:

1. **Hop-by-hop headers** — Connection, Keep-Alive, Transfer-Encoding must not be blindly forwarded incorrectly (httputil handles many cases).
2. **X-Forwarded-\*** — only append client IP if the immediate peer is a trusted hop; otherwise clients spoof IPs for rate limits/ACLs.
3. **Strip internal auth headers** — clients must not set X-User-Id that backends trust.
4. **Request IDs** — generate if missing; propagate to backends for correlation.

```
func ensureRequestID(r *http.Request) string {
 if id := r.Header.Get("X-Request-Id"); id != "" {
 return id
 }
 return uuidNew() // crypto/rand based id
}
```



## Buffering vs Streaming

- Default reverse proxy streams bodies; good for large uploads/downloads.
- Buffering enables retries with body replay but costs memory — only for small, idempotent requests with retained bodies.
- For POST with bodies, retries require `GetBody` / rewind support.

## Observability at the Edge

Metrics:

- `edge_requests_total{route,code}`
- `edge_request_duration_seconds{route}`
- `edge_upstream_errors_total{upstream,reason}`
- `edge_upstream_alive{upstream}`
- `edge_retry_total{route}`

Logs: request id, route, chosen upstream, status, duration, error class.

Tracing: create a span at the edge; inject propagation headers into upstream requests (see part 16).

## Production Checklist

- ☐ Explicit route table with deterministic matching
- ☐ Health-aware upstream pool with hysteresis on recovery
- ☐ Transport timeouts on proxy → backend
- ☐ Edge-level total timeout budget
- ☐ Conservative retry policy (idempotent only)
- ☐ Header allow/deny lists for trusted identity
- ☐ Separate admin listener for proxy diagnostics
- ☐ Metrics per route and per upstream
- ☐ Safe config reload story (or blue/green proxy instances)
- ☐ Load tests for failover and thundering herd on recovery

## Common Pitfalls

1. **Retry storms** when all instances degraded.
2. **Routing on liveness** while app is not ready.
3. **Trusting client X-Forwarded-For** on a public edge.
4. **One giant proxy binary** without resource limits — edge is critical path.
5. **WebSocket / upgrade mishandling** — ensure hijack/upgrade paths are tested.
6. **Hidden double timeouts** — edge 1s + backend 30s still fails at 1s; document the tighter bound.
7. **Inconsistent path stripping** — `/api/v1` stripped twice → wrong backend path.

## Minimal Multi-Upstream Example

```
type multiHost struct {
 pool *UpstreamPool
 tr http.RoundTripper
}

func (m *multiHost) ServeHTTP(w http.ResponseWriter, r *http.Request) {
 addr, ok := m.pool.pick()
 if !ok {
 http.Error(w, "unavailable", http.StatusServiceUnavailable)
 return
 }
 rp := &httputil.ReverseProxy{
 Director: func(req *http.Request) {
 req.URL.Scheme = "http"
 req.URL.Host = addr
 req.Host = addr
 },
 Transport: m.tr,
 ErrorHandler: func(w http.ResponseWriter, r *http.Request, err error) {
 m.pool.noteFailure(addr)
 http.Error(w, "bad gateway", http.StatusBadGateway)
 },
 }
 rp.ServeHTTP(w, r)
}
```

## Exercises

1. Proxy a single backend; kill the backend; confirm 502 and a metric increment.
2. Implement round-robin across three `httptest` backends; log which host served each request.
3. Add active health checks; mark one down; prove traffic shifts within one interval.
4. Implement passive health: three consecutive errors → unhealthy; two successes → healthy.
5. Send a client-supplied `X-Internal-User` header; strip it at the proxy; verify backend never sees the spoofed value.
6. Add edge timeout of 200ms; backend sleeps 500ms; confirm client gets timeout/502, not a hang.
7. Enable a single retry on GET only; prove POST is not retried.
8. Load-test with one slow upstream in the pool; compare RR vs least-conn if you implement both.
9. Add request IDs and propagate them; grep logs across proxy and backend for one request.
10. Document a failure drill: “all upstreams unhealthy” — expected status, alerts, runbook.

## More examples

### Round-robin reverse proxy (stdlib)

```
mkdir -p /tmp/go-rr-proxy && cd /tmp/go-rr-proxy
go mod init example.com/rr-proxy
```

Save as main.go:

```
package main

import (
 "fmt"
 "net/http"
 "net/http/httptest"
 "net/http/httputil"
 "net/url"
 "sync/atomic"
)

func main() {
 b1 := httptest.NewServer(http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
 fmt.Fprintln(w, "b1")
 }))
 defer b1.Close()
 b2 := httptest.NewServer(http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
 fmt.Fprintln(w, "b2")
 }))
 defer b2.Close()

 targets := []*url.URL{mustURL(b1.URL), mustURL(b2.URL)}
 var i atomic.Uint64
 proxy := &httputil.ReverseProxy{
 Rewrite: func(pr *httputil.ProxyRequest) {
 t := targets[i.Add(1)%uint64(len(targets))]
 pr.SetURL(t)
 pr.Out.Host = t.Host
 },
 }

 front := httptest.NewServer(proxy)
 defer front.Close()

 for n := 0; n < 4; n++ {
 resp, err := http.Get(front.URL + "/")
 if err != nil {
```

```

 panic(err)
 }
 var buf [8]byte
 k, _ := resp.Body.Read(buf[:])
 resp.Body.Close()
 fmt.Print(string(buf[:k]))
}
}

func mustURL(s string) *url.URL {
 u, err := url.Parse(s)
 if err != nil {
 panic(err)
 }
 return u
}

go run .

```

Expected output:

```

b2
b1
b2
b1

```

## Skip unhealthy upstream

```

mkdir -p /tmp/go-proxy-health && cd /tmp/go-proxy-health
go mod init example.com/proxy-health

```

Save as main.go:

```

package main

import (
 "fmt"
 "net/http"
 "net/http/httptest"
 "sync/atomic"
)

func main() {
 var healthy atomic.Bool
 healthy.Store(false)

```

```

up := httptest.NewServer(http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
 if r.URL.Path == "/healthz" {
 if !healthy.Load() {
 w.WriteHeader(http.StatusServiceUnavailable)
 return
 }
 fmt.Fprintln(w, "ok")
 return
 }
 fmt.Fprintln(w, "served")
}))
defer up.Close()

pick := func() (string, bool) {
 resp, err := http.Get(up.URL + "/healthz")
 if err != nil {
 return "", false
 }
 resp.Body.Close()
 return up.URL, resp.StatusCode == 200
}

_, ok := pick()
fmt.Println("cold ok:", ok)
healthy.Store(true)
u, ok := pick()
fmt.Println("warm ok:", ok, "url set:", u != "")
}

go run .

```

### Expected output:

```

cold ok: false
warm ok: true url set: true

```

## Runnable example

In-process reverse proxy with round-robin across three `httptest` backends and passive failure skip—stdlib `httputil.ReverseProxy`.

```

mkdir -p /tmp/go-rproxy && cd /tmp/go-rproxy
go mod init example.com/rproxy

```

Save as `main.go`:

```

package main

import (
 "fmt"
 "net/http"
 "net/http/httpptest"
 "net/http/httpputil"
 "net/url"
 "sync"
 "sync/atomic"
)

type pool struct {
 mu sync.Mutex
 addrs []string
 down map[string]bool
 rr uint64
}

func (p *pool) next() (string, bool) {
 p.mu.Lock()
 defer p.mu.Unlock()
 n := len(p.addrs)
 for i := 0; i < n; i++ {
 idx := int(atomic.AddUint64(&p.rr, 1)-1) % n
 a := p.addrs[idx]
 if !p.down[a] {
 return a, true
 }
 }
 return "", false
}

func (p *pool) markDown(a string) {
 p.mu.Lock()
 p.down[a] = true
 p.mu.Unlock()
}

func main() {
 var hits [3]atomic.Int64
 var backends []*httpptest.Server
 var addrs []string
 for i := 0; i < 3; i++ {
 i := i
 ts := httpptest.NewServer(http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
 if i == 1 && hits[1].Load() == 0 {

```

```

 // first touch on backend 1 fails once
 }
 hits[i].Add(1)
 fmt.Fprintf(w, "backend-%d", i)
})))
backends = append(backends, ts)
u, _ := url.Parse(ts.URL)
addrs = append(addrs, u.Host)
}
defer func() {
 for _, b := range backends {
 b.Close()
 }
}()

p := &pool{addrs: addrs, down: map[string]bool{}}
// Mark middle backend down to show skip.
p.markDown(addrs[1])

proxy := http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
 addr, ok := p.next()
 if !ok {
 http.Error(w, "unavailable", http.StatusServiceUnavailable)
 return
 }
 rp := &httputil.ReverseProxy{
 Director: func(req *http.Request) {
 req.URL.Scheme = "http"
 req.URL.Host = addr
 req.Host = addr
 req.Header.Del("X-Internal-User") // strip spoofable header
 },
 ErrorHandler: func(w http.ResponseWriter, r *http.Request, err error) {
 p.markDown(addr)
 http.Error(w, "bad gateway", http.StatusBadGateway)
 },
 }
 rp.ServeHTTP(w, r)
})

front := httptest.NewServer(proxy)
defer front.Close()

for i := 0; i < 6; i++ {
 resp, err := http.Get(front.URL + "/")
 if err != nil {
 fmt.Println("err", err)
 }
}

```

```

 continue
 }
 var body [32]byte
 n, _ := resp.Body.Read(body[:])
 resp.Body.Close()
 fmt.Printf("req %d -> %s\n", i+1, string(body[:n]))
}
fmt.Printf("hits: b0=%d b1=%d b2=%d\n", hits[0].Load(), hits[1].Load(), hits[2].Load())
}

go run .

```

**Expected output** (illustrative):

```

req 1 -> backend-0
req 2 -> backend-2
req 3 -> backend-0
req 4 -> backend-2
req 5 -> backend-0
req 6 -> backend-2
hits: b0=3 b1=0 b2=3

```

### What to notice

- Edge policy (skip down backends, strip unsafe headers) belongs in the proxy, not every service.
- Round-robin with health state is enough to reason about blast radius before adopting Envoy.
- Director rewrites scheme/host; never trust client-supplied internal identity headers.

### Try next

- Active health checks every 200ms that clear down when /healthz returns 200.
- Edge timeout: wrap transport with a short `ResponseHeaderTimeout`.

## Further Reading

- `net/http/httputil.ReverseProxy` source (Director, rewrite, errors)
- Envoy / nginx docs on retries and outlier detection (concepts transfer)
- Previous: HTTP resilience budgets
- Next part: **Systems Programming** — process and filesystem discipline that network daemons rely on



# **TLS Clients and Servers**

# TLS Clients and Servers

## Overview

TLS protects data in transit. In Go, `crypto/tls` sits under `net/http` and raw `tls.Conn`. Get certificate verification right; timeouts still apply.

## HTTPS server (file certs)

```
srv := &http.Server{Addr: ":8443", Handler: mux, ReadHeaderTimeout: 5 * time.Second}
log.Fatal(srv.ListenAndServeTLS("cert.pem", "key.pem"))
```

## Custom `tls.Config`

```
cfg := &tls.Config{
 MinVersion: tls.VersionTLS12,
 // Certificates: []tls.Certificate{cert},
 // GetCertificate: ... // SNI
 // ClientAuth: tls.RequireAndVerifyClientCert, // mTLS
}
ln, err := tls.Listen("tcp", ":8443", cfg)
```

## Client verification

```
tr := &http.Transport{
 TLSClientConfig: &tls.Config{
 MinVersion: tls.VersionTLS12,
 // RootCAs: pool, // private PKI
 // InsecureSkipVerify: true, // NEVER in production
 },
 ForceAttemptHTTP2: true,
}
client := &http.Client{Transport: tr, Timeout: 10 * time.Second}
```

## mTLS sketch

Server: `ClientAuth: tls.RequireAndVerifyClientCert + ClientCAs.`

Client: `Certificates: []tls.Certificate{clientCert}.`

See [171 TLS/mTLS deep dive](#).

## Pinning (careful)

Pinning SPKI hashes is brittle with rotation. Prefer public CA + short-lived certs or private PKI. If you pin, automate rotation.

## Rules of thumb

Do	Don't
MinVersion TLS1.2+	Skip verify to “fix CI” permanently
Timeouts on handshake path	Unlimited dial
Separate server/client certs for mTLS	Reuse one cert everywhere

## Try next

1. `mkcert` local certs; serve HTTPS.
2. Client against bad name → verify error.
3. Log peer cert CN on mTLS server.

## DNS Resolution Patterns

# DNS Resolution Patterns

## Overview

Every dial starts with a name. Bad DNS assumptions cause mysterious latency and sticky black holes. Control resolvers, dual-stack, and caching consciously.

## Default resolver

```
ips, err := net.DefaultResolver.LookupIPAddr(ctx, "example.com")
```

Always pass **context** with timeout.

## Custom resolver

```
r := &net.Resolver{
 PreferGo: true,
 Dial: func(ctx context.Context, network, address string) (net.Conn, error) {
 d := net.Dialer{Timeout: 2 * time.Second}
 return d.DialContext(ctx, "udp", "1.1.1.1:53")
 },
}
ips, err := r.LookupIPAddr(ctx, host)
```

## Dialer and Happy Eyeballs

```
d := net.Dialer{Timeout: 5 * time.Second, KeepAlive: 30 * time.Second}
conn, err := d.DialContext(ctx, "tcp", net.JoinHostPort(host, "443"))
```

Go's dialer handles much of dual-stack; still set overall timeouts.

## Caching

`net.Resolver` does not give you full app-level cache control. For high-QPS:

- Prefer connection reuse (HTTP Transport) over re-resolve every call
- Short-lived process caches with TTL if you roll your own (invalidate!)

## Failure modes

Symptom	Cause
Sporadic dial timeout	Resolver/network blip
Sticky bad IP	Long-lived conn to dead backend; need health + re-resolve
IPv6 blackhole	Broken AAAA path

## Minimal dig-like

```
for _, ip := range ips {
 fmt.Println(ip.IP)
}
```

(See CLI `netkit dns` command.)

## Rules of thumb

Do	Don't
ctx timeout on Lookup	Block forever on DNS
Re-resolve on new connections	Assume one IP forever for dynamic backends
Log resolved IP on errors	Only log hostname

## Try next

1. Lookup with 1ms timeout—handle error.
2. Compare `LookupIP` vs `LookupCNAME`.
3. Force custom DNS Dial to an internal resolver.

# HTTP Transport Tuning

# HTTP Transport Tuning

## Overview

`http.Transport` owns connection pooling, TLS, proxies, and HTTP/2. Defaults are fine for many apps; services with high fan-out need explicit limits.

## Sensible client

```
func NewClient() *http.Client {
 t := &http.Transport{
 Proxy: http.ProxyFromEnvironment,
 ForceAttemptHTTP2: true,
 MaxIdleConns: 100,
 MaxIdleConnsPerHost: 10,
 MaxConnsPerHost: 0, // 0 = unlimited; set in tight systems
 IdleConnTimeout: 90 * time.Second,
 TLSHandshakeTimeout: 5 * time.Second,
 ExpectContinueTimeout: 1 * time.Second,
 DialContext: (&net.Dialer{
 Timeout: 5 * time.Second,
 KeepAlive: 30 * time.Second,
 }).DialContext,
 }
 return &http.Client{Transport: t, Timeout: 15 * time.Second}
}
```

## One client per process

Reuse `*http.Client`. Creating per-request clients throws away the pool.



## Timeouts layers

```
Client.Timeout → entire request including body
Dialer.Timeout → TCP connect
TLSHandshakeTimeout → TLS
Response header → via context on request
```

Prefer `NewRequestWithContext` + outer timeout.

## Disable keep-alives (rare)

```
t.DisableKeepAlives = true // short CLI one-shots maybe
```

## HTTP/2

Enabled automatically for HTTPS with default transport settings. HTTP/2 multiplexes streams on one conn—still bound total concurrency at app layer.

## Rules of thumb

Do	Don't
Share Transport	<code>http.Get</code> without timeout forever
Cap per-host conns under load	Unlimited fan-out to one dependency
Close response bodies	Leak connections (unread body)

## Try next

1. Compare latency 100 serial gets with shared vs new client.
2. Set `MaxConnsPerHost: 2` and observe queuing.
3. Forget `Body.Close`; watch FD/conn growth.

## **Project: netcheck Toolkit**

# Project: netcheck Toolkit

## Overview

Build **netcheck**: combine TCP dial, TLS cert days, DNS lookup, and HTTP probe into one ops binary (sibling to CLI **netkit/netsec**).

## Commands

Cmd	Behavior
dial host:port	TCP connect RTT
dns name	print IPs
tls host	days left + version
http url	status + RTT
all host	dns + dial :443 + tls + https probe

## Requirements

- Context + 5s default timeout
- JSON optional (`-json`)
- Exit 1 if any check fails in `all`
- Stdlib + crypto/tls only

## Acceptance

```
go run ./cmd/netcheck all example.com
go run ./cmd/netcheck http -expect 200 https://example.com
```

# **Systems Programming**

# **Systems Programming Overview**

# Systems Programming Overview

This part covers how Go programs interact with the operating system: processes, signals, files, and stream-oriented command design. The emphasis is **operational correctness** — tools that behave predictably in production shells, CI, containers, and long-running supervisors.

Start with **Go as a systems language** (chapter 140a) if you want the *why*: on Linux, a `CGO_ENABLED=0` binary talks to the kernel itself and does not need `libc`. That is the property that makes the rest of this part deployable as one file.

## Why Systems Programming in a Go Book?

Go is often introduced as a cloud/API language. It is also a **userspace systems language**: the runtime issues Linux syscalls directly, so Docker, Kubernetes, and most of the tools in this part ship as a single static binary. In practice, that same binary may:

- Run as a PID 1-ish service under `systemd` or Kubernetes
- Manage child processes (plugins, helpers, compilers)
- Rewrite thousands of config files safely
- Sit in a Unix pipeline next to `grep` and `jq`

If your HTTP handlers are solid but your process mishandles `SIGTERM`, you still page at 3 a.m.

```

 +-----+
signals -----> | Go process | ----> child processes
stdin/stdout --> | main + runtime | ----> filesystem
env/config ----> | | ----> network (other parts)
 +-----+
```

## Learning Path

Chapter	Focus	Outcome
140a Go as a systems language	No <code>libc</code> on Linux, <code>CGO_ENABLED=0</code> , Go vs C	Why a Go binary runs on Alpine, <b>scratch</b> , and NixOS
141 Processes & signals	Lifecycle, graceful stop, supervisors	Clean shutdown and restart policy
142 Safe filesystem IO	Atomic writes, path jails, fsync tradeoffs	Automation that does not corrupt state

Chapter	Focus	Outcome
143 Unix pipeline CLIs	stdin/stdout/stderr contracts, exit codes	Tools that compose in shell and CI
144 Env, user, CWD, umask	Identity and path resolution	Correct behavior under systemd/CI
145 File descriptors	FDs, pipes, inheritance, limits	No leaks; correct redirection
146 Memory-mapped files	mmap RO/RW tradeoffs	Random access without ReadAll
147 Unix domain sockets	Local IPC, HTTP over UDS	Sidecars without TCP ports
148 Locks & pidfiles	flock, single-instance tools	Safe cron/agents
149 Project: systool	Multi-command mini-tools	Capstone practice
150 Time & timers	Wall vs monotonic, ticker hygiene	Correct deadlines
151 Filesystem watching	Poll/fsnotify, debounce, reload	Hot config without races
152 Pseudo-terminals	PTY vs pipes, interactive wrap	Automate tty-only tools carefully
153 Resource limits	rlimit, GOMEMLIMIT, caps	Survive cgroup ceilings
154 Privileges	Non-root, capabilities, modes	Least privilege defaults
155 Ops logging	stderr/journal/syslog	Findable production logs
156 Project: sysops	deadline, reload, cronish, ...	Ops toolkit capstone
157 Socket activation	systemd LISTEN_FDS	Privilege-separated bind
158 Linux procfs	/proc/self, cgroup peeks	Debug introspection
159 Project: minisup	multi-process supervisor	Restart + signal fanout

## Core Concepts Map

### Process as a state machine

```

starting -> running -> stopping -> exited
 | ^
 +-> crash -+ (restart policy lives here)

```

Every production process needs explicit answers for: how do I start, how do I stop, what is my exit code vocabulary, and who restarts me?

### The filesystem is not a database (but you can still be careful)

Partial writes and torn updates are normal under crash. Atomic rename patterns and directory fsync exist because people lost data.

## CLI tools are network services for humans and scripts

Stable stdout, noisy stderr, and meaningful exit codes are the “API” of a CLI. Breaking them breaks automation.

## Go Building Blocks

Area	Packages
Process / exec	os/exec, os, syscall (sparingly)
Signals	os/signal, signal.NotifyContext
Files	os, io, io/fs, path/filepath, embed
Env / identity	os.Getenv, os/user, os.UserHomeDir
FDs / pipes	os.Pipe, os.File.Fd, cmd.ExtraFiles
Local IPC	net.Listen("unix", ...), unixgram
Locks / mmap	golang.org/x/sys/unix (Flock, Mmap)
Time	time, context deadlines
Watch / reload	poll Stat, optional fsnotify
PTY	creack/pty (when needed)
Limits	RLIMIT_*, GOMEMLIMIT, GOMAXPROCS
Permissions	os.FileMode, platform differences
Structured ops logs	log/slog on stderr; optional log/syslog

## Suggested schedule

Day 0	140a	why Go binaries need no libc (Linux)
Days 1-2	141 → 142 → 143	core lifecycle + files + pipelines
Day 3	144 → 145	env/cwd + file descriptors
Day 4	146 → 147 → 148	mmap, UDS, locks
Day 5	149	build systool
Day 6	150 → 151 → 152	time, watch, PTY
Day 7	153 → 154 → 155	limits, privilege, logging
Day 8	156	build sysops toolkit
Day 9	157 → 158 → 159	activation, procfs, minisup

## Modern signal handling skeleton

```
ctx, stop := signal.NotifyContext(context.Background(), os.Interrupt, syscall.SIGTERM)
defer stop()

// pass ctx to servers, workers, exec.CommandContext
<-ctx.Done()
```



```
// drain with a second hard-deadline context
```

Prefer `signal.NotifyContext` over manual channel plumbing in new code (Go 1.16+).

## Production Mindset

Ask for every systems feature:

1. **What happens on SIGTERM?** (deadline? drain? exit code?)
2. **What happens if power fails mid-write?**
3. **Can a malicious path escape the working root?**
4. **Can this tool run non-interactively in CI?**
5. **What do metrics/logs say when a child exits 137 (OOM)?**

## Relationship to Other Parts

- **Network systems (13)** — servers need the shutdown patterns from 141.
- **Distributed infra (15)** — workers are processes with queues and backpressure.
- **Security (11, 17)** — path safety and least privilege are hardening.
- **Projects (99)** — many Linux-clone labs exercise these chapters.

## How to Study

1. Implement a long-running worker that exits cleanly on SIGTERM within N seconds.
2. Write a config rewriter that survives `kill -9` mid-write (atomic replace).
3. Build a filter CLI used as `cat data.log | yourtool | jq`.

## Literacy Checklist

- ☐ Explain why a Linux Go binary can omit libc, and when cgo puts it back
- ☐ Explain SIGTERM vs SIGKILL
- ☐ Use `CommandContext` so children die with parent cancel
- ☐ Perform write-temp-fsync-rename safely
- ☐ Jail user paths under a root directory
- ☐ Design stdout/stderr/exit-code contracts for scripting
- ☐ Avoid shell injection via `sh -c` with untrusted input
- ☐ Reason about CWD vs binary-relative paths under `systemd`
- ☐ Avoid FD leaks; know stdin/stdout/stderr roles
- ☐ When to mmap vs stream reads
- ☐ Serve or dial a Unix domain socket safely (permissions)
- ☐ Single-instance exclusion with flock (not pidfile alone)
- ☐ Use monotonic-friendly timing and cancelable timers

- ☐ Hot-reload config with debounce and last-good retention
- ☐ Know when a PTY is required vs a plain pipe
- ☐ Name the limit behind EMFILE / OOMKill
- ☐ Prefer non-root + clear EPERM errors
- ☐ Log to stderr/journal with structure; no secrets

## Part Warm-Up Exercises

1. On a sample service, document current signal handling (or lack of it).
2. Trace what Kubernetes sends on pod termination and the `terminationGracePeriodSeconds` interaction.
3. Find one script in your life that greps program stdout for errors — that program violated the stderr rule.
4. List three files your systems write that would hurt if truncated mid-write.
5. Compare `os.Exit` in library code vs returning errors to `main`.

## Platform Notes (Unix-first)

These chapters assume a Unix-like environment (Linux, macOS, BSD) because that is where most Go servers and CLIs run in production. Windows supports many of the same Go APIs (`os/exec`, signals subset, paths) but:

- Signal semantics differ (no true `SIGTERM` from all supervisors)
- Atomic rename and file locking differ
- Path separators and reserved names differ

When writing cross-platform tools, isolate OS-specific code behind small files (`path_unix.go` / `path_windows.go`) and test both in CI.

## Container and CI Reality

Environment	Systems concern
Kubernetes pod	<code>SIGTERM</code> + grace period; probes; non-root
systemd unit	<code>Type=notify</code> optional; restart policy
GitHub Actions	non-interactive; no TTY; pipe stdout carefully
Developer laptop	<code>Ctrl-C</code> = <code>SIGINT</code> ; human-readable defaults OK

Design for the **non-interactive** case first; make interactive polish optional.

## Mini Project Thread

Build one coherent ops utility across chapters:

1. **141** — long-running worker with graceful shutdown
2. **142** — atomically persists checkpoints
3. **143** — also usable as a filter in a pipeline
4. **144–145** — correct CWD/env and FD hygiene
5. **147–148** — optional control socket + single-instance lock
6. **149** — assemble as multi-command **systool**
7. **150–155** — deadlines, hot reload, limits, least privilege, logging
8. **156** — assemble **sysops** (deadline, reload, cronish, healthsock)

That combination is the skeleton of many internal ops utilities.

## Failure Stories Worth Remembering

- Config rewrite crashed mid-write → empty file → mass restart loop
- CLI printed logs to stdout → jq parse failures in CI
- Parent killed without waiting → zombie children filled PID table
- Command with `sh -c` and user input → shell injection

Each chapter exists to prevent one of these classes of failure.

## Checklist Before Shipping a Systems Tool

- ☐ `main` returns errors; `os.Exit` only at the top
- ☐ SIGTERM/SIGINT cancel a root context
- ☐ Children use `CommandContext` or explicit term/kill
- ☐ Durable state uses atomic write patterns
- ☐ User paths cannot escape the data root
- ☐ stdout is data; stderr is diagnostics
- ☐ Exit codes documented for automation
- ☐ Works with stdin piped (no TTY required)
- ☐ Logs never include secrets from env files

## Sample Directory Layout

```
cmd/mytool/main.go
internal/app/run.go
internal/app/run_test.go
internal/fsutil/atomic.go
internal/fsutil/path.go
```

Keep OS edge cases in `fsutil`; keep business logic testable with `io.Reader/Writer`.

## How This Part Connects to Network Services

Your HTTP server is a process:

- Chapter 141 shutdown patterns plug into `http.Server.Shutdown`
- Chapter 142 patterns protect TLS key material drops and config reloads
- Chapter 143 discipline applies to `kubectl`-style companion CLIs you ship beside the service

Do not treat “systems programming” as only embedded or C territory — production Go is systems software. Chapter 140a spells out where Go replaces C (portable userspace) and where it does not (kernel, hard real-time).

## Glossary

- **Graceful shutdown** — stop accepting; drain; then exit.
- **Atomic replace** — rename-based file update without torn reads.
- **Path jail** — force paths under a root directory.
- **Exit code** — process-level API for scripts and supervisors.
- **SIGPIPE** — write to closed pipe; common in `| head` pipelines.
- **PID 1** — init process in a container; reaps children / signal quirks.
- **TOCTOU** — time-of-check to time-of-use race on files.

## More examples

### Tour: signal context + atomic temp file

```
mkdir -p /tmp/go-sys-tour && cd /tmp/go-sys-tour
go mod init example.com/sys-tour
```

Save as `main.go`:

```

package main

import (
 "context"
 "fmt"
 "os"
 "os/signal"
 "path/filepath"
 "syscall"
 "time"
)

func atomicWrite(path, body string) error {
 dir := filepath.Dir(path)
 f, err := os.CreateTemp(dir, ".tmp-*")
 if err != nil {
 return err
 }
 tmp := f.Name()
 if _, err := f.WriteString(body); err != nil {
 f.Close()
 os.Remove(tmp)
 return err
 }
 if err := f.Close(); err != nil {
 os.Remove(tmp)
 return err
 }
 return os.Rename(tmp, path)
}

func main() {
 ctx, stop := signal.NotifyContext(context.Background(), os.Interrupt, syscall.SIGTERM)
 defer stop()
 go func() {
 time.Sleep(20 * time.Millisecond)
 stop()
 }()
 <-ctx.Done()
 fmt.Println("signal path:", ctx.Err())

 dir, _ := os.MkdirTemp("", "sys")
 defer os.RemoveAll(dir)
 path := filepath.Join(dir, "cfg.json")
 if err := atomicWrite(path, `{"ok":true}`); err != nil {
 panic(err)
 }
}

```

```

 b, _ := os.ReadFile(path)
 fmt.Println("file:", string(b))
}

go run .

```

### Expected output:

```

signal path: context canceled
file: {"ok":true}

```

## Runnable example

Tour of this part: print process identity, catch a simulated shutdown via context, and write a line to stdout with a non-zero exit path—the process contract network services and CLIs share.

```

mkdir -p /tmp/go-sys-tour && cd /tmp/go-sys-tour
go mod init example.com/sys-tour

```

Save as `main.go`:

```

package main

import (
 "context"
 "fmt"
 "os"
 "time"
)

func run(ctx context.Context) error {
 fmt.Fprintf(os.Stdout, "pid=%d\n", os.Getpid())
 select {
 case <-ctx.Done():
 fmt.Fprintln(os.Stderr, "shutting down:", ctx.Err())
 return nil
 case <-time.After(50 * time.Millisecond):
 fmt.Fprintln(os.Stdout, "work done")
 return nil
 }
}

func main() {
 ctx, cancel := context.WithTimeout(context.Background(), 100*time.Millisecond)
 defer cancel()

```

```

 if err := run(ctx); err != nil {
 fmt.Fprintln(os.Stderr, err)
 os.Exit(1)
 }
 fmt.Fprintln(os.Stdout, "tour: process + context + exit codes")
}

go run .

```

### Expected output:

```

pid=<n>
work done
tour: process + context + exit codes

```

### What to notice

- Stdout is for data/results; stderr for diagnostics—pipeline-friendly tools depend on that split.
- Context cancellation is the in-process form of SIGTERM handling (chapter 141).

### Try next

- Wire `signal.NotifyContext` and send `Ctrl-C`.
- Continue with atomic file writes (142) and stdin filters (143).

## Related in this book

- How Go reaches the kernel / static binaries: [140a Go as a Systems Language](#)
- Low-level syscalls / eBPF / Delve: [92 Systems Programming \(specialized\)](#)
- CLI curriculum (flags, Cobra, net tools): [23 CLI Tools](#)
- Runtime deep dives: [20 Go Deep Dives](#)

## Next Chapter

**Processes, Signals, and Supervisors** — the lifecycle model everything else hangs on.

## **Go as a Systems Language**



# Go as a Systems Language

Go is often introduced as a cloud and API language. That understates the design. Docker, containerd, Kubernetes, etcd, Prometheus, and Terraform are **userspace systems software**: they create processes, talk to the kernel, ship as a single file, and have to run on machines the author never configured.

This chapter is the *why* behind that property. Later chapters in this part teach processes, files, pipelines, and supervisors. Here you learn how a Go binary reaches the kernel **without libc**, why that matters for portability, and when C is still the right tool.

Structure inspired by public systems writeups (notably [Using Go for Systems Programming](#) on iximiuz Labs). Prose, diagrams, and examples here are original to this book.

## Why this belongs in a Go book

A C programmer learning process creation, `mmap`, or file descriptors usually goes through **libc**: the C standard library. POSIX is specified in C. The Linux kernel is written in C. libc is the conventional bridge from userspace to the kernel.

Go takes a different route **on Linux**: the runtime and standard library issue system calls themselves. The resulting binary does not need `libc.so` on the target. That is why a `CGO_ENABLED=0` Linux binary can run on Debian, Alpine, NixOS, or a **scratch** container as long as the kernel ABI matches.

You still cannot write kernel modules or firmware in Go. “Systems language” here means **portable userspace that speaks the OS**, not “replaces C everywhere.”

```
C (typical Linux build)

your code --> libc (printf, malloc, fopen)
 |
 v
 kernel (write, brk/mmap, open)

Go on Linux (CGO_ENABLED=0)

your code --> Go stdlib + runtime
 |
 v
 kernel (same syscalls, no libc in the middle)
```

## How a C program depends on libc

`printf` is not a system call. It is a libc function. libc formats the string, then eventually invokes the kernel's `write`. `malloc` eventually becomes `brk` or `mmap`. `fopen` becomes `open`. Your C source talks to libc; libc talks to the kernel.

A default `gcc hello.c -o hello` on Linux produces a **dynamically linked** ELF. The binary does not contain libc. At start-up the kernel loads a **dynamic linker** (the ELF interpreter) which then maps shared libraries:

Distro family	Dynamic linker you typically see
Debian, Ubuntu, Fedora, RHEL (glibc)	<code>/lib64/ld-linux-x86-64.so.2</code>
Alpine (musl)	<code>/lib/ld-musl-x86_64.so.1</code>

Inspect it:

```
file ./hello
ELF 64-bit ... dynamically linked, interpreter /lib64/ld-linux-x86-64.so.2 ...

ldd ./hello
libc.so.6 => /lib/x86_64-linux-gnu/libc.so.6
```

`ldd` lists shared-library dependencies. If the interpreter path or a required `.so` is missing, the program does not run.

### The “not found” surprise

Copy a glibc-linked binary onto Alpine and run it. The shell often prints **not found** even though the file exists and is executable. The kernel is not looking for *your* file — it is looking for the **interpreter** named in the ELF header (`/lib64/ld-linux-x86-64.so.2`). Alpine does not ship that file.

glibc and musl implement the same C API on paper and are **not binary-compatible**. Symbol versions differ. Dynamic linker paths differ. A dynamically linked C tool built on Ubuntu is not a portable Linux tool.

You *can* statically link C (`gcc -static`). That removes the runtime `.so` search, but the binary still embeds **one** libc's implementation and assumptions. It is larger, licensing of glibc-static is awkward, and you still do not get Go's own scheduler, allocator, and syscall wrappers.

## How a Go program reaches the kernel

Compile a tiny program with the Go toolchain:

```
package main

import "fmt"

func main() {
 fmt.Println("hello from a self-contained binary")
}
```

```
CGO_ENABLED=0 go build -o hello-go .
file hello-go
On Linux: ELF 64-bit ... statically linked ...
```

**Statically linked** means the kernel can map this one file and run it. There is no ELF interpreter for libc, no `libc.so` to locate.

```
ldd hello-go
Linux glibc ldd: "not a dynamic executable" or "statically linked"
Alpine musl ldd: "Not a valid dynamic program"
```

Both messages mean the same thing: there is no dynamic section to inspect.

Copy the binary to `/tmp` on a machine that does not have the Go toolchain installed. It still runs. The host libc does not matter. The kernel syscall interface does.

## What is actually inside

Every Go program includes the **Go runtime**:

- goroutine scheduler (Gs on Ms, with Ps)
- memory allocator and garbage collector
- signal handling and stack growth
- the assembly stubs that place syscall arguments in the right registers and execute `syscall` (Linux)

On Linux those stubs live in the standard library / runtime (`Syscall`, `RawSyscall`, and architecture-specific `asm_linux_*.s` files). You do not call them for ordinary I/O — `os`, `fmt`, `net`, and `io` do.

`strace` makes the difference visible. A C `printf` hello shows a short list (`write`, `exit_group`, a bit of libc setup). A Go hello shows more: `mmap`, `mprotect`, `clone`, `futex`, `rt_sigaction` — the runtime bringing up its own world — then `write` for the line of text. Those calls originate in the Go binary, not in libc.

```
C hello

loader + libc init
write("hello\n")
exit_group
```

```
Go hello (Linux, no cgo)

runtime init (mmap, clone, futex, ...)
write("hello\n")
exit_group
```

## Linux vs other kernels

This “no libc” story is **Linux-specific**:

GOOS	How Go talks to the OS
linux	Direct syscalls; <code>CGO_ENABLED=0</code> binaries are fully static
darwin	Through <code>libSystem</code> — Apple does not treat the raw syscall table as a stable ABI
windows	Windows APIs / <code>ntdll</code> , not a Unix libc
freebsd / others	OS-specific; do not assume Linux static behaviour

The portability win that made Go famous in containers — one file, Alpine or Debian, **scratch** image — is the Linux + `CGO_ENABLED=0` combination.

## CGO puts libc back in the picture

**cgo** is how Go calls C. When it is enabled, the linker may pull in libc and you are back to dynamic dependencies.

On a machine with a C compiler, `CGO_ENABLED` defaults to 1. A package that looks like pure Go can still pick up `cgo` through `net` (system DNS resolver) or `os/user` (NSS lookups) unless you disable it.

```
Guarantee a pure-Go, fully static Linux binary:
CGO_ENABLED=0 go build -o hello-go .

Confirm what the toolchain actually used:
go env CGO_ENABLED
go version -m hello-go | grep CGO
```

With `CGO_ENABLED=0`, networking and user lookup use **pure-Go implementations** (Go’s DNS resolver, Go’s `/etc/passwd` parsing). That is usually what you want for containers and cross-compiled CLIs. Use `cgo` when you *must* call a C library that has no acceptable Go equivalent — and then accept a C toolchain, harder cross-compiles, and a libc on the target. See [cgo](#).

## Cross-compilation is a systems feature

Because the toolchain ships the runtime and standard library for many `GOOS/GOARCH` pairs, you can build a Linux AMD64 binary on a Mac or a Windows box:

```
GOOS=linux GOARCH=amd64 CGO_ENABLED=0 go build -o hello-linux-amd64 .
GOOS=linux GOARCH=arm64 CGO_ENABLED=0 go build -o hello-linux-arm64 .
```

The compiler emits **Plan 9 pseudo-assembly** (an architecture-neutral IR with SB for static base, SP for the stack) and the Go linker writes the final ELF/Mach-O/PE file. You do not need a cross-`gcc` unless `cgo` is on.

Inspect what the compiler produced:

```
go build -gcflags=-S . # Plan 9-ish listing on stderr
go tool objdump -s main.main hello-linux-amd64
```

`go tool objdump` understands every architecture the toolchain can emit. You can disassemble an ARM64 binary on an x86-64 laptop; the text uses Go’s register names (R0..., R28 as the current `g` on ARM64), not necessarily the vendor ISA manual’s names.

Deeper ABI and NOSPLIT notes live in [ABI, assembly, and go:nosplit](#).

## Trade-offs: Go vs C for systems work

### Why teams pick Go

Strength	What it buys you
Static Linux binaries	One file; no glibc/musl lottery; <b>scratch</b> / distroless images
Memory safety by default	Bounds checks, no pointer arithmetic; fewer class-of-bug CVEs
Goroutines	Concurrent servers and CLIs without hand-rolled pthreads
Fast builds + first-class cross-compile	CI matrix without a C cross-toolchain
Batteries in the stdlib	HTTP, TLS, JSON, crypto — no hunt for a hash map

That combination is why so much **cloud infrastructure** is Go. The programs need to run in many environments, talk to the kernel, and stay easy to build.

### What you pay

Cost	Consequence
Runtime + GC in every binary	Larger files; CPU/memory for the scheduler and collector
GC is not hard-real-time	Fine for almost all servers and CLIs; wrong for hard $\mu$ s deadlines
Less control of layout	C still wins for packed DMA structures and bare metal
Not for the kernel	Linux modules and firmware stay C (or Rust, in some trees)

The Go GC has improved a lot (including the Green Tea collector becoming the default in Go 1.26). “GC pause” is rarely the reason to reject Go for a network service. It *is* a reason to reject Go for a hard real-time controller.

## Choose C when

- You are writing kernel code, a driver, or firmware
- Every byte and cycle is budgeted (tiny MCUs, some embedded)
- You need deterministic allocation with no collector
- You must control exact memory layout and registers

## Choose Go when

- You are building network services, CLIs, or DevOps / cloud control planes
- You want safe concurrency without managing OS threads by hand
- You need one Linux binary that runs across distros and in **scratch**
- You would rather not manage memory by hand

Go does not replace C. It occupies the large space of **userspace systems software** where C's portability tax is higher than Go's runtime tax.

## Literacy checklist

- ☐ Explain why `printf` is not a syscall
- ☐ Name glibc vs musl and why a dynamically linked Ubuntu binary fails on Alpine
- ☐ Read `file` and `ldd` well enough to spot a libc dependency
- ☐ Explain why `./binary: not found` can mean “missing dynamic linker”
- ☐ Build with `CGO_ENABLED=0` and know what it changes (DNS, `os/user`, linkage)
- ☐ Cross-compile with `GOOS` / `GOARCH` without a C cross-compiler
- ☐ Know that the no-libc story is Linux-specific (Darwin uses `libSystem`)
- ☐ List one reason to pick C anyway (kernel, hard real-time, layout)

## Exercises

1. **Link story** — On Linux, compile a 5-line C hello and a 5-line Go hello. Run `file` and `ldd` on both. Write one sentence about each output.
2. **Force static Go** — Build the same Go program twice, once with `CGO_ENABLED=0` and once with the default. Compare `ldd` and `ls -l`. If they match, check `go env CGO_ENABLED` and whether any imported package uses `cgo`.
3. **Cross-compile** — From whatever OS you use, produce `GOOS=linux GOARCH=amd64 CGO_ENABLED=0` and `GOOS=linux GOARCH=arm64 CGO_ENABLED=0` binaries. Disassemble `main.main` of both with `go tool objdump -s main.main`.
4. **strace (Linux)** — `strace -c` both hellos. Which extra syscalls does the Go runtime need just to start?
5. **The Alpine thought experiment** — Without a spare Alpine box: given an ELF interpreter of `/lib64/ld-linux-x86-64.so.2`, predict the exact failure mode on a musl-only rootsfs.

## Runnable example

This program reports what the binary believes about its build, then writes one line. After `go build`, inspect the artifact with `file` / `ldd` (Linux) or `otool -L` (macOS).

```
mkdir -p /tmp/go-as-sys && cd /tmp/go-as-sys
go mod init example.com/as-sys
```

Save as `main.go`:

```
package main

import (
 "fmt"
 "os"
 "runtime"
 "runtime/debug"
)

func cgoEnabled() string {
 info, ok := debug.ReadBuildInfo()
 if !ok {
 return "unknown"
 }
 for _, s := range info.Settings {
 if s.Key == "CGO_ENABLED" {
 return s.Value
 }
 }
 return "unknown"
}

func main() {
 fmt.Fprintf(os.Stdout, "goos=%s goarch=%s cgo=%s\n",
 runtime.GOOS, runtime.GOARCH, cgoEnabled())
 fmt.Fprintln(os.Stdout, "this write is a kernel syscall on Linux - no libc required")
}
```

```
CGO_ENABLED=0 go build -o as-sys .
./as-sys
```

```
Linux:
file ./as-sys
ldd ./as-sys || true
```

```
Cross-compile a Linux static binary from any host:
```

```
GOOS=linux GOARCH=amd64 CGO_ENABLED=0 go build -o as-sys-linux-amd64 .
file ./as-sys-linux-amd64
```

**Expected output** (host fields vary):

```
goos=linux goarch=amd64 cgo=0
this write is a kernel syscall on Linux - no libc required
```

### What to notice

- `runtime.GOOS` is the *target* you compiled for, not necessarily the machine you typed on.
- `CGO_ENABLED=0` is a build-time switch; `ReadBuildInfo` records what the linker used.
- On Linux, `file` should say statically linked. On macOS the same source still links `libSystem` — that is Darwin, not a failed build.

### Try next

- `go tool objdump -s main.main ./as-sys` and find the `CALL` to `fmt.Fprintf`.
- Continue with [Processes, signals, and supervisors](#): the binary you just built *is* the process those chapters supervise.

## Related in this book

- [Why Go exists](#) — single-binary design goal
- [cgo](#) — how libc comes back
- [Building and releasing](#) — `GOOS` / `GOARCH` / strip flags
- [Containerization](#) — `scratch` and distroless
- [Go on NixOS and FreeBSD](#) — static binaries vs missing FHS loaders
- [Specialized: syscalls, Delve, eBPF](#)
- [Linking and build modes](#)
- [ABI and assembly](#)

## Next Chapter

**Systems Programming Overview** — map of processes, files, pipelines, and the ops toolkit. Then start with processes and signals.



## **Processes, Signals, and Supervisors**

# Processes, Signals, and Supervisors

Process supervision is about controlling **state transitions under uncertainty**. Crashes, deploys, OOMs, and hung IO all force transitions; production software defines what happens next.

## Why / Overview

A Go binary is a process — on Linux, usually a **self-contained** one ([Go as a Systems Language](#)). Orchestrators (systemd, Kubernetes, Nomad, Docker) are supervisors. Your code must cooperate with them:

- Start quickly enough to pass readiness
- Serve while healthy
- On stop signal: drain work, close listeners, exit
- On failure: exit with a meaningful code so the supervisor can decide restart policy

```
starting -> running -> stopping -> exited
 | ^
 +--> crash--+
```

## Signals You Must Handle

Signal	Typical meaning	Your response
SIGTERM	Graceful stop (K8s, systemd)	Drain + exit 0 if clean
SIGINT	Ctrl-C	Same as SIGTERM for services
SIGHUP	Reload config (classic Unix)	Optional: reload, or ignore
SIGKILL	Hard kill (uncatchable)	Cannot handle; keep critical writes atomic
SIGPIPE	Write to closed pipe	Often ignored; CLI tools should handle EPIPE

```
ctx, stop := signal.NotifyContext(context.Background(), os.Interrupt, syscall.SIGTERM)
defer stop()
```

`NotifyContext` cancels when a signal arrives. Use that `ctx` everywhere: HTTP servers, worker loops, `exec.CommandContext`.

## Graceful Shutdown Pattern

```
package main

import (
 "context"
 "errors"
 "log/slog"
 "net/http"
 "os"
 "os/signal"
 "syscall"
 "time"
)

func main() {
 ctx, stop := signal.NotifyContext(context.Background(), os.Interrupt, syscall.SIGTERM)
 defer stop()

 mux := http.NewServeMux()
 mux.HandleFunc("GET /work", func(w http.ResponseWriter, r *http.Request) {
 // Honor cancellation so Shutdown can finish.
 select {
 case <-r.Context().Done():
 return
 case <-time.After(2 * time.Second):
 w.Write([]byte("done"))
 }
 })

 srv := &http.Server{Addr: ":8080", Handler: mux, ReadHeaderTimeout: 5 * time.Second}

 errCh := make(chan error, 1)
 go func() {
 slog.Info("listening", "addr", srv.Addr)
 errCh <- srv.ListenAndServe()
 }()

 select {
 case <-ctx.Done():
 slog.Info("signal received, shutting down")
 case err := <-errCh:
 if err != nil && !errors.Is(err, http.ErrServerClosed) {
 slog.Error("server failed", "err", err)
 os.Exit(1)
 }
 }
```

```

 }

 shutdownCtx, cancel := context.WithTimeout(context.Background(), 15*time.Second)
 defer cancel()
 if err := srv.Shutdown(shutdownCtx); err != nil {
 slog.Error("graceful shutdown failed", "err", err)
 _ = srv.Close()
 os.Exit(1)
 }
 slog.Info("bye")
}

```

## Two-phase stop

1. **Soft deadline** (e.g. 15s): stop accepting, drain in-flight.
2. **Hard deadline**: cancel contexts, close forcibly, exit non-zero if work abandoned.

Kubernetes: `terminationGracePeriodSeconds` must be **greater** than your soft deadline, or you get SIGKILL mid-drain.

## Exit Codes as API

Code	Convention
0	Success
1	General failure
2	Misuse / bad flags (CLI)
130	Interrupted by SIGINT (128+2) often
137	SIGKILL / OOM kill (128+9) often
143	SIGTERM (128+15) often

Supervisors and CI branch on these. Do not `os.Exit(0)` after a failed deploy migration.

```

func main() {
 if err := run(); err != nil {
 slog.Error("exit", "err", err)
 os.Exit(1)
 }
}

```

Keep `os.Exit` in main only so deferred cleanup runs.

## Child Processes

### Always use context

```
cmd := exec.CommandContext(ctx, "ffmpeg", args...)
cmd.Stdout = &buf
cmd.Stderr = &errBuf
if err := cmd.Run(); err != nil {
 return fmt.Errorf("ffmpeg: %w: %s", err, errBuf.String())
}
```

When `ctx` cancels, Go sends SIGKILL to the child by default (after `CommandContext` cancel). For graceful child stop, start with `cmd.Start()`, signal SIGTERM, wait, then kill.

```
func runGraceful(ctx context.Context, name string, args ...string) error {
 cmd := exec.Command(name, args...)
 if err := cmd.Start(); err != nil {
 return err
 }
 done := make(chan error, 1)
 go func() { done <- cmd.Wait() }()

 select {
 case err := <-done:
 return err
 case <-ctx.Done():
 _ = cmd.Process.Signal(syscall.SIGTERM)
 select {
 case err := <-done:
 return err
 case <-time.After(5 * time.Second):
 _ = cmd.Process.Kill()
 return <-done
 }
 }
}
```

### Avoid shell

```
// BAD: injection + opaque failures
// exec.Command("sh", "-c", "do-thing "+user)

// GOOD
exec.CommandContext(ctx, "/usr/bin/do-thing", userArg)
```

## Process groups

When your child spawns grandchildren, killing only the direct child can leave orphans. On Unix, set a process group and signal the group (advanced; platform-specific). Document behavior for plugin systems.

## Supervisor Policies

Whether you write a supervisor or rely on K8s:

Policy	Behavior	Risk
Always restart	Crash loop	Masks config bugs; thrashes
On-failure	Restart non-zero only	Good default
Backoff + jitter	Delay restarts	Prevents restart storms
Max attempts	Give up / page	Surfaces poison configs
Rate limit	N restarts / window	Protects cluster

```
crash -> wait min(cap, base*2^n) + jitter -> restart
 if n > max: alert and stop
```

**Never** restart blindly without backoff when the crash cause is bad config — you will burn CPU and spam logs.

## Readiness vs Liveness (process view)

- **Liveness failure** → supervisor restarts process (hung deadlock).
- **Readiness failure** → leave process up but remove from LB (warming, dependency down).

Implementing readiness that always equals “process started” defeats the purpose. See distributed infra health-check chapter for HTTP shapes.

## Observability for Process Lifecycle

Emit structured events:

```
slog.Info("process_start", "version", version, "pid", os.Getpid())
slog.Info("signal", "sig", "SIGTERM")
slog.Info("shutdown_begin")
slog.Info("shutdown_end", "dur_ms", dur.Milliseconds(), "exit_hint", 0)
```

Metrics: `process_start_time`, `shutdown_duration_seconds`, `child_exit_total{code,cmd}`, restart counts if you supervise.

## PID 1 and Containers

In containers, your app may be PID 1. PID 1 has special signal and zombie-reaping responsibilities. Prefer a tiny init (`tini`, `dumb-init`) or ensure you wait on children. Go apps that spawn children without reaping can accumulate zombies.

## Production Checklist

- ☐ `signal.NotifyContext` for `SIGTERM/SIGINT`
- ☐ Soft shutdown deadline < orchestrator grace period
- ☐ Handlers/workers honor context cancel
- ☐ Children launched with `CommandContext` or explicit `term/kill`
- ☐ No `os.Exit` in libraries
- ☐ Meaningful exit codes
- ☐ Restart backoff at supervisor layer
- ☐ Logs for signal, shutdown duration, child failures
- ☐ Liveness readiness semantics documented

## Common Pitfalls

1. **Ignoring `SIGTERM`** — K8s waits, then `SIGKILL` mid-request.
2. **Shutdown longer than grace period** — always killed.
3. **Goroutines without context** — drain never finishes.
4. **`log.Fatal` in hot paths** — skips defers.
5. **Restart loop on bad config** — fix with crash backoff + config validation at start.
6. **Orphan children** after parent exit.
7. **Treating OOM kills as app bugs only** — sometimes limits are wrong; still exit 137 is a signal.

## Exercises

1. Write a server that sleeps 10s per request; send `SIGTERM`; show that without context the process hangs until kill.
2. Fix it with context-aware handlers and `Shutdown`; measure drain time.
3. Spawn `sleep 60` with `CommandContext`; cancel after 1s; confirm child dies.
4. Implement graceful child termination (`SIGTERM` then `SIGKILL`).
5. Simulate crash loop with a binary that exits 1; write a supervisor with exponential backoff + max 5 attempts.
6. Document your cluster's `terminationGracePeriodSeconds` vs app shutdown timeout.
7. Create a child that spawns another child; observe zombies without `Wait`; fix.
8. Map exit codes in CI for a CLI tool (0/1/2).
9. Add slog events for start/signal/shutdown; reconstruct a deploy timeline from logs alone.
10. Compare systemd `Restart=on-failure` vs Kubernetes `restartPolicy` for the same binary.

## More examples

### HTTP graceful shutdown on signal context

```
mkdir -p /tmp/go-graceful-http && cd /tmp/go-graceful-http
go mod init example.com/graceful-http
```

Save as main.go:

```
package main

import (
 "context"
 "fmt"
 "net"
 "net/http"
 "os"
 "os/signal"
 "syscall"
 "time"
)

func main() {
 ctx, stop := signal.NotifyContext(context.Background(), os.Interrupt, syscall.SIGTERM)
 defer stop()

 mux := http.NewServeMux()
 mux.HandleFunc("GET /ok", func(w http.ResponseWriter, r *http.Request) {
 fmt.Fprintln(w, "ok")
 })
 ln, err := net.Listen("tcp", "127.0.0.1:0")
 if err != nil {
 panic(err)
 }
 srv := &http.Server{Handler: mux}

 errCh := make(chan error, 1)
 go func() { errCh <- srv.Serve(ln) }()

 // Simulate orchestrator SIGTERM without needing an external kill.
 go func() {
 time.Sleep(30 * time.Millisecond)
 stop()
 }()

 resp, err := http.Get("http://" + ln.Addr().String() + "/ok")
```



```

 if err != nil {
 panic(err)
 }
 resp.Body.Close()
 fmt.Println("served:", resp.StatusCode)

 <-ctx.Done()
 shCtx, cancel := context.WithTimeout(context.Background(), 200*time.Millisecond)
 defer cancel()
 if err := srv.Shutdown(shCtx); err != nil {
 panic(err)
 }
 fmt.Println("shutdown:", <-errCh)
}

go run .

```

### Expected output:

```

served: 200
shutdown: http: Server closed

```

### Exit codes for CLI contract

```

mkdir -p /tmp/go-exit-codes && cd /tmp/go-exit-codes
go mod init example.com/exit-codes

```

Save as main.go:

```

package main

import (
 "errors"
 "fmt"
 "os"
)

func run(args []string) error {
 if len(args) < 1 {
 return errUsage
 }
 if args[0] == "fail" {
 return errors.New("runtime boom")
 }
 fmt.Println("ok:", args[0])
 return nil
}

```

```

}

var errUsage = errors.New("usage: tool <name>")

func main() {
 // Demo mapping without os.Exit so the process stays testable.
 for _, args := range [][]string{{}, {"fail"}, {"ship"}} {
 err := run(args)
 code := 0
 switch {
 case errors.Is(err, errUsage):
 code = 2
 case err != nil:
 code = 1
 }
 fmt.Printf("args=%v code=%d err=%v\n", args, code, err)
 }
}

go run .

```

### Expected output:

```

args=[] code=2 err=usage: tool <name>
args=[fail] code=1 err=runtime boom
ok: ship
args=[ship] code=0 err=<nil>

```

## Runnable example

Graceful signal handling with `signal.NotifyContext`, a child process under `CommandContext`, and a tiny supervisor backoff loop—all in one stdlib program.

```

mkdir -p /tmp/go-signals && cd /tmp/go-signals
go mod init example.com/signals

```

Save as `main.go`:

```

package main

import (
 "context"
 "fmt"
 "os"
 "os/exec"

```

```

 "os/signal"
 "syscall"
 "time"
)

func supervisor(run func() error, maxAttempts int) error {
 var delay = 50 * time.Millisecond
 for attempt := 1; attempt <= maxAttempts; attempt++ {
 err := run()
 if err == nil {
 return nil
 }
 fmt.Printf("attempt %d failed: %v\n", attempt, err)
 if attempt == maxAttempts {
 return fmt.Errorf("gave up after %d attempts: %w", maxAttempts, err)
 }
 time.Sleep(delay)
 delay *= 2
 }
 return nil
}

func main() {
 // 1) Signal-aware context (demo: cancel after short timer to avoid interactive wait).
 ctx, stop := signal.NotifyContext(context.Background(), os.Interrupt, syscall.SIGTERM)
 defer stop()

 go func() {
 time.Sleep(30 * time.Millisecond)
 // Simulate supervisor/orchestrator stop without needing a real signal in CI.
 stop()
 }()

 <-ctx.Done()
 fmt.Println("shutdown signal path:", ctx.Err())

 // 2) Child with deadline
 cctx, cancel := context.WithTimeout(context.Background(), 100*time.Millisecond)
 defer cancel()
 cmd := exec.CommandContext(cctx, "sleep", "2")
 err := cmd.Run()
 fmt.Println("child cancelled:", err != nil)

 // 3) Supervisor backoff: fail twice then succeed
 var n int
 err = supervisor(func() error {
 n++

```

```

 if n < 3 {
 return fmt.Errorf("boom %d", n)
 }
 fmt.Println("worker healthy")
 return nil
 }, 5)
 fmt.Println("supervisor err:", err)
}

go run .

```

### Expected output:

```

shutdown signal path: context canceled
child cancelled: true
attempt 1 failed: boom 1
attempt 2 failed: boom 2
worker healthy
supervisor err: <nil>

```

### What to notice

- Kubernetes/docker stop send SIGTERM first; apps must drain within the grace period or get SIGKILL.
- `CommandContext` is how you avoid orphan helpers after cancel.
- Supervisors need capped exponential backoff or they thundering-herd a bad config forever.

### Try next

- Build a mini HTTP server and call `Shutdown` when the signal context cancels.
- Map exit codes: 0 success, 1 runtime error, 2 usage error for a CLI.

## Further Reading

- `os/signal` package docs
- Kubernetes termination lifecycle
- Next: **Filesystem Automation and Safe IO**

## **Filesystem Automation and Safe IO**

# Filesystem Automation and Safe IO

Filesystem automation fails in subtle ways: partial writes, interrupted updates, TOCTOU races, and path traversal. For ops tooling, **predictability beats peak throughput**.

## Why / Overview

Go services and CLIs constantly touch disks:

- Rewrite config and state files
- Rotate logs and checkpoints
- Unpack artifacts
- Cache blobs

A crash mid-write can leave empty or half-written files that break the next start. Untrusted paths can escape into `/etc` or secrets mounts.

```
unsafe: open(path) -> write -> close # torn file on crash
safe: write temp -> fsync -> rename -> fsync dir
```

## Atomic Replace Pattern

POSIX `rename` within the same filesystem is atomic for the directory entry. Combined with temp files:

```
package atomicfile

import (
 "fmt"
 "os"
 "path/filepath"
)

func WriteFile(path string, data []byte, perm os.FileMode) error {
 dir := filepath.Dir(path)
 tmp, err := os.CreateTemp(dir, "."+filepath.Base(path)+".tmp-*")
 if err != nil {
 return err
 }
}
```

```

tmpName := tmp.Name()
defer func() { _ = os.Remove(tmpName) }() // no-op if rename succeeded

if _, err := tmp.Write(data); err != nil {
 tmp.Close()
 return err
}
if err := tmp.Chmod(perm); err != nil {
 tmp.Close()
 return err
}
if err := tmp.Sync(); err != nil { // fsync file
 tmp.Close()
 return err
}
if err := tmp.Close(); err != nil {
 return err
}
if err := os.Rename(tmpName, path); err != nil {
 return err
}
// Best-effort fsync parent directory so the rename itself is durable.
return syncDir(dir)
}

func syncDir(dir string) error {
 d, err := os.Open(dir)
 if err != nil {
 return err
 }
 defer d.Close()
 if err := d.Sync(); err != nil {
 // Some FS (NFS, certain mounts) may not support directory sync.
 return fmt.Errorf("sync dir: %w", err)
 }
 return nil
}

```

Notes:

- Create the temp file **in the same directory** as the target so **rename** does not cross devices.
- Sync costs latency; for pure caches you may skip durability.
- On Windows, atomic replace semantics differ; test platform-specific paths.

## Path Safety (Root Jail)

Treat every user-supplied path as hostile.

```
package safepath

import (
 "fmt"
 "os"
 "path/filepath"
 "strings"
)

func UnderRoot(root, userPath string) (string, error) {
 rootAbs, err := filepath.Abs(root)
 if err != nil {
 return "", err
 }
 // Disallow absolute user paths escaping intent.
 if filepath.IsAbs(userPath) {
 return "", fmt.Errorf("absolute paths not allowed")
 }
 joined := filepath.Join(rootAbs, userPath)
 clean := filepath.Clean(joined)
 rel, err := filepath.Rel(rootAbs, clean)
 if err != nil {
 return "", err
 }
 if rel == ".." || strings.HasPrefix(rel, ".."+string(os.PathSeparator)) {
 return "", fmt.Errorf("path escapes root")
 }
 return clean, nil
}
```

Symlinks: `filepath.EvalSymlinks` can help but introduces its own races and platform quirks. For high-security tools, open with `O_NOFOLLOW` where available, or reject symlinks after `Lstat`.

## Safe Read Patterns

```
// Bound reads from untrusted files.
f, err := os.Open(path)
if err != nil {
 return err
}
defer f.Close()
```



```
data, err := io.ReadAll(io.LimitReader(f, 8<<20)) // 8 MiB cap
```

For line processing of huge logs, stream with `bufio.Scanner` but raise `MaxTokenSize` only deliberately — default token limits exist for a reason.

## Permissions and Umask

```
// Secrets on disk: owner read/write only
if err := atomicfile.WriteFile(secretPath, b, 0o600); err != nil { ... }

// Shared but not world-writable configs
if err := atomicfile.WriteFile(cfgPath, b, 0o644); err != nil { ... }
```

Never use `0o777` “to make it work.” Fix ownership instead.

## Directory Creation

```
if err := os.MkdirAll(dir, 0o755); err != nil {
 return err
}
```

`MkdirAll` is fine; still validate `dir` is under your root when driven by user input.

## Check-Then-Act Races (TOCTOU)

```
// racy
if _, err := os.Stat(path); os.IsNotExist(err) {
 f, _ = os.Create(path) // another process may have created it
}
```

Prefer open flags:

```
f, err := os.OpenFile(path, os.O_CREATE|os.O_EXCL|os.O_WRONLY, 0o600)
```

Or accept overwrite with atomic replace.

## Streaming Copy and Spool

```
func copyFile(src, dst string) error {
 in, err := os.Open(src)
 if err != nil {
 return err
 }
 defer in.Close()

 tmp := dst + ".tmp"
 out, err := os.OpenFile(tmp, os.O_CREATE|os.O_TRUNC|os.O_WRONLY, 0o644)
 if err != nil {
 return err
 }
 if _, err := io.Copy(out, in); err != nil {
 out.Close()
 os.Remove(tmp)
 return err
 }
 if err := out.Sync(); err != nil {
 out.Close()
 return err
 }
 if err := out.Close(); err != nil {
 return err
 }
 return os.Rename(tmp, dst)
}
```

For large artifacts, consider checksums (SHA-256) after write before rename promotion.

## Walking Trees

```
err := filepath.WalkDir(root, func(path string, d fs.DirEntry, err error) error {
 if err != nil {
 return err // or skip
 }
 if d.Type()&os.ModeSymlink != 0 {
 return nil // skip or carefully resolve
 }
 // process...
 return nil
})
```

Or `fs.WalkDir` with `os.DirFS` for testability.

## Embed for Static Assets

```
//go:embed schemas/*.json
var schemas embed.FS
```

Prefer `embed` over shipping loose files when content is part of the binary release — fewer path problems in production.

## Durability vs Speed Tradeoffs

Workload	fsync?	Pattern
Wallet / ledger checkpoint	Yes	full atomic + dir sync
CDN cache fill	Often no	write + rename optional
Queue spill to disk	Yes on commit	group commits if needed
CLI rewrite of dotfiles	Rename enough	fsync nice-to-have

Measure: fsync can dominate latency on spinning disks and some cloud volumes.

## Production Checklist

- ☐ Atomic write for any file that must be valid after crash
- ☐ Temp files on same filesystem as destination
- ☐ Max size limits on untrusted reads
- ☐ Path jail for user-controlled paths
- ☐ Restrictive permissions on secrets
- ☐ No world-writable dirs in the data path
- ☐ Explicit symlink policy
- ☐ Disk-full errors handled (not ignored)
- ☐ Tests for traversal (`../`) and partial-write recovery

## Common Pitfalls

1. **Write in place** to config files — crash leaves empty file, service won't start.
2. **Temp in `/tmp` then rename to another mount** — EXDEV, copy fallback needed.
3. **Trusting `Clean` alone** without root prefix check.
4. **Following symlinks** into sensitive locations.
5. **Ignoring `ENOSPC`** — loop of failed writes can spam logs forever.
6. **Scanner default token size** on huge lines — silent early exit if not checking `scanner.Err()`.
7. **Closing without sync** when durability was required.

## Exercises

1. Implement `WriteFile` atomic helper; kill -9 mid-write in a loop; prove the target is always old-good or new-good, never empty torn.
2. Build a path jail; unit-test `../..etc/passwd`, absolute paths, nested ...
3. Write a file with `0o600`; verify mode with `stat`.
4. Copy a 1GB file with streaming; compare memory RSS to `ReadAll`.
5. Create a symlink escape scenario; decide reject vs `EvalSymlinks` policy; document it.
6. Simulate disk full (`ulimit` or tiny loop mount); ensure errors surface.
7. Use `O_EXCL` to implement a simple lock file; handle stale locks carefully (advanced).
8. Walk a tree skipping symlinks; count regular files only.
9. Add SHA-256 verify-after-write before rename for a download tool.
10. Benchmark `fsync` vs no-`fsync` for 1k small config updates; record tradeoff.

## More examples

### Atomic write via temp + rename

```
mkdir -p /tmp/go-atomic-write && cd /tmp/go-atomic-write
go mod init example.com/atomic-write
```

Save as `main.go`:

```
package main

import (
 "fmt"
 "os"
 "path/filepath"
)

func writeAtomic(path string, data []byte) error {
 dir := filepath.Dir(path)
 f, err := os.CreateTemp(dir, ".w-*")
 if err != nil {
 return err
 }
 tmp := f.Name()
 if _, err := f.Write(data); err != nil {
 f.Close()
 os.Remove(tmp)
 return err
 }
 if err := f.Sync(); err != nil {
 f.Close()
 }
}
```

```

 os.Remove(tmp)
 return err
 }
 if err := f.Close(); err != nil {
 os.Remove(tmp)
 return err
 }
 return os.Rename(tmp, path)
}

func main() {
 dir, _ := os.MkdirTemp("", "atom")
 defer os.RemoveAll(dir)
 path := filepath.Join(dir, "state.json")
 if err := writeAtomic(path, []byte(`{"v":1}`)); err != nil {
 panic(err)
 }
 if err := writeAtomic(path, []byte(`{"v":2}`)); err != nil {
 panic(err)
 }
 b, _ := os.ReadFile(path)
 fmt.Println(string(b))
}

go run .

```

**Expected output:**

```
{"v":2}
```

## Walk files, skip dirs and symlinks to dirs

```
mkdir -p /tmp/go-walk-files && cd /tmp/go-walk-files
go mod init example.com/walk-files

```

Save as main.go:

```

package main

import (
 "fmt"
 "io/fs"
 "os"
 "path/filepath"
)

```

```

func main() {
 root, _ := os.MkdirTemp("", "walk")
 defer os.RemoveAll(root)
 _ = os.WriteFile(filepath.Join(root, "a.txt"), []byte("a"), 0o644)
 _ = os.Mkdir(filepath.Join(root, "sub"), 0o755)
 _ = os.WriteFile(filepath.Join(root, "sub", "b.txt"), []byte("b"), 0o644)

 var files []string
 _ = filepath.WalkDir(root, func(path string, d fs.DirEntry, err error) error {
 if err != nil {
 return err
 }
 if d.Type().IsRegular() {
 rel, _ := filepath.Rel(root, path)
 files = append(files, rel)
 }
 return nil
 })
 fmt.Println("files:", files)
}

go run .

```

**Expected output:**

```
files: [a.txt sub/b.txt]
```

## Runnable example

Atomic replace via temp file + rename, a path jail, and restrictive file modes—crash-safe config write pattern in pure stdlib.

```

mkdir -p /tmp/go-safe-fs && cd /tmp/go-safe-fs
go mod init example.com/safe-fs

```

Save as main.go:

```

package main

import (
 "fmt"
 "os"
 "path/filepath"
 "strings"
)

```

```

func writeFileAtomic(path string, data []byte, mode os.FileMode) error {
 dir := filepath.Dir(path)
 if err := os.MkdirAll(dir, 0o750); err != nil {
 return err
 }
 f, err := os.CreateTemp(dir, ".tmp-*")
 if err != nil {
 return err
 }
 tmp := f.Name()
 cleanup := true
 defer func() {
 if cleanup {
 _ = os.Remove(tmp)
 }
 }()
 if _, err := f.Write(data); err != nil {
 f.Close()
 return err
 }
 if err := f.Chmod(mode); err != nil {
 f.Close()
 return err
 }
 if err := f.Sync(); err != nil {
 f.Close()
 return err
 }
 if err := f.Close(); err != nil {
 return err
 }
 if err := os.Rename(tmp, path); err != nil {
 return err
 }
 cleanup = false
 return nil
}

func underRoot(root, userPath string) (string, error) {
 rootAbs, err := filepath.Abs(filepath.Clean(root))
 if err != nil {
 return "", err
 }
 joined := filepath.Join(rootAbs, userPath)
 abs, err := filepath.Abs(joined)
 if err != nil {

```

```

 return "", err
 }
 rel, err := filepath.Rel(rootAbs, abs)
 if err != nil || strings.HasPrefix(rel, "..") {
 return "", fmt.Errorf("escapes root")
 }
 return abs, nil
}

func main() {
 root := filepath.Join(os.TempDir(), "safe-fs-demo")
 _ = os.MkdirAll(root, 0o750)
 target := filepath.Join(root, "config.json")

 if err := writeFileAtomic(target, []byte(`{"v":1}`+"\n"), 0o600); err != nil {
 panic(err)
 }
 b, _ := os.ReadFile(target)
 fmt.Printf("config: %s", b)
 fi, _ := os.Stat(target)
 fmt.Printf("mode: %o\n", fi.Mode().Perm())

 ok, err := underRoot(root, "nested/a.txt")
 fmt.Println("jail ok:", ok, err)
 _, err = underRoot(root, "../../etc/passwd")
 fmt.Println("jail block:", err != nil)

 // Second atomic write replaces cleanly
 _ = writeFileAtomic(target, []byte(`{"v":2}`+"\n"), 0o600)
 b, _ = os.ReadFile(target)
 fmt.Printf("config after: %s", b)
}

go run .

```

**Expected output** (paths vary):

```

config: {"v":1}
mode: 600
jail ok: ../../safe-fs-demo/nested/a.txt <nil>
jail block: true
config after: {"v":2}

```

### What to notice

- Rename within the same filesystem is atomic for readers; in-place truncate is how configs go empty on crash.



- 0o600 for secrets/config; never world-writable automation output.
- Path jail before any user-influenced open/read/write.

### Try next

- Kill -9 mid-write in a loop and prove the target is always old-good or new-good.
- Handle EXDEV (cross-device rename) with copy+rename fallback.

## Further Reading

- POSIX rename and fsync semantics articles
- Previous: process lifecycle (crash = torn write source)
- Next: **Unix Pipeline Style CLI Tools**

## Unix Pipeline Style CLI Tools

# Unix Pipeline Style CLI Tools

Good CLI tools are composable components in a data stream. Go is excellent for CLIs: single static binary, fast startup, solid `bufio` and encoding libraries.

## Why / Overview

If your tool only works as an interactive TUI, it cannot sit in CI. If it mixes logs into `stdout`, it breaks `jq`. Pipeline culture is an API design problem.

```
stdin -> transform / filter -> stdout (data only)
 |
 +--> stderr (diagnostics, progress)
exit code -> automation branch
```

## The Pipeline Contract

1. **Read `stdin`** when no file args (or when arg is `-`).
2. **Write data to `stdout`** — nothing else.
3. **Write diagnostics to `stderr`**.
4. **Exit non-zero** on failure; document codes.
5. **Be quiet on success** unless `--verbose`.
6. **Support non-TTY** operation (no assumed terminal width unless queried).

```
func main() {
 if err := run(os.Args[1:]); err != nil {
 fmt.Fprintln(os.Stderr, "mytool:", err)
 os.Exit(1)
 }
}
```

## Input Sources

```
func openInput(path string) (io.ReadCloser, error) {
 if path == "" || path == "-" {
 return io.NopCloser(os.Stdin), nil
 }
 return os.Open(path)
}
```

Multiple files: process sequentially like `cat`, or document parallel behavior carefully (order).

## Streaming Filters

Prefer stream processing over `ReadAll` for unbounded input.

```
func filter(r io.Reader, w io.Writer, pred func(string) bool) error {
 sc := bufio.NewScanner(r)
 // Optional: sc.Buffer(make([]byte, 64*1024), 1024*1024)
 out := bufio.NewWriter(w)
 defer out.Flush()
 for sc.Scan() {
 line := sc.Text()
 if pred(line) {
 if _, err := out.WriteString(line); err != nil {
 return err
 }
 if err := out.WriteByte('\n'); err != nil {
 return err
 }
 }
 }
 return sc.Err()
}
```

Always check `sc.Err()` — a failed scan can look like EOF.

## Output Modes: Human vs Machine

```
default (TTY): human tables, colors optional
--json / pipe: NDJSON or JSON array
--tsv: stable columns for cut/awk
```

Detect TTY:

```
func stdoutIsTTY() bool {
 fi, err := os.Stdout.Stat()
 if err != nil {
 return false
 }
 return (fi.Mode() & os.ModeCharDevice) != 0
}
```

Rules:

- Never emit ANSI color when not a TTY (or when NO\_COLOR is set).
- Machine formats must be **stable** across versions (version field if needed).
- Do not interleave progress bars on stdout.

```
type row struct {
 User string `json:"user"`
 OK bool `json:"ok"`
}

func writeNDJSON(w io.Writer, rows []row) error {
 enc := json.NewEncoder(w)
 for _, r := range rows {
 if err := enc.Encode(r); err != nil { // Encode adds newline
 return err
 }
 }
 return nil
}
```

## Flags and Subcommands

Use `flag` for simple tools; `cobra/ff` etc. for larger surfaces. Conventions:

- `--help` works
- Short flags where traditional (`-n`, `-v`)
- `--version`
- Options before/after args documented

```
fs := flag.NewFlagSet("grep-lite", flag.ExitOnError)
pattern := fs.String("e", "", "pattern")
invert := fs.Bool("v", false, "invert match")
_ = fs.Parse(args)
```

## Exit Codes

Code	Meaning
0	Success (and for grep-like: match found)
1	Generic error <b>or</b> grep-like: no match
2	Usage error

Document grep-like special cases; they surprise API people.

```
const (
 exitOK = 0
 exitFail = 1
 exitUsage = 2
)
```

## Handling SIGPIPE

When writing to a closed pipe (**head** closes early), writes fail with `EPIPE` / `ErrClosed`. Good tools exit quietly with SIGPIPE semantics rather than dumping a panic stack.

```
if _, err := os.Stdout.Write(buf); err != nil {
 if errors.Is(err, syscall.EPIPE) {
 os.Exit(0) // or 141 depending on tradition
 }
 return err
}
```

## Context and Cancellation

Long CLIs should honor SIGINT:

```
ctx, stop := signal.NotifyContext(context.Background(), os.Interrupt)
defer stop()

// pass ctx into work; on cancel, flush what is safe and exit non-zero
```

## Structure of a Solid Small Tool

```
cmd/mytool/main.go flags, exit codes
internal/app/run.go pure-ish logic with io.Reader/Writer
internal/app/run_test.go table tests with bytes.Buffer
```

Dependency inject IO for tests:

```
type App struct {
 In io.Reader
 Out io.Writer
 Err io.Writer
}

func (a *App) Run(ctx context.Context, args []string) error {
 // ...
}
```

## Progress and Logging

```
log := slog.New(slog.NewTextHandler(os.Stderr, &slog.HandlerOptions{Level: slog.LevelInfo}))
log.Info("processed", "lines", n)
```

Never `fmt.Println` debug to stdout in library paths.

## Composition Examples

```
NDJSON pipeline
mytool --json < events.log | jq 'select(.level=="error")'

Classic filters
cat access.log | mytool -e 'timeout' | wc -l

Explicit files
mytool -e 'panic' app.log app.log.1
```

## Production Checklist (CLI)

- ☐ stdin default; - supported
- ☐ stdout data-only
- ☐ stderr for errors/logs

- ☐ Documented exit codes
- ☐ `--help` / usage on bad flags
- ☐ Stream large inputs
- ☐ Stable machine-readable mode
- ☐ `NO_COLOR` / TTY-aware color
- ☐ `SIGINT` cancels work
- ☐ `SIGPIPE` handled
- ☐ Tests use buffers, not real terminal

## Common Pitfalls

1. **Logging to stdout** — breaks pipes.
2. **Loading entire stdin into memory.**
3. **Interactive prompts** when stdin is a pipe (hangs CI).
4. **Unstable JSON key order / floating timestamps** for golden tests — prefer normalized output.
5. **Relative paths** depending on launch directory without documenting.
6. **Buffered stdout not flushed** on early exit — use `bufio.Writer + Flush` or defer carefully.
7. **Assuming UTF-8** without documenting binary behavior.

## Mini Implementation: line counter

```
package main

import (
 "bufio"
 "fmt"
 "io"
 "os"
)

func countLines(r io.Reader) (int, error) {
 sc := bufio.NewScanner(r)
 n := 0
 for sc.Scan() {
 n++
 }
 return n, sc.Err()
}

func main() {
 n, err := countLines(os.Stdin)
 if err != nil {
 fmt.Fprintln(os.Stderr, err)
 }
}
```



```

 os.Exit(1)
 }
 fmt.Println(n)
}

```

Extend with file args, `-v`, and JSON `{"lines":N}` as exercises.

## Exercises

1. Build a filter that prints lines containing a substring; support stdin and file args.
2. Add `--json` NDJSON output of `{line,no}`; keep default plain text.
3. Implement invert match (`-v`) and appropriate exit codes.
4. Pipe through `head -n 1` and ensure no ugly panic on SIGPIPE.
5. Add `slog` diagnostics on stderr behind `-v`; prove `jq` still parses stdout.
6. Refactor logic to `App` struct; unit-test with `strings.NewReader` / `bytes.Buffer`.
7. Handle lines longer than 64KiB safely (configure scanner buffer).
8. Add `--fail-empty` that exits 1 when zero lines matched.
9. Benchmark counting 10M lines; compare `Scanner` vs raw `bufio.Reader` `ReadBytes`.
10. Write a README section “Scripting interface” documenting flags, exit codes, and examples.

## More examples

### Stdin filter: prefix match

```

mkdir -p /tmp/go-pipe-filter && cd /tmp/go-pipe-filter
go mod init example.com/pipe-filter

```

Save as `main.go`:

```

package main

import (
 "bufio"
 "fmt"
 "strings"
)

func filter(in string, prefix string) []string {
 var out []string
 sc := bufio.NewScanner(strings.NewReader(in))
 for sc.Scan() {
 line := sc.Text()
 if strings.HasPrefix(line, prefix) {
 out = append(out, line)
 }
 }
 return out
}

```

```

 }
}
return out
}

func main() {
 in := "ERR boom\nINFO ok\nERR again\n"
 for _, line := range filter(in, "ERR") {
 fmt.Println(line)
 }
}

go run .

```

### Expected output:

```

ERR boom
ERR again

```

### Exit code when no matches (grep-like)

```

mkdir -p /tmp/go-pipe-exit && cd /tmp/go-pipe-exit
go mod init example.com/pipe-exit

```

Save as main.go:

```

package main

import (
 "fmt"
 "strings"
)

func matchCount(in, needle string) int {
 n := 0
 for _, line := range strings.Split(strings.TrimSuffix(in, "\n"), "\n") {
 if strings.Contains(line, needle) {
 n++
 }
 }
 return n
}

func main() {
 for _, needle := range []string{"go", "rust"} {

```

```

 n := matchCount("go tools\ngo test\n", needle)
 code := 0
 if n == 0 {
 code = 1
 }
 fmt.Printf("needle=%q matches=%d exit=%d\n", needle, n, code)
 }
}

go run .

```

### Expected output:

```

needle="go" matches=2 exit=0
needle="rust" matches=0 exit=1

```

## Runnable example

A pipeline-friendly line filter: read stdin (or an in-memory reader), match a substring, write matches to stdout, diagnostics to stderr, exit codes for scripts.

```

mkdir -p /tmp/go-pipeline-cli && cd /tmp/go-pipeline-cli
go mod init example.com/pipeline-cli

```

Save as `main.go`:

```

package main

import (
 "bufio"
 "fmt"
 "io"
 "os"
 "strings"
)

func filter(r io.Reader, w io.Writer, errW io.Writer, substr string, invert bool) (matches int) {
 sc := bufio.NewScanner(r)
 // Raise token size for long log lines
 buf := make([]byte, 0, 64*1024)
 sc.Buffer(buf, 1024*1024)
 for sc.Scan() {
 line := sc.Text()
 hit := strings.Contains(line, substr)
 if invert {

```

```

 hit = !hit
 }
 if hit {
 if _, err := fmt.Fprintln(w, line); err != nil {
 return matches, err
 }
 matches++
 }
}
if err := sc.Err(); err != nil {
 fmt.Fprintln(errW, "scan:", err)
 return matches, err
}
return matches, nil
}

func main() {
 // Demo without relying on external pipes: feed a buffer, print results.
 input := "alpha\nerror: disk full\nbeta\nerror: timeout\ngamma\n"
 var out, errB strings.Builder
 n, err := filter(strings.NewReader(input), &out, &errB, "error:", false)
 if err != nil {
 fmt.Fprintln(os.Stderr, err)
 os.Exit(1)
 }
 fmt.Fprint(os.Stdout, out.String())
 fmt.Fprintf(os.Stderr, "matches=%d\n", n)

 // Exit code convention for CLIs: 0 found, 1 not found (grep-like optional).
 if n == 0 {
 os.Exit(1)
 }
}

```

```

go run .
real pipeline:
printf 'a\nerror: x\nb\n' | go run . # after wiring os.Stdin/Args

```

### Expected output:

```

error: disk full
error: timeout
matches=2

```

(stderr line may interleave depending on buffering)

### What to notice

- Business output on stdout; counts/diagnostics on stderr so `| jq` keeps working.
- Configure **Scanner** buffer for long lines; check `sc.Err()`.
- Exit codes are part of the scripting API.

### Try next

- Accept optional file args; default to stdin when none.
- Add `-v` invert and `--json` NDJSON mode without breaking plain default.

## Further Reading

- Rob Pike / Unix philosophy notes on tools
- Previous: safe file IO for CLIs that write state
- Next part: **Distributed Infrastructure** patterns for long-running workers

## **Environment, User Identity, CWD, and umask**

# Environment, User Identity, CWD, and umask

## Overview

Every process inherits an **environment block**, a **user/group identity**, a **current working directory**, and (on Unix) a **umask**. Systems tools that ignore these surprise operators: relative paths resolve wrong, files get world-readable, and “works on my laptop” fails under systemd.

## Environment variables

```
os.Getenv("HOME")
os.Setenv("APP_MODE", "prod") // process-local
os.LookupEnv("DEBUG") // value, ok
os.Environ() // []string KEY=VAL
```

## Patterns

Pattern	Use
Config via env	12-factor services (DATABASE_URL)
Feature flags	DEBUG=1, APP_LOG_LEVEL=debug
Child isolation	cmd.Env = append(os.Environ(), "LC_ALL=C")
Clear inheritance	cmd.Env = []string{"PATH=/usr/bin"} (explicit only)

```
cmd := exec.Command("git", "status")
cmd.Env = append(os.Environ(), "GIT_TERMINAL_PROMPT=0")
```

Never log secrets pulled from env (TOKEN, PASSWORD, \*\_KEY).

## PATH and LookPath

```
path, err := exec.LookPath("docker")
// searches PATH; fails closed if missing
```

In minimal containers, PATH may lack tools your laptop has—document dependencies.

## User and group identity (Unix)

```
fmt.Println("uid", os.Getuid(), "euid", os.Geteuid())
fmt.Println("gid", os.Getgid())
```

ID	Meaning
UID / GID	Real user/group
EUID / EGID	Effective (permission checks)

Drop privileges in privileged starters (rare in pure Go services; more common in installers). Prefer running the whole process as non-root under Kubernetes/systemd.

```
u, err := user.Current()
// u.Username, u.HomeDir, u.Uid

// expand ~
if strings.HasPrefix(path, "~/") {
 home, _ := os.UserHomeDir()
 path = filepath.Join(home, path[2:])
}
```

Prefer `os.UserHomeDir()` / `os.UserConfigDir()` over hardcoding `/home/...`

## Current working directory

```
wd, err := os.Getwd()
err = os.Chdir("/var/lib/myapp")
```

Relative paths (`./data`, `config.yaml`) are relative to **CWD**, not the binary location.



## Binary-relative assets

```
exe, err := os.Executable()
dir := filepath.Dir(exe)
cfg := filepath.Join(dir, "config.yaml")
```

Note: `os.Executable` may be a symlink path; `filepath.EvalSymlinks` if you need the real directory.

## Children and Dir

```
cmd := exec.Command("./tool")
cmd.Dir = "/work/repo" // child CWD
```

## umask and file permissions

Unix **umask** masks permission bits on create. Go's `os.OpenFile` / `WriteFile` pass a mode that is then masked by umask.

```
// request 0666 → often becomes 0644 with umask 0022
os.WriteFile("out.txt", data, 0o666)
```

For secrets:

```
f, err := os.OpenFile(path, os.O_CREATE|os.O_WRONLY|os.O_TRUNC, 0o600)
```

Still subject to umask (e.g. `0o600 & ^umask`). If you need exact bits, `Chmod` after create:

```
_ = os.Chmod(path, 0o600)
```

## Hostname and process identity

```
host, _ := os.Hostname()
pid := os.Getpid()
ppid := os.Getppid()
```

Useful in logs: `slog.Info("start", "host", host, "pid", pid)`.

## Minimal example: show process context

```
func printContext(w io.Writer) {
 wd, _ := os.Getwd()
 fmt.Fprintf(w, "pid=%d ppid=%d\n", os.Getpid(), os.Getppid())
 fmt.Fprintf(w, "uid=%d euid=%d\n", os.Getuid(), os.Geteuid())
 fmt.Fprintf(w, "cwd=%s\n", wd)
 fmt.Fprintf(w, "home=%s\n", mustHome())
}

func mustHome() string {
 h, err := os.UserHomeDir()
 if err != nil {
 return ""
 }
 return h
}
```

## Rules of thumb

Do	Don't
Resolve paths with <code>UserConfigDir</code> / absolute roots	Assume CWD is the repo root in systemd
Pass explicit <code>cmd.Env</code> when isolation matters	Leak host secrets into every child
Create secrets as <code>0600 + chmod</code>	Default world-readable state files
Log pid/host for multi-replica debug	Log full <code>os.Environ()</code>

## Try next

1. Run the same binary from two different CWDs; print `Getwd` and a relative open error.
2. Start a child with a stripped `Env` and show it cannot find `git` without `PATH`.
3. Write a file with mode `0o666` and inspect actual mode with `stat`.

## **File Descriptors and Redirection**

# File Descriptors and Redirection

## Overview

Unix processes talk to the world through **file descriptors** (FDs): integers indexing a per-process table. FD 0/1/2 are stdin/stdout/stderr. Understanding FDs explains pipes, redirection, and why “close the file” matters for long-running services.

## The classic trio

FD	Go	Role
0	<code>os.Stdin</code>	Input
1	<code>os.Stdout</code>	Data output
2	<code>os.Stderr</code>	Diagnostics

```
fmt.Fprintln(os.Stdout, "data")
fmt.Fprintln(os.Stderr, "debug")
```

Shell: `tool >out.txt 2>err.txt` remaps FDs before `exec`.

## Opening files yields FDs

```
f, err := os.Open("/etc/hosts")
// f.Fd() → uintptr underlying FD (Unix)
defer f.Close() // returns FD to the kernel
```

Leaking FDs (`Open` without `Close`) exhausts the process limit (`ulimit -n`).

## Inheritance across `exec`

By default, child processes inherit open FDs unless marked close-on-exec.

```
cmd := exec.Command("child")
cmd.Stdin = os.Stdin
cmd.Stdout = os.Stdout
```

```
cmd.Stderr = os.Stderr
```

`os/exec` sets up pipes or inheritance for the three standard streams. Extra FDs you opened in the parent may remain open in the child if not close-on-exec—usually undesirable.

```
f, _ := os.OpenFile("log.txt", os.O_APPEND|os.O_CREATE|os.O_WRONLY, 0o644)
// syscall.CloseOnExec(int(f.Fd())) // low-level; prefer not sharing logs via inheritance
cmd.ExtraFiles = []*os.File{f} // intentional FD 3+ in child (Unix)
```

`ExtraFiles` maps to child FDs starting at 3—used by some protocols (e.g. systemd socket activation patterns).

## Pipes as FDs

```
r, w, err := os.Pipe()
go func() {
 defer w.Close()
 fmt.Fprintln(w, "hello")
}()
buf := make([]byte, 64)
n, _ := r.Read(buf)
fmt.Println(string(buf[:n]))
r.Close()
```

`io.Pipe` is in-process (not OS FDs). `os.Pipe` is a real kernel pipe—shareable with children.

## Redirection patterns in Go

### Capture child stdout

```
var stdout, stderr bytes.Buffer
cmd.Stdout = &stdout
cmd.Stderr = &stderr
err := cmd.Run()
```

### Stream child to parent stdout

```
cmd.Stdout = os.Stdout
cmd.Stderr = os.Stderr
```

## Tee logs

```
cmd.Stdout = io.MultiWriter(os.Stdout, logFile)
```

## Checking TTY vs pipe

```
fi, err := os.Stdout.Stat()
isChar := fi.Mode()&os.ModeCharDevice != 0
```

Color and progress bars: only when stdout is a TTY (and respect NO\_COLOR).

## Limits and EMFILE

```
too many open files → EMFILE / "too many open files"
```

Mitigation	How
Always Close	<code>defer</code> or <code>t.Cleanup</code>
Bound concurrency	worker pools for file openers
Raise limit carefully	container <code>ulimit</code> , not a substitute for leaks
Reuse readers	stream instead of open-all

## /dev/null and discard

```
cmd.Stdout = io.Discard
cmd.Stderr = io.Discard
// or os.OpenFile(os.DevNull, os.O_WRONLY, 0)
```

## Minimal tool: fdcount sketch

```
// Linux: count entries in /proc/self/fd
entries, err := os.ReadDir("/proc/self/fd")
fmt.Println(len(entries))
```

Use in tests to catch FD leaks (open N files, close, count should return).

## Rules of thumb

Do	Don't
Close files and pipe ends	Rely on GC finalizers for FDs
Keep stdout/stderr roles pure	Mix binary data and logs on FD 1
Use <b>ExtraFiles</b> only intentionally	Leak sockets into helpers
Handle <b>EPIPE</b> on writes	Crash on   <b>head</b> pipelines

## Try next

1. Pipe a Go program into **head -n 1** and ensure it exits cleanly on short write.
2. Spawn a child with **ExtraFiles** and print FD 3 in the child.
3. Open 10k files without close; observe the error; fix with a pool.

# Memory-Mapped Files



# Memory-Mapped Files

## Overview

**mmap** maps a file (or anonymous memory) into the process address space so you read and write with normal memory operations while the kernel pages data to/from disk. Go exposes this via [golang.org/x/sys/unix](https://golang.org/x/sys/unix) (Unix) or careful use of platform APIs—not as a one-liner in the core `stdlib`.

When to use `mmap` vs `os.Read` / `bufio`:

Prefer read/stream	Prefer mmap
Sequential scans of huge files	Random access into large read-mostly files
Simple code, small files	Shared memory between processes (advanced)
Portable <code>stdlib</code> only	OS-specific performance work

Deep dive context: [251 mmap vs pread](#).

## Mental model

```
process virtual memory
+-----+
| mapped region (pages) | <--> file on disk
+-----+
 page fault loads
```

Writes to a shared map may update the file (depending on flags). Always understand `PROT_*` and `MAP_*` flags.

## Read-only map (Unix sketch)

```
//go:build unix

package mmapfile
```

```

import (
 "fmt"
 "os"

 "golang.org/x/sys/unix"
)

type RO struct {
 data []byte
}

func OpenRO(path string) (*RO, error) {
 f, err := os.Open(path)
 if err != nil {
 return nil, err
 }
 defer f.Close()

 st, err := f.Stat()
 if err != nil {
 return nil, err
 }
 size := int(st.Size())
 if size == 0 {
 return &RO{data: []byte{}}, nil
 }

 data, err := unix.Mmap(int(f.Fd()), 0, size, unix.PROT_READ, unix.MAP_SHARED)
 if err != nil {
 return nil, err
 }
 return &RO{data: data}, nil
}

func (m *RO) Bytes() []byte { return m.data }

func (m *RO) Close() error {
 if len(m.data) == 0 {
 return nil
 }
 return unix.Munmap(m.data)
}

m, err := mmapfile.OpenRO("large.bin")
if err != nil {
 log.Fatal(err)
}

```

```

}
defer m.Close()
fmt.Printf("first 16: %x\n", m.Bytes()[:min(16, len(m.Bytes()))])

```

**Critical:** after `Munmap`, the slice is invalid—do not retain sub-slices.

## Write map (careful)

```

data, err := unix.Mmap(int(f.Fd()), 0, size,
 unix.PROT_READ|unix.PROT_WRITE, unix.MAP_SHARED)
// mutate data[i] = ...
// unix.Msync(data, unix.MS_SYNC) // durability

```

Crash consistency is harder than write-temp-rename. For config files, prefer atomic replace (chapter 142). Use writable mmap for specialized stores/databases, not casual state.

## Anonymous maps

```

data, err := unix.Mmap(-1, 0, size, unix.PROT_READ|unix.PROT_WRITE,
 unix.MAP_ANON|unix.MAP_PRIVATE)

```

Useful for large scratch buffers; still process-private memory.

## Safety with Go

1. **No GC pointers into foreign maps incorrectly** — mmap slices are plain `[]byte`; fine for bytes, not for storing Go pointers.
2. **Bounds** — map length is fixed at map time; file growth needs remap.
3. **Concurrent access** — concurrent reads OK for RO; writers need sync.
4. **Windows** — different API ([golang.org/x/sys/windows](http://golang.org/x/sys/windows)); isolate with build tags.

## When mmap loses

- One-pass line scan of a log → `bufio.Scanner` is simpler and fine
- Network filesystems with weird consistency
- Tiny files → overhead dominates

## Minimal CLI: mmapsum

```
// sha256 of mmapmed file without ReadAll
h := sha256.New()
_, _ = h.Write(m.Bytes())
fmt.Printf("%x\n", h.Sum(nil))
```

Compare memory profile vs `io.Copy(h, f)` for multi-GB inputs.

## Rules of thumb

Do	Don't
Munmap exactly once	Use slice after unmap
Prefer RO maps for inspection tools	mmap for every config write
Document OS build tags	Assume mmap code is portable without tags
Measure vs <code>pread/Read</code>	Assume mmap is always faster

## Try next

1. Map `/etc/hosts` read-only and print line count without `ReadFile`.
2. Benchmark mmap vs `io.Copy` SHA-256 on a 500MB file.
3. Read the deep-dive chapter on mmap vs `pread` for allocator interaction.

## **Unix Domain Sockets and Local IPC**

# Unix Domain Sockets and Local IPC

## Overview

**Unix domain sockets** (UDS) provide local IPC with stream or datagram semantics—like TCP/UDP but bound to filesystem paths (or abstract names on Linux). Go's `net` package supports them as networks `"unix"` and `"unixgram"`.

Use UDS when:

- Local sidecar app communication
- Privileged helper (root) spoken to by unprivileged client
- Avoid exposing a TCP port on 127.0.0.1

## Stream server and client

```
// server
path := "/tmp/myapp.sock"
_ = os.Remove(path) // clean stale socket file
ln, err := net.Listen("unix", path)
if err != nil {
 log.Fatal(err)
}
defer ln.Close()
_ = os.Chmod(path, 0o660) // restrict who can connect

for {
 conn, err := ln.Accept()
 if err != nil {
 continue
 }
 go func(c net.Conn) {
 defer c.Close()
 io.Copy(c, c) // echo
 }(conn)
}
```

```
// client
conn, err := net.Dial("unix", "/tmp/myapp.sock")
```

## Lifecycle of the socket file

Event	Action
Before listen	Remove stale path if safe
After listen	Chmod / directory permissions
Shutdown	Close listener; optionally Remove path

If the process dies without removing the path, next start gets `address already in use` until `unlink`.

## Abstract namespace (Linux)

Linux allows abstract sockets: path starts with `@` or null byte—**no filesystem node**.

```
ln, err := net.Listen("unix", "@myapp.rpc") // Go uses @ convention
```

Pros: no leftover files. Cons: Linux-only; different permission model (not file mode).

## Datagram unixgram

```
conn, err := net.ListenPacket("unixgram", "/tmp/myapp.dgram.sock")
buf := make([]byte, 2048)
n, addr, err := conn.ReadFrom(buf)
_, err = conn.WriteTo(buf[:n], addr)
```

Message-oriented; size limits apply.

## HTTP over Unix sockets

```
ln, err := net.Listen("unix", path)
server := &http.Server{Handler: mux}
go server.Serve(ln)

// client
httpc := http.Client{
 Transport: &http.Transport{
 DialContext: func(ctx context.Context, _, _ string) (net.Conn, error) {
```

```

 var d net.Dialer
 return d.DialContext(ctx, "unix", path)
 },
},
}
resp, err := httpc.Get("http://localhost/healthz") // host ignored by dialer

```

Docker engine API and many local agents use this pattern.

## Credentials and peer auth (advanced)

On Linux, `SO_PEERCREC` can report peer UID/PID—useful for authorizing local clients. Requires [golang.org/x/sys/unix](https://golang.org/x/sys/unix) and careful API use. For many apps, **filesystem permissions on the socket path** are enough.

## Timeouts

```

conn, err := d.DialContext(ctx, "unix", path)
_ = conn.SetDeadline(time.Now().Add(5 * time.Second))

```

Same discipline as TCP—don't hang forever on a stuck peer.

## Minimal tool pair: udsecho

```

terminal 1
go run ./cmd/udsecho serve -path /tmp/echo.sock
terminal 2
go run ./cmd/udsecho client -path /tmp/echo.sock -msg hello

```

## Rules of thumb

Do	Don't
Chmod socket paths tightly	World-writable <code>/tmp/*.sock</code> for secrets
Remove stale sockets on start	Ignore <code>EADDRINUSE</code> forever
Prefer UDS for local-only APIs	Bind Docker API-style services on <code>0.0.0.0</code> by accident
Document path location	Scatter sockets without <code>XDGRuntimeDir</code>

Runtime dir tip: `XDGRuntimeDir` or `/run/user/$(id -u)/` for user services.



## Try next

1. HTTP health endpoint served only on a Unix socket.
2. Two processes: one sends JSON lines over `unixgram`.
3. Start twice without `Remove`; fix the stale socket handling.

## **File Locks, Pidfiles, and Single-Instance Tools**

# File Locks, Pidfiles, and Single-Instance Tools

## Overview

Ops tools often need **mutual exclusion**: only one compactor, one migrator, one agent per host. Classic patterns are **pidfiles** and **advisory file locks** (`flock`). Both are imperfect; know the failure modes.

## Advisory flock (Unix)

```
//go:build unix

package single

import (
 "fmt"
 "os"

 "golang.org/x/sys/unix"
)

type Lock struct {
 f *os.File
}

func Acquire(path string) (*Lock, error) {
 f, err := os.OpenFile(path, os.O_CREATE|os.O_RDWR, 0o644)
 if err != nil {
 return nil, err
 }
 // LOCK_EX exclusive; LOCK_NB non-blocking
 if err := unix.Flock(int(f.Fd()), unix.LOCK_EX|unix.LOCK_NB); err != nil {
 f.Close()
 return nil, fmt.Errorf("already running: %w", err)
 }
 return &Lock{f: f}, nil
}

func (l *Lock) Close() error {
```

```

 if l == nil || l.f == nil {
 return nil
 }
 _ = unix.Flock(int(l.f.Fd()), unix.LOCK_UN)
 err := l.f.Close()
 l.f = nil
 return err
}

lock, err := single.Acquire("/var/lock/myapp.lock")
if err != nil {
 log.Fatal(err)
}
defer lock.Close()
// only one instance holds the lock

```

Property	Note
Advisory	Cooperation required; another process can ignore
Released on close/exit	Kernel drops locks when FD closes
NFS	Historically unreliable; prefer local disk

## Pidfiles (classic, racy)

```

func WritePid(path string) error {
 pid := os.Getpid()
 return os.WriteFile(path, []byte(strconv.Itoa(pid)+"\n"), 0o644)
}

func ReadPid(path string) (int, error) {
 b, err := os.ReadFile(path)
 if err != nil {
 return 0, err
 }
 return strconv.Atoi(strings.TrimSpace(string(b)))
}

```

Problems:

1. Stale pidfile after crash
2. PID reuse (new process gets same PID)
3. TOCTOU between check and write

## Slightly better pidfile

```
// open O_CREATE|O_EXCL for create-only; if exists, read and probe process
f, err := os.OpenFile(path, os.O_CREATE|os.O_EXCL|os.O_WRONLY, 0o644)
if err == nil {
 fmt.Fprintf(f, "%d\n", os.Getpid())
 return f, nil // keep open; delete on exit
}
// else: read pid, kill(pid, 0) to test existence - still imperfect
```

**Prefer flock** for single-instance when available; use pidfiles for human operators (“what PID is running?”) **in addition**, not alone.

## Combining lock + pid

```
lock, err := Acquire(lockPath)
// write pid into lock file while holding lock
fmt.Fprintf(lock.f, "%d\n", os.Getpid())
_ = lock.f.Sync()
```

Readers open the file read-only and parse the PID for status tools.

## Cross-process mutex via lock directory (portable sketch)

```
// mkdir is atomic on POSIX for create
err := os.Mkdir(lockDir, 0o700)
if err != nil {
 if os.IsExist(err) {
 return fmt.Errorf("locked")
 }
 return err
}
// hold: keep directory; release: Remove
defer os.Remove(lockDir)
```

Works without flock but needs crash cleanup of empty lock dirs (stale locks).

## Signal-safe release

Always defer `lock.Close()`. On SIGTERM, unlock as part of shutdown so a supervisor restart can acquire immediately.

```
ctx, stop := signal.NotifyContext(...)
defer stop()
defer lock.Close()
<-ctx.Done()
```

## Minimal CLI: once

```
once -lock /tmp/job.lock -- /usr/bin/rsync ...
```

```
lock, err := Acquire(*lockPath)
if err != nil {
 os.Exit(75) // EX_TEMPFAIL - try later
}
defer lock.Close()
cmd := exec.Command(args[0], args[1:]...)
cmd.Stdout, cmd.Stderr = os.Stdout, os.Stderr
err = cmd.Run()
```

Cron-friendly: non-zero if already running.

## Rules of thumb

Do	Don't
flock on local disk	Trust pidfiles alone for exclusion
Document lock path	Scatter locks in world-writable dirs without care
Exit distinct code when busy	Hang forever waiting (unless intentional)
Defer unlock	Leave exclusive lock across network calls for minutes without need

## Try next

1. Run two instances of a flock demo; confirm the second fails fast.
2. Kill -9 the holder; confirm the second instance can acquire after.
3. Implement **once** and schedule it in cron every minute safely.

## **Project: Systems Mini-Tools**

# Project: Systems Mini-Tools

## Overview

Build **systool**, a multi-command systems utility that practices this part end-to-end: process info, signals, atomic files, pipelines, locks, and local IPC. Keep each verb small.

Command	Skills
<code>id</code>	pid, uid, cwd, env peek
<code>watchdog</code>	run a child; restart on exit; SIGTERM drain
<code>atomic-write</code>	write-temp-rename
<code>lines</code>	stdin filter pipeline
<code>once</code>	flock single-instance runner
<code>serve-sock</code>	HTTP over Unix socket
<code>tailf</code>	poll file growth (simple)

Stdlib first; use [golang.org/x/sys/unix](https://golang.org/x/sys/unix) only for flock.

## Scaffold

```
mkdir -p systool/cmd/systool systool/internal/app
cd systool
go mod init example.com/systool
```

Dispatch pattern: same as CLI part (`FlagSet` per command + `NotifyContext`).

---

## 1. id — process context

```
func cmdID(stdout io.Writer) error {
 wd, err := os.Getwd()
 if err != nil {
 return err
 }
 host, _ := os.Hostname()
```



```

 fmt.Fprintf(stdout, "pid\t%d\n", os.Getpid())
 fmt.Fprintf(stdout, "ppid\t%d\n", os.Getppid())
 fmt.Fprintf(stdout, "uid\t%d\n", os.Getuid())
 fmt.Fprintf(stdout, "cwd\t%s\n", wd)
 fmt.Fprintf(stdout, "host\t%s\n", host)
 if h, err := os.UserHomeDir(); err == nil {
 fmt.Fprintf(stdout, "home\t%s\n", h)
 }
 return nil
}

```

```
go run ./cmd/systool id
```

---

## 2. watchdog — supervise a child

```

func cmdWatchdog(ctx context.Context, args []string) error {
 fs := flag.NewFlagSet("watchdog", flag.ContinueOnError)
 backoff := fs.Duration("backoff", time.Second, "restart delay")
 if err := fs.Parse(args); err != nil {
 return err
 }
 if fs.NArg() < 1 {
 return fmt.Errorf("usage: systool watchdog [--] <cmd> [args]")
 }
 for {
 if err := ctx.Err(); err != nil {
 return err
 }
 cmd := exec.CommandContext(ctx, fs.Arg(0), fs.Args()[1:]...)
 cmd.Stdout = os.Stdout
 cmd.Stderr = os.Stderr
 err := cmd.Run()
 fmt.Fprintf(os.Stderr, "watchdog: child exited: %v\n", err)
 select {
 case <-ctx.Done():
 return ctx.Err()
 case <-time.After(*backoff):
 }
 }
}

```

```
go run ./cmd/systool watchdog -backoff 500ms -- sleep 2
Ctrl-C should stop restarts
```

**Stretch:** max restarts; exponential backoff; don't restart on exit 0.

---

### 3. atomic-write

```
func atomicWrite(path string, data []byte, mode os.FileMode) error {
 dir := filepath.Dir(path)
 f, err := os.CreateTemp(dir, ".tmp-*")
 if err != nil {
 return err
 }
 tmp := f.Name()
 cleanup := true
 defer func() {
 if cleanup {
 _ = os.Remove(tmp)
 }
 }()
 if _, err := f.Write(data); err != nil {
 f.Close()
 return err
 }
 if err := f.Chmod(mode); err != nil {
 f.Close()
 return err
 }
 if err := f.Sync(); err != nil {
 f.Close()
 return err
 }
 if err := f.Close(); err != nil {
 return err
 }
 if err := os.Rename(tmp, path); err != nil {
 return err
 }
 cleanup = false
 // optional: fsync directory for durability
 return nil
}
```

```
echo hello | go run ./cmd/systool atomic-write -path /tmp/x.txt
```

---

## 4. lines — pipeline filter

```
func cmdLines(r io.Reader, w io.Writer, contains string) error {
 sc := bufio.NewScanner(r)
 for sc.Scan() {
 line := sc.Text()
 if contains == "" || strings.Contains(line, contains) {
 fmt.Fprintln(w, line)
 }
 }
 return sc.Err()
}
```

```
printf 'a\nb\nac\n' | go run ./cmd/systool lines -c a
```

Exit 0 always if IO OK; use grep-like exit codes only if you document them.

---

## 5. once — single instance

```
// acquire flock; run command; release
// exit 75 if lock busy
```

See chapter 148 for Acquire. Wire:

```
go run ./cmd/systool once -lock /tmp/job.lock -- echo only-one
```

---

## 6. serve-sock — local HTTP

```
func cmdServeSock(ctx context.Context, path string) error {
 _ = os.Remove(path)
 ln, err := net.Listen("unix", path)
 if err != nil {
```

```

 return err
}
_ = os.Chmod(path, 0o600)
mux := http.NewServeMux()
mux.HandleFunc("GET /healthz", func(w http.ResponseWriter, r *http.Request) {
 w.Write([]byte("ok"))
})
srv := &http.Server{Handler: mux}
go func() {
 <-ctx.Done()
 shutdownCtx, cancel := context.WithTimeout(context.Background(), 3*time.Second)
 defer cancel()
 _ = srv.Shutdown(shutdownCtx)
}()
err = srv.Serve(ln)
if err == http.ErrServerClosed {
 return nil
}
return err
}

```

```

go run ./cmd/systool serve-sock -path /tmp/systool.sock
curl --unix-socket /tmp/systool.sock http://localhost/healthz

```

---

## 7. tailf — follow file growth

```

func tailf(ctx context.Context, path string, w io.Writer) error {
 f, err := os.Open(path)
 if err != nil {
 return err
 }
 defer f.Close()
 // seek end
 _, _ = f.Seek(0, io.SeekEnd)
 buf := make([]byte, 4096)
 for {
 n, err := f.Read(buf)
 if n > 0 {
 _, _ = w.Write(buf[:n])
 }
 if err == io.EOF {
 select {

```

```

 case <-ctx.Done():
 return nil
 case <-time.After(200 * time.Millisecond):
 }
 continue
 }
 if err != nil {
 return err
 }
}
}

```

Not a full `tail -F` (no rotate handling)—enough to practice polls + signals.

---

## Acceptance checklist

- ☐ `id` prints pid/uid/cwd
- ☐ `watchdog` restarts `false` until Ctrl-C
- ☐ `atomic-write` leaves no torn file if killed mid-run (manual chaos)
- ☐ `lines` works in a pipe with `jq` or `wc`
- ☐ `once` second instance fails while first sleeps
- ☐ `serve-sock` answers curl via `--unix-socket`
- ☐ `tailf` prints new lines as you append to a file

## Layout recap

```

cmd/systool/main.go # signals + exit codes
internal/app/*.go # commands
internal/lock/flock_unix.go # build-tagged

```

## What you practiced

Chapter	Tool
141	watchdog, serve-sock shutdown
142	atomic-write
143	lines
144	id
145	pipes, stdout/stderr
147	serve-sock
148	once

Ship `systool` as a learning binary; promote individual commands into dedicated tools when they grow.

**Next project:** [156 Systems ops toolkit](#) (deadline, reload, cronish, healthsock, ...).

## **Time, Clocks, and Timers**

# Time, Clocks, and Timers

## Overview

Systems programs schedule work, measure latency, and enforce deadlines. Go’s `time` package is enough for almost all of this—if you know **wall clock vs monotonic** behavior and avoid classic timer leaks.

## Wall clock vs monotonic

```
t1 := time.Now()
// ... work ...
elapsed := time.Since(t1) // uses monotonic reading when possible
```

`time.Now()` carries both wall and monotonic components on modern Go. **Subtraction** (`Since`, `Sub`) prefers monotonic—so NTP steps don’t invent negative durations mid-measurement.

Need	Use
Logs, certs, file mtimes	Wall clock ( <code>Format</code> , <code>Truncate</code> )
Timeouts, RTT, deadlines	Monotonic-friendly ( <code>Since</code> , <code>Until</code> , <code>context + timer</code> )
“Run at 03:00 wall”	Wall clock + recompute after sleep

Never store only `UnixNano()` for elapsed timing across clock steps if you care about accuracy—prefer `time.Time` from `Now()`.

## Sleep and timers

```
time.Sleep(100 * time.Millisecond) // blocks goroutine

// prefer for cancelable wait:
t := time.NewTimer(100 * time.Millisecond)
defer t.Stop()
select {
case <-ctx.Done():
 return ctx.Err()
case <-t.C:
```



```
 // fired
}
```

## Stop and drain

```
if !t.Stop() {
 select {
 case <-t.C:
 default:
 }
}
```

Failing to stop/drain can wake a select later and cause subtle double-fire bugs.

## Ticker

```
tk := time.NewTicker(time.Second)
defer tk.Stop()
for {
 select {
 case <-ctx.Done():
 return
 case <-tk.C:
 // periodic work - keep it shorter than period
 }
}
```

Slow handlers + ticker = **tick pile-up** (channel buffer size 1 drops intermediate ticks in recent Go for **Ticker**—still don't do multi-second work on every tick without backpressure).

## Deadlines with context

```
ctx, cancel := context.WithTimeout(parent, 5*time.Second)
defer cancel()
// pass ctx to Dial, Query, HTTP
```

Compose: signal cancel + timeout cancel (**NotifyContext** outer, timeout inner or vice versa).

## Parsing and formatting

```
t, err := time.Parse(time.RFC3339, "2026-08-04T12:00:00Z")
s := t.UTC().Format(time.RFC3339Nano)
```

For CLI flags: `flag.Duration` already parses 300ms, 2s, 1h.

## File mtimes and caching

```
st, err := os.Stat(path)
mod := st.ModTime()
if mod.After(lastLoad) {
 // reload
}
```

Clock skew across NFS nodes can make mtime comparisons surprising—prefer content hashes for critical caches.

## Minimal tool: deadline

```
// run command with max wall time
ctx, cancel := context.WithTimeout(context.Background(), *max)
defer cancel()
cmd := exec.CommandContext(ctx, name, args...)
err := cmd.Run()
if errors.Is(ctx.Err(), context.DeadlineExceeded) {
 return fmt.Errorf("timeout after %s", *max)
}
```

## Rules of thumb

Do	Don't
Since for latency	Compare wall <code>Unix()</code> deltas for RTT
Stop timers you create	Leak <code>time.After</code> in tight loops
Bound periodic work	Do unbounded IO every tick without skip/logic
UTC in logs/storage	Mix local TZ without documenting

## Try next

1. Measure a sleep with `Since` while changing system time (VM lab)—note stability.
2. Rewrite a `time.After` loop to `NewTimer + Reset`.
3. Build `deadline 2s -- sleep 10` and confirm kill via `CommandContext`.

## **Filesystem Watching and Reload**

# Filesystem Watching and Reload

## Overview

Agents and servers often **reload config** when a file changes. Approaches:

1. **Polling** `Stat` `mtime`/size (portable, simple)
2. **OS watchers** (`inotify`, `FSEvents`, `ReadDirectoryChanges`)—via `github.com/fsnotify/fsnotify` or raw `x/sys`
3. **Signal** `SIGHUP` for classic Unix reload

This chapter keeps patterns **minimal** and production-safe (debounce, atomic replace races).

## Polling watcher (stdlib only)

```
type pollWatch struct {
 path string
 lastMod time.Time
 lastSz int64
}

func (p *pollWatch) changed() (bool, error) {
 st, err := os.Stat(p.path)
 if err != nil {
 return false, err
 }
 mod, sz := st.ModTime(), st.Size()
 if mod.Equal(p.lastMod) && sz == p.lastSz {
 return false, nil
 }
 p.lastMod, p.lastSz = mod, sz
 return true, nil
}

func watchLoop(ctx context.Context, path string, every time.Duration, onChange func() error) {
 var w pollWatch{path: path}
 _, _ = w.changed() // seed
```

```

t := time.NewTicker(every)
defer t.Stop()
for {
 select {
 case <-ctx.Done():
 return ctx.Err()
 case <-t.C:
 ok, err := w.changed()
 if err != nil {
 // missing file: keep trying
 continue
 }
 if ok {
 if err := onChange(); err != nil {
 slog.Error("reload", "err", err)
 }
 }
 }
}
}

```

**Atomic replace** (write temp + rename) may produce one or two events; always re-read the final path.

## Debounce

Editors and write tools can burst events:

```

func debounce(ctx context.Context, in <-chan struct{}, d time.Duration) <-chan struct{} {
 out := make(chan struct{}, 1)
 go func() {
 defer close(out)
 var timer *time.Timer
 for {
 select {
 case <-ctx.Done():
 return
 case _, ok := <-in:
 if !ok {
 return
 }
 if timer != nil {
 timer.Stop()
 }
 timer = time.AfterFunc(d, func() {

```

```

 select {
 case out <- struct{}{}:
 default:
 }
 })
 }
 }
 }()
 return out
 }

```

Reload **after quiet period** (e.g. 200ms).

## fsnotify sketch (common library)

```

// go get github.com/fsnotify/fsnotify
w, err := fsnotify.NewWatcher()
_ = w.Add(filepath.Dir(configPath))
for {
 select {
 case ev := <-w.Events:
 if ev.Name == configPath && (ev.Has(fsnotify.Write) || ev.Has(fsnotify.Create) || ev.Has(fsnotify.Remove)) {
 // signal reload (debounced)
 }
 case err := <-w.Errors:
 slog.Error("watch", "err", err)
 case <-ctx.Done():
 return
 }
}

```

Watch the **directory** if atomic rename replaces the inode—watching only the file can miss renames on some OSes.

## SIGHUP reload

```

hup := make(chan os.Signal, 1)
signal.Notify(hup, syscall.SIGHUP)
go func() {
 for {
 select {
 case <-ctx.Done():
 return
 }
 }
}

```

```

 case <-hup:
 _ = reload()
 }
 }
}()

```

Works when you control the operator (`kill -HUP`); pairs well with file watch.

## Safe reload checklist

1. Read entire config into memory
2. Validate
3. Swap atomically (`atomic.Value` or `mutex + pointer`)
4. Never half-apply on error—keep previous good config

```

type Config struct{ /* ... */ }

var current atomic.Pointer[Config]

func reload(path string) error {
 b, err := os.ReadFile(path)
 if err != nil {
 return err
 }
 cfg, err := parseConfig(b)
 if err != nil {
 return err
 }
 current.Store(&cfg)
 return nil
}

```

## Minimal tool: reload-demo

```

terminal 1
go run . -config /tmp/c.json
terminal 2
echo '{"n":1}' > /tmp/c.json
process logs "reloaded"

```



## Rules of thumb

Do	Don't
Debounce reloads	Parse on every partial write
Keep last good config	Crash process on bad temporary JSON
Prefer dir watch + rename awareness	Assume one Write event == complete file
Bound poll interval	Poll every 1ms on huge fleets

## Try next

1. Poll-watch a config; prove atomic rename still reloads.
2. Feed invalid JSON; process must keep serving old config.
3. Add SIGHUP path alongside poll.

## Pseudo-Terminals (PTY)

# Pseudo-Terminals (PTY)

## Overview

A **pseudo-terminal** makes a program believe it is attached to a real terminal: line discipline, window size, and `isatty` checks. You need PTYs when automating interactive tools (`ssh`, `sudo`, password prompts, TUI apps) from a non-interactive parent.

Stdlib does **not** ship a full PTY API. Common approach: [github.com/creack/pty](https://github.com/creack/pty) (or platform `x/sys` + careful setup).

## TTY detection (stdlib)

```
func isTerminal(f *os.File) bool {
 fi, err := f.Stat()
 if err != nil {
 return false
 }
 return fi.Mode() & os.ModeCharDevice != 0
}
```

CLIs use this for color, paging, and prompts (chapter 145).

## Why pipes are not enough

```
parent --pipe--> child
```

Many programs disable interactive features when stdin is a pipe (Not a `tty`). A PTY pair fixes that:

```
parent <-> PTY master <-> PTY slave <-> child (thinks it's a terminal)
```

## Start a command under a PTY (creack/pty)

```
go get github.com/creack/pty@latest

import (
 "io"
 "os"
 "os/exec"

 "github.com/creack/pty"
)

func runInPTY(name string, args ...string) error {
 cmd := exec.Command(name, args...)
 ptmx, err := pty.Start(cmd)
 if err != nil {
 return err
 }
 defer ptmx.Close()

 // copy PTY <-> user terminal
 go io.Copy(ptmx, os.Stdin)
 _, err = io.Copy(os.Stdout, ptmx)
 return err
}

go run . -- bash -l # interactive shell session sketch
```

## Window size

```
// propagate terminal resize (Unix)
_ = pty.InheritSize(os.Stdin, ptmx)

// listen for SIGWINCH and call InheritSize again
```

Without this, full-screen TUIs mis-draw after resize.

## Scripted interaction

```
ptmx, err := pty.Start(exec.Command("mysql", "-u", "root", "-p"))
// carefully write password + "\n" after prompt detection
// prefer non-interactive flags when possible
```

**Security:** automating passwords is fragile and audit-hostile—prefer socket auth, env files with strict modes, or API tokens.

## When to avoid PTYs

Prefer	Avoid PTY when
cmd.Stdin + flags/env	App has real non-interactive mode
Expect scripts ( <code>npm test</code> )	You only need stdout capture
Stdlib pipes	Cross-platform simplicity matters

## Minimal tool: runtty

```
runtty -- <command>
allocates PTY, connects to current terminal
```

Useful for wrapping tools that refuse pipes.

## Rules of thumb

Do	Don't
Prefer non-interactive APIs	PTY-scrape fragile prompts in CI
Handle <code>SIGWINCH</code> for TUIs	Assume 80x24 forever
Close master FD	Leak PTYs (FD exhaustion)
Document platform deps	Expect identical PTY behavior on Windows without code

## Try next

1. Run `ls --color=auto` under pipe vs PTY; compare color.
2. Automate a simple `read` shell script via PTY write.
3. Wire resize handling and test with a TUI (`top`, `htop`).

## Resource Limits, rlimit, and Go Knobs

# Resource Limits, rlimit, and Go Knobs

## Overview

Production processes run under **resource ceilings**: open files, memory, CPU, process count. Some are OS-enforced (**rlimit**, **cgroups**); some are Go runtime knobs (**GOMAXPROCS**, **GOMEMLIMIT**). Systems tools should fail clearly when limits bite—and avoid creating the conditions that hit them.

## Discovering limits (Unix)

```
ulimit -n # open files
ulimit -u # max user processes
```

In Go (via `x/sys/unix`):

```
//go:build unix

var lim unix.Rlimit
if err := unix.Getrlimit(unix.RLIMIT_NOFILE, &lim); err == nil {
 fmt.Printf("nofile soft=%d hard=%d\n", lim.Cur, lim.Max)
}
```

## Raising NOFILE (when allowed)

```
lim.Cur = lim.Max
_ = unix.Setrlimit(unix.RLIMIT_NOFILE, &lim)
```

Containers may still enforce a lower cgroup/nproc limit—raising soft rlimit cannot exceed hard or cgroup.

## Common failure modes

Symptom	Typical limit
too many open files	NOFILE / FD leak
cannot allocate memory / OOMKill	RSS cgroup / host RAM
Slow scheduling	CPU quota / throttling

Symptom	Typical limit
fork: resource temporarily unavailable	nproc / PIDs

## Go runtime knobs

Knob	Role
GOMAXPROCS	OS threads for Go code (auto-tuned in containers on recent Go)
GOMEMLIMIT	Soft memory limit; GC aims to stay under
GOGC	GC percent trigger
runtime/debug.SetMemoryLimit	Same family as GOMEMLIMIT

```
import "runtime/debug"

debug.SetMemoryLimit(2 << 30) // 2 GiB soft limit
```

See deep dives for pacer/GC details; here the systems takeaway is: **set limits intentionally in containers.**

## Bound your own resource use

```
// concurrency cap
sem := make(chan struct{}, 32)

// HTTP client
t := http.DefaultTransport.(*http.Transport).Clone()
t.MaxIdleConns = 100
t.MaxConnsPerHost = 10

// file workers: don't open 100k files at once
```

## Minimal tool: limits

```
func printLimits(w io.Writer) {
 fmt.Fprintf(w, "gomaxprocs\t%d\n", runtime.GOMAXPROCS(0))
 fmt.Fprintf(w, "numcpu\t%d\n", runtime.NumCPU())
 fmt.Fprintf(w, "numgoroutine\t%d\n", runtime.NumGoroutine())
 var ms runtime.MemStats
 runtime.ReadMemStats(&ms)
```



```

 fmt.Fprintf(w, "heap_alloc\t%d\n", ms.HeapAlloc)
 // optional: Getrlimit NOFILE
}

```

Ship in debug endpoints (GET /debug/limits) carefully—auth or localhost only.

## cgroups (conceptual)

Kubernetes sets CPU/memory requests/limits via cgroups. Go 1.22+ improved GOMAXPROCS container awareness. Still verify:

```

pod memory limit < GOMEMLIMIT strategy
liveness does not restart during GC thrash

```

## Rules of thumb

Do	Don't
Cap concurrency near I/O boundaries	Unbounded <code>go func</code> per request/file
Close FDs; monitor NOFILE	“Just raise ulimit” as only fix
Set GOMEMLIMIT in memory-capped pods	Assume unlimited heap in sidecars
Log clear errors on EMFILE	Busy-loop retry without backoff

## Try next

1. Print soft/hard NOFILE in a binary.
2. Open files until failure; confirm error type.
3. Run with GOMEMLIMIT=100MiB and allocate; observe GC behavior under load.

## **Privileges, Capabilities, and Least Privilege**

# Privileges, Capabilities, and Least Privilege

## Overview

Systems software often needs a **little** privilege (bind port 80, open raw sockets, read another user's files)—not a permanent root shell. This chapter is a **minimal, Go-oriented** map of least privilege: run as non-root, drop what you can, and prefer orchestrator policy over home-grown setuid.

Not a full Linux security course—pair with [17 Security hardening](#).

## Prefer external policy

Mechanism	Who configures	Example
Kubernetes <code>securityContext</code>	Platform	<code>runAsNonRoot</code> , drop caps
systemd <code>User=</code> / <code>AmbientCapabilities=</code>	Operator	<code>DynamicUser</code> , <code>CAP_NET_BIND_SERVICE</code>
Container rootless	Runtime	podman/docker user ns

Your Go code should **assume it might be non-root** and fail with a clear error when a needed privilege is missing.

## Check identity early

```
if os.Geteuid() == 0 {
 slog.Warn("running as root - prefer dedicated user")
}

ln, err := net.Listen("tcp", ":80")
if err != nil {
 return fmt.Errorf("listen :80 (need privilege or CAP_NET_BIND_SERVICE): %w", err)
}
```

## Bind low ports without full root

Linux **capabilities** split root power. `CAP_NET_BIND_SERVICE` allows bind <1024 without full UID 0.

Typical systemd:

```
[Service]
User=myapp
AmbientCapabilities=CAP_NET_BIND_SERVICE
CapabilityBoundingSet=CAP_NET_BIND_SERVICE
NoNewPrivileges=true
```

Go app still just calls `Listen`—no special code.

## setuid binaries (avoid if possible)

Historically, `setuid-root` helpers did privileged ops then dropped. Risks are high (TOCTOU, env attacks). Prefer:

- systemd socket activation (privileged unit hands over FD)
- privileged sidecar
- `CAP_*` ambient on the service user

If you must touch UIDs:

```
// conceptual - platform-specific, error-prone
// unix.Setuid, Setgid after privileged setup
```

Test extensively; do not invent `setuid` stacks casually.

## File permissions as privilege

```
config dir 0750 root:myapp
config file 0640 root:myapp
data dir 0700 myapp:myapp
socket 0660 myapp:myapp
```

Go: create with tight modes (chapter 144 `umask` + `chmod`).

## Secrets

Prefer	Avoid
Files 0600 + dedicated user	World-readable env in ps dump discussions
Runtime mounts (K8s secrets)	Baking secrets into images
Short-lived tokens	Long-lived root API keys in config

## Minimal pattern: refuse unsafe start

```
func requireSecurePaths(cfgDir string) error {
 st, err := os.Stat(cfgDir)
 if err != nil {
 return err
 }
 if st.Mode().Perm() & 0o077 != 0 {
 return fmt.Errorf("%s mode %#o is too open", cfgDir, st.Mode().Perm())
 }
 return nil
}
```

Optional hard fail in production profiles.

## Rules of thumb

Do	Don't
Run non-root by default	Require root “for convenience”
Use capabilities/socket activation	Custom setuid without review
Clear errors on EPERM/EACCES	Silent fallback to insecure paths
Tight file modes on secrets	<code>chmod 777</code> to “make it work”

## Try next

1. Run a server as non-root; bind `:8080` vs `:80`; document the error.
2. Read your systemd unit / K8s securityContext for one service.
3. Add a startup check that warns on world-writable config dirs.

## **Syslog, Journal, and Ops Logging**

# Syslog, Journal, and Ops Logging

## Overview

Systems daemons historically logged to **syslog**; under systemd, messages often land in the **journal**. Go services commonly use **log/slog** to stderr (captured by the supervisor) **or** write to syslog. Pick one clear path so operators know where to **tail**.

## Default modern pattern: stderr + supervisor

```
h := slog.NewJSONHandler(os.Stderr, &slog.HandlerOptions{Level: slog.LevelInfo})
slog.SetDefault(slog.New(h))
slog.Info("started", "pid", os.Getpid())
```

Supervisor	Where logs go
systemd	journal ( <code>journalctl -u myapp</code> )
Kubernetes	container runtime → log driver
Docker	<b>docker logs</b>
cron	mail / captured by cron daemon

**Prefer this** for new services: no syslog dependency, works in all containers.

## When to use syslog

- Policy requires `/dev/log`
- Multi-process host with centralized rsyslog/syslog-ng
- Very old deployment molds

```
import "log/syslog"

w, err := syslog.Dial("", "", syslog.LOG_INFO|syslog.LOG_DAEMON, "myapp")
if err != nil {
 // fall back to stderr
}
```

```
defer w.Close()
slog.SetDefault(slog.New(slog.NewTextHandler(io.MultiWriter(os.Stderr, w), nil)))
```

Network syslog (Dial("udp", "logger:514", ...)) needs reliability/security thought (TLS, auth)—often better handled by a local agent.

## Levels and severity

slog	syslog-ish
Debug	DEBUG
Info	INFO
Warn	WARNING
Error	ERR

Don't log at Error for routine misses (cache miss   outage).

## Structured fields for ops

```
slog.Info("child_exit",
 "cmd", name,
 "code", exitCode,
 "dur_ms", dur.Milliseconds(),
)
```

Include: request/job id, host, version, signal reason.

## Avoiding log loops

If your program ships logs by watching files it also writes, guard paths (don't ingest your own output). For agents, use a separate logging sink.

## PID 1 and stdout

In containers, PID 1's stdout **is** the log stream. Don't background and discard stdout. Don't write secrets.



## Minimal dual-sink setup

```
func setupLog(json bool) {
 var h slog.Handler
 opts := &slog.HandlerOptions{Level: slog.LevelInfo}
 if json {
 h = slog.NewJSONHandler(os.Stderr, opts)
 } else {
 h = slog.NewTextHandler(os.Stderr, opts)
 }
 slog.SetDefault(slog.New(h))
}
```

CLI flag `--log-json` for production; text for local.

## Rules of thumb

Do	Don't
stderr + supervisor for new apps	Invent custom log files without rotation plan
Structured key/value	Multiline free-form only
One correlation id field	Log passwords, tokens, cookies
Document <code>journalctl</code> / <code>kubectl logs</code>	Assume operators know your private log path

## Try next

1. Run under `systemd-run --user` and find logs with `journalctl --user`.
2. Emit JSON slog; parse with `jq`.
3. Add `version` and `pid` to every startup log line.

## **Project: Systems Ops Toolkit**

# Project: Systems Ops Toolkit

## Overview

Extend the systems story with **sysops**, a second multi-command project focused on **runtime ops**: deadlines, config reload, limits, locks + jobs, and local control sockets. Builds on [149 systool](#).

Command	Chapter skills
<code>deadline</code>	timers, <code>CommandContext</code>
<code>reload</code>	file poll + atomic config swap
<code>limits</code>	runtime + optional rlimit
<code>cronish</code>	ticker + once lock
<code>healthsock</code>	UDS HTTP health
<code>rotate</code>	simple size-based log copy/truncate (careful)
<code>envcheck</code>	required env vars present

## Scaffold

```
mkdir -p sysops/cmd/sysops sysops/internal/app
cd sysops
go mod init example.com/sysops
```

Use `signal.NotifyContext`, exit 2 on usage, stderr diagnostics.

### 1. deadline — run with timeout

```
func cmdDeadline(ctx context.Context, args []string) error {
 fs := flag.NewFlagSet("deadline", flag.ContinueOnError)
 max := fs.Duration("t", 30*time.Second, "max runtime")
 if err := fs.Parse(args); err != nil {
 return errUsage
 }
 if fs.NArg() < 1 {
 return fmt.Errorf("usage: sysops deadline -t 5s -- cmd args")
 }
}
```

```

 cctx, cancel := context.WithTimeout(ctx, *max)
 defer cancel()
 cmd := exec.CommandContext(cctx, fs.Arg(0), fs.Args()[1:]...)
 cmd.Stdout, cmd.Stderr = os.Stdout, os.Stderr
 err := cmd.Run()
 if cctx.Err() == context.DeadlineExceeded {
 return fmt.Errorf("deadline exceeded (%s)", *max)
 }
 return err
}

go run ./cmd/sysops deadline -t 1s -- sleep 10

```

---

## 2. reload — demo config watcher

```

// poll path every 200ms; on change print new contents length
// store last good bytes in atomic.Value

type box struct{ b []byte }

func cmdReload(ctx context.Context, path string) error {
 var v atomic.Value
 load := func() error {
 b, err := os.ReadFile(path)
 if err != nil {
 return err
 }
 if !json.Valid(b) { // example: require JSON
 return fmt.Errorf("invalid json")
 }
 v.Store(box{b: append([]byte(nil), b...)})
 fmt.Fprintf(os.Stderr, "reloaded %d bytes\n", len(b))
 return nil
 }
 _ = load()
 var lastMod time.Time
 var lastSz int64
 t := time.NewTicker(200 * time.Millisecond)
 defer t.Stop()
 for {
 select {

```

```

 case <-ctx.Done():
 return nil
 case <-t.C:
 st, err := os.Stat(path)
 if err != nil {
 continue
 }
 if st.ModTime().Equal(lastMod) && st.Size() == lastSz {
 continue
 }
 lastMod, lastSz = st.ModTime(), st.Size()
 if err := load(); err != nil {
 fmt.Fprintln(os.Stderr, "reload keep old:", err)
 }
 }
}

```

---

### 3. limits — print runtime snapshot

```

func cmdLimits(w io.Writer) {
 fmt.Fprintf(w, "gomaxprocs\t%d\n", runtime.GOMAXPROCS(0))
 fmt.Fprintf(w, "numcpu\t%d\n", runtime.NumCPU())
 fmt.Fprintf(w, "goroutines\t%d\n", runtime.NumGoroutine())
 var m runtime.MemStats
 runtime.ReadMemStats(&m)
 fmt.Fprintf(w, "heap_alloc\t%d\n", m.HeapAlloc)
 fmt.Fprintf(w, "sys\t%d\n", m.Sys)
}

```

---

### 4. cronish — interval runner with flock

```

sysops cronish -every 1m -lock /tmp/job.lock -- /path/backup.sh

```

```

// each tick: try Acquire lock; if busy skip; else run command
// respects ctx cancel between ticks

```

Prevents stacked cron jobs (chapter 148).

---

## 5. healthsock — UDS health for local probes

```
// Listen unix; GET /healthz returns 200 + limits snapshot
// Chmod 660; remove stale socket on start
// Shutdown on ctx

curl --unix-socket /tmp/sysops.sock http://localhost/healthz
```

---

## 6. rotate — minimal size rotate (teaching only)

```
// if file size > max: rename to .1 (overwrite), create new file
// NOT a full logrotate replacement - no copytruncate races covered fully

func rotateIfLarge(path string, max int64) error {
 st, err := os.Stat(path)
 if err != nil {
 return err
 }
 if st.Size() < max {
 return nil
 }
 bak := path + ".1"
 _ = os.Remove(bak)
 if err := os.Rename(path, bak); err != nil {
 return err
 }
 f, err := os.OpenFile(path, os.O_CREATE|os.O_WRONLY, 0o644)
 if err != nil {
 return err
 }
 return f.Close()
}
```

Document that writers must reopen after rotate (or use stderr→journal instead).

---

## 7. envcheck — required environment

```
func cmdEnvCheck(keys []string) error {
 var missing []string
 for _, k := range keys {
 if os.Getenv(k) == "" {
 missing = append(missing, k)
 }
 }
 if len(missing) > 0 {
 return fmt.Errorf("missing env: %s", strings.Join(missing, ", "))
 }
 fmt.Println("ok")
 return nil
}
```

```
go run ./cmd/sysops envcheck DATABASE_URL API_TOKEN
```

Useful as a container entrypoint preflight.

---

## Acceptance checklist

- ☐ deadline -t 1s -- sleep 5 fails with timeout
- ☐ reload keeps last good config on bad JSON
- ☐ cronish skips overlapping runs under lock
- ☐ healthsock + curl works
- ☐ envcheck exits non-zero when vars missing
- ☐ Ctrl-C stops long-running commands promptly

## Suggested implementation order

```
envcheck → limits → deadline → healthsock → reload → cronish → rotate
```

## Stretch

1. Merge `systool` + `sysops` into one binary with command groups
2. JSON output for `limits` / `healthz`
3. Debounce reload at 300ms
4. systemd unit file example in README

You now have two project tracks: **149** `systool` (core Unix hygiene) and **156** `sysops` (runtime operations)—together a practical systems programming lab in pure Go style.



## **systemd Socket Activation**

# systemd Socket Activation

## Overview

**Socket activation:** systemd (or another supervisor) binds the socket and passes the FD to your process. Benefits: on-demand start, privilege separation (bind :80 as root, run app as user), faster parallel startup.

## How FDs arrive

systemd sets:

```
LISTEN_FDS=1
LISTEN_PID=<pid>
```

FDs start at **3** (SD\_LISTEN\_FDS\_START).

## Minimal accept (stdlib)

```
func listenersFromSystemd() ([]net.Listener, error) {
 pid, _ := strconv.Atoi(os.Getenv("LISTEN_PID"))
 if pid != os.Getpid() && os.Getenv("LISTEN_PID") != "" {
 return nil, fmt.Errorf("LISTEN_PID mismatch")
 }
 n, _ := strconv.Atoi(os.Getenv("LISTEN_FDS"))
 if n < 1 {
 return nil, nil
 }
 var out []net.Listener
 for i := 0; i < n; i++ {
 fd := 3 + i
 f := os.NewFile(uintptr(fd), "systemd")
 ln, err := net.FileListener(f)
 f.Close() // FileListener dups
 if err != nil {
 return nil, err
 }
 out = append(out, ln)
 }
}
```

```

 }
 return out, nil
}

lns, err := listenersFromSystemd()
if len(lns) == 0 {
 // fallback Listen for local dev
 ln, _ := net.Listen("tcp", *addr)
 lns = []net.Listener{ln}
}
srv := &http.Server{Handler: mux}
_ = srv.Serve(lns[0])

```

Libraries: coreos/go-systemd/activation for production edge cases.

## Unit sketch

```

myapp.socket
[Socket]
ListenStream=80

myapp.service
[Service]
ExecStart=/usr/local/bin/myapp
NonBlocking=true

```

## Notify readiness

```

// optional: sd_notify READY=1 via NOTIFY_SOCKET

```

## Rules

Do	Don't
Fallback Listen for dev	Require systemd on laptops
Verify LISTEN_PID	Assume FD 3 always valid
Shutdown via Server.Shutdown	Ignore inherited FDs on exit

## Try next

1. Local: `systemd-socket-activate -l 8080 ./myapp`
2. Dual mode: env set  $\rightarrow$  use FD; else TCP.
3. Pass Unix socket via activation.

# Linux procfs Introspection

# Linux procfs Introspection

## Overview

On Linux, `/proc` exposes live kernel views of processes, FDs, and networking. Handy for **debug CLIs** and self-inspection—not a portable API (guard with build tags).

## Useful paths

Path	Info
<code>/proc/self/status</code>	Pid, VmRSS, threads, fds
<code>/proc/self/fd</code>	Open file descriptors
<code>/proc/self/limits</code>	rlimits
<code>/proc/self/net/tcp</code>	TCP sockets (parse carefully)
<code>/proc/meminfo</code>	Host memory

## Count FDs

```
//go:build linux

func fdCount() (int, error) {
 ents, err := os.ReadDir("/proc/self/fd")
 if err != nil {
 return 0, err
 }
 return len(ents), nil
}
```

## Read VmRSS

```
b, err := os.ReadFile("/proc/self/status")
// parse line "VmRSS:\t12345 kB"
```

## Minimal tool: procsel

```
systool procsel
 fds N
 vmrss N kB
```

## Cgroup memory (containers)

```
/sys/fs/cgroup/memory.max # v2
/sys/fs/cgroup/memory/memory.limit_in_bytes # v1
```

Parse when present; fall back if missing (not in cgroup).

## Rules

Do	Don't
Build-tag Linux code	Assume <code>/proc</code> on macOS
Treat format as unstable	Parse without tests
Prefer metrics APIs for prod	Scrape proc in hot path every $\mu$ s

## Try next

1. Print fd count before/after intentional leak.
2. Read cgroup memory limit inside Docker.
3. Cross-compile with `//go:build linux` stubs elsewhere.

## **Project: Mini Process Supervisor**



# Project: Mini Process Supervisor

## Overview

Build **minisup**: run multiple child commands, restart on crash, forward signals, aggregate logs—a teaching systemd/overmind lite.

## Spec

```
procfile-ish
web: ./bin/web -addr :8080
worker: ./bin/worker
```

Or flags:

```
minisup -cmd "web=./web" -cmd "worker=./worker"
```

## Requirements

1. Start all children with `Command`
2. Prefix stdout/stderr lines with name
3. On child exit `0` → restart with backoff (cap)
4. `SIGTERM` → signal children, wait with deadline, then `SIGKILL`
5. Exit when all stopped after shutdown

## Signal fanout

```
cmd := exec.Command(bin, args...)
cmd.SysProcAttr = &syscall.SysProcAttr{Setpgid: true} // Unix: kill group
// On stop: syscall.Kill(-pgid, SIGTERM)
```

## Log prefix

```
cmd.Stdout = prefixWriter{name: "web", w: os.Stdout}
```

## Acceptance

- ☐ Two children stay up
- ☐ Kill one; it restarts
- ☐ Ctrl-C stops both within 5s
- ☐ Max restarts then give up with error

## Stretch

- Health: restart if TCP port fails dial
- Procfile parser
- Status UDS: `minisup status`

Chapters: 141, 145, 150, 153.

# **Distributed Infrastructure**

# Distributed Infrastructure Overview

# Distributed Infrastructure Overview

This part introduces distributed reliability patterns used in production Go systems. The objective is to reason about **failure as a normal operating condition**, not an exception path you only sketch in design docs.

## Why / Overview

A single-process Go service is relatively easy. A system of services is not:

- Instances appear and disappear
- Networks delay, drop, and duplicate
- Dependencies degrade partially
- Retries turn brownouts into outages
- Queues hide problems until they explode

```
clients
 |
 v
[discovery / LB] -> service A -> service B
 | |
 queue/workers cache/db
```

This part focuses on three control planes you will reimplement (or configure) forever:

1. **Where is healthy capacity?** (discovery + health)
2. **How do we call others safely?** (retry + circuit breaking)
3. **How do we absorb load spikes?** (queues + backpressure)

## Learning Path

Chapter	Focus	Outcome
151 Discovery & health	Registration, TTL, readiness	Route only to safe instances
152 Retry & circuit breaker	Control loops	Avoid amplification cascades
153 Queues & workers	Buffering, concurrency, poison messages	Degrade gracefully under overload
154 Caching	TTL, singleflight, cache-aside	Cut load without consistency disasters

Chapter	Focus	Outcome
155 Idempotency	Keys, dedupe, unique constraints	Safe retries on POST/payments
156 Leader election	Leases, renew/cancel	Singleton loops without split brain
157 Transactional outbox	Same-TX events + relay	Reliable side effects
158 Project: resilient worker	Retry, DLQ, shutdown	End-to-end worker lab

## Core Mental Models

### Partial failure

In distributed systems, “up” is not boolean. An instance can:

- Accept TCP but fail application readiness
- Pass health while serving  $10\times$  latency
- Succeed writes but fail reads

Design for **degraded modes**: serve stale cache, disable non-critical features, fail fast on optional deps.

### Amplification

```

1 user request
 -> 1 API
 -> 3 retries
 -> each fans out to 5 backends
 = 15 backend calls (and more if each retries)

```

Always multiply retry policy  $\times$  fan-out  $\times$  client count.

### Backpressure vs buffering

Approach	Effect	Risk
Drop	Protect system	Data loss
Queue	Smooth spikes	Latency + memory
Reject (503)	Signal clients	Client must handle
Slow down producers	True backpressure	Needs cooperation

Go channels are in-process queues; they do not replace durable external queues when you need restart survival.

## Go Building Blocks

- context for deadlines across hops
- net/http clients with tuned transports
- golang.org/x/sync/errgroup for bounded fan-out
- sync.Mutex / atomics for in-memory breakers
- Workers: goroutines + channels or external brokers (NATS, SQS, Kafka, RabbitMQ)

```
g, ctx := errgroup.WithContext(ctx)
g.SetLimit(8) // Go 1.20+
for _, id := range ids {
 id := id
 g.Go(func() error {
 return fetch(ctx, id)
 })
}
if err := g.Wait(); err != nil {
 return err
}
```

## Production Principles

1. **Timeouts everywhere** — no unbounded waits.
2. **Idempotency** — safe retries need safe operations.
3. **Explicit queues** — size, DLQ, visibility timeout documented.
4. **Health is routing data** — not a vanity endpoint.
5. **Observable control state** — breaker open, queue depth, retry counts as first-class metrics.
6. **Load test failure** — chaos and dependency brownouts in staging.

## Relationship to Other Parts

- **Network systems (13)** — transport budgets feed retry decisions.
- **Observability (16)** — you cannot tune breakers without metrics/traces.
- **Security (17)** — identity and mTLS between discovered peers.
- **Performance (18)** — worker counts and contention under load.

## Literacy Checklist

- ☐ Distinguish liveness vs readiness
- ☐ Explain TTL registration and stale endpoint risk
- ☐ Design retry with jitter and attempt caps
- ☐ Draw circuit breaker states and transitions
- ☐ Size a worker pool against dependency capacity

- ☐ Define poison-message handling and DLQ
- ☐ Compute amplification for a proposed retry policy

## Part Warm-Up Exercises

1. Inventory outbound calls in a service; list timeout and retry for each.
2. Compute worst-case fan-out for your hottest API.
3. Find your health endpoints; do they check dependencies?
4. Measure queue depth (if any) during a past incident.
5. Write a one-paragraph policy: “when dependency X is down, we ...”

## Worked Example: Amplification Math

Suppose:

- 1,000 RPS at the edge
- Each request fans out to 4 dependencies
- Each hop retries up to 3 times on failure
- Failure rate at one dependency is 10%

Rough upper bound if every layer retries naively:

```
edge RPS × fan-out × attempts
= 1000 × 4 × 3
= 12,000 dependency calls/sec peak intent
```

During a brownout, effective load can climb further as queues rebuild. This is why attempt caps, breakers, and single-owner retry policy matter more than micro-optimizing JSON.

## Decision Guide

Problem	First pattern to consider
Clients hardcode hosts	Discovery + health
Blips become user errors	Retry with jitter (idempotent)
Dependency melt under retries	Circuit breaker
Spikes OOM the API process	Queue + backpressure / 503
Duplicate side effects	Idempotency keys / dedupe store
Poison payload spins forever	DLQ + max receive count



## What This Part Will Not Cover

- Consensus algorithms (Raft/Paxos) in depth
- Exactly-once distributed transactions
- Full service-mesh product configuration

Those matter, but the everyday Go engineer wins more reliability from timeouts, health, retries, and queues done correctly.

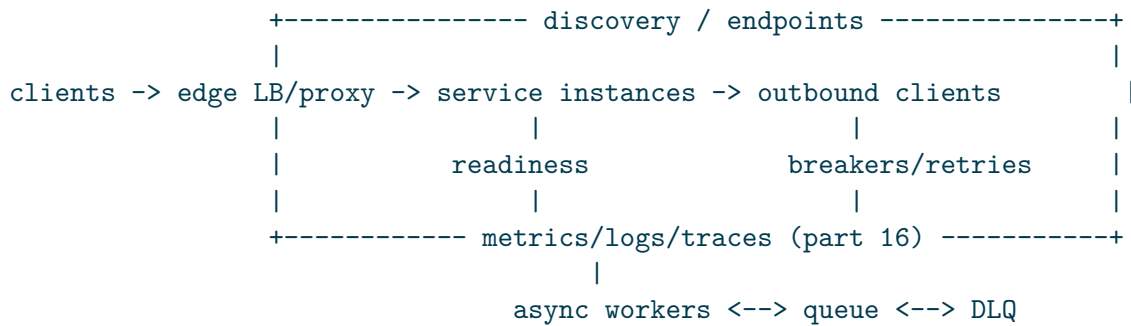
## Incident Starter Questions

When pages fire, ask in order:

1. Is the dependency **down**, **slow**, or **wrong**?
2. Are we **amplifying** with retries?
3. Is **discovery** pointing at zombies?
4. Is a **queue** lagging or a worker stuck?
5. What do **breakers** and **depths** show right now?

If you cannot answer from dashboards, instrument before rewriting architecture.

## Reference Architecture (Mental Model)



You do not need every box on day one. Add boxes when incidents demand them — but know which box is missing when they do.

## Configuration Knobs Inventory

Keep a living document per service:

Knob	Example	Owner
outbound timeout	300ms	service
retry max	2	service

Knob	Example	Owner
breaker threshold	5 fails / 10s	service
worker count	32	service
queue depth alert	10k	platform
probe interval	5s	platform

Unknown knobs default to dangerous framework defaults.

## Testing Strategy

Layer	Technique
Unit	classify errors; pure policy functions
Integration	fake dependency with flaky latency
Chaos	kill pods; network delay; certificate expiry
Load	sustain + spike; watch depth and p99

Policy without tests regresses the first time someone “simplifies” retries.

## Glossary

- **Readiness** — safe to receive traffic.
- **TTL registration** — auto-expire stale instances.
- **Idempotent** — safe to apply twice.
- **Half-open** — breaker probing recovery.
- **Backpressure** — slowing or rejecting producers.
- **DLQ** — dead-letter queue for poison messages.
- **Amplification** — retries × fan-out × clients multiplying load.
- **Visibility timeout** — lease duration before a queue redelivers.

## More examples

### Tour: health gate + retry budget

```
mkdir -p /tmp/go-dist-tour && cd /tmp/go-dist-tour
go mod init example.com/dist-tour
```

Save as `main.go`:

```
package main
```

```

import (
 "context"
 "fmt"
 "time"
)

func healthy() bool { return true }

func call(ctx context.Context, n *int) error {
 *n++
 if *n < 2 {
 return fmt.Errorf("transient")
 }
 return ctx.Err()
}

func main() {
 if !healthy() {
 fmt.Println("skip call: not ready")
 return
 }
 ctx, cancel := context.WithTimeout(context.Background(), time.Second)
 defer cancel()
 var n int
 var err error
 for attempt := 1; attempt <= 3; attempt++ {
 err = call(ctx, &n)
 if err == nil {
 break
 }
 time.Sleep(5 * time.Millisecond)
 }
 fmt.Println("attempts:", n, "err:", err)
}

go run .

```

**Expected output:**

```
attempts: 2 err: <nil>
```

## Runnable example

Tour of this part: registry of instances with TTL expiry, a failed call that trips a mini breaker, and a bounded queue reject—three control planes in one process.

```
mkdir -p /tmp/go-dist-tour && cd /tmp/go-dist-tour
go mod init example.com/dist-tour
```

Save as main.go:

```
package main

import (
 "fmt"
 "sync"
 "time"
)

type instance struct {
 addr string
 expires time.Time
}

type registry struct {
 mu sync.Mutex
 inst map[string]instance
}

func (r *registry) heartbeat(addr string, ttl time.Duration) {
 r.mu.Lock()
 r.inst[addr] = instance{addr: addr, expires: time.Now().Add(ttl)}
 r.mu.Unlock()
}

func (r *registry) healthy() []string {
 r.mu.Lock()
 defer r.mu.Unlock()
 now := time.Now()
 var out []string
 for a, i := range r.inst {
 if now.Before(i.expires) {
 out = append(out, a)
 } else {
 delete(r.inst, a)
 }
 }
 return out
}

type breaker struct {
 fails int
 open bool
}
```

```

}

func (b *breaker) call(fn func() error) error {
 if b.open {
 return fmt.Errorf("breaker open")
 }
 if err := fn(); err != nil {
 b.fails++
 if b.fails >= 3 {
 b.open = true
 }
 return err
 }
 b.fails = 0
 return nil
}

func main() {
 reg := ®istry{inst: map[string]instance{}}
 reg.heartbeat("10.0.0.1:8080", 50*time.Millisecond)
 reg.heartbeat("10.0.0.2:8080", time.Second)
 time.Sleep(60 * time.Millisecond)
 fmt.Println("healthy:", reg.healthy())

 var b breaker
 for i := 0; i < 4; i++ {
 err := b.call(func() error { return fmt.Errorf("dep down") })
 fmt.Printf("call %d: %v open=%v\n", i+1, err, b.open)
 }

 q := make(chan string, 2)
 send := func(s string) error {
 select {
 case q <- s:
 return nil
 default:
 return fmt.Errorf("queue full")
 }
 }

 fmt.Println("enq a:", send("a"))
 fmt.Println("enq b:", send("b"))
 fmt.Println("enq c:", send("c"))
 fmt.Println("tour: discovery + breaker + backpressure")
}

go run .

```

### Expected output:

```
healthy: [10.0.0.2:8080]
call 1: dep down open=false
call 2: dep down open=false
call 3: dep down open=true
call 4: breaker open open=true
enq a: <nil>
enq b: <nil>
enq c: queue full
tour: discovery + breaker + backpressure
```

### What to notice

- TTL discovery expires zombies; breakers stop retry storms; bounded queues apply backpressure.
- Chapters 151–153 deepen each control plane.

### Try next

- Add half-open probe recovery to the breaker.
- Drain the queue with N workers and measure lag.

## Next Chapter

**Service Discovery and Health Checks** — knowing who is safe to call.

## **Service Discovery and Health Checks**

# Service Discovery and Health Checks

Discovery systems answer two questions: **where instances are**, and **whether they are currently safe to route to**. Getting either wrong produces systematic 5xx and mysterious latency.

## Why / Overview

Hard-coded IPs die in dynamic environments. DNS, kube endpoints, Consul, Eureka, or a custom registry all solve:

```
instance -> register + heartbeat -> registry
router/client -> resolve -> filter healthy -> call
```

Health is not optional metadata; it is **routing input**.

## Discovery Mechanisms

Mechanism	Pros	Cons
DNS (A/AAAA/SRV)	Simple, universal	TTL lag, limited health
Platform endpoints (K8s)	Integrated readiness	Cluster-centric
Service mesh / lb	Rich policy	Operational weight
Client-side registry	Flexible	Dual-write complexity
Config static list	Debuggable	No dynamism

Many Go services use **DNS or platform discovery** plus **client-side health-aware pools**.

## Registration and TTL

Without TTL/heartbeat, crashed instances remain targets forever.

```
type Registration struct {
 ID string `json:"id"`
 Addr string `json:"addr" // host:port
 Expires time.Time `json:"expires"`
 Meta map[string]string `json:"meta,omitempty"`
}
```



```
// Registry.Put sets Expires = now + TTL; instances heartbeat before expiry.
```

Heartbeat loop:

```
func (c *Agent) heartbeat(ctx context.Context) error {
 t := time.NewTicker(c.ttl / 3)
 defer t.Stop()
 for {
 select {
 case <-ctx.Done():
 return c.registry.Deregister(context.Background(), c.id)
 case <-t.C:
 if err := c.registry.Refresh(ctx, c.id, c.ttl); err != nil {
 slog.Error("heartbeat failed", "err", err)
 }
 }
 }
}
```

On shutdown: best-effort deregister **and** rely on TTL (network can drop deregister).

## Liveness vs Readiness vs Startup

Probe	Question	Failure action
Startup	Has init finished?	Wait; don't kill yet
Liveness	Is process stuck?	Restart process
Readiness	Can it serve traffic?	Remove from LB only

```
var (
 ready atomic.Bool
 started atomic.Bool
)

func handlers() http.Handler {
 mux := http.NewServeMux()
 mux.HandleFunc("GET /healthz", func(w http.ResponseWriter, r *http.Request) {
 // Liveness: process up; avoid dependency checks here.
 w.WriteHeader(http.StatusOK)
 })
 mux.HandleFunc("GET /readyz", func(w http.ResponseWriter, r *http.Request) {
 if !ready.Load() {
 http.Error(w, "not ready", http.StatusServiceUnavailable)
 return
 }
 })
}
```

```

 // Optional: shallow dependency ping with short timeout.
 ctx, cancel := context.WithTimeout(r.Context(), 200*time.Millisecond)
 defer cancel()
 if err := db.PingContext(ctx); err != nil {
 http.Error(w, "db", http.StatusServiceUnavailable)
 return
 }
 w.WriteHeader(http.StatusOK)
})
return mux
}

```

**Anti-pattern:** readiness that always returns 200; liveness that checks the database (DB blip restarts all pods).

## Warmup

```

func main() {
 // listen first for probes if needed, ready=false
 go server.Listen()
 mustConnectDB()
 primeCaches()
 ready.Store(true)
 <-ctx.Done()
 ready.Store(false) // stop new traffic
 shutdown()
}

```

## Client-Side Discovery Pool

```

type Pool struct {
 mu sync.RWMutex
 addrs []string
}

func (p *Pool) Update(addrs []string) {
 p.mu.Lock()
 p.addrs = append([]string(nil), addrs...)
 p.mu.Unlock()
}

func (p *Pool) Pick() (string, error) {
 p.mu.RLock()

```

```

 defer p.mu.RUnlock()
 if len(p.addrs) == 0 {
 return "", errors.New("no endpoints")
 }
 // simple RR or random
 return p.addrs[rand.Intn(len(p.addrs))], nil
}

```

Refresh from DNS:

```

func watchDNS(ctx context.Context, host string, every time.Duration, p *Pool) {
 t := time.NewTicker(every)
 defer t.Stop()
 resolver := net.DefaultResolver
 for {
 addrs, err := resolver.LookupHost(ctx, host)
 if err != nil {
 slog.Warn("dns lookup", "err", err)
 } else {
 // attach port, update pool
 p.Update(withPort(addrs, "8080"))
 }
 select {
 case <-ctx.Done():
 return
 case <-t.C:
 }
 }
}

```

Respect DNS TTL when possible; aggressive refresh can stampede resolvers.

## Passive Health and Outlier Detection

Active probes cost load; passive detection uses real traffic:

```

connect error or 5xx streak >= N -> mark bad for cooldown
after cooldown, allow probe request (half-open style)

```

Combine with circuit breakers (next chapter) so discovery and breakers do not fight each other — define a single “eligible endpoints” set.

## Consistency and Split-Brain

Registries can lag:

- Client A sees endpoints {1,2}
- Client B sees {2,3}

Design for **eventual consistency**: idempotent handlers, no single-instance assumptions without leases/locks.

Leader election (if needed) is a separate, harder problem — prefer cloud primitives or etcd/Consul sessions over home-grown locks unless you must.

## Security Notes

- Authenticate registration if the registry is multi-tenant.
- Prefer mTLS to discovered peers (part 17).
- Do not put secrets in registry metadata.
- Validate advertised addresses (SSRF: instance claims 169.254.169.254).

## Observability

Metrics:

- `discovery_endpoints{service}` gauge
- `discovery_refresh_errors_total`
- `healthcheck_fail_total{type=live|ready}`
- `endpoint_selected_total{addr}` (careful cardinality — hash or sample)

Logs: registration, deregistration, ready transitions.

## Production Checklist

- ☐ Distinct live vs ready endpoints
- ☐ Ready checks dependencies with short timeouts
- ☐ Live checks do not restart on dep failure
- ☐ TTL or platform removal on crash
- ☐ Graceful deregister + ready=false before shutdown
- ☐ Client pool handles empty set (fail fast)
- ☐ Outlier / passive health for sticky failures
- ☐ Metrics for endpoint count and probe failures
- ☐ Document discovery mode (DNS/K8s/custom) in runbooks

## Common Pitfalls

1. **Only liveness** — traffic during warmup/migrations fails.
2. **Deep checks on liveness** — coordinated restarts under DB blip.
3. **No TTL** — zombie instances.
4. **Long DNS TTL** — slow failover (or too short — extra load).
5. **Ready depends on optional downstream** — optional features take the whole service out.
6. **Health handler allocates heavily** — becomes its own outage under probe load.
7. **Publishing localhost** in registry from misconfigured agents.

## Exercises

1. Implement `/healthz` and `/readyz`; toggle ready with an atomic; prove LB behavior with a tiny proxy.
2. Add DB ping to ready with 100ms timeout; kill DB; confirm live still 200, ready 503.
3. Write a TTL registry in memory; stop heartbeats; observe expiry.
4. DNS pool: resolve a name with multiple A records; pull one instance; see traffic shift (use `/etc/hosts` hacks in lab).
5. Simulate slow ready handler (2s); set probe timeout 200ms; fix the handler.
6. On `SIGTERM`, set ready false, sleep 2s, then exit; describe how this helps connection draining.
7. Implement passive health: three failures mark endpoint down for 10s.
8. Attempt SSRF-style registration address; add allowlist validation.
9. Graph endpoint count during a rolling deploy.
10. Write a runbook: “endpoint count dropped to zero.”

## More examples

### In-memory registry with health filter

```
mkdir -p /tmp/go-sd-reg && cd /tmp/go-sd-reg
go mod init example.com/sd-reg
```

Save as `main.go`:

```
package main

import (
 "fmt"
 "sync"
)

type Endpoint struct {
 ID string
 Addr string
```

```

 Healthy bool
}

type Registry struct {
 mu sync.RWMutex
 byID map[string]Endpoint
}

func NewRegistry() *Registry {
 return &Registry{byID: map[string]Endpoint{}}
}

func (r *Registry) Upsert(e Endpoint) {
 r.mu.Lock()
 r.byID[e.ID] = e
 r.mu.Unlock()
}

func (r *Registry) Healthy() []Endpoint {
 r.mu.RLock()
 defer r.mu.RUnlock()
 var out []Endpoint
 for _, e := range r.byID {
 if e.Healthy {
 out = append(out, e)
 }
 }
 return out
}

func main() {
 reg := NewRegistry()
 reg.Upsert(Endpoint{ID: "a", Addr: "10.0.0.1:8080", Healthy: true})
 reg.Upsert(Endpoint{ID: "b", Addr: "10.0.0.2:8080", Healthy: false})
 reg.Upsert(Endpoint{ID: "c", Addr: "10.0.0.3:8080", Healthy: true})
 h := reg.Healthy()
 fmt.Println("healthy count:", len(h))
 reg.Upsert(Endpoint{ID: "b", Addr: "10.0.0.2:8080", Healthy: true})
 fmt.Println("after b recovers:", len(reg.Healthy()))
}

go run .

```

### Expected output:

```

healthy count: 2
after b recovers: 3

```

## /healthz and /readyz with dependency flag

```
mkdir -p /tmp/go-health-ep && cd /tmp/go-health-ep
go mod init example.com/health-ep
```

Save as main.go:

```
package main

import (
 "fmt"
 "net/http"
 "net/http/httptest"
 "sync/atomic"
)

func main() {
 var ready atomic.Bool
 mux := http.NewServeMux()
 mux.HandleFunc("GET /healthz", func(w http.ResponseWriter, _ *http.Request) {
 w.WriteHeader(200)
 fmt.Fprintln(w, "ok")
 })
 mux.HandleFunc("GET /readyz", func(w http.ResponseWriter, _ *http.Request) {
 if !ready.Load() {
 http.Error(w, "not ready", 503)
 return
 }
 fmt.Fprintln(w, "ready")
 })
 ts := httptest.NewServer(mux)
 defer ts.Close()

 r, _ := http.Get(ts.URL + "/healthz")
 r.Body.Close()
 fmt.Println("live:", r.StatusCode)
 r, _ = http.Get(ts.URL + "/readyz")
 r.Body.Close()
 fmt.Println("ready cold:", r.StatusCode)
 ready.Store(true)
 r, _ = http.Get(ts.URL + "/readyz")
 r.Body.Close()
 fmt.Println("ready warm:", r.StatusCode)
}

go run .
```

## Expected output:

```
live: 200
ready cold: 503
ready warm: 200
```

## Runnable example

In-memory registry with heartbeats/TTL plus /healthz vs /readyz on an `httptest` server—live during warmup, ready only when marked.

```
mkdir -p /tmp/go-discovery && cd /tmp/go-discovery
go mod init example.com/discovery
```

Save as `main.go`:

```
package main

import (
 "fmt"
 "net/http"
 "net/http/httptest"
 "sync"
 "sync/atomic"
 "time"
)

type registry struct {
 mu sync.Mutex
 ttl map[string]time.Time
}

func newRegistry() *registry { return ®istry{ttl: map[string]time.Time{}} }

func (r *registry) upsert(addr string, ttl time.Duration) {
 r.mu.Lock()
 r.ttl[addr] = time.Now().Add(ttl)
 r.mu.Unlock()
}

func (r *registry) list() []string {
 r.mu.Lock()
 defer r.mu.Unlock()
 now := time.Now()
 var out []string
 for a, exp := range r.ttl {
```



```

 if now.Before(exp) {
 out = append(out, a)
 } else {
 delete(r.ttl, a)
 }
 }
 return out
}

func main() {
 var ready atomic.Bool
 mux := http.NewServeMux()
 mux.HandleFunc("GET /healthz", func(w http.ResponseWriter, _ *http.Request) {
 w.WriteHeader(http.StatusOK)
 _, _ = w.Write([]byte("ok"))
 })
 mux.HandleFunc("GET /readyz", func(w http.ResponseWriter, _ *http.Request) {
 if !ready.Load() {
 http.Error(w, "not ready", http.StatusServiceUnavailable)
 return
 }
 w.WriteHeader(http.StatusOK)
 _, _ = w.Write([]byte("ready"))
 })
 ts := httptest.NewServer(mux)
 defer ts.Close()

 check := func(path string) int {
 resp, err := http.Get(ts.URL + path)
 if err != nil {
 return 0
 }
 defer resp.Body.Close()
 return resp.StatusCode
 }

 fmt.Println("live:", check("/healthz"), "ready:", check("/readyz"))
 ready.Store(true)
 fmt.Println("after warmup ready:", check("/readyz"))

 reg := newRegistry()
 reg.upsert(ts.URL, 40*time.Millisecond)
 reg.upsert("http://10.0.0.9:8080", time.Second)
 time.Sleep(50 * time.Millisecond)
 fmt.Println("registry after ttl:", reg.list())
}

```

```
go run .
```

### Expected output:

```
live: 200 ready: 503
after warmup ready: 200
registry after ttl: [http://10.0.0.9:8080]
```

### What to notice

- Liveness readiness; LB should wait on ready during migrations.
- TTL registration removes instances that stop heartbeating (zombies).
- Health handlers must stay cheap—slow probes become outages.

### Try next

- On shutdown: set ready false, sleep 2s, then exit (connection drain).
- Passive health: three consecutive proxy errors mark an addr down for 10s.

## Further Reading

- Kubernetes probes documentation
- Previous part: reverse proxy health-aware pools
- Next: **Retry and Circuit Breaker Patterns**

## Retry and Circuit Breaker Patterns

# Retry and Circuit Breaker Patterns

Retries and circuit breakers are **control systems**. Misconfigured control loops create amplification and cascades. Well-tuned ones improve availability during partial outages.

## Why / Overview

Dependencies fail in bursts. Naive reactions:

- Retry immediately forever → melt the dependency
- Never retry → convert blips into user errors
- Cache open circuits without half-open probes → permanent brownout

```
call fails -> classify -> retryable?
 | |
 no yes -> backoff + jitter -> budget left? -> retry
 |
 no -> fail
```

Circuit breaker adds a fast-path reject when failure rate is high:

```
closed --(too many failures)--> open --(cooldown)--> half-open --(success)--> closed
 ^
 +------(fail)-----+
```

## Error Classification

```
type Class int

const (
 ClassSuccess Class = iota
 ClassRetryable
 ClassFatal
 ClassCaller // 4xx-ish; do not retry
)

func classify(err error, status int) Class {
 if err == nil && status >= 200 && status < 400 {
 return ClassSuccess
 }
```

```

 }
 if errors.Is(err, context.Canceled) {
 return ClassFatal
 }
 if errors.Is(err, context.DeadlineExceeded) {
 return ClassRetryable
 }
 if status == 429 || status == 408 || status >= 500 {
 return ClassRetryable
 }
 if status >= 400 && status < 500 {
 return ClassCaller
 }
 if err != nil {
 return ClassRetryable // network blip heuristic
 }
 return ClassFatal
}

```

Idempotency is orthogonal: a retryable transport failure on a non-idempotent POST is still dangerous.

## Retry with Exponential Backoff and Jitter

```

func retry[T any](ctx context.Context, maxAttempts int, base time.Duration, fn func(context.
 var zero T
 var last error
 backoff := base
 for attempt := 1; attempt <= maxAttempts; attempt++ {
 v, err := fn(ctx)
 if err == nil {
 return v, nil
 }
 last = err
 if classify(err, 0) != ClassRetryable || attempt == maxAttempts {
 return zero, last
 }
 // Full jitter: sleep random [0, backoff]
 delay := time.Duration(rand.Int63n(int64(backoff)))
 select {
 case <-ctx.Done():
 return zero, ctx.Err()
 case <-time.After(delay):
 }
 backoff *= 2
 }

```

```

 if backoff > 5*time.Second {
 backoff = 5 * time.Second
 }
 }
 return zero, last
}

```

## Policy parameters

Param	Guidance
maxAttempts	2–3 for user-facing; more only for background
base delay	20–100ms typical
max delay	Cap to protect tail latency
total budget	Parent context deadline is the true cap
methods	GET/HEAD/PUT safe; POST needs idempotency key

## Decorrelated jitter (alternative)

Often better under correlated failure than pure exponential; see AWS architecture blog on backoff. Implement when you have many clients.

## Idempotency Keys

```

req.Header.Set("Idempotency-Key", key) // client-generated UUID
// server stores key -> result for retention window

```

Without server support, client retries of POST are a product decision, not just a transport decision.

## Circuit Breaker Implementation (Educational)

```

type state int

const (
 stateClosed state = iota
 stateOpen
 stateHalfOpen
)

type Breaker struct {

```

```

 mu sync.Mutex
 st state
 failures int
 successes int
 openUntil time.Time
 failThreshold int
 successThreshold int
 cooldown time.Duration
}

func (b *Breaker) Allow() bool {
 b.mu.Lock()
 defer b.mu.Unlock()
 now := time.Now()
 switch b.st {
 case stateOpen:
 if now.After(b.openUntil) {
 b.st = stateHalfOpen
 b.successes = 0
 return true // probe
 }
 return false
 default:
 return true
 }
}

func (b *Breaker) Report(err error) {
 b.mu.Lock()
 defer b.mu.Unlock()
 if err == nil {
 b.failures = 0
 if b.st == stateHalfOpen {
 b.successes++
 if b.successes >= b.successThreshold {
 b.st = stateClosed
 }
 }
 return
 }
 b.failures++
 if b.st == stateHalfOpen || b.failures >= b.failThreshold {
 b.st = stateOpen
 b.openUntil = time.Now().Add(b.cooldown)
 b.failures = 0
 }
}

```

Usage:

```
if !breaker.Allow() {
 return ErrFailFast
}
v, err := call(ctx)
breaker.Report(err)
```

Production libraries (`sony/gobreaker`, resilience middleware) add sliding windows and metrics — use them when possible; understand the state machine first.

## Combining Retry and Breaker

Order matters:

```
if !breaker.Allow() -> fail fast (no retry storm)
else attempt with retry **inside** half-open carefully (often single probe)
```

Do **not** let every request retry while open; the breaker already decided.

## Budgets and Hedging

**Hedging** (send a second request if the first is slow) can reduce tail latency but doubles load — only for idempotent, cheap reads, with strict limits.

Prefer fixing the slow dependency and using proper deadlines.

## Multi-Layer Retry Coordination

Layer	Should retry?
Browser	Limited
Edge proxy	GET only, 1×
Service mesh	Align with app
App HTTP client	Primary policy owner
DB driver	Often internal; know it

Document **one primary** retry layer for each hop to avoid multiplicative attempts.



## Observability

Metrics:

- `client_calls_total{dep,result}`
- `client_retries_total{dep}`
- `client_retry_exhaust_total{dep}`
- `breaker_state{dep}` (0/1/2)
- `breaker_open_total{dep}`
- latency histograms **including** and **excluding** retries if possible

Logs: attempt number, delay, final classification, correlation id.

## Production Checklist

- ☐ Classify errors; do not retry fatal/caller errors
- ☐ Cap attempts and honor parent context
- ☐ Jittered backoff with max delay
- ☐ Idempotency story for mutating retries
- ☐ Breaker on high-traffic dependencies
- ☐ Half-open probe limits
- ☐ Metrics for retries and breaker state
- ☐ Coordinated policy with mesh/proxy
- ☐ Load tests with injected 50% failure

## Common Pitfalls

1. **Retry on everything** including 400 and cancel.
2. **No jitter** — synchronized herds.
3. **Infinite retries** in background workers without poison handling.
4. **Breaker thresholds too sensitive** — flapping.
5. **Breaker without half-open** — never recovers automatically.
6. **Hiding all errors as retryable network errors.**
7. **Retry after context deadline already passed.**

## Exercises

1. Implement **retry** with full jitter; unit-test attempt count and context cancel mid-backoff.
2. Simulate 50% failure; compare success rate with 1 vs 3 attempts; measure dependency QPS.
3. Add a breaker; drive failure rate above threshold; prove fail-fast latency is low.
4. Show flapping: threshold 1 failure — fix with windowed counts.
5. Implement idempotent POST with an in-memory key store; retry safely.
6. Trace a bug where proxy and client both retry twice (4 attempts); write a policy doc fixing it.
7. Export breaker state as a Prometheus gauge; alert when open > 2 minutes.

8. Compare exponential vs decorrelated jitter under synchronized clients (simple simulation).
9. Ensure `context.Canceled` never increments retry metrics as dependency failure.
10. Chaos: kill dependency for 30s; plot error rate with/without breaker.

## More examples

### Classify errors before retrying

```
mkdir -p /tmp/go-retry-class && cd /tmp/go-retry-class
go mod init example.com/retry-class
```

Save as `main.go`:

```
package main

import (
 "context"
 "errors"
 "fmt"
)

type class int

const (
 ok class = iota
 retryable
 fatal
 caller
)

func classify(err error, status int) class {
 if err == nil && status >= 200 && status < 400 {
 return ok
 }
 if errors.Is(err, context.Canceled) {
 return fatal
 }
 if status == 429 || status >= 500 {
 return retryable
 }
 if status >= 400 {
 return caller
 }
 if err != nil {
 return retryable
 }
}
```

```

 }
 return fatal
}

func main() {
 cases := []struct {
 err error
 status int
 }{
 {nil, 200},
 {nil, 503},
 {nil, 400},
 {context.Canceled, 0},
 {fmt.Errorf("net"), 0},
 }
 for _, c := range cases {
 fmt.Printf("status=%d err=%v -> %v\n", c.status, c.err, classify(c.err, c.status))
 }
}

go run .

```

### Expected output:

```

status=200 err=<nil> -> 0
status=503 err=<nil> -> 1
status=400 err=<nil> -> 3
status=0 err=context canceled -> 2
status=0 err=net -> 1

```

### Budget-aware retry (honor parent deadline)

```

mkdir -p /tmp/go-retry-budget && cd /tmp/go-retry-budget
go mod init example.com/retry-budget

```

Save as `main.go`:

```

package main

import (
 "context"
 "fmt"
 "time"
)

```

```

func retry(ctx context.Context, attempts int, fn func() error) (int, error) {
 var err error
 for i := 1; i <= attempts; i++ {
 if err = ctx.Err(); err != nil {
 return i - 1, err
 }
 err = fn()
 if err == nil {
 return i, nil
 }
 select {
 case <-ctx.Done():
 return i, ctx.Err()
 case <-time.After(15 * time.Millisecond):
 }
 }
 return attempts, err
}

func main() {
 ctx, cancel := context.WithTimeout(context.Background(), 25*time.Millisecond)
 defer cancel()
 n, err := retry(ctx, 10, func() error { return fmt.Errorf("down") })
 fmt.Println("stopped attempts:", n, "err:", err != nil)
}

go run .

```

Expected output:

```
stopped attempts: 2 err: true
```

(Exact attempt count may be 1–3 depending on scheduling; it must be **less than 10**.)

## Runnable example

Retries with full jitter plus a circuit breaker (closed → open → half-open) against a flaky in-process dependency.

```

mkdir -p /tmp/go-breaker && cd /tmp/go-breaker
go mod init example.com/breaker

```

Save as `main.go`:

```

package main

import (
 "context"
 "fmt"
 "math/rand"
 "sync"
 "time"
)

func retry(ctx context.Context, attempts int, fn func() error) error {
 var err error
 for i := 0; i < attempts; i++ {
 if err = ctx.Err(); err != nil {
 return err
 }
 err = fn()
 if err == nil {
 return nil
 }
 if i+1 == attempts {
 break
 }
 // full jitter backoff
 d := time.Duration(rand.Intn(int(20*time.Millisecond)<<i)) + time.Millisecond
 t := time.NewTimer(d)
 select {
 case <-ctx.Done():
 t.Stop()
 return ctx.Err()
 case <-t.C:
 }
 }
 return err
}

type CircuitBreaker struct {
 mu sync.Mutex
 failures int
 threshold int
 openUntil time.Time
 halfOpen bool
 coolDown time.Duration
}

func NewCB(threshold int, coolDown time.Duration) *CircuitBreaker {
 return &CircuitBreaker{threshold: threshold, coolDown: coolDown}
}

```

```

}

func (c *CircuitBreaker) Do(fn func() error) error {
 c.mu.Lock()
 now := time.Now()
 if now.Before(c.openUntil) {
 c.mu.Unlock()
 return fmt.Errorf("breaker open")
 }
 if !c.openUntil.IsZero() && now.After(c.openUntil) {
 c.halfOpen = true
 }
 c.mu.Unlock()

 err := fn()

 c.mu.Lock()
 defer c.mu.Unlock()
 if err != nil {
 c.failures++
 if c.halfOpen || c.failures >= c.threshold {
 c.openUntil = time.Now().Add(c.coolDown)
 c.halfOpen = false
 c.failures = 0
 }
 return err
 }
 c.failures = 0
 c.halfOpen = false
 c.openUntil = time.Time{}
 return nil
}

func main() {
 var n int
 flaky := func() error {
 n++
 if n < 3 {
 return fmt.Errorf("transient")
 }
 return nil
 }
 err := retry(context.Background(), 5, flaky)
 fmt.Println("retry ok:", err, "calls:", n)

 cb := NewCB(3, 50*time.Millisecond)
 fail := func() error { return fmt.Errorf("down") }

```

```

 for i := 0; i < 5; i++ {
 err := cb.Do(fail)
 fmt.Printf("cb %d: %v\n", i+1, err)
 }
 time.Sleep(60 * time.Millisecond)
 // half-open success resets
 err = cb.Do(func() error { return nil })
 fmt.Println("after cooldown success:", err)
 err = cb.Do(fail)
 fmt.Println("still closed after success, one fail:", err)
}

go run .

```

**Expected output** (illustrative):

```

retry ok: <nil> calls: 3
cb 1: down
cb 2: down
cb 3: down
cb 4: breaker open
cb 5: breaker open
after cooldown success: <nil>
still closed after success, one fail: down

```

### What to notice

- Breakers fail fast when open—latency collapses vs waiting on a dead dependency.
- Half-open allows a single probe after cooldown so the system can recover automatically.
- Retries without jitter synchronize clients and create herds.

### Try next

- Do not retry `context.Canceled` or HTTP 400.
- Export breaker state as a gauge (0/1) in the metrics chapter style.

## Further Reading

- Release It! (circuit breaker popularization)
- AWS architecture blog: exponential backoff and jitter
- Previous: discovery/health as inputs to call eligibility
- Next: **Queue Workers and Backpressure**

## Queue Workers and Backpressure



# Queue Workers and Backpressure

Queue systems decouple producer and consumer rates — **only if** capacity and failure behavior are explicit. Otherwise queues become infinite latency sinks or memory bombs.

## Why / Overview

Synchronous request paths force producers to wait. Queues enable:

- Spike absorption
- Asynchronous workflows
- Retries isolated from user latency
- Fan-out to multiple worker types

```
producer -> queue (buffer) -> workers -> dependency
 |
 +--> backpressure when full / depth high
```

In Go you will use:

- **In-process:** channels + worker pools
- **Durable:** SQS, Pub/Sub, NATS JetStream, Kafka, RabbitMQ, Redis streams

The control theory is the same; durability and delivery semantics differ.

## Delivery Semantics

Semantic	Meaning	App duty
At most once	May drop	Accept loss
At least once	May duplicate	Idempotent handlers
Exactly once	Rare/expensive	Often “effectively once” via idempotency

Assume **at-least-once** unless you have a strong proof otherwise. Design handlers to be safe under duplicate delivery.

```
// Idempotent handler sketch
func handle(ctx context.Context, job Job) error {
```

```

 ok, err := store.MarkIfNew(ctx, job.ID)
 if err != nil {
 return err
 }
 if !ok {
 return nil // duplicate
 }
 return doWork(ctx, job)
}

```

## In-Process Worker Pool

```

type Pool struct {
 jobs chan Job
 wg sync.WaitGroup
}

func NewPool(ctx context.Context, workers int, h func(context.Context, Job) error) *Pool {
 p := &Pool{jobs: make(chan Job, 1024)}
 for i := 0; i < workers; i++ {
 p.wg.Add(1)
 go func() {
 defer p.wg.Done()
 for {
 select {
 case <-ctx.Done():
 return
 case job, ok := <-p.jobs:
 if !ok {
 return
 }
 if err := h(ctx, job); err != nil {
 slog.Error("job failed", "id", job.ID, "err", err)
 }
 }
 }
 }()
 }
 return p
}

func (p *Pool) Submit(ctx context.Context, job Job) error {
 select {
 case p.jobs <- job:
 return nil
 }
}

```

```

 case <-ctx.Done():
 return ctx.Err()
 default:
 return ErrQueueFull // backpressure to caller
}

func (p *Pool) Close() {
 close(p.jobs)
 p.wg.Wait()
}

```

## Blocking vs dropping vs rejecting

Submit policy	Behavior
Block on full channel	Applies backpressure; can stall HTTP
Non-blocking + 503	Protects memory; client retries
Drop oldest/newest	Lossy; only for metrics/logs

Choose consciously. Silent drops for payment events are unacceptable.

## Sizing Levers

```
throughput workers × (1 / mean_job_latency)
```

But dependency capacity is the real ceiling. Oversubscribing workers increases contention and error rates.

Lever	Too small	Too large
Queue depth	Reject under mild spikes	Huge latency / RAM
Workers	Low throughput	Dep overload, lock contention
Visibility timeout (external)	Duplicate processing	Stuck messages if worker dies
Max receive count	Early DLQ	Infinite poison retry

## External Queue Worker Loop (Shape)

```
func worker(ctx context.Context, q Queue, h func(context.Context, Msg) error) error {
 for {
 msgs, err := q.Receive(ctx, 10, 20*time.Second) // batch, wait
 if err != nil {
 if ctx.Err() != nil {
 return ctx.Err()
 }
 slog.Error("receive", "err", err)
 time.Sleep(time.Second)
 continue
 }
 for _, m := range msgs {
 msgCtx, cancel := context.WithTimeout(ctx, 30*time.Second)
 err := h(msgCtx, m)
 cancel()
 if err != nil {
 // leave for retry / nack depending on system
 slog.Error("handle", "id", m.ID, "err", err)
 continue
 }
 if err := q.Ack(ctx, m); err != nil {
 slog.Error("ack", "err", err)
 }
 }
 }
}
```

### Ack timing

- **Ack after success** — at-least-once if crash before ack
- **Ack before work** — at-most-once loss on crash
- **Extend visibility** for long jobs

## Poison Messages and DLQ

Not all failures are transient.

```
transient: timeout, 503, deadlock -> retry with backoff
poison: invalid payload, permanent 400 -> DLQ after N attempts
```

```
func handle(ctx context.Context, m Msg) error {
 var job Job
 if err := json.Unmarshal(m.Body, &job); err != nil {
 return errPoison // route to DLQ, do not infinite retry
 }
 return process(ctx, job)
}
```

Alert on DLQ depth; empty it with a runbook, not silence.

## Backpressure Across Tiers

```
HTTP handler -> Submit(job)
 if ErrQueueFull -> 503 Retry-After
consumer lag high -> scale workers / shed non-critical producers
dependency circuit open -> slow receive rate or park workers
```

True end-to-end backpressure requires producers to **observe** consumer health (queue depth metrics, not only local channel fullness).

## Graceful Shutdown

```
cancel() // stop receiving new jobs
close(jobs) // or stop Submit
wait with timeout // finish in-flight
// for external queues: stop Receive; in-flight still acked or returned by visibility expiry
```

Never exit immediately while holding unacked messages if you can avoid it — you will cause stampedes of redelivery.

## Observability

Metrics:

- queue\_depth / channel length if available
- jobs\_enqueued\_total{result=ok|full}
- jobs\_processed\_total{result}
- job\_duration\_seconds
- jobs\_in\_flight
- dlq\_depth
- receive\_errors\_total

Lag: time between enqueue timestamp and process start (requires timestamp in payload).

## Production Checklist

- ☐ Document delivery semantics and idempotency
- ☐ Bounded queue / explicit full behavior
- ☐ Worker count tied to dependency capacity
- ☐ Timeouts on job handlers
- ☐ Poison path + DLQ + alerts
- ☐ Ack/visibility policy documented
- ☐ Graceful drain on shutdown
- ☐ Metrics for depth, lag, failures, DLQ
- ☐ Load test: spike producers, kill workers, duplicate delivery

## Common Pitfalls

1. **Unbounded `make(chan T)`** or growing slices as queues — OOM.
2. **More workers = faster** without measuring the dependency.
3. **Non-idempotent handlers** with at-least-once queues.
4. **Infinite retries** of bad payloads.
5. **Blocking HTTP on full in-process queue** without timeout — cascading latency.
6. **Ack before durable side effect completes.**
7. **No lag metric** — discover the backlog from customer tickets.

## Exercises

1. Implement a bounded worker pool; prove `Submit` returns `ErrQueueFull` under overload.
2. Change policy to block with context timeout; compare HTTP p99 under spike.
3. Inject duplicate deliveries; show double side effects; add idempotency store.
4. Generate invalid JSON jobs; after 3 failures move to DLQ slice; alert when `DLQ > 0`.
5. Measure throughput vs worker count for a 50ms simulated dependency; find the plateau.
6. Shutdown mid-job; confirm message redelivery with external queue or simulated visibility.
7. Add enqueue timestamp; export lag histogram.
8. Combine with a circuit breaker: when open, stop pulling messages for that dependency.
9. Use `errgroup` with limit for per-batch parallel handle; ensure `ctx cancel` stops the batch.
10. Write a capacity plan: peak produce rate, target lag, mean job time → workers and depth.

## More examples

### Bounded queue + worker pool

```
mkdir -p /tmp/go-queue-bp && cd /tmp/go-queue-bp
go mod init example.com/queue-bp
```

Save as `main.go`:

```

package main

import (
 "fmt"
 "sync"
 "time"
)

func main() {
 jobs := make(chan int, 4) // backpressure depth
 var wg sync.WaitGroup
 var done int
 var mu sync.Mutex

 for w := 0; w < 2; w++ {
 wg.Add(1)
 go func() {
 defer wg.Done()
 for j := range jobs {
 time.Sleep(5 * time.Millisecond)
 mu.Lock()
 done++
 _ = j
 mu.Unlock()
 }
 }()
 }

 for i := 1; i <= 10; i++ {
 jobs <- i // blocks when buffer full → producer slows
 }
 close(jobs)
 wg.Wait()
 fmt.Println("processed:", done)
}

go run .

```

**Expected output:**

processed: 10

## Drop vs block when full (load shed)

```
mkdir -p /tmp/go-queue-shed && cd /tmp/go-queue-shed
go mod init example.com/queue-shed
```

Save as `main.go`:

```
package main

import "fmt"

func tryEnqueue(ch chan int, v int) bool {
 select {
 case ch <- v:
 return true
 default:
 return false
 }
}

func main() {
 ch := make(chan int, 2)
 accepted, dropped := 0, 0
 for i := 0; i < 5; i++ {
 if tryEnqueue(ch, i) {
 accepted++
 } else {
 dropped++
 }
 }
 fmt.Println("accepted:", accepted, "dropped:", dropped, "len:", len(ch))
}

go run .
```

Expected output:

```
accepted: 2 dropped: 3 len: 2
```

## Runnable example

Bounded in-memory queue, worker pool, `ErrQueueFull` backpressure, and a tiny dead-letter list for poison messages.



```
mkdir -p /tmp/go-queue && cd /tmp/go-queue
go mod init example.com/queue
```

Save as main.go:

```
package main

import (
 "context"
 "errors"
 "fmt"
 "sync"
 "time"
)

var ErrQueueFull = errors.New("queue full")

type Job struct {
 ID string
 Payload string
 Attempt int
}

type Pool struct {
 jobs chan Job
 dlq []Job
 mu sync.Mutex
 wg sync.WaitGroup
 handle func(context.Context, Job) error
}

func NewPool(depth, workers int, handle func(context.Context, Job) error) *Pool {
 p := &Pool{jobs: make(chan Job, depth), handle: handle}
 for i := 0; i < workers; i++ {
 p.wg.Add(1)
 go func() {
 defer p.wg.Done()
 for j := range p.jobs {
 ctx, cancel := context.WithTimeout(context.Background(), 100*time.Millisecond)
 err := p.handle(ctx, j)
 cancel()
 if err != nil {
 j.Attempt++
 if j.Attempt >= 3 {
 p.mu.Lock()
 p.dlq = append(p.dlq, j)
 p.mu.Unlock()
 }
 }
 }
 }()
 }
}
```

```

 continue
 }
 // re-queue best-effort
 select {
 case p.jobs <- j:
 default:
 p.mu.Lock()
 p.dlq = append(p.dlq, j)
 p.mu.Unlock()
 }
}
}
}()
}
return p
}

func (p *Pool) Submit(j Job) error {
 select {
 case p.jobs <- j:
 return nil
 default:
 return ErrQueueFull
 }
}

func (p *Pool) Close() {
 close(p.jobs)
 p.wg.Wait()
}

func main() {
 var okCount int
 var mu sync.Mutex
 p := NewPool(2, 2, func(ctx context.Context, j Job) error {
 if j.Payload == "poison" {
 return fmt.Errorf("bad payload")
 }
 mu.Lock()
 okCount++
 mu.Unlock()
 return nil
 })

 fmt.Println("submit a:", p.Submit(Job{ID: "1", Payload: "ok"}))
 fmt.Println("submit b:", p.Submit(Job{ID: "2", Payload: "ok"}))
 fmt.Println("submit c:", p.Submit(Job{ID: "3", Payload: "ok"})) // may fill

```

```

 // drain a bit
 time.Sleep(50 * time.Millisecond)
 _ = p.Submit(Job{ID: "4", Payload: "poison"})
 time.Sleep(50 * time.Millisecond)
 for i := 0; i < 5; i++ {
 if err := p.Submit(Job{ID: fmt.Sprintf("x%d", i), Payload: "ok"}); err != nil {
 fmt.Println("backpressure:", err)
 break
 }
 }
 p.Close()
 p.mu.Lock()
 fmt.Println("ok_count:", okCount, "dlq:", len(p.dlq))
 p.mu.Unlock()
}

go run .

```

**Expected output** (timing-sensitive; illustrative):

```

submit a: <nil>
submit b: <nil>
submit c: queue full
backpressure: queue full
ok_count: ... dlq: >=1

```

### What to notice

- Bounded channels are the simplest backpressure tool; unbounded queues OOM under spike.
- Poison messages need attempt caps and a DLQ—infinite retry burns workers forever.
- Closing the job channel and `Wait` is orderly worker shutdown.

### Try next

- Block with `select` + context timeout instead of immediate `ErrQueueFull`.
- Export queue depth gauge and lag from enqueue timestamp.

## Further Reading

- Queueing theory basics (Little's law:  $L = W$ ) for lag intuition
- Previous: retries/breakers on the dependency side of workers
- Next part: **Observability and SRE** — measuring all of the above

## Caching Patterns

# Caching Patterns

## Overview

Caches cut latency and load—and create consistency bugs. Start with **explicit TTLs**, single-flight fills, and clear invalidation.

## In-process TTL map

```
type entry struct {
 val any
 exp time.Time
}

type Cache struct {
 mu sync.Mutex
 m map[string]entry
}

func (c *Cache) Get(k string) (any, bool) {
 c.mu.Lock()
 defer c.mu.Unlock()
 e, ok := c.m[k]
 if !ok || time.Now().After(e.exp) {
 delete(c.m, k)
 return nil, false
 }
 return e.val, true
}

func (c *Cache) Set(k string, v any, ttl time.Duration) {
 c.mu.Lock()
 c.m[k] = entry{val: v, exp: time.Now().Add(ttl)}
 c.mu.Unlock()
}
```

## Single-flight (stampede control)

```
go get golang.org/x/sync/singleflight@latest

var g singleflight.Group

v, err, _ := g.Do(key, func() (any, error) {
 return loadFromDB(ctx, key)
})
```

## Cache-aside

```
get cache → miss → load DB → set cache → return
```

Write path: update DB then **delete** cache key (or short TTL).

## Negative caching

Cache “not found” briefly to protect DB from repeated misses—short TTL.

## Rules of thumb

Do	Don't
TTL + max entries	Unbounded map growth
Invalidate on write	Assume cache == truth forever
singleflight on hot keys	Thundering herd on expiry

## Try next

1. Wrap Book repository with TTL cache.
2. Benchmark stampede with/without singleflight.
3. Add max keys with simple LRU eviction sketch.

# **Idempotency and Deduplication**

# Idempotency and Deduplication

## Overview

Networks **retry**. Clients double-submit. Without idempotency, you charge twice or create duplicate rows. Key idea: same logical request → same effect.

## Idempotency-Key header

```
POST /payments
Idempotency-Key: 9f3c...
```

Server stores key → response (or in-flight marker) with TTL.

```
type store interface {
 Get(ctx context.Context, key string) (status int, body []byte, ok bool, err error)
 Put(ctx context.Context, key string, status int, body []byte, ttl time.Duration) error
}

func withIdempotency(s store, next http.Handler) http.Handler {
 return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
 if r.Method != http.MethodPost {
 next.ServeHTTP(w, r)
 return
 }
 key := r.Header.Get("Idempotency-Key")
 if key == "" {
 http.Error(w, "missing Idempotency-Key", http.StatusBadRequest)
 return
 }
 if st, body, ok, err := s.Get(r.Context(), key); err == nil && ok {
 w.WriteHeader(st)
 _, _ = w.Write(body)
 return
 }
 // capture response → Put (use response recorder middleware)
 next.ServeHTTP(w, r)
 })
}
```



## DB unique constraints

Natural idempotency: `UNIQUE(idempotency_key)` or `UNIQUE(user_id, client_request_id)`.

```
INSERT ... ON CONFLICT (idempotency_key) DO NOTHING
```

## At-least-once consumers

Queue workers: store processed `message_id`; skip duplicates.

## Rules of thumb

Do	Don't
Require keys on money POSTs	Retry POSTs blindly without keys
TTL the key store	Infinite growth of keys
Prefer DB uniqueness	Only in-memory dedupe multi-instance

## Try next

1. Memory idempotency store + double POST test.
2. Unique index migration for order client IDs.
3. Worker skips duplicate delivery.

## **Leader Election Basics**

# Leader Election Basics

## Overview

Sometimes **exactly one** instance should run a loop (compactions, cron, singleton consumer). Leader election picks that instance. Keep it simple: lease in a database/etcd/K8s lock—not custom Paxos in app code.

## Lease pattern (DB)

```
UPDATE leases SET owner=?, expires=?
WHERE name=? AND (expires < now() OR owner=?)
```

Winner renews before expiry; losers idle and retry.

```
type LeaseStore interface {
 TryAcquire(ctx context.Context, name, owner string, ttl time.Duration) (bool, error)
 Renew(ctx context.Context, name, owner string, ttl time.Duration) (bool, error)
 Release(ctx context.Context, name, owner string) error
}

func RunAsLeader(ctx context.Context, s LeaseStore, name, owner string, ttl time.Duration, w
 t := time.NewTicker(ttl / 2)
 defer t.Stop()
 for {
 ok, err := s.TryAcquire(ctx, name, owner, ttl)
 if err != nil {
 return err
 }
 if !ok {
 select {
 case <-ctx.Done():
 return ctx.Err()
 case <-time.After(ttl / 2):
 continue
 }
 }
 // lead
 wctx, cancel := context.WithCancel(ctx)
```

```

errCh := make(chan error, 1)
go func() { errCh <- work(wctx) }()
for {
 select {
 case err := <-errCh:
 cancel()
 _ = s.Release(ctx, name, owner)
 return err
 case <-t.C:
 ok, err := s.Renew(ctx, name, owner, ttl)
 if err != nil || !ok {
 cancel()
 <-errCh
 break
 }
 case <-ctx.Done():
 cancel()
 _ = s.Release(context.Background(), name, owner)
 return ctx.Err()
 }
}
}
}
}

```

## Kubernetes

Lease objects / leader-election libraries (client-go tools) — prefer platform primitives in K8s.

## Rules

Do	Don't
TTL + renew	Infinite leadership without heartbeat
Work cancel on loss	Continue writes after lease loss
Idempotent leader work	Assume exactly-once without more design

## Try next

1. Memory lease store + two goroutines racing.
2. Kill leader; follower takes over within TTL.
3. Ensure work stops on lease loss.

## Transactional Outbox Pattern

# Transactional Outbox Pattern

## Overview

If you UPDATE the DB then Publish to a queue, crashes create lost or double events. The **outbox** writes the event in the **same transaction** as business data; a relay publishes asynchronously.

```
begin
 update orders
 insert outbox(event)
commit
 → relay reads outbox → publish → mark sent
```

## Schema sketch

```
CREATE TABLE outbox (
 id BIGSERIAL PRIMARY KEY,
 topic TEXT NOT NULL,
 payload JSONB NOT NULL,
 created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
 published_at TIMESTAMPTZ
);
```

## Write path

```
tx, _ := db.BeginTx(ctx, nil)
_, err = tx.ExecContext(ctx, `UPDATE orders SET status=$1 WHERE id=$2`, "paid", id)
_, err = tx.ExecContext(ctx, `INSERT INTO outbox(topic,payload) VALUES($1,$2)`, "order.paid"
err = tx.Commit()
```

## Relay worker

```
rows, _ := db.QueryContext(ctx, `
 SELECT id, topic, payload FROM outbox
 WHERE published_at IS NULL
 ORDER BY id LIMIT 100
 FOR UPDATE SKIP LOCKED`)
// for each: publish; UPDATE outbox SET published_at=now() WHERE id=?
```

SKIP LOCKED allows multiple relays safely.

## At-least-once

Consumers still need **idempotency** (chapter 155)—outbox gives reliable *publish attempt*, not exactly-once end-to-end.

## Rules

Do	Don't
Same TX as state change	Publish then commit (or reverse) without outbox
Idempotent consumers	Assume no duplicates
Metric lag of unpublished rows	Silent outbox growth

## Try next

1. Simulate crash after commit before publish; relay catches up.
2. Two relays with SKIP LOCKED.
3. Dashboard: count unpublished > 5m.

## **Project: Resilient Worker**



# Project: Resilient Worker

## Overview

Build a **worker** that pulls jobs from an in-memory (then Redis/SQS) queue with: retries, backoff, poison handling, concurrency cap, and graceful shutdown.

## Features

Feature	Spec
Concurrency	<code>-w 8</code> workers
Retry	max 5, exponential backoff
Poison	after max → dead-letter slice/log
Shutdown	finish current job or requeue
Metrics	processed, failed, dlq counters

## Interface

```
type Job struct {
 ID string
 Payload []byte
 Attempt int
}

type Queue interface {
 Receive(ctx context.Context) (Job, error)
 Ack(ctx context.Context, id string) error
 Retry(ctx context.Context, j Job, after time.Duration) error
 DeadLetter(ctx context.Context, j Job, reason string) error
}
```

## Loop

```
for i := 0; i < workers; i++ {
 go func() {
 for {
 j, err := q.Receive(ctx)
 if err != nil { return }
 if err := handle(ctx, j); err != nil {
 if j.Attempt+1 >= max {
 _ = q.DeadLetter(ctx, j, err.Error())
 } else {
 j.Attempt++
 _ = q.Retry(ctx, j, backoff(j.Attempt))
 }
 continue
 }
 _ = q.Ack(ctx, j.ID)
 }
 }()
}
```

## Acceptance

- ☐ Inject fail-N-times job; succeeds after retries
- ☐ Poison goes to DLQ
- ☐ SIGTERM stops receiving; no new jobs
- ☐ Race test clean

Connects chapters 152–153 + 155.

# **Observability and SRE**

## **Observability and SRE Overview**

# Observability and SRE Overview

This part explains how to make Go services **operable in real incidents**. Observability is a design property: if you cannot ask new questions of a running system, you do not have it — you have a fixed dashboard.

## Why / Overview

Production failures are rarely “the server is off.” They are:

- Elevated tail latency on one route
- Error budgets burning from one dependency
- A deploy that only breaks for a subset of tenants
- Retries hiding the true failure rate

You need three complementary signals:

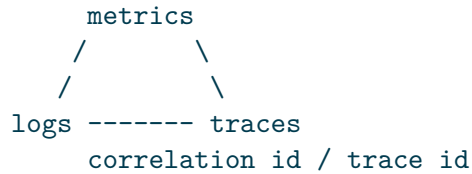
```
logs - discrete events with context (what happened)
metrics - aggregates over time (is it bad, how bad)
traces - causal latency across services (where is the time)
```

SRE practice ties those signals to **SLIs/SLOs**, alerts, and runbooks — not to collecting data for its own sake.

## Learning Path

Chapter	Focus	Outcome
161 Structured logging	slog, fields, correlation, redaction	Debuggable event streams
162 Metrics / Prometheus	RED/USE, labels, histograms	Alertable, cheap aggregates
163 Tracing	Spans, context propagation	Cross-service latency attribution
164 SLI/SLO/error budgets	Targets and burn	Reliability vs feature tradeoffs
165 Health probes	livez/readyz/drain	Correct orchestration signals
166 Alerting & runbooks	Actionable pages	Faster incident response
167 Project: instrumented service	Wire logs+metrics+probes	End-to-end lab

## The Observability Triangle



A single **correlation key** (request id / trace id) should appear in all three where possible.

## SLI / SLO Primer

Term	Meaning
SLI	Quantitative measure (e.g. % of requests < 300ms)
SLO	Target on that measure (e.g. 99.9% over 30d)
Error budget	1 – SLO; spend on velocity or freeze

Instrument what you SLO. Vanity metrics that nobody pages on are optional.

## Go Building Blocks

Signal	Libraries / stdlib
Logs	log/slog (Go 1.21+)
Metrics	Prometheus client, OpenTelemetry metrics
Traces	OpenTelemetry Go SDK
Propagation	context.Context + W3C headers

```
slog.Info("checkout",
 "request_id", rid,
 "user_hash", hash(userID), // not raw PII if avoidable
 "total_ms", dur.Milliseconds(),
)
```

## Production Principles

1. **Cardinality discipline** — unbounded label values kill metric systems.
2. **Structured fields > string paste** — enable search and aggregation.
3. **Propagate context** — lost trace context is a permanent blind spot.
4. **Sample thoughtfully** — full traces at 100% may be too expensive; errors often keep 100%.

5. **Alert on symptoms** (user pain) more than causes (CPU high).
6. **Redact secrets** — logs and spans are exfiltration channels.

## Relationship to Other Parts

- Network and distributed parts **emit** the signals designed here.
- Performance engineering **uses** profiles (adjacent to metrics/traces).
- Security hardening **constrains** what you may log.

## Literacy Checklist

- ☐ Emit JSON or structured logs with stable field names
- ☐ Carry request/trace ids across goroutines and HTTP
- ☐ Define RED metrics for a service
- ☐ Choose histogram buckets intentionally
- ☐ Avoid high-cardinality labels
- ☐ Create and propagate spans on outbound calls
- ☐ Tie an alert to an SLO and a runbook

## Part Warm-Up Exercises

1. List the top five questions you ask during incidents; map each to log/metric/trace.
2. Find one high-cardinality metric risk in an existing codebase (**user\_id** label?).
3. Grep for `log.Printf` and plan migration of hot paths to `slog`.
4. Check whether outbound HTTP clients forward trace headers.
5. Write a draft SLO for your highest-traffic endpoint.

## Instrumentation Order of Operations

When adding observability to a greenfield service, do this in order:

1. **Structured access logs** with request id (chapter 161)
2. **RED metrics** for public routes (chapter 162)
3. **Dependency client metrics** (timeouts vs errors)
4. **Trace propagation** on the hottest path (chapter 163)
5. **SLOs + alerts + runbooks** for those SLIs

Skipping to distributed tracing without metrics leaves you with pretty waterfalls and no reliable pages. Skipping logs leaves metrics without narrative.

## What “Good” Looks Like in an Incident

Within five minutes you should be able to:

1. See error rate and p99 on the affected route (metrics)
2. Grab a `request_id` / `trace_id` from a failing request (logs or edge)
3. Jump to a trace that shows the slow/failed span (traces)
4. Read surrounding log lines for that id (logs)
5. Open the runbook linked from the alert

If any hop is missing, fix instrumentation — not only the proximate bug.

## Cardinality and Cost Budget

Observability has a cloud bill. Rough discipline:

Signal	Cost driver	Control
Logs	volume × retention	sample happy path; drop debug in prod
Metrics	series count	bounded labels; careful histograms
Traces	spans × rate	head/tail sampling

Review series count and log GB/day the same way you review CPU.

## Anti-Patterns

1. Unstructured logs that require regex archaeology
2. Metrics labeled with user ids or raw URLs
3. Traces without parent context on outbound HTTP
4. Alerts with no runbook and no SLO
5. Logging secrets “just for staging” (staging leaks too)

## Minimum Dashboard Set

For each user-facing service, keep a default dashboard with:

1. Request rate (global + top routes)
2. Error rate / fraction (5xx and timeout class)
3. Latency p50 / p95 / p99
4. In-flight requests / saturation
5. Dependency latency and error panels
6. Go runtime: goroutines, heap, GC CPU (secondary)

If a panel cannot drive an action, demote it. Dashboards are for decisions, not decoration.



## Alert Quality Bar

Before shipping an alert, answer:

- What user pain does this detect?
- What is the false-positive risk?
- Who is paged and what do they run first?
- Can we silence safely during known maintenance?

Symptom-based alerts (SLO burn) beat cause-based alerts (CPU > 80%) for most services. Cause alerts are still useful for capacity planning, usually at ticket severity not page severity.

## Library Choices (2024–2026 pragmatic)

Need	Common choice
Logs	log/slog (+ JSON handler)
Metrics	Prometheus client_golang
Traces	OpenTelemetry Go
Bridges	OTel → your backend vendor

Avoid inventing a fourth proprietary telemetry API inside your org without a migration plan.

## Glossary

- **SLI / SLO / error budget** — measure, target, remaining failure allowance
- **Cardinality** — number of unique time series
- **Correlation id** — joins signals for one request
- **Exemplar** — metric sample pointing at a trace
- **Propagation** — carrying context across process boundaries
- **RED / USE** — request and resource metric recipes
- **Tail sampling** — keep interesting traces after the fact

## More examples

**Tour: slog + counter + request id**

```
mkdir -p /tmp/go-o11y-tour && cd /tmp/go-o11y-tour
go mod init example.com/o11y-tour
```

Save as `main.go`:

```

package main

import (
 "bytes"
 "fmt"
 "log/slog"
 "net/http"
 "net/http/httptest"
 "strings"
 "sync/atomic"
)

func main() {
 var hits atomic.Int64
 var buf bytes.Buffer
 log := slog.New(slog.NewJSONHandler(&buf, nil))

 h := http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
 id := r.Header.Get("X-Request-ID")
 if id == "" {
 id = "gen-1"
 }
 hits.Add(1)
 log.Info("hit", "request_id", id, "path", r.URL.Path)
 w.Header().Set("X-Request-ID", id)
 fmt.Fprintln(w, "ok")
 })
 ts := httptest.NewServer(h)
 defer ts.Close()

 req, _ := http.NewRequest(http.MethodGet, ts.URL+"/x", nil)
 req.Header.Set("X-Request-ID", "abc")
 resp, _ := http.DefaultClient.Do(req)
 resp.Body.Close()
 fmt.Println("hits:", hits.Load(), "resp id:", resp.Header().Get("X-Request-ID"))
 fmt.Println("logged:", strings.Contains(buf.String(), "abc"))
}

go run .

```

### Expected output:

```

hits: 1 resp id: abc
logged: true

```

## Runnable example

Tour of the observability triangle: one JSON log line, one counter sample, one trace-id field on a context—stdlib only.

```
mkdir -p /tmp/go-o11y-tour && cd /tmp/go-o11y-tour
go mod init example.com/o11y-tour
```

Save as `main.go`:

```
package main

import (
 "bytes"
 "context"
 "fmt"
 "log/slog"
 "time"
)

type ctxKey string

const traceIDKey ctxKey = "trace_id"

func main() {
 var buf bytes.Buffer
 log := slog.New(slog.NewJSONHandler(&buf, nil))

 ctx := context.WithValue(context.Background(), traceIDKey, "tr-demo-1")
 start := time.Now()
 // fake work
 time.Sleep(5 * time.Millisecond)
 log.InfoContext(ctx, "request",
 "trace_id", ctx.Value(traceIDKey),
 "route", "GET /checkout",
 "status", 200,
 "dur_ms", time.Since(start).Milliseconds(),
)

 // metric sample (manual exposition line)
 var requestsTotal = 1
 fmt.Printf("# TYPE demo_requests_total counter\ndemo_requests_total %d\n", requestsTotal)
 fmt.Println("--- log ---")
 fmt.Print(buf.String())
 fmt.Println("tour: logs + metrics + trace id field")
}
```

```
go run .
```

**Expected output** (illustrative):

```
TYPE demo_requests_total counter
demo_requests_total 1
--- log ---
{"time":"...", "level":"INFO", "msg":"request", "trace_id":"tr-demo-1", ...}
tour: logs + metrics + trace id field
```

**What to notice**

- The three signals answer different questions; this part teaches each without vendor lock-in first.
- Stable field names make cross-service grepping possible during incidents.

**Try next**

- Chapters 161–163: slog correlation, Prometheus-shaped metrics, W3C-ish traceparent propagation.

## Next Chapter

**Structured Logging and Correlation** — the foundation every engineer greps first.

# Structured Logging and Correlation

# Structured Logging and Correlation

Structured logs are **event records**, not formatted print statements. In Go 1.21+, `log/slog` is the standard library foundation for production logging.

## Why / Overview

`fmt.Println` and free-form `log.Printf` fail at scale:

- Cannot reliably filter by field
- Cannot join with traces
- Easy to leak secrets
- Inconsistent shapes across services

```
request_id / trace_id
 -> log fields on every line for that request
 -> (optional) metric exemplars
 -> trace span attributes
```

## slog Fundamentals

```
package main

import (
 "log/slog"
 "os"
)

func main() {
 h := slog.NewJSONHandler(os.Stdout, &slog.HandlerOptions{
 Level: slog.LevelInfo,
 })
 log := slog.New(h)
 slog.SetDefault(log)

 slog.Info("server start", "addr", ":8080", "version", version)
}
```

Levels: Debug, Info, Warn, Error. Prefer Error for handled failures that need attention; do not log+return the same error at every layer (pick a boundary).

## Loggers with attributes

```
reqLog := slog.With(
 "service", "checkout",
 "env", os.Getenv("ENV"),
)
reqLog.Info("payment authorized", "amount_cents", 4200)
```

## Correlation IDs

Generate or accept a request id at the edge; store in context; inject into logger.

```
type ctxKey struct{}

func WithRequestID(ctx context.Context, id string) context.Context {
 return ctx.WithValue(ctxKey{}, id)
}

func RequestID(ctx context.Context) string {
 if v, ok := ctx.Value(ctxKey{}).(string); ok {
 return v
 }
 return ""
}

func requestIDMiddleware(next http.Handler) http.Handler {
 return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
 id := r.Header.Get("X-Request-Id")
 if id == "" {
 id = newID() // crypto/rand hex
 }
 ctx := WithRequestID(r.Context(), id)
 w.Header().Set("X-Request-Id", id)
 // Logger bound to request
 log := slog.Default().With("request_id", id)
 ctx = withLogger(ctx, log)
 next.ServeHTTP(w, r.WithContext(ctx))
 })
}
```

Propagate X-Request-Id / traceparent on outbound calls.

## Context-aware logging helpers

```
func Info(ctx context.Context, msg string, args ...any) {
 Logger(ctx).Info(msg, args...)
}
```

Avoid putting the entire `*slog.Logger` in context if your style guide forbids context values — alternatives: explicit logger parameters. Many teams accept request-scoped logger in context for HTTP services.

## Stable Schema

Cross-service conventions beat clever one-offs:

Field	Meaning
<code>service</code>	Service name
<code>env</code>	prod/stage
<code>version</code>	build/version
<code>request_id</code>	Edge correlation
<code>trace_id</code>	Distributed trace
<code>span_id</code>	Current span
<code>method / path / route</code>	HTTP shape ( <code>route</code> = template)
<code>status</code>	HTTP status or RPC code
<code>err</code>	Error string / type
<code>dur_ms</code>	Duration

Use **route templates** (`/users/{id}`) not raw URLs in aggregatable fields.

## What to Log

Do log	Avoid
Request start/end (or access log)	Full bodies by default
Business state transitions	Secrets, tokens, passwords
Dependency failures with class	Unbounded free text dumps
Config version at start	PII without legal basis

Access log example:

```
slog.Info("http_request",
 "method", r.Method,
 "route", routePattern,
 "status", status,
```



```

 "dur_ms", time.Since(start).Milliseconds(),
 "bytes", n,
)

```

## Redaction and Security

Logs are a data store. Assume breach.

```

func redactAuth(h http.Header) http.Header {
 c := h.Clone()
 if c.Get("Authorization") != "" {
 c.Set("Authorization", "REDACTED")
 }
 if c.Get("Cookie") != "" {
 c.Set("Cookie", "REDACTED")
 }
 return c
}

```

For structured fields, prefer hashing user ids when only correlation is needed: `user_sha256=...`

## Sampling and Volume

High-QPS services may sample successful debug/info events while always keeping errors:

```

// Conceptual: custom Handler that drops Info with probability p
// unless level >= Warn

```

Or sample at the shipper (Fluent Bit, Vector) — still emit errors at 100%.

## Goroutines and Lost Context

```

// BAD: detaches without ids
go func() {
 doWork(context.Background())
}()

// GOOD: carry request context (or derived with timeout)
go func(ctx context.Context) {
 defer func() { /* recover log with request_id */ }()
 doWork(ctx)
}(ctx)

```

If work outlives the request, extract ids into the child logger before detaching, and use a new root context with timeout.

## slog Handlers and Production

- **JSON** to stdout for containers (collectors parse)
- **Text** for local dev
- Fan-out handlers exist in community packages if you must dual-write

```
// Level from env
level := slog.LevelInfo
if os.Getenv("LOG_LEVEL") == "debug" {
 level = slog.LevelDebug
}
```

## Testing Logs

```
var buf bytes.Buffer
log := slog.New(slog.NewJSONHandler(&buf, nil))
// inject log into code under test
// assert strings.Contains(buf.String(), `{"msg":"paid"}`)
```

Or use a test handler that records []slog.Record.

## Production Checklist

- ☐ JSON structured logs in prod
- ☐ Stable field vocabulary documented
- ☐ request\_id / trace\_id on every request log line
- ☐ Middleware sets and returns request id
- ☐ Outbound calls propagate correlation headers
- ☐ Secrets redacted; body logging opt-in only
- ☐ Error logged once at boundary with stack or error chain
- ☐ Levels configurable without recompile
- ☐ Volume/sampling plan for hot paths

## Common Pitfalls

1. **String interpolation only** — "user "+id loses structure.
2. **Different field names** per service (req\_id vs requestId).
3. **Logging entire http.Request** — headers leak cookies.

4. **log.Fatal in libraries** — kills process.
5. **Missing id after async hop.**
6. **Debug left at Info** — cost and noise.
7. **Duplicate error logs** every layer → alert fatigue.

## Exercises

1. Replace a `log.Printf` access log with `slog` JSON fields; query with `jq`.
2. Implement request id middleware; prove id appears in handler logs and response header.
3. Propagate id on an outbound `http.Client` call; log on both sides; join lines by id.
4. Add redaction for Authorization; unit-test.
5. Spawn a goroutine without context; show missing id; fix.
6. Define a 10-field schema doc for your team; refactor one service to match.
7. Emit `dur_ms` histogram-equivalent in logs only; discuss when metrics are better.
8. Implement level-from-env; toggle debug in a running container via restart with env.
9. Benchmark logging 10k info lines JSON vs text; note CPU.
10. Create an incident drill: given only logs, reconstruct a failed checkout path.

## More examples

### slog groups and default attrs

```
mkdir -p /tmp/go-slog-group && cd /tmp/go-slog-group
go mod init example.com/slog-group
```

Save as `main.go`:

```
package main

import (
 "bytes"
 "fmt"
 "log/slog"
 "strings"
)

func main() {
 var buf bytes.Buffer
 base := slog.New(slog.NewJSONHandler(&buf, nil))
 log := base.With("service", "checkout", "env", "dev")
 log.Info("paid",
 slog.Group("user", "id", 42, "plan", "pro"),
 "amount_cents", 999,
)
}
```

```

 line := strings.TrimSpace(buf.String())
 fmt.Println(line)
 fmt.Println("has service:", strings.Contains(line, `"service":"checkout"`))
 fmt.Println("has user group:", strings.Contains(line, `"user"`))
}

```

```
go run .
```

**Expected output** (time field varies):

```

{"time":"...", "level":"INFO", "msg":"paid", "service":"checkout", "env":"dev", "user":{"id":42, "pla
has service: true
has user group: true

```

## Child logger from context request id

```

mkdir -p /tmp/go-slog-child && cd /tmp/go-slog-child
go mod init example.com/slog-child

```

Save as main.go:

```

package main

import (
 "bytes"
 "context"
 "fmt"
 "log/slog"
 "strings"
)

type ctxKey string

const ridKey ctxKey = "rid"

func loggerWithRID(ctx context.Context, base *slog.Logger) *slog.Logger {
 if v, ok := ctx.Value(ridKey).(string); ok && v != "" {
 return base.With("request_id", v)
 }
 return base
}

func main() {
 var buf bytes.Buffer
 base := slog.New(slog.NewJSONHandler(&buf, nil))

```

```

 ctx := context.WithValue(context.Background(), ridKey, "req-9")
 log := loggerWithRID(ctx, base)
 log.Info("step", "name", "charge")
 fmt.Println(strings.Contains(buf.String(), "req-9"))
}

```

```
go run .
```

Expected output:

```
true
```

## Runnable example

JSON slog, request-id middleware with `httptest`, outbound header propagation, and Authorization redaction.

```

mkdir -p /tmp/go-slog-corr && cd /tmp/go-slog-corr
go mod init example.com/slog-corr

```

Save as `main.go`:

```

package main

import (
 "bytes"
 "context"
 "fmt"
 "log/slog"
 "net/http"
 "net/http/httptest"
 "strings"
)

type ctxKey string

const ridKey ctxKey = "request_id"

func withRID(ctx context.Context, id string) context.Context {
 return context.WithValue(ctx, ridKey, id)
}

func rid(ctx context.Context) string {
 if v, ok := ctx.Value(ridKey).(string); ok {
 return v
 }
}

```

```

 }
 return ""
}

func redactAuth(h http.Header) string {
 v := h.Get("Authorization")
 if v == "" {
 return ""
 }
 return "REDACTED"
}

func main() {
 var buf bytes.Buffer
 log := slog.New(slog.NewJSONHandler(&buf, &slog.HandlerOptions{Level: slog.LevelInfo}))

 mux := http.NewServeMux()
 mux.HandleFunc("GET /api", func(w http.ResponseWriter, r *http.Request) {
 log.Info("handler",
 "request_id", rid(r.Context()),
 "path", r.URL.Path,
 "authorization", redactAuth(r.Header),
)
 // simulate outbound call headers
 out := httptest.NewRequest(http.MethodGet, "http://dep/x", nil)
 out.Header.Set("X-Request-ID", rid(r.Context()))
 fmt.Println("outbound X-Request-ID:", out.Header.Get("X-Request-ID"))
 w.Header().Set("X-Request-ID", rid(r.Context()))
 fmt.Fprintln(w, "ok")
 })

 h := http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
 id := r.Header.Get("X-Request-ID")
 if id == "" {
 id = "generated-1"
 }
 ctx := withRID(r.Context(), id)
 mux.ServeHTTP(w, r.WithContext(ctx))
 })

 req := httptest.NewRequest(http.MethodGet, "/api", nil)
 req.Header.Set("X-Request-ID", "abc-123")
 req.Header.Set("Authorization", "Bearer super-secret")
 rr := httptest.NewRecorder()
 h.ServeHTTP(rr, req)
 fmt.Println("status:", rr.Code, "resp id:", rr.Header().Get("X-Request-ID"))
}

```

```
 fmt.Println(strings.TrimSpace(buf.String()))
}

go run .
```

### Expected output:

```
outbound X-Request-ID: abc-123
status: 200 resp id: abc-123
{"time":"...", "level":"INFO", "msg":"handler", "request_id":"abc-123", "path":"/api", "authorization": "bearer token"}
```

### What to notice

- Correlation requires the same id in middleware, handler logs, response, and outbound clients.
- Never log raw bearer tokens; redact at the edge of the logger.
- Prefer structured attrs over string interpolation.

### Try next

- Level from LOG\_LEVEL env; hide Debug when `info`.
- Spawn work with `context` so child goroutines keep the request id.

## Further Reading

- Go `log/slog` package documentation
- OpenTelemetry baggage vs attributes (next chapters)
- Next: **Metrics and Prometheus Design**

# Metrics and Prometheus Design



# Metrics and Prometheus Design

Metrics answer operational questions quickly: Is traffic normal? Are errors rising? Is latency degrading? They are cheap aggregates — if you design labels carefully.

## Why / Overview

Logs are high-cardinality events. Metrics are **pre-aggregated time series**. Prometheus scrapes pull-based endpoints; you expose `/metrics` with a small, intentional surface.

```
process -> counters/histograms/gauges
 |
 /metrics scrape
 |
Prometheus -> alerts / Grafana
```

## RED and USE

**RED** (request-driven services):

Signal	Meaning
Rate	Requests per second
Errors	Failed requests per second
Duration	Latency distribution

**USE** (resources):

Signal	Meaning
Utilization	% busy
Saturation	Queue depth / wait
Errors	Resource error counts

Export both: RED for SLIs, USE for capacity.

## Metric Types

Type	Use
Counter	Monotonic events ( <code>_total</code> suffix)
Gauge	Levels that go up/down (goroutines, depth)
Histogram	Latency/size distributions
Summary	Client-side quantiles (less preferred for prom federation)

Prefer **histograms** for latency with server-side aggregation.

## Instrumentation Example

```
package metrics

import (
 "net/http"
 "strconv"
 "time"

 "github.com/prometheus/client_golang/prometheus"
 "github.com/prometheus/client_golang/prometheus/promauto"
 "github.com/prometheus/client_golang/prometheus/promhttp"
)

var (
 httpRequests = promauto.NewCounterVec(prometheus.CounterOpts{
 Name: "http_requests_total",
 Help: "HTTP requests",
 }, []string{"method", "route", "code"})

 httpDuration = promauto.NewHistogramVec(prometheus.HistogramOpts{
 Name: "http_request_duration_seconds",
 Help: "HTTP latency",
 Buckets: []float64{.005, .01, .025, .05, .1, .25, .5, 1, 2.5, 5, 10},
 }, []string{"method", "route"})

 inFlight = promauto.NewGauge(prometheus.GaugeOpts{
 Name: "http_requests_in_flight",
 Help: "In-flight HTTP requests",
 })
)

func Middleware(next http.Handler) http.Handler {
 return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
```

```

 start := time.Now()
 inFlight.Inc()
 defer inFlight.Dec()
 ww := &statusWriter{ResponseWriter: w, code: 200}
 next.ServeHTTP(ww, r)
 route := routeOf(r) // template, not raw path
 httpRequests.WithLabelValues(r.Method, route, strconv.Itoa(ww.code)).Inc()
 httpDuration.WithLabelValues(r.Method, route).Observe(time.Since(start).Seconds())
 })
}

func Handler() http.Handler { return promhttp.Handler() }

```

Mount metrics on admin listener when possible:

```

go http.ListenAndServe("127.0.0.1:9090", metrics.Handler())

```

## Label Strategy (Cardinality)

Each unique label combination is a time series.

**Safe (bounded):**

- method, route template, code/code\_class, dependency name, breaker\_state

**Unsafe (unbounded):**

- user\_id, email, raw path, url, request\_id, pod IP as high-churn without care

```

// BAD
counter.WithLabelValues(r.URL.Path).Inc()

// GOOD
counter.WithLabelValues(routeTemplate).Inc()

```

Rule of thumb: if a label can take unlimited values from user input, it does not belong on a metric.

## Histogram Buckets

Default buckets may not fit your SLOs. If SLO is 300ms, ensure buckets around 0.1, 0.25, 0.3, 0.5, ...

```

Buckets: prometheus.ExponentialBuckets(0.005, 2, 12)
// or explicit slice aligned to SLOs

```

You cannot fix wrong buckets retroactively without a new metric name.

## Naming Conventions

```
namespace_subsystem_name_unit
http_request_duration_seconds
db_query_duration_seconds
queue_depth
client_requests_total
```

- Counters end with `_total`
- Units in seconds/bytes base units
- Avoid mixing milliseconds and seconds across metrics

## Outbound Client Metrics

```
var depDuration = promauto.NewHistogramVec(prometheus.HistogramOpts{
 Name: "dependency_request_duration_seconds",
 Help: "Outbound calls",
 Buckets: prometheus.DefBuckets,
}, []string{"dep", "method", "result"})

func observeDep(dep, method, result string, d time.Duration) {
 depDuration.WithLabelValues(dep, method, result).Observe(d.Seconds())
}
```

result: ok, timeout, error, cancel — not free-form error strings.

## Exemplars and Trace Links

Modern Prometheus + OTel can attach trace ids to histogram observations (exemplars). When available, latency spikes become one click into a trace. Configure rather than reinvent.

## Alerting from Metrics

Alert on symptoms:

```
// conceptual PromQL
rate(http_requests_total{code=~"5.."}[5m])
/ rate(http_requests_total[5m]) > 0.01
```

And latency:

```
histogram_quantile(0.99, sum by (le) (rate(http_request_duration_seconds_bucket[5m]))) > 0.3
```

Multi-window multi-burn-rate alerts (SRE workbook) reduce noise for SLO burn.

Every alert needs:

1. What is broken (SLI)
2. User impact
3. Runbook link
4. Severity / routing

## Runtime Metrics

Export Go runtime via collectors:

```
// prometheus client includes process and go collectors when using default registry
// go_goroutines, go_memstats_*, process_cpu_seconds_total
```

Useful for saturation; not a substitute for RED SLIs.

## Production Checklist

- ☐ RED metrics per public route template
- ☐ Dependency latency/error metrics
- ☐ Bounded label sets reviewed in PR
- ☐ Histogram buckets match SLOs
- ☐ `/metrics` not public without auth/network policy
- ☐ Alerts tied to SLOs + runbooks
- ☐ Dashboards use rate/increase correctly on counters
- ☐ Queue depth / in-flight gauges for backpressure
- ☐ Document metric stability (renames break alerts)

## Common Pitfalls

1. **Raw path labels** → metric explosion → Prometheus OOM.
2. **Using summaries** then trying to aggregate quantiles across instances incorrectly.
3. **Alerting on CPU only** while error rate is the user pain.
4. **Counter without `_total`** / non-monotonic “counters”.
5. **Measuring only mean latency** — tails hide outages.
6. **Scrape timeout** longer than interval; missed scrapes.
7. **High-cardinality `error_message` labels**.

## Exercises

1. Instrument a toy server with counter + histogram; scrape with a local Prometheus.
2. Add path label vs route template; show series count difference with unique ids in paths.
3. Choose buckets for a 200ms SLO; verify quantile accuracy with known sleeps.
4. Write PromQL for 5xx rate and p99 latency; graph both.
5. Add dependency metrics around an HTTP client.
6. Create an alert rule file; fire it with forced errors; follow a stub runbook.
7. Export queue depth gauge from a worker; alert when depth > N for 10m.
8. Compare `rate` vs `increase` on a slow counter; explain gaps.
9. Secure `/metrics` behind network policy or basic auth.
10. Define error budget math for 99.9% availability over 30 days.

## More examples

### Prometheus text exposition (stdlib)

```
mkdir -p /tmp/go-prom-text && cd /tmp/go-prom-text
go mod init example.com/prom-text
```

Save as `main.go`:

```
package main

import (
 "fmt"
 "net/http"
 "net/http/httptest"
 "strings"
 "sync/atomic"
)

type Counter struct{ n atomic.Int64 }

func (c *Counter) Inc() { c.n.Add(1) }
func (c *Counter) Get() int64 { return c.n.Load() }

func main() {
 var reqs Counter
 mux := http.NewServeMux()
 mux.HandleFunc("GET /", func(w http.ResponseWriter, r *http.Request) {
 reqs.Inc()
 fmt.Fprintln(w, "ok")
 })
 mux.HandleFunc("GET /metrics", func(w http.ResponseWriter, r *http.Request) {
```

```

 w.Header().Set("Content-Type", "text/plain; version=0.0.4")
 fmt.Fprintf(w, "# HELP http_requests_total Total HTTP requests\n")
 fmt.Fprintf(w, "# TYPE http_requests_total counter\n")
 fmt.Fprintf(w, "http_requests_total %d\n", reqs.Get())
 })

 ts := httptest.NewServer(mux)
 defer ts.Close()
 for i := 0; i < 3; i++ {
 r, _ := http.Get(ts.URL + "/")
 r.Body.Close()
 }
 r, _ := http.Get(ts.URL + "/metrics")
 defer r.Body.Close()
 var b [256]byte
 n, _ := r.Body.Read(b[:])
 body := string(b[:n])
 fmt.Println(strings.TrimSpace(body))
 fmt.Println("has type:", strings.Contains(body, "TYPE http_requests_total"))
}

go run .

```

**Expected output:**

```

HELP http_requests_total Total HTTP requests
TYPE http_requests_total counter
http_requests_total 3
has type: true

```

## Labeled counter map (cardinality caution)

```

mkdir -p /tmp/go-prom-labels && cd /tmp/go-prom-labels
go mod init example.com/prom-labels

```

Save as `main.go`:

```

package main

import (
 "fmt"
 "sync"
)

type LabeledCounter struct {

```

```

 mu sync.Mutex
 m map[string]int
}

func (c *LabeledCounter) Inc(code string) {
 c.mu.Lock()
 if c.m == nil {
 c.m = map[string]int{}
 }
 c.m[code]++
 c.mu.Unlock()
}

func (c *LabeledCounter) Render(name string) string {
 c.mu.Lock()
 defer c.mu.Unlock()
 var b string
 for k, v := range c.m {
 b += fmt.Sprintf("%s{code=%q} %d\n", name, k, v)
 }
 return b
}

func main() {
 var c LabeledCounter
 c.Inc("200")
 c.Inc("200")
 c.Inc("500")
 out := c.Render("http_responses_total")
 fmt.Print(out)
 fmt.Println("lines:", len(out) > 0)
}

go run .

```

**Expected output** (label order may vary):

```

http_responses_total{code="200"} 2
http_responses_total{code="500"} 1
lines: true

```

## Runnable example

Manual Prometheus text exposition: counter, gauge, and a crude histogram bucket set—stdlib only, no client\_golang.



```
mkdir -p /tmp/go-metrics && cd /tmp/go-metrics
go mod init example.com/metrics
```

Save as main.go:

```
package main

import (
 "fmt"
 "net/http"
 "net/http/httptest"
 "sort"
 "strings"
 "sync"
 "time"
)

type registry struct {
 mu sync.Mutex
 reqs map[string]float64 // status -> count
 inFlight float64
 // latency buckets in seconds
 bounds []float64
 counts []float64
 sum float64
 count float64
}

func newReg() *registry {
 b := []float64{0.005, 0.01, 0.025, 0.05, 0.1, 0.25, 0.5, 1}
 return ®istry{
 reqs: map[string]float64{},
 bounds: b,
 counts: make([]float64, len(b)+1), // +Inf
 }
}

func (r *registry) observe(status string, d time.Duration) {
 r.mu.Lock()
 defer r.mu.Unlock()
 r.reqs[status]++
 sec := d.Seconds()
 r.sum += sec
 r.count++
 placed := false
 for i, b := range r.bounds {
 if sec <= b {

```

```

 r.counts[i]++
 placed = true
 break
 }
}
if !placed {
 r.counts[len(r.bounds)]++
}
}

func (r *registry) expose() string {
 r.mu.Lock()
 defer r.mu.Unlock()
 var b strings.Builder
 fmt.Fprintf(&b, "# HELP http_requests_total Demo requests\n")
 fmt.Fprintf(&b, "# TYPE http_requests_total counter\n")
 var statuses []string
 for s := range r.reqs {
 statuses = append(statuses, s)
 }
 sort.Strings(statuses)
 for _, s := range statuses {
 fmt.Fprintf(&b, "http_requests_total{status=%q} %g\n", s, r.reqs[s])
 }
 fmt.Fprintf(&b, "# HELP http_in_flight Demo gauge\n")
 fmt.Fprintf(&b, "# TYPE http_in_flight gauge\n")
 fmt.Fprintf(&b, "http_in_flight %g\n", r.inFlight)

 fmt.Fprintf(&b, "# HELP http_request_duration_seconds Demo histogram\n")
 fmt.Fprintf(&b, "# TYPE http_request_duration_seconds histogram\n")
 var cum float64
 for i, bound := range r.bounds {
 cum += r.counts[i]
 fmt.Fprintf(&b, "http_request_duration_seconds_bucket{le=%q} %g\n", fmt.Sprintf("%g",
 }
 cum += r.counts[len(r.bounds)]
 fmt.Fprintf(&b, "http_request_duration_seconds_bucket{le=\"%+Inf\"} %g\n", cum)
 fmt.Fprintf(&b, "http_request_duration_seconds_sum %g\n", r.sum)
 fmt.Fprintf(&b, "http_request_duration_seconds_count %g\n", r.count)
 return b.String()
}

func main() {
 reg := newReg()
 mux := http.NewServeMux()
 mux.HandleFunc("GET /work", func(w http.ResponseWriter, r *http.Request) {

```

```

 reg.mu.Lock()
 reg.inFlight++
 reg.mu.Unlock()
 start := time.Now()
 time.Sleep(12 * time.Millisecond)
 reg.mu.Lock()
 reg.inFlight--
 reg.mu.Unlock()
 reg.observe("200", time.Since(start))
 w.WriteHeader(200)
 })
 mux.HandleFunc("GET /metrics", func(w http.ResponseWriter, _ *http.Request) {
 w.Header().Set("Content-Type", "text/plain; version=0.0.4")
 fmt.Fprint(w, reg.expose())
 })

 for i := 0; i < 3; i++ {
 rr := httptest.NewRecorder()
 mux.ServeHTTP(rr, httptest.NewRequest(http.MethodGet, "/work", nil))
 }
 // one error path
 reg.observe("500", 30*time.Millisecond)

 rr := httptest.NewRecorder()
 mux.ServeHTTP(rr, httptest.NewRequest(http.MethodGet, "/metrics", nil))
 fmt.Print(rr.Body.String())
}

go run .

```

**Expected output (excerpt):**

```

HELP http_requests_total Demo requests
TYPE http_requests_total counter
http_requests_total{status="200"} 3
http_requests_total{status="500"} 1
...
http_request_duration_seconds_count 4

```

### What to notice

- Use route templates as labels, never raw ids (`/user/123`)—cardinality explosion kills Prometheus.
- Histograms need `_bucket` / `_sum` / `_count`; counters use `_total`.
- Gauges go up and down (`in_flight`); counters only increase.

**Try next**

- Scrape with a local Prometheus or `curl` a real listener.
- Write PromQL: `rate(http_requests_total{status="500"}[5m])`.

## Further Reading

- Prometheus metric and labeling best practices
- Google SRE workbook: alerting on SLOs
- Previous: structured logs for event detail
- Next: **Tracing and Context Propagation**

## Tracing and Context Propagation

# Tracing and Context Propagation

Tracing makes distributed latency visible by preserving **causal context** across process boundaries. A single missing propagation hop blinds the whole request path.

## Why / Overview

Metrics say p99 is bad. Logs say a request failed. Traces answer: **where did the time go** across gateway → service → DB → dependency?

```
gateway ----> service A ----> service B ----> datastore
| | |
span span span
trace-id = same across all
```

OpenTelemetry (OTel) is the default modern approach in Go ecosystems; the ideas apply even if you use a vendor SDK.

## Core Concepts

Term	Meaning
Trace	End-to-end request / workflow
Span	Unit of work with start/end, attributes, status
Context	Carries trace/span identity in-process
Propagator	Inject/extract context over HTTP/gRPC headers
Sampler	Which traces to keep

```
W3C Trace Context headers:
traceparent: 00-<trace-id>-<span-id>-<flags>
tracestate: optional vendor data
```

## Context Propagation Rules

1. Accept `context.Context` at API boundaries.
2. Pass the same ctx (or child) into outbound calls — **never** `context.Background()` mid-request unless intentionally detaching.

3. When detaching, copy trace ids into a new context if the async work should remain in the trace (or start a linked span).
4. Instrument DB, HTTP, and queue publish/consume.

```
// BAD
go process(context.Background())

// GOOD: child span / derived context
go process(context.WithoutCancel(ctx)) // Go 1.21+ if you must outlive request cancel
// or: start a new root with explicit link for async jobs
```

`WithoutCancel` preserves values but not cancellation — use carefully for fire-and-forget that should still be attributed.

## Minimal HTTP Propagation (Conceptual)

Even without full OTel, the discipline is:

```
func propagate(req *http.Request, incoming http.Header) {
 if v := incoming.Get("Traceparent"); v != "" {
 req.Header.Set("Traceparent", v)
 }
 if v := incoming.Get("X-Request-Id"); v != "" {
 req.Header.Set("X-Request-Id", v)
 }
}
```

With OTel, use official propagators and `otelhttp` transport wrappers instead of hand-rolling.

## Span Design

Create spans around **meaningful operations**, not every function:

- Inbound HTTP/gRPC request (framework middleware)
- Outbound HTTP/RPC
- DB query / transaction
- Significant queue publish
- Expensive compute blocks

```
// Pseudocode shape with OTel API
ctx, span := tracer.Start(ctx, "db.QueryUser")
defer span.End()
span.SetAttributes(attribute.String("db.system", "postgres"))
row := db.QueryRowContext(ctx, q, id)
if err != nil {
```

```

 span.RecordError(err)
 span.SetStatus(codes.Error, err.Error())
}

```

Attributes: low-cardinality. Do not put raw SQL with unbound ids if that becomes a PII/cardinality problem — use operation names.

## Sampling

Strategy	When
Always on	Low traffic / critical paths
Probability (e.g. 1–10%)	High QPS happy path
Parent-based	Honor upstream decision
Tail sampling	Keep errors/slow traces (collector)

Always sample (or force) **errors and high latency** when the platform allows.

## Common Failure Modes

1. **New client without transport wrapper** — headers not injected.
2. **Goroutine with Background** — orphan spans or missing children.
3. **Queue consumer not extracting** — new unrelated traces per message.
4. **Clock skew** — span order looks weird (use careful interpretation).
5. **Too many tiny spans** — overhead and UI noise.
6. **Sensitive data in attributes** — tokens, PII.

## Queue propagation

Put trace context in message metadata/headers; extract on consume; start a **consumer span** as child or link.

```

producer span -> inject into message
consumer receives -> extract -> process span

```

## Correlation with Logs and Metrics

- Put `trace_id` and `span_id` into slog fields from the current span context.
- Metrics exemplars can point to traces.
- Same `service.name` and resource attributes across signals.



```
// After extracting span from ctx:
slog.InfoContext(ctx, "paid", "order_id", id)
// With a handler that appends trace ids from ctx (OTel slog bridge or custom)
```

## In-Process Performance Note

Span creation is cheap when sampled out, not free when always on with heavy attributes. Measure overhead on ultra-hot paths; batch exporters carefully.

## SDK Lifecycle (Shape)

Regardless of vendor, production services should:

```
func main() {
 ctx := context.Background()
 shutdown, err := initTracer(ctx) // set global TracerProvider + propagator
 if err != nil {
 log.Fatal(err)
 }
 defer func() {
 // Flush remaining spans on exit; bound the wait.
 ctx, cancel := context.WithTimeout(context.Background(), 5*time.Second)
 defer cancel()
 _ = shutdown(ctx)
 }()

 // run server...
}
```

Initialize **once**. Multiple providers create confusing partial traces.

## HTTP middleware and transport (conceptual)

```
// Server: wrap handler so each request becomes a span.
// handler = otelhttp.NewHandler(mux, "api")

// Client: wrap transport so outbound calls inject context.
// client.Transport = otelhttp.NewTransport(http.DefaultTransport)
```

Prefer library wrappers over copying header maps by hand — they handle version updates of the propagation format.

## Baggage vs Attributes

Mechanism	Purpose	Danger
Span attributes	Describe this operation	Cardinality if unbounded
Baggage	Propagate key-values to downstream	Becomes a global gossip channel

Use baggage sparingly (e.g. tenant tier for sampling). Never put PII or secrets in baggage — every hop may log it.

## Reading a Trace During an Incident

1. Open the trace from the failing `request_id` / exemplar.
2. Sort or scan for the longest span or first error status.
3. Check whether time is **in your service** (CPU/DB) or **waiting on a child**.
4. Confirm the slow child has correct timeout budgets (part 13).
5. If the trace stops mid-path, find the missing propagation hop and fix instrumentation first.

A broken trace is itself a reliability bug.

## Production Checklist

- ☐ OTel (or vendor) SDK initialized once at start
- ☐ W3C TraceContext propagator configured
- ☐ HTTP server middleware creates root/child spans
- ☐ HTTP client transport injects context
- ☐ DB/redis clients instrumented or wrapped
- ☐ Async/queue paths extract/inject
- ☐ Sampling policy documented
- ☐ Errors mark span status
- ☐ Logs include `trace_id`
- ☐ Admin-only or secured debug endpoints

## Common Pitfalls

1. Multiple tracer provider initializations.
2. Forgetting **Shutdown** on exporter at process exit (lost tail spans).
3. Using request body content as span names (cardinality + PII).
4. Mixing baggage (cross-cutting key-values) with unbounded values.
5. Assuming trace UI replaces metrics for alerting — it does not.

## Exercises

1. Manually pass `X-Request-Id` across two HTTP services; log it on both.
2. Add W3C `traceparent` generation/parsing in a toy form; verify header round-trip.
3. Break propagation in one client; observe “broken” trace; fix.
4. Start a span around a DB call; add attributes; export to a local collector (Jaeger/OTel).
5. Detach a goroutine with `Background`; compare trace; fix with derived context.
6. Publish a message with trace headers; consume and continue the trace.
7. Configure 1% sampling; force error sampling if available.
8. Bridge slog to include trace ids; grep logs for a trace id from the UI.
9. Measure CPU with tracing on/off under load for a hot endpoint.
10. Write a runbook: “p99 high” → open trace → identify slow span → next action.

## End-to-End Lab Outline

1. Two tiny services: **A** calls **B** over HTTP.
2. Add request ids + manual `traceparent` first; prove logs join.
3. Swap in OTel SDK + Jaeger/OTLP collector locally.
4. Break the client transport wrapper; note the cliff in the UI.
5. Add a DB span in **B**; force a slow query; confirm the waterfall.
6. Drop sample rate to 1%; force an error path to stay visible.

Document the exact env vars and ports your team uses so the lab is repeatable.

## More examples

### Trace/span ids on context and outbound headers

```
mkdir -p /tmp/go-trace-ctx && cd /tmp/go-trace-ctx
go mod init example.com/trace-ctx
```

Save as `main.go`:

```
package main

import (
 "context"
 "fmt"
 "net/http"
 "net/http/httpptest"
)

type ctxKey string
```

```

const (
 traceKey ctxKey = "trace_id"
 spanKey ctxKey = "span_id"
)

func withTrace(ctx context.Context, traceID, spanID string) context.Context {
 ctx = context.WithValue(ctx, traceKey, traceID)
 return context.WithValue(ctx, spanKey, spanID)
}

func inject(req *http.Request) {
 if v, ok := req.Context().Value(traceKey).(string); ok {
 req.Header.Set("X-Trace-Id", v)
 }
 if v, ok := req.Context().Value(spanKey).(string); ok {
 req.Header.Set("X-Span-Id", v)
 }
}

func main() {
 upstream := httptest.NewServer(http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
 fmt.Fprintf(w, "trace=%s span=%s", r.Header.Get("X-Trace-Id"), r.Header.Get("X-Span-Id"))
 }))
 defer upstream.Close()

 ctx := withTrace(context.Background(), "t-1", "s-9")
 req, _ := http.NewRequestWithContext(ctx, http.MethodGet, upstream.URL, nil)
 inject(req)
 resp, err := http.DefaultClient.Do(req)
 if err != nil {
 panic(err)
 }
 defer resp.Body.Close()
 var buf [64]byte
 n, _ := resp.Body.Read(buf[:])
 fmt.Println(string(buf[:n]))
}

go run .

```

**Expected output:**

trace=t-1 span=s-9

## Nested span timing (in-process)

```
mkdir -p /tmp/go-span-time && cd /tmp/go-span-time
go mod init example.com/span-time
```

Save as main.go:

```
package main

import (
 "fmt"
 "time"
)

type Span struct {
 Name string
 Dur time.Duration
}

func timed(name string, fn func()) Span {
 start := time.Now()
 fn()
 return Span{Name: name, Dur: time.Since(start)}
}

func main() {
 var spans []Span
 spans = append(spans, timed("handler", func() {
 spans = append(spans, timed("db", func() { time.Sleep(5 * time.Millisecond) }))
 spans = append(spans, timed("cache", func() { time.Sleep(1 * time.Millisecond) }))
 }))
 for _, s := range spans {
 fmt.Printf("%s >=1ms: %v\n", s.Name, s.Dur >= time.Millisecond)
 }
}

go run .
```

Expected output:

```
db >=1ms: true
cache >=1ms: true
handler >=1ms: true
```

## Runnable example

W3C-style traceparent generate/parse and HTTP propagation between two `httptest` services—`stdlib` only (no OTel SDK).

```
mkdir -p /tmp/go-traceprop && cd /tmp/go-traceprop
go mod init example.com/traceprop
```

Save as `main.go`:

```
package main

import (
 "crypto/rand"
 "encoding/hex"
 "fmt"
 "net/http"
 "net/http/httptest"
 "strings"
)

func newTraceparent() string {
 var tid [16]byte
 var sid [8]byte
 _, _ = rand.Read(tid[:])
 _, _ = rand.Read(sid[:])
 return fmt.Sprintf("00-%s-%s-01", hex.EncodeToString(tid[:]), hex.EncodeToString(sid[:]))
}

func parseTraceparent(s string) (traceID, spanID string, ok bool) {
 parts := strings.Split(s, "-")
 if len(parts) != 4 || parts[0] != "00" || len(parts[1]) != 32 || len(parts[2]) != 16 {
 return "", "", false
 }
 return parts[1], parts[2], true
}

func main() {
 // Service B
 b := httptest.NewServer(http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
 tp := r.Header.Get("traceparent")
 tid, sid, ok := parseTraceparent(tp)
 fmt.Printf("B saw trace_ok=%v trace_id=%s parent_span=%s\n", ok, tid, sid)
 fmt.Fprintln(w, "b-ok")
 }))
 defer b.Close()
}
```

```

// Service A calls B, propagating traceparent
a := httptest.NewServer(http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
 tp := r.Header.Get("traceparent")
 if tp == "" {
 tp = newTraceparent()
 }
 tid, _, _ := parseTraceparent(tp)
 // child span id for outbound
 var child [8]byte
 _, _ = rand.Read(child[:])
 outTP := fmt.Sprintf("00-%s-%s-01", tid, hex.EncodeToString(child[:]))
 fmt.Printf("A outbound traceparent=%s\n", outTP)

 req, _ := http.NewRequest(http.MethodGet, b.URL+"/work", nil)
 req.Header.Set("traceparent", outTP)
 resp, err := http.DefaultClient.Do(req)
 if err != nil {
 http.Error(w, err.Error(), 500)
 return
 }
 resp.Body.Close()
 w.Header().Set("traceparent", tp)
 fmt.Fprintln(w, "a-ok")
}))
defer a.Close()

req, _ := http.NewRequest(http.MethodGet, a.URL+"/", nil)
req.Header.Set("traceparent", newTraceparent())
resp, err := http.DefaultClient.Do(req)
if err != nil {
 panic(err)
}
resp.Body.Close()
fmt.Println("client status:", resp.StatusCode)

// Broken propagation demo
_, _, ok := parseTraceparent("not-a-trace")
fmt.Println("invalid rejected:", !ok)
}

go run .

```

**Expected output** (ids random):

```

A outbound traceparent=00-...
B saw trace_ok=true trace_id=... parent_span=...
client status: 200

```

`invalid rejected: true`

### What to notice

- Trace id stays stable across the request tree; span id changes per hop.
- Forgetting to copy `traceparent` on the client is the classic “broken trace” bug.
- Real systems add OTel SDK exporters; the propagation contract stays the same.

### Try next

- Put `trace_id` into slog fields for log trace joins.
- Continue a trace across a simulated queue message header map.

## Further Reading

- OpenTelemetry Go documentation
- W3C Trace Context specification
- Previous: Prometheus metrics for detection; traces for diagnosis
- Next part: **Security Hardening** — protecting the systems you can now observe



## **SLIs, SLOs, and Error Budgets**

# SLIs, SLOs, and Error Budgets

## Overview

Metrics without targets become noise. **SLIs** measure user-visible health; **SLOs** set goals; **error budgets** decide how fast you can ship vs stabilize.

## Definitions

Term	Meaning	Example
SLI	Quantitative measure	% of requests < 300ms; success ratio
SLO	Target on an SLI	99.9% success over 30 days
SLA	Contractual promise	Often looser than internal SLO
Error budget	1 - SLO failures allowed	0.1% budget for risk

## Good SLIs for HTTP APIs

```
availability = successful requests / total (exclude 4xx client errors carefully)
latency = fraction of requests under threshold (p99 or histogram)
freshness = data lag (async pipelines)
```

Prefer **user journeys** (login, checkout) over raw CPU.

## Expressing latency as an SLI

With Prometheus histograms:

```
histogram_quantile(0.99, sum(rate(http_request_duration_seconds_bucket[5m])) by (le))
```

Or “good events” style:

```
count of requests with duration <= 0.3s / count of requests
```

## Error budget policy (lightweight)

Budget remaining	Engineering posture
Healthy	Ship features, experiments OK
Burning fast	Freeze risky deploys; fix reliability
Exhausted	Reliability work only until recovery

## Minimal Go: count success/fail

```
var (
 reqTotal = expvar.NewInt("requests_total")
 reqFailed = expvar.NewInt("requests_failed")
)

// in handler:
reqTotal.Add(1)
if status >= 500 {
 reqFailed.Add(1)
}
```

Graduate to Prometheus counters/histograms (chapter 162).

## Rules of thumb

Do	Don't
Few SLOs (1–3 per critical UX)	50 red dashboards, zero owners
Align alerts to budget burn	Alert on every CPU blip
Review SLOs quarterly	Set 99.99% with no measurement

## Try next

1. Define SLIs for one service (availability + latency).
2. Compute monthly error budget from request volume.
3. Write a one-page policy: what happens when budget is empty.

## **Health, Liveness, and Readiness Probes**

# Health, Liveness, and Readiness Probes

## Overview

Orchestrators ask “is this process OK?” in two different ways:

Probe	Question	Bad answer means
<b>Liveness</b>	Should we restart the process?	Kill + restart
<b>Readiness</b>	Should we send traffic?	Remove from LB

Confusing them causes restart loops or black-hole traffic.

## Minimal endpoints

```
var ready atomic.Bool

func init() { ready.Store(true) }

func livez(w http.ResponseWriter, r *http.Request) {
 w.WriteHeader(http.StatusOK)
 _, _ = w.Write([]byte("ok"))
}

func readyz(w http.ResponseWriter, r *http.Request) {
 if !ready.Load() {
 http.Error(w, "not ready", http.StatusServiceUnavailable)
 return
 }
 // optional: ping DB with short timeout
 w.WriteHeader(http.StatusOK)
}
```

## Dependency checks

Check in readiness	Check in liveness
DB ping (short timeout)	Almost never deep deps
Required config loaded	Process deadlocked (hard)
Warmup finished	

**Do not** make liveness fail when a dependency is down—that restarts everyone during a DB blip.

## Drain on shutdown

```
// on SIGTERM:
ready.Store(false)
time.Sleep(2 * time.Second) // allow LB to observe
_ = srv.Shutdown(ctx)
```

## Startup probe (K8s)

Slow boots: separate startup probe so liveness doesn't kill during init.

## Rules of thumb

Do	Don't
Cheap liveness	Heavy DB work on livez
Ready=false while draining	Keep ready=true until process death
Bound check timeouts	Hang probe handlers

## Try next

1. Add `/livez` + `/readyz` to a service.
2. Fail readiness when DB ping fails; keep liveness OK.
3. Wire drain on SIGTERM; curl `readyz` during shutdown.

## Alerting and Runbooks

# Alerting and Runbooks

## Overview

Alerts should be **actionable, rare, and tied to user pain**. Every page should have a short **runbook**: what it means, what to check, how to mitigate.

## Alert design

Prefer	Avoid
SLO burn (error budget)	Raw CPU > 80% forever
Symptom (errors, latency)	Cause-only (GC pause) without user impact
Multi-window burn rates	Noisy single-sample thresholds

Example intent:

```
Alert if 5m error rate > 5% AND 1h error rate > 1%
→ catches both spikes and slow burns
```

## Severity

Level	Response
page	Human now
ticket	Next business hours
log-only	Not an alert

If nobody will act at 3 a.m., it is not a page.



## Runbook template

```
HighErrorRate

Symptom
5xx rate above SLO burn threshold.

Impact
Users fail to load checkout / API clients retry.

Quick checks
1. Deploy recently? (`kubectl rollout history`)
2. Dependency errors in logs (filter `dependency=`)
3. Saturation: CPU, threads, DB connections

Mitigations
1. Rollback last deploy if correlated
2. Shed non-critical traffic / enable rate limit
3. Scale if clearly capacity-bound

Escalation
Platform on-call if cluster-wide
```

## Linking code to ops

Log a stable **error code** and metric label (`code=db_timeout`) so runbooks can search.

```
slog.Error("checkout_failed", "code", "db_timeout", "err", err)
```

## Rules of thumb

Do	Don't
One runbook per paging alert	Page without owner
Test alerts in staging	Alert on untested promql
Review flappy alerts weekly	Mute forever without fixing

## Try next

1. Write runbooks for two real alerts.

2. Delete or downgrade one noisy alert.
3. Add `code=` to top error logs.

## **Project: Instrumented Mini Service**

# Project: Instrumented Mini Service

## Overview

Ship a tiny HTTP service with **slog JSON**, request IDs, **RED-ish expvar/Prometheus counters**, **livez/readyz**, and a **shutdown drain**.

## Checklist

- ☐ GET `/api/echo?msg=` returns JSON
- ☐ Middleware: request ID + access log
- ☐ Counters: `requests_total`, `requests_failed`, latency histogram (or expvar)
- ☐ `/livez` always 200 if process up
- ☐ `/readyz` fails when `-unready` file exists or after SIGTERM
- ☐ Graceful Shutdown
- ☐ README: how to curl probes + sample log line

## Stretch

- OpenTelemetry trace stub
- Prometheus `/metrics` via `promhttp`
- SLO doc: 99% success, p99 < 200ms on echo

## Acceptance

```
go test ./...
go run . &
curl -s localhost:8080/livez
```

```
curl -s localhost:8080/api/echo?msg=hi
kill -TERM $pid # readyz → 503, then exit
```

# **Security Hardening**

# Security Hardening Overview

# Security Hardening Overview

This part focuses on **practical secure-by-default design** in Go services: transport identity, access decisions, and secret lifecycle. The objective is to reduce both exploitability and operational misconfiguration risk.

## Why / Overview

Security failures in Go services are often not exotic crypto bugs. They are:

- Expired certificates during a quiet holiday
- JWT validation that skips `aud/iss`
- Authorization checks only on the UI path
- Secrets in env dumps and CI logs
- mTLS configured on one side only

```
client
 | TLS / mTLS
 v
edge -- authn --> principal
 |
 +-- authz --> allow/deny on resource+action
 |
 +-- secrets --> short-lived credentials
```

Part 11 covered tooling and application hardening baselines. This part deepens **transport**, **identity**, and **secrets**.

## Learning Path

Chapter	Focus	Outcome
171 TLS / mTLS	Trust stores, versions, rotation	Safe transport identity
172 Authn / Authz	Principals vs policies	Correct access decisions
173 Secrets	Injection, dual-read rotation	Credential lifecycle without outages
174 JWT validation	Alg/exp/aud/iss	Safe bearer tokens
175 Input validation & SSRF	Boundaries, URL fetch	No open proxies / traversal



Chapter	Focus	Outcome
176 Supply chain	govulncheck, modules, builds	Dependency hygiene
177 Project: secure checklist	Harden a real service	Evidence-based security pass

## Core Principles

1. **Authenticate then authorize** — never authorize anonymous by accident.
2. **Default deny** — missing policy is deny, not allow.
3. **Short-lived credentials** — blast radius of leaks shrinks with TTL.
4. **Encrypt in transit** everywhere that is not a deliberate exception.
5. **Separate data plane and control plane** credentials.
6. **Fail closed** on auth/crypto misconfiguration at startup when possible.

## Go Building Blocks

Area	Packages
TLS	<code>crypto/tls</code> , <code>crypto/x509</code>
HTTP security	<code>net/http</code> + middleware
Tokens	<code>golang-jwt</code> or similar (review carefully), <code>PASETO</code> , session cookies
Secrets	<code>env</code> , files, Vault/cloud SM SDKs
Random	<code>crypto/rand</code>

## Relationship to Other Parts

- **Hardening (81)** — process, headers, input limits
- **Network systems** — where TLS terminates and proxies hop
- **Observability** — audit logs for authz denials; never log secrets
- **Distributed infra** — service identity for discovery peers

## Literacy Checklist

- ☐ Configure TLS 1.2+ with sane cipher suites
- ☐ Explain mTLS client cert verification
- ☐ Rotate certs without downtime (overlap windows)
- ☐ Separate authn modules from authz policy engines
- ☐ Validate JWT/`iss`/`aud`/`exp` correctly
- ☐ Dual-read secrets during rotation
- ☐ Redact credentials in logs and errors

## Part Warm-Up Exercises

1. Find where TLS terminates in your architecture (app vs proxy vs mesh).
2. List every secret your service reads at startup.
3. Identify one endpoint that authenticates but might skip authorize.
4. Check certificate expiry monitoring exists.
5. Locate any hardcoded tokens in repo history (then rotate them).

## Threat Model Snapshot (Service-Centric)

For a typical Go API, assume:

Threat	Primary controls in this part
Network eavesdropping / tampering	TLS
Spoofed internal callers	mTLS or signed service identity
Stolen user session / token	short TTL, audience checks, revoke paths
Confused deputy / IDOR	resource-level authz
Leaked DB password	rotation, dual-read, least privilege
Misissued cert	trust pins, monitoring, automation

Attackers prefer **misconfiguration** over breaking AES-GCM. Your design should make the secure path the default path.

## Layering with Part 11

Part 11 (earlier)	Part 17 (here)
govulncheck, gosec	TLS session identity
Process/container hardening	Authn/authz architecture
Input limits, headers	Secrets lifecycle
General checklist	Rotation choreography

Use both: tools catch known issues; these chapters design the trust model.

## Minimal Secure Service Skeleton

```
listen TLS (or receive from mesh)
 -> request id middleware
 -> authenticate (session/JWT/mTLS)
 -> authorize (action + resource)
 -> business handler (context with principal)
 -> audit deny / sensitive allow
secrets: loaded at start, refreshed, never logged
```

If a route skips any of the middle boxes, that is a design smell requiring explicit justification (e.g. public /healthz).

## Compliance-Adjacent Habits (Not Legal Advice)

Even without a formal compliance program, good habits transfer:

- Inventory secrets and certs
- Record who can read production secrets
- Prefer short-lived credentials
- Keep audit logs for admin and authz deny
- Separate prod and non-prod credentials completely

## Secure Defaults Checklist (Part Preview)

Copy into PR templates for services that handle auth or secrets:

- ☐ TLS 1.2+ (or mesh-enforced equivalent)
- ☐ No `InsecureSkipVerify` outside explicit test builds
- ☐ Authn middleware before business handlers
- ☐ Authz checks resource ownership (anti-IDOR)
- ☐ Tokens validate `iss`, `aud`, `exp`, algorithm
- ☐ Cookies: `HttpOnly`, `Secure`, deliberate `SameSite`
- ☐ Secrets not in git; startup fails if missing
- ☐ Rotation dual-read tested in staging
- ☐ Authz denials audited with request id
- ☐ pprof/admin not on public interface

## Testing Security Logic

Kind	Example
Unit	<code>Authorize</code> table tests; JWT claim rejection
Integration	mTLS handshake success/fail matrices

Kind	Example
Negative	expired cert, wrong audience, cross-tenant id
Chaos	rotate secret mid-traffic with dual-read

Security tests that only cover the happy path give false confidence.

## Where Identity Lives

```
Human user -> session cookie / OIDC access token -> API
Service A -> mTLS or signed JWT (client credentials) -> Service B
Job worker -> task-scoped token or instance identity -> APIs
```

Do not reuse human user tokens for service calls. Audience and lifetime should differ.

## Mapping Incidents → Chapter

Incident	Start here
TLS handshake errors / expired cert	171
Users see other tenants' data	172 (IDOR / authz)
401 storms after deploy	172 (token validation) or 173 (secret mismatch)
DB auth failures during rotation	173 dual-read
"Works on internal network without TLS"	171 threat model

## Non-Goals

This part does not replace:

- Full application penetration testing
- Formal cryptography design reviews
- Cloud IAM deep dives for every provider

It does give Go service authors a trustworthy baseline so specialists are not fixing the same footguns repeatedly.

## Glossary

- **Authn / Authz** — identity vs permission
- **mTLS** — mutual TLS, both peers present certificates
- **Trust bundle** — set of CAs you accept
- **Dual-read** — accept old and new secret during rotation
- **Principal** — authenticated actor in your system
- **Audience (aud)** — intended recipient of a token
- **IDOR** — insecure direct object reference (authz bug)
- **SPIFFE** — workload identity URI SAN convention

## More examples

### Tour: bearer check + TLS client roots sketch

```
mkdir -p /tmp/go-sec-tour && cd /tmp/go-sec-tour
go mod init example.com/sec-tour
```

Save as main.go:

```
package main

import (
 "crypto/tls"
 "crypto/x509"
 "fmt"
 "net/http"
 "net/http/httptest"
 "strings"
)

func main() {
 // Authn stub
 h := http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
 auth := r.Header.Get("Authorization")
 if !strings.HasPrefix(auth, "Bearer ") || auth == "Bearer " {
 http.Error(w, "unauthorized", http.StatusUnauthorized)
 return
 }
 fmt.Fprintln(w, "ok")
 })
 rr := httptest.NewRecorder()
 req := httptest.NewRequest(http.MethodGet, "/", nil)
 h.ServeHTTP(rr, req)
 fmt.Println("no token:", rr.Code)
```

```

 rr = httptest.NewRecorder()
 req = httptest.NewRequest(http.MethodGet, "/", nil)
 req.Header.Set("Authorization", "Bearer demo")
 h.ServeHTTP(rr, req)
 fmt.Println("with token:", rr.Code)

 // TLS: trust httptest cert
 ts := httptest.NewTLSServer(http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
 fmt.Fprintln(w, "https")
 }))
 defer ts.Close()
 roots := x509.NewCertPool()
 roots.AddCert(ts.Certificate())
 client := &http.Client{Transport: &http.Transport{
 TLSClientConfig: &tls.Config{RootCAs: roots, MinVersion: tls.VersionTLS12},
 }}
 resp, err := client.Get(ts.URL)
 if err != nil {
 panic(err)
 }
 resp.Body.Close()
 fmt.Println("tls status:", resp.StatusCode)
}

go run .

```

### Expected output:

```

no token: 401
with token: 200
tls status: 200

```

## Runnable example

Tour of this part: HMAC token mint/verify (authn-ish), a tiny allow/deny policy (authz), and dual-secret verification (rotation)—stdlib crypto.

```

mkdir -p /tmp/go-sec-tour && cd /tmp/go-sec-tour
go mod init example.com/sec-tour

```

Save as `main.go`:

```

package main

import (

```

```

"crypto/hmac"
"crypto/sha256"
"encoding/hex"
"fmt"
)

func sign(key, msg string) string {
 m := hmac.New(sha256.New, []byte(key))
 _, _ = m.Write([]byte(msg))
 return hex.EncodeToString(m.Sum(nil))
}

func verifyDual(keys []string, msg, sig string) bool {
 raw, err := hex.DecodeString(sig)
 if err != nil {
 return false
 }
 for _, k := range keys {
 m := hmac.New(sha256.New, []byte(k))
 _, _ = m.Write([]byte(msg))
 if hmac.Equal(raw, m.Sum(nil)) {
 return true
 }
 }
 return false
}

func authorize(role, action string) bool {
 policy := map[string][]string{
 "reader": {"read"},
 "admin": {"read", "write", "delete"},
 }
 for _, a := range policy[role] {
 if a == action {
 return true
 }
 }
 return false
}

func main() {
 msg := "user=42"
 sig := sign("key-new", msg)
 fmt.Println("verify new only:", verifyDual([]string{"key-new"}, msg, sig))
 fmt.Println("dual-read old+new:", verifyDual([]string{"key-old", "key-new"}, msg, sig))
 fmt.Println("authz reader write:", authorize("reader", "write"))
 fmt.Println("authz admin write:", authorize("admin", "write"))
}

```

```
 fmt.Println("tour: transport(identity) + authz + secret rotation")
}

go run .
```

### Expected output:

```
verify new only: true
dual-read old+new: true
authz reader write: false
authz admin write: true
tour: transport(identity) + authz + secret rotation
```

### What to notice

- Authn (who) and authz (what) are separate layers; TLS is transport identity (171).
- Dual-read prevents outages during secret rotation (173).

### Try next

- Chapter 171 TLS client/server demo; 172 middleware; 173 fail-closed env secrets.

## Next Chapter

**TLS and mTLS Deep Dive** — the foundation of service identity on the wire.



# **TLS and mTLS Deep Dive**

# TLS and mTLS Deep Dive

TLS provides confidentiality and integrity for bytes on the wire. **mTLS** additionally authenticates the client with a certificate, giving strong transport-level identity between services.

## Why / Overview

Most TLS incidents are lifecycle failures, not breaks of AES:

- Expired certificates
- Wrong trust bundle after rotation
- Clients still on TLS 1.0
- Server name mismatches
- Half-applied mTLS (server requests client cert but does not verify)

```
one-way TLS: client verifies server certificate
mTLS: client verifies server AND server verifies client
```

## Go Server TLS

```
cfg := &tls.Config{
 MinVersion: tls.VersionTLS12,
 // Prefer TLS 1.3 when both sides support it (automatic if available).
 CurvePreferences: []tls.CurveID{
 tls.X25519,
 tls.CurveP256,
 },
 // Certificates: static or GetCertificate for dynamic reload.
 Certificates: []tls.Certificate{cert},
}

srv := &http.Server{
 Addr: ":8443",
 Handler: mux,
 TLSConfig: cfg,
 // timeouts still required
 ReadHeaderTimeout: 5 * time.Second,
}
```

```
log.Fatal(srv.ListenAndServeTLS("", "")) // empty if certs in TLSConfig
```

Load cert from PEM:

```
cert, err := tls.LoadX509KeyPair("server.crt", "server.key")
```

## Cipher suites

TLS 1.3 cipher suites are not configurable the same way as 1.2. For 1.2, prefer AEAD suites and avoid legacy CBC/RC4. When in doubt, set `MinVersion: tls.VersionTLS12` and let Go's defaults work; override only with a threat-model reason.

## Dynamic Certificate Reload

Long-lived processes need cert rotation without downtime:

```
type certHolder struct {
 mu sync.RWMutex
 cert tls.Certificate
}

func (h *certHolder) get(_ *tls.ClientHelloInfo) (*tls.Certificate, error) {
 h.mu.RLock()
 defer h.mu.RUnlock()
 c := h.cert
 return &c, nil
}

func (h *certHolder) reload(certFile, keyFile string) error {
 c, err := tls.LoadX509KeyPair(certFile, keyFile)
 if err != nil {
 return err
 }
 h.mu.Lock()
 h.cert = c
 h.mu.Unlock()
 return nil
}

// TLSConfig.GetCertificate = holder.get
```

Reload on SIGHUP, fs notify, or periodic poll. **Never** replace files non-atomically (see filesystem chapter).

## Client TLS

```
rootPEM, _ := os.ReadFile("ca.crt")
pool := x509.NewCertPool()
pool.AppendCertsFromPEM(rootPEM)

client := &http.Client{
 Transport: &http.Transport{
 TLSClientConfig: &tls.Config{
 RootCAs: pool,
 MinVersion: tls.VersionTLS12,
 ServerName: "api.internal", // SNI + verify name
 },
 },
 Timeout: 10 * time.Second,
}
```

Never ship `InsecureSkipVerify: true` in production. If you need custom verify (name overrides, SPIFFE IDs), use `VerifyPeerCertificate` / `VerifyConnection` carefully.

## mTLS

Server requests and verifies client certificates:

```
clientCA := x509.NewCertPool()
clientCA.AppendCertsFromPEM(clientCAPEM)

tlsCfg := &tls.Config{
 MinVersion: tls.VersionTLS12,
 ClientAuth: tls.RequireAndVerifyClientCert,
 ClientCAs: clientCA,
 Certificates: []tls.Certificate{serverCert},
}
```

Client presents cert:

```
cert, _ := tls.LoadX509KeyPair("client.crt", "client.key")
tlsCfg := &tls.Config{
 Certificates: []tls.Certificate{cert},
 RootCAs: pool,
 MinVersion: tls.VersionTLS12,
 ServerName: "payments.internal",
}
```

## Identity from client cert

```
func peerSPIFFEOrCN(r *http.Request) (string, error) {
 if r.TLS == nil || len(r.TLS.PeerCertificates) == 0 {
 return "", errors.New("no peer cert")
 }
 cert := r.TLS.PeerCertificates[0]
 // Prefer URI SANs (SPIFFE) over CN in modern designs.
 for _, u := range cert.URIs {
 return u.String(), nil
 }
 return cert.Subject.CommonName, nil
}
```

Authorize **after** extracting identity (next chapter) — transport authn   business authz.

## Trust Bundles and Rotation

Rotation sequence for CAs:

1. Distribute new CA alongside old (trust both)
2. Issue leaf certs from new CA
3. Deploy leaves
4. Remove old CA only after all leaves rotated

For leaf certs:

```
issue new leaf -> dual accept if needed -> switch serve -> revoke/expire old
```

Monitor:

- Days to expiry (cert exporter / custom metric)
- Handshake failure rates
- TLS version negotiation

## HTTP/2 and ALPN

Go enables HTTP/2 over TLS when configured correctly. ALPN negotiates protocols. Mis-set `NextProtos` can break h2.

## Proxies and TLS

Common patterns:

Pattern	Notes
TLS terminate at edge	Backend may be plain on private net (risk: flat network)
Re-encrypt to backend	Edge → backend TLS
Pass-through	Edge routes TCP; backend terminates
Mesh mTLS	Sidecars handle identity

If you trust “internal HTTP,” document the compensating control (network policy, mesh).

## Production Checklist

- ☐ MinVersion TLS 1.2+
- ☐ Private keys 0600, not in image layers with world read
- ☐ Automated expiry monitoring and paging
- ☐ Reload path tested for leaf rotation
- ☐ mTLS: `RequireAndVerifyClientCert` + correct ClientCAs
- ☐ Clients set `ServerName` / use proper URL host
- ☐ No `InsecureSkipVerify` in prod configs
- ☐ CA rotation dual-trust window documented
- ☐ Handshake errors metrics + logs (without dumping secrets)

## Common Pitfalls

1. **Clock skew** fails cert validity (`NotBefore/NotAfter`).
2. **Forgetting intermediate certs** in the server chain.
3. **File replace** of `tls.key` while process holds old fd only — need reload.
4. **CN-only verification** without SANs (modern clients expect SANs).
5. **ClientAuth RequestClientCert** without verify — provides no security.
6. **Logging private keys** during debug.
7. **Wildcard trust** too broad in ClientCAs.

## Exercises

1. Generate a local CA + server cert with `openssl` or `mkcert`; serve HTTPS with Go.
2. Connect with the system CA only; observe failure; fix RootCAs.
3. Enable mTLS; connect without client cert; confirm failure; then succeed with client cert.
4. Implement `GetCertificate` hot reload; replace files; curl still works.
5. Set `MinVersion` to TLS 1.3 only; test old client behavior.

6. Extract URI SAN identity in a handler; map to a principal.
7. Emit a gauge `tls_cert_not_after_seconds`; alert at  $< 14$  days.
8. Deliberately expire a cert; document failure symptoms in client and server logs.
9. Compare edge termination vs app termination threat models in a one-pager.
10. Rotate CA with dual-trust; prove old leaves still work until cutover completes.

## Certificate File Permissions Checklist

```
private key must not be group/world readable
chmod 600 server.key
cert can be world-readable
chmod 644 server.crt
```

In Kubernetes, use projected secrets with restrictive default modes (`defaultMode: 0400` for keys). Never `console.log` or `slog` the PEM contents.

## Handshake Failure Debugging Order

1. Clock skew (`NotBefore` / `NotAfter`)
2. Missing intermediate in server chain
3. Client trust bundle missing the issuing CA
4. SNI / `ServerName` mismatch vs certificate SAN
5. mTLS: client cert not sent or not signed by `ClientCAs`
6. Version floor: client only offers TLS 1.0, server requires 1.2+

Capture with `openssl s_client -connect host:443 -servername host -showcerts` before changing application code.

## More examples

### TLS server + RootCAs client (httptest)

```
mkdir -p /tmp/go-tls-roots && cd /tmp/go-tls-roots
go mod init example.com/tls-roots
```

Save as `main.go`:

```
package main

import (
 "crypto/tls"
 "crypto/x509"
 "fmt"
```

```

 "io"
 "net/http"
 "net/http/httptest"
)

 func main() {
 ts := httptest.NewTLSServer(http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
 fmt.Fprintf(w, "version_ok=%v", r.TLS != nil)
 }))
 defer ts.Close()

 roots := x509.NewCertPool()
 roots.AddCert(ts.Certificate())
 client := &http.Client{Transport: &http.Transport{
 TLSClientConfig: &tls.Config{RootCAs: roots, MinVersion: tls.VersionTLS12},
 }}
 resp, err := client.Get(ts.URL)
 if err != nil {
 panic(err)
 }
 b, _ := io.ReadAll(resp.Body)
 resp.Body.Close()
 fmt.Println(string(b))
 _, err = http.Get(ts.URL)
 fmt.Println("default client fails:", err != nil)
 }

 go run .

```

### Expected output:

```

version_ok=true
default client fails: true

```

### Inspect peer cert fields in handler

```

mkdir -p /tmp/go-tls-peer && cd /tmp/go-tls-peer
go mod init example.com/tls-peer

```

Save as main.go:

```

package main

import (
 "crypto/tls"

```



```

 "crypto/x509"
 "fmt"
 "net/http"
 "net/http/httptest"
)

func main() {
 ts := httptest.NewTLSServer(http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
 if r.TLS == nil {
 http.Error(w, "no tls", 500)
 return
 }
 fmt.Fprintf(w, "negotiated>=1.2=%v peers=%d",
 r.TLS.Version >= tls.VersionTLS12, len(r.TLS.PeerCertificates))
 }))
 defer ts.Close()

 roots := x509.NewCertPool()
 roots.AddCert(ts.Certificate())
 client := &http.Client{Transport: &http.Transport{
 TLSClientConfig: &tls.Config{RootCAs: roots, MinVersion: tls.VersionTLS12},
 }}
 resp, err := client.Get(ts.URL)
 if err != nil {
 panic(err)
 }
 defer resp.Body.Close()
 var buf [64]byte
 n, _ := resp.Body.Read(buf[:])
 fmt.Println(string(buf[:n]))
}

go run .

```

### Expected output:

negotiated>=1.2=true peers=0

(peers=0 without client certs; mTLS would raise this.)

## Runnable example

HTTPS with `httptest.NewTLSServer` and a client that uses the server's certificate pool (no `InsecureSkipVerify` in the happy path). Also shows a misconfigured skip-verify client for contrast.

```
mkdir -p /tmp/go-tls-demo && cd /tmp/go-tls-demo
go mod init example.com/tls-demo
```

Save as main.go:

```
package main

import (
 "crypto/tls"
 "crypto/x509"
 "fmt"
 "io"
 "net/http"
 "net/http/httpptest"
)

func main() {
 ts := httpptest.NewTLSServer(http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
 fmt.Fprintf(w, "hello tls state=%v\n", r.TLS != nil)
 }))
 defer ts.Close()

 // Trust the test server's cert (httpptest exposes it).
 roots := x509.NewCertPool()
 roots.AddCert(ts.Certificate())
 secureClient := &http.Client{
 Transport: &http.Transport{
 TLSClientConfig: &tls.Config{
 RootCAs: roots,
 MinVersion: tls.VersionTLS12,
 },
 },
 }
 resp, err := secureClient.Get(ts.URL)
 if err != nil {
 panic(err)
 }
 body, _ := io.ReadAll(resp.Body)
 resp.Body.Close()
 fmt.Printf("secure: %s", body)

 // Default client does not trust httpptest certs → error
 _, err = http.Get(ts.URL)
 fmt.Println("default client error:", err != nil)

 // InsecureSkipVerify works but is not production policy
 insecure := &http.Client{
```

```

 Transport: &http.Transport{
 TLSClientConfig: &tls.Config{InsecureSkipVerify: true, MinVersion: tls.VersionTL
 },
 }
 resp, err = insecure.Get(ts.URL)
 if err != nil {
 panic(err)
 }
 resp.Body.Close()
 fmt.Println("insecure skip verify status:", resp.StatusCode)
 fmt.Println("MinVersion TLS1.2 configured on both clients")
}

go run .

```

### Expected output:

```

secure: hello tls state=true
default client error: true
insecure skip verify status: 200
MinVersion TLS1.2 configured on both clients

```

### What to notice

- Production clients need a real trust bundle (RootCAs), not InsecureSkipVerify.
- `httptest.NewTLSServer` is the practical way to exercise TLS in unit tests.
- Set `MinVersion: tls.VersionTLS12` (or 1.3-only) deliberately.

### Try next

- Require client certs (`ClientAuth: tls.RequireAndVerifyClientCert`) for an mTLS sketch.
- Inspect `r.TLS.Version` and peer certificates on the server handler.

## Further Reading

- Go `crypto/tls` configuration documentation
- Mozilla SSL configuration generator (concepts)
- Next: **Authentication and Authorization Patterns**

# **Authentication and Authorization Patterns**

# Authentication and Authorization Patterns

Authentication answers **who are you?** Authorization answers **what are you allowed to do?**  
Mixing them produces systems that are hard to test and easy to bypass.

## Why / Overview

Common failures:

- JWT parsed but signature not verified
- Authn on API gateway, authz forgotten on internal admin routes
- Role checks as string equality scattered in handlers
- IDOR: authorized as user A, still can fetch user B's resource by id

```
request -> authenticate -> Principal
 -> authorize(Principal, action, resource) -> allow/deny
```

## Authentication Patterns

### 1. Session cookies (first-party browsers)

```
// After credential check:
sid := newSessionID()
store.Save(sid, userID, time.Now().Add(24*time.Hour))
http.SetCookie(w, &http.Cookie{
 Name: "session",
 Value: sid,
 Path: "/",
 HttpOnly: true,
 Secure: true, // HTTPS only
 SameSite: http.SameSiteLaxMode,
 MaxAge: 86400,
})
```

Use CSRF protection for cookie-based mutating requests.

## 2. Bearer tokens (APIs)

```
func bearer(r *http.Request) (string, bool) {
 h := r.Header.Get("Authorization")
 if !strings.HasPrefix(h, "Bearer ") {
 return "", false
 }
 return strings.TrimPrefix(h, "Bearer "), true
}
```

Validate JWT (or opaque token introspection) **with**:

- Signature / HMAC key or JWKS
- exp / nbf
- iss issuer allowlist
- aud audience allowlist
- algorithm allowlist (reject none)

```
// Conceptual validation steps - use a well-reviewed library.
tok, err := jwt.Parse(token, keyFunc, jwt.WithValidMethods([]string{"RS256"}))
// check claims.Issuer, claims.Audience, claims.ExpiresAt
```

## 3. mTLS identity

Peer certificate URI SAN / SPIFFE ID becomes the principal for service-to-service calls (previous chapter). Still run authz: not every client cert may call every route.

## 4. API keys

Fine for simple server-to-server; store **hashes** of keys, not raw keys; rate-limit; rotate.

## Principal Model

```
type Principal struct {
 Subject string // user id or spiffe://...
 Type string // user|service
 Roles []string // optional coarse
 Scopes []string // oauth-like
 Tenant string // multi-tenant systems
}
```

Attach to context:

```

type principalKey struct{}

func WithPrincipal(ctx context.Context, p Principal) context.Context {
 return ctx.WithValue(ctx, principalKey{}, p)
}

func PrincipalFrom(ctx context.Context) (Principal, bool) {
 p, ok := ctx.Value(principalKey{}).(Principal)
 return p, ok
}

```

## Authorization Patterns

### RBAC (role-based)

```

var perms = map[string][]string{
 "admin": {"order:read", "order:write", "user:read"},
 "user": {"order:read", "order:write:own"},
}

func allowed(p Principal, perm string) bool {
 for _, role := range p.Roles {
 for _, x := range perms[role] {
 if x == perm {
 return true
 }
 }
 }
 return false
}

```

### Resource ownership (critical for IDOR)

```

func canReadOrder(p Principal, o Order) bool {
 if allowed(p, "order:read:any") {
 return true
 }
 return allowed(p, "order:read:own") && o.UserID == p.Subject
}

```

**Always** load the resource and check ownership server-side. Never trust client-provided `user_id` alone.

## Policy as pure function

```
type Request struct {
 Principal Principal
 Action string
 Resource Resource
}

func Authorize(req Request) error {
 if !policy(req) {
 return ErrDeny
 }
 return nil
}
```

Pure functions are unit-testable with tables of allow/deny cases.

## Middleware composition

```
func requireAuth(next http.Handler) http.Handler {
 return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
 p, err := authenticate(r)
 if err != nil {
 http.Error(w, "unauthorized", http.StatusUnauthorized)
 return
 }
 next.ServeHTTP(w, r.WithContext(WithPrincipal(r.Context(), p)))
 })
}

func requirePerm(perm string, next http.Handler) http.Handler {
 return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
 p, ok := PrincipalFrom(r.Context())
 if !ok || !allowed(p, perm) {
 http.Error(w, "forbidden", http.StatusForbidden)
 return
 }
 next.ServeHTTP(w, r)
 })
}
```

Status codes: **401** unauthenticated, **403** authenticated but not allowed. Do not leak whether a resource exists if that is sensitive (sometimes always 404).



## Deny by Default

```
// Route registration: only explicitly wrapped routes are reachable.
mux.Handle("GET /admin/metrics", requireAuth(requirePerm("admin:metrics", adminHandler)))
```

Avoid a global “if admin skip checks” flag that can be left on.

## Multi-Tenant Awareness

Every query should be scoped:

```
// BAD: SELECT * FROM orders WHERE id=$1
// GOOD: SELECT * FROM orders WHERE id=$1 AND tenant_id=$2
```

Authorization + data filtering work together.

## Audit Logging

Log authz denials and sensitive allows:

```
slog.Warn("authz_deny",
 "sub", p.Subject,
 "action", action,
 "resource", resourceID,
 "request_id", RequestID(ctx),
)
```

Never log raw passwords or tokens.

## Testing Matrix

Case	Expect
No credentials	401
Bad signature / expired	401
Valid user, wrong role	403
Valid user, not owner	403
Valid owner	200
Admin override	200 if policy says so

Table-driven tests for **Authorize** catch regressions better than handler snapshots alone.

## Production Checklist

- ☐ Authn and authz modules separated
- ☐ Token validation includes iss/aud/exp/alg
- ☐ Cookies: HttpOnly, Secure, SameSite set intentionally
- ☐ CSRF strategy for cookie sessions
- ☐ Resource-level checks (anti-IDOR)
- ☐ Default deny routing
- ☐ Tenant scoping in data access
- ☐ Audit logs for deny and admin actions
- ☐ Load tests do not bypass auth in prod configs
- ☐ Service-to-service identity (mTLS or signed tokens)

## Common Pitfalls

1. **Parse-only JWT** without verify.
2. **Trust gateway headers** (X-User-Id) without authenticating the gateway hop.
3. **Frontend-only authorization.**
4. **String role checks** duplicated and drifting.
5. **403 vs 401 confusion** breaking client refresh flows.
6. **Overbroad admin role** granted to services.
7. **Debug auth bypass** left enabled.

## Exercises

1. Implement session cookie login/logout with secure flags; attempt theft via XSS hypothetical — why HttpOnly helps.
2. Validate a JWT with wrong **aud**; prove rejection.
3. Build **Authorize** pure function with 10 table cases including IDOR.
4. Create two middleware layers; write tests with **httptest**.
5. Add **tenant\_id** to queries; attempt cross-tenant read; deny.
6. Log authz deny with request id; redact token fields.
7. Convert scattered role checks into a permission map; delete duplicates.
8. Simulate expired token; ensure 401 not 500.
9. Service identity: map mTLS SPIFFE ID to Principal; authorize routes.
10. Threat-model one endpoint: list authn/authz failures and tests for each.

## More examples

### Bearer authn middleware

```
mkdir -p /tmp/go-authn-bearer && cd /tmp/go-authn-bearer
go mod init example.com/authn-bearer
```

Save as main.go:

```
package main

import (
 "context"
 "fmt"
 "net/http"
 "net/http/httptest"
 "strings"
)

type ctxKey string

const userKey ctxKey = "user"

func bearerAuth(next http.Handler) http.Handler {
 return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
 h := r.Header.Get("Authorization")
 if !strings.HasPrefix(h, "Bearer ") {
 http.Error(w, "unauthorized", http.StatusUnauthorized)
 return
 }
 token := strings.TrimPrefix(h, "Bearer ")
 if token != "good-token" {
 http.Error(w, "unauthorized", http.StatusUnauthorized)
 return
 }
 ctx := context.WithValue(r.Context(), userKey, "alice")
 next.ServeHTTP(w, r.WithContext(ctx))
 })
}

func main() {
 h := bearerAuth(http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
 fmt.Fprintln(w, r.Context().Value(userKey))
 }))

 rr := httptest.NewRecorder()
```

```

h.ServeHTTP(rr, httptest.NewRequest(http.MethodGet, "/", nil))
fmt.Println("missing:", rr.Code)

rr = httptest.NewRecorder()
req := httptest.NewRequest(http.MethodGet, "/", nil)
req.Header.Set("Authorization", "Bearer good-token")
h.ServeHTTP(rr, req)
fmt.Println("ok:", rr.Code, strings.TrimSpace(rr.Body.String()))
}

go run .

```

### Expected output:

```

missing: 401
ok: 200 alice

```

### Role-based authz gate

```

mkdir -p /tmp/go-authz-rbac && cd /tmp/go-authz-rbac
go mod init example.com/authz-rbac

```

Save as main.go:

```

package main

import (
 "fmt"
 "net/http"
 "net/http/httptest"
)

type Principal struct {
 Name string
 Roles map[string]bool
}

func requireRole(role string, p Principal, next http.Handler) http.Handler {
 return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
 if !p.Roles[role] {
 http.Error(w, "forbidden", http.StatusForbidden)
 return
 }
 next.ServeHTTP(w, r)
 })
}

```

```

}

func main() {
 admin := Principal{Name: "a", Roles: map[string]bool{"admin": true}}
 user := Principal{Name: "u", Roles: map[string]bool{"user": true}}
 core := http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
 fmt.Fprintln(w, "secret")
 })

 rr := httptest.NewRecorder()
 requireRole("admin", user, core).ServeHTTP(rr, httptest.NewRequest(http.MethodGet, "/",
 fmt.Println("user:", rr.Code)

 rr = httptest.NewRecorder()
 requireRole("admin", admin, core).ServeHTTP(rr, httptest.NewRequest(http.MethodGet, "/",
 fmt.Println("admin:", rr.Code)
}

go run .

```

### Expected output:

```

user: 403
admin: 200

```

## Runnable example

Bearer token authn middleware + pure `Authorize` function with table-driven cases (including IDOR-style deny)—`httptest` only.

```

mkdir -p /tmp/go-authz && cd /tmp/go-authz
go mod init example.com/authz

```

Save as `main.go`:

```

package main

import (
 "context"
 "fmt"
 "net/http"
 "net/http/httptest"
 "strings"
)

```

```

type Principal struct {
 ID string
 Role string
}

type ctxKey string

const principalKey ctxKey = "principal"

func authenticate(token string) (Principal, bool) {
 // Demo token map; production: verify JWT/session server-side.
 db := map[string]Principal{
 "tok-alice": {ID: "u1", Role: "user"},
 "tok-admin": {ID: "u0", Role: "admin"},
 }
 p, ok := db[token]
 return p, ok
}

func Authorize(p Principal, action, resourceOwner string) bool {
 if p.Role == "admin" {
 return true
 }
 if action == "read" || action == "write" {
 return p.ID == resourceOwner // prevent IDOR
 }
 return false
}

func withAuth(next http.Handler) http.Handler {
 return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
 h := r.Header.Get("Authorization")
 if !strings.HasPrefix(h, "Bearer ") {
 http.Error(w, "unauthorized", http.StatusUnauthorized)
 return
 }
 p, ok := authenticate(strings.TrimPrefix(h, "Bearer "))
 if !ok {
 http.Error(w, "unauthorized", http.StatusUnauthorized)
 return
 }
 ctx := context.WithValue(r.Context(), principalKey, p)
 next.ServeHTTP(w, r.WithContext(ctx))
 })
}

func main() {

```

```

mux := http.NewServeMux()
mux.Handle("GET /items/{owner}", withAuth(http.HandlerFunc(func(w http.ResponseWriter, r
 p := r.Context().Value(principalKey).(Principal)
 owner := r.PathValue("owner")
 if !Authorize(p, "read", owner) {
 http.Error(w, "forbidden", http.StatusForbidden)
 return
 }
 fmt.Fprintf(w, "ok user=%s owner=%s\n", p.ID, owner)
})))

call := func(token, path string) int {
 req := httptest.NewRequest(http.MethodGet, path, nil)
 if token != "" {
 req.Header.Set("Authorization", "Bearer "+token)
 }
 rr := httptest.NewRecorder()
 mux.ServeHTTP(rr, req)
 return rr.Code
}

fmt.Println("no token:", call("", "/items/u1"))
fmt.Println("alice own:", call("tok-alice", "/items/u1"))
fmt.Println("alice idor:", call("tok-alice", "/items/u9"))
fmt.Println("admin idor:", call("tok-admin", "/items/u9"))
fmt.Println("bad token:", call("nope", "/items/u1"))
}

go run .

```

### Expected output:

```

no token: 401
alice own: 200
alice idor: 403
admin idor: 200
bad token: 401

```

### What to notice

- 401 = not authenticated; 403 = authenticated but not allowed—clients refresh on 401, not 403.
- Authorization is a pure function—easy to table-test; do not scatter role checks only in the UI.
- Resource owner checks stop IDOR when IDs appear in URLs.

### Try next

- Add `tenant_id` to `Principal` and deny cross-tenant reads.

- Validate JWT `aud/iss/exp` with `crypto` + careful parsing libraries (or session cookies).

## Further Reading

- OWASP Authentication and Access Control cheatsheets
- Previous: TLS/mTLS as transport authn
- Next: **Secrets Management and Rotation**



# Secrets Management and Rotation

# Secrets Management and Rotation

Secrets are **dynamic credentials with a lifecycle**, not static configuration strings. Most secret-related outages come from rotation choreography errors, not from encryption algorithm choice.

## Why / Overview

Secrets include:

- Database passwords
- API tokens and HMAC keys
- TLS private keys
- OAuth client secrets
- Encryption keys (DEK/KEK hierarchy)

```
issue new secret -> readers accept both -> writers switch -> revoke old
```

If you flip both ends at once without overlap, you race yourself into downtime.

## Never Do This

Anti-pattern	Why
Secrets in git	History is forever; bots scan public repos
Secrets in Docker layers	Recoverable from images
Secrets in client apps	Extractable
Long-lived shared passwords	Huge blast radius
Logs printing env	CI and support tools leak

## Loading Secrets at Runtime

### Environment variables

Common in 12-factor apps; simple but:

- Visible in `/proc` and some debug endpoints
- Easy to dump accidentally
- Awkward dual-read of two values (use `DB_PASSWORD` + `DB_PASSWORD_OLD`)

```
func mustSecret(key string) string {
 v := os.Getenv(key)
 if v == "" {
 slog.Error("missing secret", "key", key)
 os.Exit(1)
 }
 return v
}
```

## Files (Kubernetes secrets mounts)

```
func readSecretFile(path string) (string, error) {
 b, err := os.ReadFile(path)
 if err != nil {
 return "", err
 }
 return strings.TrimSpace(string(b)), nil
}
```

Many platforms rotate by updating the file; processes must **re-read** or restart.

## Secret managers

AWS SM, GCP SM, Azure Key Vault, HashiCorp Vault, etc.:

- Short-lived credentials via IAM roles
- Central audit
- API-driven rotation

Wrap the vendor SDK behind a small interface:

```
type Source interface {
 Get(ctx context.Context, name string) (string, error)
}
```

## Fail Closed at Startup

```
func run() error {
 cfg, err := loadConfig()
 if err != nil {
 return err
 }
 if cfg.DBPassword == "" {
```

```

 return errors.New("db password required")
 }
 // open DB, then listen
 return serve(cfg)
}

```

Do not listen on :8080 and return 500 for all traffic because secrets failed — fail before ready.

## Dual-Read Rotation (Passwords / HMAC)

```

Time ->
DB password: [v1 active] [v1+v2 accepted] [v2 active]
App readers: [v1] [try v2 then v1] [v2]
App writers: [v1] [switch to v2] [v2]

type RotatingToken struct {
 mu sync.RWMutex
 cur []byte
 prev []byte
}

func (r *RotatingToken) Verify(mac, msg []byte) bool {
 r.mu.RLock()
 defer r.mu.RUnlock()
 if hmac.Equal(mac, sum(r.cur, msg)) {
 return true
 }
 if len(r.prev) > 0 && hmac.Equal(mac, sum(r.prev, msg)) {
 return true
 }
 return false
}

func (r *RotatingToken) Rotate(newKey []byte) {
 r.mu.Lock()
 defer r.mu.Unlock()
 r.prev = r.cur
 r.cur = newKey
}

```

For DB users, prefer **two users** or two passwords supported by the server during cutover.

## Runtime Refresh Without Restart

```
func (a *App) watchSecrets(ctx context.Context, every time.Duration) {
 t := time.NewTicker(every)
 defer t.Stop()
 for {
 select {
 case <-ctx.Done():
 return
 case <-t.C:
 sec, err := a.src.Get(ctx, "db-password")
 if err != nil {
 slog.Error("secret refresh failed", "err", err)
 continue // keep old; alert on repeated failure
 }
 if err := a.db.UpdatePassword(sec); err != nil {
 slog.Error("db rebind failed", "err", err)
 }
 }
 }
}
```

Some drivers need a new pool to pick up password changes — test the path.

## Encryption Keys vs Passwords

For application-level encryption:

- Use a **KEK** in a KMS to wrap **DEKs**
- Store wrapped DEKs next to ciphertext
- Rotate KEK via rewrap; rotate DEK by re-encrypting data in batches

Do not invent AES modes; use `crypto/cipher` with GCM properly (unique nonces) or higher-level libraries.

```
func encrypt(key, plaintext []byte) ([]byte, error) {
 block, err := aes.NewCipher(key)
 if err != nil {
 return nil, err
 }
 gcm, err := cipher.NewGCM(block)
 if err != nil {
 return nil, err
 }
 nonce := make([]byte, gcm.NonceSize())
 if _, err := rand.Read(nonce); err != nil {
```

```

 return nil, err
 }
 return gcm.Seal(nonce, nonce, plaintext, nil), nil
}

```

## CI/CD and Build-Time Leaks

- Inject secrets at deploy time, not build time, when possible
- Mask secrets in CI logs
- Prefer OIDC federated login to clouds over long-lived CI keys
- Scan releases with secret scanners

## Redaction Everywhere

```

type RedactedString string

func (r RedactedString) String() string { return "***" }
func (r RedactedString) GoString() string { return "***" }
func (r RedactedString) MarshalJSON() ([]byte, error) { return []byte(`***`), nil }

```

Use carefully for types that must not appear in logs.

## Production Checklist

- ☐ No secrets in git or images
- ☐ Required secrets fail startup (not silent empty)
- ☐ Dual-read/dual-trust windows for rotation
- ☐ Automated rotation where platform supports it
- ☐ Refresh or restart path tested quarterly
- ☐ Access audit on secret manager
- ☐ Least-privilege IAM for secret read
- ☐ Alert on refresh failures
- ☐ Revocation plan for leaked credentials
- ☐ TLS keys use restricted file perms / KMS

## Common Pitfalls

1. **Single global static secret** for all environments.
2. **Rotation without dual-read** → full outage.
3. **Copy-paste secrets** into tickets and Slack.
4. **Same DB password** for migrate job and app with different cutover times.

5. **Not revoking** after suspected leak (only rotating sometimes).
6. **Base64 encryption** for “hiding” secrets in config.
7. **Long-lived JWTs** as a substitute for session server state without revoke list.

## Exercises

1. Refactor a sample app to fail closed when `API_TOKEN` is missing.
2. Implement dual-key HMAC verify; rotate; prove old and new both verify during window.
3. Mount a secret file; update it; implement re-read; confirm new value used.
4. Add `RedactedString`; log a config struct; ensure secret not visible.
5. Simulate leak: document revoke + rotate steps as a runbook.
6. Compare env vs file vs Vault for your deployment model (table of tradeoffs).
7. Create two DB users; rotate app from A to B without downtime (local docker).
8. Scan a docker image history for accidental `.env` copy; fix Dockerfile.
9. Instrument `secret_refresh_failures_total`; alert on `>0` for 15m.
10. Design DEK/KEK scheme for storing personal tokens at rest; review nonce handling.

## More examples

### Dual-secret accept during rotation

```
mkdir -p /tmp/go-secret-dual && cd /tmp/go-secret-dual
go mod init example.com/secret-dual
```

Save as `main.go`:

```
package main

import (
 "crypto/hmac"
 "crypto/sha256"
 "encoding/hex"
 "fmt"
)

func sign(secret, msg string) string {
 m := hmac.New(sha256.New, []byte(secret))
 _, _ = m.Write([]byte(msg))
 return hex.EncodeToString(m.Sum(nil))
}

func valid(secrets []string, msg, sig string) bool {
 for _, s := range secrets {
 want, err1 := hex.DecodeString(sign(s, msg))
 }
}
```

```

 got, err2 := hex.DecodeString(sig)
 if err1 != nil || err2 != nil {
 continue
 }
 if hmac.Equal(want, got) {
 return true
 }
 }
 return false
}

func main() {
 old, neu := "old-secret", "new-secret"
 msg := "payload"
 // After rotation: sign with new, accept both for a window.
 sig := sign(neu, msg)
 fmt.Println("new only:", valid([]string{neu}, msg, sig))
 fmt.Println("dual window:", valid([]string{neu, old}, msg, sig))
 fmt.Println("old sig still ok in window:", valid([]string{neu, old}, msg, sign(old, msg)))
 fmt.Println("after cutover old rejected:", valid([]string{neu}, msg, sign(old, msg)))
}

go run .

```

### Expected output:

```

new only: true
dual window: true
old sig still ok in window: true
after cutover old rejected: false

```

### Load secret from file with restrictive mode check

```

mkdir -p /tmp/go-secret-file && cd /tmp/go-secret-file
go mod init example.com/secret-file

```

Save as main.go:

```

package main

import (
 "fmt"
 "os"
)

```



```

func loadSecret(path string) (string, error) {
 st, err := os.Stat(path)
 if err != nil {
 return "", err
 }
 if st.Mode().Perm() & 0o077 != 0 {
 return "", fmt.Errorf("secret %s mode too open: %v", path, st.Mode().Perm())
 }
 b, err := os.ReadFile(path)
 if err != nil {
 return "", err
 }
 return string(b), nil
}

func main() {
 path := "secret.txt"
 _ = os.WriteFile(path, []byte("s3cr3t"), 0o600)
 defer os.Remove(path)
 s, err := loadSecret(path)
 fmt.Println("ok:", s, err)

 _ = os.Chmod(path, 0o644)
 _, err = loadSecret(path)
 fmt.Println("open mode rejected:", err != nil)
}

go run .

```

### Expected output:

```

ok: s3cr3t <nil>
open mode rejected: true

```

## Runnable example

Fail-closed env loading, dual-key HMAC verify during rotation, and a `RedactedString` that never prints the secret.

```

mkdir -p /tmp/go-secrets && cd /tmp/go-secrets
go mod init example.com/secrets

```

Save as `main.go`:

```

package main

import (
 "crypto/hmac"
 "crypto/sha256"
 "encoding/hex"
 "fmt"
 "os"
)

type RedactedString string

func (r RedactedString) String() string { return "***" }
func (r RedactedString) GoString() string { return "RedactedString(***)" }
func (r RedactedString) Reveal() string { return string(r) }

func mustEnv(k string) string {
 v := os.Getenv(k)
 if v == "" {
 panic("missing required secret: " + k)
 }
 return v
}

func sign(key, msg string) string {
 m := hmac.New(sha256.New, []byte(key))
 _, _ = m.Write([]byte(msg))
 return hex.EncodeToString(m.Sum(nil))
}

func verify(keys []string, msg, sig string) bool {
 raw, err := hex.DecodeString(sig)
 if err != nil {
 return false
 }
 for _, k := range keys {
 mac := hmac.New(sha256.New, []byte(k))
 _, _ = mac.Write([]byte(msg))
 if hmac.Equal(raw, mac.Sum(nil)) {
 return true
 }
 }
 return false
}

func main() {
 // Demo defaults when unset (production: mustEnv only).

```

```

primary := os.Getenv("API_TOKEN_CURRENT")
if primary == "" {
 primary = "current-secret"
}
previous := os.Getenv("API_TOKEN_PREVIOUS") // may be empty after rotation window

secret := RedactedString(primary)
fmt.Printf("config log: token=%s\n", secret) // ***

msg := "payload"
sigOld := sign("previous-secret", msg)
sigNew := sign(primary, msg)

keys := []string{primary}
if previous != "" {
 keys = append(keys, previous)
} else {
 keys = append(keys, "previous-secret") // demo dual-read window
}
fmt.Println("accept new:", verify(keys, msg, sigNew))
fmt.Println("accept old during dual-read:", verify(keys, msg, sigOld))
fmt.Println("reject other:", verify(keys, msg, sign("other", msg)))

// Fail-closed illustration
if os.Getenv("REQUIRE_STRICT") == "1" {
 _ = mustEnv("API_TOKEN_CURRENT")
} else {
 fmt.Println("strict mode off (set REQUIRE_STRICT=1 to exercise mustEnv)")
}
}

go run .
REQUIRE_STRICT=1 API_TOKEN_CURRENT=abc go run .

```

### Expected output:

```

config log: token=***
accept new: true
accept old during dual-read: true
reject other: false
strict mode off (set REQUIRE_STRICT=1 to exercise mustEnv)

```

### What to notice

- Dual-read (current + previous) avoids total outage at rotation cutover.
- Custom `String()` prevents accidental secret leakage via `fmt` and `slog`.
- Missing required secrets should prevent process start, not run with empty keys.

### Try next

- Re-read a secret file on SIGHUP or timer; atomic swap of the active key slice.
- Metric `secret_refresh_failures_total` when reload fails.

### Further Reading

- NIST guidance on secrets management (high level)
- Previous: authn/authz that *use* these secrets
- Next part: **Performance Engineering** — speed with evidence, not secrets of folklore

## JWT and Token Validation

# JWT and Token Validation

## Overview

JWTs are common bearer tokens. Most bugs are **validation shortcuts**: skipping `exp`, `aud`, `iss`, or accepting `alg=none`. Validate explicitly.

Use a maintained library ([github.com/golang-jwt/jwt](https://github.com/golang-jwt/jwt) or similar); do not hand-roll crypto.

## Minimal validation checklist

Claim / check	Why
Signature + algorithm allowlist	Integrity
<code>exp</code> / <code>nbf</code>	Lifetime
<code>iss</code>	Who minted
<code>aud</code>	Intended recipient
<code>sub</code> present	Principal id

```
// sketch with jwt/v5 style API
tok, err := jwt.Parse(tokenStr, func(t *jwt.Token) (any, error) {
 if t.Method.Alg() != jwt.SigningMethodHS256.Alg() {
 return nil, fmt.Errorf("unexpected alg")
 }
 return hmacSecret, nil
}, jwt.WithValidMethods([]string{"HS256"}),
 jwt.WithAudience("bookstore-api"),
 jwt.WithIssuer("auth.example.com"),
)
```

## Access vs refresh

Token	Lifetime	Storage
Access	minutes	memory / short cookie
Refresh	longer	httpOnly cookie, rotate

## Service-to-service

Prefer mTLS or short-lived minted tokens over eternal shared HS256 secrets across many services.

## Rules of thumb

Do	Don't
Allowlist algorithms	Trust <code>alg</code> header blindly
Validate <code>aud</code> / <code>iss</code>	Only check signature
Short access TTL	Multi-day bearer in <code>localStorage</code> without XSS plan

## Try next

1. Reject expired tokens in a unit test.
2. Reject wrong `aud`.
3. Document key rotation for HS256/RS256.

# **Input Validation and SSRF**



# Input Validation and SSRF

## Overview

Validate at trust boundaries. **SSRF** (server-side request forgery) happens when user-controlled URLs make your server fetch internal resources (169.254.169.254, localhost, cloud metadata).

## Validation basics

```
type CreateBook struct {
 Title string `json:"title"`
 Price int `json:"price_cents"`
}

func (c CreateBook) Valid() error {
 if len(c.Title) == 0 || len(c.Title) > 200 {
 return fmt.Errorf("title length")
 }
 if c.Price < 0 {
 return fmt.Errorf("price")
 }
 return nil
}
```

- Prefer allowlists over blocklists for enums
- Bound string lengths and collection sizes
- Parse numbers with bit size (`ParseInt(..., 10, 64)`)

## Path traversal

```
// never: os.Open("/data/" + userPath)
// use safeJoin from filepath chapter
```

## SSRF defenses

When your app fetches a URL on behalf of a user:

1. **Allowlist schemes** (https only)
2. **Allowlist hosts** or resolve and block private ranges
3. **No redirects** to off-allowlist targets
4. Timeouts + size limits

```
func safeURL(raw string) (*url.URL, error) {
 u, err := url.Parse(raw)
 if err != nil {
 return nil, err
 }
 if u.Scheme != "https" {
 return nil, fmt.Errorf("scheme")
 }
 host := u.Hostname()
 ips, err := net.DefaultResolver.LookupIPAddr(ctx, host)
 // reject loopback, link-local, private RFC1918, metadata IP
 return u, nil
}

func isBadIP(ip net.IP) bool {
 return ip.IsLoopback() || ip.IsPrivate() || ip.IsLinkLocalUnicast() ||
 ip.IsLinkLocalMulticast() || ip.Equal(net.ParseIP("169.254.169.254"))
}
```

Re-check IPs after redirects (pin or disable redirects).

## Header injection

Don't put raw user strings into `Header.Set` without validation (CRLF). Prefer structured values.

## Rules of thumb

Do	Don't
Validate then use	Trust client JSON shapes unbounded
Allowlist outbound hosts	Open proxy to any URL
Bound body size	Stream forever into memory

## Try next

1. Reject `http://127.0.0.1/` in a fetch helper.
2. Reject path `../../etc/passwd` in download handler.
3. Fuzz validators with `go test -fuzz`.

# Supply Chain and Dependencies

# Supply Chain and Dependencies

## Overview

Your binary includes every module you import. Supply-chain hygiene: pin versions, review updates, scan vulns, reproduce builds.

## Minimal practices

Practice	Tool / habit
Modules	<code>go.mod</code> / <code>go.sum</code> committed
Vuln scan	<code>govulncheck ./...</code>
Minimal deps	resist “one function” modules
Checksums	<code>go.sum</code> authenticity via <code>sumdb</code>
CI	fail on known critical vulns

```
go install golang.org/x/vuln/cmd/govulncheck@latest
govulncheck ./...
```

## go.mod hygiene

```
go get module@vX.Y.Z # explicit
go mod tidy
go mod download
```

Prefer **semantic import versions** and avoid `replace` to random forks in production without review.

## Vendoring (optional)

```
go mod vendor
build with -mod=vendor for airgapped CI
```

## Reproducible builds

```
CGO_ENABLED=0 go build -trimpath -ldflags="-s -w -X main.version=$VER"
```

Document Go version (toolchain line in `go.mod`).

## Private modules

```
GOPRIVATE=github.com/myorg/*
```

Don't leak private paths to public proxies.

## Rules of thumb

Do	Don't
Run govulncheck in CI	Ignore <code>go.sum</code> conflicts blindly
Review upgrades	Auto-merge major deps without tests
Pin actions SHA in CI	<code>uses: foo@main</code> for build

## Try next

1. Run `govulncheck` on this repo's modules.
2. Add CI step.
3. Build with `-trimpath` and compare reproducibility notes.

## **Project: Secure Service Checklist**

# Project: Secure Service Checklist

## Overview

Take any small HTTP service (Bookstore or echo) and harden it against a **checklist**. Tick items with code or config evidence.

## Transport & process

- ☐ TLS in production path (or documented edge termination)
- ☐ ReadHeaderTimeout set on `http.Server`
- ☐ Non-root container user
- ☐ Graceful shutdown

## Authn / authz

- ☐ Auth on mutating routes
- ☐ Password hashing (bcrypt/argon2) if local auth
- ☐ JWT: alg allowlist + exp + aud (if used)
- ☐ CSRF strategy documented for cookie sessions

## Input / output

- ☐ MaxBytesReader on bodies
- ☐ Path jail for file ops
- ☐ No SSRF open URL fetch
- ☐ Security headers (or edge config)



## Secrets & deps

- ☐ Secrets not in git
- ☐ govulncheck clean or waived
- ☐ Structured logs without secrets

## Deliverable

SECURITY.md in the project with checklist status + residual risks.

# **Performance Engineering**

# Performance Engineering Overview

# Performance Engineering Overview

This part frames performance as an **evidence-driven workflow**. The primary rule is simple: measure first, optimize second. Guessing hotspots is a reliable way to complicate code without moving p99.

## Why / Overview

Go is fast enough for most network services out of the box. Performance work becomes necessary when:

- Tail latency violates SLOs
- Allocation rate drives GC pauses under load
- Lock contention caps throughput below CPU saturation
- A deploy regresses a critical path

```
baseline -> profile -> hypothesize -> change one thing -> remeasure -> keep/revert
```

## Learning Path

Chapter	Focus	Outcome
181 Benchmarking & profiling	bench, pprof, trace workflow	Find real hotspots
182 Memory, allocs, GC	Escape analysis, pooling, GC knobs	Control allocation pressure
183 Contention & concurrency	Mutex profiles, sharding	Raise throughput ceilings
184 Load testing & baselines	hey/vegeta, capacity	Evidence before optimize
185 Allocation recipes	Hot-path fixes	Lower allocs with discipline
186 PGO & build flags	-pgo, trimpath, ldflags	Production binary polish
187 Project: optimize endpoint	Full loop	Before/after report

## Core Principles

1. **User-visible metrics first** — p99 latency, error rate, throughput.
2. **Reproduce with stable benchmarks or load tests** before micro-tweaks.
3. **One variable per experiment.**
4. **Prefer algorithmic fixes** over micro-optimizations.

5. **Complexity costs** — pooling and sharding need proof.
6. **Regressions are bugs** — keep benchmarks in CI for critical paths.

## Go Tooling Map

Tool	Question
<code>testing.B</code>	How fast is this function?
<code>pprof CPU</code>	Where is CPU time?
<code>pprof heap</code>	Where are allocations?
<code>pprof mutex/block</code>	Who waits?
<code>go tool trace</code>	Scheduler, GC, latency gaps
<code>benchstat</code>	Are two runs different?
metrics / APM	Production truth

## Relationship to Other Parts

- Observability detects symptoms; this part deepens **local diagnosis**.
- Concurrency chapters supply patterns you will tune here.
- Network timeouts interact with performance (slow is not always “needs pprof”).

## Literacy Checklist

- ☐ Write a meaningful benchmark with stable inputs
- ☐ Capture and read a CPU profile flame graph
- ☐ Read allocs/op and B/op from benches
- ☐ Identify GC pressure symptoms
- ☐ Diagnose mutex contention with profiles
- ☐ Use `benchstat` for before/after
- ☐ Know when to stop optimizing

## Part Warm-Up Exercises

1. List your top three latency SLOs and whether you have benchmarks for them.
2. Enable pprof on a dev binary; open `/debug/pprof/`.
3. Run `go test -bench=. -benchmem` on a module you own.
4. Recall a past “optimization” that did nothing; what was missing?
5. Note GOMAXPROCS and container CPU limits in your deploy env.

## Performance vs Correctness vs Cost

Optimize in this priority order unless the product says otherwise:

1. **Correctness** under concurrency (`-race`, invariants)
2. **User SLOs** (latency, errors)
3. **Efficiency** (CPU, memory, \$)
4. **Microbenchmark bragging rights**

A faster wrong answer is still wrong. A 2% CPU win that adds a subtle race is a net loss.

## When Not to Optimize

- You have no measurement of the problem
- The code is not on a hot path (profile says <1%)
- Clarity loss is large and gain is within noise
- The real bottleneck is a dependency you do not control (fix timeouts/caching instead)

“Premature optimization” is not “never optimize” — it is “optimize without evidence.”

## Environment Parity

Benchmarks lie when:

Local	Production
8 fast cores	2 throttled vCPU
NVMe disk	network block volume
empty cache	multi-tenant noisy neighbor
<code>-race</code> off always	sometimes on in CI only

Record `GOMAXPROCS`, CPU quota, and Go version next to every performance claim.

## Suggested Lab Sequence

1. Write a deliberately slow handler (JSON + lock + alloc)
2. Load test; capture CPU + mutex + heap profiles
3. Fix the top issue only; remeasure with `benchstat` / p99
4. Repeat once more; stop when SLO is met

That sequence is the entire part in miniature.

## Regression Guardrails

Protect critical paths the way you protect correctness:

```
local loop
go test -bench=BenchmarkHotPath -benchmem -count=10 ./internal/hot > new.txt
benchstat old.txt new.txt
```

In CI, store a baseline for a small set of benches and fail on large regressions (with noise tolerance). Flaky performance CI is worse than none — invest in stable inputs and warm-up.

## Interaction with Timeouts

A “performance” incident is sometimes a **timeout budget** incident:

- Dependency p99 is 400ms; you budget 200ms → “Go is slow” tickets
- Retries multiply latency → p99 explodes without CPU hotspots

Always check SLOs, budgets, and dependency health **before** rewriting algorithms.

## Team Norms Worth Adopting

1. Performance claims require numbers (benchstat or load p99).
2. Profiles attached to optimization PRs.
3. Prefer readability until a profile forces otherwise.
4. Document GOMAXPROCS / limits when quoting throughput.
5. Revisit pools and shards annually — complexity expires.

## Mapping Symptoms → Chapter

Symptom	Start here
“Need numbers / workflow”	181 Benchmarking & profiling
High GC CPU, allocs/op	182 Memory & GC
Low CPU, high latency, lock waits	183 Contention
Dependency timeouts	Parts 13 & 15 (budgets, breakers)
Cannot find which service is slow	Part 16 tracing

Use this table in incident channels so people stop randomly rewriting JSON libraries.

## Definition of Done for a Perf Change

A performance PR is done when:

1. Before/after numbers exist (benchstat **or** load p99/RPS).
2. A profile explains *why* the change helped.
3. Correctness tests still pass (go test, race where relevant).
4. Complexity is justified in the description.
5. A rollback plan exists if production disagrees with the lab.

If any item is missing, the change is an experiment, not a finish.

## Quick Command Card

```
microbench + mem
go test -bench=BenchmarkX -benchmem -count=10 ./...

compare
benchstat old.txt new.txt

cpu profile from bench
go test -bench=BenchmarkX -cpuprofile=cpu.out
go tool pprof -http=:8080 cpu.out

live service (admin port)
go tool pprof -http=:8081 http://127.0.0.1:6060/debug/pprof/profile?seconds=20
curl -o trace.out 'http://127.0.0.1:6060/debug/pprof/trace?seconds=5'
go tool trace trace.out
```

Pin this card next to your runbooks.

## Glossary

- **ns/op**, **B/op**, **allocs/op** — benchmark cost units
- **pprof** — sampling profiler suite
- **benchstat** — statistical comparison of bench outputs
- **GOGC / GOMEMLIMIT** — GC pacing and soft memory limit
- **Contention** — waiting on shared resources
- **Flame graph** — visualization of aggregated stacks
- **p99** — 99th percentile latency (tail)
- **Escape analysis** — compiler decision stack vs heap



## More examples

### Tour: timed work + allocation awareness

```
mkdir -p /tmp/go-perf-tour && cd /tmp/go-perf-tour
go mod init example.com/perf-tour
```

Save as `main.go`:

```
package main

import (
 "fmt"
 "time"
)

func sumPrealloc(n int) int {
 s := make([]int, 0, n)
 for i := 0; i < n; i++ {
 s = append(s, i)
 }
 total := 0
 for _, v := range s {
 total += v
 }
 return total
}

func main() {
 start := time.Now()
 got := sumPrealloc(10_000)
 fmt.Println("sum:", got, "dur_ms:", time.Since(start).Milliseconds())
}

go run .
```

### Expected output:

```
sum: 49995000 dur_ms: 0
```

(dur\_ms may be 0–1 on a fast machine.)

## Runnable example

Tour of this part: a microbenchmark-style timing loop comparing two algorithms and a tiny allocation count via `testing.AllocsPerRun` pattern inlined in `main`.

```
mkdir -p /tmp/go-perf-tour && cd /tmp/go-perf-tour
go mod init example.com/perf-tour
```

Save as main.go:

```
package main

import (
 "fmt"
 "strings"
 "testing"
 "time"
)

func concatPlus(n int) string {
 s := ""
 for i := 0; i < n; i++ {
 s += "x"
 }
 return s
}

func concatBuilder(n int) string {
 var b strings.Builder
 b.Grow(n)
 for i := 0; i < n; i++ {
 b.WriteByte('x')
 }
 return b.String()
}

func timeIt(name string, n int, fn func(int) string) {
 start := time.Now()
 _ = fn(n)
 fmt.Printf("%s took %s\n", name, time.Since(start))
}

func main() {
 const n = 20000
 timeIt("plus", n, concatPlus)
 timeIt("builder", n, concatBuilder)

 allocPlus := testing.AllocsPerRun(10, func() { _ = concatPlus(1000) })
 allocBld := testing.AllocsPerRun(10, func() { _ = concatBuilder(1000) })
 fmt.Printf("allocs_per_run plus=%.0f builder=%.0f\n", allocPlus, allocBld)
 fmt.Println("tour: measure before optimize")
}
```

```
go run .
```

**Expected output** (illustrative):

```
plus took ...
builder took ...
allocs_per_run plus=... builder=1
tour: measure before optimize
```

**What to notice**

- Evidence first: builder wins on allocs for repeated concatenation—the part’s core habit.
- Chapters 181–183 deepen pprof, GC, and contention.

**Try next**

- `go test -bench=. -benchmem` on extracted functions.
- Capture a CPU profile once you have a real hotspot.

## Next Chapter

**Benchmarking and Profiling Workflow** — the loop you will reuse forever.

# Benchmarking and Profiling Workflow

# Benchmarking and Profiling Workflow

Optimization should be a controlled loop, not guesswork. Go's toolchain makes the loop tight: benchmarks, pprof, and execution traces.

## Why / Overview

Without measurement you will:

- Optimize cold code
- Add complexity for noise-level gains
- Miss regressions until customers report them

```
baseline bench/load -> profile hotspot -> focused change -> remeasure -> benchstat keep/reve
```

## Writing Good Benchmarks

```
func BenchmarkParse(b *testing.B) {
 in := fixedInput() // stable, realistic
 b.ReportAllocs()
 b.ResetTimer()
 for i := 0; i < b.N; i++ {
 if _, err := Parse(in); err != nil {
 b.Fatal(err)
 }
 }
}
```

Rules:

1. **Realistic inputs** — tiny synthetic data lies.
2. **Avoid measuring setup** — use `ResetTimer`, or do setup outside the loop.
3. **Prevent dead-code elimination** — assign to `sink` or use results.
4. **Report allocs** — `b.ReportAllocs()` or `-benchmem`.
5. **Sub-benchmarks** for sizes: `b.Run("1k", ...)`.

```
var sink int
```

```
func BenchmarkSum(b *testing.B) {
```

```

 for i := 0; i < b.N; i++ {
 sink = Sum(data)
 }
}

```

## Parallel benchmarks

```

func BenchmarkHandlerParallel(b *testing.B) {
 b.RunParallel(func(pb *testing.PB) {
 for pb.Next() {
 // concurrent path
 }
 })
}

```

Useful for mutex/contention issues; pair with CPU counts.

## Running and Comparing

```

go test -bench=BenchmarkParse -benchmem -count=10 ./... | tee old.txt
make change
go test -bench=BenchmarkParse -benchmem -count=10 ./... | tee new.txt
benchstat old.txt new.txt

```

benchstat shows geometric mean differences and statistical confidence. **Do not** trust single runs.

Flags:

- -benchtime=3s or -benchtime=1000x for stability
- -cpu=1,4 for scaling behavior
- -c for concurrent package tests carefully

## Profiling from Benchmarks

```

go test -bench=BenchmarkParse -cpuprofile=cpu.out -memprofile=mem.out
go tool pprof -http=:8080 cpu.out
go tool pprof -http=:8081 mem.out

```

Look at:

- **CPU flame graph** — widest plateaus
- **alloc\_space** — bytes allocated (retained vs ephemeral)
- **alloc\_objects** — allocation count (GC pressure)

## Profiling a Running Service

```
import _ "net/http/pprof"

// admin listener only
go func() {
 log.Println(http.ListenAndServe("127.0.0.1:6060", nil))
}()

go tool pprof -http=:8081 http://127.0.0.1:6060/debug/pprof/profile?seconds=30
curl -o heap.pb.gz http://127.0.0.1:6060/debug/pprof/heap
go tool pprof -http=:8082 heap.pb.gz
```

## Mutex and block profiles

```
runtime.SetMutexProfileFraction(1) // sample all mutex events; lower in prod if needed
runtime.SetBlockProfileRate(1) // 1 ns; too heavy for always-on prod - sample carefully

go tool pprof http://127.0.0.1:6060/debug/pprof/mutex
go tool pprof http://127.0.0.1:6060/debug/pprof/block
```

## Execution Tracer

```
curl -o trace.out 'http://127.0.0.1:6060/debug/pprof/trace?seconds=5'
go tool trace trace.out
```

Use when:

- Latency spikes with low CPU
- Suspect GC STW or scheduler delays
- Goroutines stuck in syscall or channel ops

Trace shows **timeline**, not aggregated hotspots — complementary to pprof.

## Load Testing Complements Microbenchches

Microbenchmarks miss:

- Network
- Real JSON sizes
- Multi-tenant cache effects
- GC under steady heap

Use `hey`, `vegeta`, `k6`, or custom Go load clients for end-to-end. Still attach `pprof` **during** the load.

## Workflow Playbook

1. **Define success**: e.g. `p99 < 50ms` at 2k RPS, or `bench ns/op -20%`.
2. **Baseline** with count 10 and production-like data.
3. **Profile** under that baseline load.
4. **Change one hotspot** (algorithm, alloc, lock scope).
5. **Remeasure** with `benchstat` or load test.
6. **Document** why the change is correct (not only faster).
7. **Guard** with a regression benchmark in CI if critical.

## CI Integration

```
Example: fail on large regression using benchstat + golden
go test -bench=BenchmarkCritical -count=5 ./internal/hotpath | tee ci.txt
compare against stored baseline with thresholds
```

Keep CI benches short and stable; flaky benches train people to ignore them.

## Production Checklist

- ☐ Critical paths have benchmarks with `-benchmem`
- ☐ `pprof` available on admin interface only
- ☐ Know how to capture CPU, heap, mutex, trace
- ☐ Use `benchstat` for claims
- ☐ Load test environment approximates prod limits (CPU, `GOMAXPROCS`)
- ☐ Profiles stored for major incidents (artifact)
- ☐ Optimization PRs include before/after numbers

## Common Pitfalls

1. **Benchmarking with `-race only`** — useful but not speed truth.
2. **Tiny inputs** that stay on stack and never show heap costs.
3. **Compiler optimizing away work**.
4. **Comparing different machines** without noting noise.
5. **Always-on block profiling at rate 1** in prod — heavy.
6. **Optimizing before algorithmic rewrite** ( $O(n^2) \rightarrow \text{map}$ ).
7. **Ignoring variance** — one lucky run.



## Exercises

1. Write a bad benchmark that the compiler optimizes away; fix with sink.
2. Benchmark two JSON libraries on the same payload; benchstat them.
3. Capture CPU profile of a deliberately slow function; identify it in the flame graph.
4. Add `ReportAllocs`; reduce allocs with a `sync.Pool`; remeasure.
5. Run parallel benchmark; show scaling from 1 to 4 CPUs.
6. Enable mutex profile on a contended lock demo; find the holder.
7. Capture a 5s trace during GC-heavy load; find STW-ish gaps.
8. Load-test a HTTP handler; profile mid-test; fix top hotspot.
9. Introduce a 5% performance regression; see if your CI would catch it.
10. Write a one-page playbook for your team's pprof ports and commands.

## Interpreting Flame Graphs Quickly

When the CPU profile UI opens:

1. Sort by **cum** (cumulative) to see time including callees.
2. Look for wide plateaus in your package paths — not only `runtime.*`.
3. If most time is `runtime.mallocgc` / `scanobject`, switch to heap/alloc profiles (next chapter).
4. If most time is `sync.(*Mutex).Lock`, switch to mutex profile (contention chapter).
5. If time is in `syscall.Read` / network, you may have an IO or dependency problem, not a Go micro-opt.

Always keep a copy of the profile file (`cpu.out`) next to the PR description for reviewers.

## Production Capture Safety

- Bind pprof to localhost or a private admin network.
- Prefer short profiles (15–30s) under representative load.
- Do not leave `SetBlockProfileRate(1)` on permanently in large fleets.
- Redact profile upload paths if they include customer identifiers in function args (rare but possible with bad instrumentation).

## More examples

### Mini benchmark harness (stdlib testing style)

```
mkdir -p /tmp/go-bench-mini && cd /tmp/go-bench-mini
go mod init example.com/bench-mini
```

Save as `main.go` (illustrates the measurement pattern; real benches use `testing.B`):

```

package main

import (
 "fmt"
 "strings"
 "time"
)

func concatPlus(n int) string {
 var s string
 for i := 0; i < n; i++ {
 s += "x"
 }
 return s
}

func concatBuilder(n int) string {
 var b strings.Builder
 b.Grow(n)
 for i := 0; i < n; i++ {
 b.WriteByte('x')
 }
 return b.String()
}

func timeIt(name string, n int, fn func(int) string) {
 start := time.Now()
 _ = fn(n)
 fmt.Printf("%s n=%d %s\n", name, n, time.Since(start))
}

func main() {
 timeIt("plus", 5000, concatPlus)
 timeIt("builder", 5000, concatBuilder)
}

go run .

```

**Expected output** (durations vary; builder should not be slower by orders of magnitude):

```

plus n=5000 ...
builder n=5000 ...

```

## pprof HTTP handler registration (httptest)

```
mkdir -p /tmp/go-pprof-reg && cd /tmp/go-pprof-reg
go mod init example.com/pprof-reg
```

Save as `main.go`:

```
package main

import (
 "fmt"
 "net/http"
 "net/http/httptest"
 "net/http/pprof"
)

func main() {
 mux := http.NewServeMux()
 mux.HandleFunc("GET /debug/pprof/", pprof.Index)
 mux.HandleFunc("GET /debug/pprof/cmdline", pprof.Cmdline)
 mux.HandleFunc("GET /debug/pprof/profile", pprof.Profile)
 mux.HandleFunc("GET /debug/pprof/symbol", pprof.Symbol)
 mux.HandleFunc("GET /debug/pprof/trace", pprof.Trace)

 ts := httptest.NewServer(mux)
 defer ts.Close()
 resp, err := http.Get(ts.URL + "/debug/pprof/")
 if err != nil {
 panic(err)
 }
 resp.Body.Close()
 fmt.Println("pprof index:", resp.StatusCode)
}

go run .
```

**Expected output:**

```
pprof index: 200
```

## Runnable example

A real `go test` benchmark file plus `runtime/pprof` heap profile written to a buffer—`stdlib` profiling workflow without a long-lived server.

```
mkdir -p /tmp/go-bench-pprof && cd /tmp/go-bench-pprof
go mod init example.com/bench-pprof
```

Save as work.go:

```
package benchpprof

import "strings"

func BuildReport(n int) string {
 var b strings.Builder
 for i := 0; i < n; i++ {
 b.WriteString("line\n")
 }
 return b.String()
}
```

Save as work\_test.go:

```
package benchpprof

import "testing"

func BenchmarkBuildReport(b *testing.B) {
 for i := 0; i < b.N; i++ {
 _ = BuildReport(1000)
 }
}

func BenchmarkBuildReportAlloc(b *testing.B) {
 b.ReportAllocs()
 for i := 0; i < b.N; i++ {
 _ = BuildReport(1000)
 }
}
```

Save as cmd/heapdemo/main.go:

```
package main

import (
 "bytes"
 "fmt"
 "runtime"
 "runtime/pprof"

 benchpprof "example.com/bench-pprof"
)
```

```

)

func main() {
 // Create some heap traffic
 var hold []string
 for i := 0; i < 100; i++ {
 hold = append(hold, benchpprof.BuildReport(500))
 }
 runtime.GC()

 var buf bytes.Buffer
 if err := pprof.WriteHeapProfile(&buf); err != nil {
 panic(err)
 }
 fmt.Println("heap_profile_bytes:", buf.Len())
 fmt.Println("held_reports:", len(hold))
 fmt.Println("tip: go test -bench=BenchmarkBuildReport -benchmem -count=5")
}

go test -bench=BenchmarkBuildReport -benchmem -count=3 .
go run ./cmd/heapdemo

```

Expected output (illustrative):

```

BenchmarkBuildReport-8 xxxx yyyy ns/op zzzz B/op w allocs/op
heap_profile_bytes: <nonzero>
held_reports: 100
tip: go test -bench=BenchmarkBuildReport -benchmem -count=5

```

### What to notice

- `-benchmem` reports B/op and allocs/op—the first filter before guessing.
- `pprof.WriteHeapProfile` works without binding HTTP; use `net/http/pprof` on an admin port in services.
- Run benchmarks multiple `-count` times before believing a regression.

### Try next

- `go test -cpuprofile=cpu.out -bench=.` then `go tool pprof -http=:8080 cpu.out`.
- Compare two commits with `benchstat old.txt new.txt`.

## Further Reading

- Go blog: profiling Go programs
- `benchstat` documentation
- Next: **Memory, Allocations, and GC**

# Memory, Allocations, and GC

# Memory, Allocations, and GC

Memory behavior is latency behavior. Allocation patterns influence GC pressure, which affects tail latency even when average CPU looks fine.

## Why / Overview

Go's GC is a concurrent, tri-color mark-and-sweep collector designed for low pause times. It is not free: marking work scales with **live heap** and allocation rate.

```
allocate -> object live? -> may promote to longer-lived -> GC mark -> reclaim unreachable
```

Goals of this chapter:

- See allocations in profiles and benches
- Reduce **hot-path** allocs without unreadable code
- Use pooling only with evidence
- Understand GOGC / GOMEMLIMIT knobs

## Stack vs Heap (Practical View)

Escape analysis decides whether a value can live on the stack (cheap) or must be on the heap (GC-managed).

```
go build -gcflags='-m=2' ./...
look for "escapes to heap"
```

Common escape causes:

- Returning pointers to local vars
- Interface conversion of large values
- Closures capturing variables
- Sending pointers on channels that outlive the stack frame

```
// Often heap: returned pointer
func New() *User { u := User{}; return &u }
```

```
// May stay stack if inlined and not escaped further - measure, don't guess
```

## Reading Allocation Metrics

From benchmarks:

```
BenchmarkX-8 123456 9876 ns/op 4096 B/op 12 allocs/op
```

- **B/op** — bytes allocated per op
- **allocs/op** — number of heap allocations

From runtime metrics / expvar / Prometheus Go collector:

- `go_memstats_heap_alloc_bytes`
- `go_memstats_heap_objects`
- GC pause quantiles

From pprof heap:

- `inuse_space` — currently live
- `alloc_space` — cumulative allocated (good for “who allocates a lot”)

## Hot-Path Techniques

### 1. Preallocate slices

```
// BAD: many grow copies
var out []T
for _, x := range in {
 out = append(out, f(x))
}

// GOOD
out := make([]T, 0, len(in))
```

### 2. Reuse buffers

```
buf := make([]byte, 0, 4096)
for _, item := range items {
 buf = buf[:0]
 buf = appendJSON(buf, item)
 _, _ = w.Write(buf)
}
```



### 3. Avoid fmt in hot paths

```
// fmt.Sprintf allocates; strconv and append often cheaper
b = strconv.AppendInt(b, n, 10)
```

### 4. Beware of []byte(string) / string([]byte) conversions

They allocate copies. Sometimes necessary for APIs; don't thrash.

### 5. Interfaces and methods

Storing concrete values in `interface{}` / `any` can allocate. Stream APIs with concrete types when profiling shows it matters.

## sync.Pool

```
var bufPool = sync.Pool{
 New: func() any {
 b := make([]byte, 0, 4096)
 return &b
 },
}

func process() {
 bp := bufPool.Get().(*[]byte)
 buf := (*bp)[:0]
 defer func() {
 *bp = buf
 bufPool.Put(bp)
 }()
 // use buf
}
```

Rules:

- Only for **short-lived**, frequently allocated objects
- Clear sensitive data before put if buffers hold secrets
- Pools can be wiped on GC — do not store important state only in pool
- Prove benefit with benchmarks under concurrent load

## String Builders

```
var b strings.Builder
b.Grow(n)
b.WriteString(s1)
b.WriteString(s2)
out := b.String() // may allocate once
```

Better than repeated `+` in loops.

## Maps and Reuse

Clearing maps:

```
// Go 1.21+: clear(m)
clear(m)

// Or keep map and delete keys if you need reuse without rehash cost carefully
```

Reusing maps across requests can reduce allocs; watch for leftover key retention (memory leaks of keys).

## GC Tuning Knobs

### GOGC

GOGC=100 (default) roughly means GC trigger when heap grows 100% past live set. Higher GOGC → less frequent GC, larger heap. Lower → more GC CPU, smaller heap.

```
GOGC=50 ./app # more GC, less memory
GOGC=200 ./app # less GC, more memory
```

### GOMEMLIMIT (Go 1.19+)

Soft memory limit that makes GC more aggressive as you approach the limit — excellent for containers:

```
GOMEMLIMIT=512MiB ./app
```

Prefer setting memory limits in orchestrators **and** GOMEMLIMIT to reduce OOM kills from GC softness.

## Ballast (historical)

Old trick of large unused allocation to influence GC — largely superseded by GOMEMLIMIT. Avoid new ballast designs.

## Symptoms of Allocation Problems

Symptom	Clue
High CPU in <code>runtime.gc*</code> / <code>scanobject</code>	GC pressure
p99 spikes periodic	GC pacing
Growing heap inuse	Leak or retention
High allocs/op in bench	Hot path churn
Throughput drops with more goroutines	Alloc+sched pressure

## Production Checklist

- ☐ Critical path benches report allocs
- ☐ Heap profiles captured under load
- ☐ Container memory limit + GOMEMLIMIT aligned
- ☐ GOGC only changed with measured reason
- ☐ Pooling justified by numbers
- ☐ No unbounded caches without eviction
- ☐ Escape analysis consulted for hot packages (`-m`)
- ☐ Large buffers not retained accidentally on structs

## Common Pitfalls

1. **Pooling everything** — complexity, bugs, little gain.
2. **Ignoring leaks** (slice backing arrays retained by small reslices).
3. **Global caches without bound.**
4. **String concat in loops.**
5. **defer in micro-benchmarks** affecting tiny functions (sometimes); be aware.
6. **Tuning GOGC to hide a leak.**
7. **Copying huge structs** in interfaces repeatedly.

## Exercises

1. Benchmark + string concat vs `strings.Builder` for 10k parts.
2. Find allocs/op of a JSON marshal path; reduce with streaming encoder to `Writer`.
3. Use `-gcflags=-m` on a hot file; explain three escapes.
4. Introduce a `sync.Pool` for buffers; benchstat with and without under `-cpu=8`.

5. Create a memory leak with growing slice of pointers; find it in heap profile.
6. Run with GOMEMLIMIT low; observe GC CPU vs without.
7. Compare `make([]byte, 0, n)` reuse vs new each time.
8. Profile `fmt.Sprintf("%d", i)` vs `strconv.Itoa`.
9. Clear a map between requests; ensure no key accumulation.
10. Write a short design note: when your team allows `sync.Pool`.

## Leak Patterns Specific to Go Services

1. **Unbounded caches** (map grows forever per user/id).
2. **Goroutine leaks** holding references to large buffers.
3. **time.After in loops** (pre-1.23 patterns) retaining timers — prefer `NewTimer` carefully or tickers.
4. **Registering HTTP handlers / finalizers repeatedly**.
5. **Appending to a slice shared via pointer** without bounds.

Heap profiles over two timestamps (`-base`) make growth obvious.

## More examples

### Preallocate slice capacity

```
mkdir -p /tmp/go-prealloc && cd /tmp/go-prealloc
go mod init example.com/prealloc
```

Save as `main.go`:

```
package main

import (
 "fmt"
 "runtime"
)

func allocs(fn func()) uint64 {
 var m1, m2 runtime.MemStats
 runtime.GC()
 runtime.ReadMemStats(&m1)
 fn()
 runtime.ReadMemStats(&m2)
 return m2.Mallocs - m1.Mallocs
}

func main() {
```

```

n := 1000
a := allocs(func() {
 var s []int
 for i := 0; i < n; i++ {
 s = append(s, i)
 }
 _ = s
})
b := allocs(func() {
 s := make([]int, 0, n)
 for i := 0; i < n; i++ {
 s = append(s, i)
 }
 _ = s
})
fmt.Println("grow allocs >= prealloc:", a >= b)
fmt.Println("prealloc mallocs:", b)
}

```

```
go run .
```

### Expected output:

```

grow allocs >= prealloc: true
prealloc mallocs: <small number>

```

### sync.Pool reuse

```

mkdir -p /tmp/go-pool && cd /tmp/go-pool
go mod init example.com/pool

```

Save as main.go:

```

package main

import (
 "bytes"
 "fmt"
 "sync"
)

func main() {
 var pool = sync.Pool{New: func() any { return new(bytes.Buffer) }}
 buf := pool.Get().(*bytes.Buffer)
 buf.Reset()
}

```

```

 buf.WriteString("hello")
 fmt.Println(buf.String())
 pool.Put(buf)

 buf2 := pool.Get().(*bytes.Buffer)
 // May be the same buffer; Reset before use in real code.
 buf2.Reset()
 buf2.WriteString("world")
 fmt.Println(buf2.String())
}

go run .

```

**Expected output:**

```

hello
world

```

## Runnable example

Compare allocations with `testing.AllocsPerRun`, reuse a buffer with `sync.Pool`, and print basic GC stats via `runtime.ReadMemStats`.

```

mkdir -p /tmp/go-alloc-gc && cd /tmp/go-alloc-gc
go mod init example.com/alloc-gc

```

Save as `main.go`:

```

package main

import (
 "fmt"
 "runtime"
 "sync"
 "testing"
)

var bufPool = sync.Pool{
 New: func() any {
 b := make([]byte, 0, 4096)
 return &b
 },
}

func workNew(n int) int {

```

```

 b := make([]byte, 0, n)
 for i := 0; i < n; i++ {
 b = append(b, 'x')
 }
 return len(b)
}

func workPool(n int) int {
 bp := bufPool.Get().(*[]byte)
 b := (*bp)[:0]
 for i := 0; i < n; i++ {
 b = append(b, 'x')
 }
 // keep cap; return to pool
 *bp = b
 bufPool.Put(bp)
 return len(b)
}

func main() {
 aNew := testing.AllocsPerRun(100, func() { _ = workNew(2048) })
 aPool := testing.AllocsPerRun(100, func() { _ = workPool(2048) })
 fmt.Printf("allocs workNew=%.2f workPool=%.2f\n", aNew, aPool)

 var before, after runtime.MemStats
 runtime.ReadMemStats(&before)
 sink := 0
 for i := 0; i < 10000; i++ {
 sink += workNew(1024)
 }
 runtime.GC()
 runtime.ReadMemStats(&after)
 fmt.Printf("heap_alloc_after=%d num_gc=%d total_alloc_delta=%d sink=%d\n",
 after.HeapAlloc, after.NumGC, after.TotalAlloc-before.TotalAlloc, sink)
 fmt.Println("note: Pool helps under concurrent reuse; measure in real load")
}

go run .

```

**Expected output** (illustrative):

```

allocs workNew=1.00 workPool=0.00
heap_alloc_after=... num_gc=... total_alloc_delta=...
note: Pool helps under concurrent reuse; measure in real load

```

**What to notice**

- Allocation rate drives GC CPU; reducing allocs/op often beats micro-tweaking algorithms later.
- `sync.Pool` is for *hot* reusable buffers—not a general cache; values may be dropped anytime.
- `ReadMemStats` is expensive; use sparingly or prefer runtime/metrics in production.

#### Try next

- `go build -gcflags=-m` on a file and read escape analysis notes.
- Set `GOMEMLIMIT` and observe GC behavior under a memory-hungry loop.

## Further Reading

- Go GC guide and runtime metrics
- Previous: profiling workflow to *find* alloc sites
- Next: **Contention and Concurrency Tuning**



# Contention and Concurrency Tuning

# Contention and Concurrency Tuning

Throughput often degrades due to **contention** long before CPU saturates. Adding goroutines can make the system slower when they fight over locks, shared maps, or a single connection.

## Why / Overview

```
goroutines -> shared resource -> queueing -> latency growth -> throughput collapse
```

Classic symptoms:

- Low CPU, high latency
- Throughput flat while goroutine count climbs
- Mutex profile shows long waiters
- block profile hot on channels or locks

## Sources of Contention

Resource	Example
<code>sync.Mutex</code> / <code>RWMutex</code>	Shared cache, metrics map
Channels	Single consumer bottleneck
Shared <code>map</code> without sharding	Global session table
Single DB connection / conn pool too small	Query pileup
<code>log</code> writer lock	Sync logging every request
<code>time.Now</code> + heavy formatting under lock	Lock held too long

## Diagnostic Workflow

1. Confirm problem with **load test**: RPS vs latency vs CPU.
2. If CPU low and latency high → contention or IO wait.
3. Capture **mutex** and **block** profiles under load.
4. Capture **trace** for scheduler wait.
5. Fix by reducing critical sections, sharding, or buffering.
6. Remeasure.

```

import _ "net/http/pprof"
import "runtime"

func init() {
 runtime.SetMutexProfileFraction(5) // 1/5 of events; tune
 // runtime.SetBlockProfileRate(1000) // sample; careful in prod
}

```

## Critical Section Hygiene

```

// BAD: slow work under lock
mu.Lock()
data := fetchFromDB() // network under lock!
cache[key] = data
mu.Unlock()

// GOOD: compute outside
data := fetchFromDB()
mu.Lock()
cache[key] = data
mu.Unlock()

```

Copy what you need under lock; work on the copy outside.

## RWMutex misuse

RWMutex helps when reads dominate **and** critical sections are short. If writers are frequent, RWMutex can be worse. Measure.

## Sharding

```

const shards = 16

type sharded struct {
 ms [shards]struct {
 mu sync.Mutex
 m map[string]int
 }
}

func (s *sharded) inc(key string) {
 h := fnv32(key) % shards

```

```

 sh := &s.ms[h]
 sh.mu.Lock()
 sh.m[key]++
 sh.mu.Unlock()
}

```

Sharding reduces collision probability; pick shard count relative to `GOMAXPROCS` and key distribution.

## Atomically Updated Stats

```

var hits atomic.Int64

func hit() { hits.Add(1) }

```

Prefer atomics for simple counters over mutexes. For complex metrics, use Prometheus client (already concurrent) rather than home-grown locked maps with high cardinality.

## Channel Contension and Pipelines

```

// Single channel to one worker: max throughput ~ worker speed
jobs := make(chan Job, 1024)

// Multiple workers consuming one channel: good
for i := 0; i < n; i++ {
 go worker(jobs)
}

```

Pitfalls:

- Unbuffered channels serialize producers
- Huge buffers hide deadlocks and delay backpressure
- Multiple stages with mismatched rates need intentional buffering

## Bound Fan-Out

```

g, ctx := errgroup.WithContext(ctx)
g.SetLimit(32)
for _, u := range urls {
 u := u
 g.Go(func() error {
 return fetch(ctx, u)
 })
}

```

```

 })
}
return g.Wait()

```

Unbounded `go func` per item under a hot HTTP handler is a latency and memory bomb.

## Connection Pools

Database and HTTP pools are concurrency knobs:

```

db.SetMaxOpenConns(25)
db.SetMaxIdleConns(25)
db.SetConnMaxLifetime(time.Hour)

transport.MaxConnsPerHost = 32

```

Too few → wait on pool. Too many → overload dependency. Tune against dependency capacity and latency.

## GOMAXPROCS and Containers

Go 1.5+ defaults `GOMAXPROCS` to CPU count. In cgroups, recent Go versions respect container CPU limits more carefully — still verify:

```

inside container
go env GOMAXPROCS
or log runtime.GOMAXPROCS(0) at start

```

Mismatch (thinks 64 CPUs, has 2) → oversubscription, latency noise.

## False Sharing and Cache Lines (Advanced)

Rare in app Go, more in tight numeric loops with adjacent atomics. If profiling shows multi-core scaling collapse on atomics, pad/shard counters. Don't start here.

## Work Stealing and Fairness

The Go scheduler multiplexes goroutines on `GOMAXPROCS` threads. CPU-bound tight loops without function calls can delay preemption (improved in recent Go). For fairness, insert `runtime.Gosched()` only when profiling proves need — rare.

## Production Checklist

- ☐ Load test shows CPU vs latency relationship
- ☐ Mutex/block profiles known how-to
- ☐ No IO under locks on hot paths
- ☐ Fan-out bounded (errgroup limits / semaphores)
- ☐ Pools sized for deps
- ☐ Shared maps protected **and** not global bottlenecks
- ☐ Logging async or sampled under load if contended
- ☐ GOMAXPROCS aligned with container CPUs
- ☐ After changes, remeasure p99 and throughput

## Common Pitfalls

1. **More goroutines = more speed** belief.
2. **Global mutex** around entire request handling.
3. **Map + mutex** as the only cache for everything.
4. **Holding lock across `http.Client.Do`.**
5. **Unbuffered event bus** used for high QPS metrics.
6. **Ignoring dependency pool wait** — looks like “Go is slow.”
7. **Fixing contention by removing locks** without other synchronization → data races (-race).

## Exercises

1. Build a counter with mutex vs atomic; parallel bench both.
2. Create a hot map with one mutex; shard it; benchstat.
3. Hold a lock during `time.Sleep` to simulate IO; show latency; move sleep out.
4. Capture mutex profile; identify the stack.
5. Unbounded fan-out HTTP handler vs `SetLimit`; compare memory and p99.
6. Shrink `MaxOpenConns` to 1; observe pool wait in traces/profiles.
7. Use `go test -race` on a deliberately racy counter; fix.
8. Measure throughput vs worker count curve; find the knee.
9. Replace contended `log.Printf` with async slog or buffered writer; measure.
10. Document a contention incident postmortem template (signals → profile → fix → result).

## Throughput Curve Lab (Do This Once)

Pick a handler that does shared-map updates under a mutex:

```
workers: 1, 2, 4, 8, 16, 32
measure: RPS and p99
plot: RPS rises then flattens/falls; p99 rises with contention
```

Then shard the map or switch counters to atomics and replot. The shape of that curve is the entire point of this chapter: **more concurrency is not free**.

## Combining with Timeouts and Breakers

Contention often surfaces as timeouts:

- Lock wait → handler exceeds budget → client retries → more lock wait

Fixing only retries makes it worse. Profile locks under the load that triggers pages, then reduce critical sections or capacity-limit admission (semaphore) so the system degrades with 503 instead of unlimited queueing on a mutex.

## More examples

### Hot mutex vs sharded counters

```
mkdir -p /tmp/go-shard-count && cd /tmp/go-shard-count
go mod init example.com/shard-count
```

Save as main.go:

```
package main

import (
 "fmt"
 "sync"
 "sync/atomic"
 "time"
)

func hotMutex(n, workers int) time.Duration {
 var mu sync.Mutex
 var c int
 var wg sync.WaitGroup
 start := time.Now()
 for w := 0; w < workers; w++ {
 wg.Add(1)
 go func() {
 defer wg.Done()
 for i := 0; i < n; i++ {
 mu.Lock()
 c++
 mu.Unlock()
 }
 }
 }
}
```

```

 }()
 }
 wg.Wait()
 _ = c
 return time.Since(start)
}

func atomicCount(n, workers int) time.Duration {
 var c atomic.Int64
 var wg sync.WaitGroup
 start := time.Now()
 for w := 0; w < workers; w++ {
 wg.Add(1)
 go func() {
 defer wg.Done()
 for i := 0; i < n; i++ {
 c.Add(1)
 }
 }()
 }
 wg.Wait()
 return time.Since(start)
}

func main() {
 const n, w = 20_000, 4
 fmt.Println("mutex", hotMutex(n, w))
 fmt.Println("atomic", atomicCount(n, w))
}

go run .

```

**Expected output** (durations vary; both complete):

```

mutex ...
atomic ...

```

### Admission semaphore (shed load)

```

mkdir -p /tmp/go-sem-admit && cd /tmp/go-sem-admit
go mod init example.com/sem-admit

```

Save as `main.go`:



```

package main

import (
 "fmt"
 "net/http"
 "net/http/httptest"
)

func withLimit(n int, next http.Handler) http.Handler {
 sem := make(chan struct{}, n)
 return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
 select {
 case sem <- struct{}{}:
 defer func() { <-sem }()
 next.ServeHTTP(w, r)
 default:
 http.Error(w, "busy", http.StatusServiceUnavailable)
 }
 })
}

func main() {
 entered := make(chan struct{})
 block := make(chan struct{})
 h := withLimit(1, http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
 close(entered) // signal that the slot is held
 <-block
 fmt.Fprintln(w, "ok")
 }))
 ts := httptest.NewServer(h)
 defer ts.Close()

 go func() {
 resp, _ := http.Get(ts.URL)
 resp.Body.Close()
 }()
 <-entered // wait until first request holds the semaphore
 resp, _ := http.Get(ts.URL)
 fmt.Println("shed:", resp.StatusCode)
 resp.Body.Close()
 close(block)
}

go run .

```

**Expected output:**

shed: 503

## Runnable example

Contended mutex counter vs sharded counters under many goroutines—measure elapsed time to see the throughput ceiling.

```
mkdir -p /tmp/go-contention && cd /tmp/go-contention
go mod init example.com/contention
```

Save as main.go:

```
package main

import (
 "fmt"
 "sync"
 "sync/atomic"
 "time"
)

func benchMutex(goroutines, iters int) time.Duration {
 var mu sync.Mutex
 var total int64
 var wg sync.WaitGroup
 start := time.Now()
 wg.Add(goroutines)
 for g := 0; g < goroutines; g++ {
 go func() {
 defer wg.Done()
 for i := 0; i < iters; i++ {
 mu.Lock()
 total++
 mu.Unlock()
 }
 }()
 }
 wg.Wait()
 _ = total
 return time.Since(start)
}

func benchAtomic(goroutines, iters int) time.Duration {
 var total atomic.Int64
 var wg sync.WaitGroup
 start := time.Now()
```

```

 wg.Add(goroutines)
 for g := 0; g < goroutines; g++ {
 go func() {
 defer wg.Done()
 for i := 0; i < iters; i++ {
 total.Add(1)
 }
 }()
 }
 wg.Wait()
 _ = total.Load()
 return time.Since(start)
}

func benchSharded(goroutines, iters, shards int) time.Duration {
 type pad struct {
 mu sync.Mutex
 n int64
 _ [56]byte // reduce false sharing a bit
 }
 cs := make([]pad, shards)
 var wg sync.WaitGroup
 start := time.Now()
 wg.Add(goroutines)
 for g := 0; g < goroutines; g++ {
 g := g
 go func() {
 defer wg.Done()
 s := &cs[g%shards]
 for i := 0; i < iters; i++ {
 s.mu.Lock()
 s.n++
 s.mu.Unlock()
 }
 }()
 }
 wg.Wait()
 return time.Since(start)
}

func main() {
 const g, iters = 8, 100000
 fmt.Println("mutex: ", benchMutex(g, iters))
 fmt.Println("atomic: ", benchAtomic(g, iters))
 fmt.Println("shard8: ", benchSharded(g, iters, 8))
 fmt.Println("more concurrency is not free - profile locks under real load")
}

```

```
go run .
```

**Expected output** (order of magnitude varies by machine):

```
mutex: ...
atomic: ...
shard8: ...
more concurrency is not free - profile locks under real load
```

### What to notice

- A single global mutex serializes work; atomics/sharding raise the ceiling for simple counters.
- Real contention shows up as flat RPS and rising p99—capture a mutex profile in services.
- Fix critical sections and admission control before cranking retries.

### Try next

- `go test -bench=. -cpu=1,2,4,8` and plot ns/op.
- Enable mutex profiling: `runtime.SetMutexProfileFraction(5)` and pull `pprof/mutex`.

## Further Reading

- Go blog: mutex profile, scheduler
- Previous: allocations/GC (often co-occur with contention under load)
- Concurrency part of this book for patterns (worker pools, pipelines)
- Synthesis parts for consolidating modern Go practices

## **Load Testing and Baselines**

# Load Testing and Baselines

## Overview

Microbenchmarks don't replace **load tests**. Establish a baseline: throughput, p50/p99 latency, error rate, resource use—then change one variable.

## Tools

Tool	Role
hey / vegeta / k6	HTTP load
go test -bench	Function-level
pprof under load	See real hotspots

```
hey -z 30s -c 50 http://127.0.0.1:8080/api/books
```

## Baseline template

```
Scenario: GET /api/books (cached / uncached)
Concurrency: 50
Duration: 60s
Result: QPS=... p50=... p99=... errors=...
CPU=... RSS=... (from metrics or `ps`)
Commit: abc123
```

## SLO-oriented load

Drive load until **error budget** or latency SLO breaks—know the capacity ceiling.

## Avoiding test lies

- Warm up before measuring
- Separate client machine when possible
- Watch coordinated omission (load tools that delay sending)
- Fix payloads and auth

## Rules of thumb

Do	Don't
Record env + commit with results	Compare laptop vs prod blindly
Increase load gradually	Jump to 10k connections first
Pair with profiles	Optimize without a number

## Try next

1. Baseline one endpoint; save numbers in a markdown table.
2. Add artificial `time.Sleep` and remeasure.
3. Capture CPU pprof during `hey`.

## **Allocation Hot-Path Recipes**



# Allocation Hot-Path Recipes

## Overview

High allocation rate drives GC and cache pressure. After profiles point to a site, apply **small, proven** recipes—not premature `sync.Pool` everywhere.

## Measure first

```
go test -bench=BenchmarkX -benchmem
or pprof heap under load
```

## Recipes

### 1. Avoid `fmt` in hot loops

```
// slow: fmt.Sprintf("%d", n)
strconv.Itoa(n)
strconv.AppendInt(buf[:0], int64(n), 10)
```

### 2. Grow slices with known cap

```
out := make([]T, 0, len(in))
```

### 3. Reuse buffers

```
var buf bytes.Buffer
buf.Reset()
// or sync.Pool of []byte for short-lived buffers
```

## 4. strings.Builder

```
var b strings.Builder
b.Grow(n)
b.WriteString(...)
s := b.String()
```

## 5. Interface boxing

Storing concrete values in `any`/interfaces allocates. Keep hot maps typed.

## 6. JSON

- `json.Encoder` to writer vs repeated `Marshal`
- Consider specialized codecs only when bench proves need
- Avoid `map[string]any` for hot types

## 7. io.Copy buffer

```
buf := make([]byte, 32*1024)
io.CopyBuffer(dst, src, buf)
```

## Pool discipline

```
var pool = sync.Pool{New: func() any { return make([]byte, 0, 1024) }}

b := pool.Get().([]byte)
b = b[:0]
// use b
pool.Put(b)
```

Don't put pointers to data still in use; don't assume zeroed memory.

## Rules of thumb

Do	Don't
Fix algorithm first	Pool everything
Bench before/after	Optimize cold paths

Do	Don't
Keep code readable	Unreadable zero-alloc for 0.1%

## Try next

1. `-benchmem` a JSON encode path; reduce allocs/op.
2. Replace `fmt.Sprintf` in a hot path.
3. Profile heap; fix top1 only.

## **PGO and Production Build Flags**

# PGO and Production Build Flags

## Overview

**Profile-guided optimization (PGO)** feeds CPU profiles into the compiler for better inlining/layout on hot paths. Combine with production build flags for smaller, clearer binaries.

## Production build baseline

```
CGO_ENABLED=0 go build -trimpath \
 -ldflags="-s -w -X main.version=${VERSION}" \
 -o bin/app ./cmd/app
```

Flag	Effect
CGO_ENABLED=0	Static-ish pure Go, portable
-trimpath	Reproducible paths
-s -w	Strip symbol/DWARF (harder debug)
-X	Inject version

Keep a debug build variant for crash analysis when needed.

## PGO workflow (Go 1.20+)

```
1) collect profile from realistic load (30s+)
curl -o cpu.pprof "http://localhost:6060/debug/pprof/cpu?seconds=30"

2) rebuild with profile
go build -pgo=cpu.pprof -o bin/app ./cmd/app

3) compare benchmarks / load test
```

Or default auto PGO if `default.pgo` is in the main package directory (per Go release docs).

## When PGO helps

Helps	Less impact
Large codebases, hot call chains	Tiny programs already inlined
After stable production shape	Random microbench only

## Rules of thumb

Do	Don't
Profile production-like traffic	Use toy profiles for PGO
A/B load test before/after	Assume free wins always
Document Go version + flags	Mystery binaries in prod

## Try next

1. Capture 30s CPU pprof under `hey`.
2. Build with `-pgo` and remeasure p99.
3. Add version ldflags; verify `app version`.

## **Project: Optimize a Hot Endpoint**

# Project: Optimize a Hot Endpoint

## Overview

Pick or build a deliberately inefficient endpoint, then improve it with the performance workflow.

## Setup

Serve `GET /report` that:

1. Re-reads a JSON file every request
2. Uses `fmt.Sprintf` heavily
3. Builds strings with `+` in a loop

## Steps

1. **Baseline** load test numbers (184)
2. **Profile** CPU + heap
3. **Change one thing** (cache file, Builder, precompute)
4. **Remeasure**
5. Write a short report: before/after table

## Acceptance

- ☐ 30% p99 or allocs improvement with evidence
- ☐ Tests still pass
- ☐ No correctness regression
- ☐ Profile screenshots or pprof commands recorded



## Stretch

Apply PGO and note delta vs manual fixes alone.

## **Modern Go Synthesis**

## **190 Modern Go Books (2024-2025) Index**

# 190 Modern Go Books (2024-2025) Index

This section distills practical curriculum additions from recent Go books (published in 2024 and 2025), then maps those ideas into this book's structure.

The intent is not to duplicate any book. Instead, this section extracts high-value topic patterns that repeatedly appear across recent publications and turns them into production-oriented learning content.

## Selection Criteria

- Publication date within 2024-2025.
- Reliable public metadata for release date and chapter/index signals.
- Topics that improve production readiness for Go developers.

## Source-Derived Synthesis Chapters

1. [191 Patterns from Go Programming \(2nd ed., 2024\)](#)
2. [192 Patterns from Effective Go Recipes \(2024\)](#)
3. [193 Patterns from Go in Practice \(2nd ed., 2025\)](#)
4. [194 Patterns from Mastering Go \(4th ed., 2024\)](#)
5. [195 Patterns from Let Us Go! \(2025\)](#)
6. [196 Patterns from Automate Your Home Using Go \(2024\)](#)

## Cross-Book Theme Map

```
Language fundamentals -> Systems/networking -> Web APIs -> Concurrency -> Testing/Tooling ->
```

Recent books emphasize this transition strongly: Go learning now expects developers to move from syntax to production operations quickly.

## More examples

### Tour: options + http test handler + typed error

A single demo that previews synthesis themes: constructor options, boundary HTTP, and errors.Is.

```
mkdir -p /tmp/go-synth-tour && cd /tmp/go-synth-tour
go mod init example.com/synth-tour
```

Save as main.go:

```
package main

import (
 "errors"
 "fmt"
 "net/http"
 "net/http/httptest"
)

var ErrNotFound = errors.New("not found")

type App struct {
 name string
}

type Option func(*App)

func WithName(n string) Option { return func(a *App) { a.name = n } }

func NewApp(opts ...Option) *App {
 a := &App{name: "app"}
 for _, o := range opts {
 o(a)
 }
 return a
}

func (a *App) Get(id int) error {
 if id != 1 {
 return fmt.Errorf("%w: %d", ErrNotFound, id)
 }
 return nil
}

func main() {
```

```

app := NewApp(WithName("demo"))
h := http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
 if err := app.Get(2); errors.Is(err, ErrNotFound) {
 http.Error(w, "missing", http.StatusNotFound)
 return
 }
 fmt.Fprintln(w, app.name)
})
rr := httptest.NewRecorder()
h.ServeHTTP(rr, httptest.NewRequest(http.MethodGet, "/", nil))
fmt.Println("status:", rr.Code, "name:", app.name)
}

go run .

```

### Expected output:

```
status: 404 name: demo
```

## Runnable example

Tour of the synthesis themes: config from env, a timeout budget, and a tiny HTTP handler—stdlib patterns that modern Go books keep circling.

```

mkdir -p /tmp/go-synth-tour && cd /tmp/go-synth-tour
go mod init example.com/synth-tour

```

Save as main.go:

```

package main

import (
 "context"
 "fmt"
 "net/http"
 "net/http/httptest"
 "os"
 "time"
)

func envDuration(key string, fallback time.Duration) time.Duration {
 v := os.Getenv(key)
 if v == "" {
 return fallback
 }
}

```

```

 d, err := time.ParseDuration(v)
 if err != nil {
 return fallback
 }
 return d
}

func main() {
 budget := envDuration("REQ_TIMEOUT", 200*time.Millisecond)
 h := http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
 ctx, cancel := context.WithTimeout(r.Context(), budget)
 defer cancel()
 select {
 case <-time.After(20 * time.Millisecond):
 fmt.Fprintln(w, "ok")
 case <-ctx.Done():
 http.Error(w, "timeout", http.StatusGatewayTimeout)
 }
 })
 rr := httptest.NewRecorder()
 h.ServeHTTP(rr, httptest.NewRequest(http.MethodGet, "/", nil))
 fmt.Println("status:", rr.Code, "budget:", budget)
 fmt.Println("synthesis tour: config + budgets + http")
}

go run .
REQ_TIMEOUT=50ms go run .

```

### Expected output:

```

status: 200 budget: 200ms
synthesis tour: config + budgets + http

```

### What to notice

- Recent books stress production boundaries (timeouts, config, tests) as much as syntax.
- Chapters 191–196 extract one pattern cluster each—run those examples next.

### Try next

- Skim each synthesis chapter’s runnable example and map it back to an earlier part of this book.

## **191 Patterns from Go Programming (2nd ed., 2024)**



# 191 Patterns from Go Programming (2nd ed., 2024)

This chapter extracts topics from the 2024 second edition arc: debugging, time handling, CLI work, files/systems, SQL, HTTP server/client, concurrency, testing, tooling, and cloud.

## What This Adds to Our Book

- Stronger bridge from core syntax to practical operations.
- Dedicated narrative on “time as a correctness boundary”.
- Better pairing of HTTP server and HTTP client behavior in one lifecycle.
- Cloud-readiness framing: configuration, graceful shutdown, observability, packaging.

## Learning Architecture

```
core language -> application boundary (CLI/API) -> state boundary (DB/files) -> operational
```

## Deep Integration Example: API + CLI + DB Boundary

```
package app

import (
 "context"
 "database/sql"
 "encoding/json"
 "errors"
 "fmt"
 "net/http"
 "os"
 "time"
)

var ErrNotFound = errors.New("not found")

type User struct {
```

```

 ID int64 `json:"id"`
 Email string `json:"email"`
}

type Store struct{ DB *sql.DB }

func (s Store) GetUser(ctx context.Context, id int64) (User, error) {
 ctx, cancel := context.WithTimeout(ctx, 2*time.Second)
 defer cancel()

 var u User
 err := s.DB.QueryRowContext(ctx, `SELECT id, email FROM users WHERE id=$1`, id).Scan(&u)
 if errors.Is(err, sql.ErrNoRows) {
 return User{}, ErrNotFound
 }
 if err != nil {
 return User{}, fmt.Errorf("query user %d: %w", id, err)
 }
 return u, nil
}

type API struct{ Store Store }

func (a API) GetUserHandler(w http.ResponseWriter, r *http.Request) {
 ctx, cancel := context.WithTimeout(r.Context(), 3*time.Second)
 defer cancel()

 user, err := a.Store.GetUser(ctx, 42)
 if errors.Is(err, ErrNotFound) {
 http.Error(w, "not found", http.StatusNotFound)
 return
 }
 if err != nil {
 http.Error(w, "internal error", http.StatusInternalServerError)
 return
 }
 w.Header().Set("Content-Type", "application/json")
 _ = json.NewEncoder(w).Encode(user)
}

func ReadEnvDuration(key string, fallback time.Duration) time.Duration {
 v := os.Getenv(key)
 if v == "" {
 return fallback
 }
 d, err := time.ParseDuration(v)
 if err != nil {

```

```

 return fallback
 }
 return d
}

```

## Why This Matters

The example demonstrates a full chain from request boundary to storage boundary with explicit timeouts and typed errors. That is the practical center of modern Go backend development.

## Curriculum Upgrades Recommended

1. Add a dedicated chapter linking debugging, benchmarking, and production incident triage.
2. Expand `time` chapter with deadline budgeting patterns for server+client+DB together.
3. Add side-by-side chapter pair: `net/http` server correctness and `http.Client` resilience.
4. Add cloud deployment chapter emphasizing runtime config contracts and shutdown policy.

## More examples

### Nested context budgets at the boundary

```

mkdir -p /tmp/go-gp2e-budget && cd /tmp/go-gp2e-budget
go mod init example.com/gp2e-budget

```

Save as `main.go`:

```

package main

import (
 "context"
 "fmt"
 "time"
)

func db(ctx context.Context) error {
 ctx, cancel := context.WithTimeout(ctx, 20*time.Millisecond)
 defer cancel()
 select {
 case <-time.After(50 * time.Millisecond):
 return nil
 case <-ctx.Done():
 return ctx.Err()
 }
}

```

```

 }
}

func main() {
 ctx, cancel := context.WithTimeout(context.Background(), 100*time.Millisecond)
 defer cancel()
 err := db(ctx)
 fmt.Println("db budget exceeded:", err != nil)
}

go run .

```

Expected output:

```
db budget exceeded: true
```

## Typed sentinel + wrap for handlers

```

mkdir -p /tmp/go-gp2e-err && cd /tmp/go-gp2e-err
go mod init example.com/gp2e-err

```

Save as main.go:

```

package main

import (
 "errors"
 "fmt"
)

var ErrDenied = errors.New("denied")

func authorize(user string) error {
 if user != "admin" {
 return fmt.Errorf("user %s: %w", user, ErrDenied)
 }
 return nil
}

func main() {
 err := authorize("bob")
 fmt.Println("is denied:", errors.Is(err, ErrDenied))
 fmt.Println("text:", err)
}

```

```
go run .
```

### Expected output:

```
is denied: true
text: user bob: denied
```

## Runnable example

Boundary chain without a real DB: typed errors, nested timeouts, and JSON HTTP handler test via `httptest`.

```
mkdir -p /tmp/go-gp2e && cd /tmp/go-gp2e
go mod init example.com/gp2e
```

Save as `main.go`:

```
package main

import (
 "context"
 "encoding/json"
 "errors"
 "fmt"
 "net/http"
 "net/http/httptest"
 "time"
)

var ErrNotFound = errors.New("not found")

type User struct {
 ID int64 `json:"id"`
 Email string `json:"email"`
}

type Store struct {
 users map[int64]User
}

func (s Store) GetUser(ctx context.Context, id int64) (User, error) {
 ctx, cancel := context.WithTimeout(ctx, 50*time.Millisecond)
 defer cancel()
 select {
 case <-ctx.Done():
 return User{}, ctx.Err()
 }
```

```

 default:
 }
 u, ok := s.users[id]
 if !ok {
 return User{}, ErrNotFound
 }
 return u, nil
}

func main() {
 store := Store{users: map[int64]User{42: {ID: 42, Email: "a@b.co"}}}
 h := http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
 ctx, cancel := context.WithTimeout(r.Context(), 100*time.Millisecond)
 defer cancel()
 u, err := store.GetUser(ctx, 42)
 if errors.Is(err, ErrNotFound) {
 http.Error(w, "not found", http.StatusNotFound)
 return
 }
 if err != nil {
 http.Error(w, "internal", http.StatusInternalServerError)
 return
 }
 w.Header().Set("Content-Type", "application/json")
 _ = json.NewEncoder(w).Encode(u)
 })
 rr := httptest.NewRecorder()
 h.ServeHTTP(rr, httptest.NewRequest(http.MethodGet, "/user", nil))
 fmt.Println("status:", rr.Code, "body:", rr.Body.String())
}

go run .

```

### Expected output:

```
status: 200 body: {"id":42,"email":"a@b.co"}
```

### What to notice

- Timeouts nest: handler budget > store budget.
- `errors.Is` maps domain errors to HTTP status without string matching.

### Try next

- Return `ErrNotFound` for id 7 and assert 404.
- Parse `REQ_TIMEOUT` from env for the outer budget.

## **192 Patterns from Effective Go Recipes (2024)**

# 192 Patterns from Effective Go Recipes (2024)

This source emphasizes practical recipes across I/O, JSON streaming, HTTP middleware/timeouts, text normalization, function options, errors, concurrency, sockets, C interop, testing, and shipping.

## What This Adds to Our Book

- Recipe-level production patterns that are small, composable, and immediately reusable.
- Better treatment of streaming and incremental processing.
- Stronger shipping pipeline topics: build tags, static builds, version injection, Docker packaging.

## Streaming-Oriented Mental Model

```
source -> decoder -> transform -> validator -> sink
 (stream) (bounded memory)
```

## Deep Integration Example: Streaming JSON Pipeline with Backpressure

```
package pipeline

import (
 "bufio"
 "context"
 "encoding/json"
 "errors"
 "io"
 "sync"
 "time"
)

type Event struct {
 ID string `json:"id"`
 Type string `json:"type"`
}
```



```

}

func DecodeEvents(ctx context.Context, r io.Reader) (<-chan Event, <-chan error) {
 out := make(chan Event, 128)
 errCh := make(chan error, 1)

 go func() {
 defer close(out)
 defer close(errCh)

 dec := json.NewDecoder(bufio.NewReader(r))
 for {
 var e Event
 if err := dec.Decode(&e); err != nil {
 if errors.Is(err, io.EOF) {
 return
 }
 errCh <- err
 return
 }

 select {
 case <-ctx.Done():
 errCh <- ctx.Err()
 return
 case out <- e:
 }
 }
 }()

 return out, errCh
}

func ProcessEvents(ctx context.Context, in <-chan Event, workers int, fn func(Event) error) {
 ctx, cancel := context.WithCancel(ctx)
 defer cancel()

 errCh := make(chan error, workers)
 var wg sync.WaitGroup

 for i := 0; i < workers; i++ {
 wg.Add(1)
 go func() {
 defer wg.Done()
 for e := range in {
 opCtx, stop := context.WithTimeout(ctx, 250*time.Millisecond)
 err := fn(e)
 }
 }()
 }
 wg.Wait()
 for err := range errCh {
 // ...
 }
}

```

```

 stop()
 if opCtx.Err() != nil {
 errCh <- opCtx.Err()
 cancel()
 return
 }
 if err != nil {
 errCh <- err
 cancel()
 return
 }
 }
}()
}

wg.Wait()
close(errCh)
for err := range errCh {
 if err != nil {
 return err
 }
}
return nil
}

```

## Why This Matters

Modern Go systems increasingly process streams instead of loading full datasets in memory. Recipe-style patterns are especially effective for this style because they focus on one robust tactic at a time.

## Curriculum Upgrades Recommended

1. Add a chapter on streaming-first design (`json.Decoder`, `io.Reader`, chunked processing).
2. Add chapter on function options + config normalization.
3. Add chapter on shipping pipelines: reproducible builds, tags, embedding, static binaries.
4. Add explicit `os/exec` and signal-handling narrative for operational tooling.

## More examples

### Functional options recipe

```
mkdir -p /tmp/go-egr-opts && cd /tmp/go-egr-opts
go mod init example.com/egr-opts
```

Save as main.go:

```
package main

import (
 "fmt"
 "time"
)

type Client struct {
 timeout time.Duration
 retries int
}

type Option func(*Client)

func WithTimeout(d time.Duration) Option {
 return func(c *Client) { c.timeout = d }
}

func WithRetries(n int) Option {
 return func(c *Client) { c.retries = n }
}

func NewClient(opts ...Option) *Client {
 c := &Client{timeout: time.Second, retries: 1}
 for _, o := range opts {
 o(c)
 }
 return c
}

func main() {
 c := NewClient(WithTimeout(200*time.Millisecond), WithRetries(3))
 fmt.Printf("timeout=%s retries=%d\n", c.timeout, c.retries)
}

go run .
```

Expected output:

```
timeout=200ms retries=3
```

## Table-driven validation recipe

```
mkdir -p /tmp/go-egr-table && cd /tmp/go-egr-table
go mod init example.com/egr-table
```

Save as main.go:

```
package main

import (
 "fmt"
 "strings"
)

func validEmail(s string) bool {
 return strings.Count(s, "@") == 1 && !strings.HasPrefix(s, "@") && !strings.HasSuffix(s,
}

func main() {
 cases := []struct {
 in string
 want bool
 }{
 {"a@b.co", true},
 {"@x", false},
 {"nope", false},
 }
 for _, tc := range cases {
 got := validEmail(tc.in)
 fmt.Printf("%q got=%v want=%v ok=%v\n", tc.in, got, tc.want, got == tc.want)
 }
}

go run .
```

## Expected output:

```
"a@b.co" got=true want=true ok=true
"@x" got=false want=false ok=true
"nope" got=false want=false ok=true
```

## Runnable example

Recipe-style patterns: functional options for config, streaming `json.Decoder` over a reader, and a context-aware worker fan-in.

```
mkdir -p /tmp/go-recipes && cd /tmp/go-recipes
go mod init example.com/recipes
```

Save as `main.go`:

```
package main

import (
 "context"
 "encoding/json"
 "fmt"
 "strings"
 "sync"
 "time"
)

type Config struct {
 Workers int
 Timeout time.Duration
}

type Option func(*Config)

func WithWorkers(n int) Option {
 return func(c *Config) { c.Workers = n }
}

func WithTimeout(d time.Duration) Option {
 return func(c *Config) { c.Timeout = d }
}

func NewConfig(opts ...Option) Config {
 c := Config{Workers: 2, Timeout: time.Second}
 for _, o := range opts {
 o(&c)
 }
 return c
}

func main() {
 cfg := NewConfig(WithWorkers(3), WithTimeout(200*time.Millisecond))
 fmt.Printf("config: workers=%d timeout=%s\n", cfg.Workers, cfg.Timeout)
```

```

// Streaming JSON values (NDJSON)
r := strings.NewReader("{\"n\":1}\\n{\"n\":2}\\n{\"n\":3}\\n")
dec := json.NewDecoder(r)
var sum int
for dec.More() {
 var row struct {
 N int `json:"n"`
 }
 if err := dec.Decode(&row); err != nil {
 panic(err)
 }
 sum += row.N
}
fmt.Println("stream_sum:", sum)

// Bounded workers
ctx, cancel := context.WithTimeout(context.Background(), cfg.Timeout)
defer cancel()
jobs := make(chan int, 8)
var wg sync.WaitGroup
var mu sync.Mutex
var out int
for i := 0; i < cfg.Workers; i++ {
 wg.Add(1)
 go func() {
 defer wg.Done()
 for j := range jobs {
 select {
 case <-ctx.Done():
 return
 default:
 mu.Lock()
 out += j
 mu.Unlock()
 }
 }
 }()
}
for i := 1; i <= 5; i++ {
 jobs <- i
}
close(jobs)
wg.Wait()
fmt.Println("worker_sum:", out)
}

```

```
go run .
```

### Expected output:

```
config: workers=3 timeout=200ms
stream_sum: 6
worker_sum: 15
```

### What to notice

- Options keep defaults + overrides readable without telescoping constructors.
- `json.Decoder` streams; it does not require `ReadAll` of a huge payload.
- Workers always honor `ctx.Done()` for operational cancellation.

### Try next

- Add `WithName` option and validate `Workers > 0` in `NewConfig`.
- Decode a large file with `os.Open` + `Decoder` in a loop.

## **193 Patterns from Go in Practice (2nd ed., 2025)**



# 193 Patterns from Go in Practice (2nd ed., 2025)

The 2025 edition is structured around four parts: fundamentals, robust application practices, end-to-end web systems, and cloud/advanced topics (reflection, code generation, interop).

## What This Adds to Our Book

- Strong mid-level bridge: from language fluency to production engineering.
- Better integration of testing, debugging, and benchmarking as one quality discipline.
- Explicit chapter path toward microservices and external service integration.

## End-to-End Service Model

```
HTTP edge -> business layer -> storage/external APIs -> telemetry -> deployment/runtime check
```

## Deep Integration Example: External Service Client with Typed Error Strategy

```
package external

import (
 "context"
 "encoding/json"
 "errors"
 "fmt"
 "net/http"
 "time"
)

var ErrUpstreamUnavailable = errors.New("upstream unavailable")

type Client struct {
 HTTP *http.Client
 Base string
}
```

```

type Profile struct {
 ID string `json:"id"`
 Name string `json:"name"`
}

func NewClient(base string) Client {
 return Client{
 Base: base,
 HTTP: &http.Client{Timeout: 2 * time.Second},
 }
}

func (c Client) GetProfile(ctx context.Context, id string) (Profile, error) {
 ctx, cancel := context.WithTimeout(ctx, 1200*time.Millisecond)
 defer cancel()

 req, err := http.NewRequestWithContext(ctx, http.MethodGet, c.Base+"/profiles/"+id, nil)
 if err != nil {
 return Profile{}, err
 }

 resp, err := c.HTTP.Do(req)
 if err != nil {
 return Profile{}, fmt.Errorf("http call failed: %w", ErrUpstreamUnavailable)
 }
 defer resp.Body.Close()

 if resp.StatusCode >= 500 {
 return Profile{}, fmt.Errorf("status %d: %w", resp.StatusCode, ErrUpstreamUnavailable)
 }
 if resp.StatusCode != http.StatusOK {
 return Profile{}, fmt.Errorf("unexpected status: %d", resp.StatusCode)
 }

 var p Profile
 if err := json.NewDecoder(resp.Body).Decode(&p); err != nil {
 return Profile{}, err
 }
 return p, nil
}

```

## Why This Matters

Production services fail most often at integration boundaries. This pattern teaches typed failure handling and timeout discipline in a way that scales across microservices.

## Curriculum Upgrades Recommended

1. Add one full chapter on external service client design (timeouts, retry policy, error typing).
2. Add one chapter on data ingress/egress patterns (upload/download, validation, content type).
3. Add one chapter on reflection and code generation boundaries: when useful, when harmful.

## More examples

### Middleware chain composition

```
mkdir -p /tmp/go-gip-mw && cd /tmp/go-gip-mw
go mod init example.com/gip-mw
```

Save as main.go:

```
package main

import (
 "fmt"
 "net/http"
 "net/http/httpptest"
)

type Middleware func(http.Handler) http.Handler

func chain(h http.Handler, mws ...Middleware) http.Handler {
 for i := len(mws) - 1; i >= 0; i-- {
 h = mws[i](h)
 }
 return h
}

func main() {
 var order []string
 mw := func(name string) Middleware {
 return func(next http.Handler) http.Handler {
 return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
 order = append(order, name)
 next.ServeHTTP(w, r)
 })
 }
 }
 h := chain(http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
 order = append(order, "handler")
 })
```

```

 fmt.Fprintln(w, "ok")
 }), mw("a"), mw("b"))

 rr := httptest.NewRecorder()
 h.ServeHTTP(rr, httptest.NewRequest(http.MethodGet, "/", nil))
 fmt.Println(order)
}

go run .

```

### Expected output:

```
[a b handler]
```

### Content-type gate for JSON POST

```

mkdir -p /tmp/go-gip-ct && cd /tmp/go-gip-ct
go mod init example.com/gip-ct

```

Save as main.go:

```

package main

import (
 "fmt"
 "net/http"
 "net/http/httptest"
 "strings"
)

func requireJSON(next http.Handler) http.Handler {
 return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
 ct := r.Header.Get("Content-Type")
 if !strings.HasPrefix(ct, "application/json") {
 http.Error(w, "want json", http.StatusUnsupportedMediaType)
 return
 }
 next.ServeHTTP(w, r)
 })
}

func main() {
 h := requireJSON(http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
 fmt.Fprintln(w, "json ok")
 }))
}

```

```

rr := httptest.NewRecorder()
req := httptest.NewRequest(http.MethodPost, "/", strings.NewReader(`{}`))
h.ServeHTTP(rr, req)
fmt.Println("no ct:", rr.Code)

rr = httptest.NewRecorder()
req = httptest.NewRequest(http.MethodPost, "/", strings.NewReader(`{}`))
req.Header.Set("Content-Type", "application/json")
h.ServeHTTP(rr, req)
fmt.Println("json:", rr.Code, strings.TrimSpace(rr.Body.String()))
}

go run .

```

### Expected output:

```

no ct: 415
json: 200 json ok

```

## Runnable example

External client practice: typed upstream errors, client timeout, and status classification with `httptest`.

```

mkdir -p /tmp/go-gip && cd /tmp/go-gip
go mod init example.com/gip

```

Save as `main.go`:

```

package main

import (
 "context"
 "encoding/json"
 "errors"
 "fmt"
 "net/http"
 "net/http/httptest"
 "time"
)

var ErrUpstreamUnavailable = errors.New("upstream unavailable")

type Profile struct {
 Name string `json:"name"`
}

```

```

}

func fetchProfile(ctx context.Context, client *http.Client, base string) (Profile, error) {
 req, err := http.NewRequestWithContext(ctx, http.MethodGet, base+"/profile", nil)
 if err != nil {
 return Profile{}, err
 }
 resp, err := client.Do(req)
 if err != nil {
 return Profile{}, fmt.Errorf("http: %w", ErrUpstreamUnavailable)
 }
 defer resp.Body.Close()
 if resp.StatusCode >= 500 {
 return Profile{}, fmt.Errorf("status %d: %w", resp.StatusCode, ErrUpstreamUnavailable)
 }
 if resp.StatusCode != http.StatusOK {
 return Profile{}, fmt.Errorf("unexpected status %d", resp.StatusCode)
 }
 var p Profile
 if err := json.NewDecoder(resp.Body).Decode(&p); err != nil {
 return Profile{}, err
 }
 return p, nil
}

func main() {
 var hits int
 ts := httptest.NewServer(http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
 hits++
 if hits == 1 {
 w.WriteHeader(http.StatusBadGateway)
 return
 }
 _ = json.NewEncoder(w).Encode(Profile{Name: "ada"})
 }))
 defer ts.Close()

 client := &http.Client{Timeout: 300 * time.Millisecond}
 ctx := context.Background()
 _, err := fetchProfile(ctx, client, ts.URL)
 fmt.Println("first:", err, "unavailable?", errors.Is(err, ErrUpstreamUnavailable))
 p, err := fetchProfile(ctx, client, ts.URL)
 fmt.Println("second:", p, err)
}

go run .

```

**Expected output:**

```
first: status 502: upstream unavailable unavailable? true
second: {ada} <nil>
```

**What to notice**

- Wrap with %w and a sentinel so callers decide retry vs fail without parsing strings.
- Always set `http.Client.Timeout` (or context deadline) on outbound deps.
- Close response bodies to preserve connection reuse.

**Try next**

- Retry only when `errors.Is(err, ErrUpstreamUnavailable)` with jitter.
- Map 404 to a distinct `ErrNotFound` sentinel.

## **194 Patterns from Mastering Go (4th ed., 2024)**



# 194 Patterns from Mastering Go (4th ed., 2024)

Recent 4th-edition signals emphasize advanced practice: generics, interfaces/reflection, Unix systems work, concurrency, web services, TCP/WebSocket, REST APIs, testing/profiling, fuzzing/observability, and performance.

## What This Adds to Our Book

- Better advanced track sequencing after core language chapters.
- Stronger focus on performance + fuzz + observability as one reliability loop.
- More explicit Unix-system programming integration for backend engineers.

## Advanced Engineering Loop

```
implement -> test -> fuzz -> profile -> optimize -> observe in runtime -> iterate
```

## Deep Integration Example: Fuzz-Ready Parser + Profile-Aware Fast Path

```
package parser

import (
 "errors"
 "strconv"
 "strings"
)

var ErrBadRecord = errors.New("bad record")

type Record struct {
 ID int64
 Score float64
}

func ParseRecord(line string) (Record, error) {
 parts := strings.Split(line, ",")
```

```

 if len(parts) != 2 {
 return Record{}, ErrBadRecord
 }

 id, err := strconv.ParseInt(strings.TrimSpace(parts[0]), 10, 64)
 if err != nil {
 return Record{}, ErrBadRecord
 }
 score, err := strconv.ParseFloat(strings.TrimSpace(parts[1]), 64)
 if err != nil {
 return Record{}, ErrBadRecord
 }
 if id <= 0 || score < 0 {
 return Record{}, ErrBadRecord
 }
 return Record{ID: id, Score: score}, nil
}

package parser_test

import "testing"

func FuzzParseRecord(f *testing.F) {
 f.Add("1,2.5")
 f.Add("x,y")
 f.Add("")

 f.Fuzz(func(t *testing.T, input string) {
 _, _ = ParseRecord(input)
 })
}

```

## Why This Matters

As Go systems mature, defects shift from syntax mistakes to boundary and performance defects. Fuzzing and profiling help catch these before they become incidents.

## Curriculum Upgrades Recommended

1. Add a dedicated chapter combining fuzzing, race detection, and pprof for one service.
2. Add advanced chapter on TCP/WebSocket design under network systems.
3. Expand performance track with explicit optimization workflow and rollback criteria.

## More examples

### TCP echo with deadlines (systems flavor)

```
mkdir -p /tmp/go-mg-tcp && cd /tmp/go-mg-tcp
go mod init example.com/mg-tcp
```

Save as main.go:

```
package main

import (
 "bufio"
 "fmt"
 "net"
 "time"
)

func main() {
 ln, err := net.Listen("tcp", "127.0.0.1:0")
 if err != nil {
 panic(err)
 }
 defer ln.Close()
 go func() {
 c, _ := ln.Accept()
 defer c.Close()
 _ = c.SetDeadline(time.Now().Add(time.Second))
 line, _ := bufio.NewReader(c).ReadString('\n')
 _, _ = c.Write([]byte("echo:" + line))
 }()

 c, err := net.DialTimeout("tcp", ln.Addr().String(), time.Second)
 if err != nil {
 panic(err)
 }
 defer c.Close()
 _ = c.SetDeadline(time.Now().Add(time.Second))
 _, _ = c.Write([]byte("hi\n"))
 line, _ := bufio.NewReader(c).ReadString('\n')
 fmt.Print(line)
}

go run .
```

Expected output:

echo:hi

## Worker pool fan-out / fan-in

```
mkdir -p /tmp/go-mg-pool && cd /tmp/go-mg-pool
go mod init example.com/mg-pool
```

Save as main.go:

```
package main

import (
 "fmt"
 "sync"
)

func main() {
 jobs := make(chan int, 8)
 results := make(chan int, 8)
 var wg sync.WaitGroup
 for w := 0; w < 3; w++ {
 wg.Add(1)
 go func() {
 defer wg.Done()
 for j := range jobs {
 results <- j * 2
 }
 }()
 }
 go func() {
 for i := 1; i <= 5; i++ {
 jobs <- i
 }
 close(jobs)
 wg.Wait()
 close(results)
 }()
 sum := 0
 for r := range results {
 sum += r
 }
 fmt.Println("sum:", sum)
}
```

go run .

## Expected output:

sum: 30

## Runnable example

Robust parsing + fuzz target: ParseRecord never panics on garbage; run unit checks and a short fuzz if supported.

```
mkdir -p /tmp/go-mastering && cd /tmp/go-mastering
go mod init example.com/mastering
```

Save as parse.go:

```
package mastering

import (
 "fmt"
 "strconv"
 "strings"
)

type Record struct {
 ID int
 Score float64
}

func ParseRecord(s string) (Record, error) {
 parts := strings.Split(s, ",")
 if len(parts) != 2 {
 return Record{}, fmt.Errorf("want id,score")
 }
 id, err := strconv.Atoi(strings.TrimSpace(parts[0]))
 if err != nil {
 return Record{}, fmt.Errorf("id: %w", err)
 }
 score, err := strconv.ParseFloat(strings.TrimSpace(parts[1]), 64)
 if err != nil {
 return Record{}, fmt.Errorf("score: %w", err)
 }
 return Record{ID: id, Score: score}, nil
}
```

Save as parse\_test.go:

```

package mastering

import "testing"

func TestParseRecord(t *testing.T) {
 r, err := ParseRecord("1,2.5")
 if err != nil || r.ID != 1 || r.Score != 2.5 {
 t.Fatalf("got %+v %v", r, err)
 }
 if _, err := ParseRecord("x,y"); err == nil {
 t.Fatal("expected error")
 }
}

func FuzzParseRecord(f *testing.F) {
 f.Add("1,2.5")
 f.Add("")
 f.Add(",,,")
 f.Fuzz(func(t *testing.T, input string) {
 _, _ = ParseRecord(input) // must not panic
 })
}

```

Save as cmd/demo/main.go:

```

package main

import (
 "fmt"

 "example.com/mastering"
)

func main() {
 r, err := mastering.ParseRecord("7, 3.14")
 fmt.Println(r, err)
 _, err = mastering.ParseRecord("nope")
 fmt.Println("bad:", err != nil)
}

go test .
go test -fuzz=FuzzParseRecord -fuzztime=5s .
go run ./cmd/demo

```

**Expected output** (fuzz lines vary):

PASS

```
...
{7 3.14} <nil>
bad: true
```

### What to notice

- Fuzzing finds panics and unexpected paths at boundaries—Mastering Go’s maturity theme.
- Parsers should return errors, never panic, on user input.

### Try next

- `go test -race` on concurrent map code that shares a cache.
- Add a benchmark for `ParseRecord` on long inputs.

## **195 Patterns from Let Us Go! (2025)**



# 195 Patterns from Let Us Go! (2025)

This 2025 beginner-focused arc stresses playground-first onboarding, workspace/tooling setup, version control flow, and then deep dive application development.

## What This Adds to Our Book

- Better early-stage onboarding narrative for self-learners.
- A clearer toolchain chapter that explains *why* each tool exists.
- Practical Git workflow integration earlier in the learning path.

## Onboarding Progression

Playground exploration -> local toolchain setup -> Git discipline -> deeper project work

## Deep Integration Example: Minimal Project Lifecycle from Day 1

```
package main

import (
 "flag"
 "fmt"
 "os"
)

type Config struct {
 Name string
 Env string
}

func parseConfig() Config {
 cfg := Config{}
 flag.StringVar(&cfg.Name, "name", "gopher", "name to greet")
 flag.StringVar(&cfg.Env, "env", "dev", "runtime environment")
 flag.Parse()
 return cfg
}
```

```

}

func run(cfg Config) error {
 if cfg.Env == "prod" && os.Getenv("APP_ALLOW_PROD") != "1" {
 return fmt.Errorf("prod run blocked without APP_ALLOW_PROD=1")
 }
 fmt.Printf("hello %s (%s)\n", cfg.Name, cfg.Env)
 return nil
}

func main() {
 if err := run(parseConfig()); err != nil {
 fmt.Fprintln(os.Stderr, "error:", err)
 os.Exit(1)
 }
}

```

## Why This Matters

Beginner success is mostly about reducing setup friction and establishing stable habits early. Good onboarding chapters improve completion rates and reduce learner drop-off.

## Curriculum Upgrades Recommended

1. Strengthen foundational chapters with explicit environment setup validation steps.
2. Add version-control workflow examples tied to chapter exercises/projects.
3. Add “first-week debugging habits” chapter before advanced language features.

## More examples

### Small CLI with flags and usage exit

```

mkdir -p /tmp/go-lug-cli && cd /tmp/go-lug-cli
go mod init example.com/lug-cli

```

Save as main.go:

```

package main

import (
 "flag"
 "fmt"

```

```

 "os"
)

func run(args []string) (string, error) {
 fs := flag.NewFlagSet("greet", flag.ContinueOnError)
 name := fs.String("name", "world", "who to greet")
 if err := fs.Parse(args); err != nil {
 return "", err
 }
 return "hello " + *name, nil
}

func main() {
 // Simulate argv without exiting the process.
 out, err := run([]string{"-name", "Ada"})
 fmt.Println(out, err)
 _, err = run([]string{"-bad"})
 fmt.Println("bad flag err:", err != nil)

 // Show default path
 out, _ = run(nil)
 fmt.Println(out)
 _ = os.Stderr // keep habit of writing usage to stderr in real CLIs
}

go run .

```

### Expected output:

```

hello Ada <nil>
bad flag err: true
hello world

```

### Debugging habit: log inputs and duration

```

mkdir -p /tmp/go-lug-debug && cd /tmp/go-lug-debug
go mod init example.com/lug-debug

```

Save as main.go:

```

package main

import (
 "fmt"
 "time"

```

```

)

func work(n int) int {
 sum := 0
 for i := 0; i < n; i++ {
 sum += i
 }
 return sum
}

func main() {
 start := time.Now()
 in := 1000
 out := work(in)
 fmt.Printf("work in=%d out=%d dur=%s\n", in, out, time.Since(start))
}

go run .

```

### Expected output:

work in=1000 out=499500 dur=...

## Runnable example

Beginner-friendly CLI habits: flags, env guardrails, stderr errors, and non-zero exit codes.

```

mkdir -p /tmp/go-letus && cd /tmp/go-letus
go mod init example.com/letus

```

Save as `main.go`:

```

package main

import (
 "flag"
 "fmt"
 "os"
)

type Config struct {
 Name string
 Env string
}

```

```

func parseConfig(args []string) (Config, error) {
 fs := flag.NewFlagSet("letus", flag.ContinueOnError)
 fs.SetOutput(os.Stderr)
 name := fs.String("name", "world", "who to greet")
 env := fs.String("env", "dev", "dev|prod")
 if err := fs.Parse(args); err != nil {
 return Config{}, err
 }
 return Config{Name: *name, Env: *env}, nil
}

func run(cfg Config) error {
 if cfg.Env == "prod" && os.Getenv("APP_ALLOW_PROD") != "1" {
 return fmt.Errorf("prod run blocked without APP_ALLOW_PROD=1")
 }
 if cfg.Name == "" {
 return fmt.Errorf("name must not be empty")
 }
 fmt.Printf("hello %s (%s)\n", cfg.Name, cfg.Env)
 return nil
}

func main() {
 cfg, err := parseConfig(os.Args[1:])
 if err != nil {
 os.Exit(2) // usage
 }
 if err := run(cfg); err != nil {
 fmt.Fprintln(os.Stderr, "error:", err)
 os.Exit(1)
 }
}

```

```

go run . -name Ada
go run . -env prod; echo exit:$?
APP_ALLOW_PROD=1 go run . -env prod -name Ada

```

### Expected output:

```

hello Ada (dev)
error: prod run blocked without APP_ALLOW_PROD=1
exit:1
hello Ada (prod)

```

### What to notice

- Exit 2 for usage/flags, 1 for runtime errors, 0 success—script friendly.

- Guardrails (prod allow) prevent accidents during early learning and ops.
- Diagnostics on stderr; greeting on stdout.

**Try next**

- Add `-v` verbose logging to stderr with `log/slog`.
- Write a table test for `run` covering prod blocked/allowed.

## **196 Patterns from Automate Your Home Using Go (2024)**

# 196 Patterns from Automate Your Home Using Go (2024)

This 2024 title emphasizes applied systems engineering: Raspberry Pi deployment, Dockerized services, telemetry, monitoring with Prometheus/Grafana, and practical automation workflows.

## What This Adds to Our Book

- Strong real-world edge/infrastructure use cases beyond traditional web CRUD.
- Better observability-first mindset for long-running services.
- Helpful pathway from single-node apps to homelab/multi-service operations.

## Edge Automation Topology

```
sensor/input -> Go collector -> local queue/cache -> automation decision -> action
|
+--> metrics/logs -> Prometheus/Grafana
```

## Deep Integration Example: Telemetry Collector with Health and Metrics Surface

```
package main

import (
 "encoding/json"
 "fmt"
 "log"
 "net/http"
 "sync"
 "time"
)

type Reading struct {
 Source string `json:"source"`
 Value float64 `json:"value"`
}
```



```

 At time.Time `json:"at"`
}

type State struct {
 mu sync.RWMutex
 last map[string]Reading
 ingested uint64
}

func newState() *State {
 return &State{last: make(map[string]Reading)}
}

func (s *State) ingest(r Reading) {
 s.mu.Lock()
 defer s.mu.Unlock()
 s.last[r.Source] = r
 s.ingested++
}

func (s *State) handleIngest(w http.ResponseWriter, r *http.Request) {
 if r.Method != http.MethodPost {
 http.Error(w, "method not allowed", http.StatusMethodNotAllowed)
 return
 }
 var x Reading
 if err := json.NewDecoder(r.Body).Decode(&x); err != nil {
 http.Error(w, "bad payload", http.StatusBadRequest)
 return
 }
 x.At = time.Now().UTC()
 s.ingest(x)
 w.WriteHeader(http.StatusAccepted)
}

func (s *State) handleHealth(w http.ResponseWriter, _ *http.Request) {
 w.Header().Set("Content-Type", "application/json")
 _ = json.NewEncoder(w).Encode(map[string]any{"ok": true, "ts": time.Now().UTC()})
}

func (s *State) handleMetrics(w http.ResponseWriter, _ *http.Request) {
 s.mu.RLock()
 defer s.mu.RUnlock()
 _, _ = w.Write([]byte("collector_ingested_total "))
 _, _ = w.Write([]byte(fmt.Sprintf("%d\n", s.ingested)))
}

```

```

func main() {
 st := newState()
 mux := http.NewServeMux()
 mux.HandleFunc("POST /ingest", st.handleIngest)
 mux.HandleFunc("GET /healthz", st.handleHealth)
 mux.HandleFunc("GET /metrics", st.handleMetrics)

 srv := &http.Server{
 Addr: ":8080",
 Handler: mux,
 ReadHeaderTimeout: 2 * time.Second,
 ReadTimeout: 5 * time.Second,
 WriteTimeout: 10 * time.Second,
 IdleTimeout: 30 * time.Second,
 }
 log.Fatal(srv.ListenAndServe())
}

```

## Why This Matters

Practical automation projects force integration of networking, state handling, deployment, and observability. This makes them high-value capstones for Go learners.

## Curriculum Upgrades Recommended

1. Add an edge/homelab tutorial track under specialized chapters.
2. Add observability hooks by default in systems examples.
3. Add deployment story from local process -> Docker -> lightweight orchestrated environment.

## More examples

### Device event loop with context cancel

```

mkdir -p /tmp/go-ha-loop && cd /tmp/go-ha-loop
go mod init example.com/ha-loop

```

Save as main.go:

```

package main

import (
 "context"

```

```

 "fmt"
 "time"
)

 func poll(ctx context.Context, events chan<- string) {
 t := time.NewTicker(10 * time.Millisecond)
 defer t.Stop()
 n := 0
 for {
 select {
 case <-ctx.Done():
 return
 case <-t.C:
 n++
 events <- fmt.Sprintf("tick-%d", n)
 if n == 3 {
 return
 }
 }
 }
 }

 func main() {
 ctx, cancel := context.WithTimeout(context.Background(), time.Second)
 defer cancel()
 events := make(chan string, 8)
 go poll(ctx, events)
 for i := 0; i < 3; i++ {
 fmt.Println(<-events)
 }
 }
}

go run .

```

### Expected output:

```

tick-1
tick-2
tick-3

```

### State machine for a smart switch

```

mkdir -p /tmp/go-ha-sm && cd /tmp/go-ha-sm
go mod init example.com/ha-sm

```

Save as `main.go`:

```

package main

import "fmt"

type State int

const (
 Off State = iota
 On
)

func (s State) String() string {
 if s == On {
 return "on"
 }
 return "off"
}

func transition(s State, event string) State {
 switch event {
 case "toggle":
 if s == On {
 return Off
 }
 return On
 case "off":
 return Off
 default:
 return s
 }
}

func main() {
 s := Off
 for _, e := range []string{"toggle", "toggle", "off", "toggle"} {
 s = transition(s, e)
 fmt.Println(e, "->", s)
 }
}

go run .

```

### Expected output:

```

toggle -> on
toggle -> off
off -> off

```

toggle -> on

## Runnable example

Homelab-shaped service: ingest events, healthz, and manual `/metrics` counter—stdlib HTTP with timeouts (ephemeral listen for a self-check).

```
mkdir -p /tmp/go-homeauto && cd /tmp/go-homeauto
go mod init example.com/homeauto
```

Save as `main.go`:

```
package main

import (
 "encoding/json"
 "fmt"
 "io"
 "log"
 "net"
 "net/http"
 "strings"
 "sync/atomic"
 "time"
)

type state struct{ events atomic.Int64 }

func main() {
 st := &state{}
 mux := http.NewServeMux()
 mux.HandleFunc("POST /ingest", func(w http.ResponseWriter, r *http.Request) {
 var body struct {
 Device string `json:"device"`
 Value float64 `json:"value"`
 }
 if err := json.NewDecoder(r.Body).Decode(&body); err != nil {
 http.Error(w, "bad json", http.StatusBadRequest)
 return
 }
 st.events.Add(1)
 w.WriteHeader(http.StatusAccepted)
 fmt.Fprintf(w, "accepted %s\n", body.Device)
 })
 mux.HandleFunc("GET /healthz", func(w http.ResponseWriter, _ *http.Request) {
 w.WriteHeader(http.StatusOK)
 })
}
```

```

 _, _ = w.Write([]byte("ok"))
 })
 mux.HandleFunc("GET /metrics", func(w http.ResponseWriter, _ *http.Request) {
 fmt.Fprintf(w, "# TYPE home_events_total counter\nhome_events_total %d\n", st.events)
 })

 ln, err := net.Listen("tcp", "127.0.0.1:0")
 if err != nil {
 log.Fatal(err)
 }
 srv := &http.Server{Handler: mux, ReadHeaderTimeout: 2 * time.Second}
 go func() { _ = srv.Serve(ln) }()
 base := "http://" + ln.Addr().String()

 resp, err := http.Post(base+"/ingest", "application/json", strings.NewReader(`{"device":`))
 if err != nil {
 log.Fatal(err)
 }
 io.Copy(io.Discard, resp.Body)
 resp.Body.Close()
 fmt.Println("ingest:", resp.StatusCode)

 resp, _ = http.Get(base + "/healthz")
 resp.Body.Close()
 fmt.Println("healthz:", resp.StatusCode)

 resp, _ = http.Get(base + "/metrics")
 b, _ := io.ReadAll(resp.Body)
 resp.Body.Close()
 fmt.Printf("metrics:\n%s", b)
 _ = srv.Close()
}

go run .

```

### Expected output:

```

ingest: 202
healthz: 200
metrics:
TYPE home_events_total counter
home_events_total 1

```

### What to notice

- Homelab services still need health, metrics, and timeouts—not only happy-path handlers.
- Atomic counters and JSON ingest are enough to start; wire MQTT/GPIO later.

- Ship the same binary under systemd or Docker next.

**Try next**

- Persist last reading to an atomic file write (part 14).
- Add graceful **SIGTERM** shutdown for Raspberry Pi deploys.

## **Go Deep Dives**



## **Go Deep Dives Overview**

# Go Deep Dives Overview

## Diagram: Deep-dive stack

```
flow:
 [Comp]
 |
 v
 [RT]

 [RT]
 |
 v
 [OS]
```

This part is for readers who already write idiomatic Go and want the **runtime, toolchain, and production mental models** behind real symptoms: latency cliffs, leaks, interface nil traps, timer churn, cgo taxes, and “why is this slow in the pod?”

Earlier parts teach *how to use* the language. These chapters teach *how the machine runs it* — deep enough to debug, not a full fork of `src/runtime`.

## Who This Part Is For

- You have shipped Go services or tools
- You can read basic `pprof` / `trace` output
- You want **code** → **runtime structures** → **observables**

If you are still learning syntax, finish Core, Memory, and Concurrency first.

## Learning Paths

### A — Runtime core

201 GMP → 202 memory model → 203 channels → 213 netpoll → 214 timers → 227 mutex → 239 sysmon

## B — Data representation

204 interfaces → 205 slice/map → 218 Swiss maps → 235 strings/bytes → 217 weak/unique → 236 errors

## C — Execution control

206 defer/panic → 215 stacks → 226 context → 220 leaks → 223 synctest

## D — Performance & GC

208 GC → 219 pool/zero-alloc → 222 pprof/trace → 224 zero-copy → 209 compiler/PGO → 243 GOMAXPROCS/cgroup

## E — Toolchain & platforms

210 link/race → 228 cgo → 230 ABI → 231 coverage/fuzz → 232 build cache → 233 experiments → 234 Wasm → 244 plugins

## F — Production craft

225 HTTP transport → 241 TLS → 242 slog handlers → 237 mistakes → 238 debugging → 240 reading source → 245 topic radar

## Full Chapter Map

#	Topic
201	Scheduler (GMP)
202	Memory model / happens-before
203	Channels & select
204	Interfaces (eface/iface)
205	Slice & map headers
206	Defer, panic, unwind
207	Generics implementation
208	GC internals & pacer
209	Compiler, SSA, PGO
210	Link, buildmodes, race
211	Reflect cost
212	Modules, MVS, toolchain
213	Netpoller
214	Timers / <code>time.After</code>
215	Stack growth
216	Finalizers & cleanup

#	Topic
217	<b>weak &amp; unique</b>
218	Swiss maps & maphash
219	<b>sync.Pool</b> & zero-alloc
220	Goroutine leaks
221	GODEBUG knobs
222	pprof & tracer internals
223	testing/synctest (bubbles, durable blocks, fake time)
224	Zero-copy I/O
225	HTTP Transport pool
226	Context cancel trees
227	Mutex / semaphore runtime
228	cgo + Go 1.26 scheduler
229	Iterators / range-over-func
230	ABI & assembly
231	Coverage & fuzz engine
232	Build cache / reproducibility
233	GOEXPERIMENT catalog
234	Wasm / wasip1
235	String / []byte internals
236	Error interface cost
237	Production mistakes map
238	Production debugging
239	sysmon & scavenger
240	Reading runtime source
241	TLS handshake path
242	slog handler design
243	GOMAXPROCS & cgroups
244	Plugin buildmode pitfalls
245	X/ecosystem topic radar
246	Internals-for-Interns article radar
247	Memory allocator (mcache/mcentral/mheap)
248	Runtime bootstrap
249	select + selectgo
250	Stacktraces + binary metadata
251	mmap vs pread storage I/O
252	Compiler frontend (scan/parse/types)
253	Compiler IR / SSA / link
254	Race detector internals
255	Write barriers & mark assist
256	Work stealing / findrunnable
257	Type assert / convert runtime
258	OS signals & runtime
259	HTTP/2 in Go
260	database/sql pool
261	JSON performance
262	errgroup patterns

## Stack Diagram

```
your code
 |
 v
compiler (types, SSA, inlining, escape, PGO)
 |
 v
runtime (scheduler, poller, timers, maps, GC, defer, cgo)
 |
 v
OS threads + cgroups + network stack + TLS
```

## Diagrams in this part

Prefer **ASCII diagrams** in fenced **text** blocks (works in HTML, PDF, and EPUB). Flagship topics also ship optional **SVG** under `content/20-go-deep-dives/assets/` for HTML. Mermaid is not required for this part.

## Study Method

```
hypothesis -> minimal repro -> measure (pprof/trace/race) -> model -> fix ownership/allocs/t
```

1. One chapter, one question.
2. Run the experiment; change one knob.
3. Confirm with tools, not vibes.

## Relationship to Other Parts

Concern	Prefer
Concurrency patterns	Part 07 + 201/203/220
GC knobs tutorial	Part 03/18 + 208/239
Profiling workflow	Part 18 + 222/238
Stdlib how-to	Part 98
HTTP resilience	Part 13 + 225/241
Security TLS ops	Part 17 + 241

## Honesty Boundary

Runtime details evolve (scheduler, GC, maps, experiments). Chapters teach **stable models** and version-labeled notes (e.g. Go 1.26 Green Tea GC, P-syscall simplification; Go 1.27 size-specialized malloc and `goroutineleak`). Pseudocode is a map for reading source — not a bit-identical clone of `src/runtime`.

## Checklist: Deep-Dive Literacy

- ☐ Draw G/M/P and explain work stealing
- ☐ State happens-before for unlock→lock and send→receive
- ☐ Explain typed-nil interface trap
- ☐ Fix a goroutine leak with context
- ☐ Replace hot `time.After` with timer reuse
- ☐ Tune or justify GOMAXPROCS under cgroup limits
- ☐ Read one CPU + one goroutine profile
- ☐ Find a panic string in `GOROOT/src/runtime`
- ☐ Know when cgo/plugins/Wasm are the wrong tool

When most boxes are automatic, you can track Go releases without fear.

## **Scheduler Deep Dive (GMP)**

# Scheduler Deep Dive (GMP)

## Overview

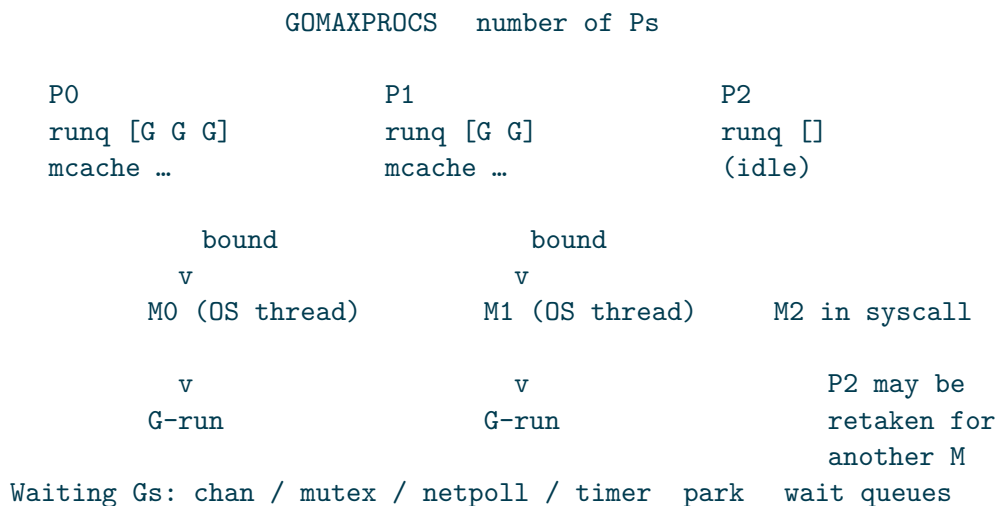
Go multiplexes millions of **goroutines (G)** onto fewer OS **threads (M)** using **logical processors (P)**. Understanding GMP turns vague advice (“don’t spawn unbounded goroutines”) into capacity math.

Diagram-rich companion series: [Internals for Interns — The Scheduler](#).

```
G = goroutine (stack + state: runnable / running / waiting / dead)
M = OS thread that executes Go code
P = scheduling context (run queue, memory caches, limits) - count GOMAXPROCS
```

Only an **M bound to a P** runs ordinary Go code. Syscalls may park the P or spin another M so other Gs keep running.

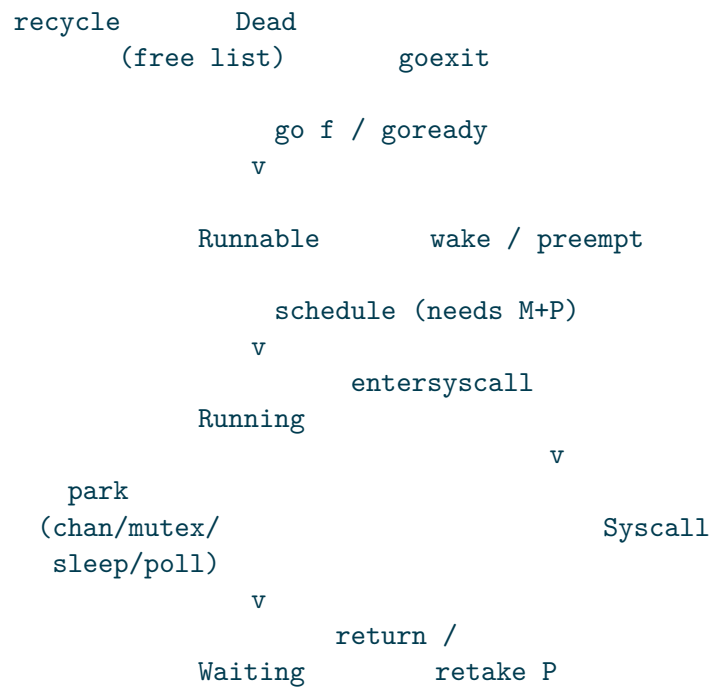
## Diagram: GMP at a glance



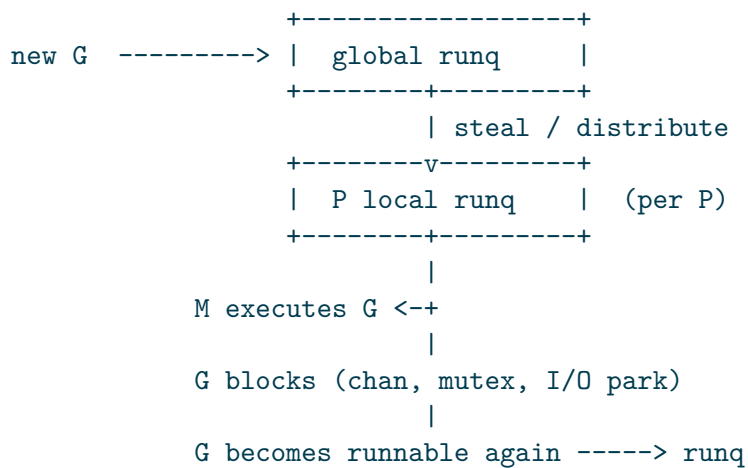
Only an **M bound to a P** runs ordinary Go code. Syscalls may free the P so other Gs keep running.



## Lifecycle of a G (creation → park → run)



## Core Loop (Mental Model)



1. Each P has a **local run queue** (cheap, no global lock).
2. Overflow / balancing uses a **global run queue**.
3. Idle Ps **work-steal** from other Ps' local queues.
4. Network poller and timers inject runnable Gs.

## GOMAXPROCS

```
runtime.GOMAXPROCS(0) // current setting
```

- Defaults to number of CPU cores (cgroup-aware on Linux in modern Go).
- Caps how many Gs run **Go code** in parallel (approx. number of Ps).
- Does **not** cap total goroutines or OS threads (threads can grow for syscalls).

Under a CPU quota (K8s), a wrong **GOMAXPROCS** causes either under-utilization or thrashing. Prefer auto-detection or set from cgroup limits intentionally.

## Preemption

Modern Go can preempt long-running Gs (async preemption) so a tight loop without function calls is less likely to starve the GC or other Gs. Still:

- Avoid owning a P forever with non-cooperative C code via cgo without care.
- `runtime.Gosched()` yields voluntarily — rare need if profiles do not show unfairness.

## Hand-off and Syscalls

When a G enters a syscall:

1. M may release its P so another M can run other Gs.
2. On return, M tries to reacquire a P; if none free, G goes runnable and M may park.

This is why blocking syscalls “all Go freezes,” but **too many** blocking syscalls create many Ms and scheduler overhead.

## Scheduling Latency Signals

Signal	Tool	Meaning
High <code>runtime.schedule / findrunnable</code>	CPU pprof	Scheduler overhead, not enough real work
Runnable Gs queueing	<code>go tool trace</code>	Over-subscription or lock/chan contention
Idle Ps + runnable Gs	trace	Load imbalance / GC / netpoll delays
Threads » GOMAXPROCS	<code>runtime.NumCgoCall</code> / OS metrics	Syscall or cgo heavy

## Design Rules From GMP

1. **Bound concurrency at the entrance** (semaphores, worker pools) — Gs are cheap, not free.
2. **Prefer non-blocking design at boundaries** — give every wait a context deadline.
3. **Match fan-out to Ps for CPU work** — GOMAXPROCS workers for pure CPU; more for I/O-bound wait.
4. **Measure under the real quota** — laptop cores pod millicores.

## Experiment

```
go mod init example
GOMAXPROCS=1 go run .
GOMAXPROCS=4 go run .
```

```
package main

import (
 "fmt"
 "runtime"
 "sync"
 "time"
)

func burn(d time.Duration) {
 end := time.Now().Add(d)
 for time.Now().Before(end) {
 // busy spin
 }
}

func main() {
 fmt.Println("GOMAXPROCS", runtime.GOMAXPROCS(0))
 var wg sync.WaitGroup
 start := time.Now()
 n := 4
 for i := 0; i < n; i++ {
 wg.Add(1)
 go func() {
 defer wg.Done()
 burn(200 * time.Millisecond)
 }()
 }
 wg.Wait()
}
```

```
 fmt.Println("elapsed", time.Since(start))
}
```

**What to notice:** With `GOMAXPROCS=1`, four 200ms CPU burns serialize (~800ms). With 4, they overlap (~200ms). I/O-bound work would not scale the same way.

**Try next:** Capture `go tool trace` around a server under load; find “Proc start/stop” and long stretches where Gs are runnable but not running.

## Related

- [Concurrency basics](#)
- [Contention tuning](#)

## Memory Model and Happens-Before

# Memory Model and Happens-Before

## Overview

Go’s memory model answers: **when is a write in goroutine A guaranteed visible to a read in goroutine B?** Without a *happens-before* edge, “it works on my machine” is data-race undefined behavior — even if the race detector is quiet today under lucky scheduling.

Reference: [Go Memory Model](#) (normative).

## Diagram: Happens-before edges

```
flow:
 [B] --HB--> [C]
 [C]
 |
 v
 [D]
```

Without an edge, concurrent write+read is a data race.

## Data Race Definition

A **data race** is concurrent access to a variable where at least one access is a write and the accesses are not synchronized by the memory model.

```
go test -race ./...
go run -race .
```

If `-race` fires, fix synchronization. Do not “make it volatile” by folklore.

## Happens-Before (Core Edges)

Informally, if event X *happens before* Y, then Y sees all effects of X.

Important edges:

Edge	Guarantee
Program order in one G	Earlier statements HB later (single goroutine)
go statement	Start of new G happens after the go statement executes
Channel send → receive	Send HB corresponding receive completion
Channel close → receive zero	Close HB receive that sees closed
Mutex unlock → later lock	Unlock HB subsequent Lock acquisition
RWMutex	Analogous rules for RLock/RUnlock
sync.Once	Do completion HB later Do returns
sync.WaitGroup	Done HB Wait return after counter hits 0
Atomic ops	Synchronizing atomics establish order per model

If you share memory, you almost always want one of: **mutex**, **channel ownership transfer**, or **atomic** with a clear protocol.

## Incorrect: Sharing Without Sync

```
// DATA RACE
var a int
go func() { a = 1 }()
fmt.Println(a) // may print 0 forever, or 1, or worse under compiler reordering assumptions
```

## Correct Patterns

### Mutex

```
var mu sync.Mutex
var a int

// writer
mu.Lock()
a = 1
mu.Unlock()

// reader
mu.Lock()
v := a
mu.Unlock()
```

Unlock *happens before* the next Lock; reader sees the write.

## Channel hand-off (share by communicating)

```
ch := make(chan int, 1)
go func() { ch <- 1 }()
v := <-ch // receive happens after send; sees 1
```

## Atomic flag

```
var ready atomic.Bool
var data int

// producer
data = 42
ready.Store(true) // release-style in practice for this pattern

// consumer
if ready.Load() {
 _ = data // safe if this is the only protocol and documented
}
```

Prefer mutexes for multi-field invariants; atomics for counters/flags with simple protocols.

## Init and Package-Level

All package `init` functions and variable initialization complete before `main` starts. Within a package, initialization order is constrained by dependency; across packages, import graph order applies. Do not rely on init order for subtle cross-package races — make dependencies explicit.

## Compiler and Hardware Reordering

Without synchronization, compilers and CPUs may reorder memory operations. The memory model is the contract that remains true; your mental model of “statements run in source order across cores” is false.

## Experiment

```
go mod init example
go run -race .
```



```

package main

import (
 "fmt"
 "sync"
)

func racy() int {
 var a int
 var wg sync.WaitGroup
 wg.Add(1)
 go func() {
 defer wg.Done()
 a = 1
 }()
 v := a // race with write
 wg.Wait()
 return v
}

func synced() int {
 var mu sync.Mutex
 var a int
 var wg sync.WaitGroup
 wg.Add(1)
 go func() {
 defer wg.Done()
 mu.Lock()
 a = 1
 mu.Unlock()
 }()
 wg.Wait()
 mu.Lock()
 v := a
 mu.Unlock()
 return v
}

func main() {
 fmt.Println("synced", synced())
 // Uncomment to see race detector:
 // fmt.Println("racy", racy())
 _ = racy
}

```

**What to notice:** -race instruments memory accesses; fixed code establishes unlock→lock or wait→read edges.

**Try next:** Build a single-producer single-consumer ring with atomics only; then rewrite with a buffered channel and compare clarity.

## Related

- [Synchronization](#)
- [sync/atomic stdlib](#)

## **Channel and Select Internals**

# Channel and Select Internals

## Overview

Channels are not magic pipes — they are runtime objects (**hchan**) with a lock, optional buffer ring, and wait queues of **sudogs** (goroutine wait records). Select is a careful algorithm over multiple channel operations so that **exactly one** case proceeds (or default).

Usage patterns: [Channel patterns](#).

## Diagram: Channel park / wake

```
sequence (top → bottom):
 actors: sender G, hchan, receiver G
 sender G --> hchan : chansend
 hchan --> receiver G : hand off / enqueue buf
 hchan --> sender G : return
 sender G --> sender G : gopark sendq
 receiver G --> hchan : chanrecv
 hchan --> sender G : goready
 note: alt buffer space or waiting recv
 note: else must wait
```

## Channel Structure (Conceptual)

```
hchan
 qcount // elements in buffer
 dataqsiz // buffer capacity (0 = unbuffered)
 buf // circular queue
 sendx/recvx // indices
 recvq // waiters for receive
 sendq // waiters for send
 lock
 closed
```

## Unbuffered (`make(chan T)`)

Send and receive must **rendezvous**: sender copies directly into receiver (or vice versa). One side parks on a sudog queue until the other arrives.

## Buffered (`make(chan T, n)`)

Send succeeds without a receiver while `qcount < n`. Receive succeeds without a sender while `qcount > 0`. Still synchronized via the channel lock and happens-before rules.

## Send / Receive Paths

Situation	Behavior
Recv, buffer non-empty	Pop buffer; maybe wake a sender
Recv, buffer empty, sender waiting	Direct hand-off
Recv, nothing available	Park on <code>recvq</code>
Send, receiver waiting	Direct hand-off
Send, buffer space	Push buffer
Send, no space	Park on <code>sendq</code>
Op on closed	Recv: zero + <code>ok=false</code> ; Send: <b>panic</b>

```
v, ok := <-ch // ok false => closed and drained
close(ch) // idempotent close panics on second close
```

## Select Algorithm (Intuition)

For `select` with multiple cases:

1. Randomize case order (fairness).
2. Probe channels for a case that can proceed **immediately**.
3. If none and there is `default`, take default.
4. Otherwise, enqueue this G on **all** involved channels' wait queues, then park.
5. On wake, dequeue from other channels; complete the winning case only.

Implications:

- There is **no priority** among cases — do not assume left-to-right wins.
- A busy channel can starve another statistically; redesign if fairness is a product requirement.
- `select` with one case quite the same codegen as bare op, but same semantics.

## Close Semantics

```
close
|
+--> fail future sends (panic)
+--> wake all recvers (they get zero values)
+--> drained buffer still readable until empty
```

Only the sender side should close (by convention). Multiple closers require a single closer protocol.

## Common Failure Modes

Bug	Symptom
Send on closed	Panic
Never closed, receivers blocked	Goroutine leak
Unbuffered send without recv	Deadlock
Select without default in tight loop	100% wake churn if misused with timers
Timer.C not drained after Stop (older patterns)	Rare wake bugs — prefer <code>time.After</code> carefully in loops

## Cost Model

- Channel ops take a **runtime lock** and may park the G — fine for coordination, expensive as a per-byte I/O bus.
- For huge fan-in of tiny messages, batch or use a shared structure under a mutex (measure).

## Experiment

```
go mod init example
go run .
```

```
package main
```

```
import (
 "fmt"
 "time"
)
```

```
func main() {
 // Rendezvous
```

```

unbuf := make(chan string)
go func() {
 unbuf <- "ping"
}()
fmt.Println("unbuf", <-unbuf)

// Buffer absorbs one send
buf := make(chan int, 1)
buf <- 1
fmt.Println("buf", <-buf)

// Select fairness demo: both cases ready each iteration
counts := map[string]int{}
for i := 0; i < 1000; i++ {
 a := make(chan int, 1)
 b := make(chan int, 1)
 a <- 1
 b <- 2
 select {
 case <-a:
 counts["a"]++
 case <-b:
 counts["b"]++
 }
}
fmt.Println("select counts", counts)

// Close
ch := make(chan int, 2)
ch <- 7
close(ch)
v, ok := <-ch
fmt.Println("after close", v, ok)
v, ok = <-ch
fmt.Println("drained", v, ok)

// Default non-block
select {
case <-time.After(10 * time.Millisecond):
 fmt.Println("timer")
default:
 fmt.Println("default path")
}
}

```

**What to notice:** Both a and b win some iterations — not always a first. Closed channel receives are safe; sends would panic.

**Try next:** Deliberately leak a goroutine blocked on send; find it with `//go:debug tracebackancestors=1` or `pprof goroutine profile`.

## Related

- [Channel patterns](#)
- [Memory model](#)



## Interface Representation (eface / iface)

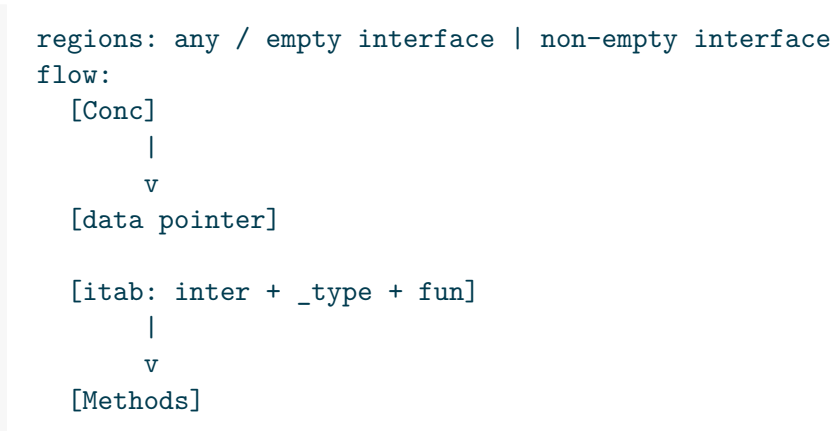
# Interface Representation (eface / iface)

## Overview

An interface value is **not** the concrete value alone. It is a small descriptor: **type metadata + data pointer** (with special cases). That design explains allocation on conversion, the famous nil-interface trap, and method-call cost.

Usage: [Interfaces](#).

## Diagram: eface vs iface



## Two Runtime Shapes

Kind	When	Conceptual fields
eface	Empty interface any / interface{}	_type *rtype, data unsafe.Pointer
iface	Non-empty interface	tab *itab, data unsafe.Pointer

itab caches the **mapping from interface method set → concrete methods** for a pair (interface type, concrete type). First conversion may populate itab; later calls reuse it.

```
iface
 tab ---> itab { inter, _type, fun[0..n] method ptrs }
 data ---> concrete value or pointer to it
```

## Dynamic Type and Dynamic Value

```
var i any = 3
// dynamic type: int
// dynamic value: 3
```

`i == nil` is true only when **both** type and value parts are unset. Assigning a typed nil sets the type part:

```
var p *int = nil
var i any = p
fmt.Println(i == nil) // false - type is *int, data is nil
```

This is the #1 production bug when returning **error**:

```
func f() error {
 var err *MyError = nil
 return err // returns non-nil error interface!
}
return nil // correct empty interface
```

## Conversion and Escape

Boxing a non-pointer into an interface often **allocates** so **data** can point at a heap copy (escape analysis decides). Hot loops that convert to **any** for logging or reflection pay for it.

```
go build -gcflags='-m' .
```

Look for **x escapes to heap** near interface conversions.

## Method Calls

Calling `i.M()`:

1. Load itab (or eface type).
2. Index method slot.
3. Call with concrete receiver in **data**.

Devirtualization / inlining may optimize some cases after SSA; do not assume every interface call is a virtual disaster — **measure**. Still, concrete types in hot paths are simpler for the compiler.

## Type Assert and Type Switch

```
v, ok := i.(T) // checks dynamic type
switch v := i.(type) { ... }
```

Failed assert without ok panics. Prefer comma-ok form at boundaries.

## Comparable Interfaces

Interfaces are comparable if dynamic types are comparable. Comparing interfaces compares dynamic type identity then values. Maps with **any** keys require comparable dynamic types at runtime or panic.

## Experiment

```
go mod init example
go run .

package main

import (
 "fmt"
 "unsafe"
)

type Stringer interface {
 String() string
}

type N int

func (n N) String() string { return fmt.Sprintf("N(%d)", n) }

func main() {
 // nil trap
 var p *N = nil
 var s Stringer = p
 var empty Stringer
 fmt.Println("typed nil iface == nil?", s == nil)
```

```

fmt.Println("empty iface == nil?", empty == nil)

// eface shape (illustrative sizes; do not depend on layout in prod code)
var a any = 42
var b any = p
fmt.Printf("any int: type-ish non-nil data non-nil? %v\n", a != nil)
fmt.Printf("any nil *N != nil? %v\n", b != nil)

// method call via iface
s = N(7)
fmt.Println("method", s.String())

// assert
if n, ok := s.(N); ok {
 fmt.Println("assert", n)
}

// show that interface is two words on 64-bit
fmt.Println("sizeof any", unsafe.Sizeof(a))
}

```

**Expected output (typical amd64/arm64):**

```

typed nil iface == nil? false
empty iface == nil? true
any int: type-ish non-nil data non-nil? true
any nil *N != nil? true
method N(7)
assert 7
sizeof any 16

```

**What to notice:** Interface is two machine words; typed nil is not interface nil.

**Try next:** Benchmark `fmt.Sprintf("%v", concrete)` vs logging with concrete fields — interface conversion shows up in alloc profiles.

## Related

- [Interface practices](#)
- [Type assertions](#)

## Slice and Map Internals

# Slice and Map Internals

## Overview

Slices and maps are the two most used runtime-backed structures. Misunderstanding headers and growth causes accidental sharing bugs, huge realloc copies, and production map panics.

Basics: [Arrays & slices](#), [Maps](#).

## Diagram: Slice header and map grow

```
regions: slice header | map
flow:
 [ptr]
 |
 v
 [Arr]

 [buckets] --load high--> [grow]
```

## Slice Header

A slice value is a small header (conceptually):

```
slice
 ptr -> underlying array
 len
 cap
```

Assignment copies the **header**, not the array:

```
a := []int{1, 2, 3}
b := a
b[0] = 9 // a[0] is also 9
```

append may allocate a new array when `len+need > cap`, then copy. Capacity growth is geometric (implementation detail; treat as amortized  $O(1)$  append).

```
s = append(s, x) // maybe new backing store; old aliases unchanged
```

## Full slice expression

```
t := s[i:j:k] // len=j-i, cap=k-i
```

Limits capacity so later `append` cannot overwrite the rest of the original array — critical when returning subslices from internal buffers.

## Nil vs empty

```
var s []int // nil, len0 cap0
t := []int{} // non-nil empty
u := make([]int, 0)
```

JSON encodes both as `[]`. Prefer one convention in APIs.

## Map Runtime Model

Maps are **hash tables** of buckets (plus overflow). Runtime may:

- Grow (reallocate buckets) when load is high
- Evacuate old buckets progressively

```
map header (hmap)
 count
 B // log2 buckets
 buckets
 oldbuckets // during grow
 ...
```

## Semantics that bite

1. **Not addressable elements** — cannot take `&m[k]` for map values in general.
2. **Iteration order is randomized** — never depend on it.
3. **Concurrent map read+write without sync panics** (fatal runtime error).
4. **Delete during range** is allowed for the current key; still no concurrent writers from other Gs.
5. **NaN float keys** are cursed — avoid float map keys.



## Growth cost

Large maps that grow under a request path create latency spikes. Pre-size when you know cardinality:

```
m := make(map[string]int, expected)
```

## Sharing and Aliasing Checklist

Action	Shares backing array?
b := a	Yes
b := a[i:j]	Yes
append that grows	No (new array for result)
slices.Clone / copy to new	No
maps.Clone	New map, same element values

When returning a slice from an internal buffer, **clone** if the buffer will be reused.

## Experiment

```
go mod init example
go run .

package main

import (
 "fmt"
 "unsafe"
)

func main() {
 // header size
 var s []int
 fmt.Println("sizeof slice header", unsafe.Sizeof(s))

 a := make([]int, 3, 6)
 a[0], a[1], a[2] = 1, 2, 3
 b := a[1:3]
 b[0] = 99
 fmt.Println("share", a) // a[1] changed

 // cap limit
```

```

c := a[0:2:2]
c = append(c, 7)
fmt.Println("full slice a after append c", a, "c", c)

// growth
var d []int
for i := 0; i < 10; i++ {
 d = append(d, i)
 fmt.Printf("len=%d cap=%d\n", len(d), cap(d))
}

// map pre-size vs not (just API demo)
m := make(map[int]int, 100)
for i := 0; i < 100; i++ {
 m[i] = i * i
}
fmt.Println("map len", len(m))

// iteration order differs across runs (print a few)
keys := 0
for k := range m {
 keys++
 if keys <= 3 {
 fmt.Println("key sample", k)
 }
}
}

```

**What to notice:** Subslice mutation mutates the original; capacity growth jumps; map key order is not sorted.

**Try next:** Force concurrent map write with two goroutines (expect fatal); fix with `sync.Mutex` or `sync.Map` only if appropriate.

## Related

- [Mutability](#)
- [slices/maps helpers](#)

## Defer, Panic, and Stack Unwind

# Defer, Panic, and Stack Unwind

## Overview

`defer` schedules a call to run when the surrounding function returns — normally or via `panic`. Modern Go **open-codes** many defers (cheap), but loops that defer still accumulate work until return. Panic unwinds the stack running defers until `recover` or process crash.

Errors: [Panic and recover](#).

## Diagram: Panic unwind

```
panic(value)

 v
run defers LIFO (this frame)

 v
caller frames ... until recover

 recover in defer stop, resume
 none crash + stacks
```

## Defer Semantics

```
func f() {
 defer fmt.Println("second")
 defer fmt.Println("first") // LIFO
 fmt.Println("body")
}
// body, first, second
```

## Arguments evaluated immediately

```
func g() {
 x := 1
 defer fmt.Println(x) // prints 1 even if x changes later
 x = 2
}
```

To see final values, defer a closure:

```
defer func() { fmt.Println(x) }()
```

## Named results

Deferred closures can assign named return values — used carefully in middleware-style wrappers:

```
func wrap() (err error) {
 defer func() {
 if err != nil {
 err = fmt.Errorf("wrap: %w", err)
 }
 }()
 return do()
}
```

## Cost Model

Pattern	Cost
Few defers per function (open-coded)	Very low
defer inside hot loop	Registers many defers; prefer explicit unlock or scoped helper
defer mu.Unlock() after Lock	Idiomatic and fine outside mega-hot paths

```
// Prefer for tight loops:
mu.Lock()
// critical
mu.Unlock()

// Prefer for error-prone multi-return:
mu.Lock()
defer mu.Unlock()
```

## Panic Unwind

```
panic(value)
-> run defers in current function (LIFO)
-> pop frame
-> run defers in caller
-> ...
-> recover in a deferred call? stop unwind, return
-> else crash process (dump stacks)
```

recover only works **directly inside a deferred function** during that panic:

```
defer func() {
 if r := recover(); r != nil {
 // log, convert to error
 }
}()
```

### What recover cannot do

- Fix corrupted program state magically
- Replace typed API errors as the default control flow
- Catch panics in **other** goroutines — each G panics independently

```
go func() {
 defer func() { recover() }() // only protects this goroutine
 mayPanic()
}()
```

## Stack Growth

Goroutine stacks start small and **grow/shrink** (copy stack). Pointers into stack frames are invisible to Go code safely — the compiler/runtime rewrites them. Do not pass pointers from Go stack to C without pinning rules (cgo).

## Experiment

```
go mod init example
go run .
```

```

package main

import "fmt"

func deferOrder() {
 defer fmt.Println("D1")
 defer fmt.Println("D2")
 fmt.Println("body")
}

func deferArgs() {
 x := 1
 defer fmt.Println("arg", x)
 defer func() { fmt.Println("closure", x) }()
 x = 9
}

func recoverDemo() (err error) {
 defer func() {
 if r := recover(); r != nil {
 err = fmt.Errorf("panic: %v", r)
 }
 }()
 panic("boom")
}

func main() {
 deferOrder()
 deferArgs()
 fmt.Println("recovered", recoverDemo())
}

```

### Expected output:

```

body
D2
D1
closure 9
arg 1
recovered panic: boom

```

**What to notice:** LIFO order; args vs closure capture; panic converted to error only with defer+recover.

**Try next:** Benchmark loop with `defer unlock` vs paired unlock for a million iterations — see if it matters in your Go version.

## Related

- [Panic recover](#)
- [Must pattern](#)



# Generics Implementation Deep Dive

# Generics Implementation Deep Dive

## Overview

Go generics (1.18+) are implemented with a hybrid strategy often described as **dictionaries + partial stenciling** (GC shape based). You do not need every compiler paper — you need the cost model: when generics are zero-cost abstraction vs when they introduce runtime dictionary calls or more code size.

Usage: [Generics intro](#) through [Idiomatic generics](#).

## Diagram: Generic call shapes

```
generic source
 v
typecheck + constraints
 v
GC shape groups + dictionaries
 v
shared/specialized machine code
```

The diagram illustrates the flow of generic call shapes. It starts with 'generic source', which leads to 'typecheck + constraints'. This then leads to 'GC shape groups + dictionaries', which finally leads to 'shared/specialized machine code'. Each step is connected by a downward arrow, indicating a sequential process.

## What the Compiler Sees

```
func Max[T cmp.Ordered](a, b T) T {
 if a > b {
 return a
 }
 return b
}
```

Calls with different type arguments may share generated code when the **GC shape** matches (same underlying representation for pointers, etc.), parameterized by a **dictionary** describing operations (comparison, conversion, ...).

```

source generic
 |
 v
type check + capture constraints
 |
 v
shape grouping (pointer-ish shapes share)
 |
 v
code + dictionary for operations on T

```

## Cost Dimensions

Dimension	Effect
Code size	Too many distinct shapes / specialized methods → binary growth
Call cost	Dictionary-indirect calls can inhibit inlining
Allocation	Generic APIs that box to <b>any</b> lose the point
Readability	Complex constraint graphs hurt more than they help

Generics excel for:

- Containers and algorithms over `~[]T`, maps, comparables
- Removing `interface{}` while keeping type checks

Generics struggle when:

- Every method needs unique runtime behavior per type without structure
- You force abstraction that a plain function + concrete types would inline better

## Constraints Are Interfaces

```

type Number interface {
 ~int | ~int64 | ~float64
}

```

Constraint satisfaction is compile-time. Method sets on type parameters follow the constraint's method set.

## Instantiation

```
f := Max[int] // function value of specialized shape
v := Max(1, 2) // inference
```

Type inference fails when the compiler cannot unify arguments — add explicit type args rather than fighting it.

## Interaction With Interfaces

```
func Print[T fmt.Stringer](v T) {
 fmt.Println(v.String())
}
```

Prefer **type parameters** when callers should keep static types through the call graph; prefer **interfaces** when you need heterogeneous collections or runtime plugin style.

## Avoid These Footguns

1. **Generic + reflect by default** — pays both complexity taxes.
2. **Over-abstracting one-use helpers** — a function with `int` is fine.
3. **Exporting huge generic surface** — every instantiation pressure multiplies for users.

## Experiment

```
go mod init example
go run .
go build -gcflags='-m' . 2>&1 | head -40
```

```
package main

import (
 "cmp"
 "fmt"
)

func Max[T cmp.Ordered](a, b T) T {
 if a > b {
 return a
 }
 return b
}
```

```

}

type Pair[K comparable, V any] struct {
 K K
 V V
}

func (p Pair[K, V]) String() string {
 return fmt.Sprintf("%v=%v", p.K, p.V)
}

func main() {
 fmt.Println(Max(3, 5))
 fmt.Println(Max(2.5, 1.2))
 fmt.Println(Pair[string, int]{K: "n", V: 1})
}

```

**What to notice:** One source definition serves multiple shapes. `-gcflags=-m` may show inlining decisions around simple generic functions.

**Try next:** Compare assembly of `Max[int]` vs a non-generic `maxInt` with `go build -gcflags=-S` and see if dictionary calls appear in your version.

## Related

- [Generic functions](#)
- [Generic types](#)
- [Compiler & SSA](#)

## **GC Internals and Pacer**

# GC Internals and Pacer

## Overview

Go's GC is a **concurrent, non-moving, mark-sweep** collector with a **pacer** that decides when to start marking so heap growth stays near a target. Go 1.26's **Green Tea GC** improves scan locality; the user-facing knobs (`GOGC`, `GOMEMLIMIT`) remain the control plane.

Practical tuning: [Garbage collection](#), [Allocations & GC](#).

Companion depth: [Internals for Interns — The Garbage Collector](#) (GreenTea / mark phase notes).

## Diagram: GC cycle



## Heap Building Blocks

```
arena / pages
 -> spans (size classes for small objects)
 -> objects
large objects: dedicated spans
```

Allocators (per-P caches) reduce cross-thread contention. When caches miss, they pull from central free lists / the heap.

## Tri-color intuition

```
white (unmarked) reach grey (to-scan) scan kids black
 not reached sweep reclaim
```

Write barriers keep the invariant safe while mutators run during concurrent mark.

## Mark and Sweep Phases

```
1. Mark setup STW (short) - enable write barrier, root prep
2. Concurrent mark - workers + mutator assists
3. Mark termination STW (short)
4. Concurrent sweep - reclaim unmarked objects by span
```

## Write barrier

While marking, pointer writes are instrumented so the GC does not lose objects. Barriers add a small tax on pointer-heavy code — one reason reducing pointer count (or packing) helps.

## Mark assist

If the mutator allocates faster than markers can scan, the allocating G **helps mark**. This couples allocation rate to GC CPU and shows up as latency on the allocating path — not only in a GC thread.

## Pacer Goals

The pacer aims roughly:



```
live heap after GC * (1 + GOGC/100) heap size that triggers next GC
```

- **GOGC=100** (default): next GC when heap roughly doubles live size.
- Lower GOGC → more frequent GC, less peak memory.
- Higher GOGC → less GC CPU, more memory.

## GOMEMLIMIT

Soft limit on Go-managed memory. When approaching the limit, GC becomes more aggressive (and may return memory to the OS more eagerly depending on version/settings). Use with container memory limits to reduce OOM-kills:

```
GOMEMLIMIT=512MiB GOGC=100 ./service
```

Prefer **limit + reasonable GOGC** over **GOGC=off** fantasies.

## Green Tea GC (1.26+)

Span-aware scanning improves cache locality for pointer-rich heaps. Opt-out for experiments:

```
GOEXPERIMENT=nogreenteagc ./service
```

Validate with your workload’s p95 and CPU profiles — do not assume every service wins equally.

## Reading GC Telemetry

```
var m runtime.MemStats
runtime.ReadMemStats(&m)
// m.NumGC, GCCPUPFraction, HeapAlloc, HeapInuse, NextGC, PauseTotalNs
```

Or metrics:

```
go_gc_duration_seconds
go_memstats_heap_alloc_bytes
go_memstats_next_gc_bytes
```

## Latency pattern

Periodic p95 spikes aligned with NumGC increases often mean **assist or STW**, not “Go is slow.” Fixes: fewer allocs, smaller pointer graphs, raise memory budget, or smooth allocation rate.

## Design Rules

1. **Allocate less** before tuning GOGC.
2. **Reuse buffers** ([]byte pools) on hot paths.
3. **Avoid pointer-heavy tiny objects** in huge graphs when possible (flatten).
4. **Set GOMEMLIMIT** in containers.
5. **Profile** — pprof heap + trace GC events.

## Experiment

```
go mod init example
go run .
```

```
package main

import (
 "fmt"
 "runtime"
 "time"
)

func stats(label string) {
 var m runtime.MemStats
 runtime.ReadMemStats(&m)
 fmt.Printf("%s alloc=%dKiB nextGC=%dKiB numGC=%d gccpu=%.3f\n",
 label,
 m.Alloc/1024,
 m.NextGC/1024,
 m.NumGC,
 m.GCCPUFraction,
)
}

func main() {
 stats("start")
 keep := make([][]byte, 0, 1000)
 for i := 0; i < 1000; i++ {
 keep = append(keep, make([]byte, 32*1024))
 }
 stats("after alloc")
 keep = nil
 runtime.GC()
 time.Sleep(50 * time.Millisecond)
 stats("after GC")
}
```

**What to notice:** `NextGC` rises with live heap; forced GC drops alloc; `GCCPUFraction` is a long-run average (not instant).

**Try next:** Run the same under `GOGC=50` vs `GOGC=200` and compare `NumGC` after identical work.

## Related

- [GC chapter](#)
- [Escape analysis](#)

## **Compiler, SSA, Inlining, and PGO**

# Compiler, SSA, Inlining, and PGO

## Overview

The Go compiler turns packages into machine code through a pipeline: parse → typecheck → mid-end optimizations (SSA) → architecture lowering → assemble. Knowing the pipeline tells you **which knobs are real** (-gcflags, inlining, escape, PGO) versus cargo cult.

## Diagram: Compiler pipeline

```
source → typecheck → IR → SSA → machine code → link

 PGO profile (hot edges)
 -gcflags=-m shows escape / inline decisions
```

## Pipeline (Simplified)

```
source .go
 -> parse / syntax
 -> typecheck
 -> IR build
 -> SSA optimize (prove, rewrite, deadcode, nilcheck elim, ...)
 -> lower to arch
 -> encode object
link -> executable
```

Escape analysis runs early enough to decide stack vs heap; inlining expands call sites so later passes see a larger region.

## Essential Flags

```
Escape and inline decisions
go build -gcflags='-m' .

More detail
```

```

go build -gcflags='-m -m' .

Disable optimizations (debug)
go build -gcflags='-N -l' .

Assembly listing
go build -gcflags='-S' . 2> asm.txt

Shared / large projects: apply to specific package
go build -gcflags='all=-m' ./...

```

Flag	Meaning
-m	Print optimization decisions
-l	Disable inlining (with -N for debug)
-N	Disable optimizations
-B	Disable bounds checks (unsafe; special cases)

## Inlining

Small functions may inline into callers, enabling further constant folding and escape improvements.

```

//go:noinline
func sink(x int) int { return x }

```

Use `//go:noinline` only for benchmarks that must measure call cost. Prefer clear small functions; the compiler is good at inlining them.

Mid-stack inlining and more aggressive budgets improved across versions — re-benchmark when upgrading major Go releases.

## Bounds Check Elimination (BCE)

SSA proves many index checks redundant:

```

for i := range s {
 s[i] = i // often single check or proved safe
}

```

Idioms that help BCE: `for i := range s`, length hoisting the compiler can see, slice expressions with proven bounds. Do not micro-obfuscate for BCE without profiles.

## Profile-Guided Optimization (PGO)

From Go 1.20+ (improved since):

```
1) Collect CPU profile from production-like load
2) Build with profile
go build -pgo=default.pgo -o app .
or auto:
go build -pgo=auto .
```

PGO feeds the compiler **hot call edges** so inlining/devirtualization favor real paths. Gains of a few percent are common; bigger when interfaces dominate.

Treat PGO profiles as **build inputs** — keep them reasonably fresh.

## What the Compiler Will Not Do

- Fix algorithmic  $O(n^2)$
- Remove channel/mutex coordination costs
- Make reflect cheap
- Cross-package miracle opts without inlining visibility

## Experiment

```
go mod init example
go build -gcflags='-m' . 2>&1 | sed -n '1,40p'
```

```
package main

import "fmt"

func add(a, b int) int { return a + b }

func main() {
 x := 3
 y := add(x, 4)
 fmt.Println(y)
}
```

Typical messages include `can inline add` and whether `x` escapes (usually does via `fmt`).

**Try next:** Compare `go test -bench=.` with and without `-pgo` after collecting a profile from the benchmark binary.

## Related

- [Escape analysis](#)
- [Performance tooling](#)



## **Linking, Build Modes, and Race ABI**

# Linking, Build Modes, and Race ABI

## Overview

After compilation, the **linker** stitches objects, resolves symbols, and emits an executable or library. Build modes and sanitizers change the ABI and cost model — important for plugins, cgo, and race-enabled CI.

## Diagram: Build modes

```
go build
 exe (default)
 pie
 c-shared / c-archive
 plugin (limited platforms)

go build -race instrumented memory + HB
```

## Default Build

```
go build -o app .
go install ./cmd/app
```

Static-ish binaries are common on Linux with pure Go (no cgo). The Go runtime issues syscalls itself, so `CGO_ENABLED=0` produces a fully static ELF with no `libc.so`. Enable cgo and **glibc vs musl** comes back: a Ubuntu-linked binary fails on Alpine because the ELF interpreter (`/lib64/ld-linux-x86-64.so.2`) is missing. See [Go as a Systems Language](#).

```
go env CGO_ENABLED
CGO_ENABLED=0 go build -o app . # force pure Go when possible
file app && ldd app # Linux: expect "statically linked"
```

## Build Modes

```
go help buildmode
```

Mode	Use
exe	Default executable
c-shared	C shared library from Go
c-archive	C archive
plugin	Go plugin (limited platforms; fragile for prod)
pie	Position-independent executable

```
go build -buildmode=pie -o app .
```

**Plugins** sound attractive and usually disappoint: version skew with main binary, limited platforms, hard ops story. Prefer explicit interfaces + separate processes.

## Build Tags and Constraints

```
//go:build linux && amd64
```

```
package onlylinux
```

```
go build -tags=debug,integration
```

Use tags for optional integration tests and platform files (`foo_windows.go`), not for sprawling feature flags that should be config.

## Linker Flags

```
go build -ldflags='-s -w' -o app . # strip symbols / DWARF
go build -ldflags='-X main.version=1.2.3' .
```

`-s -w` smaller binaries, harder debug. Keep symbols in staging if you need crash analysis.

## Race Detector

```
go test -race ./...
go build -race -o app-race .
```

Race builds:

- Use a different memory allocator instrumentation
- Cost **CPU and memory** (often 2–10×)
- Are for testing, not production traffic

Shadow memory tracks happens-before; if it reports a race, trust it.

## cgo Boundary (Preview)

cgo inserts C ABI calls and can:

- Prevent easy cross-compilation
- Block some optimizations
- Require `CGO_ENABLED=1` and a toolchain

Deep systems: [cgo](#). Prefer pure Go drivers when shipping multi-arch binaries.

## Reproducible Builds

```
go build -trimpath -ldflags='-buildid=' -o app .
```

Trim paths remove local directory noise from binaries; useful for supply-chain hygiene.

## Experiment

```
go mod init example
go build -o /tmp/app-normal .
go build -ldflags='-s -w' -o /tmp/app-strip .
ls -la /tmp/app-normal /tmp/app-strip
```

```
go test -race -c -o /tmp/app.test .
file sizes differ; race binary is larger
ls -la /tmp/app.test
```

```
package main

import "fmt"

var version = "dev"

func main() {
 fmt.Println("version", version)
}
```

```
go build -ldflags='-X main.version=1.0.0' -o /tmp/app-ver .
/tmp/app-ver
```

**What to notice:** Strip and race change binary size dramatically; `-X` injects version at link time.

**Try next:** Build with `CGO_ENABLED=0` and confirm `ldd` (Linux) shows no unexpected libc deps for a pure Go net/http binary.

## Related

- [Building & releasing](#)
- [CI/CD security](#)

## **Reflect Internals and Cost**

# Reflect Internals and Cost

## Overview

Package `reflect` exposes runtime type metadata (`rtype`) and dynamic values (`Value`). It powers encoding/json, dependency injection frameworks, and ORMs — and it is a common production hotspot when used on every request without caching.

Usage intro: [Reflection](#).

## Diagram: Reflect path

```
flow:
 [V]
 |
 v
 [Flags]
```

## Core Types

```
t := reflect.TypeOf(x) // runtime type descriptor (nil-safe only if x typed)
v := reflect.ValueOf(x) // dynamic value
```

API	Role
Type	Size, kind, fields, methods — often cacheable
Value	Read/write/call — easy to allocate
Kind	Int, Struct, Slice, ...
Interface	Box back to any

## What Metadata Lives Where

```
interface / concrete
 |
 v
 rtype (uncommonType, struct fields, method table...)
 |
 Value flags: addressable? settability? indir?
```

`ValueOf` on a non-pointer often yields a **non-addressable** copy — `Set` fails. Pass pointers for mutation:

```
rv := reflect.ValueOf(&x).Elem()
rv.SetInt(3)
```

## Allocations and Interface Boxing

Common alloc sources:

1. `Value.Interface()` boxing
2. `ValueOf` of small values that escape
3. Building `[]reflect.Value` for `Call`
4. Pathological use inside tight loops without type caches

```
// Cache field indexes by type
var cache sync.Map // reflect.Type -> []int indices
```

`encoding/json` caches struct field info per type — copy that idea.

## Method Calls via Reflect

```
m := reflect.ValueOf(obj).MethodByName("Do")
results := m.Call(nil)
```

Much slower than a direct call or interface method. Prefer interfaces for hot polymorphic paths; use reflect for glue and frameworks.

## Unsafe Adjacent

`reflect.Type` and `unsafe.Pointer` interactions exist for advanced serializers. Prefer well-tested libraries over hand-rolled unsafe reflection unless you accept maintenance cost.

See [unsafe](#).



## When Reflect Is Justified

Use	Verdict
One-time config decode	Fine
Per-request JSON (stdlib)	Fine — heavily optimized
Per-request generic mapper over large structs	Cache or codegen (go generate, json struct tags already)
Game loop / packet parse	Avoid; use codegen or concrete code

## Experiment

```
go mod init example
go test -bench=. -benchmem

package main

import (
 "reflect"
 "testing"
)

type S struct {
 A int
 B string
}

func direct(s *S) int { return s.A }

func viaReflect(s *S) int {
 return int(reflect.ValueOf(s).Elem().FieldByName("A").Int())
}

func BenchmarkDirect(b *testing.B) {
 s := &S{A: 42}
 for i := 0; i < b.N; i++ {
 _ = direct(s)
 }
}

func BenchmarkReflect(b *testing.B) {
 s := &S{A: 42}
 for i := 0; i < b.N; i++ {
 _ = viaReflect(s)
 }
}
```

```

}

// Keep a main so go run works if needed
func main() {}

```

Put benchmarks in `main_test.go` or rename package for pure test module:

```

simpler: single file package reflectbench_test

```

Alternatively run:

```

// file: cost_test.go
package cost

import (
 "reflect"
 "testing"
)

type S struct{ A int }

func BenchmarkFieldDirect(b *testing.B) {
 s := S{A: 1}
 for i := 0; i < b.N; i++ {
 _ = s.A
 }
}

func BenchmarkFieldReflect(b *testing.B) {
 s := S{A: 1}
 for i := 0; i < b.N; i++ {
 _ = reflect.ValueOf(s).Field(0).Int()
 }
}

mkdir /tmp/reflect-cost && cd /tmp/reflect-cost
go mod init example
write cost_test.go as above
go test -bench=. -benchmem

```

**What to notice:** Reflect field access is often **orders of magnitude** slower and may allocate depending on path.

**Try next:** Cache `reflect.Type` and field index; re-benchmark `FieldByName` vs `Field(i)`.

## Related

- [Reflection](#)
- [Codegen](#)

## **Modules, MVS, and Toolchain**

# Modules, MVS, and Toolchain

## Overview

Go modules are a **versioned dependency graph** with a precise selection algorithm: **Minimal Version Selection (MVS)**. Treat `go.mod` / `go.sum` as production config — not editor noise.

Basics: [Dependency management](#), [Workspaces](#).

## Diagram: MVS selection

```
flow:
 [Max]
 |
 v
 [Build]
```

## Core Files

File	Role
<code>go.mod</code>	Module path, Go version, requires, replaces, excludes, toolchain
<code>go.sum</code>	Cryptographic checksums of module contents
<code>vendor/</code>	Optional frozen source trees

```
go mod tidy # align requires with imports
go mod download
go mod verify
go mod graph
```

## Minimal Version Selection

Unlike many package managers that pull “newest compatible,” MVS picks the **minimum version that satisfies all constraints** in the build list.

```
Your module requires A v1.2
B requires A v1.5
C requires A v1.3

Selected A = v1.5 (max of minima)
```

Properties:

- **Reproducible** given the same `go.mod` build list
- **Not** “always latest”
- Upgrades are **explicit** (`go get pkg@vX`)

```
go list -m all
go list -m -versions github.com/some/dep
go get example.com/dep@v1.4.2
go get example.com/dep@none # drop
```

## The Build List

`go list -m all` shows the selected set. Understanding diamond dependencies is mostly reading that list and `go mod graph | grep dep`.

## replace and exclude

```
replace example.com/fork => ../fork

exclude example.com/bad v1.0.0
```

`replace` is powerful for local development; **avoid permanent replaces** in published modules — consumers inherit pain. Prefer releasing fixed versions.

## Retracts

Module authors can **retract** broken versions so `go get` avoids them. If you cannot upgrade, pin carefully and track advisories.

## Toolchain Directive

Modern Go can select a **toolchain** version:

```
go 1.27
toolchain go1.27.1
```

```
go env GOTOOLCHAIN # auto | local | path | version
```

CI should pin or intentionally use **auto** with care so builds do not silently jump compilers mid-incident.

## Vendoring

```
go mod vendor
go build -mod=vendor
```

Pros: offline builds, airgapped, reviewable third-party diffs. Cons: repo size, merge noise. Good for enterprises and release branches.

## Checksums and Proxies

```
GOPROXY (default proxy.golang.org)
GOSUMDB (sum.golang.org)
GOPRIVATE / GONOSUMDB for private modules
```

```
export GOPRIVATE=github.com/myorg/*
```

Private modules need auth to the host; sumdb is skipped for **GOPRIVATE**.

## Diagnosis Playbook

Symptom	Action
“missing go.sum entry”	<code>go mod tidy</code>
Unexpected old dep	<code>go mod graph</code> + who requires it
Local replace forgotten	Search <code>replace</code> in <code>go.mod</code>
CI != laptop	Print <code>go version</code> , <code>go env</code> , <code>go list -m all</code>
Ambiguous import	Module path vs directory mismatch

## Experiment

```
mkdir /tmp/mvs-demo && cd /tmp/mvs-demo
go mod init example.com/demo
go get rsc.io/quote@v1.5.2
go list -m all
go mod graph | head
cat go.mod
go mod tidy
```

**What to notice:** `go.mod` records the selected versions; `go.sum` grows with hashes; `graph` shows edges that forced versions upward.

**Try next:** Introduce a `replace` to a local stub module and build; then remove `replace` and fetch the real version.

## Related

- [Dependency management](#)
- [Workspaces](#)
- [Building & releasing](#)



## Netpoller and Network I/O

# Netpoller and Network I/O

## Overview

Blocking on a socket in Go rarely blocks an OS thread the way a naive `read()` would in a 1:1 thread model. The **netpoller** integrates with the scheduler: a G waiting on network readiness parks, its M can run other work, and the poller wakes the G when the FD is ready.

This is why Go servers scale to huge connection counts with modest `GOMAXPROCS`.

Companion: [Internals for Interns — Network Poller](#) (who calls `netpoll` from `findrunnable` / `sysmon`).

## Diagram: Netpoll park

```
sequence (top → bottom):
 actors: goroutine, runtime, epoll/kqueue/IOCP
 goroutine --> runtime : Read would block
 runtime --> goroutine : park netpoll wait
 runtime --> epoll/kqueue/IOCP : register FD
 epoll/kqueue/IOCP --> runtime : readable
 runtime --> goroutine : goready
 goroutine --> goroutine : Read completes
```

## Mental Model

```
G calls conn.Read
-> runtime sees would-block
-> G parks on netpoll wait
-> M may run other Gs
...
kernel marks FD readable
-> netpoller (epoll/kqueue/IOCP) notices
-> G becomes runnable
-> Read completes
```

Platform backends:

OS	Mechanism
Linux	epoll
BSD/macOS	kqueue
Windows	IOCP

## Deadlines vs Poller

`SetDeadline` / `context` cancellation arm timers that **also** wake blocked network ops. Without deadlines, a stuck peer can pin a G forever (goroutine + buffer memory).

```
_ = conn.SetReadDeadline(time.Now().Add(30 * time.Second))
// or
ctx, cancel := context.WithTimeout(ctx, 30*time.Second)
defer cancel()
// http and many APIs use Request.Context()
```

## Interaction With Syscalls

Not every I/O goes through netpoll. File I/O and some cgo paths still block Ms. Network sockets registered with the poller are the sweet spot for massive concurrency.

Go 1.26 simplified how **P** tracks syscall state (derive from G instead of a dedicated P-syscall state), which helps syscall/cgo-heavy paths. See [cgo & scheduler](#).

## Zero-Copy Adjacent

`io.Copy` between sockets may use `splice/sendfile` on Linux when types allow — poller still owns readiness. TLS usually **cannot** sendfile the cleartext path the same way. See [zero-copy I/O](#).

## Diagnosis

Symptom	Check
Goroutines stuck in <code>net.Read</code>	Deadlines? peer hang?
Threads exploding	Blocking syscalls / DNS / cgo, not poller waits
High latency, low CPU	Downstream wait; trace shows park on poll

```
go tool pprof http://localhost:6060/debug/pprof/goroutine
look for runtime.netpollblock, internal/poll.runtime_pollWait
```

## Experiment

```
go mod init example
go run .

package main

import (
 "fmt"
 "io"
 "net"
 "time"
)

func main() {
 ln, err := net.Listen("tcp", "127.0.0.1:0")
 if err != nil {
 panic(err)
 }
 defer ln.Close()

 go func() {
 c, err := ln.Accept()
 if err != nil {
 return
 }
 defer c.Close()
 time.Sleep(50 * time.Millisecond)
 _, _ = c.Write([]byte("hello"))
 }()

 c, err := net.Dial("tcp", ln.Addr().String())
 if err != nil {
 panic(err)
 }
 defer c.Close()
 _ = c.SetReadDeadline(time.Now().Add(time.Second))
 buf := make([]byte, 16)
 n, err := c.Read(buf)
 fmt.Printf("read %q err=%v\n", buf[:n], err)

 // deadline fire
 _ = c.SetReadDeadline(time.Now().Add(20 * time.Millisecond))
 _, err = c.Read(buf)
 fmt.Printf("timeout? %v\n", err)
 _, _ = io.WriteString(c, "x")
}
```

```
}
```

**What to notice:** Read waits without spinning a CPU core; deadlines surface as errors, not hangs.

**Try next:** Open 10k localhost connections with short deadlines; watch RSS and goroutine count stay manageable.

## **Timers, Tickers, and time.After Pitfalls**

# Timers, Tickers, and time.After Pitfalls

## Overview

Timers are a frequent source of **subtle leaks and wakeups** in hot paths. The runtime keeps a **timer heap** (per-P structures in modern Go). Stopping and resetting correctly matters as much as creating timers.

## Diagram: Timer heap cost

```
flow:
 [Heap]
 |
 v
 [GC]
```

## APIs

API	Use
<code>time.NewTimer</code>	One-shot; must <b>Stop/Reset</b> carefully
<code>time.NewTicker</code>	Periodic; always <b>Stop</b>
<code>time.After</code>	Convenient; <b>allocates a new timer</b> each call
<code>time.AfterFunc</code>	Fire a function on a timer thread-ish path

## The Classic Pitfall

```
// BAD in a busy loop / per-request select
select {
case <-ch:
case <-time.After(time.Second): // new timer every iteration
}
```

Each `time.After` creates a timer that lives until it fires (or is GC'd after fire). Under load this is allocation + timer-heap churn.

## Prefer:

```
t := time.NewTimer(time.Second)
defer t.Stop()
select {
case <-ch:
 if !t.Stop() {
 select {
 case <-t.C:
 default:
 }
 }
case <-t.C:
}
```

Or reuse with `Reset` after successful `Stop`/drain per current docs for your Go version.

## Tickers

```
tk := time.NewTicker(100 * time.Millisecond)
defer tk.Stop()
for {
 select {
 case <-ctx.Done():
 return
 case <-tk.C:
 // work
 }
}
```

A stopped ticker is not restarted with `Reset` in older Go the same way as timers — check version docs.

## Runtime View

Timers are not free threads. They are heap entries processed by the runtime; expired timers make Gs runnable. Huge numbers of far-future timers cost memory; huge numbers of near-term timers cost CPU in the timer processor.

## Context Timeouts

`context.WithTimeout` uses timers under the hood. Prefer **one timeout at the edge** and derive children, rather than stacking independent `After` calls at every layer without budget math.



## Experiment

```
go test -bench=. -benchmem

package timers_test

import (
 "testing"
 "time"
)

func BenchmarkTimeAfter(b *testing.B) {
 ch := make(chan struct{})
 for i := 0; i < b.N; i++ {
 select {
 case <-ch:
 case <-time.After(time.Hour):
 default:
 }
 }
}

func BenchmarkTimerReuse(b *testing.B) {
 t := time.NewTimer(time.Hour)
 defer t.Stop()
 ch := make(chan struct{})
 for i := 0; i < b.N; i++ {
 if !t.Stop() {
 select {
 case <-t.C:
 default:
 }
 }
 t.Reset(time.Hour)
 select {
 case <-ch:
 case <-t.C:
 default:
 }
 }
}
```

**What to notice:** `time.After` in a loop allocates; reuse patterns dominate under `-benchmem`.

**Try next:** Find `time.After` in a hot select in your codebase with `rg 'time\\.After'`.

## **Stack Growth and Copying**

# Stack Growth and Copying

## Overview

Each goroutine starts with a **small stack** (on the order of a few KB historically; exact size is version-dependent) and **grows** by allocating a larger stack and **copying** live frames when needed. Stacks also shrink when underutilized.

This design makes millions of Gs feasible — you do not pre-reserve 1–8 MiB per goroutine like many OS threads.

## Diagram: Stack growth

```
G stack nearly full

 v
morestack → larger stack

 v
copy frames + fix pointers → resume
```

## Growth Path

```
G running, stack nearly full
-> morestack
-> allocate larger stack
-> copy stack contents
-> adjust pointers into stack (compiler/runtime)
-> resume
```

Go forbids taking the address of a stack variable and using it after the frame dies; the compiler and escape analysis keep GC and stack copy correct.

## Why Deep Recursion Hurts

Deep recursion causes repeated growth and large stacks:

```
func sum(n int) int {
 if n == 0 {
 return 0
 }
 return n + sum(n-1) // grows stack with depth
}
```

Prefer iterative algorithms for unbounded depth.

## cgo and Stacks

cgo may switch to larger **system stacks** for C execution. Crossing Go C frequently is expensive (scheduler + stack). See [cgo transitions](#).

## Debugging Stacks

```
GOTRACEBACK=all ./app
// panic dumps all Gs
```

```
buf := make([]byte, 1<<16)
n := runtime.Stack(buf, true) // all goroutines if true
fmt.Printf("%s", buf[:n])
```

## Nosplit

```
//go:nosplit
func tight() { /* cannot grow stack; for runtime/asm constraints */ }
```

User code almost never needs `//go:nosplit`. Misuse panics if the frame does not fit.

## Experiment

```
go mod init example
go run .
```

```
package main
```

```
import (
 "fmt"
```

```

 "runtime"
)

func depth(n int, max int) int {
 if n == max {
 var st runtime.StackRecord
 // approximate: print goroutine stack size indirectly via Stack
 buf := make([]byte, 64*1024)
 k := runtime.Stack(buf, false)
 fmt.Printf("at depth %d stack dump bytes=%d\n", n, k)
 return n
 }
 return depth(n+1, max)
}

func main() {
 fmt.Println("GOMAXPROCS", runtime.GOMAXPROCS(0))
 _ = depth(0, 200)
}

```

**What to notice:** Deep call chains produce large stack dumps; production code should bound recursion and prefer workers with small frames.

**Try next:** Compare a recursive tree walk vs explicit stack slice for a large tree — RSS and latency under load.

## **Finalizers and runtime.AddCleanup**

# Finalizers and runtime.AddCleanup

## Overview

Go is GC-managed, but some resources are **outside the heap** (OS FDs, cgo memory). Historically `runtime.SetFinalizer` ran a function when an object became unreachable. Modern Go prefers `runtime.AddCleanup` (and explicit `Close`) because finalizers are subtle and easy to get wrong.

## Diagram: Lifecycle preference

```
flow:
 [Backstop] --timing not guaranteed--> [Explicit]
```

## Prefer Explicit Lifecycle

```
f, err := os.Open(path)
if err != nil {
 return err
}
defer f.Close() // always first choice
```

Finalizers are a **backstop**, not a primary API.

## SetFinalizer (Legacy Pattern)

```
runtime.SetFinalizer(obj, func(o *T) {
 // runs asynchronously when obj is unreachable
 o.release()
})
```

Problems:

1. **No guarantee on timing** — may run late or, in edge cases, not before process exit.
2. **Resurrection** — finalizer can store the object globally and keep it alive.
3. **Order** — interdependent objects need careful design.
4. **Performance** — finalizers delay reclaim and add GC bookkeeping.

## AddCleanup (Preferred Direction)

```
// Go 1.24+ style cleanup (check your version docs)
runtime.AddCleanup(ptr, func(res Resource) {
 res.Release()
}, resourceToken)
```

Cleanups are designed to avoid several classic finalizer pitfalls (exact API surface evolves — verify on [pkg.go.dev/runtime](https://pkg.go.dev/runtime)).

## Rules of Thumb

Need	Approach
Files, sockets, <code>body.Close</code>	<code>defer Close</code>
Cache with eviction	Explicit pool / LRU
C malloc companion	<code>runtime.SetFinalizer</code> only if library requires; prefer free in <code>Close</code>
Tests for leaks	<code>testing</code> + explicit asserts, not finalizer timing

## Experiment

```
go mod init example
go run .

package main

import (
 "fmt"
 "runtime"
 "time"
)

type blob struct {
 id int
}

func main() {
 for i := 0; i < 3; i++ {
 b := &blob{id: i}
 id := i
 runtime.SetFinalizer(b, func(_ *blob) {
 fmt.Println("finalizer", id)
 })
 }
}
```



```
 })
 b = nil
}
runtime.GC()
time.Sleep(50 * time.Millisecond)
runtime.GC()
time.Sleep(50 * time.Millisecond)
fmt.Println("done")
}
```

**What to notice:** Finalizers may print after GC, asynchronously; never rely on them for correctness of security-sensitive cleanup without also having an explicit path.

**Try next:** Wrap an `os.File` only with `Close`; use `GODEBUG=gctrace=1` while allocating to see GC activity separately from finalizers.

## **weak and unique Packages**

# weak and unique Packages

## Overview

Go 1.24+ added first-class support for patterns that used to require finalizer hacks or string maps:

Package	Role
<code>weak</code>	Weak pointers — do not keep the referent alive
<code>unique</code>	Canonicalize comparable values (interning)

These show up in caches, symbol tables, and RPC codecs.

## Diagram: Weak vs strong

```
diagram:
 Strong[strong *T] -->|keeps alive| Obj[object]
 Weak[weak.Pointer] -.->|does not keep| Obj
 GC[GC] -->|collects| Obj
 Weak2[weak.Value] -->|nil after| GC
 U1[unique.Make a] --> H[canonical handle]
 U2[unique.Make a] --> H
```

## weak.Pointer

```
import "weak"

type heavy struct{ buf []byte }

func demo() {
 h := &heavy{buf: make([]byte, 1<<20)}
 wp := weak.Make(h)
 h = nil
 runtime.GC()
 if v := wp.Value(); v != nil {
 fmt.Println("still alive", len(v.buf))
 } else {
```

```

 fmt.Println("collected")
 }
}

```

Use cases:

- Caches that should not pin memory forever
- Canonical registries keyed by identity without leaks

Do **not** replace explicit cache eviction policies when you need deterministic memory bounds — combine weak refs with size limits.

## unique.Handle

```

import "unique"

h1 := unique.Make("GET")
h2 := unique.Make("GET")
fmt.Println(h1 == h2) // true - same canonical handle

```

`unique.Make` returns a handle for a comparable value; equal values collapse to one interned copy. Good for:

- Repeated strings (method names, header keys, enums-as-strings)
- Reducing duplicate allocations in large graphs

Handles compare with `==`. Extract value with `h.Value()`.

## When Not To

- Tiny short-lived strings — interning overhead may lose
- Secrets — longer lifetime increases exposure window
- Non-comparable structures — not supported

## Experiment

```

go mod init example
go run .

```

```

package main

import (
 "fmt"

```

```

 "runtime"
 "unique"
 "weak"
)

func main() {
 a := unique.Make("content-type")
 b := unique.Make("content-type")
 fmt.Println("unique equal", a == b, a.Value())

 type node struct{ n int }
 p := &node{n: 7}
 w := weak.Make(p)
 fmt.Println("before", w.Value() != nil)
 p = nil
 runtime.GC()
 runtime.GC()
 fmt.Println("after GC value?", w.Value() != nil)
}

```

**What to notice:** Unique handles collapse equal values; weak pointers may become nil after GC once strong refs are gone (timing can vary — double GC is a demo heuristic).

**Try next:** Intern 100k duplicate strings with and without unique; compare `runtime.MemStats.HeapAlloc`.

## Swiss Maps and maphash

# Swiss Maps and maphash

## Overview

Go's map implementation has evolved. Recent versions moved toward **Swiss Table**-style layouts (open addressing with group metadata) for better CPU cache behavior versus older bucket-chain designs. Exact rollout is versioned — treat this as a **performance model**, not a guarantee of on-disk layout.

Complement: [Slice & map internals](#).

## Diagram: Map lookup path

```
key hash probe groups / buckets slot
high load grow / evacuate

maphash.Seed custom tables / sharding
```

## What Changed Conceptually

```
Legacy (simplified): buckets -> overflow lists
Swiss-style: groups of slots + control bytes (SIMD-friendly probes)
```

Implications:

- Better average lookup locality
- Different resize/evacuation behavior
- Iteration order still **randomized** — never depend on it
- Concurrent map misuse still **fatal**

## Practical Advice (Unchanged)

```
m := make(map[string]T, approxSize) // still helps
```

- Pre-size when cardinality known
- Prefer **string** / integer keys over allocations as keys

- Do not store huge structs as values if you update fields often (consider `map[K]*T`)

## maphash

Package `hash/maphash` provides **high-quality, process-randomized** hashing suitable for your own hash tables and sharding:

```
import "hash/maphash"

var seed maphash.Seed // zero value ok; or maphash.MakeSeed()

func hashString(s string) uint64 {
 var h maphash.Hash
 h.SetSeed(seed)
 h.WriteString(s)
 return h.Sum64()
}
```

Properties:

- Seed differs per process → harder hash-flooding for custom maps
- Not a crypto hash
- Fast for strings/bytes

Use for:

- Sharding keys across workers
- Custom open-address tables
- Consistent bucketing **within one process** (not cross-process stable unless you fix a seed carefully and accept risks)

## Map Iteration and Deletes

```
for k := range m {
 if shouldDrop(k) {
 delete(m, k) // allowed in range
 }
}
```



## Experiment

```
go mod init example
go test -bench=. -benchmem

package mapspeed_test

import (
 "hash/maphash"
 "testing"
)

func BenchmarkMapString(b *testing.B) {
 m := make(map[string]int, b.N)
 for i := 0; i < b.N; i++ {
 m[string(rune('a'+i%26))+string(rune(i))] = i
 }
}

func BenchmarkMaphash(b *testing.B) {
 var seed maphash.Seed
 var h maphash.Hash
 s := "authorization"
 b.ResetTimer()
 for i := 0; i < b.N; i++ {
 h.SetSeed(seed)
 h.WriteString(s)
 _ = h.Sum64()
 h.Reset()
 }
}
```

**What to notice:** Built-in maps are heavily optimized; custom maps need `maphash` + careful probing to approach them.

**Try next:** Read Go release notes for your version mentioning map changes; re-run production pprof after upgrades.

## **sync.Pool and Zero-Allocation Patterns**

# sync.Pool and Zero-Allocation Patterns

## Overview

Reducing allocations is the highest-leverage GC optimization. `sync.Pool` reuses temporary objects; **zero-allocation** patterns avoid heap traffic entirely on hot paths.

X and production war stories hammer this constantly: `pprof` → `mallocgc` → fix allocs before spinning on GOGC.

## Diagram: Pool get/put

```
sequence (top → bottom):
 actors: goroutine, sync.Pool
 goroutine --> sync.Pool : Get
 sync.Pool --> goroutine : buffer
 goroutine --> goroutine : use reslice
 goroutine --> sync.Pool : Put
 note: may drop on GC
```

## sync.Pool

```
var bufPool = sync.Pool{
 New: func() any {
 b := make([]byte, 0, 32*1024)
 return &b
 },
}

func handle() {
 bp := bufPool.Get().(*[]byte)
 buf := (*bp)[:0]
 defer func() {
 if cap(buf) > 64*1024 { // don't retain huge buffers
 return
 }
 *bp = buf
 }
```

```

 bufPool.Put(bp)
}()
// append into buf...
}

```

Rules:

1. **No correctness dependency** — pool may drop items on GC.
2. **Reset** objects before Put (clear sensitive data).
3. **Cap retention** — avoid pooling multi-MB buffers forever.
4. Prefer pooling `[]byte` or structs with slices, not giant value copies.

## Zero-Alloc Techniques

Technique	Idea
Stack arrays	<code>var a [256]byte</code> then <code>a[:n]</code> if <code>n</code> bounded
<code>sync.Pool</code>	Reuse buffers across requests
<code>strconv.Append*</code>	Append to <code>[]byte</code> without intermediate strings
<code>strings.Builder</code>	Single alloc for multi-part strings
Avoid <code>fmt</code> in hot paths	Prefer <code>AppendInt</code> / specialized encoders
Interface-free APIs	Avoid boxing in tight loops

```

b := make([]byte, 0, 64)
b = strconv.AppendInt(b, 42, 10)
b = append(b, ' ')
b = append(b, "ok"...)

```

## Measuring

```

go test -bench=. -benchmem
go test -bench=. -memprofile=mem.out
go tool pprof mem.out

```

```

testing.AllocsPerRun(1000, func() { /* hot path */ })

```

## Anti-Patterns

- Pooling when objects are already tiny and short-lived (overhead > gain)
- Sharing pooled buffers across goroutines without ownership discipline
- Forgetting to reslice to zero length (`buf[:0]`)

## Experiment

```
go test -bench=. -benchmem

package pool_test

import (
 "strconv"
 "sync"
 "testing"
)

var pool = sync.Pool{New: func() any { b := make([]byte, 0, 64); return &b }}

func BenchmarkAllocFmt(b *testing.B) {
 for i := 0; i < b.N; i++ {
 _ = strconv.Itoa(i) + "x"
 }
}

func BenchmarkAppendPool(b *testing.B) {
 for i := 0; i < b.N; i++ {
 bp := pool.Get().(*[]byte)
 buf := (*bp)[:0]
 buf = strconv.AppendInt(buf, int64(i), 10)
 buf = append(buf, 'x')
 *bp = buf
 pool.Put(bp)
 }
}
```

**What to notice:** Append + pool often wins on alloc/op versus string concat.

**Try next:** Profile a JSON encoder hot path; see if `json.Encoder` reuse or code generation helps more than pooling.

## **Goroutine Leaks and Diagnosis**

# Goroutine Leaks and Diagnosis

## Overview

Goroutines are cheap until they **never exit**. Leaks show up as monotonically rising goroutine counts, memory growth (stacks + closed-over data), and eventually latency death.

This is one of the top production issues called out in Go engineering threads: spawn without stop conditions.

## Diagram: Leak patterns

```
go worker

 forever block on chan
 missing ctx.Done
 lock never released

v
NumGoroutine ↑ pprof goroutine fix bounds + cancel
```

## Common Leak Patterns

Pattern	Why it leaks
Blocked on channel send/recv forever	Partner gone
<code>select</code> without <code>ctx.Done</code>	No cancel path
HTTP request without timeout	Stuck on body read
<code>for range ticker.C</code> without <code>Stop/exit</code>	Forever loop
Waiting on mutex held by dead owner	Deadlock-ish stall
Callback registered never removed	Global map grows

## Fix Templates

```
// Always pair spawn with context
go func() {
 select {
 case <-ctx.Done():
 return
 case jobs <- job:
 }
}()
```

```
// Bound fan-out
sem := make(chan struct{}, 32)
for _, item := range items {
 sem <- struct{}{}
 go func(it Item) {
 defer func() { <-sem }()
 process(ctx, it)
 }(item)
}
```

## Diagnosis Toolkit

```
Live
curl -s localhost:6060/debug/pprof/goroutine?debug=1 | head
go tool pprof http://localhost:6060/debug/pprof/goroutine

Go 1.27+: goroutines the GC can prove will never wake
go tool pprof http://localhost:6060/debug/pprof/goroutineleak

Binary
import _ "net/http/pprof"
```

The `goroutineleak` profile (experimental in 1.26, generally available in 1.27) reports goroutines blocked on a concurrency primitive that is unreachable from any runnable goroutine. It will **not** catch leaks parked on globals or on locals of still-runnable Gs. Use it alongside the ordinary goroutine profile, not instead of it.

```
runtime.NumGoroutine()
```

In tests:



```

func TestNoLeak(t *testing.T) {
 before := runtime.NumGoroutine()
 // run code under test with cancel
 // ...
 deadline := time.Now().Add(2 * time.Second)
 for time.Now().Before(deadline) {
 if runtime.NumGoroutine() <= before {
 return
 }
 time.Sleep(10 * time.Millisecond)
 }
 t.Fatalf("goroutines leaked: before=%d now=%d", before, runtime.NumGoroutine())
}

```

For concurrent unit tests with fake time, see [sync/ctest](#).

## Experiment

```

go mod init example
go run .

```

```

package main

import (
 "fmt"
 "runtime"
 "time"
)

func leak() {
 ch := make(chan int)
 go func() { ch <- 1 }() // blocked forever if no receiver
}

func fixed(done <-chan struct{}) {
 ch := make(chan int)
 go func() {
 select {
 case ch <- 1:
 case <-done:
 }
 }()
}

func main() {

```

```
 fmt.Println("start", runtime.NumGoroutine())
 leak()
 time.Sleep(20 * time.Millisecond)
 fmt.Println("after leak", runtime.NumGoroutine())

 done := make(chan struct{})
 fixed(done)
 close(done)
 time.Sleep(20 * time.Millisecond)
 fmt.Println("after fixed", runtime.NumGoroutine())
}
```

**What to notice:** One blocked send permanently increases the count.

**Try next:** Dump `goroutine?debug=1` in a real service after load; group stacks by `chan receive` sites.

## **GODEBUG and Runtime Knobs**

# GODEBUG and Runtime Knobs

## Overview

The runtime exposes a surprising amount of behavior through environment variables — especially **GODEBUG**. These are for diagnosis and carefully controlled experiments, not random production toggles without metrics.

## Diagram: Knob layers

```
flow:
 [Runtime]
 |
 v
 [Observe]
```

## Essential Environment

Variable	Role
GOMAXPROCS	P count / parallel Go code
GOGC	GC percent trigger
GOMEMLIMIT	Soft memory limit
GODEBUG	Comma-separated runtime debug flags
GOTRACEBACK	Panic stack verbosity ( <b>single</b> , <b>all</b> , <b>crash</b> , ...)
GOFLAGS	Default flags for go command
GORACE	Race detector options

## GODEBUG Hits Worth Knowing

Exact keys evolve — check `go doc runtime` / release notes. Common families:

```
GODEBUG=gctrace=1 ./app # GC line traces
GODEBUG=schedtrace=1000 ./app # scheduler trace every 1s
GODEBUG=asyncpreemptoff=1 ./app # debug preemption (special)
GODEBUG=http2debug=1 ./app # HTTP/2 verbose (net/http)
```

```
init-time print of effective settings sometimes available via:
go env
```

## GOTRACEBACK

```
GOTRACEBACK=all ./app # all goroutines on panic
GOTRACEBACK=crash ./app # extra crash info / abort behavior (platform)
```

## Programmatic Knobs

```
debug.SetGCPercent(50)
debug.SetMemoryLimit(512 << 20)
debug.WriteHeapDump(fd)
runtime.SetMutexProfileFraction(5)
runtime.SetBlockProfileRate(1000)
```

## Safety Rules

1. Change **one** knob at a time.
2. Compare p95 CPU memory goroutines before/after.
3. Do not ship schedtrace or http2debug at full verbosity in prod.
4. Prefer GOMEMLIMIT over folklore GOGC=off.

## Experiment

```
GODEBUG=gctrace=1 go run .
```

```
package main

import (
 "fmt"
 "runtime"
 "runtime/debug"
)

func main() {
 debug.SetGCPercent(100)
 var keep [][]byte
```

```
 for i := 0; i < 200; i++ {
 keep = append(keep, make([]byte, 256*1024))
 }
 fmt.Println("goroutines", runtime.NumGoroutine(), "kept", len(keep))
 keep = nil
 runtime.GC()
}
```

**What to notice:** `gctrace` lines show GC cycles correlating with allocation bursts.

**Try next:** Capture `schedtrace` during a latency incident; look for high latencies in scheduling vs your code.

## **pprof and Execution Tracer Internals**

# pprof and Execution Tracer Internals

## Overview

Workflows live in [benchmarking & profiling](#). This chapter is the **mechanism**: how samples appear, what they mean, and how the execution tracer differs from CPU pprof.

## Diagram: Profile vs trace

```
flow:
 [OnCPU]
 |
 v
 [Tool]

 [Events]
 |
 v
 [Tool2]
```

## CPU Profile

Rough model:

```
OS signal / interrupt (platform dependent sampling)
-> record stack of current G/M
-> aggregate into profile proto
```

```
go test -cpuprofile=cpu.out -bench=.
go tool pprof -http=:0 cpu.out
```

Implications:

- **Sampling** — short functions may be undercounted
- Inlined frames can look surprising
- CPU profile is not wall-clock wait time



## Heap / Alloc Profiles

- Heap: objects still live at sample time (retained)
- Alloc: cumulative allocations (even if freed)

```
go tool pprof http://localhost:6060/debug/pprof/heap
```

Use alloc profiles to find churn; heap profiles to find leaks/retention.

## Block and Mutex Profiles

```
runtime.SetBlockProfileRate(1) // careful in prod
runtime.SetMutexProfileFraction(5)
```

These answer “who waited on locks/channels” — complementary to CPU.

## Goroutine Profile

Stacks of all Gs — best leak and deadlock forensic tool.

## Execution Tracer (go tool trace)

Records **events** with timestamps:

- Goroutine create/block/unblock
- Syscall enter/exit
- GC STW / mark assist
- Network poll
- Processor start/stop

```
go test -trace=trace.out -bench=.
go tool trace trace.out
```

Use when CPU is low but latency is high — classic **scheduler** / **stall** story.

## Cost of Profiling

Mode	Overhead
Occasional CPU 30s	Usually acceptable
Always-on mutex rate=1	Can be heavy
Full traces long windows	Large files, overhead

Sample in production; continuous profiling products add aggregation layers.

## Experiment

```
go test -bench=BenchmarkWork -cpuprofile=/tmp/cpu.out -benchtime=1s
go tool pprof -top /tmp/cpu.out | head
```

```
package work_test

import "testing"

func work(n int) int {
 x := 0
 for i := 0; i < n; i++ {
 x += i
 }
 return x
}

func BenchmarkWork(b *testing.B) {
 for i := 0; i < b.N; i++ {
 _ = work(1000)
 }
}
```

**What to notice:** Top frames should show `work` or inlined sites; if everything is `runtime.*`, you may be measuring something else (GC, tiny benchmark).

**Try next:** Generate a 200ms trace around an HTTP handler and find a long “block” on network or channel.

**testing/synctest: Bubbles, Durable Blocks, Fake Time**

# testing/synctest: Bubbles, Durable Blocks, Fake Time

## Overview

testing/synctest is **not** a polite `time.Sleep` helper. It is a **runtime feature** exposed through a thin testing API: the scheduler tracks an isolated **bubble** of goroutines, channels, and timers, and can answer a precise question instead of guessing.

```
Has every goroutine in this test either finished, or reached a block
that only another goroutine in the *same* bubble can unblock?
```

If yes, the bubble is **settled**. Then `synctest.Wait` can return, or the **fake clock** can jump to the next timer deadline—without burning wall-clock time.

**Sources / depth.** This chapter synthesizes the public API with runtime-level concepts documented in guest runtime walkthroughs such as *Fake Clocks, Real Guarantees: Inside Go's synctest* (Alex Rios / Internals for Interns; runtime as of ~Go 1.25–1.27). Prose and diagrams here are original. Always verify symbols against your `GOROOT` and `go doc testing/synctest`.

**Availability:** experiment in 1.24 → stable package from 1.25 → `synctest.Sleep` in Go 1.27. Check `go version` before relying on APIs.

## The problem it solves

```
// Ordinary concurrent test - a guess dressed as a test
go doSomething()
time.Sleep(10 * time.Millisecond) // hope the G ran
if !done {
 t.Fatal("not done")
}
```

Longer sleeps only buy a **slower guess**. Scheduling order, machine load, and GC jitter all change whether 10ms was enough.

ordinary test

```
go doWork()
```

synctest

```
bubble { workers, timers, chans }
```

v		v
time.Sleep(10ms)	guess	Wait / clock jump      proof
v		v
assert "probably done"		assert only after SETTLED (all Gs done or durably blocked)

## Public API (small overlay)

Typical surface (confirm on your version):

```
synctest.Test(t, func(t *testing.T) { ... })
synctest.Wait()
synctest.Sleep(d) // Go 1.27+: Sleep + Wait convenience
// older / internal shape also used synctest.Run(func(){...})
```

```
testing/synctest thin public package
```

```
internal/synctest glue
```

```
runtime/synctest.go bubble, counters, event loop
+
runtime proc/chan/time/sema/select durability hooks
```

`synctest.Test` creates a bubble, adapts `*testing.T` (child `T` marked as `synctest`), and runs your function through normal `tRunner` so `Fail/Fatal/cleanup/logs/race` still work.

**Inside a bubble, some `T` methods panic** (e.g. `t.Parallel`, `t.Run` as normal nested subtests, `t.Deadline` in designs where wall deadline vs fake `time.Now` would be nonsense). Treat the bubble as a special domain, not a nested subtest tree.

## The bubble

Conceptual structure (names follow runtime ideas; fields evolve):

```
synctestBubble
now fake clock (ns) - starts at a fixed epoch (often 2000-01-01 UTC)
timers private timer heap driven by bubble.now
root controller G (event loop; not your test body)
main G that runs the test function
running count of Gs not yet proven durably blocked
```

```

active bookkeeping for mid-park / wake gaps
...

 syncctest bubble
root G (event loop; not test body)

 bubble.now (fake clock)
 bubble.timers (private heap)

main G (test function)
 worker Gs (inherit bubble on go)

running / active counters → settled?

 may touch (not durable)
 v
 outside world / globals / net

```

## Membership

Every `g` may carry bubble `*syncctestBubble`.

On `go f()` (`newproc1`):

- **User Gs inherit** the parent's bubble pointer.
- **System Gs** (GC, netpoll helpers, ...) **do not** inherit membership.

So membership spreads automatically down the spawn tree without tagging every `go` statement.

```

Test body (in bubble)
 go worker → in bubble
 go nested → in bubble
Runtime GC G → not in bubble

```

## Root vs main

`Run/Test` does **not** run your function on the root. Root becomes the **controller**; a **main** goroutine runs the test (and for `Test`, `adapters/tRunner` are also members). Root owns the fake-time event loop.

## Durable blocking = wait reason

**Durable block (docs language):** blocked and can only be unblocked by another goroutine in the **same** bubble.

Runtime implements this with `waitReason` when a G parks (`_Gwaiting`). A table marks which reasons are **idle in synctest**.

Typically durable (idle)	Typically <b>not</b> durable
<code>time.Sleep</code> (fake timer)	<code>sync.Mutex.Lock</code> / <code>RWMutex</code>
send/recv on <b>bubbled</b> channel	send/recv on <b>outside</b> channel
select where <b>every</b> case is bubbled	select with any outside channel
<code>sync.Cond.Wait</code> (with bubble wake checks)	—
<code>WaitGroup.Wait</code> when <b>associated</b> to this bubble	<code>WaitGroup</code> not associated / wrong bubble
nil-chan send/recv, empty <code>select{}</code>	(deadlock path if nothing else can finish)
<code>synctest.Wait</code> itself	—

G parks

```

 v
 status == _Gwaiting? no counts as running (not settled)

 yes
 v
 waitReason idle-in-synctest? no still "running" for bubble

 yes
 v
 DURABLE block → running--

```

**Why mutex fails the test:** a mutex has **no bubble ownership**. An outside G could `Unlock`. The runtime refuses to call that settled, so `synctest.Wait` may **hang** until the outer go test timeout—not always a nice bubble deadlock panic.

## Two counters, one invariant

```
running > 0 || active > 0 "something still going on"
```

Counter	Meaning
<code>running</code>	Gs not yet proven durably blocked (not the same as “on CPU”)
<code>active</code>	Scheduler bookkeeping during <b>mid-park</b> , wakes, root loop tokens

Parking is not atomic: status may drop to waiting **before** a commit function finalizes the park. Without `active`, the bubble could see `running == 0` in that gap and **false-settle**.

```

worker G bubble counters

 incActive (mid-park)
 active > 0
 status → waiting
 (running may hit 0 early) running==0, active>0 ← not settled yet
 park committed / aborted
 decActive
 only now both may be 0 → settled

```

Status changes go through the scheduler's status CAS path; bubbled Gs call into bubble accounting (changegstatus-style hooks).

## Root event loop (fake time)

When settled (and timers remain), the root:

1. Runs fake timers due at `bubble.now`
2. Tries to park (handing back its `active` token)
3. Looks at next timer deadline
4. Either **jumps** `bubble.now = next` or exits (done / no timers)

```

root loop

 v
 run timers due at bubble.now

 v
 try park (release active token)

 v
 next timer deadline?

 none some

 v v
 stop loop bubble.now = next

 v
 main done & no leftover Gs?
 yes → clean exit
 no → deadlock panic (durable stuck / leaked Gs)

```

**Fake time never wall-waits once settled.** Ten “seconds” of Sleep can finish in microseconds of wall time: `now` is assigned, not slept.



```
Fake clock starts ~2000-01-01 00:00:00 UTC
```

```
G2: Sleep(3s) timer @ 00:00:03
main: Sleep(10s) timer @ 00:00:10
```

```
both park → settled
root jumps now → 00:00:03 → G2 runs
settled again
root jumps now → 00:00:10 → main runs
```

## time.Now inside a bubble

Runtime returns **bubble.now** for wall time. Monotonic component is often **zero** so mixing inside/outside times does not invent a second confusing clock. **time.Since** still works because the bubble wall clock is under runtime control.

## Fake timers

Timers created in a bubble are marked fake and live on the **bubble timer heap**, not a normal per-P heap. Callbacks (e.g. **AfterFunc**) briefly count as bubble activity so spawned Gs inherit membership.

## Channels: married at make

```
hchan.bubble set in makechan if creator is in a bubble
```

Action	Effect
send/recv/close from <b>outside</b> bubble	<b>fatal</b> (process-killing runtime error)
block on bubbled chan	wait reason is <b>syncctest</b> send/recv → durable
block on unbubbled chan	not durable

```
// wait reason conceptually:
// unbubbled: waitReasonChanSend
// bubbled: waitReasonSyncctestChanSend // idle-in-syncctest
```

## Select

Durable only if **every** non-nil case is a bubbled channel in the **current** bubble. One outside case spoils the proof → non-durable select wait.

```
select {
case <-bubbledCh: // ok
case <-time.After(d): // After's chan is bubbled if created inside
case <-outsideCh: // spoils durability
}
```

## WaitGroup association (heap special)

WaitGroup packs a **bubble membership bit** into state and, on Add inside a bubble, associates via a **heap special** (same family of side notes as finalizers).

```
Durable WaitGroup.Wait requires roughly:
1. Add happened in a bubble
2. Wait called in a bubble
3. Same bubble association still valid
```

Package-level var `wg sync.WaitGroup` lives in the data segment—**no heap object** to attach the special to. Association can fail; waits may never count as durable.

Prefer:

```
wg := new(sync.WaitGroup) // heap-allocated
// or local var that escapes to heap as needed
```

Miss association → bubble may never settle while Wait is blocked.

## sync.Cond

Cond.Wait parks with a reason marked idle. Ownership is checked on **wake**: signaling a bubbled waiter from outside its bubble is **fatal**. Durability rides on parked `g.bubble`, not a field on Cond itself.

## Mutex: deliberately non-durable

```
var mu sync.Mutex
syncctest.Test(t, func(t *testing.T) {
 mu.Lock()
 go func() {
 mu.Lock() // blocks - NOT durable
 }()
 syncctest.Wait() // may hang forever (outer test timeout)
})
```

```

flow:
 [ND]
 |
 v
 [R]

 [R]
 |
 v
 [H]

```

### Debug fork:

Symptom	Look for
Named <b>deadlock panic</b> from bubble	All <b>durable</b> blocks, no timers left / leaked Gs after main exit
<b>Hang</b> until go test timeout	Non-durable wait: mutex, unbubbled channel, bad WaitGroup association

## Deadlock detection (when root can prove it)

When the root stops and leftover Gs remain:

- “all goroutines in bubble are blocked” — durable deadlock, no timer escape
- “main exited but blocked goroutines remain” — leak after test function return

Stricter than ordinary tests: a forever-blocked **bubbled** channel is a loud failure, not silent process exit.

## Isolation cheat sheet

```

Created inside bubble:
 channel → hchan.bubble
 timer → isFake, bubble heap
 WaitGroup → associate on Add (heap)

From outside:
 send/recv/close bubbled chan → fatal
 fake timer stop/reset from out → fatal
 WaitGroup Add from wrong bubble → fatal

Outside objects used inside:
 allowed, but waits usually NOT durable

```

## Isolation is not a sandbox

The test can still touch globals, disk, network. Those ops are **not** part of the settling proof. Keep I/O out of unit bubbles or accept non-determinism.

## End-to-end narrative

```
func TestWorkerTimeout(t *testing.T) {
 synctest.Test(t, func(t *testing.T) {
 ch := make(chan string) // bubbled
 go func() {
 select {
 case msg := <-ch:
 t.Log("got", msg)
 case <-time.After(5 * time.Second): // fake timer
 t.Error("timed out")
 }
 }()
 time.Sleep(3 * time.Second) // fake
 ch <- "hello"
 })
}
```

```
sequence (top → bottom):
 actors: Root, Main, Worker, bubble.now
 Root --> Main : start test G
 Main --> Worker : go select
 Worker --> Worker : park durable select
 Main --> Main : Sleep 3s park durable
 Root --> bubble.now : jump to +3s
 Main --> Worker : send hello
 Worker --> Worker : log and exit
 Main --> Root : return clean
 note: settled running=0
```

No wall wait for “3 seconds”; no sleep-and-hope after the send.

## synctest.Wait and the race detector

When Wait un parks, the runtime establishes a **happens-before** edge (raceacquire-style) so reads after Wait of data written by now-blocked Gs are not false races. Matching releases fire when Gs durably block, exit, or finish fake timer callbacks.

`sync_test.Sleep(d)` (Go 1.27+) `time.Sleep(d) + Wait()`: advance clock, then wait for reactions to settle. Same-instant timer order remains unspecified; trailing `Wait` makes that OK for most tests.

## Practical rules

1. Create channels/timers **inside** the bubble you care about.
2. Use **heap** `WaitGroups` if you need durable `Wait`.
3. Do not expect `Mutex` waits to settle the bubble.
4. Prefer `sync_test.Wait` over wall `Sleep` for “work finished”.
5. Hang vs panic: hang → non-durable; panic → durable deadlock/leak.
6. Still run `go test -race`.
7. Don’t nest `sync_test.Run/Test` (runtime panics).

## Relation to other chapters

Topic	Chapter
Flaky concurrent tests	<a href="#">260 Testing</a>
GMP / parking	<a href="#">201 Scheduler</a>
Timers	<a href="#">214 Timers</a>
Channel internals	<a href="#">203 Channels</a>
Context timeouts	<a href="#">255 Context</a>

## Reading the source (same path as the article)

1. `testing/sync_test` public API
2. `testing` hook / T adaptation
3. `runtime.sync_testRun` / bubble struct
4. status hooks + wait reason table
5. `time` linknames + fake timers
6. `makechan` / `chansend` / `selectgo`
7. `sync.WaitGroup` + `semacquire` durable flag
8. Package tests under `internal/sync_test`

## Experiment

```
go version
go doc testing/synctest

func TestFakeSleep(t *testing.T) {
 // Use the API your version exports: Test and/or Run
 synctest.Test(t, func(t *testing.T) {
 start := time.Now()
 done := make(chan struct{})
 go func() {
 time.Sleep(30 * time.Second)
 close(done)
 }()
 synctest.Wait() // or Sleep if available
 <-done
 // wall time should be tiny; bubble time advanced ~30s
 if time.Since(start) > time.Second {
 t.Fatalf("wall clock moved too much: %v", time.Since(start))
 }
 })
}

go test -race -count=100 -run TestFakeSleep
```

**What to notice:** 30s of fake sleep does not cost 30s of CI.

**Try next:** Add a mutex-blocked G and observe Wait hang vs durable channel deadlock panic.

## Recap

Idea	One line
Bubble	Isolated scheduler domain for one test
Membership	Inherited on <code>go</code> ; system Gs excluded
Durable block	Wait reason marked idle-in-synctest
Settled	<code>running==0</code> and <code>active==0</code>
Root	Advances fake clock / wakes Wait / detects deadlock
Fake time	Jump <code>now</code> on private timer heap
Objects	Chan/timer/WG associated; outside touch often fatal
Mutex	Never durable—by design

The tiny API is small on purpose. The **proof** that a concurrent test is settled lives in the runtime—membership, wait reasons, timer heaps, and isolation—exactly what user-space mocks cannot see.

## **Zero-Copy I/O (sendfile, splice, Copy)**

# Zero-Copy I/O (sendfile, splice, Copy)

## Overview

Moving bytes through user space burns CPU. Linux offers **sendfile** and **splice** to reduce copies. Go's `io.Copy` tries optimized paths when concrete types implement `ReaderFrom` / `WriterTo` (and OS support exists).

Popular writeups (widely shared among systems Go engineers) stress: TLS and some wrappers disable the fast path.

## Diagram: Copy fast path

```
io.Copy(dst, src)

 dst has ReadFrom? yes optimized path

 src has WriteTo? yes optimized path

 else buffered copy loop
TLS / custom wrappers often force the slow path
```

## Copy Dispatch

```
io.Copy(dst, src)
-> if dst has ReaderFrom: dst.ReadFrom(src)
-> else if src has WriterTo: src.WriteTo(dst)
-> else generic buffer loop
```

`*os.File` and `*net.TCPConn` participate in platform-specific fast paths when combinations allow.

## sendfile / splice (Linux)

Typical wins:

- File → socket static file serving



- Socket → socket proxying (sometimes via splice)

Usually **not** available end-to-end when:

- TLS wraps the connection (encryption needs bytes in user space)
- Custom `io.Reader` wrappers break type assertions
- Non-Linux platforms fall back to buffered copy

## Practical Patterns

```
// Prefer typed paths
if _, err := io.Copy(dstFile, srcFile); err != nil { ... }

// Cap memory even on slow path
lr := io.LimitReader(src, max)
_, err = io.Copy(dst, lr)

// HTTP: FileServer / ServeContent already try efficient paths
http.ServeContent(w, r, name, modtime, content)
```

## Measuring

```
perf stat -e cycles,instructions,cache-misses ./proxy # Linux
go test -bench=Copy -benchmem
```

Compare:

1. `io.Copy` file→file
2. Manual Read/Write 32KiB buffer
3. File→TLS conn (expect less zero-copy)

## Experiment

```
go mod init example
go test -bench=. -benchmem
```

```
package zc_test
```

```
import (
 "bytes"
 "io"
```

```

 "os"
 "path/filepath"
 "testing"
)

func BenchmarkCopyFile(b *testing.B) {
 dir := b.TempDir()
 src := filepath.Join(dir, "s")
 dst := filepath.Join(dir, "d")
 data := bytes.Repeat([]byte("x"), 1<<20)
 if err := os.WriteFile(src, data, 0o644); err != nil {
 b.Fatal(err)
 }
 b.SetBytes(int64(len(data)))
 b.ResetTimer()
 for i := 0; i < b.N; i++ {
 in, _ := os.Open(src)
 out, _ := os.Create(dst)
 _, _ = io.Copy(out, in)
 in.Close()
 out.Close()
 }
}

```

**What to notice:** Kernel paths dominate large copies; wrappers and TLS change the story more than micro-tuning buffer sizes alone.

**Try next:** Wrap the reader in a custom type without `WriterTo` and observe any bench regression.

# HTTP Transport and Connection Pool

# HTTP Transport and Connection Pool

## Overview

`http.Client` performance is mostly **`http.Transport`**: dialing, TLS, idle connection pools, HTTP/2 multiplexing. Using `http.DefaultClient` without timeouts is a top production mistake called out constantly in Go ops threads.

## Diagram: Client stack

```
Client.Do(req)
 v
Transport
 Dial (+ TLS)
 idle pool (per host)
 HTTP/1.1 or HTTP/2 streams
 v
resp.Body.Close() return conn to pool
```

## Client Stack

```
Client.Timeout (optional overall)
-> Transport.RoundTrip
 DialContext
 TLSHandshake
 conn pool (idle conns per host)
 HTTP/1.1 or HTTP/2
```

```
tr := &http.Transport{
 Proxy: http.ProxyFromEnvironment,
 DialContext: (&net.Dialer{Timeout: 3 * time.Second, KeepAlive: 30 * time.Second,
 ForceAttemptHTTP2: true,
 MaxIdleConns: 100,
 MaxIdleConnsPerHost: 10,
```

```

 MaxConnsPerHost: 0, // 0 = unlimited
 IdleConnTimeout: 90 * time.Second,
 TLSHandshakeTimeout: 5 * time.Second,
 ExpectContinueTimeout: 1 * time.Second,
 ResponseHeaderTimeout: 10 * time.Second,
}
client := &http.Client{Transport: tr, Timeout: 15 * time.Second}

```

## Pool Behaviors

Knob	Effect
MaxIdleConns	Global idle cache size
MaxIdleConnsPerHost	Default is low (2) — often raise for chatty APIs
MaxConnsPerHost	Hard cap; backpressure when hit
IdleConnTimeout	Drop idle to free FDs
DisableKeepAlives	Every request new dial — debug only

Failing to `Body.Close()` **leaks** connections out of the pool.

## HTTP/2

When negotiated, multiple streams share one TCP connection. Benefits under many parallel requests to one host; watch head-of-line at other layers and flow control. Debug with `GODEBUG=http2debug=1` carefully.

## Server Side (Brief)

```

srv := &http.Server{
 ReadHeaderTimeout: 5 * time.Second,
 ReadTimeout: 15 * time.Second,
 WriteTimeout: 30 * time.Second,
 IdleTimeout: 60 * time.Second,
}

```

Timeouts are load shedding, not pedantry.

## Experiment

```
go mod init example
go run .
```

```
package main
```

```
import (
 "fmt"
 "io"
 "net/http"
 "net/http/httptest"
 "time"
)
```

```
func main() {
 ts := httptest.NewServer(http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
 _, _ = io.WriteString(w, "ok")
 }))
 defer ts.Close()

 tr := ts.Client().Transport.(*http.Transport).Clone()
 tr.MaxIdleConnsPerHost = 5
 client := &http.Client{Transport: tr, Timeout: 2 * time.Second}

 for i := 0; i < 3; i++ {
 resp, err := client.Get(ts.URL)
 if err != nil {
 panic(err)
 }
 _, _ = io.Copy(io.Discard, resp.Body)
 resp.Body.Close()
 }
 fmt.Println("requests ok", time.Now().Format(time.RFC3339))
}
```

**What to notice:** Closing bodies returns conns to the idle pool; omitting close eventually exhausts FDs under load.

**Try next:** Load test with MaxIdleConnsPerHost=2 vs 32 against a single upstream; compare dial rates in traces.

## Context Cancel Trees Internals

# Context Cancel Trees Internals

## Overview

`context.Context` is a **tree of cancellation and values**. Understanding parent→child propagation explains goroutine leaks, deadline bugs, and why `context.Background()` at random depths is an outage pattern.

## Diagram: Cancel flows down

```
parent ctx
 child (timeout)
 child (values)
cancel(parent) cancels all children
cancel(child) does NOT cancel parent
```

## Tree Shape

```
Background
 WithValue(request_id)
 WithTimeout(2s) <- parent cancel cancels children
 WithCancel <- child cancel does NOT cancel parent
 WithTimeout(500ms)
```

Rules:

1. Cancel/timeout flows **down**.
2. Child cancel does **not** cancel siblings/parent.
3. Values flow by lookup up the parent chain.
4. Always defer `cancel()` for `WithCancel/Timeout/Deadline`.

## Implementation Intuition

Each cancelable context holds:

- A done channel (closed on cancel)
- Parent linkage / children set



- Optional timer for deadlines
- Optional cause error (`WithCancelCause`)

Closing parent walks children. This is why leaking cancel funcs can retain subtrees longer than you expect.

## Values

```
type key struct{} // unexported key type avoids collisions
ctx = context.WithValue(ctx, key{}, rid)
```

Do not store optional params that belong in function arguments. Values are for request-scoped data (IDs, loggers), not control flow.

## Error Taxonomy

```
ctx.Err() == context.Canceled
ctx.Err() == context.DeadlineExceeded
// WithCancelCause: context.Cause(ctx)
```

Map these to client retries carefully — cancel from client disconnect    retryable server error necessarily.

## Experiment

```
go mod init example
go run .
```

```
package main

import (
 "context"
 "fmt"
 "time"
)

func main() {
 parent, cancel := context.WithCancel(context.Background())
 defer cancel()
 child, cancelChild := context.WithTimeout(parent, 50*time.Millisecond)
 defer cancelChild()
```

```

stop := make(chan struct{})
go func() {
 <-child.Done()
 fmt.Println("child", child.Err())
 close(stop)
}()

<-stop
cancel() // parent cancel after child already done
fmt.Println("parent", parent.Err())

// child cancel does not kill parent
p2, c2 := context.WithCancel(context.Background())
ch, c3 := context.WithCancel(p2)
c3()
fmt.Println("child2", ch.Err(), "parent2", p2.Err())
c2()
}

```

**What to notice:** Timeout sets child error first; canceling a child leaves parent active until its own cancel.

**Try next:** Pass `context.WithoutCancel(ctx)` (Go 1.21+) when you must detach a background job intentionally — and document why.

## Mutex and Semaphore Runtime

# Mutex and Semaphore Runtime

## Overview

`sync.Mutex` is not a pure userspace spin forever. After a short path, contending lockers **park** through runtime semaphores (sudog queues), integrating with the scheduler like channel waits.

## Diagram: Lock fast/slow path

```
Lock attempt
 uncontended CAS enter critical section
 contended park (semacquire)
Unlock wake one waiter runnable
```

## Fast Path / Slow Path

```
Lock attempt
-> atomic CAS (uncontended) success → return
-> spin briefly (optional, implementation)
-> enter runtime semacquire queue
-> G parks
Unlock
-> hand off / wake waiter
-> woken G runnable
```

Implications:

- Uncontended locks are extremely cheap
- High contention shows up in **block/mutex profiles**, not only CPU
- Holding locks across I/O serializes your service (top production mistake)

## RWMutex

Multiple readers; writers exclusive. Writer starvation possible under continuous readers — measure.

## Cond

`sync.Cond` builds on mutex + notification; easy to misuse. Prefer channels for many app-level designs; Cond appears in specialized queues.

## Atomics vs Mutex

Need	Prefer
Counter / flag	<code>atomic</code>
Multi-field invariant	<code>Mutex</code>
Coordination / ownership transfer	<code>Channel</code>

## Avoid

```
mu.Lock()
resp, err := http.Get(url) // BAD: lock held across I/O
mu.Unlock()
```

## Experiment

```
go test -bench=. -benchmem

package lock_test

import (
 "sync"
 "sync/atomic"
 "testing"
)

func BenchmarkMutex(b *testing.B) {
 var mu sync.Mutex
 var x int
 b.RunParallel(func(pb *testing.PB) {
```

```

 for pb.Next() {
 mu.Lock()
 x++
 mu.Unlock()
 }
 })
}

func BenchmarkAtomic(b *testing.B) {
 var x atomic.Int64
 b.RunParallel(func(pb *testing.PB) {
 for pb.Next() {
 x.Add(1)
 }
 })
}

```

**What to notice:** Atomics win for simple counters; mutexes win for complex critical sections — profile contention, don't guess.

**Try next:** Enable mutex profile on a service and find the top `Unlock` edge under load.

## **cgo and Scheduler Transitions (incl. Go 1.26)**

# cgo and Scheduler Transitions (incl. Go 1.26)

## Overview

cgo lets Go call C (and reverse), but each call crosses **ABI**, **stack**, and **scheduler** boundaries. High-frequency cgo is a classic latency and throughput tax.

Go 1.26 runtime work simplified how processors track syscall/cgo blocking: **P no longer uses a dedicated syscall state**; the scheduler derives blocking from **G state**. That reduces bookkeeping on hot syscall/cgo paths (community writeups cite meaningful wins especially for cgo-heavy workloads).

## Diagram: cgo call path

```
Go code

 v
runtime.cgocall

 v
system stack → C function (may block M)

 v
return to Go → reacquire P if needed
```

## Call Path (Conceptual)

```
Go code
-> runtime.cgocall
-> switch to system stack
-> C function runs (may block OS thread)
-> return to Go
-> reacquire P if needed
```

While C runs, that **M** is busy in C. The runtime may reassign **P** so other Go code continues — but threads can accumulate if many Ms stick in C.



## Costs

1. Call overhead (argument copy, stack switch)
2. Loss of some Go compiler visibility
3. Harder cross-compilation
4. GC interactions with C-allocated memory (you free C memory)

## Rules

Prefer	Avoid
Pure Go libraries	cgo in per-request tiny helpers
Batch work into fewer cgo calls	Chatty call loops
Explicit C resource Close	Relying only on finalizers
Build tags for optional cgo	Silent cgo in libraries users don't expect

```
CGO_ENABLED=0 go build ./... # fail if cgo required
go env CGO_ENABLED
```

## Debugging

```
see if binary links libc unexpectedly
ldd ./app # Linux

profiles often show runtime.cgocall
go tool pprof cpu.out
```

## Relation To Non-cgo Syscalls

Blocking syscalls from Go also park/reassign; netpoll avoids blocking Ms for sockets. cgo is closer to “unknown blocking foreign code.”

## Experiment

```
pure Go path baseline
CGO_ENABLED=0 go build -o /tmp/pure .
```

Write a tiny package without cgo and confirm it builds with CGO\_ENABLED=0. If you have a cgo dependency, compare:

```
go test -bench=. -tags=cgo ./...
```

**What to notice:** Many services never need cgo; forcing `CGO_ENABLED=0` in CI catches accidental deps early.

**Try next:** Read Go 1.26 release notes scheduler section alongside [GMP scheduler](#). For the *deployment* cost of cgo (libc, Alpine, `scratch`), start at [Go as a Systems Language](#).

## **Iterators and range-over-func Deep Dive**

# Iterators and range-over-func Deep Dive

## Overview

Go 1.22–1.23 made **for range over functions** a language feature, with package `iter` defining `Seq` / `Seq2`. This is a deep dive on semantics and costs — not just syntax.

Also: [slices/maps/cmp/iter](#), [loops](#).

## Diagram: Push iterator

```
flow:
 [Yield] --true--> [Next]
 [Yield] --false--> [Stop]
```

## Types

```
type Seq[V any] func(yield func(V) bool)
type Seq2[K, V any] func(yield func(K, V) bool)
```

The iterator **pushes** values into `yield`. Returning `false` from `yield` stops early (break).

## Desugaring Intuition

```
for v := range All(s) {
 if v < 0 {
 break
 }
}
// roughly calls All(s)(func(v T) bool { ...; return continue? })
```

Panic safety: iterator code should treat `yield` like a callback that may not return normally if the consumer panics — `stdlib` iterators are careful; custom ones should be too.

## Pull Iterators

```
next, stop := iter.Pull(seq)
defer stop()
for {
 v, ok := next()
 if !ok {
 break
 }
}
```

Pull converts push-style to pull-style via a small coroutine-like parking protocol — convenient adapters, slight overhead.

## When Iterators Shine

- Lazy pipelines over large or infinite streams
- Library APIs that used to allocate intermediate slices
- Single implementation for `range` and `slices.Collect`

## When They Don't

- Tiny hot loops where a classic index for is clearer and fully BCE-friendly
- Need for random access
- Over-abstracting one-off loops

## Experiment

```
go mod init example
go run .
```

```
package main
```

```
import (
 "fmt"
 "iter"
 "slices"
)
```

```
func Filter[V any](s []V, ok func(V) bool) iter.Seq[V] {
 return func(yield func(V) bool) {
 for _, v := range s {
```

```

 if !ok(v) {
 continue
 }
 if !yield(v) {
 return
 }
 }
}

func main() {
 nums := []int{1, 2, 3, 4, 5}
 for v := range Filter(nums, func(n int) bool { return n%2 == 0 }) {
 fmt.Println("even", v)
 }
 fmt.Println("collect", slices.Collect(Filter(nums, func(n int) bool { return n > 3 })))
}

```

**What to notice:** No intermediate filtered slice until Collect.

**Try next:** Implement Map and chain Map(Filter(...)); compare allocs to nested slice loops.

## **ABI, Assembly, and go:nosplit**

# ABI, Assembly, and go:nosplit

## Overview

Most Go code never touches assembly. When you do (crypto, math, codecs, runtime), you meet **ABIInternal**, register conventions, and stack growth rules.

## Diagram: ABI layers

```
diagram:
 GoFunc[Go function] --> ABIInt[ABIInternal]
 Asm[assembly TEXT] --> ABIO[ABIO / NOSPLIT]
 Boundary[boundary] --> Transition[ABI transition]
```

## ABIs (High Level)

ABI	Role
ABIO	Stable-ish assembly convention used historically
ABIInternal	Compiler's internal register ABI for Go functions

The compiler transitions between them at certain boundaries. Assembly files use special headers:

```
// func Add(a, b int) int
TEXT ·Add(SB), NOSPLIT, $0-24
// ...
```

## go:nosplit

Prevents stack growth checks in a frame. Required for some runtime entry paths; dangerous in app code if the frame is large.

```
//go:nosplit
func tiny() { }
```



## go:uintptrescapes / go:uintptrkeepalive

Document pointer lifetime when smuggling pointers as `uintptr` through syscalls — get this wrong and GC frees live memory.

## SIMD Experiments

Go 1.26 introduced experimental `simd/archsimd`. Go 1.27 adds a portable `simd` package (vector-size-agnostic types such as `Int8s` / `Float32s`) and extends `archsimd` to arm64 Neon and Wasm SIMD. Both still require `GOEXPERIMENT=simd` and are not production-stable. Prefer them over hand asm for new vector experiments.

## When To Write Asm

1. Profile proves a hot leaf
2. Algorithm is fixed-width crypto/math
3. You can maintain per-GOARCH files
4. You accept review cost

Otherwise prefer pure Go + compiler autovectorization where it exists, or cgo to a maintained C library (with eyes open).

## Plan 9 listing and go tool objdump

The compiler emits **Plan 9-style pseudo-assembly** (SB = static base, SP = stack) and the Go linker writes the final binary. That is why you can inspect any GOARCH on any host without a cross-GDB.

```
Compiler IR (stderr):
go build -gcflags='-S' . 2>&1 | head -80

Real machine code, still printed in Plan 9 register names:
CGO_ENABLED=0 go build -o hello .
go tool objdump -s main.main hello

Cross-arch: build ARM64, disassemble it on amd64/arm64/darwin alike:
GOOS=linux GOARCH=arm64 CGO_ENABLED=0 go build -o hello-arm64 .
go tool objdump -s main.main hello-arm64
```

Calls in that listing point at Go symbols compiled into the binary (`fmt.Fprintln`, `runtime.morestack`) — not at `printf` or `libc.so`. That is the same “no libc in the middle” fact as [Go as a Systems Language](#). On ARM64, R28 holds the current goroutine (g).

## Experiment

```
go build -gcflags='-S' . 2>&1 | head -80
```

```
package main

func add(a, b int) int { return a + b }

func main() {
 println(add(2, 3))
}
```

**What to notice:** Even simple functions show ABI prologue/epilogue; inlining may erase `add` entirely with optimizations.

**Try next:** Compare `-gcflags='-l -S'` (no inline) vs default for the same function. Then `go tool objdump -s main.main` on a `GOARCH=arm64` binary built from this machine.

## Coverage Instrumentation and Fuzz Engine

# Coverage Instrumentation and Fuzz Engine

## Overview

`go test` is not only assert runners — it can **instrument binaries for coverage** and drive a **fuzzing engine** that mutates inputs to hit new edges.

## Diagram: Fuzz loop

```
seed corpus

v
mutate → run FuzzXxx → new coverage?

 yes → keep input

crash/hang testdata/fuzz/...
```

## Coverage

```
go test -cover ./...
go test -coverprofile=c.out ./...
go tool cover -html=c.out
go test -covermode=atomic ./... # concurrent-safe counts
```

Integration-style:

```
go build -cover -o app .
GOCOVERDIR=covdata ./app # exercise
go tool covdata textfmt -i=covdata -o=c.out
```

Use coverage to find **untested error paths**, not as a vanity percentage.

## Fuzz Engine Model

```
seed corpus
-> mutate
-> execute FuzzXxx
-> instrumentation: new coverage edge?
 yes -> keep interesting input
-> crash/hang -> fail and write testdata/fuzz/...
```

```
go test -fuzz=FuzzParse -fuzztime=30s
```

Commit failing inputs under `testdata/fuzz` for regression.

## Writing Good Fuzz Targets

1. Round-trip properties (`decode(encode(x))`)
2. Invariants (no panic, bounds)
3. Avoid non-determinism and network
4. Keep work per input small

## Coordination With Race

```
go test -race -fuzz=FuzzX -fuzztime=10s
```

Heavier but catches races fuzz alone may miss.

## Experiment

```
mkdir /tmp/fuzzdemo && cd /tmp/fuzzdemo
go mod init example
```

```
// reverse.go
package example

func Reverse(s string) string {
 r := []rune(s)
 for i, j := 0, len(r)-1; i < j; i, j = i+1, j-1 {
 r[i], r[j] = r[j], r[i]
 }
 return string(r)
}
```

```
// reverse_test.go
package example

import "testing"

func FuzzReverse(f *testing.F) {
 f.Add("gopher")
 f.Fuzz(func(t *testing.T, s string) {
 if Reverse(Reverse(s)) != s {
 t.Fatalf("%q", s)
 }
 })
}

go test -fuzz=FuzzReverse -fuzztime=5s -cover
```

**What to notice:** Fuzz spends time exploring UTF-8 edges; coverage rises as new branches hit.

**Try next:** Fuzz a parser you own; keep the first crash input as a unit test.

## **Build Cache and Reproducibility**

# Build Cache and Reproducibility

## Overview

The Go toolchain caches build actions aggressively. Understanding the cache explains “why CI rebuilt everything” and how to aim for **bit-reproducible** artifacts.

## Diagram: Reproducible build

```
sources + flags + go version
 v
 GOCACHE
 v
objects link (-trimpath) binary
```

## Cache Location

```
go env GOCACHE
go clean -cache
go clean -modcache # modules, separate
```

Cache keys include source, build flags, env that affects compile, cgo toolchains, tags, GOOS/GOARCH, etc.

## What Busts Cache

- Touching files unnecessarily
- Changing CGO\_\* flags
- Unstable generated code (timestamps in source)
- Different Go patch versions
- `-trimpath` vs not when comparing binaries



## Reproducible Flags

```
go build -trimpath -ldflags='-buildid= -s -w' -o app .
```

Flag	Effect
-trimpath	Remove local paths from binaries
-buildid=	Empty buildid for stabler hashes
-s -w	Strip symbols (optional)
CGO_ENABLED=0	Fewer host dependencies

## Module Reproducibility

```
go mod download
go mod verify
go build -mod=readonly
or
go build -mod=vendor
```

Lock with `go.sum`. Pin `toolchain` in `go.mod` for compiler version policy.

## Experiment

```
go mod init example
echo 'package main; func main(){}' > main.go
go build -trimpath -ldflags='-buildid=' -o a1 .
go build -trimpath -ldflags='-buildid=' -o a2 .
shasum a1 a2 # often identical pure-Go
```

**What to notice:** Pure Go + `trimpath` often matches; `cgo` and `ld` versioning frequently break reproducibility.

**Try next:** In CI, print `go version`, `go env`, and `go list -m all` on mismatch incidents.

# GOEXPERIMENT Catalog

# GOEXPERIMENT Catalog

## Overview

GOEXPERIMENT enables incomplete or alternate runtime/compiler features. Experiments graduate, change, or vanish — always pair with release notes.

```
GOEXPERIMENT=name go test ./...
go doc go/token # not helpful; use:
go tool dist list
list experiments for your version:
go doc internal/goexperiment # may be unavailable; see src/internal/goexperiment
```

From user-facing docs / release notes (illustrative — verify for your version):

Experiment	Theme
greenteagc / nogreenteagc	Green Tea GC default since 1.26; opt-out remains
simd	Experimental portable <code>simd</code> + <code>simd/archsimd</code> (1.26–1.27)
nosizespecializedmalloc	Opt out of 1.27 size-specialized small allocs (expected gone in 1.28)
nojsonv2	Restore pre-1.27 <code>encoding/json</code> engine (temporary)
arenas	Historical arena experiment (do not bet new code without checking status)
loopvar	Historical loop var semantics (graduated)
rangefunc	Historical range-over-func (graduated)
newinliner	Inliner experiments

## Diagram: Experiment lifecycle

```
flow:
 [Test]
 |
 v
 [Grad]
```

```
[Test]
 |
 v
[Drop]
```

## Rules

1. **Never** enable random experiments in production without a rollback plan.
2. Test performance and correctness under the experiment **and** default.
3. Prefer graduated language features over experiments for library APIs.

## JSON v2 and Friends

encoding/json/v2 graduated in Go 1.27. The v1 package uses the v2 engine; `GOEXPERIMENT=nojsonv2` is an emergency opt-out. `goroutineleakprofile` was deleted when the leak profile became generally available.

## Experiment

```
go version
GOEXPERIMENT=nogreenteagc go test ./... # only if valid on your version
or simply read:
go help build | less
```

Document in your runbook which experiments you intentionally ship (ideally none).

**Try next:** Skim the Go 1.27 release notes “Runtime” + “Compiler” sections and map each experiment bullet to a chapter in this part.

# WebAssembly and wasip1

# WebAssembly and wasip1

## Overview

Go can target **js/wasm** (browser + wasm\_exec.js) and **wasip1** (WASI preview1) for server-side Wasm runtimes. Constraints differ sharply from linux/amd64.

## Diagram: Wasm targets

```
Go source
 GOOS=js GOARCH=wasm browser + wasm_exec
 GOOS=wasip1 GOARCH=wasm WASI runtime
```

## Targets

```
GOOS=js GOARCH=wasm go build -o main.wasm .
GOOS=wasip1 GOARCH=wasm go build -o main.wasm .
```

## Constraints

Area	Reality
Threads	Limited / host-dependent
Networking	WASI and browser models differ
Filesystem	Capability-based under WASI
cgo	Generally unavailable
Binary size	Larger than tiny Go-less Wasm; use strip/trimpath

## Use Cases

- Policy plugins
- Edge filters
- Portable CLI sandboxes
- Browser tools (js/wasm)

## Experiment

```
if wasip1 supported:
printf 'package main\nimport "fmt"\nfunc main(){ fmt.Println("hi wasm") }\n' > /tmp/w.go
cd /tmp && GOOS=wasip1 GOARCH=wasm go build -o hi.wasm w.go
ls -la hi.wasm
run with a WASI runtime if installed, e.g. wasmtime hi.wasm
```

**What to notice:** Build succeeds without a local WASI runner; size is non-trivial compared to a hello C program.

**Try next:** Compare GOOS=wasip1 vs native binary size with `-ldflags='-s -w'`.

## **String and []byte Internals**



# String and []byte Internals

## Overview

Strings are immutable `ptr+len` headers over bytes (usually UTF-8). `[]byte` is `ptr+len+cap`. Converting between them often **allocates** — a hot-path footgun.

## Diagram: Headers

```
diagram:
 S[string ptr+len] --> Imm[immutable bytes]
 B[slice ptr+len+cap] --> Mut[mutable]
 Conv[string b / byte s] --> Copy[usually copy]
```

## Headers

```
string: { data *byte, len int } // immutable
[]byte: { data *byte, len int, cap int } // mutable
```

```
s := "go"
b := []byte(s) // copy
s2 := string(b) // copy
```

Compiler sometimes proves conversions are temporary and elides copies (e.g. map lookup `m[string(b)]` optimizations in modern Go) — **measure**, don't assume.

## Range and Indexing

```
for i, r := range s { /* r is rune; i is byte index */ }
s[i] // byte, not rune
```

## unsafe Conversions (Advanced)

```
// historical patterns; prefer safer APIs / newer runtime helpers when available
// wrong lifetime → memory corruption
```

Only with full understanding of immutability: never mutate a `[]byte` view of a string.

## Builder and Buffer

- `strings.Builder` — string assembly
- `bytes.Buffer` — also `io.Writer`
- `bytealg` runtime helpers power `strings/bytes` (assembly on some arch)

## Experiment

```
go test -bench=. -benchmem

package conv_test

import "testing"

var sink string

func BenchmarkByteToString(b *testing.B) {
 raw := []byte("authorization: bearer ...")
 for i := 0; i < b.N; i++ {
 sink = string(raw)
 }
}

func BenchmarkMapKey(b *testing.B) {
 m := map[string]int{"authorization: bearer ...": 1}
 raw := []byte("authorization: bearer ...")
 for i := 0; i < b.N; i++ {
 _ = m[string(raw)]
 }
}
```

**What to notice:** Map key optimization may reduce allocs vs naive assign to `sink`.

**Try next:** Profile a JSON logger converting large `[]byte` bodies to string.

## Error Interface Cost and Wrapping

# Error Interface Cost and Wrapping

## Overview

`error` is an interface — values box dynamic types. Wrapping builds a chain inspected by `errors.Is` / `As`. Correctness first; cost matters in ultra-hot paths.

## Diagram: Error interface

```
flow:
 [Chain]
 |
 v
 [Is]
```

## Representation

```
type error interface{ Error() string }
// concrete: *errors.errorString, fmt.wrapError, custom types
```

Returning `error` from a function may allocate when constructing `fmt.Errorf` or custom structs.

## Wrapping

```
return fmt.Errorf("open %s: %w", path, err)
errors.Is(err, fs.ErrNotExist)
errors.As(err, &pathErr)
errors.Join(err1, err2)
```

## Performance Notes

Action	Cost driver
<code>errors.New</code> sentinel	One-time
<code>fmt.Errorf</code> per call	Alloc + formatting
Deep <code>Is</code> chains	Walk + interface asserts
<code>err.Error()</code> strings	May alloc; avoid in hot success path

Pattern: construct rich errors on **failure only**; keep success path alloc-free.

## Sentinel vs Types

```
var ErrNotFound = errors.New("not found")

type NotFoundError struct{ Key string }
func (e *NotFoundError) Error() string { return "not found: " + e.Key }
```

## Experiment

```
go test -bench=. -benchmem

package errcost_test

import (
 "errors"
 "fmt"
 "testing"
)

var ErrX = errors.New("x")

func BenchmarkSentinel(b *testing.B) {
 for i := 0; i < b.N; i++ {
 _ = ErrX
 }
}

func BenchmarkWrap(b *testing.B) {
 for i := 0; i < b.N; i++ {
 _ = fmt.Errorf("wrap: %w", ErrX)
 }
}
```

```
func BenchmarkIs(b *testing.B) {
 err := fmt.Errorf("wrap: %w", ErrX)
 for i := 0; i < b.N; i++ {
 _ = errors.Is(err, ErrX)
 }
}
```

**What to notice:** Reusing sentinels is free; wrapping each time allocates.

**Try next:** Audit a hot path that does `fmt.Errorf` on success-adjacent branches.

## **Production Mistakes Deep Dive**

# Production Mistakes Deep Dive

## Overview

Recurring themes from production Go engineering discussions (including widely shared checklists on X): timeouts, leaks, default clients, lock+I/O, and observability gaps. This chapter maps each mistake to **runtime symptoms** and **fixes** covered elsewhere in this part.

## Diagram: Incident loop

```
flow:
 [Prof]
 |
 v
 [Trace]

 [Trace]
 |
 v
 [Fix]
```

## The List (Mapped)

Mistake	Symptom	Deep dive / fix
No context timeouts	Goroutine + FD growth	<a href="#">Context trees</a> , <a href="#">netpoller</a>
Unbounded goroutines	RSS↑, sched latency	<a href="#">Scheduler</a> , <a href="#">leaks</a>
Default <code>http.Client</code>	Hung dials/TLS	<a href="#">Transport pool</a>
Errors as strings	Untestable retries	<a href="#">Error cost</a>
<code>nil</code> vs empty JSON	Client contract breaks	<a href="#">encoding stdlib</a>
Unbounded channels	Memory spikes	<a href="#">Channel internals</a>
Lock held over I/O	Throughput collapse	<a href="#">Mutex runtime</a>
<code>time.After</code> in hot select	Timer/alloc churn	<a href="#">Timers</a>
DB without limits/context	Connection storms	<a href="#">HTTP/DB practices</a>
Logs without IDs/metrics	Blind incidents	<a href="#">slog</a> , observability parts



## Incident Playbook

1. NumGoroutine / FD count / RSS chart
2. goroutine pprof (wait sites)
3. mutex/block profile if CPU low
4. trace 200-500ms around incident
5. check client/server timeouts
6. check deploy diff for "go func" and "time.After"

## Engineering Questions (From “knowing Go”)

1. Do you need concurrency at all?
2. Goroutine or worker pool?
3. Channel or mutex?
4. How does context propagate?
5. Did you run `-race`, `pprof`, `-gcflags=-m`?

## Mini Lab

Take any service and grep:

```
rg -n 'http\\.Get|http\\.DefaultClient|time\\.After|go func|context\\.Background\\(\\)' --gl
```

Classify each hit as safe or debt.

**Try next:** Add a CI check that fails on `http.Get` in non-test packages (project policy).

## **Production Debugging Tooling**

# Production Debugging Tooling

## Overview

When production hurts, you rarely attach a debugger first. You grab **profiles, traces, stacks, and core-ish dumps** — fast.

## Diagram: Debug surfaces

```
flow:
 [Pprof]
 |
 v
 [Heap]

 [Pprof]
 |
 v
 [Goroutine]
```

## Always-On Endpoints (Carefully)

```
import _ "net/http/pprof"
// bind to internal listener only
go http.ListenAndServe("127.0.0.1:6060", nil)
```

Expose via secure admin network / port-forward, never public internet.

## Signals

Signal	Common action
SIGQUIT	Stack dump (often)
SIGTERM	Graceful shutdown
panic	GOTRACEBACK controls detail

```
GOTRACEBACK=all ./app
kill -QUIT $pid # platform dependent behavior
```

## Delve (Dev / Staging)

```
dlv attach $pid
dlv dap # editor
```

Prefer staging for breakpoints; prod attach can freeze worlds.

## Core / Crash

```
Linux examples - ops dependent
ulimit -c unlimited
GOTRACEBACK=crash
```

Pair with symbolized binaries (don't strip staging).

## Continuous Profiling

Products sample CPU/heap cheaply over time — use alongside on-demand pprof. Compare versions after deploys.

## Checklist

1. Secure pprof listener
2. Mutex profile fraction documented
3. Version/build stamped (-X)
4. Runbooks for “goroutine explosion” and “GC CPU”

## Experiment

```
go mod init example
tiny server with pprof
```

```
package main
```

```
import (
```

```

 "net/http"
 _ "net/http/pprof"
 "time"
)

func main() {
 go func() {
 for {
 time.Sleep(10 * time.Millisecond)
 }
 }()
 _ = http.ListenAndServe("127.0.0.1:6060", nil)
}

other terminal
go tool pprof http://127.0.0.1:6060/debug/pprof/goroutine

```

**What to notice:** Sleeping G still appears in goroutine profile; counts matter more than one stack.

**Try next:** Automate capturing 30s CPU profile on SIGUSR1 in a side project.

## **sysmon, Scavenger, and Background Runtime**

# sysmon, Scavenger, and Background Runtime

## Overview

Besides your goroutines, the runtime runs **background machinery**: monitoring long preemption needs, retaking Ps stuck in syscalls, returning memory to the OS (**scavenger**), and coordinating GC.

## Diagram: Background runtime

```
flow:
 [Sysmon]
 |
 v
 [Netpoll]
```

## sysmon (System Monitor)

A runtime thread that periodically:

- Retakes Ps from Ms blocked in syscalls too long
- Helps with preemption / netpoll wakeups (version-dependent details)
- Notices sleeping conditions

You do not call sysmon — but its work appears when syscall-heavy loads misbehave.

## Memory Scavenging

After GC frees spans, memory may remain reserved to the process. The scavenger **returns idle heap pages to the OS** (with hysteresis) so RSS can fall.

```
HeapInuse vs HeapSys vs RSS
HeapInuse - occupied objects
HeapSys - virtual from OS for heap
RSS - OS resident (includes scavenged lag)
```

GOMEMLIMIT and GC interact with how aggressively memory returns.

## GC Workers

Mark workers and assist run as special Gs. High `GCCPUFraction` means the mutator is paying for GC.

## What You Control

Control	Effect
<code>GOMAXPROCS</code>	Parallel Go + related resources
<code>GOGC</code> / <code>GOMEMLIMIT</code>	GC frequency & memory ceiling
Allocation rate	Drives all of the above
Syscall/cgo rate	Pressure on Ms and sysmon retake

## Experiment

```
GODEBUG=gctrace=1 go run .
```

```
package main

import (
 "fmt"
 "runtime"
 "time"
)

func main() {
 var m runtime.MemStats
 hold := make([][]byte, 0, 500)
 for i := 0; i < 500; i++ {
 hold = append(hold, make([]byte, 100*1024))
 }
 runtime.ReadMemStats(&m)
 fmt.Printf("hold HeapInuse=%dMiB HeapSys=%dMiB\n", m.HeapInuse>>20, m.HeapSys>>20)
 hold = nil
 runtime.GC()
 time.Sleep(200 * time.Millisecond)
 runtime.ReadMemStats(&m)
 fmt.Printf("after HeapInuse=%dMiB HeapSys=%dMiB\n", m.HeapInuse>>20, m.HeapSys>>20)
}
```

**What to notice:** HeapInuse drops quickly after GC; HeapSys/RSS may lag until scavenging.

**Try next:** Compare with `GOMEMLIMIT` set near working set size.



## Reading the Go Runtime Source

# Reading the Go Runtime Source

## Overview

The ultimate deep dive is **src/runtime** in the Go tree. You do not need to memorize it — you need a map, a debugger for experiments, and humility about version drift.

Workshop-style learning (modifying the toolchain) is increasingly popular for advanced gophers.

## Diagram: Source map

```
symptom (latency / leak / panic)
```

```
v
```

```
pick runtime area
```

```
proc.go scheduler
```

```
chan.go channels
```

```
mgc*.go GC
```

```
netpoll*.go network
```

## Entry Points

Area	Files (names evolve)
Scheduler	proc.go, runtime2.go
Channels	chan.go
Maps	map*.go
GC	mgc*.go, mheap.go
Netpoll	netpoll*.go
Stacks	stack.go
Timers	time.go
Interface	iface.go / abi
Assembly	asm_*.s, sys_*.s

Clone matching your version:

```
git clone --depth 1 --branch go1.27.1 https://github.com/golang/go
cd go/src/runtime
rg -n 'type hchan'
rg -n 'func schedule'
```

## How To Study

1. Pick a symptom (e.g. send on closed panics).
2. Find the user-facing panic string in runtime.
3. Walk callers one level at a time.
4. Write a 10-line program that hits the path.
5. Optional: dlv on a rebuilt go tool (advanced).

## go tool objdump / nm

```
go build -o app .
go tool nm app | rg 'runtime\\.'
go tool objdump -s 'main\\.main' app | head
```

## Experiments Without Patching

- GODEBUG traces
- `go build -gcflags=-m`
- Race detector as dynamic analysis
- Breakpoints in **your** code at API boundaries, infer runtime

## Experiments With Patching (Optional)

- Add a `println` in a runtime helper (local toolchain)
- Use `go install` from a branch
- Never ship patched runtimes without extreme need

## Field Guide: This Book → Source

Chapter	Start reading
201 GMP	<code>proc.go</code> <code>schedule/findrunnable</code>
203 channels	<code>chan.go</code>
208 GC	<code>mgc.go</code>

Chapter	Start reading
213 netpoll	<code>netpoll_*.go</code>
214 timers	<code>time.go</code>
228 cgo	<code>cgocall.go</code>

## Experiment

```
on a full go checkout matching your version
rg -n "panic\\(.*close of closed channel" "$(go env GOROOT)/src/runtime"
rg -n "func makeslice" "$(go env GOROOT)/src/runtime"
```

**What to notice:** Panic strings and helper names are breadcrumbs; follow them instead of random file browsing.

**Try next:** Pick one production bug class from [mistakes](#) and find the runtime check that enforces it.

## **crypto/tls Handshake Path**

# crypto/tls Handshake Path

## Overview

TLS is often the hidden cost in “simple HTTP” clients. Handshake CPU, session resumption, and certificate verification dominate cold connections; pooling and HTTP/2 amortize them.

## Diagram: TLS then HTTP

```
flow:
 [HS]
 |
 v
 [App]
```

## Path

```
Dial TCP
-> TLS client hello
-> server hello / certs
-> key exchange / finished
-> Application data (HTTP)
```

```
cfg := &tls.Config{
 MinVersion: tls.VersionTLS12,
 // ServerName for SNI verification when not in address
}
d := tls.Dialer{Config: cfg, NetDialer: &net.Dialer{Timeout: 5 * time.Second}}
conn, err := d.DialContext(ctx, "tcp", "example.com:443")
```

## Cost Drivers

Factor	Effect
New TCP+TLS each request	Highest latency

Factor	Effect
Session tickets / resumption	Cheaper reconnects
Huge trust stores / custom Verify	CPU
HTTP/2 reuse	One handshake, many streams
mTLS	Client certs + private key ops

## Client Integration

`http.Transport.TLSClientConfig` and idle pools determine whether you pay handshake per request. See [Transport](#).

## Security Notes

- Pin `MinVersion`
- Avoid `InsecureSkipVerify` outside tests
- Prefer system roots or explicit pinned CAs for internal mesh

Deep mTLS ops: [TLS/mTLS](#).

## Experiment

```
go mod init example
optional network:
go run . example.com:443

package main

import (
 "crypto/tls"
 "fmt"
 "net"
 "os"
 "time"
)

func main() {
 addr := "example.com:443"
 if len(os.Args) > 1 {
 addr = os.Args[1]
 }
 start := time.Now()
 conn, err := tls.DialWithDialer(
```

```

 &net.Dialer{Timeout: 5 * time.Second},
 "tcp",
 addr,
 &tls.Config{MinVersion: tls.VersionTLS12},
)
 if err != nil {
 fmt.Println("err", err)
 return
 }
 defer conn.Close()
 st := conn.ConnectionState()
 fmt.Printf("version=%x resumed=%v alpn=%q time=%s\n",
 st.Version, st.DidResume, st.NegotiatedProtocol, time.Since(start))
}

```

**What to notice:** First dial slower; second process may still be cold unless session cache configured at higher layers.

**Try next:** Compare `http.Transport` with `ForceAttemptHTTP2` true/false under parallel GETs.



## **slog Handler Design Deep Dive**

# slog Handler Design Deep Dive

## Overview

log/slog separates **Logger** (API) from **Handler** (serialization + output). Custom handlers power redaction, sampling, OpenTelemetry bridges, and multi-sink fanout.

Basics: [fmt/log/slog](#), [structured logging](#).

## Diagram: Logger vs Handler

```
slog.Logger

 v
Handler
 Enabled(level)?
 Handle(Record) JSON / text / OTel
 WithAttrs / WithGroup
```

## Interfaces

```
type Handler interface {
 Enabled(ctx context.Context, level slog.Level) bool
 Handle(ctx context.Context, r slog.Record) error
 WithAttrs(attrs []slog.Attr) Handler
 WithGroup(name string) Handler
}
```

Rules for custom handlers:

1. Honor **Enabled** to avoid expensive attr work
2. **WithAttrs/WithGroup** return new handlers (immutability)
3. Be concurrent-safe — loggers are shared
4. Do not panic on normal records

## Patterns

Pattern	Idea
Middleware handler	Wrap another handler; filter/redact
Sampling	Drop debug under load
Fanout	Write JSON + ship metrics counter
Context attrs	Pull request_id from ctx in Handle

```
type RedactHandler struct{ slog.Handler }

func (h RedactHandler) Handle(ctx context.Context, r slog.Record) error {
 r.Attrs(func(a slog.Attr) bool {
 if a.Key == "password" || a.Key == "token" {
 // rebuild record without secrets - production code clones carefully
 }
 return true
 })
 return h.Handler.Handle(ctx, r)
}
```

## Performance

- JSON handler cost encoding + I/O
- Avoid `fmt.Sprintf` in hot attr values
- Precompute constant attrs via `With`

## Experiment

```
go mod init example
go run .

package main

import (
 "context"
 "log/slog"
 "os"
)

type MinLevel struct {
 slog.Handler
```

```

 min slog.Level
}

func (h MinLevel) Enabled(ctx context.Context, l slog.Level) bool {
 return l >= h.min && h.Handler.Enabled(ctx, l)
}

func main() {
 base := slog.NewJSONHandler(os.Stdout, nil)
 log := slog.New(MinLevel{Handler: base, min: slog.LevelInfo})
 log.Debug("hidden")
 log.Info("visible", "n", 1)
}

```

**What to notice:** Debug suppressed without formatting work beyond Enabled checks on the wrapper.

**Try next:** Add attribute redaction for keys `authorization` and `cookie`.

## **GOMAXPROCS, cgroups, and Containers**

# GOMAXPROCS, cgroups, and Containers

## Overview

In Kubernetes and cgroup-limited environments, **CPU quota** **host core count**. Older Go versions defaulted GOMAXPROCS to machine CPUs, causing thrashing inside small pods. Modern Go improves cgroup awareness — still verify in your version and platform.

## Diagram: Quota mismatch

```
flow:
 [Pod]
 |
 v
 [Bad]

 [Pod]
 |
 v
 [Good]
```

## Failure Mode

```
Pod limit: 500m CPU (0.5 core)
GOMAXPROCS=16 (node size)
-> 16 threads fight for 0.5 CPU
-> latency noise, steal time, GC assists hurt more
```

## Practices

```
// log at boot
slog.Info("sched", "gomaxprocs", runtime.GOMAXPROCS(0), "numCPU", runtime.NumCPU())
```

- Prefer runtime auto-detect on current Go
- Or set explicitly from downward API / cgroup read

- Libraries like `automaxprocs` existed historically for this gap

## Memory Side

Pair CPU limits with `GOMEMLIMIT` near container memory limit (leave headroom for non-Go RSS, caches).

```
GOMEMLIMIT=400MiB ./app
```

## Experiment

```
go run .

package main

import (
 "fmt"
 "runtime"
)

func main() {
 fmt.Println("NumCPU", runtime.NumCPU())
 fmt.Println("GOMAXPROCS", runtime.GOMAXPROCS(0))
}
```

Run inside and outside a constrained container if you have Docker:

```
docker run --rm --cpus=0.5 -v "$PWD":/app -w /app golang:1.27 go run .
```

**What to notice:** Compare printed values to `--cpus` quota on your platform/Go version.

**Try next:** Load test a CPU-bound endpoint with `GOMAXPROCS` 1 vs 4 under a 1 CPU quota.

## **Plugins and buildmode Pitfalls**



# Plugins and buildmode Pitfalls

## Overview

`buildmode=plugin` looks like a dynamic extension story. In production it is usually a **trap**: brittle ABI, limited platforms, and operational pain. Prefer processes, Wasm, or explicit interfaces.

## Diagram: Prefer process plugins

need extension point?

side process + RPC	(preferred)
Wasm sandbox	
buildmode=plugin	(fragile: skew, platforms)

## Reality Check

Issue	Detail
Platforms	Not universal
Version skew	Plugin must match main toolchain/deps closely
Dependency bloat	Duplicated modules risk
Debugging	Harder stacks and deploys
Security	Loading code = trust boundary

## Prefer Alternatives

1. **Side process + RPC** (gRPC/HTTP) — isolation, language freedom
2. **Wasm plugins** — sandboxing story
3. **Compile-time interfaces** + separate binaries
4. **HashiCorp go-plugin** style (subprocess) if you need polyglot

## If You Must

```
go build -buildmode=plugin -o plug.so ./plug
open via plugin.Open; lookup symbols
```

Pin exact Go version + `go.mod` policy; test load on the **identical** base image as production.

## Experiment

Skip coding plugins by default. Instead document a design choice:

```
feature flag -> separate binary -> RPC
vs
plugin.Open
```

Write a 10-line ADR in your repo choosing RPC.

**Try next:** Read `go help buildmode` end-to-end once; note which modes you actually need (usually `exe` only).

## Topic Radar (X + Ecosystem Signals)

# Topic Radar (X + Ecosystem Signals)

## Overview

This chapter captures **themes that repeatedly surface** in Go engineering discourse on X and in adjacent blogs — used as a backlog radar for this deep-dives part. Signals change; the curriculum should track *problems*, not hype.

## Diagram: Radar loop

```
flow:
 [Repro]
 |
 v
 [Chapter]

 [Chapter]
 |
 v
 [Link]
```

## Hot Themes Observed

### Runtime & scheduler

- GMP explainers and GOMAXPROCS capacity math
- Go 1.26 removal of P-level syscall state / cgo path simplification
- Go 1.27 generic methods, json/v2 GA, goroutineleak profile
- Channels as runtime queues (sudog), not magic
- GC pauses as product latency (Green Tea GC chatter)

### Correctness under concurrency

- Context timeouts as non-optional
- Goroutine leak checklists
- Race detector as culture, not CI checkbox
- `testing/synctest` replacing `sleep` tests

## Performance craft

- Escape analysis + pprof as literacy
- `sync.Pool` and zero-alloc append patterns
- Zero-copy `io.Copy` / sendfile limits with TLS
- `time.After` abuse in hot selects

## Production engineering

- Default `http.Client` considered harmful
- Structured logs with request IDs
- Mutex held across I/O
- Container CPU limits vs GOMAXPROCS

## Learning meta

- “Features engineering”
- Read `src/runtime` workshops
- Interview lists that mix runtime + design judgment

## Mapped Into This Part

Signal	Chapter
GMP / syscall P changes	201, 228
Channel internals posts	203
synctest runtime (e.g. internals-for-interns.com)	223
Zero-copy / sendfile	224
Production top mistakes	237
GC / pprof	208, 222
Pool / allocs	219
Reading source	240

## Still-Fertile Backlog (Future)

- `encoding/json/v2` migration cookbooks now that 1.27 shipped stable defaults
- Full Swiss-map implementation walk with diagrams per release
- Runtime metrics export (`/debug` → OTel) cookbook
- FIPS / boringcrypto build graphs for regulated shops
- Delve scripting for runtime bugs
- Port of `runtime` execution tracer event schema reference
- Arena alternatives pattern library (`sync.Pool` arenas, slab allocators)
- CPU profile vs wall-clock eBPF cross-check on Linux

## How To Extend This Book

When a topic trends:

1. Write a **minimal repro** that shows the symptom.
2. Add a chapter under `20-go-deep-dives/` with experiment + cross-links.
3. Link from this radar and from [index](#).
4. Avoid duplicating Part 07/18 tutorials — keep the *why/runtime* angle.

## Experiment

```
personal radar loop
rg -n 'TODO|FIXME|race|timeout' --glob '*.go' | head
```

Convert one production footgun into a regression test this week.

**Try next:** Subscribe to Go release notes + one runtime blog; every release, tick experiments in [GOEXPERIMENT](#).

## **Article Radar: Internals for Interns + Kin**

# Article Radar: Internals for Interns + Kin

## Overview

This radar catalogs **diagram-rich Go internals writing** in the same vein as *Inside Go's sync\_test*—runtime source, compiler pipeline, and systems guests. Use it to prioritize reading and to track what this monorepo has absorbed into local chapters.

## Diagram: map sources → book chapters

IFI runtime series	this book
bootstrap	248
allocator	247
scheduler	201, 256
GC	208, 255
netpoll / sysmon	213, 239
sync_test	223

## Internals for Interns — Runtime series

Article	URL	Book landing
Bootstrap	<a href="#">understanding-go-runtime</a>	248
Memory allocator	<a href="#">go-memory-allocator</a>	247
Scheduler	<a href="#">go-runtime-scheduler</a>	201
Garbage collector	<a href="#">go-garbage-collector</a>	208
System monitor	<a href="#">go-sysmon</a>	239
Network poller	<a href="#">go-netpoller</a>	213
Slices, maps, channels	<a href="#">go-runtime-slices-maps-channels</a>	203, 205
select	<a href="#">go-runtime-select</a>	249



Article	URL	Book landing
Stacktraces	<a href="#">go-runtime-stacktraces</a>	<a href="#">250</a>
Reflect	<a href="#">go-runtime-reflect</a>	<a href="#">211</a>
Profiling	<a href="#">go-runtime-profiling</a>	<a href="#">222</a>
synctest (guest)	<a href="#">inside-go-synctest</a>	<a href="#">223</a>
mmap vs pread (guest)	<a href="#">mmap-vs-pread-go-storage-engine</a>	<a href="#">251</a>

Series hub: [Understanding the Go Runtime](#).

## Internals for Interns — Compiler series

Phase	URL	Book landing
Scanner	<a href="#">the-go-lexer</a>	<a href="#">209</a> + radar
Parser	<a href="#">the-go-parser</a>	<a href="#">209</a>
Type checker	<a href="#">the-go-type-checker</a>	<a href="#">209</a>
Unified IR	<a href="#">go-compiler-unified-ir</a>	<a href="#">209</a>
IR	<a href="#">the-go-ir</a>	<a href="#">209</a>
SSA	<a href="#">the-go-ssa</a>	<a href="#">209</a>
Machine code	<a href="#">the-go-machine-code</a>	<a href="#">209</a>
Linker	<a href="#">the-go-linker</a>	<a href="#">210</a>

## Kindred sources (same “diagram + source” energy)

Source	Why
<a href="#">antonz.org Gist of Go: Concurrency</a>	Interactive curriculum → part <a href="#">21</a>
<a href="#">iximiuz Labs: Using Go for Systems Programming</a>	libc vs direct syscalls, static binaries, CGO_ENABLED=0 → <a href="#">140a</a>
<a href="#">Go scheduler talk (Vyukov)</a>	Original lightweight concurrency design
<a href="#">go.dev blog</a>	Official GC, GOMAXPROCS, experiments
Channel runtime posts (e.g. gaborkoos / similar)	hchan latency mental models → <a href="#">203</a>

Source	Why
Zero-copy sendfile/splice writeups	→ <a href="#">224</a>

## How we absorb articles

1. Extract durable models (not copy prose)
2. Add mermaid/ASCII diagrams in-repo
3. Cross-link experiments + GOROOT reading tips
4. Attribute the source series at chapter tops where used heavily
5. Re-check after each Go minor release

## Experiment

Pick one unread row in the tables above; after reading, open the matching book chapter and add one personal note (sticky TODO in a `_planning` file if you keep one) for anything the monorepo still under-explains.

**Try next:** On each Go release, re-open the GC + scheduler posts and diff against [208](#) / [201](#).

# Memory Allocator Deep Dive

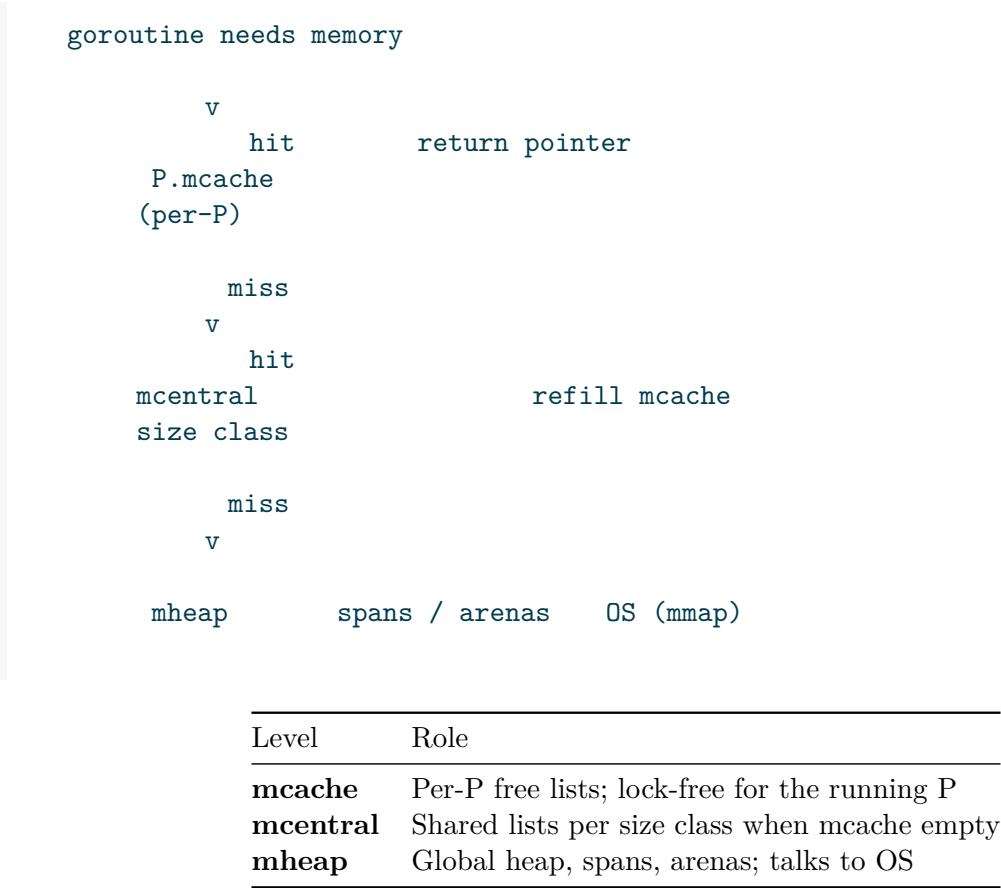
# Memory Allocator Deep Dive

## Overview

Go’s allocator is a **warehouse manager**: hand out differently sized objects fast, keep fragmentation in check, and cooperate with the GC. Most small allocations hit a **per-P cache** and avoid locks.

Teaching map aligned with runtime walkthroughs such as [Internals for Interns — Memory Allocator](#). Original prose for this book.

## Diagram: three-level hierarchy



## Size classes and spans

```
tiny / small objects → fixed size classes → span of pages carved into slots
large objects → dedicated spans (page-rounded)
```

Go 1.27 generates **size-specialized** allocation calls for some objects under 80 bytes (up to ~30% cheaper on that path; ~1% overall on alloc-heavy programs). Disable with `GOEXPERIMENT=nosizespecializedmalloc` if you need a bisect.

```
flow:
 [Class]
 |
 v
 [Slot]

 [Slot]
 |
 v
 [Ptr]
```

## Arenas and address space

Modern Go reserves large **arenas** of virtual address space, then commits pages as needed. That keeps metadata dense and growth predictable compared to many tiny `mmaps`.

## Interaction with GC and P

- Each **P** owns an mcache so allocation does not bounce on a global lock.
- GC **sweeps** spans; scavenger returns idle pages to the OS (239).
- Allocation pressure drives the **pacer** (208).

## Escape analysis still wins

The fastest allocation is **no heap allocation**. Stack frames and SSA prove many values need no allocator at all (215, 209).

## Diagnosis

Signal	Meaning
<code>runtime.mallocgc</code> in CPU pprof	alloc-heavy path

Signal	Meaning
Heap profile top	retained objects
Alloc profile top	churn even if freed
High GCCPUFraction	pacer reacting to alloc rate

## Experiment

```
go test -bench=. -benchmem

func BenchmarkHeap(b *testing.B) {
 for i := 0; i < b.N; i++ {
 _ = make([]byte, 1024)
 }
}

func BenchmarkStack(b *testing.B) {
 for i := 0; i < b.N; i++ {
 var a [1024]byte
 sink = a[0]
 }
}

var sink byte
```

**What to notice:** stack-ish patterns show fewer alloc/op when the compiler keeps data off-heap.

**Try next:** GODEBUG=allocfreetrace=1 on a tiny program (noisy; educational only).

## Runtime Bootstrap

# Runtime Bootstrap

## Overview

Before `main.main`, the OS jumps into **runtime startup**: init the scheduler, allocator, GC metadata, threads, and package `init` graph. Every Go binary embeds this runtime.

Series-style depth: [Understanding the Go Runtime — Bootstrap](#).

## Diagram: from exec to main

```
OS loader
 v
runtime entry (asm)
 v
init scheduler + heap + main G
 v
package init() order
 v
main.main
```

## What gets set up

Subsystem	Early job
Allocator	arenas, spans ready for first <b>new</b>
Scheduler	<b>GOMAXPROCS</b> Ps, main M, main G
GC	metadata, write barrier off until needed
Signals	platform signal plumbing
Netpoll	poll descriptor (platform)



## Package init order

```
imported packages first (dependency order)
→ file-level var initializers
→ init() functions
→ main.main
```

Do not rely on clever cross-package init races; make dependencies explicit.

## Main goroutine

`main.main` runs on the **main goroutine**. When it returns, the runtime shuts down the process (other Gs are not gracefully joined unless you waited).

## Experiment

```
go build -o /tmp/hello .
optional: go tool objdump / nm to see runtime symbols
go tool nm /tmp/hello | rg 'runtime\\.(main|schedinit|newproc)' | head
```

```
package main

import "fmt"

func init() { fmt.Println("init") }
func main() { fmt.Println("main") }
```

**What to notice:** `init` always before `main`; runtime symbols dominate even tiny binaries.

**Try next:** Compare binary size `CGO_ENABLED=0` `hello` vs one that imports `net/http`.

**select: Compiler and selectgo**

# select: Compiler and selectgo

## Overview

`select` looks like `switch` but is a **compiler** + **runtime** duet. Simple shapes are rewritten; hard cases call `runtime.selectgo`.

Depth companion: [Internals for Interns — The select Statement](#).

## Diagram: two paths

```
select { ... }

 one case / simple compiler rewrite

 general runtime.selectgo

 park until one case wins
```

## Rules that matter

Rule	Meaning
Ready cases	If several ready, <b>uniform random</b> pick
All blocked	Wait until one ready (unless <b>default</b> )
<b>default</b>	Non-blocking poll
Nil channel case	Never selected (disable pattern)

## Pseudo-algorithm (teaching)

1. randomize case order
2. lock involved channels carefully (runtime does ordered locking)
3. probe for immediately ready op
4. if none and default → default
5. else enqueue this G on all wait queues; park
6. on wake: dequeue others; complete winner only

## Synctest note

Inside a synctest bubble, select is durable only if **every** non-nil case is bubbled ([223](#)).

## Experiment

```
go run .

package main

import "fmt"

func main() {
 a, b := make(chan int, 1), make(chan int, 1)
 counts := map[string]int{}
 for i := 0; i < 1000; i++ {
 a <- 1
 b <- 1
 select {
 case <-a:
 counts["a"]++
 case <-b:
 counts["b"]++
 }
 // drain the other so next iter both ready
 select {
 case <-a:
 default:
 }
 select {
 case <-b:
 default:
 }
 }
 fmt.Println(counts)
}
```

**What to notice:** Both cases win a substantial share—not left-to-right priority.

**Try next:** go build -gcflags=-S on a one-case select and a multi-case select; compare call sites.

## **Stacktraces and Runtime Metadata**

# Stacktraces and Runtime Metadata

## Overview

A stack trace is not magic: the **compiler and linker freeze metadata** into the binary; the runtime **unwinds** live frames using that data. Reflect uses a sibling idea—type descriptors instead of PC→func maps.

Companion read: [Internals for Interns — Stacktraces](#).

## Diagram: build time vs run time

```
regions: build time | run time
flow:
 [Unwind]
 |
 v
 [Meta]

 [Unwind]
 |
 v
 [Frames]
```

## Who prints stacks

Trigger	Behavior
panic	dump (controlled by GOTRACEBACK)
runtime.Stack	buffer of current or all Gs
pprof goroutine	stacks of live Gs
SIGQUIT (often)	all-G dump

```
GOTRACEBACK=all ./app
```

## Inlining and wrappers

Optimized binaries **inline** functions; traces may show fewer frames or wrapper names. Disable inlining for forensics:

```
go build -gcflags='-l' .
```

## Relation to profiling

CPU profiles sample stacks with the same unwinding family of mechanisms ([222](#)).

## Experiment

```
package main

import (
 "fmt"
 "runtime"
)

func deep(n int) {
 if n == 0 {
 buf := make([]byte, 4096)
 k := runtime.Stack(buf, false)
 fmt.Printf("%s", buf[:k])
 return
 }
 deep(n - 1)
}

func main() { deep(5) }
```

**What to notice:** Frames list `deep` repeatedly with decreasing depth.

**Try next:** Compare stacks with and without `-gcflags=-l`.

## **mmap vs pread in Go Storage**



# mmap vs pread in Go Storage

## Overview

Storage engines eventually ask: **how should we read bytes from files?** In Go the answer is rarely absolute—“always mmap” or “always pread”. The interesting design is an **abstraction** that prefers mmap when pages are warm and falls back when a fault would **block an M**.

Guest-systems depth: [mmap vs pread in a real Go storage engine](#) (Phuong Le / Internals for Interns). Patterns below are teaching synthesis.

## Diagram: two I/O styles

```
mmap path: map pages → load byte
 page fault?
 may block M on major fault

pread path: syscall ReadAt → copy into []byte
 G park rules better understood
```

## Tradeoffs

	mmap	pread
API	pointer-like access	explicit ReadAt
Warm data	very fast	syscall + copy
Cold data	major fault risk	controlled syscall
Go runtime	fault on M is painful	netpoll/syscall paths understood
Portability	OS-specific edges	straightforward

## Go-shaped recommendation

```
abstraction ReadAt(off, buf)
 if region cached / known resident → mmap view
 else → pread into buf
never let page faults surprise a hot P/M without budget
```

Prefer measuring **tail latency** under cold cache and concurrent queries, not only average throughput.

## Experiment

```
conceptual microbench outline
1) os.ReadAt loop over file
2) unix.Mmap + index loads
compare under: warm cache vs drop_caches (Linux ops only)
```

**What to notice:** Warm mmap wins; cold fault storms can look like “Go scheduler freeze” when Ms stuck in fault handling.

**Try next:** Read the guest post’s VictoriaLogs-oriented rationale end-to-end; map it to your own on-disk layout.

## **Compiler Frontend: Scan, Parse, Typecheck**

# Compiler Frontend: Scan, Parse, Typecheck

## Overview

go build is a pipeline. The **frontend** turns source text into a type-checked AST (and related forms) before IR/SSA. Understanding this half explains compile errors, `go/types`, and why some “optimizations” never see your source.

Series map: [Internals for Interns — Compiler](#) (scanner → type checker). Companion: [209 SSA/PGO](#).

## Diagram: frontend stages

```
.go source

 v
scanner → tokens → parser → AST

 v
 type checker

 v
 export / IR bridge
```

## Scanner (lexer)

- Characters → **tokens** (`package`, `IDENT`, `INT`, `+`, ...)
- Tracks positions for error messages and stack traces later
- No meaning yet—`fmt` and `Println` are just `idents` and a dot

## Parser

- Tokens → **AST** (`*ast.File`, `decls`, `stmts`, `exprs`)
- Enforces grammar; produces structure without full type knowledge
- `go/parser` in the standard library mirrors the idea for tools

```
fset := token.NewFileSet()
f, err := parser.ParseFile(fset, "main.go", src, 0)
```

## Type checker

- Resolves identifiers, assigns types, checks assignability
- Builds method sets, interfaces satisfaction, constant folding
- Emits the errors you actually fix day to day

```
undeclared name / cannot use X as Y / missing method M
```

Tools (`gopls`, `go vet`, many linters) reuse **go/types** rather than reimplementing rules.

## Unified IR / export data (bridge)

After typecheck, the compiler serializes a compact representation for:

- Build cache reuse
- Export data for importers of the package
- Feeding the mid-end (IR) without re-parsing forever

## What the frontend will not do

Expectation	Reality
Cross-package inlining decisions	Later (IR/SSA)
Register allocation	Backend
Fix algorithm bugs	Never

## Experiment

```
go install golang.org/x/tools/cmd/goimports@latest # optional tools
Dump AST for a file with the go/ast printer pattern, or:
go doc go/parser
go doc go/types
```

Write a 10-line program that uses `go/parser` + `ast.Inspect` to count `*ast.CallExpr` nodes in a file.

**Try next:** Feed the type checker a file with a deliberate interface mismatch; print `types.Config.Check` errors.

## **Compiler Mid/Back: IR, SSA, Machine Code, Link**

# Compiler Mid/Back: IR, SSA, Machine Code, Link

## Overview

After typecheck, the compiler optimizes in **IR**, lowers to **SSA**, emits **machine code** objects, then the **linker** produces a binary. This is where inlining, escape analysis, and PGO bite.

## Diagram: mid and back end

```
typed frontend
 v
IR (inline, escape, ...)
 v
SSA opts + arch lower
 v
.o objects linker binary
```

## IR responsibilities

Pass family	Effect
Inlining	Expand small callees; enable more opts
Escape analysis	Stack vs heap
Devirtualization	Concrete calls when type known
Walk / order	Prepare for SSA

Flags: `go build -gcflags='-m -m'` shows escape/inline decisions.

## SSA

- Every value assigned once → easy dataflow
- Dozens of rewrite passes (nilcheck elim, bounds prove, ...)
- Then **lower** to AMD64... / ARM64... ops

```
GOSSAFUNC=main go build . # classic SSA HTML dump when supported
go build -gcflags='-S' . 2>&1 | head
```

## Machine code + object files

- Encode instructions, relocations, PC line metadata
- One package → one (or more) object; still not runnable alone

## Linker

```
resolve symbols across packages
→ apply relocations
→ layout sections
→ emit ELF/Mach-O/PE + buildid
```

```
go build -ldflags='-s -w -X main.version=dev' -trimpath -o app .
```

See also [210 link/race](#), [248 bootstrap](#).

## Experiment

```
go build -gcflags='-m' . 2>&1 | head -40
go tool nm ./app | rg 'main\\.|runtime\\.main' | head
```

**What to notice:** Even tiny mains reference large `runtime.*` symbol sets after link.

**Try next:** Build twice with/without `-pgo=auto` after a CPU profile; bench a hot path.



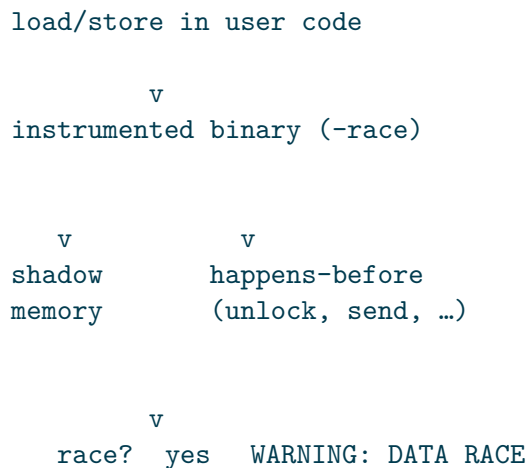
## Race Detector Internals

# Race Detector Internals

## Overview

`go test -race` is not a linter—it is a **different ABI/runtime** that shadows memory and tracks **happens-before**. It catches concurrent unsynchronized access with at least one write.

## Diagram: instrumentation model



## What creates happens-before (reminder)

Event	Edge
Unlock → later Lock	yes
Channel send → receive	yes
WaitGroup.Done → Wait return	yes
go start	parent before child start
Atomics (sync/atomic)	model-defined

No edge + concurrent write/read → race report.

## Cost model

Mode	Overhead
<code>-race</code> tests	often 2–10× CPU/mem
Production race binary	rarely shipped

Requires CGO toolchain on many platforms historically—CI images need gcc/clang.

## False confidence

- Passing once no races
- Coverage matters: races on rare branches need fuzz/load
- Synctest settles logical concurrency; **still run `-race`**

## Experiment

```
cat > /tmp/race.go <<'EOF'
package main
import ("fmt"; "sync")
func main() {
 var x int
 var wg sync.WaitGroup
 wg.Go(func() { x++ })
 wg.Go(func() { x++ })
 wg.Wait()
 fmt.Println(x)
}
EOF
go run -race /tmp/race.go
```

**What to notice:** Report names both goroutines and stack sites even if printed total looks “fine”.

**Try next:** Fix with `atomic.Int64` or `mutex`; confirm clean under `-race`.

**Write Barriers and Mark Assist**

# Write Barriers and Mark Assist

## Overview

During concurrent mark, mutators keep running. **Write barriers** intercept pointer stores so the GC does not lose objects. When allocation outruns marking, **mark assist** makes allocating Gs help mark—coupling alloc rate to GC CPU on the request path.

## Diagram: barrier keeps tri-color safe

```
mutator: x.f = p

 v
write barrier (during concurrent mark)

 v
shade / publish pointer to GC

alloc while GC behind mark assist on same G
```

Without barriers, a black object could gain a white pointer the GC will never see → use-after-free class bugs.

## When barriers are on

- Enabled around concurrent mark
- Cost shows up as extra work on **pointer-heavy** writes
- Reducing pointer density (flatten structs, fewer interfaces in hot graphs) helps

## Mark assist

```
mutator allocates fast
→ GC behind on mark
→ allocator charges assist credit
→ G marks a bit before continuing
```

```
sequence (top → bottom):
 actors: mutator G, allocator, mark work
 mutator G --> allocator : mallocgc
 allocator --> mutator G : assist debt
 mutator G --> mark work : scan some greys
 mutator G --> mutator G : continue user code
```

**Symptom:** latency spikes on alloc-heavy handlers correlating with GC cycles—not only STW.

## Tuning angle

1. Allocate less (best)
2. GOMEMLIMIT / GOGC shape frequency
3. Profile runtime.gcAssistAlloc / mark symbols
4. Avoid panicking into “just raise GOGC” without heap proof

## Experiment

```
GODEBUG=gctrace=1 go run .
```

```
package main
import ("fmt"; "runtime")
func main() {
 var hold [][]byte
 for i := 0; i < 2000; i++ {
 hold = append(hold, make([]byte, 64*1024))
 }
 var m runtime.MemStats
 runtime.ReadMemStats(&m)
 fmt.Println("NumGC", m.NumGC, "GCCPUFraction", m.GCCPUFraction)
 _ = hold
}
```

**What to notice:** gctrace lines and rising NumGC under allocation bursts.

**Try next:** Compare assist-heavy run vs pre-sized slice with fewer growths.

## **Work Stealing and findrunnable**

# Work Stealing and findrunnable

## Overview

When a P needs work, it does not only look at its local run queue. The scheduler's **findrunnable** path checks local queue, global queue, **work stealing**, netpoll, and idle logic. This is how Go keeps cores busy without a single giant lock.

## Diagram: where is the next G?

```
P needs a runnable G

 v
local runq? yes run it
 no
 v
global runq? yes run it
 no
 v
steal from other P? yes run it
 no
 v
netpoll ready? yes run it
 no
 v
idle / wait for work
```

## Local vs global queues

Queue	Property
Per-P local	Fast, little contention
Global	Overflow / balancing; more shared

New Gs often land local to the creator's P; load imbalance triggers steal.



## Work stealing

Idle P **steals half** (teaching model) of another P's local queue when possible—classic multiprocessor scheduler trick.

```
P_idle looks at P_victim.runq
→ take some Gs
→ run them locally
```

## Netpoll integration

Before declaring the machine idle, try **non-blocking netpoll**: “any FD ready?” Ready network Gs become runnable without a dedicated thread blocked in `read`.

`sysmon` also helps poll when all Ps are busy ([213](#), [239](#)).

## Why you care

Symptom	Scheduler angle
Runnable Gs, idle CPUs	steal/netpoll delay, GOMAXPROCS, cgroup
Latency + low CPU	parking on chan/IO, not missing steal
Syscall heavy	extra Ms; P handoff ( <a href="#">201</a> )

## Experiment

```
GODEBUG=schedtrace=1000 go run .
```

```
package main
import ("sync"; "time")
func main() {
 var wg sync.WaitGroup
 for i := 0; i < 1000; i++ {
 wg.Go(func() { time.Sleep(50 * time.Millisecond) })
 }
 wg.Wait()
}
```

**What to notice:** `schedtrace` lines show runqueue lengths and idle/spinning threads (format version-dependent).

**Try next:** Capture `go tool trace` and inspect Proc start/stop + Syscall blocks.

## **Type Assert and Convert at Runtime**

# Type Assert and Convert at Runtime

## Overview

Type assertions and conversions are not only compile-time checks. Failed asserts panic (or return `ok=false`); interface asserts consult **dynamic type** metadata in the interface word.

## Diagram: assert path

```
interface value

 i.(T) ok-form value or ok=false
 i.(T) bare value or panic
 type switch branch on dynamic type
```

```
var i any = "x"
s := i.(string) // panic if wrong
s, ok := i.(string) // safe
switch v := i.(type) { case string: _ = v }
```

## Convert vs assert

	Convert T(x)	Assert i.(T)
Static types	both known	interface → concrete/interface
Failure	compile error	panic or ok=false
Cost	often pure codegen	type metadata compare

## Interface → interface

```
var r io.Reader = bytes.NewBuffer(nil)
rc, ok := r.(io.ReadCloser) // may fail
```

Runtime checks method set / itab compatibility for the pair of interface types.

## Performance notes

- Hot asserts on varying types can miss predict and cost more than type switch with few cases
- Prefer static types at package boundaries when possible
- `any` + `assert` in loops is a smell (also escapes/allocs)

## Experiment

```
go run .

package main
import "fmt"
func main() {
 var i any = 3
 if _, ok := i.(string); ok {
 fmt.Println("string")
 } else {
 fmt.Println("not string")
 }
 defer func() {
 if r := recover(); r != nil {
 fmt.Println("panic:", r)
 }
 }()
 _ = i.(string)
}
```

**What to notice:** comma-ok is quiet; bare assert panics.

**Try next:** Benchmark type switch vs chained asserts on a closed set of types.

# OS Signals and the Runtime

# OS Signals and the Runtime

## Overview

Unix signals interrupt processes asynchronously. Go multiplexes them through the runtime so user code can use `os/signal` channels instead of raw C handlers—while still coordinating with the scheduler and `cgo`.

## Diagram: signal path (teaching)

```
kernel signal
 v
runtime notes signal
 v
os/signal demux → Notify channel
 v
user G handles (e.g. Shutdown)
```

## os/signal API

```
ctx, stop := signal.NotifyContext(context.Background(), os.Interrupt, syscall.SIGTERM)
defer stop()
<-ctx.Done()
// graceful shutdown
```

```
ch := make(chan os.Signal, 1)
signal.Notify(ch, syscall.SIGTERM)
```

**Buffer the channel**—signals can be missed if send would block.

## Interactions

Area	Note
Multiple Notify	Same signal can fan out
Reset/Stop	Undo registration
Windows	Different signal story; Interrupt exists
cgo	Foreign code may install handlers—careful
Panic dumps	GOTRACEBACK, SIGQUIT often stack dump

## Graceful shutdown pattern

```
sequence (top → bottom):
 actors: SIGTERM, main, http.Server
 SIGTERM --> main : NotifyContext cancel
 main --> http.Server : Shutdown(ctx)
 http.Server --> main : drains then return
```

## Experiment

```
terminal 1
go run .
terminal 2
kill -TERM $(pidof yourbin)
```

```
package main
import (
 "context"
 "fmt"
 "os"
 "os/signal"
 "time"
)
func main() {
 ctx, stop := signal.NotifyContext(context.Background(), os.Interrupt)
 defer stop()
 fmt.Println("running; press Ctrl+C")
 select {
 case <-ctx.Done():
 fmt.Println("got", ctx.Err())
 case <-time.After(30 * time.Second):
 fmt.Println("timeout")
 }
}
```

**What to notice:** Cancel is process-wide coordination; pair with server `Shutdown`, not `os.Exit` mid-request.

**Try next:** Trap `SIGTERM` in a binary under Docker `docker stop` and confirm clean logs.



## HTTP/2 in Go

# HTTP/2 in Go

## Overview

Go's `net/http` speaks HTTP/2 when TLS+ALPN (or h2c in limited cases) allows it. One TCP connection **multiplexes streams**, changing connection pool math and failure modes.

## Diagram: multiplex

```
regions: single TLS conn
flow:
 [Client]
 |
 v
 [conn]

 [conn]
 |
 v
 [Server]
```

## Client knobs

```
tr := &http.Transport{
 ForceAttemptHTTP2: true,
 // MaxIdleConnsPerHost less critical for h2 multiplexing
 // but MaxConnsPerHost still bounds total conns
}
```

```
GODEBUG=http2debug=1 ./app # noisy; debug only
```

## Practical differences vs HTTP/1.1

Topic	HTTP/1.1	HTTP/2
Parallel requests / host	many conns or pipeline rare	streams on one conn
Head-of-line	per connection TCP HOL	stream independence (TCP still HOL)
Server push	N/A / deprecated paths	limited/rare in Go clients
Debugging	simple	need h2 frames / debug flags

## Server

`http.Server` + TLS config with `NextProtos` often enough. Timeouts still mandatory:

```
ReadHeaderTimeout, ReadTimeout, WriteTimeout, IdleTimeout
```

## Experiment

```
go run .
curl -v --http2 https://...

// print negotiated protocol from a real client response
// resp.Proto == "HTTP/2.0" when h2 used
```

**What to notice:** Idle pool settings interact with h2 differently; leaking `Body` still kills reuse.

**Try next:** Load test one host with many parallel requests; compare conn count h2 vs forced h1.

## **database/sql Pool and Context**

# database/sql Pool and Context

## Overview

database/sql is a **generic pool** over driver connections. Most production bugs are pool limits, missing context, or holding `*sql.Rows` open—not SQL syntax.

## Diagram: pool

```
db.QueryContext
 v
 sql.DB pool
 idle conn
 open new (MaxOpen)

 v
 Rows / Tx Close/Commit return to pool
```

## Essential settings

```
db.SetMaxOpenConns(25)
db.SetMaxIdleConns(25)
db.SetConnMaxLifetime(time.Hour)
db.SetConnMaxIdleTime(10 * time.Minute)
```

Knob	If wrong
MaxOpen too low	queueing latency
MaxOpen too high	DB overload
MaxIdle 0	reconnect thrash
No context	hung queries pin conns

## Context everywhere

```
ctx, cancel := context.WithTimeout(ctx, 3*time.Second)
defer cancel()
rows, err := db.QueryContext(ctx, q, args...)
```

Cancel should abort driver ops (driver-dependent quality).

## Rows and transactions

```
defer rows.Close() // always
tx.Commit / Rollback // always pair
```

Leaking rows = leaking conns = pool exhaustion under load.

## pgx note

Many services prefer `jackc/pgx` pool directly; patterns still mirror: max conns, timeouts, close. See [web db chapter](#).

## Experiment

```
// conceptual: open sqlite or pg, set MaxOpenConns(1),
// run two concurrent QueryContext with sleep in SQL,
// observe second waits for conn
```

**What to notice:** Pool size becomes a concurrency throttle.

**Try next:** Add `db.Stats()` logging of `InUse`, `Idle`, `WaitCount` in a service.

# JSON Performance Patterns

# JSON Performance Patterns

## Overview

`encoding/json` is correct and convenient; hot paths often spend CPU in **reflection** and allocations. Patterns: reuse encoders, pre-size, avoid `map[string]any`. In Go 1.27, v1 is backed by the v2 engine (faster unmarshal) and `encoding/json/v2` is the supported new API.

## Diagram: cost centers

```
flow:
 [Marshal]
 |
 v
 [Alloc]
```

## Patterns

Pattern	Why
<code>json.NewEncoder(w).Encode</code>	stream to writer
<code>Decoder.DisallowUnknownFields</code>	strict APIs
<code>Pool bytes.Buffer</code>	cut allocs
Typed structs	field cache
<code>json.RawMessage</code>	delay parse
Codegen (easyjson etc.)	last resort

## Encoder reuse caveats

Stateful encoders/decoders need care if concurrent—prefer per-goroutine or mutex; pooling buffers is safer than sharing one Encoder across Gs.



## json/v2 (GA in Go 1.27)

encoding/json/v2 is no longer an experiment. Marshal is roughly at parity with the old v1 engine; unmarshal is significantly faster. Stricter defaults (UTF-8, duplicate names, case-sensitive fields) are the usual migration cost.

```
import jsonv2 "encoding/json/v2"

b, err := jsonv2.Marshal(e)
err = jsonv2.Unmarshal(b, &e, jsonv2.RejectUnknownMembers(true))
```

Keep v1 imports for existing handlers; migrate hot paths and new APIs to v2 after golden-file tests pass. Emergency rollback: `GOEXPERIMENT=nojsonv2` (233).

## Experiment

```
go test -bench=. -benchmem

type E struct {
 A int `json:"a"`
 B string `json:"b"`
}

func BenchmarkMarshal(b *testing.B) {
 e := E{1, "x"}
 for i := 0; i < b.N; i++ {
 _, _ = json.Marshal(e)
 }
}
```

**What to notice:** alloc/op and ns/op; compare with streaming encode to `io.Discard`.

**Try next:** Profile a handler that Marshals large slices every request.

## **errgroup Patterns**

# errgroup Patterns

## Overview

[golang.org/x/sync/errgroup](https://golang.org/x/sync/errgroup) runs a set of goroutines and returns the **first error**, optionally canceling siblings via derived context. Prefer it over ad-hoc WaitGroup+mutex error slots for fan-out.

## Diagram: cancel on first error

```
sequence (top → bottom):
 actors: errgroup, task A, task B
 errgroup --> task A : Go
 errgroup --> task B : Go
 task A --> errgroup : err
 errgroup --> task B : ctx cancel
 errgroup --> errgroup : Wait returns err
```

## Basic API

```
g, ctx := errgroup.WithContext(parent)
g.Go(func() error {
 return work(ctx)
})
g.Go(func() error {
 return other(ctx)
})
if err := g.Wait(); err != nil {
 return err
}
```

## Limits

```
g.SetLimit(8) // max concurrent Gs in this group (modern x/sync)
```

Bounds fan-out without a hand-rolled semaphore.

## Rules

1. Pass `ctx` into every blocking call
2. Return errors; do not log+ignore inside Go funcs without policy
3. `Wait` once; do not reuse group
4. First error wins—others may still run until cancel observed

## vs WaitGroup

	WaitGroup	errgroup
Error prop	manual	built-in
Cancel siblings	manual	<code>WithContext</code>
Limit concurrency	manual	<code>SetLimit</code>

## Experiment

```
go get golang.org/x/sync/errgroup
```

```
g, ctx := errgroup.WithContext(context.Background())
g.Go(func() error {
 <-ctx.Done()
 return ctx.Err()
})
g.Go(func() error {
 return errors.New("boom")
})
fmt.Println(g.Wait())
```

**What to notice:** First error cancels context; the other returns `context.Canceled` or exits promptly if it watches `ctx`.

**Try next:** Crawl N URLs with `SetLimit(5)` and a shared `ctx` timeout.

# Assets

# **Concurrency Ground Up**

# Concurrency From the Ground Up

# Concurrency From the Ground Up

This part is a **diagram-first curriculum** for Go concurrency: goroutines, channels, pipelines, time, context, wait groups, races, semaphores, atomics, testing, a simplified scheduler model, race-condition reasoning, and condition-variable signaling.

It is designed for programmers who already know Go basics (types, interfaces, errors) and want the *application* of concurrency primitives—not a feature laundry list.

**Pedagogy note.** The chapter map and teaching emphasis follow the structure popularized by interactive concurrency guides such as [Gist of Go: Concurrency](#) (Anton Zhiyanov). Explanations, diagrams, and code here are original to this book; they are not a copy of that site. Use both: that site for interactive exercises, this part for a library-style reference with ASCII diagrams integrated into the monorepo.

**Tooling baseline (late 2026):** examples assume a recent Go toolchain (Go 1.27-class: `WaitGroup.Go`, generic methods, `go test -race`). Use `go mod init` with go 1.27 when creating lab modules.

## Learning Outcomes

After this part you can:

1. Separate **business logic** from **concurrency mechanics**
2. Choose among wait groups, done/cancel channels, and context
3. Build cancel-safe pipelines without leaked goroutines
4. Apply throttling, backpressure, and timeouts correctly
5. Detect **data races** and fix them with ownership, mutexes, or atomics
6. Distinguish **race conditions** from data races (including `-race` clean logic bugs)
7. Use channels first; reach for `sync.Cond` only when predicates need broadcast
8. Explain (at a teaching level) how Gs map onto OS threads

## Contents Map

Basics:	goroutines → channels → pipelines → time → context
Sync:	wait groups → races/mutex → semaphores → atomics
Practice:	testing → scheduler model → race conditions → Cond



Chapter	File	Focus
251	<a href="#">251-goroutines-fundamentals</a>	go, main lifetime, WaitGroup, first channels
252	<a href="#">252-channels-complete</a>	close, range, directions, done, deadlocks, nil channels
253	<a href="#">253-pipelines-select-cancel</a>	leaks, cancel+select, merge, fan-out/fan-in
254	<a href="#">254-time-throttle-backpressure</a>	throttle, backpressure, timeouts, timers
255	<a href="#">255-context-complete</a>	context tree, deadline, values, vs cancel channel
256	<a href="#">256-waitgroups-encapsulation</a>	WaitGroup mental model, pointers, encapsulation
257	<a href="#">257-data-races-mutex</a>	data races, race detector, mutex, RWMutex
258	<a href="#">258-semaphores-rendezvous</a>	semaphore N, rendezvous, barriers
259	<a href="#">259-atomics-complete</a>	atomic RMW, composition traps, CAS
260	<a href="#">260-testing-concurrent-code</a>	testing strategies, determinism
261	<a href="#">261-scheduler-model-diagrams</a>	cores → threads → Gs, GOMAXPROCS
262	<a href="#">262-race-conditions-vs-data-races</a>	race conditions vs data races
263	<a href="#">263-signaling-cond-broadcast</a>	<code>sync.Cond</code> , Signal vs Broadcast

## Suggested path

Week-shaped path (adjust pace):

Day 1-2	251 → 252	start + talk
Day 3-4	253 → 254 → 255	pipelines + time + context
Day 5	256 → 257	WaitGroup + races
Day 6	258 → 259	limits + atomics
Day 7	260 → 261	test + scheduler model
Day 8	262 → 263	logic races + Cond

Always keep `go test -race` (or `go run -race`) in the loop for shared-memory chapters (257+).

## Core Mental Models (Preview)

### Three layers of concurrency

```
instr instr instr instr
Core 1 Core 2 Core 3 Core 4 CPU (true parallel ops)

Thr A Thr B Thr C Thr D OS threads (OS schedules)

G11 G15 G12 G16 G13 G14 G17... Goroutines (Go schedules)
```

### Channel as data + synchronization

```
goroutine A ch goroutine B
 recv send

unbuffered send blocks until recv (rendezvous)
```

### Cancel vs done

```
done: worker signals waiter "I finished"
cancel: waiter signals worker "stop now"
```

### Data race vs race condition

```
data race: concurrent mem access, 1 write, no happens-before
race condition: wrong result due to interleaving (may be -race clean)
```

See [262](#).

## How to Study

1. Read a chapter, redraw one diagram from memory.
2. Run every runnable example; then break it (remove close, drop cancel, share a map).
3. Prefer one solid pipeline lab over ten passive reads.
4. Always keep `go test -race` in the loop for Part B (sync and later).
5. When stuck, write the **sequential** version first, then add concurrency at the edges.

## Lab module template

```
mkdir conc-lab && cd conc-lab
go mod init example.com/conc-lab
go.mod: set `go 1.27` (or your installed version)
```

```
// go.mod
module example.com/conc-lab
```

```
go 1.27
```

```
go run -race .
go test -race ./...
```

## Relationship to Other Parts

Need	Also see
Runtime-deep GMP / netpoll / timers	<a href="#">20 Go Deep Dives</a>
Worker pools in production services	<a href="#">07 Concurrency</a>
Stdlib <code>sync</code> / <code>context</code> recipes	<a href="#">98 Stdlib</a>
Capstone-style builds	<a href="#">99 Projects</a>

## Attribution

Teaching structure inspired by public concurrency curricula (notably antonz.org's *Gist of Go: Concurrency*). All prose, diagrams, and examples in this monorepo chapter set are written for this book under the same author voice as the rest of the Go library.

## Goroutines Fundamentals

# Goroutines Fundamentals

## Overview

A **goroutine** is a function executing independently under the Go runtime. The keyword `go` starts that execution; it does not create an OS thread by itself. Thousands of goroutines can multiplex onto far fewer threads.

## Sequential vs concurrent

```
func say(id int, phrase string) {
 for _, word := range strings.Fields(phrase) {
 fmt.Printf("Worker #%d: %s\n", id, word)
 time.Sleep(time.Duration(rand.Intn(50)) * time.Millisecond)
 }
}

// Sequential: second talker starts only after the first finishes.
func sequential() {
 say(1, "go is awesome")
 say(2, "cats are cute")
}

// Concurrent: both run; main must still wait somehow.
func concurrent() {
 go say(1, "go is awesome")
 go say(2, "cats are cute")
 time.Sleep(300 * time.Millisecond) // fragile - prefer WaitGroup
}
```

```
sequence (top → bottom):
actors: Main, say#1, say#2
Main --> say#1 : call
say#1 --> Main : return
Main --> say#2 : call
say#2 --> Main : return
Main --> say#1 : go
Main --> say#2 : go
Main --> Main : must wait or exit
```

```
note: sequential
note: concurrent
```

## Main is a goroutine

When `main` returns, the **process** ends. Other goroutines are abandoned—even if they had more work.

Without wait:

```
main go G1
 go G2
 return process exit (G1/G2 may print nothing)
```

## WaitGroup: correct waiting

```
func concurrentWait() {
 var wg sync.WaitGroup
 wg.Add(2)
 go func() {
 defer wg.Done()
 say(1, "go is awesome")
 }()
 go func() {
 defer wg.Done()
 say(2, "cats are cute")
 }()
 wg.Wait()
}
```

**Separate concurrency from business logic.** Prefer wrapping `say` so `say` itself does not know about `WaitGroup`.

## WaitGroup.Go (Go 1.25+)

```
var wg sync.WaitGroup
wg.Go(func() { say(1, "go is awesome") })
wg.Go(func() { say(2, "cats are cute") })
wg.Wait()
```

Under the hood: `Add(1) + go + defer Done()`.

## First channel: producer and consumer

```
goroutine B goroutine A
 send X recv X

func main() {
 messages := make(chan string)
 go func() { messages <- "ping" }()
 msg := <-messages
 fmt.Println(msg)
}
```

**Unbuffered send blocks** until a receiver is ready (and vice versa). Channels transfer data **and** synchronize.

```
sequence (top → bottom):
 actors: sender, channel, receiver
 sender --> channel : send "ping" (blocks)
 receiver --> channel : receive
 channel --> receiver : "ping"
 channel --> sender : send completes
```

## Producer-consumer skeleton

```
func countDigitsInWords(phrase string) map[string]int {
 words := strings.Fields(phrase)
 type pair struct {
 word string
 count int
 }
 out := make(chan pair)
 go func() {
 defer close(out)
 for _, w := range words {
 out <- pair{w, countDigits(w)}
 }
 }()
 stats := map[string]int{}
 for p := range out {
 stats[p.word] = p.count
 }
 return stats
}
```

```
}
```

## Rules of thumb

Do	Don't
Wait with WaitGroup / channel / context	<code>time.Sleep</code> to “hope” work finished
Keep pure functions free of sync types	Pass WaitGroup into every helper
Start with unbuffered channels	Buffer “just in case” without a reason

## Runnable example

```
go mod init example && go run .

package main

import (
 "fmt"
 "sync"
 "time"
)

func main() {
 var wg sync.WaitGroup
 ch := make(chan string)
 wg.Go(func() {
 time.Sleep(20 * time.Millisecond)
 ch <- "ready"
 })
 fmt.Println(<-ch)
 wg.Wait()
 fmt.Println("done")
}
```

**What to notice:** Receive synchronizes with send; WaitGroup covers the worker lifetime if you need both result and join.

**Try next:** Remove the receive and observe deadlock; remove WaitGroup and observe whether exit is still safe (with only channel sync it can be).



## **Channels Complete**

# Channels Complete

## Overview

Channels are multi-purpose: **data transfer**, **completion (done)**, and (with care) **cancellation**. This chapter covers close semantics, range, directional types, deadlocks, and nil-channel tricks.

## End-of-stream: why close exists

Infinite receive loops deadlock when the producer finishes:

```
producer sends all words consumer for { recv }
producer exits
consumer blocks forever on recv → "all goroutines are asleep - deadlock"
```

Magic sentinel values ("EOF") are fragile when the sentinel is also valid data. **Close** is the structured signal.

## Closing a channel

```
// writer
for _, w := range words {
 in <- w
}
close(in)

// reader
for {
 w, ok := <-in
 if !ok {
 break // closed and drained
 }
 // use w
}
```

Operation	Open channel	Closed channel
Send	may block	<b>panic</b>

Operation	Open channel	Closed channel
Receive	value, ok=true	zero value, ok=false (repeatable)
Close	ok once	<b>panic</b> on second close

```
state machine:
 start --> Open
 Open --> Closed
 Closed --> Closed
 Open --> Open
```

## Rules

1. **Only the writer closes** (sole ownership).
2. **Close is a signal**, not a resource free—GC collects unused channels either way.
3. Close when readers need end-of-data; otherwise optional.

## Range over channel

```
for w := range in {
 // until closed
}
```

Unlike slice range, channel range yields **one** value (not index).

## Directional channels

Catch misuse at **compile time**:

```
func submit(str string, stream chan<- string) { /* send + close */ }
func receive(stream <-chan string) { /* range only */ }

stream := make(chan string) // bidirectional at creation
go submit(str, stream) // converts to send-only param
receive(stream) // converts to recv-only param
```

Type	Allowed
chan T	send, recv, close
chan<- T	send, close
<-chan T	recv only

## Done channel

Wait for work without a WaitGroup:

```
func say(done chan<- struct{}, id int, phrase string) {
 // ... work ...
 done <- struct{}{} // or close(done) for multi-waiter broadcast
}

done := make(chan struct{})
go say(done, 1, "hello")
<-done

done: worker waiter "finished"
```

struct{} uses zero bytes of payload—signal only.

## Deadlocks

Go runtime detects when **all** goroutines are blocked:

```
fatal error: all goroutines are asleep - deadlock!
```

Common causes:

- Unbuffered send without receiver
- Receive without sender
- WaitGroup counter never reaches zero
- Mutual wait (A waits B, B waits A)

## Nil channels

```
var ch chan int // nil
// <-ch blocks forever
// ch <- 1 blocks forever
```

**Useful in select:** set a case's channel to **nil** to **disable** that case after it closes (see pipelines merge).

## Buffered channels

```
ch := make(chan int, 3) // send up to 3 without a receiver
```

capacity 3:

```
send s1 s2 s3 → buffer full → 4th send blocks
recv frees a slot
```

Buffer size is a **capacity** / **decoupling** choice, not free async magic.

## Runnable example

```
go mod init example && go run .
```

```
package main
```

```
import "fmt"
```

```
func main() {
 in := make(chan string)
 go func() {
 for _, w := range []string{"one", "two", "", "four"} {
 in <- w
 }
 close(in)
 }()
 for w := range in {
 if w != "" {
 fmt.Print(w, " ")
 }
 }
 fmt.Println()
 // closed receives
 v, ok := <-in
 fmt.Printf("after close: %q ok=%v\n", v, ok)
}
```

**Expected:**

```
one two four
after close: "" ok=false
```

**Try next:** Make receive take `<-chan string` and try to close inside it—confirm compile error.

## **Pipelines, Select, and Cancel**

# Pipelines, Select, and Cancel

## Overview

Pipelines compose stages connected by channels. The hard part is **not leaking goroutines** when a consumer stops early, and **merging** multiple streams correctly.

## Leak: early exit without cancel

```
func rangeGen(start, stop int) <-chan int {
 out := make(chan int)
 go func() {
 for i := start; i < stop; i++ {
 out <- i // blocks if nobody receives
 }
 close(out)
 }()
 return out
}

// Consumer breaks early → producer stuck on send forever.
for v := range rangeGen(41, 46) {
 fmt.Println(v)
 if v == 42 {
 break
 }
}
```

```
sequence (top → bottom):
actors: rangeGen, consumer
rangeGen --> consumer : 41
rangeGen --> consumer : 42
consumer --> consumer : break
note: LEAKED G
```

Goroutines are cheap but not free; leaks accumulate.

## Cancel channel + select

```
func rangeGen(cancel <-chan struct{}, start, stop int) <-chan int {
 out := make(chan int)
 go func() {
 defer close(out)
 for i := start; i < stop; i++ {
 select {
 case out <- i:
 case <-cancel:
 return
 }
 }
 }()
 return out
}

func main() {
 cancel := make(chan struct{})
 defer close(cancel) // signal on exit

 for v := range rangeGen(cancel, 41, 46) {
 fmt.Println(v)
 if v == 42 {
 break
 }
 }
}
```

select rules (teaching model):

1. run a case that can proceed
2. if several ready → choose at random (fairness)
3. if none ready → wait (or take default if present)

regions: select  
flow:

```
[out left-arrow i] --consumer reading--> [RunA]
[left-arrow cancel] --cancel closed--> [Exit]
```

## Cancel vs done

Name	Direction	Meaning
done	worker → waiter	“I finished”
cancel	waiter → worker	“Stop now”



Both are often named `done` in the wild—be explicit in your APIs.

## Merge sequentially (slow)

```
// Reads in1 fully, then in2 - second producer stalls.
for v := range in1 {
 out <- v
}
for v := range in2 {
 out <- v
}
```

## Merge concurrently (WaitGroup)

```
func merge(in1, in2 <-chan int) <-chan int {
 out := make(chan int)
 var wg sync.WaitGroup
 forward := func(in <-chan int) {
 defer wg.Done()
 for v := range in {
 out <- v
 }
 }
 wg.Add(2)
 go forward(in1)
 go forward(in2)
 go func() {
 wg.Wait()
 close(out)
 }()
 return out
}
```

## Merge with select + nil disable

```
func mergeSelect(in1, in2 <-chan int) <-chan int {
 out := make(chan int)
 go func() {
 defer close(out)
 for in1 != nil || in2 != nil {
 select {
```

```

 case v, ok := <-in1:
 if !ok {
 in1 = nil // disable case
 continue
 }
 out <- v
 case v, ok := <-in2:
 if !ok {
 in2 = nil
 continue
 }
 out <- v
 }
}
}()
return out
}

```

Why nil?

closed channel is always ready with ok=false  
 without nil, select spins forever on closed cases  
 nil channel cases never select → clean exit when both nil

## Fan-out / fan-in pipeline sketch

```

producer chan stage chan consumer

cancel closed early select exits (no leak)

```

Each stage: func(in <-chan T) <-chan U starting its own goroutine, closing out on exit, accepting cancel when needed.

## Runnable example

```

go mod init example && go run .

package main

import "fmt"

func rangeGen(cancel <-chan struct{}, start, stop int) <-chan int {
 out := make(chan int)

```

```

 go func() {
 defer close(out)
 for i := start; i < stop; i++ {
 select {
 case out <- i:
 case <-cancel:
 return
 }
 }
 }()
 return out
}

func main() {
 cancel := make(chan struct{})
 defer close(cancel)
 for v := range rangeGen(cancel, 41, 46) {
 fmt.Println(v)
 if v == 42 {
 break
 }
 }
 fmt.Println("consumer done; producer exits via cancel")
}

```

**What to notice:** Early break no longer leaks the generator.

**Try next:** Implement `mergeSelect` for three channels by generalizing the nil pattern.

**Time: Throttle, Backpressure, Timeouts**

# Time: Throttle, Backpressure, Timeouts

## Overview

Time tools in concurrent Go: **limit parallelism**, **reject overload**, **bound wait**, and **schedule delayed work**—without leaking timers.

## Throttle ( N concurrent)

Buffered channel as semaphore:

```
sema capacity N:

handle: sema <- token // acquire
 go work(); <-sema // release in worker

wait: fill all slots again to ensure idle

func throttle(n int, fn func()) (handle func(), wait func()) {
 sema := make(chan struct{}, n)
 handle = func() {
 sema <- struct{}{}
 go func() {
 fn()
 <-sema
 }()
 }
 wait = func() {
 for i := 0; i < n; i++ {
 sema <- struct{}{} // re-acquire all slots
 }
 }
 return handle, wait
}

throttle (N):
 acquire slot → run work → release
 full? → block
```

```
backpressure:
 full? → return "busy" immediately (select default)
```

With N=2 and 4×100ms jobs → ~200ms total.

## Backpressure (fail fast)

```
handle = func() error {
 select {
 case sema <- struct{}{}:
 go func() {
 fn()
 <-sema
 }()
 return nil
 default:
 return errors.New("busy")
 }
}
```

```
select + default:
 ready case → take it
 else → default immediately (no wait)
```

Clients must handle "busy" (retry later, shed load, degrade).

## Operation timeout with select

```
func withTimeout(timeout time.Duration, fn func() int) (int, error) {
 done := make(chan struct{})
 var result int
 go func() {
 result = fn()
 close(done)
 }()
 select {
 case <-done:
 return result, nil
 case <-time.After(timeout):
 return 0, errors.New("timeout")
 }
}
```

```
sequence (top → bottom):
 actors: caller, worker, timer
 caller --> worker : start fn
 caller --> timer : After(timeout)
 worker --> caller : done
 timer --> caller : fire
 note: alt fn finishes first
 note: else timeout first
 note: return error; worker may still run!
```

**Caveat:** on timeout the worker goroutine may continue unless `fn` also observes cancel (prefer context—next chapter).

## Timers vs time.After

API	Notes
<code>time.After</code>	Convenient; <b>new timer every call</b> —bad in hot loops
<code>time.NewTimer</code>	Reuse with Stop/Reset carefully
<code>time.AfterFunc</code>	Run function after delay; Stop cancels

```
timer := time.NewTimer(100 * time.Millisecond)
defer timer.Stop()
select {
case <-timer.C:
 // fired
case <-cancel:
 if !timer.Stop() {
 select {
 case <-timer.C:
 default:
 }
 }
}
```

Hot path anti-pattern:

```
for {
 select {
 case <-in:
 case <-time.After(time.Hour): // allocates every iteration
 }
}
```

## Runnable example

```
go mod init example && go run .

package main

import (
 "errors"
 "fmt"
 "time"
)

func work() { time.Sleep(80 * time.Millisecond) }

func main() {
 sema := make(chan struct{}, 2)
 handle := func() error {
 select {
 case sema <- struct{}{}:
 go func() {
 work()
 <-sema
 }()
 return nil
 default:
 return errors.New("busy")
 }
 }
 for i := 1; i <= 4; i++ {
 fmt.Println(i, handle())
 }
 time.Sleep(200 * time.Millisecond)
}
```

**Expected:** first two nil, next two busy.

**Try next:** Replace timeout demo with `context.WithTimeout` so the worker can exit early.



**Context Complete**

# Context Complete

## Overview

`context.Context` carries **cancellation**, **deadlines**, and optional **request-scoped values**. Prefer it over ad-hoc cancel channels for public APIs.

## From cancel channel to context

```
// Channel style
func execute(cancel <-chan struct{}, fn func() int) (int, error) {
 ch := make(chan int, 1)
 go func() { ch <- fn() }()
 select {
 case res := <-ch:
 return res, nil
 case <-cancel:
 return 0, errors.New("canceled")
 }
}

// Context style
func execute(ctx context.Context, fn func() int) (int, error) {
 ch := make(chan int, 1)
 go func() { ch <- fn() }()
 select {
 case res := <-ch:
 return res, nil
 case <-ctx.Done():
 return 0, ctx.Err()
 }
}

ctx, cancel := context.WithCancel(context.Background())
defer cancel() // always
```

Multiple `cancel()` calls are safe (unlike double close on a channel).

## Context is a tree

```
Background
 WithCancel / WithTimeout
 children inherit cancel downward
 shorter deadline always wins
```

Rule	Detail
Immutable nodes	“Add” property → new child
Parent cancel	cancels all children
Child cancel	does <b>not</b> cancel parent/siblings
Timeouts nest	<b>shorter</b> deadline wins; child cannot extend parent

```
parent, cancel := context.WithTimeout(context.Background(), 200*time.Millisecond)
defer cancel()
child, cancel2 := context.WithTimeout(parent, 50*time.Millisecond)
defer cancel2()
// work with child: max wait is 50ms
```

Soft child timeout longer than parent is pointless—parent fires first.

## Timeout and deadline

```
ctx, cancel := context.WithTimeout(parent, 150*time.Millisecond)
// equivalent core:
// context.WithDeadline(parent, time.Now().Add(150*time.Millisecond))
defer cancel()

// errors
// context.Canceled
// context.DeadlineExceeded
```

execute need not know *why* Done closed—only that it should stop waiting.

## Values

```
type key int
const requestID key = 1
ctx = context.WithValue(ctx, requestID, "abc")
id, _ := ctx.Value(requestID).(string)
```

Use for request metadata (IDs, loggers)—not for optional parameters that belong in function args.

## Propagate into work

Best practice: `fn` accepts `ctx` and checks `ctx.Done()` or passes `ctx` to I/O:

```
func work(ctx context.Context) (int, error) {
 select {
 case <-time.After(100 * time.Millisecond):
 return 42, nil
 case <-ctx.Done():
 return 0, ctx.Err()
 }
}
```

Otherwise timeout only abandons the **waiter**, not the **worker**.

## Runnable example

```
go mod init example && go run .
```

```
package main

import (
 "context"
 "fmt"
 "time"
)

func execute(ctx context.Context, fn func() int) (int, error) {
 ch := make(chan int, 1)
 go func() { ch <- fn() }()
 select {
 case v := <-ch:
 return v, nil
 case <-ctx.Done():
 return 0, ctx.Err()
 }
}

func main() {
 ctx, cancel := context.WithTimeout(context.Background(), 50*time.Millisecond)
 defer cancel()
 v, err := execute(ctx, func() int {
 time.Sleep(100 * time.Millisecond)
 return 42
 })
}
```

```
 fmt.Println(v, err)
}
```

**Expected:** 0 context deadline exceeded

**Try next:** Nest a 10ms child under a 1s parent and confirm the child wins.

## **Wait Groups and Encapsulation**

# Wait Groups and Encapsulation

## Overview

A **wait group** waits for a set of goroutines to finish. Prefer it over done channels when you need join without a result payload. Hide it behind APIs so callers never manage counters.

## Done channel vs WaitGroup

```
// done channel
done := make(chan struct{}, 1)
go func() {
 work()
 done <- struct{}{}
}()
<-done

// wait group
var wg sync.WaitGroup
wg.Add(1)
go func() {
 defer wg.Done()
 work()
}()
wg.Wait()
```

Tool	Best for
done channel	single completion + optional result on another channel
WaitGroup	N workers, no per-worker result at join site
wg.Go	less boilerplate (Go 1.25+)

## Mental model (naive)

```
// conceptual - real impl is concurrent-safe and parks, not busy-waits
type WaitGroup struct{ n int }

func (wg *WaitGroup) Add(delta int) {
 wg.n += delta
 if wg.n < 0 {
 panic("negative WaitGroup counter")
 }
}

func (wg *WaitGroup) Done() { wg.Add(-1) }
func (wg *WaitGroup) Wait() {
 for wg.n > 0 { /* real code blocks efficiently */ }
}

state machine:
start --> Zero
Zero --> Positive
Positive --> Positive
Positive --> Zero
Zero --> Zero
```

Properties that match the real `sync.WaitGroup`:

- Knows **nothing** about goroutines—only a counter
- Wait before any Add does not block (counter already 0)
- Reusable after returning to zero
- Negative counter panics

## Pass by pointer

```
// BUG: copy of WaitGroup - Wait sees counter 0
func runWork(wg sync.WaitGroup) {
 wg.Add(1)
 go func() {
 defer wg.Done()
 work()
 }()
}

// OK
func runWork(wg *sync.WaitGroup) { ... }
```



```
value pass: main.wg runWork.wg_copy
pointer: both share same counter
```

## Encapsulation

### Hide inside a function

```
func RunConc(fns ...func()) {
 var wg sync.WaitGroup
 wg.Add(len(fns))
 for _, fn := range fns {
 go func(f func()) {
 defer wg.Done()
 f()
 }(fn)
 }
 wg.Wait()
}
// client: RunConc(work, work, work)
```

### Hide inside a type

```
type ConcRunner struct {
 wg sync.WaitGroup
 funcs []func()
}

func (c *ConcRunner) Add(fn func()) { c.funcs = append(c.funcs, fn) }

func (c *ConcRunner) Run() {
 c.wg.Add(len(c.funcs))
 for _, fn := range c.funcs {
 go func(f func()) {
 defer c.wg.Done()
 f()
 }(fn)
 }
 c.wg.Wait()
}
```

Clients should not see `WaitGroup` unless they are writing infrastructure.

## Add after Wait (advanced)

Technically possible if a waiter blocks on `Wait` while other goroutines still `Add`—but easy to get wrong. Prefer all `Add` calls **before** `Wait` from the coordinating goroutine, or use `wg.Go` which pairs `add/start/done`.

## Runnable example

```
go mod init example && go run .

package main

import (
 "fmt"
 "sync"
 "time"
)

func RunConc(fns ...func()) {
 var wg sync.WaitGroup
 for _, fn := range fns {
 wg.Go(fn)
 }
 wg.Wait()
}

func main() {
 start := time.Now()
 RunConc(
 func() { time.Sleep(50 * time.Millisecond); fmt.Print(".") },
 func() { time.Sleep(50 * time.Millisecond); fmt.Print(".") },
 func() { time.Sleep(50 * time.Millisecond); fmt.Print(".") },
)
 fmt.Printf("\ntook %v\n", time.Since(start).Round(time.Millisecond))
}
```

**What to notice:** Three sleeps overlap → ~50ms, not 150ms.

**Try next:** Deliberately pass `WaitGroup` by value and watch `Wait` return early.

## Data Races and Mutexes

# Data Races and Mutexes

## Overview

When multiple goroutines access the same variable and **at least one writes**, you have a **data race** unless synchronization creates happens-before. Maps can **fatal** on concurrent write; other races may silently corrupt.

## Concurrent map writes

```
counter := map[string]int{}
// two goroutines: counter[word]++ → often:
// fatal error: concurrent map writes
```

`counter[word]++` is **not** one atomic step (hash, read, add, write, maybe grow).

## Race detector

```
go run -race .
go test -race ./...
```

```
WARNING: DATA RACE
Read at ... by goroutine 6
Previous write at ... by goroutine 7
```

Races are **timing-dependent**—passing once means nothing. Always CI with `-race` on supported platforms (needs CGO toolchain historically).

Channels are race-free for their send/receive protocol; sharing the *payload pointers* can still race.

## Fix 1: avoid shared mutation

```
each worker → private map
main → merge maps after Wait
```

```
// counters[i] written only by worker i - OK for distinct indices
// do not append to shared slice from many Gs without sync
```

Or send partial maps on a channel and merge in one owner goroutine (**share by communicating**).

## Fix 2: mutex

```
var mu sync.Mutex
mu.Lock()
counter[word]++
mu.Unlock()

sequence (top → bottom):
 actors: G1, Mutex, G2
 G1 --> Mutex : Lock
 G2 --> Mutex : Lock (blocks)
 G1 --> G1 : critical section
 G1 --> Mutex : Unlock
 Mutex --> G2 : acquired
 G2 --> G2 : critical section
 G2 --> Mutex : Unlock
```

Use mutex when:

- Multiple writers, or
- Writer + readers without other sync

Readers-only → no mutex needed for that data.

## Non-reentrant

```
mu.Lock()
mu.Lock() // deadlock - Go mutex is not reentrant
```

## Pass by pointer

Same rule as WaitGroup—copies break exclusion.

## RWMutex

```
var rw sync.RWMutex
// readers
rw.RLock(); _ = m[k]; rw.RUnlock()
// writer
rw.Lock(); m[k]=v; rw.Unlock()
```

Many concurrent readers; writers exclusive. Measure—RWMutex is not always faster.

```
Writer W Readers R1 R2 R3 R4
```

```
Mutex: W excludes everyone; each R serializes
RWMutex: R1-R4 may overlap; W waits for readers
```

## Critical section hygiene

Do	Don't
Lock only around shared data	Hold lock across HTTP/DB
Unlock via defer when multi-return	Forget unlock on error path
Document what mu protects	One global mutex for everything

## Runnable example

```
go run -race .

package main

import (
 "fmt"
 "sync"
)

func main() {
 var mu sync.Mutex
 counter := map[string]int{}
 var wg sync.WaitGroup
 inc := func(word string) {
 defer wg.Done()
 mu.Lock()
 counter[word]++
 mu.Unlock()
 }
```

```
}
wg.Add(2)
go inc("go")
go inc("go")
wg.Wait()
fmt.Println(counter)
}
```

**What to notice:** `-race` is clean; remove mutex and it may fail or warn.

**Try next:** Rewrite with per-worker maps + merge; compare to mutex under `-bench`.

# Semaphores, Rendezvous, Barriers



# Semaphores, Rendezvous, Barriers

## Overview

Limit concurrency to **N** (semaphore), meet at a **sync point** (rendezvous), or wait for **everyone** before next phase (barrier).

## Mutex is N=1

```
type External struct{ lock sync.Mutex }

func (e *External) Call() {
 e.lock.Lock()
 defer e.lock.Unlock()
 // only one call at a time
}
```

When the dependency allows 4 parallel calls, a mutex is too strict.

## Semaphore model

```
slots: [free free free free] capacity 4
```

```
Acquire: take a free slot or block
Release: free a slot; wake one waiter
```

```
semaphore capacity N: [free free ...]
Acquire → take slot or block
Release → free slot, wake waiter
```

```
rendezvous: each waits until the other arrives
```

## Channel implementation

Stdlib has no `Semaphore` type; a buffered channel is the idiomatic simple form ([golang.org/x/sync/semaphore](https://golang.org/x/sync/semaphore) for weighted cases).

```

type Semaphore chan struct{}

func NewSemaphore(n int) Semaphore {
 return make(chan struct{}, n)
}

func (s Semaphore) Acquire() { s <- struct{}{} }
func (s Semaphore) Release() { <-s }

func (s Semaphore) TryAcquire() bool {
 select {
 case s <- struct{}{}:
 return true
 default:
 return false
 }
}

```

## Where to acquire

Pattern	Effect
Acquire <b>inside</b> worker	May spawn huge number of blocked Gs
Acquire <b>before</b> go	Bounds <b>goroutine count</b> as well as concurrency

```

for range nCalls {
 sem.Acquire()
 wg.Go(func() {
 defer sem.Release()
 ex.Call()
 })
}

```

## Rendezvous

Two parties wait for each other before continuing:

```

G1 reaches point wait
 both proceed
G2 reaches point wait

```

```

ready1 := make(chan struct{})
ready2 := make(chan struct{})

// G1:
close(ready1)
<-ready2

// G2:
close(ready2)
<-ready1

sequence (top → bottom):
 actors: G1, G2
 G1 --> G1 : close ready1
 G1 --> G2 : wait ready2
 G2 --> G2 : close ready2
 G2 --> G1 : wait ready1
 note: both unblocked

```

**Caution:** concurrent `fmt` itself synchronizes and can hide races when debugging.

## Barrier (N parties)

Generalize rendezvous to N workers: each `Add(1)` to a cycle, last arriver releases all. Patterns:

- `sync.WaitGroup` for one-shot phase joins
- Two `WaitGroups` or condition variables for reusable barriers
- Channel of capacity 0 with careful N-handshake (easy to get wrong)

Teaching pattern for one phase:

```

var start, done sync.WaitGroup
start.Add(n)
done.Add(n)
for i := 0; i < n; i++ {
 go func() {
 start.Done()
 start.Wait() // all started
 // phase work
 done.Done()
 }()
}
done.Wait()

```

## Runnable example

```
go mod init example && go run .

package main

import (
 "fmt"
 "sync"
 "time"
)

func main() {
 sem := make(chan struct{}, 4)
 var wg sync.WaitGroup
 start := time.Now()
 for i := 0; i < 12; i++ {
 sem <- struct{}{}
 wg.Go(func() {
 defer func() { <-sem }()
 time.Sleep(10 * time.Millisecond)
 fmt.Print(".")
 })
 }
 wg.Wait()
 fmt.Printf("\n%d ms\n", time.Since(start).Milliseconds())
}
```

**Expected:** ~30ms ( $12/4 \times 10\text{ms}$ ), twelve dots.

**Try next:** Implement rendezvous so two printers always print “sync” before either prints “after”.

# Atomics Complete

# Atomics Complete

## Overview

`sync/atomic` provides lock-free operations that the CPU can perform safely under concurrency. **Composition of atomics is not atomic**—that distinction separates counters from broken “clever” state machines.

## Non-atomic increment

```
total++ // read-modify-write: can lose updates
// 5 Gs × 10000 → often total < 50000
```

```
G1 reads 42
G2 reads 42
G1 writes 43
G2 writes 43 // lost update
```

## Atomic types and ops

Type	Ops
<code>atomic.Bool</code>	Load, Store, Swap, CompareAndSwap
<code>atomic.Int32/64, Uint...</code>	+ Add, And, Or (1.23+)
<code>atomic.Pointer[T]</code>	typed pointer
<code>atomic.Value</code>	any type (consistent concrete type)

```
var total atomic.Int32
total.Add(1)
fmt.Println(total.Load())

// CAS
var flag atomic.Bool
if flag.CompareAndSwap(false, true) {
 // first closer wins
}
```

Pass atomics by **pointer** (or embed in structs used via pointer).

## Composition traps

```
// Guaranteed sum of +1/+1 even with sleep between (sequence-independent Adds)
counter.Add(1)
sleep()
counter.Add(1)

// NOT a reliable algorithm: Load then branch then Add
if counter.Load()%2 == 0 {
 counter.Add(1)
} else {
 counter.Add(2)
}
// No data race, but race CONDITION → unpredictable result
```

```
regions: OK | Race condition without data race
flow:
 [Br]
 |
 v
 [Ad]
```

**Bulletproof composite update:** mutex around the whole sequence, or a careful CAS loop that retries.

## CAS gate (mutex alternative for once)

```
type Gate struct{ closed atomic.Bool }

func (g *Gate) Close() {
 if !g.closed.CompareAndSwap(false, true) {
 return // already closed
 }
 // free resources once
}
```

Great for “first caller wins / early exit”. Not a substitute when waiters must block until unlock.

## When to use atomics

Use	Prefer
Counters, flags	atomics
Multi-field invariants	mutex
Hand-off of ownership	channel
Complex lock-free structures	expert only + heavy tests

## Runnable example

```

go run -race .

package main

import (
 "fmt"
 "sync"
 "sync/atomic"
)

func main() {
 var total atomic.Int32
 var wg sync.WaitGroup
 for i := 0; i < 5; i++ {
 wg.Go(func() {
 for j := 0; j < 10000; j++ {
 total.Add(1)
 }
 })
 }
 wg.Wait()
 fmt.Println("total", total.Load())
}

```

**Expected:** total 50000 and race-clean.

**Try next:** Implement a CAS loop that doubles a value only if still equal to an observed snapshot.



# Testing Concurrent Code

# Testing Concurrent Code

## Overview

Well-designed concurrent code is testable with normal tools: channels, wait groups, and fakes. Flaky tests usually mean **missing synchronization** or **time.Sleep as assertion**.

## Diagram: Assert after barrier

```
flow:
 [Barrier]
 |
 v
 [Assert]
```

## Principles

1. **Make state observable** after a barrier (Wait, channel close, context cancel).
2. **Inject clocks and dependencies** (interfaces) so tests control timing.
3. **Run with -race** always in CI.
4. **Prefer determinism** over “sleep and hope”.

```
go test -race -count=100 ./...
```

## Pattern: wait for completion

```
func TestWorker(t *testing.T) {
 done := make(chan struct{})
 go func() {
 defer close(done)
 work()
 }()
 select {
 case <-done:
 case <-time.After(time.Second):
```

```

 t.Fatal("timeout")
 }
}

```

## Pattern: collect results

```

out := make(chan int, n)
// workers send to out
// close when done
var sum int
for v := range out {
 sum += v
}

```

## Pattern: assert no leak

```

before := runtime.NumGoroutine()
// run cancellable system
// cancel and wait
deadline := time.Now().Add(2 * time.Second)
for time.Now().Before(deadline) {
 if runtime.NumGoroutine() <= before+margin {
 return
 }
 time.Sleep(10 * time.Millisecond)
}
t.Fatal("goroutine leak")

```

## Avoid sleeps as correctness

```

// brittle
go doWork()
time.Sleep(10 * time.Millisecond)
assertState()

```

Prefer:

- channel signal when state is ready
- polling with timeout only when API is legacy
- `testing/synctest` bubbles when available (full runtime model: [synctest deep dive](#))

## Testing cancel paths

```
ctx, cancel := context.WithCancel(context.Background())
errc := make(chan error, 1)
go func() { errc <- server.Run(ctx) }()
cancel()
select {
case err := <-errc:
 if !errors.Is(err, context.Canceled) { t.Fatal(err) }
case <-time.After(time.Second):
 t.Fatal("hang")
}
```

## Table tests still work

Concurrency does not forbid table tests—ensure each case uses fresh channels/groups and does not share mutable fixtures across parallel `t.Run` without isolation.

```
t.Parallel() // only if no shared mutable state
```

## Runnable example

```
mkdir /tmp/conctest && cd /tmp/conctest
go mod init example
```

inc\_test.go:

```
package example

import (
 "sync"
 "sync/atomic"
 "testing"
 "time"
)

func TestAtomicInc(t *testing.T) {
 var n atomic.Int64
 var wg sync.WaitGroup
 for i := 0; i < 100; i++ {
 wg.Go(func() { n.Add(1) })
 }
}
```

```

 wg.Wait()
 if n.Load() != 100 {
 t.Fatalf("got %d", n.Load())
 }
}

func TestCancelPath(t *testing.T) {
 done := make(chan struct{})
 go func() {
 defer close(done)
 time.Sleep(5 * time.Millisecond)
 }()
 select {
 case <-done:
 case <-time.After(time.Second):
 t.Fatal("timeout")
 }
}

go test -race -count=50 -v

```

**What to notice:** Barrier (Wait / channel close) then assert—no Sleep as the correctness signal.

**Try next:** Write a test that fails under -race when two Gs append to the same slice without sync.

## Scheduler Model with Diagrams

# Scheduler Model with Diagrams

## Overview

A teaching model of how Go runs many goroutines on few threads—enough to reason about blocking, GOMAXPROCS, and syscalls. For production-depth GMP, also read [Scheduler deep dive](#).

## Three layers

Hardware - cores run instructions in parallel

instr A      instr B      instr C      instr D

Core 1      Core 2      Core 3      Core 4

OS - many threads share cores (OS scheduler)

Thread E

Thread F

Thread A

Thread B

Thread C

Thread D

Go - many goroutines share threads (Go scheduler)

G15   G16   G17   G18   G19   G20   runnable queue

G11           G12           G13           G14   running

Thread A

Thread B

Thread C

Thread D

Go does **not** assign threads to cores; the OS does.

## Simplified scheduler loop

1. Queue all runnable Gs
2. Run up to N of them on worker threads (N = GOMAXPROCS)
3. If a G blocks (channel, mutex, park) → run another
4. Preempt long runners so others don't starve

```
cores run threads; threads run Gs
GOMAXPROCS max Ps running Go code
block (chan/IO) → park G → run another
```

## Starvation and preemption

If G11–G14 never block, G15+ could starve. The scheduler **preempts** running goroutines so others get CPU (async preemption in modern Go improves tight loops).

## System calls

While a G is in a blocking syscall, its OS thread may be stuck. Runtime can **detach** scheduling capacity and run other Gs on additional threads, then collapse extras later.

```
Before: 4 threads run Go code
G11,G12 enter syscalls
Runtime: Thread E,F run G15,G16
G11,G12 return → rebalance
```

## GOMAXPROCS

Maximum OS threads running **Go code** simultaneously (excluding those stuck in syscalls).

```
runtime.GOMAXPROCS(0) // get
runtime.GOMAXPROCS(1) // set
// Go 1.25+: runtime.SetDefaultGOMAXPROCS()
```

Default = logical CPUs (with cgroup awareness improving in recent versions).

```
GOMAXPROCS=1 go run .
docker run --cpus=4 ... # verify GOMAXPROCS vs host
```

```
regions: Host 16 CPUs | Pod cpus=2
flow:
 [Go app]
```



|  
v  
[Q]

Wrong pairing (tiny quota, huge GOMAXPROCS) → thrashing.

## Concurrency primitives vs scheduler

Primitive	Scheduler view
channel send/recv	may park G; wake on peer
mutex	park on contention
sleep / timer	park until timer heap fires
syscall	may occupy M longer

## Metrics, profiles, traces

Tool	Shows
<code>runtime.NumGoroutine</code>	live Gs
CPU pprof	on-CPU stacks
mutex/block profile	contention
<code>go tool trace</code>	STW, park, proc start/stop
<code>GODEBUG=schedtrace=1000</code>	scheduler trace lines

## Runnable experiment

```
GOMAXPROCS=1 go run .
GOMAXPROCS=4 go run .
```

```
package main

import (
 "fmt"
 "runtime"
 "sync"
 "time"
)

func burn(d time.Duration) {
 end := time.Now().Add(d)
 for time.Now().Before(end) {
```

```

 }
}

func main() {
 fmt.Println("GOMAXPROCS", runtime.GOMAXPROCS(0))
 var wg sync.WaitGroup
 start := time.Now()
 for i := 0; i < 4; i++ {
 wg.Go(func() { burn(200 * time.Millisecond) })
 }
 wg.Wait()
 fmt.Println("elapsed", time.Since(start))
}

```

**What to notice:** With GOMAXPROCS=1, ~800ms; with 4, ~200ms for CPU-bound burns.

**Try next:** Open go tool trace on a server under load and find runnable Gs waiting while a syscall holds threads.

## Keep going

You now have the full **ground-up** path: primitives → sync → atomics → tests → scheduler model. Production depth continues in part **20 Go Deep Dives** and part **07** applied patterns.

## Race Conditions vs Data Races

# Race Conditions vs Data Races

## Overview

These terms are not interchangeable. A **data race** is unsynchronized concurrent memory access where at least one access is a write—undefined behavior under the Go memory model, and usually reported by the race detector (`-race`). A **race condition** is a **logic timing bug**: the program's outcome depends on interleaving in a way the author did not intend. Race conditions can exist **without** a data race when every memory access is synchronized (atomics, mutexes, channels) but the *algorithm* is still wrong.

```
concurrent bug
 data race → memory model violation; -race often finds it
 race condition → wrong logic under interleaving
 (can be -race clean!)
```

Related chapters: [257 data races & mutex](#), [259 atomics](#).

## Teaching rule

```
-race clean correct concurrent algorithm
correct algorithm define happens-before for shared memory
 AND make multi-step decisions atomic as a unit
```

Symptom	Likely class	Detector
WARNING: DATA RACE	data race	<code>go test -race</code>
Concurrent map writes / fatal	data race	runtime + <code>-race</code>
Wrong totals, double-insert, lost update with atomics	race condition	logic tests, stress
TOCTOU / check-then-act	race condition	design review + stress

## Data race: classic counter

Unsynchronized ++ is a read-modify-write. Two goroutines can both read the same value and both write the same next value—**lost updates**—and the accesses form a data race.

```
mkdir race-lab && cd race-lab
go mod init example.com/race-lab
```

data\_race.go:

```
package main

import (
 "fmt"
 "sync"
)

func main() {
 var x int
 var wg sync.WaitGroup
 for i := 0; i < 2; i++ {
 wg.Go(func() {
 for j := 0; j < 100_000; j++ {
 x++ // data race: concurrent read+write without sync
 }
 })
 }
 wg.Wait()
 fmt.Println("x =", x, "(often not 200000)")
}

go run -race .
WARNING: DATA RACE
x = <something> 200000>
```

**What to notice:** Without `-race` you might only see a wrong number; with `-race` the detector pins the concurrent accesses.

### Fixes (any one is enough for the race)

```
// 1) Mutex
var mu sync.Mutex
mu.Lock()
x++
mu.Unlock()
```

```
// 2) Atomic
var x atomic.Int64
x.Add(1)

// 3) Ownership: only one goroutine mutates; others send ops on a channel
```

## Race condition without data race: atomic composition

Each Load / Add is race-free, but the **pair** is not a single atomic decision. Two goroutines can interleave between Load and Add—classic TOCTOU on a “policy” that depends on the observed value.

```
package main

import (
 "fmt"
 "sync"
 "sync/atomic"
)

func main() {
 var counter atomic.Int64
 var wg sync.WaitGroup

 // Intent: if even, +1; if odd, +2. Not atomic as a unit.
 bump := func() {
 if counter.Load()%2 == 0 {
 counter.Add(1)
 } else {
 counter.Add(2)
 }
 }

 for i := 0; i < 1000; i++ {
 wg.Go(bump)
 }
 wg.Wait()
 fmt.Println("final", counter.Load())
 // Re-run many times: totals vary. go test -race stays clean.
}

go run -race .
clean - no data race
final value still non-deterministic relative to a sequential model
```

## Correct patterns

```
// Mutex around the whole decision
mu.Lock()
if counter%2 == 0 {
 counter++
} else {
 counter += 2
}
mu.Unlock()

// Or CAS loop that retries the whole policy
for {
 old := counter.Load()
 var next int64
 if old%2 == 0 {
 next = old + 1
 } else {
 next = old + 2
 }
 if counter.CompareAndSwap(old, next) {
 break
 }
}
```

## Check-then-act

```
G1: if !exists { create() }
G2: if !exists { create() }
 both pass check → duplicate / conflict
```

This appears in maps, files, databases, and caches:

```
// Broken even if map is protected by separate Lock per line incorrectly,
// or if you use atomics only for the flag and not the create body.
if _, ok := m[k]; !ok {
 m[k] = expensiveInit() // two Gs both see !ok
}
```

Domain	Broken pattern	Fix
In-process map	check then insert	single mutex around both; <code>sync.Map</code> <code>LoadOrStore</code>

Domain	Broken pattern	Fix
Files	Stat then Create	O_EXCL, or lock file
SQL	SELECT then INSERT	UNIQUE constraint + handle conflict;
HTTP	“if not cached then fetch”	transaction single-flight, mutex, or CAS

### LoadOrStore sketch

```

package main

import (
 "fmt"
 "sync"
)

func main() {
 var m sync.Map
 var wg sync.WaitGroup
 for i := 0; i < 8; i++ {
 wg.Go(func() {
 actual, loaded := m.LoadOrStore("cfg", "once")
 fmt.Println("loaded_existing=", loaded, "value=", actual)
 })
 }
 wg.Wait()
}

```

Only one store wins; others see `loaded=true`. That closes the check-then-act hole for this map pattern.

### Diagram: three layers of “correct”

Algorithm / invariants (idempotent create, unique keys, CAS)	← race conditions live here
Synchronization (mutex, chan, atomic)	← kills data races



## Runnable lab: compare three versions

Create `racecond_test.go` in a module:

```
package racecond

import (
 "sync"
 "sync/atomic"
 "testing"
)

// Broken: data race on plain int.
func badInc(n *int, workers, per int) {
 var wg sync.WaitGroup
 for w := 0; w < workers; w++ {
 wg.Go(func() {
 for i := 0; i < per; i++ {
 *n++
 }
 })
 }
 wg.Wait()
}

// Race-free memory, wrong if you expected a specific composite policy;
// here simple Add is actually correct for a counter.
func atomicInc(n *atomic.Int64, workers, per int) {
 var wg sync.WaitGroup
 for w := 0; w < workers; w++ {
 wg.Go(func() {
 for i := 0; i < per; i++ {
 n.Add(1)
 }
 })
 }
 wg.Wait()
}

// Race condition: double-checked init without proper once.
func racyInit(ready *atomic.Bool, initCount *atomic.Int32) {
```

```

var wg sync.WaitGroup
for i := 0; i < 32; i++ {
 wg.Go(func() {
 if !ready.Load() {
 // pretend expensive init
 initCount.Add(1)
 ready.Store(true)
 }
 })
}
wg.Wait()
}

func TestBadIncRace(t *testing.T) {
 // Run with: go test -race -run TestBadIncRace
 var n int
 badInc(&n, 4, 10_000)
 // Do not assert equality under race; detector is the signal.
 t.Log("n=", n)
}

func TestAtomicInc(t *testing.T) {
 var n atomic.Int64
 atomicInc(&n, 4, 10_000)
 if n.Load() != 40_000 {
 t.Fatalf("got %d", n.Load())
 }
}

func TestRacyInitCondition(t *testing.T) {
 // Often initCount > 1 even though -race is clean.
 // Fix with sync.Once or mutex around check+init.
 var ready atomic.Bool
 var initCount atomic.Int32
 racyInit(&ready, &initCount)
 if c := initCount.Load(); c != 1 {
 t.Logf("race condition: init ran %d times (expected 1)", c)
 }
}

func TestOnceInit(t *testing.T) {
 var once sync.Once
 var initCount atomic.Int32
 var wg sync.WaitGroup
 for i := 0; i < 32; i++ {
 wg.Go(func() {
 once.Do(func() { initCount.Add(1) })
 })
 }
 wg.Wait()
}

```

```

 })
}
wg.Wait()
if initCount.Load() != 1 {
 t.Fatalf("once failed: %d", initCount.Load())
}
}
}

go test -race -count=20 -v

```

**Experiment:** Run the atomic composition example from [259 atomics](#) under `-race` (clean) and print varying totals (condition).

## Pitfalls

1. **Believing “atomics make the program correct.”** They make *operations* race-free, not *policies*.
2. **Sleeping to “fix” races.** Masks bugs; fails under load or different GOMAXPROCS.
3. **Only testing happy serial paths.** Concurrent bugs need stress (`-count=N`, `-race`, high worker counts).
4. **Confusing channel safety with payload safety.** Sending a pointer does not protect the pointee.

## Checklist

- ☐ Can I name the shared memory and who writes it?
- ☐ Is every multi-step decision under one lock, one CAS loop, or one owner goroutine?
- ☐ Does CI run `go test -race ./...` on supported platforms?
- ☐ Are uniqueness / idempotency enforced at the storage boundary?

## Keep going

Next	Why
<a href="#">257 Data races &amp; mutex</a>	Detector + critical sections
<a href="#">259 Atomics</a>	RMW and composition traps
<a href="#">260 Testing concurrent code</a>	Deterministic tests
<a href="#">263 Cond &amp; Broadcast</a>	Waiting on predicates safely

**Try next:** Find a production `if err := find(); err == notFound { insert }` and document the uniqueness constraint (DB unique index, etcd create, etc.) that makes the race condition harmless.

## **Signaling: Cond and Broadcast**

# Signaling: Cond and Broadcast

## Overview

Sometimes many waiters must wake when shared state changes. **Channels** cover most Go code; `sync.Cond` is the low-level “wait for a condition under a mutex” tool. Prefer channels unless you need **broadcast to many waiters** that all re-check a shared predicate protected by the same lock.

Related: [252 channels](#), [257 mutex](#), [258 semaphores](#).

## Diagram: wait / signal

```
sequence (top → bottom):
 actors: waiter, Cond, producer
 waiter --> Cond : L.Lock
 waiter --> Cond : Wait (unlocks + parks)
 producer --> Cond : L.Lock; change state
 producer --> Cond : Signal or Broadcast
 producer --> Cond : Unlock
 Cond --> waiter : wake; re-Lock
 waiter --> waiter : recheck condition in for-loop
```

**Always wait in a loop**—spurious wakes, broad Broadcast, and stale signals happen.

```
mu.Lock()
for !ready {
 cond.Wait() // atomically unlocks mu while waiting; re-locks on wake
}
// invariant: ready == true while holding mu
mu.Unlock()
```

## Signal vs Broadcast

Method	Wakes	Use when
Signal	<b>one</b> waiter	single consumer can take the item

Method	Wakes	Use when
Broadcast	<b>all</b> waiters	many must re-check (e.g. state machine flip)

Wrong choice:

- **Signal** when many waiters each need to proceed → **starvation** / **stuck waiters**
- **Broadcast** when one consumer would suffice → **thundering herd**

```
producers write queue
```

```

 Signal one waiter takes item
Cond others stay parked

```

```
Broadcast
```

```
all waiters wake, re-lock, recheck predicate
```

## vs channel

Need	Prefer
One result / pipeline stage	channel
Close = fan-out “done”	<code>close(ch)</code>
<code>select</code> on many events	channels
Complex shared-state predicate under one mutex	<b>Cond</b> <b>or</b> redesign with channels
Bounded queue with many waiters	often channel; Cond for classic textbooks

Rule of thumb in modern Go: **if a channel models the problem cleanly, use a channel.**

## Runnable: ready flag + Broadcast

```
mkdir cond-lab && cd cond-lab
go mod init example.com/cond-lab
```

main.go:

```
package main
```

```

import (
 "fmt"
 "sync"
 "time"
)

func main() {
 var mu sync.Mutex
 cond := sync.NewCond(&mu)
 ready := false

 var wg sync.WaitGroup
 for id := 1; id <= 3; id++ {
 wg.Go(func() {
 mu.Lock()
 for !ready {
 fmt.Println("waiter", id, "waiting")
 cond.Wait()
 }
 fmt.Println("waiter", id, "proceeding")
 mu.Unlock()
 })
 }

 time.Sleep(50 * time.Millisecond)
 mu.Lock()
 ready = true
 fmt.Println("producer: broadcast")
 cond.Broadcast()
 mu.Unlock()

 wg.Wait()
}

go run .
waiter N waiting (×3)
producer: broadcast
waiter N proceeding (×3)

```

**What to notice:** Waiters block without busy-spin; Broadcast unblocks all; each re-checks `ready` under the lock.

## Runnable: bounded buffer with Cond

Classic producer/consumer where both “not full” and “not empty” are predicates:

```

package main

import (
 "fmt"
 "sync"
 "time"
)

type Buffer struct {
 mu sync.Mutex
 cond *sync.Cond
 q []int
 cap int
}

func NewBuffer(cap int) *Buffer {
 b := &Buffer{cap: cap}
 b.cond = sync.NewCond(&b.mu)
 return b
}

func (b *Buffer) Put(v int) {
 b.mu.Lock()
 for len(b.q) == b.cap {
 b.cond.Wait()
 }
 b.q = append(b.q, v)
 b.cond.Broadcast() // wake getters (and other putters if needed)
 b.mu.Unlock()
}

func (b *Buffer) Get() int {
 b.mu.Lock()
 for len(b.q) == 0 {
 b.cond.Wait()
 }
 v := b.q[0]
 b.q = b.q[1:]
 b.cond.Broadcast() // wake putters
 b.mu.Unlock()
 return v
}

func main() {
 b := NewBuffer(2)
 var wg sync.WaitGroup

```



```

 wg.Go(func() {
 for i := 1; i <= 5; i++ {
 b.Put(i)
 fmt.Println("put", i)
 time.Sleep(10 * time.Millisecond)
 }
 })
 wg.Go(func() {
 for i := 0; i < 5; i++ {
 v := b.Get()
 fmt.Println("get", v)
 time.Sleep(30 * time.Millisecond)
 }
 })
 wg.Wait()
}

go run -race .

```

**What to notice:** Full buffer blocks `Put`; empty blocks `Get`; no spinning. Race detector stays clean.

## Channel equivalent (usually clearer)

```

ch := make(chan int, 2) // capacity = bound
// put: ch <- v
// get: v := <-ch

```

Prefer this unless you already hold a larger mutex protecting more state than the queue alone.

## Spurious and broad wakes

```

// WRONG: if instead of for
mu.Lock()
if !ready {
 cond.Wait()
}
// ready might still be false after Broadcast from another condition
mu.Unlock()

```

Always:

```

for !predicate() {
 cond.Wait()
}

```

## Signal vs Broadcast lab

```

package main

import (
 "fmt"
 "sync"
 "time"
)

func main() {
 var mu sync.Mutex
 cond := sync.NewCond(&mu)
 items := 0

 // Three consumers each want one item.
 var wg sync.WaitGroup
 for id := 1; id <= 3; id++ {
 wg.Go(func() {
 mu.Lock()
 for items == 0 {
 cond.Wait()
 }
 items--
 fmt.Println("consumer", id, "took item; left", items)
 mu.Unlock()
 })
 }

 time.Sleep(20 * time.Millisecond)
 mu.Lock()
 items = 3
 // Try Signal() once - only one consumer proceeds; others hang.
 // cond.Signal()
 cond.Broadcast() // correct here: three items, three waiters
 mu.Unlock()

 wg.Wait()
}

```

Comment switch: with a single **Signal** and `items=3`, two waiters never wake unless someone signals

again.

## Pitfalls

1. **Waiting without holding the lock.** Undefined / panics / lost signals.
2. **Forgetting the loop.** Spurious wake or multi-predicate Broadcast.
3. **Copying `sync.Cond`.** Always pass by pointer; create with `sync.NewCond(&mu)`.
4. **Using `Cond` when a channel would do.** Harder to `select`, harder to cancel.
5. **Broadcast thundering herd** on hot paths—consider per-waiter channels or a work queue channel.

## Cancellation and Cond

`Cond.Wait` does not take a context. Patterns:

- Use channels for cancelable waits (`select` on `ctx.Done()`).
- Or set a done flag under the mutex and Broadcast, then check `ctx / done` in the wait loop.

```
mu.Lock()
for !ready && !closed {
 cond.Wait()
}
ok := ready && !closed
mu.Unlock()
```

## Checklist

- ☐ Predicate checked in a `for` loop under the mutex
- ☐ Same mutex for state and `Cond`
- ☐ Signal vs Broadcast matches waiter count semantics
- ☐ `-race` clean on producer/consumer tests
- ☐ Documented why channel was not enough (if using `Cond`)

## Keep going

Next	Why
<a href="#">252 Channels</a>	Preferred signaling for most apps
<a href="#">258 Semaphores</a>	Bound concurrency with channels
<a href="#">262 Race vs data race</a>	Predicate bugs under interleaving
<a href="#">260 Testing</a>	Stress the wait loops

**Try next:** Rewrite the ready-flag example with `close(done)` and compare readability, cancelability, and `select` composition.

# **Web Development in Go**

# Web Development in Go

# Web Development in Go

This part is a **hands-on curriculum** for Go web development: project structure, HTTP and routing, templates, databases, concurrency at the edge, sessions and auth, frontend/backend contracts, and testing—all explained in simple, direct language.

The real code and examples make the concepts easy to use. Coverage spans nearly every major aspect of Go web work, but keeps examples straightforward and avoids complex edge cases. That makes it ideal for learners and practitioners who want clarity and reliable progress.

You will learn to structure projects, design backend services, manage sessions, secure authentication, and integrate external APIs with step-by-step **Bookstore** examples. Techniques cover routing, concurrency, testing, logging, performance, and error handling—with code that is easy to understand and explanations that are easy to follow.

This edition focuses on **modern Go features without niche scenarios**, so lessons transfer to real projects. Simplicity over unnecessary detail takes you from setup toward deployment.

**How this part relates to the rest of the book.** Part 08-**web** covers modern stacks (GOTH, gRPC, Huma, Postgres/Redis). Part 13-**network-systems** and deep dives go lower. **This part** is the practical spine: net/http first, clear modules, Bookstore REST + HTML, then secure auth and tests. Read it when you want a full web app path without framework overload.

**Tooling baseline:** Go 1.22+ routing (ServeMux method patterns) is the default. Gorilla Mux appears where method/path variables were historically taught; prefer stdlib mux for new work unless you need Mux-specific features. Use `go mod init`, `go test -race`, and structured logging (`log/slog`).

## Learning outcomes

After this part you can:

1. Scaffold a modular Go web project with clear package boundaries
2. Serve HTTP, route requests, and return JSON or HTML consistently
3. Render templates and static assets without mixing business logic into handlers
4. Talk to a database through interfaces that stay testable
5. Use concurrency for I/O-bound work without leaking goroutines
6. Implement sessions, password hashing, and role-based access
7. Define stable API contracts for frontend/backend interchange
8. Mock dependencies and write focused automated tests
9. Log, surface errors, and keep incident response simple

## Key features

Focus	What you practice
Project shape	Organised code, modular design, real workflows
REST APIs	<b>net/http</b> , optional Gorilla Mux, JSON habits
Security	Auth, roles, password hashing, secure cookies
Contracts	Clear API shapes, simple JSON interchange
Concurrency	Scale data fetching, avoid deadlocks
Testing	Mock DB/APIs/deps for fast, isolated tests
Sessions	Cookies, resilient login flows
Ops	Logging, tracing events, find problems quickly
Integration	HTTP client patterns for payments and third parties
Errors	Understandable messages, standard response plans

## Contents map

Setup:      intro → structure → HTTP/routing  
Content:    templates → databases  
Runtime:    concurrency → sessions/auth  
Boundary:   frontend contracts → testing/debugging  
Production: middleware → shutdown → rate limit → WS → uploads → capstone

Chapter	File	Focus
270	this page	overview, path, Bookstore map
271	<a href="#">271-intro-web-dev-go</a>	why Go for web, request lifecycle, first server
272	<a href="#">272-structuring-go-web-app</a>	modules, packages, config, dependency wiring
273	<a href="#">273-http-requests-routing</a>	handlers, mux, middleware, JSON REST
274	<a href="#">274-templating-rendering</a>	<b>html/template</b> , layouts, static files
275	<a href="#">275-databases</a>	<b>database/sql</b> , repositories, Bookstore catalog
276	<a href="#">276-concurrency-web</a>	parallel fetches, timeouts, safe patterns
277	<a href="#">277-sessions-auth-authz</a>	cookies, sessions, passwords, roles
278	<a href="#">278-frontend-backend-communication</a>	API contracts, clients, external APIs
279	<a href="#">279-testing-debugging</a>	httptest, mocks, logging, performance checks



Chapter	File	Focus
280	<a href="#">280-middleware-patterns</a>	chain, recover, request ID, access log
281	<a href="#">281-graceful-shutdown-web</a>	Shutdown, readiness drain, K8s
282	<a href="#">282-rate-limiting</a>	concurrency cap, token bucket
283	<a href="#">283-websockets-basics</a>	upgrade, heartbeats, hubs
284	<a href="#">284-file-uploads-downloads</a>	multipart, ServeContent, static
285	<a href="#">285-project-bookstore-api-capstone</a>	full Bookstore API project

## Suggested path

Week-shaped path (adjust pace):

Day 1	271 → 272	first server + layout
Day 2	273	routing + REST bookstore API
Day 3	274 → 275	HTML views + database
Day 4	276 → 277	concurrency + auth
Day 5	278 → 279	contracts + tests + logging
Day 6	280 → 282	middleware, shutdown, limits
Day 7	283 → 285	websockets, uploads, capstone

## Running example: Bookstore

Throughout this part you build pieces of a small bookstore service:

Browser / SPA / curl

HTTP API	/api/books, /api/auth/...
+ HTML UI	/books, /login

domain + repository	Book, User, Order
---------------------	-------------------

database / mocks

Keep each chapter's snippets runnable with `go mod init` and a short `main` or `_test.go`. Prefer interfaces at boundaries so chapter 279 can mock cleanly.

# Introduction to Web Development in Go

# Introduction to Web Development in Go

## Overview

Go is a strong fit for web backends: one binary, a solid standard library HTTP stack, clear concurrency, and boring reliability. This chapter sets the mental model—how a request moves through your process—and gets a minimal server running.

## Why Go for web services

Strength	What it means in practice
<code>net/http</code> in stdlib	Ship APIs without a framework tax
Goroutines	Handle many concurrent connections simply
Static typing + <code>go test</code>	Refactor routes and handlers with confidence
Single binary	Deploy the same artifact to VM, container, or bare metal
Explicit errors	Surface failures at the boundary instead of hidden exceptions

You still need good structure, timeouts, and tests. Go does not replace design; it keeps the implementation honest.

## Request lifecycle



**Rule:** Handlers should be thin. Parse input → call application code → write response. Keep SQL, payment calls, and business rules out of the raw `http.Handler` body when the app grows.

## First server

```
mkdir -p /tmp/bookstore-intro && cd /tmp/bookstore-intro
go mod init example.com/bookstore
```

Save as main.go:

```
package main

import (
 "encoding/json"
 "log"
 "net/http"
 "time"
)

func main() {
 mux := http.NewServeMux()
 mux.HandleFunc("GET /healthz", healthz)
 mux.HandleFunc("GET /api/hello", hello)

 srv := &http.Server{
 Addr: ":8080",
 Handler: mux,
 ReadHeaderTimeout: 5 * time.Second,
 ReadTimeout: 10 * time.Second,
 WriteTimeout: 10 * time.Second,
 IdleTimeout: 60 * time.Second,
 }

 log.Println("listening on :8080")
 log.Fatal(srv.ListenAndServe())
}

func healthz(w http.ResponseWriter, r *http.Request) {
 w.WriteHeader(http.StatusOK)
 _, _ = w.Write([]byte("ok"))
}

func hello(w http.ResponseWriter, r *http.Request) {
 w.Header().Set("Content-Type", "application/json")
 _ = json.NewEncoder(w).Encode(map[string]string{
 "service": "bookstore",
 "message": "welcome",
 })
}
```

Run and probe:

```
go run .
other terminal
curl -s localhost:8080/healthz
curl -s localhost:8080/api/hello
```

## What to notice

- **Method-aware patterns** (GET /path) need Go 1.22+. Older Go used path-only matching.
- **Timeouts on `http.Server`** are not optional for production; they bound stuck clients.
- **JSON via `encoding/json`** is enough for most REST bodies.

## Status codes you will use constantly

Code	When
200	Success with body
201	Created resource
204	Success, no body
400	Client sent bad input
401	Not authenticated
403	Authenticated but not allowed
404	Missing resource
409	Conflict (duplicate ISBN, etc.)
500	Unexpected server failure

Prefer **specific 4xx** over a generic 500 when the client can fix the request.

## Logging from day one

```
import "log/slog"

func withRequestLog(next http.Handler) http.Handler {
 return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
 start := time.Now()
 next.ServeHTTP(w, r)
 slog.Info("request",
 "method", r.Method,
 "path", r.URL.Path,
 "duration_ms", time.Since(start).Milliseconds(),
)
 })
}
```

```
}
```

Wire it as `Handler: withRequestLog(mux)` on the server. Chapter 279 expands correlation and tests.

## Error shape for APIs

Agree early on a small JSON error envelope:

```
type APIError struct {
 Error string `json:"error"`
 Code string `json:"code,omitempty"`
}

func writeError(w http.ResponseWriter, status int, code, msg string) {
 w.Header().Set("Content-Type", "application/json")
 w.WriteHeader(status)
 _ = json.NewEncoder(w).Encode(APIError{Error: msg, Code: code})
}
```

Clients and frontends can branch on `code` without scraping free-text messages.

## Bookstore north star

By the end of this part the same process will expose:

- GET `/api/books` — list catalog
- GET `/api/books/{id}` — book detail
- POST `/api/books` — create (admin)
- POST `/api/auth/login` — session cookie
- HTML pages that reuse the same domain layer

## Rules of thumb

Do	Don't
Set server timeouts	Use bare <code>http.ListenAndServe</code> in production forever
Return structured JSON errors	Mix ad-hoc string bodies
Keep handlers thin	Put SQL and payment logic only in handlers
Log method, path, duration	Log secrets, cookies, full credit cards

## Try next

1. Add `GET /api/version` returning a build string.
2. Return 404 JSON for unknown paths via a catch-all or custom `NotFoundHandler`.
3. Hit the server with concurrent `curl` loops and watch logs stay readable.



# Structuring Go Web Applications

# Structuring Go Web Applications

## Overview

A modular layout keeps web apps maintainable: HTTP stays at the edge, domain rules stay pure, and databases hide behind interfaces. This chapter shows a practical Bookstore layout—not the only valid layout, but one that scales from a tutorial to a small product.

## Goals of structure

1. **Find things fast** — routes, handlers, domain, persistence in predictable places
2. **Test in isolation** — swap real DB for mocks without rewriting handlers
3. **Avoid circular imports** — packages depend inward (domain) not sideways forever
4. **Ship one binary** — `cmd/bookstore` is the process entry; libraries live under `internal/`

## Recommended layout

```
bookstore/
 go.mod
 cmd/
 bookstore/
 main.go # wiring only: config, DI, ListenAndServe
 internal/
 config/
 config.go # env + defaults
 domain/
 book.go # types + pure validation
 user.go
 store/
 store.go # interfaces (BookRepository, ...)
 memory/
 book_repo.go # in-memory for demos/tests
 postgres/
 book_repo.go # real SQL later
 httpapi/
 server.go # routes + middleware stack
 books.go # book handlers
 auth.go
```

```

 respond.go # JSON helpers
web/
 templates/ # html/template files
 static/ # css/js
migrations/ # optional SQL migrations

```

`internal/` is invisible to other modules—good for app-private code.

## Dependency direction

```

cmd/bookstore

httpapi uses store (interfaces)

 implements

domain store/memory, store/postgres

```

- **domain** knows nothing about HTTP or SQL.
- **store** defines interfaces and implementations.
- **httpapi** depends on interfaces + domain types.
- **main** constructs concrete types and injects them.

## Config from environment

```

package config

import (
 "os"
 "time"
)

type Config struct {
 Addr string
 DatabaseURL string
 SessionSecret string
 ShutdownTimeout time.Duration
}

func Load() Config {
 return Config{
 Addr: getenv("ADDR", ":8080"),

```

```

 DatabaseURL: os.Getenv("DATABASE_URL"),
 SessionSecret: getenv("SESSION_SECRET", "dev-only-change-me"),
 ShutdownTimeout: 10 * time.Second,
 }
}

func getenv(k, def string) string {
 if v := os.Getenv(k); v != "" {
 return v
 }
 return def
}

```

Never commit production secrets. Dev defaults are fine if loudly unsafe.

## Domain types (Bookstore)

```

package domain

import "fmt"

type Book struct {
 ID string `json:"id"`
 Title string `json:"title"`
 Author string `json:"author"`
 ISBN string `json:"isbn"`
 Price int `json:"price_cents" // store money as integer cents
}

func (b Book) Validate() error {
 if b.Title == "" {
 return fmt.Errorf("title is required")
 }
 if b.Author == "" {
 return fmt.Errorf("author is required")
 }
 if b.Price < 0 {
 return fmt.Errorf("price_cents must be >= 0")
 }
 return nil
}

```

Validation on the domain type keeps handlers short and reuses the same rules for HTML forms and JSON APIs.

## Repository interface

```
package store

import (
 "context"

 "example.com/bookstore/internal/domain"
)

type BookRepository interface {
 List(ctx context.Context) ([]domain.Book, error)
 Get(ctx context.Context, id string) (domain.Book, error)
 Create(ctx context.Context, b domain.Book) (domain.Book, error)
}
```

Handlers depend on `BookRepository`, not on Postgres. Chapter 275 implements memory and SQL; chapter 279 mocks this interface.

## Wiring in main

```
package main

import (
 "log"
 "net/http"
 "time"

 "example.com/bookstore/internal/config"
 "example.com/bookstore/internal/httpapi"
 "example.com/bookstore/internal/store/memory"
)

func main() {
 cfg := config.Load()
 books := memory.NewBookRepo()
 api := httpapi.NewServer(books)

 srv := &http.Server{
 Addr: cfg.Addr,
 Handler: api.Handler(),
 ReadHeaderTimeout: 5 * time.Second,
 }
 log.Printf("bookstore listening on %s", cfg.Addr)
 log.Fatal(srv.ListenAndServe())
}
```

```
}
```

**main is assembly, not business logic.** When you add auth, sessions, and Postgres, only wiring and constructors grow here.

## Package naming habits

Prefer	Avoid
<code>httpapi</code> , <code>store</code> , <code>domain</code> Short import paths under <code>internal/</code> One concern per package	<code>utils</code> , <code>helpers</code> , <code>common</code> dumping grounds Giant <code>models</code> package everything imports Circular handler <code>service</code> handler

If two packages need each other constantly, merge them or extract a smaller shared types package.

## Modular design without over-engineering

For a learning Bookstore you do **not** need:

- Full clean-architecture ceremony with six layers
- A DI framework
- Microservices split by “books” vs “users”

You **do** need:

- Interfaces at I/O boundaries
- `context.Context` on store methods
- Config outside code
- A single place that registers routes

## Graceful shutdown sketch

```
go func() {
 log.Fatal(srv.ListenAndServe())
}()

// on signal:
ctx, cancel := context.WithTimeout(context.Background(), cfg.ShutdownTimeout)
defer cancel()
_ = srv.Shutdown(ctx)
```

Production processes should finish in-flight requests instead of hard-killing connections.

## Rules of thumb

Do	Don't
Put app code under <code>internal/</code>	Export everything as a public library by accident
Inject repositories into handlers	Reach for package-level global DB from every file
Keep <code>main</code> thin	Grow a 2k-line <code>main.go</code> of handlers
Validate in domain or a small service	Duplicate validation in every handler

## Try next

1. Scaffold the directories above and compile with an empty mux.
2. Move `healthz` into `httpapi` and construct it from `main`.
3. Add a `UserRepository` interface stub for the auth chapter.

# Handling HTTP Requests and Routing



# Handling HTTP Requests and Routing

## Overview

Routing maps method + path to a handler. Handlers parse input, call your domain/store, and write responses. This chapter builds Bookstore REST endpoints with **net/http**, shows middleware, and notes when Gorilla Mux still appears in older material.

## ServeMux (Go 1.22+)

```
mux := http.NewServeMux()
mux.HandleFunc("GET /api/books", s.listBooks)
mux.HandleFunc("POST /api/books", s.createBook)
mux.HandleFunc("GET /api/books/{id}", s.getBook)
```

Path wildcards: {id} is available as `r.PathValue("id")`.

## Handler shape

```
type Server struct {
 books store.BookRepository
}

func NewServer(books store.BookRepository) *Server {
 return &Server{books: books}
}

func (s *Server) Handler() http.Handler {
 mux := http.NewServeMux()
 mux.HandleFunc("GET /api/books", s.listBooks)
 mux.HandleFunc("GET /api/books/{id}", s.getBook)
 mux.HandleFunc("POST /api/books", s.createBook)
 return withRecover(withRequestLog(mux))
}
```

## List and get (JSON)

```
func (s *Server) listBooks(w http.ResponseWriter, r *http.Request) {
 items, err := s.books.List(r.Context())
 if err != nil {
 writeError(w, http.StatusInternalServerError, "store_error", "could not list books")
 return
 }
 writeJSON(w, http.StatusOK, items)
}

func (s *Server) getBook(w http.ResponseWriter, r *http.Request) {
 id := r.PathValue("id")
 book, err := s.books.Get(r.Context(), id)
 if err != nil {
 writeError(w, http.StatusNotFound, "not_found", "book not found")
 return
 }
 writeJSON(w, http.StatusOK, book)
}
```

## Create with validation

```
func (s *Server) createBook(w http.ResponseWriter, r *http.Request) {
 defer r.Body.Close()
 r.Body = http.MaxBytesReader(w, r.Body, 1<<20) // 1 MiB

 var in domain.Book
 dec := json.NewDecoder(r.Body)
 dec.DisallowUnknownFields()
 if err := dec.Decode(&in); err != nil {
 writeError(w, http.StatusBadRequest, "bad_json", "invalid JSON body")
 return
 }
 if err := in.Validate(); err != nil {
 writeError(w, http.StatusBadRequest, "validation", err.Error())
 return
 }

 out, err := s.books.Create(r.Context(), in)
 if err != nil {
 writeError(w, http.StatusConflict, "create_failed", err.Error())
 return
 }
}
```

```

 writeJSON(w, http.StatusCreated, out)
}

```

### Habits that prevent production pain:

- Limit body size (MaxBytesReader)
- DisallowUnknownFields when you want strict clients
- Use `r.Context()` for cancel on client disconnect

## Response helpers

```

func writeJSON(w http.ResponseWriter, status int, v any) {
 w.Header().Set("Content-Type", "application/json; charset=utf-8")
 w.WriteHeader(status)
 _ = json.NewEncoder(w).Encode(v)
}

```

Always set `Content-Type` before `WriteHeader`.

## Middleware

Middleware wraps `http.Handler`—logging, auth, CORS, panic recovery.

```

func withRequestLog(next http.Handler) http.Handler {
 return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
 start := time.Now()
 next.ServeHTTP(w, r)
 slog.Info("http",
 "method", r.Method,
 "path", r.URL.Path,
 "ms", time.Since(start).Milliseconds(),
)
 })
}

func withRecover(next http.Handler) http.Handler {
 return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
 defer func() {
 if rec := recover(); rec != nil {
 slog.Error("panic", "err", rec)
 writeError(w, http.StatusInternalServerError, "panic", "internal error")
 }
 }()
 next.ServeHTTP(w, r)
 })
}

```

```
 })
}
```

Order matters: recover outermost so panics in logging still convert to 500s.

```
request → recover → log → auth → mux → handler
response
```

## Query parameters and headers

```
q := r.URL.Query().Get("q") // search term
auth := r.Header.Get("Authorization")
```

For Bookstore search:

```
// GET /api/books?q=gopher
func (s *Server) listBooks(w http.ResponseWriter, r *http.Request) {
 items, err := s.books.List(r.Context())
 // ... filter by title containing q ...
}
```

## Gorilla Mux (when you see it)

Many tutorials and older codebases use [Gorilla Mux](#):

```
r := mux.NewRouter()
r.HandleFunc("/api/books/{id}", getBook).Methods(http.MethodGet)
// id := mux.Vars(r)["id"]
```

**Today:** prefer stdlib `ServeMux` method patterns for new Bookstore code. Reach for Mux (or another router) only if you need features you truly miss (subrouters with custom matchers, complex host rules). Concepts transfer: method restriction, path vars, middleware.

## Method not allowed vs not found

With Go 1.22 patterns, a path that exists for GET but not POST can yield **405** when another method is registered for that path. Unknown paths stay **404**. Test both; clients depend on them.

## REST habits for Bookstore

Method	Path	Action
GET	<code>/api/books</code>	list / search
GET	<code>/api/books/{id}</code>	detail
POST	<code>/api/books</code>	create
PUT/PATCH	<code>/api/books/{id}</code>	update
DELETE	<code>/api/books/{id}</code>	remove

Keep collections plural. Use nouns, not verbs (`/api/books` not `/api/getBooks`).

## Rules of thumb

Do	Don't
Parse → validate → act → respond	Stream huge bodies into memory blindly
Propagate <code>r.Context()</code>	Ignore cancellation mid-query
Centralize JSON write/error helpers	Copy-paste header/status in every handler
Prefer stdlib mux for new work	Add a router dependency with no gain

## Try next

1. Implement DELETE `/api/books/{id}` returning 204.
2. Add middleware that rejects non-JSON `Content-Type` on POST.
3. Write a table of curl commands for every route as a smoke checklist.

# Templating and Rendering Content

# Templating and Rendering Content

## Overview

Not every client is JSON. Server-rendered HTML remains a clear way to ship admin UIs and simple Bookstore pages. Go's `html/template` auto-escapes content by default—use it for user-facing HTML. Keep templates dumb: pass view models, not open database handles.

## `html/template` vs `text/template`

Package	Use
<code>html/template</code>	Browsers (escapes HTML, JS, CSS contexts)
<code>text/template</code>	Emails, configs, code gen—not HTML pages

Never put untrusted user input into `text/template` and serve as HTML.

## Minimal page

`internal/web/templates/layout.tmpl:`

```
{{define "layout"}}
<!DOCTYPE html>
<html lang="en">
<head>
 <meta charset="utf-8">
 <title>{{block "title" .}}Bookstore{{end}}</title>
 <link rel="stylesheet" href="/static/site.css">
</head>
<body>
 <header>Bookstore</header>
 <main>
 {{block "content" .}}{{end}}
 </main>
</body>
</html>
{{end}}
```

internal/web/templates/books\_list.tmpl:

```
{{define "title"}}Catalog{{end}}
{{define "content"}}
<h1>Books</h1>

 {{range .Books}}
 {{.Title}} - {{.Author}}
 {{else}}
 No books yet.
 {{end}}

{{end}}
```

## Parse once, execute many

```
package httpapi

import (
 "html/template"
 "io/fs"
 "net/http"
)

type Server struct {
 books store.BookRepository
 tmpl *template.Template
}

func NewServer(books store.BookRepository, templateFS fs.FS) (*Server, error) {
 t, err := template.ParseFS(templateFS, "templates/*.tmpl")
 if err != nil {
 return nil, err
 }
 return &Server{books: books, tmpl: t}, nil
}

func (s *Server) booksPage(w http.ResponseWriter, r *http.Request) {
 items, err := s.books.List(r.Context())
 if err != nil {
 http.Error(w, "unavailable", http.StatusInternalServerError)
 return
 }
 data := struct {
 Books []domain.Book
 }
```



```

 }{Books: items}

 w.Header().Set("Content-Type", "text/html; charset=utf-8")
 if err := s.tmpl.ExecuteTemplate(w, "layout", data); err != nil {
 // header may already be sent; log and move on
 slog.Error("template", "err", err)
 }
}

```

Embed templates in the binary:

```

//go:embed templates/* static/*
var webFS embed.FS

```

## Static files

```

mux.Handle("GET /static/", http.StripPrefix("/static/",
 http.FileServer(http.FS(staticFS))))

```

Set long cache headers only for fingerprinted assets in production.

## Forms (create book)

Template snippet:

```

<form method="POST" action="/books">
 <label>Title <input name="title" required></label>
 <label>Author <input name="author" required></label>
 <label>Price (cents) <input name="price_cents" type="number" min="0"></label>
 <button type="submit">Add book</button>
</form>

```

Handler:

```

func (s *Server) createBookForm(w http.ResponseWriter, r *http.Request) {
 if err := r.ParseForm(); err != nil {
 http.Error(w, "bad form", http.StatusBadRequest)
 return
 }
 price, _ := strconv.Atoi(r.FormValue("price_cents"))
 b := domain.Book{
 Title: r.FormValue("title"),
 Author: r.FormValue("author"),
 Price: price,
 }
}

```

```

 }
 if err := b.Validate(); err != nil {
 http.Error(w, err.Error(), http.StatusBadRequest)
 return
 }
 if _, err := s.books.Create(r.Context(), b); err != nil {
 http.Error(w, err.Error(), http.StatusConflict)
 return
 }
 http.Redirect(w, r, "/books", http.StatusSeeOther)
}

```

**POST** → **redirect** → **GET** avoids duplicate form submits on refresh.

## Escaping and safety

html/template escapes `{{.Title}}`. If you must inject trusted HTML:

```

// only for trusted, sanitized HTML
type page struct {
 Body template.HTML
}

```

Treat `template.HTML` as a sharp tool—never cast raw user input.

## Sharing data with JSON APIs

Prefer one domain layer for both HTML and JSON:

```

GET /books → HTML list (templates)
GET /api/books → JSON list (same BookRepository)

```

Do not reimplement list logic twice.

## Partial rendering / HTMX (optional)

If you later adopt HTMX (see part 08-web), return **fragments** from the same templates (`{{define "book_row"}}`) instead of full layouts. The structure here still holds: parse once, execute named templates.

## Rules of thumb

Do	Don't
Parse templates at startup	Parse from disk on every request
Pass view structs	Pass <code>*sql.DB</code> into templates
Use <code>html/template</code> for pages	Build HTML with string concat + user data
Redirect after POST	Re-render POST responses that create data

## Try next

1. Add a book detail page at `GET /books/{id}`.
2. Show a flash-style error message when validation fails (query param or session).
3. Embed templates with `//go:embed` and confirm the binary serves pages without the source tree.

# Interaction with Databases

# Interaction with Databases

## Overview

Bookstore data needs a durable store. Go's `database/sql` is the standard entry point: open a pool, run queries with context, scan into domain types. Keep SQL behind a repository so handlers stay free of driver details—and so tests can swap an in-memory implementation.

## Principles

1. **Context on every query** — cancel when the client disconnects
2. **Interfaces at the boundary** — `BookRepository` in chapter 272
3. **Money as integers** — `price_cents`, not `float64`
4. **Pool settings matter** — open connections, idle lifetime, max lifetime
5. **Migrations outside the hot path** — version schema deliberately
6. **IDs** — Go 1.27+ can use `stdlib uuid` (`uuid.New().String()` or `uuid.NewV7().String()`) instead of `github.com/google/uuid`

## Schema (Postgres-flavored)

```
CREATE TABLE books (
 id TEXT PRIMARY KEY,
 title TEXT NOT NULL,
 author TEXT NOT NULL,
 isbn TEXT NOT NULL UNIQUE,
 price_cents INT NOT NULL CHECK (price_cents >= 0),
 created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

SQLite works for local demos; production Bookstores often use Postgres.

## Open a pool

```
import (
 "database/sql"
 "time"

 _ "github.com/jackc/pgx/v5/stdlib" // or modernc.org/sqlite, etc.
)

func OpenDB(url string) (*sql.DB, error) {
 db, err := sql.Open("pgx", url)
 if err != nil {
 return nil, err
 }
 db.SetMaxOpenConns(10)
 db.SetMaxIdleConns(5)
 db.SetConnMaxLifetime(time.Hour)

 ctx, cancel := context.WithTimeout(context.Background(), 3*time.Second)
 defer cancel()
 if err := db.PingContext(ctx); err != nil {
 _ = db.Close()
 return nil, err
 }
 return db, nil
}
```

`sql.Open` does not always connect immediately—**Ping** (with timeout) fails fast at boot.

## Postgres repository

```
package postgres

import (
 "context"
 "database/sql"
 "errors"
 "fmt"

 "example.com/bookstore/internal/domain"
 "github.com/google/uuid"
)

type BookRepo struct {
```

```

 db *sql.DB
}

func NewBookRepo(db *sql.DB) *BookRepo {
 return &BookRepo{db: db}
}

func (r *BookRepo) List(ctx context.Context) ([]domain.Book, error) {
 rows, err := r.db.QueryContext(ctx,
 `SELECT id, title, author, isbn, price_cents FROM books ORDER BY title`)
 if err != nil {
 return nil, err
 }
 defer rows.Close()

 var out []domain.Book
 for rows.Next() {
 var b domain.Book
 if err := rows.Scan(&b.ID, &b.Title, &b.Author, &b.ISBN, &b.Price); err != nil {
 return nil, err
 }
 out = append(out, b)
 }
 return out, rows.Err()
}

func (r *BookRepo) Get(ctx context.Context, id string) (domain.Book, error) {
 var b domain.Book
 err := r.db.QueryRowContext(ctx,
 `SELECT id, title, author, isbn, price_cents FROM books WHERE id = $1`, id,
).Scan(&b.ID, &b.Title, &b.Author, &b.ISBN, &b.Price)
 if errors.Is(err, sql.ErrNoRows) {
 return domain.Book{}, fmt.Errorf("book %s: %w", id, errNotFound)
 }
 return b, err
}

func (r *BookRepo) Create(ctx context.Context, b domain.Book) (domain.Book, error) {
 if b.ID == "" {
 b.ID = uuid.NewString()
 }
 _, err := r.db.ExecContext(ctx,
 `INSERT INTO books (id, title, author, isbn, price_cents)
 VALUES ($1,$2,$3,$4,$5)`,
 b.ID, b.Title, b.Author, b.ISBN, b.Price,
)
 return b, err
}

```

```
}
```

Define a package-level `errNotFound` (or domain sentinel) so handlers map to 404 with `errors.Is`.

## In-memory repository (demo + tests)

```
package memory

import (
 "context"
 "fmt"
 "sync"

 "example.com/bookstore/internal/domain"
 "github.com/google/uuid"
)

type BookRepo struct {
 mu sync.RWMutex
 byID map[string]domain.Book
}

func NewBookRepo() *BookRepo {
 return &BookRepo{byID: map[string]domain.Book{}}
}

func (r *BookRepo) List(ctx context.Context) ([]domain.Book, error) {
 r.mu.RLock()
 defer r.mu.RUnlock()
 out := make([]domain.Book, 0, len(r.byID))
 for _, b := range r.byID {
 out = append(out, b)
 }
 return out, nil
}

func (r *BookRepo) Get(ctx context.Context, id string) (domain.Book, error) {
 r.mu.RLock()
 defer r.mu.RUnlock()
 b, ok := r.byID[id]
 if !ok {
 return domain.Book{}, fmt.Errorf("not found")
 }
 return b, nil
}
```



```
func (r *BookRepo) Create(ctx context.Context, b domain.Book) (domain.Book, error) {
 r.mu.Lock()
 defer r.mu.Unlock()
 if b.ID == "" {
 b.ID = uuid.NewString()
 }
 if _, exists := r.byID[b.ID]; exists {
 return domain.Book{}, fmt.Errorf("duplicate id")
 }
 r.byID[b.ID] = b
 return b, nil
}
```

Use memory in early chapters and unit tests; switch `main` to Postgres when ready.

## Transactions

Checkout / multi-table writes need transactions:

```
tx, err := db.BeginTx(ctx, nil)
if err != nil {
 return err
}
defer tx.Rollback() // no-op after Commit

// tx.ExecContext(...); tx.ExecContext(...)
return tx.Commit()
```

Keep transactions short. Do not hold a tx open across external HTTP calls (payment gateway)—prepare locally, then commit, or use outbox patterns later.

## N+1 and simple query design

Bad: list books, then one query per author detail in a loop.

Good: join or batch IDs in one query.

```
SELECT b.id, b.title, a.name
FROM books b
JOIN authors a ON a.id = b.author_id
WHERE b.id = ANY($1);
```

## Connection errors vs not found

Situation	Handler response
<code>sql.ErrNoRows</code>	404
Unique violation	409
Context deadline	504 or 503
Driver/network	500 + log

Log the real error; return a safe client message.

## Rules of thumb

Do	Don't
Parameterized queries ( <code>\$1, ?</code> )	String-concatenate user input into SQL
Set pool limits	Leave unlimited open connections
Hide SQL behind repositories	Scatter queries through every handler
Test with memory or testcontainers	Require a full prod DB for every unit test

## Try next

1. Add `Update` and `Delete` to both memory and SQL repos.
2. Seed three sample books at startup in memory mode.
3. Map unique ISBN conflicts to HTTP 409 in the create handler.

# Concurrency in Go Web Services

# Concurrency in Go Web Services

## Overview

Each HTTP request already runs on its own goroutine under `net/http`. Extra concurrency pays off when a single handler must fetch several independent resources, call external APIs, or process batches—without blocking the whole response on serial I/O. This chapter applies Go concurrency tools to Bookstore-style backends and avoids the usual deadlocks and leaks.

For a full concurrency curriculum (diagrams, `Cond`, `atomics`), use part **21 (Concurrency From the Ground Up)**. Here we stay web-focused.

## What the server already gives you

Client A	goroutine	handler A
Client B	goroutine	handler B
Client C	goroutine	handler C

You do **not** start a goroutine per request yourself. You start goroutines **inside** a handler when work can run in parallel, then wait before writing the response.

## Parallel independent fetches

Example: book detail page needs book metadata + review summary + stock from three services.

```
func (s *Server) bookDashboard(ctx context.Context, id string) (Dashboard, error) {
 var (
 book domain.Book
 reviews []Review
 stock int
 eg errgroup.Group
)

 eg.Go(func() error {
 var err error
 book, err = s.books.Get(ctx, id)
 return err
 })
}
```

```

 eg.Go(func() error {
 var err error
 reviews, err = s.reviews.ListForBook(ctx, id)
 return err
 })
 eg.Go(func() error {
 var err error
 stock, err = s.inventory.Stock(ctx, id)
 return err
 })

 if err := eg.Wait(); err != nil {
 return Dashboard{}, err
 }
 return Dashboard{Book: book, Reviews: reviews, Stock: stock}, nil
}

```

Use [golang.org/x/sync/errgroup](https://golang.org/x/sync/errgroup) (or `errgroup.WithContext`) so the first failure cancels siblings when you wire context.

### With cancel on first error

```

eg, ctx := errgroup.WithContext(ctx)
// eg.Go(...) as above
err := eg.Wait()

```

## Timeouts and context

Always bound outbound work:

```

ctx, cancel := context.WithTimeout(r.Context(), 2*time.Second)
defer cancel()

req, err := http.NewRequestWithContext(ctx, http.MethodGet, url, nil)
// client.Do(req)

```

Pattern:

```

request context (client gone)

timeout child (2s)

```

```
reviews inventory
```

If the browser disconnects, `r.Context()` cancels and well-behaved queries stop.

## Worker pool for bulk work

Importing a CSV of books: do not spawn one goroutine per row unbounded.

```
func importBooks(ctx context.Context, rows []BookRow, workers int, save func(context.Context, BookRow) error) error {
 jobs := make(chan BookRow)
 eg, ctx := errgroup.WithContext(ctx)

 for i := 0; i < workers; i++ {
 eg.Go(func() error {
 for row := range jobs {
 if err := save(ctx, row); err != nil {
 return err
 }
 }
 return nil
 })
 }

 eg.Go(func() error {
 defer close(jobs)
 for _, row := range rows {
 select {
 case <-ctx.Done():
 return ctx.Err()
 case jobs <- row:
 }
 }
 return nil
 })

 return eg.Wait()
}
```

## Shared state in handlers

In-memory repositories and caches need synchronization:

```

type Counter struct {
 mu sync.Mutex
 n int
}

func (c *Counter) Inc() {
 c.mu.Lock()
 c.n++
 c.mu.Unlock()
}

```

Prefer:

1. **No shared mutable state** (DB owns truth)
2. **Channels / ownership** for pipelines
3. **Mutex** for small in-process caches
4. **atomics** only for simple counters

Run tests with `go test -race`.

## Deadlock patterns to avoid

Bug	Symptom	Fix
Unbuffered send with no receiver	Hang	Ensure consumer, or buffer deliberately
<code>WaitGroup</code> Add inside goroutine	Under-count	Add before <code>go</code> , or use <code>WaitGroup.Go</code>
Lock held across slow I/O	Latency spike	Copy data under lock, I/O outside
Waiting on channel that never closes	Hang	<code>defer close</code> , or select on <code>ctx.Done()</code>

## HTTP client reuse

```

var paymentClient = &http.Client{
 Timeout: 10 * time.Second,
 Transport: &http.Transport{
 MaxIdleConns: 100,
 MaxIdleConnsPerHost: 10,
 IdleConnTimeout: 90 * time.Second,
 },
}

```

Do **not** create a new `http.Client` per request without reason—connection pooling lives on the client/transport.

## Fire-and-forget (careful)

Logging metrics asynchronously is fine if you bound queues. Starting a goroutine that can block forever without tracking is not:

```
// risky: no backpressure, no shutdown
go sendEmail(order)

// better: queue + worker, or wait with timeout in request when email is critical
```

For web handlers, prefer completing critical work within the request or enqueueing to a durable job system.

## Rules of thumb

Do	Don't
Parallelize independent I/O with <code>errgroup</code>	Spawn unlimited goroutines per upload
Propagate and timeout contexts	Ignore <code>r.Context()</code>
Protect shared maps with <code>mutex</code>	Assume request isolation covers package-level maps
Reuse HTTP clients	<code>http.Get</code> in a tight loop without reuse thinking

## Try next

1. Add a handler that fetches book + fake review service in parallel with a 1s timeout.
2. Break it intentionally (remove `Wait`) and observe incomplete responses or races.
3. Run `go test -race` on your memory repository under concurrent `Create/List`.



# Sessions, Authentication, and Authorization

# Sessions, Authentication, and Authorization

## Overview

Authentication answers *who are you?* Authorization answers *what may you do?* Sessions keep users logged in across requests. This chapter implements a straightforward Bookstore flow: password hashing, login, secure cookies, and role checks—without drowning in every OAuth edge case.

## Concepts

```
Authn: credentials → identity (user id)
Session: identity stored server-side or in signed cookie
Authz: identity + role/permission → allow or deny
```

Term	Bookstore example
Authentication	Email + password login
Session	Cookie <code>session_id</code> → user in store
Authorization	Only <code>admin</code> may POST <code>/api/books</code>
Password hash	<code>bcrypt</code> / <code>argon2</code> of password; never store plaintext

## Password hashing

```
import "golang.org/x/crypto/bcrypt"

func HashPassword(plain string) (string, error) {
 b, err := bcrypt.GenerateFromPassword([]byte(plain), bcrypt.DefaultCost)
 return string(b), err
}

func CheckPassword(hash, plain string) bool {
 return bcrypt.CompareHashAndPassword([]byte(hash), []byte(plain)) == nil
}
```

Never log passwords. Prefer constant-time compare (bcrypt already does).

## User model

```
type User struct {
 ID string `json:"id"`
 Email string `json:"email"`
 PasswordHash string `json:"-"` // never serialize
 Role string `json:"role"` // "customer" | "admin"
}
```

## Session store (server-side)

Simple and clear for learning:

```
type Session struct {
 UserID string
 ExpiresAt time.Time
}

type SessionStore struct {
 mu sync.Mutex
 byID map[string]Session
}

func (s *SessionStore) Create(userID string, ttl time.Duration) (string, error) {
 id, err := randomID(32)
 if err != nil {
 return "", err
 }
 s.mu.Lock()
 s.byID[id] = Session{UserID: userID, ExpiresAt: time.Now().Add(ttl)}
 s.mu.Unlock()
 return id, nil
}

func (s *SessionStore) Get(id string) (Session, bool) {
 s.mu.Lock()
 defer s.mu.Unlock()
 sess, ok := s.byID[id]
 if !ok || time.Now().After(sess.ExpiresAt) {
 delete(s.byID, id)
 return Session{}, false
 }
 return sess, true
}
```

```
func (s *SessionStore) Delete(id string) {
 s.mu.Lock()
 delete(s.byID, id)
 s.mu.Unlock()
}
```

Production often uses Redis or signed/encrypted cookies (JWT or iron-cookie style). Server-side sessions are easy to revoke.

## Secure cookie

```
func setSessionCookie(w http.ResponseWriter, sessionID string, secure bool) {
 http.SetCookie(w, &http.Cookie{
 Name: "session_id",
 Value: sessionID,
 Path: "/",
 HttpOnly: true, // not readable from JS
 Secure: secure, // HTTPS only in prod
 SameSite: http.SameSiteLaxMode, // CSRF mitigation baseline
 MaxAge: int((24 * time.Hour).Seconds()),
 })
}
```

Flag	Why
HttpOnly	Stops XSS from stealing the cookie via <code>document.cookie</code>
Secure	Cookie only on HTTPS
SameSite	Reduces cross-site cookie sends

## Login handler

```
func (s *Server) login(w http.ResponseWriter, r *http.Request) {
 var in struct {
 Email string `json:"email"`
 Password string `json:"password"`
 }
 if err := json.NewDecoder(r.Body).Decode(&in); err != nil {
 writeError(w, http.StatusBadRequest, "bad_json", "invalid body")
 return
 }

 user, err := s.users.ByEmail(r.Context(), in.Email)
```

```

if err != nil || !CheckPassword(user.PasswordHash, in.Password) {
 // same message for unknown user and bad password
 writeError(w, http.StatusUnauthorized, "auth_failed", "invalid email or password")
 return
}

sid, err := s.sessions.Create(user.ID, 24*time.Hour)
if err != nil {
 writeError(w, http.StatusInternalServerError, "session", "could not create session")
 return
}
setSessionCookie(w, sid, s.secureCookies)
writeJSON(w, http.StatusOK, map[string]string{"status": "ok"})
}

```

## Auth middleware

```

type ctxKey int

const userIDKey ctxKey = 1

func (s *Server) requireAuth(next http.Handler) http.Handler {
 return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
 c, err := r.Cookie("session_id")
 if err != nil {
 writeError(w, http.StatusUnauthorized, "auth_required", "login required")
 return
 }
 sess, ok := s.sessions.Get(c.Value)
 if !ok {
 writeError(w, http.StatusUnauthorized, "auth_required", "login required")
 return
 }
 ctx := context.WithValue(r.Context(), userIDKey, sess.UserID)
 next.ServeHTTP(w, r.WithContext(ctx))
 })
}

func UserIDFrom(ctx context.Context) (string, bool) {
 id, ok := ctx.Value(userIDKey).(string)
 return id, ok
}

```

## Role-based access

```
func (s *Server) requireRole(role string, next http.Handler) http.Handler {
 return s.requireAuth(http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
 uid, _ := UserIDFrom(r.Context())
 user, err := s.users.Get(r.Context(), uid)
 if err != nil || user.Role != role {
 writeError(w, http.StatusForbidden, "forbidden", "insufficient role")
 return
 }
 next.ServeHTTP(w, r)
 })))
}

// registration
mux.Handle("POST /api/books", s.requireRole("admin", http.HandlerFunc(s.createBook)))
```

401 = not logged in. 403 = logged in but not allowed. Keep them distinct.

## Logout

```
func (s *Server) logout(w http.ResponseWriter, r *http.Request) {
 if c, err := r.Cookie("session_id"); err == nil {
 s.sessions.Delete(c.Value)
 }
 http.SetCookie(w, &http.Cookie{
 Name: "session_id", Value: "", Path: "/", MaxAge: -1, HttpOnly: true,
 })
 writeJSON(w, http.StatusOK, map[string]string{"status": "logged_out"})
}
```

## CSRF note (HTML forms)

Cookie sessions + browser forms need CSRF tokens (double-submit or synchronizer token). JSON APIs called with custom headers from SPAs have a different threat model. For Bookstore HTML admin, add a CSRF token in forms before production.

## Rules of thumb

Do	Don't
Hash passwords with <code>bcrypt/argon2</code>	Store plaintext or reversible encryption as “hash”
<code>HttpOnly + Secure + SameSite</code> cookies	Put tokens in <code>localStorage</code> without XSS plan
Generic login failure messages	Reveal “email not found” vs “bad password”
Check roles server-side every request	Trust a client-sent <code>role=admin</code> field

## Try next

1. Add `POST /api/auth/register` with email uniqueness.
2. Protect `POST /api/books` with admin role.
3. Expire sessions and confirm `/api/me` returns 401 after expiry.

# Frontend and Backend Communication



# Frontend and Backend Communication

## Overview

Frontends and backends talk through **contracts**: URLs, methods, status codes, and JSON shapes. Keep those contracts stable, document them lightly, and use the same habits when your Bookstore calls payment gateways or other third-party APIs. This chapter covers JSON interchange, client patterns, and external integration without framework lock-in.

## API contract basics

Agree on:

1. **Resource paths** — /api/books, /api/books/{id}
2. **Methods and status codes** — 200/201/4xx/5xx meanings
3. **JSON field names** — snake\_case or camelCase; pick one and stick to it
4. **Error envelope** — { "error": "...", "code": "..." }
5. **Auth** — cookie session vs Authorization: Bearer ...

Example list response:

```
[
 {
 "id": "b1",
 "title": "The Go Programming Language",
 "author": "Donovan & Kernighan",
 "isbn": "978-0134190440",
 "price_cents": 4499
 }
]
```

Example error:

```
{
 "error": "title is required",
 "code": "validation"
}
```

## JSON tags and stability

```
type BookDTO struct {
 ID string `json:"id"`
 Title string `json:"title"`
 Author string `json:"author"`
 ISBN string `json:"isbn"`
 Price int `json:"price_cents"`
}
```

- Omit sensitive fields with `json:"-"` (password hashes).
- Prefer additive changes (new optional fields) over renames.
- Version only when you must (`/api/v2/...`)—many Bookstores never need it early.

## CORS (browser frontends on another origin)

If the UI is `http://localhost:5173` and API is `http://localhost:8080`, browsers enforce CORS.

```
func withCORS(next http.Handler, allowedOrigin string) http.Handler {
 return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
 w.Header().Set("Access-Control-Allow-Origin", allowedOrigin)
 w.Header().Set("Access-Control-Allow-Credentials", "true")
 w.Header().Set("Access-Control-Allow-Headers", "Content-Type")
 w.Header().Set("Access-Control-Allow-Methods", "GET,POST,PUT,DELETE,OPTIONS")
 if r.Method == http.MethodOptions {
 w.WriteHeader(http.StatusNoContent)
 return
 }
 next.ServeHTTP(w, r)
 })
}
```

For cookie auth cross-origin, the frontend must use `credentials: "include"` and the server must not use `Allow-Origin: *` with credentials.

**Simpler path for learning:** serve HTML from the same Go process (chapter 274) and skip CORS entirely.

## Fetch from a browser (sketch)

```
const res = await fetch("/api/books", { credentials: "same-origin" });
if (!res.ok) {
 const err = await res.json();
 throw new Error(err.error || res.statusText);
}
const books = await res.json();
```

Always check `res.ok` before assuming JSON is a success payload.

## Go HTTP client patterns

Reusable client for third parties (payments, shipping, ISBN lookup):

```
type PaymentGateway struct {
 base string
 apiKey string
 http *http.Client
}

func NewPaymentGateway(base, apiKey string) *PaymentGateway {
 return &PaymentGateway{
 base: strings.TrimRight(base, "/"),
 apiKey: apiKey,
 http: &http.Client{
 Timeout: 15 * time.Second,
 },
 }
}

type ChargeRequest struct {
 AmountCents int `json:"amount_cents"`
 Currency string `json:"currency"`
 OrderID string `json:"order_id"`
}

type ChargeResponse struct {
 ChargeID string `json:"charge_id"`
 Status string `json:"status"`
}

func (g *PaymentGateway) Charge(ctx context.Context, in ChargeRequest) (ChargeResponse, error) {
 body, err := json.Marshal(in)
 if err != nil {
```

```

 return ChargeResponse{}, err
 }
 req, err := http.NewRequestWithContext(ctx, http.MethodPost, g.base+"/v1/charges", bytes)
 if err != nil {
 return ChargeResponse{}, err
 }
 req.Header.Set("Content-Type", "application/json")
 req.Header.Set("Authorization", "Bearer "+g.apiKey)

 res, err := g.http.Do(req)
 if err != nil {
 return ChargeResponse{}, err
 }
 defer res.Body.Close()

 if res.StatusCode >= 300 {
 b, _ := io.ReadAll(io.LimitReader(res.Body, 4096))
 return ChargeResponse{}, fmt.Errorf("payment gateway %s: %s", res.Status, b)
 }

 var out ChargeResponse
 if err := json.NewDecoder(res.Body).Decode(&out); err != nil {
 return ChargeResponse{}, err
 }
 return out, nil
}

```

## Client checklist

Practice	Why
Context on every call	Timeouts and cancel
Explicit <code>http.Client</code> timeout	No hung requests forever
Limit error body reads	Avoid memory blowups
Interface for the gateway	Mock in tests (chapter 279)
Idempotency keys (when API supports)	Safe retries on POST

## Retries (simple, careful)

Retry **idempotent** GETs on network blips. Do not blindly retry POSTs that charge cards unless the API is idempotent.

```

func getWithRetry(ctx context.Context, client *http.Client, url string, attempts int) (*http
 var last error

```

```

 for i := 0; i < attempts; i++ {
 req, err := http.NewRequestWithContext(ctx, http.MethodGet, url, nil)
 if err != nil {
 return nil, err
 }
 res, err := client.Do(req)
 if err == nil {
 return res, nil
 }
 last = err
 select {
 case <-ctx.Done():
 return nil, ctx.Err()
 case <-time.After(time.Duration(i+1) * 100 * time.Millisecond):
 }
 }
 return nil, last
}

```

## Content negotiation

Same resource, two representations:

- Accept: application/json → JSON handler
- Browser navigation → HTML handler

Or separate paths (/api/books vs /books)—clearer for many teams.

## Rules of thumb

Do	Don't
Document JSON fields your UI depends on	Rename fields casually
Same error shape everywhere	Mix string bodies and JSON errors
Interface external APIs	Call payment URLs deep inside handlers with no seam
Prefer same-origin for learning apps	Enable wide-open CORS in production

## Try next

1. Write a short markdown “API contract” for five Bookstore endpoints.

2. Implement a fake `ISBNClient` interface that returns fixed metadata.
3. From curl, exercise login cookie + authenticated create book as a frontend would.

# Testing and Debugging

# Testing and Debugging

## Overview

Reliable web services need fast, isolated tests: handlers with `httptest`, repositories mocked behind interfaces, and a few integration checks. Logging and simple performance checks help you find problems when tests pass but production does not. This chapter closes the Bookstore path with practical routines—not a full testing theory course (see part 06-testing for that).

## What to test

Layer	Technique	Speed
Domain validation	Table-driven unit tests	Fast
Handlers	<code>httptest</code> + mock store	Fast
Repository SQL	Test DB or <code>testcontainers</code>	Medium
Full process	<code>httptest</code> server or real port	Slower

Aim for many fast tests; keep a thin integration suite.

## Table-driven domain tests

```
func TestBookValidate(t *testing.T) {
 tests := []struct {
 name string
 book domain.Book
 wantErr bool
 }{
 {"ok", domain.Book{Title: "Go", Author: "A", Price: 100}, false},
 {"no title", domain.Book{Author: "A", Price: 100}, true},
 {"neg price", domain.Book{Title: "Go", Author: "A", Price: -1}, true},
 }
 for _, tt := range tests {
 t.Run(tt.name, func(t *testing.T) {
 err := tt.book.Validate()
 if (err != nil) != tt.wantErr {
 t.Fatalf("Validate() err=%v wantErr=%v", err, tt.wantErr)
 }
 })
 }
}
```



```

 })
 }
}

```

## Mock repository

```

type mockBooks struct {
 list []domain.Book
 err error
}

func (m mockBooks) List(ctx context.Context) ([]domain.Book, error) {
 return m.list, m.err
}

func (m mockBooks) Get(ctx context.Context, id string) (domain.Book, error) {
 return domain.Book{}, errors.New("not implemented")
}

func (m mockBooks) Create(ctx context.Context, b domain.Book) (domain.Book, error) {
 return b, m.err
}

```

Or generate mocks later; hand-written stubs stay clear for teaching.

## httptest handler test

```

func TestListBooks(t *testing.T) {
 s := &Server{books: mockBooks{list: []domain.Book{
 {ID: "1", Title: "Go Book", Author: "Ada", Price: 1999},
 }}}

 req := httptest.NewRequest(http.MethodGet, "/api/books", nil)
 rec := httptest.NewRecorder()
 s.listBooks(rec, req)

 if rec.Code != http.StatusOK {
 t.Fatalf("status %d", rec.Code)
 }

 var got []domain.Book
 if err := json.NewDecoder(rec.Body).Decode(&got); err != nil {
 t.Fatal(err)
 }

 if len(got) != 1 || got[0].Title != "Go Book" {
 t.Fatalf("unexpected body: %#v", got)
 }
}

```

```

 }
}

```

## Full mux test

```

func TestRoutes(t *testing.T) {
 s := NewServer(memory.NewBookRepo())
 ts := httptest.NewServer(s.Handler())
 t.Cleanup(ts.Close)

 res, err := http.Get(ts.URL + "/api/books")
 if err != nil {
 t.Fatal(err)
 }
 defer res.Body.Close()
 if res.StatusCode != http.StatusOK {
 t.Fatalf("status %d", res.StatusCode)
 }
}

```

## Testing auth

1. Create user + session in the test store.
2. Attach cookie: `req.AddCookie(&http.Cookie{Name: "session_id", Value: sid})`.
3. Assert 401 without cookie, 403 with wrong role, 201 with admin.

Keep clocks controllable if you test expiry (inject a `now` function).

## Mocking external APIs

```

type PaymentClient interface {
 Charge(ctx context.Context, in ChargeRequest) (ChargeResponse, error)
}

type fakePay struct {
 res ChargeResponse
 err error
}

func (f fakePay) Charge(ctx context.Context, in ChargeRequest) (ChargeResponse, error) {
 return f.res, f.err
}

```

Handler tests inject `fakePay{err: errors.New("down")}` and expect 502/503—not a real network call.

## Logging and tracing events

Structured logs make incidents greppable:

```
slog.Info("book_created",
 "book_id", book.ID,
 "user_id", userID,
 "request_id", requestID,
)
```

## Request ID middleware

```
func withRequestID(next http.Handler) http.Handler {
 return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
 id := r.Header.Get("X-Request-ID")
 if id == "" {
 id = uuid.NewString()
 }
 w.Header().Set("X-Request-ID", id)
 ctx := context.WithValue(r.Context(), reqIDKey, id)
 next.ServeHTTP(w, r.WithContext(ctx))
 })
}
```

Return the same ID to clients; paste it into log searches during incidents.

## Error messages and incident response

Audience	Message
Client	Safe, actionable ( <code>validation</code> , <code>not_found</code> )
Logs	Full error chain, request id, user id
On-call	Runbook: check logs by request id → DB → dependency status

Do not dump stack traces or SQL to public JSON bodies.

## Light performance checks

```
func BenchmarkListBooks(b *testing.B) {
 s := &Server{books: mockBooks{list: make([]domain.Book, 100)}}
 b.ResetTimer()
 for i := 0; i < b.N; i++ {
 req := httptest.NewRequest(http.MethodGet, "/api/books", nil)
 rec := httptest.NewRecorder()
 s.listBooks(rec, req)
 }
}
```

For real hotspots: `go test -bench . -cpuprofile=cpu.out` and part 09-performance-tooling / 18-performance-engineering.

Smoke load with curl/hey:

```
hey -n 1000 -c 50 http://localhost:8080/api/books
```

## Debug checklist

1. Reproduce with curl + request id
2. Check status code and error code JSON
3. Search logs for request\_id
4. Confirm auth cookie / role
5. Confirm DB row exists
6. Confirm downstream timeout vs 500
7. `go test ./... and go test -race ./...`

## Rules of thumb

Do	Don't
Mock at interfaces	Hit real payment APIs in unit tests
Use httptest for handlers	Require full Docker for every commit
Log request ids	Log passwords, session secrets, card numbers
Race-test concurrent stores	Assume single-threaded handlers mean no races in packages

## Try next

1. Write tests for create book: 201 happy path, 400 validation, 401 unauthenticated.
2. Force a panic in a handler and assert recover middleware returns 500.

3. Add a failing fake payment client test for checkout.

## Where to go next

- Part **08-web** — GOTH, gRPC, Huma, Postgres/Redis recipes
- Part **06-testing** — organization, integration, advanced patterns
- Part **16-observability-sre** — metrics, tracing, correlation
- Part **17-security-hardening** — TLS, deeper authn/authz

You now have a clear path from empty module to a testable, session-aware Bookstore API with HTML and JSON edges—simple enough to learn, solid enough to grow.

# Middleware Patterns

# Middleware Patterns

## Overview

Middleware wraps `http.Handler` to add cross-cutting behavior: logging, auth, recovery, request IDs, CORS, metrics. Keep each piece small and composable.

## The shape

```
type Middleware func(http.Handler) http.Handler

func chain(h http.Handler, ms ...Middleware) http.Handler {
 for i := len(ms) - 1; i >= 0; i-- {
 h = ms[i](h)
 }
 return h
}
```

```
request → recover → requestID → log → auth → mux → handler
response ←
```

Order matters: recover outermost so panics in other middleware still convert to 500.

## Recover

```
func Recover(next http.Handler) http.Handler {
 return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
 defer func() {
 if rec := recover(); rec != nil {
 slog.Error("panic", "err", rec, "path", r.URL.Path)
 http.Error(w, "internal error", http.StatusInternalServerError)
 }
 }()
 next.ServeHTTP(w, r)
 })
}
```

## Request ID

```
type ctxKey int
const reqIDKey ctxKey = 1

func RequestID(next http.Handler) http.Handler {
 return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
 id := r.Header.Get("X-Request-ID")
 if id == "" {
 id = strconv.FormatInt(time.Now().UnixNano(), 36)
 }
 w.Header().Set("X-Request-ID", id)
 ctx := context.WithValue(r.Context(), reqIDKey, id)
 next.ServeHTTP(w, r.WithContext(ctx))
 })
}
```

## Access log with status

Capture status by wrapping `ResponseWriter`:

```
type statusWriter struct {
 http.ResponseWriter
 code int
}

func (w *statusWriter) WriteHeader(code int) {
 w.code = code
 w.ResponseWriter.WriteHeader(code)
}

func AccessLog(next http.Handler) http.Handler {
 return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
 sw := &statusWriter{ResponseWriter: w, code: 200}
 start := time.Now()
 next.ServeHTTP(sw, r)
 slog.Info("http",
 "method", r.Method,
 "path", r.URL.Path,
 "status", sw.code,
 "ms", time.Since(start).Milliseconds(),
)
 })
}
```



Implement `Unwrap()` / `Flush` if you need `http.Hijacker` or `Flusher` passthrough in production.

## Max body

```
func MaxBytes(n int64) Middleware {
 return func(next http.Handler) http.Handler {
 return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
 r.Body = http.MaxBytesReader(w, r.Body, n)
 next.ServeHTTP(w, r)
 })
 }
}
```

## Timeout (handler budget)

```
func Timeout(d time.Duration) Middleware {
 return func(next http.Handler) http.Handler {
 return http.TimeoutHandler(next, d, "timeout")
 }
}
```

Prefer also setting `http.Server` timeouts (chapter 271).

## Rules of thumb

Do	Don't
One concern per middleware	Mega-middleware that logs+auth+metrics+...
Document chain order	Auth after handler “for speed”
Propagate context values	Global request maps without locks

## Try next

1. Build a chain: `Recover` → `RequestID` → `AccessLog` → `mux`.
2. Add unit tests with `httptest` asserting status on panic.
3. Log request ID from a handler via context.

# Graceful Shutdown for Web Servers

# Graceful Shutdown for Web Servers

## Overview

On deploy or Ctrl-C, stop accepting new connections, finish in-flight requests, then exit. Go's `http.Server.Shutdown` does the hard part—wire it to signals and a deadline.

## Pattern

```
func main() {
 mux := http.NewServeMux()
 // register routes...

 srv := &http.Server{
 Addr: ":8080",
 Handler: mux,
 ReadHeaderTimeout: 5 * time.Second,
 }

 go func() {
 slog.Info("listen", "addr", srv.Addr)
 if err := srv.ListenAndServe(); err != nil && !errors.Is(err, http.ErrServerClosed) {
 slog.Error("serve", "err", err)
 os.Exit(1)
 }
 }()

 ctx, stop := signal.NotifyContext(context.Background(), os.Interrupt, syscall.SIGTERM)
 defer stop()
 <-ctx.Done()
 slog.Info("shutdown signal")

 shutdownCtx, cancel := context.WithTimeout(context.Background(), 15*time.Second)
 defer cancel()
 if err := srv.Shutdown(shutdownCtx); err != nil {
 slog.Error("shutdown", "err", err)
 _ = srv.Close()
 }
}
```

## Kubernetes notes

- **SIGTERM** on pod stop
- `terminationGracePeriodSeconds` must exceed your shutdown timeout
- Readiness fail first (optional sleep) so LB drains before Shutdown

```
// optional: sleep 2s after SIGTERM so endpoints controller removes you
time.Sleep(2 * time.Second)
_ = srv.Shutdown(shutdownCtx)
```

## In-flight work beyond HTTP

Background workers need the same root context:

```
root, stop := signal.NotifyContext(...)
// workers select on root.Done()
// then Shutdown HTTP
```

## Health endpoints during drain

- Liveness: still 200 until process dies
- Readiness: 503 after signal so new traffic stops

```
var ready atomic.Bool
ready.Store(true)

mux.HandleFunc("GET /readyz", func(w http.ResponseWriter, r *http.Request) {
 if !ready.Load() {
 http.Error(w, "draining", http.StatusServiceUnavailable)
 return
 }
 w.WriteHeader(http.StatusOK)
})
// on signal: ready.Store(false)
```

## Rules of thumb

Do	Don't
Shutdown with timeout	Only <code>os.Exit</code> on SIGTERM
Fail readiness early	Keep accepting traffic while dying
Close DB pools after HTTP drains	Cut DB under active handlers

## Try next

1. Slow handler (5s sleep); send SIGTERM; confirm request completes.
2. Set shutdown timeout 1s; prove forced close path.
3. Add `/readyz` drain behavior.

## **Rate Limiting and Concurrency Caps**

# Rate Limiting and Concurrency Caps

## Overview

Protect the process from overload: limit request **rate** (token bucket) and **in-flight concurrency** (semaphore). Start simple—stdlib + [golang.org/x/time/rate](http://golang.org/x/time/rate) or a tiny token bucket.

## Concurrency semaphore

```
func MaxInFlight(n int) func(http.Handler) http.Handler {
 sem := make(chan struct{}, n)
 return func(next http.Handler) http.Handler {
 return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
 select {
 case sem <- struct{}{}:
 defer func() { <-sem }()
 next.ServeHTTP(w, r)
 default:
 http.Error(w, "busy", http.StatusServiceUnavailable)
 }
 })
 }
}
```

## Token bucket (x/time/rate)

```
go get golang.org/x/time/rate@latest
```

```
func RateLimit(r rate.Limit, burst int) func(http.Handler) http.Handler {
 lim := rate.NewLimiter(r, burst)
 return func(next http.Handler) http.Handler {
 return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
 if !lim.Allow() {
 w.Header().Set("Retry-After", "1")
 http.Error(w, "rate limit", http.StatusTooManyRequests)
 return
 }
 next.ServeHTTP(w, r)
 })
 }
}
```

```

 }
 next.ServeHTTP(w, r)
 })
}
}
// e.g. RateLimit(10, 20) → 10 req/s, burst 20

```

## Per-IP limit (minimal)

```

type ipLimiter struct {
 mu sync.Mutex
 m map[string]*rate.Limiter
 r rate.Limit
 b int
}

func (l *ipLimiter) get(ip string) *rate.Limiter {
 l.mu.Lock()
 defer l.mu.Unlock()
 if lim, ok := l.m[ip]; ok {
 return lim
 }
 lim := rate.NewLimiter(l.r, l.b)
 l.m[ip] = lim
 return lim
}

```

Production: bound map size, use X-Forwarded-For only behind trusted proxies.

## Client-side (outbound)

```

lim := rate.NewLimiter(5, 5)
_ = lim.Wait(ctx) // before third-party API call

```

## Rules of thumb

Do	Don't
503/429 with Retry-After	Hang requests forever in queue without bound
Cap expensive routes tighter	One global limit for static + checkout
Limit outbound fan-out	Unlimited parallel dependency calls



## Try next

1. Burst 5 then 429.
2. Combine `MaxInFlight(50) + RateLimit`.
3. Load test with `hey` and tune.

# WebSockets Basics

# WebSockets Basics

## Overview

WebSockets upgrade an HTTP connection to a long-lived bidirectional channel. In Go, use [golang.org/x/net/websocket](https://golang.org/x/net/websocket) (older) or the popular [github.com/gorilla/websocket](https://github.com/gorilla/websocket) / [nhooyr.io/websocket](https://nhooyr.io/websocket). This chapter shows a **minimal** gorilla-style echo and operational pitfalls.

## Upgrade sketch (gorilla)

```
go get github.com/gorilla/websocket@latest

var upgrader = websocket.Upgrader{
 CheckOrigin: func(r *http.Request) bool {
 // tighten in production
 return true
 },
 ReadBufferSize: 1024,
 WriteBufferSize: 1024,
}

func echoWS(w http.ResponseWriter, r *http.Request) {
 c, err := upgrader.Upgrade(w, r, nil)
 if err != nil {
 return
 }
 defer c.Close()
 for {
 mt, msg, err := c.ReadMessage()
 if err != nil {
 break
 }
 if err := c.WriteMessage(mt, msg); err != nil {
 break
 }
 }
}
```

## Heartbeats

Idle proxies kill quiet connections. Send pings:

```
c.SetReadDeadline(time.Now().Add(60 * time.Second))
c.SetPongHandler(func(string) error {
 c.SetReadDeadline(time.Now().Add(60 * time.Second))
 return nil
})
// writer goroutine: c.WriteControl(websocket.PingMessage, ...)
```

## Concurrency

One reader goroutine; one writer goroutine; never concurrent writers without a mutex or dedicated write pump.

```
read loop → app → write pump (single)
```

## Auth

Authenticate on the **HTTP upgrade** (cookie/session or `Sec-WebSocket-Protocol` / query token). Don't invent a second weak auth after upgrade without TLS.

## When not to use WebSockets

Prefer WS	Prefer HTTP/SSE
True bidirectional chat/games	Server→client events only (SSE)
Low-latency duplex	Simple request/response

## Rules of thumb

Do	Don't
Limit message size	Read unbounded frames
Ping/pong deadlines	Assume connections live forever
Serialize writes	Write from many goroutines bare

## Try next

1. Echo server + browser `WebSocket` client.
2. Reject oversized messages.
3. Broadcast hub with register/unregister channels.

## **File Uploads and Downloads**

# File Uploads and Downloads

## Overview

Uploads and downloads stress memory, disk, and trust boundaries. Stream, bound sizes, and never trust client filenames.

## Upload (multipart)

```
func upload(w http.ResponseWriter, r *http.Request) {
 r.Body = http.MaxBytesReader(w, r.Body, 32<<20) // 32MiB
 if err := r.ParseMultipartForm(8 << 20); err != nil {
 http.Error(w, "too large or bad multipart", http.StatusBadRequest)
 return
 }
 file, hdr, err := r.FormFile("file")
 if err != nil {
 http.Error(w, "missing file", http.StatusBadRequest)
 return
 }
 defer file.Close()

 name := filepath.Base(hdr.Filename) // strip paths
 dstPath := filepath.Join(uploadDir, name)
 // better: random name + store original in DB
 dst, err := os.Create(dstPath)
 if err != nil {
 http.Error(w, "storage", 500)
 return
 }
 defer dst.Close()
 if _, err := io.Copy(dst, io.LimitReader(file, 32<<20)); err != nil {
 http.Error(w, "write", 500)
 return
 }
 fmt.Fprintln(w, "stored", name)
}
```

Sniff type: `http.DetectContentType` on first 512 bytes.

## Download

```
func download(w http.ResponseWriter, r *http.Request) {
 // resolve id → path inside jail (chapter 142 path jail)
 f, err := os.Open(safePath)
 if err != nil {
 http.NotFound(w, r)
 return
 }
 defer f.Close()
 st, _ := f.Stat()
 w.Header().Set("Content-Type", "application/octet-stream")
 w.Header().Set("Content-Disposition", `attachment; filename="report.bin"`)
 http.ServeContent(w, r, st.Name(), st.ModTime(), f) // supports Range
}
```

## Static files

```
mux.Handle("GET /static/", http.StripPrefix("/static/",
 http.FileServer(http.Dir("static"))))
```

For embed: `http.FileServer(http.FS(embedFS))`. Disable directory listing in production if needed (custom FS).

## Rules of thumb

Do	Don't
MaxBytesReader	Buffer whole upload in memory blindly
filepath.Base + jail	Use raw <code>hdr.Filename</code> paths
ServeContent for Range	Hand-roll partial downloads first

## Try next

1. Upload form + size limit test.
2. Download with `curl -C -` - resume via `ServeContent`.
3. Store under random UUID names.



## **Project: Bookstore API Capstone**

# Project: Bookstore API Capstone

## Overview

Combine this part into one **Bookstore API** binary: modular layout, REST, middleware, auth, DB interface, tests, graceful shutdown.

## Scope

Route	Access
GET /healthz	public
GET /readyz	public
GET /api/books	public
GET /api/books/{id}	public
POST /api/books	admin
POST /api/auth/login	public
POST /api/auth/logout	auth
GET /api/me	auth

## Checklist

- ☐ internal/ layout (272)
- ☐ memory store + interface (275)
- ☐ middleware chain: recover, request ID, log, max body (280)
- ☐ bcrypt login + session cookie (277)
- ☐ role check on create (277)
- ☐ graceful Shutdown (281)
- ☐ table tests + httptest (279)
- ☐ optional rate limit on login (282)

## Milestone order

1. health + list books (memory)
2. create book (no auth)
3. login + session + protect create
4. middleware + shutdown
5. tests
6. JSON error envelope consistency

## Stretch

- HTML list page (274)
- Postgres driver switch (275)
- `-strict` security headers via reverse proxy notes
- OpenAPI via Huma later (part 08-web)

## Done when

```
go test ./...
go run ./cmd/bookstore
curl -s localhost:8080/api/books | jq .
```

Login, set cookie jar, create book as admin; anonymous create returns 401.

## CLI Tools in Go

## Building CLI Tools in Go

# Building CLI Tools in Go

This part is a **hands-on curriculum for command-line tools**: flags, subcommands, exit codes, pipes, config, signals, tests—first with **stdlib only**, then with **Cobra**, **pflag**, **Viper**, and brief notes on other frameworks.

Go is one of the best languages for CLIs: single static binary, fast startup, excellent `flag / os / io / os/exec`, and a culture of small, composable tools.

**Related parts.** Stdlib surface: [987 flag / exec / signal](#). Pipeline philosophy: [143 Unix pipeline CLIs](#). Capstones in this part: [317 Network CLI tools](#) (ping, dns, scan, probe) and [318 Network security mini-tools](#) (tls, headers, banner, ...). More labs: [99 projects](#) (goping, linux clones, log shipper).

## Learning outcomes

After this part you can:

1. Ship a correct `main` that parses flags, writes stdout/stderr properly, and exits with useful codes
2. Implement subcommands with only `flag.FlagSet`
3. Read env vars and simple config files without a framework
4. Compose stdin/stdout pipelines and spawn subprocesses safely
5. Cancel work on SIGINT with `context + os/signal`
6. Test CLIs without flaky subprocess soup
7. Grow into Cobra when subcommands and completion matter
8. Wire Viper-style config only when env/file/flag layering earns its keep
9. Build multi-command **network diagnostic** tools (TCP ping, DNS, port scan, HTTP probe)
10. Build **simple network security** mini-tools (TLS cert, headers, banner, secrets hygiene)

## Path map

Stdlib core:	<code>anatomy → flag → subcommands → exits/streams</code>
Real tools:	<code>config → pipes → exec → signals → tests → layout</code>
Ecosystem:	<code>Cobra → advanced Cobra → Viper → other libs</code>
Practice:	<code>shipping → cookbook → netkit → netsec mini-tools</code>

Chapter	Focus
300	This overview
301	First tools with <code>os.Args</code>
302	<code>flag</code> package in depth
303	Subcommands with <code>FlagSet</code>
304	Exit codes, stdout vs stderr
305	Env vars and config files
306	Stdin, files, and pipes
307	<code>os/exec</code> orchestration
308	Signals and context
309	Testing CLIs
310	Project layout for larger CLIs
311	Cobra basics
312	Cobra advanced (persist flags, hooks, completion)
313	Viper and config layering
314	Other frameworks (urfave/cli, pflag, charm notes)
315	Version, install, completion, release
316	Cookbook: many end-to-end examples
317	<b>Project:</b> network CLI tools (ping, dns, scan, probe)
318	<b>Project:</b> simple network security mini-tools (tls, headers, ...)
319	<b>Project:</b> text/log tools ( <code>textkit</code> )
320	<b>Project:</b> HTTP debug CLI ( <code>httpdbg</code> )
321	Cross-platform CLI notes (Windows/paths/signals)

## Suggested schedule

Days 1-2	301-304	<code>stdlib</code> CLI hygiene
Days 3-4	305-308	config, pipes, exec, signals
Day 5	309-310	tests + package layout
Day 6	311-313	Cobra (+ Viper if needed)
Day 7	314-316	ecosystem + cookbook tools
Day 8	317	build <code>netkit</code> (ping/dns/scan/probe)
Day 9	318	build <code>netsec</code> (tls/headers/banner/...)
Day 10	319-321	<code>textkit</code> , <code>httpdbg</code> , portability

## Mental model: a CLI is a function

```
argv + env + stdin + files
```

```
parse flags / subcommand / config
```

```
run pure-ish domain work (testable)
```

```
stdout (data) · stderr (diagnostics) · exit code
```

Keep `run(args, stdin, stdout, stderr)` error (or a small `App` struct) so tests do not need `os.Exit` or `global flag.CommandLine`.

## Stdlib first, libraries later

Stay on stdlib when...	Reach for Cobra/Viper when...
One binary, few flags	Many nested subcommands
Internal tools / scripts	Public developer product CLI
You want zero deps	Shell completion + man-page ecosystem
Teaching / small utilities	Flag inheritance across command trees

Most famous Go CLIs started simple. Add libraries when the **command tree** or **config surface** hurts—not on day one of a toy.

## Running examples

Unless noted, examples assume:

```
mkdir -p /tmp/gocli && cd /tmp/gocli
go mod init example.com/gocli
paste main.go or package files
go run .
```



## First CLI Tools with os.Args

# First CLI Tools with os.Args

## Overview

Before `flag`, understand raw arguments: `os.Args[0]` is the program name, `os.Args[1:]` is everything the user typed. Many tiny tools need only this.

## Hello CLI

```
package main

import (
 "fmt"
 "os"
)

func main() {
 fmt.Println("args:", os.Args)
 if len(os.Args) < 2 {
 fmt.Fprintln(os.Stderr, "usage: greeter <name>")
 os.Exit(2)
 }
 fmt.Printf("hello, %s\n", os.Args[1])
}
```

```
go run . Ada
hello, Ada
go run .
usage: greeter <name> (exit 2)
```

## Separate main from logic

```
package main

import (
 "fmt"
 "io"
```

```

 "os"
)

func main() {
 if err := run(os.Args[1:], os.Stdout); err != nil {
 fmt.Fprintln(os.Stderr, "greeter:", err)
 os.Exit(1)
 }
}

func run(args []string, stdout io.Writer) error {
 if len(args) < 1 {
 return fmt.Errorf("usage: greeter <name>")
 }
 _, err := fmt.Fprintf(stdout, "hello, %s\n", args[0])
 return err
}

```

Now tests call `run([]string{"Ada"}, &buf)` without exiting the test process.

## Positional args patterns

### Required path

```

func run(args []string) error {
 if len(args) != 1 {
 return fmt.Errorf("usage: cat1 <file>")
 }
 b, err := os.ReadFile(args[0])
 if err != nil {
 return err
 }
 _, err = os.Stdout.Write(b)
 return err
}

```

### Variadic files (like cat)

```

func run(args []string, stdout io.Writer) error {
 if len(args) == 0 {
 _, err := io.Copy(stdout, os.Stdin)
 return err
 }
}

```

```

 for _, path := range args {
 f, err := os.Open(path)
 if err != nil {
 return err
 }
 _, err = io.Copy(stdout, f)
 f.Close()
 if err != nil {
 return err
 }
 }
 return nil
}

```

## Options before flags package (-n style DIY)

```

func parse(args []string) (count int, rest []string, err error) {
 count = 1
 i := 0
 for i < len(args) {
 a := args[i]
 if a == "--" {
 i++
 break
 }
 if len(a) > 1 && a[0] == '-' {
 // fragile DIY - prefer flag package next chapter
 return 0, nil, fmt.Errorf("unknown option %q", a)
 }
 break
 }
 return count, args[i:], nil
}

```

**Lesson:** hand-parsing flags gets messy quickly. Use `flag` (or `Cobra`) for real options.

## Program name for usage

```

name := filepath.Base(os.Args[0])
fmt.Fprintf(os.Stderr, "usage: %s <file>\n", name)

```

## Environment as “args”

```
if os.Getenv("DEBUG") != "" {
 fmt.Fprintln(os.Stderr, "debug on")
}
```

Env is global config; args are per-invocation. Chapter 305 covers layering.

## Example: add calculator

```
package main

import (
 "fmt"
 "os"
 "strconv"
)

func main() {
 if err := run(os.Args[1:]); err != nil {
 fmt.Fprintln(os.Stderr, err)
 os.Exit(1)
 }
}

func run(args []string) error {
 if len(args) < 2 {
 return fmt.Errorf("usage: add <a> ")
 }
 a, err := strconv.ParseFloat(args[0], 64)
 if err != nil {
 return fmt.Errorf("a: %w", err)
 }
 b, err := strconv.ParseFloat(args[1], 64)
 if err != nil {
 return fmt.Errorf("b: %w", err)
 }
 fmt.Println(a + b)
 return nil
}
```

## Example: whichdir — print working directory variants

```
func run(args []string) error {
 wd, err := os.Getwd()
 if err != nil {
 return err
 }
 fmt.Println(wd)
 if len(args) > 0 && args[0] == "--base" {
 fmt.Println(filepath.Base(wd))
 }
 return nil
}
```

## Rules of thumb

Do	Don't
Put logic in <code>run(...)</code>	Scatter <code>os.Exit</code> deep in packages
Usage on stderr, exit 2 for misuse	Print usage to stdout
Validate <code>len(args)</code> early	Index <code>args[1]</code> without checks

## Try next

1. Write `echo1` that joins args with spaces (like `echo`).
2. Write `headn` that prints the first line of a file (no flags yet).
3. Add a unit test for `run` with a `bytes.Buffer` as stdout.

## **flag Package in Depth**

# flag Package in Depth

## Overview

Package `flag` is the `stdlib` CLI parser: `bool/int/string/duration` flags, custom `Value` types, and usage text. Master this before `Cobra`—`Cobra`'s flags are powered by **pflag**, a POSIX-friendly fork of the same ideas.

## Minimal example

```
package main

import (
 "flag"
 "fmt"
 "os"
 "time"
)

func main() {
 var (
 name = flag.String("name", "world", "name to greet")
 times = flag.Int("n", 1, "repeat count")
 verbose = flag.Bool("v", false, "verbose")
 timeout = flag.Duration("timeout", 2*time.Second, "budget")
)
 flag.Parse()

 if *verbose {
 fmt.Fprintf(os.Stderr, "timeout=%s args=%v\n", *timeout, flag.Args())
 }
 for i := 0; i < *times; i++ {
 fmt.Printf("hello, %s\n", *name)
 }
}
```

```
go run . -name Ada -n 2 -v -- leftover
flags after -- are not parsed as flags in some styles;
with flag, non-flag args are flag.Args() after Parse
```



## Var forms (bind to your own vars)

```
var cfg struct {
 Addr string
 Port int
}
flag.StringVar(&cfg.Addr, "addr", "127.0.0.1", "listen address")
flag.IntVar(&cfg.Port, "port", 8080, "port")
flag.Parse()
```

Prefer Var when flags map into a config struct.

## Custom Usage

```
flag.Usage = func() {
 fmt.Fprintf(flag.CommandLine.Output(),
 "Usage: %s [flags] <path>\n\nFlags:\n", filepath.Base(os.Args[0]))
 flag.PrintDefaults()
}
```

`flag.CommandLine.Output()` defaults to `stderr`—keep it that way.

## Error handling modes

```
fs := flag.NewFlagSet("tool", flag.ContinueOnError)
// flag.ExitOnError - print usage, os.Exit(2) (CommandLine default)
// flag.PanicOnError - panic (rare)
// flag.ContinueOnError - return error from Parse (best for tests)
```

For testable tools:

```
fs := flag.NewFlagSet(os.Args[0], flag.ContinueOnError)
fs.SetOutput(io.Discard) // or a buffer when asserting usage
err := fs.Parse(args)
```

## Custom flag types (flag.Value)

```
type stringsFlag []string

func (s *stringsFlag) String() string { return fmt.Sprint(*s) }
func (s *stringsFlag) Set(v string) error {
 *s = append(*s, v)
 return nil
}

func main() {
 var tags stringsFlag
 flag.Var(&tags, "tag", "repeatable tag (use multiple -tag)")
 flag.Parse()
 fmt.Println(tags)
}

go run . -tag a -tag b
[a b]
```

## Enum flag

```
type level string

func (l *level) String() string { return string(*l) }
func (l *level) Set(v string) error {
 switch v {
 case "debug", "info", "error":
 *l = level(v)
 return nil
 default:
 return fmt.Errorf("level must be debug|info|error")
 }
}

var lvl level = "info"
flag.Var(&lvl, "level", "log level")
```

## Bool flags

```
verbose := flag.Bool("verbose", false, "verbose")
// -verbose, -verbose=true, -verbose=false
// short -v needs a separate flag or pflag/Cobra
flag.Bool("v", false, "verbose (short)")
```

Stdlib flag does **not** combine short options (-abc). Use pflag/Cobra for GNU style.

## Positional args after flags

```
flag.Parse()
paths := flag.Args()
if len(paths) == 0 {
 fmt.Fprintln(os.Stderr, "need at least one path")
 os.Exit(2)
}
```

Order: users typically pass flags first. With stdlib flag, a non-flag stops flag parsing for the rest (classic Go behavior).

## Example: httpget (stdlib-only client)

```
func main() {
 url := flag.String("url", "", "URL to GET")
 timeout := flag.Duration("timeout", 10*time.Second, "client timeout")
 flag.Parse()
 if *url == "" {
 flag.Usage()
 os.Exit(2)
 }
 client := &http.Client{Timeout: *timeout}
 resp, err := client.Get(*url)
 if err != nil {
 fmt.Fprintln(os.Stderr, err)
 os.Exit(1)
 }
 defer resp.Body.Close()
 fmt.Println(resp.Status)
 _, _ = io.Copy(os.Stdout, io.LimitReader(resp.Body, 1<<20))
}
```

## Example: sleepfor

```
d := flag.Duration("d", time.Second, "sleep duration")
flag.Parse()
time.Sleep(*d)
```

## Example: lines — count lines in files

```
func main() {
 verbose := flag.Bool("v", false, "print per-file counts")
 flag.Parse()
 total := 0
 for _, path := range flag.Args() {
 n, err := countLines(path)
 if err != nil {
 fmt.Fprintln(os.Stderr, err)
 os.Exit(1)
 }
 if *verbose {
 fmt.Printf("%d\t%s\n", n, path)
 }
 total += n
 }
 fmt.Println(total)
}
```

## Rules of thumb

Do	Don't
Document every flag default in help	Undocumented env-only secret flags as the only API
<code>ContinueOnError</code> + <code>run()</code> for tests	Parse global flag in <code>init</code> of libraries
Custom <code>Value</code> for repeatable/enum	Parse enums with silent fallbacks

## Try next

1. Add a repeatable `-header k=v` flag using `flag.Value`.
2. Write tests that parse a private `FlagSet` with `ContinueOnError`.

3. Build `greet -n 3 -name X` and verify usage on bad `-n=abc`.

## Subcommands with `flag.FlagSet`

# Subcommands with flag.FlagSet

## Overview

Real tools grow verbs: `git status`, `docker compose up`, `kubectl get pods`. You can implement subcommands with **only** the standard library via `flag.NewFlagSet`—no Cobra required.

## Dispatch pattern

```
os.Args[0] program
os.Args[1] subcommand (get|set|list)
os.Args[2:] flags + args for that subcommand

package main

import (
 "fmt"
 "os"
)

func main() {
 if len(os.Args) < 2 {
 usage()
 os.Exit(2)
 }
 var err error
 switch os.Args[1] {
 case "get":
 err = cmdGet(os.Args[2:])
 case "set":
 err = cmdSet(os.Args[2:])
 case "list":
 err = cmdList(os.Args[2:])
 case "help", "-h", "--help":
 usage()
 return
 default:
 fmt.Fprintf(os.Stderr, "unknown command %q\n", os.Args[1])
 }
```

```

 usage()
 os.Exit(2)
 }
 if err != nil {
 fmt.Fprintln(os.Stderr, err)
 os.Exit(1)
 }
}

func usage() {
 fmt.Fprintf(os.Stderr, `Usage: kv <command> [flags]

Commands:
 get get a key
 set set a key
 list list keys
`)
}

```

## Per-command FlagSet

```

func cmdGet(args []string) error {
 fs := flag.NewFlagSet("get", flag.ContinueOnError)
 fs.SetOutput(os.Stderr)
 raw := fs.Bool("raw", false, "print raw value only")
 if err := fs.Parse(args); err != nil {
 return err
 }
 if fs.NArg() != 1 {
 return fmt.Errorf("usage: kv get [-raw] <key>")
 }
 key := fs.Arg(0)
 val, ok := store[key]
 if !ok {
 return fmt.Errorf("key not found: %s", key)
 }
 if *raw {
 fmt.Print(val)
 return nil
 }
 fmt.Printf("%s=%s\n", key, val)
 return nil
}

```



```

func cmdSet(args []string) error {
 fs := flag.NewFlagSet("set", flag.ContinueOnError)
 fs.SetOutput(os.Stderr)
 if err := fs.Parse(args); err != nil {
 return err
 }
 if fs.NArg() != 2 {
 return fmt.Errorf("usage: kv set <key> <value>")
 }
 store[fs.Arg(0)] = fs.Arg(1)
 return nil
}

```

## Shared “global” flags

Parse globals **before** the subcommand, or require globals after the verb (simpler with FlagSet-per-command).

### Globals before verb (manual)

```

// mytool -verbose get -id 1
// harder with stdlib flag alone - often document:
// mytool get -verbose -id 1

```

### Globals on every FlagSet (duplicate)

```

func addGlobal(fs *flag.FlagSet, verbose *bool) {
 fs.BoolVar(verbose, "verbose", false, "verbose output")
 fs.BoolVar(verbose, "v", false, "verbose output (short)")
}

```

Cobra’s persistent flags solve this more cleanly (chapter 312). For stdlib, **prefer flags after the subcommand**.

## Nested subcommands

```

// tool remote add
// tool remote list
func cmdRemote(args []string) error {
 if len(args) < 1 {
 return fmt.Errorf("usage: tool remote <add|list>")
 }
}

```

```

 }
 switch args[0] {
 case "add":
 return cmdRemoteAdd(args[1:])
 case "list":
 return cmdRemoteList(args[1:])
 default:
 return fmt.Errorf("unknown remote command %q", args[0])
 }
}

```

## Example: mini todo CLI

```

// commands: add, list, done
// storage: JSON file in OS user config dir

func cmdAdd(args []string) error {
 fs := flag.NewFlagSet("add", flag.ContinueOnError)
 pri := fs.Int("pri", 0, "priority")
 if err := fs.Parse(args); err != nil {
 return err
 }
 if fs.NArg() < 1 {
 return fmt.Errorf("usage: todo add [-pri N] <text>")
 }
 text := strings.Join(fs.Args(), " ")
 return saveNewTask(text, *pri)
}

```

## Help that lists commands

```

var commands = map[string]string{
 "get": "get a key",
 "set": "set a key",
 "list": "list all keys",
}

func usage() {
 fmt.Fprintln(os.Stderr, "Usage: kv <command> [flags]")
 fmt.Fprintln(os.Stderr, "\nCommands:")
 keys := slices.Sorted(maps.Keys(commands))
 for _, k := range keys {

```

```

 fmt.Fprintf(os.Stderr, " %-8s %s\n", k, commands[k])
 }
}

```

## Alias map

```

aliases := map[string]string{"ls": "list", "rm": "delete"}
cmd := os.Args[1]
if a, ok := aliases[cmd]; ok {
 cmd = a
}

```

## When FlagSet is enough

Enough	Grow to Cobra
~10 commands, flat or shallow	Deep trees, many shared flags
Internal tooling	Public CLI with completion
You want zero dependencies	Man pages, docs generators

## Rules of thumb

Do	Don't
One <code>FlagSet</code> per command	One global <code>flag.Parse</code> for all verbs
<code>ContinueOnError</code> in libraries/tests	<code>ExitOnError</code> inside <code>run</code> used by tests
Document command list in usage	Silent unknown commands

## Try next

1. Implement `kv get|set|list` with an in-memory map (then JSON file).
2. Add `kv help get` that prints only the `get` `FlagSet` defaults.
3. Add nested config `get` / `config set`.

## **Exit Codes, Stdout, and Stderr**

# Exit Codes, Stdout, and Stderr

## Overview

Automation treats your CLI as a machine API: **stdout is data, stderr is diagnostics, exit code is success/failure**. Break that contract and scripts, CI, and pipes fail in confusing ways.

Also see [Unix pipeline CLI tools](#).

## The contract

```
exit 0 success
exit 1 runtime failure (file missing, network error)
exit 2 misuse (bad flags, wrong arity) - common convention
exit 130 interrupted by SIGINT (128+2) - platform dependent
```

Go's flag with `ExitOnError` exits **2** on parse errors—good.

```
func main() {
 if err := run(os.Args[1:]); err != nil {
 fmt.Fprintln(os.Stderr, "mytool:", err)
 var code int
 switch {
 case errors.Is(err, errUsage):
 code = 2
 default:
 code = 1
 }
 os.Exit(code)
 }
}
```

## Stdout vs stderr

Stream	Put here
stdout	JSON results, tables meant for pipes, primary output
stderr	Usage, progress, logs, warnings

```

fmt.Fprintln(os.Stderr, "fetching...") // progress
fmt.Println(string(jsonBytes)) // data for jq

```

```

mytool 2>log.txt | jq . # logs aside, data through pipe

```

## Quiet / verbose / JSON modes

```

type mode int

const (
 modeText mode = iota
 modeJSON
)

func run(stdout, stderr io.Writer, verbose bool, m mode, data Result) error {
 if verbose {
 fmt.Fprintln(stderr, "processing", data.ID)
 }
 if m == modeJSON {
 enc := json.NewEncoder(stdout)
 enc.SetIndent("", " ")
 return enc.Encode(data)
 }
 fmt.Fprintf(stdout, "%s\t%d\n", data.Name, data.Count)
 return nil
}

```

## Is stdout a TTY?

```

func stdoutIsTTY() bool {
 fi, err := os.Stdout.Stat()
 if err != nil {
 return false
 }
 return fi.Mode() & os.ModeCharDevice != 0
}

```

Use TTY detection for:

- color (only if TTY and NO\_COLOR unset)
- progress bars
- human tables vs TSV for pipes

```

if stdoutIsTTY() && os.Getenv("NO_COLOR") == "" {
 // optional color codes
}

```

Respect `NO_COLOR` when you add color (stdlib has no color package—use raw ANSI sparingly or a small lib later).

## Don't call `os.Exit` in libraries

```

// package greeter
func Greet(w io.Writer, name string) error { ... }

// package main
func main() {
 if err := greeter.Greet(os.Stdout, name); err != nil {
 os.Exit(1)
 }
}

```

`os.Exit` skips deferred cleanup in the current goroutine's stack for the process—keep it at the edge.

## Example: correct `grep`-shaped output

```

// matches to stdout, errors to stderr, exit 0 if any match, 1 if none (grep-like)
matched := false
for _, line := range lines {
 if strings.Contains(line, pattern) {
 fmt.Fprintln(stdout, line)
 matched = true
 }
}
if !matched {
 return errNoMatch // main maps to exit 1
}

```

Document whether you follow `grep`'s exit semantics—they surprise some users.

## Machine-readable errors

```
type cliError struct {
 Code int `json:"code"`
 Message string `json:"message"`
}

// when --json-errors and failing:
json.NewEncoder(os.Stderr).Encode(cliError{Code: 1, Message: err.Error()})
```

## Rules of thumb

Do	Don't
Data → stdout	Mix log lines into JSON stdout
Diagnostics → stderr	Require users to scrape stdout for errors
Non-zero on failure	Always exit 0 and bury errors in text
Map usage errors to 2	Use exit 1 for everything without docs

## Try next

1. Build a tool that prints JSON on stdout and timing on stderr; pipe through jq.
2. Add `--quiet` that silences stderr progress only.
3. Verify exit codes in a shell: `go run . bad; echo $?`.



# Environment Variables and Config Files

# Environment Variables and Config Files

## Overview

Flags are per-invocation. Environment variables and config files hold **defaults** that follow the user or deployment. Stdlib-only tooling can load both cleanly; Viper (chapter 313) automates layering when complexity grows.

## Precedence (recommended)

```
flags > env > config file > built-in defaults
```

Document this in `--help`. Users should never wonder why a value “stuck.”

## Environment variables

```
func getenv(key, def string) string {
 if v := os.Getenv(key); v != "" {
 return v
 }
 return def
}

func getenvInt(key string, def int) int {
 v := os.Getenv(key)
 if v == "" {
 return def
 }
 n, err := strconv.Atoi(v)
 if err != nil {
 return def // or return error - prefer fail-fast for CLIs
 }
 return n
}
```

## 12-factor style names

```
MYTOOL_API_URL=https://api.example.com
MYTOOL_TOKEN=...
MYTOOL_TIMEOUT=10s
```

```
apiURL := getenv("MYTOOL_API_URL", "http://127.0.0.1:8080")
```

## Parse duration from env

```
func envDuration(key string, def time.Duration) (time.Duration, error) {
 v := os.Getenv(key)
 if v == "" {
 return def, nil
 }
 return time.ParseDuration(v)
}
```

## Booleans

```
func envBool(key string, def bool) bool {
 v := strings.ToLower(strings.TrimSpace(os.Getenv(key)))
 switch v {
 case "":
 return def
 case "1", "true", "yes", "y", "on":
 return true
 case "0", "false", "no", "n", "off":
 return false
 default:
 return def
 }
}
```

Prefer failing on garbage if the var is critical.

## Config file locations

```
func defaultConfigPath(app string) (string, error) {
 dir, err := os.UserConfigDir() // platform-correct
 if err != nil {
 return "", err
 }
}
```

```

 }
 return filepath.Join(dir, app, "config.json"), nil
}

```

OS	UserConfigDir typical
macOS	~/Library/Application Support
Linux	~/.config
Windows	%AppData%

Also support explicit `--config path`.

## JSON config (stdlib)

```

type Config struct {
 APIURL string `json:"api_url"`
 Timeout string `json:"timeout"` // parse as duration
 Verbose bool `json:"verbose"`
}

func loadConfig(path string) (Config, error) {
 var c Config
 b, err := os.ReadFile(path)
 if err != nil {
 if os.IsNotExist(err) {
 return Config{}, nil // missing file OK
 }
 return Config{}, err
 }
 if err := json.Unmarshal(b, &c); err != nil {
 return Config{}, fmt.Errorf("config %s: %w", path, err)
 }
 return c, nil
}

```

## Merge with flags

```

type Options struct {
 APIURL string
 Verbose bool
 Timeout time.Duration
}

```

```

func parse(args []string) (Options, error) {
 fs := flag.NewFlagSet("mytool", flag.ContinueOnError)
 cfgPath := fs.String("config", "", "config file path")
 api := fs.String("api-url", "", "API base URL")
 verbose := fs.Bool("verbose", false, "verbose")
 timeout := fs.Duration("timeout", 0, "timeout (0=default)")
 if err := fs.Parse(args); err != nil {
 return Options{}, err
 }

 path := *cfgPath
 if path == "" {
 path, _ = defaultConfigPath("mytool")
 }
 fileCfg, err := loadConfig(path)
 if err != nil {
 return Options{}, err
 }

 opt := Options{
 APIURL: firstNonEmpty(*api, os.Getenv("MYTOOL_API_URL"), fileCfg.APIURL, "http://12
 Verbose: *verbose || envBool("MYTOOL_VERBOSE", false) || fileCfg.Verbose,
 Timeout: 10 * time.Second,
 }
 if *timeout > 0 {
 opt.Timeout = *timeout
 } else if d, err := envDuration("MYTOOL_TIMEOUT", 0); err == nil && d > 0 {
 opt.Timeout = d
 } else if fileCfg.Timeout != "" {
 d, err := time.ParseDuration(fileCfg.Timeout)
 if err != nil {
 return Options{}, err
 }
 opt.Timeout = d
 }
 return opt, nil
}

func firstNonEmpty(vals ...string) string {
 for _, v := range vals {
 if v != "" {
 return v
 }
 }
 return ""
}

```

## Dotenv? Not in stdlib

.env files are not standard. Options:

1. Document that users `export $(cat .env | xargs)` in shell
2. Parse a simple `KEY=VAL` file yourself (~30 lines)
3. Use a small library later

A minimal dotenv:

```
func loadDotEnv(path string) error {
 b, err := os.ReadFile(path)
 if err != nil {
 return err
 }
 for _, line := range strings.Split(string(b), "\n") {
 line = strings.TrimSpace(line)
 if line == "" || strings.HasPrefix(line, "#") {
 continue
 }
 k, v, ok := strings.Cut(line, "=")
 if !ok {
 continue
 }
 k = strings.TrimSpace(k)
 v = strings.TrimSpace(v)
 if os.Getenv(k) == "" { // don't override existing env
 _ = os.Setenv(k, v)
 }
 }
 return nil
}
```

## Secrets

- Prefer env or OS keychain for tokens—not committed JSON
- Support `MYTOOL_TOKEN_FILE` pointing at a file (Kubernetes-friendly)

```
func loadToken() (string, error) {
 if t := os.Getenv("MYTOOL_TOKEN"); t != "" {
 return t, nil
 }
 if p := os.Getenv("MYTOOL_TOKEN_FILE"); p != "" {
 b, err := os.ReadFile(p)
 return strings.TrimSpace(string(b)), err
 }
}
```

```
 return "", fmt.Errorf("set MYTOOL_TOKEN or MYTOOL_TOKEN_FILE")
}
```

## Rules of thumb

Do	Don't
Document precedence	Silent flag < env surprises
Allow missing config file	Crash if optional file absent
Fail on invalid config JSON	Ignore corrupt files
Use <code>UserConfigDir</code>	Hardcode <code>~/.mytool</code> only

## Try next

1. Implement `parse` with flag > env > JSON defaults.
2. Add `--print-config` that dumps resolved options (redact tokens).
3. Support `TOKEN_FILE` and test with `t.Setenv`.

## **Stdin, Files, and Pipes**



# Stdin, Files, and Pipes

## Overview

Great CLIs work in pipelines: read files or stdin, write transform results to stdout. This chapter is pure stdlib patterns for input discovery, streaming, and safe file handling.

## Input priority (common UX)

1. Explicit file args
2. If no args (or arg is "-") → stdin

```
func openInputs(args []string) ([]io.ReadCloser, error) {
 if len(args) == 0 {
 return []io.ReadCloser{io.NopCloser(os.Stdin)}, nil
 }
 var out []io.ReadCloser
 for _, a := range args {
 if a == "-" {
 out = append(out, io.NopCloser(os.Stdin))
 continue
 }
 f, err := os.Open(a)
 if err != nil {
 // close already opened on error
 for _, c := range out {
 c.Close()
 }
 return nil, err
 }
 out = append(out, f)
 }
 return out, nil
}
```

## Stream line by line

```
func process(r io.Reader, w io.Writer, pred func(string) bool) error {
 sc := bufio.NewScanner(r)
 // optional: larger tokens
 // buf := make([]byte, 0, 64*1024)
 // sc.Buffer(buf, 1024*1024)
 for sc.Scan() {
 line := sc.Text()
 if pred(line) {
 if _, err := fmt.Fprintln(w, line); err != nil {
 return err
 }
 }
 }
 return sc.Err()
}
```

## Example: grep1 (stdlib)

```
func main() {
 n := flag.Bool("n", false, "print line numbers")
 flag.Parse()
 if flag.NArg() < 1 {
 fmt.Fprintln(os.Stderr, "usage: grep1 [-n] <pattern> [files...]")
 os.Exit(2)
 }
 pat := flag.Arg(0)
 files := flag.Args()[1:]
 inputs, err := openInputs(files)
 if err != nil {
 fmt.Fprintln(os.Stderr, err)
 os.Exit(1)
 }
 matched := false
 lineNo := 0
 for _, in := range inputs {
 sc := bufio.NewScanner(in)
 for sc.Scan() {
 lineNo++
 line := sc.Text()
 if strings.Contains(line, pat) {
 matched = true
 if *n {
```

```

 fmt.Printf("%d:%s\n", lineNo, line)
 } else {
 fmt.Println(line)
 }
}
}
err = sc.Err()
in.Close()
if err != nil {
 fmt.Fprintln(os.Stderr, err)
 os.Exit(1)
}
}
if !matched {
 os.Exit(1)
}
}

```

```

echo -e "go\nrust\ngo lang" | go run . go
printf 'a\nb\n' > t.txt && go run . -n b t.txt

```

## Example: wc1 lines/words/bytes

```

func count(r io.Reader) (lines, words, bytes int, err error) {
 br := bufio.NewReader(r)
 for {
 b, e := br.ReadByte()
 if e != nil {
 if e == io.EOF {
 return lines, words, bytes, nil
 }
 return 0, 0, 0, e
 }
 bytes++
 if b == '\n' {
 lines++
 }
 }
}
// words: use bufio.Scanner with Split(bufio.ScanWords) in a second pass
// or track isSpace transitions in the byte loop

```

## Example: JSON filter from stdin

```
func run(r io.Reader, w io.Writer, field string) error {
 dec := json.NewDecoder(r)
 enc := json.NewEncoder(w)
 for {
 var m map[string]any
 if err := dec.Decode(&m); err != nil {
 if err == io.EOF {
 return nil
 }
 return err
 }
 if v, ok := m[field]; ok {
 if err := enc.Encode(v); err != nil {
 return err
 }
 }
 }
}
```

```
echo '{"name":"a"}{"name":"b"}' | go run . -field name
```

## Detect pipe vs TTY for stdin

```
func stdinIsPipe() bool {
 fi, err := os.Stdin.Stat()
 if err != nil {
 return false
 }
 return fi.Mode()&os.ModeCharDevice == 0
}
```

If stdin is a TTY and no files given, you may print usage instead of blocking forever:

```
if len(args) == 0 && !stdinIsPipe() {
 return fmt.Errorf("usage: tool <files> or pipe stdin")
}
```

(Some tools intentionally wait for interactive stdin—document either way.)

## Atomic write of output files

```
func writeFileAtomic(path string, data []byte, mode os.FileMode) error {
 dir := filepath.Dir(path)
 tmp, err := os.CreateTemp(dir, ".tmp-*")
 if err != nil {
 return err
 }
 tmpName := tmp.Name()
 defer func() { _ = os.Remove(tmpName) }() // if not renamed

 if _, err := tmp.Write(data); err != nil {
 tmp.Close()
 return err
 }
 if err := tmp.Chmod(mode); err != nil {
 tmp.Close()
 return err
 }
 if err := tmp.Close(); err != nil {
 return err
 }
 return os.Rename(tmpName, path)
}
```

## Progress on stderr only

```
fmt.Fprintf(os.Stderr, "\rprocessed %d", n)
// final newline
fmt.Fprintln(os.Stderr)
```

Never put `\r` progress on stdout if users pipe to files.

## Rules of thumb

Do	Don't
Stream with <code>bufio</code>	<code>ReadAll</code> multi-GB stdin
Accept - as stdin	Force temp files for every pipeline
Close files	Leak FDs in long loops
Errors → stderr	Corrupt the data stream

## Try next

1. Implement `uniq1` adjacent-line dedupe from stdin.
2. Add `cut1 -f 2 -d ,` for CSV-ish lines.
3. Pipe `grep1` into `wc -l` and confirm exit codes compose.

## **os/exec Orchestration**

# os/exec Orchestration

## Overview

CLIs often wrap other tools: `git`, `docker`, `kubectl`, compilers. Package `os/exec` runs subprocesses with explicit `argv` (never shell strings with untrusted input).

## Basics

```
cmd := exec.CommandContext(ctx, "git", "status", "--short")
cmd.Dir = repoDir
cmd.Env = append(os.Environ(), "LC_ALL=C")
out, err := cmd.Output() // stdout; stderr in ExitError.Stderr if failed
```

Method	Captures
<code>Run</code>	exit only; you wire <code>Stdout/Stderr</code>
<code>Output</code>	stdout bytes; err on non-zero
<code>CombinedOutput</code>	stdout+stderr merged
<code>Start/Wait</code>	async

## Stream to the user

```
cmd := exec.Command("go", "test", "./...")
cmd.Stdout = os.Stdout
cmd.Stderr = os.Stderr
cmd.Stdin = os.Stdin
err := cmd.Run()
```

## Capture and inspect exit code

```
err := cmd.Run()
if err != nil {
 var ee *exec.ExitError
 if errors.As(err, &ee) {
```



```

 return fmt.Errorf("git exit %d", ee.ExitCode())
 }
 return err // not found, killed, etc.
}

```

## Pipes between processes

```

func pipeline(ctx context.Context) error {
 ps := exec.CommandContext(ctx, "ps", "aux")
 grep := exec.CommandContext(ctx, "grep", "go")

 r, w := io.Pipe()
 ps.Stdout = w
 grep.Stdin = r
 var grepOut bytes.Buffer
 grep.Stdout = &grepOut

 if err := ps.Start(); err != nil {
 return err
 }
 if err := grep.Start(); err != nil {
 return err
 }
 errPs := ps.Wait()
 _ = w.Close() // signal EOF to grep
 errGrep := grep.Wait()
 if errPs != nil {
 return errPs
 }
 if errGrep != nil {
 return errGrep
 }
 fmt.Print(grepOut.String())
 return nil
}

```

Prefer shell for one-off scripts; prefer argv pipelines in production wrappers for safety.

## Never shell untrusted input

```
// BAD
exec.Command("sh", "-c", "echo "+userInput)

// GOOD
exec.Command("echo", userInput)
```

If you must use a shell, allowlist and escape carefully—usually you should not.

## LookPath

```
path, err := exec.LookPath("docker")
if err != nil {
 return fmt.Errorf("docker not installed: %w", err)
}
_ = path
```

## Example: gofmt-write wrapper

```
func run(paths []string) error {
 args := append([]string{"-w"}, paths...)
 cmd := exec.Command("gofmt", args...)
 cmd.Stdout = os.Stdout
 cmd.Stderr = os.Stderr
 return cmd.Run()
}
```

## Example: parallel commands with errgroup

```
eg, ctx := errgroup.WithContext(context.Background())
for _, pkg := range packages {
 pkg := pkg
 eg.Go(func() error {
 cmd := exec.CommandContext(ctx, "go", "test", pkg)
 out, err := cmd.CombinedOutput()
 if err != nil {
 return fmt.Errorf("%s: %s: %w", pkg, out, err)
 }
 return nil
 })
}
```

```

 })
}
return eg.Wait()

```

Bound parallelism if the child tools are heavy.

## Example: runjson — exec and parse JSON stdout

```

cmd := exec.CommandContext(ctx, "docker", "inspect", id)
out, err := cmd.Output()
if err != nil {
 return err
}
var payload any
if err := json.Unmarshal(out, &payload); err != nil {
 return fmt.Errorf("parse docker json: %w", err)
}

```

## Timeouts

```

ctx, cancel := context.WithTimeout(context.Background(), 30*time.Second)
defer cancel()
cmd := exec.CommandContext(ctx, "make", "test")
err := cmd.Run()
if errors.Is(ctx.Err(), context.DeadlineExceeded) {
 return fmt.Errorf("timed out")
}

```

CommandContext sends kill when ctx cancels (details differ by OS for process groups).

## Rules of thumb

Do	Don't
Argv slices	<code>sh -c</code> with user strings
CommandContext	Untimed long children
Surface exit codes	Swallow <code>ExitError</code> without code
Set <code>Dir/Env</code> explicitly	Rely on ambient cwd silently

## Try next

1. Wrap `git rev-parse --short HEAD` and print version.
2. Run `go test` with `timeout 5s`; assert cancel path.
3. Build `whichall` that LookPaths a list of tools and exits 1 if any missing.

## Signals and Context in CLIs

# Signals and Context in CLIs

## Overview

Users hit Ctrl-C. Deployments send SIGTERM. Good CLIs cancel work, close files, and exit promptly. Wire `os/signal` to `context` and pass that context through I/O and `exec`.

## Minimal graceful cancel

```
func main() {
 ctx, stop := signal.NotifyContext(context.Background(), os.Interrupt, syscall.SIGTERM)
 defer stop()

 if err := run(ctx, os.Args[1:]); err != nil {
 if errors.Is(err, context.Canceled) {
 fmt.Fprintln(os.Stderr, "interrupted")
 os.Exit(130)
 }
 fmt.Fprintln(os.Stderr, err)
 os.Exit(1)
 }
}
```

`signal.NotifyContext` (Go 1.16+) is the preferred pattern.

## Pass ctx everywhere

```
func run(ctx context.Context, args []string) error {
 // HTTP
 req, err := http.NewRequestWithContext(ctx, http.MethodGet, url, nil)
 // ...
 // Exec
 cmd := exec.CommandContext(ctx, "sleep", "60")
 // Files: poll ctx in loops
 return workLoop(ctx)
}
```

```

func workLoop(ctx context.Context) error {
 for i := 0; i < 1000; i++ {
 select {
 case <-ctx.Done():
 return ctx.Err()
 default:
 }
 // unit of work
 }
 return nil
}

```

## Long-running workers

```

func runWorkers(ctx context.Context, jobs <-chan Job) error {
 var wg sync.WaitGroup
 defer wg.Wait()

 for i := 0; i < 4; i++ {
 wg.Add(1)
 go func() {
 defer wg.Done()
 for {
 select {
 case <-ctx.Done():
 return
 case j, ok := <-jobs:
 if !ok {
 return
 }
 process(ctx, j)
 }
 }
 }()
 }
 // producer closes jobs when done; main waits via defer wg.Wait
 // cancel via signal cancels ctx so workers return
 return nil
}

```

## Ignore SIGPIPE? (advanced)

Writing to a closed pipe (downstream head exited) can raise SIGPIPE. Go's runtime usually converts write errors to EPIPE—check write errors on stdout:

```
if _, err := fmt.Fprintln(os.Stdout, line); err != nil {
 return err // broken pipe - exit quietly or 1
}
```

## Example: cancellable download CLI

```
func download(ctx context.Context, url, dest string) error {
 req, err := http.NewRequestWithContext(ctx, http.MethodGet, url, nil)
 if err != nil {
 return err
 }
 resp, err := http.DefaultClient.Do(req)
 if err != nil {
 return err
 }
 defer resp.Body.Close()

 f, err := os.Create(dest)
 if err != nil {
 return err
 }
 defer f.Close()

 _, err = io.Copy(f, resp.Body) // Copy stops when ctx cancels the body
 return err
}
```

Use a client with timeout as a backstop even when signals work.

## Double Ctrl-C

Some tools escalate: first signal cancels gracefully; second forces `os.Exit(1)`. Keep it simple for most CLIs—one cancel is enough if work respects ctx.

```
go func() {
 <-ctx.Done()
 // optional: force exit after grace period
 t := time.NewTimer(5 * time.Second)
```



```

defer t.Stop()
select {
case <-t.C:
 fmt.Fprintln(os.Stderr, "forcing exit")
 os.Exit(1)
}
}()

```

## Rules of thumb

Do	Don't
NotifyContext at main	Catch signals deep in libraries
Propagate ctx to HTTP/exec	Ignore cancel in tight loops
Map cancel to clear message	Hang forever after Ctrl-C
Check write errors	Assume stdout always works

## Try next

1. `sleep 30` under your CLI with `ctx`; Ctrl-C and confirm quick exit.
2. Download a large file; cancel mid-way; ensure partial file is removed or kept intentionally.
3. Unit-test `run` with a pre-canceled context.

## Testing CLI Tools

# Testing CLI Tools

## Overview

Test CLIs as pure functions of args and streams. Avoid `os.Exit` and global `flag.CommandLine` in the unit under test. Use `FlagSet` with `ContinueOnError`, `bytes.Buffer`, and table tests.

## Golden structure

```
// cmd/mytool/main.go
func main() {
 os.Exit(realMain(os.Args[1:], os.Stdin, os.Stdout, os.Stderr))
}

func realMain(args []string, in io.Reader, out, errOut io.Writer) int {
 if err := run(args, in, out, errOut); err != nil {
 fmt.Fprintln(errOut, "mytool:", err)
 if errors.Is(err, errUsage) {
 return 2
 }
 return 1
 }
 return 0
}

func TestRunHello(t *testing.T) {
 var out bytes.Buffer
 code := realMain([]string{"-name", "Ada"}, strings.NewReader(""), &out, io.Discard)
 if code != 0 {
 t.Fatalf("code=%d", code)
 }
 if got := out.String(); got != "hello, Ada\n" {
 t.Fatalf("got %q", got)
 }
}
```

## FlagSet in run()

```
func run(args []string, in io.Reader, out, errOut io.Writer) error {
 fs := flag.NewFlagSet("mytool", flag.ContinueOnError)
 fs.SetOutput(errOut)
 name := fs.String("name", "world", "name")
 if err := fs.Parse(args); err != nil {
 return errUsage
 }
 _, err := fmt.Fprintf(out, "hello, %s\n", *name)
 return err
}
```

Never call `flag.Parse()` on the global set inside library tests—it races and mutates process state.

## Table-driven cases

```
func TestCLI(t *testing.T) {
 tests := []struct {
 name string
 args []string
 in string
 code int
 want string
 }{
 {"ok", []string{"-name", "Zo"}, "", 0, "hello, Zo\n"},
 {"default", nil, "", 0, "hello, world\n"},
 {"badflag", []string{"-nope"}, "", 2, ""},
 }
 for _, tt := range tests {
 t.Run(tt.name, func(t *testing.T) {
 var out, errB bytes.Buffer
 code := realMain(tt.args, strings.NewReader(tt.in), &out, &errB)
 if code != tt.code {
 t.Fatalf("code=%d err=%s", code, errB.String())
 }
 if tt.want != "" && out.String() != tt.want {
 t.Fatalf("out=%q want %q", out.String(), tt.want)
 }
 })
 }
}
```

## Temp files and env

```
func TestWithConfig(t *testing.T) {
 dir := t.TempDir()
 path := filepath.Join(dir, "c.json")
 if err := os.WriteFile(path, []byte(`{"verbose":true}`), 0o600); err != nil {
 t.Fatal(err)
 }
 t.Setenv("MYTOOL_API_URL", "http://example")
 // call run with --config path
}
```

## Subprocess tests (optional)

When you must test main packaging:

```
func TestMainBinary(t *testing.T) {
 if os.Getenv("BE_CRASHER") == "1" {
 main()
 return
 }
 cmd := exec.Command(os.Args[0], "-test.run=TestMainBinary")
 cmd.Env = append(os.Environ(), "BE_CRASHER=1")
 err := cmd.Run()
 // assert exit code via ExitError
}
```

Prefer in-process tests; subprocess tests are slower and messier.

## Golden files

```
func TestOutputGolden(t *testing.T) {
 var out bytes.Buffer
 _ = run([]string{"list"}, nil, &out, io.Discard)
 golden := filepath.Join("testdata", "list.golden")
 if os.Getenv("UPDATE_GOLDEN") != "" {
 _ = os.WriteFile(golden, out.Bytes(), 0o644)
 }
 want, err := os.ReadFile(golden)
 if err != nil {
 t.Fatal(err)
 }
 if !bytes.Equal(out.Bytes(), want) {
```

```

 t.Fatalf("mismatch")
 }
}

```

## Race and parallel

```

func TestX(t *testing.T) {
 t.Parallel()
 // only if run uses no process-global state
}

```

## Rules of thumb

Do	Don't
Inject Reader/Writer	Read/write only <code>os.Stdout</code> in core logic
Return exit codes from <code>realMain</code>	<code>os.Exit</code> inside tests
Private <code>FlagSet</code>	Global <code>flag.Parse</code> in packages
<code>t.Setenv</code> / <code>t.TempDir</code>	Touch real <code>\$HOME</code> config

## Try next

1. Refactor any sample CLI to `realMain` and add three table cases.
2. Add a usage test that expects code 2 and stderr containing "usage".
3. Golden-test JSON output with `UPDATE_GOLDEN=1 go test`.

# Structuring Larger CLI Apps

# Structuring Larger CLI Apps

## Overview

Small tools fit in `main.go`. Larger CLIs need packages so commands, domain logic, and I/O stay testable. This chapter shows a stdlib-friendly layout that also migrates cleanly to Cobra.

## Recommended layout

```
mytool/
 go.mod
 cmd/
 mytool/
 main.go # os.Exit(realMain(...)) only
 internal/
 app/
 app.go # App struct, Run(args)
 root.go # dispatch
 cmd/
 get.go
 set.go
 list.go
 config/
 config.go
 domain/
 store.go # business logic
 scripts/
 README.md
```

`internal/` keeps API private to the module.

## App struct

```
package app

type App struct {
 Stdin io.Reader
```



```

Stdout io.Writer
Stderr io.Writer
// optional: Version string, FS for embed, HTTP client
}

func New() *App {
 return &App{
 Stdin: os.Stdin,
 Stdout: os.Stdout,
 Stderr: os.Stderr,
 }
}

func (a *App) Run(args []string) error {
 if len(args) < 1 {
 return a.usage()
 }
 switch args[0] {
 case "get":
 return a.cmdGet(args[1:])
 case "set":
 return a.cmdSet(args[1:])
 case "version":
 fmt.Fprintln(a.Stdout, version)
 return nil
 default:
 return fmt.Errorf("unknown command %q", args[0])
 }
}

// cmd/mytool/main.go
func main() {
 a := app.New()
 if err := a.Run(os.Args[1:]); err != nil {
 fmt.Fprintln(os.Stderr, err)
 os.Exit(1)
 }
}

```

## Dependency injection for domain

```
type Store interface {
 Get(ctx context.Context, key string) (string, error)
 Set(ctx context.Context, key, val string) error
}

type App struct {
 Store Store
 Stdout io.Writer
 // ...
}

func (a *App) cmdGet(args []string) error {
 // parse flags...
 val, err := a.Store.Get(context.Background(), key)
 // write to a.Stdout
 return err
}
```

Tests inject a memory store.

## Cobra migration path

When you adopt Cobra later:

```
internal/app → cobra root command constructor
internal/cmd → cobra subcommands calling domain
domain/ → unchanged
```

Keep **domain** free of cobra/flag types.

## Version and commit via ldflags

```
// internal/buildinfo/buildinfo.go
var (
 Version = "dev"
 Commit = "none"
 Date = "unknown"
)
```

```
go build -ldflags "-X example.com/mytool/internal/buildinfo.Version=1.2.3 \
-X example.com/mytool/internal/buildinfo.Commit=$(git rev-parse --short HEAD)" \
-o mytool ./cmd/mytool
```

## Multi-binary monorepo

```
cmd/
 mytool/
 mytool-helper/
```

Share `internal/` packages; thin mains.

## Example: split packages sketch

```
// internal/cmd/get.go
package cmd

func Get(stdout io.Writer, store Store, args []string) error {
 fs := flag.NewFlagSet("get", flag.ContinueOnError)
 // ...
}

// internal/app/root.go
case "get":
 return cmd.Get(a.Stdout, a.Store, args[1:])
```

## Config package

```
package config

type Config struct {
 Path string
 // ...
}

func Load(path string) (Config, error) { /* json */ }
```

Main/app resolve path; commands receive `Config` values, not global state.

## Rules of thumb

Do	Don't
<code>cmd/</code> + <code>internal/</code>	Everything in <code>package main</code> forever
Inject <code>Store/IO</code>	Global <code>var db *sql.DB</code> in all files
Keep domain pure	Import <code>flag</code> into domain validators
One binary entry per UX	Hidden side-effect <code>init()</code> parsers

## Try next

1. Split the kv CLI into `cmd/mytool` + `internal/app`.
2. Add `version` subcommand from `ldflags`.
3. Test `App.Run` with a fake `Store`.

## Cobra Basics

# Cobra Basics

## Overview

[Cobra](#) is the most common Go library for rich CLIs (kubectl, hugo, gh patterns). It gives you **command trees**, **POSIX flags** (via pflag), **help generation**, and hooks for completion.

Use Cobra when stdlib `FlagSet` dispatch becomes repetitive—not for every 50-line script.

## Install

```
mkdir mytool && cd mytool
go mod init example.com/mytool
go get github.com/spf13/cobra@latest
optional generator:
go install github.com/spf13/cobra-cli@latest
```

## Minimal root + subcommand

```
package main

import (
 "fmt"
 "os"

 "github.com/spf13/cobra"
)

func main() {
 if err := rootCmd.Execute(); err != nil {
 os.Exit(1)
 }
}

var rootCmd = &cobra.Command{
 Use: "greet",
 Short: "A friendly greeter",
}
```

```

 Long: "greet prints a greeting with optional flair.",
 // Run when user types bare `greet` (optional)
 RunE: func(cmd *cobra.Command, args []string) error {
 return cmd.Help()
 },
}

var helloCmd = &cobra.Command{
 Use: "hello [name]",
 Short: "Say hello",
 Args: cobra.MaximumNArgs(1),
 RunE: func(cmd *cobra.Command, args []string) error {
 name := "world"
 if len(args) == 1 {
 name = args[0]
 }
 times, _ := cmd.Flags().GetInt("times")
 for i := 0; i < times; i++ {
 fmt.Printf("hello, %s\n", name)
 }
 return nil
 },
}

func init() {
 helloCmd.Flags().IntP("times", "n", 1, "repeat count")
 rootCmd.AddCommand(helloCmd)
}

go run . hello Ada -n 2
go run . hello --help

```

## Run vs RunE

Field	Behavior
Run	func(cmd, args) — errors must be handled inside
RunE	returns error — Cobra prints and sets exit code

Prefer **RunE** for testable, consistent errors.

## Args validators

```
Args: cobra.ExactArgs(1),
Args: cobra.MinimumNArgs(1),
Args: cobra.NoArgs,
Args: cobra.OnlyValidArgs, // with ValidArgs
```

Custom:

```
Args: func(cmd *cobra.Command, args []string) error {
 if len(args) < 1 {
 return fmt.Errorf("requires a key")
 }
 return nil
},
```

## Local flags

```
// only on this command
helloCmd.Flags().String("lang", "en", "language")
```

## Project layout with Cobra

```
cmd/
 root.go // rootCmd, Execute()
 hello.go // helloCmd + init AddCommand
 version.go
main.go // cmd.Execute()
```

```
// main.go
package main
```

```
import "example.com/mytool/cmd"
```

```
func main() {
 cmd.Execute()
}
```

```
// cmd/root.go
package cmd
```



```

import (
 "os"
 "github.com/spf13/cobra"
)

var rootCmd = &cobra.Command{Use: "mytool", Short: "demo"}

func Execute() {
 if err := rootCmd.Execute(); err != nil {
 os.Exit(1)
 }
}

```

## Example: kv with Cobra

```

var getCmd = &cobra.Command{
 Use: "get KEY",
 Short: "Get a value",
 Args: cobra.ExactArgs(1),
 RunE: func(cmd *cobra.Command, args []string) error {
 raw, _ := cmd.Flags().GetBool("raw")
 val, ok := store[args[0]]
 if !ok {
 return fmt.Errorf("not found")
 }
 if raw {
 fmt.Print(val)
 return nil
 }
 fmt.Printf("%s=%s\n", args[0], val)
 return nil
 },
}

func init() {
 getCmd.Flags().Bool("raw", false, "raw value only")
 rootCmd.AddCommand(getCmd)
}

```

## Silencing usage on errors

```
rootCmd.SilenceUsage = true // don't dump full help on every runtime error
rootCmd.SilenceErrors = true // handle printing yourself
```

Useful once commands stabilize—usage on every “not found” is noisy.

## Mapping to stdlib skills

Stdlib	Cobra
<code>switch os.Args[1]</code>	<code>AddCommand tree</code>
<code>flag.FlagSet</code>	<code>cmd.Flags()</code>
<code>flag.Args()</code>	args in <code>RunE</code>
manual help text	auto <code>--help</code>

## Rules of thumb

Do	Don't
Keep domain out of <code>cmd</code> package	Put SQL in <code>RunE</code> closures forever
Use <code>RunE</code>	Ignore errors in <code>Run</code>
One file per command group as you grow	2k-line <code>root.go</code>

## Try next

1. Port the stdlib `greet` tool to Cobra with `-n`.
2. Add `version` subcommand.
3. Set `SilenceUsage` and compare error UX.

## **Cobra Advanced: Persistent Flags, Hooks, Completion**

# Cobra Advanced: Persistent Flags, Hooks, Completion

## Overview

After basics: **persistent flags** (inherited by children), **PreRun/PostRun hooks**, **required flags**, **groups**, and **shell completion**. These are the features that justify Cobra over hand-rolled FlagSets.

## Persistent flags

```
var (
 cfgFile string
 verbose bool
)

func init() {
 rootCmd.PersistentFlags().StringVar(&cfgFile, "config", "", "config file")
 rootCmd.PersistentFlags().BoolVarP(&verbose, "verbose", "v", false, "verbose")
 // children see --config and -v without redefining
}
```

```
mytool --verbose get key
mytool get key --verbose # also works for persistent flags
```

Local flags stay on one command:

```
getCmd.Flags().Bool("raw", false, "raw")
```

## Required flags

```
getCmd.Flags().String("ns", "", "namespace")
_ = getCmd.MarkFlagRequired("ns")
```

## PreRun / PostRun chain

Order for a subcommand:

```
PersistentPreRun → PreRun → Run → PostRun → PersistentPostRun
```

```
rootCmd.PersistentPreRunE = func(cmd *cobra.Command, args []string) error {
 return loadConfig(cfgFile)
}

getCmd.PreRunE = func(cmd *cobra.Command, args []string) error {
 if verbose {
 fmt.Fprintln(os.Stderr, "get", args)
 }
 return nil
}
```

Use PreRun for auth checks, config validation, and client construction.

## Bind flags to structs cleanly

```
type GetOpts struct {
 Raw bool
 NS string
}

func newGetCmd(store Store) *cobra.Command {
 var opts GetOpts
 cmd := &cobra.Command{
 Use: "get KEY",
 Args: cobra.ExactArgs(1),
 RunE: func(cmd *cobra.Command, args []string) error {
 return store.Get(cmd.Context(), args[0], opts)
 },
 }
 cmd.Flags().BoolVar(&opts.Raw, "raw", false, "raw")
 cmd.Flags().StringVar(&opts.NS, "ns", "default", "namespace")
 return cmd
}
```

Factory functions make testing easier than package-level var.

## Context on commands

```
// Cobra sets cmd.Context() from ExecuteContext
func Execute() error {
 ctx, stop := signal.NotifyContext(context.Background(), os.Interrupt)
 defer stop()
 return rootCmd.ExecuteContext(ctx)
}

// in RunE:
req, _ := http.NewRequestWithContext(cmd.Context(), ...)
```

## Command groups (Cobra 1.6+)

```
rootCmd.AddGroup(&cobra.Group{ID: "core", Title: "Core Commands"})
getCmd.GroupID = "core"
rootCmd.AddCommand(getCmd)
```

Help output clusters commands by group.

## Custom usage / version templates

```
rootCmd.SetVersionTemplate(`{{printf "mytool %s\n" .Version}}`)
rootCmd.Version = buildinfo.Version
// enables --version on root when Version is non-empty
```

## Shell completion

```
// cmd/completion.go
var completionCmd = &cobra.Command{
 Use: "completion [bash|zsh|fish|powershell]",
 Short: "Generate completion script",
 Args: cobra.ExactArgs(1),
 RunE: func(cmd *cobra.Command, args []string) error {
 switch args[0] {
 case "bash":
 return cmd.Root().GenBashCompletion(os.Stdout)
 case "zsh":
 return cmd.Root().GenZshCompletion(os.Stdout)
 case "fish":

```

```

 return cmd.Root().GenFishCompletion(os.Stdout, true)
 case "powershell":
 return cmd.Root().GenPowerShellCompletionWithDesc(os.Stdout)
 default:
 return fmt.Errorf("unsupported shell %q", args[0])
 }
},
}

```

Dynamic args:

```

getCmd.ValidArgsFunction = func(cmd *cobra.Command, args []string, toComplete string) ([]string, cobra.ShellCompDirective) {
 keys := store.KeysWithPrefix(toComplete)
 return keys, cobra.ShellCompDirectiveNoFileComp
}

```

## Traverse children for help

```

rootCmd.SetHelpCommand(helpCmd) // optional customize

```

## Example: parent remote with children

```

remoteCmd := &cobra.Command{Use: "remote", Short: "Manage remotes"}
remoteAddCmd := &cobra.Command{
 Use: "add NAME URL",
 Args: cobra.ExactArgs(2),
 RunE: func(cmd *cobra.Command, args []string) error {
 return remotes.Add(args[0], args[1])
 },
}
remoteCmd.AddCommand(remoteAddCmd)
rootCmd.AddCommand(remoteCmd)
// mytool remote add origin https://...

```

## Testing Cobra commands

```

func TestHello(t *testing.T) {
 cmd := newHelloCmd()
 b := bytes.NewBufferString("")
 cmd.SetOut(b)
}

```

```

cmd.SetErr(io.Discard)
cmd.SetArgs([]string{"-n", "2", "Ada"})
if err := cmd.Execute(); err != nil {
 t.Fatal(err)
}
// assert b.String()
}

```

Prefer constructing commands via factories rather than executing package-global `rootCmd` (mutable).

## Rules of thumb

Do	Don't
Persistent flags for globals	Copy the same flags on 20 commands
<code>ExecuteContext</code> + signals	Ignore cancel in long <code>RunE</code>
Factories for commands	Global mutable option vars only
Completion for public CLIs	Skip docs for install paths

## Try next

1. Add `--config` persistent flag and load JSON in `PersistentPreRunE`.
2. Mark a flag required; confirm error UX.
3. Generate `zsh` completion and install to your `fpath`.



## Viper and Config Layering

# Viper and Config Layering

## Overview

[Viper](#) is often paired with Cobra for **layered config**: files, env, flags, defaults. It is powerful and also easy to overuse. Prefer the stdlib merge from chapter 305 until you feel real pain—then Viper (or a thinner alternative) earns its dependency.

## When Viper helps

Helps	Overkill
Many file formats (YAML/TOML/JSON)	Single JSON + few flags
Automatic env binding	Three getenv calls
Live watch (rare for CLIs)	One-shot tools
Large apps with many keys	Tiny scripts

## Install

```
go get github.com/spf13/viper@latest
```

## Minimal Viper + Cobra

```
import (
 "strings"
 "github.com/spf13/cobra"
 "github.com/spf13/viper"
)

var cfgFile string

var rootCmd = &cobra.Command{
 Use: "mytool",
 PersistentPreRunE: func(cmd *cobra.Command, args []string) error {
 return initConfig()
 }
}
```



## Precedence (Viper)

Typical order (highest last wins when binding flags correctly):

```
defaults → config file → env → flags
```

Confirm with your version's docs; always document for users.

## Example config.yaml

```
api_url: https://api.example.com
timeout: 15s
verbose: true
```

## Unmarshal into struct

```
type Config struct {
 APIURL string `mapstructure:"api_url"`
 Timeout string `mapstructure:"timeout"`
 Verbose bool `mapstructure:"verbose"`
}

var c Config
if err := viper.Unmarshal(&c); err != nil {
 return err
}
```

## Bind env explicitly

```
_ = viper.BindEnv("api_url", "MYTOOL_API_URL")
```

Useful when AutomaticEnv naming is awkward.

## Pitfalls

1. **Global viper** — the package-level instance is convenient and hard to test. Prefer `viper.New()` per app in larger codebases.
2. **Type confusion** — timeouts as strings vs durations; parse explicitly.
3. **Flag default vs viper default** — set one source of truth.
4. **Sensitive values** — don't dump full viper settings into logs.

## Isolated Viper for tests

```
func newViper() *viper.Viper {
 v := viper.New()
 v.SetDefault("api_url", "http://127.0.0.1:8080")
 return v
}
```

Pass `*viper.Viper` on your App struct instead of using globals.

## Lighter alternatives

Approach	Notes
Stdlib JSON + env (ch 305)	Zero dep, enough for most tools
caarlos0/env	Struct tags for env only
knadh/koanf	Modular, explicit providers
pelletier/go-toml / yaml.v3	Parse files yourself

## Stdlib-first checklist before Viper

- ☐ 15 config keys
- ☐ JSON or single format is fine
- ☐ Flags + env + one file path cover deploys

If all true, stay stdlib.

## Rules of thumb

Do	Don't
Document precedence	Surprise users with silent file overrides
<code>viper.New()</code> when testing	Global state across parallel tests
Redact secrets in debug dumps	Log entire config maps
Fail on corrupt files	Ignore parse errors

## Try next

1. Bind `--api-url` to viper and override with `MYTOOL_API_URL`.
2. Load YAML from `UserConfigDir`.
3. Refactor to `viper.New()` injected into commands.

## **Other CLI Libraries: urfave/cli, pflag, and Friends**

# Other CLI Libraries: `urfave/cli`, `pflag`, and Friends

## Overview

Cobra is not the only option. This chapter surveys common libraries so you can choose deliberately—and recognize them in the wild.

## Comparison matrix

Library	Style	Best for
<code>stdlib flag</code>	minimal	small tools, zero deps
<code>spf13/pflag</code>	POSIX/GNU flags	drop-in richer flags without full Cobra
<code>spf13/cobra</code>	command tree	large public CLIs
<code>urfave/cli</code>	declarative apps	compact multi-command tools
<code>alecthomas/kong</code>	struct-tagged	strongly typed CLIs
<code>charmbracelet/</code>	TUI	interactive terminals (not pure pipes)

## `pflag` alone

Cobra uses `pflag`. You can use `pflag` without Cobra:

```
import flag "github.com/spf13/pflag"

var (
 name = flag.StringP("name", "n", "world", "name")
 verbose = flag.BoolP("verbose", "v", false, "verbose")
)

flag.Parse()
```

Differences from `stdlib flag`:

- Short + long flags (`-n` / `--name`)
- Combined shorts (`-abc`) for bools
- Slightly different parsing rules (more GNU-like)

```
go get github.com/spf13/pflag@latest
```

## urfave/cli v2/v3 sketch

```
package main

import (
 "fmt"
 "os"

 "github.com/urfave/cli/v2"
)

func main() {
 app := &cli.App{
 Name: "greet",
 Usage: "fight the loneliness!",
 Commands: []*cli.Command{
 {
 Name: "hello",
 Usage: "say hello",
 Flags: []cli.Flag{
 &cli.IntFlag{Name: "times", Aliases: []string{"n"}, Value: 1},
 },
 Action: func(c *cli.Context) error {
 name := "world"
 if c.NArg() > 0 {
 name = c.Args().Get(0)
 }
 for i := 0; i < c.Int("times"); i++ {
 fmt.Printf("hello, %s\n", name)
 }
 return nil
 },
 },
 },
 }

 if err := app.Run(os.Args); err != nil {
 fmt.Fprintln(os.Stderr, err)
 os.Exit(1)
 }
}
```

**Feel:** more “app struct” oriented; less boilerplate for simple trees.



## Kong (struct-first)

```
var cli struct {
 Debug bool `help:"Enable debug mode"`
 Hello struct {
 Name string `arg:"" default:"world"`
 Times int `help:"Repeat" short:"n" default:"1"`
 } `cmd:"" help:"Say hello"`
}

// ctx := kong.Parse(&cli)
// switch ctx.Command() { case "hello": ... }
```

Excellent when you want flags as typed fields; different aesthetics from Cobra.

## Charm ecosystem (when CLI becomes TUI)

Package	Role
bubbletea	Elm-style TUI framework
lipgloss	styling
huh	forms
bubbletea apps	interactive selectors, dashboards

**Rule:** keep pipe-friendly commands as non-TUI; offer a separate `mytool tui` if needed. Don't break `jq` pipelines with alternate screens.

## mow.cli, kingpin, ff

Older or niche libs still appear in codebases:

- [jawher/mow.cli](#) — fluent API
- [alecthomas/kingpin](#) — predecessor ideas to kong
- [peterbourgon/ff](#) — flags + env + file in a small package

Read their READMEs before adopting; Cobra/urfave cover most new work.

## Choosing

```
zero deps, few flags → stdlib flag
need -v/--verbose only → pflag
nested commands, public → cobra
compact declarative → urfave/cli
struct tags / typed → kong
full screen UI → charm/bubbletea
```

## Hybrid pattern

Many teams:

1. Domain in pure Go packages
2. cmd layer uses Cobra **or** urfave
3. Stdlib tests against domain + thin command factories

Switching frameworks then costs little.

## Rules of thumb

Do	Don't
Pick one framework per binary	Mix Cobra + urfave in one tree
Match ecosystem (k8s → Cobra familiarity)	Add Cobra “because tutorials do”
Keep TUI optional	Force interactive UI for CI tools

## Try next

1. Reimplement `greet hello -n` with pflag only.
2. Skim kubectrl or gh command layout on GitHub (Cobra patterns).
3. Port one cookbook tool (ch 316) to urfave/cli and compare LOC.

**Shipping: Version, Install, Completion, Release**

# Shipping: Version, Install, Completion, Release

## Overview

A CLI users trust has a clear **version**, easy **install**, optional **completion**, and reproducible **releases**. Most of this works with stdlib + Go toolchain; GoReleaser and package managers finish the last mile.

## Version command / flag

```
var (
 version = "dev"
 commit = "none"
 date = "unknown"
)

// go build -ldflags "-X main.version=1.0.0 -X main.commit=abc -X main.date=2026-01-01"

var versionCmd = &cobra.Command{
 Use: "version",
 Short: "Print version",
 Run: func(cmd *cobra.Command, args []string) {
 fmt.Printf("mytool %s (%s) built %s\n", version, commit, date)
 },
}
```

Also support `mytool --version` via `Cobra rootCmd.Version`.

## From build info (no ldflags)

```
import "runtime/debug"

func versionString() string {
 if bi, ok := debug.ReadBuildInfo(); ok {
 return bi.Main.Version // often "(devel)" for go run
 }
 return "dev"
}
```

```
}
```

Use ldflags for release binaries; buildinfo helps modules.

## Install paths

```
module install
go install example.com/mytool/cmd/mytool@v1.2.3

local
go build -o dist/mytool ./cmd/mytool
sudo cp dist/mytool /usr/local/bin/
```

Document GOBIN / PATH issues on macOS/Linux.

## Man pages and docs

- Cobra: community tools generate man/markdown from command tree
- At minimum: `mytool --help` and README examples with copy-paste commands

## Completions (install notes)

```
zsh example
mytool completion zsh > "${fpath[1]}/_mytool"
or
mytool completion zsh > ~/.zsh/completions/_mytool
compinit

bash
mytool completion bash | sudo tee /etc/bash_completion.d/mytool
```

Document one path per shell; link to Cobra chapter 312.

## Cross-compile

```
GOOS=linux GOARCH=amd64 go build -o mytool-linux-amd64 ./cmd/mytool
GOOS=darwin GOARCH=arm64 go build -o mytool-darwin-arm64 ./cmd/mytool
GOOS=windows GOARCH=amd64 go build -o mytool-windows-amd64.exe ./cmd/mytool
```

CGO\_ENABLED=0 for static-ish pure Go binaries (usual for CLIs). On Linux that means **no libc** on the target: the same file runs on Debian, Alpine, and NixOS. Leave cgo on and a Ubuntu-built CLI can fail on Alpine with `not found` because the glibc dynamic linker is missing. See [Go as a Systems Language](#).

## Makefile sketch

```
VERSION ?= $(shell git describe --tags --always --dirty)
LDLFLAGS := -s -w -X main.version=$(VERSION)

.PHONY: build test install
build:
 go build -ldflags "$$(LDLFLAGS)" -o bin/mytool ./cmd/mytool

test:
 go test ./...

install: build
 install -m 755 bin/mytool $(HOME)/bin/mytool
```

## GoReleaser (overview)

[GoReleaser](#) automates multi-arch builds, checksums, changelog, GitHub releases, Homebrew taps, etc.

```
.goreleaser.yaml (sketch)
builds:
 - main: ./cmd/mytool
 ldflags:
 - -s -w -X main.version={{.Version}} -X main.commit={{.Commit}}
 env: [CGO_ENABLED=0]
 goos: [linux, darwin, windows]
 goarch: [amd64, arm64]

tag and release in CI
git tag v1.2.3 && git push --tags
```

## Docker (optional for CLIs)

Often unnecessary for pure Go CLIs; useful for hermetic CI:

```
FROM golang:1.22 AS build
WORKDIR /src
COPY . .
RUN CGO_ENABLED=0 go build -o /out/mytool ./cmd/mytool
FROM gcr.io/distroless/static
COPY --from=build /out/mytool /mytool
ENTRYPOINT ["/mytool"]
```

## Checksums and supply chain

```
sha256sum mytool-* > checksums.txt
cosign / slsa later for high-security tools
```

Publish checksums next to release assets.

## Rules of thumb

Do	Don't
Embed version in binary	Rely on filename alone
Provide install one-liners	Assume <code>go install</code> only
Ship completion for public tools	Require manual flag memorization
CGO_ENABLED=0 for CLIs	Accidental <code>cgo</code> → fragile cross-build

## Try next

1. Add ldflags version to a sample tool; verify `mytool version`.
2. Cross-compile for linux/amd64 from your Mac/Linux host.
3. Generate completion and smoke-test tab complete for a subcommand.

## CLI Cookbook: Stdlib and Cobra Examples



# CLI Cookbook: Stdlib and Cobra Examples

## Overview

Runnable sketches you can paste into a module. §A–§F are stdlib-only. §G–§H use Cobra. Trim imports and split packages as needed.

```
mkdir -p /tmp/cookbook && cd /tmp/cookbook
go mod init example.com/cookbook
```

---

### A. hello — flags + exit codes (stdlib)

```
package main

import (
 "errors"
 "flag"
 "fmt"
 "io"
 "os"
)

var errUsage = errors.New("usage")

func main() {
 os.Exit(realMain(os.Args[1:], os.Stdout, os.Stderr))
}

func realMain(args []string, stdout, stderr io.Writer) int {
 if err := run(args, stdout, stderr); err != nil {
 fmt.Fprintln(stderr, err)
 if errors.Is(err, errUsage) {
 return 2
 }
 return 1
 }
 return 0
}
```

```

}

func run(args []string, stdout, stderr io.Writer) error {
 fs := flag.NewFlagSet("hello", flag.ContinueOnError)
 fs.SetOutput(stderr)
 n := fs.Int("n", 1, "times")
 name := fs.String("name", "world", "name")
 if err := fs.Parse(args); err != nil {
 return errUsage
 }
 for i := 0; i < *n; i++ {
 fmt.Fprintf(stdout, "hello, %s\n", *name)
 }
 return nil
}

go run . -name Ada -n 2

```

---

## B. catn — files or stdin (stdlib)

```

package main

import (
 "flag"
 "fmt"
 "io"
 "os"
)

func main() {
 flag.Parse()
 args := flag.Args()
 if len(args) == 0 {
 if _, err := io.Copy(os.Stdout, os.Stdin); err != nil {
 fail(err)
 }
 return
 }
 for _, p := range args {
 if p == "-" {
 if _, err := io.Copy(os.Stdout, os.Stdin); err != nil {
 fail(err)
 }
 }
 }
}

```

```

 }
 continue
 }
 f, err := os.Open(p)
 if err != nil {
 fail(err)
 }
 _, err = io.Copy(os.Stdout, f)
 f.Close()
 if err != nil {
 fail(err)
 }
}

}

func fail(err error) {
 fmt.Fprintln(os.Stderr, "catn:", err)
 os.Exit(1)
}

echo hi | go run .
go run . /etc/hosts

```

---

## C. ffind — walk files by suffix (stdlib)

```

package main

import (
 "flag"
 "fmt"
 "io/fs"
 "os"
 "path/filepath"
 "strings"
)

func main() {
 root := flag.String("root", ".", "start directory")
 suffix := flag.String("ext", ".go", "file suffix")
 flag.Parse()

 err := filepath.WalkDir(*root, func(path string, d fs.DirEntry, err error) error {

```

```

 if err != nil {
 return err
 }
 if d.IsDir() {
 if d.Name() == ".git" || d.Name() == "node_modules" {
 return fs.SkipDir
 }
 return nil
 }
 if strings.HasSuffix(path, *suffix) {
 fmt.Println(path)
 }
 return nil
 })
 if err != nil {
 fmt.Fprintln(os.Stderr, err)
 os.Exit(1)
 }
}

```

## D. jq — extract JSON field from NDJSON stdin (stdlib)

```

package main

import (
 "encoding/json"
 "flag"
 "fmt"
 "os"
)

func main() {
 field := flag.String("f", "", "field name")
 flag.Parse()
 if *field == "" {
 fmt.Fprintln(os.Stderr, "usage: jq -f field < ndjson")
 os.Exit(2)
 }
 dec := json.NewDecoder(os.Stdin)
 enc := json.NewEncoder(os.Stdout)
 for {
 var m map[string]any
 if err := dec.Decode(&m); err != nil {

```

```

 break
 }
 if v, ok := m[*field]; ok {
 _ = enc.Encode(v)
 }
}
}

printf '%s\n' '{"user":"a"}' '{"user":"b"}' | go run . -f user

```

---

## E. httpc — GET with timeout (stdlib)

```

package main

import (
 "context"
 "flag"
 "fmt"
 "io"
 "net/http"
 "os"
 "time"
)

func main() {
 url := flag.String("url", "", "URL")
 timeout := flag.Duration("timeout", 10*time.Second, "timeout")
 flag.Parse()
 if *url == "" {
 fmt.Fprintln(os.Stderr, "usage: httpc -url URL")
 os.Exit(2)
 }
 ctx, cancel := context.WithTimeout(context.Background(), *timeout)
 defer cancel()
 req, err := http.NewRequestWithContext(ctx, http.MethodGet, *url, nil)
 if err != nil {
 fail(err)
 }
 resp, err := http.DefaultClient.Do(req)
 if err != nil {
 fail(err)
 }
}

```

```

 defer resp.Body.Close()
 fmt.Fprintln(os.Stderr, resp.Status)
 _, _ = io.Copy(os.Stdout, io.LimitReader(resp.Body, 2<<20))
}

func fail(err error) { fmt.Fprintln(os.Stderr, err); os.Exit(1) }

```

---

## F. kv — subcommands with FlagSet (stdlib)

```

package main

import (
 "encoding/json"
 "flag"
 "fmt"
 "os"
 "sync"
)

type DB struct {
 mu sync.Mutex
 Data map[string]string `json:"data"`
 path string
}

func openDB(path string) (*DB, error) {
 db := &DB{Data: map[string]string{}, path: path}
 b, err := os.ReadFile(path)
 if err != nil {
 if os.IsNotExist(err) {
 return db, nil
 }
 return nil, err
 }
 if err := json.Unmarshal(b, db); err != nil {
 return nil, err
 }
 if db.Data == nil {
 db.Data = map[string]string{}
 }
 return db, nil
}

```

```

func (db *DB) save() error {
 db.mu.Lock()
 defer db.mu.Unlock()
 b, err := json.MarshalIndent(db, "", " ")
 if err != nil {
 return err
 }
 return os.WriteFile(db.path, b, 0o600)
}

func main() {
 if len(os.Args) < 2 {
 fmt.Fprintln(os.Stderr, "usage: kv <get|set|list> ...")
 os.Exit(2)
 }
 path := os.Getenv("KV_DB")
 if path == "" {
 path = "kv.json"
 }
 db, err := openDB(path)
 if err != nil {
 fail(err)
 }
 switch os.Args[1] {
 case "get":
 err = cmdGet(db, os.Args[2:])
 case "set":
 err = cmdSet(db, os.Args[2:])
 case "list":
 err = cmdList(db, os.Args[2:])
 default:
 fmt.Fprintln(os.Stderr, "unknown command")
 os.Exit(2)
 }
 if err != nil {
 fail(err)
 }
}

func cmdGet(db *DB, args []string) error {
 fs := flag.NewFlagSet("get", flag.ContinueOnError)
 if err := fs.Parse(args); err != nil {
 return err
 }
 if fs.NArg() != 1 {
 return fmt.Errorf("usage: kv get KEY")
 }
}

```

```

 db.mu.Lock()
 v, ok := db.Data[fs.Arg(0)]
 db.mu.Unlock()
 if !ok {
 return fmt.Errorf("not found")
 }
 fmt.Println(v)
 return nil
}

func cmdSet(db *DB, args []string) error {
 fs := flag.NewFlagSet("set", flag.ContinueOnError)
 if err := fs.Parse(args); err != nil {
 return err
 }
 if fs.NArg() != 2 {
 return fmt.Errorf("usage: kv set KEY VALUE")
 }
 db.mu.Lock()
 db.Data[fs.Arg(0)] = fs.Arg(1)
 db.mu.Unlock()
 return db.save()
}

func cmdList(db *DB, args []string) error {
 db.mu.Lock()
 defer db.mu.Unlock()
 for k, v := range db.Data {
 fmt.Printf("%s\t%s\n", k, v)
 }
 return nil
}

func fail(err error) { fmt.Fprintln(os.Stderr, err); os.Exit(1) }

go run . set hello world
go run . get hello
go run . list

```

---



## G. Cobra todo — add / list (library)

```
go get github.com/spf13/cobra@latest

package main

import (
 "encoding/json"
 "fmt"
 "os"
 "time"

 "github.com/spf13/cobra"
)

type Task struct {
 Text string `json:"text"`
 At time.Time `json:"at"`
}

func load(path string) ([]Task, error) {
 b, err := os.ReadFile(path)
 if err != nil {
 if os.IsNotExist(err) {
 return nil, nil
 }
 return nil, err
 }
 var t []Task
 return t, json.Unmarshal(b, &t)
}

func save(path string, tasks []Task) error {
 b, err := json.MarshalIndent(tasks, "", " ")
 if err != nil {
 return err
 }
 return os.WriteFile(path, b, 0o600)
}

func main() {
 path := "tasks.json"
 root := &cobra.Command{Use: "todo", Short: "Tiny todo list"}

 add := &cobra.Command{
 Use: "add TEXT...",
 }
```

```

 Args: cobra.MinimumNArgs(1),
 RunE: func(cmd *cobra.Command, args []string) error {
 tasks, err := load(path)
 if err != nil {
 return err
 }
 text := args[0]
 for _, a := range args[1:] {
 text += " " + a
 }
 tasks = append(tasks, Task{Text: text, At: time.Now().UTC()})
 return save(path, tasks)
 },
}

list := &cobra.Command{
 Use: "list",
 RunE: func(cmd *cobra.Command, args []string) error {
 tasks, err := load(path)
 if err != nil {
 return err
 }
 for i, t := range tasks {
 fmt.Printf("%d.\t%s\t(%s)\n", i+1, t.Text, t.At.Format(time.RFC3339))
 }
 return nil
 },
}

root.AddCommand(add, list)
if err := root.Execute(); err != nil {
 os.Exit(1)
}
}

go run . add write docs
go run . list

```

---

## H. Cobra parent + persistent verbose

```
package main

import (
 "fmt"
 "os"

 "github.com/spf13/cobra"
)

func main() {
 var verbose bool
 root := &cobra.Command{Use: "tool"}
 root.PersistentFlags().BoolVarP(&verbose, "verbose", "v", false, "verbose")

 ping := &cobra.Command{
 Use: "ping",
 RunE: func(cmd *cobra.Command, args []string) error {
 if verbose {
 fmt.Fprintln(os.Stderr, "pinging...")
 }
 fmt.Println("pong")
 return nil
 },
 }
 root.AddCommand(ping)
 _ = root.Execute()
}

go run . -v ping
```

---

## I. Ideas to implement yourself

Tool	Skills
sha1dir	walk + crypto/sha256 + parallel with errgroup
portwait	dial loop + timeout + signals
envshow	filter <code>os.Environ</code> with regex
jwtlook	base64 decode payload (no verify) for debugging
modgraph	go mod graph via exec + format
tailf	poll file size + print deltas

Tool	Skills
<code>csv2json</code>	<code>encoding/csv</code> + json encoder
<code>watchrun</code>	poll mtime + exec command

## J. Full project: network diagnostics CLI

For a multi-command **ping** / **dns** / **scan** / **probe** toolkit with layout, stats, workers, and tests, follow the dedicated project chapter:

→ [317 Project: Network CLI Tools](#)

## K. Full project: network security mini-tools

For **minimal** security helpers (TLS cert, headers, banner, redirects, CIDR, cookies, hash, secret scan, rand, listen):

→ [318 Project: Simple Network Security Mini-Tools](#)

Use `stdlib` until the command tree aches—then port the same domain to Cobra.

---

## Cookbook checklist

- ☐ `stdout` data / `stderr` logs
- ☐ non-zero exit on failure
- ☐ `ContinueOnError` `FlagSet` or `Cobra RunE`
- ☐ context on `network/exec`
- ☐ at least one table test on `run` / `realMain`

When those hold, your CLI is ready to grow.

## **Project: Network CLI Tools (Ping, DNS, Scan, Probe)**

# Project: Network CLI Tools (Ping, DNS, Scan, Probe)

## Overview

Build **netkit**, a multi-command networking CLI using **stdlib only** (plus optional Cobra later). You implement tools people actually run while debugging connectivity:

Subcommand	What it does
<b>ping</b>	TCP connect RTT (works without root; not ICMP)
<b>dns</b>	Resolve A/AAAA (and optional reverse)
<b>scan</b>	Concurrent TCP port scan on a host
<b>probe</b>	HTTP(S) latency + status code check
<b>whoami</b>	Local addresses / outbound path sketch

This project ties together flags, subcommands, exit codes, timeouts, concurrency, and signals from earlier chapters.

**Related.** Single-tool lab in projects: [CLI Ping Tool](#). Deeper TCP: [131 TCP services](#). Stdlib sockets: [994 net dial/listen](#).

## Goals

1. Ship one binary with several network diagnostic verbs
2. Always use **timeouts** and **context cancel** (Ctrl-C)
3. Keep **data on stdout**, diagnostics on stderr
4. Exit **0** on success, **1** on failure, **2** on misuse
5. Structure code so probes are unit-testable without the network (interfaces + fakes)

## Scaffold

```
mkdir -p netkit/cmd/netkit netkit/internal/{app,netutil}
cd netkit
go mod init example.com/netkit

netkit/
cmd/netkit/main.go
internal/app/app.go # dispatch + shared flags
internal/app/ping.go
internal/app/dns.go
internal/app/scan.go
internal/app/probe.go
internal/netutil/dial.go # shared dial helpers
internal/netutil/stats.go
```

## main.go

```
package main

import (
 "context"
 "fmt"
 "os"
 "os/signal"
 "syscall"

 "example.com/netkit/internal/app"
)

func main() {
 ctx, stop := signal.NotifyContext(context.Background(), os.Interrupt, syscall.SIGTERM)
 defer stop()

 a := app.New(os.Stdout, os.Stderr)
 if err := a.Run(ctx, os.Args[1:]); err != nil {
 fmt.Fprintln(os.Stderr, "netkit:", err)
 code := 1
 if app.IsUsage(err) {
 code = 2
 }
 if ctx.Err() != nil {
 code = 130
 }
 }
}
```

```

 os.Exit(code)
 }
}

```

## app.go (dispatch)

```

package app

import (
 "context"
 "errors"
 "fmt"
 "io"
)

var ErrUsage = errors.New("usage")

func IsUsage(err error) bool { return errors.Is(err, ErrUsage) }

type App struct {
 Stdout io.Writer
 Stderr io.Writer
}

func New(stdout, stderr io.Writer) *App {
 return &App{Stdout: stdout, Stderr: stderr}
}

func (a *App) Run(ctx context.Context, args []string) error {
 if len(args) < 1 {
 return a.usage()
 }
 switch args[0] {
 case "ping":
 return a.cmdPing(ctx, args[1:])
 case "dns":
 return a.cmdDNS(ctx, args[1:])
 case "scan":
 return a.cmdScan(ctx, args[1:])
 case "probe":
 return a.cmdProbe(ctx, args[1:])
 case "whoami":
 return a.cmdWhoami(ctx, args[1:])
 case "help", "-h", "--help":
 return a.usage()
 }
}

```



```

 default:
 fmt.Fprintf(a.Stderr, "unknown command %q\n", args[0])
 return a.usage()
 }
}

func (a *App) usage() error {
 fmt.Fprintf(a.Stderr, `Usage: netkit <command> [flags]

Commands:
 ping TCP connect ping (RTT stats)
 dns DNS lookup (A/AAAA)
 scan TCP port scan
 probe HTTP(S) status + latency
 whoami local network identity

Run "netkit <command> -h" for command flags.
`)
 return ErrUsage
}

```

---

## Tool 1: ping — TCP connect RTT

Classic ICMP ping needs raw sockets (often root). **TCP ping** dials a port (default 443) and measures connect time—enough for “is the service reachable?” diagnostics.

### netutil helpers

```

package netutil

import (
 "context"
 "net"
 "time"
)

type ProbeResult struct {
 OK bool
 RTT time.Duration
 Err error
}

```

```

func TCPping(ctx context.Context, address string, timeout time.Duration) ProbeResult {
 d := net.Dialer{Timeout: timeout}
 start := time.Now()
 conn, err := d.DialContext(ctx, "tcp", address)
 if err != nil {
 return ProbeResult{OK: false, Err: err}
 }
 _ = conn.Close()
 return ProbeResult{OK: true, RTT: time.Since(start)}
}

type Stats struct {
 Sent, Recv int
 Min, Max, Sum time.Duration
}

func (s *Stats) Add(r ProbeResult) {
 s.Sent++
 if !r.OK {
 return
 }
 s.Recv++
 if s.Recv == 1 || r.RTT < s.Min {
 s.Min = r.RTT
 }
 if r.RTT > s.Max {
 s.Max = r.RTT
 }
 s.Sum += r.RTT
}

func (s Stats) LossPct() float64 {
 if s.Sent == 0 {
 return 0
 }
 return float64(s.Sent-s.Recv) / float64(s.Sent) * 100
}

func (s Stats) Avg() time.Duration {
 if s.Recv == 0 {
 return 0
 }
 return s.Sum / time.Duration(s.Recv)
}

```

## cmdPing

```
package app

import (
 "context"
 "flag"
 "fmt"
 "net"
 "time"

 "example.com/netkit/internal/netutil"
)

func (a *App) cmdPing(ctx context.Context, args []string) error {
 fs := flag.NewFlagSet("ping", flag.ContinueOnError)
 fs.SetOutput(a.Stderr)
 count := fs.Int("c", 4, "number of probes")
 interval := fs.Duration("i", time.Second, "interval between probes")
 timeout := fs.Duration("t", 2*time.Second, "per-probe timeout")
 port := fs.String("p", "443", "TCP port")
 jsonOut := fs.Bool("json", false, "machine-readable summary on stdout")
 if err := fs.Parse(args); err != nil {
 return ErrUsage
 }
 if fs.NArg() != 1 {
 fmt.Fprintln(a.Stderr, "usage: netkit ping [flags] <host>")
 fs.PrintDefaults()
 return ErrUsage
 }
 host := fs.Arg(0)

 ips, err := net.DefaultResolver.LookupIPAddr(ctx, host)
 if err != nil || len(ips) == 0 {
 return fmt.Errorf("resolve %s: %w", host, err)
 }
 ip := ips[0].IP.String()
 addr := net.JoinHostPort(ip, *port)

 fmt.Fprintf(a.Stderr, "PING %s (%s) TCP/%s\n", host, ip, *port)

 var st netutil.Stats
 samples := make([]time.Duration, 0, *count)

 for i := 1; i <= *count; i++ {
 if err := ctx.Err(); err != nil {
 return err
 }
 }
}
```

```

 }
 r := netutil.TCPPing(ctx, addr, *timeout)
 st.Add(r)
 if r.OK {
 samples = append(samples, r.RTT)
 fmt.Fprintf(a.Stdout, "%d: open rtt=%v\n", i, r.RTT)
 } else {
 fmt.Fprintf(a.Stdout, "%d: fail err=%v\n", i, r.Err)
 }
 if i < *count {
 select {
 case <-ctx.Done():
 return ctx.Err()
 case <-time.After(*interval):
 }
 }
}

fmt.Fprintf(a.Stderr, "--- %s ping stats ---\n", host)
fmt.Fprintf(a.Stderr, "%d sent, %d ok, %.1f%% loss\n", st.Sent, st.Recv, st.LossPct())
fmt.Fprintf(a.Stderr, "rtt min/avg/max = %v/%v/%v\n", st.Min, st.Avg(), st.Max)

if *jsonOut {
 // summary JSON for scripts (optional: encode struct)
 fmt.Fprintf(a.Stdout, `{"host":%q,"ip":%q,"port":%q,"sent":%d,"ok":%d,"loss_pct":%.2
 host, ip, *port, st.Sent, st.Recv, st.LossPct(), st.Min, st.Avg(), st.Max)
}

if st.Recv == 0 {
 return fmt.Errorf("all probes failed")
}
_ = samples
return nil
}

```

```

go run ./cmd/netkit ping example.com
go run ./cmd/netkit ping -c 5 -p 80 -i 200ms example.com

```

**Stretch:** probe every resolved IP; compute jitter (stddev of samples).

## Tool 2: dns — lookup

```
func (a *App) cmdDNS(ctx context.Context, args []string) error {
 fs := flag.NewFlagSet("dns", flag.ContinueOnError)
 fs.SetOutput(a.Stderr)
 typ := fs.String("type", "ip", "ip|host (reverse)")
 if err := fs.Parse(args); err != nil {
 return ErrUsage
 }
 if fs.NArg() != 1 {
 fmt.Fprintln(a.Stderr, "usage: netkit dns [-type ip|host] <name-or-ip>")
 return ErrUsage
 }
 name := fs.Arg(0)

 switch *typ {
 case "ip":
 ips, err := net.DefaultResolver.LookupIPAddr(ctx, name)
 if err != nil {
 return err
 }
 for _, ip := range ips {
 fmt.Fprintln(a.Stdout, ip.IP.String())
 }
 case "host":
 hosts, err := net.DefaultResolver.LookupAddr(ctx, name)
 if err != nil {
 return err
 }
 for _, h := range hosts {
 fmt.Fprintln(a.Stdout, h)
 }
 default:
 return fmt.Errorf("unknown -type %q", *typ)
 }
 return nil
}
```

```
go run ./cmd/netkit dns example.com
go run ./cmd/netkit dns -type host 1.1.1.1
```

Stretch: LookupMX, LookupTXT, LookupCNAME behind -type mx|txt|cname.

### Tool 3: scan — concurrent TCP ports

```
func (a *App) cmdScan(ctx context.Context, args []string) error {
 fs := flag.NewFlagSet("scan", flag.ContinueOnError)
 fs.SetOutput(a.Stderr)
 ports := fs.String("ports", "22,80,443,8080", "comma-separated ports or start-end range")
 timeout := fs.Duration("t", 500*time.Millisecond, "dial timeout per port")
 workers := fs.Int("w", 32, "concurrent workers")
 if err := fs.Parse(args); err != nil {
 return ErrUsage
 }
 if fs.NArg() != 1 {
 fmt.Fprintln(a.Stderr, "usage: netkit scan [flags] <host>")
 return ErrUsage
 }
 host := fs.Arg(0)

 list, err := parsePorts(*ports)
 if err != nil {
 return err
 }

 type job struct{ port int }
 type res struct {
 port int
 open bool
 }

 jobs := make(chan job)
 results := make(chan res)
 var wg sync.WaitGroup

 worker := func() {
 defer wg.Done()
 for j := range jobs {
 addr := net.JoinHostPort(host, strconv.Itoa(j.port))
 d := net.Dialer{Timeout: *timeout}
 conn, err := d.DialContext(ctx, "tcp", addr)
 if err == nil {
 _ = conn.Close()
 results <- res{port: j.port, open: true}
 } else {
 results <- res{port: j.port, open: false}
 }
 }
 }

}
```

```

n := *workers
if n > len(list) {
 n = len(list)
}
wg.Add(n)
for i := 0; i < n; i++ {
 go worker()
}

go func() {
 defer close(jobs)
 for _, p := range list {
 select {
 case <-ctx.Done():
 return
 case jobs <- job{port: p}:
 }
 }
}()

go func() {
 wg.Wait()
 close(results)
}()

openN := 0
for r := range results {
 if r.open {
 openN++
 fmt.Fprintf(a.Stdout, "%d/tcp open\n", r.port)
 }
}
fmt.Fprintf(a.Stderr, "scanned %d ports, %d open\n", len(list), openN)
if ctx.Err() != nil {
 return ctx.Err()
}
return nil
}

func parsePorts(s string) ([]int, error) {
 var out []int
 for _, part := range strings.Split(s, ",") {
 part = strings.TrimSpace(part)
 if part == "" {
 continue
 }
 }
}

```

```

 if lo, hi, ok := strings.Cut(part, "-"); ok {
 a, err1 := strconv.Atoi(lo)
 b, err2 := strconv.Atoi(hi)
 if err1 != nil || err2 != nil || a < 1 || b > 65535 || a > b {
 return nil, fmt.Errorf("bad range %q", part)
 }
 for p := a; p <= b; p++ {
 out = append(out, p)
 }
 continue
 }
 p, err := strconv.Atoi(part)
 if err != nil || p < 1 || p > 65535 {
 return nil, fmt.Errorf("bad port %q", part)
 }
 out = append(out, p)
}
if len(out) == 0 {
 return nil, fmt.Errorf("no ports")
}
return out, nil
}

```

```

go run ./cmd/netkit scan -ports 80,443,22 example.com
go run ./cmd/netkit scan -ports 1-1024 -w 64 -t 200ms 127.0.0.1

```

**Ethics note:** only scan hosts you own or have permission to test. Rate-limit (-w, -t) on shared networks.

---

## Tool 4: probe — HTTP status + latency

```

func (a *App) cmdProbe(ctx context.Context, args []string) error {
 fs := flag.NewFlagSet("probe", flag.ContinueOnError)
 fs.SetOutput(a.Stderr)
 timeout := fs.Duration("t", 10*time.Second, "total timeout")
 method := fs.String("X", http.MethodGet, "HTTP method")
 expect := fs.Int("expect", 0, "expected status (0=any 2xx/3xx)")
 if err := fs.Parse(args); err != nil {
 return ErrUsage
 }
 if fs.NArg() != 1 {
 fmt.Fprintln(a.Stderr, "usage: netkit probe [flags] <url>")
 }
}

```



```

 return ErrUsage
}
rawURL := fs.Arg(0)

ctx, cancel := context.WithTimeout(ctx, *timeout)
defer cancel()

req, err := http.NewRequestWithContext(ctx, *method, rawURL, nil)
if err != nil {
 return err
}
req.Header.Set("User-Agent", "netkit/1.0")

client := &http.Client{
 Timeout: *timeout,
 CheckRedirect: func(req *http.Request, via []*http.Request) error {
 if len(via) >= 10 {
 return fmt.Errorf("too many redirects")
 }
 return nil
 },
}

start := time.Now()
resp, err := client.Do(req)
rtt := time.Since(start)
if err != nil {
 return err
}
defer resp.Body.Close()
_, _ = io.Copy(io.Discard, io.LimitReader(resp.Body, 64<<10))

fmt.Fprintf(a.Stdout, "status=%d rtt=%v url=%s\n", resp.StatusCode, rtt, rawURL)

if *expect > 0 && resp.StatusCode != *expect {
 return fmt.Errorf("status %d want %d", resp.StatusCode, *expect)
}
if *expect == 0 && resp.StatusCode >= 400 {
 return fmt.Errorf("status %d", resp.StatusCode)
}
return nil
}

```

```

go run ./cmd/netkit probe https://example.com
go run ./cmd/netkit probe -expect 200 -t 3s https://example.com

```

## Tool 5: whoami — local identity

```
func (a *App) cmdWhoami(ctx context.Context, args []string) error {
 ifaces, err := net.Interfaces()
 if err != nil {
 return err
 }
 for _, iface := range ifaces {
 if iface.Flags&net.FlagUp == 0 {
 continue
 }
 addrs, err := iface.Addrs()
 if err != nil {
 continue
 }
 for _, addr := range addrs {
 fmt.Fprintf(a.Stdout, "%s\t%s\n", iface.Name, addr.String())
 }
 }

 // optional: discover outbound IP by dialing UDP (no packets need succeed)
 conn, err := net.Dial("udp", "8.8.8.8:80")
 if err == nil {
 if ua, ok := conn.LocalAddr().(*net.UDPAddr); ok {
 fmt.Fprintf(a.Stdout, "outbound\t%s\n", ua.IP)
 }
 _ = conn.Close()
 }
 return nil
}
```

---

## Optional: Cobra wrapper

When the command tree grows, map each cmdX to a Cobra command (chapter 311) but **keep** netutil pure:

```
// sketch
pingCmd := &cobra.Command{
 Use: "ping HOST",
 Args: cobra.ExactArgs(1),
 RunE: func(cmd *cobra.Command, args []string) error {
 return a.cmdPing(cmd.Context(), append(flagArgs, args...))
 },
}
```

```
}
```

Domain stays in `netutil`; Cobra only parses and calls.

---

## Tests (no live network required)

```
func TestParsePorts(t *testing.T) {
 got, err := parsePorts("22,80-82")
 if err != nil {
 t.Fatal(err)
 }
 want := []int{22, 80, 81, 82}
 if !slices.Equal(got, want) {
 t.Fatalf("got %v want %v", got, want)
 }
}

func TestStats(t *testing.T) {
 var s netutil.Stats
 s.Add(netutil.ProbeResult{OK: true, RTT: 10 * time.Millisecond})
 s.Add(netutil.ProbeResult{OK: false, Err: context.DeadlineExceeded})
 if s.Recv != 1 || s.Sent != 2 {
 t.Fatalf("stats %+v", s)
 }
}
```

For dial logic, introduce:

```
type Dialer interface {
 DialContext(ctx context.Context, network, address string) (net.Conn, error)
}
```

Inject a fake that returns after N ms or errors—table-test ping loop without the internet.

---

## Acceptance checklist

- ☐ `netkit ping example.com` prints per-probe RTT and summary
- ☐ Ctrl-C stops mid-scan / mid-ping promptly

- ❑ `scan -ports 80,443` lists only open ports on stdout
- ❑ `dns` prints one IP per line (pipe-friendly)
- ❑ `probe -expect 200` fails non-zero on 404
- ❑ Misuse (`netkit`, `netkit ping`) exits 2 with usage on stderr
- ❑ At least `parsePorts` + `Stats` unit tests pass

```
go test ./...
go build -o netkit ./cmd/netkit
./netkit ping -c 3 1.1.1.1
./netkit scan -ports 53,80,443 1.1.1.1
./netkit probe https://1.1.1.1
```

## Stretch goals

1. **JSON mode** for every command (`-json`) for CI scripts
2. **ICMP ping** via `golang.org/x/net/icmp` (needs privileges; document)
3. **Traceroute-style** increasing TTL (OS-specific; advanced)
4. **CIDR scan** for lab networks only
5. **TLS probe**: dial TLS, print cert expiry (`crypto/tls`)
6. Publish with version ldflags (chapter 315)

## What you practiced

Skill	Where
Subcommands + FlagSet	each <code>cmd*</code>
Timeouts / Dialer	<code>TCPping</code> , <code>scan</code> , <code>probe</code>
Concurrency + cancel	<code>scan workers</code> + <code>ctx</code>
Stdout/stderr contract	<code>results</code> vs <code>stats</code>
Testable packages	<code>netutil</code> , <code>parsePorts</code>

You now have a real **network diagnostics CLI**—the same genre as `ping`, `dig`, and port scanners—implemented the Go way: simple protocols, explicit deadlines, one static binary.

**Next:** simple security helpers (TLS cert days left, security headers, banners, secret scan) in [318 Network security mini-tools](#).

## **Project: Simple Network Security Mini-Tools**

# Project: Simple Network Security Mini-Tools

## Overview

Build **netsec**, a small multi-command CLI of **minimal security helpers**—not a scanner suite, not Metasploit. Each subcommand is short, stdlib-only, and useful for day-to-day checks: TLS certs, security headers, banners, redirects, and local hygiene.

Use only against systems you own or have permission to test.

Subcommand	Purpose
<b>tls</b>	Show cert expiry, SANs, issuer
<b>headers</b>	Check common HTTP security headers
<b>banner</b>	Read first bytes from a TCP service
<b>redirects</b>	Follow HTTP redirect chain
<b>cidr</b>	Test if an IP is inside a CIDR
<b>cookies</b>	Inspect <b>Set-Cookie</b> flags
<b>hash</b>	SHA-256 a file (integrity)
<b>secret</b>	Naive secret-pattern scan on files
<b>rand</b>	Generate a random token (crypto/rand)
<b>listen</b>	Bind a port (check free / demos)

Pair with diagnostics from [317 netkit](#). Hardening depth: [17 Security](#).

## Goals

1. One binary, many tiny verbs
2. Timeouts on every network call
3. Clear pass/fail exit codes for CI (**headers**, **tls --min-days**)
4. Stdout = facts; stderr = warnings
5. Keep each command under ~80 lines of real logic

## Scaffold

```
mkdir -p netsec/cmd/netsec netsec/internal/app
cd netsec
go mod init example.com/netsec
```

```
netsec/
 cmd/netsec/main.go
 internal/app/app.go
 internal/app/tls.go
 internal/app/headers.go
 internal/app/banner.go
 # ... one file per command is fine
```

### main + dispatch (same pattern as netkit)

```
package main

import (
 "context"
 "fmt"
 "os"
 "os/signal"
 "syscall"

 "example.com/netsec/internal/app"
)

func main() {
 ctx, stop := signal.NotifyContext(context.Background(), os.Interrupt, syscall.SIGTERM)
 defer stop()
 if err := app.New(os.Stdout, os.Stderr).Run(ctx, os.Args[1:]); err != nil {
 fmt.Fprintln(os.Stderr, "netsec:", err)
 os.Exit(app.ExitCode(err))
 }
}

// internal/app/app.go - switch on tls|headers|banner|...
// ErrUsage → exit 2; other errors → exit 1; nil → 0
```

---

## 1. tls — certificate peek

```
func (a *App) cmdTLS(ctx context.Context, args []string) error {
 fs := flag.NewFlagSet("tls", flag.ContinueOnError)
 fs.SetOutput(a.Stderr)
 addr := fs.String("addr", "", "host:port (default host:443)")
 serverName := fs.String("servername", "", "SNI override")
 minDays := fs.Int("min-days", 0, "fail if not valid at least this many days (0=off)")
 timeout := fs.Duration("t", 8*time.Second, "timeout")
 if err := fs.Parse(args); err != nil {
 return ErrUsage
 }
 if fs.NArg() != 1 {
 fmt.Fprintln(a.Stderr, "usage: netsec tls [flags] <host>")
 return ErrUsage
 }
 host := fs.Arg(0)
 target := *addr
 if target == "" {
 target = net.JoinHostPort(host, "443")
 }
 sni := *serverName
 if sni == "" {
 sni = host
 }

 d := &net.Dialer{Timeout: *timeout}
 raw, err := d.DialContext(ctx, "tcp", target)
 if err != nil {
 return err
 }
 defer raw.Close()
 _ = raw.SetDeadline(time.Now().Add(*timeout))

 cfg := &tls.Config{ServerName: sni, MinVersion: tls.VersionTLS12}
 conn := tls.Client(raw, cfg)
 if err := conn.HandshakeContext(ctx); err != nil {
 return err
 }
 defer conn.Close()

 state := conn.ConnectionState()
 if len(state.PeerCertificates) == 0 {
 return fmt.Errorf("no peer certificates")
 }
 cert := state.PeerCertificates[0]
```



```

days := int(time.Until(cert.NotAfter).Hours() / 24)

fmt.Fprintf(a.Stdout, "subject\t%s\n", cert.Subject.CommonName)
fmt.Fprintf(a.Stdout, "issuer\t%s\n", cert.Issuer.CommonName)
fmt.Fprintf(a.Stdout, "not_before\t%s\n", cert.NotBefore.Format(time.RFC3339))
fmt.Fprintf(a.Stdout, "not_after\t%s\n", cert.NotAfter.Format(time.RFC3339))
fmt.Fprintf(a.Stdout, "days_left\t%d\n", days)
fmt.Fprintf(a.Stdout, "version\t%s\n", tls.VersionName(state.Version))
for _, san := range cert.DNSNames {
 fmt.Fprintf(a.Stdout, "san\t%s\n", san)
}

if *minDays > 0 && days < *minDays {
 return fmt.Errorf("cert expires in %d days (need >= %d)", days, *minDays)
}
if time.Now().After(cert.NotAfter) {
 return fmt.Errorf("certificate expired")
}
return nil
}

go run ./cmd/netsec tls example.com
go run ./cmd/netsec tls -min-days 14 example.com # CI gate

```

## 2. headers — security header checklist

Minimal checks only—no full browser security model.

```

var interesting = []string{
 "Strict-Transport-Security",
 "Content-Security-Policy",
 "X-Content-Type-Options",
 "X-Frame-Options",
 "Referrer-Policy",
 "Permissions-Policy",
 "Cross-Origin-Opener-Policy",
}

func (a *App) cmdHeaders(ctx context.Context, args []string) error {
 fs := flag.NewFlagSet("headers", flag.ContinueOnError)
 fs.SetOutput(a.Stderr)
 timeout := fs.Duration("t", 10*time.Second, "timeout")

```

```

strict := fs.Bool("strict", false, "exit 1 if any recommended header missing")
if err := fs.Parse(args); err != nil {
 return ErrUsage
}
if fs.NArg() != 1 {
 fmt.Fprintln(a.Stderr, "usage: netsec headers [-strict] <url>")
 return ErrUsage
}
url := fs.Arg(0)

ctx, cancel := context.WithTimeout(ctx, *timeout)
defer cancel()
req, err := http.NewRequestWithContext(ctx, http.MethodGet, url, nil)
if err != nil {
 return err
}
resp, err := http.DefaultClient.Do(req)
if err != nil {
 return err
}
defer resp.Body.Close()
_, _ = io.Copy(io.Discard, io.LimitReader(resp.Body, 64<<10))

missing := 0
for _, h := range interesting {
 v := resp.Header.Get(h)
 if v == "" {
 missing++
 fmt.Fprintf(a.Stdout, "MISSING\t%s\n", h)
 } else {
 fmt.Fprintf(a.Stdout, "OK\t%s\t%s\n", h, v)
 }
}
if *strict && missing > 0 {
 return fmt.Errorf("%d recommended headers missing", missing)
}
return nil
}

```

```

go run ./cmd/netsec headers https://example.com
go run ./cmd/netsec headers -strict https://example.com

```

### 3. banner — first bytes (service fingerprint sketch)

```
func (a *App) cmdBanner(ctx context.Context, args []string) error {
 fs := flag.NewFlagSet("banner", flag.ContinueOnError)
 fs.SetOutput(a.Stderr)
 timeout := fs.Duration("t", 3*time.Second, "timeout")
 max := fs.Int("n", 256, "max bytes to read")
 if err := fs.Parse(args); err != nil {
 return ErrUsage
 }
 if fs.NArg() != 1 {
 fmt.Fprintln(a.Stderr, "usage: netsec banner host:port")
 return ErrUsage
 }
 addr := fs.Arg(0)

 d := net.Dialer{Timeout: *timeout}
 conn, err := d.DialContext(ctx, "tcp", addr)
 if err != nil {
 return err
 }
 defer conn.Close()
 _ = conn.SetDeadline(time.Now().Add(*timeout))

 // some services need a poke (HTTP)
 _, _ = io.WriteString(conn, "\r\n")

 buf := make([]byte, *max)
 n, err := conn.Read(buf)
 if n > 0 {
 // printable-ish for terminals
 s := strings.Map(func(r rune) rune {
 if r >= 32 && r < 127 || r == '\n' || r == '\r' || r == '\t' {
 return r
 }
 return '.'
 }, string(buf[:n]))
 fmt.Fprint(a.Stdout, s)
 if !strings.HasSuffix(s, "\n") {
 fmt.Fprintln(a.Stdout)
 }
 }
 if err != nil && !errors.Is(err, io.EOF) {
 // timeout with partial banner is still useful
 if n == 0 {
 return err
 }
 }
}
```

```

 }
}
return nil
}

go run ./cmd/netsec banner example.com:80
go run ./cmd/netsec banner 127.0.0.1:22

```

---

## 4. redirects — chain walk

```

func (a *App) cmdRedirects(ctx context.Context, args []string) error {
 fs := flag.NewFlagSet("redirects", flag.ContinueOnError)
 fs.SetOutput(a.Stderr)
 max := fs.Int("max", 10, "max hops")
 timeout := fs.Duration("t", 10*time.Second, "timeout")
 if err := fs.Parse(args); err != nil {
 return ErrUsage
 }
 if fs.NArg() != 1 {
 return ErrUsage
 }
 url := fs.Arg(0)
 client := &http.Client{
 Timeout: *timeout,
 CheckRedirect: func(req *http.Request, via []*http.Request) error {
 return http.ErrUseLastResponse // manual walk
 },
 }

 for i := 0; i < *max; i++ {
 req, err := http.NewRequestWithContext(ctx, http.MethodGet, url, nil)
 if err != nil {
 return err
 }
 resp, err := client.Do(req)
 if err != nil {
 return err
 }
 loc := resp.Header.Get("Location")
 fmt.Fprintf(a.Stdout, "%d\t%d\t%s\n", i, resp.StatusCode, url)
 _, _ = io.Copy(io.Discard, io.LimitReader(resp.Body, 8<<10))
 resp.Body.Close()
 }
}

```

```

 if resp.StatusCode < 300 || resp.StatusCode >= 400 || loc == "" {
 return nil
 }
 next, err := resp.Request.URL.Parse(loc)
 if err != nil {
 return err
 }
 // weak open-redirect hint: scheme jump to different host
 if i == 0 {
 // continue
 }
 url = next.String()
}
return fmt.Errorf("too many redirects (>%d)", *max)
}

go run ./cmd/netsec redirects http://example.com

```

---

## 5. cidr — membership check

```

func (a *App) cmdCIDR(ctx context.Context, args []string) error {
 fs := flag.NewFlagSet("cidr", flag.ContinueOnError)
 fs.SetOutput(a.Stderr)
 if err := fs.Parse(args); err != nil {
 return ErrUsage
 }
 // netsec cidr <ip> <cidr>
 if fs.NArg() != 2 {
 fmt.Fprintln(a.Stderr, "usage: netsec cidr <ip> <cidr>")
 return ErrUsage
 }
 ip := net.ParseIP(fs.Arg(0))
 if ip == nil {
 return fmt.Errorf("bad ip")
 }
 _, network, err := net.ParseCIDR(fs.Arg(1))
 if err != nil {
 return err
 }
 ok := network.Contains(ip)
 fmt.Fprintf(a.Stdout, "%t\n", ok)
 if !ok {

```

```

 return fmt.Errorf("not in network")
 }
 return nil
}

go run ./cmd/netsec cidr 10.0.0.5 10.0.0.0/8 ; echo $?

```

Useful in allowlist scripts and tiny policy checks.

---

## 6. cookies — flag hygiene

```

func (a *App) cmdCookies(ctx context.Context, args []string) error {
 fs := flag.NewFlagSet("cookies", flag.ContinueOnError)
 fs.SetOutput(a.Stderr)
 timeout := fs.Duration("t", 10*time.Second, "timeout")
 strict := fs.Bool("strict", false, "fail if session-like cookie lacks Secure+HttpOnly")
 if err := fs.Parse(args); err != nil {
 return ErrUsage
 }
 if fs.NArg() != 1 {
 return ErrUsage
 }
 ctx, cancel := context.WithTimeout(ctx, *timeout)
 defer cancel()
 req, _ := http.NewRequestWithContext(ctx, http.MethodGet, fs.Arg(0), nil)
 resp, err := http.DefaultClient.Do(req)
 if err != nil {
 return err
 }
 defer resp.Body.Close()
 _, _ = io.Copy(io.Discard, io.LimitReader(resp.Body, 64<<10))

 bad := 0
 for _, c := range resp.Cookies() {
 fmt.Fprintf(a.Stdout, "name=%s secure=%v httponly=%v samesite=%v\n",
 c.Name, c.Secure, c.HttpOnly, c.SameSite)
 // heuristic: names containing session/token
 n := strings.ToLower(c.Name)
 if strings.Contains(n, "session") || strings.Contains(n, "token") || strings.Contains(n, "token") {
 if !c.Secure || !c.HttpOnly {
 bad++
 fmt.Fprintf(a.Stderr, "warn: %s should be Secure+HttpOnly\n", c.Name)
 }
 }
 }
}

```

```

 }
}
if *strict && bad > 0 {
 return fmt.Errorf("%d weak cookies", bad)
}
return nil
}

```

---

## 7. hash — file integrity

```

func (a *App) cmdHash(ctx context.Context, args []string) error {
 fs := flag.NewFlagSet("hash", flag.ContinueOnError)
 fs.SetOutput(a.Stderr)
 if err := fs.Parse(args); err != nil {
 return ErrUsage
 }
 if fs.NArg() < 1 {
 fmt.Fprintln(a.Stderr, "usage: netsec hash <files...>")
 return ErrUsage
 }
 for _, path := range fs.Args() {
 f, err := os.Open(path)
 if err != nil {
 return err
 }
 h := sha256.New()
 if _, err := io.Copy(h, f); err != nil {
 f.Close()
 return err
 }
 f.Close()
 fmt.Fprintf(a.Stdout, "%x %s\n", h.Sum(nil), path)
 }
 return nil
}

```

```
go run ./cmd/netsec hash ./cmd/netsec/main.go
```

---

## 8. secret — naive leak hunt (local files)

Not a replacement for gitleaks/trufflehog—teaching regex + walk.

```
var patterns = []*regexp.Regexp{
 regexp.MustCompile(`(?i)api[_-]?key\s*[:=]\s*["']["']{8,}`),
 regexp.MustCompile(`(?i)secret\s*[:=]\s*["']["']{8,}`),
 regexp.MustCompile(`AKIA[0-9A-Z]{16}`), // AWS key id shape
 regexp.MustCompile(`-----BEGIN (RSA |EC )?PRIVATE KEY-----`),
}

func (a *App) cmdSecret(ctx context.Context, args []string) error {
 fs := flag.NewFlagSet("secret", flag.ContinueOnError)
 fs.SetOutput(a.Stderr)
 root := fs.String("root", ".", "directory")
 if err := fs.Parse(args); err != nil {
 return ErrUsage
 }
 found := 0
 err := filepath.WalkDir(*root, func(path string, d fs.DirEntry, err error) error {
 if err != nil {
 return nil
 }
 if d.IsDir() {
 name := d.Name()
 if name == ".git" || name == "node_modules" || name == "vendor" {
 return fs.SkipDir
 }
 return nil
 }
 // skip large / binary-ish
 info, err := d.Info()
 if err != nil || info.Size() > 1<<20 {
 return nil
 }
 b, err := os.ReadFile(path)
 if err != nil {
 return nil
 }
 if bytes.IndexByte(b, 0) >= 0 {
 return nil // binary
 }
 for _, re := range patterns {
 if re.Find(b) != nil {
 found++
 fmt.Fprintf(a.Stdout, "%s\t%s\n", path, re.String())
 }
 }
 })
}
```



```

 }
 return nil
})
if err != nil {
 return err
}
if found > 0 {
 return fmt.Errorf("found %d potential secret matches", found)
}
return nil
}

go run ./cmd/netsec secret -root .

```

---

## 9. rand — tokens with crypto/rand

```

func (a *App) cmdRand(ctx context.Context, args []string) error {
 fs := flag.NewFlagSet("rand", flag.ContinueOnError)
 fs.SetOutput(a.Stderr)
 n := fs.Int("n", 32, "bytes of entropy")
 hexOut := fs.Bool("hex", true, "hex encode (false=raw base64)")
 if err := fs.Parse(args); err != nil {
 return ErrUsage
 }
 b := make([]byte, *n)
 if _, err := rand.Read(b); err != nil {
 return err
 }
 if *hexOut {
 fmt.Fprintf(a.Stdout, "%x\n", b)
 } else {
 fmt.Fprintln(a.Stdout, base64.RawURLEncoding.EncodeToString(b))
 }
 return nil
}

```

Never use `math/rand` for tokens or session IDs.

## 10. listen — bind check / tiny sink

```
func (a *App) cmdListen(ctx context.Context, args []string) error {
 fs := flag.NewFlagSet("listen", flag.ContinueOnError)
 fs.SetOutput(a.Stderr)
 addr := fs.String("addr", "127.0.0.1:0", "listen address (:0 = ephemeral)")
 if err := fs.Parse(args); err != nil {
 return ErrUsage
 }
 ln, err := net.Listen("tcp", *addr)
 if err != nil {
 return err // port in use / permission
 }
 defer ln.Close()
 fmt.Fprintf(a.Stdout, "listening\t%s\n", ln.Addr().String())
 // wait until cancel - useful smoke for firewall demos
 <-ctx.Done()
 return nil
}
```

```
go run ./cmd/netsec listen -addr 127.0.0.1:9999
Ctrl-C to stop
```

---

## Wire the switch

```
func (a *App) Run(ctx context.Context, args []string) error {
 if len(args) < 1 {
 return a.usage()
 }
 switch args[0] {
 case "tls":
 return a.cmdTLS(ctx, args[1:])
 case "headers":
 return a.cmdHeaders(ctx, args[1:])
 case "banner":
 return a.cmdBanner(ctx, args[1:])
 case "redirects":
 return a.cmdRedirects(ctx, args[1:])
 case "cidr":
 return a.cmdCIDR(ctx, args[1:])
 case "cookies":
 return a.cmdCookies(ctx, args[1:])
 }
```

```

 case "hash":
 return a.cmdHash(ctx, args[1:])
 case "secret":
 return a.cmdSecret(ctx, args[1:])
 case "rand":
 return a.cmdRand(ctx, args[1:])
 case "listen":
 return a.cmdListen(ctx, args[1:])
 default:
 return a.usage()
}
}

```

---

## Minimal tests

```

func TestCIDR(t *testing.T) {
 _, n, _ := net.ParseCIDR("10.0.0.0/8")
 if !n.Contains(net.ParseIP("10.1.2.3")) {
 t.Fatal("expected contain")
 }
}

func TestRandLen(t *testing.T) {
 b := make([]byte, 16)
 if _, err := rand.Read(b); err != nil {
 t.Fatal(err)
 }
}

```

Network-backed commands: test with `httptest` for `headers` / `cookies` (inject a test server that sets headers).

---

## Suggested lab order

- |                       |              |
|-----------------------|--------------|
| 1. hash + rand        | (no network) |
| 2. cidr               | (pure logic) |
| 3. tls + headers      | (HTTPS)      |
| 4. banner + redirects | (TCP/HTTP)   |
| 5. cookies + secret   | (hygiene)    |

## Ethics & scope

Do	Don't
Test your apps and labs	Scan random internet hosts
Use <code>-strict</code> in <b>your</b> CI	Treat this as a pentest platform
Document false positives ( <b>secret</b> )	Commit real secrets to fix tests

## Stretch (still minimal)

1. `tls --json` for monitors
2. `headers` allowlist config file
3. Merge `netsec` under `netkit sec ...` as a Cobra parent
4. `tls` print leaf + intermediates briefly
5. Simple **HSTS** max-age parser

## Acceptance checklist

- ☐ `tls example.com` prints `days_left`
- ☐ `tls -min-days 30` fails near expiry (simulate with low threshold)
- ☐ `headers -strict` non-zero when headers missing
- ☐ `hash` matches `shasum -a 256` on a file
- ☐ `secret` finds a planted fake `api_key=...` in a temp file
- ☐ `rand -n 16` prints 32 hex chars
- ☐ Misuse exits 2

These tools stay **small on purpose**: enough to learn `crypto/tls`, `net/http`, and safe CLI habits—without building a security product.

## **Project: Text and Log CLI Tools**

# Project: Text and Log CLI Tools

## Overview

**textkit**: small pipeline tools—stdlib only—for daily log/text work.

Cmd	Behavior
<b>freq</b>	word/line frequency counts
<b>cut</b>	-f fields -d delim
<b>ts</b>	prefix lines with RFC3339 timestamp
<b>sample</b>	keep every Nth line or rate %
<b>jsonl-keys</b>	list keys seen in NDJSON

## Contracts

- stdin if no files
- data stdout / errors stderr
- exit 0 if IO ok

## freq core

```
sc := bufio.NewScanner(r)
counts := map[string]int{}
for sc.Scan() {
 for _, w := range strings.Fields(sc.Text()) {
 counts[w]++
 }
}
// print sorted by count desc
```

**ts**

```
fmt.Fprintf(w, "%s %s\n", time.Now().Format(time.RFC3339), line)
```

## Acceptance

```
printf 'a b\na\n' | go run . freq
echo hello | go run . ts
```

## **Project: HTTP Debug CLI**



# Project: HTTP Debug CLI

## Overview

`httpdbg`: curl-like teaching client with timing breakdowns.

## Features

```
httpdbg get URL
httpdbg get -H "Accept: application/json" -v URL
httpdbg timings URL # DNS / connect / TLS / TTFB sketch
```

## Timings sketch

Use `http.Transport` + `httptrace.ClientTrace`:

```
trace := &httptrace.ClientTrace{
 DNSDone: func(info httptrace.DNSDoneInfo) { /* mark */ },
 ConnectDone: func(network, addr string, err error) {},
 GotFirstResponseByte: func() {},
}
req = req.WithContext(httptrace.WithClientTrace(ctx, trace))
```

## Requirements

- Timeouts
- `-o` write body to file
- Print status and selected headers on `-v`
- Never follow redirects beyond `-max-redirect`

## Acceptance

```
go run . get -v https://example.com
go run . timings https://example.com
```

## Cross-Platform CLI Notes

# Cross-Platform CLI Notes

## Overview

Go builds everywhere; OS details still leak into CLIs. Isolate differences.

## Paths

```
filepath.Join(a, b) // not strings + "/"
filepath.Separator
os.UserConfigDir() // not hardcode ~/.config
```

## Line endings

```
// accept both when reading tools
// write "\n" unless you must speak CRLF to Windows tools
```

## Signals

Windows: `os.Interrupt` works for Ctrl-C; SIGTERM semantics differ. Use `signal.NotifyContext` with available signals; build-tag extra Unix signals.

## Shells

- Avoid `cmd /c` with user strings
- PowerShell quoting `bash`
- Document examples for both

## Color

```
// enable only if TTY && NO_COLOR unset && TERM != dumb
```

Windows: optional virtual terminal processing for ANSI.

## Build tags

```
//go:build windows
// flock_windows.go stub → error "unsupported"
```

## File locking

`flock` Unix-only; Windows has different APIs—feature-detect or document Unix-only commands.

## Rules

Do	Don't
<code>filepath</code> + <code>os</code> helpers	Assume <code>/tmp</code> exists as on Linux
CI matrix linux/mac/windows	Test only on your laptop OS
Clear “unix only” in help	Silent fail on Windows

## Try next

1. `GOOS=windows go build` your CLI.
2. Replace hardcoded `/tmp` with `os.TempDir()`.
3. Stub `flock` commands on Windows with message.

# Stdlib

# Standard Library Tour Overview

# Standard Library Tour Overview

This part is a **stdlib-first map** of the packages you will use on almost every real Go program. Frameworks and third-party libraries come later; fluency here is what makes those tools feel thin and optional.

Earlier parts already teach language mechanics. Here the unit of study is the **package contract**: what it guarantees, what it leaves to you, and which defaults are production-safe.

## Why This Part Exists

Most “I know Go” gaps are not syntax. They are:

- Using `fmt.Println` where `log/slog` belongs
- Loading whole files when `io` streams would bound memory
- `http.Get` with no timeout
- Stringly-typed paths instead of `filepath` / `io/fs`
- Hand-rolled JSON error handling that loses type safety
- Concurrency without `context` deadlines or `sync` ownership rules

The standard library already solves these — if you know where to look.

## Learning Path

Chapter	Packages	Outcome
981 <code>fmt</code> / <code>log</code> / <code>slog</code>	<code>fmt</code> , <code>log</code> , <code>log/slog</code>	Readable output and structured events
982 Text & numbers	<code>bytes</code> , <code>strings</code> , <code>strconv</code> , <code>unicode</code> , <code>utf8</code>	Correct UTF-8 and conversions
983 I/O composition	<code>io</code> , <code>bufio</code>	Bounded streams and efficient copies
984 Files & embed	<code>os</code> , <code>path/filepath</code> , <code>io/fs</code> , <code>embed</code>	Safe paths and single-binary assets
985 Encoding	<code>encoding/json</code> , <code>encoding/json/v2</code> , <code>csv</code> , <code>xml</code> , <code>base64</code> , <code>hex</code>	Data at rest and on the wire
986 <code>net/http</code>	<code>net/http</code> , <code>net/url</code>	Production-shaped clients and servers



Chapter	Packages	Outcome
987 CLI & process	<code>flag, os/exec, os/signal</code>	Tools that parse, run, and shut down cleanly
988 time & context	<code>time, context</code>	Budgets, clocks, and cancellation
989 sync & atomic	<code>sync, sync/atomic</code>	Shared state without races
990 Collections	<code>slices, maps, cmp, iter</code>	Modern helpers (Go 1.21+)
991 testing	<code>testing, testing/fstest, net/http/httptest</code>	Tests, examples, fuzz, benches
992 Crypto & text scan	<code>crypto/*, uuid, hash, math/rand/v2, regexp</code>	Integrity, IDs, secrets hygiene, matching
993 database/sql	<code>database/sql</code>	Pools, queries, transactions
994 net dial/listen	<code>net</code>	TCP/UDP/Unix under HTTP
995 templates	<code>text/template, html/template</code>	Safe rendering and codegen
996 compress/archive	<code>compress/*, archive/zip, tar</code>	gzip, zip, tar streams
997 binary / gob / pem	<code>encoding/binary, gob, pem</code>	Frames, Go blobs, cert armor
998 math / bits / big	<code>math, math/bits, math/big</code>	Floats, bit ops, big integers
999 mime / ops	<code>mime, multipart, expvar, runtime/debug</code>	Uploads, debug vars, build info
999a containers	<code>container/heap, list, ring</code>	Priority queues, lists
999b path/filepath	<code>path, path/filepath</code>	OS paths vs slash paths, jails
999c bufio advanced	<code>bufio Scanner/Reader/Writer</code>	Long lines, peek, flush

## Mental Model

```

input -> decode / parse -> domain logic -> encode / write -> output
 | | |
 encoding/* context io / os / net
 flag / json time slog for events

```

Almost every program is that pipeline with different endpoints (CLI, HTTP, file, pipe).

## Package Selection Cheatsheet

Need	Prefer	Avoid by default
Logs for ops	<code>log/slog</code>	<code>fmt.Println</code> in libraries
HTTP service	<code>net/http</code> + std mux (1.22+)	Framework until you need it
SQL access	<code>database/sql</code> + driver	String-built queries
Raw sockets	<code>net.Dialer</code> / <code>Listen</code> + deadlines	Untimed <code>Accept/Read</code> loops
HTML pages	<code>html/template</code>	<code>text/template</code> for untrusted HTML

Need	Prefer	Avoid by default
gzip / zip / tar	<code>compress/gzip</code> , <code>archive/*</code>	Shelling out for common formats
Binary frames	<code>encoding/binary</code> + max length	Trust remote length blindly
Go-to-Go cache	<code>gob</code> (versioned)	<code>gob</code> as a public multi-lang API
Config flags	<code>flag</code> (or small wrapper)	Global env soup without docs
File walk	<code>fs.WalkDir</code> / <code>filepath.WalkDir</code>	Manual recursion without errors
JSON API	<code>encoding/json</code>	<code>interface{}</code> everywhere
Timeouts	<code>context</code> + <code>http.Client.Timeout</code>	Fire-and-forget goroutines
Shared map	<code>sync.Mutex</code> or <code>sync.Map</code> (when justified)	Unsynchronized maps
Sort / search	<code>slices</code>	Hand-rolled $O(n^2)$ loops
Big integers	<code>math/big</code>	<code>float64</code> for money / huge ints
Uploads	<code>mime/multipart</code> + size limits	Unbounded <code>ParseMultipartForm</code>
Secrets material	<code>crypto/rand</code> , careful logging	<code>math/rand</code> for tokens

## Relationship to Other Parts

Topic	Deeper part
Streaming I/O patterns	<a href="#">12 Specialized — I/O</a>
Concurrency design	<a href="#">07 Concurrency</a>
HTTP resilience	<a href="#">13 Network systems</a>
TCP services	<a href="#">13 Network systems — TCP</a>
SQL pools	<a href="#">20 Deep dives — database/sql</a>
Web templates / DB	<a href="#">22 Web development</a>
slog in production	<a href="#">16 Observability</a>
TLS / crypto hardening	<a href="#">17 Security</a>
Profiling	<a href="#">18 Performance</a>

Use this part as the **default toolkit**. Jump to specialized parts when the problem is no longer “which package?” but “how does this fail under load?”

## How to Study

1. Read the index map; pick one vertical (CLI tool or tiny HTTP service).

2. For each chapter, run the runnable example once, then change one assumption (timeout, limit, encoding).
3. Rebuild a small project using **only** stdlib until you feel friction — that friction is when a library earns its place.

stdlib literacy loop

read package docs -> run example -> break limits -> fix with contracts -> ship tiny tool

## Checklist: Stdlib Literacy

- ☐ Can structure logs with `slog` and attach request-scoped fields
- ☐ Can stream with `io.Copy / LimitReader` instead of unbounded `ReadAll`
- ☐ Can open, walk, and embed files without path traversal bugs
- ☐ Can marshal/unmarshal JSON with sensible error surfaces
- ☐ Can serve and call HTTP with timeouts and context
- ☐ Can build a multi-flag CLI that exits non-zero on failure
- ☐ Can cancel work with `context` and wait with `sync.WaitGroup`
- ☐ Can write table tests, an example, and a basic fuzz target
- ☐ Can generate tokens with `crypto/rand` and hash with `crypto/sha256`
- ☐ Can open `database/sql`, query with context, and map `ErrNoRows`
- ☐ Can dial/listen on TCP with deadlines (not only HTTP)
- ☐ Can render HTML safely with `html/template` (parse once)
- ☐ Can gzip a stream and extract zip/tar without path traversal
- ☐ Can write a length-prefixed binary frame with a max size
- ☐ Can accept a size-limited multipart upload
- ☐ Can expose version/`expvar` or read `debug.ReadBuildInfo`

When most boxes are checked, third-party libraries become choices — not crutches.

## Suggested extensions path

Core loop (981-992)

Data & wire (993-997)

Math & ops (998-999)

Extras (999a-c)

text → io → files → json → http → cli → time → sync → tests → crypto

sql → net → templates → compress → binary/gob/pem

bits/big → mime/uploads → expvar/pprof/buildinfo

container heap/list/ring → path/filepath → bufio advanced

**fmt, log, and log/slog**

# fmt, log, and log/slog

## Overview

Three packages cover **output**:

Package	Role
fmt	Formatting values for humans and strings
log	Legacy line logger (still fine for tiny tools)
log/slog	Structured logging (Go 1.21+ default for services)

Rule of thumb: **libraries return errors; programs log**. Prefer **slog** for anything that runs as a service or daemon.

## fmt Essentials

```
fmt.Print("a", 1) // a1 (no spaces between args of different kinds as you'd expect)
fmt.Println("hello", 42) // hello 42\n
s := fmt.Sprintf("%s=%d", "n", 3)
fmt.Fprintf(w, "status=%v\n", err)
```

## Verbs you actually need

Verb	Meaning
%v	Default format
%+v	Struct with field names
%#v	Go-syntax representation
%T	Type
%s / %q	String / quoted
%d / %x	Decimal / hex
%f / %g	Float
%w	Wrap error ( <code>fmt.Errorf</code> )
%+v on errors	Implementation-defined detail

```
err := fmt.Errorf("open %s: %w", path, err)
```

## Strings and builders

For hot paths assembling many pieces, prefer `strings.Builder` or `fmt.Fprintf` into a builder — not repeated `+` in a loop.

## log (legacy)

```
import "log"

log.SetFlags(log.LstdFlags | log.Lshortfile)
log.Println("starting")
log.Printf("port=%d", port)
log.Fatal(err) // log + os.Exit(1)
```

Fine for CLIs and examples. Lacks structured fields and levels as first-class data.

## log/slog (preferred)

```
import "log/slog"

slog.Info("server start", "addr", addr)
slog.Error("query failed", "err", err, "query", q)
```

## Handlers

```
// Human text (dev)
slog.SetDefault(slog.New(slog.NewTextHandler(os.Stderr, &slog.HandlerOptions{
 Level: slog.LevelDebug,
})))

// JSON (prod)
slog.SetDefault(slog.New(slog.NewJSONHandler(os.Stdout, &slog.HandlerOptions{
 Level: slog.LevelInfo,
})))
```

## Levels and groups

```
slog.Debug("cache hit", "key", key)
slog.Warn("retrying", "attempt", n)
```

```
logger := slog.Default().With("service", "billing")
logger.Info("charge", "user_id", id, "cents", 199)
```

## Context-aware logging

```
func handle(ctx context.Context, w http.ResponseWriter, r *http.Request) {
 log := slog.Default()
 if rid, ok := ctx.Value(requestIDKey).(string); ok {
 log = log.With("request_id", rid)
 }
 log.Info("request", "path", r.URL.Path, "method", r.Method)
}
```

For production correlation patterns, see [Structured logging](#).

## Anti-patterns

1. **Logging and returning the same error without a policy** — pick one place that owns the user-visible message.
2. **Logging secrets** — tokens, passwords, full auth headers.
3. **fmt.Sprintf then log the string** — lose structure; pass key-value pairs.
4. **log.Fatal deep in libraries** — exits the process; return **error** instead.

## Summary

Situation	Use
Build a message string	fmt.Sprintf / Errorf
One-off script noise	log or fmt
Service / long-running tool	log/slog
Wrap errors	fmt.Errorf("...: %w", err)

## Runnable example

Save as main.go:

```
go mod init example
go run .
```

```

package main

import (
 "fmt"
 "log/slog"
 "os"
)

func main() {
 // fmt: formatting
 name := "gopher"
 fmt.Printf("hello %s\n", name)
 fmt.Printf("type=%T value=%#v\n", name, struct{ N int }{N: 7})

 err := fmt.Errorf("open config: %w", os.ErrNotExist)
 fmt.Printf("wrapped: %v\n", err)

 // slog text
 text := slog.New(slog.NewTextHandler(os.Stdout, &slog.HandlerOptions{Level: slog.LevelDebug}))
 text.Debug("boot", "pid", os.Getpid())
 text.Info("ready", "port", 8080)

 // slog JSON
 jsonLog := slog.New(slog.NewJSONHandler(os.Stdout, nil))
 jsonLog.Info("request",
 "method", "GET",
 "path", "/health",
 "status", 200,
 "ms", 3,
)

 // grouped attrs
 svc := jsonLog.With("service", "demo")
 svc.Warn("slow dependency", "dep", "db", "ms", 120)
}

```

**Expected output** (timestamps/pid vary):

```

hello gopher
type=string value=struct { N int }{N:7}
wrapped: open config: file does not exist
level=DEBUG msg=boot pid=...
level=INFO msg=ready port=8080
{"time":"...", "level":"INFO", "msg":"request", "method":"GET", "path":"/health", "status":200, "ms":3}
{"time":"...", "level":"WARN", "msg":"slow dependency", "service":"demo", "dep":"db", "ms":120}

```

**What to notice:** - `%#v` is invaluable when debugging unexpected types. - `slog` fields are queryable; free-form strings are not. - Same event shape works for text (local) and JSON (shipped).



**Try next:** Add a custom `slog.Handler` wrapper that redacts any attribute key named `password` or `token`.

**bytes, strings, strconv, unicode**

# bytes, strings, strconv, unicode

## Overview

Text in Go is **UTF-8 bytes** with optional rune interpretation. This chapter is the stdlib toolkit for slicing, searching, converting, and validating that text without corrupting multi-byte characters.

Related language intro: [Strings](#).

## Package map

Package	Use for
<code>strings</code>	Immutable string ops, builders, readers
<code>bytes</code>	Same ops on <code>[]byte</code> (often I/O buffers)
<code>strconv</code>	Parse/format numbers and quotes
<code>unicode</code> / <code>unicode/utf8</code>	Character classes and UTF-8 decode

Many `strings` functions have `bytes` twins (`Contains`, `Split`, `TrimSpace`, ...). Prefer `[]byte` when you already have buffers from I/O.

## strings cookbook

```
strings.Contains(s, sub)
strings.HasPrefix(s, "http")
strings.Cut(s, "=") // before, after, ok (Go 1.18+)
strings.CutPrefix(s, "Bearer ") // Go 1.20+
strings.CutLast(s, "/") // last separator (Go 1.27+); bytes.CutLast twin
strings.Fields(s) // split on unicode whitespace
strings.Join(parts, ",")
strings.ReplaceAll(s, old, new)
strings.ToValidUTF8(s, "\uFFFD")
```

## Builder vs buffer

```
var b strings.Builder
b.Grow(64)
b.WriteString("id=")
b.WriteString(id)
out := b.String() // copies once
```

`bytes.Buffer` implements `io.Writer` and is better when mixing writes from encoders.

## bytes cookbook

```
bytes.Contains(buf, []byte("ERROR"))
bytes.Split(buf, []byte{'\n'})
bytes.TrimSpace(buf)
reader := bytes.NewReader(buf) // io.Reader, io.Seeker
```

## strconv: explicit conversions

```
n, err := strconv.Atoi("42")
n64, err := strconv.ParseInt("ff", 16, 64)
f, err := strconv.ParseFloat("3.14", 64)
b, err := strconv.ParseBool("true")

s := strconv.Itoa(42)
s = strconv.FormatInt(255, 16) // "ff"
s = strconv.FormatFloat(f, 'f', 2, 64) // "3.14"
q := strconv.Quote("a\nb") // "\"a\\nb\""
```

Never use `fmt.Sprintf("%d", n)` in hot paths when `strconv` will do — it is clearer and often faster.

## unicode and utf8

```
import (
 "unicode"
 "unicode/utf8"
)

utf8.RuneCountInString(s)
r, size := utf8.DecodeRuneInString(s)
```

```
unicode.IsLetter(r)
unicode.Is(unicode.Latin, r)
```

range over a string already yields runes; use `utf8` when decoding partial buffers from the network.

## Validation pattern

```
func isIdent(s string) bool {
 if s == "" {
 return false
 }
 for i, r := range s {
 if i == 0 && !unicode.IsLetter(r) && r != '_' {
 return false
 }
 if i > 0 && !unicode.IsLetter(r) && !unicode.IsDigit(r) && r != '_' {
 return false
 }
 }
 return utf8.ValidString(s)
}
```

## Summary

Task	API
Search / split strings	<code>strings.*</code>
Same on buffers	<code>bytes.*</code>
Number parse/format	<code>strconv.*</code>
Rune properties	<code>unicode.*</code>
Partial UTF-8	<code>unicode/utf8</code>

## Runnable example

```
go mod init example
go run .
```

```
package main
```

```
import (
 "bytes"
```

```

 "fmt"
 "strconv"
 "strings"
 "unicode"
 "unicode/utf8"
)

func main() {
 s := "name=ada; role=admin"
 k, v, ok := strings.Cut(s, "; ")
 fmt.Println("cut:", k, "|", v, ok)

 var b strings.Builder
 b.WriteString("user:")
 b.WriteString(strings.ToUpper("ada"))
 fmt.Println("builder:", b.String())

 raw := []byte(" line1\nline2\n")
 fmt.Printf("trim=%q fields=%q\n", bytes.TrimSpace(raw), bytes.Fields(raw))

 n, err := strconv.ParseInt("2a", 16, 64)
 fmt.Println("parse:", n, err, "format:", strconv.FormatInt(n, 10))

 msg := "Go "
 fmt.Println("bytes", len(msg), "runes", utf8.RuneCountInString(msg))
 for _, r := range msg {
 fmt.Printf(" %q letter=%v\n", r, unicode.IsLetter(r))
 }

 fmt.Println("valid utf8?", utf8.ValidString("ok\xff"))
}

```

### Expected output:

```

cut: name=ada | role=admin true
builder: user:ADA
trim="line1\nline2" fields=["line1" "line2"]
parse: 42 <nil> format: 42
bytes 5 runes 3
'G' letter=true
'o' letter=true
' ' letter=false
valid utf8? false

```

**What to notice:** - Cut is clearer than SplitN for single separators. - bytes and strings mirror each other — pick based on your data form. - Invalid UTF-8 is detectable; do not assume network input is clean.

**Try next:** Write `splitKV(s string) (map[string]string, error)` using `strings.Cut` on `=` and `;` pairs, rejecting duplicates.

## **io and bufio Composition**



# io and bufio Composition

## Overview

The `io` package is Go's **streaming contract**. Small interfaces (`Reader`, `Writer`, `Closer`) let files, sockets, compressors, and crypto share the same pipelines.

Deep patterns also appear in [I/O and Streaming](#). This chapter is the `stdlib` composition cheat-sheet.

## Core contracts

```
type Reader interface {
 Read(p []byte) (n int, err error)
}

type Writer interface {
 Write(p []byte) (n int, err error)
}
```

- Process `n > 0` **before** checking `err` (including `io.EOF`).
- `io.EOF` means end of stream, not a hard failure for `Read` loops.
- Prefer `io.Copy` over manual loops — it uses `WriterTo` / `ReaderFrom` when available.

## Helpers worth memorizing

Helper	Purpose
<code>io.Copy</code> / <code>CopyBuffer</code>	Stream $A \rightarrow B$ efficiently
<code>io.ReadAll</code>	Entire stream to memory (cap it!)
<code>io.LimitReader</code>	Hard cap bytes read
<code>io.TeeReader</code>	Read + fork to a <code>Writer</code> (checksums)
<code>io.MultiReader</code>	Concatenate readers
<code>io.MultiWriter</code>	Fan-out writes
<code>io.Pipe</code>	In-process streaming between goroutines
<code>io.NopCloser</code>	Wrap <code>Reader</code> as <code>ReadCloser</code>
<code>io.Discard</code>	Drain without storing

```

const max = 1 << 20 // 1 MiB
data, err := io.ReadAll(io.LimitReader(r, max+1))
if err != nil {
 return err
}
if len(data) > max {
 return fmt.Errorf("payload exceeds %d bytes", max)
}

```

## bufio: amortize syscalls

```

br := bufio.NewReader(f)
line, err := br.ReadString('\n')

bw := bufio.NewWriter(f)
fmt.Fprintln(bw, "hello")
if err := bw.Flush(); err != nil {
 return err
}

```

Type	Role
bufio.Reader	Peek, ReadBytes, ReadString
bufio.Writer	Buffered writes; always Flush
bufio.Scanner	Tokenize lines/words (watch token size)

```

sc := bufio.NewScanner(f)
// optional: sc.Buffer(buf, 1024*1024) for long lines
for sc.Scan() {
 line := sc.Text()
 _ = line
}
return sc.Err()

```

Default `Scanner` max token is 64KiB — raise it for pathological log lines or reject them.

## Composition examples

### Checksum while uploading

```
h := sha256.New()
tr := io.TeeReader(src, h)
if _, err := io.Copy(dst, tr); err != nil {
 return err
}
sum := h.Sum(nil)
```

### Bound + discard remainder

```
// Read at most n bytes; ensure caller can still Close the body
lr := io.LimitReader(resp.Body, n)
_, err := io.Copy(dst, lr)
```

## Pitfalls

1. Forgetting `Close` on `resp.Body` / files → connection or FD leaks.
2. Unbounded `ReadAll` on user input → memory exhaustion.
3. Nested `bufio.Reader` without understanding buffered leftovers.
4. Ignoring short writes from custom `Writers`.

## Runnable example

```
go mod init example
go run .
```

```
package main

import (
 "bufio"
 "bytes"
 "crypto/sha256"
 "encoding/hex"
 "fmt"
 "io"
 "strings"
)
```

```

func main() {
 src := strings.NewReader("hello stream\nsecond line\n")

 // LimitReader + ReadAll
 limited, _ := io.ReadAll(io.LimitReader(src, 5))
 fmt.Printf("limited=%q\n", limited)

 // reset-like: new reader
 src = strings.NewReader("hello stream\nsecond line\n")
 h := sha256.New()
 var out bytes.Buffer
 tr := io.TeeReader(src, h)
 if _, err := io.Copy(&out, tr); err != nil {
 panic(err)
 }
 fmt.Printf("copied=%q\n", out.Bytes())
 fmt.Println("sha256", hex.EncodeToString(h.Sum(nil))[:16]+"...")

 // Scanner lines
 sc := bufio.NewScanner(strings.NewReader("a\nb\nc\n"))
 for sc.Scan() {
 fmt.Println("line:", sc.Text())
 }
 fmt.Println("scan err:", sc.Err())

 // MultiWriter
 var a, b bytes.Buffer
 mw := io.MultiWriter(&a, &b)
 io.WriteString(mw, "x")
 fmt.Printf("a=%q b=%q\n", a.String(), b.String())
}

```

### Expected output:

```

limited="hello"
copied="hello stream\nsecond line\n"
sha256 f74a77476883f531...
line: a
line: b
line: c
scan err: <nil>
a="x" b="x"

```

**What to notice:** - `LimitReader` stops early without error when the cap is hit mid-stream. - `TeeReader` is the clean way to hash or log while still delivering bytes. - `Scanner` is for tokens; `io.Copy` is for bulk.

**Try next:** Use `io.Pipe` with two goroutines: one writes lines, one counts them with

bufio.Scanner.

**os, filepath, io/fs, and embed**

# os, filepath, io/fs, and embed

## Overview

These packages cover **process environment and files**:

Package	Role
os	Files, env, args, process exit, working directory
path/filepath	OS-aware path manipulation
io/fs	Abstract filesystem interface
embed	Ship files inside the binary

Path safety and portable tools depend on getting this layer right.

## os basics

```
data, err := os.ReadFile("config.json") // Go 1.16+
err = os.WriteFile("out.txt", data, 0o644)
f, err := os.OpenFile(path, os.O_RDWR|os.O_CREATE, 0o644)
info, err := os.Stat(path)
os.MkdirAll("a/b/c", 0o755)
os.Remove(path)
os.RemoveAll(dir) // careful
```

## Env and args

```
os.Args // argv; Args[0] is program name
os.Getenv("HOME")
os.LookupEnv("PORT") // value, ok
os.Exit(1) // skips deferred cleanup in other goroutines
```

Prefer returning errors to `main` and a single `os.Exit` there.

## filepath: always OS-aware

Use `path/filepath` for local disk paths. Use `path` (slash-only) for URL-like paths.

```
filepath.Join("a", "b", "c.txt")
filepath.Clean("../etc/passwd")
filepath.Abs(".")
filepath.Ext("main.go") // ".go"
filepath.Base("/tmp/x") // "x"
filepath.Dir("/tmp/x") // "/tmp"
filepath.WalkDir(root, fn) // preferred over Walk
```

## Prevent path traversal

```
func safeJoin(root, name string) (string, error) {
 clean := filepath.Clean("/") + name // force absolute-like clean
 rel := strings.TrimPrefix(clean, string(filepath.Separator))
 full := filepath.Join(root, rel)
 rootAbs, err := filepath.Abs(root)
 if err != nil {
 return "", err
 }
 fullAbs, err := filepath.Abs(full)
 if err != nil {
 return "", err
 }
 sep := string(os.PathSeparator)
 if fullAbs != rootAbs && !strings.HasPrefix(fullAbs, rootAbs+sep) {
 return "", fmt.Errorf("path escapes root")
 }
 return fullAbs, nil
}
```

## io/fs abstraction

```
var fsys fs.FS = os.DirFS("/data")
b, err := fs.ReadFile(fsys, "hello.txt")
entries, err := fs.ReadDir(fsys, ".")
err = fs.WalkDir(fsys, ".", func(path string, d fs.DirEntry, err error) error {
 return err
})
```

Benefits: tests can swap `fstest.MapFS`; `embed.FS` implements `fs.FS`.



## embed

```
import "embed"

//go:embed static/*
var staticFS embed.FS

//go:embed VERSION
var version string

http.Handle("/static/", http.FileServer(http.FS(staticFS)))
```

Rules:

- Paths are relative to the source file.
- No .. in embed patterns.
- Embedded content is read-only at runtime.

See also [codegen](#) / [embed](#).

## Runnable example

```
go mod init example
go run .

package main

import (
 "fmt"
 "io/fs"
 "os"
 "path/filepath"
 "testing/fstest"
)

func main() {
 dir, err := os.MkdirTemp("", "stdlib-fs-*)
 if err != nil {
 panic(err)
 }
 defer os.RemoveAll(dir)

 path := filepath.Join(dir, "note.txt")
 if err := os.WriteFile(path, []byte("hello fs\n"), 0o644); err != nil {
```

```

 panic(err)
 }

 data, _ := os.ReadFile(path)
 fmt.Printf("disk=%q\n", data)

 // Walk
 _ = filepath.WalkDir(dir, func(p string, d fs.DirEntry, err error) error {
 if err != nil {
 return err
 }
 rel, _ := filepath.Rel(dir, p)
 fmt.Println("walk:", rel, "dir?", d.IsDir())
 return nil
 })

 // Abstract FS (great for tests)
 mapFS := fstest.MapFS{
 "a.txt": {Data: []byte("A")},
 "b/c.txt": {Data: []byte("C")},
 }
 b, _ := fs.ReadFile(mapFS, "b/c.txt")
 fmt.Printf("mapfs=%q\n", b)

 fmt.Println("join:", filepath.Join("var", "log", "app.log"))
 fmt.Println("clean:", filepath.Clean("a//b/../c"))
}

```

**Expected output** (temp path names vary):

```

disk="hello fs\n"
walk: . dir? true
walk: note.txt dir? false
mapfs="C"
join: var/log/app.log
clean: a/c

```

**What to notice:** - DirFS / MapFS / embed.FS share one fs.FS API. - WalkDir reports the root as "." with IsDir()==true. - Cleaning alone does not make a path safe relative to a root — join + prefix check.

**Try next:** Serve embed.FS with http.FileServer(http.FS(...)) and verify a missing file returns 404.

**encoding: JSON, CSV, XML, base64**

# encoding: JSON, CSV, XML, base64

## Overview

The `encoding/*` packages turn Go values into bytes and back. JSON dominates APIs; CSV dominates data interchange; base64/hex appear in tokens and digests.

Package	Common use
<code>encoding/json</code>	HTTP APIs, config (v1 API; v2 engine in Go 1.27+)
<code>encoding/json/v2</code>	New JSON API: options, stricter UTF-8/duplicates
<code>encoding/csv</code>	Exports, bulk import
<code>encoding/xml</code>	Legacy integrations
<code>encoding/base64</code>	Tokens, data URLs
<code>encoding/hex</code>	Digests, debug
<code>encoding/gob</code>	Go-only RPC caches (avoid for public APIs)

## JSON

### Struct tags

```
type User struct {
 ID int64 `json:"id"`
 Name string `json:"name"`
 Email string `json:"email,omitempty"`
 Internal string `json:"- "` // never serialized
}
```

### Encode / decode

```
b, err := json.Marshal(user)
err = json.Unmarshal(b, &user)

// Streams (preferred for large or HTTP bodies)
enc := json.NewEncoder(w)
enc.SetIndent("", " ")
err = enc.Encode(user) // adds trailing newline
```

```
dec := json.NewDecoder(r)
dec.DisallowUnknownFields() // strict APIs
err = dec.Decode(&user)
```

## Useful options

```
dec.UseNumber() // keep numbers as json.Number
```

## Dynamic JSON

Prefer typed structs. When needed:

```
var m map[string]any
json.Unmarshal(b, &m)
```

## Custom marshalers

```
func (id UserID) MarshalText() ([]byte, error) { /* ... */ }
func (id *UserID) UnmarshalText(b []byte) error { /* ... */ }
```

Or MarshalJSON / UnmarshalJSON for full control.

## encoding/json/v2 (Go 1.27+)

v1 remains supported. v2 is the API you want for **new** strict JSON:

```
import jsonv2 "encoding/json/v2"

b, err := jsonv2.Marshal(user)
err = jsonv2.Unmarshal(b, &user, jsonv2.RejectUnknownMembers(true))
err = jsonv2.UnmarshalRead(r, &user)
err = jsonv2.MarshalWrite(w, user)
```

v2 rejects invalid UTF-8 and duplicate object names by default, and matches struct fields case-sensitively. The v1 package is now implemented on top of v2, so existing `json.Marshal` / `json.Unmarshal` calls get the faster unmarshal path without an import change. Compatibility problems: `GOEXPERIMENT=nojsonv2`.

## CSV

```
r := csv.NewReader(f)
r.FieldsPerRecord = 3
records, err := r.ReadAll()

w := csv.NewWriter(f)
_ = w.Write([]string{"name", "age"})
_ = w.Write([]string{"ada", "36"})
w.Flush()
return w.Error()
```

Always check `w.Error()` after `Flush`.

## XML (when you must)

```
type Note struct {
 XMLName xml.Name `xml:"note"`
 To string `xml:"to"`
 Body string `xml:",chardata"`
}
b, err := xml.MarshalIndent(note, "", " ")
```

## base64 and hex

```
s := base64.StdEncoding.EncodeToString(raw)
raw, err = base64.StdEncoding.DecodeString(s)
s = base64.RawURLEncoding.EncodeToString(raw) // JWT-style

h := hex.EncodeToString(sum[:])
```

## Error handling pattern

```
if err := json.NewDecoder(r).Decode(&in); err != nil {
 var syn *json.SyntaxError
 var typeErr *json.UnmarshalTypeError
 switch {
 case errors.As(err, &syn):
 return fmt.Errorf("json syntax at %d: %w", syn.Offset, err)
 case errors.As(err, &typeErr):
```

```

 return fmt.Errorf("json field %s: %w", typeErr.Field, err)
 default:
 return fmt.Errorf("json decode: %w", err)
 }
}

```

## Runnable example

```

go mod init example
go run .

```

```

package main

import (
 "bytes"
 "encoding/base64"
 "encoding/csv"
 "encoding/hex"
 "encoding/json"
 "fmt"
 "strings"
)

type Item struct {
 SKU string `json:"sku"`
 Qty int `json:"qty"`
 Note string `json:"note,omitempty"`
}

func main() {
 it := Item{SKU: "ABC", Qty: 2, Note: "rush"}
 var buf bytes.Buffer
 enc := json.NewEncoder(&buf)
 enc.SetEscapeHTML(false)
 _ = enc.Encode(it)
 fmt.Print("json:", buf.String())

 var back Item
 _ = json.NewDecoder(strings.NewReader(buf.String())).Decode(&back)
 fmt.Printf("decoded=%+v\n", back)

 var cbuf strings.Builder
 cw := csv.NewWriter(&cbuf)
 _ = cw.Write([]string{"sku", "qty"})
}

```

```

_ = cw.Write([]string{it.SKU, fmt.Sprintf("%d", it.Qty)})
cw.Flush()
fmt.Print("csv:\n", cbuf.String())

raw := []byte("hello")
fmt.Println("b64:", base64.StdEncoding.EncodeToString(raw))
fmt.Println("hex:", hex.EncodeToString(raw))
}

```

### Expected output:

```

json:{"sku":"ABC","qty":2,"note":"rush"}
decoded={SKU:ABC Qty:2 Note:rush}
csv:
sku,qty
ABC,2
b64: aGVsbG8=
hex: 68656c6c6f

```

**What to notice:** - Encode adds a newline; handy for NDJSON-style logs/APIs. - CSV writer buffers — Flush + Error are mandatory. - Pick base64 **variant** deliberately (std vs URL vs raw).

**Try next:** Decode JSON with `DisallowUnknownFields` and show the error when an extra field is present.



**net/http Standard Library**

# net/http Standard Library

## Overview

net/http is a complete HTTP client and server. Since Go 1.22, the standard multiplexer supports **method-aware patterns** (GET /items/{id}), which covers most API routing without a framework.

Resilience deep dive: [HTTP client/server resilience](#).

## Server (Go 1.22+ patterns)

```
mux := http.NewServeMux()
mux.HandleFunc("GET /health", func(w http.ResponseWriter, r *http.Request) {
 w.WriteHeader(http.StatusOK)
 _, _ = w.Write([]byte("ok"))
})
mux.HandleFunc("GET /items/{id}", func(w http.ResponseWriter, r *http.Request) {
 id := r.PathValue("id")
 fmt.Fprintln(w, id)
})
mux.HandleFunc("POST /items", createItem)

srv := &http.Server{
 Addr: ":8080",
 Handler: mux,
 ReadHeaderTimeout: 5 * time.Second,
 ReadTimeout: 15 * time.Second,
 WriteTimeout: 30 * time.Second,
 IdleTimeout: 60 * time.Second,
}
log.Fatal(srv.ListenAndServe())
```

Always set timeouts. Defaults can hang forever under slow clients.

## Middleware pattern

```
func withRequestID(next http.Handler) http.Handler {
 return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
 id := r.Header.Get("X-Request-ID")
 if id == "" {
 id = strconv.FormatInt(time.Now().UnixNano(), 36)
 }
 w.Header().Set("X-Request-ID", id)
 ctx := context.WithValue(r.Context(), ctxKeyID, id)
 next.ServeHTTP(w, r.WithContext(ctx))
 })
}
```

## Client

```
client := &http.Client{
 Timeout: 10 * time.Second,
 Transport: &http.Transport{
 Proxy: http.ProxyFromEnvironment,
 DialContext: (&net.Dialer{Timeout: 3 * time.Second}).DialContext,
 MaxIdleConns: 100,
 IdleConnTimeout: 90 * time.Second,
 TLSHandshakeTimeout: 5 * time.Second,
 ForceAttemptHTTP2: true,
 },
}

req, err := http.NewRequestWithContext(ctx, http.MethodGet, url, nil)
req.Header.Set("Accept", "application/json")
resp, err := client.Do(req)
if err != nil {
 return err
}
defer resp.Body.Close()
body, err := io.ReadAll(io.LimitReader(resp.Body, 1<<20))
```

Never use the bare package-level `http.Get` in production code without timeouts — it uses `http.DefaultClient` with no overall timeout.

## JSON helpers

```
func writeJSON(w http.ResponseWriter, status int, v any) {
 w.Header().Set("Content-Type", "application/json; charset=utf-8")
 w.WriteHeader(status)
 _ = json.NewEncoder(w).Encode(v)
}

func readJSON(r *http.Request, dst any) error {
 defer r.Body.Close()
 dec := json.NewDecoder(io.LimitReader(r.Body, 1<<20))
 dec.DisallowUnknownFields()
 return dec.Decode(dst)
}
```

## net/url

```
u, err := url.Parse("https://example.com/search?q=go+lang")
q := u.Query()
q.Set("page", "2")
u.RawQuery = q.Encode()
```

## httptest (for unit tests)

```
req := httptest.NewRequest(http.MethodGet, "/health", nil)
rr := httptest.NewRecorder()
mux.ServeHTTP(rr, req)
if rr.Code != 200 { t.Fatal(rr.Code) }
```

## Runnable example

```
go mod init example
go run .
```

```
package main

import (
 "encoding/json"
 "fmt"
```

```

 "io"
 "net/http"
 "net/http/httptest"
 "strings"
)

func main() {
 mux := http.NewServeMux()
 mux.HandleFunc("GET /health", func(w http.ResponseWriter, r *http.Request) {
 w.Header().Set("Content-Type", "application/json")
 _ = json.NewEncoder(w).Encode(map[string]string{"status": "ok"})
 })
 mux.HandleFunc("GET /echo/{msg}", func(w http.ResponseWriter, r *http.Request) {
 fmt.Fprint(w, r.PathValue("msg"))
 })
 mux.HandleFunc("POST /sum", func(w http.ResponseWriter, r *http.Request) {
 var in struct{ A, B int }
 if err := json.NewDecoder(io.LimitReader(r.Body, 4096)).Decode(&in); err != nil {
 http.Error(w, "bad json", http.StatusBadRequest)
 return
 }
 _ = json.NewEncoder(w).Encode(map[string]int{"sum": in.A + in.B})
 })

 // health
 req := httptest.NewRequest(http.MethodGet, "/health", nil)
 rr := httptest.NewRecorder()
 mux.ServeHTTP(rr, req)
 fmt.Println("health", rr.Code, strings.TrimSpace(rr.Body.String()))

 // path value
 req = httptest.NewRequest(http.MethodGet, "/echo/hi", nil)
 rr = httptest.NewRecorder()
 mux.ServeHTTP(rr, req)
 fmt.Println("echo", rr.Body.String())

 // POST JSON
 body := strings.NewReader(`{"A":2,"B":40}`)
 req = httptest.NewRequest(http.MethodPost, "/sum", body)
 req.Header.Set("Content-Type", "application/json")
 rr = httptest.NewRecorder()
 mux.ServeHTTP(rr, req)
 fmt.Println("sum", strings.TrimSpace(rr.Body.String()))
}

```

**Expected output:**

```
health 200 {"status":"ok"}
echo hi
sum {"sum":42}
```

**What to notice:** - Method + path patterns keep routing in the `stdlib`. - `httptest` exercises handlers without binding a port. - Body decoding should always be size-limited.

**Try next:** Add middleware that rejects requests missing `Authorization` with 401, and test it with `httptest`.

**flag, os/exec, and os/signal**

# flag, os/exec, and os/signal

## Overview

Most Go programs start as **tools**: parse flags, run work, maybe spawn subprocesses, and shut down cleanly on SIGINT/SIGTERM.

Package	Role
flag	CLI flags and usage
os/exec	Subprocesses
os/signal	Graceful shutdown signals

## flag

```
var (
 addr = flag.String("addr", ":8080", "listen address")
 n = flag.Int("n", 1, "repeat count")
 v = flag.Bool("v", false, "verbose")
)
flag.Parse()
args := flag.Args() // non-flag args
```

## Custom FlagSet (subcommands)

```
fs := flag.NewFlagSet("greet", flag.ExitOnError)
name := fs.String("name", "world", "who to greet")
_ = fs.Parse(os.Args[2:])
```

## Good CLI hygiene

- Provide defaults and help text.
- Exit non-zero on misuse (flag.ExitOnError or manual os.Exit(2)).
- Prefer explicit flags over positional soup for optional config.



```

flag.Usage = func() {
 fmt.Fprintf(os.Stderr, "usage: %s [flags] <path>\n", filepath.Base(os.Args[0]))
 flag.PrintDefaults()
}

```

## os/exec

```

cmd := exec.CommandContext(ctx, "git", "status", "--short")
cmd.Dir = repo
cmd.Env = append(os.Environ(), "LC_ALL=C")
out, err := cmd.Output() // stdout; err may be *exec.ExitError

```

## Stream pipes

```

cmd := exec.Command("grep", "TODO")
cmd.Stdin = strings.NewReader(src)
var stdout, stderr bytes.Buffer
cmd.Stdout = &stdout
cmd.Stderr = &stderr
err := cmd.Run()

```

## Rules

1. Prefer `CommandContext` so `cancel` kills the process tree (platform nuances apply).
2. Never build shell command strings with untrusted input — pass `argv` slices.
3. Check exit codes via `errors.As(err, &exitErr)`.

```

if err := cmd.Run(); err != nil {
 var ee *exec.ExitError
 if errors.As(err, &ee) {
 return fmt.Errorf("exit %d: %w", ee.ExitCode(), err)
 }
 return err
}

```

## os/signal and graceful shutdown

```
ctx, stop := signal.NotifyContext(context.Background(), os.Interrupt, syscall.SIGTERM)
defer stop()

go func() {
 <-ctx.Done()
 shutdownCtx, cancel := context.WithTimeout(context.Background(), 10*time.Second)
 defer cancel()
 _ = srv.Shutdown(shutdownCtx)
}()
```

`signal.NotifyContext` (Go 1.16+) is the clean default for servers and long CLIs.

## Runnable example

```
go mod init example
go run . -n 2 hello
```

```
package main

import (
 "bytes"
 "context"
 "flag"
 "fmt"
 "os"
 "os/exec"
 "time"
)

func main() {
 n := flag.Int("n", 1, "repeat")
 flag.Parse()
 msg := "world"
 if flag.NArg() > 0 {
 msg = flag.Arg(0)
 }
 for i := 0; i < *n; i++ {
 fmt.Println("hello", msg)
 }

 ctx, cancel := context.WithTimeout(context.Background(), 2*time.Second)
 defer cancel()
```

```

 cmd := exec.CommandContext(ctx, "echo", "from-subprocess")
 var buf bytes.Buffer
 cmd.Stdout = &buf
 if err := cmd.Run(); err != nil {
 fmt.Fprintln(os.Stderr, "exec:", err)
 os.Exit(1)
 }
 fmt.Print("exec out:", buf.String())
}

```

### Expected output:

```

hello hello
hello hello
exec out:from-subprocess

```

(With `-n 2 hello` you get two greeting lines.)

**What to notice:** - `flag.Arg` / `Args` are only valid after `Parse`. - `CommandContext` ties subprocess lifetime to a deadline. - Real services combine `NotifyContext` with `http.Server.Shutdown`.

**Try next:** Add a timeout duration flag and pass it into `context.WithTimeout` around a long `exec.Command`.

## **time and context Recipes**

# time and context Recipes

## Overview

`time` measures and schedules; `context` propagates **deadlines** and **cancel** across API boundaries. Together they implement request budgets.

Language-oriented time intro: [Time](#). Concurrency-focused context: [Context](#).

## time essentials

```
now := time.Now()
t := time.Date(2026, 8, 3, 12, 0, 0, 0, time.UTC)
d := 1500 * time.Millisecond
d, err := time.ParseDuration("1h30m")
```

## Format and parse

Go's reference time is Mon Jan 2 15:04:05 MST 2006 (layout by example):

```
const layout = time.RFC3339
s := now.UTC().Format(layout)
t, err := time.Parse(layout, s)
```

Prefer RFC3339 / RFC3339Nano for APIs.

## Timers and tickers

```
timer := time.NewTimer(2 * time.Second)
defer timer.Stop()
select {
case <-timer.C:
case <-ctx.Done():
 if !timer.Stop() {
 <-timer.C // drain if already fired (pre-1.23 habit; see release notes)
 }
}
```

```

ticker := time.NewTicker(time.Second)
defer ticker.Stop()
for {
 select {
 case <-ctx.Done():
 return ctx.Err()
 case t := <-ticker.C:
 _ = t
 }
}

```

Always Stop tickers to avoid leaks.

## context essentials

```

ctx := context.Background()
ctx, cancel := context.WithCancel(ctx)
defer cancel()

ctx, cancel = context.WithTimeout(ctx, 3*time.Second)
defer cancel()

ctx, cancel = context.WithDeadline(ctx, time.Now().Add(time.Second))
defer cancel()

```

## Values (use sparingly)

```

type key int
const requestID key = 1
ctx = context.WithValue(ctx, requestID, "abc")
id, _ := ctx.Value(requestID).(string)

```

Store request-scoped metadata (IDs), not optional parameters that belong in function args.

## Waiting on cancel

```

select {
case <-ctx.Done():
 return ctx.Err() // context.Canceled or DeadlineExceeded
case res := <-results:
 return res, nil
}

```

## Recipes

### Bound a function

```
func fetch(ctx context.Context, url string) ([]byte, error) {
 req, err := http.NewRequestWithContext(ctx, http.MethodGet, url, nil)
 if err != nil {
 return nil, err
 }
 resp, err := http.DefaultClient.Do(req)
 // ...
}
```

### Remaining budget

```
if dl, ok := ctx.Deadline(); ok {
 remaining := time.Until(dl)
 if remaining < 50*time.Millisecond {
 return errBudgetExhausted
 }
}
```

### Sleep that respects cancel

```
func sleep(ctx context.Context, d time.Duration) error {
 t := time.NewTimer(d)
 defer t.Stop()
 select {
 case <-ctx.Done():
 return ctx.Err()
 case <-t.C:
 return nil
 }
}
```

### Runnable example

```
go mod init example
go run .
```

```

package main

import (
 "context"
 "fmt"
 "time"
)

func work(ctx context.Context, name string, d time.Duration) error {
 select {
 case <-time.After(d):
 fmt.Println(name, "done")
 return nil
 case <-ctx.Done():
 fmt.Println(name, "canceled:", ctx.Err())
 return ctx.Err()
 }
}

func main() {
 now := time.Now().UTC()
 fmt.Println("rfc3339:", now.Format(time.RFC3339))
 d, _ := time.ParseDuration("250ms")
 fmt.Println("duration:", d)

 ctx, cancel := context.WithTimeout(context.Background(), 200*time.Millisecond)
 defer cancel()

 _ = work(ctx, "slow", 500*time.Millisecond)

 ctx2, cancel2 := context.WithTimeout(context.Background(), time.Second)
 defer cancel2()
 _ = work(ctx2, "fast", 50*time.Millisecond)

 if dl, ok := ctx2.Deadline(); ok {
 fmt.Println("remaining>0?", time.Until(dl) > 0)
 }
}

```

### Expected output:

```

rfc3339: 2026-...
duration: 250ms
slow canceled: context deadline exceeded
fast done
remaining>0? true

```

**What to notice:** - Timeout cancels waiters blocked in `select` on `ctx.Done()`. - Always defer



`cancel()` to free timer resources even on success. - Format APIs with RFC3339 unless you have a domain-specific layout.

**Try next:** Nest a child `WithTimeout` shorter than the parent and observe which deadline fires first.

**sync and sync/atomic**

# sync and sync/atomic

## Overview

When goroutines share memory, the stdlib gives two complementary tools:

Package	Style
sync	Mutexes, WaitGroups, Once, Cond, Map, Pool
sync/atomic	Lock-free counters and pointers for simple cases

Prefer **channels for ownership transfer** and **mutexes for shared structures**. Concurrency design: [Part 07](#).

## WaitGroup

```
var wg sync.WaitGroup
for _, job := range jobs {
 wg.Add(1)
 go func(j Job) {
 defer wg.Done()
 process(j)
 }(job)
}
wg.Wait()
```

Call `Add` before spawning; never negative. Prefer `wg.Go` (Go 1.25+) when available:

```
wg.Go(func() { process(job) })
```

## Mutex and RWMutex

```
type Cache struct {
 mu sync.Mutex
 m map[string]string
}
```

```
func (c *Cache) Get(k string) (string, bool) {
 c.mu.Lock()
 defer c.mu.Unlock()
 v, ok := c.m[k]
 return v, ok
}
```

RWMutex helps read-heavy maps; measure before assuming it wins.

## Once

```
var once sync.Once
var client *http.Client

func Client() *http.Client {
 once.Do(func() {
 client = &http.Client{Timeout: 10 * time.Second}
 })
 return client
}
```

## Pool

```
var bufPool = sync.Pool{
 New: func() any { return make([]byte, 32*1024) },
}
b := bufPool.Get().([]byte)
defer bufPool.Put(b[:cap(b)]) // reset length if you resliced
```

Use for high-churn buffers; do not assume pooled items are zeroed.

## Map

`sync.Map` is specialized (append-only keys, many disjoint keys). For ordinary caches, a mutex + map is clearer.

## Cond (advanced)

```
mu := sync.Mutex{}
cond := sync.NewCond(&mu)
// Waiters: cond.Wait(); Signal/Broadcast after state change under the same mutex
```

See [pipelines / Cond](#).

## atomic

```
var hits atomic.Int64
hits.Add(1)
fmt.Println(hits.Load())

var flag atomic.Bool
flag.Store(true)
```

Typed atomics (`atomic.Int64`, `Bool`, `Pointer[T]`) are preferred over `AddInt64` free functions.

## Compare-and-swap

```
var p atomic.Pointer[Config]
p.Store(&Config{V: 1})
for {
 old := p.Load()
 next := *old
 next.V++
 if p.CompareAndSwap(old, &next) {
 break
 }
}
```

## Race detector

```
go test -race ./...
go run -race .
```

If the race detector complains, fix the ownership story — do not “hope” atomics alone fix complex structs.

## Runnable example

```
go mod init example
go run .

package main

import (
 "fmt"
 "sync"
 "sync/atomic"
)

func main() {
 var hits atomic.Int64
 var wg sync.WaitGroup
 var mu sync.Mutex
 seen := make(map[int]bool)

 for i := 0; i < 8; i++ {
 wg.Add(1)
 go func(id int) {
 defer wg.Done()
 hits.Add(1)
 mu.Lock()
 seen[id] = true
 mu.Unlock()
 }(i)
 }
 wg.Wait()

 fmt.Println("hits", hits.Load())
 fmt.Println("unique", len(seen))

 var once sync.Once
 once.Do(func() { fmt.Println("init once") })
 once.Do(func() { fmt.Println("never again") })
}
```

### Expected output:

```
hits 8
unique 8
init once
```

**What to notice:** - Counters can be atomic; maps still need a mutex (or channels). - `Once` runs exactly one successful init path. - Run with `-race` to validate this pattern under the detector.

**Try next:** Replace the mutex map with a channel that owns the map in a single goroutine (share-by-communication).

**slices, maps, cmp, and iter**



# slices, maps, cmp, and iter

## Overview

Go 1.21+ added first-class helpers for common collection work. Go 1.23 popularized **range-over-func** iterators via package `iter`.

Package	Role
<code>slices</code>	Search, sort, clone, compact, delete
<code>maps</code>	Keys/values clone, equal, copy
<code>cmp</code>	Ordered comparisons for sort helpers
<code>iter</code>	Seq / Seq2 iterator types

Basics of ranging: [Loops](#).

## slices

```
import "slices"

s := []int{3, 1, 2}
slices.Sort(s) // [1 2 3]
slices.SortFunc(s, cmp.Compare) // same for ordered types
i, ok := slices.BinarySearch(s, 2)
s2 := slices.Clone(s)
s = slices.Delete(s, 1, 2) // remove s[1]
s = slices.Compact(s) // adjacent dups (sorted first)
max := slices.Max(s)
```

## Useful helpers

Func	Notes
<code>Contains</code> / <code>Index</code>	Linear search
<code>Compact</code> / <code>CompactFunc</code>	Needs sorted input for uniqueness
<code>Grow</code> / <code>Clip</code>	Capacity management
<code>Replace</code>	Splice
<code>Collect</code>	From <code>iter.Seq</code> (Go 1.23+)

```
s = slices.DeleteFunc(s, func(n int) bool { return n%2 == 0 })
```

## maps

```
import "maps"

a := map[string]int{"x": 1, "y": 2}
b := maps.Clone(a)
maps.Copy(b, map[string]int{"y": 9, "z": 3})
eq := maps.Equal(a, b)

for k := range maps.Keys(a) { // Go 1.23+ iterators
 _ = k
}
```

`maps.Keys` / `Values` return iterators — collect with `slices.Collect` if you need a slice.

## cmp

```
import "cmp"

cmp.Compare(a, b) // -1, 0, 1 for ordered types
cmp.Or(x, y, z) // first non-zero (useful defaults)
cmp.Less(a, b)

slices.SortFunc(people, func(a, b Person) int {
 if c := cmp.Compare(a.Last, b.Last); c != 0 {
 return c
 }
 return cmp.Compare(a.First, b.First)
})
```

## iter and range-over-func

```
import "iter"

func Backward[E any](s []E) iter.Seq[E] {
 return func(yield func(E) bool) {
 for i := len(s) - 1; i >= 0; i-- {
 if !yield(s[i]) {
 return
 }
 }
 }
}
```

```

 return
 }
}

for v := range Backward([]string{"a", "b", "c"}) {
 fmt.Println(v)
}

```

Pull-style when you need manual advancement:

```

next, stop := iter.Pull(Backward([]int{1, 2, 3}))
defer stop()
for {
 v, ok := next()
 if !ok {
 break
 }
 fmt.Println(v)
}

```

## When not to use them

- Tiny one-off loops are fine as plain `for`.
- Do not force iterators everywhere — they shine for **lazy** pipelines and library APIs.
- Sorting huge slices of structs: consider indices or `sort.Slice` patterns if allocations dominate (benchmark).

## Runnable example

```

go mod init example
go run .

```

```

package main

import (
 "cmp"
 "fmt"
 "iter"
 "maps"
 "slices"
)

```

```

func Countdown(n int) iter.Seq[int] {
 return func(yield func(int) bool) {
 for i := n; i >= 0; i-- {
 if !yield(i) {
 return
 }
 }
 }
}

func main() {
 s := []int{5, 1, 5, 3, 2}
 slices.Sort(s)
 fmt.Println("sorted", s)
 fmt.Println("max", slices.Max(s))
 s = slices.Compact(s)
 fmt.Println("compact", s)

 type pair struct{ K string; V int }
 ps := []pair{{"b", 2}, {"a", 1}, {"a", 0}}
 slices.SortFunc(ps, func(a, b pair) int {
 if c := cmp.Compare(a.K, b.K); c != 0 {
 return c
 }
 return cmp.Compare(a.V, b.V)
 })
 fmt.Println("pairs", ps)

 m := map[string]int{"x": 1, "y": 2}
 fmt.Println("clone equal?", maps.Equal(m, maps.Clone(m)))

 fmt.Print("countdown:")
 for v := range Countdown(3) {
 fmt.Print(" ", v)
 }
 fmt.Println()

 collected := slices.Collect(Countdown(2))
 fmt.Println("collect", collected)
}

```

### Expected output:

```

sorted [1 2 3 5 5]
max 5
compact [1 2 3 5]

```

```
pairs [{a 0} {a 1} {b 2}]
clone equal? true
countdown: 3 2 1 0
collect [2 1 0]
```

**What to notice:** - `Compact` only removes **adjacent** duplicates — sort first for set-like uniqueness.  
- `cmp.Compare` composes clean multi-key sorts. - `slices.Collect` materializes a lazy `iter.Seq`.

**Try next:** Write `Filter[T any](s []T, ok func(T) bool) iter.Seq[T]` and range over it without allocating an intermediate slice.

## **testing Package: Examples, Fuzz, Bench**

# testing Package: Examples, Fuzz, Bench

## Overview

Package `testing` is both a runner and a small toolkit: tests, benchmarks, examples, fuzz targets, temp dirs, and HTTP helpers via `net/http/httptest`.

Fundamentals also live in [Part 06](#). This chapter is the `stdlib` surface checklist.

## Test functions

```
func TestSum(t *testing.T) {
 t.Parallel()
 got := Sum(2, 3)
 if got != 5 {
 t.Fatalf("Sum(2,3)=%d want 5", got)
 }
}
```

Method	Use
Error / Errorf	Fail, continue
Fatal / Fatalf	Fail, stop this test
Helper	Mark assertion helpers
Cleanup	Register teardown (LIFO)
TempDir	Isolated temp directory
Setenv	Env var for this test
Context	Canceled when test ends (Go 1.24+)
Deadline	Time left if <code>-timeout</code> set

## Table tests

```
tests := []struct {
 name string
 a, b int
 want int
}{
 {"zero", 0, 0, 0},
```

```

 {"pos", 2, 3, 5},
}
for _, tt := range tests {
 t.Run(tt.name, func(t *testing.T) {
 if got := Sum(tt.a, tt.b); got != tt.want {
 t.Fatalf("got %d want %d", got, tt.want)
 }
 })
}

```

## Examples

```

func ExampleSum() {
 fmt.Println(Sum(1, 2))
 // Output:
 // 3
}

```

Examples run as tests and appear in `pkg.go.dev` docs. Unordered output:

```

// Unordered output:
// a
// b

```

## Benchmarks

```

func BenchmarkSum(b *testing.B) {
 for b.Loop() { // Go 1.24+; or for i := 0; i < b.N; i++
 Sum(2, 3)
 }
}

```

```

go test -bench=BenchmarkSum -benchmem ./...

```

## Fuzzing

```

func FuzzReverse(f *testing.F) {
 f.Add("gopher")
 f.Fuzz(func(t *testing.T, s string) {
 rev := Reverse(s)
 })
}

```



```

 dbl := Reverse(rev)
 if dbl != s {
 t.Fatalf("roundtrip %q -> %q -> %q", s, rev, dbl)
 }
 })
}

go test -fuzz=FuzzReverse -fuzztime=10s

```

Failing inputs land under `testdata/fuzz/` — commit them as regression seeds.

## fstest and httptest

```

fsys := fstest.MapFS{"a.txt": {Data: []byte("x")}}
b, err := fs.ReadFile(fsys, "a.txt")

req := httptest.NewRequest(http.MethodGet, "/x", nil)
rr := httptest.NewRecorder()
handler.ServeHTTP(rr, req)

// Go 1.27+: in-memory server (synctest-safe, auto-cleanup)
ts := httptest.NewTestServer(t, handler)
resp, err := ts.Client().Get("http://example.com/x")

```

## Commands

```

go test ./...
go test -v -run TestSum
go test -race ./...
go test -coverprofile=c.out && go tool cover -html=c.out
go test -bench=. -benchmem
go test -fuzz=. -fuzztime=30s

```

## Runnable example

Save as `sum.go` and `sum_test.go` in a module:

```

mkdir /tmp/stdlib-test && cd /tmp/stdlib-test
go mod init example

```

`sum.go`:

```

package example

func Sum(a, b int) int { return a + b }

func Reverse(s string) string {
 r := []rune(s)
 for i, j := 0, len(r)-1; i < j; i, j = i+1, j-1 {
 r[i], r[j] = r[j], r[i]
 }
 return string(r)
}

```

sum\_test.go:

```

package example

import (
 "fmt"
 "testing"
)

func TestSum(t *testing.T) {
 if Sum(2, 3) != 5 {
 t.Fatal("bad sum")
 }
}

func ExampleSum() {
 fmt.Println(Sum(1, 2))
 // Output:
 // 3
}

func FuzzReverse(f *testing.F) {
 f.Add("ab")
 f.Fuzz(func(t *testing.T, s string) {
 if Reverse(Reverse(s)) != s {
 t.Fatal("roundtrip")
 }
 })
}

func BenchmarkSum(b *testing.B) {
 for i := 0; i < b.N; i++ {
 Sum(i, i)
 }
}

```

```
go test -v
go test -bench=BenchmarkSum -benchmem
go test -fuzz=FuzzReverse -fuzztime=3s
```

**What to notice:** - Examples are documentation that cannot drift silently. - Fuzz finds encoding edge cases (`[]rune` vs `bytes`) faster than hand-written tables. - Benchmarks need stable loops; avoid timing setup inside `b.N` work.

**Try next:** Use `t.TempDir` and `os.WriteFile` to test a function that reads a config path.

**crypto, hash, rand, and regexp**

# crypto, hash, rand, and regexp

## Overview

Last stdlib essentials for **integrity, randomness, and text matching**.

Package	Use
crypto/sha256 (etc.)	Content hashes, integrity
crypto/hmac	Message authentication
crypto/rand	Secure random bytes
math/rand/v2	Non-secure randomness / sims
crypto/subtle	Constant-time compare
crypto/mldsa	Post-quantum ML-DSA signatures (Go 1.27+; also TLS/x509)
uuid	Generate/parse UUIDs (Go 1.27+; RFC 9562)
regexp	Regular expressions
net/url	URL parse (with HTTP chapter)

TLS deep dive: [Security hardening](#).

## Hashing

```
sum := sha256.Sum256([]byte("hello"))
fmt.Printf("%x\n", sum)

h := sha256.New()
io.Copy(h, r)
digest := h.Sum(nil)
```

Streaming hash large inputs — do not `ReadAll` multi-GB files first.

## HMAC

```
mac := hmac.New(sha256.New, key)
mac.Write(msg)
sig := mac.Sum(nil)
if !hmac.Equal(sig, expected) {
 return errUnauthorized
}
```

Use `hmac.Equal` / `subtle.ConstantTimeCompare` — not `==` on strings for secrets.

## Randomness

```
// Secure tokens / keys
b := make([]byte, 16)
if _, err := rand.Read(b); err != nil { // crypto/rand
 return err
}
token := hex.EncodeToString(b)

// Simulations, randomized tests (not secrets)
n := rand.IntN(100) // math/rand/v2
```

Never use `math/rand` for session IDs, API keys, or CSRF tokens.

## uuid (Go 1.27+)

Stdlib `uuid` covers the common cases that used to need [github.com/google/uuid](https://github.com/google/uuid):

```
import "uuid"

id := uuid.New() // currently v4
id = uuid.NewV7() // time-ordered; good for DB keys
s := id.String()
parsed, err := uuid.Parse(s)
```

`New()` is the default; use `NewV7()` when insertion order / index locality matters. Random bits come from a cryptographically secure generator.

Go 1.27 also adds `crypto/mldsa` (FIPS 204) with `crypto/x509` and `crypto/tls` integration for post-quantum signatures. Treat it as an opt-in for new PKI work, not a drop-in replacement for existing ECDSA certs.

## regexp

```
re := regexp.MustCompile(`(?i)^[a-z0-9._%+\-]+@[a-z0-9.\-]+\.[a-z]{2,}$`)
ok := re.MatchString(email)
parts := re.FindStringSubmatch(s)
```

## Performance tips

1. Compile once (`var re = regexp.MustCompile(...)` at package scope).
2. Prefer `strings` / `bytes` for fixed-prefix checks.
3. Beware catastrophic backtracking on untrusted patterns/input — keep patterns simple.
4. Use `regexp.Compile` when pattern comes from config; handle errors.

```
const max = 1024
if len(s) > max {
 return false
}
return re.MatchString(s)
```

## Password hashing note

Stdlib does **not** include `bcrypt`/`argon2`. For passwords use [golang.org/x/crypto/bcrypt](https://golang.org/x/crypto/bcrypt) or `argon2` — do not roll `SHA256(password)`.

## Runnable example

```
go mod init example
go run .
```

```
package main

import (
 "crypto/hmac"
 "crypto/rand"
 "crypto/sha256"
 "crypto/subtle"
 "encoding/hex"
 "fmt"
 mrand "math/rand/v2"
 "regexp"
)
```

```

func main() {
 sum := sha256.Sum256([]byte("hello"))
 fmt.Println("sha256", hex.EncodeToString(sum[:])[:16]+"...")

 key := []byte("secret-key")
 msg := []byte("payload")
 mac := hmac.New(sha256.New, key)
 mac.Write(msg)
 sig := mac.Sum(nil)

 mac2 := hmac.New(sha256.New, key)
 mac2.Write(msg)
 fmt.Println("hmac ok?", hmac.Equal(sig, mac2.Sum(nil)))

 // constant-time compare demo
 a := []byte("abcd")
 b := []byte("abce")
 fmt.Println("subtle", subtle.ConstantTimeCompare(a, b) == 1)

 tok := make([]byte, 8)
 _, _ = rand.Read(tok)
 fmt.Println("token", hex.EncodeToString(tok))
 fmt.Println("sim roll", rand.IntN(6)+1)

 re := regexp.MustCompile(`^item-(\d+)$`)
 m := re.FindStringSubmatch("item-42")
 fmt.Println("submatch", m)
}

```

**Expected output** (token varies):

```

sha256 2cf24dba5fb0a30e...
hmac ok? true
subtle false
token a1b2c3d4e5f67890
sim roll 4
submatch [item-42 42]

```

**What to notice:** - HMAC equality is constant-time via `hmac.Equal`. - `crypto/rand` for tokens; `math/rand/v2` for games/sims. - Submatch index 0 is the full match; 1+ are groups.

**Try next:** Stream-hash a file with `sha256.New + io.Copy` and compare to `shasum -a 256` on the command line.



## Stdlib tour wrap-up

You now have a map of the packages that appear in nearly every Go binary. Next steps:

1. Rebuild a small CLI using only parts 980–992.
2. Rebuild a tiny JSON API with timeouts and `slog`.
3. Jump to specialized parts only when stdlib friction is real.

Return to the [stdlib overview](#) checklist and mark what you can do without docs.

**database/sql**

# database/sql

## Overview

`database/sql` is the standard **database access layer**: connection pools, prepared statements, transactions, and context-aware queries. Drivers register themselves; your code stays mostly driver-agnostic.

Deeper pool and production notes: [database/sql pool deep dive](#) and Bookstore use in [web databases](#).

## Mental model

```
sql.DB = pool of connections (safe for concurrent use)
```

```
QueryContext / QueryRowContext / ExecContext
PrepareContext → sql.Stmt
BeginTx → sql.Tx → Commit / Rollback
```

- `sql.DB` is not a single connection. It is a pool.
- `sql.Open` may not dial until the first use — always `PingContext` at startup.
- Pass `context.Context` so cancel/timeout reaches the driver.

## Open, configure, ping

```
import (
 "context"
 "database/sql"
 "time"

 _ "modernc.org/sqlite" // or pgx stdlib, etc.
)

func openDB(dsn string) (*sql.DB, error) {
 db, err := sql.Open("sqlite", dsn)
 if err != nil {
 return nil, err
 }
}
```

```

db.SetMaxOpenConns(10)
db.SetMaxIdleConns(5)
db.SetConnMaxLifetime(time.Hour)
db.SetConnMaxIdleTime(15 * time.Minute)

ctx, cancel := context.WithTimeout(context.Background(), 3*time.Second)
defer cancel()
if err := db.PingContext(ctx); err != nil {
 _ = db.Close()
 return nil, err
}
return db, nil
}

```

Setting	Role
MaxOpenConns	Cap concurrent DB usage
MaxIdleConns	Keep warm connections
ConnMaxLifetime	Recycle to avoid server-side timeouts
ConnMaxIdleTime	Drop idle conns (Go 1.15+)

## Query patterns

### Many rows

```

rows, err := db.QueryContext(ctx,
 `SELECT id, title FROM books WHERE author = ? ORDER BY title`, author)
if err != nil {
 return nil, err
}
defer rows.Close()

var out []Book
for rows.Next() {
 var b Book
 if err := rows.Scan(&b.ID, &b.Title); err != nil {
 return nil, err
 }
 out = append(out, b)
}
return out, rows.Err() // always check rows.Err()

```

## One row

```
var b Book
err := db.QueryRowContext(ctx,
 `SELECT id, title FROM books WHERE id = ?`, id,
).Scan(&b.ID, &b.Title)
if errors.Is(err, sql.ErrNoRows) {
 return Book{}, errNotFound
}
return b, err
```

## Exec (insert/update/delete)

```
res, err := db.ExecContext(ctx,
 `INSERT INTO books (id, title) VALUES (?, ?)`, id, title)
if err != nil {
 return err
}
n, _ := res.RowsAffected()
_ = n
```

Always use placeholders (`?` or `$1` depending on driver). Never concatenate user input into SQL.

## Nullables

Database NULL is not Go zero. Use:

```
var maybeTitle sql.NullString
// Scan into maybeTitle; use maybeTitle.Valid && maybeTitle.String
```

Or pointer fields (`*string`) with some drivers/scanners. Be consistent.

## Transactions

```
tx, err := db.BeginTx(ctx, &sql.TxOptions{Isolation: sql.LevelReadCommitted})
if err != nil {
 return err
}
defer tx.Rollback() // no-op after successful Commit

if _, err := tx.ExecContext(ctx, `UPDATE accounts SET bal = bal - ? WHERE id = ?`, amt, from
```

```

 return err
 }
 if _, err := tx.ExecContext(ctx, `UPDATE accounts SET bal = bal + ? WHERE id = ?`, amt, to);
 return err
 }
 return tx.Commit()

```

Keep transactions **short**. Do not hold a `tx` open across slow external HTTP calls.

## Prepared statements

```

stmt, err := db.PrepareContext(ctx, `SELECT title FROM books WHERE id = ?`)
if err != nil {
 return err
}
defer stmt.Close()

var title string
err = stmt.QueryRowContext(ctx, id).Scan(&title)

```

Prepare when you run the same SQL many times in one process. For one-shot queries, `QueryContext` is fine.

## Errors worth mapping

Condition	Typical HTTP / API mapping
<code>sql.ErrNoRows</code>	Not found
Unique constraint (driver-specific)	Conflict
<code>context.DeadlineExceeded</code>	Timeout / unavailable
Connection refused	Unavailable; log + retry policy

Log the **real** error; return a safe client message.

## Rules of thumb

Do	Don't
* <code>Context</code> methods + timeouts	Bare <code>Query</code> in request handlers without budget
Close <code>rows</code> (defer)	Forget <code>rows.Err()</code> after the loop
Parameterized SQL	String-build queries from user input
One <code>*sql.DB</code> per process/DSN	Open a new DB per request

## Try next

1. Open SQLite in-memory, create a table, insert, list.
2. Force `ErrNoRows` and map it with `errors.Is`.
3. Run two concurrent `QueryContext` calls against one `*sql.DB` (pool is concurrent-safe).

## **net: Dial, Listen, and UDP**



# net: Dial, Listen, and UDP

## Overview

Package `net` is the foundation under `net/http`: TCP/UDP addresses, listeners, dialers, and resolution. When HTTP is too high-level—or you need a custom protocol—you work here.

TCP services deep dive: [131 TCP services](#).

## Addresses and resolution

```
// TCPAddr, UDPAddr, UnixAddr
tcpAddr, err := net.ResolveTCPAddr("tcp", "127.0.0.1:9000")
ips, err := net.LookupIP(ctx, "example.com") // prefer context variants where available
```

Prefer `net.Dialer` over bare `net.Dial` so you can set timeouts and keep-alives.

```
d := net.Dialer{
 Timeout: 3 * time.Second,
 KeepAlive: 30 * time.Second,
}
conn, err := d.DialContext(ctx, "tcp", "127.0.0.1:9000")
```

## TCP server (line protocol sketch)

```
ln, err := net.Listen("tcp", "127.0.0.1:9000")
if err != nil {
 log.Fatal(err)
}
defer ln.Close()

for {
 conn, err := ln.Accept()
 if err != nil {
 log.Println("accept:", err)
 continue
 }
}
```

```

 go handle(conn)
}

func handle(c net.Conn) {
 defer c.Close()
 _ = c.SetDeadline(time.Now().Add(30 * time.Second))
 br := bufio.NewReader(c)
 line, err := br.ReadString('\n')
 if err != nil {
 return
 }
 _, _ = io.WriteString(c, "echo: "+line)
}

```

Habit	Why
Deadline per connection	Avoid stuck clients forever
One goroutine per conn (or limited pool)	Bound resource use
Always Close	Prevent FD leaks
bufio for line protocols	Fewer syscalls

## TCP client

```

conn, err := d.DialContext(ctx, "tcp", addr)
if err != nil {
 return err
}
defer conn.Close()
_ = conn.SetDeadline(time.Now().Add(5 * time.Second))
_, err = io.WriteString(conn, "ping\n")
// read response...

```

## UDP

```

// listen
pc, err := net.ListenPacket("udp", "127.0.0.1:9001")
// or DialUDP for connected UDP

buf := make([]byte, 1500)
n, addr, err := pc.ReadFrom(buf)
_, err = pc.WriteTo(buf[:n], addr) // echo

```

UDP is message-oriented: no streams, no automatic retransmit. Size buffers for your MTU/use case.

## Unix domain sockets

```
ln, err := net.Listen("unix", "/tmp/app.sock")
// client: Dial("unix", "/tmp/app.sock")
```

Useful for local IPC (sidecar, same-host tools). Remember file permissions on the socket path.

## net.Conn as io.Reader / Writer

```
// stream copy with limit
n, err := io.Copy(dst, io.LimitReader(conn, 1<<20))
```

Composition with `io` is the Go style: your protocol parsers should take `io.Reader`/`io.Writer` when possible, not only `net.Conn`.

## Errors and temporary failures

```
var ne net.Error
if errors.As(err, &ne) && ne.Timeout() {
 // retry or return 504-style
}
```

DNS and dial failures are often wrapped—log with `%+v` / `slog` attributes; branch with `errors.As`.

## ip and ipnet helpers

```
ip := net.ParseIP("2001:db8::1")
_, cidr, _ := net.ParseCIDR("10.0.0.0/8")
ok := cidr.Contains(net.ParseIP("10.1.2.3"))
```

Handy for allowlists and basic network policy in tools.

## Rules of thumb

Do	Don't
Dial/Listen with timeouts/deadlines	Block forever on Accept/Read
Propagate <code>context</code> on Dial	Ignore cancel when user hits Ctrl-C
Limit concurrent handlers	Accept unbounded <code>go handle</code> under load without a cap
Close connections	Leak FDs on error paths

## Try next

1. Build a TCP echo server and client with 2s deadlines.
2. Send a UDP packet to yourself and print the peer address.
3. Wrap `handle` with a semaphore channel of size 100 and load-test `Accept`.

**text/template and html/template**

# text/template and html/template

## Overview

The template packages render **text with actions** (`{{...}}`). Use `html/template` for browser HTML (context-aware escaping). Use `text/template` for configs, emails-as-text, code generation, and non-HTML output.

Web-focused usage: [Templating and rendering](#).

## When to use which

Package	Escaping	Use
<code>html/template</code>	Yes (HTML/JS/CSS/URL contexts)	Web pages, HTML emails
<code>text/template</code>	No	Nginx configs, Markdown, codegen, plain text

Serving user data as HTML with `text/template` is an XSS footgun.

## Core loop: parse once, execute many

```
import "html/template"

tmpl, err := template.New("hi").Parse(`Hello, {{.Name}}!`)
if err != nil {
 log.Fatal(err)
}
err = tmpl.Execute(os.Stdout, struct{ Name string }{"Gopher"})
// Hello, Gopher!
```

From files:

```
tmpl, err := template.ParseFiles("layout.tmpl", "page.tmpl")
// or template.ParseFS(fsys, "templates/*.tmpl")
```

## Actions you will use constantly

```
{{.Field}} // print field
{{if .OK}}yes{{else}}no{{end}}
{{range .Items}}{{.}}{{end}}
{{with .User}}{{.Email}}{{end}}
{{/* comment */}}
```

Pipelines:

```
{{.Title | printf "%q" | html}}
```

## Define and execute named templates

```
{{define "layout"}}
<html><body>{{template "content" .}}</body></html>
{{end}}
```

```
{{define "content"}}
<h1>{{.Title}}</h1>
{{end}}
```

```
err := tpl.ExecuteTemplate(w, "layout", data)
```

## FuncMap

```
funcs := template.FuncMap{
 "upper": strings.ToUpper,
 "cents": func(n int) string {
 return fmt.Sprintf("$%.2f", float64(n)/100)
 },
}
tpl, err := template.New("price").Funcs(funcs).Parse(`{{cents .Price}} {{upper .Title}}`)
```

Register **Funcs** before **Parse**. Only expose pure, trusted helpers.

## html/template safety

```
// Auto-escaped:
// {{.Comment}} from user → <script>...

// Escape hatch (trusted HTML only):
type page struct {
 Body template.HTML
}
```

Type	Meaning
template.HTML	Trusted HTML fragment
template.JS	Trusted JS
template.CSS	Trusted CSS
template.URL	Trusted URL

Never cast raw request input to these types.

## Whitespace and formatting

```
{{- /* trim left */ -}}
{{.Name -}} {{/* trim right */}}
```

- next to {{ / }} trims adjacent whitespace—useful for tidy HTML or generated code.

## text/template for codegen

```
const stub = `package {{.Pkg}}

func {{.Name}}() string { return {{printf "%q" .Msg}} }
`

t := template.Must(template.New("stub").Parse(stub))
var buf bytes.Buffer
_ = t.Execute(&buf, struct {
 Pkg, Name, Msg string
}{ "demo", "Hello", "hi" })
// gofmt the result in real tools
```



## Common errors

Error	Cause
complete and no parse result	Empty parse / wrong name
no template "X"	ExecuteTemplate name mismatch
Missing data prints <no value>	Field name wrong or nil pipeline
Func not defined	Funcs registered after Parse

## Rules of thumb

Do	Don't
Parse at startup	Parse templates on every request
html/template for HTML	Build HTML with fmt + user strings
Pass view structs	Pass *sql.DB into templates
template.Must only at init	Must in request paths (panic under load)

## Try next

1. Parse a layout + content from `embed.FS` and execute both HTML and a JSON-free page.
2. Add a `FuncMap` that formats `time.Time` as RFC3339.
3. Intentionally inject `<script>` via data and confirm `html/template` escapes it.

**compress and archive**

# compress and archive

## Overview

The standard library covers everyday compression and packaging: **gzip**, **zlib**, **flate**, **bzip2** (reader), **lzw**, plus **zip** and **tar** archives. Use these for log shipping, backup tools, HTTP Content-Encoding, and release artifacts—without pulling a third-party zipper for common cases.

## Package map

Package	Role
<code>compress/gzip</code>	<code>.gz</code> streams (HTTP, logs)
<code>compress/zlib</code>	zlib wrappers
<code>compress/flate</code>	raw DEFLATE
<code>compress/bzip2</code>	bzip2 <b>reader</b> only
<code>archive/zip</code>	ZIP read/write
<code>archive/tar</code>	tar read/write (often + gzip)

## Gzip: compress a file

```
func gzipFile(src, dst string) error {
 in, err := os.Open(src)
 if err != nil {
 return err
 }
 defer in.Close()

 out, err := os.Create(dst)
 if err != nil {
 return err
 }
 defer out.Close()

 zw := gzip.NewWriter(out)
 zw.Name = filepath.Base(src)
 if _, err := io.Copy(zw, in); err != nil {
```

```

 _ = zw.Close()
 return err
 }
 return zw.Close() // flush trailer; check this error
}

```

## Decompress

```

func gunzip(src io.Reader, dst io.Writer) error {
 zr, err := gzip.NewReader(src)
 if err != nil {
 return err
 }
 defer zr.Close()
 _, err = io.Copy(dst, zr)
 return err
}

```

## HTTP Content-Encoding sketch

```

// response: write gzip when client accepts it
func gzipMiddleware(next http.Handler) http.Handler {
 return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
 if !strings.Contains(r.Header.Get("Accept-Encoding"), "gzip") {
 next.ServeHTTP(w, r)
 return
 }
 w.Header().Set("Content-Encoding", "gzip")
 gz := gzip.NewWriter(w)
 defer gz.Close()
 gzw := gzipResponseWriter{Writer: gz, ResponseWriter: w}
 next.ServeHTTP(gzw, r)
 })
}

```

(Production middleware also handles `Content-Length`, status codes, and `Vary: Accept-Encoding`—keep this as a pattern, not a drop-in.)

## tar + gzip (.tar.gz)

```
func untarGz(r io.Reader, destDir string) error {
 gr, err := gzip.NewReader(r)
 if err != nil {
 return err
 }
 defer gr.Close()
 tr := tar.NewReader(gr)

 for {
 hdr, err := tr.Next()
 if errors.Is(err, io.EOF) {
 return nil
 }
 if err != nil {
 return err
 }
 // Prevent path traversal: clean + ensure under destDir
 target, err := safeJoin(destDir, hdr.Name)
 if err != nil {
 return err
 }
 switch hdr.Typeflag {
 case tar.TypeDir:
 if err := os.MkdirAll(target, 0o755); err != nil {
 return err
 }
 case tar.TypeReg:
 if err := os.MkdirAll(filepath.Dir(target), 0o755); err != nil {
 return err
 }
 f, err := os.OpenFile(target, os.O_CREATE|os.O_WRONLY|os.O_TRUNC, hdr.FileInfo().Mode())
 if err != nil {
 return err
 }
 if _, err := io.Copy(f, io.LimitReader(tr, hdr.Size)); err != nil {
 f.Close()
 return err
 }
 f.Close()
 }
 }
}
```

**Path traversal:** never write `hdr.Name` that escapes `destDir` (`../` tricks). Always clean and verify a common root.

## ZIP

```
// read
r, err := zip.OpenReader("site.zip")
defer r.Close()
for _, f := range r.File {
 rc, err := f.Open()
 // io.Copy to safe path under dest; then rc.Close()
}

// write
f, err := os.Create("out.zip")
zw := zip.NewWriter(f)
w, err := zw.Create("readme.txt")
_, err = io.WriteString(w, "hello")
err = zw.Close()
```

Same traversal rules as tar when extracting.

## Streaming vs memory

```
good: io.Copy(gzipWriter, file) // bounded memory
bad: gzip.Compress whole []byte // for huge inputs
```

Prefer stream composition (`io.Reader`  $\rightarrow$  compressor  $\rightarrow$  `io.Writer`).

## Rules of thumb

Do	Don't
Check <code>Close</code> on gzip writers	Ignore close errors (corrupt archives)
Limit extracted sizes	Trust archive headers blindly
Sanitize entry paths	Extract zip-slip paths
Stream large payloads	<code>ReadAll</code> multi-GB archives

## Try next

1. Gzip a log file and gunzip it; compare SHA-256.
2. Build a tiny `tar.gz` of a directory listing.
3. Attempt a zip-slip path and ensure your extractor rejects it.

**encoding/binary, gob, and pem**

# encoding/binary, gob, and pem

## Overview

Beyond JSON/CSV/XML (chapter 985), the stdlib encodes **binary layouts**, **Go-native gob streams**, and **PEM** blocks (certs/keys). Pick the format for the consumer: wire protocols → binary; Go-to-Go caches → gob (carefully); TLS material → pem + x509.

## encoding/binary

Read and write numbers in a fixed endianness—file headers, network protocols, length-prefixed frames.

```
var buf bytes.Buffer
// write
if err := binary.Write(&buf, binary.BigEndian, uint32(42)); err != nil {
 return err
}
if err := binary.Write(&buf, binary.BigEndian, int16(-3)); err != nil {
 return err
}

// read
var u uint32
var s int16
r := bytes.NewReader(buf.Bytes())
if err := binary.Read(r, binary.BigEndian, &u); err != nil {
 return err
}
if err := binary.Read(r, binary.BigEndian, &s); err != nil {
 return err
}
```



## Fixed size helpers

```
b := make([]byte, 8)
binary.BigEndian.PutUint64(b, 0xdeadbeef)
n := binary.BigEndian.Uint64(b)
_ = n
```

Endian	Common use
<code>binary.BigEndian</code>	Network byte order, many file formats
<code>binary.LittleEndian</code>	x86-native dumps, some formats
<code>binary.NativeEndian</code>	Same-machine only (Go 1.19+)

## Length-prefixed message

```
func writeFrame(w io.Writer, payload []byte) error {
 if err := binary.Write(w, binary.BigEndian, uint32(len(payload))); err != nil {
 return err
 }
 _, err := w.Write(payload)
 return err
}

func readFrame(r io.Reader, max int) ([]byte, error) {
 var n uint32
 if err := binary.Read(r, binary.BigEndian, &n); err != nil {
 return nil, err
 }
 if int(n) > max {
 return nil, fmt.Errorf("frame %d exceeds max %d", n, max)
 }
 buf := make([]byte, n)
 _, err := io.ReadFull(r, buf)
 return buf, err
}
```

Always bound `max`—never trust remote lengths blindly.

## encoding/gob

Gob is Go's built-in binary serialization for **Go types** (including interfaces with registration). Great for same-version Go services and caches; **not** a long-term public API format.

```

type Item struct {
 SKU string
 Count int
}

var buf bytes.Buffer
enc := gob.NewEncoder(&buf)
if err := enc.Encode(Item{SKU: "go-book", Count: 2}); err != nil {
 log.Fatal(err)
}
dec := gob.NewDecoder(&buf)
var out Item
if err := dec.Decode(&out); err != nil {
 log.Fatal(err)
}

```

## Interfaces

```

gob.Register(Item{}) // once; before encode/decode of interface values

```

## Caveats

Prefer gob when	Avoid gob when
Go Go, controlled versions	Public multi-language APIs
Short-lived cache blobs	Long-term archival formats
Internal RPC experiments	Anything needs schema evolution guarantees

For public APIs prefer JSON, Protobuf, or similar.

## encoding/pem

PEM is the ASCII armor around DER bytes (-----BEGIN ...-----).

```

block, rest := pem.Decode(pemBytes)
if block == nil {
 return fmt.Errorf("no PEM block")
}
_ = rest
// block.Type e.g. "CERTIFICATE", "RSA PRIVATE KEY"
// block.Bytes is DER

```

Encode:

```
var buf bytes.Buffer
_ = pem.Encode(&buf, &pem.Block{Type: "CERTIFICATE", Bytes: der})
```

Pair with crypto/x509:

```
cert, err := x509.ParseCertificate(block.Bytes)
```

Never log private key PEM in application logs.

## encoding/hex and base64 (recap)

```
h := hex.EncodeToString(sum[:])
raw, err := hex.DecodeString(h)

s := base64.StdEncoding.EncodeToString(data)
// URL-safe: base64.RawURLEncoding
```

Chapter 985 covers these in the JSON/CSV track; they pair naturally with binary digests and tokens.

## Rules of thumb

Do	Don't
Cap length-prefixed frames	Allocate <code>make([]byte, n)</code> from network without max
Document endianness	Mix Big/Little without a spec
Use gob only	Expose gob as a mobile client protocol
in-process/versioned Go	
Keep PEM keys off disk	Commit private keys into git
world-readable	

## Try next

1. Implement `writeFrame/readFrame` over a `bytes.Buffer` and fuzz lengths.
2. Gob-encode a slice of structs and decode into a new value.
3. PEM-decode a test certificate and print `Subject` via `x509.ParseCertificate`.

**math, math/bits, and math/big**

# math, math/bits, and math/big

## Overview

- **math**: floating-point helpers (`Sin`, `Sqrt`, `Inf`, `NaN`, rounding).
- **math/bits**: integer bit operations (ones count, rotate, leading zeros)—often used for hashers, allocators, and compact encodings.
- **math/big**: arbitrary-precision integers/floats/rats—crypto sizes, money-like rationals, huge counters.

Most application code needs little of **math**; systems and crypto-adjacent tools use **bits** and **big** more.

## math: floats with eyes open

```
import "math"

x := math.Hypot(3, 4) // 5
y := math.Round(2.5) // 3 (half away from zero in recent Go - check docs for edge cases)
```

## NaN and Inf

```
nan := math.NaN()
inf := math.Inf(1)
if math.IsNaN(nan) { /* ... */ }
if math.IsInf(inf, 1) { /* +Inf */ }
```

NaN compares unequal to itself. Prefer `math.IsNaN` over `==`.

## Float equality

```
func almostEqual(a, b, eps float64) bool {
 return math.Abs(a-b) <= eps
}
```

Do not use `==` for computed floats in tests without a tolerance.

## Integer-ish helpers

```
math.Max(a, b) // float64 only in classic math
// For ordered types prefer cmp / min/max builtins (Go 1.21+)
m := max(a, b) // builtin for integers etc.
```

## math/bits: integer bit tricks

```
import "math/bits"

bits.OnesCount64(x) // popcount
bits.LeadingZeros64(x) // for log2-ish sizing
bits.TrailingZeros64(x) // power-of-two alignment
bits.RotateLeft64(x, 7)
bits.ReverseBytes64(x)
bits.Len64(x) // bit length
```

Example: next power of two sizing

```
func nextPow2(n uint64) uint64 {
 if n <= 1 {
 return 1
 }
 return 1 << bits.Len64(n-1)
}
```

These compile to efficient CPU instructions on common architectures.

## math/big: big integers

```
import "math/big"

a := new(big.Int).SetString("12345678901234567890", 10)
b := big.NewInt(42)
sum := new(big.Int).Add(a, b)
fmt.Println(sum.String())
```

## Methods mutate and return receiver

```
// sum = a + b; Add returns sum for chaining
sum.Add(a, b)
// careful: overlapping aliases - read docs for each method
```

## Modular exponent (crypto-shaped)

```
result := new(big.Int).Exp(base, exp, mod) // base^exp mod mod
```

For production crypto, prefer `crypto/*` packages over hand-rolled `big` math.

## Rationals and floats

```
r := big.NewRat(1, 3)
f, _ := new(big.Float).SetString("3.14159265358979323846")
_ = r
_ = f
```

## Money warning

`float64` is a poor ledger type. Prefer:

- integer `cents` (`int64`), or
- `big.Rat` / decimal libraries for fractional currency rules.

```
// cents
total := int64(1999) + int64(500) // $19.99 + $5.00
```

## Performance notes

Tool	Cost
<code>math float ops</code>	Cheap; watch NaN paths
<code>math/bits</code>	Very cheap
<code>math/big</code>	Heap, slower; fine for rare big numbers

Do not use `big.Int` for every loop counter.

## Rules of thumb

Do	Don't
Use builtins <code>min/max</code> for integers	Force float <code>math.Max</code> for int
Test floats with epsilon	Assert exact binary equality
Use <code>bits</code> for masks/popcount	Hand-roll portable popcount
Use <code>big</code> for true big integers	Store money in <code>float64</code>

## Try next

1. Implement `nextPow2` and table-test edges (0, 1, 2, 3, `MaxUint32`).
2. Add two 50-digit integers with `big.Int`.
3. Show that `0.1+0.2 != 0.3` in `float64` and fix with integer cents.



**mime, multipart, expvar, and runtime/debug**

# mime, multipart, expvar, and runtime/debug

## Overview

This chapter groups **edge-of-stdlib** packages every serious tool eventually touches:

- **mime** / **mime/multipart** — content types and file uploads
- **expvar** — process variables for quick ops scrapes
- **net/http/pprof** — profile endpoints (import for side effect)
- **runtime/debug** / **debug/buildinfo** — GC knobs, stack dumps, embed build metadata

## mime: content types

```
import "mime"

ctype := mime.TypeByExtension(".json") // application/json
exts, _ := mime.ExtensionsByType("image/png")

mediatype, params, err := mime.ParseMediaType(
 `multipart/form-data; boundary=----abcd`)
// mediatype == "multipart/form-data"
// params["boundary"] == "----abcd"
```

Set types explicitly on responses when you know them:

```
w.Header().Set("Content-Type", "application/json; charset=utf-8")
```

## multipart: form uploads

```
func upload(w http.ResponseWriter, r *http.Request) {
 // Bound memory: 32 MiB in memory, rest to temp files
 if err := r.ParseMultipartForm(32 << 20); err != nil {
 http.Error(w, "bad multipart", http.StatusBadRequest)
 return
 }
}
```

```

}
file, hdr, err := r.FormFile("file")
if err != nil {
 http.Error(w, "missing file", http.StatusBadRequest)
 return
}
defer file.Close()

// Optional: sniff type from head bytes
head := make([]byte, 512)
n, _ := file.Read(head)
ctype := http.DetectContentType(head[:n])
// re-open or use io.MultiReader if you need full stream again

dst, err := os.CreateTemp("", "upload-*")
if err != nil {
 http.Error(w, "storage", http.StatusInternalServerError)
 return
}
defer dst.Close()
if _, err := io.Copy(dst, io.MultiReader(bytes.NewReader(head[:n]), file)); err != nil {
 http.Error(w, "write", http.StatusInternalServerError)
 return
}
fmt.Fprintf(w, "saved %s (%s) as %s\n", hdr.Filename, ctype, dst.Name())
}

```

Habit	Why
Size limits	Avoid memory exhaustion
Don't trust <code>Filename</code> alone	Path traversal / odd characters
Detect content type	Clients lie about MIME

## Writing multipart (client)

```

var buf bytes.Buffer
w := multipart.NewWriter(&buf)
fw, _ := w.CreateFormFile("file", "notes.txt")
_, _ = io.WriteString(fw, "hello")
_ = w.Close()

req, _ := http.NewRequest(http.MethodPost, url, &buf)
req.Header.Set("Content-Type", w.FormDataContentType())

```

## expvar: export process vars

```
import "expvar"

var (
 requests = expvar.NewInt("requests_total")
 version = expvar.NewString("version")
)

func init() {
 version.Set("1.2.3")
}

func handle(w http.ResponseWriter, r *http.Request) {
 requests.Add(1)
 // ...
}

// http.Handle("/debug/vars", expvar.Handler()) // JSON map of vars
```

Useful for **simple** scrapes. For production metrics at scale, prefer Prometheus-style instrumentation (part 16)—but `expvar` is zero-dep and always there.

## net/http/pprof

```
import _ "net/http/pprof"

// If you use DefaultServeMux:
go func() {
 log.Println(http.ListenAndServe("127.0.0.1:6060", nil))
}()
// Then: go tool pprof http://127.0.0.1:6060/debug/pprof/heap
// go tool pprof http://127.0.0.1:6060/debug/pprof/goroutineleak (Go 1.27+)
```

Bind pprof to **localhost** or protect it—profiles leak internals.

## runtime/debug

```
import "runtime/debug"

debug.SetGCPercent(50) // GC target; trade CPU vs memory
debug.SetMemoryLimit(2 << 30) // soft memory limit (Go 1.19+)
```

```

debug.WriteHeapDump(fd) // heavy; support tooling
debug.PrintStack() // stderr stack of current goroutine

```

Build info at runtime:

```

if bi, ok := debug.ReadBuildInfo(); ok {
 fmt.Println(bi.Main.Path, bi.Main.Version)
 for _, s := range bi.Settings {
 // vcs.revision, -ldflags, CGO_ENABLED, ...
 }
}

```

## debug/buildinfo (binaries on disk)

```

import "debug/buildinfo"

bi, err := buildinfo.ReadFile("/path/to/binary")
// bi.GoVersion, bi.Path, bi.Deps

```

Handy for “what version is deployed?” without a `/version` endpoint.

## Rules of thumb

Do	Don't
Bound multipart memory	Parse unlimited uploads
Lock down <code>/debug/*</code>	Expose pprof on 0.0.0.0 publicly
Export a version string	Guess deploy version from timestamps
Prefer slog + metrics for ops	Rely only on <code>fmt</code> prints in prod

## Try next

1. Accept a multipart upload and reject bodies over 1 MiB.
2. Register an `expvar` counter and curl `/debug/vars`.
3. Print `debug.ReadBuildInfo()` from your binary's `main`.

**container/heap, list, and ring**

# container/heap, list, and ring

## Overview

The `container` packages implement classic structures not in the language: **heap** (priority queue), **doubly linked list**, and **ring**.

## container/heap

```
type intHeap []int
func (h intHeap) Len() int { return len(h) }
func (h intHeap) Less(i, j int) bool { return h[i] < h[j] }
func (h intHeap) Swap(i, j int) { h[i], h[j] = h[j], h[i] }
func (h *intHeap) Push(x any) { *h = append(*h, x.(int)) }
func (h *intHeap) Pop() any {
 old := *h
 n := len(old)
 x := old[n-1]
 *h = old[:n-1]
 return x
}

h := &intHeap{3, 1, 2}
heap.Init(h)
heap.Push(h, 0)
x := heap.Pop(h).(int) // 0
```

Use for schedulers, merge-k-sorted, Dijkstra.

## container/list

```
l := list.New()
e := l.PushBack("b")
l.PushFront("a")
l.InsertAfter("c", e)
for e := l.Front(); e != nil; e = e.Next() {
 fmt.Println(e.Value)
}
```

```
}
```

Prefer slices unless you need  $O(1)$  remove from middle with handles.

## container/ring

```
r := ring.New(3)
for i := 0; i < r.Len(); i++ {
 r.Value = i
 r = r.Next()
}
r.Do(func(v any) { fmt.Println(v) })
```

Round-robin buffers.

## Rules

Prefer	When
slices + sort	Simple ordered data
heap	Repeated min/max extract
list	Heavy middle splice with element pointers

## Try next

1. Top-K largest with min-heap of size K.
2. LRU sketch: map + list.
3. Ring for N worker round-robin.



## **path and path/filepath Deep Dive**

# path and path/filepath Deep Dive

## Overview

path is slash-separated (URLs, import paths). path/filepath is **OS paths**. Mixing them is a common bug.

## Quick rules

Package	Use
path/filepath	Files on disk
path	URL paths, logical slash paths

```
filepath.Join("a", "b") // a/b or a\b
path.Join("a", "b") // always a/b
filepath.Base("/tmp/x.go")
filepath.Ext("x.go") // .go
filepath.Clean("../etc/passwd")
filepath.Abs(".")
```

## Safe join (jail)

```
func safeJoin(root, unsafe string) (string, error) {
 c := filepath.Clean("/") + unsafe // force abs clean form
 rel := strings.TrimPrefix(c, string(filepath.Separator))
 full := filepath.Join(root, rel)
 absRoot, _ := filepath.Abs(root)
 absFull, _ := filepath.Abs(full)
 if !strings.HasPrefix(absFull, absRoot+string(filepath.Separator)) && absFull != absRoot {
 return "", fmt.Errorf("escape")
 }
 return absFull, nil
}
```

## Walk

```
filepath.WalkDir(root, func(p string, d fs.DirEntry, err error) error {
 if err != nil {
 return err
 }
 if d.IsDir() && d.Name() == ".git" {
 return fs.SkipDir
 }
 return nil
})
```

## Match / Glob

```
filepath.Glob("*.go")
filepath.Match("a/*.txt", name)
```

## Rules

Do	Don't
<code>filepath</code> for files	<code>path.Join</code> for disk on Windows
Clean + jail user paths	Concatenate <code>root + "/" + user</code>

## Try next

1. Prove `path.Join` vs `filepath.Join` docs on Windows GOOS.
2. `WalkDir` skip `node_modules`.
3. `SafeJoin` reject `../../etc/passwd`.

## **bufio Scanner and Readers Advanced**

# bufio Scanner and Readers Advanced

## Overview

bufio sits between raw `os.File/net.Conn` and high-level logic. Master Scanner token sizes, Reader peek, and Writer flush.

## Scanner limits

Default max token ~64K. Long lines fail with `bufio.ErrTooLong`.

```
sc := bufio.NewScanner(r)
buf := make([]byte, 0, 1024*1024)
sc.Buffer(buf, 10*1024*1024) // max 10MiB tokens
sc.Split(bufio.ScanLines)
for sc.Scan() {
 _ = sc.Text()
}
return sc.Err()
```

## Custom split

```
// split on NUL
sc.Split(func(data []byte, atEOF bool) (advance int, token []byte, err error) {
 if i := bytes.IndexByte(data, 0); i >= 0 {
 return i + 1, data[:i], nil
 }
 if atEOF && len(data) > 0 {
 return len(data), data, nil
 }
 return 0, nil, nil
})
```

## Reader Peek / Discard

```
br := bufio.NewReader(r)
b, _ := br.Peek(4) // next bytes without consume
_, _ = br.Discard(4)
```

## Writer

```
bw := bufio.NewWriter(w)
fmt.Fprintln(bw, "line")
_ = bw.Flush() // required before close for full delivery
```

## Rules

Do	Don't
Raise buffer for long lines	Ignore Scan errors
Flush writers	Assume OS sees buffered data
Prefer Scanner for lines	ReadAll multi-GB logs

## Try next

1. File with 1MB line; fix with Buffer.
2. Peek magic bytes for file type.
3. Benchmark Scanner vs ReadString.

# Projects

## **027 Project 27: Kubernetes Event Watcher**



# 027 Build a Kubernetes Event Watcher

Stream cluster Events in real time with `client-go` Watch APIs. Print concise, timestamped lines operators can follow during incident response or CI rollout debugging.

```
kubeconfig -> clientset -> Events.Watch -> ResultChan -> print
```

## Problem statement

CLI `kevents`:

- Watch Events in one namespace or all
- Optional field selector (e.g. involved object kind)
- Print: `TIME TYPE REASON OBJECT MESSAGE`
- Handle watch reconnects on error
- Graceful shutdown on `SIGINT`

## Acceptance criteria

- ☐ Opens a Watch and prints events until interrupted
- ☐ Namespace flag works
- ☐ Context cancel stops cleanly (`Stop()` watch)
- ☐ Errors are visible; process does not silent-exit on first blip without log
- ☐ Builds with modules

## Setup

```
mkdir kevents && cd kevents
go mod init example.com/kevents
go get k8s.io/client-go@latest
go get k8s.io/apimachinery@latest
go mod tidy
go 1.27
```

## Full main.go

```
package main

import (
 "context"
 "flag"
 "fmt"
 "os"
 "os/signal"
 "path/filepath"
 "syscall"
 "time"

 corev1 "k8s.io/api/core/v1"
 metav1 "k8s.io/apimachinery/pkg/apis/meta/v1"
 "k8s.io/apimachinery/pkg/watch"
 "k8s.io/client-go/kubernetes"
 "k8s.io/client-go/tools/clientcmd"
 "k8s.io/client-go/util/homedir"
)

func formatEvent(e *corev1.Event) string {
 obj := e.InvolvedObject.Kind + "/" + e.InvolvedObject.Name
 if e.InvolvedObject.Namespace != "" {
 obj = e.InvolvedObject.Namespace + "/" + obj
 }
 ts := e.LastTimestamp.Time
 if ts.IsZero() {
 ts = e.EventTime.Time
 }
 if ts.IsZero() {
 ts = time.Now()
 }
 return fmt.Sprintf("%s %-12s %-20s %-40s %s",
 ts.Format(time.RFC3339),
 e.Type,
 e.Reason,
 obj,
 e.Message,
)
}

func main() {
 var kubeconfig string
 if home := homedir.HomeDir(); home != "" {
```

```

 kubeconfig = filepath.Join(home, ".kube", "config")
}
kc := flag.String("kubeconfig", kubeconfig, "kubeconfig path")
ns := flag.String("n", "", "namespace (empty=all)")
fieldSelector := flag.String("field-selector", "", "field selector")
flag.Parse()

cfg, err := clientcmd.BuildConfigFromFlags("", *kc)
if err != nil {
 fmt.Fprintln(os.Stderr, err)
 os.Exit(1)
}
cli, err := kubernetes.NewForConfig(cfg)
if err != nil {
 fmt.Fprintln(os.Stderr, err)
 os.Exit(1)
}

ctx, stop := signal.NotifyContext(context.Background(), os.Interrupt, syscall.SIGTERM)
defer stop()

namespace := *ns
if namespace == "" {
 namespace = metav1.NamespaceAll
}

fmt.Fprintln(os.Stderr, "watching events; Ctrl+C to stop")

for {
 if ctx.Err() != nil {
 return
 }
 w, err := cli.CoreV1().Events(namespace).Watch(ctx, metav1.ListOptions{
 FieldSelector: *fieldSelector,
 })
 if err != nil {
 fmt.Fprintln(os.Stderr, "watch error:", err)
 select {
 case <-ctx.Done():
 return
 case <-time.After(2 * time.Second):
 }
 continue
 }

 func() {
 defer w.Stop()

```

```

 for {
 select {
 case <-ctx.Done():
 return
 case ev, ok := <-w.ResultChan():
 if !ok {
 fmt.Fprintln(os.Stderr, "watch closed; reconnecting")
 return
 }
 if ev.Type == watch.Error {
 fmt.Fprintf(os.Stderr, "watch event error: %v\n", ev.Object)
 return
 }
 e, ok := ev.Object.(*corev1.Event)
 if !ok {
 fmt.Printf("%s %T\n", ev.Type, ev.Object)
 continue
 }
 fmt.Printf("%-8s %s\n", ev.Type, formatEvent(e))
 }
 }
 }()
}
}

```

## Step-by-step build path

1. Client from kubeconfig.
2. `Events(ns).Watch` with `ListOptions`.
3. Range `ResultChan`; type-assert `*corev1.Event`.
4. On channel close / error, backoff and reconnect.
5. SIGINT via context cancels `Watch`.

## Run and verification

```

go run .
go run . -n default
generate events:
kubectl run wget --image=busybox --restart=Never -- wget -O- https://kubernetes.default
kubectl delete pod wget --force --grace-period=0

```

## Stretch goals

1. Filter `type=Warning` only.
2. JSON lines for log aggregation.
3. Informer/factory instead of raw Watch.
4. Metrics: events/sec counter.

## Pitfalls

Pitfall	Fix
Not calling <code>w.Stop()</code>	leak; always defer Stop
Ignoring watch close	reconnect loop
Printing only <code>%T</code>	format useful fields
No cancel	hang on Ctrl+C

## Learning goals

- Watch API lifecycle
- Operator-facing event streams
- Reconnect patterns for long-lived clients

## **028 Project 28: Kubernetes Rollout Checker**

## 028 Build a Kubernetes Rollout Checker

Check whether a Deployment is fully rolled out: desired replicas match updated and ready counts. Exit 0 when healthy, 1 when pending, 2 on usage/API errors—ideal for CI gates after `kubectl apply`.

```
kubeconfig -> Get Deployment -> compare desired/updated/ready -> exit code
```

### Problem statement

```
kroll -name DEPLOY [-n NS] [-wait DURATION] [-interval D]
```

- One-shot check by default
- Optional `-wait` polls until healthy or timeout
- Print a one-line status for logs

### Acceptance criteria

- ☐ Reads Deployment status fields correctly
- ☐ Exit 0 only when `ready == desired && updated == desired` (and `desired > 0` or allow 0)
- ☐ Exit 1 when still rolling
- ☐ Exit 2 on bad flags / API errors
- ☐ Optional wait loop with timeout

### Setup

```
mkdir kroll && cd kroll
go mod init example.com/kroll
go get k8s.io/client-go@latest
go get k8s.io/apimachinery@latest
go mod tidy
go 1.27
```

## Full main.go

```
package main

import (
 "context"
 "flag"
 "fmt"
 "os"
 "path/filepath"
 "time"

 metav1 "k8s.io/apimachinery/pkg/apis/meta/v1"
 "k8s.io/client-go/kubernetes"
 "k8s.io/client-go/tools/clientcmd"
 "k8s.io/client-go/util/homedir"
)

type status struct {
 Desired int32
 Updated int32
 Ready int32
 Avail int32
}

func (s status) Healthy() bool {
 return s.Ready == s.Desired && s.Updated == s.Desired && s.Avail == s.Desired
}

func getStatus(ctx context.Context, cli *kubernetes.Clientset, ns, name string) (status, error) {
 d, err := cli.AppsV1().Deployments(ns).Get(ctx, name, metav1.GetOptions{})
 if err != nil {
 return status{}, err
 }
 var desired int32 = 1
 if d.Spec.Replicas != nil {
 desired = *d.Spec.Replicas
 }
 return status{
 Desired: desired,
 Updated: d.Status.UpdatedReplicas,
 Ready: d.Status.ReadyReplicas,
 Avail: d.Status.AvailableReplicas,
 }, nil
}
```



```

func main() {
 var kubeconfig string
 if home := homedir.HomeDir(); home != "" {
 kubeconfig = filepath.Join(home, ".kube", "config")
 }
 kc := flag.String("kubeconfig", kubeconfig, "kubeconfig path")
 name := flag.String("name", "", "deployment name")
 ns := flag.String("n", "default", "namespace")
 wait := flag.Duration("wait", 0, "max wait for healthy (0=check once)")
 interval := flag.Duration("interval", 2*time.Second, "poll interval")
 flag.Parse()

 if *name == "" {
 fmt.Fprintln(os.Stderr, "usage: kroll -name DEPLOY [-n NS] [-wait 2m]")
 os.Exit(2)
 }

 cfg, err := clientcmd.BuildConfigFromFlags("", *kc)
 if err != nil {
 fmt.Fprintln(os.Stderr, err)
 os.Exit(2)
 }
 cli, err := kubernetes.NewForConfig(cfg)
 if err != nil {
 fmt.Fprintln(os.Stderr, err)
 os.Exit(2)
 }

 deadline := time.Now().Add(*wait)
 for {
 ctx, cancel := context.WithTimeout(context.Background(), 10*time.Second)
 st, err := getStatus(ctx, cli, *ns, *name)
 cancel()
 if err != nil {
 fmt.Fprintln(os.Stderr, "get deployment:", err)
 os.Exit(2)
 }

 fmt.Printf("deployment=%s/%s desired=%d updated=%d ready=%d available=%d\n",
 *ns, *name, st.Desired, st.Updated, st.Ready, st.Avail)

 if st.Healthy() {
 fmt.Println("rollout: healthy")
 os.Exit(0)
 }
 fmt.Println("rollout: pending")
 }
}

```

```

 if *wait == 0 || time.Now().After(deadline) {
 if *wait > 0 {
 fmt.Fprintln(os.Stderr, "timeout waiting for rollout")
 }
 os.Exit(1)
 }
 time.Sleep(*interval)
 }
}

```

## Run and verification

```

kubectl create deploy roll-demo --image=nginx:1.27 --replicas=2
go run . -name roll-demo -n default
go run . -name roll-demo -wait 60s -interval 1s

kubectl set image deploy/roll-demo nginx=nginx:1.27-alpine
go run . -name roll-demo -wait 2m

```

## Tests

Unit-test `status.Healthy` without a cluster:

```

package main

import "testing"

func TestHealthy(t *testing.T) {
 if !(status{2, 2, 2, 2}).Healthy() {
 t.Fatal("expected healthy")
 }
 if (status{2, 1, 1, 1}).Healthy() {
 t.Fatal("expected pending")
 }
}

```

## Stretch goals

1. Also check `DeploymentProgressing` condition.
2. Support `StatefulSet` / `DaemonSet`.
3. JSON output for CI annotations.
4. Fail if `UnavailableReplicas > 0` after deadline.

## Pitfalls

Pitfall	Fix
Ignoring AvailableReplicas	include in healthy check
Nil Spec.Replicas	default 1 per API conventions
Wait without timeout	always bound wait
Wrong exit codes for CI	document 0/1/2

## Learning goals

- Deployment status fields for rollout gates
- Polling with timeout for CI
- Scriptable exit codes

## 029 Project 29: Terraform Plan Parser

## 029 Build a Terraform Plan Parser

Parse `terraform show -json` output and summarize create/update/delete/replace actions. This is the foundation for CI policy gates and human-readable plan digests.

```
plan.json -> decode resource_changes -> classify actions -> print + summary
```

### Problem statement

```
tf-plan-parse <plan.json>
```

- Read Terraform plan JSON
- For each `resource_changes[]`, print address and actions
- Detect replace (`delete+create` in one change)
- Print summary counts
- Exit 2 on usage/IO/JSON errors; 0 otherwise

### Acceptance criteria

- ☐ Handles empty plans
- ☐ Counts create/update/delete/replace
- ☐ Replace not double-counted as delete+create in summary (your policy: document)
- ☐ Stable stdout suitable for logs
- ☐ Stdlib only

### Setup

```
mkdir tfplanparse && cd tfplanparse
go mod init example.com/tfplanparse
go 1.27
```

Prerequisite sample:

```
in a terraform root
terraform plan -out=tfplan
terraform show -json tfplan > plan.json
```

Minimal fixture testdata/plan.json:

```
{
 "resource_changes": [
 {
 "address": "aws_instance.web",
 "type": "aws_instance",
 "change": { "actions": ["create"] }
 },
 {
 "address": "aws_instance.old",
 "type": "aws_instance",
 "change": { "actions": ["delete", "create"] }
 }
]
}
```

## Full main.go

```
package main

import (
 "encoding/json"
 "fmt"
 "os"
)

type Plan struct {
 ResourceChanges []ResourceChange `json:"resource_changes"`
}

type ResourceChange struct {
 Address string `json:"address"`
 Type string `json:"type"`
 Change struct {
 Actions []string `json:"actions"`
 } `json:"change"`
}

func classify(actions []string) string {
 if len(actions) == 2 && actions[0] == "delete" && actions[1] == "create" {
 return "replace"
 }
 if len(actions) == 1 {
 return actions[0]
 }
}
```

```

 }
 if len(actions) == 0 {
 return "no-op"
 }
 return "mixed:" + fmt.Sprint(actions)
}

func analyze(p Plan) (counts map[string]int, lines []string) {
 counts = map[string]int{
 "create": 0, "update": 0, "delete": 0, "replace": 0, "no-op": 0, "other": 0,
 }
 for _, rc := range p.ResourceChanges {
 kind := classify(rc.Change.Actions)
 switch kind {
 case "create", "update", "delete", "replace", "no-op":
 counts[kind]++
 default:
 counts["other"]++
 }
 lines = append(lines, fmt.Sprintf("%-8s %s", kind, rc.Address))
 }
 return counts, lines
}

func main() {
 if len(os.Args) != 2 {
 fmt.Fprintln(os.Stderr, "usage: tf-plan-parse <plan.json>")
 os.Exit(2)
 }
 b, err := os.ReadFile(os.Args[1])
 if err != nil {
 fmt.Fprintln(os.Stderr, err)
 os.Exit(2)
 }
 var p Plan
 if err := json.Unmarshal(b, &p); err != nil {
 fmt.Fprintln(os.Stderr, "json:", err)
 os.Exit(2)
 }

 counts, lines := analyze(p)
 for _, line := range lines {
 fmt.Println(line)
 }
 fmt.Printf("summary create=%d update=%d delete=%d replace=%d no-op=%d other=%d\n",
 counts["create"], counts["update"], counts["delete"], counts["replace"], counts["no-

```

## Run and verification

```
go run . testdata/plan.json
REPLACE aws_instance.old
create aws_instance.web
summary create=1 ...
```

## Tests

```
package main

import "testing"

func TestClassifyReplace(t *testing.T) {
 if classify([]string{"delete", "create"}) != "replace" {
 t.Fatal()
 }
}

func TestAnalyze(t *testing.T) {
 p := Plan{ResourceChanges: []ResourceChange{
 {Address: "a", Change: struct {
 Actions []string `json:"actions"`
 }{Actions: []string{"create"}}},
 }}
 // fix: use proper struct init
 _ = p
}
```

Better test:

```
func TestAnalyzeCounts(t *testing.T) {
 var p Plan
 p.ResourceChanges = []ResourceChange{
 {Address: "x", Change: struct {
 Actions []string `json:"actions"`
 }{Actions: []string{"create"}}},
 }
 // Simpler: set Actions via temporary
 rc := ResourceChange{Address: "x"}
 rc.Change.Actions = []string{"create"}
 p.ResourceChanges = []ResourceChange{rc}
 rc2 := ResourceChange{Address: "y"}
 rc2.Change.Actions = []string{"delete", "create"}
```



```

 p.ResourceChanges = append(p.ResourceChanges, rc2)

 c, _ := analyze(p)
 if c["create"] != 1 || c["replace"] != 1 {
 t.Fatalf("%v", c)
 }
}

go test ./...

```

## Stretch goals

1. Group by provider/type.
2. Filter addresses by regex.
3. Markdown report for PR comments.
4. Read plan from stdin (-).

## Pitfalls

Pitfall	Fix
Binary <code>tfplan</code> not JSON	always <code>terraform show -json</code>
Plan format version drift	pin terraform; tolerate missing fields
Counting replace as delete+create	classify first

## Learning goals

- Deterministic IaC review tooling
- JSON plan shape for Terraform
- Building blocks for policy-as-code

# 030 Project 30: Terraform Risk Reporter

## 030 Build a Terraform Risk Reporter

Use a Terraform plan JSON to **fail CI** when deletes or replacements touch critical resources. This sits on top of the plan parser idea and turns analysis into a gate.

```
plan.json -> find delete/replace -> match risk rules -> exit 1 if risky
```

### Problem statement

```
tf-risk [-max-risky N] [-deny-regex RE] <plan.json>
```

- Flag every delete or replace as risky by default
- Optional deny regex on address (e.g. `aws_db_instance`)
- Exit 1 if risky count > max (default 0)
- Exit 0 if clean; 2 on usage/parse errors

### Acceptance criteria

- ☐ Detects delete and replace
- ☐ Prints each risky address with actions
- ☐ Exit code suitable for CI
- ☐ Deny regex optional
- ☐ Stdlib only

### Setup

```
mkdir tfrisk && cd tfrisk
go mod init example.com/tfrisk
go 1.27
```

## Full main.go

```
package main

import (
 "encoding/json"
 "flag"
 "fmt"
 "os"
 "regexp"
)

type plan struct {
 ResourceChanges []struct {
 Address string `json:"address"`
 Type string `json:"type"`
 Change struct {
 Actions []string `json:"actions"`
 } `json:"change"`
 } `json:"resource_changes"`
}

func hasAction(actions []string, x string) bool {
 for _, a := range actions {
 if a == x {
 return true
 }
 }
 return false
}

func isReplace(actions []string) bool {
 return len(actions) == 2 && actions[0] == "delete" && actions[1] == "create"
}

func isRisky(actions []string) bool {
 return hasAction(actions, "delete") || isReplace(actions)
}

func main() {
 maxRisky := flag.Int("max-risky", 0, "allowed risky changes before fail")
 deny := flag.String("deny-regex", "", "if set, only these addresses are risky when delet")
 flag.Parse()
 if flag.NArg() != 1 {
 fmt.Fprintln(os.Stderr, "usage: tf-risk [flags] <plan.json>")
 os.Exit(2)
 }
}
```

```

}

var denyRE *regexp.Regexp
if *deny != "" {
 var err error
 denyRE, err = regexp.Compile(*deny)
 if err != nil {
 fmt.Fprintln(os.Stderr, "deny-regex:", err)
 os.Exit(2)
 }
}

b, err := os.ReadFile(flag.Arg(0))
if err != nil {
 fmt.Fprintln(os.Stderr, err)
 os.Exit(2)
}

var p plan
if err := json.Unmarshal(b, &p); err != nil {
 fmt.Fprintln(os.Stderr, "json:", err)
 os.Exit(2)
}

risky := 0
for _, rc := range p.ResourceChanges {
 a := rc.Change.Actions
 if !isRisky(a) {
 continue
 }
 if denyRE != nil && !denyRE.MatchString(rc.Address) && !denyRE.MatchString(rc.Type)
 // when deny-regex set: only matching resources are "policy risks"
 // still count all deletes as informational? here: skip non-matching
 fmt.Printf("INFO: %s actions=%v (not in deny-regex)\n", rc.Address, a)
 continue
 }
 risky++
 kind := "delete"
 if isReplace(a) {
 kind = "replace"
 }
 fmt.Printf("RISK(%s): %s actions=%v\n", kind, rc.Address, a)
}

fmt.Printf("risky changes=%d max=%d\n", risky, *maxRisky)
if risky > *maxRisky {
 os.Exit(1)
}

```

```
}
```

## Policy modes

Mode	Flags	Behavior
Strict	default	any delete/replace fails
Budget	-max-risky 2	allow up to 2
Critical only	-deny-regex 'aws_db\ aws_rds'	only matching addresses gate

## Run and verification

```
fixture from project 29
go run . -max-risky 0 plan.json; echo exit:$?

go run . -deny-regex 'aws_db_instance' plan.json
```

CI sketch:

```
- run: terraform show -json tfplan > plan.json
- run: go run ./tfrisk -max-risky 0 plan.json
```

## Tests

```
package main

import "testing"

func TestIsReplace(t *testing.T) {
 if !isReplace([]string{"delete", "create"}) {
 t.Fatal()
 }
 if isReplace([]string{"delete"}) {
 t.Fatal()
 }
}

func TestIsRisky(t *testing.T) {
 if !isRisky([]string{"delete"}) || !isRisky([]string{"delete", "create"}) {
 t.Fatal()
 }
}
```

```
 }
 if isRisky([]string{"create"}) {
 t.Fatal()
 }
}
```

## Stretch goals

1. Severity levels (destroy production tagged resources).
2. Allowlist file of addresses that may be destroyed.
3. GitHub PR comment body markdown.
4. Combine with cost estimator (project 38).

## Pitfalls

Pitfall	Fix
Treating update as risk	only delete/replace
Silent success on bad JSON	exit 2
Regex on wrong field	match address and type

## Learning goals

- Enforce infrastructure policy as code
- CI exit codes for plan gates
- Reduce risky deploys with automated checks

## **031 Project 31: Prometheus Exporter**



# 031 Build a Prometheus Exporter

Expose custom application metrics at `/metrics` using the official Prometheus Go client. Instrument a small HTTP handler with counter, histogram, and gauge—the three metric types you will use constantly.

```
/work -> update metrics
/metrics -> promhttp scrape text
```

## Problem statement

Build a demo service on `:9100`:

- GET `/work` simulates work and updates metrics
- GET `/metrics` Prometheus text exposition
- Metrics: `demo_requests_total`, `demo_latency_seconds`, `demo_inflight`

## Acceptance criteria

- ☐ `/metrics` returns Prometheus text format
- ☐ Counter increases on `/work`
- ☐ Histogram observes latency
- ☐ Gauge tracks in-flight correctly (inc/dec with defer)
- ☐ Module builds with `client_golang`

## Setup

```
mkdir exporter && cd exporter
go mod init example.com/exporter
go get github.com/prometheus/client_golang/prometheus@latest
go get github.com/prometheus/client_golang/prometheus/promhttp@latest
go 1.27
go mod tidy
```

## Full main.go

```
package main

import (
 "log"
 "math/rand"
 "net/http"
 "time"

 "github.com/prometheus/client_golang/prometheus"
 "github.com/prometheus/client_golang/prometheus/promhttp"
)

func main() {
 reqTotal := prometheus.NewCounter(prometheus.CounterOpts{
 Name: "demo_requests_total",
 Help: "Total completed /work requests",
 })
 latency := prometheus.NewHistogram(prometheus.HistogramOpts{
 Name: "demo_latency_seconds",
 Help: "Request latency for /work",
 Buckets: prometheus.DefBuckets,
 })
 inflight := prometheus.NewGauge(prometheus.GaugeOpts{
 Name: "demo_inflight",
 Help: "In-flight /work requests",
 })
 prometheus.MustRegister(reqTotal, latency, inflight)

 mux := http.NewServeMux()
 mux.Handle("/metrics", promhttp.Handler())
 mux.HandleFunc("/work", func(w http.ResponseWriter, r *http.Request) {
 inflight.Inc()
 defer inflight.Dec()
 start := time.Now()
 time.Sleep(time.Duration(50+rand.Intn(200)) * time.Millisecond)
 reqTotal.Inc()
 latency.Observe(time.Since(start).Seconds())
 w.WriteHeader(http.StatusOK)
 _, _ = w.Write([]byte("ok\n"))
 })

 addr := ":9100"
 log.Println("listening", addr, " metrics=/metrics work=/work")
 log.Fatal(http.ListenAndServe(addr, mux))
}
```

```
}
```

## Run and verification

```
go run .
terminal B
curl -s localhost:9100/work
curl -s localhost:9100/metrics | grep demo_
```

Expected fragments:

```
demo_requests_total ...
demo_inflight ...
demo_latency_seconds_bucket ...
```

Load a bit:

```
for i in $(seq 1 20); do curl -s localhost:9100/work >/dev/null; done
curl -s localhost:9100/metrics | grep demo_requests_total
```

## Step-by-step build path

1. Define metric types and Help strings.
2. `MustRegister` with default registry.
3. Instrument handler with defer for gauges.
4. Expose `promhttp.Handler()`.
5. Scrape with curl; later add Prometheus scrape config.

## Label cardinality warning

```
// BAD: user_id label → unbounded series
// GOOD: low-cardinality labels: method, code, route template
```

## Stretch goals

1. `CounterVec` by HTTP method/status.
2. Custom registry (not global) for tests.
3. `/healthz` separate from metrics.
4. Exemplars / native histograms (advanced).

## Pitfalls

Pitfall	Fix
High-cardinality labels	static enum labels only
Forgetting gauge Dec	always defer
Blocking default mux pollution	use dedicated <code>ServeMux</code>
Measuring after sleep only	observe full handler duration

## Learning goals

- Practical service metrics design
- Counter / histogram / gauge roles
- Observability by default for Go services

## **032 Project 32: Prometheus Probe Service**

# 032 Build a Prometheus Probe Service

Probe a target URL on demand and export health/latency metrics—similar in spirit to blackbox exporter, but tiny and educational.

```
GET /probe?target=URL -> HTTP GET target -> set probe_up + probe_duration_seconds
GET /metrics -> scrape
```

## Problem statement

Service on :9115:

- `/probe?target=https://example.com` performs a GET with timeout
- Updates labeled gauges `probe_up` and `probe_duration_seconds`
- Returns up/down body and HTTP 200/503
- `/metrics` for Prometheus

## Acceptance criteria

- ☐ Missing target → 400
- ☐ Successful GET (2xx/3xx policy you document) → `probe_up=1`
- ☐ Error or 4xx/5xx → `probe_up=0`
- ☐ Duration always recorded when attempt finishes
- ☐ HTTP client timeout configured

## Setup

```
mkdir probe && cd probe
go mod init example.com/probe
go get github.com/prometheus/client_golang/prometheus@latest
go get github.com/prometheus/client_golang/prometheus/promhttp@latest
go mod tidy
go 1.27
```

## Full main.go

```
package main

import (
 "log"
 "net/http"
 "time"

 "github.com/prometheus/client_golang/prometheus"
 "github.com/prometheus/client_golang/prometheus/promhttp"
)

func main() {
 up := prometheus.NewGaugeVec(prometheus.GaugeOpts{
 Name: "probe_up",
 Help: "1 if last probe succeeded",
 }, []string{"target"})
 dur := prometheus.NewGaugeVec(prometheus.GaugeOpts{
 Name: "probe_duration_seconds",
 Help: "Duration of last probe",
 }, []string{"target"})
 prometheus.MustRegister(up, dur)

 client := &http.Client{Timeout: 2 * time.Second}

 mux := http.NewServeMux()
 mux.Handle("/metrics", promhttp.Handler())
 mux.HandleFunc("/probe", func(w http.ResponseWriter, r *http.Request) {
 target := r.URL.Query().Get("target")
 if target == "" {
 http.Error(w, "missing target", http.StatusBadRequest)
 return
 }
 start := time.Now()
 resp, err := client.Get(target)
 elapsed := time.Since(start).Seconds()
 dur.WithLabelValues(target).Set(elapsed)

 if err != nil {
 up.WithLabelValues(target).Set(0)
 w.WriteHeader(http.StatusServiceUnavailable)
 _, _ = w.Write([]byte("down\n"))
 return
 }
 defer resp.Body.Close()
 })
}
```

```

 if resp.StatusCode >= 400 {
 up.WithLabelValues(target).Set(0)
 w.WriteHeader(http.StatusServiceUnavailable)
 _, _ = w.Write([]byte("down\n"))
 return
 }
 up.WithLabelValues(target).Set(1)
 _, _ = w.Write([]byte("up\n"))
})

log.Println("probe service on :9115 /probe /metrics")
log.Fatal(http.ListenAndServe(":9115", mux))
}

```

## Run and verification

```

go run .
curl -s 'http://localhost:9115/probe?target=https://example.com'
curl -s 'http://localhost:9115/probe?target=http://127.0.0.1:1'
curl -s localhost:9115/metrics | grep probe_

```

Prometheus scrape config sketch:

```

scrape_configs:
- job_name: blackbox-lite
 metrics_path: /probe
 params:
 target: [https://example.com]
 static_configs:
 - targets: ['localhost:9115']

```

(Real blackbox uses `relabel_configs`; this lab keeps metrics on the probe process itself.)

## Stretch goals

1. Histogram of probe durations instead of gauge-only.
2. Support `module=http_2xx|tcp_connect`.
3. Limit label cardinality (hash target or fixed list).
4. Concurrent probe limit with semaphore.

## Pitfalls



Pitfall	Fix
Unbounded <code>target</code> labels	allowlist or drop after scrape
No timeout	hang worker
Not closing body	connection leak
Following redirects to evil URLs	custom <code>CheckRedirect</code>

## Learning goals

- On-demand probes vs push metrics
- Labeled gauges for latest status
- Building blocks for SLO synthetic checks

## **033 Project 33: eBPF Tracepoint Basics**

## 033 Build eBPF Tracepoint Basics

Capture process exec events from `sys_enter_execve` using eBPF: a minimal kernel program, userspace loader with cilium/ebpf, and a perf/ring buffer reader. **Linux lab only**; requires privileges.

```
kernel tracepoint -> eBPF program -> perf/ring buffer -> userspace Go reader
```

### Problem statement

On a Linux VM/lab host:

1. Load a BPF object that attaches to `syscalls/sys_enter_execve`
2. Emit `{pid, comm}` events to userspace
3. Print events until Ctrl+C
4. Detach cleanly on shutdown

### Acceptance criteria

- ☐ Documented toolchain install (clang, llvm, bpf2go or prebuilt)
- ☐ Userspace program loads, attaches, reads events
- ☐ Running `ls / echo` produces lines
- ☐ SIGINT stops without kernel leftover panic (best-effort detach)
- ☐ Clear notes if hardware/CI cannot run BPF

### Setup

```
Debian/Ubuntu-ish lab
sudo apt-get install -y clang llvm libbpf-dev linux-headers-$(uname -r)

mkdir ebpf-trace && cd ebpf-trace
go mod init example.com/ebpf-trace
go get github.com/cilium/ebpf@latest
go install github.com/cilium/ebpf/cmd/bpf2go@latest
go 1.27
```

## Kernel program `exec.bpf.c` (sketch)

```
//go:build ignore

#include "vmlinux.h"
#include <bpf/bpf_helpers.h>

char LICENSE[] SEC("license") = "Dual BSD/GPL";

struct event {
 __u32 pid;
 char comm[16];
};

struct {
 __uint(type, BPF_MAP_TYPE_PERF_EVENT_ARRAY);
 __uint(key_size, sizeof(__u32));
 __uint(value_size, sizeof(__u32));
} events SEC(".maps");

SEC("tracepoint/syscalls/sys_enter_execve")
int handle_exec(struct trace_event_raw_sys_enter *ctx) {
 struct event e = {};
 e.pid = bpf_get_current_pid_tgid() >> 32;
 bpf_get_current_comm(&e.comm, sizeof(e.comm));
 bpf_perf_event_output(ctx, &events, BPF_F_CURRENT_CPU, &e, sizeof(e));
 return 0;
}
```

Generate bindings (names vary by bpf2go version):

```
go generate ./...
//go:generate bpf2go -cc clang -cflags "-O2 -g -Wall" bpf exec.bpf.c -- -I/usr/include
```

## Userspace `main.go`

```
package main

import (
 "bytes"
 "encoding/binary"
 "fmt"
 "log"
 "os"
```

```

 "os/signal"
 "syscall"

 "github.com/cilium/ebpf/link"
 "github.com/cilium/ebpf/perf"
 "github.com/cilium/ebpf/rlimit"
)

type event struct {
 Pid uint32
 Comm [16]byte
}

func main() {
 // Allow the current process to lock memory for eBPF maps.
 if err := rlimit.RemoveMemlock(); err != nil {
 log.Fatal(err)
 }

 objs := bpfObjects{}
 if err := loadBpfObjects(&objs, nil); err != nil {
 log.Fatalf("load: %v", err)
 }
 defer objs.Close()

 tp, err := link.Tracepoint("syscalls", "sys_enter_execve", objs.HandleExec, nil)
 if err != nil {
 log.Fatalf("attach: %v", err)
 }
 defer tp.Close()

 rd, err := perf.NewReader(objs.Events, os.Getpagesize())
 if err != nil {
 log.Fatal(err)
 }
 defer rd.Close()

 sig := make(chan os.Signal, 1)
 signal.Notify(sig, os.Interrupt, syscall.SIGTERM)
 go func() {
 <-sig
 _ = rd.Close()
 }()

 fmt.Println("listening for execve; Ctrl+C to stop")
 for {
 rec, err := rd.Read()

```

```

 if err != nil {
 break
 }
 if rec.LostSamples != 0 {
 fmt.Fprintf(os.Stderr, "lost %d samples\n", rec.LostSamples)
 }
 var e event
 if err := binary.Read(bytes.NewBuffer(rec.RawSample), binary.LittleEndian, &e); err != nil {
 continue
 }
 fmt.Printf("pid=%d comm=%s\n", e.Pid, bytes.TrimRight(e.Comm[:], "\x00"))
 }
}

```

**Note:** bpf0objects, loadBpf0objects, and field names come from bpf2go output. Adjust to your generated API (objs.HandleExec vs objs.bpfPrograms.HandleExec).

## Step-by-step build path

1. Install kernel headers + clang.
2. Write minimal .bpf.c with one tracepoint.
3. bpf2go generate Go + embed.
4. Load, attach, read perf events.
5. Add filtering (UID, comm prefix) next.

## Run and verification

```

go generate ./...
go build -o ebpf-trace .
sudo ./ebpf-trace
other terminal:
ls /tmp; echo hi
expect comm lines for shell tools

```

## Pitfalls

Pitfall	Fix
macOS/Windows host	use Linux VM
Missing CAP_BPF/root	sudo or capabilities
Header/vmlinux mismatch	generate vmlinux.h via bpftool
Lost samples	larger perf buffer; reduce event rate
License section missing	BPF verifier rejects

Pitfall	Fix
---------	-----

## Stretch goals

1. Include filename from `execve` args (careful helper limits).
2. Switch to ringbuf map.
3. Filter by cgroup.
4. Export Prometheus counters of execs/sec.

## Learning goals

- Event-driven observability near the kernel
- Userspace loader lifecycle
- Safe iteration practices for low-level tooling

## **034 Project 34: eBPF XDP Packet Counter**



## 034 Build an eBPF XDP Packet Counter

Attach an XDP program to a network interface, increment a packet counter in a BPF map, and poll the counter from userspace every second. **Linux lab only**—use a VM or spare interface.

```
NIC RX -> XDP program -> map++ -> userspace Lookup every 1s
```

### Problem statement

Goals:

- Attach XDP to an interface index (or name)
- Kernel program counts packets (and optionally bytes)
- Userspace prints **packets=N** each second
- Clean detach on exit

### Acceptance criteria

- ☐ Document interface selection (`ip link`)
- ☐ Counter increases under traffic (`ping`, `curl`)
- ☐ Userspace reads map without racey panics
- ☐ Defer close/detach on SIGINT
- ☐ Lab safety notes (do not brick remote prod NIC)

### Setup

```
mkdir xdp-count && cd xdp-count
go mod init example.com/xdp-count
go get github.com/cilium/ebpf@latest
go install github.com/cilium/ebpf/cmd/bpf2go@latest
clang/llvm + headers as in project 33
go 1.27
```

## Kernel sketch count.bpf.c

```
//go:build ignore

#include "vmlinux.h"
#include <bpf/bpf_helpers.h>

char LICENSE[] SEC("license") = "Dual BSD/GPL";

struct {
 __uint(type, BPF_MAP_TYPE_ARRAY);
 __uint(max_entries, 1);
 __type(key, __u32);
 __type(value, __u64);
} packet_count SEC(".maps");

SEC("xdp")
int xdp_count(struct xdp_md *ctx) {
 __u32 key = 0;
 __u64 *val = bpf_map_lookup_elem(&packet_count, &key);
 if (val) {
 __sync_fetch_and_add(val, 1);
 }
 return XDP_PASS; // do not drop traffic
}
```

Generate:

```
//go:generate bpf2go -cc clang bpf count.bpf.c -- -I...
go generate ./...
```

## Userspace main.go

```
package main

import (
 "flag"
 "fmt"
 "log"
 "net"
 "os"
 "os/signal"
 "syscall"
 "time"
```

```

 "github.com/cilium/ebpf/link"
 "github.com/cilium/ebpf/rlimit"
)

func ifaceIndex(name string) (int, error) {
 iface, err := net.InterfaceByName(name)
 if err != nil {
 return 0, err
 }
 return iface.Index, nil
}

func main() {
 ifname := flag.String("iface", "lo", "interface name")
 flag.Parse()

 if err := rlimit.RemoveMemlock(); err != nil {
 log.Fatal(err)
 }

 idx, err := ifaceIndex(*ifname)
 if err != nil {
 log.Fatal(err)
 }

 objs := bpfObjects{}
 if err := loadBpfObjects(&objs, nil); err != nil {
 log.Fatalf("load: %v", err)
 }
 defer objs.Close()

 l, err := link.AttachXDP(link.XDPOptions{
 Program: objs.XdpCount, // name from bpf2go
 Interface: idx,
 })
 if err != nil {
 log.Fatalf("attach xdp on %s (%d): %v", *ifname, idx, err)
 }
 defer l.Close()

 fmt.Printf("XDP attached on %s index=%d; Ctrl+C to stop\n", *ifname, idx)

 sig := make(chan os.Signal, 1)
 signal.Notify(sig, os.Interrupt, syscall.SIGTERM)

 ticker := time.NewTicker(time.Second)

```

```

defer ticker.Stop()

for {
 select {
 case <-sig:
 fmt.Println("detaching")
 return
 case <-ticker.C:
 var key uint32 = 0
 var value uint64
 if err := objs.PacketCount.Lookup(&key, &value); err != nil {
 log.Println("lookup:", err)
 continue
 }
 fmt.Printf("packets=%d\n", value)
 }
}
}

```

Field names (`XdpCount`, `PacketCount`) must match `bpf2go` output—rename to match generated code.

## Run and verification

```

ip -br link
go generate ./... && go build -o xdp-count .
sudo ./xdp-count -iface lo
other terminal:
ping -c 20 127.0.0.1
counter should rise

```

## Safety

- Prefer **lab VMs** and `lo` or a throwaway veth pair.
- Always return `XDP_PASS` until you know what you are dropping.
- Remote servers: a bad XDP program can cut SSH—use out-of-band console.

## Stretch goals

1. Per-protocol counters (parse Ethernet/IP safely with helpers).
2. Bytes counter as well as packets.
3. `bpf2go` `prog show` / `ip link show` verification notes.
4. Pin maps under `/sys/fs/bpf` for multi-process readers.

## Pitfalls

Pitfall	Fix
Hard-coded ifindex	resolve by name
XDP_DROP by mistake	start with PASS
No memlock rlimit	<code>rlimit.RemoveMemlock</code>
Generated names mismatch	read <code>*_bpfel.go</code>

## Learning goals

- XDP attach path vs tracepoints
- Array maps for counters
- Operational safety for datapath programs

## **035 Project 35: Proxmox Multi-Node Scheduler**

# 035 Build a Proxmox Multi-Node Scheduler

Schedule VM placement across Proxmox nodes by free memory and CPU load. Produce an explainable ranking (dry-run) before you ever call migrate/create APIs.

```
fetch node stats -> score nodes -> rank placement order -> print plan
```

## Problem statement

CLI that:

- Authenticates with PVEAPIToken (env or flags)
- GET /api2/json/nodes for CPU/memory stats
- Scores each node: prefer free memory + CPU headroom
- Prints ranked placement list
- Never mutates the cluster in the base project (dry-run only)

## Acceptance criteria

- ☐ Fails clearly if env/flags missing
- ☐ Deterministic score function (unit-tested)
- ☐ Sorted highest score first
- ☐ HTTP client timeout + optional TLS skip for lab
- ☐ Dry-run only by default

## Setup

```
mkdir pvesched && cd pvesched
go mod init example.com/pvesched
go 1.27

export PVE_BASE_URL='https://pve.example:8006'
export PVE_TOKEN='user@pam!tokenid=secret'
```

## Full main.go

```
package main

import (
 "crypto/tls"
 "encoding/json"
 "flag"
 "fmt"
 "net/http"
 "os"
 "sort"
 "time"
)

type NodeStat struct {
 Node string `json:"node"`
 CPU float64 `json:"cpu"`
 Mem float64 `json:"mem"`
 MaxMem float64 `json:"maxmem"`
 Status string `json:"status"`
}

type apiResp struct {
 Data []NodeStat `json:"data"`
}

func score(n NodeStat) float64 {
 if n.MaxMem <= 0 {
 return -1
 }
 freeMemRatio := 1 - (n.Mem / n.MaxMem)
 if freeMemRatio < 0 {
 freeMemRatio = 0
 }
 cpuHeadroom := 1 - n.CPU
 if cpuHeadroom < 0 {
 cpuHeadroom = 0
 }
 // weight memory higher for VM placement heuristics
 return freeMemRatio*0.7 + cpuHeadroom*0.3
}

func fetchNodes(base, token string, insecure bool) ([]NodeStat, error) {
 tr := http.DefaultTransport.(*http.Transport).Clone()
 if insecure {
```



```

 tr.TLSClientConfig = &tls.Config{InsecureSkipVerify: true} // lab only
}
hc := &http.Client{Timeout: 8 * time.Second, Transport: tr}

req, err := http.NewRequest(http.MethodGet, base+"/api2/json/nodes", nil)
if err != nil {
 return nil, err
}
req.Header.Set("Authorization", "PVEAPIToken="+token)
resp, err := hc.Do(req)
if err != nil {
 return nil, err
}
defer resp.Body.Close()
if resp.StatusCode >= 300 {
 return nil, fmt.Errorf("status %d", resp.StatusCode)
}
var out apiResp
if err := json.NewDecoder(resp.Body).Decode(&out); err != nil {
 return nil, err
}
return out.Data, nil
}

func main() {
 base := flag.String("base", os.Getenv("PVE_BASE_URL"), "proxmox base url")
 token := flag.String("token", os.Getenv("PVE_TOKEN"), "api token")
 insecure := flag.Bool("insecure", true, "skip TLS verify (lab)")
 flag.Parse()

 if *base == "" || *token == "" {
 fmt.Fprintln(os.Stderr, "set -base/-token or PVE_BASE_URL/PVE_TOKEN")
 os.Exit(2)
 }

 nodes, err := fetchNodes(*base, *token, *insecure)
 if err != nil {
 fmt.Fprintln(os.Stderr, err)
 os.Exit(1)
 }

 sort.SliceStable(nodes, func(i, j int) bool {
 return score(nodes[i]) > score(nodes[j])
 })

 fmt.Println("placement order (dry-run):")

```

```

 for i, n := range nodes {
 fmt.Printf("%d) node=%s score=%.3f cpu=%.2f mem=%.0f/%.0f status=%s\n",
 i+1, n.Node, score(n), n.CPU, n.Mem, n.MaxMem, n.Status)
 }
 if len(nodes) > 0 {
 fmt.Printf("recommended: %s\n", nodes[0].Node)
 }
}

```

## Run and verification

```
go run . -base "$PVE_BASE_URL" -token "$PVE_TOKEN"
```

Offline unit test without API:

```

package main

import "testing"

func TestScorePrefersFreeMem(t *testing.T) {
 a := NodeStat{Mem: 10, MaxMem: 100, CPU: 0.5}
 b := NodeStat{Mem: 90, MaxMem: 100, CPU: 0.1}
 if score(a) <= score(b) {
 t.Fatalf("a=%f b=%f", score(a), score(b))
 }
}

```

## Stretch goals

1. Exclude nodes with `status != online`.
2. Factor disk free space into score.
3. Emit JSON plan for a later apply step.
4. Actually create VM on top-ranked node (dangerous—gate with `-apply`).

## Pitfalls

Pitfall	Fix
No timeout	hung CLI
Trusting lab TLS skip in prod	proper CA
Non-explainable ML scores	keep weighted formula printed
Mutating by default	dry-run first

## Learning goals

- Explainable scheduling scores
- Proxmox API auth patterns
- Safe automation with dry-run defaults

## **036 Project 36: Proxmox Capacity Balancer**

# 036 Build a Proxmox Capacity Balancer (Dry-Run)

Generate **migration suggestions** from overloaded nodes to healthier nodes. Start with an in-memory model you can unit-test; optionally wire Proxmox APIs later.

```
node CPU map -> find hottest + coolest -> suggest migration path
```

## Problem statement

Given per-node CPU utilization (and optionally a list of VMs per node):

1. Identify source = highest CPU above threshold
2. Identify destination = lowest CPU with headroom
3. Print a dry-run migration suggestion
4. Never call migrate unless `-apply` (stretch)

## Acceptance criteria

- ☐ Deterministic suggestion from fixed input
- ☐ No suggestion when cluster is balanced
- ☐ Threshold flags (`-hot`, `-cool`)
- ☐ Table output suitable for operators
- ☐ Stdlib only for base version

## Setup

```
mkdir pvebalance && cd pvebalance
go mod init example.com/pvebalance
go 1.27
```

## Full main.go

```
package main

import (
 "encoding/json"
 "flag"
 "fmt"
 "os"
 "sort"
)

type Node struct {
 Name string `json:"name"`
 CPU float64 `json:"cpu" // 0..1
}

type Suggestion struct {
 From string `json:"from"`
 To string `json:"to"`
 FromCPU float64 `json:"from_cpu"`
 ToCPU float64 `json:"to_cpu"`
 Reason string `json:"reason"`
}

func suggest(nodes []Node, hot, cool float64) *Suggestion {
 if len(nodes) < 2 {
 return nil
 }
 src := nodes[0]
 dst := nodes[0]
 for _, n := range nodes[1:] {
 if n.CPU > src.CPU {
 src = n
 }
 if n.CPU < dst.CPU {
 dst = n
 }
 }
 if src.Name == dst.Name {
 return nil
 }
 if src.CPU < hot {
 return nil // nothing hot enough
 }
 if dst.CPU > cool {
```

```

 return nil // no cool target
 }
 return &Suggestion{
 From: src.Name, To: dst.Name,
 FromCPU: src.CPU, ToCPU: dst.CPU,
 Reason: fmt.Sprintf("src cpu %.2f >= hot %.2f; dst cpu %.2f <= cool %.2f",
 src.CPU, hot, dst.CPU, cool),
 }
}

func main() {
 hot := flag.Float64("hot", 0.85, "source CPU threshold")
 cool := flag.Float64("cool", 0.50, "destination max CPU")
 demo := flag.Bool("demo", true, "use built-in demo nodes")
 flag.Parse()

 var nodes []Node
 if *demo {
 nodes = []Node{
 {"pve1", 0.92},
 {"pve2", 0.35},
 {"pve3", 0.41},
 }
 } else {
 // optional: read JSON array from stdin
 if err := json.NewDecoder(os.Stdin).Decode(&nodes); err != nil {
 fmt.Fprintln(os.Stderr, "stdin json:", err)
 os.Exit(2)
 }
 }

 sort.Slice(nodes, func(i, j int) bool { return nodes[i].Name < nodes[j].Name })
 fmt.Println("cluster:")
 for _, n := range nodes {
 fmt.Printf(" %-8s cpu=%.2f\n", n.Name, n.CPU)
 }

 s := suggest(nodes, *hot, *cool)
 if s == nil {
 fmt.Println("suggest: none (balanced or thresholds not met)")
 return
 }
 fmt.Printf("suggest migration: %s -> %s (cpu %.2f -> %.2f)\n", s.From, s.To, s.FromCPU,
 fmt.Println("reason:", s.Reason)
 fmt.Println("mode: dry-run (no API calls)")
}

```

## Run and verification

```
go run .
suggest migration: pve1 -> pve2 ...

go run . -hot 0.99
none

echo '["name":"a","cpu":0.9],["name":"b","cpu":0.2]' | go run . -demo=false
```

## Tests

```
package main

import "testing"

func TestSuggest(t *testing.T) {
 nodes := []Node{{"pve1", 0.92}, {"pve2", 0.35}, {"pve3", 0.41}}
 s := suggest(nodes, 0.85, 0.5)
 if s == nil || s.From != "pve1" || s.To != "pve2" {
 t.Fatalf("%+v", s)
 }
}

func TestNoSuggestWhenCool(t *testing.T) {
 nodes := []Node{{"a", 0.9}, {"b", 0.8}}
 if s := suggest(nodes, 0.85, 0.5); s != nil {
 t.Fatal(s)
 }
}

go test ./...
```

## Stretch goals

1. Pick a specific VM on the hot node (largest CPU VM).
2. Fetch live stats from Proxmox API (project 35 client).
3. Simulate after-move CPU estimates.
4. -apply with confirmation prompt.

## Pitfalls



Pitfall	Fix
Migrating without headroom check	cool threshold
Oscillation (ping-pong)	hysteresis / cooldown timer
Ignoring maintenance nodes	status filter

## Learning goals

- Capacity balancing as scored suggestions
- Dry-run first automation
- Testable pure functions for ops tools

## **037 Project 37: Kubernetes HPA Simulator**

## 037 Build a Kubernetes HPA Simulator

Simulate Horizontal Pod Autoscaler-style replica scaling from a CPU load time series. Learn the control loop (target utilization, min/max replicas, scale-up/down rules) without a cluster.

```
cpu samples -> compare to target -> adjust replicas within [min,max] -> log
```

### Problem statement

Model a simplified HPA:

- Target average CPU percent (e.g. 60%)
- Current replicas
- Each tick: observe CPU, scale up if sustained high, scale down if sustained low
- Enforce min/max replicas
- Optional stabilization: require N consecutive ticks before scale down

### Acceptance criteria

- ☐ Replicas never leave [min, max]
- ☐ High CPU increases replicas (until max)
- ☐ Low CPU decreases replicas (until min) with optional hysteresis
- ☐ Deterministic mode with fixed seed or fixed series
- ☐ Unit tests for the pure step function

### Setup

```
mkdir hpasim && cd hpasim
go mod init example.com/hpasim
go 1.27
```

## Full main.go

```
package main

import (
 "flag"
 "fmt"
 "math/rand"
 "os"
)

type Config struct {
 TargetCPU float64
 MinReplicas int
 MaxReplicas int
 UpBand float64 // scale up if cpu > target+UpBand
 DownBand float64 // scale down if cpu < target-DownBand
 DownStable int // consecutive low ticks required
}

type State struct {
 Replicas int
 LowStreak int
}

func step(cfg Config, s State, cpu float64) (State, string) {
 if s.Replicas < cfg.MinReplicas {
 s.Replicas = cfg.MinReplicas
 }
 action := "noop"

 if cpu > cfg.TargetCPU+cfg.UpBand && s.Replicas < cfg.MaxReplicas {
 s.Replicas++
 s.LowStreak = 0
 return s, "scale_up"
 }
 if cpu < cfg.TargetCPU-cfg.DownBand {
 s.LowStreak++
 if s.LowStreak >= cfg.DownStable && s.Replicas > cfg.MinReplicas {
 s.Replicas--
 s.LowStreak = 0
 return s, "scale_down"
 }
 return s, "hold_low"
 }
 s.LowStreak = 0
}
```

```

 return s, action
}

func main() {
 ticks := flag.Int("ticks", 30, "simulation ticks")
 target := flag.Float64("target", 60, "target CPU percent")
 minR := flag.Int("min", 1, "min replicas")
 maxR := flag.Int("max", 20, "max replicas")
 seed := flag.Int64("seed", 42, "rng seed")
 demo := flag.Bool("demo-series", false, "use fixed series instead of random")
 flag.Parse()

 if *minR < 1 || *maxR < *minR {
 fmt.Fprintln(os.Stderr, "invalid min/max")
 os.Exit(2)
 }

 cfg := Config{
 TargetCPU: *target,
 MinReplicas: *minR,
 MaxReplicas: *maxR,
 UpBand: 10,
 DownBand: 20,
 DownStable: 2,
 }
 s := State{Replicas: *minR}
 rng := rand.New(rand.NewSource(*seed))

 fixed := []float64{70, 82, 90, 88, 91, 72, 55, 40, 38, 35, 60, 65}

 for t := 1; t <= *ticks; t++ {
 var cpu float64
 if *demo {
 cpu = fixed[(t-1)%len(fixed)]
 } else {
 cpu = 30 + rng.Float64()*70
 }
 var action string
 s, action = step(cfg, s, cpu)
 fmt.Printf("tick=%02d cpu=%5.1f%% replicas=%d action=%s\n", t, cpu, s.Replicas, action)
 }
}

```

## Run and verification

```
go run . -demo-series -ticks 12
go run . -seed 1 -ticks 30
```

## Tests

```
package main

import "testing"

func TestScaleUp(t *testing.T) {
 cfg := Config{TargetCPU: 60, MinReplicas: 1, MaxReplicas: 5, UpBand: 10, DownBand: 20, D
 s := State{Replicas: 1}
 s, a := step(cfg, s, 90)
 if a != "scale_up" || s.Replicas != 2 {
 t.Fatalf("%s %d", a, s.Replicas)
 }
}

func TestRespectMax(t *testing.T) {
 cfg := Config{TargetCPU: 60, MinReplicas: 1, MaxReplicas: 2, UpBand: 0, DownBand: 20, Do
 s := State{Replicas: 2}
 s, a := step(cfg, s, 99)
 if a != "noop" || s.Replicas != 2 {
 t.Fatalf("%s %d", a, s.Replicas)
 }
}

func TestScaleDownNeedsStreak(t *testing.T) {
 cfg := Config{TargetCPU: 60, MinReplicas: 1, MaxReplicas: 10, UpBand: 10, DownBand: 20,
 s := State{Replicas: 3}
 s, _ = step(cfg, s, 20)
 if s.Replicas != 3 {
 t.Fatal("scaled too early")
 }
 s, a := step(cfg, s, 20)
 if a != "scale_down" || s.Replicas != 2 {
 t.Fatalf("%s %d", a, s.Replicas)
 }
}

go test ./...
```

## Stretch goals

1. DesiredReplicas formula: `ceil(current * cpu/target)` like real HPA.
2. Separate scale-up/scale-down cooldown timers.
3. Multi-metric (CPU + RPS).
4. Compare against recorded metrics CSV.

## Pitfalls

Pitfall	Fix
Flapping replicas	hysteresis bands + down stable streak
Ignoring min/max	clamp every step
Random-only demos	fixed series for docs/tests

## Learning goals

- HPA control-loop intuition
- Stabilization windows
- Pure step functions for simulations

## **038 Project 38: Terraform Cost Estimator**



## 038 Build a Terraform Cost Estimator (Simple)

Estimate monthly cost **delta** from a Terraform plan JSON by mapping resource types to rough unit prices. Not a substitute for Infracost—but excellent for learning plan analysis and CI annotations.

```
plan.json -> actions × unit price table -> monthly delta $
```

### Problem statement

```
tf-cost [-prices prices.json] <plan.json>
```

- **create** adds unit monthly cost
- **delete** subtracts unit monthly cost
- **replace** treat as delete+create (net depends on type change; base: same type → ~0)
- Unknown types contribute 0 (list them as warnings)

### Acceptance criteria

- ☐ Parses plan JSON `resource_changes`
- ☐ Prints estimated monthly delta
- ☐ Warns on unknown types that change
- ☐ Exit 2 on parse errors; 0 on success
- ☐ Prices overridable via JSON file

### Setup

```
mkdir tfcost && cd tfcost
go mod init example.com/tfcost
go 1.27
```

prices.json example:

```
{
 "aws_instance": 15.0,
 "aws_db_instance": 120.0,
 "aws_lb": 25.0
```

```
}
```

## Full main.go

```
package main

import (
 "encoding/json"
 "flag"
 "fmt"
 "os"
)

type Plan struct {
 ResourceChanges []struct {
 Address string `json:"address"`
 Type string `json:"type"`
 Change struct {
 Actions []string `json:"actions"`
 } `json:"change"`
 } `json:"resource_changes"`
}

func loadPrices(path string) (map[string]float64, error) {
 defaults := map[string]float64{
 "aws_instance": 15.0,
 "aws_db_instance": 120.0,
 "aws_lb": 25.0,
 }
 if path == "" {
 return defaults, nil
 }
 b, err := os.ReadFile(path)
 if err != nil {
 return nil, err
 }
 var m map[string]float64
 if err := json.Unmarshal(b, &m); err != nil {
 return nil, err
 }
 for k, v := range defaults {
 if _, ok := m[k]; !ok {
 m[k] = v
 }
 }
}
```

```

 return m, nil
}

func estimate(p Plan, cost map[string]float64) (delta float64, unknown []string) {
 seenUnknown := map[string]bool{}
 for _, rc := range p.ResourceChanges {
 unit, ok := cost[rc.Type]
 if !ok {
 if !seenUnknown[rc.Type] && len(rc.Change.Actions) > 0 && rc.Change.Actions[0] != "" {
 // track types that actually change
 for _, a := range rc.Change.Actions {
 if a == "create" || a == "delete" {
 seenUnknown[rc.Type] = true
 unknown = append(unknown, rc.Type)
 break
 }
 }
 }
 continue
 }
 actions := rc.Change.Actions
 if len(actions) == 2 && actions[0] == "delete" && actions[1] == "create" {
 // same type replace → approx net 0 for this toy model
 continue
 }
 for _, a := range actions {
 switch a {
 case "create":
 delta += unit
 fmt.Printf("+ %.2f %s (%s)\n", unit, rc.Address, rc.Type)
 case "delete":
 delta -= unit
 fmt.Printf("- %.2f %s (%s)\n", unit, rc.Address, rc.Type)
 }
 }
 }
 return delta, unknown
}

func main() {
 pricesPath := flag.String("prices", "", "optional prices JSON")
 flag.Parse()
 if flag.NArg() != 1 {
 fmt.Fprintln(os.Stderr, "usage: tf-cost [-prices prices.json] <plan.json>")
 os.Exit(2)
 }
}

```

```

cost, err := loadPrices(*pricesPath)
if err != nil {
 fmt.Fprintln(os.Stderr, err)
 os.Exit(2)
}
b, err := os.ReadFile(flag.Arg(0))
if err != nil {
 fmt.Fprintln(os.Stderr, err)
 os.Exit(2)
}
var p Plan
if err := json.Unmarshal(b, &p); err != nil {
 fmt.Fprintln(os.Stderr, "json:", err)
 os.Exit(2)
}

delta, unknown := estimate(p, cost)
for _, u := range unknown {
 fmt.Fprintf(os.Stderr, "warn: no price for type %s\n", u)
}
fmt.Printf("estimated monthly delta: $%.2f\n", delta)
}

```

## Run and verification

```

go run . plan.json
go run . -prices prices.json plan.json

```

## Tests

```

package main

import "testing"

func TestEstimateCreate(t *testing.T) {
 var p Plan
 rc := p.ResourceChanges
 _ = rc
 item := struct {
 Address string `json:"address"`
 Type string `json:"type"`
 Change struct {
 Actions []string `json:"actions"`
 }
 }
}

```

```

 } `json:"change"`
 }{Address: "aws_instance.web", Type: "aws_instance"}
 item.Change.Actions = []string{"create"}
 p.ResourceChanges = append(p.ResourceChanges, item)
 delta, _ := estimate(p, map[string]float64{"aws_instance": 15})
 if delta != 15 {
 t.Fatalf("%v", delta)
 }
}

```

## Stretch goals

1. Regional price multipliers.
2. Count module expansion addresses.
3. Fail CI if delta > budget (-max-delta 50).
4. Integrate with risk reporter output.

## Pitfalls

Pitfall	Fix
Treating prices as accurate	label as rough estimate
Ignoring replace	define policy (net 0 vs full)
Double-counting	classify actions carefully

## Learning goals

- Plan-driven financial signals
- Extensible price tables
- Honest limits of toy cost models

## **039 Project 39: Prometheus Alert Rule Tester**

# 039 Build a Prometheus Alert Rule Tester

Evaluate synthetic metric streams against threshold rules with a **for**-style duration (consecutive steps). This teaches alert flapping, hysteresis, and unit-testing rules without a live Prometheus.

```
values[] + threshold + forSteps -> streak -> alert bool per tick
```

## Problem statement

Given:

- A series of float samples (CPU%, error rate, ...)
- **threshold** and **forSteps** (like **for: 3m** if step=1m)
- Emit per-tick: value, streak, alert firing?

Also support loading samples from a CSV file.

## Acceptance criteria

- ☐ Alert fires only after **forSteps** consecutive breaches
- ☐ Streak resets when value drops to/ below threshold
- ☐ Pure function is unit-tested
- ☐ CLI demo with built-in series
- ☐ Stdlib only

## Setup

```
mkdir alertrule && cd alertrule
go mod init example.com/alertrule
go 1.27
```

## Full main.go

```
package main

import (
 "encoding/csv"
 "flag"
 "fmt"
 "io"
 "os"
 "strconv"
)

// shouldAlert updates streak based on value and returns whether alert is firing.
// Pass streak by pointer so callers keep state across ticks.
func shouldAlert(v, threshold float64, forSteps int, streak *int) bool {
 if v > threshold {
 *streak++
 } else {
 *streak = 0
 }
 return *streak >= forSteps
}

func evalSeries(vals []float64, threshold float64, forSteps int) []bool {
 out := make([]bool, len(vals))
 streak := 0
 for i, v := range vals {
 out[i] = shouldAlert(v, threshold, forSteps, &streak)
 }
 return out
}

func loadCSV(path string) ([]float64, error) {
 f, err := os.Open(path)
 if err != nil {
 return nil, err
 }
 defer f.Close()
 r := csv.NewReader(f)
 var vals []float64
 for {
 rec, err := r.Read()
 if err == io.EOF {
 break
 }
 }
}
```



```

 if err != nil {
 return nil, err
 }
 if len(rec) == 0 {
 continue
 }
 v, err := strconv.ParseFloat(rec[0], 64)
 if err != nil {
 continue // skip header/bad
 }
 vals = append(vals, v)
 }
 return vals, nil
}

func main() {
 threshold := flag.Float64("threshold", 85, "alert if value > threshold")
 forSteps := flag.Int("for", 3, "consecutive steps above threshold")
 csvPath := flag.String("csv", "", "optional CSV of values (first column)")
 flag.Parse()

 vals := []float64{70, 82, 90, 88, 91, 72, 95, 96, 97}
 if *csvPath != "" {
 var err error
 vals, err = loadCSV(*csvPath)
 if err != nil {
 fmt.Fprintln(os.Stderr, err)
 os.Exit(1)
 }
 }

 streak := 0
 fmt.Printf("threshold=%.1f for=%d steps\n", *threshold, *forSteps)
 for i, v := range vals {
 alert := shouldAlert(v, *threshold, *forSteps, &streak)
 fmt.Printf("t=%d val=%.1f streak=%d alert=%v\n", i, v, streak, alert)
 }
}

```

## Run and verification

```

go run .
t=0.. alert becomes true once streak hits 3

CSV:

```

```
printf '10\n90\n91\n92\n50\n' > s.csv
go run . -csv s.csv -threshold 85 -for 3
```

## Tests

```
package main

import "testing"

func TestForDuration(t *testing.T) {
 vals := []float64{90, 90, 90, 10}
 got := evalSeries(vals, 85, 3)
 want := []bool{false, false, true, false}
 for i := range want {
 if got[i] != want[i] {
 t.Fatalf("i=%d got %v want %v", i, got, want)
 }
 }
}

func TestReset(t *testing.T) {
 vals := []float64{90, 10, 90, 90}
 got := evalSeries(vals, 85, 2)
 if got[1] || got[2] || !got[3] {
 t.Fatalf("%v", got)
 }
}

go test ./...
```

## Stretch goals

1. Dual thresholds (warn vs critical).
2. `avg_over_time` style window instead of consecutive.
3. Parse a tiny subset of Prometheus rule YAML.
4. Generate a markdown alert timeline for PR review.

## Pitfalls

Pitfall	Fix
Firing on first sample	enforce <code>forSteps</code>

Pitfall	Fix
Off-by-one streak	unit tests with tables
Flapping	longer for + cool-down (stretch)

## Learning goals

- Alert for duration semantics
- Deterministic rule testing
- Avoiding noisy pages with streaks

## **040 Project 40: Infra Reconciler Daemon**

# 040 Build an Infra Reconciler Daemon

Model a control-loop daemon that continuously moves **actual** state toward **desired** state. This is the same mental model as Kubernetes controllers and GitOps agents, stripped to a few dozen lines you can reason about.

```
desired config -> diff -> apply one action -> observe -> repeat
```

## Problem statement

Daemon-style process:

- Holds **Desired** and **Actual** integers (stand-in for replica counts, firewall rules, ...)
- Every tick, apply **at most one** idempotent action
- Desired can change mid-run (config reload simulation)
- Actual can drift (external change)
- Logs every action for auditability
- Stops after max ticks or SIGINT

## Acceptance criteria

- ☐ `diff` and `apply` are pure/testable
- ☐ Converges when desired stable
- ☐ Survives desired changes and drift
- ☐ Graceful shutdown on signal
- ☐ Stdlib only

## Setup

```
mkdir reconcilerd && cd reconcilerd
go mod init example.com/reconcilerd
go 1.27
```

## Full main.go

```
package main

import (
 "context"
 "flag"
 "fmt"
 "math/rand"
 "os"
 "os/signal"
 "syscall"
 "time"
)

type state struct {
 Desired int
 Actual int
}

func diff(s state) int { return s.Desired - s.Actual }

func apply(s *state) string {
 d := diff(*s)
 switch {
 case d > 0:
 s.Actual++
 return "scale_up"
 case d < 0:
 s.Actual--
 return "scale_down"
 default:
 return "noop"
 }
}

func main() {
 desired := flag.Int("desired", 5, "initial desired")
 actual := flag.Int("actual", 1, "initial actual")
 ticks := flag.Int("ticks", 40, "max ticks (0=until signal)")
 interval := flag.Duration("interval", 200*time.Millisecond, "tick interval")
 seed := flag.Int64("seed", 1, "drift rng seed")
 driftProb := flag.Float64("drift", 0.1, "per-tick drift probability")
 flag.Parse()

 ctx, stop := signal.NotifyContext(context.Background(), os.Interrupt, syscall.SIGTERM)
```

```

defer stop()

s := state{Desired: *desired, Actual: *actual}
rng := rand.New(rand.NewSource(*seed))
ticker := time.NewTicker(*interval)
defer ticker.Stop()

fmt.Printf("reconciler start desired=%d actual=%d\n", s.Desired, s.Actual)

for i := 0; *ticks == 0 || i < *ticks; i++ {
 select {
 case <-ctx.Done():
 fmt.Println("shutdown")
 return
 case <-ticker.C:
 }

 // simulate config reload
 if i == 10 {
 s.Desired = 3
 fmt.Printf("tick=%02d CONFIG desired=%d\n", i, s.Desired)
 }

 // simulate external drift
 if rng.Float64() < *driftProb {
 delta := rng.Intn(3) - 1
 s.Actual += delta
 if s.Actual < 0 {
 s.Actual = 0
 }
 fmt.Printf("tick=%02d DRIFT delta=%+d actual=%d\n", i, delta, s.Actual)
 }

 action := apply(&s)
 fmt.Printf("tick=%02d desired=%d actual=%d action=%s diff=%d\n",
 i, s.Desired, s.Actual, action, diff(s))
}
fmt.Println("max ticks reached")
}

```

## Architecture

```
 watch/load
desired.yaml reconciler
 loop

 apply

 actual world
```

## Run and verification

```
go run . -desired 5 -actual 0 -ticks 25 -drift 0.15 -interval 50ms
expect scale_up to 5, config change to 3, scale_down, occasional drift recovery
```

## Tests

```
package main

import "testing"

func TestApplyConverges(t *testing.T) {
 s := state{Desired: 3, Actual: 0}
 for i := 0; i < 10; i++ {
 apply(&s)
 }
 if s.Actual != 3 || diff(s) != 0 {
 t.Fatalf("%+v", s)
 }
}

func TestNoop(t *testing.T) {
 s := state{Desired: 2, Actual: 2}
 if apply(&s) != "noop" {
 t.Fatal()
 }
}

go test -race ./...
```



## Stretch goals

1. Load desired from JSON file; reload on `SIGHUP`.
2. Workqueue of resource IDs.
3. Per-resource last-error backoff.
4. Metrics: `reconciles_total`, `drift_total`.
5. Multi-field state (not just int).

## Pitfalls

Pitfall	Fix
Applying huge diffs in one tick	one step; requeue
No drift handling	re-observe actual every tick
Busy loop	ticker / event driven
Non-idempotent apply	same desired → safe noop

## Learning goals

- Level-triggered reconciliation
- Drift and config change handling
- Daemon lifecycle with signals

## **041 Project 41: TinyGo Blink & Toolchain**

# 041 TinyGo: Toolchain, Targets, and Blink

Build your first **TinyGo** firmware loop: install the toolchain, pick a board target, flash (or run in the WASM/simulator path), and understand how TinyGo differs from desktop Go.

**Baseline:** TinyGo current stable (track [tinygo.org](https://tinygo.org) release notes); Go 1.22+ host toolchain recommended for building TinyGo itself if you compile from source.

## Why TinyGo

Desktop Go	TinyGo
GC, large runtime, OS threads	Smaller runtime, selective GC / freelist modes by target
net/http, rich stdlib	Subset of stdlib + <code>machine</code> / board packages
<code>go build</code> → OS binary	<code>tinygo build -target=...</code> → firmware / WASM
Soft realtime only	Soft realtime loops + hardware interrupts (not hard RTOS guarantees)

TinyGo is still **Go syntax and modules**, but the **target** decides available packages and size.

Host (dev machine)	Device / WASM
<code>tinygo build -target=pico</code>	<code>flash</code> MCU firmware
<code>tinygo run -target=wasip1</code>	<code>wasmtime</code> / browser

## Step 1: Install TinyGo

```
macOS (Homebrew example - verify current formula)
brew install tinygo

or follow https://tinygo.org/getting-started/install/

tinygo version
go version # host Go still useful for module tools
```

List targets:

```
tinygo targets | head
tinygo info pico # Raspberry Pi Pico example
tinygo info wasip1 # WASM for host experiments without hardware
```

## Step 2: Module + blink (board-agnostic pattern)

```
mkdir tiny-blink && cd tiny-blink
go mod init example.com/tiny-blink
```

main.go — replace pin with your board's LED (see board docs):

```
package main

import (
 "machine"
 "time"
)

func main() {
 led := machine.LED // many boards define LED; else machine.GPIO25 etc.
 led.Configure(machine.PinConfig{Mode: machine.PinOutput})

 for {
 led.High()
 time.Sleep(500 * time.Millisecond)
 led.Low()
 time.Sleep(500 * time.Millisecond)
 }
}
```

## No hardware? Use WASM smoke build

```
Compile-only check that TinyGo accepts the package shape
(blink needs machine.LED - for pure host, use the wasip1 project below)
tinygo build -o /tmp/check.elf -target=pico .
```

## Host-only hello (no GPIO):

```
// main_wasip1.go - build with -target=wasip1
package main
```

```
import "fmt"

func main() {
 fmt.Println("tinygo wasip1 ok")
}

tinygo run -target=wasip1 .
or: tinygo build -o hello.wasm -target=wasip1 . && wasmtime hello.wasm
```

## Step 3: Flash (hardware)

Examples (exact flags change by board/probe):

```
Pico UF2 style (common workflow)
tinygo flash -target=pico .

Some boards use -port
tinygo flash -target=arduino-nano33 -port=/dev/ttyACM0 .
```

If flash fails: check udev rules (Linux), cable is data-capable, bootloader mode (Pico BOOTSEL).

## Step 4: Size and optimization awareness

```
tinygo build -o firmware.elf -target=pico -size=full .
```

Note code/data/BSS. Optimization flags (`-opt=z` etc.) matter on small MCUs—read current TinyGo docs for supported `-opt` values.

## Learning Goals

- Install and verify TinyGo; list targets
- Separate **host Go** vs **device build**
- Produce a continuous loop firmware or WASM hello
- Know where to look for pin definitions (**machine**, board packages)

## Extensions

1. Blink two LEDs out of phase.
2. Print `time.Now()` over USB serial if the target supports it (`machine.Serial`).
3. Compare binary size with `-opt` variants.

## Common gotchas

Issue	Fix
<code>machine.LED</code> undefined	Wrong target or board; set pin explicitly
Flash permission denied	udev / cable / bootloader mode
Import works in <code>go build</code> but not TinyGo	Package not in TinyGo stdlib subset
Expect hard realtime	Soft RT only; measure jitter on real hardware

## Checkpoint

- ☐ `tinygo` version works
- ☐ Built for at least one target
- ☐ Blink **or** `wasip1` hello runs
- ☐ Notes: board name + pin used

## 042 Project 42: TinyGo Button + Interrupt

# 042 TinyGo: Button Input and Interrupt-Driven Events

Move from a pure **sleep** loop to **event-driven** MCU code: poll a button, then (where the target supports it) use a pin interrupt so the main loop stays free for other work.

## Realtime framing

Style	Latency	CPU use	Typical use
Polling in <code>for { }</code>	Bounded by loop rate	Busy or delayed by sleeps	Simple labs
Interrupt on edge	Faster reaction	Idle-friendly	Buttons, encoders, IRQs
Timer IRQ / PWM	Periodic deadlines	Predictable ticks	Soft realtime control

TinyGo is **not** a certified hard-realtime OS. Treat deadlines as **best-effort**: measure, keep ISRs short, defer heavy work to the main loop.

Edge on BUTTON pin

IRQ handler    set flag / push event    main loop reacts (debounce, LED, UART)

## Step 1: Polling baseline (all targets)

```
package main

import (
 "machine"
 "time"
)

func main() {
 led := machine.LED
 led.Configure(machine.PinConfig{Mode: machine.PinOutput})
```



```

btn := machine.BUTTON // or explicit GPIO; active-low common with pull-up
btn.Configure(machine.PinConfig{Mode: machine.PinInputPullup})

var last bool = true // pull-up idle high
for {
 v := btn.Get()
 if v != last {
 // simple debounce
 time.Sleep(20 * time.Millisecond)
 if btn.Get() == v {
 last = v
 if !v { // pressed (active low)
 led.Set(!led.Get())
 }
 }
 }
 time.Sleep(5 * time.Millisecond)
}
}

tinygo flash -target=YOUR_BOARD .

```

## Step 2: Interrupt pattern (when supported)

API names vary by TinyGo version/board. Sketch:

```

package main

import (
 "machine"
 "time"
)

var pressed = make(chan struct{}, 1)

func main() {
 led := machine.LED
 led.Configure(machine.PinConfig{Mode: machine.PinOutput})

 btn := machine.BUTTON
 btn.Configure(machine.PinConfig{Mode: machine.PinInputPullup})

 // SetInterrupt API differs by target - consult tinygo.org machine docs for your board.
 // Conceptual:

```

```

// btn.SetInterrupt(machine.PinFalling, func(p machine.Pin) {
// select {
// case pressed <- struct{}{}:
// default:
// }
// })

// Fallback if interrupt API unavailable: keep polling in a tight loop
// and only document the interrupt path for boards that support it.

for {
 select {
 case <-pressed:
 led.Set(!led.Get())
 time.Sleep(50 * time.Millisecond) // debounce after event
 default:
 // optional idle work / low-power sleep if target supports
 time.Sleep(1 * time.Millisecond)
 }
}
}

```

**Lab requirement:** Implement **working** code for **your** board. If interrupts are unsupported, complete polling + debounce and write a short note: *“Interrupt API N/A on target X; measured poll latency ...”*

### Step 3: Debounce theory

Mechanical buttons bounce for 5–50 ms. Strategies:

1. **Ignore window** after edge (software).
2. **Require stable samples** N times.
3. **Hardware RC** (optional).

Never toggle LEDs raw in an ISR without debounce policy—you will get multi-fires.

### Step 4: Measure soft-realtime behavior

Metric	How
Reaction latency	Scope or toggle second pin; or timestamp over UART
Jitter	Stddev of latency over 50 presses

Metric	How
Missed events	Channel full / default branch count

Log 50 presses; record min/avg/max latency if you have a serial console.

## Learning Goals

- Poll vs interrupt trade-offs
- Debounce as a correctness requirement
- Keep interrupt work minimal
- Honest soft-realtime expectations

## Extensions

1. Long-press vs short-press detection.
2. Two buttons with different actions.
3. Disable interrupts during critical section if API allows.

## Checkpoint

- ☐ Reliable press detection (no multi-toggle spam)
- ☐ Debounce implemented and explained
- ☐ Interrupt **or** documented fallback
- ☐ 50-press notes (even qualitative)

## **043 Project 43: TinyGo UART Sensor Loop**

# 043 TinyGo: UART Console + Timed Sensor Loop

Build a **periodic control loop**: sample a sensor (or simulated ADC), emit CSV over serial, and keep loop timing steady enough for soft realtime logging.

## Architecture

```
tick every T ms

read sensor / mock filter (EMA) UART CSV line

deadline miss counter++
```

## Step 1: Serial hello

```
package main

import (
 "fmt"
 "machine"
 "time"
)

func main() {
 machine.Serial.Configure(machine.UARTConfig{BaudRate: 115200})
 for {
 fmt.Fprintf(machine.Serial, "hello tinygo %d\n", time.Now().UnixNano())
 time.Sleep(time.Second)
 }
}
```

Host:

```
Linux example - adjust device
screen /dev/ttyACM0 115200
```

```
or: picocom -b 115200 /dev/ttyACM0
```

## Step 2: Fixed-rate loop with miss detection

```
package main

import (
 "fmt"
 "machine"
 "time"
)

const period = 50 * time.Millisecond // 20 Hz

func readMockSensor() int {
 // Replace with machine.ADC when available on your board
 return int(time.Now().UnixNano()%1000) / 10 // 0..99 fake
}

func main() {
 machine.Serial.Configure(machine.UARTConfig{BaudRate: 115200})

 var ema float32
 var misses int
 next := time.Now()

 for {
 start := time.Now()
 if start.After(next.Add(period / 2)) {
 misses++
 }

 raw := readMockSensor()
 // EMA: y = x + (1-y)y
 const alpha = 0.2
 ema = alpha*float32(raw) + (1-alpha)*ema

 fmt.Fprintf(machine.Serial, "t_ns=%d raw=%d ema=%.2f misses=%d\n",
 start.UnixNano(), raw, ema, misses)

 next = next.Add(period)
 if d := time.Until(next); d > 0 {
 time.Sleep(d)
 } else {
```

```

 // fell behind - resync
 next = time.Now().Add(period)
 misses++
 }
}
}

```

### Step 3: Real ADC (when target supports it)

Concept:

```

import "machine"

var adc machine.ADC

func initADC() {
 adc = machine.ADC{Pin: machine.ADC0} // board-specific
 adc.Configure(machine.ADCCfg{})
}

func readMockSensor() int {
 return int(adc.Get()) // width depends on target
}

```

Consult your board's pinout. If no ADC: keep mock and still complete timing/miss metrics.

### Step 4: Soft realtime checklist

Practice	Why
Fixed period	Stable sample rate for filters
Count deadline misses	Visibility when loop overruns
Avoid allocs in hot loop	GC/heap pressure on tiny heaps
Short UART lines	Don't block forever on full buffers

Optional: build with size report and note RAM pressure.

### Learning Goals

- UART as primary debug channel
- Timed loops and miss accounting

- Simple digital filter (EMA)
- Path to real ADC sensors

## Extensions

1. Change rate via serial command (`r 100`  $\rightarrow$  100 ms).
2. Threshold alarm: print **ALERT** when ema exceeds N.
3. Dual-rate: fast sample, slow print.

## Checkpoint

- ☐ Serial output visible on host
- ☐ Fixed-rate loop with miss counter
- ☐ EMA or similar filter
- ☐ Notes: period chosen and why



## **044 Project 44: TinyGo Cooperative Tasks (Soft RT)**

# 044 TinyGo: Cooperative Multi-Tasking Without a Full RTOS

Implement a **tiny cooperative scheduler**: multiple tasks share one core by yielding, with per-task periods—useful mental model for soft realtime systems and for understanding why preemptive RTOS exists.

## Theory (short)

Model	Description
<b>Cooperative</b>	Task runs until it yields; bad task starves others
<b>Preemptive RTOS</b>	Timer IRQ switches tasks; needs careful shared data
<b>This lab</b>	Cooperative + fixed time slices via <code>time</code> checks

```
loop forever:
 for each task:
 if now >= task.next:
 task.run()
 task.next += task.period
```

## Step 1: Task table

```
package main

import (
 "fmt"
 "machine"
 "time"
)

type Task struct {
 Name string
 Period time.Duration
 Next time.Time
 Run func()
}
```

```

func main() {
 machine.Serial.Configure(machine.UARTConfig{BaudRate: 115200})
 led := machine.LED
 led.Configure(machine.PinConfig{Mode: machine.PinOutput})

 var blinkOn bool
 var ticks int

 tasks := []Task{
 {
 Name: "blink",
 Period: 500 * time.Millisecond,
 Run: func() {
 blinkOn = !blinkOn
 led.Set(blinkOn)
 },
 },
 {
 Name: "heartbeat",
 Period: 1 * time.Second,
 Run: func() {
 ticks++
 fmt.Fprintf(machine.Serial, "hb ticks=%d\n", ticks)
 },
 },
 {
 Name: "sensor",
 Period: 100 * time.Millisecond,
 Run: func() {
 // pretend sample
 _ = time.Now().UnixNano() % 100
 },
 },
 },
}

now := time.Now()
for i := range tasks {
 tasks[i].Next = now.Add(tasks[i].Period)
}

for {
 now = time.Now()
 for i := range tasks {
 t := &tasks[i]
 if !now.Before(t.Next) {
 t.Run()
 }
 }
}

```

```

 t.Next = t.Next.Add(t.Period)
 // if still behind, skip ahead
 if now.After(t.Next) {
 t.Next = now.Add(t.Period)
 }
 }
}
time.Sleep(1 * time.Millisecond) // cooperative idle
}
}

```

## Step 2: Inject a “bad” task

Add a task that `time.Sleep(200 * time.Millisecond)` occasionally. Observe heartbeat jitter.  
**Lesson:** cooperative scheduling fails open when a task hogs the CPU.

## Step 3: Metrics

Track:

- runs per task
- late count when `now.Sub(scheduled) > threshold`
- Print every 5 s over UART

## Step 4: Design write-up

In `NOTES.md`:

1. When cooperative is enough (UI blink, slow sensors).
2. When you need interrupts/RTOS (motor control, strict deadlines).
3. How TinyGo’s model maps to your board.

## Learning Goals

- Multi-rate task tables
- Soft deadline tracking

- Starvation under cooperative scheduling
- Honest boundary vs FreeRTOS-class systems

## Extensions

1. Priority: run shortest period first each cycle.
2. Disable a task via serial command.
3. Port one task to pin interrupt (from Project 42).

## Checkpoint

- ☐ 3 tasks with different periods
- ☐ UART shows heartbeat
- ☐ Bad-task experiment documented
- ☐ NOTES.md with RT conclusions

## **045 Project 45: TinyGo I2C Realtime Monitor**

# 045 TinyGo: I2C Device + Realtime Status Monitor

Wire an **I2C sensor or peripheral** (or a mock if no hardware), read it on a fixed schedule, and publish a compact status line over UART—combining bus IO, timing, and error handling for embedded Go.

## Hardware options

Path	Hardware	Notes
A	BME280 / SHT30 / MPU6050 etc.	Real I2C; check address + pull-ups
B	I2C EEPROM / expander	Simpler register IO
C	<b>Mock</b>	Software sensor if no bus

Many boards: `machine.I2C0` with SDA/SCL pins—**read your pinout**.

## Step 1: Bus init sketch

```
package main

import (
 "fmt"
 "machine"
 "time"
)

func main() {
 machine.Serial.Configure(machine.UARTConfig{BaudRate: 115200})

 i2c := machine.I2C0
 err := i2c.Configure(machine.I2CConfig{
 Frequency: 100_000, // 100 kHz start; 400 kHz if stable
 })
 if err != nil {
 fmt.Fprintf(machine.Serial, "i2c configure: %v\n", err)
 }
}
```

```

 // continue with mock if desired
 }

 const addr uint16 = 0x76 // example - change per device

 ticker := time.NewTicker(200 * time.Millisecond)
 defer ticker.Stop() // if supported; else ignore

 for range ticker.C {
 // Real read example (device-specific registers!):
 // buf := make([]byte, 2)
 // err := i2c.ReadRegister(uint8(addr), 0xD0, buf)
 // Mock:
 v := int(time.Now().UnixNano()%500) / 5
 ok := true
 fmt.Fprintf(machine.Serial, "ok=%v val=%d t=%d\n", ok, v, time.Now().Unix())
 _ = addr
 _ = i2c
 }
}

```

Replace mock with **one** real register read for your chip using the datasheet.

## Step 2: Error budget (soft RT)

```

type Monitor struct {
 Period time.Duration
 Failures int
 LastOK time.Time
}

func (m *Monitor) Sample(read func() (int, error)) {
 v, err := read()
 if err != nil {
 m.Failures++
 return
 }
 m.LastOK = time.Now()
 // publish v
 _ = v
}

```

Rules:

- Cap consecutive failures before **SAFE** mode (LED pattern / UART alert).



- Do not block forever on bus errors—timeout if API allows.
- Keep allocations out of the 200 ms path if RAM is tight (`buf` reused).

### Step 3: Status LED semantics

LED	Meaning
Slow blink	Healthy samples
Fast blink	Bus errors
Solid	Fatal / halt (optional)

### Step 4: Host-side plot (optional)

Log UART to file and plot with any tool:

```
picocom -b 115200 /dev/ttyACM0 | tee samples.log
```

## Learning Goals

- I2C configure + address/register mindset
- Timed sampling with failure counts
- Operator-visible status (UART + LED)
- Datasheet-driven integration

## Extensions

1. Two sensors on one bus.
2. Change sample rate over UART.
3. Simple threshold alarm relay pin.

## Safety

- 3.3 V vs 5 V logic levels—**do not** overvolt pins.
- Shared GND; pull-ups usually 4.7k on SDA/SCL.

## Checkpoint

- ☐ Bus init or explicit mock path
- ☐ Fixed-rate samples with failure counter
- ☐ UART status lines
- ☐ Datasheet note or mock justification

## 000 Projects Index

# 000 Projects: Build Real Software

This section is hands-on. Most chapters are **build projects** (runnable code + steps). A few are **checklist** / **drill** projects (harden or verify an existing service against a list)—those are labeled in the chapter title (e.g. *Secure Service Checklist*).

## Project tiers

Tier	What you get
<b>Build</b>	Module layout, full or nearly full source, run instructions, extensions
<b>Checklist</b>	Requirements and evidence to collect; you supply or reuse a small service
<b>Index / ideas</b>	Catalogues and workout lists (not a single binary)

## How To Use These Projects

1. Read the architecture diagram.
2. Type the code yourself first.
3. Run it.
4. Break it intentionally.
5. Add one extension before moving to the next project.

Learning loop

```
read -> build -> run -> debug -> extend -> repeat
```

## Project Path (Beginner -> Advanced)

1. [CLI Ping Tool](#)
2. [URL Shortener API](#)
3. [Concurrent Log Analyzer](#)
4. [File Integrity Checker](#)
5. [Concurrent Port Scanner](#)
6. [Mini Job Runner](#)
7. [Linux ls Clone](#)
8. [Linux cat Clone](#)

9. [Linux grep Clone](#)
10. [Linux tail -f Clone](#)
11. [Linux du Clone](#)
12. [Linux find Clone](#)
13. [Linux wc Clone](#)
14. [HTTP Load Tester](#)
15. [TUI System Monitor](#)
16. [Proxmox TUI Manager](#)
17. [SSH Remote Orchestrator](#)
18. [Go Workout: 200 Ten-Minute Exercises](#)
19. [Linux ps-lite](#)
20. [KV HTTP Store](#)
21. [WebSocket Chat](#)
22. [Log Shipper CLI](#)
23. [Proxmox Batch Operations CLI](#)
24. [Controller Pattern Simulator](#)
25. [Resource Index + 150 More Ideas](#)
26. [Kubernetes Pod Lister](#)
27. [Kubernetes Event Watcher](#)
28. [Kubernetes Rollout Checker](#)
29. [Terraform Plan Parser](#)
30. [Terraform Risk Reporter](#)
31. [Prometheus Exporter](#)
32. [Prometheus Probe Service](#)
33. [eBPF Tracepoint Basics](#)
34. [eBPF XDP Packet Counter](#)
35. [Proxmox Multi-Node Scheduler](#)
36. [Proxmox Capacity Balancer](#)
37. [Kubernetes HPA Simulator](#)
38. [Terraform Cost Estimator](#)
39. [Prometheus Alert Rule Tester](#)
40. [Infra Reconciler Daemon](#)

## **TinyGo & soft realtime (embedded)**

41. [TinyGo Blink & Toolchain](#)
42. [TinyGo Button + Interrupt](#)
43. [TinyGo UART Sensor Loop](#)
44. [TinyGo Cooperative Tasks \(Soft RT\)](#)
45. [TinyGo I2C Realtime Monitor](#)

TinyGo track

toolchain/blink -> button/IRQ -> timed sensor UART -> cooperative tasks -> I2C monitor

## What You Will Practice

- CLI flags and argument parsing
- Networking and HTTP servers
- Goroutines, channels, and worker pools
- File I/O and hashing
- Error handling and retries
- Production-friendly project structure
- **TinyGo** targets, **machine** pins, soft-realtime loops, interrupts, UART/I2C

## Step-by-Step Explanation

1. Pick a small set of exercises by track.
2. Timebox each attempt to ten minutes.
3. Record one takeaway and one weakness after each exercise.
4. Revisit chapter references when blocked.
5. Re-solve selected problems from memory weekly.

## Learning Goals

- Build consistency, not one-time intensity.
- Improve retrieval and transfer of Go patterns.
- Progress from syntax fluency to engineering fluency.

## 001 Project 1: CLI Ping Tool

# 001 Build a CLI Ping Tool (TCP Ping)

We will build a portable ping tool using the standard library. It measures round-trip time by opening a TCP connection.

## Why TCP Ping?

Classic ICMP ping needs raw socket permissions. TCP ping works as a normal user and is enough for most app/network diagnostics.

```
CLI flow
```

```
input host -> resolve DNS -> connect N times -> record RTT -> print stats
```

## Step 1: Create the Module

```
mkdir goping && cd goping
go mod init example.com/goping
```

## Step 2: Full main.go

```
package main

import (
 "flag"
 "fmt"
 "math"
 "net"
 "os"
 "time"
)

type result struct {
 ok bool
 rtt time.Duration
 err error
```



```

}

func tcpPing(host, port string, timeout time.Duration) result {
 start := time.Now()
 conn, err := net.DialTimeout("tcp", net.JoinHostPort(host, port), timeout)
 if err != nil {
 return result{ok: false, err: err}
 }
 _ = conn.Close()
 return result{ok: true, rtt: time.Since(start)}
}

func main() {
 count := flag.Int("c", 4, "number of probes")
 interval := flag.Duration("i", 1*time.Second, "interval between probes")
 timeout := flag.Duration("t", 2*time.Second, "connection timeout")
 port := flag.String("p", "443", "tcp port to probe")
 flag.Parse()

 if flag.NArg() != 1 {
 fmt.Println("usage: goping [flags] <host>")
 flag.PrintDefaults()
 os.Exit(2)
 }

 host := flag.Arg(0)
 ips, err := net.LookupIP(host)
 if err != nil || len(ips) == 0 {
 fmt.Fprintf(os.Stderr, "resolve failed: %v\n", err)
 os.Exit(1)
 }

 fmt.Printf("PING %s (%s) over TCP:%s\n", host, ips[0].String(), *port)

 var sent, recv int
 var minRTT time.Duration = time.Hour
 var maxRTT time.Duration
 var sumRTT time.Duration

 for i := 1; i <= *count; i++ {
 sent++
 r := tcpPing(host, *port, *timeout)
 if r.ok {
 recv++
 if r.rtt < minRTT {
 minRTT = r.rtt
 }
 }
 }
}

```

```

 if r.rtt > maxRTT {
 maxRTT = r.rtt
 }
 sumRTT += r.rtt
 fmt.Printf("%d: connected time=%v\n", i, r.rtt)
 } else {
 fmt.Printf("%d: timeout/error: %v\n", i, r.err)
 }

 if i < *count {
 time.Sleep(*interval)
 }
}

loss := float64(sent-recv) / float64(sent) * 100
avg := time.Duration(0)
if recv > 0 {
 avg = sumRTT / time.Duration(recv)
} else {
 minRTT = 0
}

stdDev := time.Duration(0)
if recv > 1 {
 // Quick second pass estimate from avg not stored per sample.
 // Keep simple for CLI output consistency.
 _ = math.Sqrt(1)
}

fmt.Println("--- stats ---")
fmt.Printf("%d probes sent, %d received, %.1f%% loss\n", sent, recv, loss)
fmt.Printf("rtt min/avg/max = %v/%v/%v\n", minRTT, avg, maxRTT)
_ = stdDev
}

```

## Step 3: Run

```

go run . google.com
go run . -c 10 -i 500ms -t 1s -p 443 cloudflare.com

```

## What to Extend

1. Add per-IP probing when DNS returns multiple addresses.

2. Add JSON output (`-json`) for scripting.
3. Track jitter by storing all RTT samples.

## Step-by-Step Explanation

1. Parse probe flags and validate host input.
2. Resolve DNS and print target IP for transparency.
3. Dial TCP with timeout for each probe.
4. Measure latency and aggregate min/avg/max.
5. Print packet loss and summary stats.

## Code Anatomy

- `tcpPing` performs one probe and returns a typed result.
- Main loop controls pacing and collects metrics.
- Final summary computes packet loss and latency profile.

## Learning Goals

- Network diagnostics with standard library only.
- Timeouts and resilient error reporting.
- Building practical CLI observability tools.

## **002 Project 2: URL Shortener API**

## 002 Build a URL Shortener API

You will build a small HTTP service with two endpoints:

- POST /shorten -> create short URL
- GET /:code -> redirect to original URL

Request flow

```
client -> POST /shorten -> store (code -> url) -> return short URL
client -> GET /abc123 -> lookup code -> 302 redirect
```

### Full main.go

```
package main

import (
 "crypto/rand"
 "encoding/base64"
 "encoding/json"
 "fmt"
 "log"
 "net/http"
 "net/url"
 "strings"
 "sync"
)

type store struct {
 mu sync.RWMutex
 data map[string]string
}

type shortenRequest struct {
 URL string `json:"url"`
}

type shortenResponse struct {
 Code string `json:"code"`
}
```

```

 URL string `json:"short_url"`
}

func newCode(n int) string {
 b := make([]byte, n)
 if _, err := rand.Read(b); err != nil {
 panic(err)
 }
 return strings.TrimRight(base64.RawURLEncoding.EncodeToString(b), "=")
}

func main() {
 s := &store{data: make(map[string]string)}
 mux := http.NewServeMux()

 mux.HandleFunc("POST /shorten", func(w http.ResponseWriter, r *http.Request) {
 var req shortenRequest
 if err := json.NewDecoder(r.Body).Decode(&req); err != nil {
 http.Error(w, "invalid json", http.StatusBadRequest)
 return
 }

 u, err := url.ParseRequestURI(req.URL)
 if err != nil || u.Scheme == "" || u.Host == "" {
 http.Error(w, "invalid url", http.StatusBadRequest)
 return
 }

 code := newCode(5)
 s.mu.Lock()
 s.data[code] = req.URL
 s.mu.Unlock()

 resp := shortenResponse{
 Code: code,
 URL: fmt.Sprintf("http://%s/%s", r.Host, code),
 }
 w.Header().Set("Content-Type", "application/json")
 _ = json.NewEncoder(w).Encode(resp)
 })

 mux.HandleFunc("GET /{code}", func(w http.ResponseWriter, r *http.Request) {
 code := r.PathValue("code")
 s.mu.RLock()
 longURL, ok := s.data[code]
 s.mu.RUnlock()
 if !ok {

```

```

 http.NotFound(w, r)
 return
 }
 http.Redirect(w, r, longURL, http.StatusFound)
})

log.Println("listening on :8080")
log.Fatal(http.ListenAndServe(":8080", mux))
}

```

## Run and Test

```

go run .

curl -s -X POST localhost:8080/shorten \
 -H 'content-type: application/json' \
 -d '{"url":"https://go.dev/doc/"}'

curl -i localhost:8080/<code-from-response>

```

## What to Extend

1. Persistent storage (BoltDB/Postgres).
2. Expiry timestamps.
3. Rate limiting middleware.

## Step-by-Step Explanation

1. Define request and response contracts first.
2. Validate inbound input before doing any state changes.
3. Keep handler logic short and move reusable logic into helper functions.
4. Add timeouts and clear error paths.
5. Return consistent responses and status codes.

## Code Anatomy

- Handlers parse input, call domain logic, write response.
- Shared state uses synchronization where needed.
- Transport concerns stay separate from business rules.

## Learning Goals

- Build reliable service endpoints in Go.
- Understand API ergonomics and operational safety.
- Prepare code structure for tests and persistence later.



## **003 Project 3: Concurrent Log Analyzer**

## 003 Build a Concurrent Log Analyzer

Read a log file and count log levels using a **worker pool**. Practice fan-out (lines to workers), local aggregation, and fan-in merge without shared mutable maps.

```
file -> producer -> jobs channel -> N workers -> partial counts -> merge -> report
```

### Problem statement

```
log-analyzer [-w N] <file>
```

- Stream lines from a file
- Workers count INFO/WARN/ERROR/DEBUG substrings (case-sensitive tokens)
- Merge partial maps in main
- Print a summary table

### Acceptance criteria

- ☐ Worker count configurable (-w, default NumCPU)
- ☐ No shared map writes from workers (each has a local map)
- ☐ Correct totals on a fixture file
- ☐ Terminates cleanly (channels closed, WaitGroup)
- ☐ `go run -race clean`

### Setup

```
mkdir logan && cd logan
go mod init example.com/logan
go 1.27
```

## Full main.go

```
package main

import (
 "bufio"
 "flag"
 "fmt"
 "os"
 "runtime"
 "strings"
 "sync"
)

var levels = []string{"INFO", "WARN", "ERROR", "DEBUG"}

func worker(lines <-chan string, out chan<- map[string]int) {
 local := map[string]int{}
 for _, lvl := range levels {
 local[lvl] = 0
 }
 for line := range lines {
 for _, level := range levels {
 if strings.Contains(line, level) {
 local[level]++
 }
 }
 }
 out <- local
}

func analyze(path string, workers int) (map[string]int, error) {
 f, err := os.Open(path)
 if err != nil {
 return nil, err
 }
 defer f.Close()

 if workers < 1 {
 workers = 1
 }

 jobs := make(chan string, 1024)
 partials := make(chan map[string]int, workers)
 var wg sync.WaitGroup
```

```

for i := 0; i < workers; i++ {
 wg.Go(func() {
 worker(jobs, partials)
 })
}

go func() {
 scanner := bufio.NewScanner(f)
 buf := make([]byte, 0, 64*1024)
 scanner.Buffer(buf, 1024*1024)
 for scanner.Scan() {
 jobs <- scanner.Text()
 }
 close(jobs)
 wg.Wait()
 close(partial)
 // surface scan error via second channel in production; simplified here
 _ = scanner.Err()
}()

total := map[string]int{}
for _, lvl := range levels {
 total[lvl] = 0
}
for p := range partials {
 for k, v := range p {
 total[k] += v
 }
}
return total, nil
}

func main() {
 workers := flag.Int("w", runtime.NumCPU(), "worker count")
 flag.Parse()
 if flag.NArg() != 1 {
 fmt.Fprintln(os.Stderr, "usage: log-analyzer [-w N] <file>")
 os.Exit(2)
 }

 total, err := analyze(flag.Arg(0), *workers)
 if err != nil {
 fmt.Fprintf(os.Stderr, "open failed: %v\n", err)
 os.Exit(1)
 }
}

```

```

 fmt.Println("Log Summary")
 for _, lvl := range levels {
 fmt.Printf("%-6s %d\n", lvl+":", total[lvl])
 }
}

```

## Step-by-step build path

1. Sequential single-goroutine counter first (correctness baseline).
2. Introduce jobs channel + N workers with **local** maps.
3. Merge partials in one owner goroutine/main.
4. Add **-race** and large fixture stress.
5. Extend matchers (regex, JSON field).

## Run and verification

```

printf 'INFO start\nERROR boom\nWARN x\nINFO ok\nDEBUG d\nERROR e2\n' > app.log
go run . -w 4 app.log
go run -race . -w 8 app.log

```

## Tests

```

package main

import (
 "os"
 "path/filepath"
 "testing"
)

func TestAnalyze(t *testing.T) {
 dir := t.TempDir()
 path := filepath.Join(dir, "a.log")
 content := "INFO a\nINFO b\nERROR x\n"
 if err := os.WriteFile(path, []byte(content), 0o644); err != nil {
 t.Fatal(err)
 }
 total, err := analyze(path, 2)
 if err != nil {
 t.Fatal(err)
 }
}

```

```

 if total["INFO"] != 2 || total["ERROR"] != 1 {
 t.Fatalf("%v", total)
 }
}

go test -race ./...

```

## Stretch goals

1. JSON output (`-json`).
2. Per-service breakdown if lines contain `service=`.
3. Top-N error messages (heap).
4. Multi-file args.

## Pitfalls

Pitfall	Fix
Shared map++ from workers	local maps + merge
Forgetting close(partials)	close after Wait
Scanner default 64K	Buffer increase
Substring false positives	word boundaries / regex

## Learning goals

- Leak-free worker pools
- Fan-out / fan-in architecture
- Race-free aggregation patterns

## **004 Project 4: File Integrity Checker**

## 004 Build a File Integrity Checker

This CLI calculates SHA-256 checksums and verifies files against a manifest.

```
manifest mode: file -> hash -> write "hash path"
verify mode: read manifest -> re-hash files -> compare -> report
```

### Full main.go

```
package main

import (
 "bufio"
 "crypto/sha256"
 "encoding/hex"
 "flag"
 "fmt"
 "io"
 "os"
 "path/filepath"
 "strings"
)

func hashFile(path string) (string, error) {
 f, err := os.Open(path)
 if err != nil {
 return "", err
 }
 defer f.Close()

 h := sha256.New()
 if _, err := io.Copy(h, f); err != nil {
 return "", err
 }
 return hex.EncodeToString(h.Sum(nil)), nil
}

func createManifest(root, out string) error {
 mf, err := os.Create(out)
```



```

 if err != nil {
 return err
 }
 defer mf.Close()

 return filepath.WalkDir(root, func(path string, d os.DirEntry, err error) error {
 if err != nil {
 return err
 }
 if d.IsDir() {
 return nil
 }
 h, err := hashFile(path)
 if err != nil {
 return err
 }
 _, err = fmt.Fprintf(mf, "%s %s\n", h, path)
 return err
 })
}

func verifyManifest(path string) error {
 f, err := os.Open(path)
 if err != nil {
 return err
 }
 defer f.Close()

 s := bufio.NewScanner(f)
 failed := 0
 for s.Scan() {
 line := s.Text()
 parts := strings.SplitN(line, " ", 2)
 if len(parts) != 2 {
 return fmt.Errorf("bad line: %q", line)
 }
 expected, filePath := parts[0], parts[1]
 got, err := hashFile(filePath)
 if err != nil || got != expected {
 failed++
 fmt.Printf("FAIL %s\n", filePath)
 } else {
 fmt.Printf("OK %s\n", filePath)
 }
 }
 if err := s.Err(); err != nil {
 return err
 }
}

```

```

 }
 if failed > 0 {
 return fmt.Errorf("%d file(s) failed verification", failed)
 }
 return nil
}

func main() {
 mode := flag.String("mode", "manifest", "manifest|verify")
 root := flag.String("root", ".", "root directory for manifest mode")
 manifest := flag.String("manifest", "checksums.txt", "manifest path")
 flag.Parse()

 var err error
 switch *mode {
 case "manifest":
 err = createManifest(*root, *manifest)
 case "verify":
 err = verifyManifest(*manifest)
 default:
 err = fmt.Errorf("unknown mode: %s", *mode)
 }

 if err != nil {
 fmt.Fprintln(os.Stderr, "error:", err)
 os.Exit(1)
 }
}

```

## Run

```

go run . -mode manifest -root ./data -manifest checksums.txt
go run . -mode verify -manifest checksums.txt

```

## What to Extend

1. Support ignore patterns (`.git`, `node_modules`).
2. Output machine-readable JSON reports.
3. Add optional HMAC signing for manifest authenticity.

## Step-by-Step Explanation

1. Parse command-line flags and validate inputs early.
2. Keep the core operation in a small, testable function.
3. Process data as a stream when possible to reduce memory use.
4. Print stable output and meaningful exit codes.
5. Add one extension feature and test edge cases.

## Code Anatomy

- `main` handles flags, orchestration, and errors.
- Worker/helper functions hold business logic.
- Output section should be deterministic for scripting and CI usage.

## Learning Goals

- Write composable Unix-style Go tools.
- Improve error messages and operator experience.
- Practice iterative improvement over one clear baseline.

## **005 Project 5: Concurrent Port Scanner**

## 005 Build a Concurrent Port Scanner

Build a CLI that scans a host across a TCP port range using a **bounded worker pool**, collects open ports, and prints them sorted. This is a classic fan-out/fan-in exercise: many dials in flight, controlled concurrency, clean shutdown.

```
ports range -> jobs channel -> workers dial tcp -> open ports channel -> sort -> print
```

### Problem statement

Given `-host`, `-from`, `-to`, `-w` (workers), and `-t` (dial timeout), probe every port in range with `net.DialTimeout("tcp", ...)`. Print open ports in ascending order. Exit non-zero on invalid flags.

### Acceptance criteria

- ☐ Invalid range (`from > to`, out of 1–65535) exits with code 2 and a clear message
- ☐ Worker count bounds concurrency (not one goroutine per port without limit)
- ☐ Open ports printed sorted; closed/filtered ports silent
- ☐ Program terminates (no goroutine leak after range completes)
- ☐ `go build` succeeds with `stdlib` only
- ☐ Optional: `-json` or banner grab as stretch (not required for pass)

### Setup

```
mkdir portscan && cd portscan
go mod init example.com/portscan
go.mod
module example.com/portscan
go 1.27
```

## Full main.go

```
package main

import (
 "flag"
 "fmt"
 "net"
 "os"
 "sort"
 "sync"
 "time"
)

func scan(host string, timeout time.Duration, jobs <-chan int, openPorts chan<- int) {
 for p := range jobs {
 addr := net.JoinHostPort(host, fmt.Sprintf("%d", p))
 conn, err := net.DialTimeout("tcp", addr, timeout)
 if err != nil {
 continue
 }
 _ = conn.Close()
 openPorts <- p
 }
}

func main() {
 host := flag.String("host", "127.0.0.1", "target host")
 from := flag.Int("from", 1, "start port")
 to := flag.Int("to", 1024, "end port")
 workers := flag.Int("w", 200, "worker count")
 timeout := flag.Duration("t", 300*time.Millisecond, "dial timeout")
 flag.Parse()

 if *from < 1 || *to > 65535 || *from > *to {
 fmt.Fprintln(os.Stderr, "invalid port range: need 1 <= from <= to <= 65535")
 os.Exit(2)
 }

 if *workers < 1 {
 fmt.Fprintln(os.Stderr, "workers must be >= 1")
 os.Exit(2)
 }

 jobs := make(chan int, *workers)
 openPorts := make(chan int, 256)
 var wg sync.WaitGroup
```

```

for i := 0; i < *workers; i++ {
 wg.Go(func() {
 scan(*host, *timeout, jobs, openPorts)
 })
}

go func() {
 for p := *from; p <= *to; p++ {
 jobs <- p
 }
 close(jobs)
 wg.Wait()
 close(openPorts)
}()

var open []int
for p := range openPorts {
 open = append(open, p)
}
sort.Ints(open)

fmt.Printf("open ports on %s (%d-%d):\n", *host, *from, *to)
for _, p := range open {
 fmt.Println(p)
}
fmt.Printf("total open: %d\n", len(open))
}

```

## Step-by-step build path

1. **Validate flags** before starting any goroutine.
2. **Create job and result channels**; buffer jobs roughly to worker count to reduce scheduling chatter.
3. **Start N workers** that only dial and optionally send open ports—no shared mutable state except channels.
4. **Producer goroutine** pushes ports, closes jobs, waits for workers, then closes results (fan-in join).
5. **Main aggregates**, sorts, prints. Prefer `sync.WaitGroup.Go` (Go 1.25+) for worker startup.

```

sequence (top → bottom):
 actors: main, producer, workers, openCh
 main --> workers : start N
 main --> producer : go fill jobs
 producer --> workers : jobs

```

```
workers --> openCh : open ports
producer --> openCh : close after Wait
main --> openCh : range + sort
```

## Run and verification

```
Local: open something you control, e.g. a local HTTP server on :8080
go run . -host 127.0.0.1 -from 1 -to 9000 -w 300 -t 150ms

Public lab target (be polite; small range)
go run . -host scanme.nmap.org -from 20 -to 100 -w 50 -t 500ms

go build -o portscan .
./portscan -host 127.0.0.1 -from 1 -to 100
```

Quick self-check with a known listener:

```
terminal A
python3 -m http.server 8765
terminal B
go run . -host 127.0.0.1 -from 8760 -to 8770 -w 20 -t 200ms
expect 8765 in the list
```

Race-friendly structure (channels only for shared coordination):

```
go run -race . -host 127.0.0.1 -from 1 -to 200 -w 50 -t 50ms
```

## Tests (optional unit)

Extract dial behind an interface for pure tests, or keep a smoke test:

```
// scan_test.go - integration-style, skip in short mode
package main

import (
 "net"
 "testing"
 "time"
)

func TestDialLocalListener(t *testing.T) {
 ln, err := net.Listen("tcp", "127.0.0.1:0")
 if err != nil {
```



```

 t.Fatal(err)
 }
 defer ln.Close()
 _, port, _ := net.SplitHostPort(ln.Addr().String())

 conn, err := net.DialTimeout("tcp", net.JoinHostPort("127.0.0.1", port), time.Second)
 if err != nil {
 t.Fatal(err)
 }
 _ = conn.Close()
}

go test -race ./...

```

## Stretch goals

1. **CIDR / multi-host**: expand 10.0.0.0/30 into hosts; nest host×port jobs.
2. **Banner grab**: after dial, set read deadline and print first line.
3. **Rate limit**: token bucket so `-w` high does not flood a single target.
4. **UDP probe** (best-effort) or service name map for common ports.
5. **JSON output** for CI: `{"host": "...", "open": [22,80]}`.

## Pitfalls

Pitfall	Why it hurts	Fix
Unbounded goroutines	file descriptor / memory blowup	worker pool
No timeout	hangs on filtered ports	<code>DialTimeout</code>
Shared slice append without sync	data race	send on channel, append in one owner
Forgetting to close <code>openPorts</code>	main blocks forever	close after <code>Wait</code>
Scanning without permission	legal/ToS issues	only scan hosts you own or have OK for

## Code anatomy

- **Producer** pushes port numbers into jobs.
- **Workers** consume jobs and emit open ports.
- **Aggregator** (main) merges results and prints a deterministic summary.

## Learning goals

- Build leak-free fan-out/fan-in with WaitGroup + channel close.
- Balance throughput ( $-w$ ) against timeout and target load.
- Keep coordination in channels; avoid shared mutable maps without locks.

## **006 Project 6: Mini Job Runner**

## 006 Build a Mini Job Runner with Retries

This project builds a small in-process job runner with retries and backoff.

```
enqueue jobs -> workers pick jobs -> run handler -> retry on failure -> final status
```

### Full main.go

```
package main

import (
 "context"
 "errors"
 "fmt"
 "math/rand"
 "sync"
 "time"
)

type Job struct {
 ID int
 Payload string
 Attempts int
}

type Runner struct {
 workers int
 maxTry int
 jobs chan Job
}

func NewRunner(workers, maxTry, queueSize int) *Runner {
 return &Runner{
 workers: workers,
 maxTry: maxTry,
 jobs: make(chan Job, queueSize),
 }
}
```

```

func (r *Runner) Enqueue(j Job) {
 r.jobs <- j
}

func process(ctx context.Context, j Job) error {
 select {
 case <-ctx.Done():
 return ctx.Err()
 case <-time.After(100 * time.Millisecond):
 // simulate intermittent failure
 if rand.Intn(3) == 0 {
 return errors.New("temporary failure")
 }
 fmt.Printf("job %d ok payload=%q\n", j.ID, j.Payload)
 return nil
 }
}

func (r *Runner) Start(ctx context.Context) {
 var wg sync.WaitGroup
 for w := 1; w <= r.workers; w++ {
 wg.Add(1)
 go func(workerID int) {
 defer wg.Done()
 for {
 select {
 case <-ctx.Done():
 return
 case j, ok := <-r.jobs:
 if !ok {
 return
 }

 err := process(ctx, j)
 if err == nil {
 continue
 }

 j.Attempts++
 if j.Attempts < r.maxTry {
 backoff := time.Duration(j.Attempts*200) * time.Millisecond
 time.AfterFunc(backoff, func() { r.Enqueue(j) })
 fmt.Printf("worker=%d retry job=%d attempt=%d err=%v\n", workerID, j.ID, j.Attempts, err)
 } else {
 fmt.Printf("worker=%d dead-letter job=%d err=%v\n", workerID, j.ID, j.Attempts, err)
 }
 }
 }
 }(workerID)
 }
}

```

```

 }
 }(w)
}

wg.Wait()
}

func (r *Runner) Stop() {
 close(r.jobs)
}

func main() {
 rand.Seed(time.Now().UnixNano())
 ctx, cancel := context.WithTimeout(context.Background(), 5*time.Second)
 defer cancel()

 r := NewRunner(4, 4, 64)
 for i := 1; i <= 20; i++ {
 r.Enqueue(Job{ID: i, Payload: fmt.Sprintf("task-%d", i)})
 }

 go func() {
 <-ctx.Done()
 r.Stop()
 }()

 r.Start(ctx)
}

```

## Run

```
go run .
```

## What to Extend

1. Persistent queue storage.
2. Dead-letter queue file.
3. Metrics endpoint (`/metrics`) with processing stats.

## Step-by-Step Explanation

1. Model jobs, workers, and outputs explicitly.

2. Bound concurrency using worker pools and buffered channels.
3. Use `sync.WaitGroup` for lifecycle control.
4. Aggregate worker results in one place.
5. Verify behavior under both normal and failure paths.

## Code Anatomy

- Producer pushes jobs into a channel.
- Workers consume jobs and emit results.
- Aggregator merges results and prints summary.

## Learning Goals

- Build leak-free goroutine patterns.
- Balance throughput and resource limits.
- Understand fan-out/fan-in architecture.

## **007 Project 7: Build an Is Clone**



## 007 Build an ls Clone

Build a practical `ls`-style CLI with flags for hidden files, long format, and sorting. Focus on clean formatting and deterministic ordering for scripts.

```
path -> read entries -> filter -> sort -> format -> print
```

### Problem statement

```
gls [-a] [-l] [-sort name|size|time] [path]
```

- Default path .
- Hide dotfiles unless `-a`
- `-l` shows mode, size, modtime, name
- Directories get a trailing / in long mode

### Acceptance criteria

- ☐ Lists directory entries
- ☐ `-a` shows hidden
- ☐ `-l` long format columns
- ☐ Sort by name/size/time
- ☐ Non-zero exit on unreadable path
- ☐ Stdlib only

### Setup

```
mkdir gls && cd gls
go mod init example.com/gls
go 1.27
```

## Full main.go

```
package main

import (
 "flag"
 "fmt"
 "io/fs"
 "os"
 "path/filepath"
 "sort"
 "strings"
 "time"
)

type item struct {
 Name string
 Info fs.FileInfo
}

func list(path string, showAll bool, sortBy string) ([]item, error) {
 entries, err := os.ReadDir(path)
 if err != nil {
 return nil, err
 }
 var items []item
 for _, e := range entries {
 name := e.Name()
 if !showAll && strings.HasPrefix(name, ".") {
 continue
 }
 info, err := e.Info()
 if err != nil {
 continue
 }
 items = append(items, item{Name: name, Info: info})
 }
 sort.SliceStable(items, func(i, j int) bool {
 switch sortBy {
 case "size":
 if items[i].Info.Size() == items[j].Info.Size() {
 return strings.ToLower(items[i].Name) < strings.ToLower(items[j].Name)
 }
 return items[i].Info.Size() < items[j].Info.Size()
 case "time":
 ti, tj := items[i].Info.ModTime(), items[j].Info.ModTime()
```

```

 if ti.Equal(tj) {
 return strings.ToLower(items[i].Name) < strings.ToLower(items[j].Name)
 }
 return ti.Before(tj)
 default: // name
 return strings.ToLower(items[i].Name) < strings.ToLower(items[j].Name)
 }
})
return items, nil
}

func main() {
 showAll := flag.Bool("a", false, "show hidden files")
 longFmt := flag.Bool("l", false, "long listing format")
 sortBy := flag.String("sort", "name", "name|size|time")
 flag.Parse()

 path := "."
 if flag.NArg() > 0 {
 path = flag.Arg(0)
 }
 switch *sortBy {
 case "name", "size", "time":
 default:
 fmt.Fprintln(os.Stderr, "invalid -sort; want name|size|time")
 os.Exit(2)
 }

 items, err := list(path, *showAll, *sortBy)
 if err != nil {
 fmt.Fprintf(os.Stderr, "read dir failed: %v\n", err)
 os.Exit(1)
 }

 for _, it := range items {
 if *longFmt {
 mode := it.Info.Mode().String()
 sz := it.Info.Size()
 mod := it.Info.ModTime().Format(time.DateTime)
 name := it.Name
 if it.Info.IsDir() {
 name += string(filepath.Separator)
 }
 fmt.Printf("%s %10d %s %s\n", mode, sz, mod, name)
 } else {
 fmt.Println(it.Name)
 }
 }
}

```

```
}
}
```

## Step-by-step build path

1. ReadDir + filter dots.
2. Collect FileInfo for size/time sorts.
3. Stable sort with tie-break on name.
4. Long vs short formatters.
5. Validate `-sort` early.

## Run and verification

```
go run .
go run . -a -l -sort size /var/log
go run . -sort time -l .
```

## Tests

```
package main

import (
 "os"
 "path/filepath"
 "testing"
)

func TestListHidesDot(t *testing.T) {
 dir := t.TempDir()
 _ = os.WriteFile(filepath.Join(dir, "a.txt"), []byte("x"), 0o644)
 _ = os.WriteFile(filepath.Join(dir, ".secret"), []byte("y"), 0o644)
 items, err := list(dir, false, "name")
 if err != nil {
 t.Fatal(err)
 }
 if len(items) != 1 || items[0].Name != "a.txt" {
 t.Fatalf("%v", items)
 }
 items, _ = list(dir, true, "name")
 if len(items) != 2 {
 t.Fatalf("%d", len(items))
 }
}
```

```
}
```

```
go test ./...
```

## Stretch goals

1. Colorize dirs when stdout is a TTY.
2. `-R` recursive.
3. Human sizes (`-h`) in long mode.
4. Multiple path arguments.

## Pitfalls

Pitfall	Fix
Locale-dependent sort	document case-fold policy
Following symlinks unexpectedly	<code>ReadDir</code> metadata; document
Unstable sort ties	<code>sort.SliceStable</code> + name tie-break

## Learning goals

- Directory listing APIs
- Formatting operator-friendly tables
- Flag-driven CLI design

## **008 Project 8: Build a cat Clone**

# 008 Build a cat Clone

Create a streaming file printer with line numbers and optional end-of-line markers—stdlib only, Unix-friendly exit codes, and multi-file support like classic `cat`.

```
stdin/files -> scanner -> transform lines -> stdout
```

## Problem statement

Implement `gcat` that:

- Reads **stdin** when no file args are given
- Concatenates multiple files in order
- Supports `-n` (number lines) and `-E` (show \$ at end of each line)
- Continues after a missing file (stderr message) like many Unix tools
- Streams line-by-line (does not load whole files)

## Acceptance criteria

- ☐ `go run . file.txt` prints file contents
- ☐ No args → read stdin until EOF
- ☐ `-n` numbers lines; numbering continues across files
- ☐ `-E` appends \$ before newline semantics (end of logical line)
- ☐ Missing file: error on stderr, non-zero if any failure (your policy: document it)
- ☐ Works on empty files without hanging

## Setup

```
mkdir gcat && cd gcat
go mod init example.com/gcat
go 1.27
```

## Full main.go

```
package main

import (
 "bufio"
 "flag"
 "fmt"
 "io"
 "os"
)

func cat(r io.Reader, number, showEnds bool, start int) (int, error) {
 s := bufio.NewScanner(r)
 // Increase token size for long lines (default 64K may be tight for logs).
 buf := make([]byte, 0, 64*1024)
 s.Buffer(buf, 1024*1024)

 lineNo := start
 for s.Scan() {
 line := s.Text()
 if showEnds {
 line += "$"
 }
 if number {
 fmt.Printf("%6d\t%s\n", lineNo, line)
 lineNo++
 } else {
 fmt.Println(line)
 }
 }
 return lineNo, s.Err()
}

func main() {
 number := flag.Bool("n", false, "number output lines")
 showEnds := flag.Bool("E", false, "show $ at end of line")
 flag.Parse()

 lineNo := 1
 var hadErr bool

 if flag.NArg() == 0 {
 if _, err := cat(os.Stdin, *number, *showEnds, lineNo); err != nil {
 fmt.Fprintln(os.Stderr, "stdin:", err)
 os.Exit(1)
 }
 }
}
```



```

 }
 return
}

for _, path := range flag.Args() {
 f, err := os.Open(path)
 if err != nil {
 fmt.Fprintf(os.Stderr, "%s: %v\n", path, err)
 hadErr = true
 continue
 }
 n, err := cat(f, *number, *showEnds, lineNo)
 _ = f.Close()
 if err != nil {
 fmt.Fprintf(os.Stderr, "%s: %v\n", path, err)
 hadErr = true
 continue
 }
 lineNo = n
}

if hadErr {
 os.Exit(1)
}
}

```

## Step-by-step build path

1. **Flags first** — `-n`, `-E`, then positional paths.
2. **Core** `cat(io.Reader, ...)` — pure stream helper; easy to test with `strings.NewReader`.
3. **Stdin path** when `flag.NArg() == 0`.
4. **Multi-file loop** with continuous line numbers when `-n`.
5. **Scanner buffer** for long log lines; surface `s.Err()`.

## Run and verification

```

printf 'a\nb\nc\n' > sample.txt
go run . sample.txt
go run . -n -E sample.txt

multi-file numbering
printf 'x\n' > a.txt; printf 'y\n' > b.txt
go run . -n a.txt b.txt

```

```
expect line numbers 1 then 2

stdin
echo hello | go run . -n
```

## Tests

```
package main

import (
 "strings"
 "testing"
)

func TestCatNumbers(t *testing.T) {
 // Capture via redirect in integration tests, or refactor cat to write to io.Writer.
 r := strings.NewReader("one\ntwo\n")
 // Smoke: no error on scan
 _, err := cat(r, false, false, 1)
 if err != nil {
 t.Fatal(err)
 }
}

func TestCatEmpty(t *testing.T) {
 _, err := cat(strings.NewReader(""), true, true, 1)
 if err != nil {
 t.Fatal(err)
 }
}
```

Refactor tip for stronger tests: change signature to `cat(w io.Writer, r io.Reader, ...)`.

```
go test ./...
```

## Stretch goals

1. `-b` number non-blank lines only.
2. `-A` show tabs as `^I` and ends as `$`.
3. Binary-safe mode using `io.Copy` when no transforms.
4. Squeeze blank lines (`-s`).
5. Read from `-` as stdin mixed with files.

## Pitfalls

Pitfall	Fix
Loading whole file with <code>ReadFile</code>	Use scanner / streaming
Default scanner max token 64K	<code>Scanner.Buffer</code>
Resetting line numbers per file unintentionally	Pass / return <code>lineNo</code>
Silent open errors	Always print to stderr
Treating binary as lines	Document limitation or use raw copy

## Learning goals

- Composable Unix-style CLIs with `io.Reader`
- Flag ergonomics and multi-file edge cases
- Streaming over buffering for large inputs

## **009 Project 9: Build a grep Clone**

# 009 Build a grep Clone

Search text with regex, case-insensitive mode, and line numbers. Stream each file line-by-line and print matches in a stable, script-friendly format.

```
files -> scan line by line -> regex match -> print matches
```

## Problem statement

Implement `ggrep`:

```
ggrep [flags] <pattern> <file...>
```

Flags:

- `-i` ignore case (prefix `(?i)` or compile with case-folding)
- `-n` show line numbers as `file:line:text`

Exit codes (grep-compatible style is nice-to-have):

- 0 if any match
- 1 if no match
- 2 on usage/regex/IO hard failure (choose and document)

## Acceptance criteria

- ☐ Matches printed for each matching line
- ☐ `-i` works for mixed-case patterns
- ☐ `-n` includes line numbers
- ☐ Invalid regex → clear error, exit 2
- ☐ Multiple files supported
- ☐ Streaming (no full-file regex required)

## Setup

```
mkdir ggrep && cd ggrep
go mod init example.com/ggrep
go 1.27
```

## Full main.go

```
package main

import (
 "bufio"
 "flag"
 "fmt"
 "os"
 "regexp"
)

func grepFile(path string, re *regexp.Regexp, showLineNo bool) (matched bool, err error) {
 f, err := os.Open(path)
 if err != nil {
 return false, err
 }
 defer f.Close()

 s := bufio.NewScanner(f)
 buf := make([]byte, 0, 64*1024)
 s.Buffer(buf, 1024*1024)

 lineNo := 0
 for s.Scan() {
 lineNo++
 line := s.Text()
 if !re.MatchString(line) {
 continue
 }
 matched = true
 if showLineNo {
 fmt.Printf("%s:%d:%s\n", path, lineNo, line)
 } else {
 fmt.Printf("%s:%s\n", path, line)
 }
 }
 return matched, s.Err()
}
```

```

}

func main() {
 ignoreCase := flag.Bool("i", false, "ignore case")
 showLineNo := flag.Bool("n", false, "show line number")
 flag.Parse()

 if flag.NArg() < 2 {
 fmt.Fprintln(os.Stderr, "usage: grep [flags] <pattern> <file...>")
 os.Exit(2)
 }

 pattern := flag.Arg(0)
 if *ignoreCase {
 pattern = "(?i)" + pattern
 }
 re, err := regexp.Compile(pattern)
 if err != nil {
 fmt.Fprintln(os.Stderr, "invalid regex:", err)
 os.Exit(2)
 }

 any := false
 hadErr := false
 for _, file := range flag.Args()[1:] {
 matched, err := grepFile(file, re, *showLineNo)
 if err != nil {
 fmt.Fprintf(os.Stderr, "%s: %v\n", file, err)
 hadErr = true
 continue
 }
 if matched {
 any = true
 }
 }

 if hadErr {
 os.Exit(2)
 }
 if !any {
 os.Exit(1)
 }
}

```

## Step-by-step build path

1. Parse flags; require pattern + 1 file.
2. Compile regex once (with optional (?i)).
3. Per file: open, scan, match, print.
4. Track **any** match and IO errors for exit codes.
5. Raise scanner buffer for long lines.

## Run and verification

```
printf 'Error: boom\ninfo ok\nERROR again\n' > app.log
go run . error app.log
go run . -i -n "timeout|error" app.log

no match → exit 1
go run . zzzzz app.log; echo exit:$?
```

## Tests

```
package main

import (
 "os"
 "path/filepath"
 "regexp"
 "testing"
)

func TestGrepFile(t *testing.T) {
 dir := t.TempDir()
 path := filepath.Join(dir, "t.log")
 if err := os.WriteFile(path, []byte("alpha\nBeta\nalpha2\n"), 0o644); err != nil {
 t.Fatal(err)
 }
 re := regexp.MustCompile(`(?i)alpha`)
 ok, err := grepFile(path, re, true)
 if err != nil || !ok {
 t.Fatalf("matched=%v err=%v", ok, err)
 }
}

go test -race ./...
```



## Stretch goals

1. Recursive directory walk (`-r`) using `filepath.WalkDir`.
2. Invert match (`-v`).
3. Count only (`-c`).
4. Colorize matches on TTY.
5. Read stdin when files omitted.

## Pitfalls

Pitfall	Fix
Compiling regex per line	Compile once
Shell globs not expanded by Go	Pass files or implement walk
Catastrophic backtracking	Prefer simpler patterns; timeouts hard in stdlib
Binary files spam	Detect NUL and skip (stretch)

## Learning goals

- `regexp` package and RE2 flavor limits
- Streaming text tools
- Grep-compatible exit codes for scripts

## **010 Project 10: Build a tail -f Clone**

# 010 Build a tail -f Clone

Read the last N lines of a file, then optionally follow appended content—core log-watching skill for operators.

```
open file -> ring buffer last N -> print -> poll growth -> print new data
```

## Problem statement

```
gtail [-n N] [-f] <file>
```

- Print last N lines (default 10)
- -f follow: after initial print, stream new bytes as the file grows
- Exit 2 on usage errors; 1 on IO errors

## Acceptance criteria

- ☐ Last N lines correct for files with N and <N lines
- ☐ Follow mode prints newly appended lines
- ☐ Does not busy-spin (sleep/poll)
- ☐ Missing file → clear error
- ☐ Stdlib only

## Setup

```
mkdir gtail && cd gtail
go mod init example.com/gtail
go 1.27
```

## Full main.go

```
package main

import (
 "bufio"
 "context"
 "flag"
 "fmt"
 "io"
 "os"
 "os/signal"
 "syscall"
 "time"
)

func lastNLines(path string, n int) ([]string, error) {
 if n < 0 {
 n = 0
 }
 f, err := os.Open(path)
 if err != nil {
 return nil, err
 }
 defer f.Close()

 buf := make([]string, 0, n)
 s := bufio.NewScanner(f)
 sbuf := make([]byte, 0, 64*1024)
 s.Buffer(sbuf, 1024*1024)
 for s.Scan() {
 if n == 0 {
 continue
 }
 if len(buf) == n {
 copy(buf, buf[1:])
 buf[n-1] = s.Text()
 } else {
 buf = append(buf, s.Text())
 }
 }
 return buf, s.Err()
}

func follow(ctx context.Context, path string, offset int64, poll time.Duration) error {
 f, err := os.Open(path)
```

```

 if err != nil {
 return err
 }
 defer f.Close()

 if _, err := f.Seek(offset, io.SeekStart); err != nil {
 return err
 }

 for {
 select {
 case <-ctx.Done():
 return nil
 default:
 }
 n, err := io.Copy(os.Stdout, f)
 if err != nil {
 return err
 }
 if n == 0 {
 // optional: detect truncation/rotation via Stat size < offset
 select {
 case <-ctx.Done():
 return nil
 case <-time.After(poll):
 }
 }
 }
}

func main() {
 n := flag.Int("n", 10, "show last n lines")
 followMode := flag.Bool("f", false, "follow file")
 poll := flag.Duration("poll", 500*time.Millisecond, "follow poll interval")
 flag.Parse()

 if flag.NArg() != 1 {
 fmt.Fprintln(os.Stderr, "usage: gtail [-n N] [-f] <file>")
 os.Exit(2)
 }

 path := flag.Arg(0)
 lines, err := lastNLines(path, *n)
 if err != nil {
 fmt.Fprintln(os.Stderr, err)
 os.Exit(1)
 }
}

```

```

 for _, line := range lines {
 fmt.Println(line)
 }

 if !*followMode {
 return
 }

 st, err := os.Stat(path)
 if err != nil {
 fmt.Fprintln(os.Stderr, err)
 os.Exit(1)
 }

 ctx, stop := signal.NotifyContext(context.Background(), os.Interrupt, syscall.SIGTERM)
 defer stop()

 if err := follow(ctx, path, st.Size(), *poll); err != nil {
 fmt.Fprintln(os.Stderr, err)
 os.Exit(1)
 }
}

```

## Step-by-step build path

1. Ring buffer for last N lines (or container/ring).
2. Print initial window.
3. Seek to end size; Copy loop with poll sleep.
4. SIGINT via context.
5. Stretch: re-open on truncation/rotation.

## Run and verification

```

printf '1\n2\n3\n4\n5\n' > app.log
go run . -n 3 app.log
3 4 5

follow:
go run . -n 2 -f app.log
other terminal:
echo new >> app.log

```

## Tests

```
package main

import (
 "os"
 "path/filepath"
 "testing"
)

func TestLastN(t *testing.T) {
 dir := t.TempDir()
 path := filepath.Join(dir, "f")
 _ = os.WriteFile(path, []byte("a\nb\nc\nd\n"), 0o644)
 lines, err := lastNLines(path, 2)
 if err != nil {
 t.Fatal(err)
 }
 if len(lines) != 2 || lines[0] != "c" || lines[1] != "d" {
 t.Fatalf("%v", lines)
 }
}

func TestLastNShortFile(t *testing.T) {
 dir := t.TempDir()
 path := filepath.Join(dir, "f")
 _ = os.WriteFile(path, []byte("only\n"), 0o644)
 lines, err := lastNLines(path, 10)
 if err != nil || len(lines) != 1 {
 t.Fatalf("%v %v", lines, err)
 }
}

go test ./...
```

## Stretch goals

1. Detect truncate (size < offset) and reopen from start.
2. Inotify/fsnotify instead of poll (Linux).
3. Multiple files (like `tail -f a b`).
4. `-F` forever reopen by name after rotation.

## Pitfalls

Pitfall	Fix
Loading whole file for last N	ring buffer while scanning
Busy loop on EOF	sleep/poll or fsnotify
Ignoring rotation	track size/inode
No Ctrl+C in <code>-f</code>	signal.NotifyContext

## Learning goals

- Ring buffers for trailing windows
- Follow-mode IO
- Operator UX for log tools



## **011 Project 11: Build a du Clone**

# 011 Build a du Clone

Calculate recursive directory size and print human-readable or raw byte output. Use `filepath.WalkDir` and tolerate permission errors without aborting the whole walk.

```
dir walk -> sum file sizes -> print per path + total
```

## Problem statement

```
gdu [-h] [path...]
```

- Default path .
- Sum sizes of all files under each root
- `-h` human-readable (default true in this lab; use `-h=false` for bytes)
- Multiple roots → print total line

## Acceptance criteria

- ☐ Recursive size for directories
- ☐ Human and raw modes
- ☐ Skips unreadable entries without crash
- ☐ Multi-path grand total
- ☐ Stdlib only

## Setup

```
mkdir gdu && cd gdu
go mod init example.com/gdu
go 1.27
```

## Full main.go

```
package main

import (
 "flag"
 "fmt"
 "io/fs"
 "os"
 "path/filepath"
)

func human(n int64) string {
 units := []string{"B", "KB", "MB", "GB", "TB"}
 v := float64(n)
 i := 0
 for v >= 1024 && i < len(units)-1 {
 v /= 1024
 i++
 }
 return fmt.Sprintf("%.1f%s", v, units[i])
}

func dirSize(root string) (int64, error) {
 var total int64
 err := filepath.WalkDir(root, func(path string, d fs.DirEntry, err error) error {
 if err != nil {
 fmt.Fprintf(os.Stderr, "skip %s: %v\n", path, err)
 return nil
 }
 if d.IsDir() {
 return nil
 }
 info, err := d.Info()
 if err != nil {
 return nil
 }
 total += info.Size()
 return nil
 })
 return total, err
}

func format(sz int64, humanFmt bool) string {
 if humanFmt {
 return fmt.Sprintf("%8s", human(sz))
 }
}
```

```

 }
 return fmt.Sprintf("%8d", sz)
}

func main() {
 humanFmt := flag.Bool("h", true, "human readable")
 flag.Parse()

 args := flag.Args()
 if len(args) == 0 {
 args = []string{"."}
 }

 var grand int64
 var hadErr bool
 for _, p := range args {
 sz, err := dirSize(p)
 if err != nil {
 fmt.Fprintf(os.Stderr, "%s: %v\n", p, err)
 hadErr = true
 continue
 }
 grand += sz
 fmt.Printf("%8s %s\n", format(sz, *humanFmt), p)
 }

 if len(args) > 1 {
 fmt.Printf("%8s total\n", format(grand, *humanFmt))
 }
 if hadErr {
 os.Exit(1)
 }
}

```

## Step-by-step build path

1. Implement `human` with unit table.
2. `WalkDir` summing file sizes only.
3. Multi-root CLI + total.
4. Log skips to `stderr`; keep walking.
5. Tests with temp dirs of known size.

## Run and verification

```
go run . -h /var/log
go run . -h=false .
go run . -h /tmp /var/tmp
```

## Tests

```
package main

import (
 "os"
 "path/filepath"
 "testing"
)

func TestDirSize(t *testing.T) {
 dir := t.TempDir()
 _ = os.WriteFile(filepath.Join(dir, "a"), []byte("12345"), 0o644)
 _ = os.Mkdir(filepath.Join(dir, "sub"), 0o755)
 _ = os.WriteFile(filepath.Join(dir, "sub", "b"), []byte("xy"), 0o644)
 sz, err := dirSize(dir)
 if err != nil {
 t.Fatal(err)
 }
 if sz != 7 {
 t.Fatalf("got %d", sz)
 }
}

func TestHuman(t *testing.T) {
 if human(1024) != "1.0KB" {
 t.Fatal(human(1024))
 }
}

go test ./...
```

## Stretch goals

1. Apparent size vs allocated blocks (platform-specific).
2. Depth summary (-d 1 like du --max-depth).
3. Exclude globs.

4. Parallel walk with care for FS limits.

## Pitfalls

Pitfall	Fix
Aborting walk on permission error	return nil after log
Counting directory entries as files	skip <code>IsDir</code>
Binary vs decimal units	document 1024 base

## Learning goals

- Recursive FS walks
- Human-readable formatting
- Resilient error handling on real trees

## **012 Project 12: Build a find Clone**

# 012 Build a find Clone

Search a directory tree by name substring, entry type, and minimum size. Use `filepath.WalkDir` for efficient streaming walks without loading the whole tree into memory.

```
walk tree -> filter (name/type/size) -> print matching paths
```

## Problem statement

```
gfind [root] [-name SUBSTR] [-type f|d] [-min-size BYTES]
```

Default root is `..`. Filters compose as AND. Name match is case-insensitive substring on the base name (not full path)—document this; classic `find` uses globs.

## Acceptance criteria

- ☐ Walks recursively from root
- ☐ `-name` filters by basename contains (case-insensitive)
- ☐ `-type f` files only; `-type d` dirs only; empty = both
- ☐ `-min-size` applies to files only
- ☐ Permission errors: skip entry (or print to `stderr`) without aborting whole walk
- ☐ Stdlib only; `go build` clean

## Setup

```
mkdir gfind && cd gfind
go mod init example.com/gfind
go 1.27
```



## Full main.go

```
package main

import (
 "flag"
 "fmt"
 "io/fs"
 "os"
 "path/filepath"
 "strings"
)

func match(path string, d fs.DirEntry, name, typeFlag string, minSize int64) (bool, error) {
 base := filepath.Base(path)
 if name != "" && !strings.Contains(strings.ToLower(base), strings.ToLower(name)) {
 return false, nil
 }
 switch typeFlag {
 case "f":
 if d.IsDir() {
 return false, nil
 }
 case "d":
 if !d.IsDir() {
 return false, nil
 }
 case "":
 // ok
 default:
 return false, fmt.Errorf("invalid -type %q (want f, d, or empty)", typeFlag)
 }
 if minSize > 0 && !d.IsDir() {
 info, err := d.Info()
 if err != nil {
 return false, nil // skip unreadable
 }
 if info.Size() < minSize {
 return false, nil
 }
 }
 return true, nil
}

func main() {
 name := flag.String("name", "", "substring to match in filename")
}
```

```

typeFlag := flag.String("type", "", "f=file d=dir")
minSize := flag.Int64("min-size", 0, "minimum file size in bytes")
flag.Parse()

root := "."
if flag.NArg() > 0 {
 root = flag.Arg(0)
}

// Validate type early
if *typeFlag != "" && *typeFlag != "f" && *typeFlag != "d" {
 fmt.Fprintf(os.Stderr, "invalid -type %q\n", *typeFlag)
 os.Exit(2)
}

err := filepath.WalkDir(root, func(path string, d fs.DirEntry, err error) error {
 if err != nil {
 fmt.Fprintf(os.Stderr, "skip %s: %v\n", path, err)
 return nil // keep walking
 }
 ok, mErr := match(path, d, *name, *typeFlag, *minSize)
 if mErr != nil {
 return mErr
 }
 if ok {
 fmt.Println(path)
 }
 return nil
})
if err != nil {
 fmt.Fprintln(os.Stderr, err)
 os.Exit(1)
}
}

```

## Step-by-step build path

1. Flags + optional root argument.
2. `WalkDir` callback: handle walk errors without failing the whole run.
3. Centralize filters in `match` for unit tests.
4. Print paths as you go (streaming).
5. Later: add prune (`-maxdepth`) and glob `filepath.Match`.

## Run and verification

```
go run . /etc -name conf -type f
go run . . -name go -type f
go run . . -min-size 1048576 -type f # files 1MiB

empty name matches everything (careful on huge trees)
go run . /tmp -type d | head
```

## Tests

```
package main

import (
 "os"
 "path/filepath"
 "testing"
)

func TestMatchNameAndType(t *testing.T) {
 dir := t.TempDir()
 f := filepath.Join(dir, "App.LOG")
 if err := os.WriteFile(f, []byte("hi"), 0o644); err != nil {
 t.Fatal(err)
 }
 info, err := os.Stat(f)
 if err != nil {
 t.Fatal(err)
 }
 // DirEntry from WalkDir; use type from os.DirEntry via ReadDir
 entries, _ := os.ReadDir(dir)
 var d os.DirEntry
 for _, e := range entries {
 if e.Name() == "App.LOG" {
 d = e
 break
 }
 }
 ok, err := match(f, d, "log", "f", 0)
 if err != nil || !ok {
 t.Fatalf("ok=%v err=%v size=%d", ok, err, info.Size())
 }
}
```

```
go test ./...
```

## Stretch goals

1. Glob `-name '*.go'` via `filepath.Match`.
2. `-maxdepth N` by counting path separators relative to root.
3. `-exec` style: print null-terminated for `xargs -0`.
4. Concurrent walk with worker pool (careful with FS limits).
5. Skip directories (`.git`, `node_modules`) with a skip list.

## Pitfalls

Pitfall	Fix
Returning walk <code>err</code> aborts tree	Log and <code>return nil</code>
<code>Info()</code> on every dir for size	Only when min-size set and file
Symlink loops	<code>WalkDir</code> doesn't follow symlinks by default—good
Case-sensitive surprise	Document case-insensitive policy

## Learning goals

- `filepath.WalkDir` and `fs.DirEntry`
- Filter composition for CLI tools
- Resilient walks over imperfect trees

## **013 Project 13: Build a wc Clone**

# 013 Build a wc Clone

Count lines, words, and bytes across files—streaming with `bufio.Scanner` and Unicode-aware word boundaries via `unicode.IsSpace`.

```
file -> scanner/read -> count L/W/B -> per file + total
```

## Problem statement

```
gwc [file...]
```

- No files → read stdin
- Print `lines words bytes [filename]`
- Multiple files → final `total` row
- Words: maximal non-space sequences (Unicode spaces)

## Acceptance criteria

- ☐ Correct counts on simple fixtures
- ☐ Stdin mode works
- ☐ Multi-file total row
- ☐ Missing file: `stderr + continue`
- ☐ Stdlib only

## Setup

```
mkdir gwc && cd gwc
go mod init example.com/gwc
go 1.27
```

## Full main.go

```
package main

import (
 "bufio"
 "flag"
 "fmt"
 "io"
 "os"
 "unicode"
)

type counts struct {
 lines int
 words int
 bytes int
}

func countReader(r io.Reader) (counts, error) {
 var c counts
 s := bufio.NewScanner(r)
 // Note: Scanner strips newlines; we approximate bytes as len(line)+1 per line.
 // For exact byte counts matching GNU wc, use raw Read and dual-pass logic (stretch).
 buf := make([]byte, 0, 64*1024)
 s.Buffer(buf, 1024*1024)
 for s.Scan() {
 line := s.Text()
 c.lines++
 c.bytes += len(line) + 1 // +1 for '\n' (last line without newline is imperfect)
 inWord := false
 for _, ch := range line {
 if unicode.IsSpace(ch) {
 inWord = false
 continue
 }
 if !inWord {
 c.words++
 inWord = true
 }
 }
 }
 return c, s.Err()
}

func printCounts(c counts, name string) {
```

```

 if name == "" {
 fmt.Printf("%8d %8d %8d\n", c.lines, c.words, c.bytes)
 return
 }
 fmt.Printf("%8d %8d %8d %s\n", c.lines, c.words, c.bytes, name)
}

func main() {
 flag.Parse()
 if flag.NArg() == 0 {
 c, err := countReader(os.Stdin)
 if err != nil {
 fmt.Fprintln(os.Stderr, err)
 os.Exit(1)
 }
 printCounts(c, "")
 return
 }

 var total counts
 var hadErr bool
 for _, path := range flag.Args() {
 f, err := os.Open(path)
 if err != nil {
 fmt.Fprintf(os.Stderr, "%s: %v\n", path, err)
 hadErr = true
 continue
 }
 c, err := countReader(f)
 _ = f.Close()
 if err != nil {
 fmt.Fprintf(os.Stderr, "%s: %v\n", path, err)
 hadErr = true
 continue
 }
 total.lines += c.lines
 total.words += c.words
 total.bytes += c.bytes
 printCounts(c, path)
 }
 if flag.NArg() > 1 {
 printCounts(total, "total")
 }
 if hadErr {
 os.Exit(1)
 }
}

```



## Step-by-step build path

1. `countReader(io.Reader)` returns struct.
2. Word state machine with `unicode.IsSpace`.
3. Stdin vs files branching.
4. Aggregate totals.
5. Document newline/byte caveats vs GNU `wc`.

## Run and verification

```
printf 'hello world\n\nfoo\n' > t.txt
go run . t.txt
echo 'a b c' | go run .
go run . t.txt t.txt
```

## Tests

```
package main

import (
 "strings"
 "testing"
)

func TestCountReader(t *testing.T) {
 c, err := countReader(strings.NewReader("hi there\n"))
 if err != nil {
 t.Fatal(err)
 }
 if c.lines != 1 || c.words != 2 {
 t.Fatalf("%+v", c)
 }
}

func TestEmpty(t *testing.T) {
 c, err := countReader(strings.NewReader(""))
 if err != nil || c.lines != 0 || c.words != 0 {
 t.Fatalf("%+v %+v", c, err)
 }
}

go test ./...
```

## Stretch goals

1. Exact byte count with `io.Copy` to `byteCounter` writer.
2. Flags `-l`, `-w`, `-c` to select columns.
3. Read from multiple compressed files.
4. Match GNU `wc` on files without trailing newline.

## Pitfalls

Pitfall	Fix
Byte count vs scanner	document or dual-count
ASCII-only words	use unicode
Huge lines	<code>Scanner.Buffer</code>

## Learning goals

- Streaming text metrics
- Unicode word splitting
- Multi-file CLI totals

## **014 Project 14: HTTP Load Tester**

# 014 Build an HTTP Load Tester

Generate concurrent requests and report throughput and latency.

```
N workers -> request loop -> result channel -> aggregate metrics
```

## Full main.go

```
package main

import (
 "flag"
 "fmt"
 "net/http"
 "os"
 "sync"
 "time"
)

type result struct {
 ok bool
 dur time.Duration
}

func main() {
 url := flag.String("url", "http://localhost:8080/health", "target URL")
 concurrency := flag.Int("c", 20, "concurrent workers")
 requests := flag.Int("n", 500, "total requests")
 timeout := flag.Duration("t", 3*time.Second, "request timeout")
 flag.Parse()

 client := &http.Client{Timeout: *timeout}
 results := make(chan result, *requests)

 start := time.Now()
 var wg sync.WaitGroup
 jobs := make(chan struct{}, *requests)
 for i := 0; i < *requests; i++ {
 jobs <- struct{}{}
 }
```

```

}
close(jobs)

for w := 0; w < *concurrency; w++ {
 wg.Add(1)
 go func() {
 defer wg.Done()
 for range jobs {
 t0 := time.Now()
 resp, err := client.Get(*url)
 d := time.Since(t0)
 if err != nil {
 results <- result{ok: false, dur: d}
 continue
 }
 _ = resp.Body.Close()
 results <- result{ok: resp.StatusCode < 500, dur: d}
 }
 }()
}

wg.Wait()
close(results)

var okCount int
var min, max, sum time.Duration
min = 1<<63 - 1
for r := range results {
 if r.ok {
 okCount++
 }
 if r.dur < min {
 min = r.dur
 }
 if r.dur > max {
 max = r.dur
 }
 sum += r.dur
}

elapsed := time.Since(start)
avg := time.Duration(int64(sum) / int64(*requests))
rps := float64(*requests) / elapsed.Seconds()

fmt.Printf("target: %s\n", *url)
fmt.Printf("requests: %d, success: %d, failed: %d\n", *requests, okCount, *requests-okCount)

```

```
 fmt.Printf("latency min/avg/max: %v/%v/%v\n", min, avg, max)
 fmt.Printf("elapsed: %v, throughput: %.2f req/s\n", elapsed, rps)

 if okCount == 0 {
 os.Exit(1)
 }
}
```

## Run

```
go run . -url http://localhost:8080/health -c 50 -n 5000
```

## Step-by-Step Explanation

1. Model jobs, workers, and outputs explicitly.
2. Bound concurrency using worker pools and buffered channels.
3. Use `sync.WaitGroup` for lifecycle control.
4. Aggregate worker results in one place.
5. Verify behavior under both normal and failure paths.

## Code Anatomy

- Producer pushes jobs into a channel.
- Workers consume jobs and emit results.
- Aggregator merges results and prints summary.

## Learning Goals

- Build leak-free goroutine patterns.
- Balance throughput and resource limits.
- Understand fan-out/fan-in architecture.

## **015 Project 15: TUI System Monitor**

# 015 Build a TUI System Monitor

Create a live terminal UI for CPU, memory, and goroutine stats using Bubble Tea + gopsutil.

```
ticker -> collect metrics -> update model -> render tui frame
```

## Setup

```
go mod init example.com/gotop
go get github.com/charmbracelet/bubbletea@latest
go get github.com/charmbracelet/lipgloss@latest
go get github.com/shirou/gopsutil/v4/cpu@latest
go get github.com/shirou/gopsutil/v4/mem@latest
```

## Full main.go

```
package main

import (
 "fmt"
 "runtime"
 "time"

 tea "github.com/charmbracelet/bubbletea"
 "github.com/charmbracelet/lipgloss"
 "github.com/shirou/gopsutil/v4/cpu"
 "github.com/shirou/gopsutil/v4/mem"
)

type tickMsg struct{}

type sample struct {
 cpuPct float64
 memPct float64
 gorCount int
 ts time.Time
}
```



```

type model struct {
 last sample
 err error
}

func tickCmd() tea.Cmd {
 return tea.Tick(1*time.Second, func(time.Time) tea.Msg { return tickMsg{} })
}

func (m model) Init() tea.Cmd { return tickCmd() }

func (m model) Update(msg tea.Msg) (tea.Model, tea.Cmd) {
 switch msg := msg.(type) {
 case tea.KeyMsg:
 if msg.String() == "q" || msg.String() == "ctrl+c" {
 return m, tea.Quit
 }
 case tickMsg:
 cp, err := cpu.Percent(0, false)
 if err != nil || len(cp) == 0 {
 m.err = err
 return m, tickCmd()
 }
 vm, err := mem.VirtualMemory()
 if err != nil {
 m.err = err
 return m, tickCmd()
 }
 m.last = sample{
 cpuPct: cp[0],
 memPct: vm.UsedPercent,
 gorCount: runtime.NumGoroutine(),
 ts: time.Now(),
 }
 m.err = nil
 return m, tickCmd()
 }
 return m, nil
}

func (m model) View() string {
 title := lipgloss.NewStyle().Bold(true).Foreground(lipgloss.Color("86")).Render("GoTop -")
 if m.err != nil {
 return fmt.Sprintf("%s\n\nerror: %v\n\npress q to quit\n", title, m.err)
 }
 return fmt.Sprintf(

```

```

 "%s\n\nCPU: %6.2f%%\nMemory: %6.2f%%\nGoroutines: %d\nUpdated: %s\n\n",
 title,
 m.last.cpuPct,
 m.last.memPct,
 m.last.gorCount,
 m.last.ts.Format(time.TimeOnly),
)
}

func main() {
 p := tea.NewProgram(model{})
 if _, err := p.Run(); err != nil {
 panic(err)
 }
}

```

## Run

```
go run .
```

## Step-by-Step Explanation

1. Collect node/resource metrics from API sources.
2. Compute deterministic scores per target.
3. Rank candidates by score and policy constraints.
4. Produce dry-run placement/migration decisions.
5. Apply gradually with canary and rollback plan.

## Code Anatomy

- Data collection stage fetches capacity and load.
- Scoring stage turns metrics into comparable values.
- Decision stage emits ranked scheduling actions.

## Learning Goals

- Build scheduling logic that is explainable and auditable.
- Balance utilization, reliability, and operational safety.
- Prepare for production-grade orchestration workflows.

## **016 Project 16: Proxmox TUI Manager**

# 016 Build a Proxmox TUI Manager

This project builds an advanced terminal UI that talks to Proxmox API.

Features: - list nodes - list QEMU VMs on selected node - start/stop VM - refresh state

```
TUI keypress -> API client -> Proxmox endpoint -> update model -> redraw UI
```

## Setup

```
go mod init example.com/proxmox-tui
go get github.com/charmbracelet/bubbletea@latest
go get github.com/charmbracelet/lipgloss@latest
```

Set env vars:

```
export PVE_BASE_URL="https://proxmox.local:8006"
export PVE_TOKEN="user@pam!token=secret"
```

## Full main.go

```
package main

import (
 "crypto/tls"
 "encoding/json"
 "fmt"
 "io"
 "net/http"
 "os"
 "strconv"
 "time"

 tea "github.com/charmbracelet/bubbletea"
 "github.com/charmbracelet/lipgloss"
)
```

```

type node struct {
 Node string `json:"node"`
}

type vm struct {
 VMID int `json:"vmid"`
 Name string `json:"name"`
 Status string `json:"status"`
}

type apiResp[T any] struct {
 Data T `json:"data"`
}

type client struct {
 base string
 hc *http.Client
}

func newClient(base string) *client {
 return &client{
 base: base,
 hc: &http.Client{Timeout: 8 * time.Second, Transport: &http.Transport{
 TLSClientConfig: &tls.Config{InsecureSkipVerify: true},
 }},
 }
}

func (c *client) do(method, path string, body io.Reader) (*http.Response, error) {
 req, err := http.NewRequest(method, c.base+path, body)
 if err != nil {
 return nil, err
 }
 req.Header.Set("Authorization", "PVEAPIToken="+os.Getenv("PVE_TOKEN"))
 return c.hc.Do(req)
}

func (c *client) listNodes() ([]node, error) {
 resp, err := c.do(http.MethodGet, "/api2/json/nodes", nil)
 if err != nil {
 return nil, err
 }
 defer resp.Body.Close()
 var out apiResp[[]node]
 if err := json.NewDecoder(resp.Body).Decode(&out); err != nil {
 return nil, err
 }
}

```

```

 return out.Data, nil
}

func (c *client) listVMs(nodeName string) ([]vm, error) {
 resp, err := c.do(http.MethodGet, "/api2/json/nodes/"+nodeName+"/qemu", nil)
 if err != nil {
 return nil, err
 }
 defer resp.Body.Close()
 var out apiResp[[]vm]
 if err := json.NewDecoder(resp.Body).Decode(&out); err != nil {
 return nil, err
 }
 return out.Data, nil
}

func (c *client) vmAction(nodeName string, vmid int, action string) error {
 resp, err := c.do(http.MethodPost, "/api2/json/nodes/"+nodeName+"/qemu/"+strconv.Itoa(vmid)+"/"+action, nil)
 if err != nil {
 return err
 }
 defer resp.Body.Close()
 if resp.StatusCode >= 300 {
 b, _ := io.ReadAll(resp.Body)
 return fmt.Errorf("action failed: %s", string(b))
 }
 return nil
}

type loadedMsg struct {
 nodes []node
 vms []vm
 err error
}

type actionMsg struct{ err error }

type model struct {
 cli *client
 nodes []node
 vms []vm
 sel int
 node string
 status string
}

func loadCmd(c *client, n string) tea.Cmd {

```

```

return func() tea.Msg {
 ns, err := c.listNodes()
 if err != nil {
 return loadedMsg{err: err}
 }
 nodeName := n
 if nodeName == "" && len(ns) > 0 {
 nodeName = ns[0].Node
 }
 vs, err := c.listVMs(nodeName)
 return loadedMsg{nodes: ns, vms: vs, err: err}
}
}

func (m model) Init() tea.Cmd { return loadCmd(m.cli, "") }

func (m model) Update(msg tea.Msg) (tea.Model, tea.Cmd) {
 switch msg := msg.(type) {
 case tea.KeyMsg:
 switch msg.String() {
 case "q", "ctrl+c":
 return m, tea.Quit
 case "r":
 return m, loadCmd(m.cli, m.node)
 case "down", "j":
 if m.sel < len(m.vms)-1 {
 m.sel++
 }
 case "up", "k":
 if m.sel > 0 {
 m.sel--
 }
 case "s": // start
 if len(m.vms) == 0 {
 return m, nil
 }
 v := m.vms[m.sel]
 return m, func() tea.Msg { return actionMsg{err: m.cli.vmAction(m.node, v.VMID,
 case "x": // stop
 if len(m.vms) == 0 {
 return m, nil
 }
 v := m.vms[m.sel]
 return m, func() tea.Msg { return actionMsg{err: m.cli.vmAction(m.node, v.VMID,
 }
 case loadedMsg:

```

```

 if msg.err != nil {
 m.status = "load error: " + msg.err.Error()
 return m, nil
 }
 m.nodes = msg.nodes
 m.vms = msg.vms
 if m.node == "" && len(msg.nodes) > 0 {
 m.node = msg.nodes[0].Node
 }
 if m.sel >= len(m.vms) {
 m.sel = len(m.vms) - 1
 if m.sel < 0 {
 m.sel = 0
 }
 }
 m.status = "loaded"
 case actionMsg:
 if msg.err != nil {
 m.status = "action error: " + msg.err.Error()
 return m, nil
 }
 m.status = "action submitted"
 return m, loadCmd(m.cli, m.node)
 }
 return m, nil
}

func (m model) View() string {
 title := lipgloss.NewStyle().Bold(true).Foreground(lipgloss.Color("39")).Render("Proxmox")
 s := title + "\n"
 s += fmt.Sprintf("node: %s | status: %s\n", m.node, m.status)
 s += "keys: j/k move | s start | x stop | r refresh | q quit\n\n"
 for i, v := range m.vms {
 cur := " "
 if i == m.sel {
 cur = ">"
 }
 s += fmt.Sprintf("%s vmid=%d name=%s status=%s\n", cur, v.VMID, v.Name, v.Status)
 }
 return s
}

func main() {
 base := os.Getenv("PVE_BASE_URL")
 if base == "" || os.Getenv("PVE_TOKEN") == "" {
 fmt.Println("set PVE_BASE_URL and PVE_TOKEN")
 os.Exit(2)
 }
}

```



```
}
m := model{cli: newClient(base), status: "booting"}
if _, err := tea.NewProgram(m).Run(); err != nil {
 panic(err)
}
}
```

## Run

```
go run .
```

## Notes

- This sample uses `InsecureSkipVerify` for lab environments. Use proper TLS validation in production.
- Use API tokens with least privilege.

## Step-by-Step Explanation

1. Collect node/resource metrics from API sources.
2. Compute deterministic scores per target.
3. Rank candidates by score and policy constraints.
4. Produce dry-run placement/migration decisions.
5. Apply gradually with canary and rollback plan.

## Code Anatomy

- Data collection stage fetches capacity and load.
- Scoring stage turns metrics into comparable values.
- Decision stage emits ranked scheduling actions.

## Learning Goals

- Build scheduling logic that is explainable and auditable.
- Balance utilization, reliability, and operational safety.
- Prepare for production-grade orchestration workflows.

## **017 Project 17: SSH Remote Orchestrator**

# 017 Build an SSH Remote Orchestrator

Run commands on many hosts concurrently over SSH and aggregate results.

```
host list -> worker pool -> ssh exec per host -> collect stdout/stderr -> summary
```

## Setup

```
go mod init example.com/gossh-orchestrator
go get golang.org/x/crypto/ssh@latest
```

## Full main.go

```
package main

import (
 "flag"
 "fmt"
 "os"
 "strings"
 "sync"
 "time"

 "golang.org/x/crypto/ssh"
)

type result struct {
 host string
 out string
 err error
}

func run(host, user, keyPath, cmd string, timeout time.Duration) result {
 key, err := os.ReadFile(keyPath)
 if err != nil {
 return result{host: host, err: err}
 }
}
```

```

 signer, err := ssh.ParsePrivateKey(key)
 if err != nil {
 return result{host: host, err: err}
 }

 cfg := &ssh.ClientConfig{
 User: user,
 Auth: []ssh.AuthMethod{ssh.PublicKeys(signer)},
 HostKeyCallback: ssh.InsecureIgnoreHostKey(),
 Timeout: timeout,
 }

 c, err := ssh.Dial("tcp", host+":22", cfg)
 if err != nil {
 return result{host: host, err: err}
 }
 defer c.Close()

 s, err := c.NewSession()
 if err != nil {
 return result{host: host, err: err}
 }
 defer s.Close()

 out, err := s.CombinedOutput(cmd)
 return result{host: host, out: string(out), err: err}
}

func main() {
 hostsArg := flag.String("hosts", "", "comma-separated host list")
 user := flag.String("user", "root", "ssh user")
 key := flag.String("key", os.Getenv("HOME")+"/.ssh/id_rsa", "private key path")
 cmd := flag.String("cmd", "uname -a", "command to run")
 workers := flag.Int("w", 10, "worker count")
 timeout := flag.Duration("t", 5*time.Second, "ssh timeout")
 flag.Parse()

 if *hostsArg == "" {
 fmt.Fprintln(os.Stderr, "provide -hosts")
 os.Exit(2)
 }
 hosts := strings.Split(*hostsArg, ",")

 jobs := make(chan string)
 results := make(chan result)
 var wg sync.WaitGroup

```

```

for i := 0; i < *workers; i++ {
 wg.Add(1)
 go func() {
 defer wg.Done()
 for h := range jobs {
 results <- run(strings.TrimSpace(h), *user, *key, *cmd, *timeout)
 }
 }()
}

go func() {
 for _, h := range hosts {
 jobs <- h
 }
 close(jobs)
 wg.Wait()
 close(results)
}()

ok := 0
for r := range results {
 if r.err != nil {
 fmt.Printf("[%s] ERROR: %v\n", r.host, r.err)
 continue
 }
 ok++
 fmt.Printf("[%s]\n%s\n", r.host, strings.TrimSpace(r.out))
}
fmt.Printf("done: %d/%d succeeded\n", ok, len(hosts))
}

```

## Run

```
go run . -hosts "10.0.0.11,10.0.0.12" -user root -cmd "uptime"
```

## Security Notes

- Replace `ssh.InsecureIgnoreHostKey` with known-hosts validation in production.
- Use dedicated restricted SSH keys for automation.

## Step-by-Step Explanation

1. Model jobs, workers, and outputs explicitly.
2. Bound concurrency using worker pools and buffered channels.
3. Use `sync.WaitGroup` for lifecycle control.
4. Aggregate worker results in one place.
5. Verify behavior under both normal and failure paths.

## Code Anatomy

- Producer pushes jobs into a channel.
- Workers consume jobs and emit results.
- Aggregator merges results and prints summary.

## Learning Goals

- Build leak-free goroutine patterns.
- Balance throughput and resource limits.
- Understand fan-out/fan-in architecture.

## **018 Project 18: Go Workout - 200 Ten-Minute Exercises**

# 018 Go Workout: 200 Ten-Minute Exercises

This chapter uses a **short-exercise format** inspired by workout-style learning books: small tasks, tight scope, immediate practice.

## How to Use

- Spend 10 minutes per exercise.
- Do not copy old answers.
- Prefer standard library first.
- After every 10 exercises, write a short reflection.

```
10-minute loop
```

```
read task -> code -> run -> test edge case -> note takeaway
```

## Foundation Warmup (1-20)

1. Parse two integers from CLI args and print sum.
2. Parse `-name` flag and print greeting.
3. Build a `switch` on day number (1-7).
4. Convert Celsius to Fahrenheit with proper formatting.
5. Reverse a string safely for ASCII.
6. Count vowels in a string.
7. Find min/max in an `[]int`.
8. Remove duplicates from a slice.
9. Implement `contains([]string, string) bool`.
10. Build a frequency map for words in a sentence.
11. Sort a slice of ints ascending.
12. Sort a slice of structs by one field.
13. Write a function returning `(value, error)` for divide.
14. Use `defer` to time function execution.
15. Create a custom error type with context.
16. Marshal struct to JSON with tags.
17. Unmarshal JSON into struct and validate fields.
18. Read env var with fallback default.
19. Parse RFC3339 timestamp.
20. Format duration nicely (`1h2m3s`).



## Files and CLI Tools (21-40)

21. Read file and print line count.
22. Read file and print byte count.
23. Read from stdin and uppercase output.
24. Build tiny `head -n` clone.
25. Build tiny `tail -n` clone.
26. Filter lines containing a substring.
27. Replace substring in each line and print.
28. Walk directory and count files.
29. Walk directory and sum file sizes.
30. Print top 10 largest files in a directory tree.
31. Build file extension counter (`.go`, `.md`, etc.).
32. Skip hidden files while walking.
33. Add `-json` output mode to a CLI tool.
34. Add `-quiet` and `-verbose` log levels.
35. Create a checksum (SHA-256) for one file.
36. Compare two files by checksum.
37. Build tiny `wc` (lines/words/bytes).
38. Build tiny `grep` with regex input.
39. Add case-insensitive flag to `grep` clone.
40. Print first match index and line number.

## Structs, Methods, Interfaces (41-60)

41. Define `User` struct and constructor.
42. Add `String()` method to struct.
43. Add pointer receiver mutator method.
44. Add value receiver read-only method.
45. Implement small interface (`Runner`) with two types.
46. Compose two structs via embedding.
47. Resolve method name conflict in embedded structs.
48. Implement `io.Writer` that counts bytes.
49. Implement `io.Reader` over in-memory string.
50. Build a decorator function for timing any function.
51. Build `Option` pattern for server config.
52. Validate config and return combined error.
53. Compare two structs for equality constraints.
54. Parse CSV into typed struct slice.
55. Convert struct slice to map by key field.
56. Implement generic `Map` helper for slices.
57. Implement generic `Filter` helper.
58. Implement generic `Reduce` helper.
59. Create custom type alias with methods.
60. Add unit tests for all methods.

## Errors and Reliability (61-80)

61. Wrap errors with `%w` across 3 layers.
62. Detect sentinel error with `errors.Is`.
63. Detect custom type with `errors.As`.
64. Build retry helper with max attempts.
65. Add exponential backoff to retry helper.
66. Add jitter to retry backoff.
67. Build circuit-breaker lite (open/close states).
68. Add timeout via `context.WithTimeout`.
69. Add cancellation via parent context.
70. Build `must` helper for internal tooling only.
71. Build panic recovery middleware for HTTP.
72. Capture stack trace on panic and log.
73. Return structured JSON error response.
74. Add error codes enum for API errors.
75. Validate all CLI input and fail fast.
76. Add dead-letter file for failed jobs.
77. Add graceful shutdown on SIGINT.
78. Add idempotency key check for commands.
79. Add bounded retries for network call.
80. Add fallback path when primary call fails.

## Concurrency Basics (81-100)

81. Launch 5 goroutines and wait with `WaitGroup`.
82. Fan-out tasks to worker pool.
83. Fan-in results from workers.
84. Add context cancellation to worker pool.
85. Add timeout to a blocking goroutine.
86. Use buffered channel to smooth bursts.
87. Use unbuffered channel to enforce handoff.
88. Prevent goroutine leak on early return.
89. Build bounded semaphore using channel.
90. Merge two channels into one output.
91. Implement producer-consumer with backpressure.
92. Add `select` with timeout case.
93. Add `select` with default non-blocking case.
94. Protect shared map with mutex.
95. Replace mutex map with `sync.Map` and compare.
96. Use `sync.Once` for singleton init.
97. Use `sync.Cond` for queue notifications.
98. Build rate limiter with ticker.
99. Build token bucket limiter.
100. Profile goroutine count during load.

## Networking and HTTP (101-120)

101. Build `/health` endpoint.
102. Add `/ready` endpoint with dependency check.
103. Parse query params safely.
104. Parse JSON request body with size limit.
105. Return JSON response helper function.
106. Add middleware for request logging.
107. Add middleware for panic recovery.
108. Add middleware for request ID.
109. Add middleware for timeout context.
110. Add simple in-memory rate limit middleware.
111. Build URL shortener `POST /shorten`.
112. Build URL shortener `GET /{code}` redirect.
113. Add TTL to short links.
114. Add short-link collision handling.
115. Add ETag support to one endpoint.
116. Add graceful server shutdown with context.
117. Build HTTP client with timeout + retries.
118. Build HTTP client with custom transport.
119. Add gzip compression middleware.
120. Benchmark one endpoint with your load tester.

## Data and Persistence (121-140)

121. Use `encoding/csv` to read sample data.
122. Use `encoding/csv` to write report file.
123. Store key-value data in BoltDB (or bbolt).
124. Build repository interface over in-memory store.
125. Add file-backed repository implementation.
126. Add migration version field to data model.
127. Add optimistic locking field (`version`).
128. Add simple cache with TTL map.
129. Add cache invalidation on write.
130. Serialize map to JSON snapshot file.
131. Recover state from snapshot on startup.
132. Add periodic snapshot goroutine.
133. Add checksum to snapshot file.
134. Add snapshot restore validation.
135. Build append-only event log file.
136. Replay event log to rebuild state.
137. Add compaction command for old events.
138. Add corruption detection for events.
139. Add CLI export to CSV.
140. Add CLI import from CSV.

## Testing Workout (141-160)

141. Write table-driven tests for parser.
142. Add subtests by category.
143. Add golden file test for CLI output.
144. Add benchmark with `b.Loop`.
145. Add benchmark with `-benchmem` analysis.
146. Add fuzz test for JSON decode path.
147. Add race test and fix discovered issue.
148. Add test helper for temporary fixture files.
149. Add `t.Cleanup` in all integration tests.
150. Use `httptest` for handler test.
151. Use `httptest.Server` for client test.
152. Use `testing/synctest` for flaky timer logic.
153. Add tests for retry backoff boundaries.
154. Add tests for context timeout path.
155. Add tests for panic recovery middleware.
156. Add tests for rate limiter edge case.
157. Add tests for short-link collision behavior.
158. Add tests for file checksum mismatch path.
159. Add integration test with build tag `integration`.
160. Add CI test command matrix in README.

## Linux Tooling Track (161-180)

161. Extend `ls` clone with `-R` recursion.
162. Extend `ls` clone with colorized output.
163. Extend `cat` clone with `-T` tab markers.
164. Extend `grep` clone with invert match `-v`.
165. Extend `grep` clone with count-only `-c`.
166. Extend `tail` clone to handle file rotation.
167. Extend `du` clone with max depth flag.
168. Extend `find` clone with regex match mode.
169. Extend `wc` clone with rune count flag.
170. Build mini `xargs` clone.
171. Build mini `cut` clone.
172. Build mini `sort` clone.
173. Build mini `uniq` clone.
174. Build mini `tee` clone.
175. Build mini `which` clone.
176. Build mini `ps` clone using `/proc` (Linux).
177. Build mini `free` clone from `/proc/meminfo`.
178. Build mini `uptime` clone from `/proc/uptime`.
179. Build mini `df` clone using `syscall.Statfs`.
180. Build mini `watch` clone for repeated commands.

## Advanced TUI and Infra Track (181-200)

181. Add CPU history sparkline to TUI monitor.
182. Add memory history sparkline to TUI monitor.
183. Add keybinding help modal in TUI.
184. Add theme toggle (light/dark terminal palette).
185. Add status bar with connection state.
186. Add periodic refresh ticker with pause key.
187. Add filtering by VM status in Proxmox TUI.
188. Add node selector screen in Proxmox TUI.
189. Add VM details pane in Proxmox TUI.
190. Add confirmation dialog before VM stop.
191. Add async action queue and progress states.
192. Add retry-on-429 for Proxmox API calls.
193. Add structured logs for every API request.
194. Add audit log file for start/stop actions.
195. Add SSH orchestrator output to JSON report.
196. Add SSH orchestrator parallel limit per subnet.
197. Add host reachability precheck before SSH.
198. Add rollout mode: canary then full batch.
199. Add rollback command set for failed rollout.
200. Build capstone: TUI panel that triggers SSH jobs and shows live results.

## Chapter References in This Book

Use these while solving workouts:

- Foundations: `content/01-foundations/`
- Core: `content/02-core/`
- Concurrency: `content/07-concurrency-parallelism/`
- Testing: `content/06-testing/`
- Projects: `content/99-projects/`

## Step-by-Step Explanation

1. Pick a small set of exercises by track.
2. Timebox each attempt to ten minutes.
3. Record one takeaway and one weakness after each exercise.
4. Revisit chapter references when blocked.
5. Re-solve selected problems from memory weekly.

## Learning Goals

- Build consistency, not one-time intensity.

- Improve retrieval and transfer of Go patterns.
- Progress from syntax fluency to engineering fluency.

## **019 Project 19: Linux ps-lite**

# 019 Build a Linux ps-Lite Tool

Read process info from `/proc` and print a compact process table. Linux-only; detect unsupported platforms cleanly.

```
/proc -> parse /proc/<pid>/stat + comm -> rows -> sort by pid -> table
```

## Problem statement

```
gps [-n N]
```

- Scan `/proc` numeric directories
- Parse `stat` for state and RSS (pages → bytes)
- Print PID, state, RSS MiB, comm
- Limit rows with `-n`

## Acceptance criteria

- ☐ Works on Linux with `/proc`
- ☐ Clear message on macOS/Windows
- ☐ Sorted by PID
- ☐ Handles processes vanishing mid-scan
- ☐ Stdlib only

## Setup

```
mkdir gps && cd gps
go mod init example.com/gps
go 1.27
```



## Full main.go

```
package main

import (
 "flag"
 "fmt"
 "os"
 "path/filepath"
 "sort"
 "strconv"
 "strings"
)

type proc struct {
 PID int
 Comm string
 RSS int64
 State string
}

func parseProc(pid int) (proc, error) {
 statPath := fmt.Sprintf("/proc/%d/stat", pid)
 b, err := os.ReadFile(statPath)
 if err != nil {
 return proc{}, err
 }
 // comm is in parentheses and may contain spaces: (chrome --type=...)
 s := string(b)
 l := strings.IndexByte(s, '(')
 r := strings.LastIndexByte(s, ')')
 if l < 0 || r < 0 || r <= l {
 return proc{}, fmt.Errorf("bad stat format")
 }
 comm := s[l+1 : r]
 rest := strings.Fields(s[r+2:]) // after " "
 if len(rest) < 22 {
 return proc{}, fmt.Errorf("short stat")
 }
 state := rest[0]
 // rss is field 24 in /proc/[pid]/stat → index 21 in rest (after state=rest[0])
 rssPages, err := strconv.ParseInt(rest[21], 10, 64)
 if err != nil {
 return proc{}, err
 }
 pageSize := int64(os.Getpagesize())
```

```

 return proc{PID: pid, Comm: comm, State: state, RSS: rssPages * pageSize}, nil
}

func main() {
 limit := flag.Int("n", 50, "max rows")
 flag.Parse()

 entries, err := os.ReadDir("/proc")
 if err != nil {
 fmt.Fprintln(os.Stderr, "/proc unavailable (Linux only):", err)
 os.Exit(1)
 }

 var rows []proc
 for _, e := range entries {
 if !e.IsDir() {
 continue
 }
 pid, err := strconv.Atoi(e.Name())
 if err != nil {
 continue
 }
 p, err := parseProc(pid)
 if err != nil {
 continue // process exited
 }
 rows = append(rows, p)
 }

 sort.Slice(rows, func(i, j int) bool { return rows[i].PID < rows[j].PID })
 if *limit > len(rows) {
 *limit = len(rows)
 }

 fmt.Printf("%-8s %-4s %-10s %s\n", "PID", "S", "RSS(MB)", "COMM")
 for i := 0; i < *limit; i++ {
 r := rows[i]
 fmt.Printf("%-8d %-4s %-10.1f %s\n",
 r.PID, r.State, float64(r.RSS)/1024.0/1024.0, filepath.Base(r.Comm))
 }
}

```

## Step-by-step build path

1. List `/proc` and filter numeric PIDs.
2. Parse `stat` carefully (comm parentheses).

3. Convert RSS pages with `os.Getpagesize()`.
4. Sort and format table.
5. Skip races where PID disappears.

## Run and verification

```
go run . -n 40
compare:
ps -eo pid,stat,rss,comm | head
```

## Tests

On non-Linux, skip:

```
package main

import (
 "os"
 "runtime"
 "testing"
)

func TestParseSelf(t *testing.T) {
 if runtime.GOOS != "linux" {
 t.Skip("linux only")
 }
 p, err := parseProc(os.Getpid())
 if err != nil {
 t.Fatal(err)
 }
 if p.PID == 0 || p.Comm == "" {
 t.Fatalf("%+v", p)
 }
}
```

## Stretch goals

1. Sort by RSS descending (`-sort rss`).
2. Filter by comm regex.
3. Read `/proc/pid/cmdline` null-separated.
4. TUI refresh every second.

## Pitfalls

Pitfall	Fix
Naive Fields on whole stat	parse comm via parentheses
Hard-coded page size 4096	<code>os.Getpagesize()</code>
Crashing when process exits	ignore read errors

## Learning goals

- `/proc` as an API
- Robust parsing of kernel text formats
- Linux-specific tooling with clean failure elsewhere

## 020 Project 20: KV HTTP Store

## 020 Build a Key-Value HTTP Store

A production-style mini service with CRUD endpoints and periodic snapshot persistence.

```
HTTP API <-> in-memory map + mutex <-> snapshot.json
```

### Full main.go

```
package main

import (
 "encoding/json"
 "log"
 "net/http"
 "os"
 "sync"
 "time"
)

type store struct {
 mu sync.RWMutex
 m map[string]string
}

func (s *store) load(path string) {
 b, err := os.ReadFile(path)
 if err != nil {
 return
 }
 s.mu.Lock()
 defer s.mu.Unlock()
 _ = json.Unmarshal(b, &s.m)
}

func (s *store) save(path string) error {
 s.mu.RLock()
 defer s.mu.RUnlock()
 b, err := json.MarshalIndent(s.m, "", " ")
 if err != nil {
```

```

 return err
 }
 return os.WriteFile(path, b, 0o644)
}

func main() {
 s := &store{m: map[string]string{}}
 snap := "snapshot.json"
 s.load(snap)

 go func() {
 t := time.NewTicker(10 * time.Second)
 defer t.Stop()
 for range t.C {
 _ = s.save(snap)
 }
 }()

 mux := http.NewServeMux()
 mux.HandleFunc("PUT /kv/{key}", func(w http.ResponseWriter, r *http.Request) {
 key := r.PathValue("key")
 var body struct{ Value string `json:"value"` }
 if err := json.NewDecoder(r.Body).Decode(&body); err != nil {
 http.Error(w, "bad json", http.StatusBadRequest)
 return
 }
 s.mu.Lock()
 s.m[key] = body.Value
 s.mu.Unlock()
 w.WriteHeader(http.StatusNoContent)
 })

 mux.HandleFunc("GET /kv/{key}", func(w http.ResponseWriter, r *http.Request) {
 key := r.PathValue("key")
 s.mu.RLock()
 v, ok := s.m[key]
 s.mu.RUnlock()
 if !ok {
 http.NotFound(w, r)
 return
 }
 _ = json.NewEncoder(w).Encode(map[string]string{"key": key, "value": v})
 })

 mux.HandleFunc("DELETE /kv/{key}", func(w http.ResponseWriter, r *http.Request) {
 key := r.PathValue("key")
 s.mu.Lock()

```

```

 delete(s.m, key)
 s.mu.Unlock()
 w.WriteHeader(http.StatusNoContent)
 })

 mux.HandleFunc("GET /kv", func(w http.ResponseWriter, r *http.Request) {
 s.mu.RLock()
 defer s.mu.RUnlock()
 _ = json.NewEncoder(w).Encode(s.m)
 })

 log.Println("listening :8081")
 log.Fatal(http.ListenAndServe(":8081", mux))
}

```

## Run

```

go run .
curl -X PUT localhost:8081/kv/name -d '{"value":"king"}' -H 'content-type: application/json'
curl localhost:8081/kv/name

```

## Step-by-Step Explanation

1. Define request and response contracts first.
2. Validate inbound input before doing any state changes.
3. Keep handler logic short and move reusable logic into helper functions.
4. Add timeouts and clear error paths.
5. Return consistent responses and status codes.

## Code Anatomy

- Handlers parse input, call domain logic, write response.
- Shared state uses synchronization where needed.
- Transport concerns stay separate from business rules.

## Learning Goals

- Build reliable service endpoints in Go.
- Understand API ergonomics and operational safety.
- Prepare code structure for tests and persistence later.



## **021 Project 21: WebSocket Chat**

## 021 Build a WebSocket Chat Server

A minimal real-time chat backend with a broadcast hub. Clients connect to `/ws`, send text frames, and receive every message fan-out to all peers.

```
clients <-> hub (mutex + map) <-> broadcast WriteMessage
```

### Problem statement

- Upgrade HTTP → WebSocket on `/ws`
- Track connected clients
- Broadcast each inbound text message to all clients
- Remove clients on read error/close
- Lab-friendly: `CheckOrigin` allows all (document prod hardening)

### Acceptance criteria

- ☐ Two clients can exchange messages
- ☐ Disconnect does not crash the server
- ☐ Mutex protects the client map (race-clean under `-race`)
- ☐ Builds with gorilla/websocket or nhooyr/coder websocket
- ☐ Clear run instructions

### Setup

```
mkdir gochat && cd gochat
go mod init example.com/gochat
go get github.com/gorilla/websocket@latest
go 1.27
go mod tidy
```

## Full main.go

```
package main

import (
 "log"
 "net/http"
 "sync"
 "time"

 "github.com/gorilla/websocket"
)

var upgrader = websocket.Upgrader{
 CheckOrigin: func(r *http.Request) bool { return true }, // lab only
}

type hub struct {
 mu sync.Mutex
 clients map[*websocket.Conn]struct{}
}

func (h *hub) add(c *websocket.Conn) {
 h.mu.Lock()
 defer h.mu.Unlock()
 h.clients[c] = struct{}{}
}

func (h *hub) del(c *websocket.Conn) {
 h.mu.Lock()
 defer h.mu.Unlock()
 delete(h.clients, c)
 _ = c.Close()
}

func (h *hub) broadcast(msg []byte) {
 h.mu.Lock()
 defer h.mu.Unlock()
 for c := range h.clients {
 _ = c.SetWriteDeadline(time.Now().Add(5 * time.Second))
 if err := c.WriteMessage(websocket.TextMessage, msg); err != nil {
 delete(h.clients, c)
 _ = c.Close()
 }
 }
}
```

```

func main() {
 h := &hub{clients: map[*websocket.Conn]struct{}{}}

 http.HandleFunc("/ws", func(w http.ResponseWriter, r *http.Request) {
 c, err := upgrader.Upgrade(w, r, nil)
 if err != nil {
 return
 }
 h.add(c)
 defer h.del(c)

 for {
 _, msg, err := c.ReadMessage()
 if err != nil {
 return
 }
 h.broadcast(msg)
 }
 })

 http.HandleFunc("/", func(w http.ResponseWriter, r *http.Request) {
 w.Header().Set("Content-Type", "text/html; charset=utf-8")
 _, _ = w.Write([]byte(indexHTML))
 })

 log.Println("chat on http://localhost:8090/ ws://localhost:8090/ws")
 log.Fatal(http.ListenAndServe(":8090", nil))
}

const indexHTML = `<!doctype html>
<title>gochat</title>
<pre id="log"></pre>
<input id="m" placeholder="message"/><button onclick="send()">send</button>
<script>
const log = (t) => { logEl.textContent += t + "\n"; };
const logEl = document.getElementById("log");
const ws = new WebSocket("ws://" + location.host + "/ws");
ws.onmessage = (e) => log("< " + e.data);
ws.onopen = () => log("connected");
function send() {
 const v = document.getElementById("m").value;
 ws.send(v);
 log("> " + v);
}
</script>`

```

## Run and verification

```
go run .
open two browser tabs to http://localhost:8090/
send messages; both tabs should show traffic

go test -race ./... # if you extract hub tests
```

Manual wscat-style:

```
if you have websocat or similar
websocat ws://localhost:8090/ws
```

## Tests

```
package main

import (
 "testing"
)

func TestHubAddDel(t *testing.T) {
 h := &hub{clients: map[*websocket.Conn]struct{}{}}
 // without real conn, just ensure map ops are safe with nil key edge
 // prefer integration test with httptest.Server for deeper coverage
 if len(h.clients) != 0 {
 t.Fatal()
 }
}
```

Better: use `httptest.NewServer` + gorilla dialer (stretch in repo).

## Step-by-step build path

1. Upgrade handler.
2. Hub with mutexed set of conns.
3. Read loop → broadcast.
4. Remove on error; write deadlines.
5. Tiny HTML client for demo.

## Stretch goals

1. Per-client send goroutine + outbound queue (avoid slow-client blocking hub).

2. Nicknames and join/leave system messages.
3. Rooms via path `/ws/{room}`.
4. Origin allowlist for production.
5. Ping/pong keepalive.

## Pitfalls

Pitfall	Fix
Concurrent map without mutex	data race
Broadcast write blocks hub	per-conn writer queues
CheckOrigin true in prod	CSRF-ish WS risks
No write deadline	hung peers stall all

## Learning goals

- WebSocket upgrade and framing
- Fan-out hub patterns
- Synchronization around connection sets

## 022 Project 22: Log Shipper CLI

## 022 Build a Log Shipper CLI

Tail a log file and POST new lines to an HTTP ingestion endpoint. This project combines follow-mode I/O, JSON payloads, HTTP client timeouts, and optional batching for efficiency.

```
file tail -> line channel -> (optional batch) -> HTTP POST /ingest
```

### Problem statement

Build logship:

- Seek to end of file (or start with `-from-start`)
- Follow new lines (like `tail -f`)
- POST each line (or batch) as JSON to `-endpoint`
- Survive transient network errors without exiting the process
- Exit cleanly on SIGINT

### Acceptance criteria

- ☐ New lines after start are POSTed
- ☐ HTTP client has a timeout
- ☐ Failed POSTs log to stderr and continue following
- ☐ Ctrl+C stops the loop
- ☐ Module builds with stdlib only

### Setup

```
mkdir logship && cd logship
go mod init example.com/logship
go 1.27
```



## Full main.go

```
package main

import (
 "bufio"
 "bytes"
 "context"
 "encoding/json"
 "flag"
 "fmt"
 "io"
 "net/http"
 "os"
 "os/signal"
 "syscall"
 "time"
)

type payload struct {
 TS string `json:"ts"`
 Line string `json:"line"`
 File string `json:"file"`
}

func shipLine(ctx context.Context, client *http.Client, endpoint, file, line string) error {
 body, err := json.Marshal(payload{
 TS: time.Now().Format(time.RFC3339Nano),
 Line: line,
 File: file,
 })
 if err != nil {
 return err
 }
 req, err := http.NewRequestWithContext(ctx, http.MethodPost, endpoint, bytes.NewReader(b
 if err != nil {
 return err
 }
 req.Header.Set("Content-Type", "application/json")
 resp, err := client.Do(req)
 if err != nil {
 return err
 }
 defer resp.Body.Close()
 _, _ = io.Copy(io.Discard, resp.Body)
 if resp.StatusCode >= 300 {
```

```

 return fmt.Errorf("status %d", resp.StatusCode)
 }
 return nil
}

func main() {
 file := flag.String("file", "app.log", "log file")
 endpoint := flag.String("endpoint", "http://localhost:8080/ingest", "ingest URL")
 fromStart := flag.Bool("from-start", false, "ship existing content first")
 poll := flag.Duration("poll", 300*time.Millisecond, "poll interval on EOF")
 flag.Parse()

 ctx, stop := signal.NotifyContext(context.Background(), os.Interrupt, syscall.SIGTERM)
 defer stop()

 f, err := os.Open(*file)
 if err != nil {
 fmt.Fprintln(os.Stderr, err)
 os.Exit(1)
 }
 defer f.Close()

 if !*fromStart {
 if _, err := f.Seek(0, io.SeekEnd); err != nil {
 fmt.Fprintln(os.Stderr, err)
 os.Exit(1)
 }
 }

 r := bufio.NewReader(f)
 client := &http.Client{Timeout: 3 * time.Second}

 fmt.Fprintf(os.Stderr, "shipping %s -> %s\n", *file, *endpoint)

 for {
 select {
 case <-ctx.Done():
 fmt.Fprintln(os.Stderr, "shutdown")
 return
 default:
 }

 line, err := r.ReadString('\n')
 if err == io.EOF {
 // partial line without newline: keep in buffer via Reader
 select {
 case <-ctx.Done():

```

```

 return
 case <-time.After(*poll):
 }
 continue
}
if err != nil {
 fmt.Fprintln(os.Stderr, "read error:", err)
 select {
 case <-ctx.Done():
 return
 case <-time.After(*poll):
 }
 continue
}

if err := shipLine(ctx, client, *endpoint, *file, line); err != nil {
 fmt.Fprintln(os.Stderr, "post failed:", err)
 // continue tailing
}
}
}

```

## Minimal ingest receiver (for local test)

ingest.go (separate program or build tag):

```

//go:build ignore

package main

import (
 "fmt"
 "io"
 "log"
 "net/http"
)

func main() {
 http.HandleFunc("/ingest", func(w http.ResponseWriter, r *http.Request) {
 b, _ := io.ReadAll(r.Body)
 fmt.Printf("got: %s\n", b)
 w.WriteHeader(http.StatusNoContent)
 })
 log.Println("ingest :8080")
 log.Fatal(http.ListenAndServe(":8080", nil))
}

```

```
}
```

```
go run ingest.go
other terminal:
echo 'hello ship' >> app.log
go run . -file app.log -endpoint http://localhost:8080/ingest
another terminal:
echo 'line two' >> app.log
```

## Step-by-step build path

1. Open file; seek end unless `-from-start`.
2. Loop `ReadString('\n')`; on EOF sleep/poll.
3. Marshal JSON; POST with context + timeout.
4. Wire SIGINT via `signal.NotifyContext`.
5. Add batching: accumulate N lines or T ms, one POST.

## Batching stretch (sketch)

```
// flush []string every 100 lines or 1s via time.Ticker
type batchPayload struct {
 Lines []payload `json:"lines"`
}
```

## Verification

```
go build -o logship .
with ingest running:
printf 'a\nb\n' >> app.log
expect two POSTs (or one batch)
```

## Pitfalls

Pitfall	Fix
No HTTP timeout	hang forever
Exit on first POST error	use continue + log
Truncate/rotation not handled	detect inode/size shrink; reopen (stretch)
Busy loop on EOF	poll sleep / ticker

Pitfall	Fix
Missing newline at EOF	keep partial buffer (bufio does)

## Learning goals

- Follow-mode file IO
- Resilient HTTP clients
- Graceful shutdown with context
- Path to production shippers (batch, backoff, rotation)

## **023 Project 23: Proxmox Batch Operations CLI**

## 023 Build a Proxmox Batch CLI

Operate many VMs in one command (start/stop/reboot) using Proxmox API token auth and a bounded worker pool. Print a per-VM result summary for automation logs.

```
parse vmid list -> jobs channel -> workers POST status API -> result lines
```

### Problem statement

```
pvebatch -node NODE -op start|stop|reboot -ids 101,102,103 [-w 8]
```

- Auth via `-base` / `-token` or `PVE_BASE_URL` / `PVE_TOKEN`
- Concurrent API calls with worker limit
- Non-zero exit if any VM fails (optional policy)

### Acceptance criteria

- ☐ Validates required flags
- ☐ Worker pool bounds concurrency
- ☐ Prints ok/fail per VMID
- ☐ HTTP timeout set
- ☐ Lab TLS skip documented

### Setup

```
mkdir pvebatch && cd pvebatch
go mod init example.com/pvebatch
go 1.27

export PVE_BASE_URL='https://pve.example:8006'
export PVE_TOKEN='user@pam!tokenid=secret'
```

## Full main.go

```
package main

import (
 "crypto/tls"
 "flag"
 "fmt"
 "io"
 "net/http"
 "os"
 "strconv"
 "strings"
 "sync"
 "sync/atomic"
 "time"
)

func action(hc *http.Client, base, token, node string, vmid int, op string) error {
 url := fmt.Sprintf("%s/api2/json/nodes/%s/qemu/%d/status/%s", base, node, vmid, op)
 req, err := http.NewRequest(http.MethodPost, url, nil)
 if err != nil {
 return err
 }
 req.Header.Set("Authorization", "PVEAPIToken="+token)
 resp, err := hc.Do(req)
 if err != nil {
 return err
 }
 defer resp.Body.Close()
 body, _ := io.ReadAll(io.LimitReader(resp.Body, 4096))
 if resp.StatusCode >= 300 {
 return fmt.Errorf("status %d: %s", resp.StatusCode, strings.TrimSpace(string(body)))
 }
 return nil
}

func main() {
 base := flag.String("base", os.Getenv("PVE_BASE_URL"), "proxmox base url")
 token := flag.String("token", os.Getenv("PVE_TOKEN"), "api token")
 node := flag.String("node", "pve", "node name")
 op := flag.String("op", "start", "start|stop|reboot")
 ids := flag.String("ids", "", "comma list vmids")
 workers := flag.Int("w", 8, "parallel workers")
 insecure := flag.Bool("insecure", true, "skip TLS verify (lab)")
 flag.Parse()
}
```



```

switch *op {
case "start", "stop", "reboot":
default:
 fmt.Fprintln(os.Stderr, "op must be start|stop|reboot")
 os.Exit(2)
}
if *base == "" || *token == "" || *ids == "" {
 fmt.Fprintln(os.Stderr, "need -base, -token, -ids (or env PVE_*)")
 os.Exit(2)
}
if *workers < 1 {
 os.Exit(2)
}

tr := http.DefaultTransport.(*http.Transport).Clone()
if *insecure {
 tr.TLSClientConfig = &tls.Config{InsecureSkipVerify: true}
}
hc := &http.Client{Timeout: 8 * time.Second, Transport: tr}

jobs := make(chan int)
var wg sync.WaitGroup
var fail atomic.Int64

for i := 0; i < *workers; i++ {
 wg.Go(func() {
 for id := range jobs {
 err := action(hc, *base, *token, *node, id, *op)
 if err != nil {
 fmt.Printf("vm %d fail: %v\n", id, err)
 fail.Add(1)
 } else {
 fmt.Printf("vm %d ok\n", id)
 }
 }
 })
}

for _, s := range strings.Split(*ids, ",") {
 id, err := strconv.Atoi(strings.TrimSpace(s))
 if err != nil {
 fmt.Fprintf(os.Stderr, "bad id %q\n", s)
 fail.Add(1)
 continue
 }
 jobs <- id
}

```

```

 }
 close(jobs)
 wg.Wait()

 if fail.Load() > 0 {
 os.Exit(1)
 }
}

```

## Step-by-step build path

1. Flag/env config and op allowlist.
2. Shared `http.Client` with timeout.
3. Worker pool consuming VMID jobs.
4. Atomic fail counter for exit code.
5. Document token scopes (VM.PowerMgmt).

## Run and verification

```

go run . -node pve -op start -ids 101,102,103 -w 4
go run . -op stop -ids 101

```

## Tests

Unit-test ID parsing logic by extracting a helper:

```

func parseIDs(s string) ([]int, error) {
 var out []int
 for _, p := range strings.Split(s, ",") {
 p = strings.TrimSpace(p)
 if p == "" {
 continue
 }
 id, err := strconv.Atoi(p)
 if err != nil {
 return nil, err
 }
 out = append(out, id)
 }
 return out, nil
}

```

## Stretch goals

1. Read VMIDs from file.
2. Status poll until guest agent up.
3. Dry-run mode printing URLs only.
4. JSON summary output.

## Pitfalls

Pitfall	Fix
Unbounded goroutines	worker pool
No timeout	hung batch
Insecure TLS in prod	proper CA pool
Ignoring partial failure	exit 1 if any fail

## Learning goals

- Concurrent API automation
- WaitGroup + job channel pools
- Operator-friendly batch tooling

## **024 Project 24: Controller Pattern Simulator**

# 024 Build a Controller/Reconciler Simulator

Practice the Kubernetes-style reconciliation loop **without** Kubernetes dependencies: desired state vs actual state, idempotent steps, external drift, and convergence under a ticker.

```
desired state + actual state -> reconcile loop -> actions -> converge
```

## Problem statement

Simulate a Deployment-like controller:

- DesiredReplicas is the spec
- ActualReplicas is observed status
- Each tick, apply **one** reconcile action: scale up, scale down, or noop
- Occasionally inject **drift** (external delete/create)
- Print a clear timeline so operators can see convergence

## Acceptance criteria

- ☐ Reconcile is idempotent: repeated noops when in sync
- ☐ Actual moves toward desired by at most one step per tick (or document multi-step)
- ☐ Drift can move actual away; controller recovers
- ☐ Program ends after N ticks or when stable for M ticks
- ☐ Stdlib only; deterministic mode via fixed RNG seed flag

## Setup

```
mkdir reconsim && cd reconsim
go mod init example.com/reconsim
go 1.27
```

## Full main.go

```
package main

import (
 "flag"
 "fmt"
 "math/rand"
 "os"
 "time"
)

type State struct {
 DesiredReplicas int
 ActualReplicas int
}

func reconcile(s *State) string {
 if s.ActualReplicas < s.DesiredReplicas {
 s.ActualReplicas++
 return "scale_up"
 }
 if s.ActualReplicas > s.DesiredReplicas {
 s.ActualReplicas--
 return "scale_down"
 }
 return "noop"
}

func clampNonNeg(n int) int {
 if n < 0 {
 return 0
 }
 return n
}

func main() {
 desired := flag.Int("desired", 3, "desired replicas")
 actual := flag.Int("actual", 0, "initial actual replicas")
 ticks := flag.Int("ticks", 20, "max ticks")
 driftEvery := flag.Int("drift-every", 6, "inject drift every N ticks (0=off)")
 seed := flag.Int64("seed", 1, "rng seed for drift")
 delay := flag.Duration("delay", 200*time.Millisecond, "tick delay (0 for fast)")
 flag.Parse()

 if *desired < 0 || *actual < 0 {
```

```

 fmt.Fprintln(os.Stderr, "desired/actual must be >= 0")
 os.Exit(2)
}

rng := rand.New(rand.NewSource(*seed))
s := State{DesiredReplicas: *desired, ActualReplicas: *actual}

stable := 0
for i := 0; i < *ticks; i++ {
 if *driftEvery > 0 && i > 0 && i%*driftEvery == 0 {
 delta := rng.Intn(3) - 1 // -1,0,1
 s.ActualReplicas = clampNonNeg(s.ActualReplicas + delta)
 fmt.Printf("tick=%02d DRIFT delta=%+d actual=%d\n", i, delta, s.ActualReplicas)
 }

 action := reconcile(&s)
 fmt.Printf("tick=%02d desired=%d actual=%d action=%s\n",
 i, s.DesiredReplicas, s.ActualReplicas, action)

 if action == "noop" {
 stable++
 if stable >= 3 {
 fmt.Println("converged")
 return
 }
 } else {
 stable = 0
 }

 if *delay > 0 {
 time.Sleep(*delay)
 }

 // mid-run desired change demo
 if i == *ticks/2 && *ticks >= 4 {
 s.DesiredReplicas = max(1, s.DesiredReplicas-1)
 fmt.Printf("tick=%02d DESIRED_CHANGED desired=%d\n", i, s.DesiredReplicas)
 }
}
fmt.Println("finished ticks")
}

func max(a, b int) int {
 if a > b {
 return a
 }
 return b
}

```

```
}
```

## Step-by-step build path

1. Model `State{Desired, Actual}`.
2. Write pure `reconcile(*State)` action with unit tests.
3. Drive a ticker/loop with logging.
4. Add drift and desired changes.
5. Add convergence detection (stable noops).

## Architecture

```
Desired observe Actual

 reconcile
 diff apply one

 action log
```

## Run and verification

```
go run . -desired 3 -actual 0 -ticks 25 -seed 42 -delay 0
expect scale_up until actual=3, noops, maybe drift recovery

go run . -desired 5 -actual 8 -delay 0
expect scale_down
```

## Tests

```
package main

import "testing"

func TestReconcileScaleUp(t *testing.T) {
 s := State{DesiredReplicas: 2, ActualReplicas: 0}
 if a := reconcile(&s); a != "scale_up" || s.ActualReplicas != 1 {
 t.Fatalf("action=%s actual=%d", a, s.ActualReplicas)
 }
}
```



```

 }
}

func TestReconcileNoop(t *testing.T) {
 s := State{DesiredReplicas: 2, ActualReplicas: 2}
 if a := reconcile(&s); a != "noop" {
 t.Fatal(a)
 }
}

func TestReconcileScaleDown(t *testing.T) {
 s := State{DesiredReplicas: 1, ActualReplicas: 3}
 if a := reconcile(&s); a != "scale_down" || s.ActualReplicas != 2 {
 t.Fatalf("action=%s actual=%d", a, s.ActualReplicas)
 }
}

go test -race ./...

```

## Stretch goals

1. Multiple resources (map of name  $\rightarrow$  state).
2. Exponential backoff after failed apply (simulate errors).
3. Workqueue with rate limiting (client-go style mental model).
4. Level-triggered vs edge-triggered event notes in README.
5. JSON status dump each tick for later graphing.

## Pitfalls

Pitfall	Fix
One-shot script instead of loop	Always re-observe actual
Non-idempotent apply	Same desired $\rightarrow$ noop safely
Unbounded scale steps	Cap max replicas; step by 1
Non-deterministic tests	Fixed seed; pure reconcile

## Learning goals

- Controller pattern used in modern platforms
- Convergence over one-shot automation
- Drift handling and observability of actions

## **025 Project 25: Resource Index + 150 More Ideas**

# 025 Resource Index and Project Idea Backlog

This chapter is a curated idea generator based on well-known Go learning resources and book/tutorial indexes.

## Resource Anchors

1. Go by Example: [gobyexample.com](http://gobyexample.com)
2. Exercism Go Track (141 exercises): [exercism.org/tracks/go](http://exercism.org/tracks/go)
3. Exercism Go docs/testing: [exercism.org/docs/tracks/go](http://exercism.org/docs/tracks/go)
4. Build Your Own X list: [github.com/codecrafters-io/build-your-own-x](https://github.com/codecrafters-io/build-your-own-x)
5. Awesome Go: [github.com/uhub/awesome-go](https://github.com/uhub/awesome-go)
6. Go by Example (book index/public page): [simonandschuster.com/.../Go-by-Example](http://simonandschuster.com/.../Go-by-Example)

## 150 Additional Project Ideas

### Linux CLI (1-30)

1. `chmod` helper wrapper with safety checks.
2. `chown` dry-run simulator.
3. `tree` command with depth filtering.
4. file deduplicator by checksum.
5. symlink verifier.
6. broken symlink cleaner.
7. directory watcher + action runner.
8. command history analyzer.
9. shell session recorder parser.
10. cron expression validator.
11. mini `top` clone.
12. mini `iotop` style estimator.
13. mini `netstat` parser.
14. mini `ss` parser.
15. process killer by regex.
16. zombie process detector.
17. service health checker.
18. `journalctl` filter wrapper.
19. colorized log viewer.
20. rotating file logger CLI.
21. backup snapshot tool.

22. rsync-like copy planner.
23. permission drift checker.
24. stale temp-file cleaner.
25. bulk rename utility.
26. grep over compressed files.
27. large-file splitter/merger.
28. file entropy scanner.
29. duplicate line remover.
30. path sanitizer utility.

## **Networking/Web (31-60)**

31. DNS lookup CLI with JSON output.
32. TLS certificate inspector.
33. HTTP header security scanner.
34. simple API gateway with auth key.
35. reverse proxy with rate limit.
36. websocket event broadcaster.
37. webhook receiver with signature check.
38. link checker crawler.
39. HTTP cache proxy.
40. REST pagination helper.
41. OpenAPI schema diff checker.
42. JSON API contract tester.
43. API replay tool from HAR.
44. request signer (HMAC).
45. JWT issue/verify CLI.
46. OAuth device-flow sample app.
47. multipart upload service.
48. resumable upload protocol demo.
49. static site server with live reload.
50. image resize API.
51. URL metadata extractor.
52. API latency SLO dashboard backend.
53. HTTP retry middleware benchmark.
54. gRPC health check service.
55. gRPC proxy to REST bridge.
56. protobuf evolution checker.
57. distributed trace demo app.
58. log correlation ID middleware.
59. CORS policy validator.
60. synthetic uptime checker.

## **Data/Storage (61-90)**

61. sqlite migration runner.

62. seed data loader CLI.
63. query timing profiler.
64. slow-query report generator.
65. CSV-to-SQL importer.
66. SQL-to-CSV exporter.
67. data masking utility.
68. pii scanner for CSV/JSON.
69. key rotation simulation tool.
70. append-only ledger store.
71. LSM-tree toy storage engine.
72. B-tree toy index.
73. bloom filter demo service.
74. write-ahead log simulator.
75. compaction strategy benchmark.
76. cache stampede protector demo.
77. id generator service (ksuid/ulid).
78. event sourcing sample.
79. CQRS read model projector.
80. dead letter queue consumer.
81. stream deduplication worker.
82. retention policy enforcer.
83. object storage sync tool.
84. folder-to-s3 mirror (mock).
85. schema migration drift detector.
86. JSON schema validator CLI.
87. XML-to-JSON converter.
88. parquet writer demo.
89. backup restore verifier.
90. data lineage tracer toy.

## **Concurrency/Systems (91-120)**

91. pipeline backpressure simulator.
92. bounded queue benchmark.
93. lock contention visualizer.
94. goroutine leak detector wrapper.
95. worker auto-scaler simulation.
96. scheduler fairness simulator.
97. token bucket distributed simulation.
98. local message broker toy.
99. pub/sub with retries.
100. exactly-once semantics demo.
101. raft consensus toy model.
102. leader election simulator.
103. heartbeat failure detector.
104. distributed lock prototype.
105. partial failure injection tool.

- 106. chaos monkey for local services.
- 107. process supervisor.
- 108. service dependency graph parser.
- 109. config hot-reload framework.
- 110. plugin loader architecture demo.
- 111. dynamic rule engine toy.
- 112. policy-as-code evaluator.
- 113. command bus/event bus demo.
- 114. saga workflow simulator.
- 115. batch scheduler prototype.
- 116. resource quota enforcer.
- 117. cgroup stats reader (Linux).
- 118. seccomp profile inspector.
- 119. ptrace-based syscall tracer (toy).
- 120. mini container runtime PoC.

## **TUI/Infra/Platform (121-150)**

- 121. TUI docker image browser.
- 122. TUI git commit explorer.
- 123. TUI log tail with filters.
- 124. TUI database query runner.
- 125. TUI feature-flag manager.
- 126. TUI release dashboard.
- 127. TUI CI pipeline monitor.
- 128. TUI Kubernetes pod viewer.
- 129. TUI helm values diff tool.
- 130. TUI terraform plan viewer.
- 131. Proxmox node capacity dashboard.
- 132. Proxmox VM template manager.
- 133. Proxmox snapshot orchestrator.
- 134. VM startup order manager.
- 135. host inventory manager.
- 136. SSH key rotation orchestrator.
- 137. parallel command runner with canary.
- 138. rollout status dashboard.
- 139. blue/green deployment simulator.
- 140. incident timeline builder.
- 141. SRE runbook executor CLI.
- 142. service dependency outage simulator.
- 143. alert noise reduction analyzer.
- 144. SLO burn-rate calculator.
- 145. policy compliance reporter.
- 146. artifact promotion workflow.
- 147. SBOM generator wrapper.
- 148. CVE triage dashboard.
- 149. secure secrets sync tool.

150. platform control plane toy app.

## **Suggested Build Order**

1. Pick 10 CLI tasks.
2. Pick 5 network/API tasks.
3. Pick 5 concurrency/system tasks.
4. Pick 3 TUI/infra tasks.
5. Finish with one capstone controller + dashboard.

## **Step-by-Step Explanation**

1. Pick a small set of exercises by track.
2. Timebox each attempt to ten minutes.
3. Record one takeaway and one weakness after each exercise.
4. Revisit chapter references when blocked.
5. Re-solve selected problems from memory weekly.

## **Learning Goals**

- Build consistency, not one-time intensity.
- Improve retrieval and transfer of Go patterns.
- Progress from syntax fluency to engineering fluency.

## **026 Project 26: Kubernetes Pod Lister**



## 026 Build a Kubernetes Pod Lister

List pods across namespaces using `client-go`. Produce operator-friendly tabular output suitable for scripts and CI smoke checks against a real or kind/minikube cluster.

```
kubeconfig -> clientset -> Pods.List -> print ns/name/phase/node
```

### Problem statement

CLI `kpods`:

- Load kubeconfig from `-kubeconfig` (default `~/.kube/config`)
- Optional `-n` namespace (empty = all namespaces)
- Optional label selector `-l`
- Print: `NAMESPACE/NAME PHASE NODE READY`
- Non-zero exit if API call fails

### Acceptance criteria

- ☐ Lists pods with `client-go`
- ☐ Namespace filter works
- ☐ Clear errors when kubeconfig/cluster unavailable
- ☐ Context with timeout (no hang forever)
- ☐ Builds with Go modules

### Setup

```
mkdir kpods && cd kpods
go mod init example.com/kpods
go get k8s.io/client-go@latest
go get k8s.io/apimachinery@latest
go 1.27
go mod tidy
```

Requires a reachable cluster (`kubectl get pods` works).

## Full main.go

```
package main

import (
 "context"
 "flag"
 "fmt"
 "os"
 "path/filepath"
 "time"

 corev1 "k8s.io/api/core/v1"
 metav1 "k8s.io/apimachinery/pkg/apis/meta/v1"
 "k8s.io/client-go/kubernetes"
 "k8s.io/client-go/tools/clientcmd"
 "k8s.io/client-go/util/homedir"
)

func readyCount(p corev1.Pod) (ready, total int) {
 total = len(p.Status.ContainerStatuses)
 for _, cs := range p.Status.ContainerStatuses {
 if cs.Ready {
 ready++
 }
 }
 return ready, total
}

func main() {
 var kubeconfig string
 if home := homedir.HomeDir(); home != "" {
 kubeconfig = filepath.Join(home, ".kube", "config")
 }
 kc := flag.String("kubeconfig", kubeconfig, "kubeconfig path")
 namespace := flag.String("n", "", "namespace (empty=all)")
 label := flag.String("l", "", "label selector")
 timeout := flag.Duration("timeout", 10*time.Second, "API timeout")
 flag.Parse()

 cfg, err := clientcmd.BuildConfigFromFlags("", *kc)
 if err != nil {
 fmt.Fprintln(os.Stderr, "kubeconfig:", err)
 os.Exit(1)
 }
 cli, err := kubernetes.NewForConfig(cfg)
```

```

if err != nil {
 fmt.Fprintln(os.Stderr, "client:", err)
 os.Exit(1)
}

ctx, cancel := context.WithTimeout(context.Background(), *timeout)
defer cancel()

ns := *namespace
if ns == "" {
 ns = metav1.NamespaceAll
}

pods, err := cli.CoreV1().Pods(ns).List(ctx, metav1.ListOptions{
 LabelSelector: *label,
})
if err != nil {
 fmt.Fprintln(os.Stderr, "list pods:", err)
 os.Exit(1)
}

fmt.Printf("%-48s %-12s %-20s %s\n", "POD", "PHASE", "NODE", "READY")
for _, p := range pods.Items {
 r, t := readyCount(p)
 fmt.Printf("%-48s %-12s %-20s %d/%d\n",
 p.Namespace+"/"+p.Name,
 string(p.Status.Phase),
 p.Spec.NodeName,
 r, t,
)
}
fmt.Fprintf(os.Stderr, "total: %d\n", len(pods.Items))
}

```

## Step-by-step build path

1. Build rest config from kubeconfig.
2. Create `kubernetes.Clientset`.
3. Call `Pods(ns).List` with context timeout.
4. Format rows for humans/scripts.
5. Add label selector and readiness columns.

## Run and verification

```
go run .
go run . -n kube-system
go run . -l app=myapp
go run . -timeout 5s

compare with kubectl
kubectl get pods -A --no-headers | wc -l
```

Without a cluster:

```
go run . -kubeconfig /nonexistent
expect non-zero exit and readable error
```

## Pitfalls

Pitfall	Fix
No timeout	use <code>context.WithTimeout</code>
Hard-coded HOME	<code>homedir.HomeDir</code> + flag
Panic on errors	print <code>stderr</code> + <code>exit 1</code>
Huge output	add limit/continue token (stretch)
Version skew	pin <code>client-go</code> close to cluster version

## Stretch goals

1. Watch mode (`Watch` + print events).
2. JSON output (`-o json`).
3. Filter by phase (`-field-selector`).
4. In-cluster config path for running as a Pod.
5. Table of restarts from `ContainerStatuses`.

## Learning goals

- Use `client-go` safely with context
- Map API objects to operator-facing columns
- Build automation-friendly cluster checks